

웹스퀘어 AI 개발 가이드

웹스퀘어 AI 개발 가이드

차례

차례	v
문서 이력	xli

Part I. 소개 1

1. 스튜디오	3
1.1 개요	3
1.2 PC 요구 사양	3
1.3 Eclipse 지원	4
1.4 설치	4
1.4.1. Windows용 스튜디오 설치 (인스톨러)	4
1.4.1.1. 설치 전 준비	4
1.4.1.2. 설치 방법	5
1.4.2. Eclipse 플러그인 설치	9
1.4.2.1. 설치 전 준비	9
1.4.2.2. 설치 방법	10
1.5 환경 설정	21
1.6 엔진 교체	23
2. 스튜디오 서버	25
2.1 개요	25
2.2 사양	25
2.3 점검	26
2.3.1. 설정	26
2.3.2. 접속	26
2.4 Quick Code 생성	28
2.4.1. 프로퍼티 설정	28
2.4.1.1. DefaultWebSquareSourceGenerateAdapter 구현체 설정	28
2.4.1.2. WebSquare 화면 다국어 설정 및 WebSquare 화면 template 디렉토리 위치 프로퍼티 설정	29
2.4.2. Adapter 구현	30
2.4.3. 소스 생성	32
2.4.3.1. 소스 생성 유형 목록 조회 (GetWebSquareSourceTypes)	32
2.4.3.2. 소스 생성 (CreateWebSquareSource)	33
2.4.3.2.1. getWebsquare()	33
2.4.3.2.2. getWebsquareList()	33
2.4.3.3. 소스 적용	35

3. 엔진	39
3.1 설치	39
3.2 사양	39
3.3 앱 실행 클라이언트 사양	40
3.4 브라우저 지원	40
3.5 설정	41
3.5.1. 클라이언트 (client.config.xml)	42
3.5.2. 서버 (server.config.xml)	42
3.6 설치 관련 문제 해결	43
3.6.1. "WebSquare.startApplication is not function"	44
3.6.2. WebServer와 WAS 분리	45
3.6.3. /javascriptLoader.wq 404 error	46
3.6.4. /javascriptLoader.wq 400(Bad Request) error	46
3.6.5. websquare.html 전송 이후 반응이 없을 때	47
3.6.6. WEBSQUARE_HOME 설정	47
3.6.7. 솔라리스OS 에서 CPU코어 개수 확인	47
3.6.8. 설치/패치 시 충돌 가능 jar 목록	48
3.6.9. JBOSS 설치 시 유의사항	48
3.6.10. 설치 할 때 확인해야 할 contentType	49
3.6.11. 웹로직 설치의 경우 web.xml 파일에 mime-mapping 설정	49
3.6.12. JEUS에서 한글이 깨지는 경우	49
3.6.13. 웹서버가 SUN iplanet(혹은 웹티어)인 경우	50
3.6.14. WAS가 websphere 일 <i>server.config.xml</i> 추가 설정	51
4. 동작	53
4.1 코드 구조	53
4.2 스크립트	54
4.2.1. <code>scwin.onpageload</code>	54
4.2.2. <code>scwin.onbeforepageunload</code>	55
4.2.3. <code><initScript></code>	56
4.2.4. <code><postScript></code>	57
4.2.5. 스크립트 실행 순서	58
4.2.5.1. WFrame이 중첩된 경우	58
4.2.5.2. Scope 기능 미적용 시	59
4.3 W-Pack (JS 변환)	60
4.4 브라우저 호출	61
4.5 CSS	63
4.5.1. <code>class</code> selector 사용	63
4.5.2. jQuery의 id selector 사용	63
4.6 웹표준 호환성	64

4.6.1. 비표준 태그 처리	64
4.6.2. GridView의 <code><nobr></code> 태그	65
4.6.3. XML의 namespace 제거	65
5. SPA	67
5.1 Scope 설정	67
5.1.1. scriptPrecedence	67
5.1.2. strictMode	68
5.1.2.1. 참고 (JavaScript "with"문)	69
5.1.3. postDrawMode	70
5.1.4. SPA Variable	71
5.1.5. <code>lazy="false"</code>	72
5.1.6. <code><getWindow searchTarget="windowId"/></code>	73
5.2 ID 변경	73
5.2.1. 최상단 메인 화면	73
5.2.1.1. 예제 (1)	74
5.2.1.2. 예제 (2)	75
5.2.1.3. 예제 (3)	76
5.2.2. WFrame 내부	77
5.2.2.1. ID 중복 허용	78
5.3 <code>\$w</code> Vs. <code>\$p</code>	78
5.4 WFrame 간의 화면 참조	79
5.4.1. 주요 API	79
5.4.2. 원칙	80
5.4.2.1. 자식 → 부모 (<code>\$p.parent();</code>)	80
5.4.2.2. 형제	80
5.4.2.3. 부모 → 자식 (<code>wframe.getWindow();</code>)	81
5.4.3. 정리	81
5.4.3.1. 부모 → 자식	81
5.4.3.2. 자식 → 부모	82
5.4.3.3. TabControl	82
5.4.3.3.1. TabControl에서 각 탭 접근	82
5.4.3.4. WindowContainer	83
5.4.3.4.1. WindowContainer에서 각 Window 접근	83
5.4.4. 예제	83
5.5 WFrame 생성 옵션 전달	84
5.5.1. 설정 방법	85
5.5.1.1. 직접 설정	86
5.5.1.2. <code>setDataObject</code> 를 통해 동적으로 설정	86
5.5.2. 관련 컴포넌트	87

5.5.3. 제약 사항	87
5.5.3.1. 객체에 함수 직접 입력 금지	87
5.5.3.2. 순환 참조 금지	88
5.6 WFrame 사용 시 유의 사항	88
5.6.1. 외부 라이브러리 수동 릴리즈	88
5.6.2. <code>onpageload</code> 에서 <code>setSrc</code> 호출 시 <code>src</code> 속성 비우기	88
5.6.3. Scope 권장	89
5.6.4. IFrame 제한	89
5.7 공통 함수	89
5.7.1. 객체 생성 방식 (권장)	89
5.7.1.1. 예제	90
5.7.2. 변수 전달 방식	91
5.7.2.1. 예제	91
5.8 외부 라이브러리 연동	92
5.8.1. <code>scopeExternal</code>	92
5.8.2. <code>scopeVariable</code>	93
5.9 CSS	93
5.9.1. <code>class</code> selector 사용	93
5.9.2. jQuery의 id selector 사용	94
5.10 SPA	95
5.11 디버깅	95
5.11.1. 예제	96
6. 이전 버전과의 비교	97
6.1 페이지 컴포넌트 기능 확대	97
6.2 다양한 리소스 및 예제 제공	98
6.2.1. Design Kit	98
6.2.2. Dev Kit	99
6.2.3. WRE (WebSquare Reference Example)	100
6.3 Perspective 개선	101
6.4 비주얼스크립트 탭 분할 기능	103
6.5 Source 탭 숨김/표시 옵션 제공	103
6.6 MAC 및 Linux 지원	104
6.7 다양한 검색 지원	104
6.8 웹스퀘어5 Vs. 웹스퀘어 AI	105
Part II. 스튜디오	107
7. 뷰 구성	109
7.1 초기 화면	109

7.2	뷰 설정	110
7.3	뷰 초기화	111
7.4	디자인 에디터	112
7.4.1.	API	112
7.4.1.1.	문서화 정보	112
7.4.1.1.1.	일반 정보	113
7.4.1.1.2.	개정 이력	115
7.4.1.2.	컴포넌트	116
7.4.1.2.1.	정보	116
7.4.1.2.2.	레이아웃	117
7.4.1.2.2.	레이아웃 적용	117
7.4.1.2.2.	레이아웃 생성	119
7.4.1.3.	속성	121
7.4.1.3.1.	속성 정의하기	121
7.4.1.4.	메소드	122
7.4.1.4.1.	메소드 정의하기	123
7.4.1.5.	이벤트	125
7.4.1.5.1.	이벤트 정의하기	125
7.4.2.	모듈	126
7.4.2.1.	공통 컴포넌트	126
7.4.2.1.1.	전역 공통 컴포넌트 (config.js)	127
7.4.2.1.2.	MSA 공통 컴포넌트 (config.js)	128
7.4.2.1.3.	페이지 공통 컴포넌트	130
7.4.2.2.	스크립트	132
7.4.2.2.1.	전역 라이브러리 (config.js)	132
7.4.2.2.2.	외부 라이브러리	134
7.4.2.3.	스타일	134
7.4.2.3.1.	전역 스타일 시트 (config.js)	135
7.4.2.3.2.	MSA 스타일 시트 (config.js)	136
7.4.2.3.3.	외부 스타일시트	137
7.4.2.3.4.	내부 스타일 시트	137
7.4.2.4.	데이터컬렉션	137
7.4.2.4.1.	DataList	138
7.4.2.4.2.	DataMap	141
7.4.2.4.3.	LinkedDataList	143
7.4.2.5.	워크플로우컬렉션	146
7.4.2.6.	서브미션	147
7.4.2.7.	MSA	149
7.4.2.7.1.	채널	149

7.4.2.7.2. 발행자	150
7.4.2.7.3. 구독자	150
7.4.3. 디자인	150
7.4.3.1. 컴포넌트	151
7.4.3.2. 데이터컬렉션	153
7.4.3.3. 스크립트	154
7.4.4. 스크립트	155
7.4.4.1. 테마 설정	156
7.4.4.2. 자동 완성	156
7.4.4.3. 컨텍스트 메뉴	157
7.4.4.4. 툴팁	157
7.4.5. 비주얼스크립트	157
7.4.6. 소스	158
7.4.7. 미리보기	160
7.5 Project Explorer	160
7.6 Component	162
7.6.1. 속성	162
7.6.1.1. 속성 설명 툴팁	163
7.6.1.2. 카테고리 별 분류	164
7.6.1.3. 속성 경렬	166
7.6.2. 이벤트	168
7.6.3. 메소드	169
7.6.4. 스타일	170
7.7 Design Kit	170
7.7.1. 컴포넌트	171
7.7.1.1. 즐겨찾기	172
7.7.1.2. 목록 새로 고침	172
7.7.2. 스니펫	172
7.7.2.1. 즐겨찾기	174
7.7.2.2. 미리보기	175
7.7.3. 템플릿	176
7.7.3.1. 즐겨찾기	178
7.7.4. 디자인 가이드	179
7.7.4.1. 색상 적용	180
7.7.4.2. 폰트 적용	181
7.8 Dev Kit	181
7.8.1. 동영상	182
7.8.2. 문서	184
7.8.3. 검색	184

7.8.4. 연관 검색	185
7.8.4.1. 입력 필드	185
7.8.4.2. 속성	185
7.8.4.3. 메소드	186
8. 화면 그리기	187
8.1 그리기 모드	187
8.2 Static 모드 (클릭 & 클릭)	187
8.2.1. 스타일 제거	189
8.3 Absolute 모드 (클릭 & 드래그)	191
8.4 이벤트 추가	194
8.5 컴포넌트 동적 생성	195
8.6 객체 추가	196
8.6.1. 브라우저 객체	196
8.6.2. 사용자 정의	196
8.6.3. WebSquare 객체	197
8.6.3.1. 유틸리티	197
8.6.3.2. UI 객체	197
8.6.3.3. 데이터 객체	197
8.7 브라우저에서 확인하기	198
9. 유용한 기능	199
9.1 드래그 & 드롭을 통한 컴포넌트 제어	199
9.1.1. 리사이징	199
9.1.2. 여러 컴포넌트 선택	200
9.1.3. 앞/뒤 이동	202
9.1.3.1. Static 모드	202
9.1.3.1.1. 뒤로 이동	202
9.1.3.1.2. 앞으로 이동	202
9.1.3.2. Absolute 모드	203
9.1.4. 여러 컴포넌트를 한번에 이동	204
9.1.4.1. Static 모드	205
9.1.4.2. Absolute 모드	206
9.1.5. 하위로 이동	206
9.2 컴포넌트	208
9.2.1. 컴포넌트 정보	208
9.2.1.1. 툴팁	208
9.2.1.2. 이벤트 표시	209
9.2.1.3. <code>ref</code> 바인딩 표시	210
9.2.2. 컴포넌트 쿼리 톨바	210
9.2.3. 표 편집	212

9.2.4. 컴포넌트 복사	213
9.3 컨텍스트 메뉴	214
9.3.1. 메뉴 구성	214
9.3.2. 등록 이벤트 표시	215
9.3.3. 컨텍스트 메뉴 편집	216
9.4 눈금선, 눈금자, 안내선	217
9.4.1. 눈금자 (Ruler)	217
9.4.2. 눈금선 (Gridlines)	218
9.4.3. 안내선 (Guides)	219
9.5 미리 보기	220
9.6 브라우저로 보기	220
9.7 파일 검색	222
9.8 기타	224
9.8.1. 고유 ID 부여	224
9.8.2. 디자인 탭 확대/축소	224
9.8.3. 청사진	225
9.8.4. 신규 아이콘	226
9.8.5. SVN 및 Git 플러그인	227
9.8.6. 메시지 표시	229
9.8.6.1. 저장 확인 메시지	229
9.8.6.2. 오류 메시지	229

Part III. 개발 프로세스 231

10. 프로젝트 생성	233
10.1 WebSquare Project	233
10.1.1. 생성	233
10.1.2. 폴더 구조	238
10.1.2.1. 웹스퀘어 리소스	240
10.1.2.1.1. 공통 모듈	240
10.1.2.1.1.1. 전역 공통 컴포넌트	240
10.1.2.1.1.1. MSA 공통 컴포넌트	240
10.1.2.1.1.1. 전역 라이브러리	240
10.1.2.1.1.1. 전역 스타일시트	240
10.1.2.1.2. 사용자 정의 컴포넌트	240
10.1.2.2. 웹스퀘어 환경 설정	241
10.1.2.2.1. 스튜디오	242
10.1.2.2.2. 클라이언트	242
10.1.2.2.3. 서버	242

10.2	WebSquare Example Project	242
10.2.1.	생성	242
10.3	WebSquare TTC Project	244
10.3.1.	생성	245
10.3.2.	폴더 구조	250
10.3.3.	라이선스 및 아이콘 등록	250
10.3.4.	TTC 등록	253
10.3.5.	TTC 자원 등록	254
10.3.6.	TTC 샘플 등록	256
10.4	프로젝트 설정	257
11.	컴포넌트 생성	259
11.1	WebSquare Common Component	259
11.1.1.	생성하기	259
11.1.2.	사용하기	261
11.2	WebSquare Page Component	262
11.2.1.	소개	262
11.2.2.	생성하기	263
11.2.3.	메소드 추가하기	266
11.2.4.	PageFrame 소스로 사용하기	270
11.3	User-Defined Ccomponent (UDC)	272
11.3.1.	생성하기	272
11.3.2.	사용하기	280
11.4	Extended Component (EXC)	284
11.4.1.	소개	284
11.4.2.	UDC Vs. EXC	284
11.4.3.	생성하기	284
11.4.4.	사용하기	288
11.5	Trusted Third-party Component (TTC)	289
11.5.1.	소개	289
11.5.2.	생성하기	290
11.5.3.	내보내기	292
11.5.4.	가져오기	294
12.	DataCollection	299
12.1	DataCollection	299
12.1.1.	개요	299
12.1.2.	동적 생성	300
12.1.3.	현재 화면에 포함된 DataCollection 반환	300
12.1.4.	복사 및 삭제	300
12.1.5.	Studio에서 생성하고 디버깅 메뉴로 확인하기	302

12.2	DataList	304
12.2.1.	생성	304
12.2.1.1.	디자인 탭에서 생성	304
12.2.1.2.	모듈 탭에서 생성	309
12.2.2.	데이터 타입 자동 변경	313
12.2.2.1.	<code>string</code> 타입 데이터를 <code>number</code> 타입으로 설정 (<code>keepDataType</code>)	313
12.2.2.2.	<code>number</code> 타입 데이터를 <code>string</code> 타입으로 설정 (<code>preserveType</code>)	314
12.2.3.	필터링	315
12.2.3.1.	<code>getFilterList()</code>	315
12.2.3.2.	<code>getFilteredRowIndex()</code>	316
12.2.4.	데이터 반환	316
12.2.4.1.	DataList의 전체 데이터 반환	316
12.2.4.2.	DataList의 행 수 반환	317
12.2.4.3.	셀 데이터 반환	317
12.2.5.	사용자 데이터 설정 및 반환	317
12.2.6.	여러 DataList를 Excel 파일로 다운로드	318
12.2.7.	공백 데이터	318
12.2.7.1.	공백 데이터 처리	318
12.2.7.2.	<code>null</code> 값과 공백(<code>" "</code>) 값 구분	319
12.2.8.	컬럼 데이터 포맷 (<code>dataList.column - importFormatter</code>)	320
12.2.9.	이벤트	320
12.2.9.1.	<code>ondataload</code>	320
12.2.9.2.	<code>onbeforecelldatachange</code>	321
12.2.9.3.	<code>onbeforerowpositionchange</code> & <code>onrowpositionchange</code>	321
12.2.9.4.	<code>oninsertrow</code>	322
12.2.9.5.	<code>onremoverow</code>	323
12.3	DataMap	323
12.3.1.	생성	323
12.3.1.1.	디자인 탭에서 생성	323
12.3.1.2.	모듈 탭에서 생성	329
12.3.2.	데이터 타입 자동 변경	332
12.3.2.1.	<code>string</code> 타입 데이터를 <code>number</code> 타입으로 설정 (<code>keepDataType</code>)	333
12.3.2.2.	<code>number</code> 타입 데이터를 <code>string</code> 타입으로 설정 (<code>preserveType</code>)	333
12.3.3.	동적 바인딩	334
12.3.4.	바인딩된 컴포넌트 확인	335
12.3.5.	공백 데이터	336
12.3.5.1.	공백 데이터 처리	336
12.3.5.2.	<code>null</code> 값과 공백(<code>" "</code>) 값 구분	337
12.4	LinkedDataList	338

12.4.1. 권장 사용법	338
12.4.2. 필터링 및 정렬 조건 설정	339
12.4.3. 생성	340
12.4.3.1. 디자인 탭에서 생성	340
12.4.3.2. 모듈 탭에서 생성	345
13. AliasDataCollection	349
13.1 정의	349
13.2 주요 API	350
13.2.1. 속성	350
13.2.2. 함수	351
13.3 제약 사항	351
13.4 생성	351
13.4.1. Studio에서 생성	352
13.4.2. 동적 생성 (Script)	356
14. Submission	359
14.1 정의	359
14.2 Submission vs. AJAX	360
14.3 Submission & DataCollection	362
14.4 Submission 생성	362
14.4.1. Studio에서 생성	362
14.4.2. 동적 생성	367
14.5 Submission 실행, 중단, 확인	368
14.5.1. Submission 실행	368
14.5.2. Submission 중단	368
14.5.3. Submission 확인	369
14.6 이벤트	369
14.6.1. submitdone(e)	370
14.6.2. submiterror(e)	371
14.7 기타	371
14.7.1. request header 설정	371
14.7.2. 데이터가 없는 경우 전송 데이터에서 제외	372
15. JS로 전환	373
15.1 JS 파일 생성 폴더	373
15.2 스튜디오에서 변환하기	373
15.3 Stand-alone 모듈 사용하기	375
15.3.1. 디렉토리를 빌드할 경우	376
15.3.2. 여러 파일을 빌드할 경우	376
16. 디버깅	379

16.1	로그	379
16.1.1.	기본 로그	379
16.1.2.	브라우저 콘솔 로그	380
16.1.3.	브라우저에서 로그 확인 (Chrome & IE 기준)	380
16.1.4.	확인할 로그 타입 지정	381
16.2	웹스퀘어 AI 제공 기능	381
16.2.1.	디버깅용 컨텍스트 메뉴	381
16.2.2.	디버깅용 컨텍스트 메뉴 허용 및 차단	382
16.2.3.	각 디버깅 메뉴 설명	383
16.2.3.1.	로그 보기	384
16.2.3.2.	DataCollection 보기	384
16.2.3.3.	페이지 소스 보기	385
16.2.3.4.	현재 화면 이벤트 출력 실행/중단	386
16.3	브라우저 개발자 도구를 활용한 디버깅 (Chrome 기준)	387

Part IV. 컴포넌트 389

17.	Forms	391
17.1	Anchor	391
17.2	AutoComplete	391
17.3	Calendar	392
17.4	CheckBox	393
17.5	CheckComboBox	394
17.6	DatePicker	394
17.7	Editor	395
17.8	FlipToggle	395
17.9	Image	396
17.10	InputBox	396
17.11	InputCalendar	397
17.12	MultiSelect	398
17.13	MultiUpload	398
17.14	Radio	399
17.15	ScheduleCalendar	400
17.16	SearchBox	401
17.17	Secret	401
17.18	SelectBox	402
17.19	Slider	402
17.20	Span	403
17.21	Spinner	403

17.22	TextArea	404
17.23	TextBox	404
17.24	Trigger	405
17.25	Upload	405
18.	Grid	407
18.1	GridView	407
18.2	Pivot	410
19.	Container	413
19.1	FloatingLayer	413
19.2	Generator	413
19.3	Group	415
19.4	ScrollView	415
19.5	SlideHide	416
19.6	Switch	417
19.7	TabControl	417
19.8	TableLayout	418
19.9	WidgetContainer	418
19.10	WindowContainer	420
20.	Navigation	421
20.1	Accordion	421
20.2	PageControl	421
20.3	PageList	422
20.4	TreeView	422
21.	Chart	425
21.1	FusionChart	425
21.2	MapChart	426
22.	Frame	427
22.1	IFrame	427
22.2	PageFrame	427
22.3	WFrame	428
23.	기타 컴포넌트	429
23.1	ActiveX	429
23.2	Applet	429
23.3	Flash	429
23.4	Output	429
23.5	ProgressBar	430
23.6	RoundedRectangle	430
23.7	Tag	430

23.8	XHTML	431
23.9	XSL	431
23.10	HTML5 태그 지원	431
24.	유틸리티	433
24.1	\$p	433
24.2	WebSquare.date	433
24.3	WebSquare.util	434
24.4	팝업	434

Part V. 설정 437

25.	스튜디오 설정	439
25.1	개요	439
25.2	Component ID Rule	440
25.3	Design Editor	441
25.4	Layout Info	442
25.5	Preview Setting	443
25.6	Project Common	444
25.7	Resource Folder	445
25.8	User Properties	446
25.9	W-Docs	447
25.9.1.	Document Generator	448
25.9.2.	Meta Dictionary	449
25.10	W-Pack	450
26.	클라이언트 설정 (client.config.xml)	453
26.1	개요	453
26.2	Overview	454
26.2.1.	W-Pack	454
26.2.1.1.	contextRoot	454
26.2.1.2.	srcExtension	454
26.2.1.3.	destExtension	454
26.2.1.4.	destRoot	454
26.2.1.5.	babelDestRoot	454
26.2.1.6.	loadingMode	455
26.2.1.7.	common	455
26.2.1.8.	name	455
26.2.1.9.	commonPath	455
26.2.1.9.1.	path	455
26.2.2.	strictMode	456

26.2.2.1. strictMode	456
26.2.2.2. scope variable	456
26.2.2.3. scriptPrecedence	456
26.2.3. page (SP3) / SPA (SP2)	457
26.2.3.1. variable	457
26.2.3.2. clone	457
26.2.3.3. spa	457
26.2.4. welcome-file	457
26.2.5. clearMemory	457
26.2.5.1. clearMemory	458
26.2.5.2. userFunction	458
26.2.5.3. userAfterFunction	458
26.2.6. Process Message	458
26.2.6.1. processMsgURL	458
26.2.6.2. processMsgHeight	458
26.2.6.3. processMsgWidth	458
26.2.6.4. useProcessMsgBackground	459
26.2.6.5. processMsgBackground	459
26.2.6.6. processMsgType	459
26.2.6.7. processMsgFrameURL	459
26.2.7. date	459
26.2.7.1. maxYear	459
26.2.7.2. minYear	459
26.2.7.3. enable	460
26.2.7.4. autoupdateinterval	460
26.2.8. pageFaultMsg	460
26.2.8.1. pageFaultMsg	460
26.2.9. cloneTab	460
26.2.9.1. cloneTab	460
26.2.10. editor	461
26.2.10.1. version	461
26.2.11. fusionchart	461
26.2.11.1. version	461
26.2.12. encoding	461
26.2.12.1. byteCheckEncoding	461
26.2.13. encodeParameter	461
26.2.13.1. value	461
26.2.13.2. fallback	462
26.2.13.3. onlyParam	462

26.2.13.4. key	462
26.2.14. language	462
26.2.14.1. language	462
26.2.14.2. useLanguagePack	462
26.2.14.3. multilanguage	462
26.2.14.3.1. lang	463
26.2.14.3.2. file	463
26.2.15. pageURLprefix	463
26.2.15.1. pageURLprefix	463
26.2.15.2. type	463
26.2.15.3. ignoreW2xPath	463
26.2.16. escape	464
26.2.16.1. escape	464
26.2.17. requirejs	464
26.2.17.1. timeout	464
26.3 Common	464
26.3.1. Project UDC	464
26.3.1.1. Project UDC	465
26.3.1.1.1. name	465
26.3.1.1.2. src	465
26.3.2. MSA	465
26.3.2.1. MSA Server	465
26.3.2.1.1. name	465
26.3.2.1.2. url	465
26.3.2.1.3. w2xHome	465
26.3.2.2. MSA UDC	466
26.3.2.2.1. MSA Server	466
26.3.2.2.2. name	466
26.3.2.2.3. src	466
26.3.3. UDC	466
26.3.3.1. Global UDC	466
26.3.3.1.1. name	466
26.3.3.1.2. src	466
26.3.3.1.3. type	467
26.4 dev	467
26.4.1. debug	467
26.4.1.1. debug	467
26.4.1.2. remoteConsole	467
26.4.1.3. console	467

26.4.1.4.	errorConsole	467
26.4.1.5.	skipLog	468
26.4.1.6.	skipCrossOrigin	468
26.4.1.7.	debugKey	468
26.4.1.8.	debugMenu	468
26.4.1.9.	fiddle	468
26.4.2.	scriptLoading	469
26.4.2.1.	merge	469
26.4.3.	exceptionHandler	469
26.4.3.1.	exceptionHandler	469
26.5	Js	469
26.5.1.	script	469
26.5.1.1.	initScriptSupport	469
26.5.1.2.	initScript	470
26.5.1.3.	postScriptSupport	470
26.5.1.4.	postScript	470
26.5.1.5.	external	470
26.5.1.5.1.	module	470
26.5.1.6.	jquery	470
26.6	UI/UX	471
26.6.1.	반응형/적응형	471
26.6.1.1.	media	471
26.6.1.1.1.	name	471
26.6.1.1.2.	minWidth	471
26.6.1.1.3.	maxWidth	471
26.6.2.	stylesheet	471
26.6.2.1.	enable	471
26.6.2.2.	import	472
26.6.2.3.	earlyImportList	472
26.6.3.	mobile	472
26.6.3.1.	preventTouch	472
26.6.3.2.	hideAddressBar	472
26.6.3.3.	bindMobileMouseEvent	472
26.6.4.	html	473
26.6.4.1.	postDrawMode	473
26.6.4.2.	parseHtml	473
26.6.4.3.	emptyTag	473
26.6.4.4.	visibleHelper	473
26.6.5.	tooltip	474

26.6.5.1. hideTooltipDisabledComp	474
26.6.6. itemTable	474
26.6.6.1. allValue	474
26.7 Comm	474
26.7.1. submission	474
26.7.1.1. processMSg	474
26.7.1.2. hideProcessMsg	475
26.7.1.3. errorHandler	475
26.7.1.4. showSubmissionTime	475
26.7.1.5. customHandler	475
26.7.1.6. action	475
26.7.1.7. mode	475
26.7.1.8. method	476
26.7.1.9. mediatype	476
26.7.1.10. timeout	476
26.7.1.11. encoding	476
26.7.1.12. preSubmitFunction	476
26.7.1.13. callbackSubmitFunction	477
26.7.1.14. submitErrorHandler	477
26.7.1.15. submitErrorHandlerMode	477
26.7.1.16. externalHandler	477
26.7.1.17. externalResponseHandler	477
26.7.1.18. requestID	478
26.7.1.19. makeGlobalObject	478
26.7.1.20. requestHeaderFunction	478
26.7.1.21. abortTrigger	478
26.7.1.22. useModal	478
26.7.1.23. customHeader	478
26.7.1.24. requestJSONHandler	479
26.7.2. ajax	479
26.7.2.1. externalHandler	479
26.7.3. workflow	479
26.7.3.1. processMSg	479
26.7.3.2. makeGlobalObject	479
26.7.4. hybrid	479
26.7.4.1. submissionURL	479
26.8 Data	480
26.8.1. dataList	480
26.8.1.1. removeDummyRowStatus	480

26.8.1.2.	removeDataMatch	480
26.8.1.3.	decreaseGetRangeXML	481
26.8.1.4.	excelErrorHandler	481
26.8.2.	dataMap	481
26.8.2.1.	jsonArray	481
26.9	Resource	481
26.9.1.	cache	481
26.9.1.1.	cache	481
26.9.1.2.	postfix	482
26.9.1.3.	scriptPostfix	482
26.9.2.	engineCache	482
26.9.2.1.	enable	482
26.9.2.2.	compression	482
26.9.2.3.	postfix	482
26.9.3.	engineLoadingModule	483
26.9.3.1.	ie	483
26.9.3.2.	chrome	483
26.9.3.3.	moz	483
26.9.3.4.	safari	483
26.9.3.5.	opera	484
26.9.3.6.	android	484
26.9.3.7.	iphone	484
26.9.4.	userAgentPattern	484
26.9.4.1.	xPathParser	484
26.9.4.2.	serializer	485
26.9.4.3.	URLEncoder	485
26.10	notRecommended	485
26.10.1.	notRecommended	485
26.10.1.1.	warningSync	485
26.11	컴포넌트 (config 전용)	485
26.11.1.	Forms	486
26.11.1.1.	autoComplete	486
26.11.1.1.1.	overrideKey	486
26.11.1.2.	checkcombobox	486
26.11.1.2.1.	overrideKey	486
26.11.1.3.	inputCalendar	487
26.11.1.3.1.	calendarDisplayFormat[yearMonthDateHour]/@defaultValue	487
26.11.1.3.2.	calendarDisplayFormat[yearMonthDateTimeSec]/@defaultValue	487
26.11.1.3.3.	calendarDisplayFormat[yearMonthDateTime]/@defaultValue	487

26.11.1.3.4.	calendarDisplayFormat[yearMonthDate]/@defaultValue	487
26.11.1.3.5.	calendarDisplayFormat[yearMonth]/@defaultValue	487
26.11.1.3.6.	calendarDisplayFormat[year]/@defaultValue	487
26.11.1.3.7.	displayFormat[yearMonthDateHour]/@defaultValue	487
26.11.1.3.8.	displayFormat[yearMonthDateTimeSec]/@defaultValue	488
26.11.1.3.9.	displayFormat[yearMonthDateTimeTime]/@defaultValue	488
26.11.1.3.10.	displayFormat[yearMonthDate]/@defaultValue	488
26.11.1.3.11.	displayFormat[yearMonth]/@defaultValue	488
26.11.1.3.12.	displayFormat[year]/@defaultValue	488
26.11.1.3.13.	ioFormat[yearMonthDateHour]/@defaultValue	488
26.11.1.3.14.	ioFormat[yearMonthDateTimeSec]/@defaultValue	488
26.11.1.3.15.	ioFormat[yearMonthDateTime]/@defaultValue	488
26.11.1.3.16.	ioFormat[yearMonthDate]/@defaultValue	489
26.11.1.3.17.	ioFormat[yearMonth]/@defaultValue	489
26.11.1.3.18.	ioFormat[year]/@defaultValue	489
26.11.1.3.19.	placeholder[yearMonthDateHour]/@defaultValue	489
26.11.1.3.20.	placeholder[yearMonthDateTimeSec]/@defaultValue	489
26.11.1.3.21.	placeholder[yearMonthDateTime]/@defaultValue	489
26.11.1.3.22.	placeholder[yearMonthDate]/@defaultValue	489
26.11.1.3.23.	placeholder[yearMonth]/@defaultValue	490
26.11.1.3.24.	placeholder[year]/@defaultValue	490
26.11.1.4.	selectBox	490
26.11.1.4.1.	mobileRenderType	490
26.11.1.4.2.	overrideKey	490
26.11.2.	Grid	490
26.11.2.1.	GridView	490
26.11.2.1.1.	colIdToColIndex	490
26.11.2.1.2.	column/chooseOptionLabel	491
26.11.2.1.3.	column/displayFormat	491
26.11.2.1.4.	column/editType	491
26.11.2.1.5.	column/textAlign	491
26.11.2.1.6.	column[calendar]/dateValidCheck	491
26.11.2.1.7.	column[calendar]/ioFormat[yearMonthDateTimeSec]/@defaultValue	491
26.11.2.1.8.	column[calendar]/ioFormat[yearMonthDateTime]/@defaultValue	491
26.11.2.1.9.	column[calendar]/ioFormat[yearMonthDate]/@defaultValue	492
26.11.2.1.10.	column[calendar]/ioFormat[yearMonth]/@defaultValue	492
26.11.2.1.11.	column[expression]/NaNFormat	492
26.11.2.1.12.	column[text]/dataType[euro]/customModelFormatter	492
26.11.2.1.13.	column[text]/dataType[euro]/customModelUnformatter	492

26.11.2.1.14.	column[textarea]/keepCellClass	492
26.11.2.1.15.	dataType[date]/@displayFormat	493
26.11.2.1.16.	excel/download/errorHandler	493
26.11.2.1.17.	getColumnVisible/@useRealColIndex	493
26.11.2.1.18.	header/column/hideCustomFilter	493
26.11.2.1.19.	header/column/useFilter	493
26.11.2.1.20.	mainZIndex	493
26.11.2.1.21.	useDataDragDropGuideLine	493
26.11.2.1.22.	xScrollZIndex	494
26.11.2.1.23.	yScrollZIndex	494
26.11.3.	Container	494
26.11.3.1.	group	494
26.11.3.1.1.	adaptive/trimText	494
26.11.3.1.2.	applyScopeHeaders	494
26.11.3.2.	TabControl	494
26.11.3.2.1.	overrideKey	494
26.11.3.3.	WindowContainer	495
26.11.3.3.1.	getWindow/@searchTarget	495
26.12	Deprecated	495
26.12.1.	stylesheet	495
26.12.1.1.	stylesheet	495
26.12.1.2.	relativePath	495
26.12.1.3.	removeDefaultClass	495
26.12.2.	docType	496
26.12.2.1.	docType	496
26.12.3.	mode	496
26.12.3.1.	mode	496
26.12.4.	debug	496
26.12.4.1.	fixErrorLineNumber	496
26.12.5.	선행 처리	496
26.12.5.1.	systemPreProcessor	496
26.12.5.2.	businessPreProcessor	497
26.12.6.	컴포넌트	497
26.12.6.1.	웹스퀘어 컴포넌트	497
26.12.6.1.1.	id	497
26.12.6.1.2.	namespace	497
26.12.6.1.3.	className	497
26.12.6.1.4.	value	497
26.12.7.	UDC	498

26.12.7.1. SP1 UDC 스펙 지원	498
26.12.8. 데이터 변환	498
26.12.8.1. xml2jsonFunction	498
26.12.8.2. json2xmlFunction	498
26.12.8.3. removeXmlNameSpace	498
26.12.9. Instance	498
26.12.9.1. copyChildrenNodes	499
26.12.9.2. removeInstanceNode	499
26.12.9.3. forcedValueSetting	499
26.12.10. checkUpdate	499
26.12.10.1. enable	499
26.12.10.2. mode	499
26.12.10.3. interval	500
26.12.11. imageCache	500
26.12.11.1. enable	500
26.12.11.2. version	500
26.12.11.3. serverUrl	500
26.12.11.4. hostUrl	500
26.12.12. localStorage	501
26.12.12.1. defer	501
26.12.13. SPA	501
26.12.13.1. onpageunload	501
26.12.13.2. scriptCache	501
26.12.13.3. autoReload	501
26.12.13.4. count	502
26.12.13.5. editorCount	502
26.12.13.6. fusionchartCount	502
26.12.13.7. userFunction	502
26.12.14. errorPage	502
26.12.14.1. errorPage	502
26.12.14.2. E0	502
26.12.14.3. E400	503
26.12.14.4. E401	503
26.12.14.5. E402	503
26.12.14.6. E403	503
26.12.14.7. E404	503
26.12.14.8. E405	503
26.12.14.9. E500	503
26.12.14.10. E501	504

26.12.14.11.	E502	504
26.12.14.12.	E503	504
26.12.14.13.	E504	504
26.12.14.14.	Else	504
26.12.15.	url	504
26.12.15.1.	DocumentRoot	504
26.12.15.2.	logMsgMobileURL	505
26.12.15.3.	logMsgURL	505
26.12.15.4.	debugMsgURL	505
26.12.15.5.	errorMsgURL	505
26.12.15.6.	viewSourceURL	505
26.12.15.7.	viewCollectionURL	505
26.12.15.8.	viewInstanceURL	505
26.12.15.9.	fiddleURL	506
26.12.15.10.	printRangeURL	506
26.12.15.11.	redirectUrl	506
26.12.16.	etc	506
26.12.16.1.	processMsgTitle	506
26.12.16.2.	parseChild	506
26.12.16.3.	scriptEncoding	506
26.12.16.4.	showLogPopup	507
26.12.16.5.	license	507
26.12.16.6.	requestHeaderFunction	507
26.12.16.7.	preserveWhiteSpace	507
26.12.16.8.	codebase	507
26.12.16.9.	globalExposure	507
26.12.16.10.	layerForceClose	508
26.12.16.11.	blurAll	508
26.12.16.12.	nextTabFunction	508
26.12.16.13.	loadErrorHandler	508
26.12.16.14.	useAppConfig	508
26.12.16.15.	domainFrameURL	508
26.12.16.16.	imageUrlPrefix	509
26.12.16.17.	clearMemoryCloseChildPopup	509
26.12.16.18.	serverTimeURL	509
26.12.16.19.	modal	509
26.12.16.20.	forcedValueSetting	509
26.12.16.21.	parseHtml	509
26.12.16.22.	postDrawMode	510

26.13 Source	510
27. 서버 설정 (server.config.xml)	511
27.1 개요	511
27.2 overview	512
27.2.1. encoding	512
27.2.1.1. default_encoding	512
27.2.1.2. encoding	512
27.2.2. engine	512
27.2.2.1. engineType	512
27.2.2.2. etag	516
27.2.2.3. wqDefiner	516
27.2.3. date	517
27.2.3.1. encoding	517
27.3 upload	517
27.3.1. file upload	517
27.3.1.1. support	517
27.3.1.2. useFileUpload	517
27.3.1.3. encoding	517
27.3.1.4. baseDir	518
27.3.1.5. subDir	518
27.3.1.5.1. name	519
27.3.1.5.2. path	519
27.3.1.6. firstfilePostFix	519
27.3.1.7. duplicatedFilePostFix	519
27.3.1.8. sequence	519
27.3.1.9. maxUploadSize	519
27.3.1.10. subDirMaxUpload	520
27.3.1.10.1. name	520
27.3.1.10.2. path	520
27.3.1.11. allowedExtension	520
27.3.1.12. deniedExtension	520
27.3.1.13. uploadMode	520
27.3.1.14. folderName	521
27.3.1.15. removeKeyPath	521
27.3.1.16. fileDefiner	521
27.3.1.17. decrypt	522
27.3.1.18. decryptTempDir	523
27.3.2. image upload	523
27.3.2.1. baseDir	523

27.3.2.2.	multiSubDir	523
27.3.2.3.	verifyHeader	524
27.3.2.4.	allowedExtension	524
27.3.2.5.	deniedExtension	524
27.3.2.6.	imageDefiner	524
27.3.2.7.	subDir (deprecated)	524
27.4	excel	525
27.4.1.	excel upload	525
27.4.1.1.	maxRowCount	525
27.4.1.2.	maxCellCount	525
27.4.1.3.	maxUploadSize	525
27.4.1.4.	loadingMode	525
27.4.1.5.	saxDecimal	525
27.4.1.6.	decrypt	526
27.4.1.7.	temp	526
27.4.1.8.	dataConvertor	527
27.4.1.9.	validExtension	527
27.4.1.10.	cellDataConvertor	528
27.4.1.11.	applyExcelFormat	528
27.4.1.12.	dataRefine	528
27.4.2.	excel download	528
27.4.2.1.	headerColor	528
27.4.2.2.	bodyColor	529
27.4.2.3.	footerColor	529
27.4.2.4.	subtotalColor	529
27.4.2.5.	maxRowCount	530
27.4.2.6.	allowedExtension	530
27.4.2.7.	rowAccessWindowSize	530
27.4.2.8.	encrypt	530
27.4.2.9.	tempDir	531
27.4.2.10.	dataConvertor	531
27.4.2.11.	dataXMLProvider	532
27.4.2.12.	baseDir	532
27.4.2.13.	dataTypeDate	533
27.4.3.	mobile download	533
27.4.3.1.	keyMode	533
27.4.3.2.	preFix	533
27.4.3.3.	userAgent	534
27.4.3.4.	definer	534

27.5	csv	534
27.5.1.	general information	534
27.5.1.1.	encoding	534
27.5.1.2.	useBOM	534
27.5.1.3.	opencsv	534
27.5.2.	csv upload	535
27.5.2.1.	maxUploadSize	535
27.5.2.2.	decrypt	535
27.5.2.3.	tempDir	535
27.5.2.4.	checkEncode	535
27.5.3.	csv download	535
27.5.3.1.	keyMode	535
27.5.3.2.	prefix	536
27.5.3.3.	userAgent	536
27.5.3.4.	dataConvertor	536
27.5.3.5.	encrypt	537
27.5.4.	mobile download	539
27.5.4.1.	definer	539
27.6	i18n	539
27.6.1.	general information	539
27.6.1.1.	baseDir	539
27.6.1.2.	defaultLocale	539
27.6.1.3.	domain	539
27.6.1.3.1.	name	539
27.6.1.3.2.	locale	540
27.6.1.4.	localeParam	540
27.6.1.5.	storageType	540
27.6.1.6.	baseUrl	540
27.6.1.7.	cache	540
27.6.2.	https	540
27.6.2.1.	keystorePath	541
27.6.2.2.	keystoreType	541
27.6.2.3.	keystoreaPass	541
27.6.3.	xmlInfo	541
27.6.3.1.	web2fileCacheImpl	541
27.6.3.2.	excludeList	541
27.6.4.	messageInfo	542
27.6.4.1.	storageType	542
27.6.4.2.	baseDir	542

27.6.4.3. <code>messageImpl</code>	542
27.6.4.4. <code>resource</code>	542
27.6.5. <code>obfuscate</code>	542
27.6.5.1. <code>encodeParameter</code>	542
27.6.5.2. <code>fileNameBase64Decode</code>	543
27.7 <code>Source</code>	543

Part VI. 심화 가이드 545

28. 다국어 설정 가이드	547
28.1 클라이언트 다국어	547
28.1.1. 언어 파일	547
28.1.2. <code>config.xml</code>	550
28.1.3. 텍스트 다국어 (<code>localeRef</code>)	551
28.1.3.1. <code>localeRef</code> 속성 값이 언어 파일에 정의되지 않은 경우	553
28.1.4. 선택 항목 다국어 (<code>useLocaleItem</code>)	555
28.1.5. 선택 항목 및 <code>chooseOption</code> 다국어 (<code>useItemLocale</code> & <code>chooseOptionLabelLocaleRef</code>)	557
28.1.6. 툴팁 다국어 (<code>tooltipLocaleRef</code>)	559
28.1.7. <code>GridView</code> 다국어	560
28.1.7.1. 행 번호 및 행 상태 컬럼의 헤더 다국어	560
28.1.7.2. 헤더 툴팁 다국어	561
28.2 서버 (WAS) 다국어	562
28.2.1. Properties 파일 사용	562
28.2.1.1. 다국어 프레임워크	562
28.2.1.2. Property	563
28.2.1.3. <code>config.xml</code>	564
28.2.1.4. Function 작성	564
28.2.1.5. <code>websquare.xml</code>	565
28.2.1.6. WebSquare 화면 파일	566
28.2.1.7. WebSquare5 Studio에서 보기	566
28.2.2. 서버 다국어 DB 사용	567
28.2.2.1. <code>websquare.xml</code> 변경	568
28.2.2.2. <code>com.inswave.common.i18n.LoadI18nMessageImpl</code> 생성/등록	569
28.2.2.3. DB 테이블 생성 / 샘플 데이터 등록 (오라클용)	569
28.2.2.4. <code>config.xml</code> 변경	569
28.2.2.5. <code>i18NCommon.js</code> 등록	570
28.2.2.6. 웹스퀘어 화면 파일	570
28.2.2.7. <code>locale</code> 설정	571
29. 동적 이벤트 가이드	573

29.1	<code>component.bind(type, function);</code>	573
29.2	<code>component.unbind([type], [function]);</code>	573
29.3	<code>component.trigger(type, [array]);</code>	574
30.	모바일 웹 개발 가이드	575
30.1	개요	575
30.2	주의사항	575
30.2.1.	여러가지 해상도 대응은 viewport나 css3의 @media를 사용	575
30.2.2.	마우스나 키보드 사용을 전제하지 말 것	576
30.2.3.	스크롤 사용시 주의 사항	576
30.2.4.	이벤트 사용시 주의	576
30.2.5.	파일 첨부 제한	576
30.2.6.	플러그인 기술 사용금지	576
30.2.7.	모바일 화면 구성 시 iframe 사용 금지	577
30.2.8.	팝업 사용시 주의 사항	577
30.2.9.	사용금지 컴포넌트	577
30.2.10.	테스트 된 모바일 OS 및 브라우저	577
30.3	모바일 개발시 사용가능 컴포넌트	578
30.3.1.	권장	579
30.3.2.	사용 가능	580
30.3.3.	비권장	580
30.3.4.	사용 금지	581
30.4	모바일 접근성	582
30.4.1.	FlipToggle	582
30.4.1.1.	accessibility	582
30.4.2.	ScrollView	583
30.4.3.	Switch	583
30.4.3.1.	accessibility	583
31.	GridView의 senseReader 속성 사용 시 유의 사항	585
31.1	성능 비교 (예시)	585
31.1.1.	Heap 메모리	585
31.1.1.1.	<code>senseReader="false"</code>	586
31.1.1.2.	<code>senseReader="true"</code>	587
31.1.2.	DOM 개체 수	587
31.1.2.1.	<code>senseReader="false"</code>	587
31.1.2.2.	<code>senseReader="true"</code>	587
31.1.3.	<code>senseReader</code> 메모리 사용량	588
31.1.3.1.	<code>senseReader="false"</code>	588
31.1.3.2.	<code>senseReader="true"</code>	589
31.1.4.	결론	589

31.2 렌더링 방식 (<code>drawType</code>)	590
31.2.1. virtual 렌더링 (<code>drawType="virtual"</code>)	590
31.2.2. native 렌더링 (<code>drawType="native"</code>)	592
32. TabControl의 탭 콘텐츠 렌더링 가이드	595
32.1 탭 콘텐츠 렌더링	595
32.2 <code>alwaysDraw</code>	595
32.2.1. <code>alwaysDraw="true"</code>	595
32.2.2. <code>alwaysDraw="false"</code> (기본 값)	597
32.2.2.1. <code>frameMode="wframe"</code>	598
32.2.2.2. <code>frameMode="wframePreload"</code>	600
32.2.2.2.1. 탭 콘텐츠 렌더링과 객체 접근	601
32.3 <code>frameMode="wframe"</code>	602
32.4 <code>frameMode="wframePreload"</code>	603
32.4.1. 탭 콘텐츠는 렌더링되지 않았지만 컴포넌트에 접근은 가능한 상황	606
32.4.1.1. 예외 상황	608
32.4.1.2. 대안	609
32.5 주의할 점	609
32.5.1. 사례 1	610
32.5.2. 사례 2	611
32.6 결론	614
33. 서버 프레임워크 연동 가이드	615
33.1 일반 원칙	615
33.1.1. 표준 웹을 준수하는 데이터 구조와 통신 방식	615
33.1.2. 웹스퀘어와 서버 간 데이터 변환	615
33.1.3. 웹스퀘어 Adapter	616
33.2 전자정부 프레임워크 연동	616
33.2.1. 배포	617
33.2.1.1. WAR 배포	617
33.2.1.2. Exploded 배포	618
33.2.2. 어플리케이션 설정	618
33.2.2.1. 인코딩	618
33.2.2.2. Auto Deploy	619
33.2.3. DataSource 설정	619
33.2.4. 로컬 개발 환경 설정	620
33.2.4.1. WAS	620
33.2.4.2. <i>Web.xml</i>	622
33.2.4.3. Spring config	623
33.2.4.3.1. WebSquareAdapter 사용	623
33.2.4.3.2. MessageConverter 사용	625

33.2.4.4. Context ROOT	627
33.2.4.4.1. 주의	628
33.2.4.5. Maven	629
33.2.4.6. Adapter 추가	631
33.2.4.7. DB Persistence	631
33.2.5. 화면 구동	636
33.2.6. websquare.xml 로그	642
33.2.7. 웹스퀘어5 데모	643
33.3 SpringBoot Jackson 연동	645
33.3.1. 개발 환경 구성	645
33.3.2. SpringBoot와 WebSquare 연계	646
33.4 Spring 연동	653
33.5 스트러츠 연동	654
33.5.1. 개요	654
33.5.2. 구성	654
33.5.2.1. RequestProcessor 구현	654
33.5.2.2. Action Class 구현	658
33.5.2.3. Action Form Class 구현	659
33.5.2.4. struts-config.xml 설정	660
33.5.2.5. WebSquare XML 구현	661
33.6 WebSquare JSON 어댑터 연동	662
33.6.1. Data Adapter 개요	662
33.6.2. 역할 및 기능	662
33.6.3. 적용 방법	663
33.6.3.1. 기본 API	663
33.6.3.1.1. websquare.system.adapter.RequestAdapter	663
33.6.3.1.2. websquare.system.adapter.ResponseAdapter	663
33.6.3.2. Servlet	664
33.6.3.2.1. websquare.system.adapter.servlet.DefaultAdapterServlet	664
33.6.3.3. Filter	666
33.6.3.3.1. websquare.system.adapter.filter.AdapterFilter	666
33.6.4. 지원 데이터 Format	668
33.6.4.1. Client Side(통신 Data Format)	670
33.6.4.1.1. XML	670
33.6.4.1.2. JSON	670
33.6.4.2. Server Side	671
33.6.4.2.1. Map	671
33.6.4.2.2. VO (Beans)	672
33.6.5. VO(Beans) Class Type 지정	675

33.6.5.1.	클라이언트에서 VO Class Type 전달	675
33.6.5.2.	서버에서 Map 객체의 형태로 VO Class Type 전달	676
33.6.5.3.	서버에서 Annotation으로 VO Class Type 정의	676
33.6.6.	Custom Converter 생성	677
33.6.6.1.	Custom Converter 적용 방법	677
33.6.6.1.1.	Custom RequestConverter	677
33.6.6.1.2.	Custom ResonseConverter	679
33.6.7.	Configuration	681
34.	성능 개선 가이드	685
34.1	코드 분석	685
34.1.1.	전역 변수	685
34.1.2.	객체	686
34.1.2.1.	API	686
34.1.2.2.	수식	688
34.1.3.	객체 생성	689
34.1.4.	스크립트 수행 시간 최소화	689
34.1.5.	가독성	690
34.1.6.	데이터 형태에 적합한 수식 사용	693
34.1.7.	변수 선언 및 참조 방식	694
34.1.8.	팝업 파라미터 전달	695
34.1.9.	실행 시간 최소화	696
34.1.10.	DataCollection 관련 API의 반복 수행	698
34.2	주요 확인 사항	699
34.2.1.	기본	699
34.2.2.	속도	699
34.2.3.	메모리	700
34.2.4.	백화 현상	700
34.3	메모리 관리	701
34.3.1.	메모리 누수 방지	701
34.3.1.1.	WebSquare 공개 API 사용	701
34.3.1.2.	jQuery의 제한적 사용	701
34.3.1.3.	최상위 scope에 함수 및 변수 선언 제한	702
34.3.2.	Submission	703
34.3.2.1.	업무화면에서 Submission 생성	703
34.3.2.2.	Async 통신 권장 (Sync 모드 경고)	704
34.3.3.	<code>\$p.ajax</code>	705
34.3.3.1.	응답 객체의 직접 할당 금지	705
34.3.3.2.	Closure 내부에서 객체 참조	707
34.3.4.	DataCollection	709

34.3.4.1. 생성	709
34.3.4.2. 초기화	710
34.3.5. 변수명	711
34.3.6. 팝업	711
34.3.6.1. 팝업의 Lifecycle	711
34.3.6.2. 타 Frame으로 객체 전달	711
34.3.7. 객체 제어	713
34.3.8. 전역 변수 선언	714
34.3.9. 공통 코드 조회	715
34.3.10. 웹사이트 성능 최적화	715
34.3.11. 기타 주의할 내용	716
34.3.11.1. WFrame	716
34.3.11.2. TabControl	716
34.3.11.3. 동기 요청 최소화	717
34.4 성능 관리	717
34.4.1. 초기 화면 네트워크 튜닝	717
34.4.1.1. 리소스 로딩	717
34.4.1.2. 네트워크 문제 진단	719
34.4.1.2.1. Queue에 저장되거나 지연된 시리즈	719
34.4.1.2.2. 느린 TTFB	719
34.4.1.2.3. 처리 용량 확보	720
34.4.1.3. 해결 방안	720
34.4.2. jQuery 셀렉터 사용	720
34.4.2.1. id 셀렉터	721
34.4.2.1.1. 사용 예 (1)	721
34.4.2.1.2. 사용 예 (2)	721
34.4.2.2. 태그 셀렉터	721
34.4.2.3. 메서드 체인	722
34.4.2.4. 셀렉터 캐싱	722
34.4.2.5. 셀렉터 범위 지정	722
34.4.3. 재귀 호출과 반복문 튜닝	723
34.4.3.1. DataCollection 제어	723
34.4.3.1.1. 반복적인 DataCollection 변경	723
34.4.3.1.2. 여러행 추가시 Array를 이용하여 처리	724
34.4.3.1.3. 반복문 내에 사용시 이벤트 중지 후 처리	725
34.4.3.2. GridView 튜닝	726
34.4.4. 성능 개선을 위한 추가 기능	728
34.4.4.1. SPA	728
34.4.4.2. WFrame	728

34.5 성능 저하 원인 및 해결 방안	728
34.5.1. 웹 서버 설정	728
34.5.1.1. JSON, JS, XML, CSS 다운로드 속도	728
34.5.1.2. 정적인 자원에 대한 Caching 설정 오류	728
34.5.1.3. SSL Handshake	729
34.5.2. WebSquare 설정	729
34.5.2.1. Cache 설정 및 압축 설정	729
34.5.2.2. engineType 설정	729
34.5.3. 통신 최적화	731
34.5.3.1. 화면 프리징	731
34.5.3.2. HTTP 오류 발생 리소스	731
34.5.3.3. HTTP 1.1 Max Connection 초과시	731
34.5.3.4. 동시에 다수의 HTTP 요청	732
34.5.3.5. 다수의 JS 파일 요청	732
34.5.4. 데이터 튜닝	732
34.5.4.1. XML 방식의 대량 데이터 사용	732
34.5.5. 리소스 중복 로딩	733
34.5.5.1. CSS, JS 중복 로딩 오류	733
34.5.6. 화면 튜닝	733
34.5.6.1. TabControl이 여러 Tab을 포함	733
34.5.6.2. 화면 로딩 속도	734
34.5.6.3. 외부 컴포넌트 수	734
34.5.6.4. 그리드 엑셀 업/다운로드	734
34.5.6.5. 그리드 컬럼 개수	734
34.5.6.6. 그리드 대량 데이터	734
34.5.6.7. IE 하위 호환성	734
34.5.6.8. 성능을 고려하지 않은 개발	734
34.5.6.9. IFrame 개수	735
34.5.6.10. WFrame 개수	735
34.5.6.11. 화면당 컴포넌트 수	735
34.5.7. IE 브라우저에서의 렌더링 속도	735
34.5.7.1. 속도 저하 원인	735
34.5.7.2. 확인 방법	735
34.5.7.3. 해결 방법	737
34.5.7.3.1. config.xml	737
34.5.7.3.2. 클라이언트	738
34.6 WebSquare 로그 분석	738
34.6.1. 로그 조회	738
34.6.2. 로그 구조	743

34.6.2.1. Step 로그	743
34.6.2.2. 오류 로그	744
34.7 점검 목록	746
34.7.1. 일반	746
34.7.1.1. JavaScript	746
34.7.1.2. 통신	747
34.7.1.3. 네트워크	747
34.7.1.4. CSS	748
34.7.1.5. jQuery	748
34.7.1.6. 메모리 누수	749
34.7.1.7. 기타	750
34.7.2. 컴포넌트	751
34.7.2.1. 일반	751
34.7.2.2. Generator	751
34.7.2.3. GridView	751
34.7.2.4. IFrame	752
34.7.2.5. Submission	753
34.7.2.6. TabControl	753
34.7.2.7. 팝업	753
34.7.3. 데이터	754
34.7.4. 외부 연동	754
35. 파일 업로드/다운로드 가이드	757
35.1 업로드	757
35.1.1. Upload/MultiUpload	757
35.1.1.1. 기본 설정	757
35.1.1.2. fileDefiner 구현	759
35.1.1.3. decrypt 구현	761
35.1.2. Editor (이미지)	762
35.1.2.1. 기본 설정	762
35.1.2.2. imageDefiner 구현	763
35.1.3. GridView	765
35.1.3.1. Excel	765
35.1.3.1.1. 기본 설정	765
35.1.3.1.2. decrypt 구현	766
35.1.3.1.3. dataConverter 구현	767
35.1.3.1.4. 위변조 방지 ¹	768

35.1.3.2. CSV	769
35.1.3.2.1. 최대 업로드 사이즈 ²	770
35.1.4. Upload시 파일명 처리	771
35.1.4.1. 중복 방지용 날짜 추가	771
35.1.4.1.1. 설정 설명	771
35.1.4.1.2. 처리 방법	771
35.1.4.2. 업로드 경로를 업무별로 분리하는 방법	772
35.1.4.2.1. 처리 방법	772
35.1.4.3. 사용자 정의 fileDefiner 설정	773
35.1.4.3.1. 처리 방법	773
35.1.5. 업로드 이미지 파일명 (Editor)	773
35.2 다운로드	774
35.2.1. <code>\$p.download</code> (Excel 및 기타 파일)	774
35.2.2. <code>WebSquare.net.download</code>	777
35.2.3. GridView	779
35.2.3.1. Excel	779
35.2.3.1.1. 기본 설정	779
35.2.3.1.2. encrypt 구현	780
35.2.3.2. CSV	784
36. Workflow 가이드	787
36.1 개요	787
36.2 생성 방법	787
36.3 관련 API	789
37. 비권장 사례 모음	791
37.1 일반	791
37.1.1. 변수명	791
37.1.2. 객체 제어	791
37.1.3. 전역 변수 선언	792
37.1.4. 동기 요청 최소화	793
37.1.5. 최상위 scope에 함수 및 변수 선언 제한	794
37.1.6. 금지 사항	794
37.2 메모리	795
37.3 모바일	798
37.3.1. 여러가지 해상도 대응은 viewport나 css3의 @media를 사용	798
37.3.2. 마우스나 키보드 사용을 전제하지 말 것	799
37.3.3. 스크롤 사용시 주의 사항	799

37.3.4. 이벤트 사용시 주의	799
37.3.5. 파일 첨부 제한	799
37.3.6. 플러그인 기술 사용금지	800
37.3.7. 모바일 화면 구성 시 iframe 사용 금지	800
37.3.8. 팝업 사용시 주의 사항	800
37.3.9. 사용 금지 컴포넌트	800
37.4 네트워크	801
37.4.1. 느린 TTFB	801
37.4.2. 처리 용량 확보	802
37.4.3. 네트워크 문제 해결	802
37.5 기타	802
37.5.1. <code>\$p.ajax</code>	803
37.5.1.1. 응답 객체의 직접 할당 금지	803
37.5.1.2. Closure 내부에서 객체 참조	804
37.5.2. <code>LinkedList</code>	806
37.5.3. <code>GridView</code>	808
37.5.3.1. 조회 건 수	808
37.5.3.2. <code>visibleRowNum</code>	808
37.5.4. 팝업	808
37.5.4.1. 팝업 종류	808
37.5.4.2. 파라미터 전달	809
37.5.5. <code>TabControl</code>	810
37.5.6. <code>WFrame</code>	811
37.5.6.1. <code>WFrame</code> 사용 권장	811
37.5.6.2. <code>class</code> selector 사용	811

문서 이력

2025

문서 변경 내역

일자	내용
2025-04-08	• 스튜디오 ◦ 1.2 PC 요구 사양 업데이트. ◦ 1.3 Eclipse 지원 업데이트. • 엔진 ◦ 3.2 사양 업데이트. ◦ 3.3 앱 실행 클라이언트 사양 업데이트. ◦ 3.4 브라우저 지원 업데이트. • DataCollection ◦ DataList ▪ 12.2.7.2. null 값과 공백("") 값 구분 추가. ◦ DataMap ▪ 12.3.5.2. null 값과 공백("") 값 구분 추가.
2025-02-20	• 예제 파일의 다운로드 링크 업데이트. • 28. 다국어 설정 가이드 ◦ 28.1.7. GridView 다국어 추가. • 스튜디오 서버 ◦ 2.4 Quick Code 생성 추가. • DataCollection ◦ DataList ▪ 12.2.7. 공백 데이터 추가. ▪ 12.2.8. 컬럼 데이터 포맷 (dataList.column - importFormatter) 추가. ◦ DataMap ▪ 12.3.5. 공백 데이터 추가.

2024

문서 변경 내역

일자	내용
2024-12-11	• 스튜디오 서버 ◦ 설치, 실행, 기능 단락 삭제. • Part IV. 컴포넌트 ◦ 22. Frame - PageInherit 삭제. • DataCollection ◦ DataMap - 12.3.4. 바인딩된 컴포넌트 확인 추가.
2024-11-06	• 2. 스튜디오 서버 - 2.2 사양 내용 변경. • 12. DataCollection ◦ 12.2 DataList ▪ 12.2.2. 데이터 타입 자동 변경 추가. ▪ 12.1.5. Studio에서 생성하고 디버깅 메뉴로 확인하기 추가. ▪ 12.1.4. 복사 및 삭제 추가. ▪ 12.2.9. 이벤트 추가. ◦ 12.3 DataMap ▪ 12.3.2. 데이터 타입 자동 변경 추가. • 13. AliasDataCollection 추가. • 14. Submission 내용 전반 보완. • Part IV. 컴포넌트 추가. • Part VI. 심화 가이드 추가.
2024-10-10	• 2. 스튜디오 서버 ◦ 2.2 사양 - Mac OS 추가. ◦ 2.3 점검 - 2.3.1. 설정 추가.
2024-09-20	• 용어 변경 ◦ 웹스퀘어 에이아이 → 웹스퀘어 AI • 2. 스튜디오 서버 ◦ 2.2 사양 - JDK 사양 변경. ◦ Windows 경로 변경. ◦ 2.3 점검 - 계정 정보 변경.
2024-08-14	• 3.6 설치 관련 문제 해결 ◦ "javascript.wq?q=bootloader" → "/javascriptLoader.wq" 변경. • 4. 동작 - 4.2.5. 스크립트 실행 순서 추가. • 5. SPA 추가. • DataCollection ◦ DataList - 12.2.2.2. number 타입 데이터를 string 타입으로 설정 (preserveType) 추가. ◦ DataMap - 12.3.2.2. number 타입 데이터를 string 타입으로 설정 (preserveType) 추가. • 25. 스튜디오 설정 ◦ 25.3 Design Editor - 최신 화면으로 업데이트.

일자	내용
	◦ 25.5 Preview Setting - 최신 화면으로 업데이트. ◦ 25.7 Resource Folder - 최신 화면으로 업데이트.
2024-07-26	• 2. 스튜디오 서버 추가.
2024-07-18	• DataCollection ◦ DataList - 12.2.2.1. string 타입 데이터를 number 타입으로 설정 (keepDataType) 추가. ◦ DataMap - 12.3.2.1. string 타입 데이터를 number 타입으로 설정 (keepDataType) 추가.
2024-06-27	• 1.4 설치 ◦ 1.4.2. Eclipse 플러그인 설치 추가. • 11.2 WebSquare Page Component ◦ 11.2.3. 메소드 추가하기 추가. ◦ 11.2.4. PageFrame 소스로 사용하기 추가.
2024-03-14	정식 버전 발행.
2024-02-29	초안 발행.

Part I. **소개**

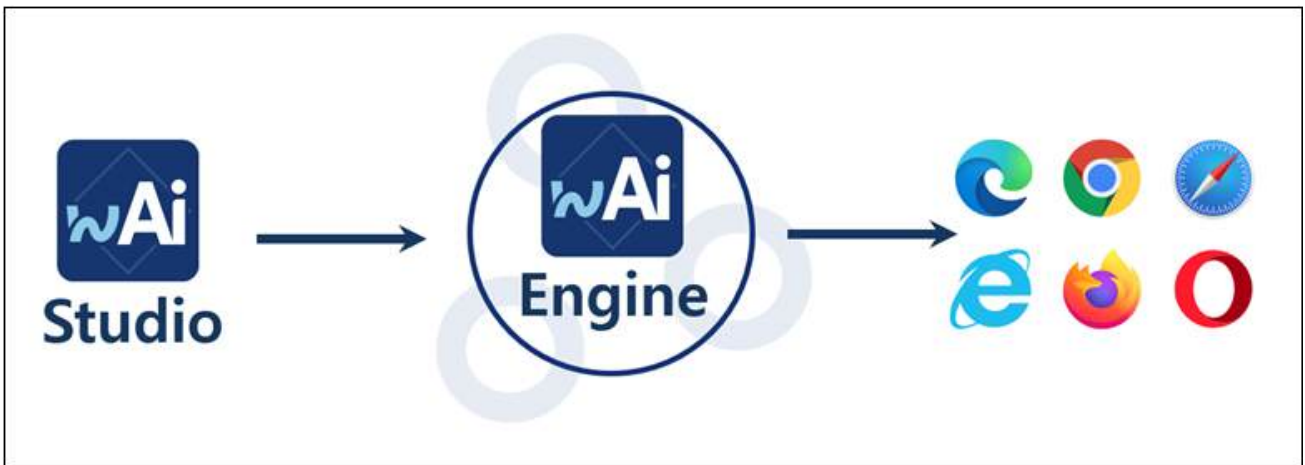
1. 스튜디오

1.1 개요

웹스퀘어 AI는 엔진과 스튜디오(개발도구)로 구성됩니다. 스튜디오는 웹스퀘어 화면 파일을 개발할 수 있는 환경을 제공하고, 엔진은 웹스퀘어 화면 파일을 브라우저에 표시합니다.

웹스퀘어 스튜디오는 WYSIWYG 방식의 통합 개발 도구로, 개발자는 컴포넌트 그리기, 스크립트 추가, 화면 확인, 디버깅을 한번에 수행할 수 있습니다.

그림 1-1 웹스퀘어 AI 구성



1.2 PC 요구 사양

Table 1-1 PC 설치 사양

항목	내용
CPU	• i5 이상
메모리	• 8G 이상
HDD 여유 공간	• 100GB 이상
OS	• Windows 7/8/10 (64-bit) • Mac OS
디스플레이	• 1280 * 1024

1.3 Eclipse 지원

Table 1-2 Eclipse 지원

항목	내용
Eclipse 지원	4.8 (Photon) ~ 2023-09 버전 까지 플러그인 설치 지원. - Eclipse 2020-12 release ~ 버전 부터 최소 jdk 11 버전 이상 동작 가능. - Eclipse 2020-09 해당버전은 버그 확인으로 사용 불가.
JDK 지원	1.8 이상. - 최소 JRE (JDK) 1.8.0_2xx 설치가 필요. - openjdk-1.8.0.242-1.b08x86_64 : 해당 JDK 버전 버그로 인해 이클립스 프로젝트 생성이 안되며 스튜디오 비정상 동작.
전자 정부 프레임워크	eGovFrameDev-3.10.0-64 ~ eGovFrameDev-4.1.0-Win-64bit
Sprint Tools	Spring Tools 4.0.0 ~ Spring Tools 4.20.0



웹스퀘어 스튜디오 최신 버전은 jre 미포함

1.4 설치

1.4.1. Windows용 스튜디오 설치 (인스톨러)

1.4.1.1. 설치 전 준비

(1) 인스톨러

사용자 PC 사양에 맞는 인스톨러를 제공 받으십시오.

(2) Java 1.8

WebSquare Studio는 Java 1.8 이상에서 동작합니다.

사용자 PC에 Java 1.8 이상을 설치하십시오.

(3) 기존 스튜디오 삭제

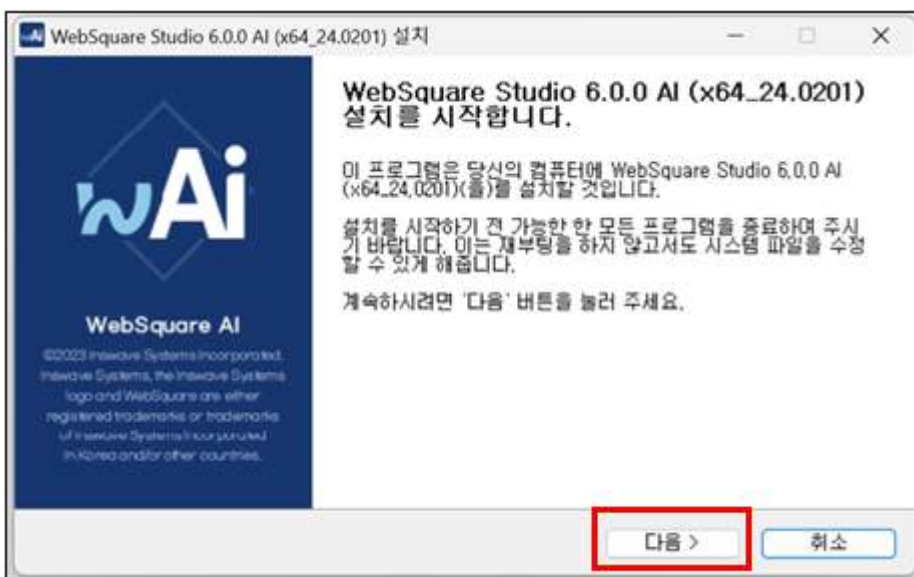
기존에 설치된 스튜디오가 있을 경우, 설치된 스튜디오를 먼저 제거하십시오.

단, workspace 폴더는 삭제하지 말고 계속 사용하십시오.

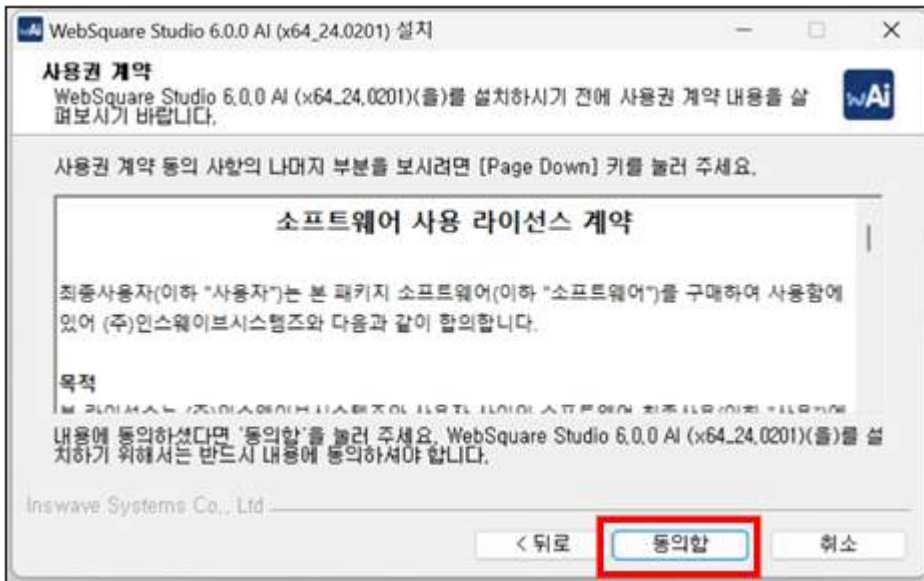
그림 1-2 YouTube 동영상 (<https://youtu.be/sHSvMKUvME4>)

1.4.1.2. 설치 방법

1. 인스톨러를 실행합니다.
2. 다음을 클릭합니다.



3. 동의함을 클릭합니다.

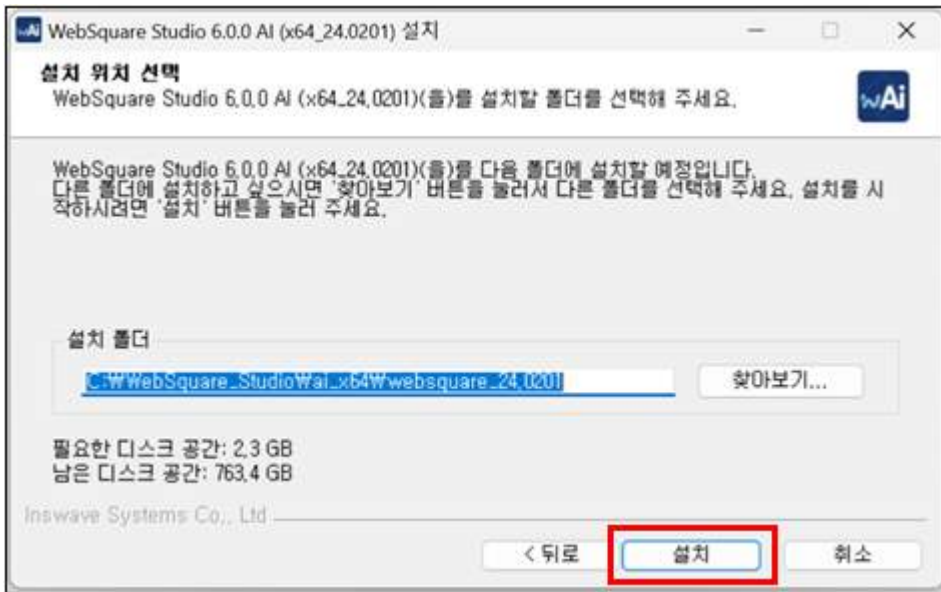


4. WebSquare Studio는 Java 1.8 이상에서 동작합니다. Java 경로를 bin 폴더까지 지정한 후 다음을 클릭합니다.

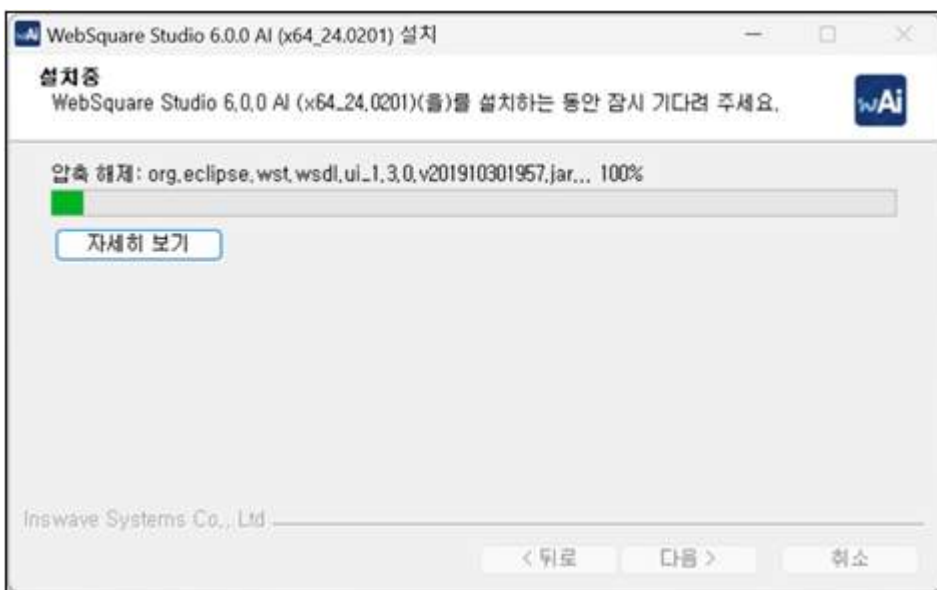


5. 설치 경로를 지정합니다.

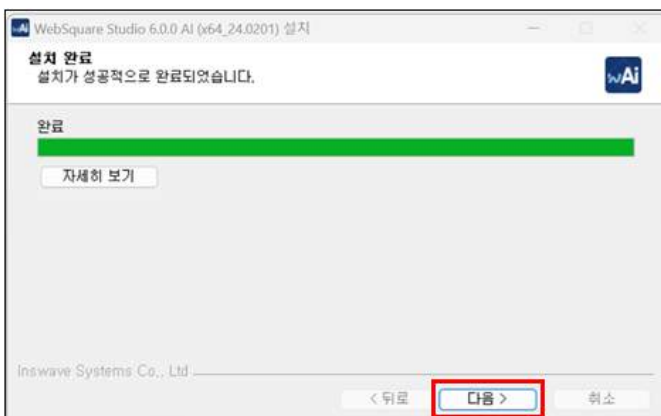
6. 설치를 클릭합니다.



7. 아래와 같이 설치가 진행됩니다.



8. Next 버튼을 클릭합니다.



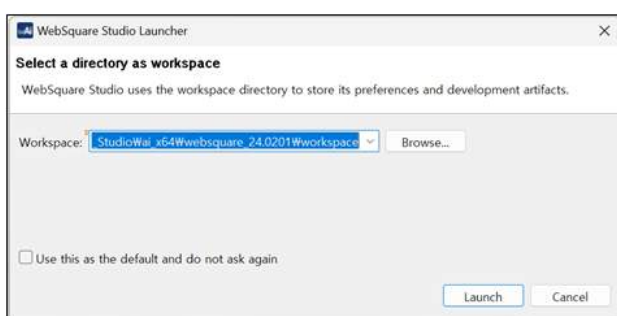
- 성공적으로 설치되면 아래와 같은 화면이 나타납니다.
- Finish** 버튼을 클릭합니다.



- 설치가 완료된 스튜디오를 실행합니다.아래와 같은 화면이 표시되고 스튜디오가 시작됩니다.



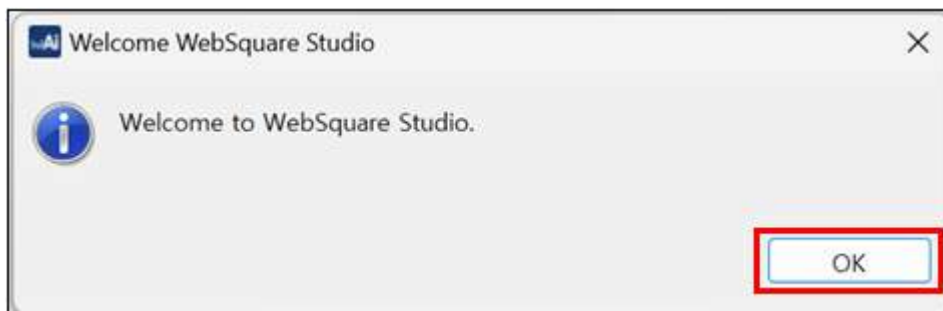
- Workspace를 지정합니다.



13. 라이선스 키를 입력합니다.



14. OK를 클릭하면 Studio가 시작되고 웰컴 페이지가 표시됩니다.



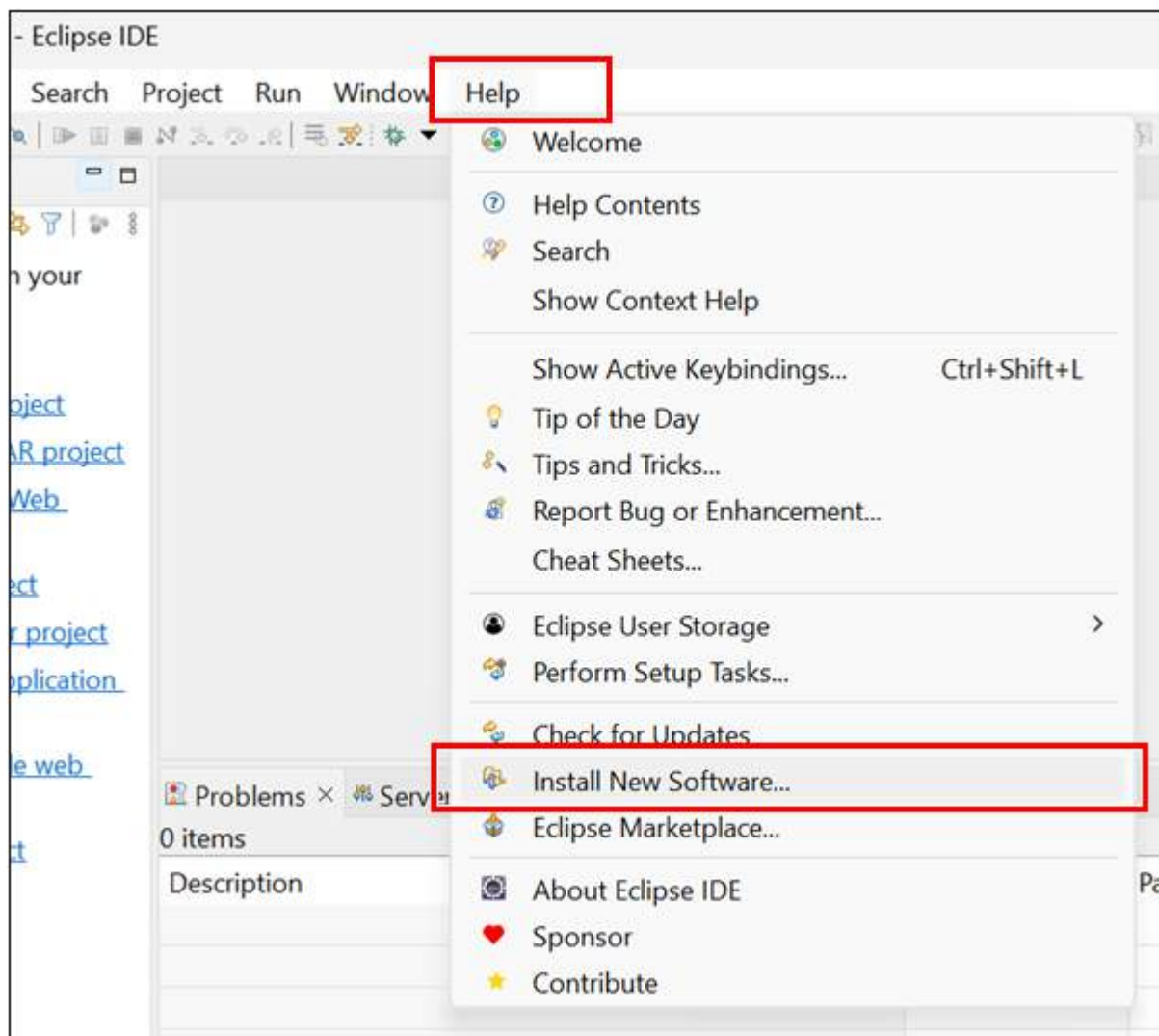
1.4.2. Eclipse 플러그인 설치

1.4.2.1. 설치 전 준비

(1) 플러그인

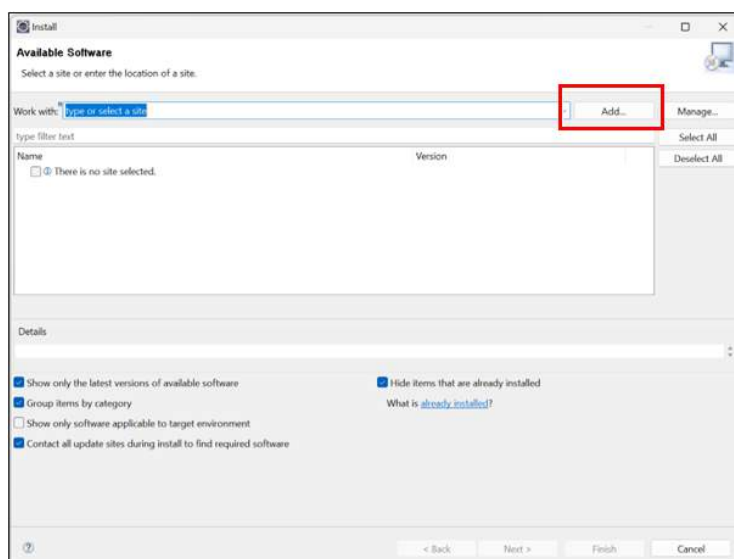
사용자 PC 사양에 맞는 웹스퀘어 플러그인을 제공 받으십시오.

그림 1-4 Help - Install New Software



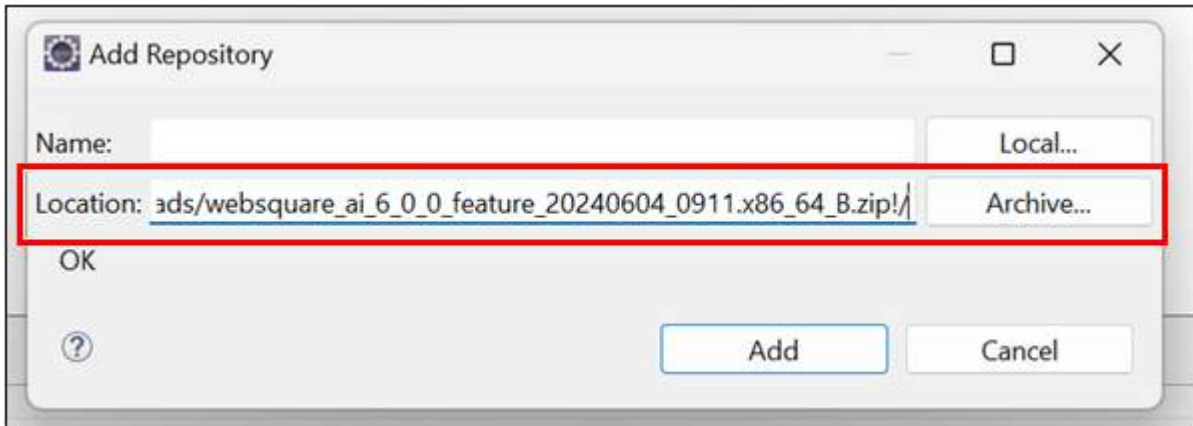
3. 소프트웨어 선택창이 표시되면 **Add** 버튼을 클릭합니다.

그림 1-5 소프트웨어 선택 창



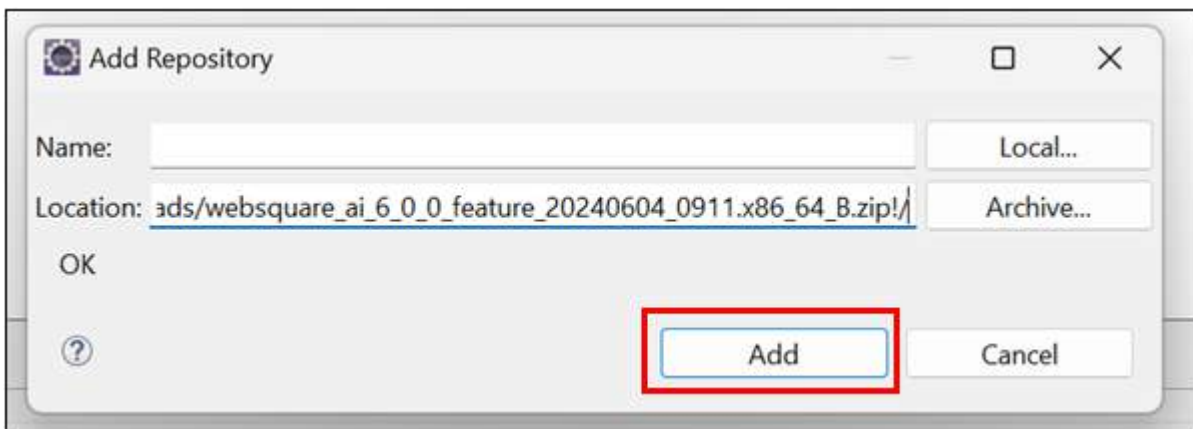
4. **Archive** 버튼을 클릭하여 다운로드한 웹스퀘어 플러그인을 선택합니다.

그림 1-6 웹스퀘어 플러그인 선택



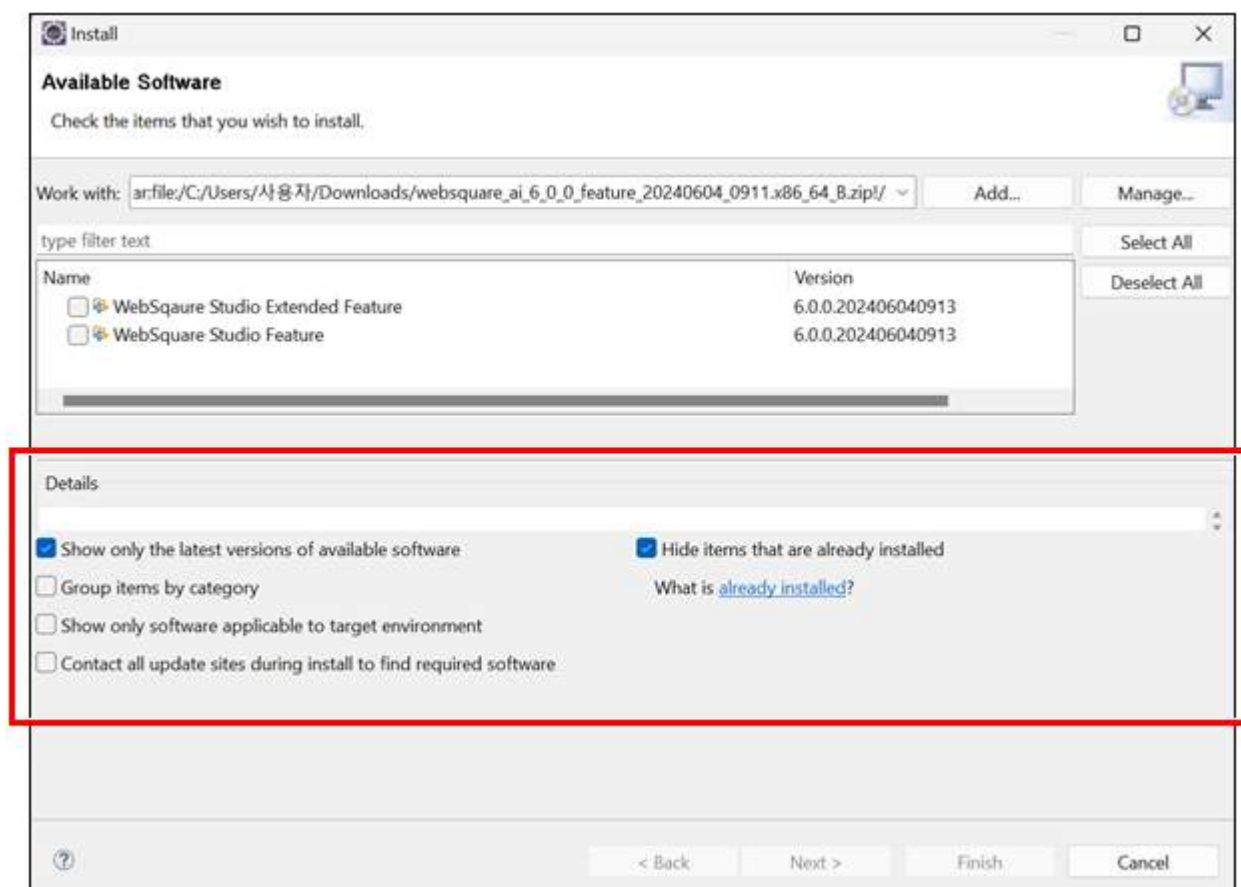
5. **Add** 버튼을 클릭합니다.

그림 1-7 웹스퀘어 플러그인 추가



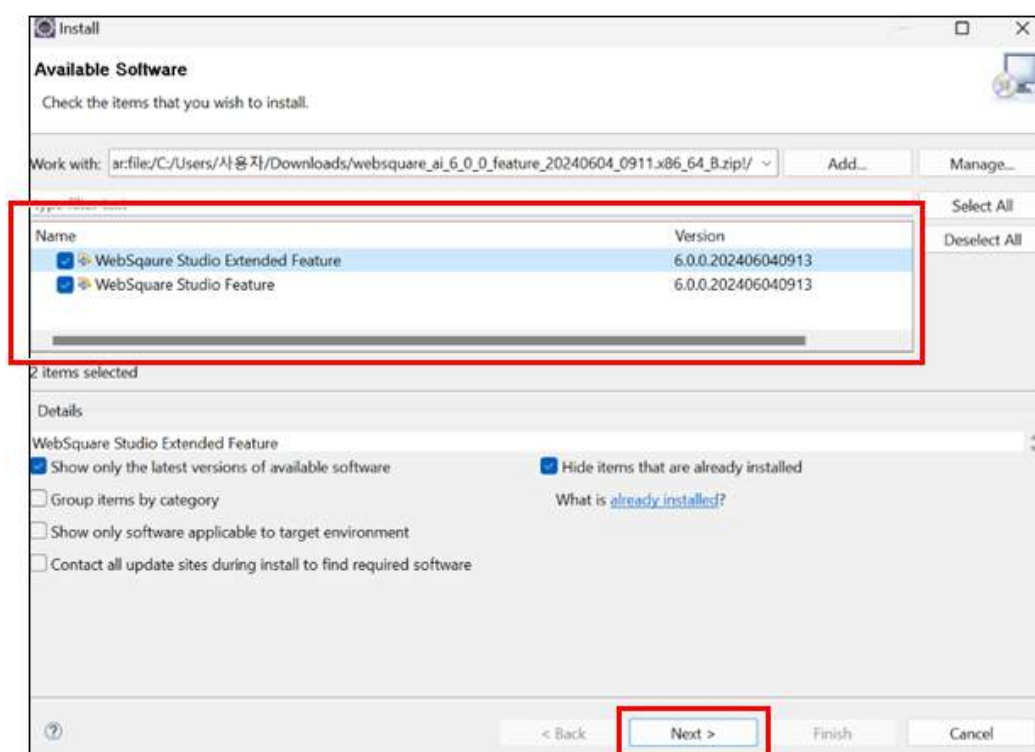
6. 하단의 옵션을 아래와 같이 설정합니다.

그림 1-8 옵션 선택



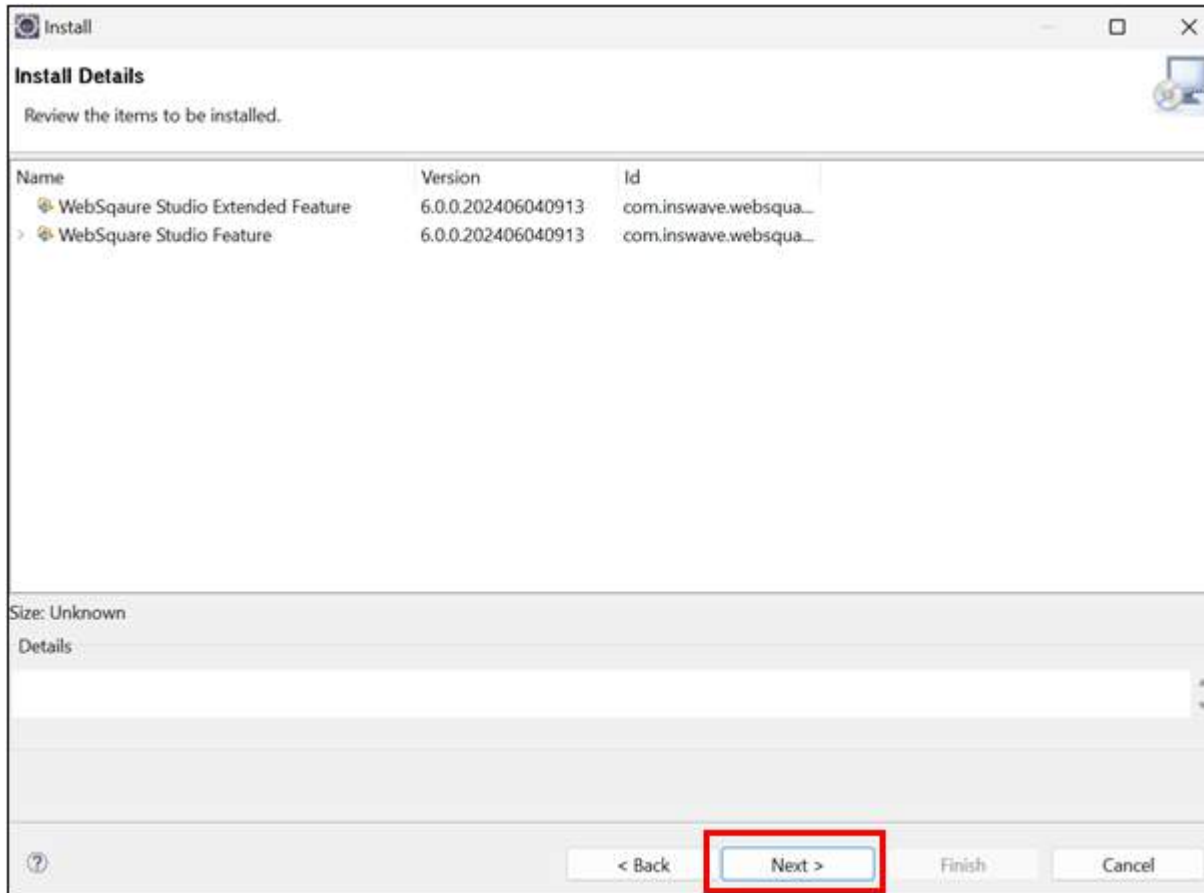
7. 설치 항목을 선택하고 **Next** 버튼을 클릭합니다.

그림 1-9 설치 항목 선택



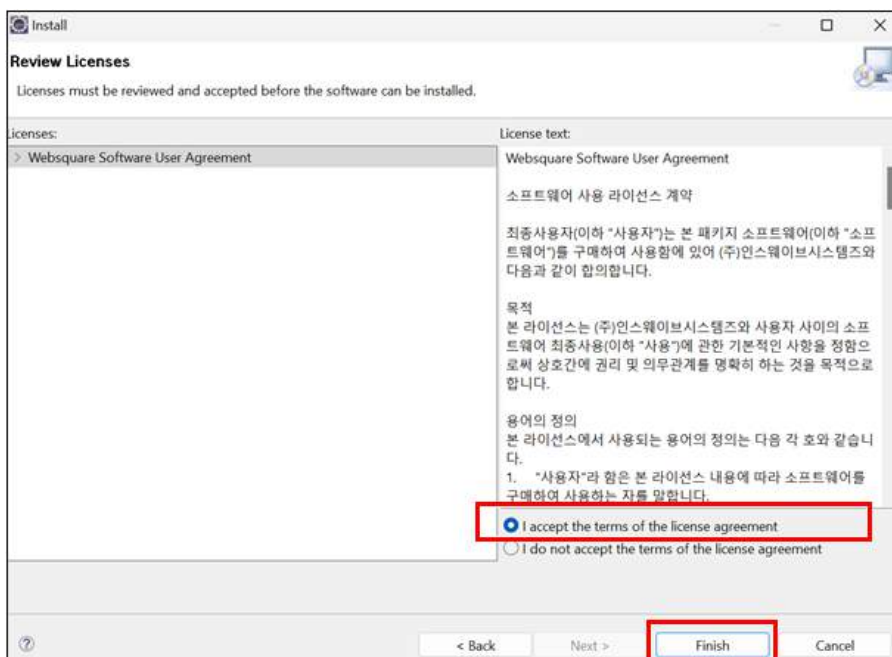
8. **Next** 버튼을 클릭합니다.

그림 1-10 설치 시작

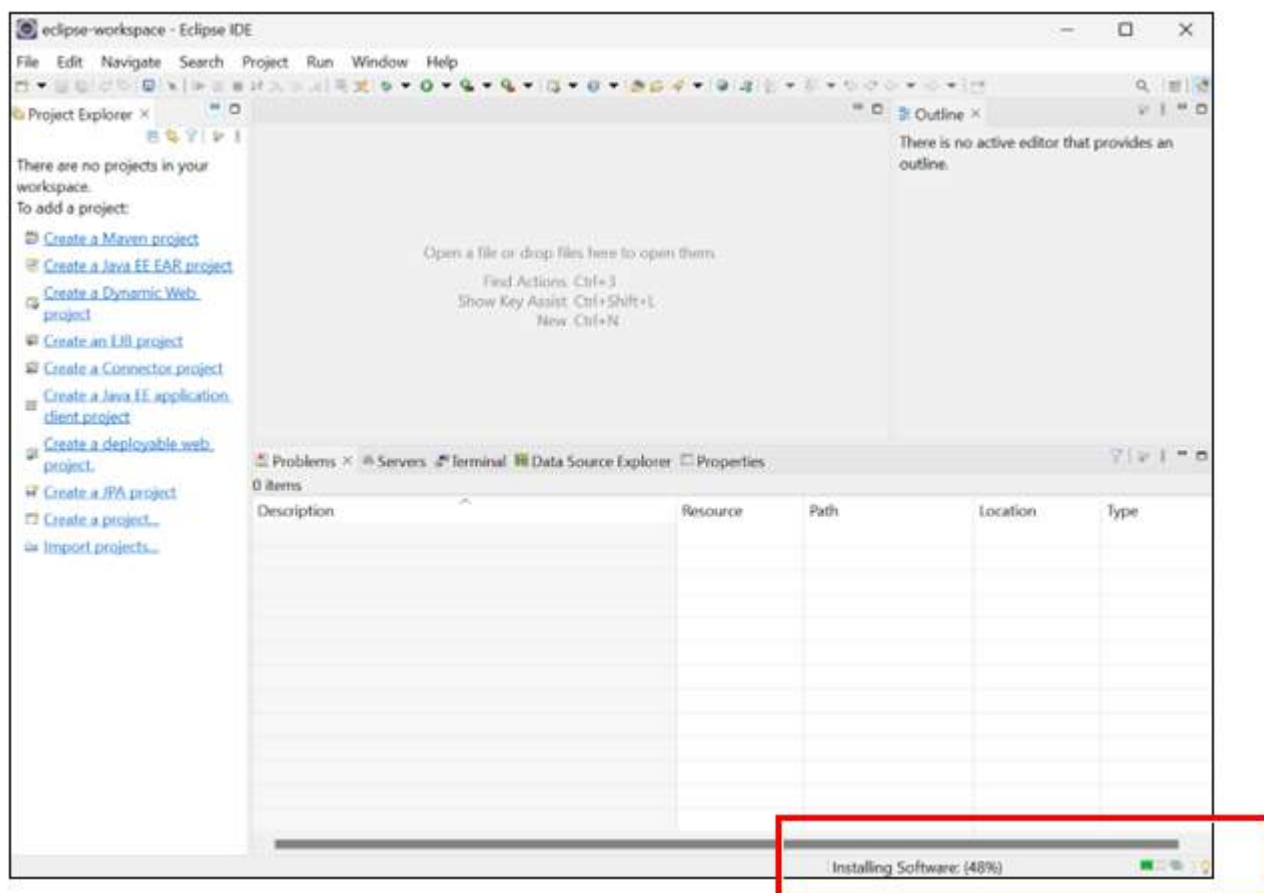


9. 라이선스 창이 표시되면, 동의를 선택하고 **Finish** 버튼을 클릭합니다.

그림 1-11 라이선스 창

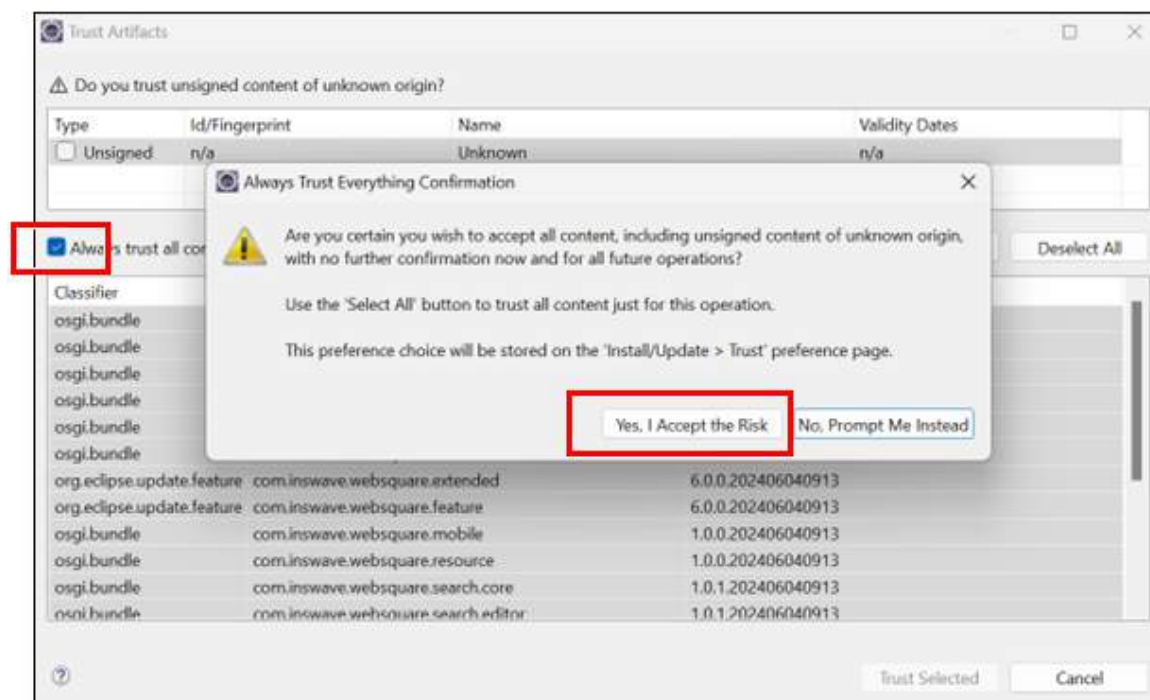


10. 설치가 시작되는지 확인합니다.



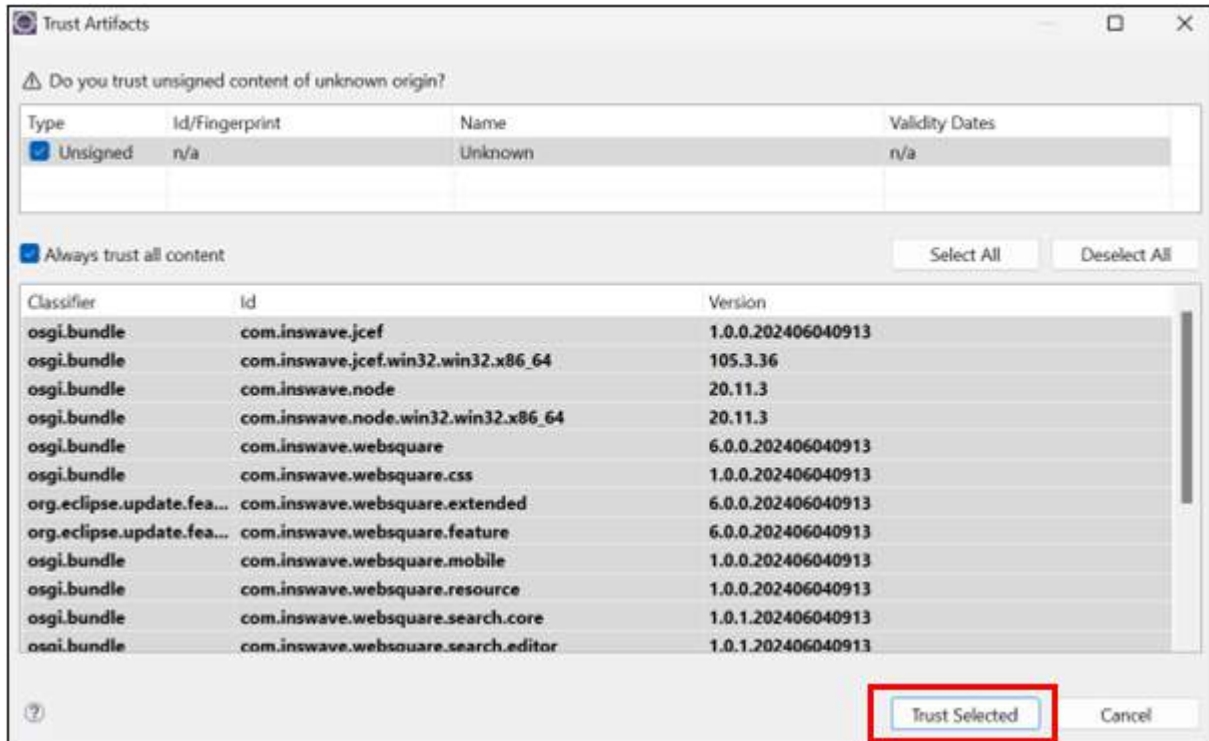
11. 설치 중 아래와 같은 창이 표시되면 **Always trust all...** 옵션을 선택하고, **Yes, I Accept...** 버튼을 클릭합니다.

그림 1-12 웹스퀘어 플러그인 확인

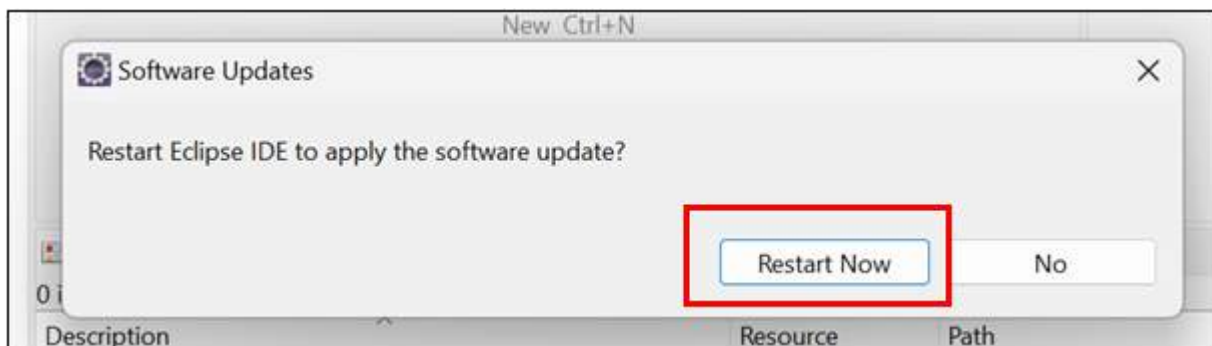


12. Trust Selected 버튼을 클릭합니다.

그림 1-13 웹스퀘어 플러그인 신뢰

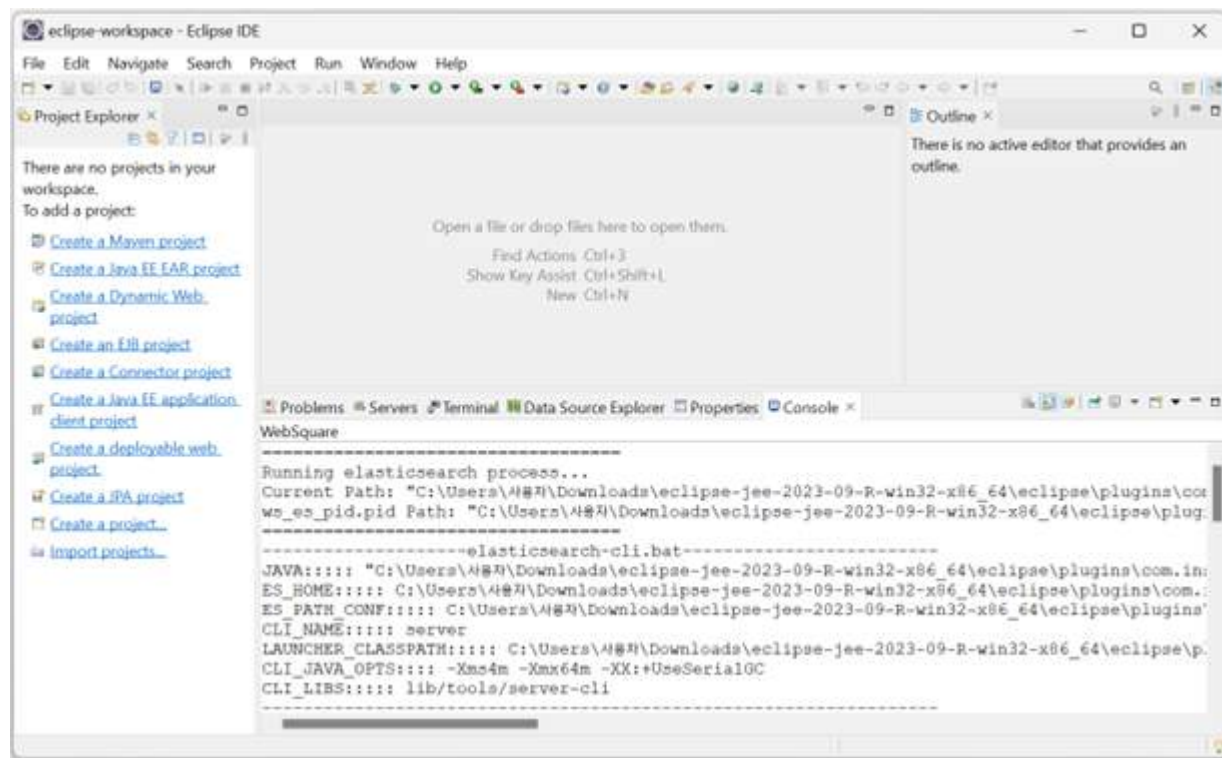


13. Restart Now 버튼을 클릭합니다.



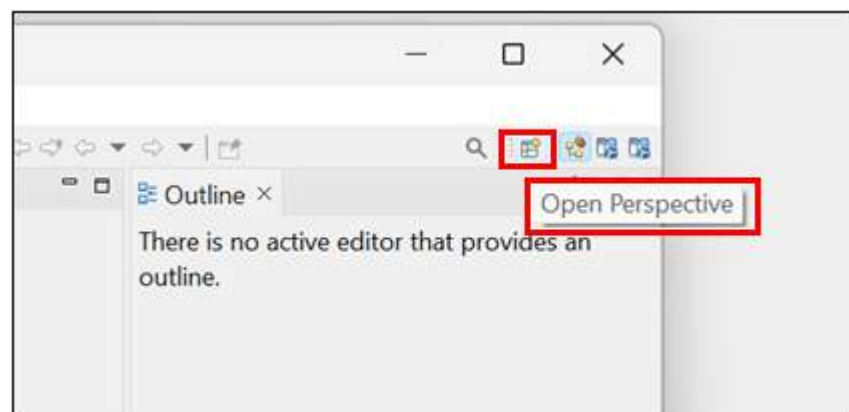
14. Eclipse를 재시작합니다.

그림 1-14 Eclipse 재시작

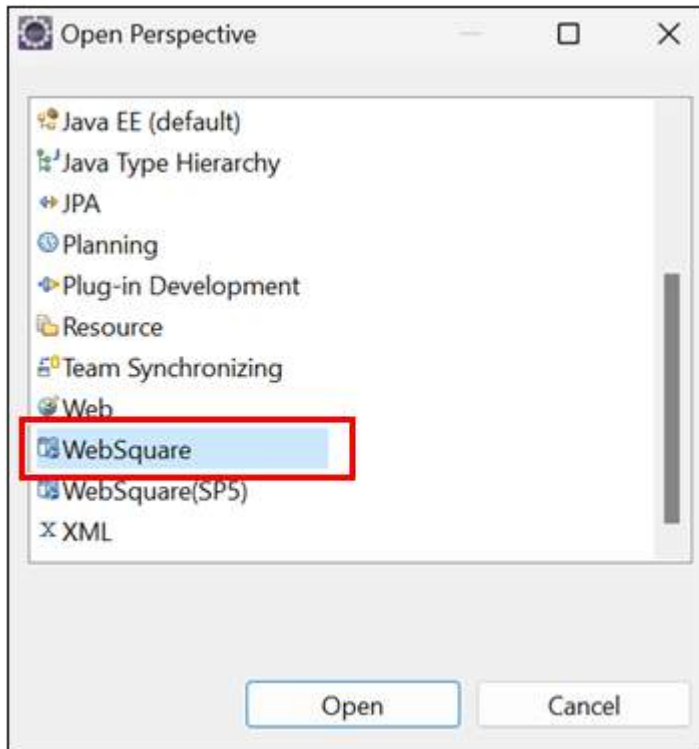


15. 우측 상단의 **Open Perspective** 버튼을 클릭합니다.

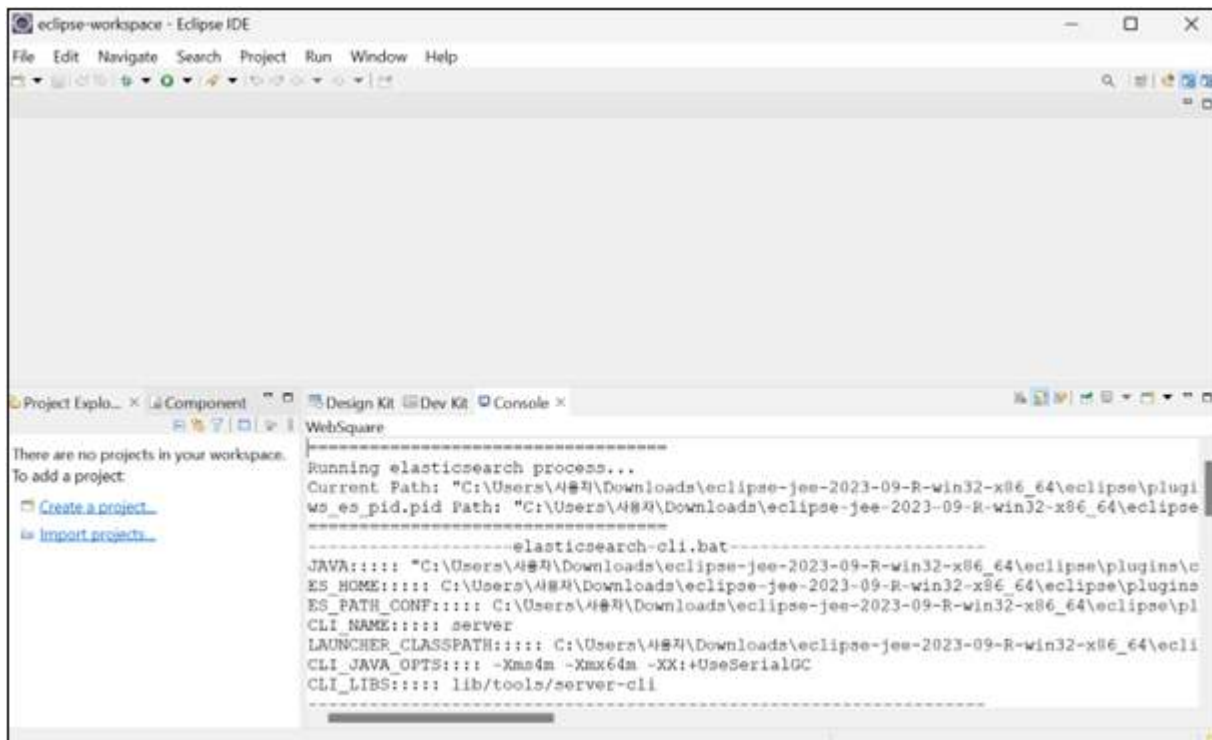
그림 1-15 Open Perspective



16. **WebSquare**를 선택하고 **Open**을 클릭합니다.

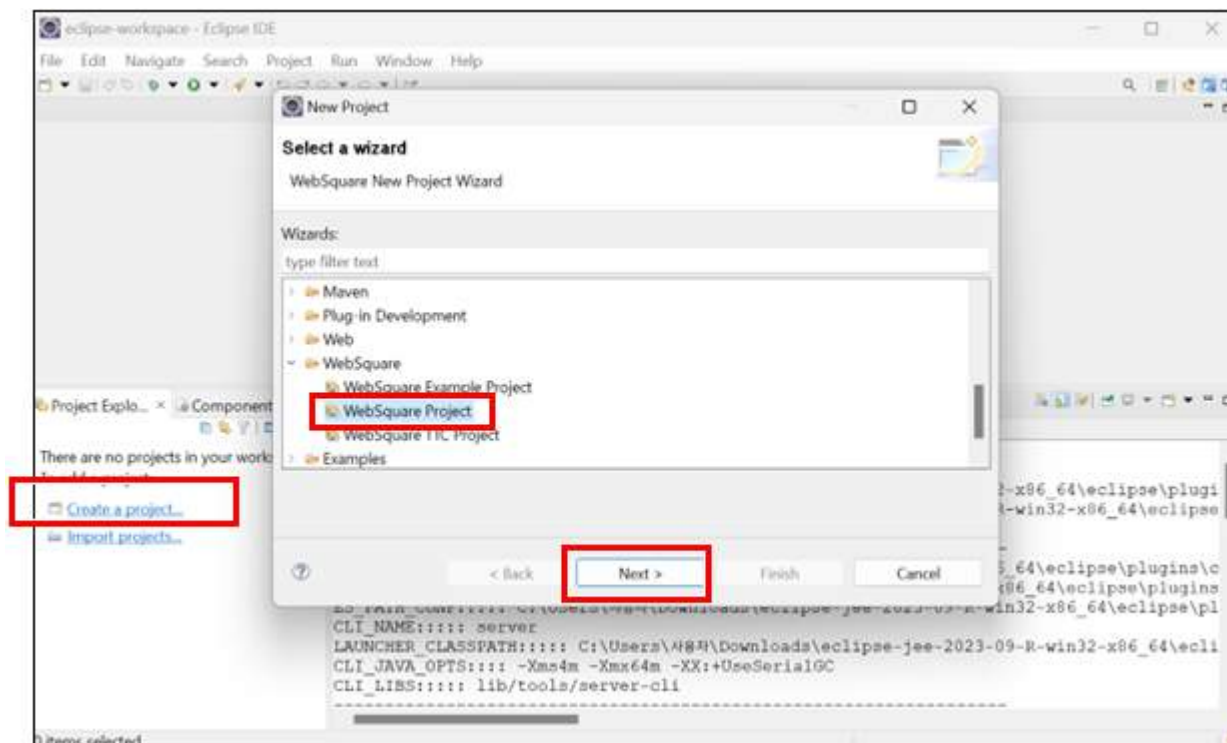


17. WebSquare Studio 화면이 표시되는지 확인합니다.

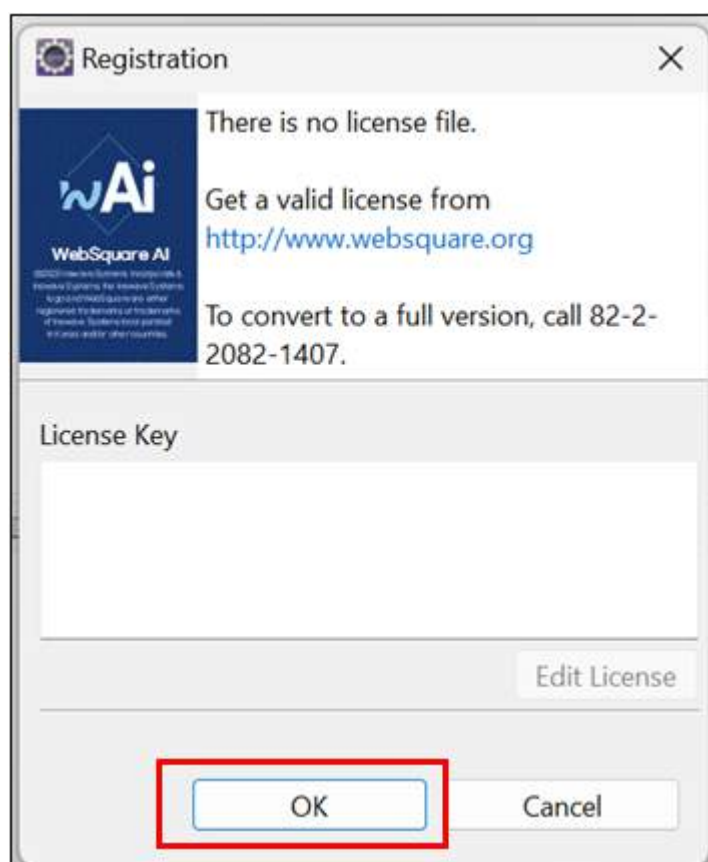


18. Project Explorer 탭에서 WebSquare 프로젝트를 생성합니다. (10. 프로젝트 생성 참고.)

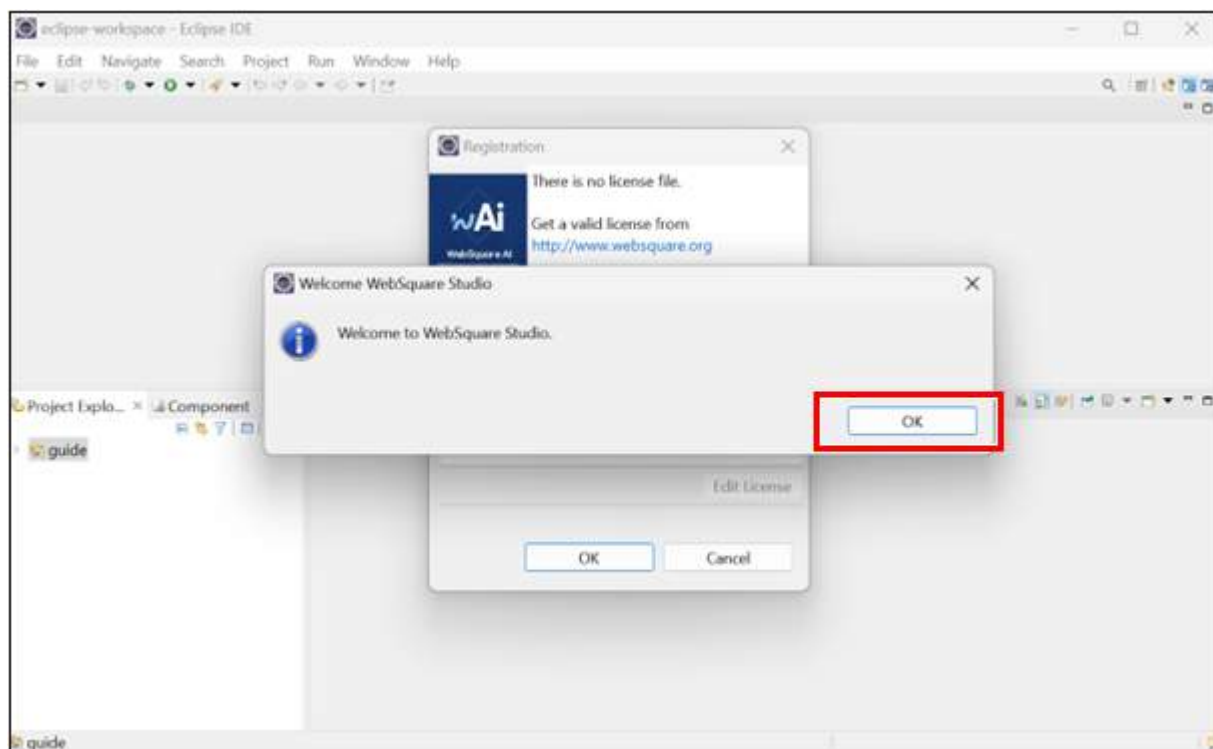
그림 1-16 웹스퀘어 프로젝트 생성



19. 부여받은 라이선스 번호를 입력하고 OK 버튼을 클릭합니다.



20. 웹스퀘어 플러그인이 정상 실행되는지 확인합니다.



21. 프로젝트의 WebContent 폴더 하위에 화면 파일을 생성합니다. (11. 컴포넌트 생성 참고.)

그림 1-17 화면 파일 생성

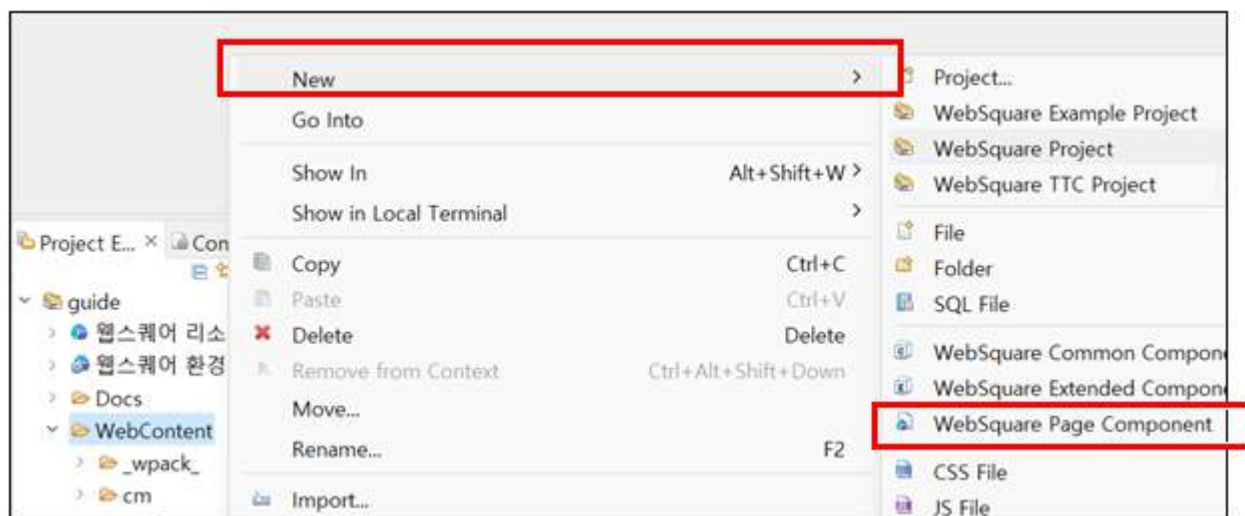
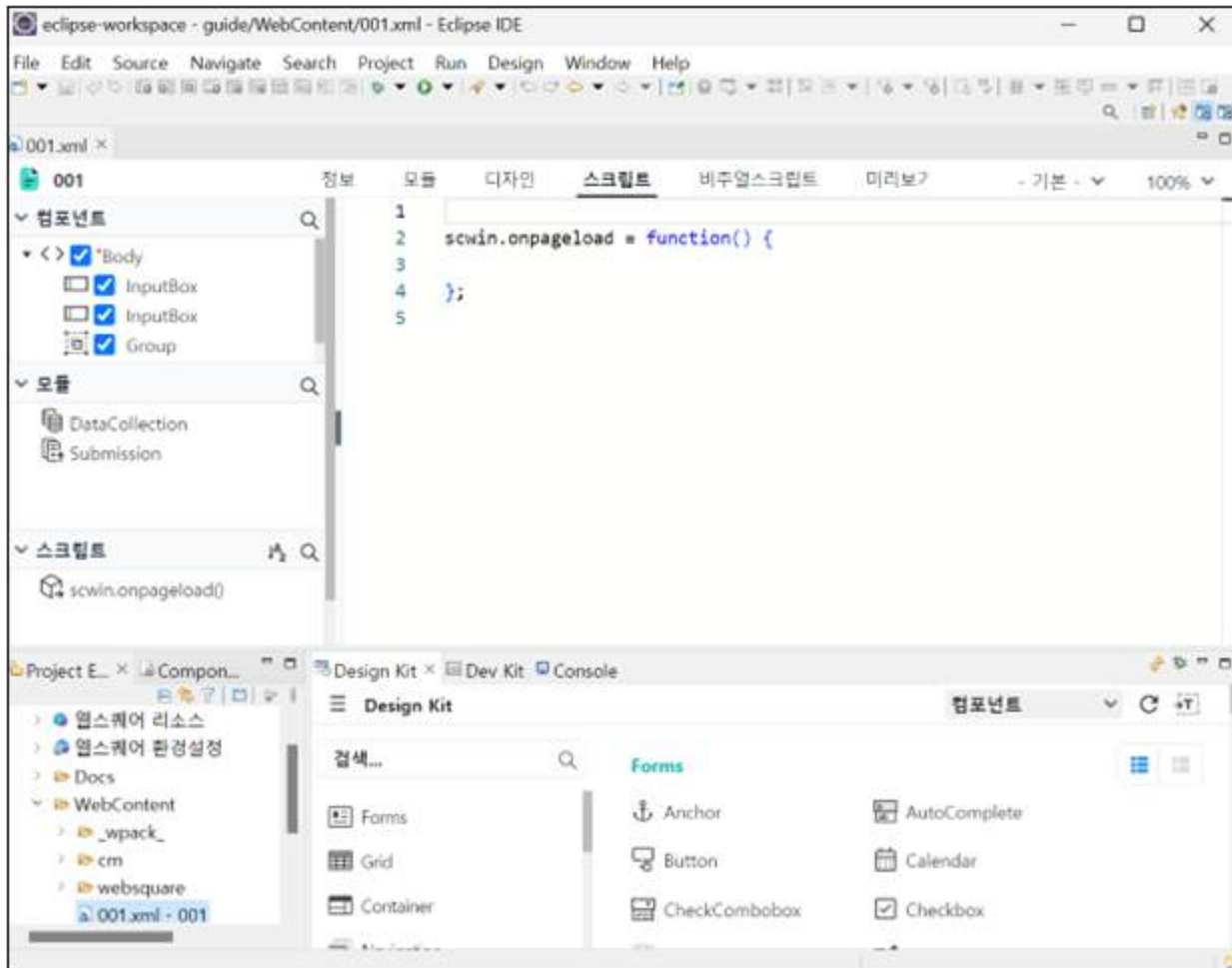


그림 1-18 화면 파일



1.5 환경 설정

1. Windows - Preferences를 선택합니다.
2. 왼쪽 목록에서 WebSquare를 클릭합니다.
3. 각 항목을 설정합니다.

그림 1-19 Window - Preferences

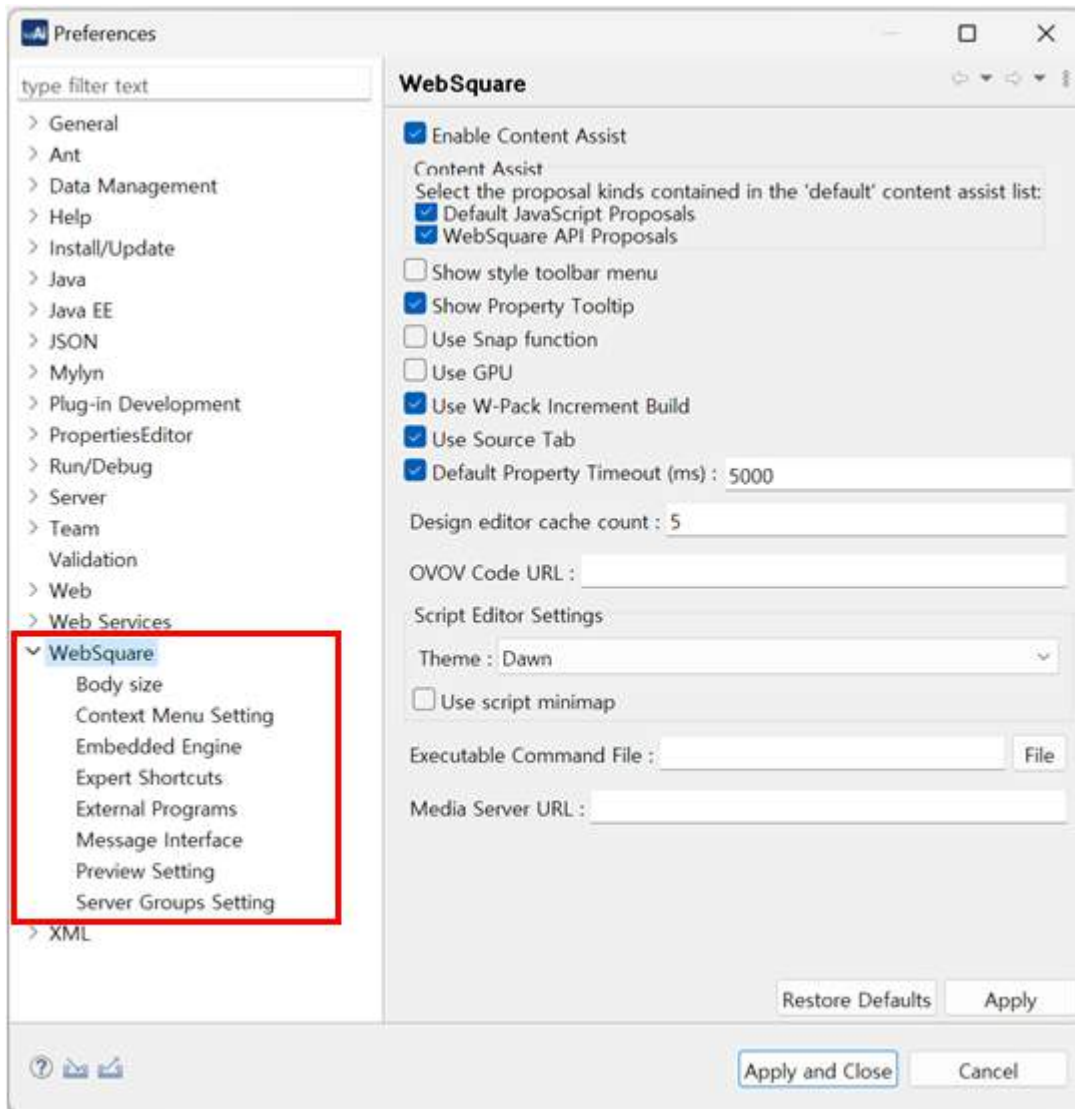


Table 1-3 전역 설정 항목

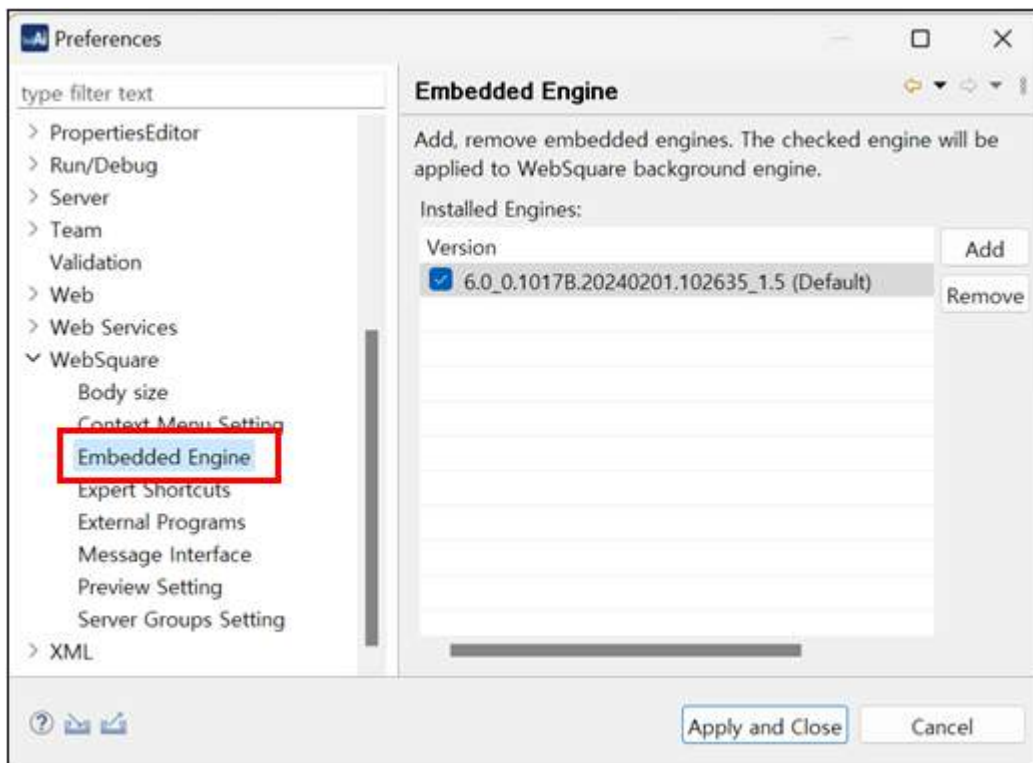
항목	설명
Body size	해상도 (단위: 픽셀) 추가한 해상도는 툴바의 Change Body Size 메뉴를 통해 확인할 수 있습니다.
Context Menu Setting	9.3 컨텍스트 메뉴 참조.
Embedded Engine	현재 스튜디오에 임베딩된 웹스퀘어 AI 엔진의 확인, 추가, 삭제가 가능.
Expert ShortCuts	특수 키 사용 여부 • Ctrl + 컴포넌트 이동: 컴포넌트 복사 • Alt + 컴포넌트 그리기: Static Position의 컴포넌트 생성 • Shift + 컴포넌트 그리기: Relative Position의 컴포넌트 생성
External Programs	미리보기 용 브라우저 추가 툴바의 Execute External Programs 메뉴를 통해 추가한 브라우저를 확인할 수 있습니다.
Preview Setting	웹스퀘어 AI의 내장 미리보기 기능 사용 여부 Use External Preview (No preview tab)을 선택 해제하면 내장 미리보기 기능을 사용할 수 있

항목	설명
	습니다.

1.6 엔진 교체

1. WebSquare Studio를 시작합니다.
2. **Window - Preferences**를 선택합니다.
3. **WebSquare - Embedded Engine**를 선택합니다.
4. 현재 설치된 Engine 버전을 확인합니다.

그림 1-20 엔진 버전 확인



5. **Add**를 클릭하여 Engine 파일을 추가합니다. (*zip* 확장명의 압축 파일로, 압축 해제할 필요는 없습니다.)
6. 새 Engine이 추가되는 과정은 오른쪽 아래의 진행 상태 창(progress view)에서 확인할 수 있습니다. 100% 완료되면 새로운 Engine이 성공적으로 추가됩니다.
7. 새 버전의 Engine을 선택하고 **OK** 버튼을 클릭하면 스튜디오가 재 시작되며 새로운 Engine으로 바뀌어 있는 것을 확인할 수 있습니다.

2. 스튜디오 서버

2.1 개요

스튜디오 서버는 인스웨이브 시스템즈가 제공하는 AI 기능을 통합 관리하는 어플리케이션입니다. 로컬/서버 환경에서 스튜디오 서버를 설치하여 웹스퀘어 AI 스튜디오에서 AI 기능을 사용할 수 있습니다.

2.2 사양

설치 환경 사양은 아래와 같습니다.

Table 2-1 하드웨어/소프트웨어 최소 사양

OS	CPU	메모리	HDD	JDK	비고
Windows Linux Unix Mac	2 Core	2 Gb	10 Gb	1.8 - 17	

Table 2-2 하드웨어/소프트웨어 권장 사양

OS	CPU	메모리	HDD	JDK	비고
Windows Linux Unix Mac	4 Core	4 Gb	10 Gb	1.8 - 17	

Table 2-3 미들웨어 사양

WAS	DB	비고
Embedded Tomcat 9.0.74	H2	

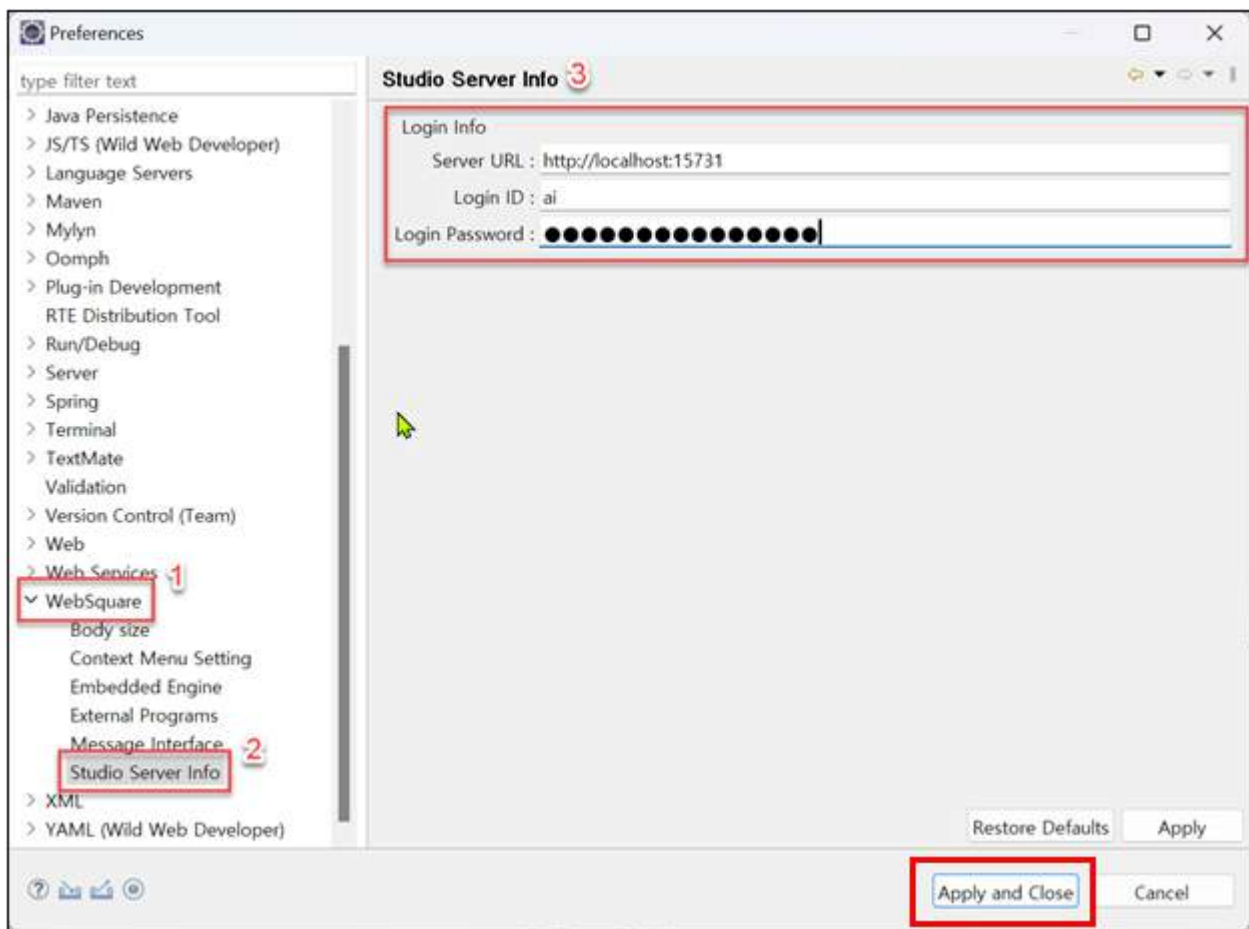


기본 미들웨어 외에 타사의 WAS 및 DB도 사용 가능합니다.

2.3 점검

2.3.1. 설정

웹스퀘어 AI 스튜디오에서 프로젝트를 우클릭한 후 **Properties**를 선택하여 스튜디오 서버의 URL, 아이디, 비밀번호를 입력하고 저장합니다.



2.3.2. 접속

스튜디오 서버 어플리케이션으로 띄운 URL에 접속하여 웹 화면이 정상적으로 표시되는지 확인합니다.

Table 2-4 접속 URL

URL	<code>http://localhost:15731/InswaveAdmin/websquare/websquare.html</code>
-----	---

 계정 정보는 관리자에게 문의하십시오.

그림 2-1 스튜디오 서버 접속 화면

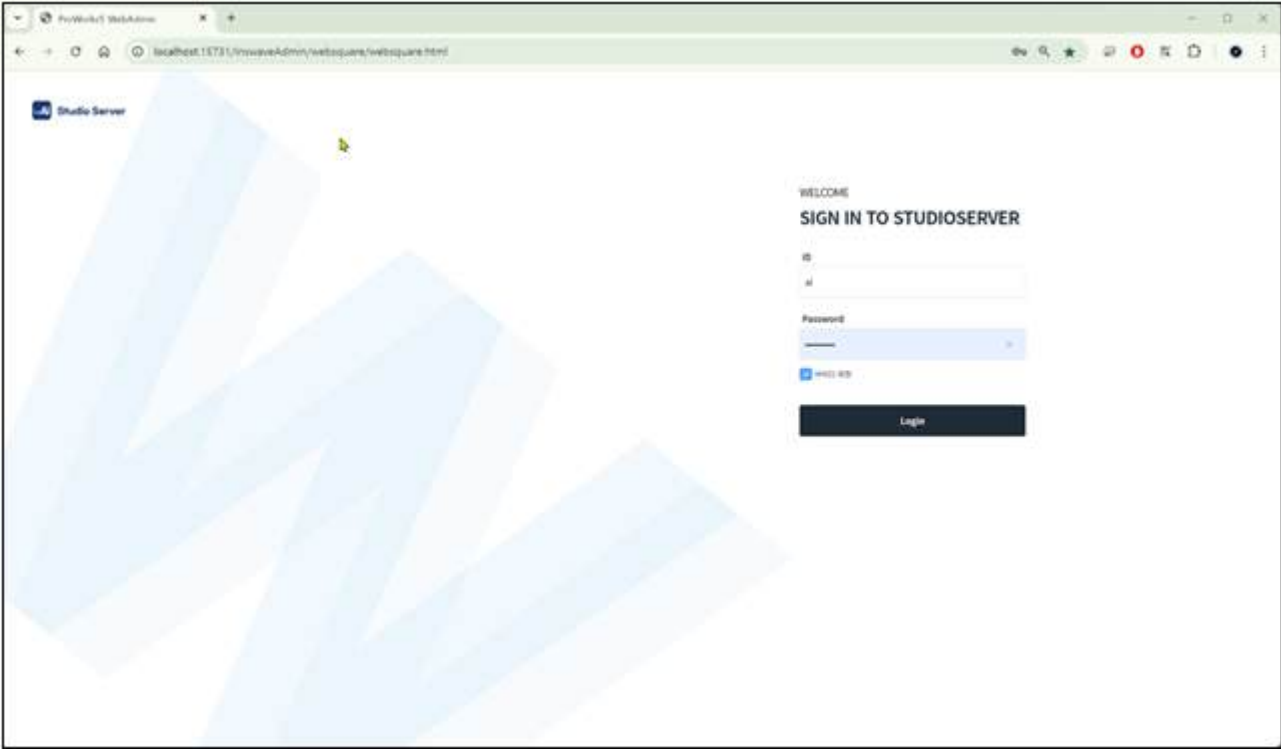
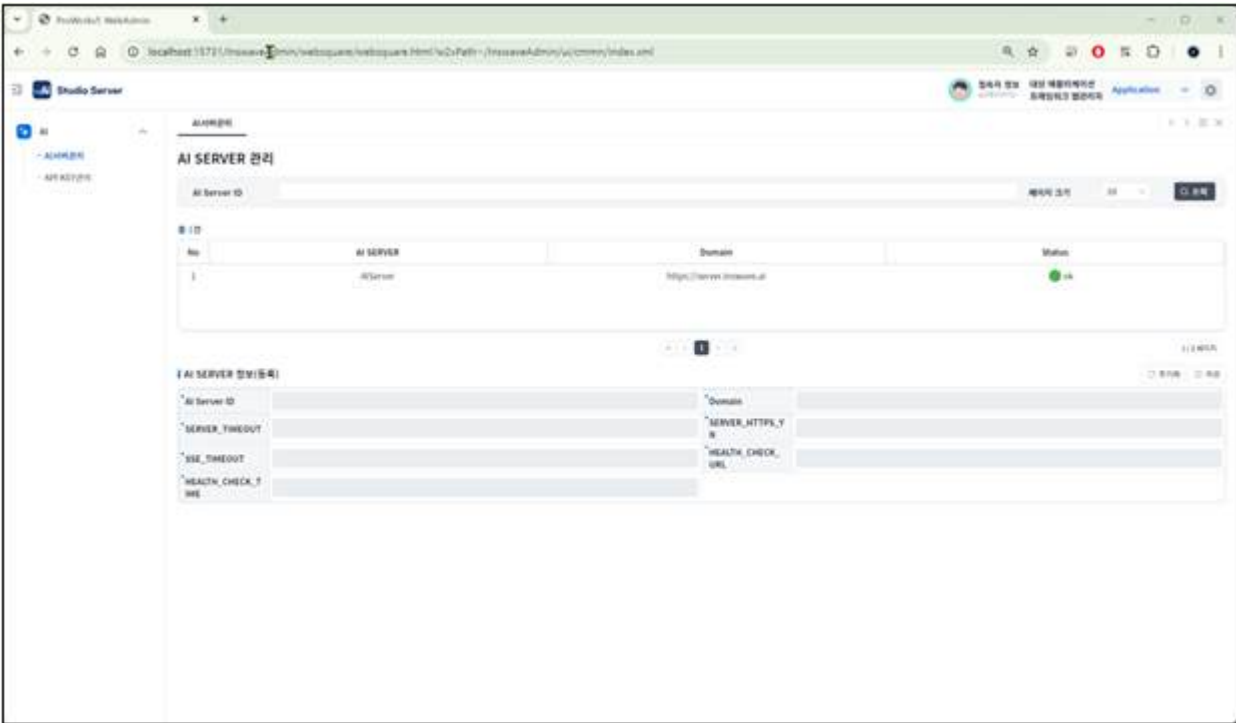


그림 2-2 스튜디오 서버에 접속한 모습



2.4 Quick Code 생성

Adapter Class를 사용하여 소스 및 템플릿을 커스터마이징 할 수 있습니다.

샘플 소스 다운로드

<https://drive.google.com/file/d/1J7AbnglcyNndfuYmnpbfTeW1lzT7uTay/view?usp=sharing>

샘플 템플릿 다운로드

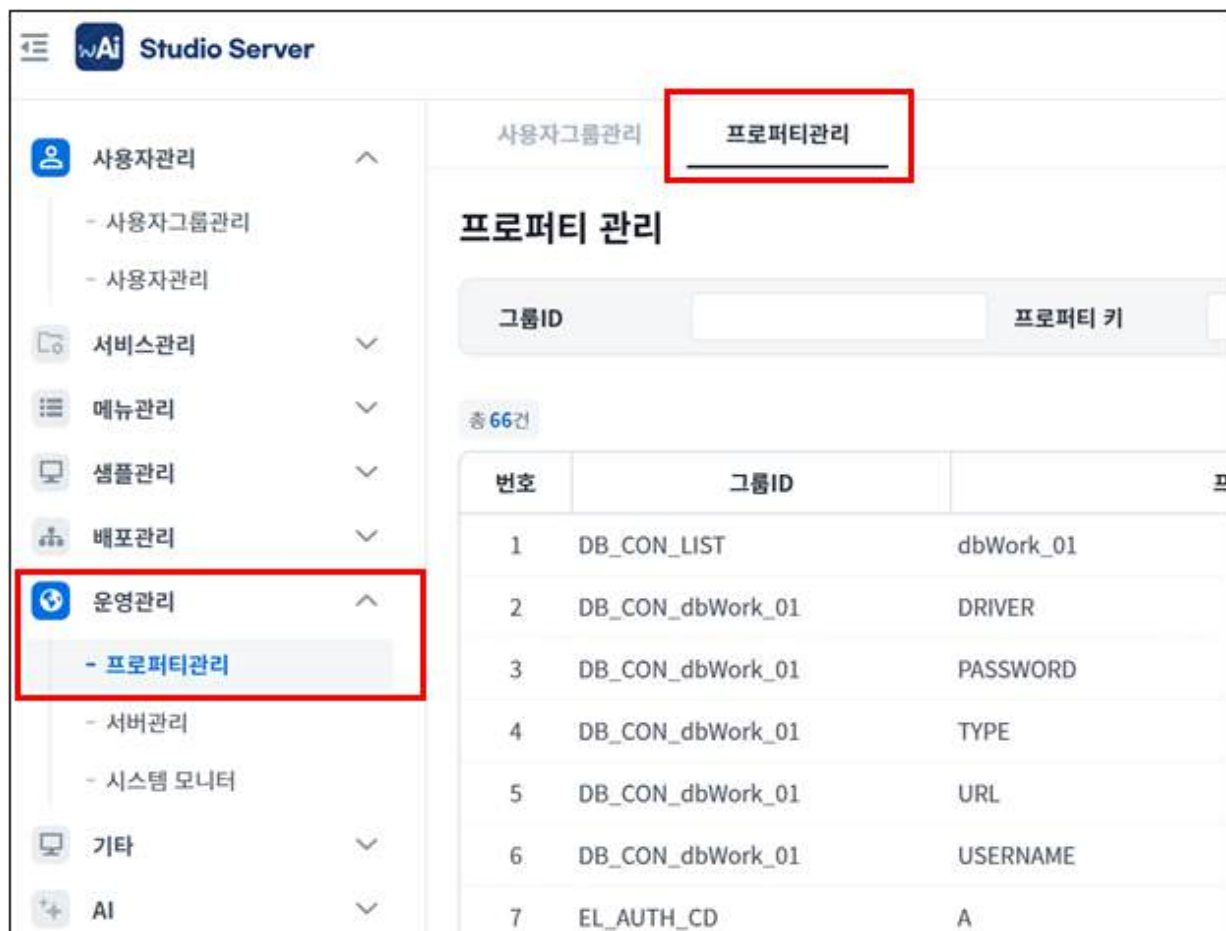
<https://drive.google.com/file/d/119dmmW5dga1Bf1w3VMVFllbJ4loizW01/view?usp=sharing>

2.4.1. 프로퍼티 설정

2.4.1.1. DefaultWebSquareSourceGenerateAdapter 구현체 설정

1. 스튜디오 서버 어플리케이션에 접속합니다.
2. 운영관리 - 프로퍼티 관리 메뉴를 선택합니다.

그림 2-3 운영관리 - 프로퍼티 관리



3. 관리할 프로퍼티를 선택하여 정보를 수정합니다.

그림 2-4 프로퍼티 관리

프로퍼티 관리

그룹ID: [] 프로퍼티 키: WEBSQUARE_SOURCE 제목: [] 페이지 크기: 10 [초기화]

총 1건

번호	그룹ID	프로퍼티 키	제목	정렬순서
1	EL_CORE_PROP	WEBSQUARE_SOURCE_GEN_ADAPTER_CLASS_NAME	WebSquare 화면 생성 어댑터	

1 / 1 페이지

프로퍼티 정보(수정)

그룹ID: EL_CORE_PROP

프로퍼티 키: WEBSQUARE_SOURCE

프로퍼티 값: com.inswave.eladmin,

제목: WebSquare 화면 생성 어

설명: WebSquare 화면 생성 어댑터

정렬순서: []



기본 구현체

기본 구현체(DefaultWebSquareSourceGenerateAdapter)는 프로퍼티 관리 메뉴에서 등록하지 않아도 동작합니다.

새로운 구현체를 생성할 경우에만 프로퍼티 관리 메뉴에서 등록하십시오.

2.4.1.2. WebSquare 화면 다국어 설정 및 WebSquare 화면 template 디렉토리 위치 프로퍼티 설정

1. WebSquare 화면 다국어 설정 및 WebSquare 화면 template 디렉토리 위치 프로퍼티를 각각 선택하여 관련 정보를 설정합니다.

그림 2-5 WebSquare 화면 다국어 설정 및 WebSquare 화면 template 디렉토리 위치 프로퍼티 설정

접속자 정보 admin@webadm.com 대상 애플리케이션프레임워크 템플릿 Application

사용자그룹관리 CRUD 템플릿 생성 프로퍼티관리

프로퍼티 관리

그룹ID: [] 프로퍼티 키: WEBSQ 제목: [] 페이지 크기: 50 [조회]

총 2건

번호	그룹ID	프로퍼티 키	제목	정렬순서
1	EL_CORE_PROP	WEBSQUARE_SOURCE_GEN_MULTI_LANG_YN	WebSquare 화면 다국어 설정	
2	EL_CORE_PROP	WEBSQUARE_SOURCE_GEN_TEMPLATE_PATH	WebSquare 화면 template 디렉토리 위치	

1 / 1 페이지

프로퍼티 정보(수정)

[초기화] [저장] [삭제]

*그룹ID: EL_CORE_PROP

*프로퍼티 키: WEBSQUARE_SOURCE

프로퍼티 값: /home/webadm/temp

제목: WebSquare 화면 templ.

설명: WebSquare 화면 template 디렉토리 위치

정렬순서: []



다국어 설정

WEBSQUARE_SOURCE_GEN_MULTI_LANG_YN (WebSquare 다국어 설정) 기본 값은 "N"입니다.

2.4.2. Adapter 구현

SourceGenerateAdapter를 상속받아 Adapter를 구현하십시오.

그림 2-6 SourceGenerateAdapter 상속

```

1 package com.inswave.eladmin.adapter;
2
3 import java.util.ArrayList;
4
5
6
7
8
9
10
11
12
13
14
15
16 public class DefaultWebSquareSourceGenerateAdapter extends SourceGenerateAdapter {
17
18     final public static String TYPE_BASE = "Base";
19     final public static String TYPE_LIST = "List";
20
21     public DefaultWebSquareSourceGenerateAdapter(Map<String, Object> adapterInfoMap) {
22         super(adapterInfoMap);
23     }
24
25     @Override
26     public List<SourceGenTypeVo> getTypes() {
27         List<SourceGenTypeVo> allTypes = new ArrayList<SourceGenTypeVo>();
28
29         SourceGenTypeVo vo = new SourceGenTypeVo();
30         vo.setCreateType(TYPE_BASE);
31         vo.setCreateTypeName(getTypeName(vo.getCreateType()));
32         allTypes.add(vo);
33
34         SourceGenTypeVo vo2 = new SourceGenTypeVo();
35         vo2.setCreateType(TYPE_LIST);
36         vo2.setCreateTypeName(getTypeName(vo2.getCreateType()));
37         allTypes.add(vo2);
38
39         return allTypes;
40     }
41
42     @Override
43     public String getSource(ReqGenCodeVo reqGenCodeVo) throws Exception {
44
45         String templateClassPathRoot = ElConfig.getValue("WEBSQUARE_SOURCE_GEN_TEMPLATE_PATH", "inswave/gen_src/websquare");
46         String multiLangYn = ElConfig.getValue("WEBSQUARE_SOURCE_GEN_MULTI_LANG_YN", "N");
47
48         if (reqGenCodeVo == null) {
49             throw new UserException("WSADM.ERROR.COMM.0002");
50         }
51
52         if (TYPE_BASE.equals(reqGenCodeVo.getCreateType())) {
53             return getWebsquare(reqGenCodeVo, templateClassPathRoot, multiLangYn);
54         } else if (TYPE_LIST.equals(reqGenCodeVo.getCreateType())) {
55             return getWebsquareList(reqGenCodeVo, templateClassPathRoot, multiLangYn);
56         } else {
57             throw new UserException("WSADM.ERROR.COMM.0003");
58         }
59     }
60 }

```

SourceGenerateAdapter를 상속 받아 아래의 추상 메소드를 구현하십시오.

커스텀 소스에 대한 TYPE 리스트

```
public abstract List<SourceGenTypeVo> getTypes();
```

입력 인자를 받는 TYPE에 따라 생성된 소스

```
public abstract String getSource(ReqGenCodeVo reqGenCodeVo) throws Exception;
```

그림 2-7 추상 메소드 구현

```

1 package com.inswave.eladmin.adapter;
2
3 import java.util.ArrayList;
4
15
16 public class DefaultWebSquareSourceGenerateAdapter extends SourceGenerateAdapter {
17
18     final public static String TYPE_BASE = "Base";
19     final public static String TYPE_LIST = "List";
20
21     public DefaultWebSquareSourceGenerateAdapter(Map<String, Object> adapterInfoMap) {
22         super(adapterInfoMap);
23     }
24
25     @Override
26     public List<SourceGenTypeVo> getTypes() {
27         List<SourceGenTypeVo> allTypes = new ArrayList<SourceGenTypeVo>();
28
29         SourceGenTypeVo vo = new SourceGenTypeVo();
30         vo.setCreateType(TYPE_BASE);
31         vo.setCreateTypeName(getTypeName(vo.getCreateType()));
32         allTypes.add(vo);
33
34         SourceGenTypeVo vo2 = new SourceGenTypeVo();
35         vo2.setCreateType(TYPE_LIST);
36         vo2.setCreateTypeName(getTypeName(vo2.getCreateType()));
37         allTypes.add(vo2);
38
39         return allTypes;
40     }
41
42     @Override
43     public String getSource(ReqGenCodeVo reqGenCodeVo) throws Exception {
44
45         String templateClassPathRoot = ElConfig.getValue("WEBSQUARE_SOURCE_GEN_TEMPLATE_PATH", "inswave/gen_src/websquare");
46         String multiLangYn = ElConfig.getValue("WEBSQUARE_SOURCE_GEN_MULTI_LANG_YN", "N");
47
48         if (reqGenCodeVo == null) {
49             throw new UserException("WSADM.ERROR.COMM.0002");
50         }
51
52         if (TYPE_BASE.equals(reqGenCodeVo.getCreateType())) {
53             return getWebsquare(reqGenCodeVo, templateClassPathRoot, multiLangYn);
54         } else if (TYPE_LIST.equals(reqGenCodeVo.getCreateType())) {
55             return getWebsquareList(reqGenCodeVo, templateClassPathRoot, multiLangYn);
56         } else {
57             throw new UserException("WSADM.ERROR.COMM.0003");
58         }
59     }
60

```

2.4.3. 소스 생성

2.4.3.1. 소스 생성 유형 목록 조회 (GetWebSquareSourceTypes)

TYPE_BASE는 현재 두 가지 타입을 지원하며, Adapter를 통해 추가 타입을 구현할 수 있습니다.

TYPE_BASE=Base

getWebsquare()

TYPE_BASE=List

getWebsquare()

그림 2-8 TYPE_BASE

```

1 package com.inswave.eladmin.adapter;
2
3 import java.util.ArrayList;
4
15
16 public class DefaultWebSquareSourceGenerateAdapter extends SourceGenerateAdapter {
17
18     final public static String TYPE_BASE = "Base";
19     final public static String TYPE_LIST = "List";
20
21     public DefaultWebSquareSourceGenerateAdapter(Map<String, Object> adapterInfoMap) {
22         super(adapterInfoMap);
23     }
24
25     @Override
26     public List<SourceGenTypeVo> getTypes() {
27         List<SourceGenTypeVo> allTypes = new ArrayList<SourceGenTypeVo>();
28
29         SourceGenTypeVo vo = new SourceGenTypeVo();
30         vo.setCreateType(TYPE_BASE);
31         vo.setCreateTypeName(getTypeName(vo.getCreateType()));
32         allTypes.add(vo);
33
34         SourceGenTypeVo vo2 = new SourceGenTypeVo();
35         vo2.setCreateType(TYPE_LIST);
36         vo2.setCreateTypeName(getTypeName(vo2.getCreateType()));
37         allTypes.add(vo2);
38
39         return allTypes;
40     }
41
42     @Override
43     public String getSource(ReqGenCodeVo reqGenCodeVo) throws Exception {
44
45         String templateClassPathRoot = ElConfig.getValue("WEBSQUARE_SOURCE_GEN_TEMPLATE_PATH", "inswave/gen_src/websquare");
46         String multiLangYn = ElConfig.getValue("WEBSQUARE_SOURCE_GEN_MULTI_LANG_YN", "N");
47
48         if (reqGenCodeVo == null) {
49             throw new UserException("WSADM.ERROR.COMM.0002");
50         }
51
52         if (TYPE_BASE.equals(reqGenCodeVo.getCreateType())) {
53             return getWebsquare(reqGenCodeVo, templateClassPathRoot, multiLangYn);
54         } else if (TYPE_LIST.equals(reqGenCodeVo.getCreateType())) {
55             return getWebsquareList(reqGenCodeVo, templateClassPathRoot, multiLangYn);
56         } else {
57             throw new UserException("WSADM.ERROR.COMM.0003");
58         }
59     }
60

```

2.4.3.2. 소스 생성 (CreateWebSquareSource)

CSS 및 공통 함수를 변경할 경우, `getWebsquare()` 및 `getWebsquareList()` 메소드를 변경하십시오. 각 메소드는 `TY` `PE_Base` 및 다국어 설정 값에 따라 아래의 템플릿을 함께 변경해야 합니다.

2.4.3.2.1. getWebsquare()

다국어 설정 값에 따라 `Base.template` 및 `Base_multiLang.template` 템플릿을 함께 변경하십시오.

Table 2-5 TYPE_BASE=Base인 경우, `getWebsquare()` 메소드 변경

TYPE_BASE	다국어 설정값	함께 수정할 템플릿
Base	"N"	Base.template
Base	"Y"	Base_multiLang.template

2.4.3.2.2. getWebsquareList()

다국어 설정 값에 따라 `List.template` 및 `List_multiLang.template` 템플릿을 함께 변경하십시오.

Table 2-6 TYPE_BASE=List인 경우, getWebsquareList() 메소드 변경

TYPE_BASE	다국어 설정값	함께 수정할 템플릿
List	"N"	List.template
List	"Y"	List_multiLang.template

그림 2-9 @~@ : 고정된 값을 치환



그림 2-10 @~@~ : 동적인 값을 치환



2.4.3.3. 소스 적용

아래의 순서로 소스를 적용하십시오.

1. Java Project Build Path를 설정합니다.
2. 구현한 Adapter를 적용합니다.
3. 서버를 재기동합니다.

그림 2-11 inswave-spring-1.1.jar / websquare-ai-admin-core-1.1.0.jar (예시)

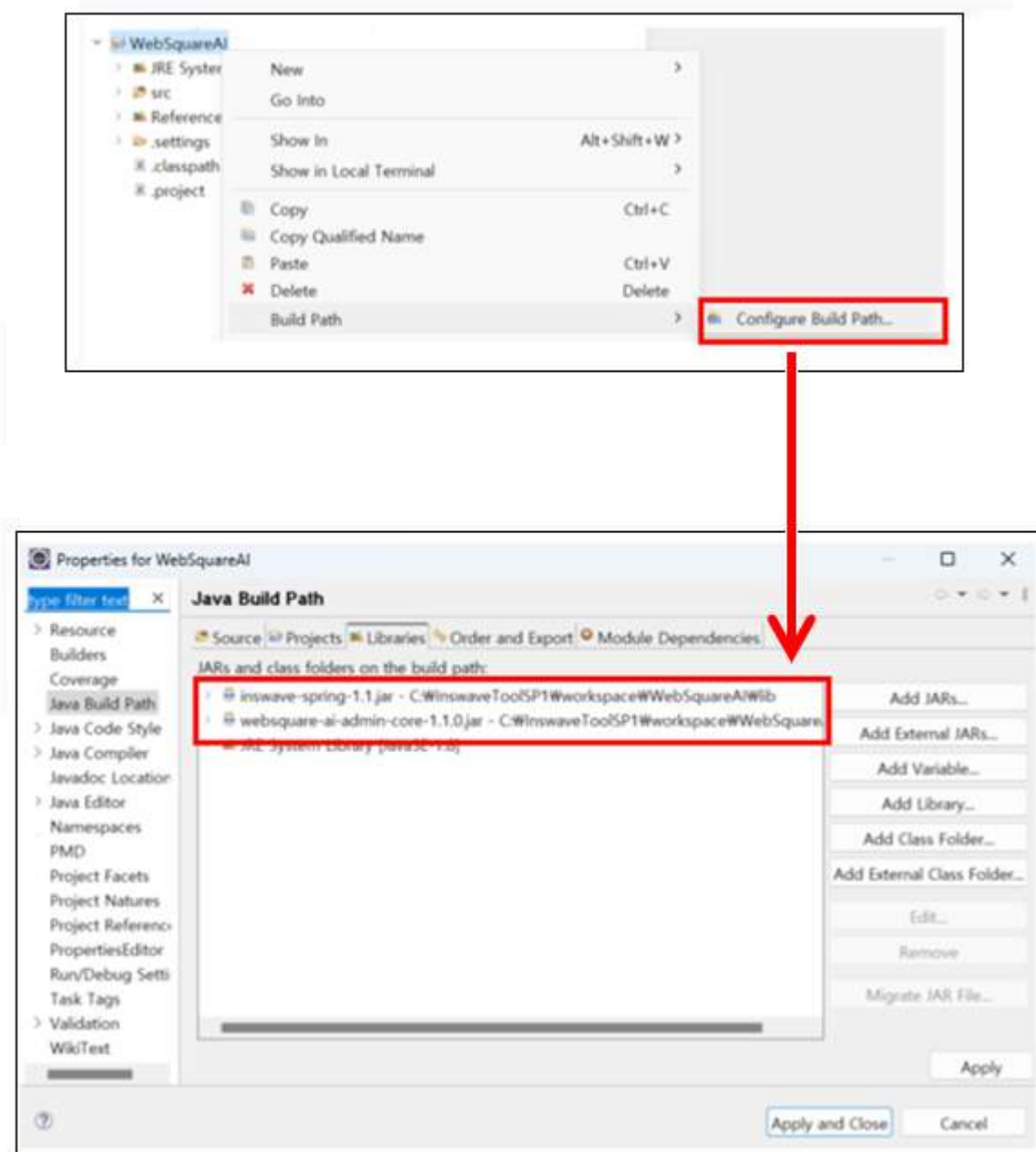


그림 2-12 Adapter 구현

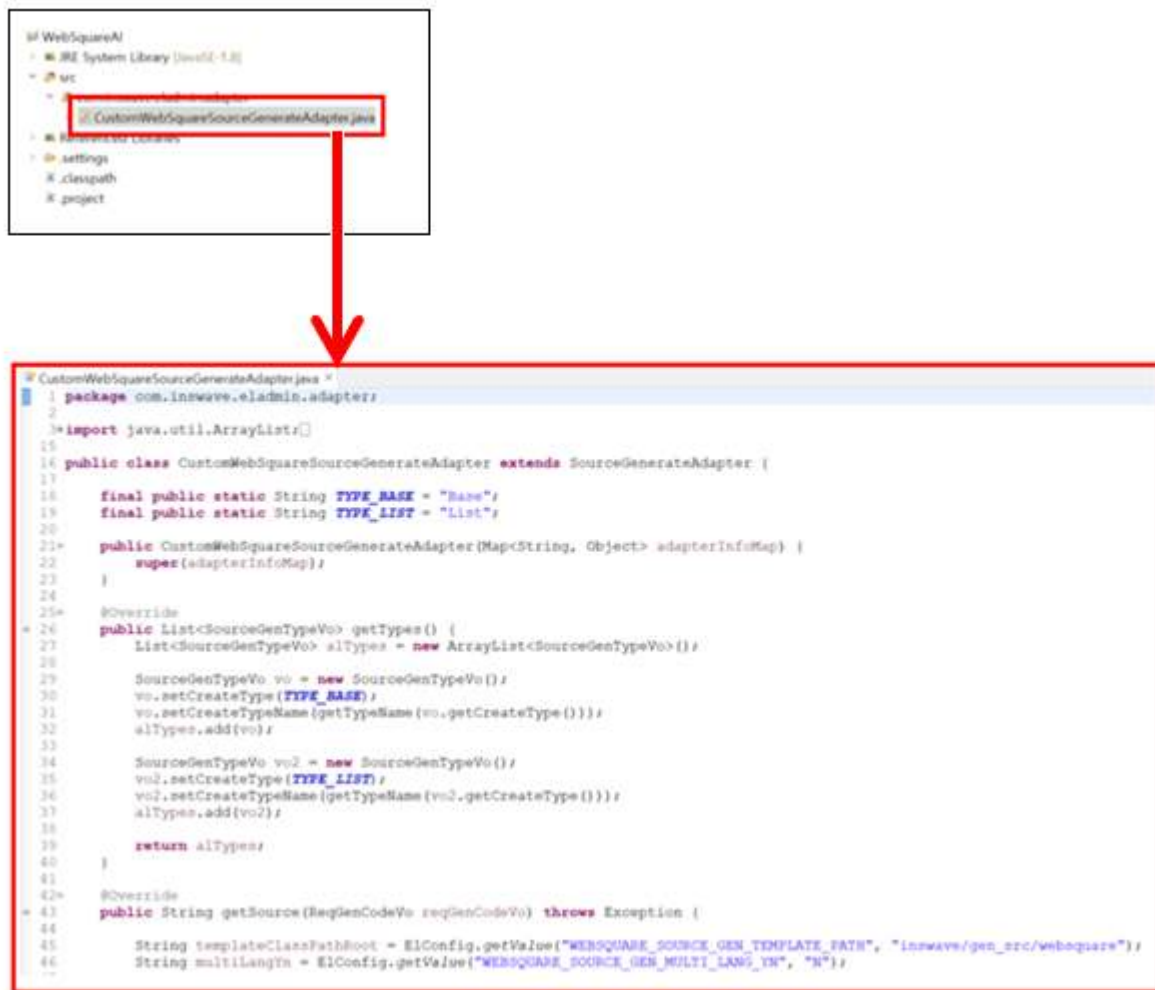


그림 2-13 구현한 어댑터 적용 (서버 class path로 설정한 위치에 class 파일 배포)

```

[webadm@proworks5-dev adapter]$ pwd
/home/webadm/classes/com/inswave/eladmin/adapter
[webadm@proworks5-dev adapter]$ ls -al
total 16
drwxr-xr-x 2 webadm sw      56 Mar 12 16:45 .
drwxr-xr-x 3 webadm sw      21 Mar 12 16:44 ..
-rw-r--r-- 1 webadm sw 15436 Mar 12 16:45 CustomWebSquareSourceGenerateAdapter.class

```


3. 엔진

3.1 설치

웹스퀘어 AI 엔진은 JAR 파일 형태로 제공되어 웹 서버에 설치되며, 설치 사양은 웹 서버 사양을 따릅니다. 단, 대용량 파일 업로드/다운로드와 같은 웹을 통해 제공하는 서비스 종류 및 규모에 따라 웹서버 환경을 구성할 것을 권장합니다.

3.2 사양

Table 3-1 지원 범위

구분	내용	
WebSquare AI 설치 서버	CPU	• 4 core 이상.
	메모리	• 8G 이상.
	디스크	• 100GB 이상.
	운영체제	• Unix • Linux (CentOS 7.x, Ubuntu, RHEL) • Windows
	JVM	• JDK 1.8 이상.
WAS	• 아래를 포함한 J2EE 기반의 모든 WAS 지원 ◦ WebLogic ◦ WebShpere ◦ JEUS ◦ Tomcat ◦ Jboss	
Web Server	• HTTP1.1 표준을 준수하는 모든 Web Server ◦ WebTier, ◦ Webtob ◦ Apache 등	
GPU 서버 (AI 증강 개발용)	GPU	Nvidia A100 또는 상위 모델
	GPU 메모리	40GB
	CPU	16 Core
	CPU 메모리	64GB

3.3 앱 실행 클라이언트 사양

Table 3-2 클라이언트 사양

항목	내용
CPU	i3 이상
메모리	4G 이상
Disk	100GB이상
운영체제	- Linux - Windows 10 이상 - Mac OS X - iOS - Android

3.4 브라우저 지원

Chrome

권장: Chrome 115 이상 (2023년 7월 릴리즈.)

최소: Chrome 90 (2021년 4월 릴리즈.)

Edge

권장: MS Edge 83 이상 (2020년 릴리즈.)

최소: MS Edge 40 (2017년 릴리즈.)

Firefox

권장: Firefox 115 ESR 이상 (2023년 7월릴리즈.)

최소: Firefox 78 ESR (2020년 6월 릴리즈.)

Safari

권장: Safari 16.0 이상 (2022년 9월 릴리즈.)

최소: Safari 14.0 (2020년 9월 릴리즈.)

iPhone

권장: iOS 16.0 이상 (2022년 9월 릴리즈.)

최소: iOS 10.0 (2016년 9월 릴리즈.)

Android

권장: Android 13 (2022년 릴리즈.)
최소: Android 8.0 (2017년 릴리즈.)



에버그린 브라우저 (Evergreen Browser) 정책

사용자에 의한 업데이트가 없어도 자동으로 브라우저를 업데이트하는 정책으로 최신 버전과 직전 버전까지 정상 동작을 보장합니다.
웹 브라우저 지원은 에버그린 브라우저 정책을 따릅니다.

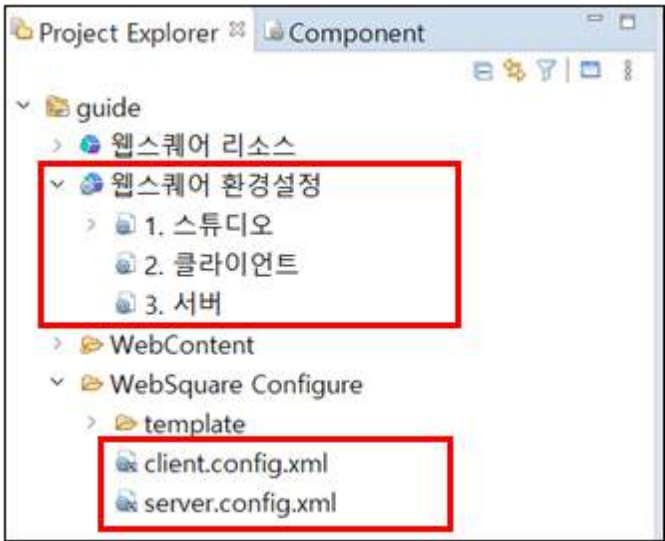
3.5 설정

프로젝트의 웹스퀘어 환경설정 혹은 **WebSquare Configure** 폴더에 있는 **client.config.xml** 및 **server.config.xml** 파일을 클릭하여 직접 설정을 변경할 수 있습니다.

Table 3-3 설정 항목

클라이언트 설정 (<i>client.config.xml</i>)	서버 설정 (<i>server.config.xml</i>)
브라우저 상의 UI 화면 출력과 관련된 내용	웹스퀘어 AI Engine의 동작과 관련된 내용 • WebSquare 엔진 모듈 처리 설정 • Excel, CSV 업로드/다운로드 설정 • 다국어 처리 설정 • WebSquare Hybrid 관련 설정

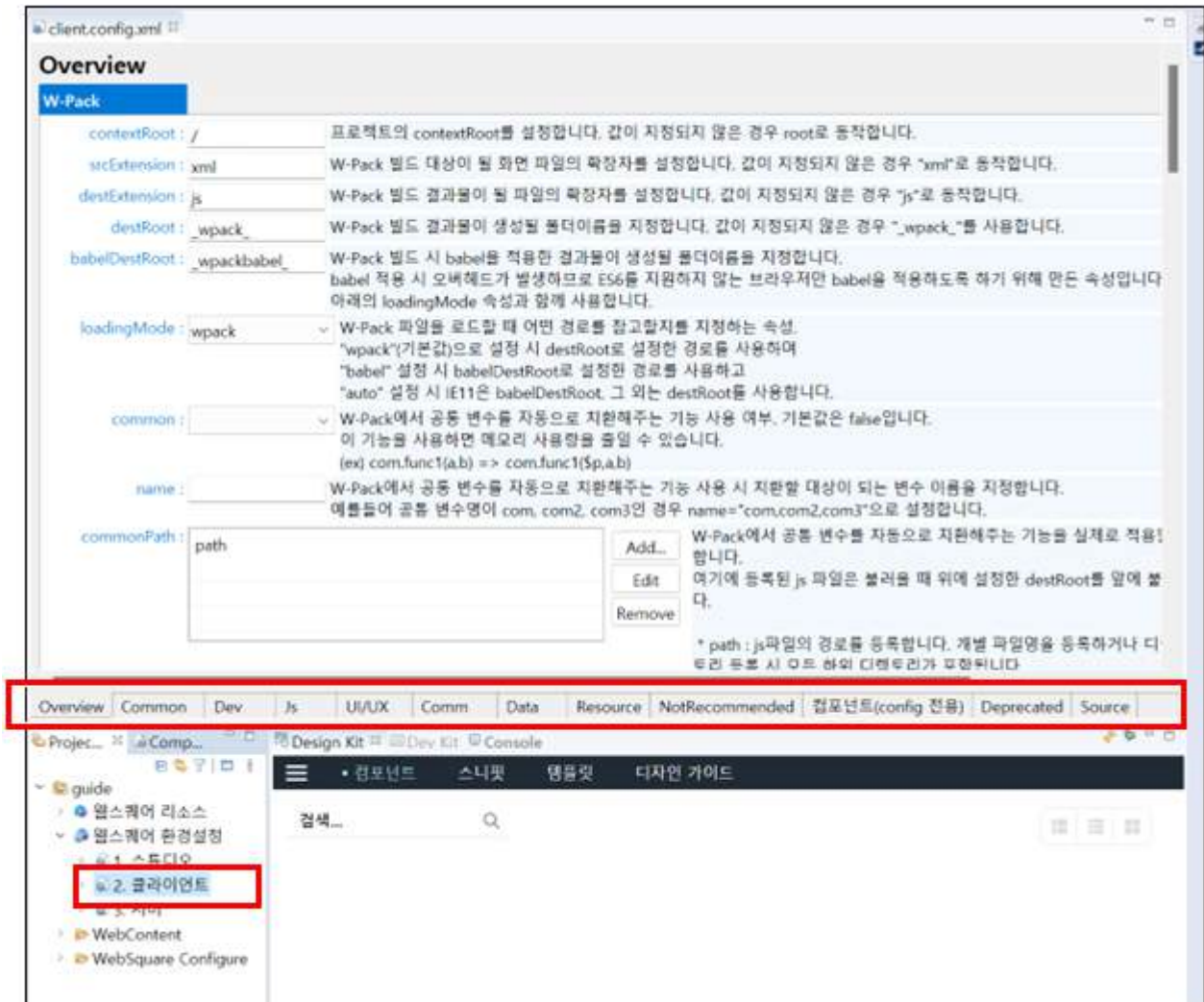
그림 3-1 웹스퀘어 환경설정



3.5.1. 클라이언트 (client.config.xml)

다음은 웹스퀘어 환경설정 - 클라이언트 혹은 `client.config.xml` 파일을 클릭하면 표시되는 화면입니다. 각 설정 항목에 대한 설명이 표시되며, 설정 값을 직접 입력하거나 선택할 수 있습니다. 하단의 카테고리를 클릭하면 해당 부분으로 바로 직접 이동할 수 있습니다.

그림 3-2 클라이언트 설정 (`client.config.xml`)



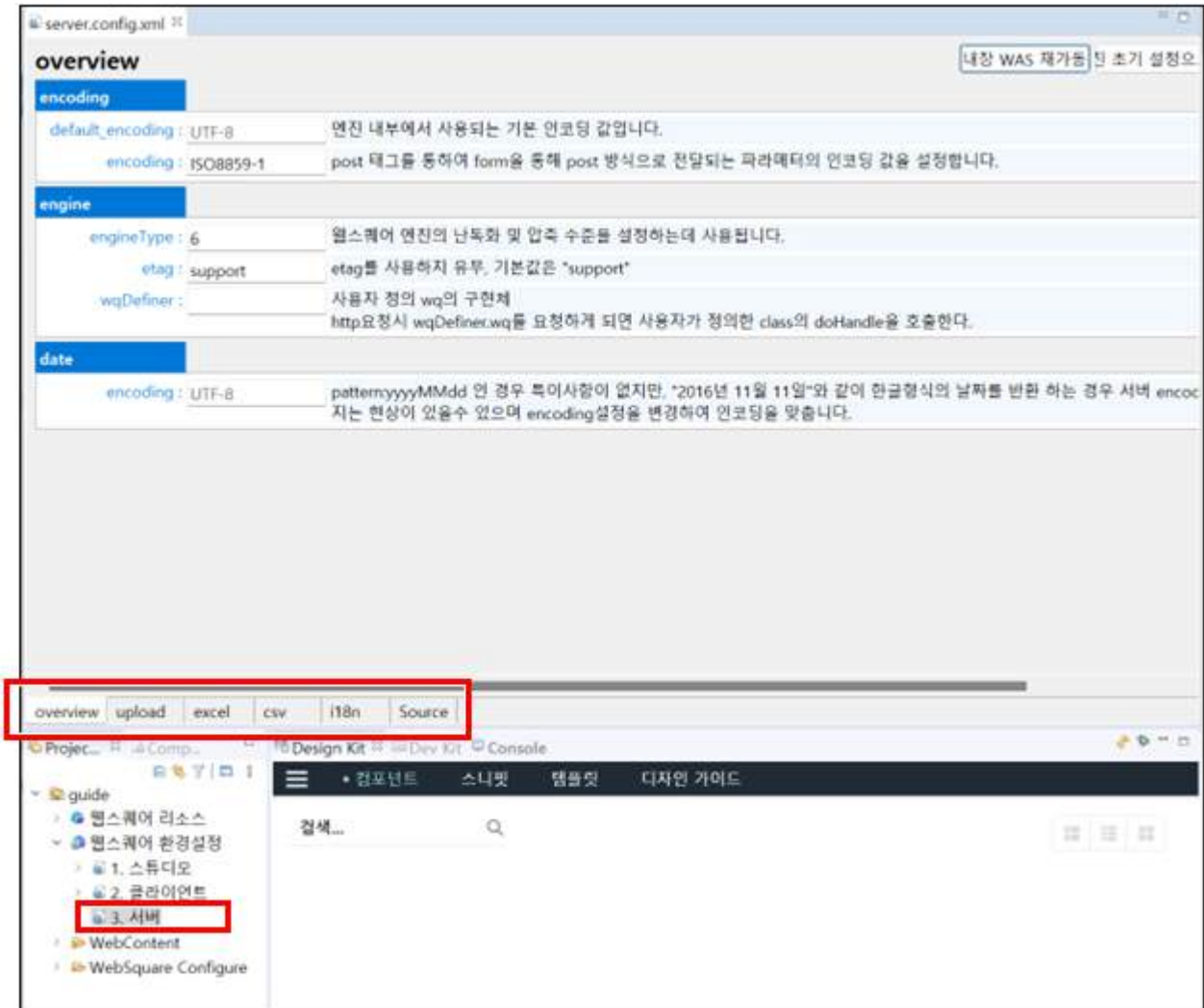
설정 항목에 대한 자세한 설명은 [26. 클라이언트 설정 \(client.config.xml\)](#) 참조.

3.5.2. 서버 (server.config.xml)

다음은 웹스퀘어 환경설정 - 서버 혹은 `server.config.xml` 파일을 클릭하면 표시되는 화면입니다. 각 설정 항목에 대한 설명이 표시되며, 설정 값을 직접 입력하거나 선택할 수 있습니다. 하단의 카테고리를 클릭하면 해당 부분으로 직

접 이동할 수 있습니다.

그림 3-3 서버 설정 (server.config.xml)



설정 항목에 대한 자세한 설명은 [27. 서버 설정 \(server.config.xml\)](#) 참조.

3.6 설치 관련 문제 해결

본 단락은 웹스퀘어 AI 엔진 설치 시 발생할 수 있는 주요 상황 및 그 해결책을 정리한 것입니다. 아래의 이슈에 대한 해결책을 참고하십시오.

Table 3-4 설치 이슈 및 해결 방법

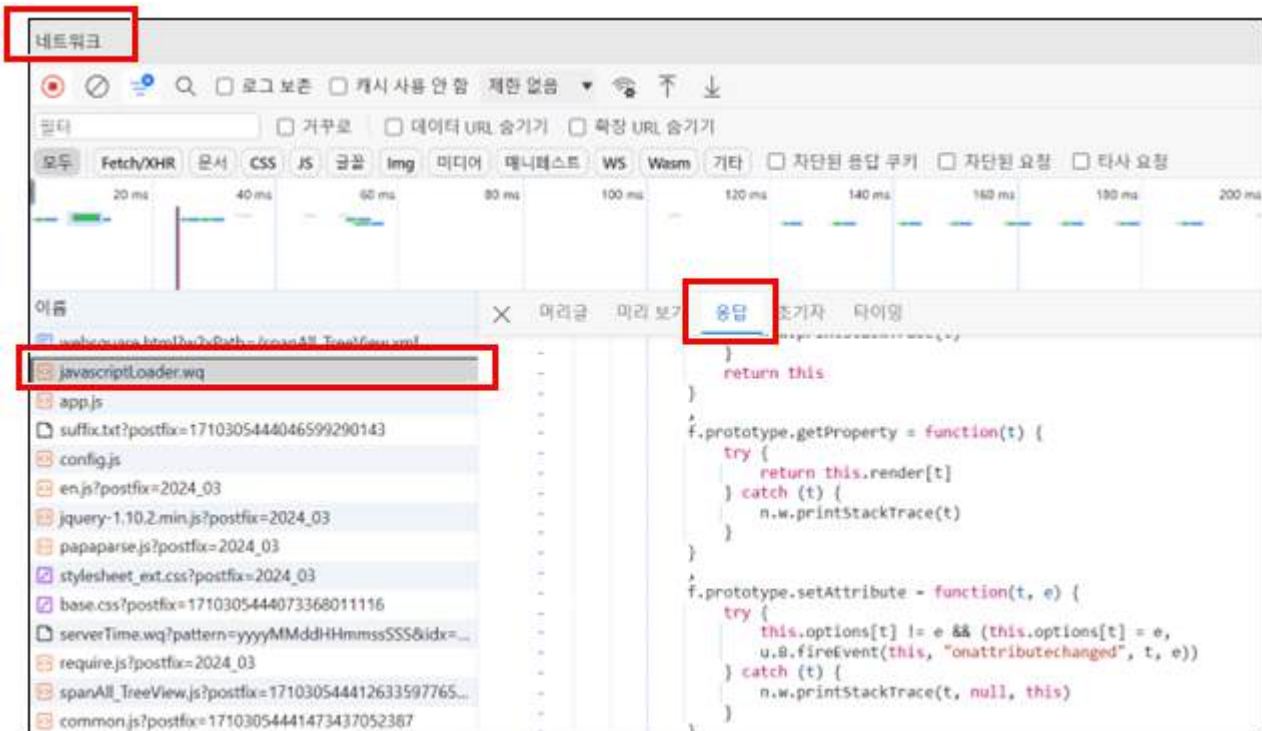
No.	이슈
1	3.6.1. "WebSquare.startApplication is not function"
2	3.6.2. WebServer와 WAS 분리
3	3.6.3. /javascriptLoader.wq 404 error
4	3.6.4. /javascriptLoader.wq 400(Bad Request) error
5	3.6.5. websquare.html 전송 이후 반응이 없을 때
6	3.6.6. WEBSQUARE_HOME 설정
7	3.6.7. 솔라리스OS 에서 CPU코어 개수 확인
8	3.6.8. 설치/패치 시 충돌 가능 jar 목록
9	3.6.9. JBOSS 설치 시 유의사항
10	3.6.10. 설치 할 때 확인해야 할 contentType
11	3.6.11. 웹로직 설치의 경우 web.xml 파일에 mime-mapping 설정
12	3.6.12. JEUS에서 한글이 깨지는 경우
13	3.6.13. 웹서버가 SUN iplanet(혹은 웹티어)인 경우
14	3.6.14. WAS가 websphere 일 server.config.xml 추가 설정

3.6.1. "WebSquare.startApplication is not function"

"WebSquare.startApplication is not function" 경고 창은 라이선스가 유효하지 않을 경우 발생합니다. "WebSquare.startApplication is not function" 경고 창이 발생할 경우 아래의 방법으로 확인하십시오.

1. 브라우저 개발자도구의 네트워크 탭을 선택합니다.
2. javascriptLoader.wq 파일을 선택하고 응답 탭을 클릭합니다.
3. 정상인 경우 응답 탭에 코드가 표시되어야 합니다. (라이선스가 유효하지 않을 경우 아무 내용도 표시되지 않습니다.)

그림 3-4 javascriptLoader.wq 확인



3.6.2. WebServer와 WAS 분리

WebServer와 WAS를 분리해서 사용할 경우, .jar 및 .jsp 파일은 WAS에 나머지 파일은 WebServer에 저장하십시오.

1. WebServer와 WAS에 동일한 폴더 구조를 구성합니다.



1. engine, externalJS, fiddle, uiplugin 폴더를 확인하십시오
2. 특히 externalJS 하위의 editor 폴더에는 같은 이름의 .js 파일이 존재할 수 있습니다.

2. 이후 웹스퀘어 화면 file은 모두 WebServer의 해당 경로로 업로드 해야합니다. (WAS에 올릴 경우, xml 헤더 중복 에러가 발생할 수 있습니다.)
3. 브라우저 개발자 도구에서 GET /websquare/javascriptLoader.wq 이상 진행하지 못하는 경우, .wq 관련 설정을 확인하십시오.

그림 3-5 WebServer 폴더 구조 (예)

externalJS	2016-03-10 오후...	파일 폴더	
fiddle	2016-03-10 오후...	파일 폴더	
images	2016-03-10 오후...	파일 폴더	
message	2016-03-10 오후...	파일 폴더	
skin	2016-03-10 오후...	파일 폴더	
theme	2016-03-10 오후...	파일 폴더	
uiplugin	2016-03-10 오후...	파일 폴더	
blank.html	2016-03-04 오전...	HTML 문서	1KB
blank.xml	2016-03-04 오전...	XML 문서	1KB
config.js	2016-03-04 오전...	JScript 스크립트 ...	7KB
config.xml	2016-03-04 오전...	XML 문서	17KB
debug.html	2016-03-04 오전...	HTML 문서	2KB
devicePreview.html	2016-03-04 오전...	HTML 문서	6KB
popup.html	2016-03-04 오전...	HTML 문서	1KB
websquare.html	2016-03-04 오전...	HTML 문서	1KB
websquare_sap.html	2016-03-04 오전...	HTML 문서	1KB

그림 3-6 WAS 폴더 구조 (예)

engine	2016-03-10 오후...	파일 폴더
externalJS	2016-03-10 오후...	파일 폴더
fiddle	2016-03-10 오후...	파일 폴더
uiplugin	2016-03-10 오후...	파일 폴더



WebServer 및 WAS 설정 과정에서 **.wq** 처리가 필요하기 때문에 서버 엔지니어 지원이 필요합니다.

3.6.3. /javascriptLoader.wq 404 error

페이지 로딩 시 **/javascriptLoader.wq 404 error**가 발생할 수 있습니다. 이 경우 다음을 확인하십시오.

1. WAS 및 WebServer가 설치 경로를 정확히 바라보고 있는가? (일반적인 html파일 등을 올려놓고 확인 가능)
2. WAS 및 WebServer에 설치된 웹스퀘어 AI의 권한은 어떠한가?

3.6.4. /javascriptLoader.wq 400(Bad Request) error

페이지 로딩 시 **/javascriptLoader.wq 400 error**가 발생할 수 있습니다. 이 경우 다음을 확인하십시오.

1. 클라이언트가 보낸 요청에 결함이 있거나 문제가 있는 경우 발생합니다.

- I. spring, @requestParm 어노테이션 사용 시 빈 파라미터 일 때 (required = false 처리 필요)
 - II. http 전송 header 크기 또는 header의 특정 값 (Apache의 경우 크기 조정 가능)
 - III. 브라우저 캐시 및 쿠키, 설정상의 문제
 - IV. 웹방화벽 문제
2. 웹 방화벽에서 해당 페이지 예외 처리 후, 정상 수행되는 것을 확인합니다.
 3. 웹 방화벽에서 해당 페이지 예외 처리 후, 정상 수행되는 것을 확인합니다.

3.6.5. websquare.html 전송 이후 반응이 없을 때

개발자 도구에서 확인 결과 **websquare.html** 은 Status 200으로 정상인 경우에도 아무런 반응이 없을 수 있습니다. Window Server에 설치를 마친 후, 서버를 기동하고 Test페이지를 열어본 순간 아무런 반응이 없을 때가 있습니다. OS에 Chrome이 설치된 경우 아래의 테스트를 수행하십시오.

1. Chrome에서는 정상, IE에서는 반응이 없다면,
 - I. IE - 인터넷 옵션 - 신뢰할 수 있는 사이트 - 사이트를 추가를 선택합니다.
 - II. 해당 주소를 신뢰할 수 있는 사이트로 등록합니다.
 - III. <http://{해당주소}/websquare/javascriptLoader.wq> 를 실행하여 메시지를 확인합니다.
2. Chrome에서도 반응이 없다면,
 - I. 설치 과정에 문제가 있는지 확인하십시오. (개발자 도구의 Network등)



Window Server는 설치 직후 기본적인 보안 관련 제약 사항이 강력하게 설정되어 있습니다. 해당 OS에 해당 브라우저로 실행 했을 때 발생하는 문제입니다.

3.6.6. WEBSQUARE_HOME 설정

해당 서버를 실행시키는 **startUCMS1.sh** 셸 스크립트를 아래와 같이 수정하십시오.

```
JAVA_OPTIONS="${JAVA_OPTIONS} -DWEBSQUARE_HOME=/app/data/UCMS/websquare_home"
export JAVA_OPTIONS
```

3.6.7. 솔라리스OS 에서 CPU코어 개수 확인

psrinfo -pv 명령어로 확인하십시오.

아래 예제의 경우 CPU 코어 개수는 총 16개입니다

```

192.168.70.102 - PuTTY
You have new mail.
Sourcing /root/.profile-EIS.....
root@MPSS-C-WAS-1:/root # psrinfo -pv
The physical processor has 8 cores and 16 virtual processors (0-15)
  The core has 2 virtual processors (0-1)
  The core has 2 virtual processors (2-3)
  The core has 2 virtual processors (4-5)
  The core has 2 virtual processors (6-7)
  The core has 2 virtual processors (8-9)
  The core has 2 virtual processors (10-11)
  The core has 2 virtual processors (12-13)
  The core has 2 virtual processors (14-15)
  SPARC64-X (chipid 0, clock 2800 MHz)
The physical processor has 8 cores and 16 virtual processors (32-47)
  The core has 2 virtual processors (32-33)
  The core has 2 virtual processors (34-35)
  The core has 2 virtual processors (36-37)
  The core has 2 virtual processors (38-39)
  The core has 2 virtual processors (40-41)
  The core has 2 virtual processors (42-43)
  The core has 2 virtual processors (44-45)
  The core has 2 virtual processors (46-47)
  SPARC64-X (chipid 1, clock 2800 MHz)
root@MPSS-C-WAS-1:/root #
  
```

3.6.8. 설치/패치 시 충돌 가능 jar 목록

기존에 서비스가 있는 곳에 설치/패치 시, 충돌 날 수 있는 jar 파일은 아래와 같습니다.

사이트의 JAR 파일	WebSquare의 JAR 파일	비고
<i>crimson.jar</i>	<i>xml_api.jar</i>	DocumentBuilderFactory.class 이하 충돌

아래와 같은 에러 메시지가 발생합니다.

Code 3-1 에러 메시지

```

1 WAS:Upload library(common-fileupload-1.2.jar) loading error occurred. If file upload wasn't
needed, turn off file upload in configuration file(websquare.xml). 이하 생략...
  
```

3.6.9. JBOSS 설치 시 유의사항

1. **web.xml**에 `<display-name>` 태그가 없으면 정상적으로 .war 폴더를 로딩할 수 없습니다. display-name에 해당 .war 이름을 명시합니다.
2. 해당 WEB-INF 폴더에 **jboss-web.xml** 파일이 없으면 정상적으로 해당 폴더를 로딩할 수 없습니다.

3. jboss 모든 버전에서 class-loading 관련 설정이 필요합니다.
4. 해당 .war 폴더에 WEB-INF를 구성해도 무방합니다. (WAS 엔지니어와 확인 필요.)

3.6.10. 설치 할 때 확인해야 할 contentType

설치 시 확인 해야 할 content-Type은 아래와 같습니다. websquare 기동 시 오류가 발생할 경우 개발자 도구를 통해 아래 내용을 확인하십시오. 잘못된 경우 WebServer 또는 WAS의 mime-type을 확인하십시오.

- *config.xml* - application/xml
- *websquare.html* - text/html
- *javascriptPluginAll.wq* - text/javascript
- 화면 xml - application/xml

3.6.11. 웹로직 설치의 경우 web.xml 파일에 mime-mapping 설정

web.xml 파일에 아래 내용을 추가하십시오.

Code 3-2 mime-mapping 설정

```
1 <mime-mapping>
2   <extension>xml</extension>
3   <mime-type>application/xml</mime-type>
4 </mime-mapping>
```

3.6.12. JEUS에서 한글이 깨지는 경우

DB 등에서 조회한 값은 정상 호출하지만 WebSquare XML 파일의 한글이 깨지는 경우가 있습니다. 이 경우 아래 내용을 확인합니다.

1. *config.xml* 등의 한글 주석 등이 깨져있는가?
2. XML 파일만 응답 헤더가 `contentType = application/xml;charset=ISO_8859_1`인가? (다른 확장자는 정상.)

이 현상은 서버 쪽의 인코딩 설정에 관련된 것일 수 있습니다. JEUS의 **WEBMain.xml**을 확인하십시오. 참고로 아래는 한글이 정상 출력되는 인코딩 예입니다.

```

1 <encoding>
2   <request-encoding>
3     <default>UTF-8</default>
4     <forced>UTF-8</forced>
5   </request-encoding>
6   <response-encoding>
7     <default>UTF-8</default>
8     <forced>UTF-8</forced>
9   </response-encoding>
10  <request-url-encoding>
11    <default>UTF-8</default>
12    <forced>UTF-8</forced>
13  </request-url-encoding>
14 </encoding>

```

3.6.13. 웹서버가 SUN iplanet(혹은 웹티어)인 경우

웹서버가 iplanet(혹은 웹티어)인 경우, .wq 확장자에 대한 처리 및 mime-type 설정이 약간 복잡합니다. 이 경우 아래 내용을 확인하십시오.

설정 파일 위치를 확인합니다.

아래 예제의 경우 `mime.types` 및 `XXXX_obj.conf` 파일을 변경해야 합니다. (경우에 따라서 `magnus.conf`를 변경할 때도 있습니다.

```

ps-BOWEB11      https-EDWEB11      https-NJEFWEB11   jdk
ps-BPWEB11      https-EEWEB11      https-QQWEB11     Legal
ps-BPWEB11      https-EPWEB11      https-SSWEB11     lib
ps-BTWEB11      https-FMWEB11      https-USWEB11     plugins
ps-COWEB11      https-HI2WEB11     https-WAWEB11     README.txt
ps-EAWEB11      https-MBWEB11      https-WSWEB11     setup
sunuser@dbrt-fwork01: /Product/webserver$ cd https-WSWEB11
sunuser@dbrt-fwork01: ? Product/webserver/https-WSWEB11$ ls
auto-deploy  collections  generated     logs          web-app
in           config       lock-db      sessions
sunuser@dbrt-fwork01: ? Product/webserver/https-WSWEB11$
sunuser@dbrt-fwork01: ? Product/webserver/https-WSWEB11$ ls
auto-deploy  collections  generated     logs          web-app
in           config       lock-db      sessions
sunuser@dbrt-fwork01: ? Product/webserver/https-WSWEB11$ cd config
sunuser@dbrt-fwork01: ? Product/webserver/https-WSWEB11/config$ ls
cert8.db      key3.db      mime.types   server.xml
certmap.conf  keyfile     obj.conf     WSWEB11-obj.conf
default-web.xml login.conf   secmod.db
default.acl   magnus.conf server.policy
sunuser@dbrt-fwork01: ? Product/webserver/https-WSWEB11/config$ pwd
/Product/webserver/https-WSWEB11/config
sunuser@dbrt-fwork01: ? Product/webserver/https-WSWEB11/config$

```

mime-type을 설정합니다. (예: `type=text/wq exts=wq`)

다음은 `XXXX_obj.conf` 설정 예시입니다. `ppath=".wq"` 부분을 참고하십시오.

```
Object name="weblogic" ppath="*.jsp">
Service fn="wl_proxy" WebLogicCluster="10.22.10.46:50131" WLIOTimeoutSecs="Unlimited" I
dempotent="OFF" DebugConfigInfo="ON" DynamicServerList="OFF"
</Object>

<Object name="weblogic" ppath="*.nhi">
Service fn="wl_proxy" WebLogicCluster="10.22.10.46:50131" WLIOTimeoutSecs="Unlimited" I
dempotent="OFF" DebugConfigInfo="ON" DynamicServerList="OFF"
</object>

<Object name="weblogic" ppath="*.wq">
Service fn="wl_proxy" WebLogicCluster="10.22.10.46:50131" WLIOTimeoutSecs="Unlimited" I
dempotent="OFF" DebugConfigInfo="ON" DynamicServerList="OFF"
```



웹로직 연계와 관련된 Object가 있다면, 복사하여 .wq 요청에 관련된 내용만 변경해서 사용할 수 있습니다.

3.6.14. WAS가 websphere 일 `server.config.xml` 추가 설정

상황	조치 사항
• WebServer(IHS, IBM Http Server)와 WAS(Websphere)가 있고, 모든 요청을 WAS에서 처리. • 일반적인 web - contextRoot에 WebSquare를 설치 한 후 WAS 재가동. • 샘플 페이지로 접속 시도. • websquare.html 파일은 정상적으로 가져오지만 java script.wq 요청에 대해 404 에러 발생.	1. 다른 곳에 있는 server.config.xml 파일에 websquare servlet을 설정하여 정상 작동 확인. server.config.xml의 대략적인 경로: Appserver/profiles/Appsvr1/config/cells/.../deployments/ ... /WEB-INF/server.config.xml.



Websphere만의 특성인지는 추가 확인이 필요.

4. 동작

4.1 코드 구조

다음은 웹스퀘어 AI로 생성되는 XML 코드와 일반적인 HTML 코드 구조를 비교한 것입니다.

그림 4-1 웹스퀘어 AI 코드 vs. HTML 코드



Table 4-1 WebSquare XML 구성 요소

요소			설명
<head>	<xf:model>	<w2:dataCollection>	- 데이터 객체를 정의하는 영역 - DataMap - DataList

요소			설명
			◦ <code>LinkedList</code> • 서버 통신을 위한 <code>request</code>, <code>response</code> 데이터와 화면에서 사용할 데이터를 정의
		<code><xf:workflow></code>	<code>submit</code> , <code>submitDone</code> 의 실행 순서 • 여러 개의 <code>Submission</code>을 실행할 경우 사용. • 실행 순서, 결과 처리 순서, 결과에 따른 이후 <code>Submission</code>의 실행 여부등을 정의. • 조회(<code>Select</code>) 용도의 통신에 사용할 것을 권장.
		<code><xf:submission></code>	서비스 호출에 필요한 <code>submit</code> 을 정의 각 <code>submit</code> 은 고유 ID를 포함 • 서버 통신을 위한 인터페이스 • 용도 별로 여러 개 정의 가능. • 통신 방식(동기/비동기) 선택 가능. • 통신 전/후에 실행할 함수 정의 가능. • 전송할 데이터(<code>request</code> 혹은 <code>ref</code>)와 수신할 데이터(<code>response</code> 혹은 <code>target</code>)를 설정
	<code><script></code>		글로벌 스크립트 정의 컴포넌트의 이벤트 함수 정의 • 업무 로직 구성 ◦ 글로벌 스크립트 정의 ◦ 컴포넌트의 이벤트 함수 정의
	<code><style></code>		스타일 지정
<code><body></code>			컴포넌트, 컴포넌트 속성, 화면 UI 정보 포함 • Design 탭에서 그리는 화면 UI 정보 ◦ 컴포넌트 배치 및 구성 ◦ 각 컴포넌트 속성

4.2 스크립트

4.2.1. `scwin.onpageload`

`onpageload`는 WebSquare Page 파일 생성 시 자동 생성되는 이벤트입니다. Script 탭에서 `onpageload` 및 `onpagenload` 이벤트를 정의할 수 있으며, `onpageload` 이벤트는 페이지 로딩 후 실행됩니다.

그림 4-2 페이지 컴포넌트 생성

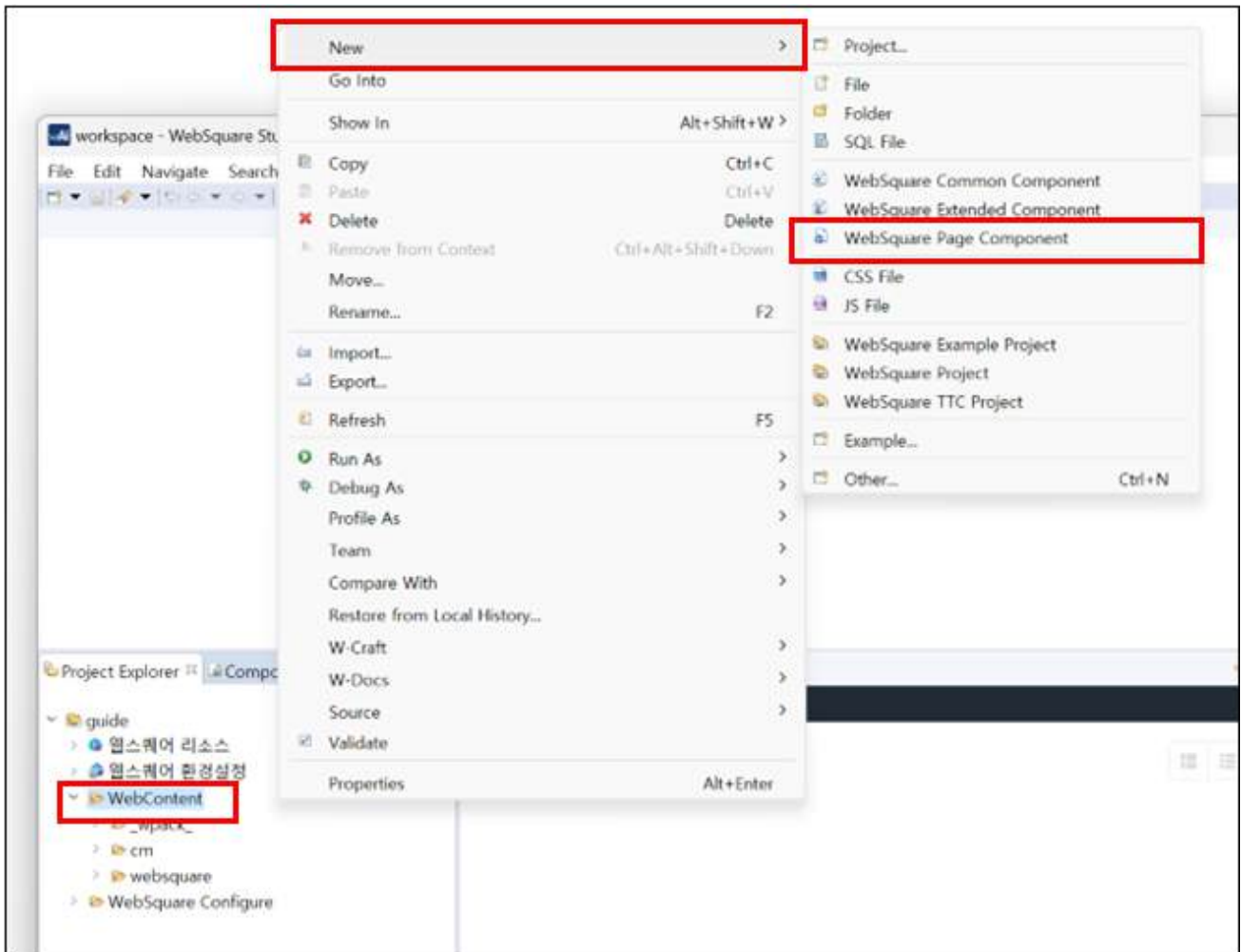
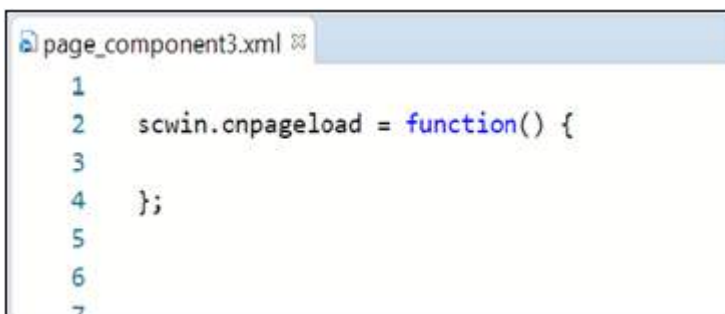


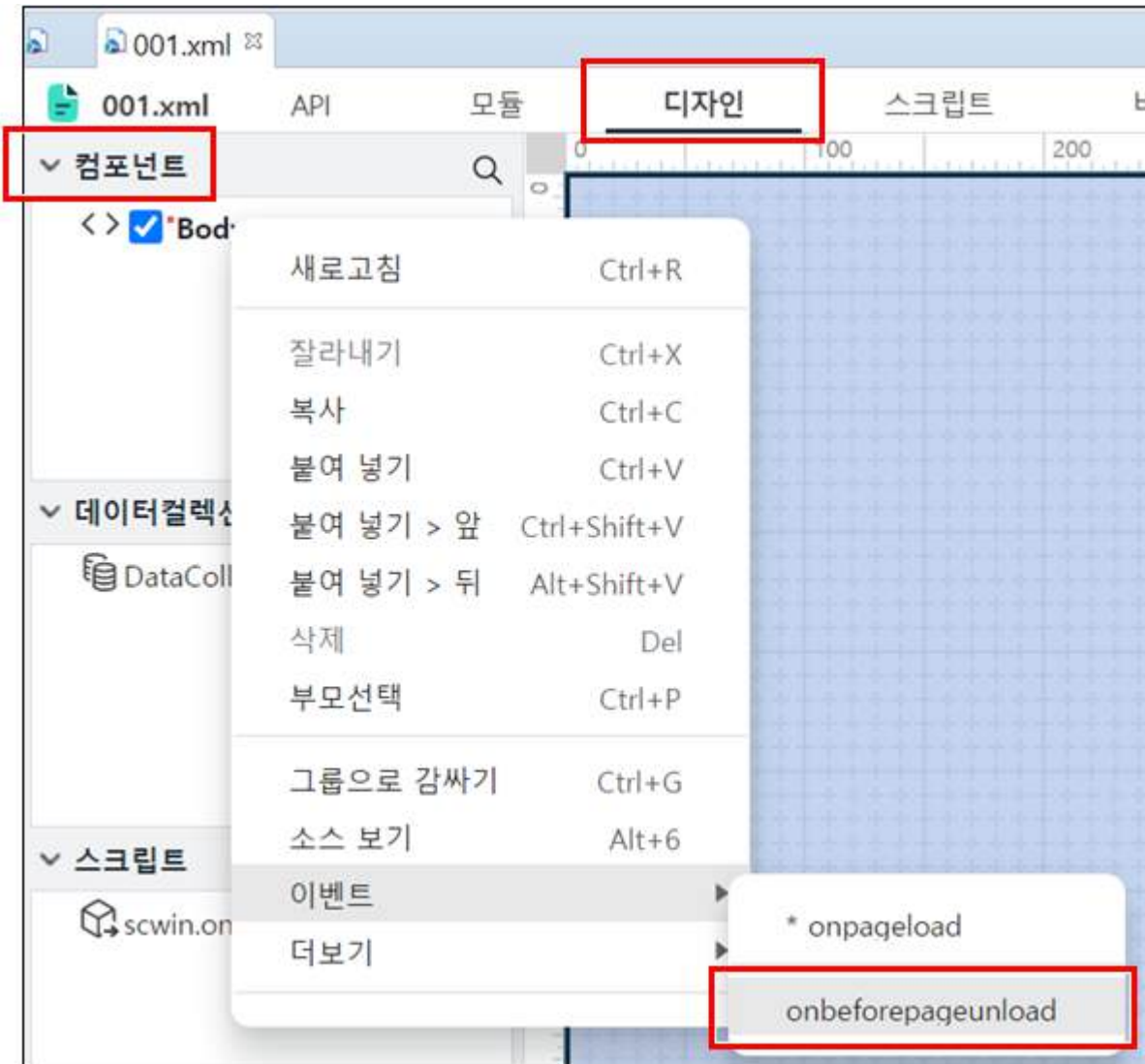
그림 4-3 페이지 컴포넌트 생성 후



4.2.2. scwin.onbeforepageunload

`onbeforepageunload` 이벤트는 WebSquare 페이지를 닫을 때 body 부분에서 발생하는 스크립트입니다. `onbeforepageunload` 이벤트는 디자인 에디터의 디자인 - 컴포넌트 탭에서 Body를 우클릭하여 추가할 수 있습니다.

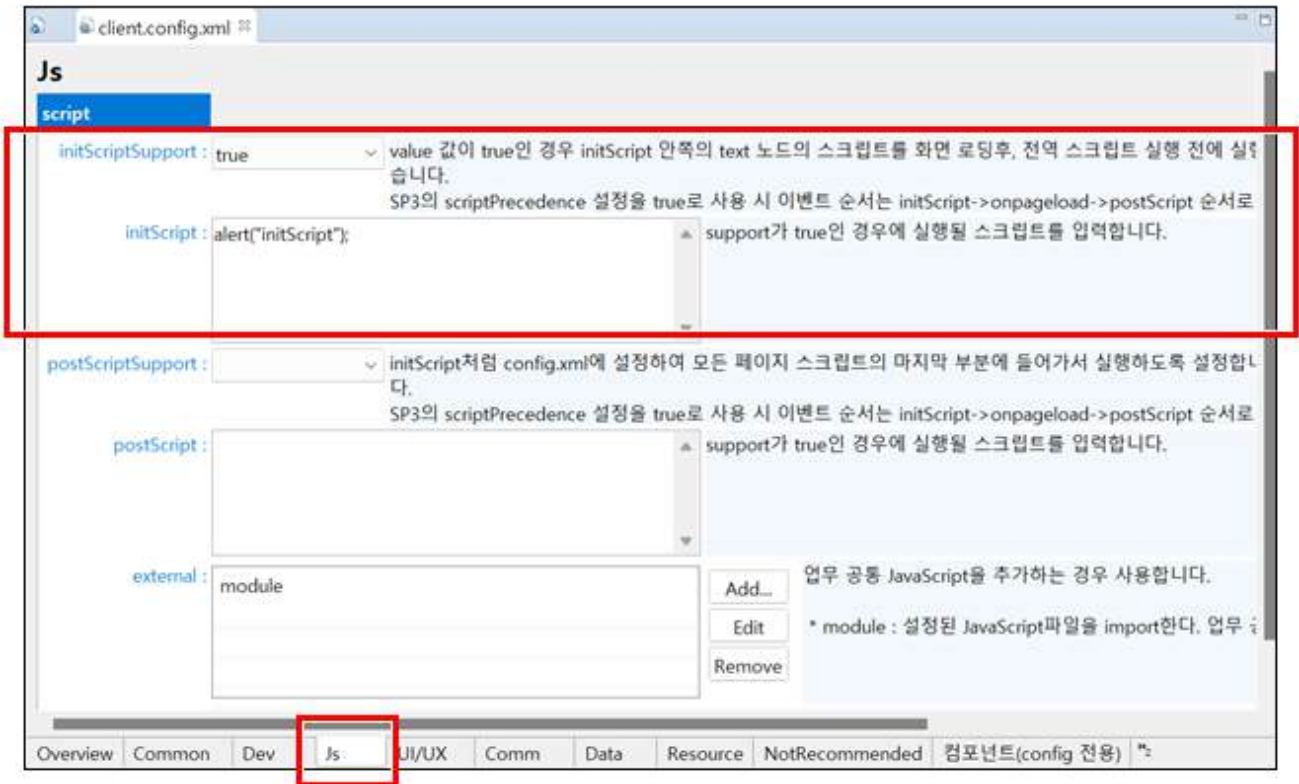
그림 4-4 Body에 이벤트 추가



4.2.3. <initScript>

아래와 같이 `client.config.xml` 파일을 설정하여 `initScript`를 등록할 수 있습니다. 페이지 로딩 직후 실행되며 `scwin.onpageload` 스크립트 이전에 실행됩니다.

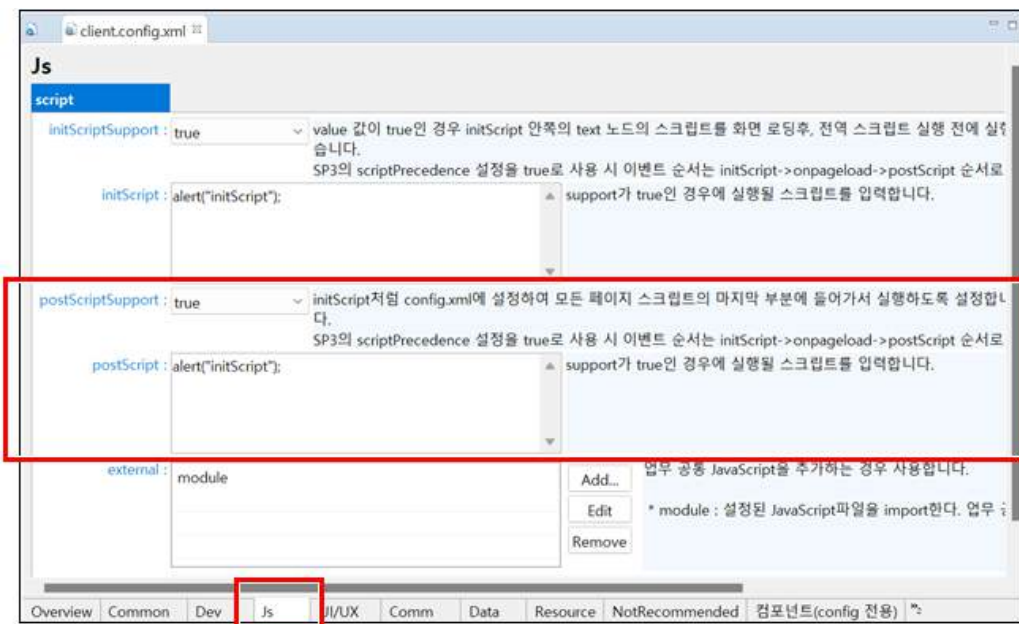
그림 4-5 client.config.xml 설정 예 (initScript)



4.2.4. <postScript>

아래와 같이 client.config.xml 파일을 설정하여 postScript를 등록할 수 있습니다. postScript는 페이지 로딩 시 `swin.onpageload` 스크립트가 실행된 후에 실행됩니다.

그림 4-6 client.config.xml 설정 예 (postScript)



4.2.5. 스크립트 실행 순서

Scope 기능 사용 시 스크립트는 아래의 순서로 동작합니다.

1. 외부 JS 파일
2. 로컬 스크립트
3. `<initScript>`
4. `onpageload`
5. `<postScript>`

메인 페이지가 WFrame을 포함하는 경우, 실행 순서는 아래와 같습니다.

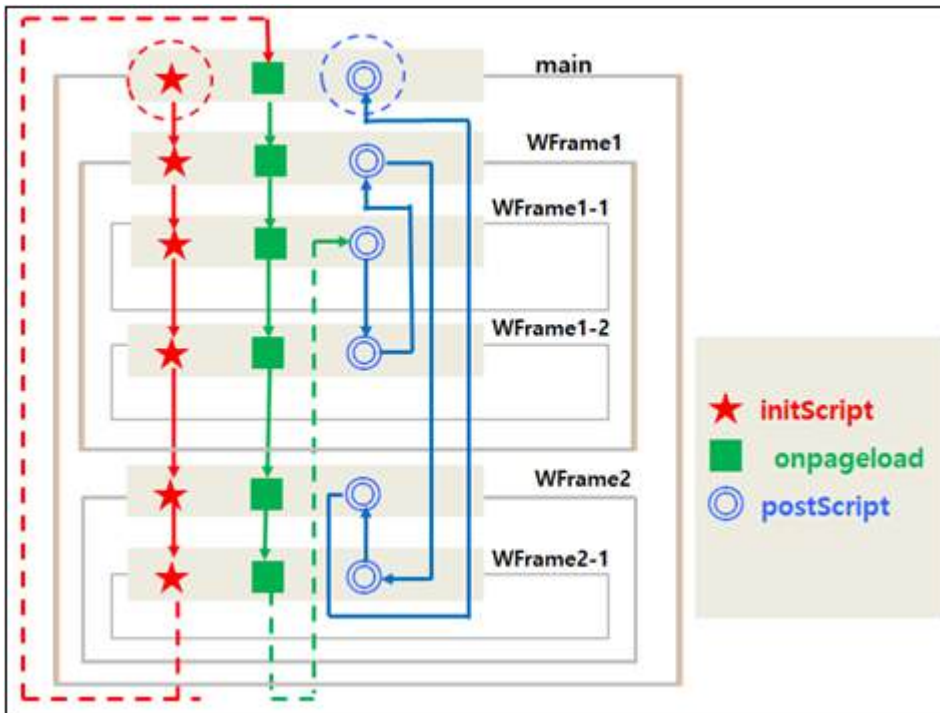
Table 4-2 스크립트 실행 순서

순서	대 항목	세부 순서	세부 항목
1	외부 JS 파일	1-1	메인 페이지
		1-2	WFrame
2	로컬 영역의 스크립트	2-1	메인 페이지
		2-2	WFrame
3	<code><initScript></code> (<i>client.config.xml</i> 파일에 정의)	3-1	메인 페이지
		3-2	WFrame
4	<code>onpageload</code>	4-1	메인 페이지
		4-2	WFrame
5	<code><postScript></code> (<i>client.config.xml</i> 파일에 정의)	5-1	WFrame
		5-2	메인 페이지

4.2.5.1. WFrame이 중첩된 경우

메인 화면이 WFrame을 중첩 포함하고 있는 경우, 스크립트 수행 순서는 아래와 같습니다.

그림 4-7 스크립트 실행 순서 (initScript, onpageload, postScript)



위의 그림의 순서를 표로 나타내면 아래와 같습니다.

Table 4-3 스크립트 실행 순서

화면		initScript	onpageload	postScript		
메인		①	⑦			⑱
	WFrame1	②	⑧	⑮		
	WFrame1-1	③	⑨	⑬		
	WFrame1-2	④	⑩	⑭		
	WFrame2	⑤	⑪		⑰	
	WFrame2-1	⑥	⑫		⑯	

4.2.5.2. Scope 기능 미적용 시

참고로 Scope 기능을 사용하지 않는 경우, 스크립트 수행 순서는 아래와 같습니다.

1. 외부 JS 파일:

- I. PageInherit 컴포넌트의 외부 JS 파일 로딩.
- II. 메인 페이지의 외부 JS 파일 로딩.
- III. WFrame 컴포넌트의 외부 JS 파일 로딩.

2. 메인 페이지:

- I. 메인 페이지의 initScript (client.config.xml 파일에 정의된 JS 파일.)
- II. 메인 페이지의 <script> 영역에 정의된 스크립트.

III. 메인 페이지의 `onpageload` 이벤트 (화면의 `<script>` 영역에 정의.)

3. WFrame 컴포넌트:

- I. WFrame 컴포넌트의 `initScript` (`client.config.xml` 파일에 정의된 JS 파일.)
- II. WFrame 컴포넌트의 `<script>` 영역에 정의된 스크립트.
- III. WFrame 컴포넌트의 `onpageload` 이벤트 (화면의 `<script>` 영역에 정의.)

4. postScript:

- I. WFrame 컴포넌트의 `postScript` (`client.config.xml` 파일에 정의된 JS 파일.)
- II. 메인 페이지의 `postScript` (`client.config.xml` 파일에 정의된 JS 파일.)

4.3 W-Pack (JS 변환)

웹스퀘어 AI는 화면 소스를 JavaScript로 빌드할 수 있는 W-Pack을 제공합니다. 스튜디오에서 개발한 화면 파일은 XML 형식으로 생성되며, W-Pack은 이를 JavaScript 파일로 변환합니다.

그림 4-8 W-Pack



파일 크기 감소

압축 및 최소화(Minify) 기능을 제공하여, 기존 XML 형식과 비교하여 파일 크기가 감소되었습니다.

보안성 개선

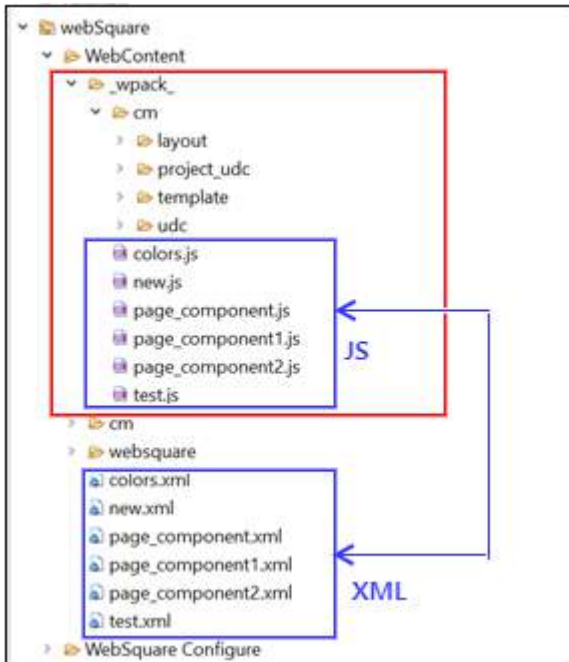
난독화(Obfuscation)를 통해 소스 코드의 보안성을 높였습니다.

속도 개선

웹 환경에 최적화된 JS 파일은 XML과 비교하여 로딩 시간이 짧고 메모리 사용량이 적어, 웹스퀘어 엔진은 JS 파일을 이용하여 더욱 빨리 화면을 브라우저에 그릴 수 있습니다.

XML 화면 파일은 JS 파일로 변환되어 **/WebContent/_wpack_/** 폴더 아래에 저장됩니다. 화면 파일 개발에 사용한 공통 리소스도 JS 파일로 변환되어 **/WebContent/_wpack_/cm/** 폴더 아래에 저장됩니다.

그림 4-9 W-Pack 폴더



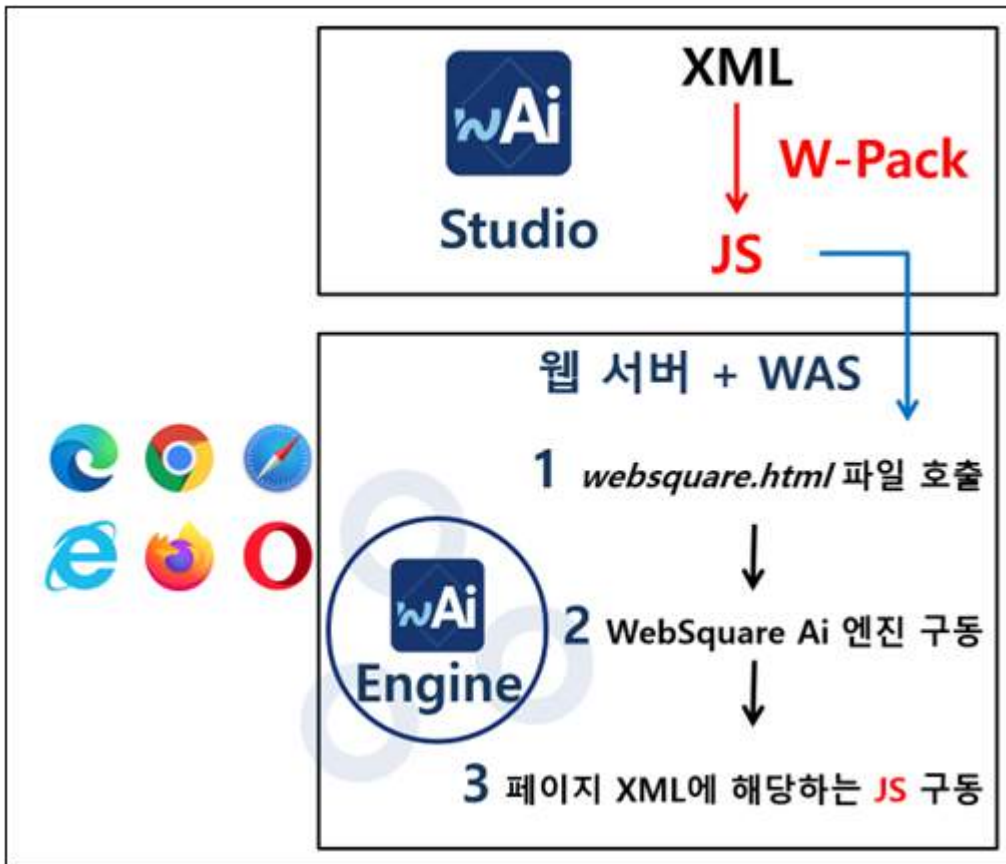
4.4 브라우저 호출

WebSquare Studio에서 작성한 화면은 XML 파일로 생성되며, W-Pack에 의해 JS 파일로 변환되며, 브라우저는 XML 파일을 표시하나 화면을 표시할 때 실제로 사용하는 파일은 JS 파일입니다.

웹스퀘어 AI로 개발한 화면의 브라우저 호출 순서는 아래와 같습니다.

1. 브라우저에서 화면을 호출합니다.
2. 서버 쪽 웹스퀘어 AI 엔진의 **websquare.html** 파일이 호출됩니다.
3. 웹스퀘어 AI 엔진이 구동됩니다.
4. 화면에 해당하는 XML 파일을 URL로 호출합니다.
5. W-Pack에 의해 변환된 JS 파일이 로딩됩니다.
6. 해당 화면이 브라우저에 표시됩니다.

그림 4-10 화면 호출 순서



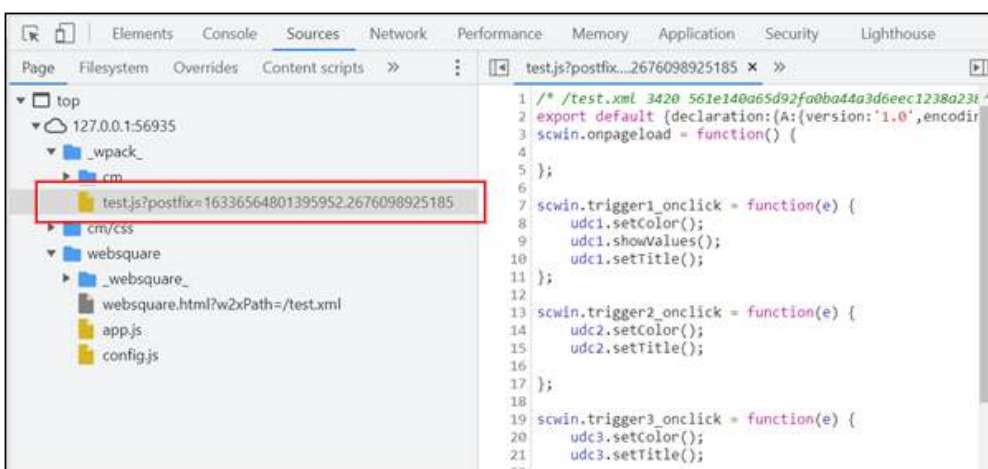
브라우저에는 호출한 화면의 XML 파일 명이 표시됩니다.

그림 4-11 브라우저 표시 주소 (test.xml)



실제 브라우저에 구동되는 파일은 아래와 같이 JS 파일입니다.

그림 4-12 실제 로딩 파일 (test.js)



4.5 CSS

4.5.1. `class` selector 사용

각 화면 내 컴포넌트의 ID는 항상 변합니다. 즉, `input1` 객체는 연결되는 화면에 따라 `frame1 input1`이 될 수도 있고 `frame2 frame22 input1`이 될 수도 있습니다. 따라서 id selector 대신 class selector 사용을 권장합니다.

Code 4-1 사용 예

```
1 // class 사용 (권장)
2 <xf:group id="group1" class="contentZ1"/> .contentZ1 { }
3
4 // id 사용 (비권장)
5 <xf:group id="group1"/> #group1 { }
```



class명 중복 주의

class selector를 사용할 경우 부모 화면에 정의된 스타일 이름과 겹치지 않도록 주의하십시오. 이름이 겹칠 경우 부모 화면의 class를 덮어 쓰게 되어 부모 화면의 스타일이 변경됩니다. class명이 겹치지 않도록 화면 ID를 class명 뒤에 붙이는 방법을 권장합니다.

4.5.2. jQuery의 id selector 사용

Scope 기능을 사용할 경우, 각 화면에 존재하는 객체의 ID는 변합니다. 그러나 jQuery와 같은 외부 모듈을 사용할 경우, ID를 통해 WebSquare 객체에 접근해야 할 경우가 있습니다. 이를 위해 SP3 이상 버전은 외부 모듈이 ID를 통해서 WebSquare 객체에 접근할 수 있는 방법을 제공합니다.

아래와 같이 `$p.$("#" + componentID)` 형식으로 ID를 통해 객체에 접근하십시오.

Code 4-2 ID를 통한 접근 방법

```
1 // Scope 적용 전 접근 방법
2 // $("#input1")
3
4 // Scope 적용 후 접근 방식
5 $p.$("#input1")
```

`$` 대신 `$p.$`를 사용하여 객체를 검색할 수 있습니다. 반환된 결과는 jQuery의 검색 결과와 동일하며, jQuery와 동일

하계 API를 사용할 수 있습니다.

Code 4-3 사용 예

```
1 $.p.$("#link_area .link_text").css("color", "red")
```

4.6 웹표준 호환성

4.6.1. 비표준 태그 처리

컴포넌트 별로 사용자 정의 attribute를 사용하는 경우, w3c validator에서 오류가 검출됩니다. client.config.xml 설정 파일에 `<dataPrefix>` 항목을 아래와 같이 설정하면 "data-" prefix가 추가되어 오류가 발생하지 않습니다.

Code 4-4 설정 예

```
1 <WebSquare>
2     <dataPrefix value="true" />
3 </WebSquare>
```

Code 4-5 적용 전

```
1 <th removeborderstyle="false" or_wd="70" inputtype="select" style="text-align: center;
position: static;" id="grid1_column1" value="" blockselect="false" displaymode="label" datatype
="" customformatter="aaaa" alloption="" chooseoption="" ref="" onchangefunction="doTest"
colindex="0" role="gridcell" class="gridHeaderTDDefault gridHeaderTDDefault_data
grid1_header__column0" col_id="column1" header_id="column1">
2 ...
3 </th>
```

Code 4-6 적용 후

```
1 <th data-or_wd="70" data-inputtype="select" style="text-align: center; position: static;" id
="grid1_column1" data-value="" data-datatype="" data-onchangefunction="doTest" data-colindex=
"0" class="gridHeaderTDDefault gridHeaderTDDefault_data grid1_header__column0" data-col_id=
"column1" data-header_id="column1">
2 ...
3 </th>
```


4.6.2. GridView의 <noBr> 태그

GridView에서 텍스트 데이터를 출력할 경우 <noBr> 태그로 출력됩니다. <noBr> 태그는 비권장 태그입니다. client.config.xml 설정 파일에 <gridView> - <dataTag> 항목을 아래와 같이 설정하여, <noBr> 태그 대신 태그로 출력하십시오.

Code 4-7 설정 예

```
1 <WebSquare>
2     <gridView>
3         <dataTag value="span" />
4     </gridView>
5 </WebSquare>
```

4.6.3. XML의 namespace 제거

client.config.xml 설정 파일에 <removeXmlNamespace> 항목을 아래와 같이 설정하면, XML 관련 namespace를 제거할 수 있습니다.

Code 4-8 설정 예

```
1 <WebSquare>
2     <removeXmlNamespace value="true" />
3 </WebSquare>
```


5. SPA

5.1 Scope 설정

Scope은 웹 페이지를 구성하는 모든 단위 화면에 유효 범위(Scope)를 설정하는 기능입니다. 모든 화면은 Scope 단위로 존재하기 때문에, Scope에 포함된 객체 및 스크립트는 Scope 내에서만 유효합니다. 즉, 동일한 컴포넌트 ID 및 스크립트를 여러 개의 화면에서 함께 사용하더라도 웹 페이지는 정상 동작합니다.

Scope 기능을 사용하기 위해서는 각 화면을 WFrame으로 구성하고 WFrame 내에 표시할 화면(XML 파일)은 `src` 및 `setSrc` API를 통해 정적/동적으로 지정하십시오.

WFrame으로 구성된 화면은 Scope으로 인식되어 각 WFrame 내에 존재하는 객체 및 스크립트는 해당 WFrame 내에서만 유효합니다.

각 WFrame을 Scope 객체로 지정하기 위해서는 SPA Variable에서 Scope 변수명을 설정해야 합니다. 기본 값은 `scwin`입니다. WebSquare5 Studio에서 Page를 생성하여 이벤트를 추가할 경우 해당 함수 앞에 `scwin.`이 자동 추가되고 `scwin.`로 시작하는 스크립트는 자신이 호출된 WFrame 내에서만 실행됩니다.

WFrame의 Scope 기능을 사용하기 위해서는 아래의 설정이 필요합니다.

Table 5-1 Scope 사용에 필요한 설정

No.	항목	설정 위치	설명
(1)	<code>scriptPrecedence</code>	<code>client.config.xml</code>	페이지 로딩 이벤트 순서 보장.
(2)	<code>strictMode</code>	<code>client.config.xml</code>	페이지 내의 객체 참조 제한.
(3)	<code>postDrawMode</code>	<code>client.config.xml</code>	페이지 내 스크립트 즉시 실행 여부.
(4)	SPA Variable	<code>client.config.xml</code>	Scope 변수 지정.
(5)	<code>lazy="false"</code>	화면 파일 (XML)	업무 스크립트 생성 시점.
(6)	<code>mode="sync", scope="true"</code>	<code>client.config.xml</code> (WFrame)	WFrame 속성. 페이지 내 스크립트 즉시 실행 여부.
(7)	<code><getWindow searchTarget="window Id"/></code>	<code>client.config.xml</code> (WindowContainer)	<code>getWindow</code> 함수가 <code>windowId</code> 를 기준으로 동작하도록 설정.

5.1.1. scriptPrecedence

Scope 기능을 사용할 경우 4.2.5. 스크립트 실행 순서에 따라 스크립트가 동작해야 합니다. 이를 보장하기 위해 `client.config.xml` 파일에서 `<scriptPrecedence>` 항목을 아래와 같이 설정하십시오.

설명	페이지 로딩 이벤트 순서 보장.
설정 위치	<code>client.config.xml</code>

Code 5-1 client.config.xml 설정 예

```
1 <scriptPrecedence value="true"/>
```

5.1.2. strictMode

Scope 기능을 사용할 경우, 각 화면은 독립적으로 동작하며 객체의 직접 참조는 WFrame 내에서만 가능합니다. **strictMode**는 메인 화면에 내부적으로 WFrame을 씌우는 기능입니다. **client.config.xml** 파일에 아래 설정을 추가하십시오.

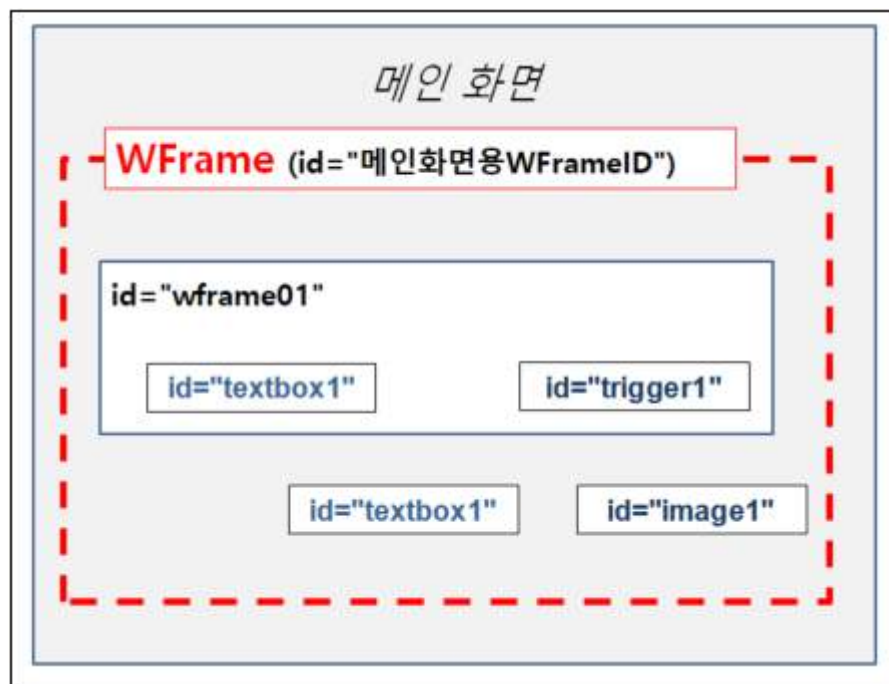
설명	페이지 내의 객체 참조 제한.
설정 위치	client.config.xml

Code 5-2 strictMode 설정 예

```
1 <strictMode value="true" id="mainFrame용WFrameID"/>
```

strictMode의 id로 지정된 값에 따라 메인 화면에 포함된 객체의 ID가 변경됩니다.

그림 5-1 <strictMode value="true" id="mainFrame용WFrameID">



위의 예제에서 **<strictMode value="true" id="mf">**로 설정할 경우, 메인 화면 내에 포함된 textbox1 및 image1 컴포넌트는, ID 앞에 **mf**가 추가됩니다.

Table 5-2 ID 변경 예시

<code><strictMode value = "false"></code>	<code><strictMode value = "true" id="mf"></code>
<code>id="textbox1"</code> <code>id="image1"</code>	<code>id="mf_textbox1"</code> <code>id="mf_image1"</code>



CSS 디자인

strictMode 사용은 최상단 메인 페이지를 WFrame으로 씌우는 것을 의미합니다.

즉, 메인 화면에 속한 컴포넌트의 ID는 "메인 화면에 사용한 WFrame의 ID" + "_" + "컴포넌트 ID"로 변경됩니다.

이는 메인 화면의 CSS 디자인에도 영향을 미칩니다.

예를 들어 #header 대신 #메인화면용WFrame의ID_header 형식을 사용해야 합니다.

좀더 자세한 내용은 [5.9 CSS - 5.9.1. class selector 사용](#) 및 [5.9.2. jQuery의 id selector 사용](#) 부분을 참고하십시오.

그림 5-2 YouTube 동영상 (<https://youtu.be/7jJ8zEbh3M>)

Goal / 목표

Wrap the top-level main page with WFrame to allow object referencing among the main page and the WFrames contained in the main page. / 최상단 메인 페이지를 WFrame으로 감싼 후, 메인 화면과 메인 화면에 포함된 WFrame 간의 객체 참조를 허용.

config.xml

```
<WebSquare>
...
<strictMode value="true" id="mf" />
...
</WebSquare>
```

YouTube

<https://youtu.be/7jJ8zEbh3M>

5.1.2.1. 참고 (JavaScript "with"문)

WFrame의 scope 기능은 JavaScript의 "with" 문을 사용하여 구현하였습니다. "with" 문은 지정한 객체 내에 있는 자원에 빨리 접근할 수 있습니다. 단, "with" 문에 내의 객체가 해당 자원을 포함하지 않을 경우, 객체 외부에서 동일한

이름의 자원을 찾으려는 시도를 합니다.

Code 5-3 "with" 문 사용 예제

```
1 function process(record) {
2     with(record) {
3         addRecord(firstName + " " + lastName, email);
4     }
5 }
```

위의 예에서 record 객체에 firstName이나 lastName 혹은 email 자원이 없을 경우, 부모를 포함한 상위 화면에서 해당 자원을 찾습니다. 그러나, 이 같은 동작은 코드의 명확도를 낮추며 의도하지 않은 동작으로 이어질 수 있습니다. **strictMode**는 자원 참조를 해당 WFrame 내부로만 제한하여 이 같은 단점을 보완합니다.

strictMode는 최상단 메인 화면을 WFrame으로 씌워서 메인 화면 내 컴포넌트에 Scope를 설정합니다. 그 결과 최상단 메인 화면을 포함한 모든 화면을 WFrame 단위로 관리하여 모든 화면에 대해 Scope 기능을 적용할 수 있습니다.

Table 5-3 **strictMode** 비교

항목	<strictMode value="false">	<strictMode value="true">
객체 참조 방법	자신이 속한 WFrame을 확인한 후 부모 및 전역을 참조. 명시적 지정이 필요 없음.	자신이 속한 WFrame 내에서만 찾음. 명시적 지정이 필요. (명시적 지정이 없을 경우, 부모나 전역의 자원을 자동으로 참조하지 않음.)
스크립트가 참조하는 자원이 자신이 속한 WFrame에 없는 경우	부모나 전역을 검색.	오류를 반환.



다른 유효 범위 참조

외부의 WFrame에 있는 객체를 참조하기 위해서는 명시적으로 해당 객체를 지정해야 합니다.
([5.4 WFrame 간의 화면 참조](#) 참고.)

5.1.3. postDrawMode

본 설정은 다중 WFrame 구조를 사용하는 경우 필요합니다. 본 설정을 추가해야 Scope 기능 적용을 위해 정의된 [4.2 .5. 스크립트 실행 순서](#)를 보장할 수 있습니다.

client.config.xml 파일에 아래의 설정을 추가하십시오.

설명	페이지 내 스크립트 즉시 실행 여부.
설정 위치	client.config.xml

Code 5-4 client.config.xml 설정

```
1 <postDrawMode value="synchronous"></postDrawMode>
```



SP1 이전 엔진의 하위 호환성을 유지하기 위해 별도로 설정하십시오.

5.1.4. SPA Variable

각 화면 파일의 스크립트를 해당 WFrame 내에서만 실행하기 위해서는 스크립트 앞에 변수를 추가해야 합니다. 이 변수를 *client.config.xml* 파일에 등록하십시오. 기본 값은 `scwin`입니다.

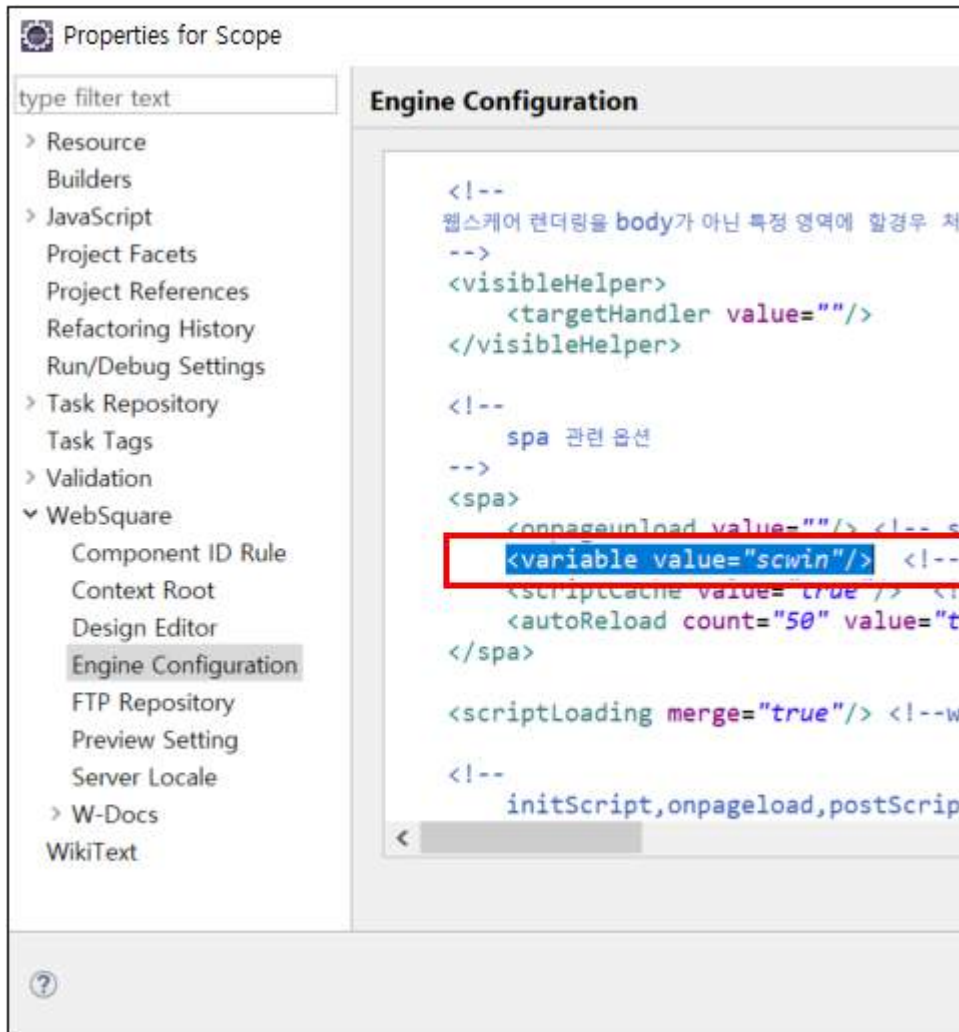
설명	Scope 변수 지정.
설정 위치	client.config.xml

Code 5-5 scope 변수 지정

```
1 <spa>
2   <variable value="scwin"/>
3   ...
4
5 </spa>
```

1. Project Explorer에서 **WebSquare Project**를 우클릭한 후 **Properties**를 클릭합니다.
2. **WebSquare - Engine Configuration**을 선택합니다.
3. `<spa>`의 `<variable>` 옵션을 설정합니다. (여러 개를 등록할 경우, 제일 앞의 변수가 적용됩니다.)
4. Project Explorer에서 **WebSquare Project**를 우클릭한 후 **Properties**를 클릭합니다.
5. **WebSquare - Engine Configuration**을 선택합니다.
6. `<spa>`의 `<variable>` 옵션을 설정합니다. (여러 개를 등록할 경우, 제일 앞의 변수가 적용됩니다.)

그림 5-3 scope 변수 지정



5.1.5. lazy="false"

화면 파일(XML)을 4.2.5. 스크립트 실행 순서에 따라 로딩하기 위해 필요한 설정으로, 각 화면 파일에서 설정합니다.



SP1 Studio 부터 WebSquare Page 선택시 기본 설정으로 선택됩니다.

Code 5-6 lazy="false" 모드

```
1 <script type="text/javascript" lazy="false">
```


5.1.6. <getWindow searchTarget="windowId"/>

WindowContainer를 위한 설정으로 `getWindow` 함수가 `windowId`를 기준으로 동작하도록 설정합니다.

설명	<code>getWindow</code> 함수가 <code>windowId</code> 를 기준으로 동작하도록 설정.
설정 위치	<code>client.config.xml</code> (WindowContainer)

```

1 <windowContainer>
2   <getWindow searchTarget="windowId"/>
3 </windowContainer>

```

5.2 ID 변경

5.2.1. 최상단 메인 화면

`strictMode`를 사용할 경우, 최상단 메인 화면은 WFrame으로 감싸지고 메인 화면에 속한 컴포넌트의 ID는 아래와 같이 변경됩니다.

Table 5-4 ID 변경 예시

<code><strictMode value = "false"></code>	<code><strictMode value = "true" id="mf"></code>
• <code>id="textbox1"</code>	• <code>id="mf_textbox1"</code>



CSS 디자인

`strictMode` 사용은 최상단 메인 페이지를 WFrame으로 씌우는 것을 의미합니다.

즉, 메인 화면에 속한 컴포넌트의 ID는 "메인 화면에 사용한 WFrame의 ID" + "_" + "컴포넌트 ID"로 변경됩니다.

이는 메인 화면의 CSS 디자인에도 영향을 미칩니다.

예를 들어 `#header` 대신 `#메인화면용WFrame의ID_header` 형식을 사용해야 합니다.

좀더 자세한 내용은 [5.9 CSS - 5.9.1. class selector 사용](#) 및 [5.9.2. jQuery의 id selector 사용](#) 부분을 참고하십시오.

5.2.1.1. 예제 (1)

그림 5-4 `strictMode` 적용에 따른 ID 변경 (변경 전)

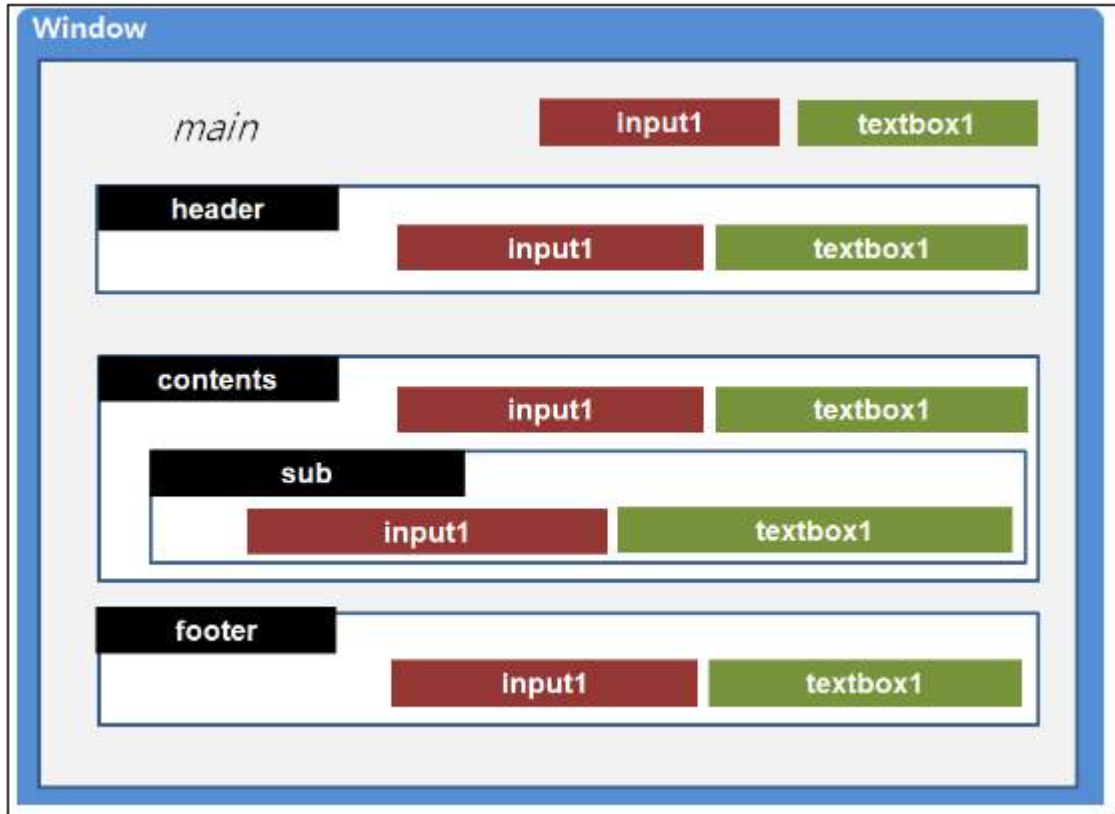
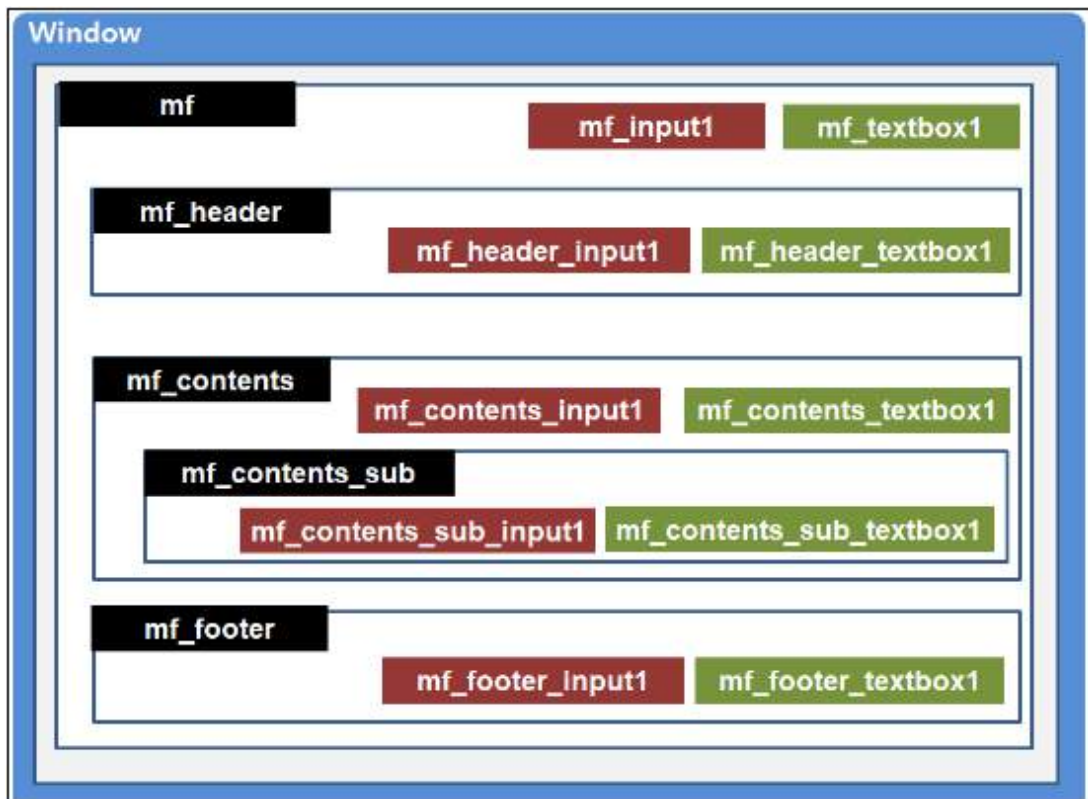


그림 5-5 `strictMode` 적용에 따른 ID 변경 (변경 후)



Code 5-7 strictMode 적용 (client.config.xml)

```
1 <strictMode value="true" id="mf"/>
```

Code 5-8 main.xml

```
1 <body ev:onpageload="scwin.onpageload">
2     <xf:input id="input1" style="width: 120px;height: 21px;"></xf:input>
3     <w2:textbox id="textbox1" label="Home" style=""></w2:textbox>
4     <w2:wframe id="header" style="" src="header.xml" scope="true"></w2:wframe>
5     <w2:wframe id="contents" style="" src="contents.xml" scope="true"></w2:wframe>
6     <w2:wframe id="footer" style="" src="footer.xml" scope="true"></w2:wframe>
7 </body>
```

Code 5-9 header.xml

```
1 <body ev:onpageload="scwin.onpageload">
2     <xf:input id="input1" style="width: 120px;height: 21px;"></xf:input>
3     <w2:textbox id="textbox1" label="Home" style=""></w2:textbox>
4 </body>
```

Code 5-10 contents.xml

```
1 <body ev:onpageload="scwin.onpageload">
2     <xf:input id="input1" style="width: 120px;height: 21px;"></xf:input>
3     <w2:textbox id="textbox1" label="Home" style=""></w2:textbox>
4     <w2:wframe id="sub" style="" src="sub.xml" scope="true"></w2:wframe>
5 </body>
```

Code 5-11 sub.xml & footer.xml

```
1 <body ev:onpageload="scwin.onpageload">
2     <xf:input id="input1" style="width: 120px;height: 21px;"></xf:input>
3     <w2:textbox id="textbox1" label="Home" style=""></w2:textbox>
4 </body>
```

5.2.1.2. 예제 (2)

Code 5-12 main.xml의 Script

```
1 scwin.param = "main.xml";
```

Code 5-13 wframe1.xml의 Script

```

scwin.pageParam = "wframe1.xml";

console.log("scwin.pageParam = " + scwin.pageParam);
console.log("scwin.param = " + scwin.param);
console.log("$p.parent().scwin.param = " + $p.parent().scwin.param);

```

Code 5-14 실행 결과 (strictMode 사용 안 함)

```

1  scwin.pageParam = wframe1.xml
2  scwin.param = main.xml
3  $p.parent().scwin.param = main.xml

```

Code 5-15 실행 결과 (strictMode 사용)

```

1  scwin.pageParam = wframe1.xml
2  scwin.param = undefined
3  $p.parent().scwin.param = main.xml

```

5.2.1.3. 예제 (3)

아래 스크립트는 `id="image1"` 인 컴포넌트의 `title` 속성 값을 찾는 동작을 수행합니다.

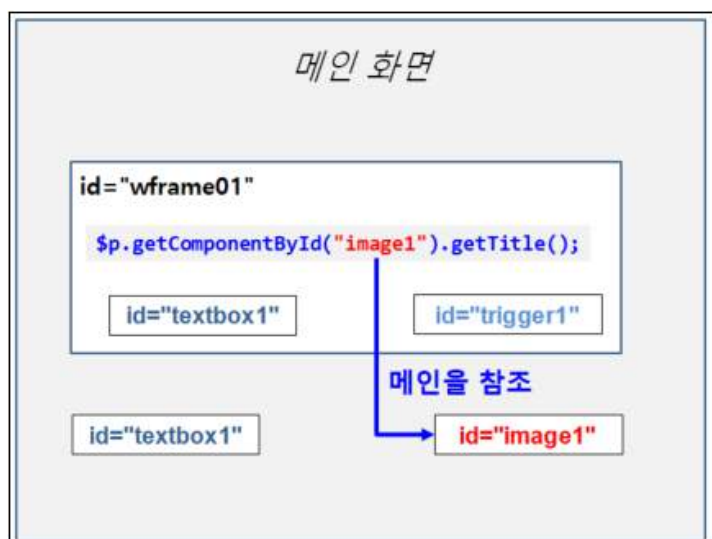
Code 5-16 예제

```

1  $p.getComponentById("image1").getTitle();

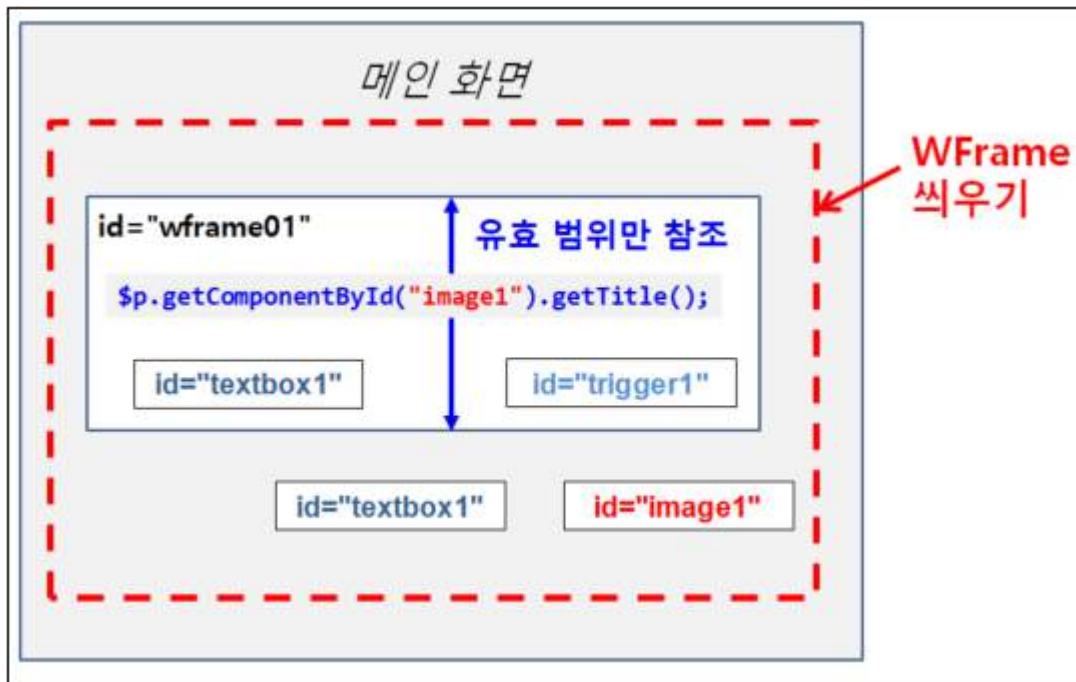
```

`strictMode`를 사용하지 않으면, `id="image1"` 컴포넌트가 존재하지 않을 경우 상위 화면에서 `id="image1"` 컴포넌트를 찾습니다.

그림 5-6 `sstrictMode="false"`

아래와 같이 `strictMode`를 사용할 경우, 스크립트가 정의된 WFrame 화면 내에는 `id="image1"` 컴포넌트가 존재하지 않기 때문에 오류가 반환됩니다.

그림 5-7 `strictMode="true"`



5.2.2. WFrame 내부

WFrame의 `scope="true"` 설정을 사용할 경우, WFrame 내부에 포함된 컴포넌트의 ID는 아래와 같이 변경됩니다. 이를 통해 ID 중복을 원칙적으로 차단할 수 있습니다.

Table 5-5 ID 변경 예시

<code>scope="false"</code>	<code>scope="true"</code>
<code>id="textbox1"</code>	<code>id="WFrame의ID_textbox1"</code>



변경된 ID 사용 금지!

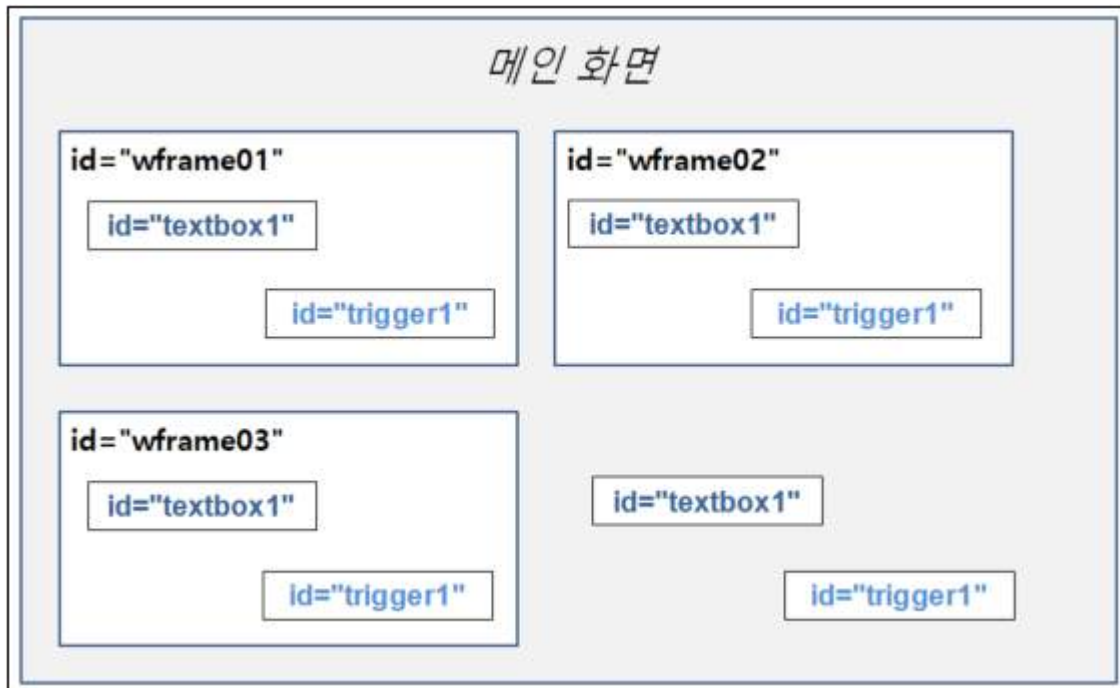
컴포넌트 ID의 변경은 내부적인 동작입니다. 즉, "WFrame의ID_" 문자열을 원래의 컴포넌트 ID 앞에 추가하는 규칙은 스펙에 따라 변경될 수도 있습니다. WFrame의 `scope`가 적용된 전체 ID 사용을 피하십시오. 엔진 업데이트 시 정상 동작하지 않을 수 있습니다.

5.2.2.1. ID 중복 허용

아래 예제에서 메인 화면은 WFrame 3개를 포함합니다. 또한 각 WFrame과 메인 화면은 동일한 ID를 사용하는 Text Box와 Trigger 컴포넌트를 포함합니다.

Scope 기능을 사용할 경우, 5.2.1. 최상단 메인 화면 및 5.2.2. WFrame 내부 화면에 포함된 컴포넌트의 ID가 변경되어 각 화면에 포함된 동일한 ID는 내부적으로는 서로 다른 ID로 변환됩니다. 그 결과 개발자는 ID 중복 걱정 없이 WFrame을 이용하여 화면을 구성할 수 있습니다.

그림 5-8 WFrame을 사용한 화면 구성 (예)



5.3 \$w Vs. \$p

Scope 기능을 적용할 경우, 각 화면은 Scope 단위로 관리됩니다. 따라서 함수 역시 Scope 객체 내에서 실행되며, 해당 함수는 자신이 어느 Scope에서 실행되는지에 대한 정보가 필요합니다. 그러나 자신이 속한 Scope 정보를 별도의 인자로 받도록 처리할 경우, 코딩이 복잡해 집니다.

이를 해결하기 위해 Scope 객체에 기존의 \$w 계열 함수를 모두 추가했습니다. 즉, 유틸리티 기능을 제공하는 \$w.XXX 부류의 API를 scwin.\$w.XXX 형태로 사용할 수 있습니다. 그러나, 이 역시 코딩이 길어지는 문제가 있습니다. 이를 위해 scwin.\$w를 \$p에 매핑하여, scwin.\$w 대신 \$p를 사용할 수 있도록 했습니다.

Code 5-17 \$p 사용

```

1 // (1) Scope 기능을 사용하지 않을 경우
2 $w.createSubmission("submission1");
3
4 // (2) Scope 기능을 사용할 경우
5 // (2-1) scwin을 붙여야 함
6 scwin.$w.createSubmission("submission1");
7
8 // (2-2) scwin.$w를 축약한 $p 사용 가능
9 $p.createSubmission("submission1");

```

5.4 WFrame 간의 화면 참조

5.4.1. 주요 API

Table 5-6 주요 API

API	설명
<code>getWindow()</code>	WFrame의 scope를 반환.
<code>parent()</code>	부모 페이지(WFrame)의 scope를 반환.
<code>top()</code>	최상위 페이지의 scope를 반환. - strictMode를 사용할 경우, 최상위에 id로 지정된 이름으로 자동 생성된 WFrame의 scope가 반환됨. - strictMode를 사용하지 않는 경우, window.top이 반환됨.
<code>main()</code>	페이지의 최상위 scope를 반환. 실제 프레임이 다른 영역까지는 검색하지 않음.

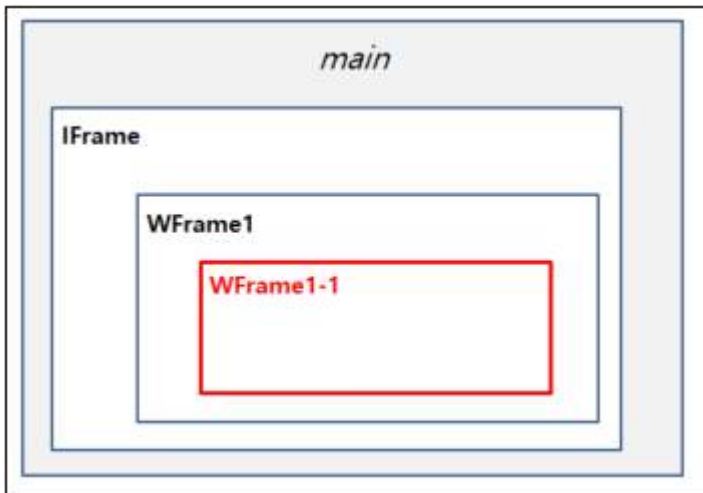


`top` vs. `main`

아래의 그림과 같은 경우 WFrame1-1에서 각 API를 실행하면 서로 다른 결과를 반환합니다.

`top`: main을 반환.

`main`: IFrame을 반환.

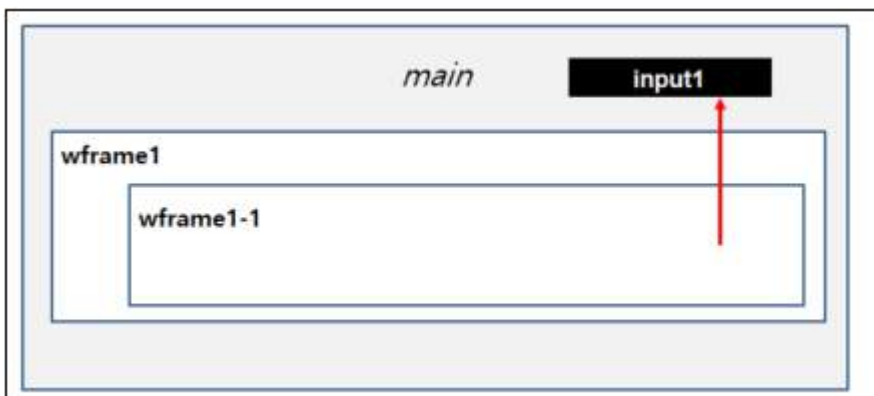


5.4.2. 원칙

5.4.2.1. 자식 → 부모 (\$p.parent();)

Code 5-18 사용 예

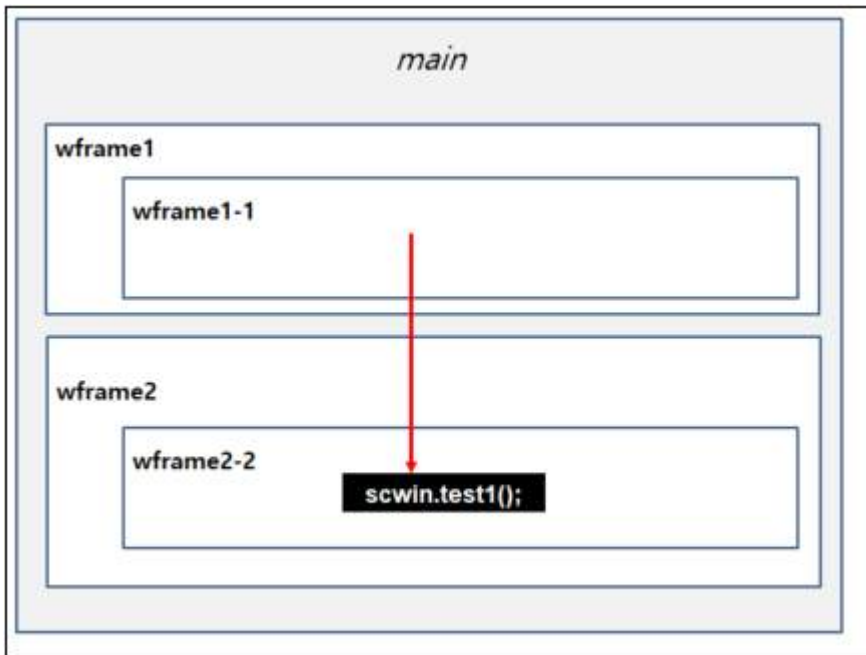
```
1 $p.parent().$p.parent().input1.setValue("test");
```



5.4.2.2. 형제

Code 5-19 사용 예

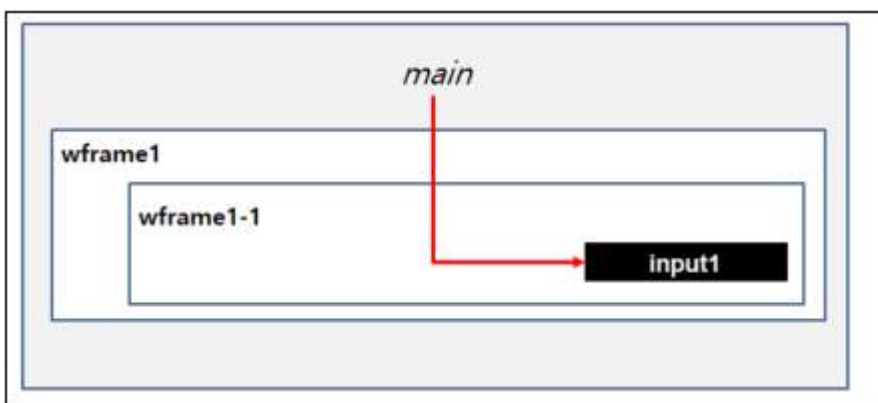
```
1 $p.parent().$p.parent().wframe2.getWindow().wframe2-2.getWindow().scwin.test1();
```

5.4.2.3. 부모 → 자식 (`wframe.getWindow();`)

Code 5-20 사용 예

```
1 wframe1.getWindow().wframe1-1.getWindow().input1.getValue();
```



5.4.3. 정리

5.4.3.1. 부모 → 자식

Table 5-7 코딩 방식 비교

SP1	SP3 이후
<code>getObj</code>	<code>getWindow</code>

Code 5-21 SP1

```
1 wframe1.getObj("wframe11").getObj("input1").getValue();
```

Code 5-22 SP3 이후

```
1 wframe1.getWindow().wframe11.getWindow().input1.getValue();
```

5.4.3.2. 자식 → 부모

Table 5-8 코딩 방식 비교

SP1	SP3 이후
<code>scwin.\$w.getRuntimeId</code>	<code>\$p.parent</code>

Code 5-23 SP1

```
1 var realId = scwin.$w.getRuntimeId("input2", "parent.parent");
2 WebSquare.util.getComponentById(realId).setValue("test");
```

Code 5-24 SP3 이후

```
1 $p.parent().$p.parent().input2.setValue("test");
```

5.4.3.3. TabControl

5.4.3.3.1. TabControl에서 각 탭 접근

Table 5-9 코딩 방식 비교

SP1	SP3 이후
- TabControl의 <code>getFrame()</code> - WFrame의 <code>getObj</code>	<code>getWindow()</code> <code>getWindow()</code>

Code 5-25 SP1

```
1 tabControl1.getFrame(tabControl1.getTabIndex("tab1")).getObj("wframe_header").getObj("scwin")
).test();
```

Code 5-26 SP3 이후

```
1 tabControl1.getWindow("tab1").wframe_header.getWindow().scwin.test();
```

5.4.3.4. WindowContainer

5.4.3.4.1. WindowContainer에서 각 Window 접근

Table 5-10 코딩 방식 비교

SP1	SP3 이후
- WindowContainer의 <code>getWinInfoObj()</code> - WFrame의 <code>getObj</code>	<code>\$p.main()</code> - WindowContainer 및 WFrame 의 <code>getWindow()</code>

Code 5-27 SP1

```
1 windowContainer1.getWinInfoObj("window2", "windowId").window.frame.getObj("wframe_header")
   .getObj("dataList1").getAllJSON();
```

Code 5-28 SP3

```
1 $p.main().windowContainer1.getWindow("window2")
   .wframe_header.getWindow().dataList1.getAllJSON();
```

5.4.4. 예제

그림 5-9 화면 구성 예

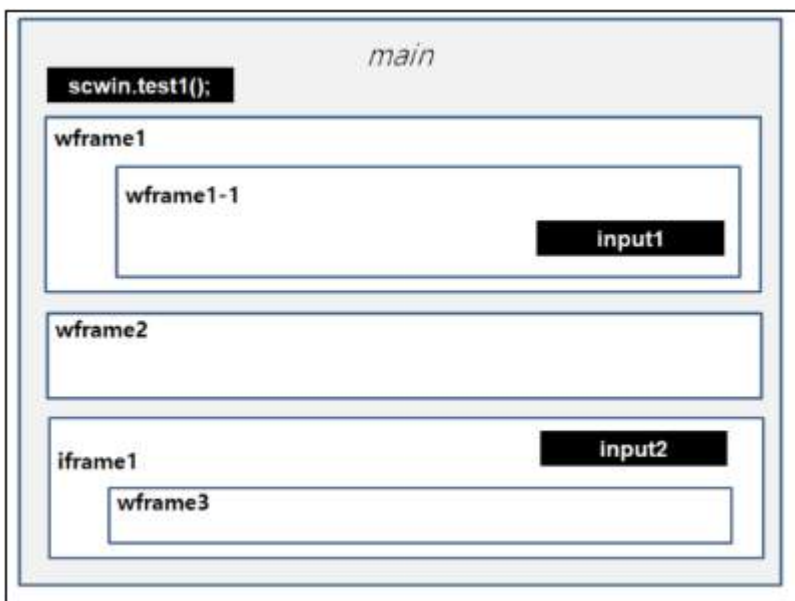


Table 5-11 wframe1-1의 input1에 접근하기

스크립트 위치	접근 방법
main	<code>wframe1.getWindow().wframe1-1.getWindow().input1.getValue();</code>
wframe1	<code>wframe1-1.getWindow().input1.getValue();</code>
wframe1-1	<code>input1.getValue();</code>
wframe2	<code>\$p.parent().wframe1.getWindow().wframe1-1.getWindow().input1.getValue();</code>
iframe1	<code>\$p.parent().wframe1.getWindow().wframe1-1.getWindow().input1.getValue();</code>

Table 5-12 main의 `scwin.test1();` 실행하기

스크립트 위치	접근 방법
main	<code>scwin.test1();</code>
wframe1	<code>\$p.parent().scwin.test1();</code> <code>\$p.top().scwin.test1();</code>
wframe1-1	<code>\$p.top().scwin.test1();</code> <code>\$p.parent().\$p.parent().scwin.test1();</code>
wframe2	<code>\$p.parent().scwin.test1();</code> <code>\$p.top().scwin.test1();</code>
iframe1	<code>\$p.parent().scwin.test1();</code>

Table 5-13 iframe1의 input2에 접근하기

스크립트 위치	접근 방법
main	<code>iframe1.getWindow().input2.getValue();</code>
wframe1	<code>\$p.top().iframe1.getWindow().input2.getValue();</code>
wframe1-1	<code>\$p.parent().\$p.parent().iframe1.getWindow().input2.getValue();</code> <code>\$p.top().iframe1.getWindow().input2.getValue();</code>
wframe2	<code>\$p.parent().iframe1.getWindow().input2.getValue();</code>
wframe3	<code>\$p.main.getWindow().input2.getValue();</code>
iframe1	<code>input2.getValue();</code>

5.5 WFrame 생성 옵션 전달

WFrame 생성 시 `dataObject` 속성을 통해 데이터를 전달할 수 있고, 호출된 페이지는 `$p.getParameter`로 해당 데이터를 참조할 수 있습니다.

아래의 예와 같이 `$p.openPopup`을 통해 팝업을 생성할 때 새로 생성할 팝업의 옵션을 전달하는 방법과 동일합니다.

Code 5-29 `$p.openPopup` 사용 예

```

1  var options = {
2      id: "popup1",
3      type: "browser",
4      width: "230px",
5      height: "250px",
6      top: "130px",
7      left: "200px",
8      popupName: "openPopup1",
9      modal: true,
10     useIFrame: useIFrame,
11     resizable: true,
12     status: true,
13     menubar: false,
14     scrollbars: false,
15     title: false,
16     dataObject: dataObj
17 };
18
19 $p.openPopup("/popup.xml", options);

```

5.5.1. 설정 방법

`dataObject`는 크게 `type`, `name`, `data` 속성으로 구성되며, 모든 속성이 존재해야 정상 동작합니다.

Table 5-14 `dataObject` 속성 설명

속성	설명
<code>type</code>	데이터 종류. "json": 권장 "xml": 비권장 "array": 비권장
<code>name</code>	<code>\$p.getParameter</code> API를 통해 데이터를 가져오기 위한 key 값.
<code>data</code>	실제 전달할 데이터. 데이터는 복제되어 전달되며 데이터를 받은 쪽에서 값을 변경해도 원래 데이터는 변경되지 않습니다.

Code 5-30 `dataObject` 예

```

1  var dataObject = {
2      "type": "json",

```

```

3     "name": "dataObject",
4     "data": {
5         "param1": "test",
6         "param2": {
7             "param22": "test2"
8         },
9         "callback": "$p.parent().scwin.windowContainer1_callback"
10    }
11 }

```

5.5.1.1. 직접 설정

`JSON.stringify`를 통해 변환된 값을 `dataObject`로 그대로 설정하면 됩니다.



단, 각 속성을 반드시 큰 따옴표 (")로 감싸야 합니다. 그렇지 않을 경우 `JSON.parse` 시에 오류가 발생합니다.

Code 5-31 직접 설정 (예)

```

1 <w2:wframe id="wframe1" src="wframe1.xml"
2     dataObject='{ "type": "json",
3         "name": "dataObject",
4         "data": { "param1": "test", "param2": { "param22": "test2" },
5         "callback": "$p.parent().scwin.wframe1_callback" } }' />

```

5.5.1.2. `setDataObject`를 통해 동적으로 설정

`wframe1.setDataObject(dataObject);` 호출하여 데이터를 설정한 후 `$p.getParameter`를 통해 해당 데이터를 가져올 수 있습니다.

Code 5-32 parameter를 받는 페이지에서 parameter를 활용하는 예제 (main.xml)

```

1 scwin.wframe1_callback = function(scope) {
2     $p.log("parent : "
3         $p.id + ", child : " + scope.id);
4 }

```

Code 5-33 parameter를 받는 페이지에서 parameter를 활용하는 예제 (wframe.xml)

```

1 scwin.trigger1_onclick = function() {
2     var obj = $p.getParameter("dataObject");

```

```

3    // $p.getParameter로 값을 가져온다.
4    $p.log(JSON.stringify(obj));
5    try {
6        eval(obj.callback)($p);
7        // parameter로 전달받은 함수를 실행할 때는 eval을 사용한다.
8        // 부모 페이지의 scwin.wframe1_callback 함수가 실행된다.
9    } catch (e) {}
10 }

```

5.5.2. 관련 컴포넌트

TabControl, WFrame, WindowContainer 컴포넌트에서 WFrame을 생성할 때 WFrame의 옵션을 `dataObject`를 통해 전달 할 수 있습니다.

각 컴포넌트 별 관련 API

TabControl: `addTab()`

WindowContainer: `createWindow()`

popup: `$p.openPopup()`

WFrame: `setSrc()`

5.5.3. 제약 사항

`dataObject`의 data 객체 전달 시 `JSON.stringify` → `JSON.parse`를 통해 값을 복사하며, 그 결과 다음 제약 사항이 존재합니다.

5.5.3.1. 객체에 함수 직접 입력 금지

객체의 값에 함수를 직접 넣을 경우, 해당 속성 값은 무시됩니다. `dataObject` 객체는 `JSON.stringify` API로 변환이 가능해야 하는데 `JSON.stringify`는 함수를 무시합니다. 함수는 string 형태로 전달한 다음, 전달 받는 페이지서 eval을 통해 실행하십시오.



함수 실행 시 **scope**는 **parameter**를 받는 **페이지**입니다. 즉, **parameter**를 전달한 페이지 내의 함수를 실행하려면 함수명 앞에 `$p.parent().`를 붙여야 합니다.

5.5.3.2. 순환 참조 금지

객체는 순환 참조를 포함할 수 없으며, 모든 속성은 반드시 유한한 depth를 가져야 합니다. 순환 참조가 있을 경우, 값을 복사할 수 없습니다. 순환 참조가 필요한 경우, 해당 객체명을 string 형태로 넘긴 후, parameter를 받는 쪽에서 별도의 처리를 해야합니다. 예를 들어 `{"test" : window}`와 같은 window 객체를 parameter로 넘겨야 하는 경우, `{"test" : "window"}`와 같이 string으로 넘긴 후, 추후 eval 등으로 별도로 처리해야 합니다.



비권장

parameter로 window 객체를 넘기는 것은 권장하지 않습니다.

5.6 WFrame 사용 시 유의 사항

5.6.1. 외부 라이브러리 수동 릴리즈

WFrame이 언로딩될 때 엔진은 아래의 자원을 자동으로 제거합니다.

- 웹스퀘어 객체
- DOM
- CSS
- SPA Variable에 등록한 변수
- 외부 스크립트 로딩 시 `scopeVariable`로 등록한 변수
- `$p` API로 생성된 동적 컴포넌트(`$p.data.create`, `$p.dynamicCreate` 등)

그러나 이외의 자원은 그대로 제거되지 않고 남아 있습니다. 예를 들어, WFrame 화면 내에서 전역 변수를 선언한 경우, 해당 변수는 페이지가 제거되어도 그대로 남아있습니다. 마찬가지로 특정 화면이 외부 라이브러리를 사용할 경우 해당 화면이 제거되어도 외부 라이브러리는 그대로 남아있습니다. 따라서 페이지의 `onpageunload` 이벤트에서 외부 라이브러리를 반드시 수동으로 릴리즈해야 합니다.

5.6.2. `onpageunload`에서 `setSrc` 호출 시 `src` 속성 비우기

`onpageunload` 이벤트가 자기 화면 WFrame에 대해 `setSrc`를 호출할 경우, WFrame의 `src` 속성을 비우십시오. `setSrc`를 호출할 경우, 기존 페이지를 먼저 언로딩해야 합니다. 즉, `src` 속성이 존재할 경우, 화면 렌더링 직후 해당 화면을 폐기하고 `onpageunload` 이벤트로 새 화면을 호출하게 되어 자원 낭비를 초래합니다.

5.6.3. Scope 권장

SP3 이상 버전은 Scope 기능 적용을 권장합니다. `scope="true"` WFrame을 사용하십시오. `scope="true"` WFrame 과 `scope="false"` WFrame의 혼용은 코딩의 복잡성을 증가시킵니다.

5.6.4. IFrame 제한

외부 솔루션과의 연계 등 브라우저를 다시 로딩하여 메모리를 비워야 하는 것과 같이 특수한 경우에만 IFrame을 사용 하십시오. 대부분의 경우, WFrame 사용을 권장합니다.

5.7 공통 함수

각 공통 함수는 자신이 실행된 Scope 내에서만 유효해야 합니다. 즉, WFrame에서 호출된 공통 함수는 해당 WFrame 내에서만 실행되어야 합니다. 그 결과, 공통 함수 개발자는 호출된 공통 함수가 어느 Scope에서 호출되었는지 파악 할 수 있는 방법을 제공해야 합니다. 아래 두 가지 방법으로 가능합니다.

1. Scope 내에 공통 함수 객체를 생성 (권장)
2. 공통 함수 호출시 scope를 변수로 전달

5.7.1. 객체 생성 방식 (권장)

*client.config.xml*의 설정 중 `<engine>`/`<module>`로 로딩되는 JS에 포함된 공통 함수는 Main에 생성됩니다. 그러나 각 Scope에서 공통 함수를 호출할 경우, 호출된 공통 함수는 해당 Scope 내에서만 유효해야 합니다.

이를 위해 SP3 이상 버전은 *client.config.xml* 파일에 각 scope 마다 서로 다른 공통 함수 객체를 자동으로 생성할 수 있는 설정 항목을 제공합니다.

아래의 예제의 경우, 각 Scope마다 각기 다른 `com` 객체가 자동 생성됩니다.

Code 5-34 client.config.xml 설정 예

```
1 <engine>
2   <module src="/common/js/common.js" />
3 </engine>
4 <spa>
5   <variable value="scwin" clone="com"/>
```

```
6 </spa>
```

객체 생성 방식은 가장 간단히 scope를 참조할 수 있는 방법으로 아래 설명할 5.7.2. 변수 전달 방식에서 발생할 수 있는 변수 전달 오류를 방지할 수 있어 권장합니다.

5.7.1.1. 예제

1. 공통 함수를 작성한 파일을 *common.js* 파일에 작성합니다.
2. 공통 함수 객체를 *com*이라고 가정합니다.
3. *client.config.xml* 파일에 아래 설정을 추가하십시오. (Scope에 생성할 공통 함수 객체를 *clone* 속성에 지정.)

Code 5-35 client.config.xml 설정 예

```
1 <engine>
2   <module src="/common/js/common.js" />
3 </engine>
4 <spa>
5   <variable value="scwin" clone="com"/>
6 </spa>
```

Code 5-36 common.js 예

```
1 com = {
2   // 통신 속성 정보
3   CONTEXT_PATH: "", // Context Path 경로
4   SERVICE_URL: "", // Service Url
5   DEFAULT_OPTIONS_MODE: "asynchronous", // 기본 통신 모드 ( "asynchronous" /
"synchronous")
6   DEFAULT_OPTIONS_MEDIATYPE: "application/json", // 기본 미디어 타입
7
8   // 통신 상태 코드
9   MESSAGE_CODE: {
10    STATUS_ERROR: "E",
11    STATUS_SUCCESS: "S",
12    STATUS_WARNING: "W"
13  }
14 }
15 com.setCommonCode = function(codeOption, callbackFunc) {
16   .....
```

5.7.2. 변수 전달 방식

공통 함수 호출시 scope를 알 수 있는 객체(예: `scwin`, `$p`, 웹스퀘어 컴포넌트)를 함수의 변수로 받을 수 있습니다. 객체 정보에서 Scope 정보를 참조하여 해당 Scope에 생성된 `$p`를 사용하십시오.

Code 5-37 사용 예

```
1 com.openPopup = function($p, url, opt, dataJson) {
2     .....
```

예를 들어, 컴포넌트 ID를 변수로 받아 해당 컴포넌트에 대해 `$w.XXX` 류의 유틸리티성 API를 실행하는 공통 함수가 존재할 수 있습니다. 이 경우, 공통 함수는 Scope 내에서만 실행되어야 하기 때문에 변수로 받은 컴포넌트에 대해 `$p.XXX` (혹은 `scwin.$w.XXX`) 형태의 API를 실행해야 합니다.

5.7.2.1. 예제

다음은 컴포넌트의 title 값을 반환하는 공통 함수 예제입니다. 공통 함수는 컴포넌트 ID를 변수로 받아 해당 컴포넌트에 대해 정의된 API를 실행합니다. 본 함수는 해당 Scope 내에서만 실행되어야 하기 때문에, 함수 앞에 `scwin.`을 추가하거나 유틸성 API 경우 `$w` 대신에 `$p`(혹은 `scwin.$w`)를 사용해야 합니다.

Code 5-38 공통 함수 정의 (예)

```
1 titleFinder = function($p, compId) {
2     return $p.getComponentById(compId).getTitle();
3 }
4
5 pathFinder = function($p, compId) {
6     var rawpath = $p.getComponentById(compId).getSrc();
7     return rawpath.substring(rawpath.lastIndexOf('/') + 1);
8 }
```

위와 같이 정의한 공통 함수를 화면 파일에서 사용할 경우 `$p`를 변수로 입력해야 합니다.

Code 5-39 공통 함수 사용 (예)

```
1 scwin.trigger1_onclick = function(e) {
2     var title = titleFinder($p, "image1");
3     input1.setValue(title);
4 };
5
6 scwin.trigger2_onclick = function(e) {
7     var imagepath = pathFinder($p, "image1");
```

```
8   input2.setValue(imagepath);
9 };
```



프로젝트의 Scope 기능 사용 여부 및 함수의 Scope 참조 여부에 따라 공통 함수의 변수가 변경될 수 있습니다. 따라서 변수 전달 방식은 공통 함수의 표준화와 개발의 편의를 위해서 권장하지 않습니다.

5.8 외부 라이브러리 연동

SP3 이상 버전은 외부 JS를 내부 화면 내에 정의된 script와 동일하게 실행할 수 있는 기능을 제공합니다. 즉, 화면이 외부 JS를 포함할 경우, 해당 외부 JS는 화면의 컴포넌트 및 scwin 객체를 직접 참조할 수 있습니다.

WebSquare 페이지 내에서 외부 라이브러리를 로딩할 경우 아래 두 가지 속성을 추가하십시오.

5.8.1. `scopeExternal`

`scopeExternal="true"`인 경우, 연동된 외부 JS는 Scope 내에 접근할 수 있습니다.

Code 5-40 설정 예

```
1 <script src="external_wframe1.js" scopeExternal="true"/>
```



객체 참조 가능 시점

스크립트 로딩 시 즉시 함수가 선언/실행 되기 때문에 로딩 직후에는 렌더링된 객체를 참조하지 못할 수 있습니다. `onpageLoad` 이벤트 이후부터 객체를 참조하십시오.



외부 라이브러리 객체 삭제

외부 라이브러리에서 생성한 객체는 Page 언로드시에 자동 삭제되지 않습니다. 메모리 문제가 우려되는 경우, `IFrame`으로 분리하여 사용할 것을 권장합니다.

5.8.2. `scopeVariable`

엔진이 외부 JS 객체를 해당 Scope 내에 생성하고 삭제합니다. 이 때 사용할 객체명을 지정하십시오. 로딩된 객체는 해당 Scope 내에서만 사용 가능하고 Page 언로딩 시에 삭제됩니다.

Code 5-41 설정 예 (*wframe1.xml*)

```
1 <script src="/common/biz/ctiCommon.js" scopeExternal="true" scopeVariable="cti"/>
```

wframe1.xml 파일 내부에 아래 예제와 같이 외부 JS를 로딩할 경우,

- (1) `cti` 변수가 자동으로 생성되고,
- (2) *ctiCommon.js* 및 *wframe1.xml* 내부 스크립트에서만 해당 변수에 접근할 수 있습니다.



변수 재정의 금지

`scopeVariable`로 등록한 변수를 외부 JS 파일 내에 재정의하지 마십시오. 이 변수를 재정의할 경우 해당 변수는 전역에 생성됩니다.

Code 5-42 *ctiCommon.js* 내 객체 선언 금지

```
1 /* 객체 선언 금지!!! */
2 /* 선언시 scope가 정상 동작하지 않습니다. */
3
4 //var cti = {}; // 금지
5
6 cti.pushCall = function() {
7     ....
```

5.9 CSS

5.9.1. `class` selector 사용

WFrame을 사용하여 화면을 구성할 경우, 각 화면 내 컴포넌트의 ID는 항상 변합니다. 즉, `input1` 객체는 연결되는 WFrame에 따라 `wframe1_input1`이 될 수도 있고 `wframe2_wframe22_input1`이 될 수도 있습니다. 따라서 id select or 대신 class selector 사용을 권장합니다.

Code 5-43 사용 예

```

1 // class 사용 (권장)
2 <xf:group id="group1" class="contentZ1"/> .contentZ1 { }
3
4 // id 사용 (비권장)
5 <xf:group id="group1"/> #group1 { }

```



class명 중복 주의

class selector를 사용할 경우 부모 화면에 정의된 스타일 이름과 겹치지 않도록 주의하십시오. 이름이 겹칠 경우 부모 화면의 class를 덮어 쓰게 되어 부모 화면의 스타일이 변경됩니다.
class명이 겹치지 않도록 화면 ID를 class명 뒤에 붙이는 방법을 권장합니다.

5.9.2. jQuery의 id selector 사용

Scope 기능을 사용할 경우, 각 화면에 존재하는 객체의 ID는 변합니다. 그러나 jQuery와 같은 외부 모듈을 사용할 경우, ID를 통해 WebSquare 객체에 접근해야 할 경우가 있습니다. 이를 위해 SP3 이상 버전은 외부 모듈이 ID를 통해서 WebSquare 객체에 접근할 수 있는 방법을 제공합니다.

아래와 같이 `$p.$("#" + componentID)` 형식으로 ID를 통해 객체에 접근하십시오.

Code 5-44 ID를 통한 접근 방법

```

1 // Scope 적용 전 접근 방법
2 // $("#input1")
3
4 // Scope 적용 후 접근 방식
5 $p.$("#input1")

```

`$` 대신 `$p.$`를 사용하여 객체를 검색할 수 있습니다. 반환된 결과는 jQuery의 검색 결과와 동일하며, jQuery와 동일하게 API를 사용할 수 있습니다.

Code 5-45 사용 예

```

1 $p.$("#link_area .link_text").css("color", "red")

```

5.10 SPA

Scope은 웹페이지 전체를 하나의 Single Page로 구현하기 위한 기능입니다. SP3 이전 엔진의 경우, IFrame을 통해 SPA를 구현했습니다. TabControl 및 WindowContainer의 경우 `spaInitCount`를 통해 엔진 재활용 가능한 IFrame 개수를 지정하고 `spaAuto` 속성을 통해 빈 IFrame을 1개씩만 미리 준비하도록 함으로써 초기 로딩 속도 저하를 방지했습니다. 그러나 SP3 이후 버전에서는 WFrame으로 구성된 화면에서 `setSrc()` 함수만 실행하여 SPA를 쉽게 구현할 수 있습니다. 즉, Scope 기능 및 WFrame을 사용하여 별도의 추가 코딩 없이 SPA를 손쉽게 구현할 수 있습니다.

Table 5-15 SP1 vs. SP3 이상

	SP1	SP3 이상
구현 수단	IFrame 사용	WFrame 사용
구현 방식	- 소스 코드를 통한 구현. - IFrame의 효율적 사용이 핵심. - TabControl 및 WindowContainer의 경우, <code>spaInitCount</code>를 통해 엔진 재활용 가능한 IFrame 개수를 지정 <code>spaAuto</code> 속성을 통해 빈 IFrame을 1개씩만 준비하여 초기 로딩 속도 저하를 방지 - 구현 방식 복잡 - SPA 적용이 제한적	<code>setSrc()</code> 함수 사용 - 구현 방식 간단. - 적용에 제한 없음

5.11 디버깅

객체에 접근하기 위해서는, 해당 객체가 존재하는 Scope을 먼저 찾아야 합니다. 이후 해당 Scope 내에 있는 객체에 접근하십시오. Scope은 아래 방법으로 찾을 수 있습니다.

1. `$p.main()` 혹은 `$p.top()`으로 상위 scope에 접근.
2. `$p.getWindow`를 이용하여 원하는 객체가 포함된 scope에 접근.

아래는 디버깅용으로 Scope을 쉽게 찾기 위해 제공하는 API입니다.

Table 5-16 디버깅용 API

API	설명
<code>\$p.debug.getScope(domElement);</code>	렌더링된 domElement가 속한 화면의 Scope 객체를 반환.
<code>\$p.debug.getFrame(domElement);</code>	렌더링된 domElement가 속한 화면의 WFrame ID를 반환.

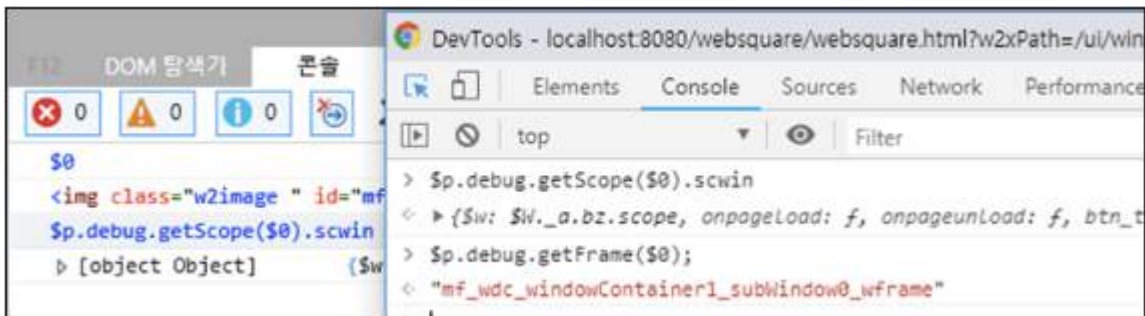
5.11.1. 예제

1. 브라우저에서 개발자 도구를 실행합니다.
2. DOM 탐색기에서 원하는 element를 클릭한다.
3. 콘솔 창에서 `$p.debug.getScope($0);` API를 실행합니다.
4. 반환되는 Scope 객체를 확인합니다.



브라우저의 디버깅 도구가 제공하는 특수 전역 객체 `$0`을 사용하면 DOM Element를 확인할 수 있습니다. `$0`은 현재 선택된 DOM Element를 반환하여 이는 디버깅용 API의 변수로 사용됩니다.

그림 5-10 DOM Element 확인



6. 이전 버전과의 비교

6.1 페이지 컴포넌트 기능 확대

종류	설명
Common Component (프로젝트 공통 컴포넌트)	내부에서 공통으로 사용하는 컴포넌트를 생성하기 위해 사용됩니다.
Extended Component (EXC)	기존의 웹스퀘어 컴포넌트를 EXC로 생성할 경우, 해당 컴포넌트의 기능은 그대로 상속받는 동시에 추가로 필요한 속성, 이벤트, 함수를 생성하여 사용할 수 있습니다.
Trusted Third-party Component (TTC)	기존의 TTC (Trusted Third-party Component) 기능을 개선하여 앱스토어 처럼 TTC를 등록하고 사용할 수 있는 기능을 제공합니다.
User-defined Component (UDC)	개발자가 웹스퀘어가 제공하는 컴포넌트를 조합하여 Group으로 묶은 뒤 직접 속성, 이벤트, 함수를 구현하는 방식으로 새로운 컴포넌트를 생성할 수 있습니다.

그림 6-1 프로젝트 공통 컴포넌트 (WebSquare Common Component)

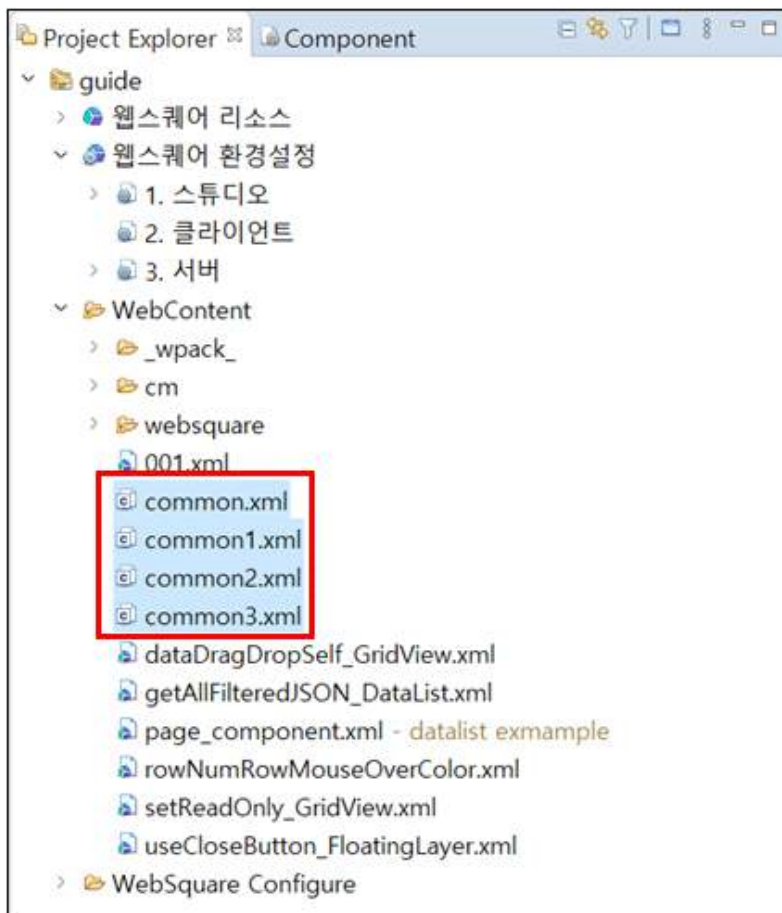
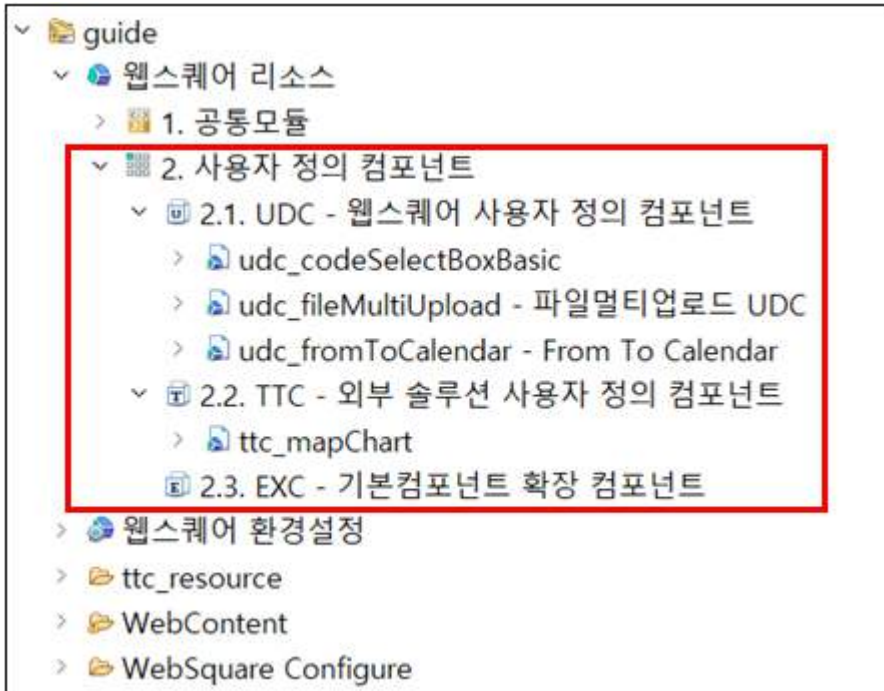


그림 6-2 사용자 정의 컴포넌트 (UDC, TTC, EXC)



6.2 다양한 리소스 및 예제 제공

6.2.1. Design Kit

Design Kit 화면의 상단에는 컴포넌트, 스니펫, 템플릿, 디자인 가이드 메뉴가 표시되고 각 메뉴를 클릭하여 관련 디자인 리소스를 썸네일 이미지 형태로 확인할 수 있습니다.

그림 6-3 Design Kit

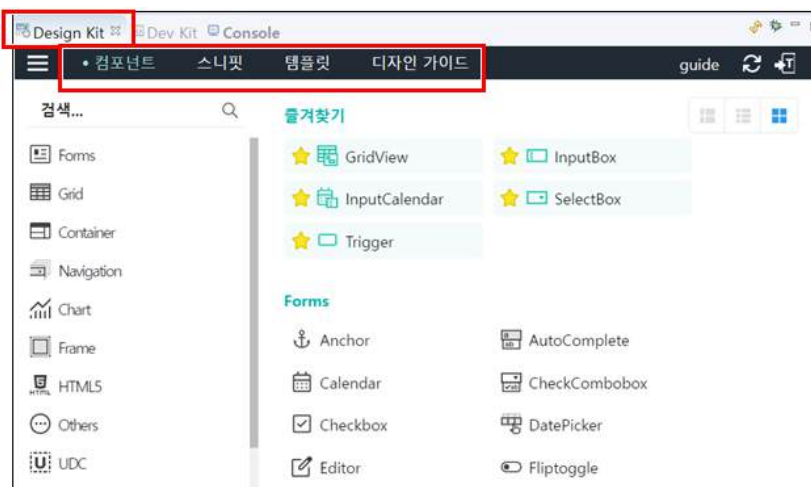


Table 6-1 Design Kit

컴포넌트	화면 개발에 사용할 수 있는 웹스퀘어 컴포넌트를 카테고리 별로 구분하여 표시.
스니펫	공동 개발에 필요한 디자인 리소스를 썸네일 이미지 형태로 제공.
템플릿	화면 파일 생성 시 선택 가능한 템플릿을 썸네일 이미지로 제공.
디자인 가이드	프로젝트에 적용할수 있는 디자인 가이드 화면을 표시

6.2.2. Dev Kit

동영상, 기술 문서, 검색 등을 통해 웹스퀘어 AI의 전반적인 개발 가이드를 제공하고, 웹스퀘어가 제공하는 다양한 기술 자료를 검색할 수 있는 기능을 제공합니다.

그림 6-4 Dev Kit

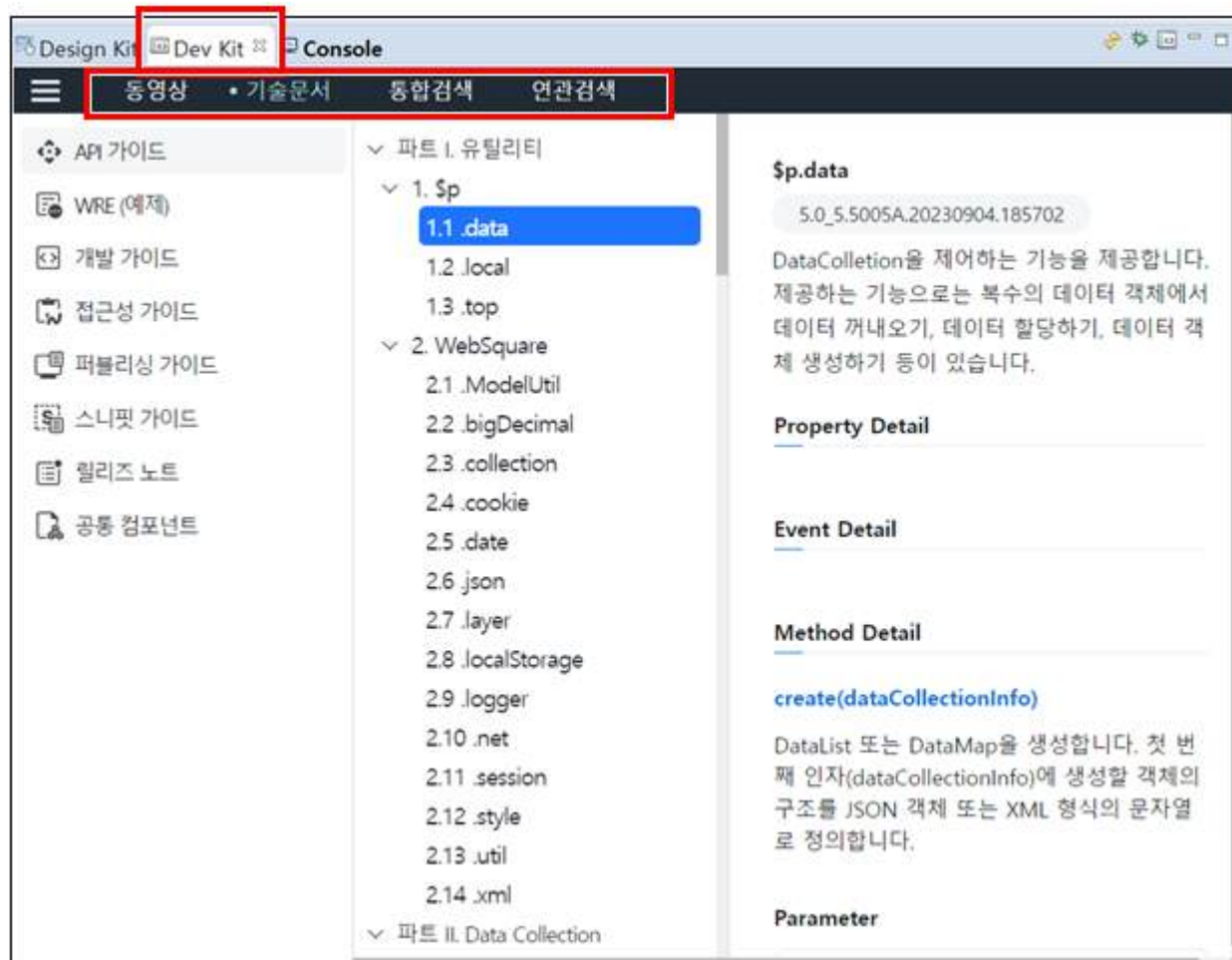


Table 6-2 Dev Kit

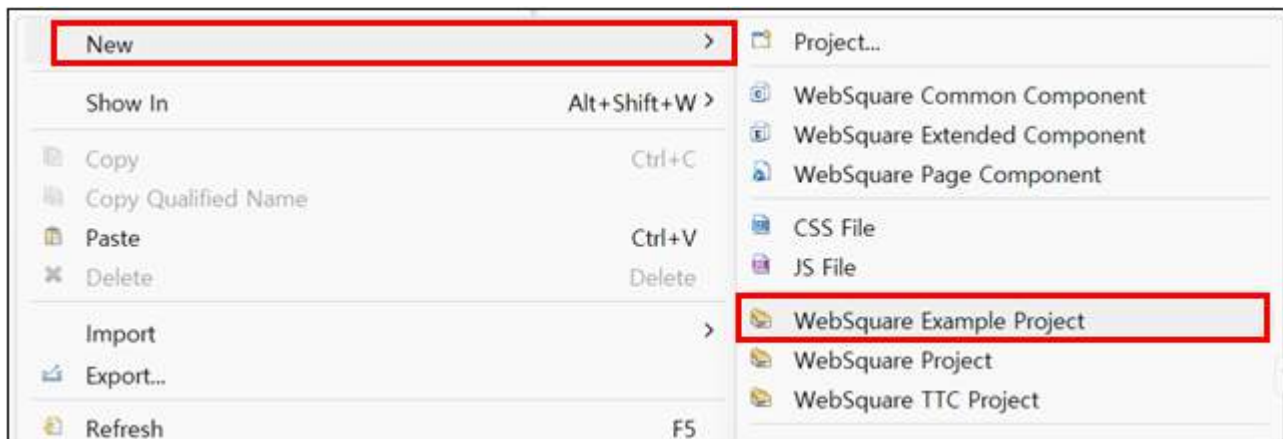
동영상	웹스퀘어 컴포넌트 사용법을 설명하는 동영상 및 쇼츠 목록을 표시합니다.
기술문서	웹스퀘어 다양한 각종 기술 문서를 제공합니다.

통합검색	키워드를 입력하여 해당 키워드와 관련된 웹스퀘어 기술 문서 및 동영상을 검색하여 표시합니다.
연관 검색	웹스퀘어 스튜디오에서 사용할 수 있는 컴포넌트의 입력 필드, 속성, 메소드등에 관해 검색 결과를 표시합니다.

6.2.3. WRE (WebSquare Reference Example)

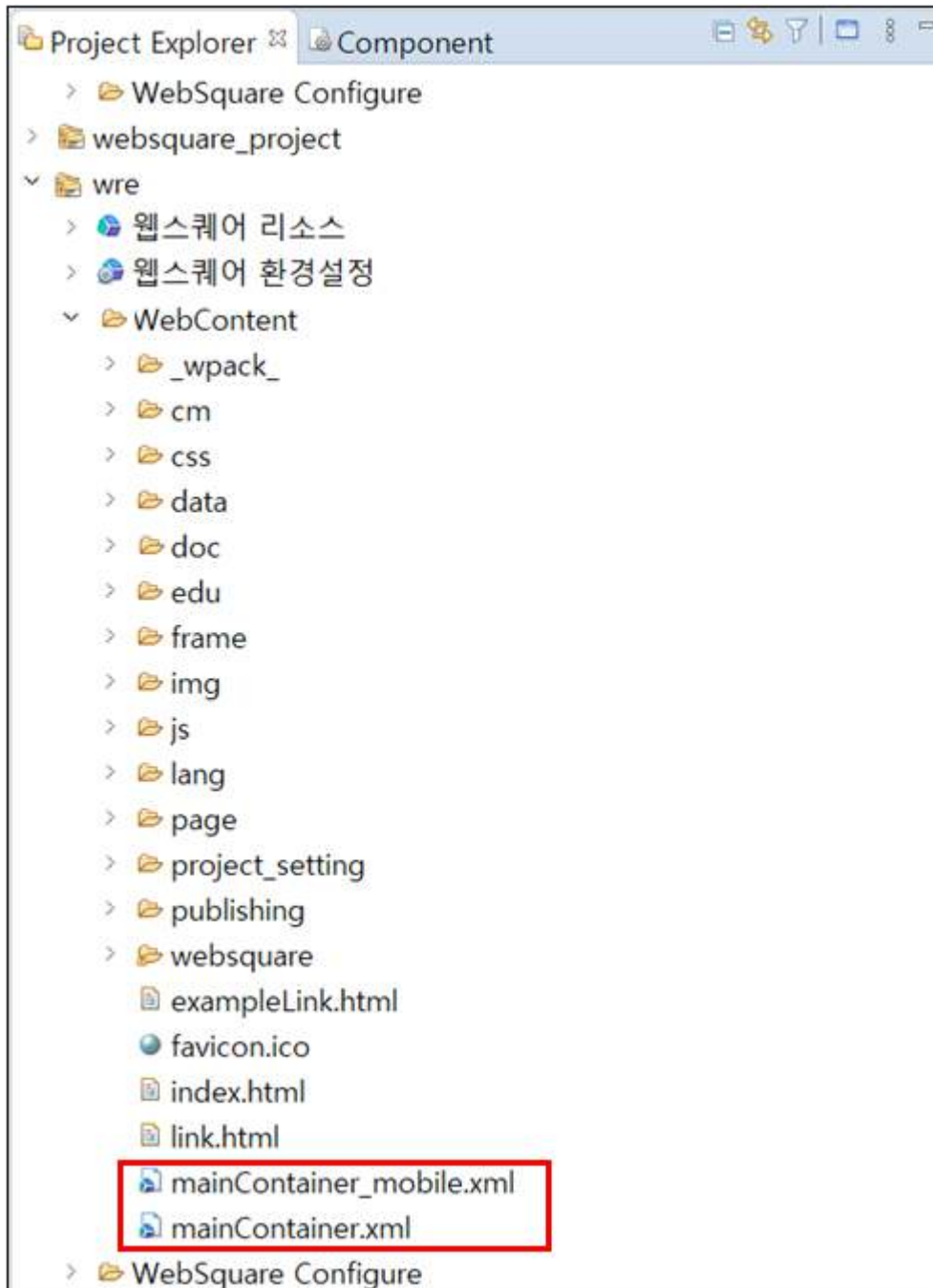
WebSquare Reference Example 프로젝트는 외부 네트워크의 사용이 불가능한 환경에서도 예제 파일을 쉽게 확인할 수 있는 환경을 제공합니다. 웹스퀘어 스튜디오에서 서버 구성 없이 쉽게 생성하고 실행할 수 있습니다.

그림 6-5 New - WebSquare Example Project



"/wre/WebContent/" 폴더 하위의 mainContainer.xml 및 mainContainer_mobile.xml 파일을 선택하여 PC 및 모바일 기기용 WebSquare Reference Example을 확인할 수 있습니다.

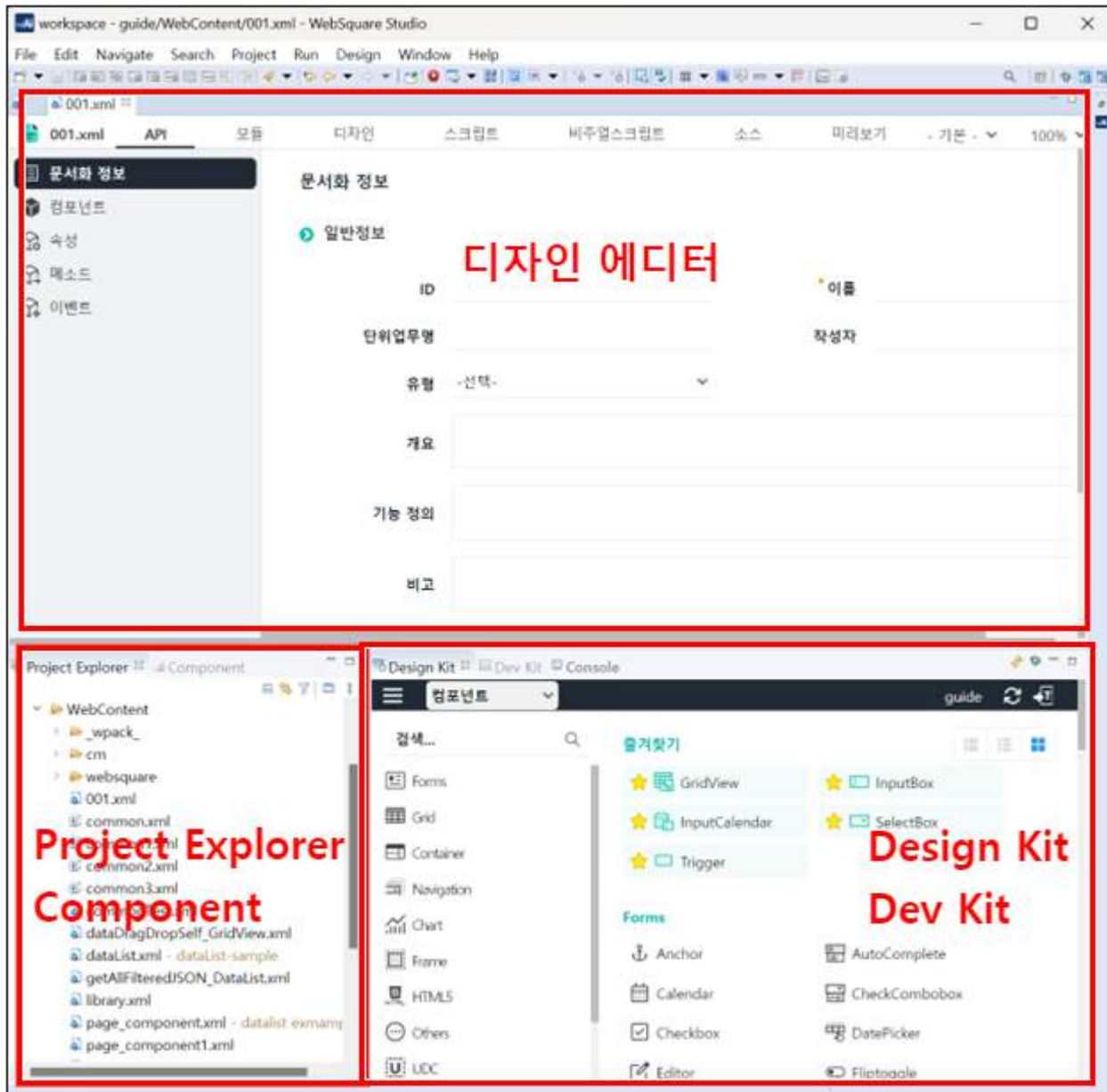
그림 6-6 mainContainer.xml 및 mainContainer_mobile.xml 파일



6.3 Perspective 개선

개발자가 화면 디자인과 스크립트를 빠르고 직관적으로 확인할 수 있도록 웹스퀘어 AI의 스튜디오 화면 구성을 아래와 같이 변경했습니다.

그림 6-7 웹스퀘어 AI 뷰 구성



- 디자인 에디터 확대 및 기능 추가.
 - 문서 정보 등록 기능 추가.
 - 공통 컴포넌트 등록 기능 추가.
 - 데이터컬렉션 등록 기능 추가.
 - 비주얼스크립 탭을 추가.
- Component 뷰 추가.
 - 각 컴포넌트의 속성, 스타일, 이벤트 확인 가능.
- Design Kit 추가.
- 기존 Palette를 Design Kit의 컴포넌트 탭으로 이동.
- Dev Kit 추가.
- 기존의 Source 뷰는 숨김 처리. (숨김 취소 옵션 제공.)

6.4 비주얼스크립트 탭 분할 기능

Design 탭의 컴포넌트를 확인하면서 해당 컴포넌트 관련 스크립트를 작성할 수 있도록, 웹스퀘어 AI 스튜디오는 비주얼스크립트 탭을 클릭할 경우, Design과 Script를 동시에 표시합니다.

그림 6-8 비주얼스크립트 탭

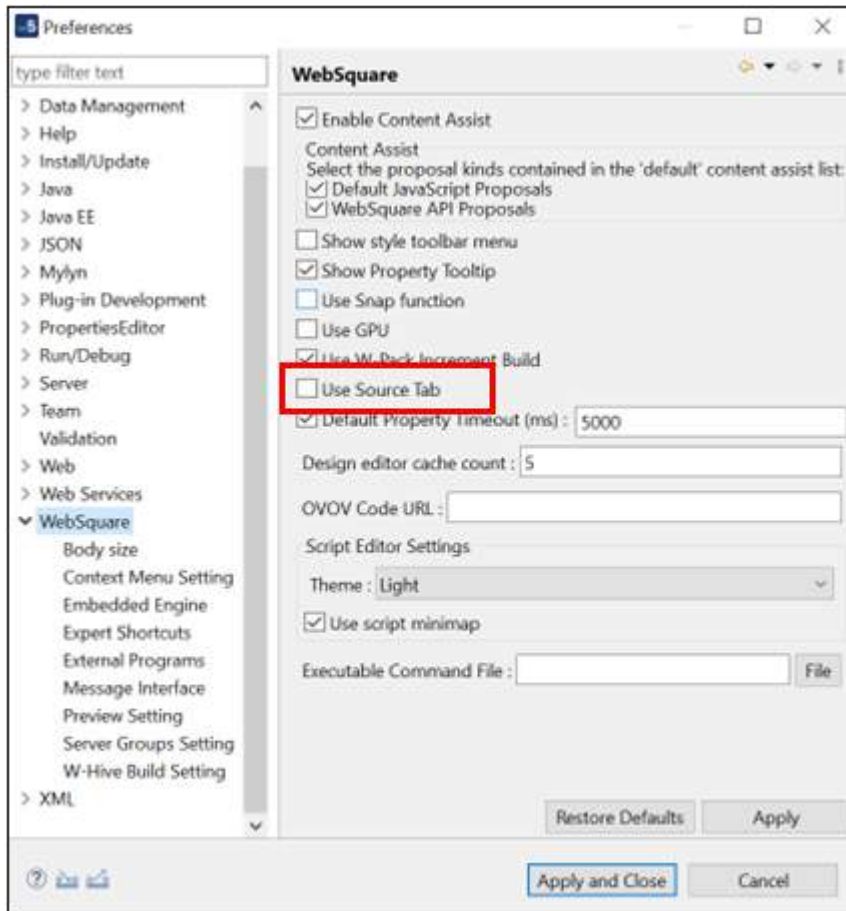


6.5 Source 탭 숨김/표시 옵션 제공

웹스퀘어 페이지의 소스 코드 보안을 높이고 및 소스 코드 수정을 통한 동작 오류를 방지하기 위해, 웹스퀘어 AI 스튜디오는 웹스퀘어 페이지를 열 경우 Source 탭을 표시하지 않는 옵션을 제공합니다.

Source 탭을 표시하려면, **Window - Preferences - WebSquare**를 선택하고 **Use Source Tab** 항목을 체크하십시오.

그림 6-9 Source 탭 표시



6.6 MAC 및 Linux 지원

MAC 및 Linux OS에서 사용 가능한 Studio를 Eclipse 플러그인 형태로 제공합니다.

6.7 다양한 검색 지원

9.7 파일 검색

특정 키워드를 포함한 웹스퀘어 화면 파일을 검색할 수 있습니다.

7.8.3. 검색 (키워드 검색)

키워드를 입력하여 해당 키워드와 관련된 기술 문서 및 동영상を検索할 수 있습니다.

7.8.4. 연관 검색 (즉시 검색)

스튜디오에서 입력 필드, 속성, 메소드를 클릭하여 관련된 내용을 즉시 검색할 수 있습니다.

6.8 웹스퀘어5 Vs. 웹스퀘어 AI

Table 6-3 웹스퀘어5 Vs. 웹스퀘어 AI

항목	웹스퀘어5	웹스퀘어 AI
Design Kit 지원	X	O
Dev Kit 지원	X	O
EXC (컴포넌트 확장)	X	O
WRE 프로젝트 (샘플 프로젝트)	X	O
Source 탭 숨김/표시 옵션	X	O
Design과 Script 동시 표시	X	O

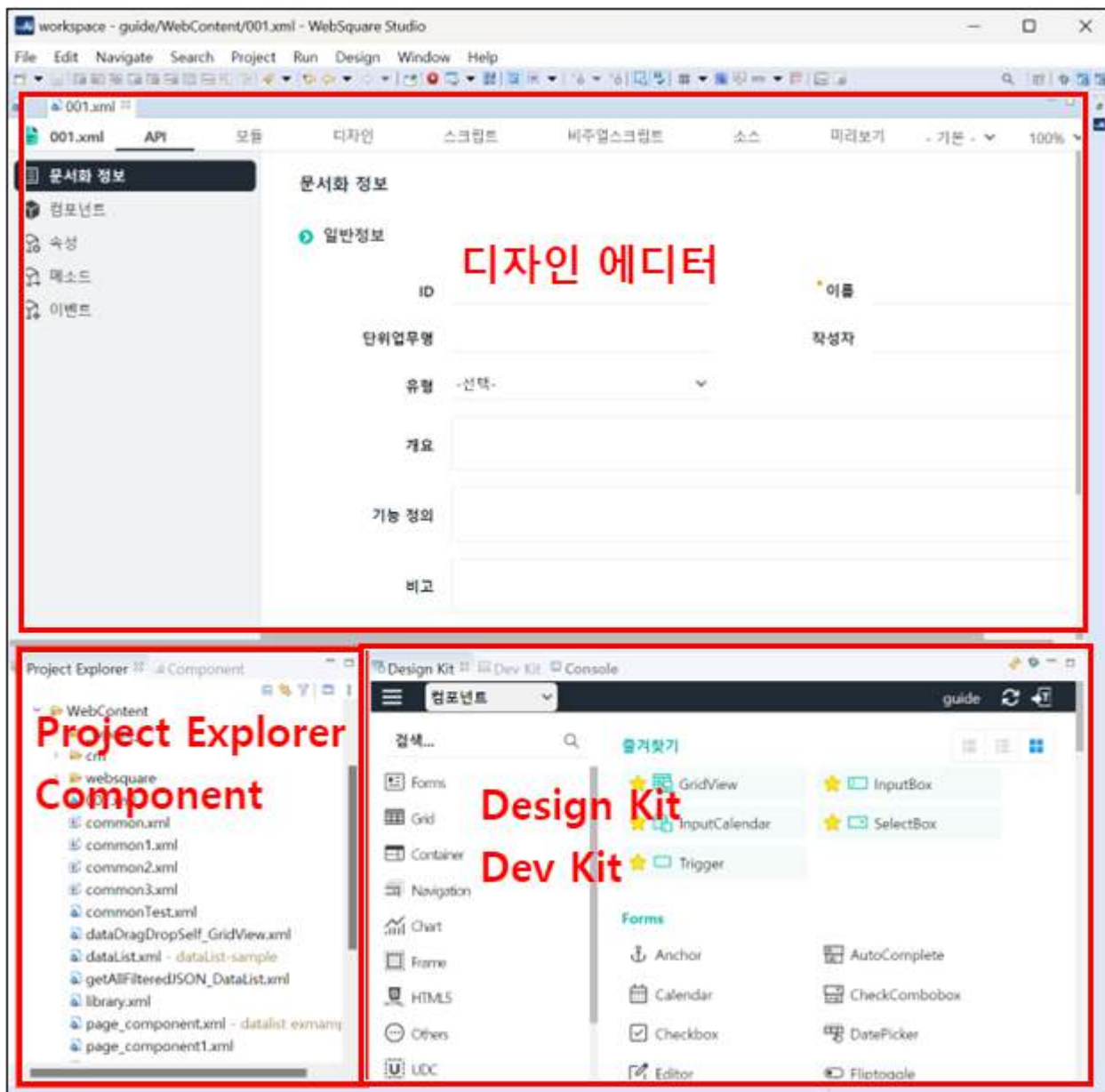
Part II. 스튜디오

7. 뷰 구성

7.1 초기 화면

웹스퀘어 AI 스튜디오의 초기 화면은 다음과 같습니다.

그림 7-1 웹스퀘어 AI 뷰 구성



7.2 뷰 설정

우측 상단의 **Open Perspective** 버튼을 클릭하면 화면 Perspective를 설정할 수 있습니다.

그림 7-2 Open Perspective

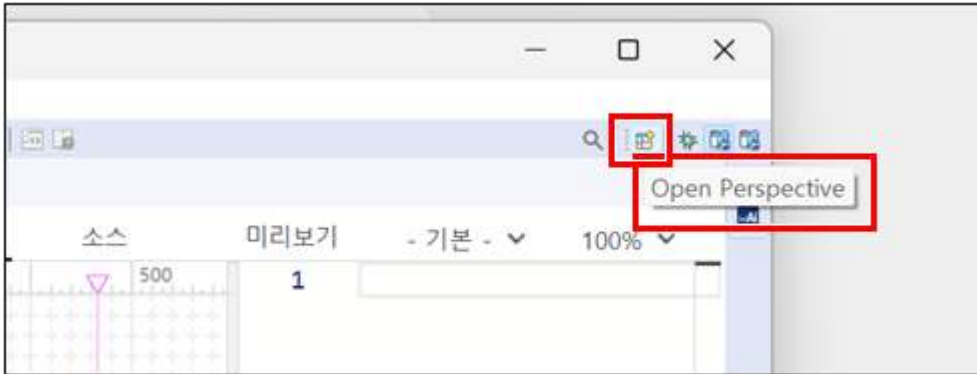
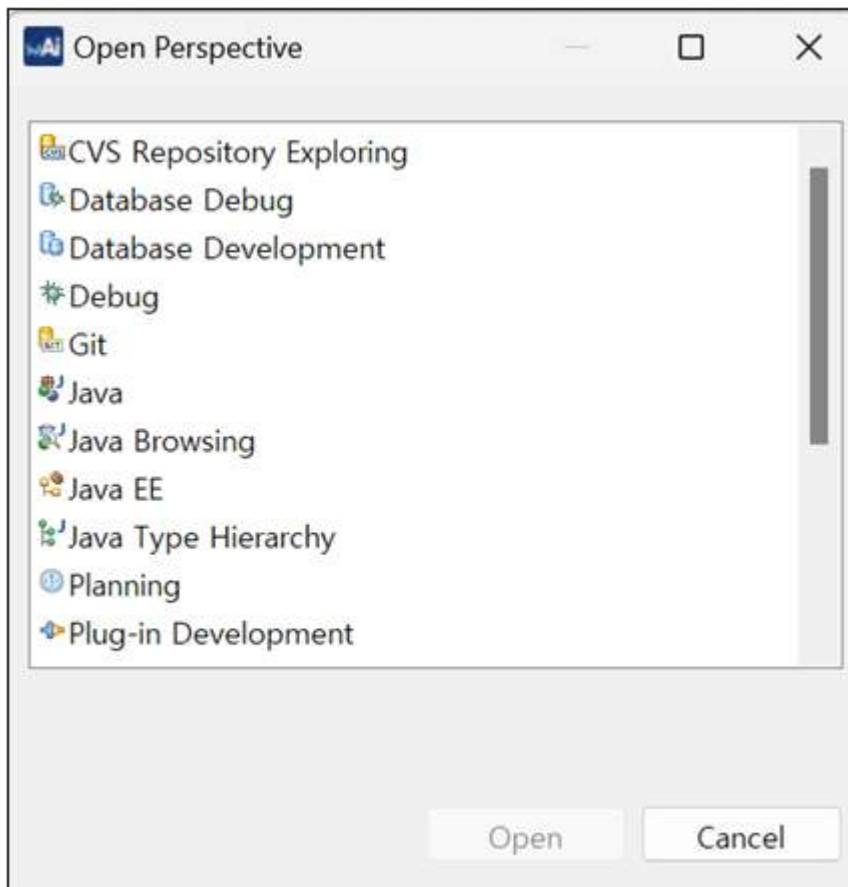


그림 7-3 Perspectives



7.3 뷰 초기화

스튜디오 우측 상단의 WebSquare 버튼을 우클릭한 후 **Reset** 메뉴를 선택하면 뷰 구성을 초기화할 수 있습니다.

그림 7-4 뷰 초기화

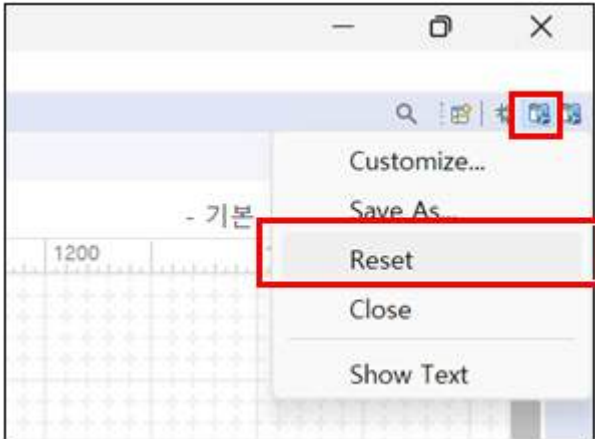
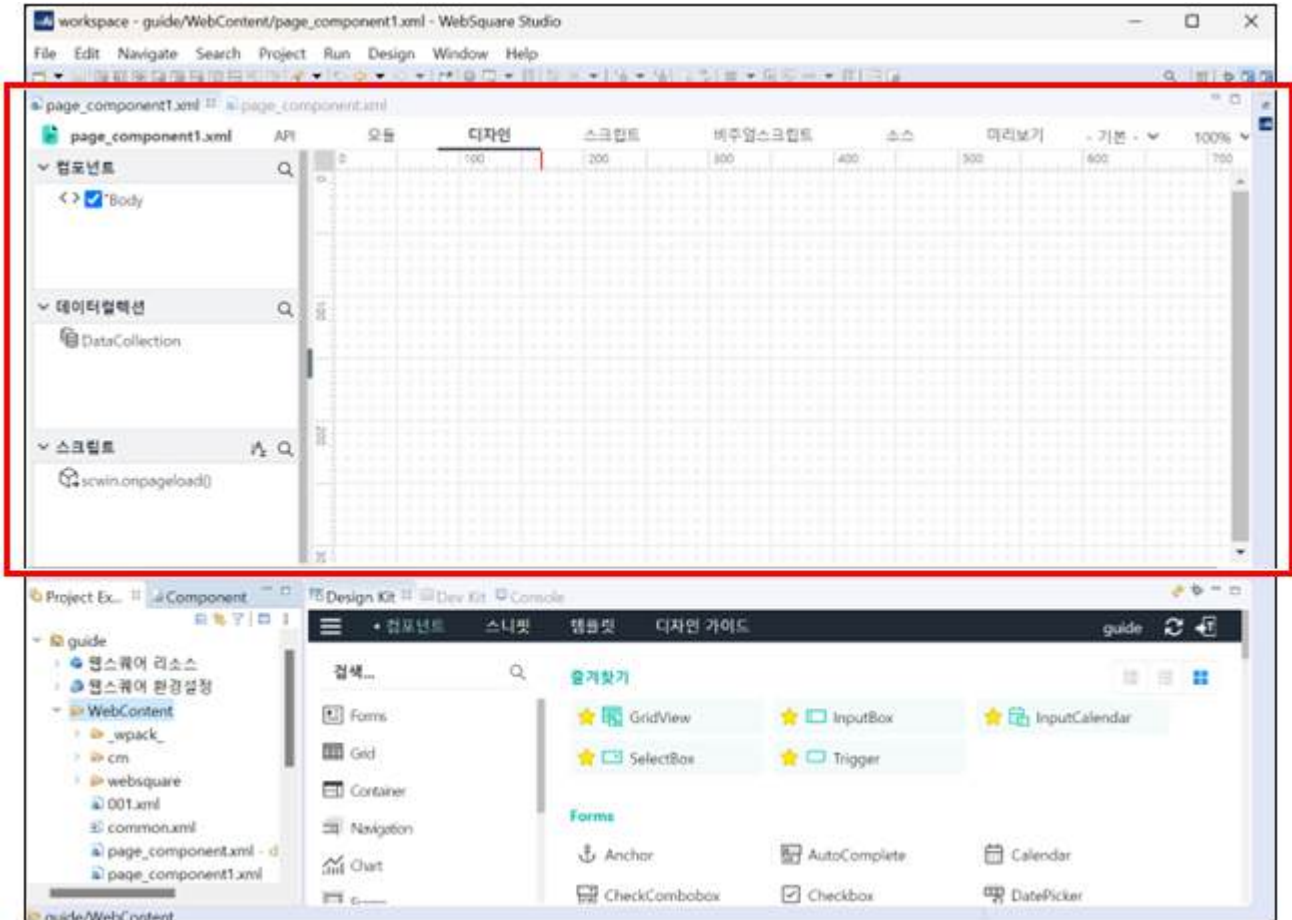


그림 7-5 뷰 초기화 확인



7.4 디자인 에디터

그림 7-6 디자인 에디터



7.4.1. API

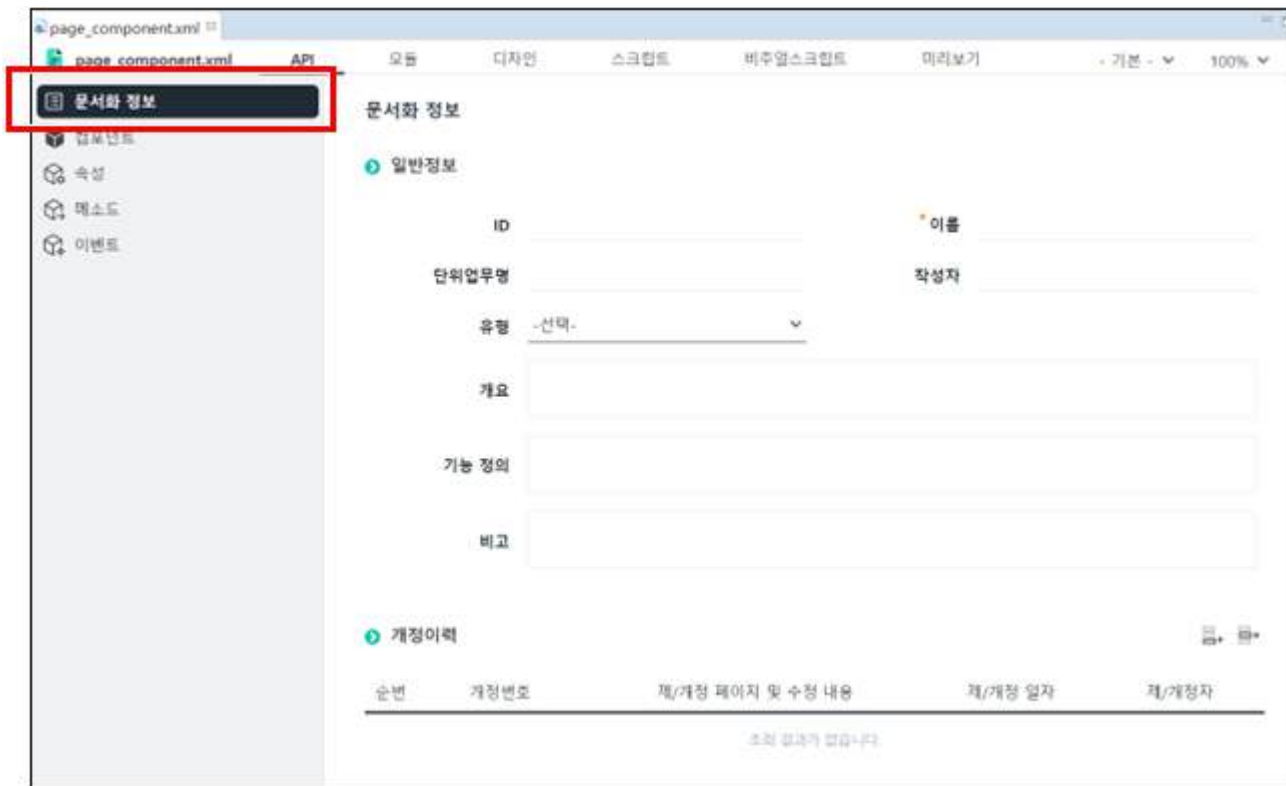
개발한 화면을 페이지 컴포넌트로 등록하여 사용할 경우, API 탭에서 해당 화면의 정보를 등록해야 합니다. 우측의 문서화 정보, 컴포넌트, 속성, 메소드, 이벤트 탭을 활용하여 해당 화면 및 페이지 컴포넌트의 일반적인 정보를 등록하고, 페이지 컴포넌트의 속성, 함수, 이벤트를 추가할 수 있습니다.

현재 활성화 되어있는 웹스퀘어 페이지의 일반적인 정보 및 페이지가 가지고 있는 속성과 메소드, 이벤트 정보를 입력합니다.

7.4.1.1. 문서화 정보

W-Doc를 사용하여 문서 정보를 생성할 때 필요한 정보를 입력합니다.

그림 7-7 문서화 정보



7.4.1.1.1. 일반 정보

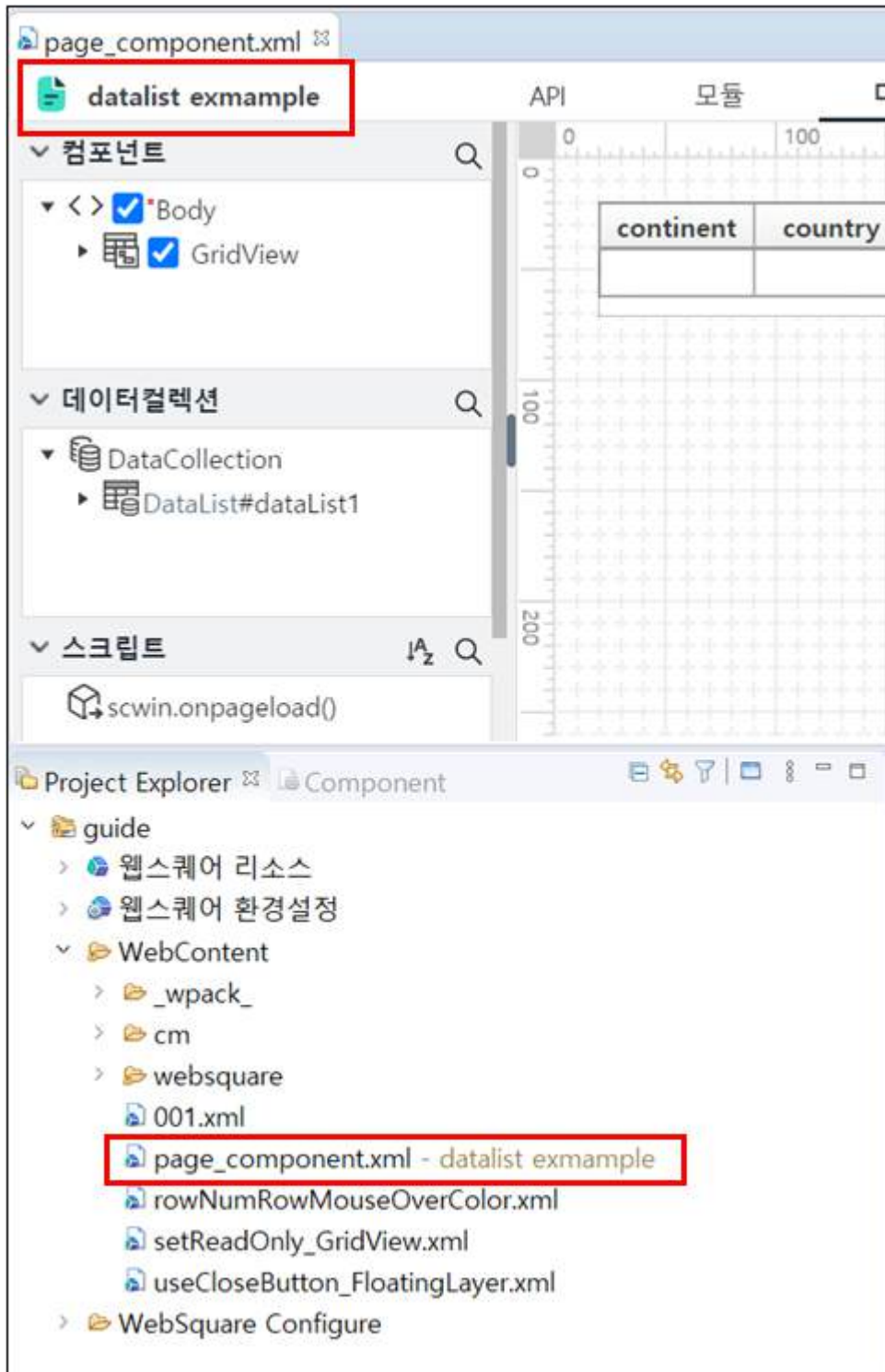
입력된 항목들은 Dev Kit의 통합검색 탭에서 검색이 가능하고 페이지 컴포넌트 탭 표시할 정보를 입력하십시오.

- ID : 페이지의 프로젝트에서 사용하는 페이지 ID.
- 이름 : 페이지 상단과 Project Explorer에 표시되는 페이지의 업무명. (필수 항목)

그림 7-8 문서화 정보 - 일반 정보 - 이름

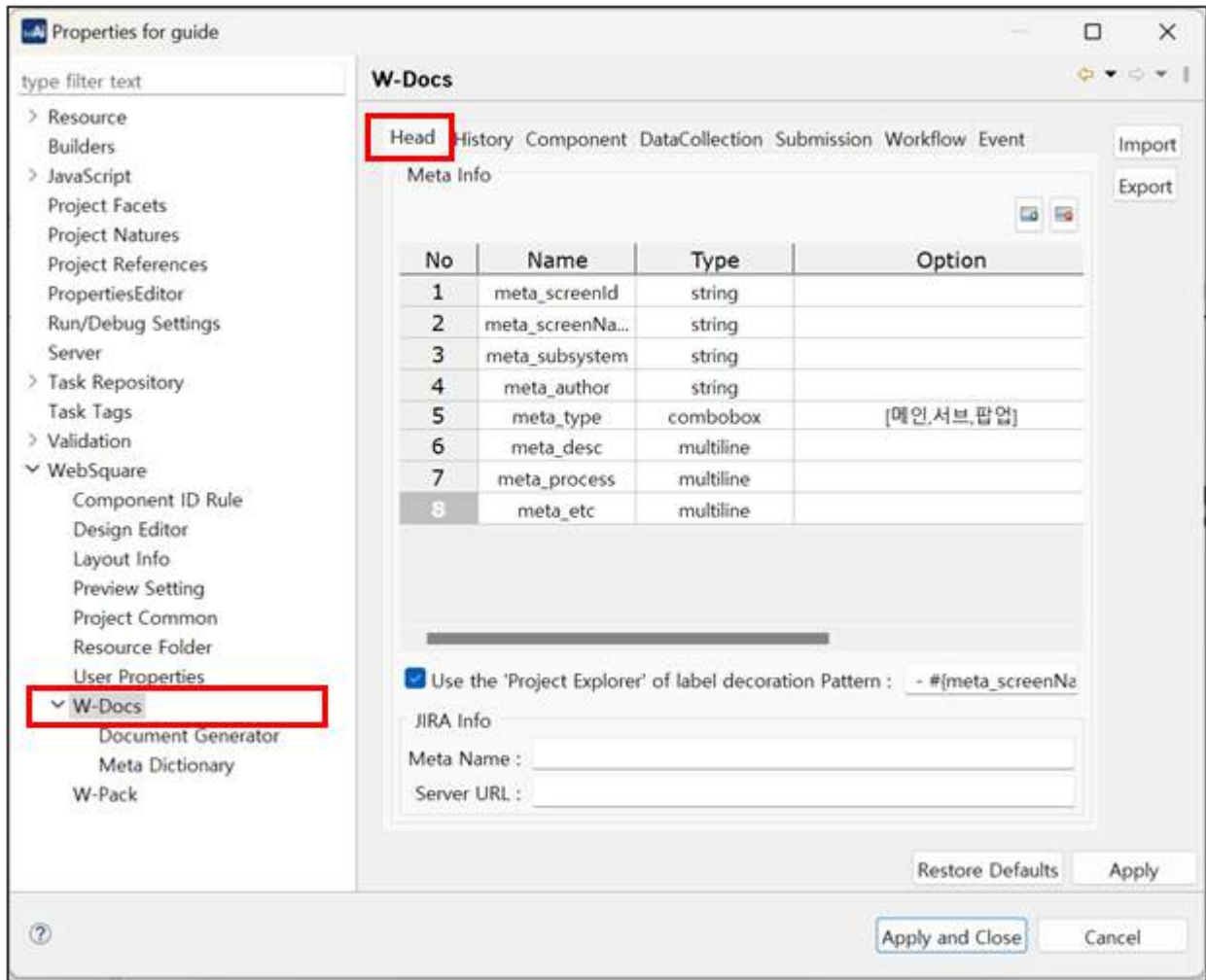


그림 7-9 문서화 정보 - 일반 정보 - 이름



프로젝트의 Properties 메뉴를 선택하여 일반 정보로 표시할 항목을 정의할 수 있습니다.

그림 7-10 프로젝트 - Properties



7.4.1.1.2. 개정 이력

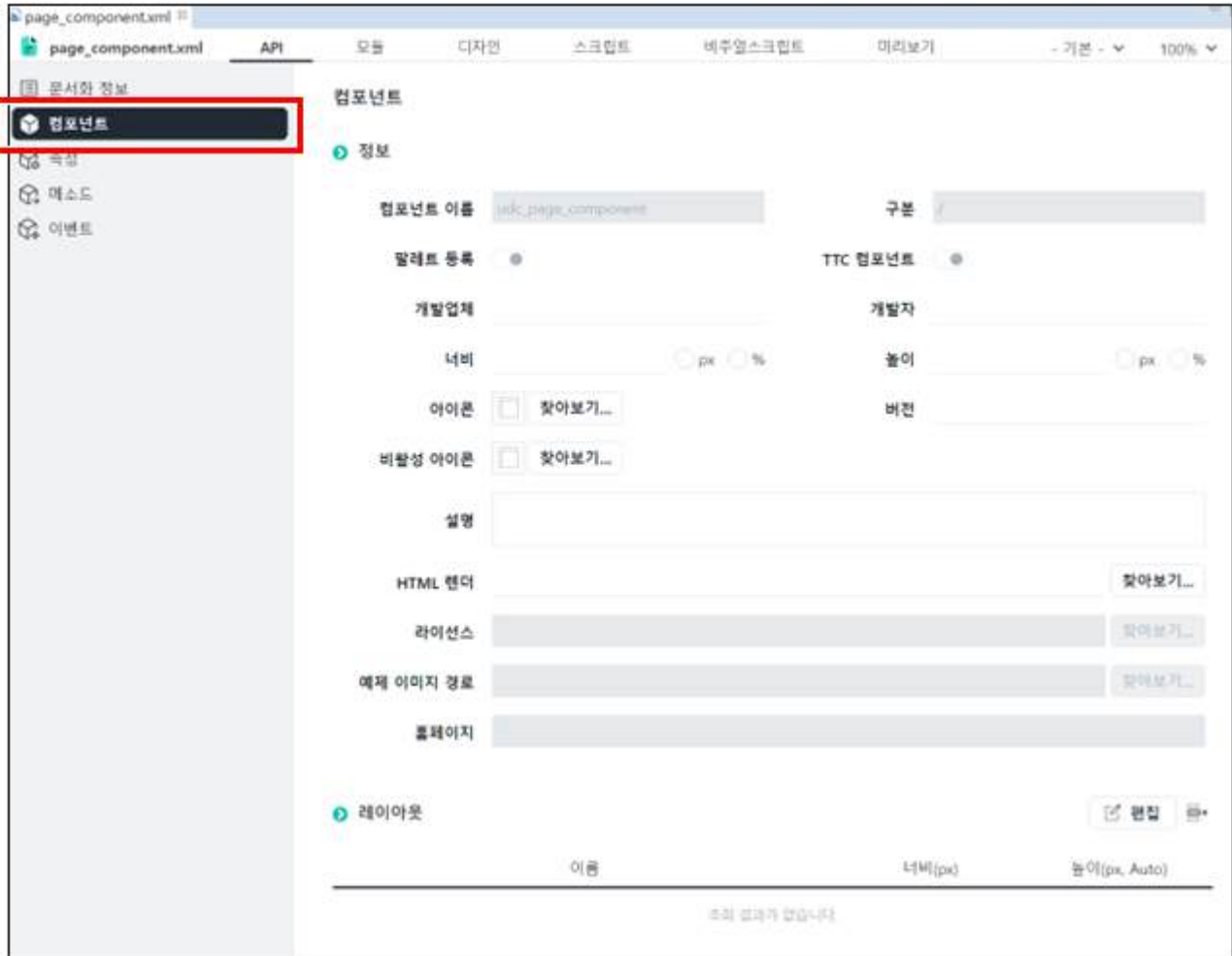
우측 상단의 버튼을 클릭하여 개정 이력을 추가하거나 삭제할 수 있습니다.

그림 7-11 문서화 정보 - 개정 이력



7.4.1.2. 컴포넌트

그림 7-12 컴포넌트



7.4.1.2.1. 정보

생성할 UDC, TTC, EXC 컴포넌트에 대한 정보를 작성한다. 항목을 입력할 시 xml 화면 스크립트에 주석으로 작성됩니다. 주석은 스크립트 및 비주얼스크립트에서 개발자가 직접 변경할 수 있으나, 유효성을 위해 **API** 탭에서 직접 작성하는 것을 권장합니다.

- 컴포넌트 이름 : 컴포넌트 이름은 현재 활성화 된 화면의 컴포넌트 타입에 따라 xml 화면 명 앞에 'udc_', 'ttc_', 'exc_'가 자동으로 입력되며 사용자가 임의로 변경할 수 없는 항목.
- 구분 : 현재 활성화된 xml의 Deploy Path의 하위 경로가 자동으로 입력되며 사용자가 임의로 변경할 수 없는 항목.
- 팔레트 등록 : 팔레트 등록을 선택할 수 있는 토글 버튼.
- TTC 컴포넌트 : TTC 컴포넌트로의 생성 여부를 선택할 수 있는 토글 버튼.
- 아이콘, 비활성 아이콘, HTML 렌더, 라이선스, 예제 이미지 경로
 - 클릭 또는 파일/폴더를 선택할 때 표시할 이미지를 등록. (16X16 사이즈의 PNG 파일만 선택 가능.) 등록된 이미지들은 스튜디오에서 UDC, TTC, EXC를 표시하는 아이콘으로 사용됨.

그림 7-13 컴포넌트

컴포넌트

정보

컴포넌트 이름

udc_001

구분

/

팔레트 등록

☒

TTC 컴포넌트

☒

개발업체

개발자

너비

☐ px ☐ %

높이

☐ px ☐ %

아이콘

☐

찾아보기...

버전

비활성 아이콘

☐

찾아보기...

설명

HTML 렌더

찾아보기...

라이선스

찾아보기...

예제 이미지 경로

찾아보기...

홈페이지

7.4.1.2.2. 레이아웃

7.4.1.2.2. 레이아웃 적용

편집 버튼을 클릭하여 현재 화면 파일에 적용할 레이아웃을 선택할 수 있습니다.

그림 7-14 레이아웃

레이아웃

편집

이름

너비(px)

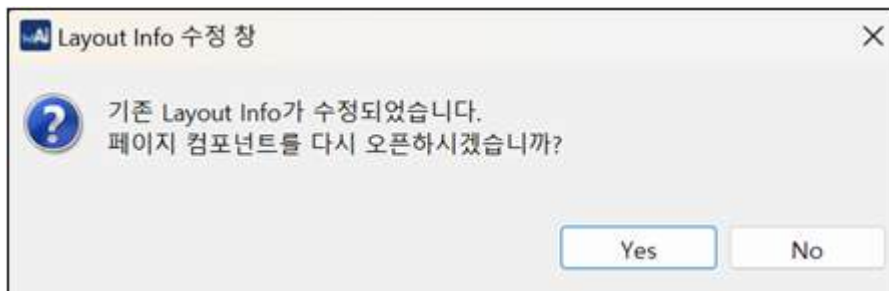
높이(px, Auto)

조회 결과가 없습니다.

그림 7-15 사용할 레이아웃 선택



그림 7-16 레이아웃 적용



등록한 레이아웃은 디자인 에디터의 우측 상단에서 선택할 수 있습니다.

그림 7-17 레이아웃 적용

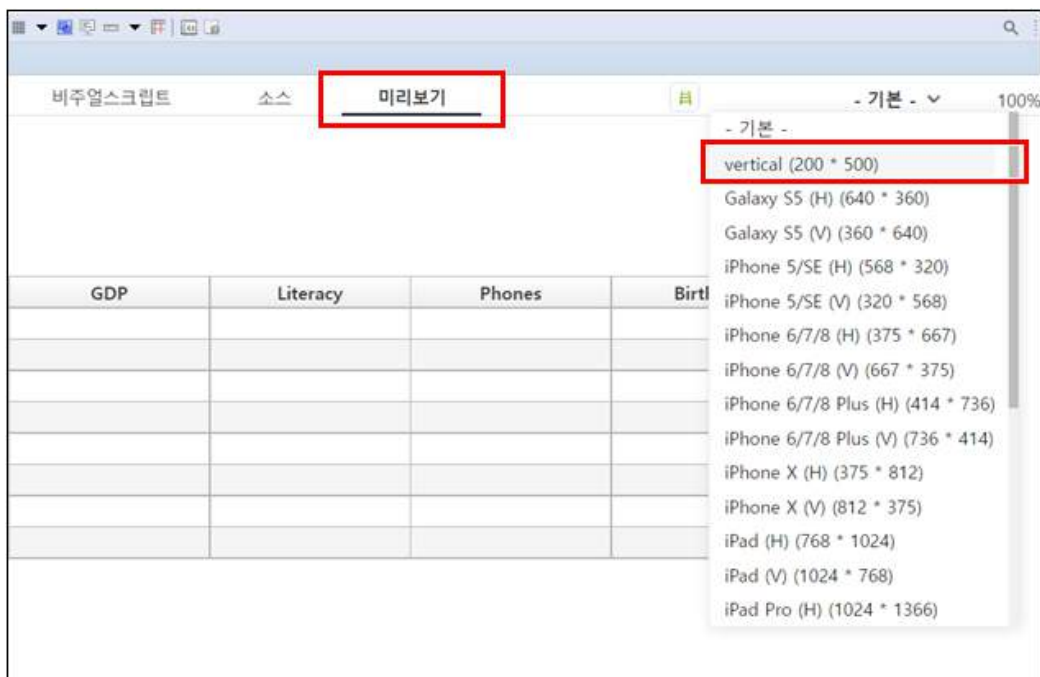
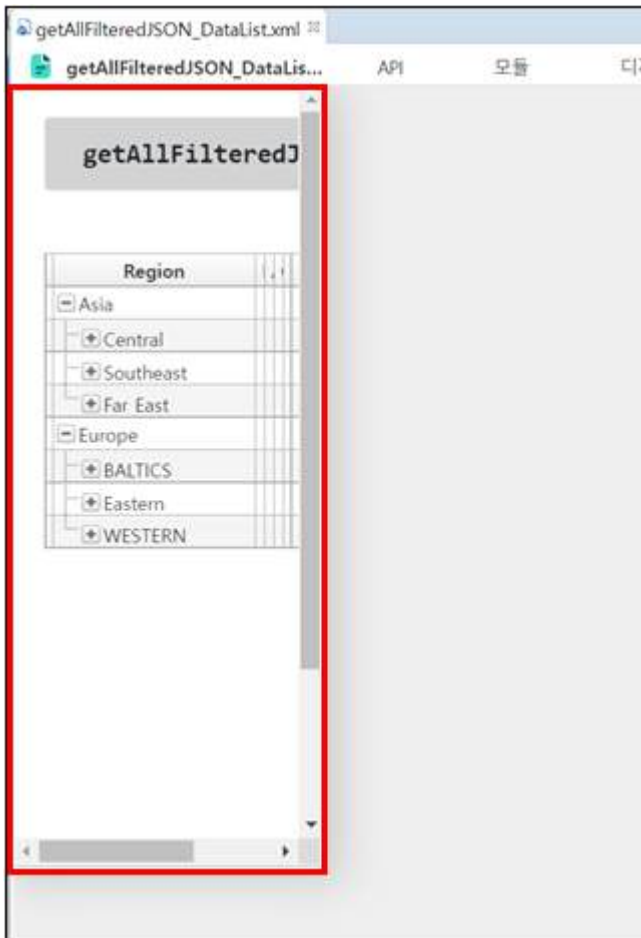


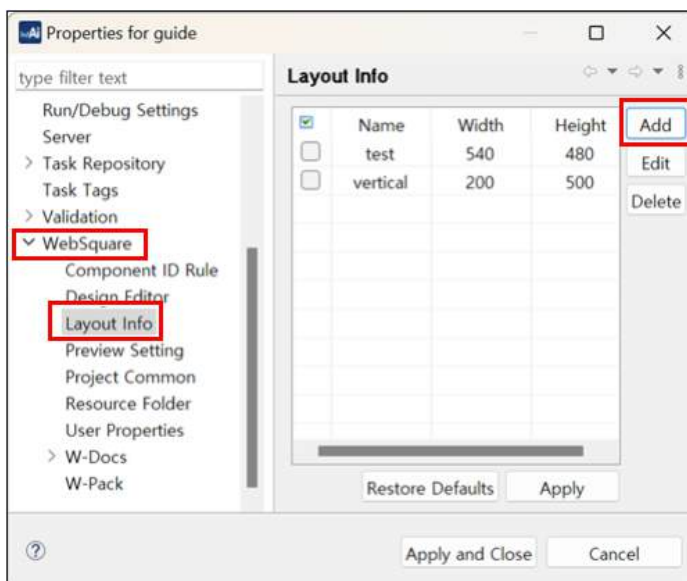
그림 7-18 레이아웃이 적용된 모습



7.4.1.2.2. 레이아웃 생성

레이아웃 편집 버튼을 클릭할 때 표시되는 레이아웃은 프로젝트의 Properties 메뉴를 사용하여 생성할 수 있습니다. **Project - Properties** 메뉴를 선택한 후, **WebSquare - Layout Info** 옵션을 선택하십시오.

그림 7-19 레이아웃 추가



위의 화면에서 **Add** 버튼을 클릭한 후,아래와 같은 창이 표시되면 레이아웃 정보를 입력하십시오.

그림 7-20 레이아웃 정보

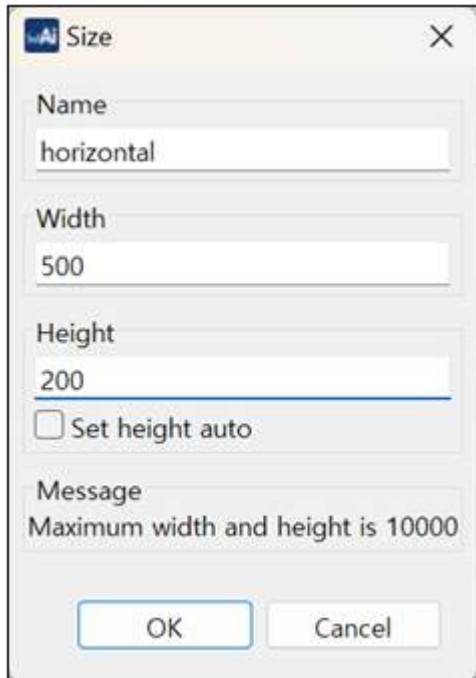
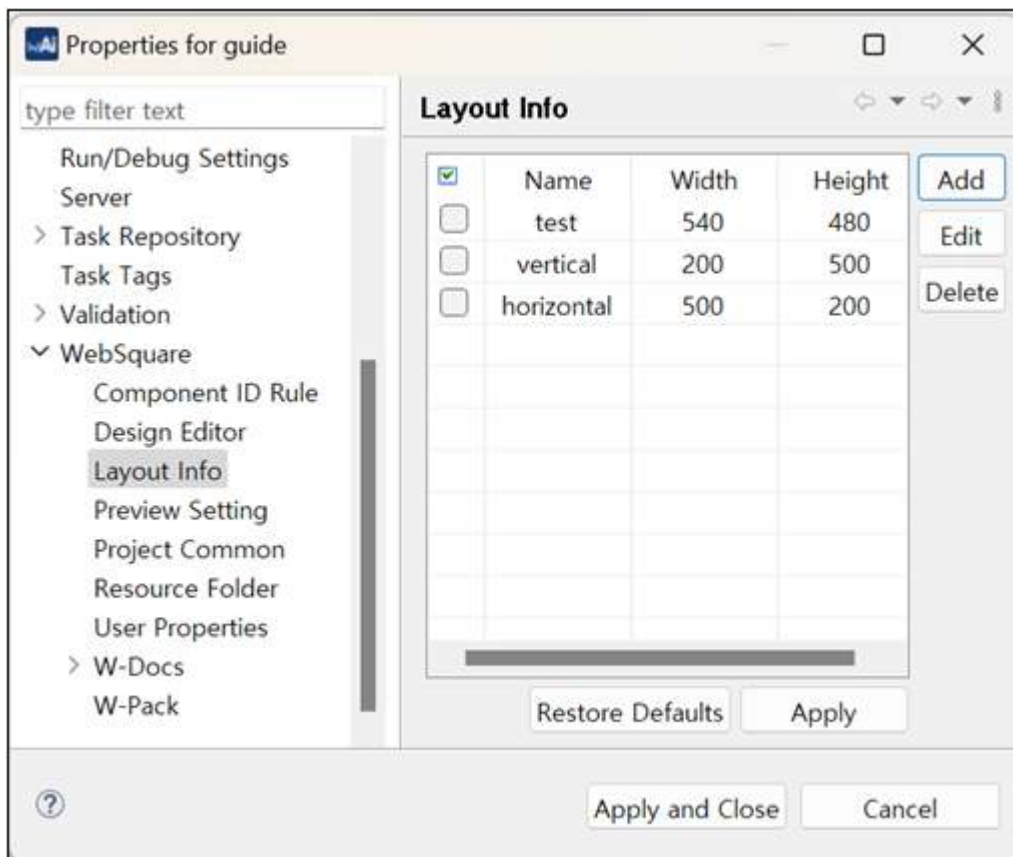


그림 7-21 레이아웃 추가



7.4.1.3. 속성

UDC, TTC, EXC 컴포넌트의 속성을 정의합니다. UDC, TTC, EXC 컴포넌트 선택시 **Component** 뷰의 속성 탭에 정의한 속성들이 표시됩니다.

그림 7-22 속성



7.4.1.3.1. 속성 정의하기

우측 상단의 버튼을 클릭하고 속성을 추가 또는 기존에 추가된 속성을 삭제할 수 있습니다.

그림 7-23 속성 정의 추가



좌측 상단의 화살표 버튼을 클릭하면 속성 설정창을 닫을 수 있습니다.

그림 7-24 속성 설정창 닫기

형식	속성 타입.
구분	Component 뷰의 속성 탭에서 분류할 때 사용할 카테고리.
기본값	속성의 기본값.
옵션	위에서 설정한 형식에 따라 표시할 옵션 값들을 정의. [value1, value2, value3] 와 같이 대괄호와 콤마로 구분해서 작성.
바인딩 대상	위에서 설정한 형식이 <code>bindDataList</code> , <code>bindDataMap</code> , 혹은 <code>bindDataColumn</code> 인 경우, 화면에 포함하고 있는 DataCollection 객체를 바인딩할 수 있음.
필수여부	속성의 필수 여부를 토글 버튼으로 지정.

7.4.1.4. 메소드

Websquare Page Component를 컴포넌트로 사용하는 경우 메소드를 직접 설정할 수 있습니다. 여기서 정의한 메소드는 Component 뷰의 메소드 탭에 표시됩니다.

그림 7-25 메소드



7.4.1.4.1. 메소드 정의하기

우측 상단의 버튼을 클릭하여 속성을 추가하거나 기존에 추가된 메소드를 삭제할 수 있습니다.

그림 7-26 메소드 정의 추가

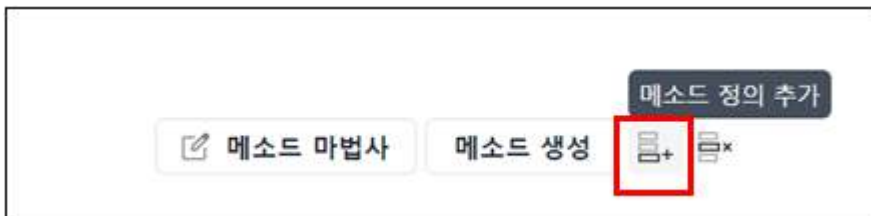


그림 7-27 메소드 정의

→ 메소드

상세 정보

이름

desc

설명

숨김

매개 변수

순번 형식 이름 설명

조회 결과가 없습니다.

반환 값

형식

이름

설명

예제

숨김 속성을 활성화 하면 **Component** 뷰에 해당 메소드가 나타나지 않으며 메소드 생성시 Source 탭의 <w2:public Info> 태그에 추가되지 않습니다. 숨김 속성을 비활성화 하면 Source 탭의 <w2:publicInfo> 태그가 추가되며 해당 태그의 임의의 값을 수정시 오류가 발생할 수 있으므로 Source 탭에서 직접 수정하지 않고 해당 화면에서 수정합니다.

7.4.1.5. 이벤트

UDC, TTC, EXC 컴포넌트의 이벤트를 정의합니다. UDC, TTC, EXC 컴포넌트 선택시 **Component** 뷰의 **이벤트** 탭에 정의한 이벤트들이 표시됩니다.



7.4.1.5.1. 이벤트 정의하기

그림 7-28 이벤트 정의하기



이벤트 생성 버튼을 클릭하여 이벤트 정의에 대응하는 JavaScript 메소드 식을 생성합니다.

그림 7-29 이벤트 생성



그림 7-30 이벤트 구현

```

56  /**
57   * @event
58   * @name onUserEvent1
59   * @description desc
60   * @param
61   * @example
62   */
63  scwin.emit_onUserEvent1 = function () {
64    |  $p.emit("onUserEvent1");
65  };
66

```

7.4.2. 모듈

7.4.2.1. 공통 컴포넌트

그림 7-31 모듈 - 공통 컴포넌트



7.4.2.1.1. 전역 공통 컴포넌트 (config.js)

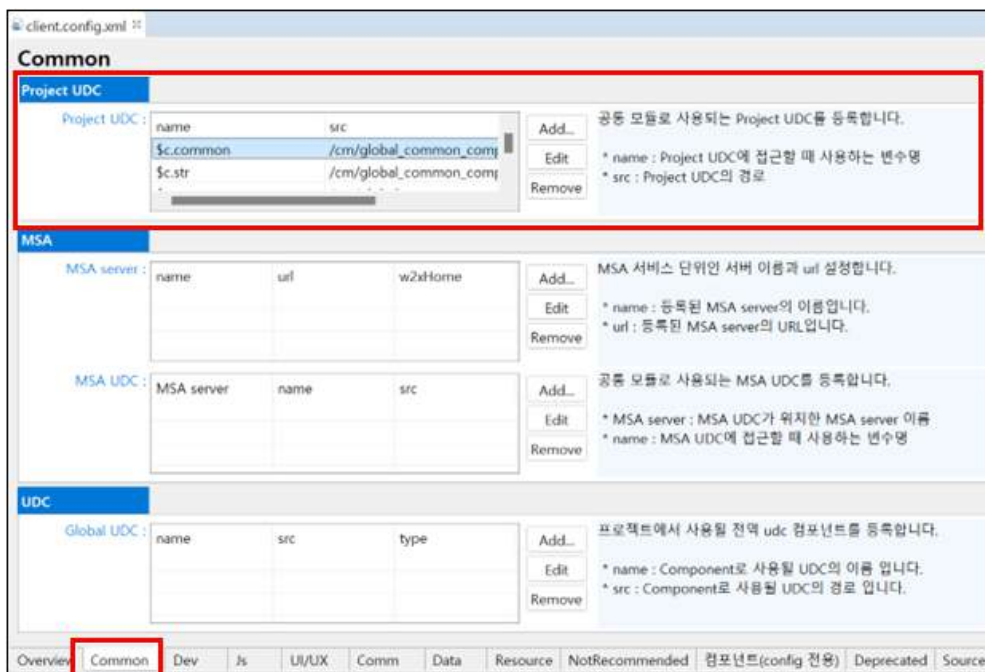
client.config.xml 설정 파일에 등록 되어있는 Project UDC의 정보를 나타내며 해당 컴포넌트는 공통 모듈로 사용됩니다.

그림 7-32 전역 공통 컴포넌트 (config.js)



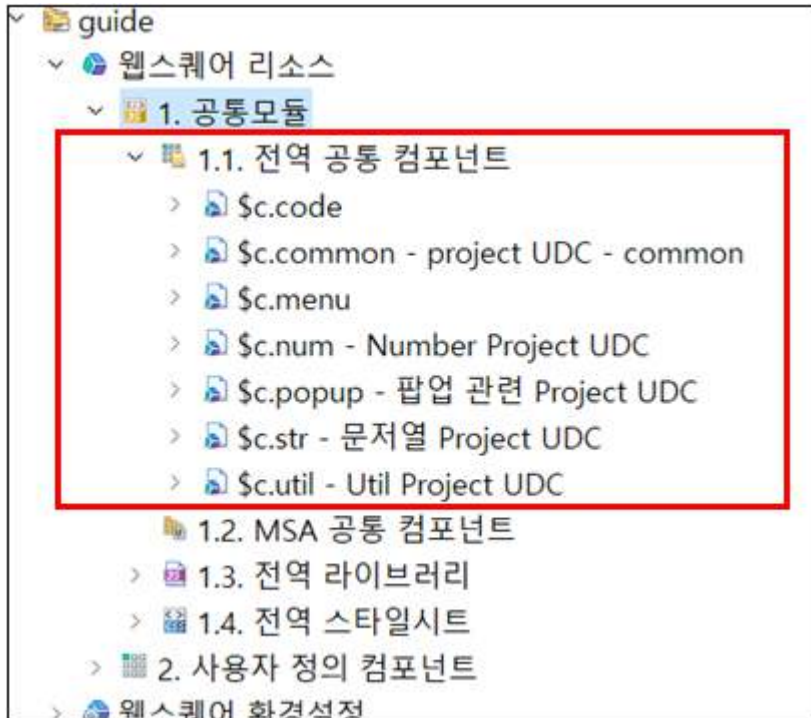
설정 버튼을 클릭하면 아래와 같이 클라이언트 설정 파일이 열립니다. 클라이언트 설정 파일에서 Project UDC 항목을 직접 추가할 수 있습니다.

그림 7-33 클라이언트 설정 (client.config.xml) - Common - Project UDC



전역 공통 컴포넌트는 Project Explorer에서도 확인할 수 있습니다.

그림 7-34 전역 공통 컴포넌트



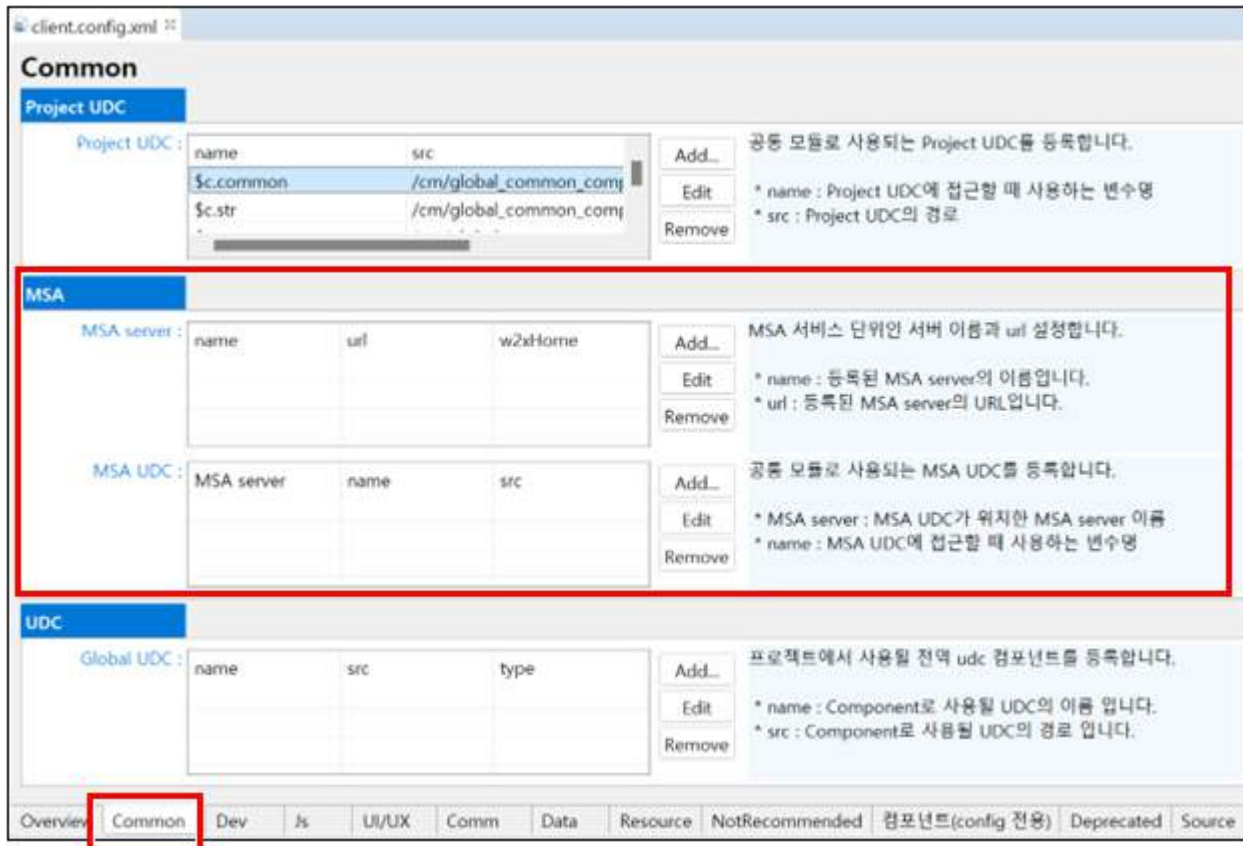
7.4.2.1.2. MSA 공통 컴포넌트 (config.js)

client.config.xml 설정 파일에 등록 되어있는 MSA의 정보를 표시합니다.

그림 7-35 MSA 공통 컴포넌트 (config.js)

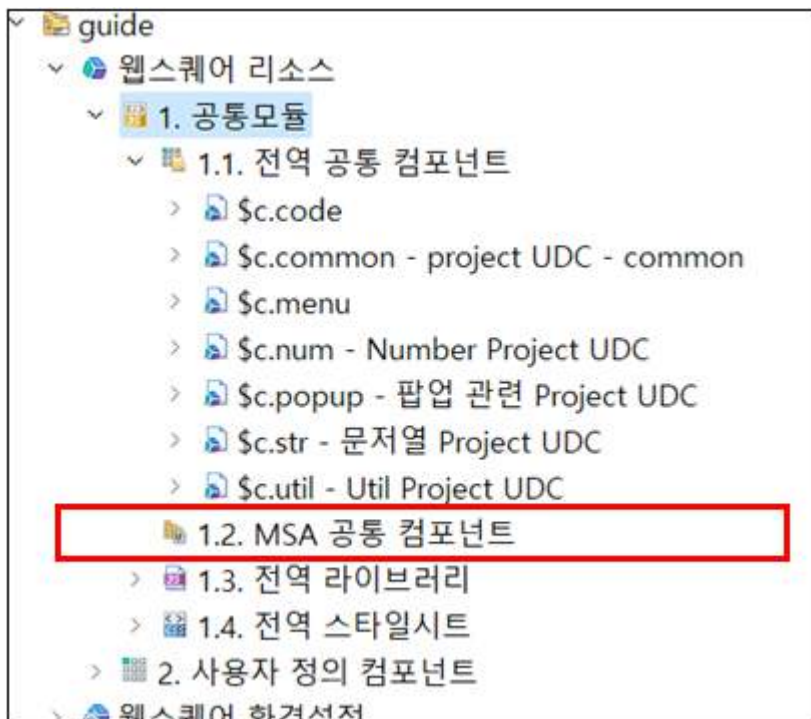


그림 7-36 클라이언트 설정 (client.config.xml) - Common - MSA



MSA 공통 컴포넌트는 Project Explorer에서도 확인할 수 있습니다.

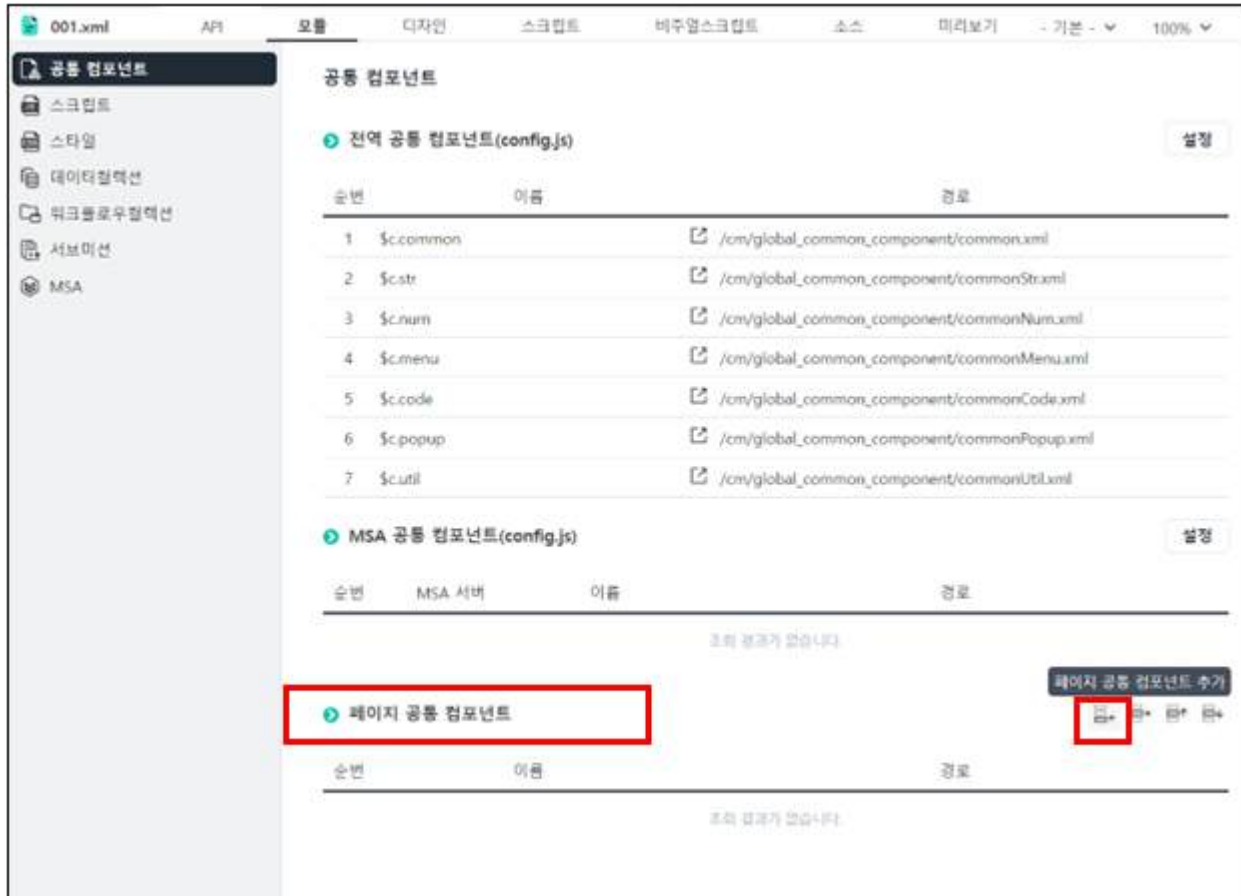
그림 7-37 MSA 공통 라이브러리



7.4.2.1.3. 페이지 공통 컴포넌트

프로젝트 내부 공통 컴포넌트를 등록하여 컴포넌트에 있는 함수를 사용할 수 있습니다. (11. 컴포넌트 생성 참고.)

그림 7-38 페이지 공통 컴포넌트



페이지 공통 컴포넌트를 추가할 경우, 아래와 같이 이미 생성된 프로젝트 내부 컴포넌트 (WebSquare Common Component) 목록이 표시됩니다.

그림 7-39 프로젝트 내부 컴포넌트 (WebSquare Common Component)

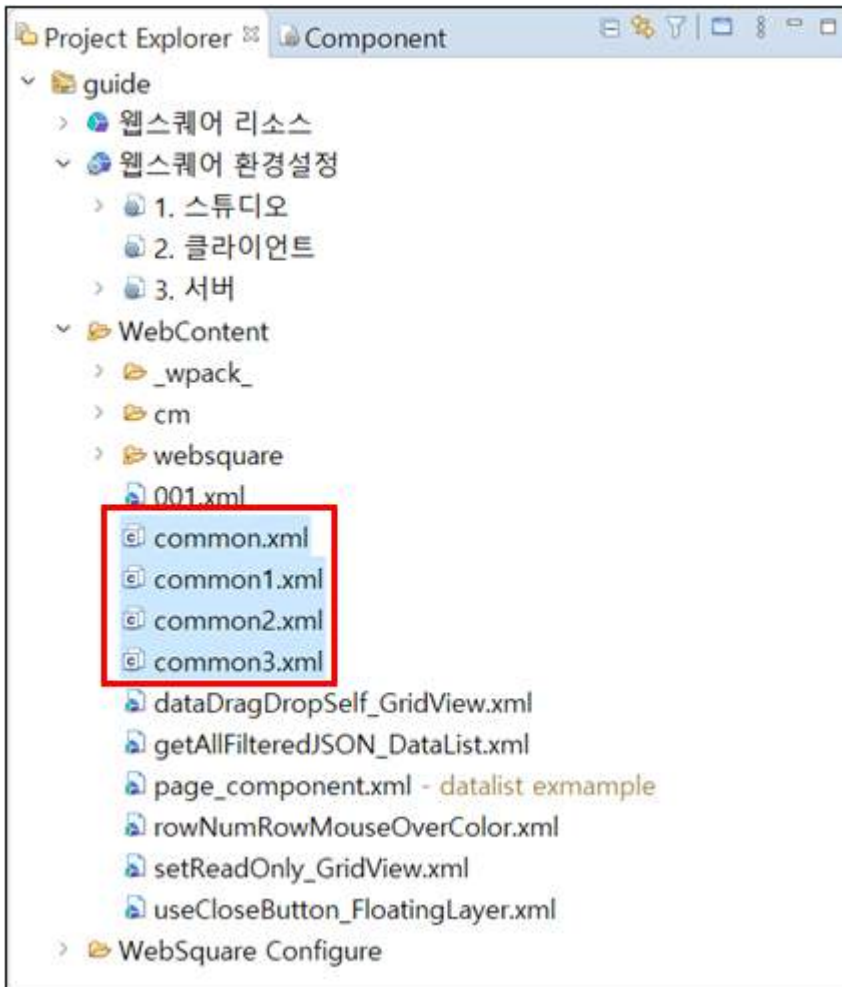


그림 7-40 공통 컴포넌트 목록



이름 항목을 더블 클릭하여 페이지의 공통 컴포넌트의 이름을 변경할 수 있습니다.

페이지 공통 컴포넌트				
순번	이름	경로		
1	common1	/common.xml		

7.4.2.2. 스크립트

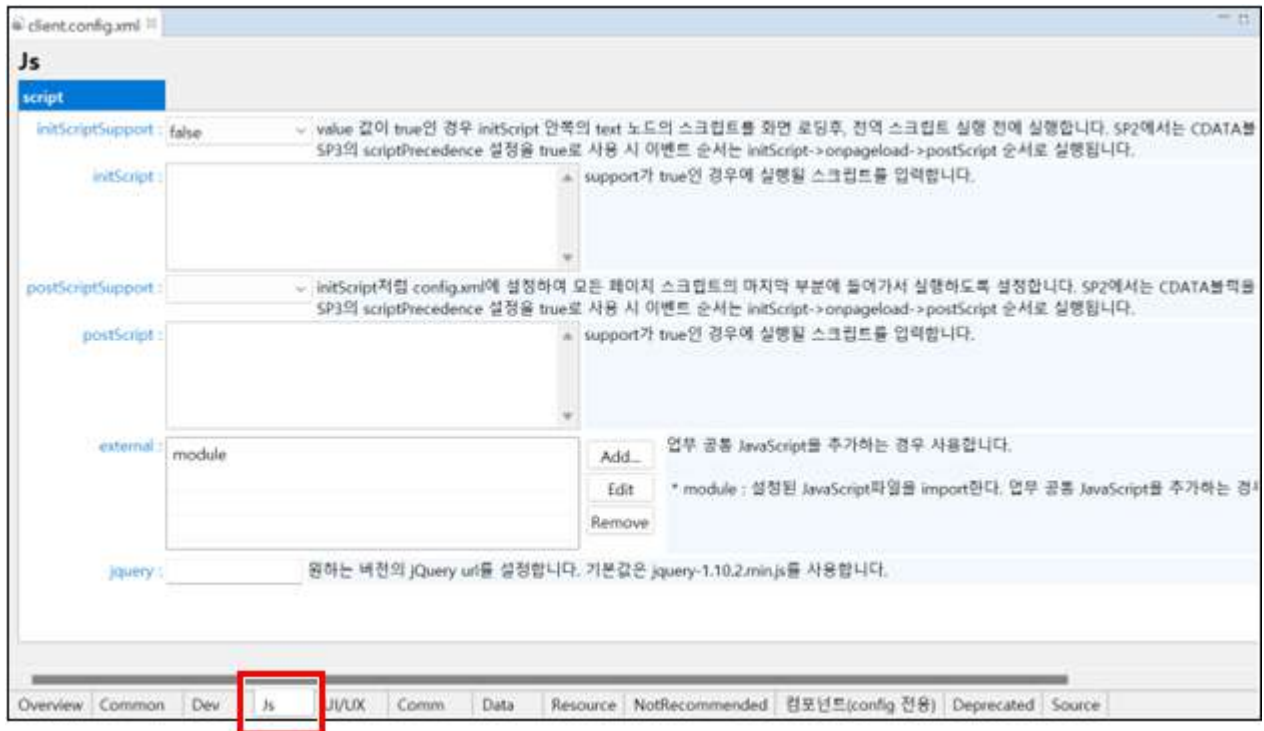


7.4.2.2.1. 전역 라이브러리 (config.js)

WebSquare Component Page에 공통으로 import 되어있는 전역 라이브러리를 표시합니다. 해당 라이브러리는 클라이언트 설정 (client.config.xml) 파일에 미리 정의되어 있으며 화면 렌더링시 갱신됩니다.

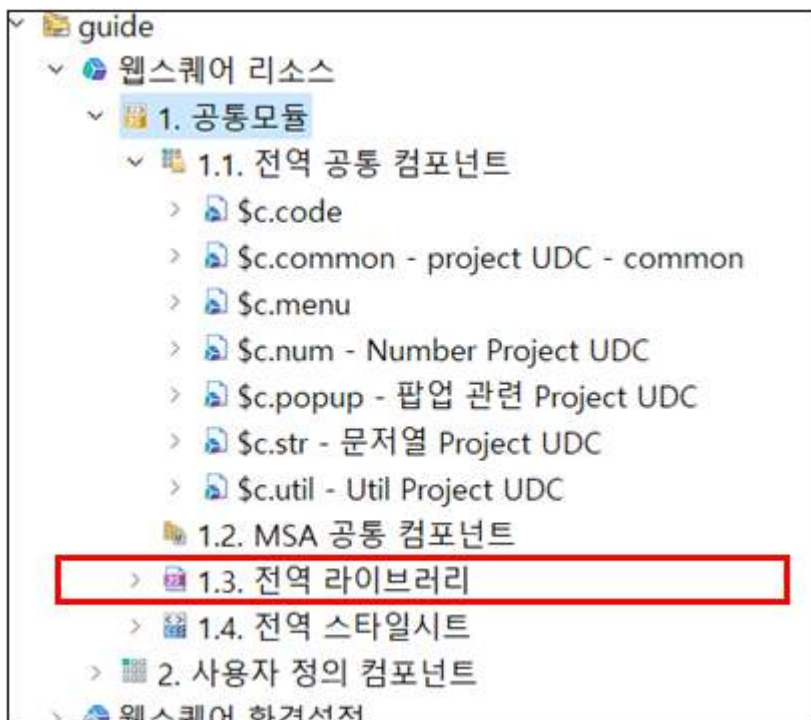


그림 7-41 client.config.xml - Js



전역 라이브러리는 Project Explorer에서도 확인할 수 있습니다.

그림 7-42 전역 라이브러리



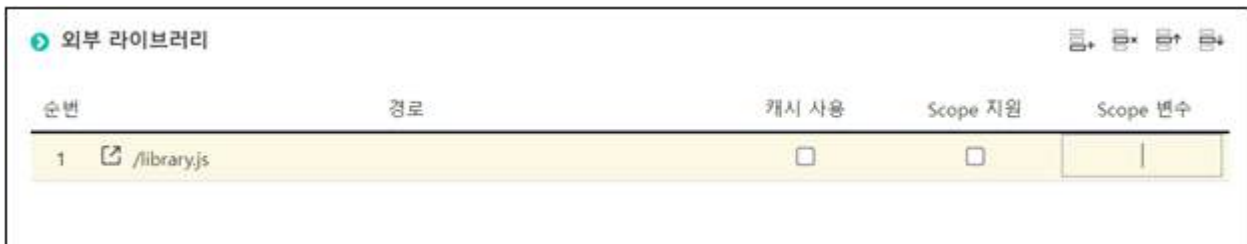
7.4.2.2.2. 외부 라이브러리

공통 라이브러리를 제외한 개별 Component Page에 추가된 라이브러리를 표시합니다. 추가된 라이브러리의 정보는 Source탭에서 확인할 수 있으며 해당 태그의 각 속성에 설정값이 표시됩니다.

그림 7-43 외부 라이브러리



그림 7-44 외부 라이브러리 추가



7.4.2.3. 스타일

그림 7-45 모듈 - 스타일



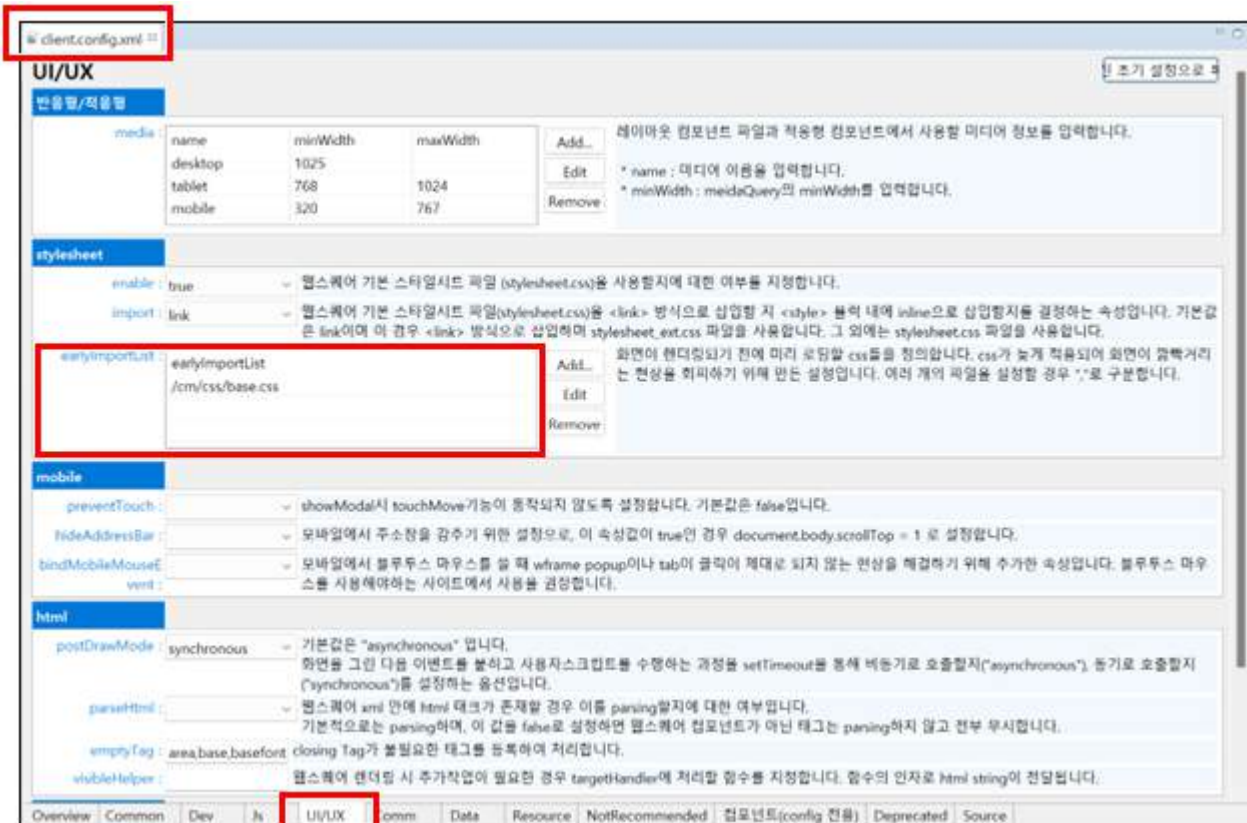
7.4.2.3.1. 전역 스타일 시트 (config.js)

client.config.xml 설정 파일에 등록 되어있는 UI/UX - stylesheet - earlyImportList 항목을 표시합니다.

그림 7-46 전역 스타일 시트 (config.js)

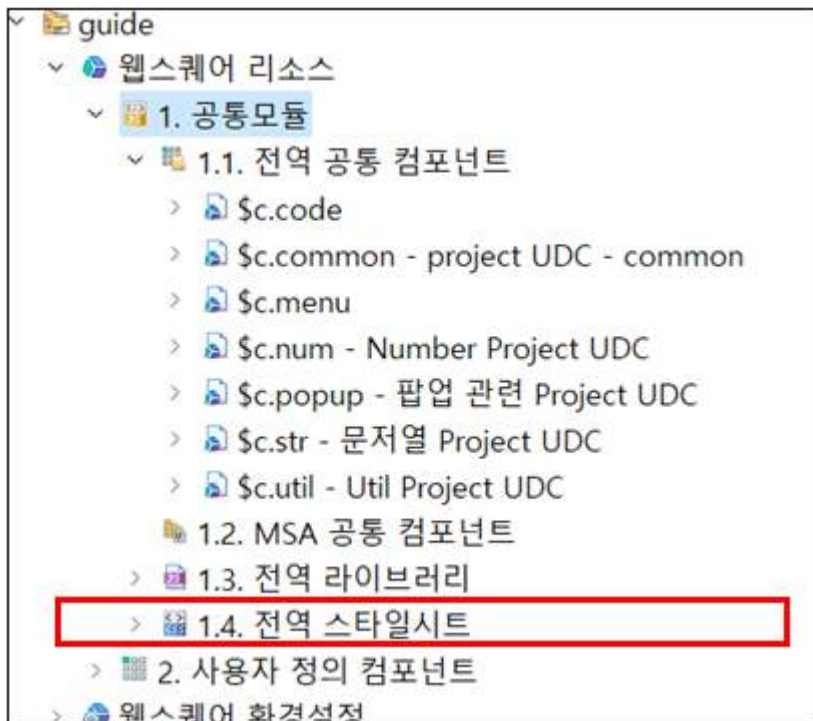


그림 7-47 client.config.xml



전역 스타일시트는 Project Explorer에서도 확인할 수 있습니다.

그림 7-48 전역 스타일시트



7.4.2.3.2. MSA 스타일 시트 (config.js)

client.config.xml 설정 파일에 등록 되어있는 Common - MSA 항목을 표시합니다.

그림 7-49 MSA 스타일 시트 (config.js)



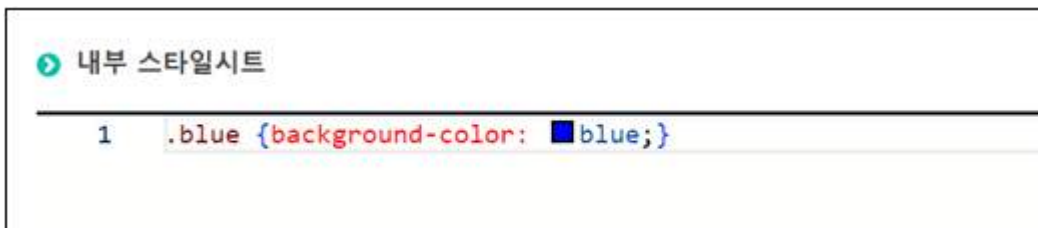
7.4.2.3.3. 외부 스타일시트

earlyImportList에 정의되지 않은 신규 CSS 파일은 외부 스타일 시트로 추가할 수 있습니다.

7.4.2.3.4. 내부 스타일 시트

내부 스타일시트 영역에 CSS를 직접 작성하여 컴포넌트의 class 속성값으로 지정할 수 있습니다.

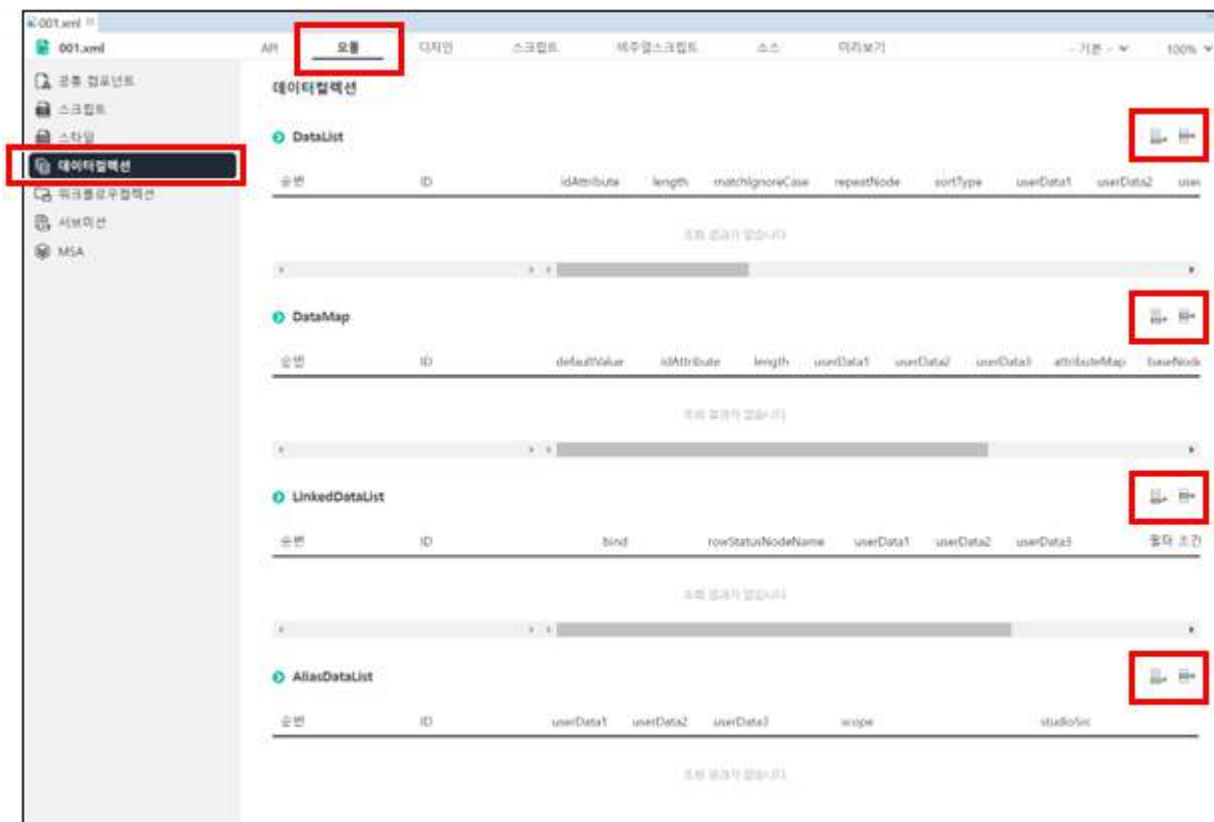
그림 7-50 내부 스타일시트 직접 지정



7.4.2.4. 데이터컬렉션

데이터컬렉션 객체를 추가하거나 삭제할 수 있습니다. 우측 상단의 추가 및 삭제 버튼을 사용하십시오.

그림 7-51 모듈 - 데이터컬렉션



7.4.2.4.1. DataList

DataList 우측의 추가 버튼을 클릭하여 DataList를 추가할 수 있습니다.



실제 DataList 추가 예제는 [12.2 DataList](#)를 참고하십시오.

그림 7-52 DataList 추가



DataList 정보를 수정하는 버튼을 클릭해서 속성 및 이벤트를 설정할 수 있습니다.

그림 7-53 DataList 정보 수정



그림 7-54 DataList 정보 수정

DataList

속성 | 이벤트 | 메소드

Basic & ETC

ID: dataList1

idAttribute:

length:

matchIgnoreCase: -선택- ▼

repeatNode: map

sortType: -선택- ▼

userData1:

userData2:

userData3:

DataList의 컬럼을 정의하는 버튼을 클릭하고 DataList 컬럼 정보를 정의할 수 있습니다.

그림 7-55 DataList 컬럼 설정

DataList

순번	ID	idAttribute	length
1	dataList1		

Navigation: < > <>

그림 7-56 DataList 컬럼 설정

→ DataList: dataList1

Columns

순번	ID	name	dataType	defaultValue	encYN
1	col1	Continent	text		
2	col2	Country	text		
3	col3	Item	text		
4	col4	Quantity	number		

정의한 컬럼에 대한 데이터를 추가하는 버튼을 클릭하고 데이터를 추가하십시오.

그림 7-57 데이터 추가

→ DataList

순번	ID	idAttribute	length
1	dataList1		

로컬에서 정의한 데이터를 사용할 경우 **데이터 사용** 옵션을 선택합니다.

그림 7-58 로컬 데이터 사용

→ DataList: dataList1

Data ☒ 데이터 사용

순번	col1	col2	col3	col4
1				
2				

7.4.2.4.2. DataMap

DataMap 우측의 추가 버튼을 클릭하여 DataMap을 추가할 수 있습니다.



실제 DataMap 추가 예제는 [12.3 DataMap](#)을 참고하십시오.

그림 7-59 DataMap 추가



DataMap을 수정하는 버튼을 클릭해서 속성 및 이벤트를 설정할 수 있습니다.

그림 7-60 DataMap 정보 수정

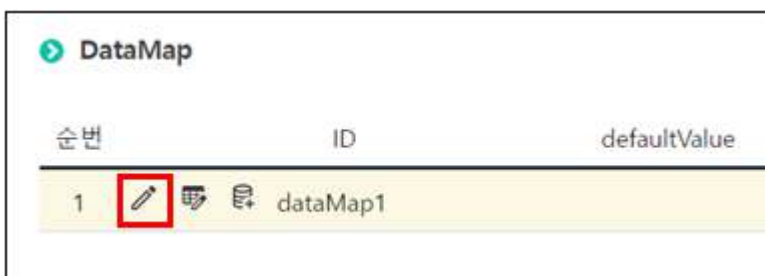


그림 7-61 DataMap 정보 수정

DataMap

속성 이벤트 메소드

Basic & ETC

ID

defaultValue

idAttribute

length

userData1

userData2

userData3

DataMap의 키 데이터를 추가하는 버튼을 클릭하여 키 데이터를 정의하십시오.

그림 7-62 DataMap 키 설정

DataMap

순번	ID	defaultValue
1	dataMap1	

그림 7-63 DataMap 키 설정

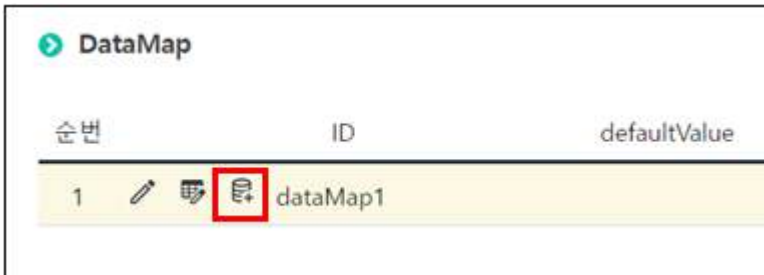
DataMap

Keys

순번	ID	name	dataType	de
1	key1	name1	text	▼
2	key2	name2	text	▼
3	key3	name3	text	▼

정의한 키에 대한 데이터를 추가하는 버튼을 클릭하고 데이터를 추가하십시오.

그림 7-64 데이터 추가



로컬에서 정의한 데이터를 사용할 경우 **데이터 사용** 옵션을 선택합니다.

그림 7-65 로컬 데이터 사용



7.4.2.4.3. LinkedList

LinkedList 우측의 추가 버튼을 클릭하여 LinkedList를 추가할 수 있습니다.



실제 LinkedList 추가 예제는 [12.4 LinkedList](#)를 참고하십시오.

그림 7-66 LinkedDataList 추가



LinkedDataList를 수정하는 버튼을 클릭해서 속성 및 이벤트를 설정할 수 있습니다.

그림 7-67 LinkedDataList 정보 수정



그림 7-68 LinkedDataList 정보 수정

→ **LinkedDataList**

속성 **메소드**

▶ **Basic & ETC**

ID

bind -선택- ▼

rowStatusNodeName

userData1

userData2

userData3

LinkedDataList의 바인딩 정보 및 필터링/정렬 조건을 설정하는 버튼을 클릭하십시오.

그림 7-69 바인딩 정보 및 필터링/정렬 조건 설정

▶ **LinkedDataList**

순번	ID	bind
1	linkedDataList1	dataList1

LinkedDataList의 바인딩 정보 및 필터링/정렬 조건을 설정합니다.

그림 7-70 바인딩 정보 및 필터링/정렬 조건 설정

→ **LinkedList: linkedDataList1**

▶ 바인드 정보

바인드 -선택-

필터 조건 정렬 조건

▶ Columns

순번	ID	name	dataType	defa
1	col1	Continent	text	
2	col2	Country	text	
3	col3	Item	text	
4	col4	Quantity	number	

7.4.2.5. 워크플로우컬렉션

우측의 워크플로우컬렉션 추가 버튼을 클릭하여 워크플로우를 추가할 수 있습니다.

그림 7-71 모듈 - 워크플로우컬렉션



워크플로우컬렉션 정보를 입력합니다.

그림 7-72 워크플로우컬렉션 정보 입력

Workflow Manager

Workflow ID : Process Message :

resolveCallback : ☐ rejectCallback : ☐

Description :

Step List

Type : Action : ☐

☒ Serial Step ☐ Parallel Step

Order	Action	Pre	Post
1	function1		

Type : Pre : ☐

Action : Post : ☐

Description :

function1 (1)

scwin.workflow2_resolveCallback

워크플로우컬렉션이 추가되어 표시되고, 콜백 함수 앞의 아이콘을 클릭하면 스크립트 탭의 해당 부분으로 이동할 수 있습니다.

그림 7-73 추가된 워크플로우컬렉션

워크플로우컬렉션

워크플로우컬렉션

순번	워크플로우 ID	설명	작업 완료 콜백	작업 실패 콜백
1	workflow2		scwin.workflow2_resolveCall...	scwin.workflow2_rejectCallba...

7.4.2.6. 서브미션

모듈 - 서브미션탭에서 신규 Submission을 추가할 수 있습니다.



실제 Submission 추가 예제는 [14.4 Submission 생성](#)를 참고하십시오.

그림 7-74 서브미션 추가



서브미션 정보를 입력합니다.

그림 7-75 서브미션 정보 입력

Table 7-1 Submission 속성

항목	설명
ID	Submission ID.
Reference	서버로 전송할 데이터 형식. (DataCollection 버튼 클릭 후 선택. 단, 해당 DataCollection을 미리 설정해야 함.)
Target	서버로부터 수신할 데이터 형식. (DataCollection 버튼 클릭 후 선택. 단, 해당 DataCollection을 미리 설정해야 함.)
URL Action	수신할 데이터가 있는 서버 URL
Process Message	Submission 실행 중 상태 창에 표시할 메시지.

서브미션이 추가되어 표시되고, 서브미션 수정 버튼을 클릭하여 서브미션 정보를 수정할 수 있습니다.

그림 7-76 추가된 서브미션

서브미션						
서브미션						
순번	ID	action	encoding	mediaType	method	mode
1	submission1		UTF-8	application/json	post	asynchronous

7.4.2.7. MSA

MSA 서비스를 추가하거나 삭제할 수 있습니다. 우측 상단의 추가 및 삭제 버튼을 사용하십시오.

그림 7-77 모듈 - MSA



7.4.2.7.1. 채널

우측 버튼을 클릭하여 MSA 채널을 추가합니다.

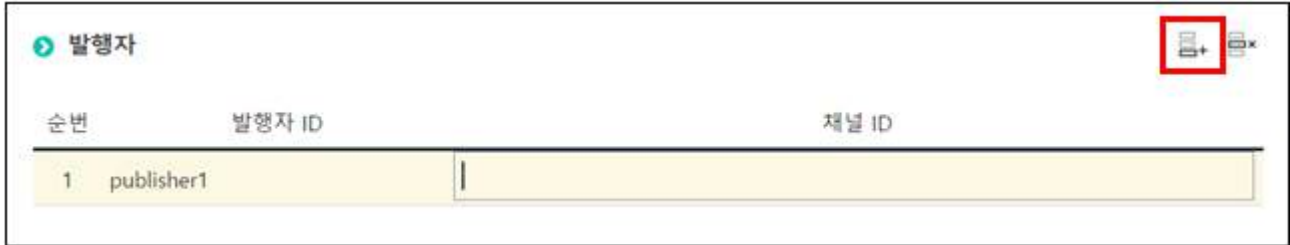
그림 7-78 MSA 채널 추가

채널	
순번	채널 ID
1	channel

7.4.2.7.2. 발행자

우측 버튼을 클릭하여 MSA 발행자와 채널 ID를 추가합니다.

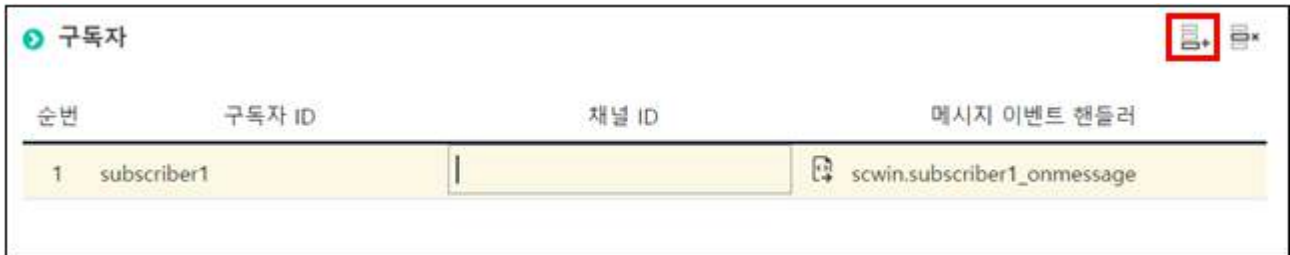
그림 7-79 MSA 발행자 및 채널 ID 추가



7.4.2.7.3. 구독자

우측 버튼을 클릭하여 구독자와 채널 ID, 메시지 이벤트 핸들러를 추가합니다.

그림 7-80 채널 ID 및 메시지 이벤트 핸들러

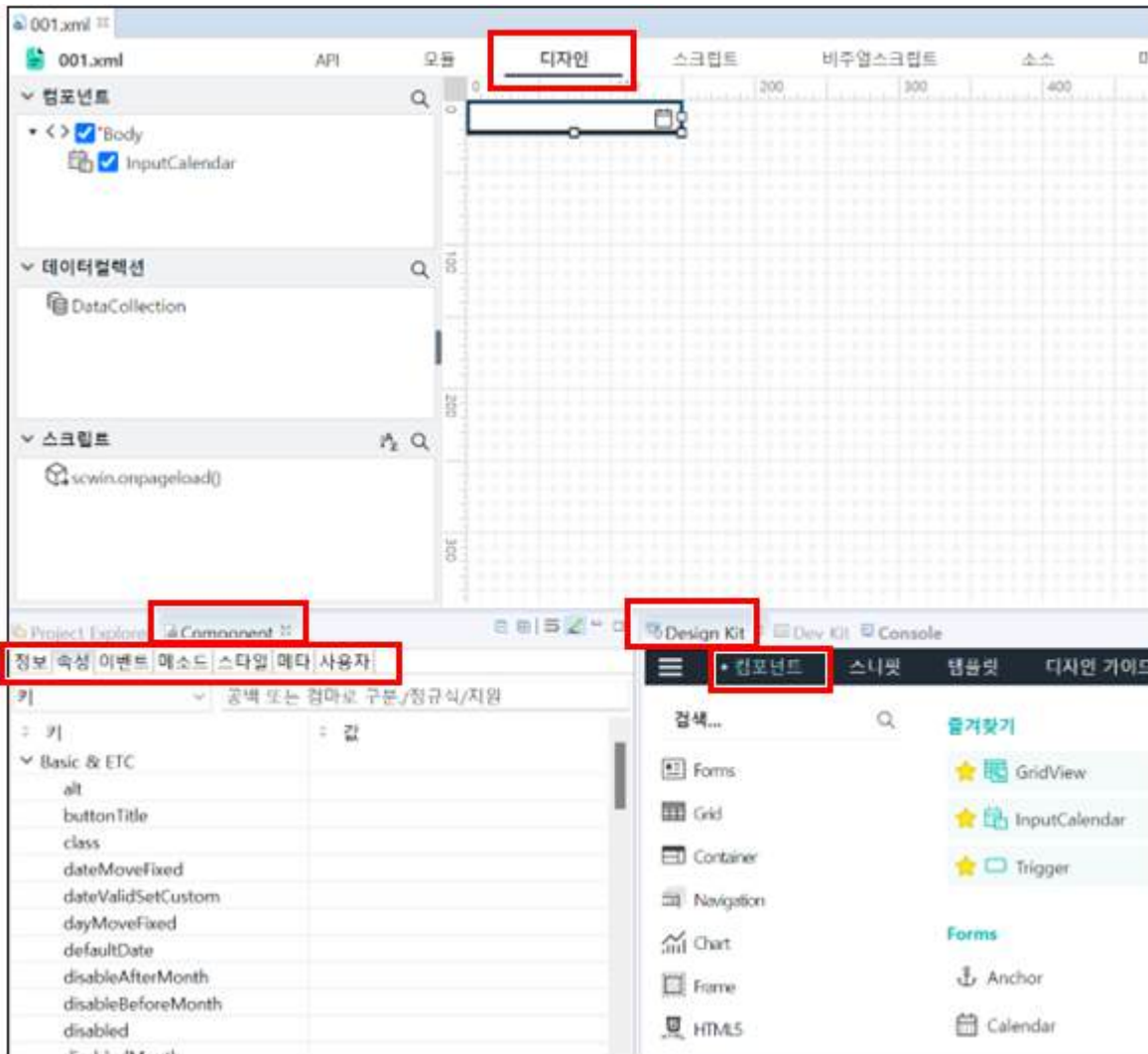


7.4.3. 디자인

WYSIWYG 방식의 컴포넌트 디자인 공간으로 **Design Kit**의 **컴포넌트** 탭에서 선택한 컴포넌트를 그릴 수 있습니다. 일반적으로 컴포넌트를 디자인하는 순서는 아래와 같습니다.

1. **Design Kit**의 **컴포넌트** 탭에서 컴포넌트를 선택합니다.
2. **디자인** 탭에서 원하는 위치에 선택한 컴포넌트를 그립니다.
3. 해당 컴포넌트의 속성 및 스타일을 **Component** 뷰에서 편집합니다.
4. 디자인 에디터의 **스크립트** 탭에서 컴포넌트의 메소드를 사용하여 원하는 스크립트를 작성합니다.

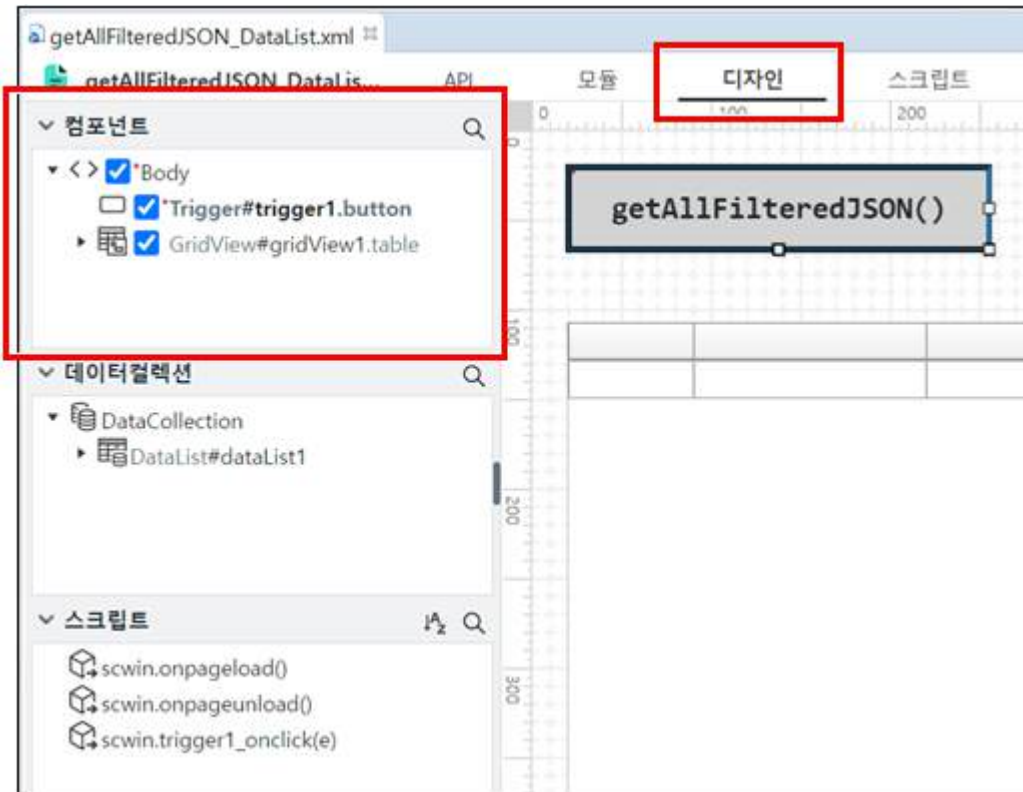
그림 7-81 디자인 탭에서 컴포넌트 추가



7.4.3.1. 컴포넌트

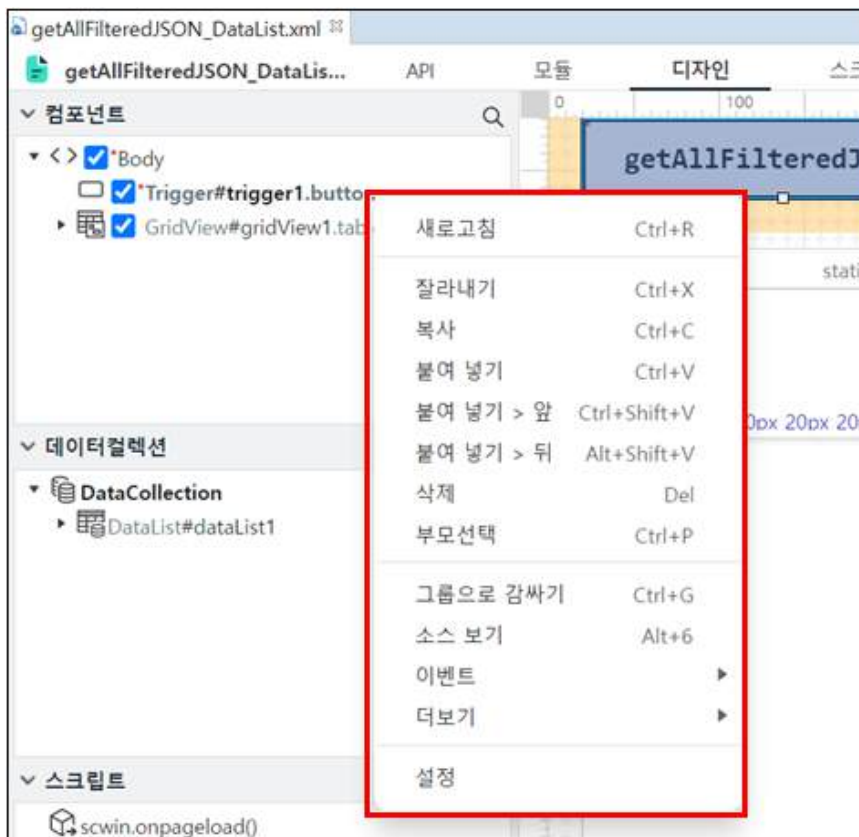
우측 컴포넌트 박스는 현재 화면 구성을 트리 형식으로 표시하고, 각 컴포넌트는 **컴포넌트명#ID.클래스명** 형식으로 표시됩니다. 각 컴포넌트 앞의 체크박스 선택을 해제하면 컴포넌트가 디자인 에디터에 표시되지 않습니다.

그림 7-82 디자인 - 컴포넌트



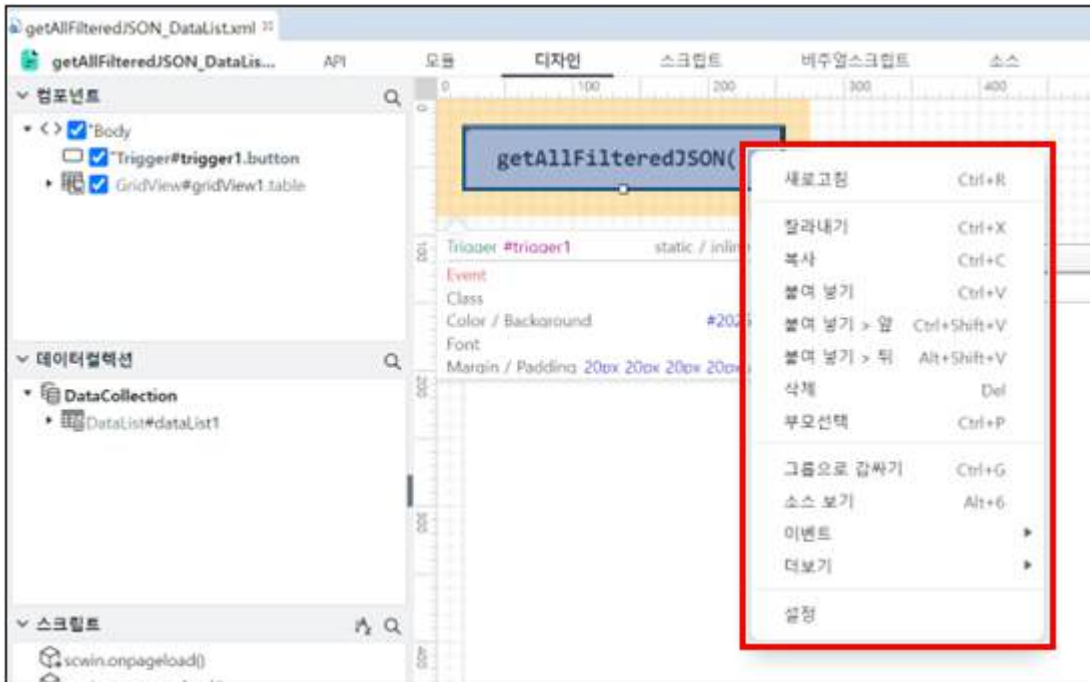
컴포넌트탭에서 각 컴포넌트를 우클릭하면 컴포넌트의 컨텍스트 메뉴가 표시됩니다.

그림 7-83 컨텍스트 메뉴 (디자인 - 컴포넌트 탭)



디자인 탭에서도 각 컴포넌트를 우클릭하면 동일한 컨텍스트 메뉴가 표시됩니다.

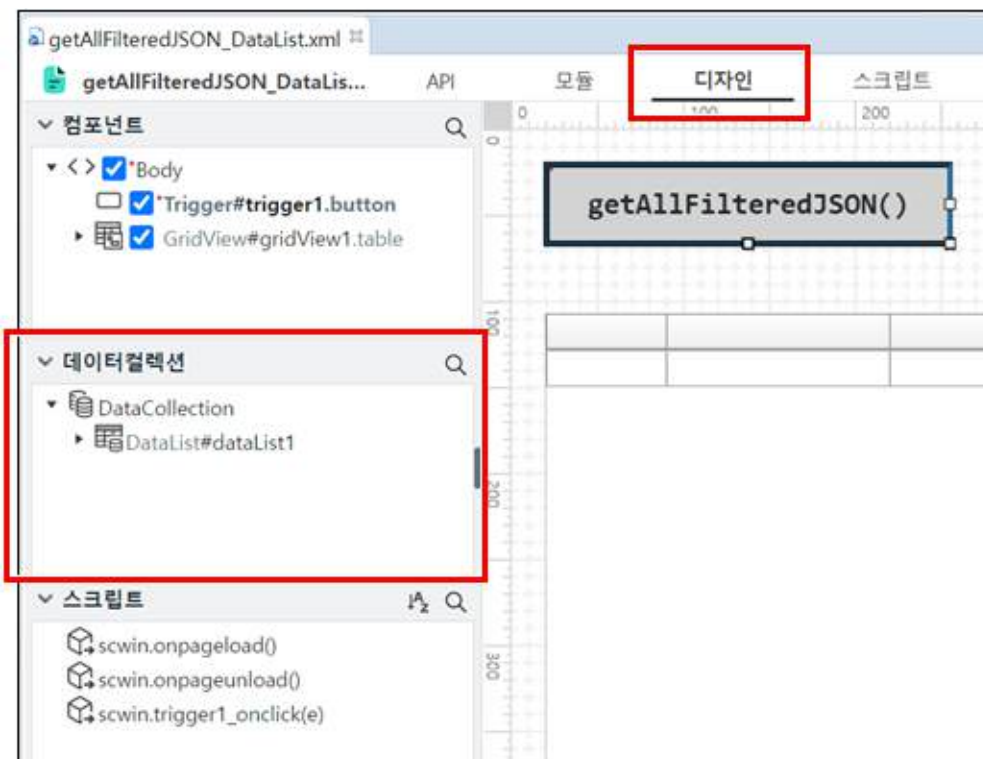
그림 7-84 컨텍스트 메뉴 (디자인 탭)



7.4.3.2. 데이터컬렉션

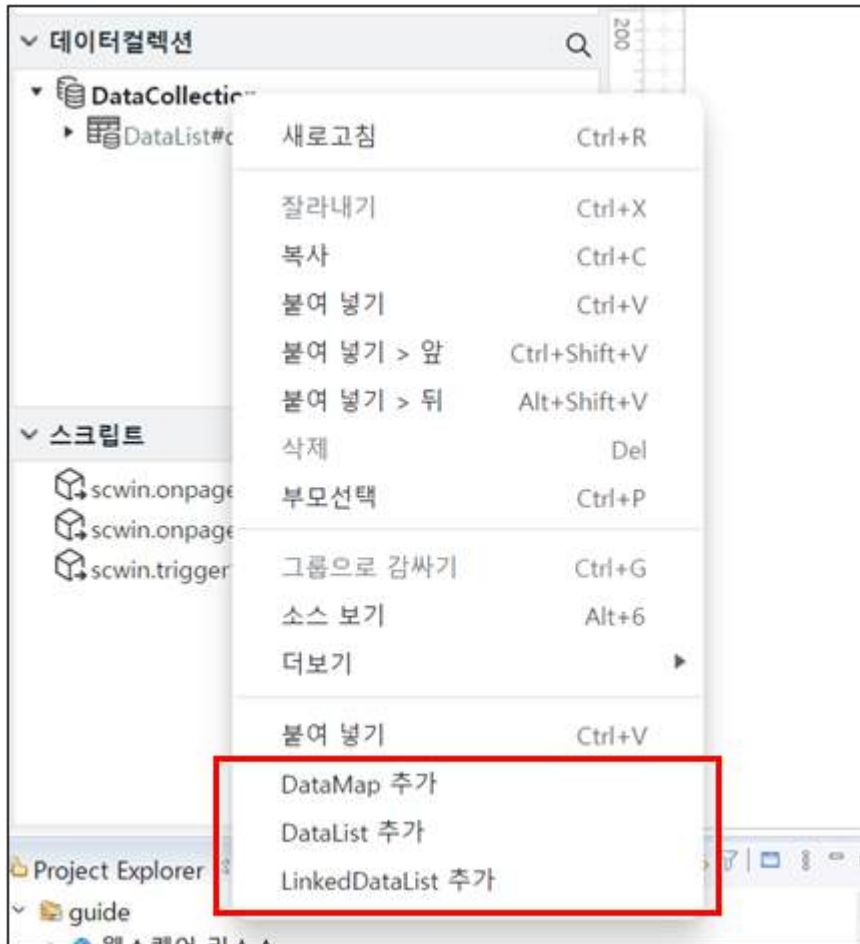
데이터컬렉션 박스는 현재 화면의 데이터컬렉션 객체를 표시합니다.

그림 7-85 디자인 - 데이터컬렉션



데이터컬렉션탭에서 DataCollection 항목을 우클릭하여 신규 DataCollection을 추가할 수 있습니다.

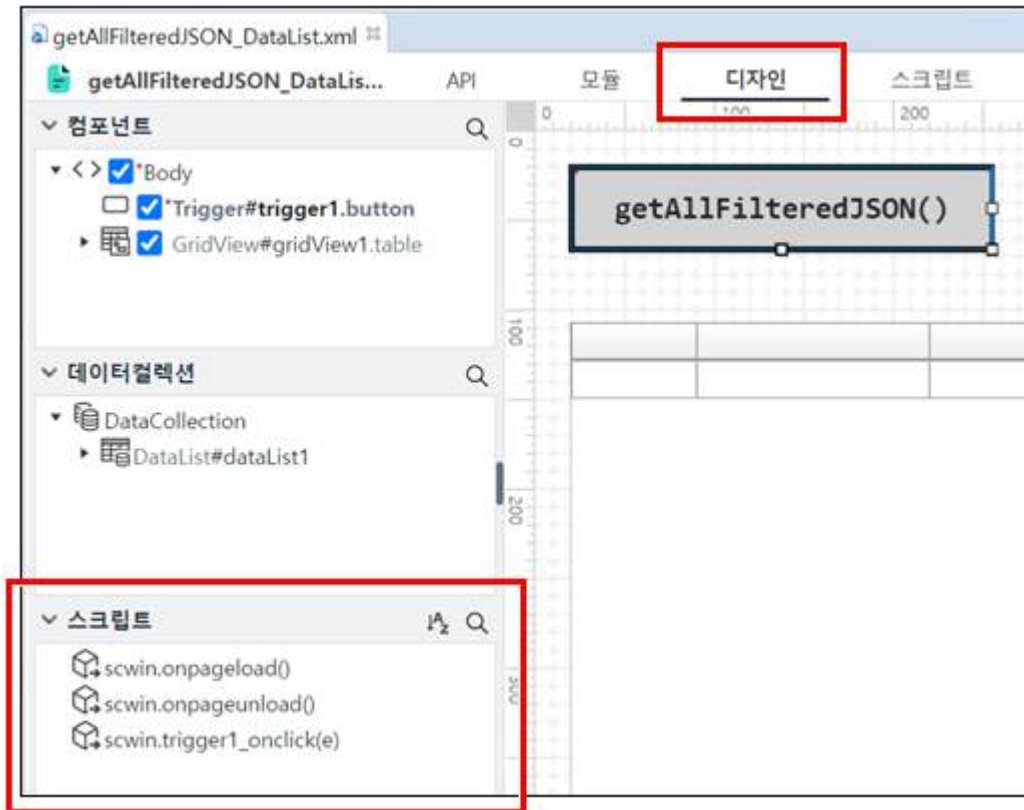
그림 7-86 데이터컬렉션 추가



7.4.3.3. 스크립트

디자인 - 스크립트 박스는 현재 페이지에 포함된 스크립트 목록을 표시합니다. 각 스크립트를 클릭하면 스크립트 탭의 해당 부분으로 이동합니다.

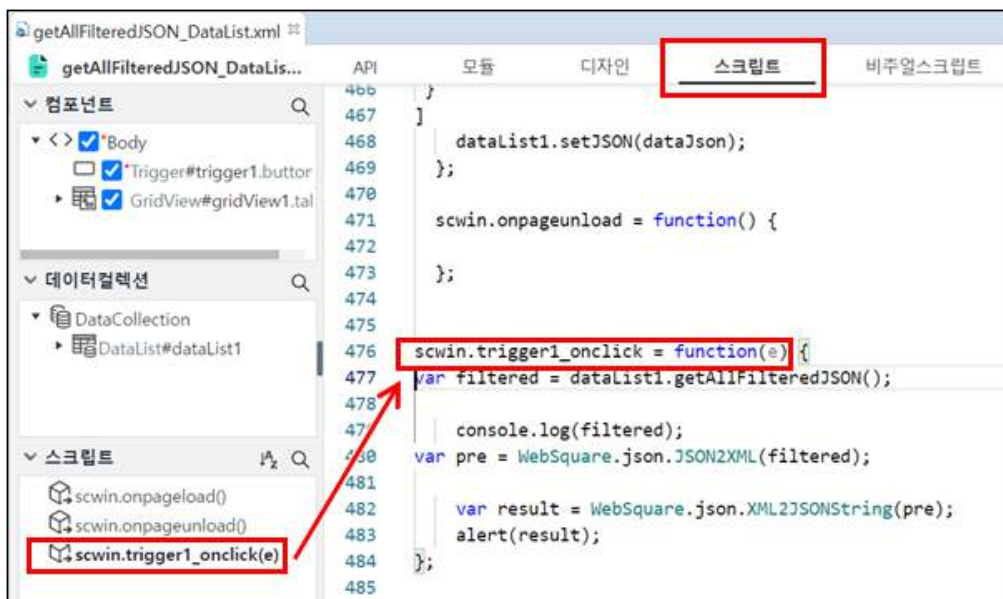
그림 7-87 디자인 - 스크립트



7.4.4. 스크립트

디자인 탭은 현재 화면에 포함된 스크립트를 표시합니다. 좌측의 스크립트 박스에서 스크립트를 클릭하면 해당 부분으로 바로 이동할 수 있습니다.

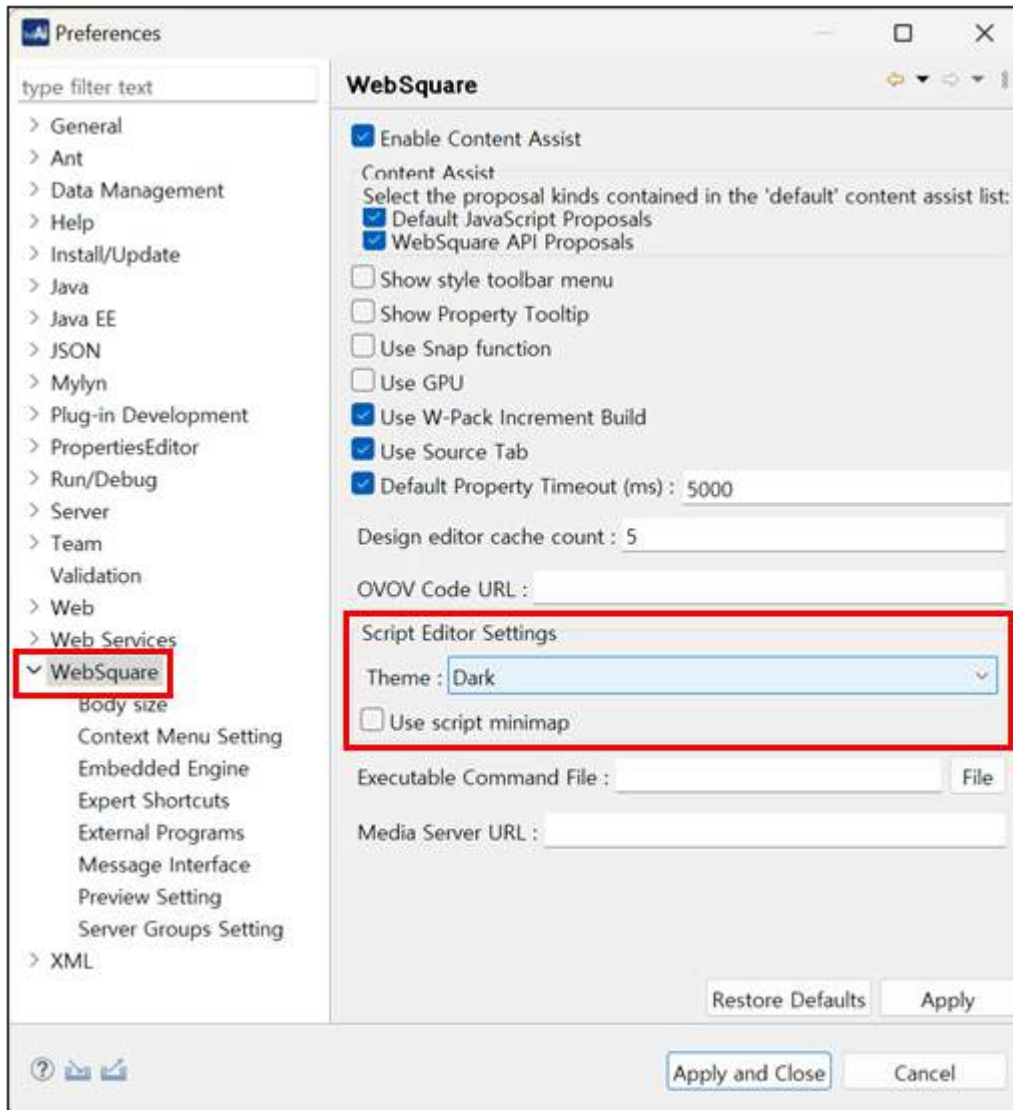
그림 7-88 스크립트



7.4.4.1. 테마 설정

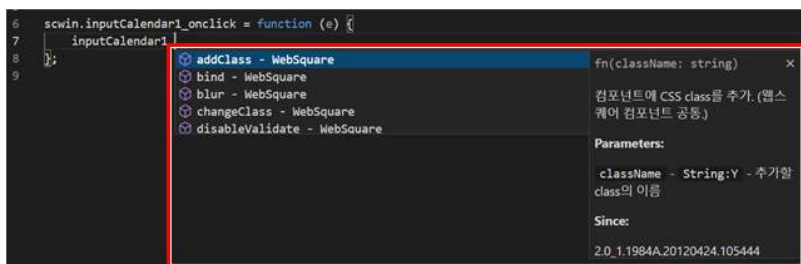
Windows - Preferences 메뉴를 선택한 후 WebSquare 항목의 Script Editing Settings 옵션을 설정하여 스크립트 및 비주얼스크립트 탭의 스타일을 설정할 수 있습니다.

그림 7-89 스크립트 및 비주얼스크립트 탭 스타일 설정



7.4.4.2. 자동 완성

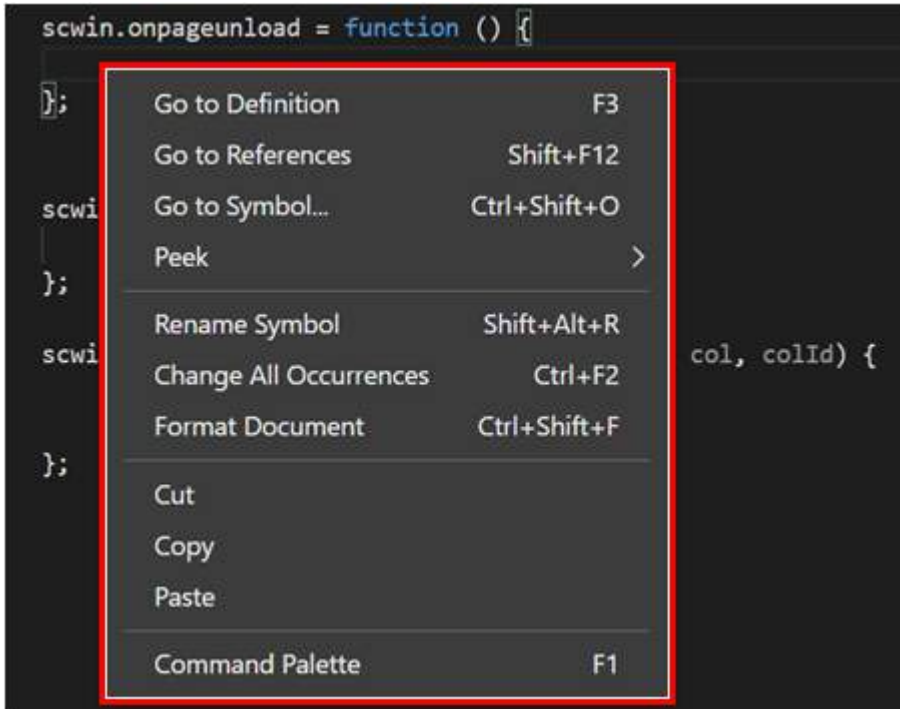
Script tab에서 Script 작성 시, Ctrl + Space를 누르면 아래와 같이 자동 완성 기능이 제공됩니다.



7.4.4.3. 컨텍스트 메뉴

아래와 같은 컨텍스트 메뉴를 사용하여 찾기, 복사, 포매팅 등의 기능을 사용할 수 있습니다.

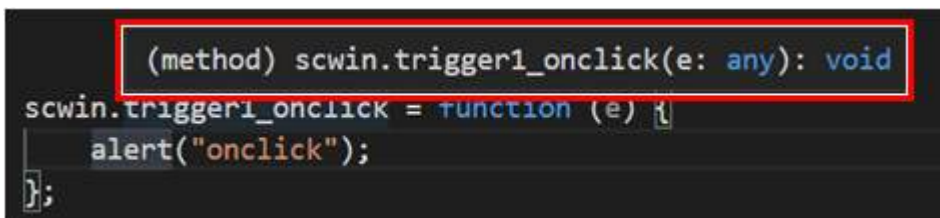
그림 7-90 컨텍스트 메뉴



7.4.4.4. 툴팁

마우스 오버 시 아래와 같이 툴팁을 제공합니다.

그림 7-91 툴팁



7.4.5. 비주얼스크립트

작업 화면 상단의 비주얼스크립트 탭을 클릭할 경우, 디자인과 스크립트를 동시에 표시합니다. 개발자는 화면의 컴포넌트를 확인하면서 해당 컴포넌트 관련 스크립트를 작성할 수 있습니다.

그림 7-92 비주얼스크립트 탭



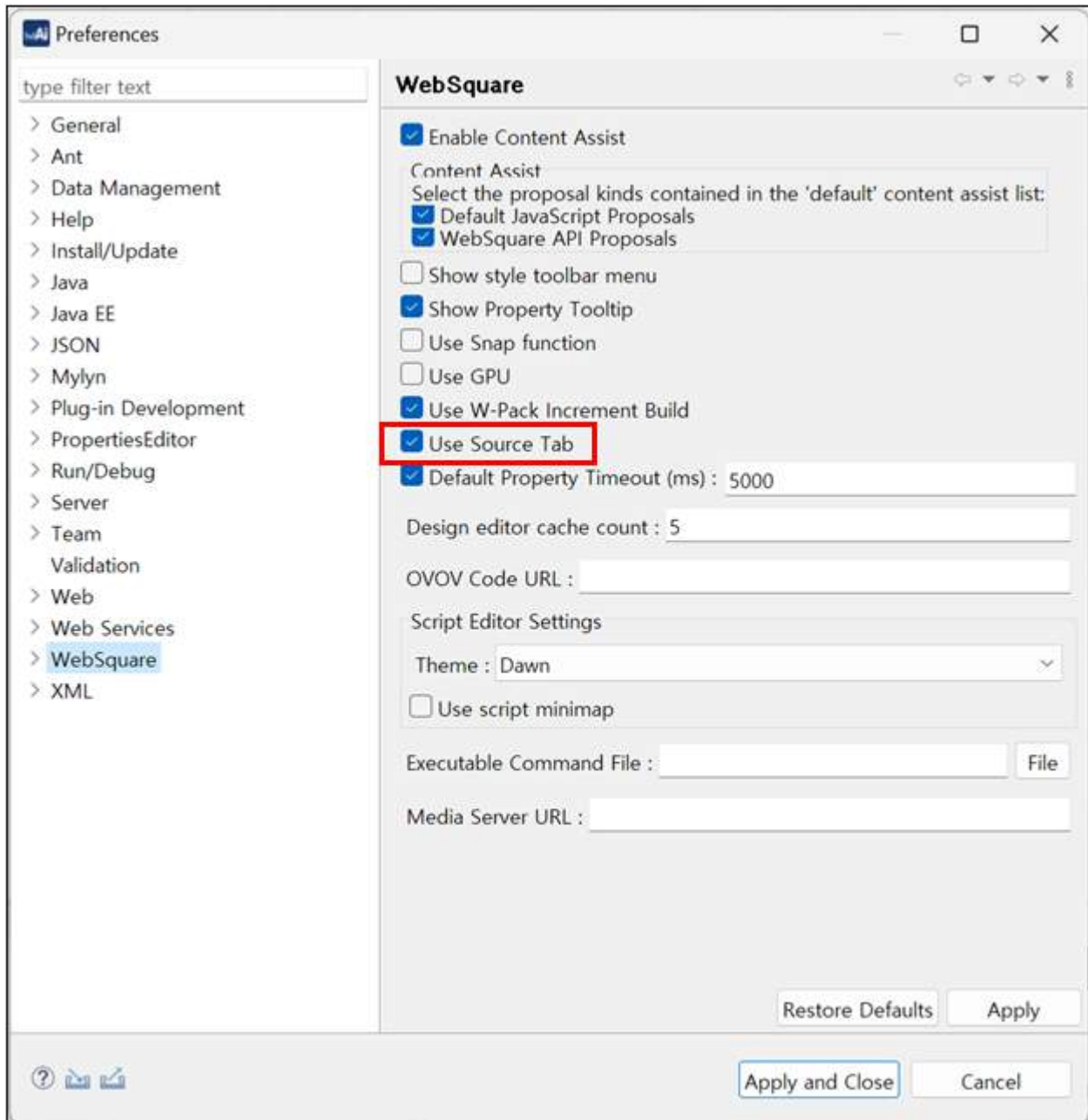
7.4.6. 소스

그림 7-93 소스



웹스퀘어 AI 스튜디오는 기본 상태에서 Source 탭을 표시하지 않습니다. Source 탭을 표시하려면, **Window - Preferences - WebSquare**를 선택하고 **Use Source Tab** 항목을 체크하십시오.

그림 7-94 소스 탭 표시



Eclipse의 Search 기능

Eclipse의 Search 기능을 사용하여 특정 텍스트를 검색할 경우, 웹스퀘어 파일이 검색결과로 표시됩니다. 해당 웹스퀘어 파일이 열려 있고 Source 탭이 표시된 경우, 검색 결과를 클릭하면 Source 탭의 해당 텍스트로 이동합니다. Source 탭이 닫혀있는 경우에는, 해당 페이지로 포커스가 이동합니다.

7.4.7. 미리보기

현재 작업 중인 페이지를 렌더링한 모습을 확인할 수 있습니다.

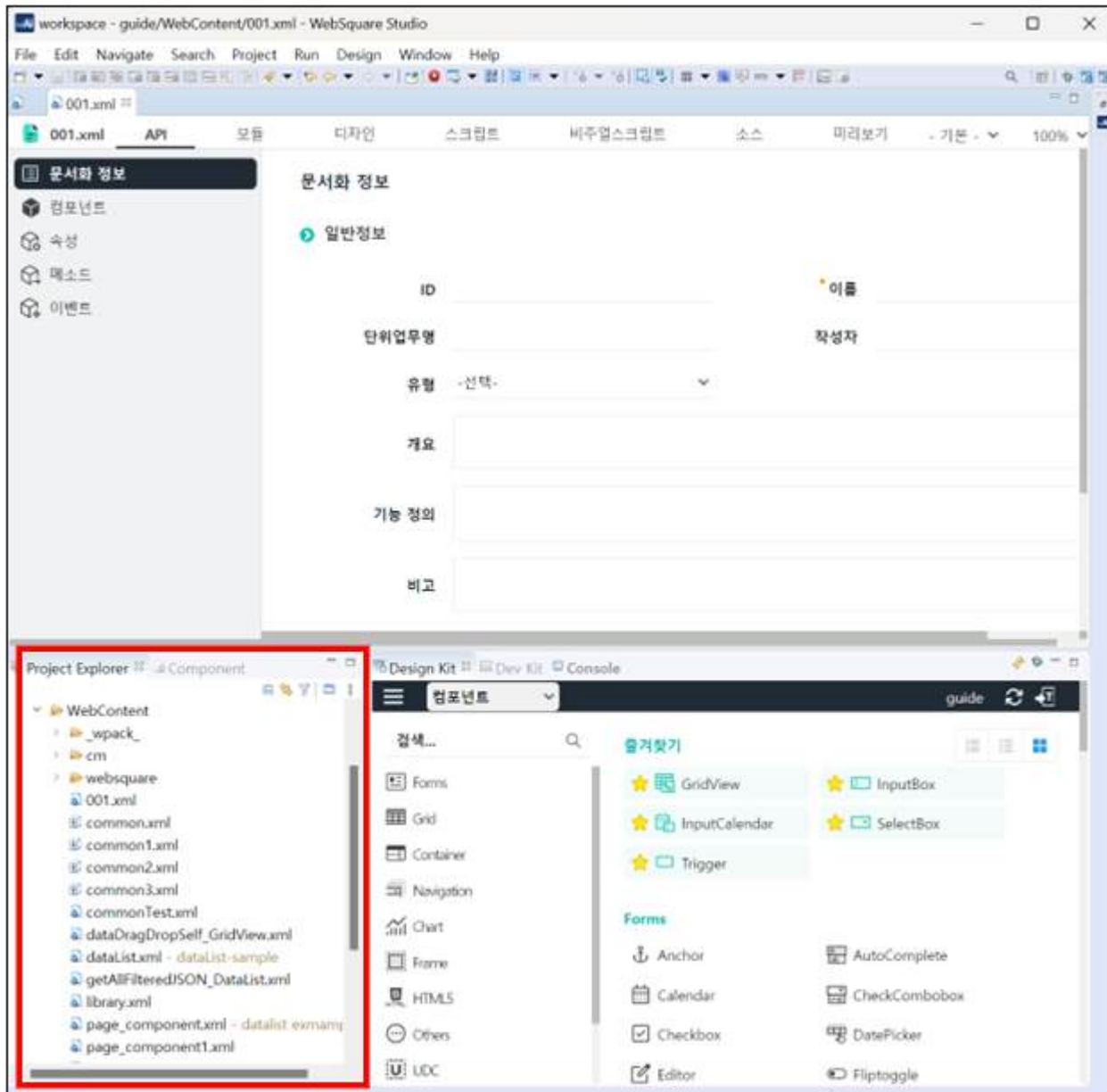
그림 7-95 미리보기



7.5 Project Explorer

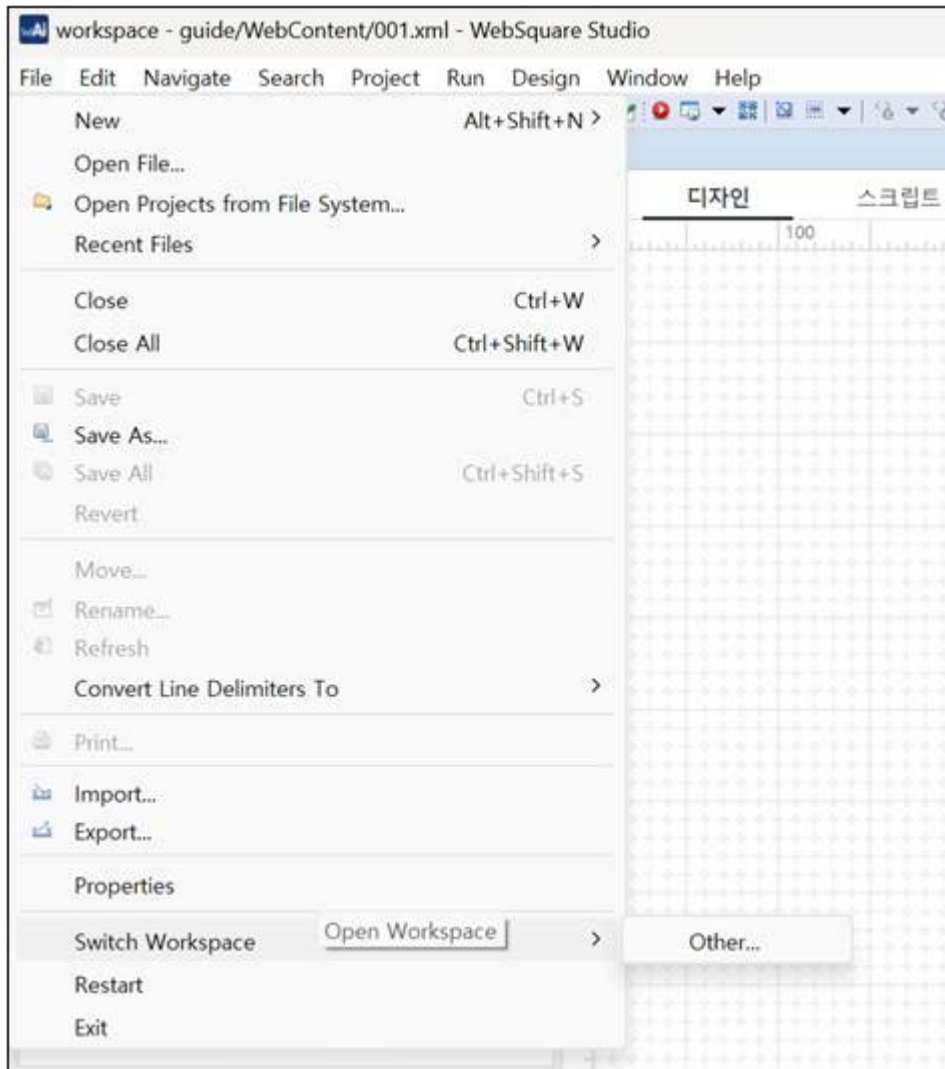
Project Explorer는 스튜디오의 좌측 하단에 위치해 있으면 현재 workspace에 있는 프로젝트와 파일을 표시합니다.

그림 7-96 Project Explorer



File - Switch Workspace 메뉴를 선택하여 Workspace를 변경할 수 있습니다.

그림 7-97 File - Switch Workspace



7.6 Component

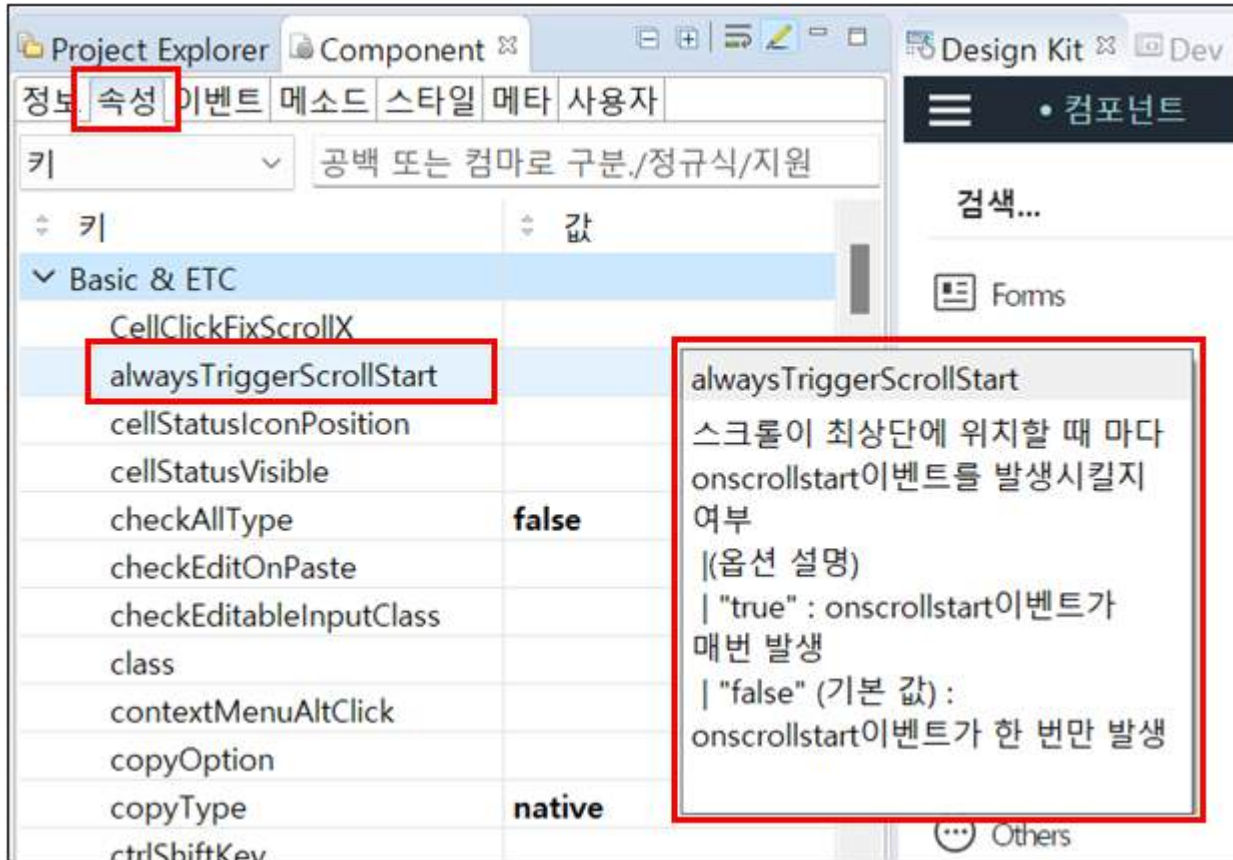
7.6.1. 속성

디자인 에디터의 **Design** 탭에서 컴포넌트를 선택하면 해당 컴포넌트의 속성이 **Component** 뷰의 속성 탭에 표시됩니다. 속성 탭에서 각 속성을 선택하면, 해당 속성에 대한 정보가 툴팁으로 표시됩니다.

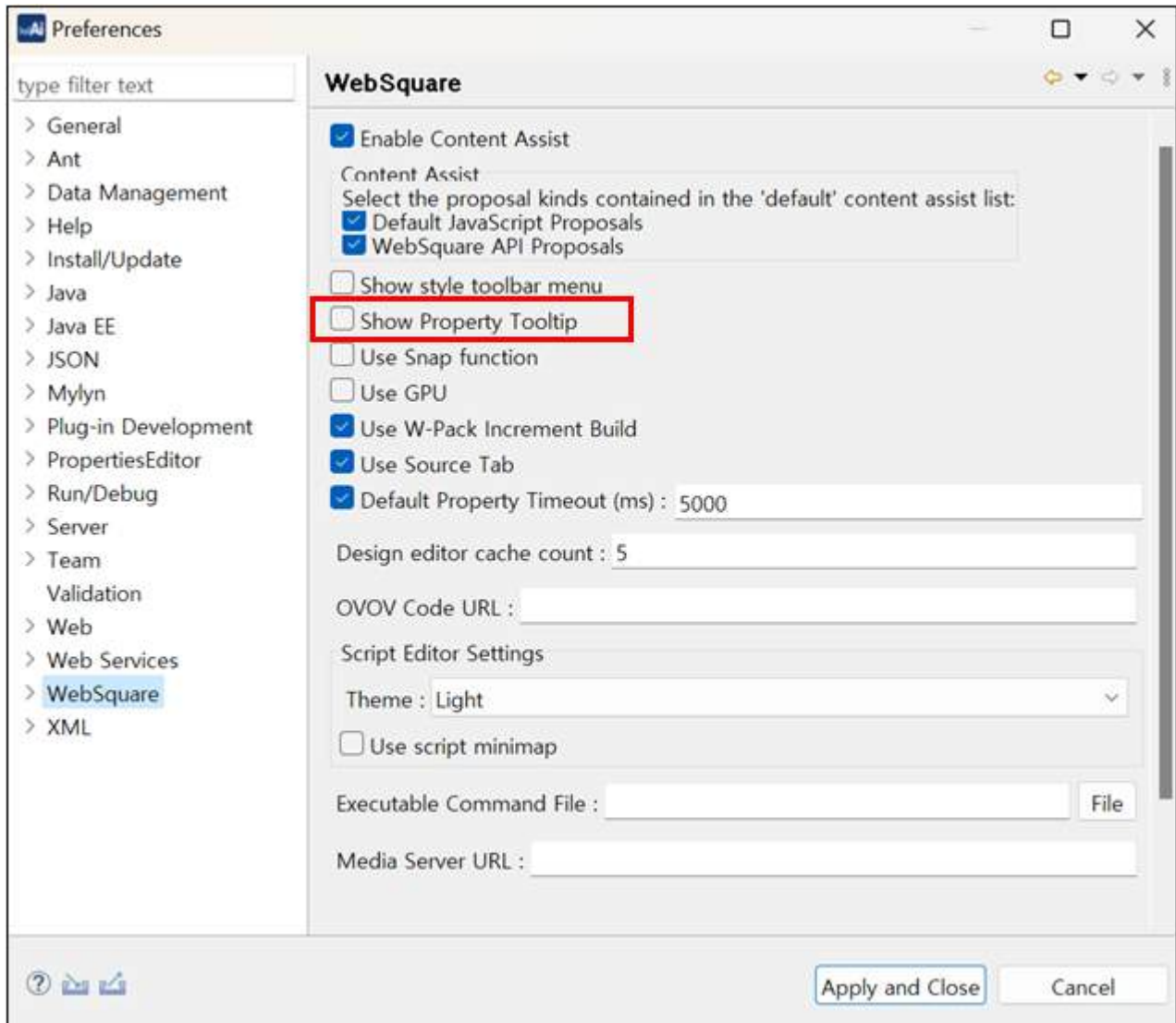
7.6.1.1. 속성 설명 툴팁

Component 뷰의 속성 탭에서 각 속성에 마우스 커서를 올리면 아래와 같이 좌측에 속성에 대한 설명이 툴팁으로 표시됩니다.

그림 7-98 툴팁



Window - Preferences - WebSquare 선택 후 Show Property Tooltip 항목을 선택하거나 선택 해제하여 속성 설명 툴팁을 표시하거나 숨길 수 있습니다.



7.6.1.2. 카테고리 별 분류

속성은 카테고리 별로 분류되어 표시됩니다.

그림 7-99 카테고리를 펼친 모습

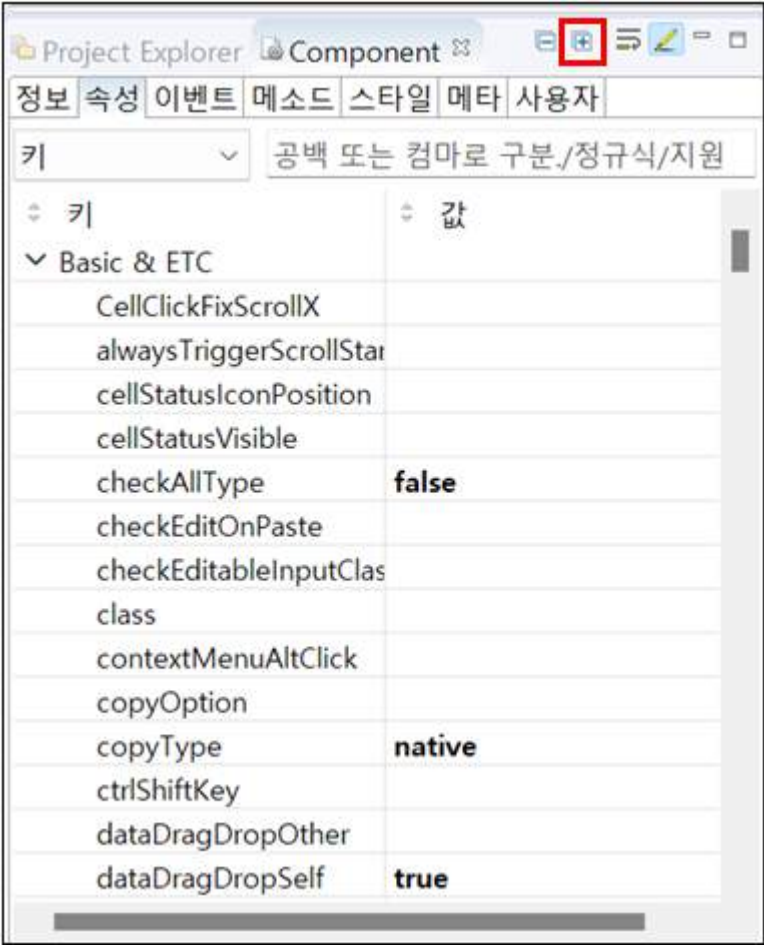
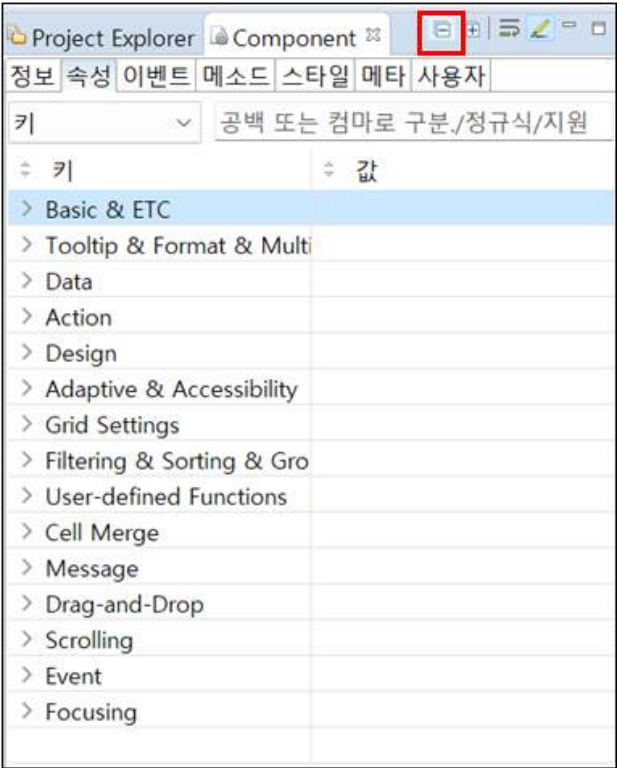


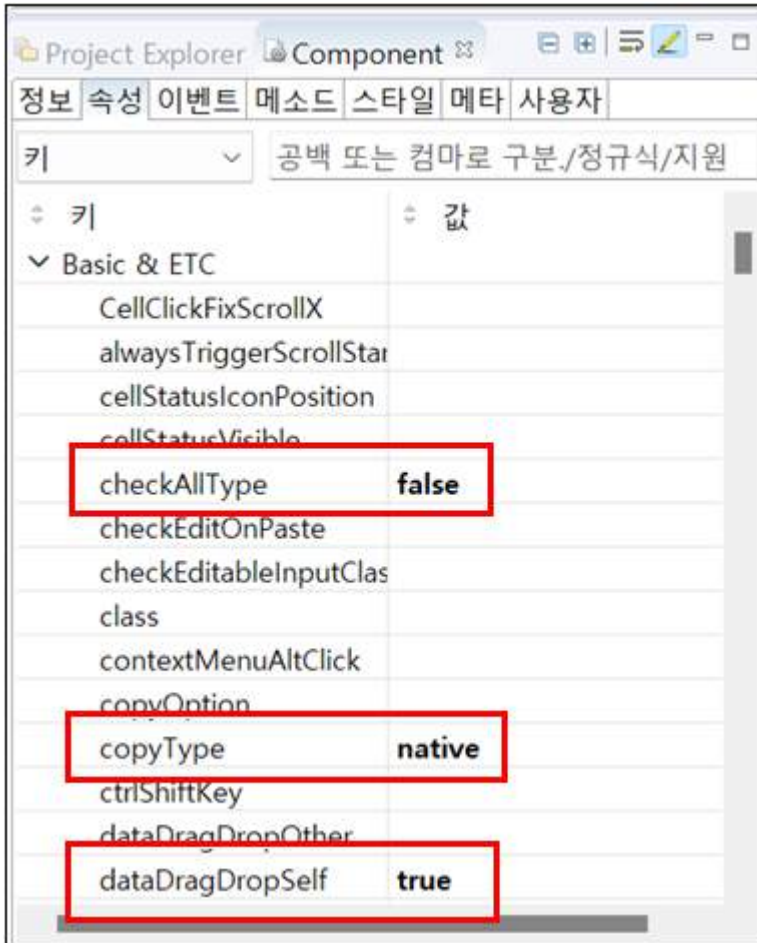
그림 7-100 카테고리를 닫은 모습



7.6.1.3. 속성 정렬

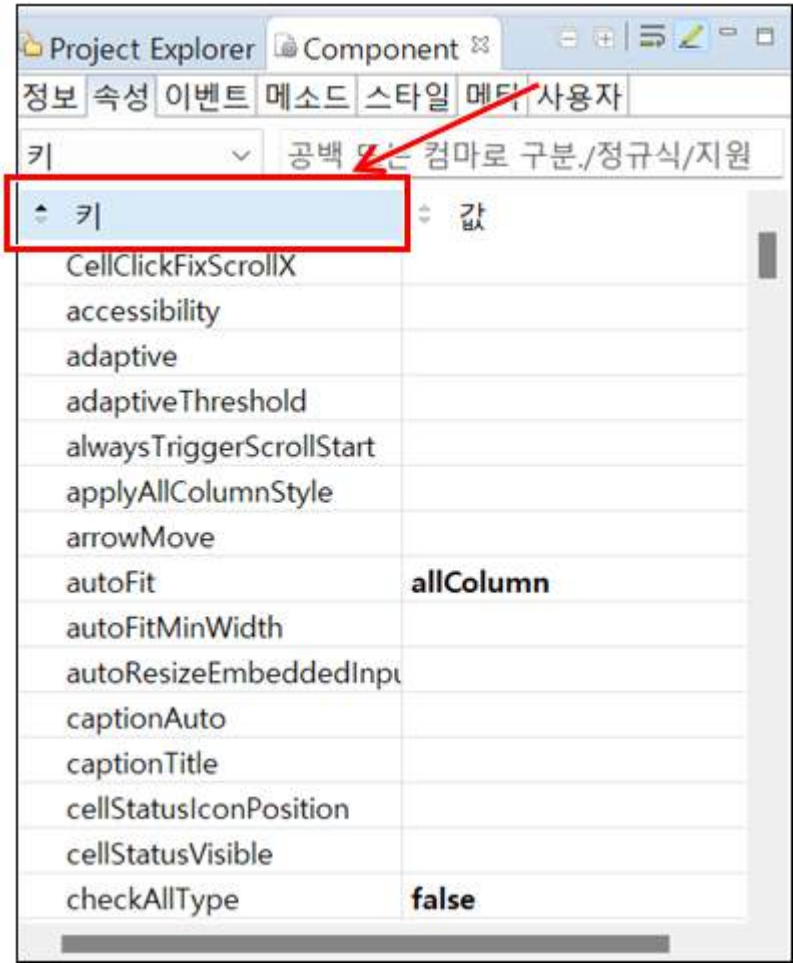
기본값은 회색으로, 사용자가 직접 입력한 값은 검정색으로 굵게 표시됩니다.

그림 7-101 속성 표시



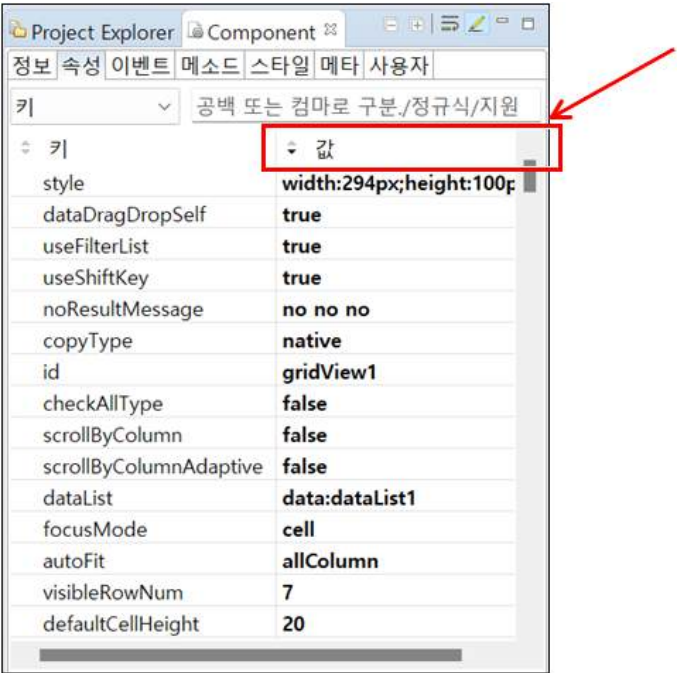
키를 클릭하면 오름차순 정렬 - 내림차순 정렬 - 카테고리 표시 순으로 속성을 표시할 수 있습니다.

그림 7-102 Key를 클릭하여 속성 정렬



값을 클릭하면 사용자가 정의한 값이 상단에 정렬됩니다.

그림 7-103 값을 클릭하여 사용자 정의 값만 정렬



7.6.2. 이벤트

디자인 데이터의 디자인 탭에서 컴포넌트를 선택한 후 Component 뷰의 이벤트 탭을 클릭하면 해당 컴포넌트가 지원하는 이벤트 목록이 표시됩니다. 각 이벤트의 script 버튼을 클릭하며, 해당 이벤트의 코드가 디자인 데이터의 스크립트 탭에 자동 생성됩니다.

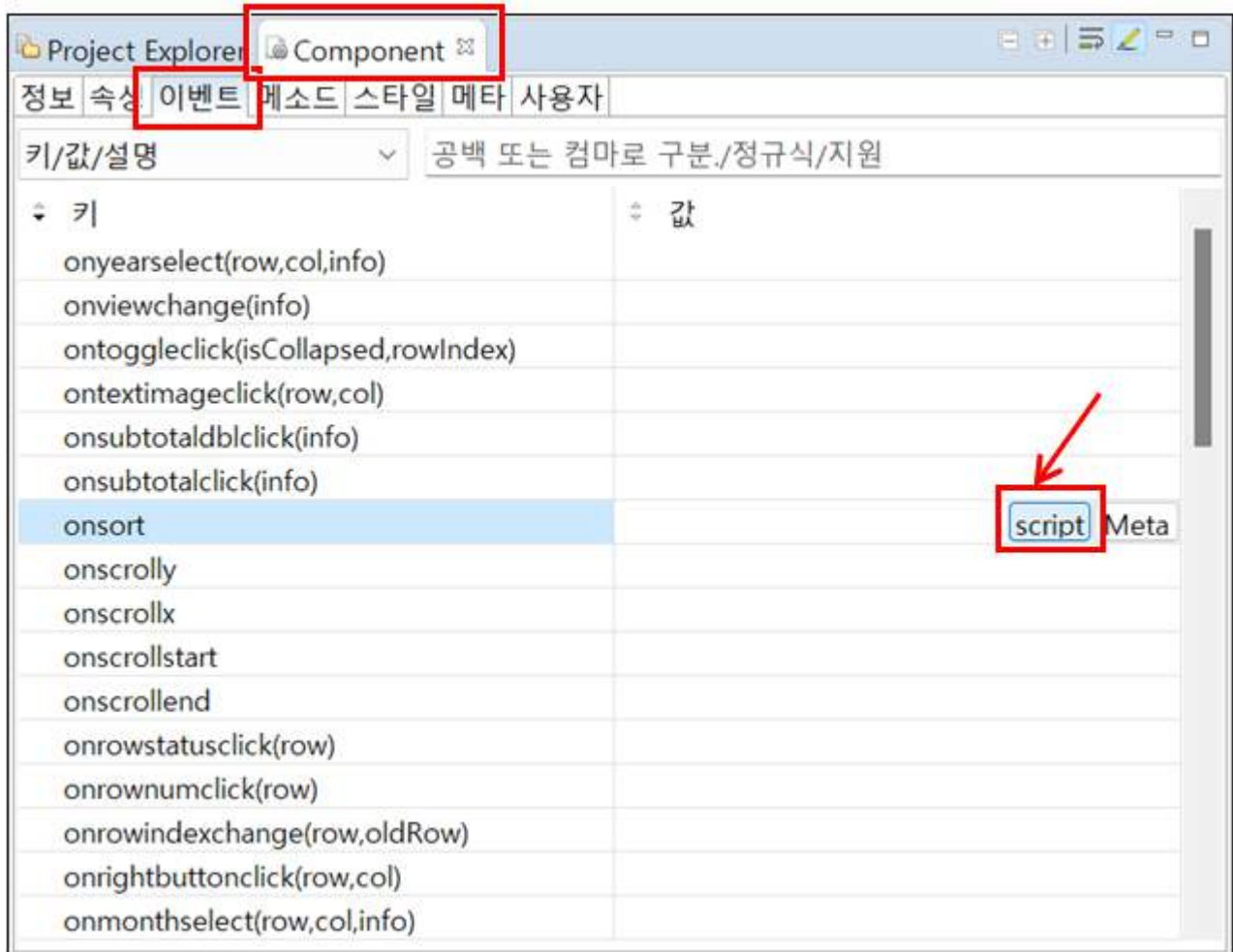


그림 7-104 Script 버튼 클릭 후 자동 생성된 이벤트 함수

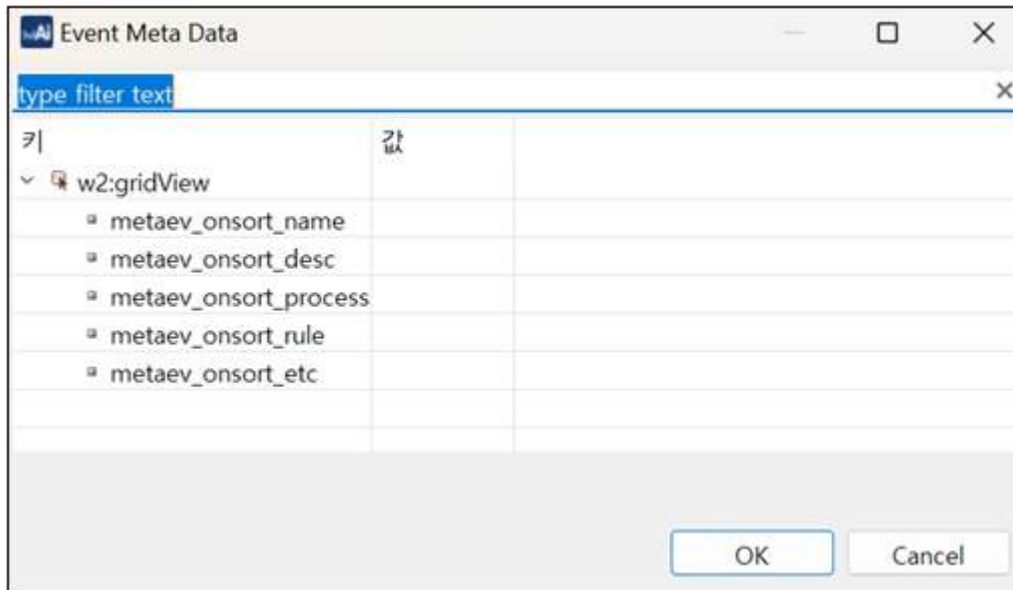
```

45
46  scwin.gridView1_onsort = function () {
47  |
48  };
49

```

script 옆의 Meta 버튼을 클릭할 경우, 해당 이벤트의 메타 정보(이름, 설명, 규칙 등)를 입력할 수 있는 창이 표시됩니다. 이벤트의 메타 정보는 W-Docs를 사용하여 화면 설계서를 생성할 Excel 파일에 포함됩니다.

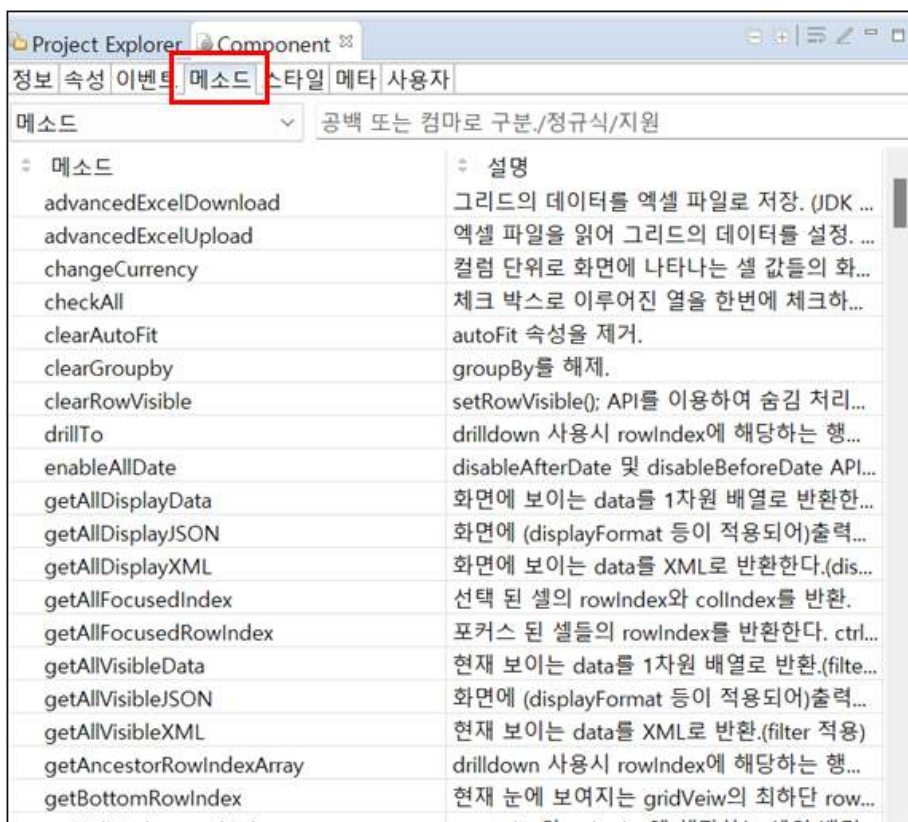
그림 7-105 Meta 버튼을 클릭할 경우



7.6.3. 메소드

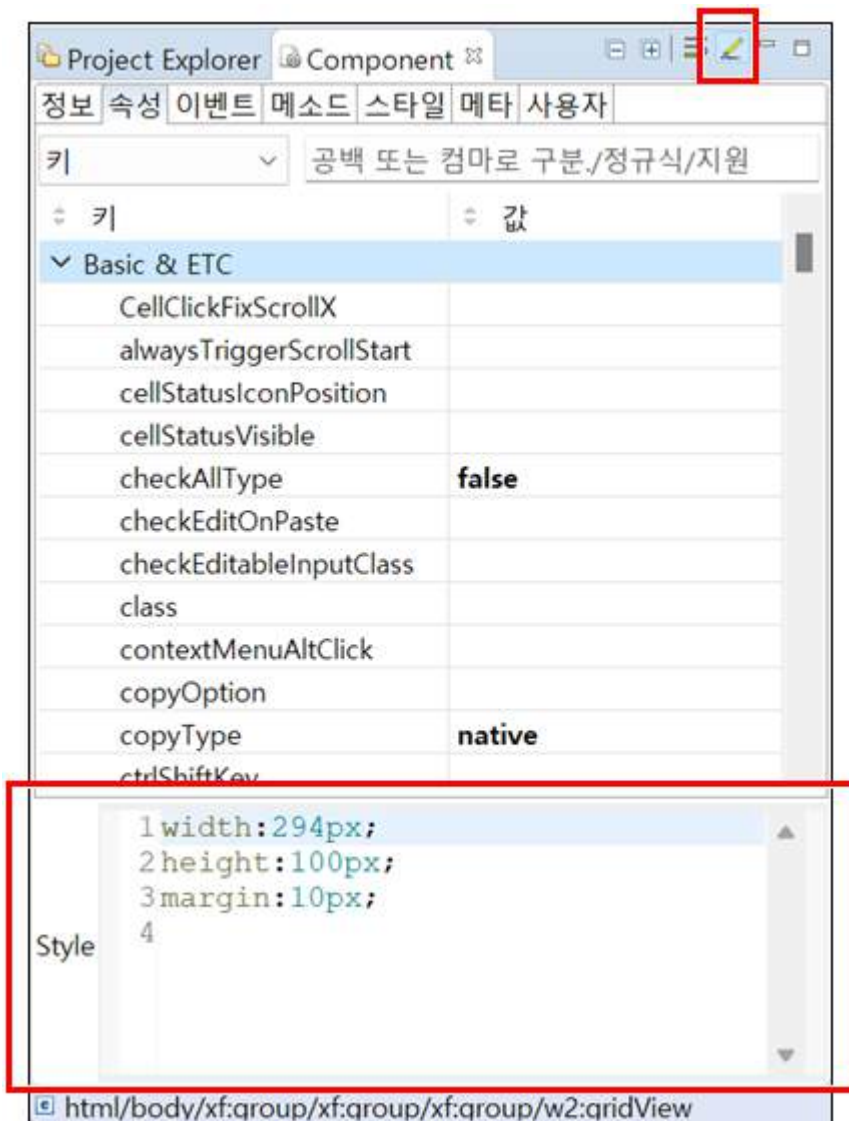
디자인 탭에서 컴포넌트를 선택한 후 Component 뷰의 메소드 탭을 클릭하면 해당 컴포넌트가 지원하는 메소드 목록이 표시됩니다.

그림 7-106 메소드 목록



7.6.4. 스타일

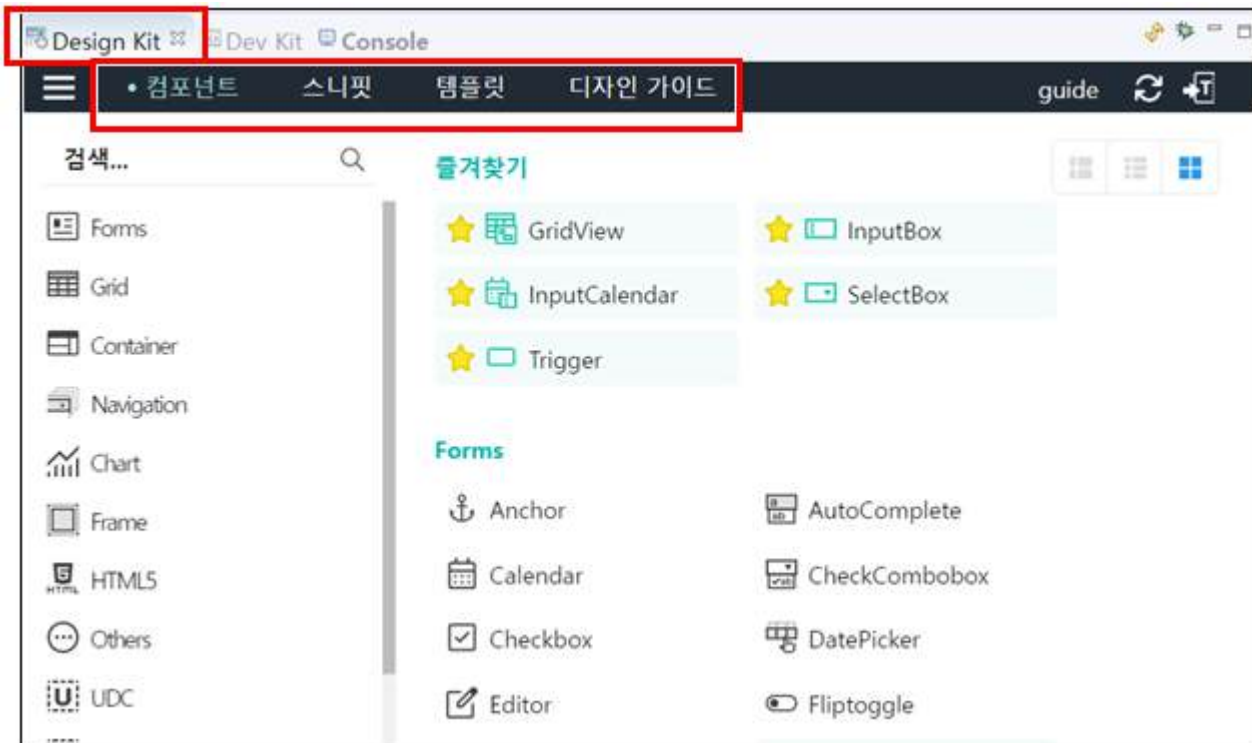
디자인 탭에서 선택한 컴포넌트의 인-라인 스타일이 아래와 같이 **Component - 속성** 탭 하단에 표시됩니다. 상단의 버튼을 클릭하면 스타일을 숨길 수 있습니다.



7.7 Design Kit

Design Kit는 디자인 관련 리소스, Template, 가이드 등을 제공하고, 프로젝트에 맞게 추가 구성이 가능합니다.

그림 7-107 Design Kit

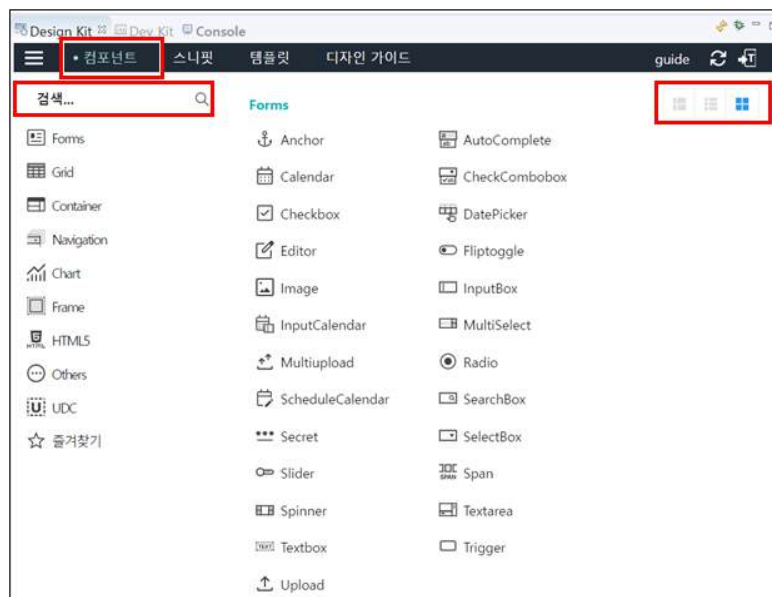


7.7.1. 컴포넌트

화면 개발에 사용할 수 있는 웹스퀘어 컴포넌트를 카테고리 별로 구분하여 표시합니다.

웹스퀘어 AI가 제공하는 컴포넌트와 UDC, TTC, Snippet을 한 눈에 확인할 수 있습니다. 검색창에 컴포넌트 이름을 입력하여 컴포넌트를 검색할 수 있습니다.

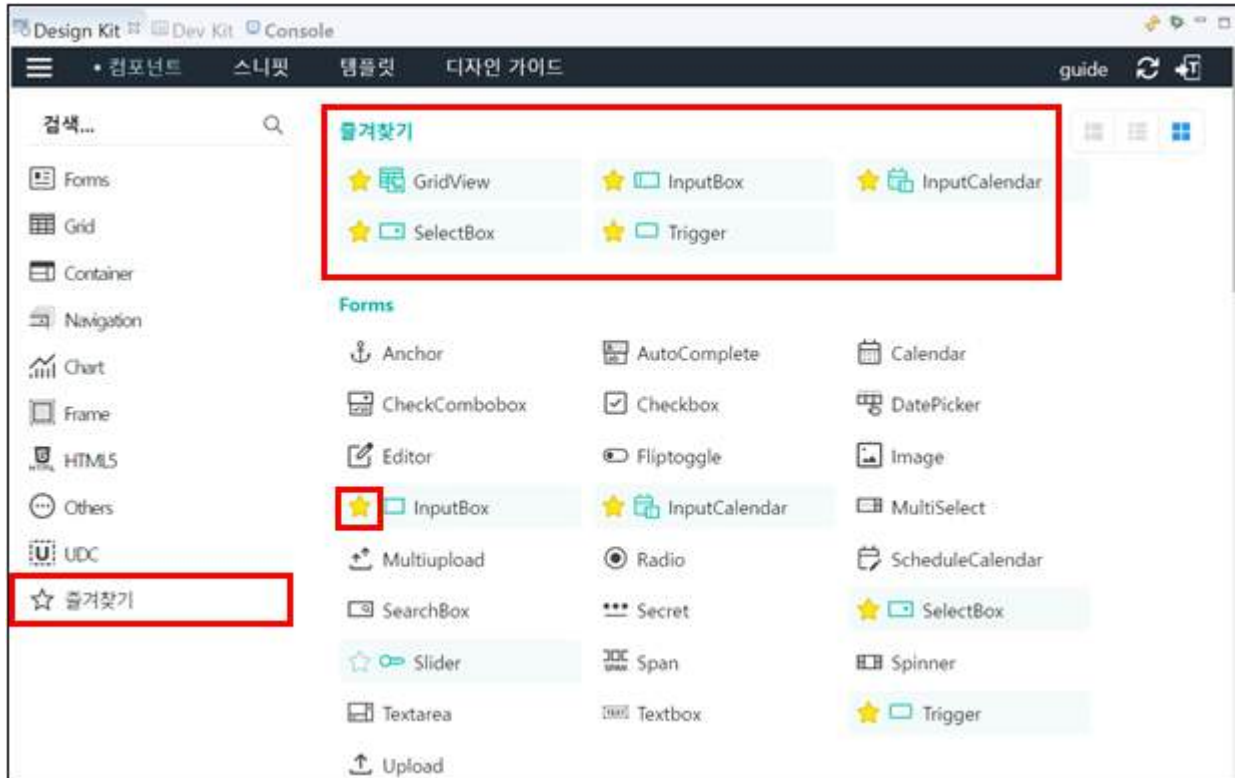
그림 7-108 Design Kit - 컴포넌트



7.7.1.1. 즐겨찾기

컴포넌트 이름 왼쪽의 별 모양 아이콘을 클릭하여 해당 컴포넌트를 즐겨찾기에 추가할 수 있습니다.

그림 7-109 즐겨찾기



7.7.1.2. 목록 새로 고침

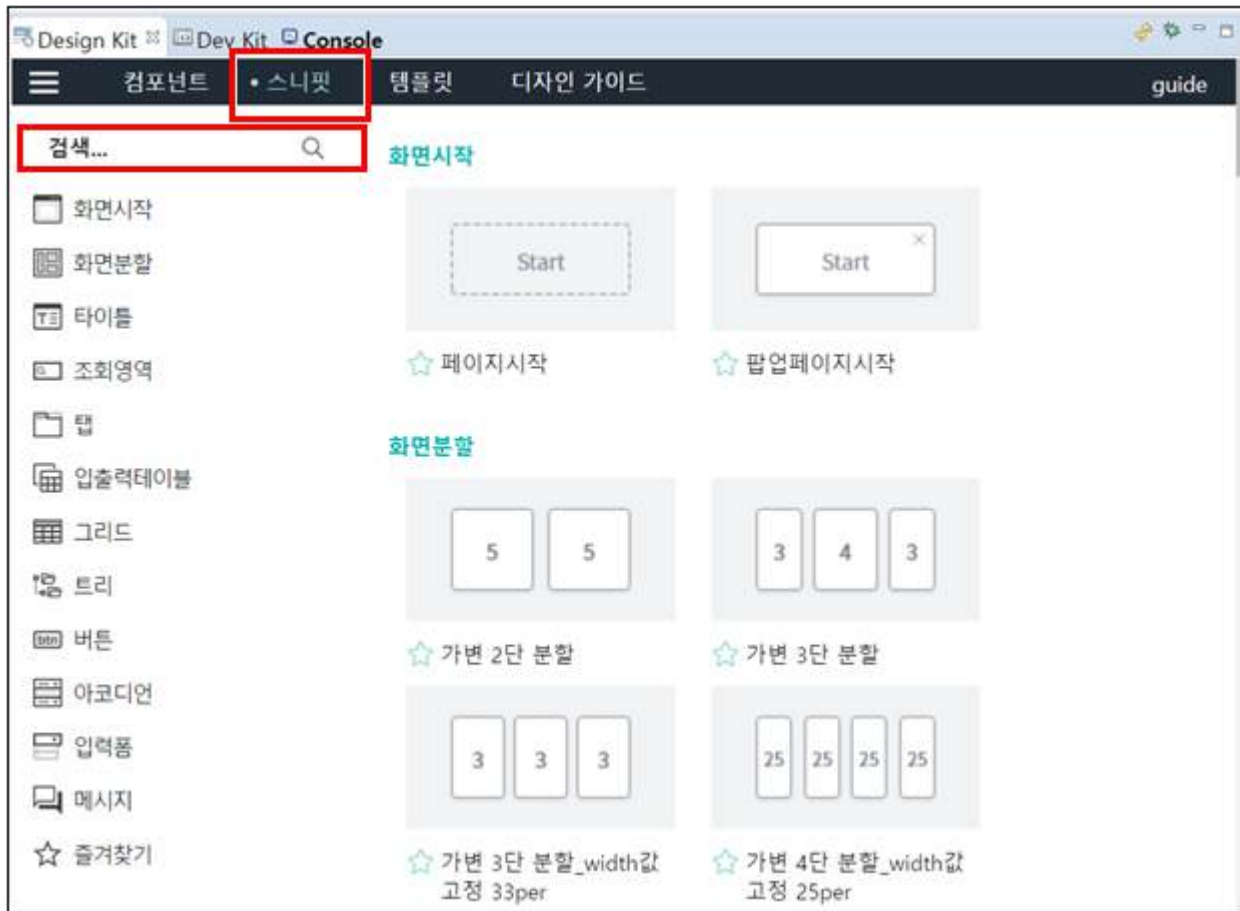
UDC나 TTC를 등록한 경우 **컴포넌트 새로 고침** 버튼을 클릭하여 컴포넌트 목록을 갱신할 수 있습니다.

그림 7-110 컴포넌트 새로 고침



7.7.2. 스니핏

등록된 스니핏을 카테고리별로 표시하며 검색기능 및 즐겨찾기 기능을 제공합니다.



Design Kit의 스니펫 탭은 `/WebContent/cm/template/snippets/` 폴더에 저장된 리소스를 썸네일 이미지로 제공합니다.

그림 7-111 `/WebContent/cm/template/snippets/` 폴더



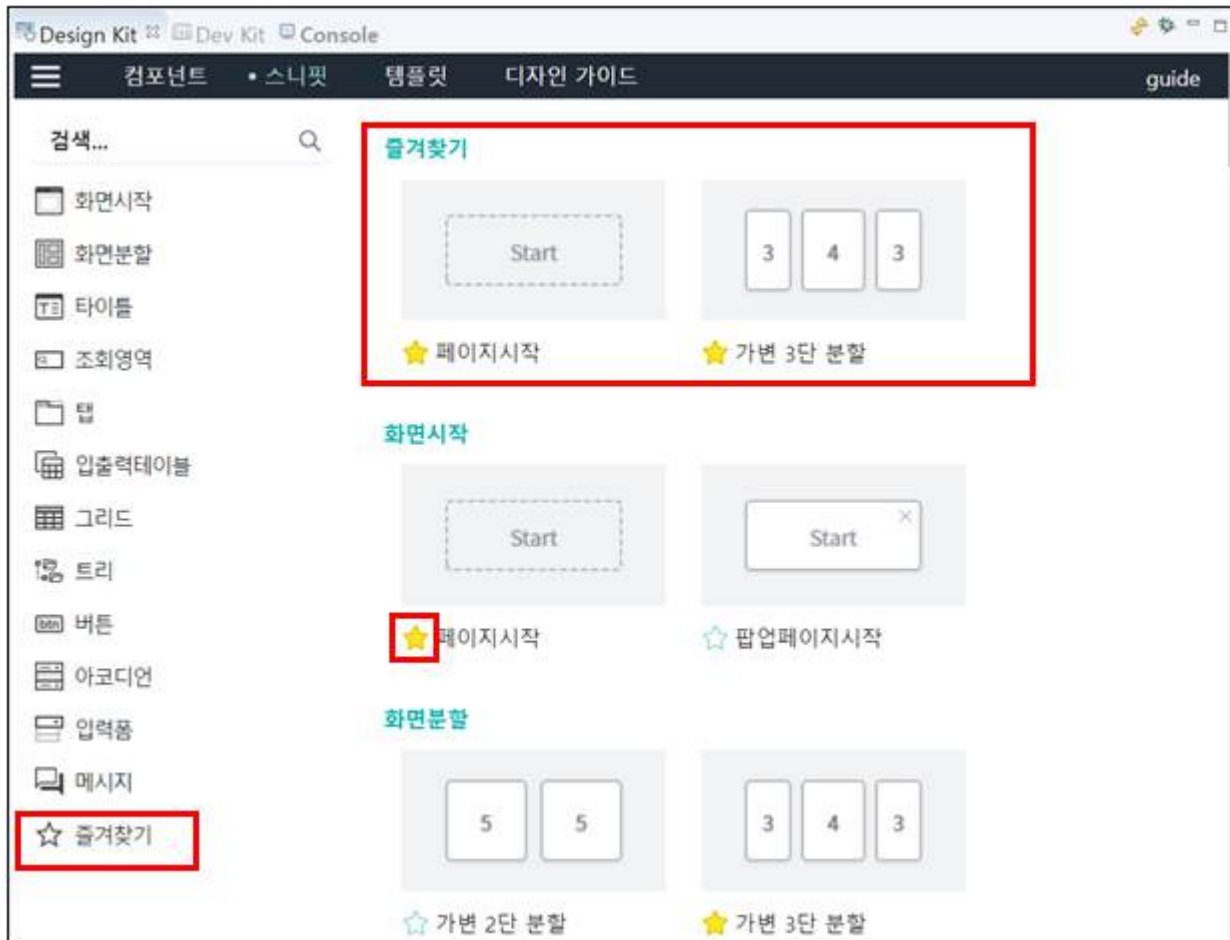
Design Kit의 스니핏 탭에서 원하는 항목을 클릭하여 해당 컴포넌트 디자인을 추가할 수 있습니다.

그림 7-112 Design Kit에서 스니핏을 화면 파일에 추가하기



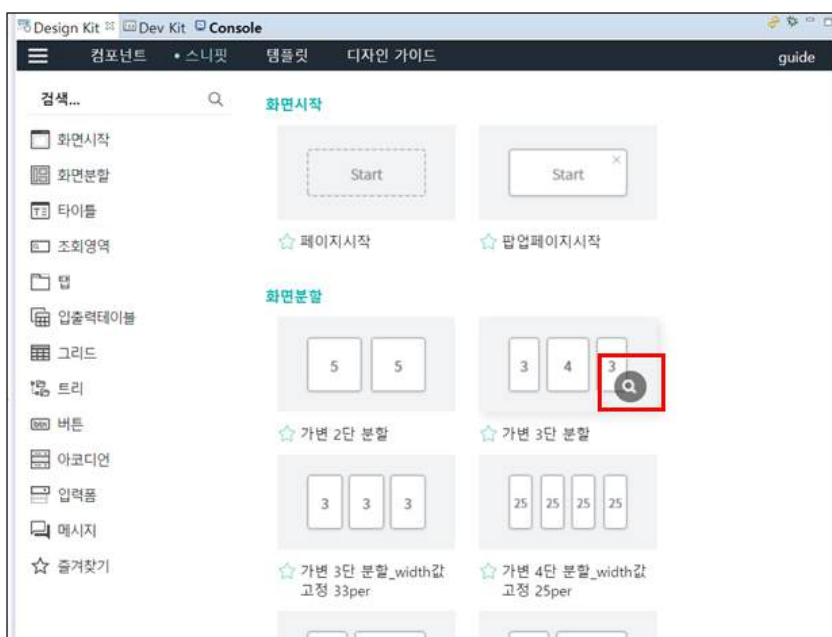
7.7.2.1. 즐겨찾기

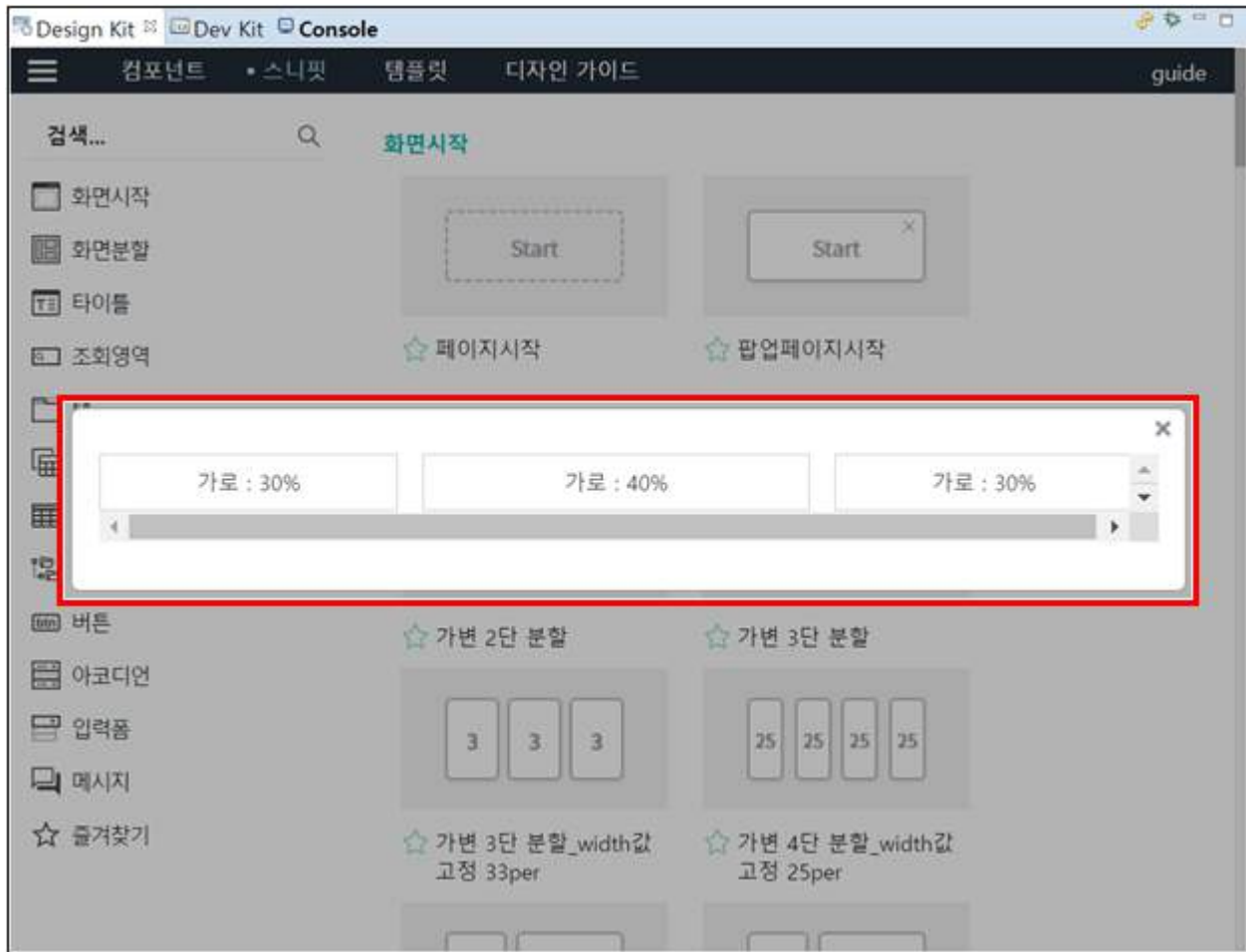
스니핏 이름 왼쪽의 별 모양 아이콘을 클릭하여 해당 스니핏을 즐겨찾기에 추가할 수 있습니다.



7.7.2.2. 미리보기

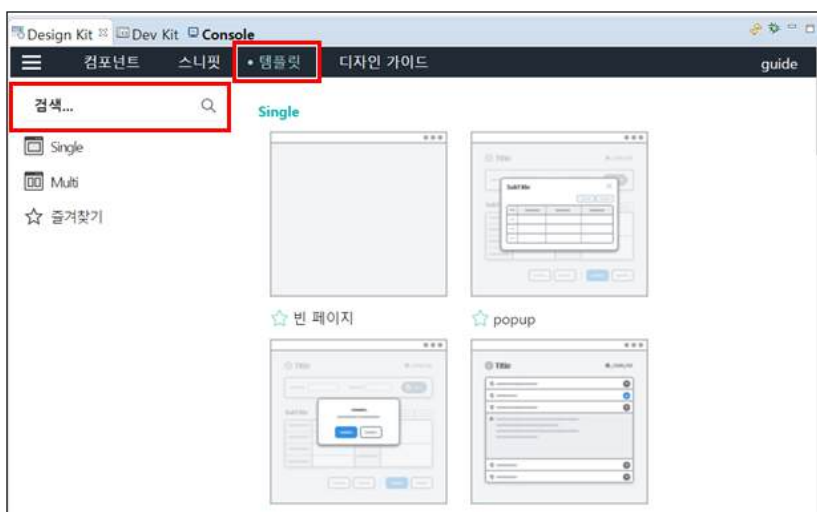
스니핏에 커서를 두면 나타나는 돋보기 모양의 미리보기 버튼이 표시됩니다. 이 버튼을 클릭하면 해당 스니핏을 확인할 수 있습니다.





7.7.3. 템플릿

효율적인 프로젝트 관리를 위해 웹스퀘어 AI는 화면 파일을 템플릿 기반으로 화면을 관리하고 생성할 수 있는 기능을 제공합니다. 템플릿으로 등록된 화면 파일들은 썸네일 이미지 형태로 제공되어 개발자들은 템플릿을 선택해서 원하는 화면을 쉽게 개발할 수 있습니다.



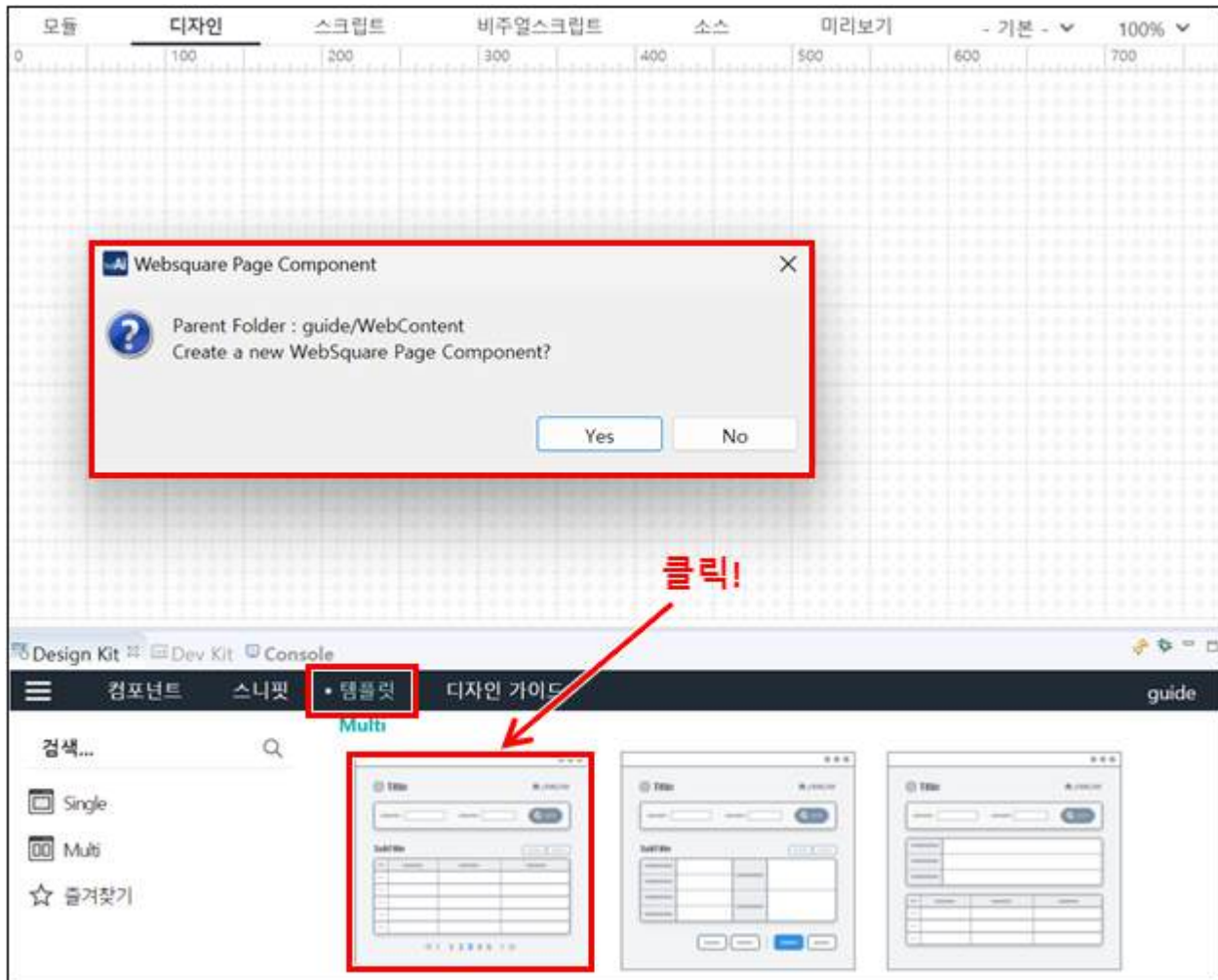
Design Kit의 **템플릿** 탭은 **/WebContent/cm/template/page/** 폴더에 저장된 리소스를 썸네일 이미지로 제공합니다.

그림 7-113 /WebContent/cm/template/page/ 폴더



Design Kit의 **컴포넌트 - 템플릿** 탭에서 템플릿을 선택할 경우, 해당 템플릿을 적용한 새로운 페이지 파일이 생성됩니다.

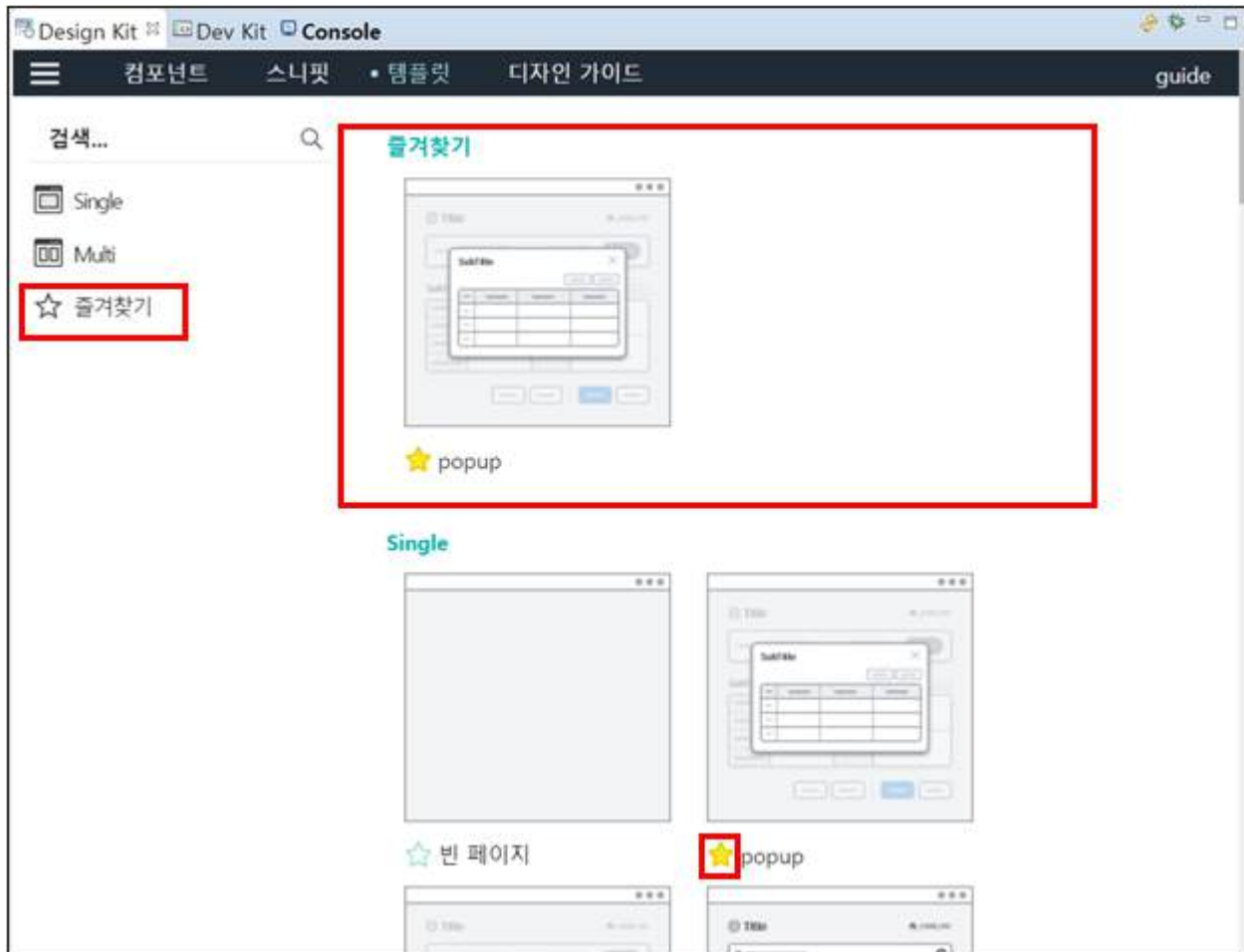
그림 7-114 Design Kit에서 템플릿을 화면 파일에 적용하기



7.7.3.1. 즐겨찾기

템플릿 이름 왼쪽의 별 모양 아이콘을 클릭하여 해당 템플릿을 즐겨찾기에 추가할 수 있습니다.

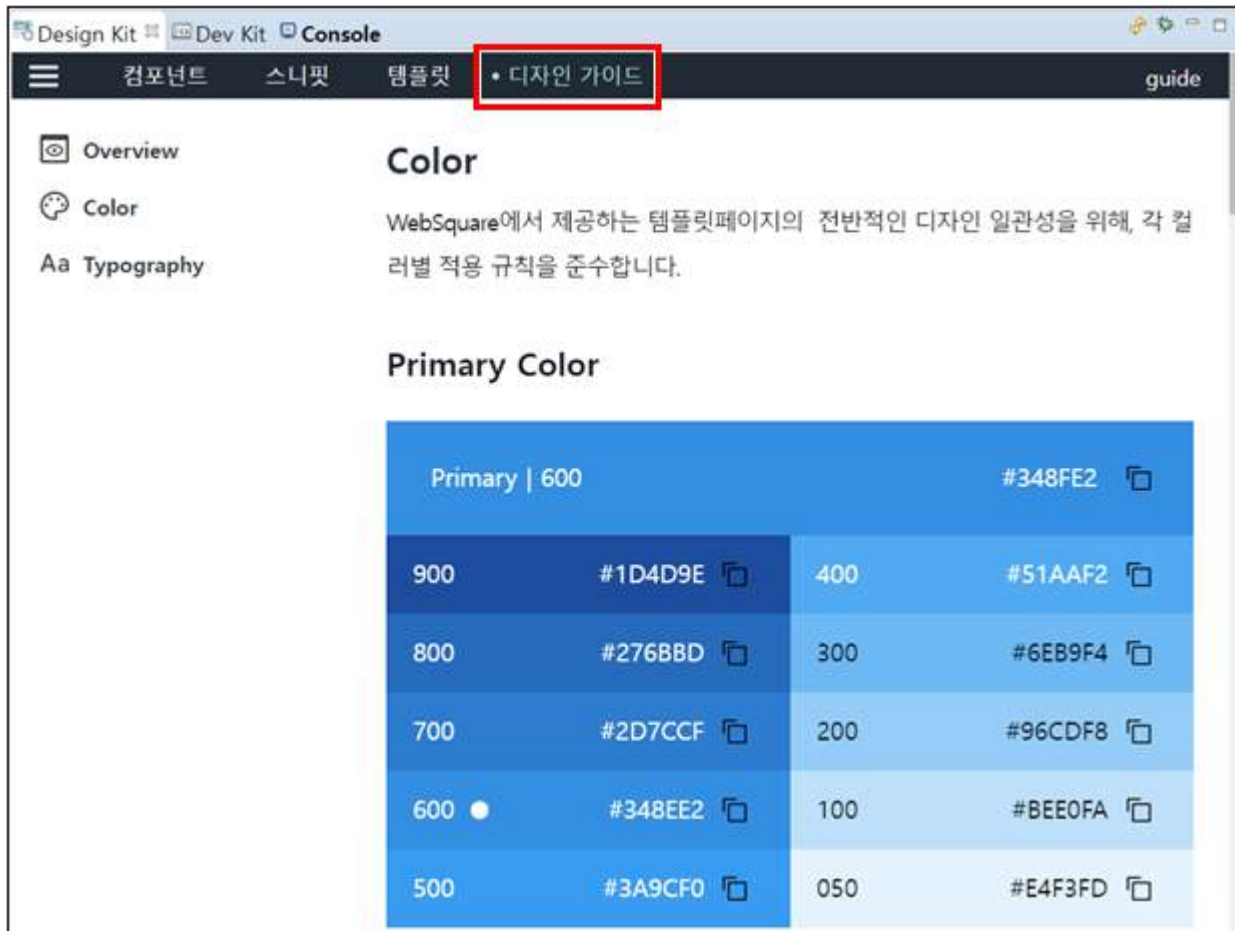
그림 7-115 즐겨찾기 추가



7.7.4. 디자인 가이드

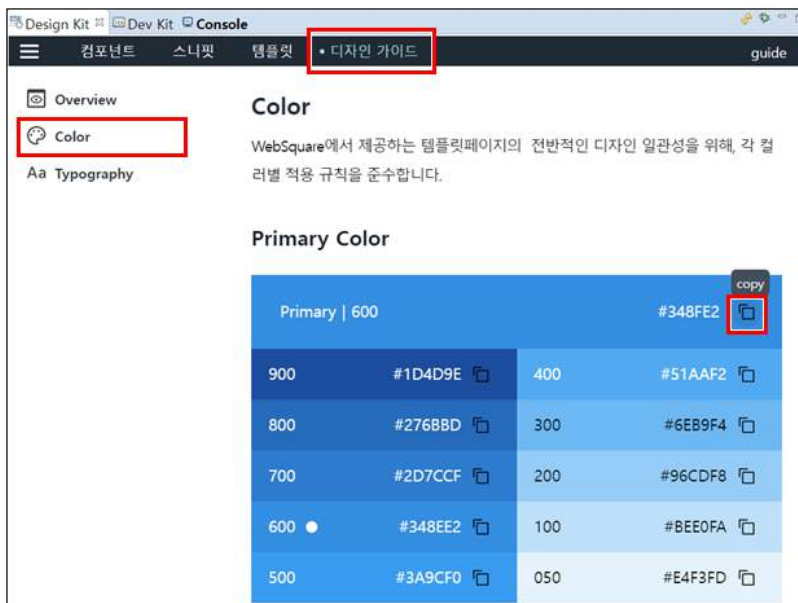
디자인 가이드 탭은 각 프로젝트에 적용할수 있는 디자인 가이드 화면을 표시합니다.

그림 7-116 디자인 가이드



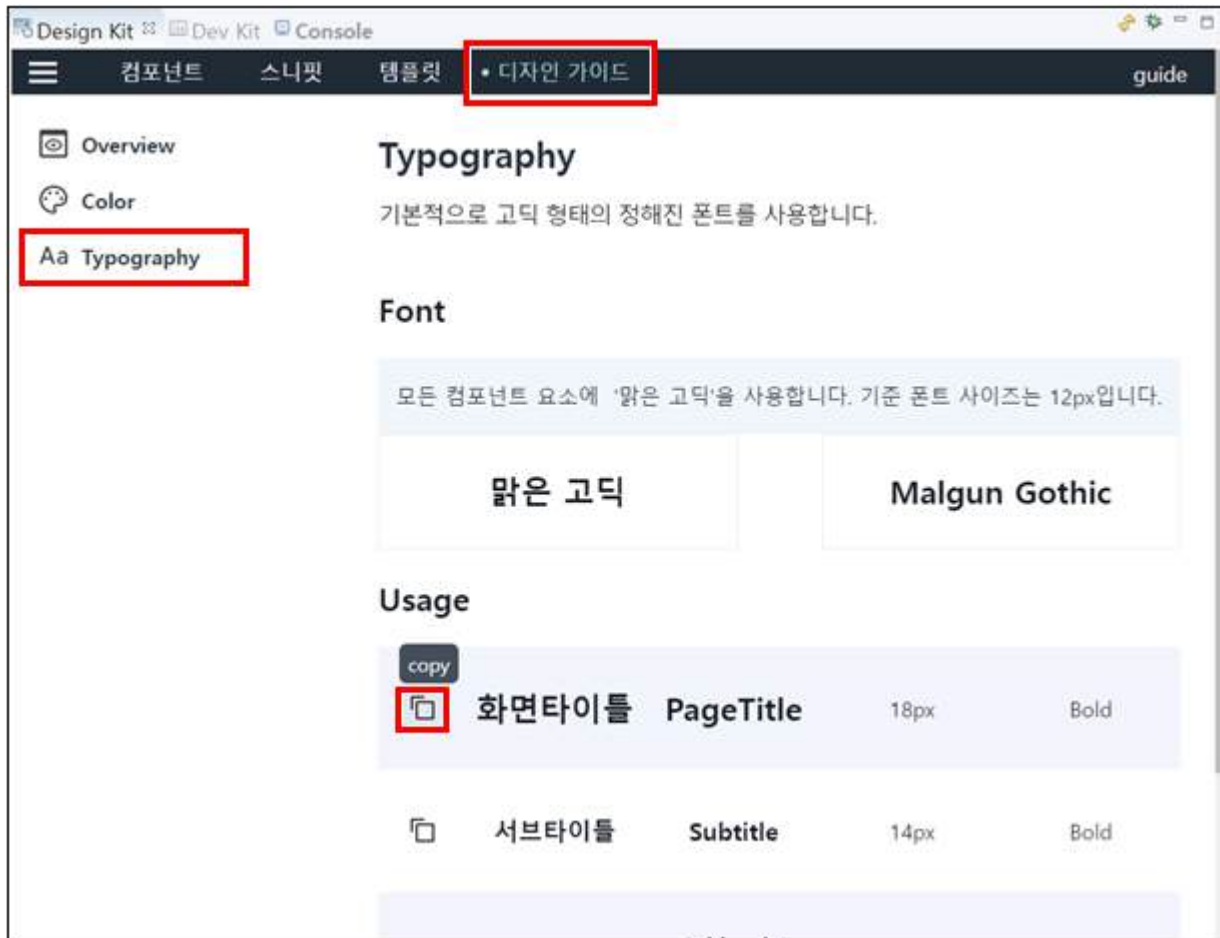
7.7.4.1. 색상 적용

디자인 탭에서 컴포넌트를 선택한 후 디자인 가이드의 색상 복사 버튼을 클릭하면 해당 색상을 선택한 컴포넌트에 적용할 수 있습니다.



7.7.4.2. 폰트 적용

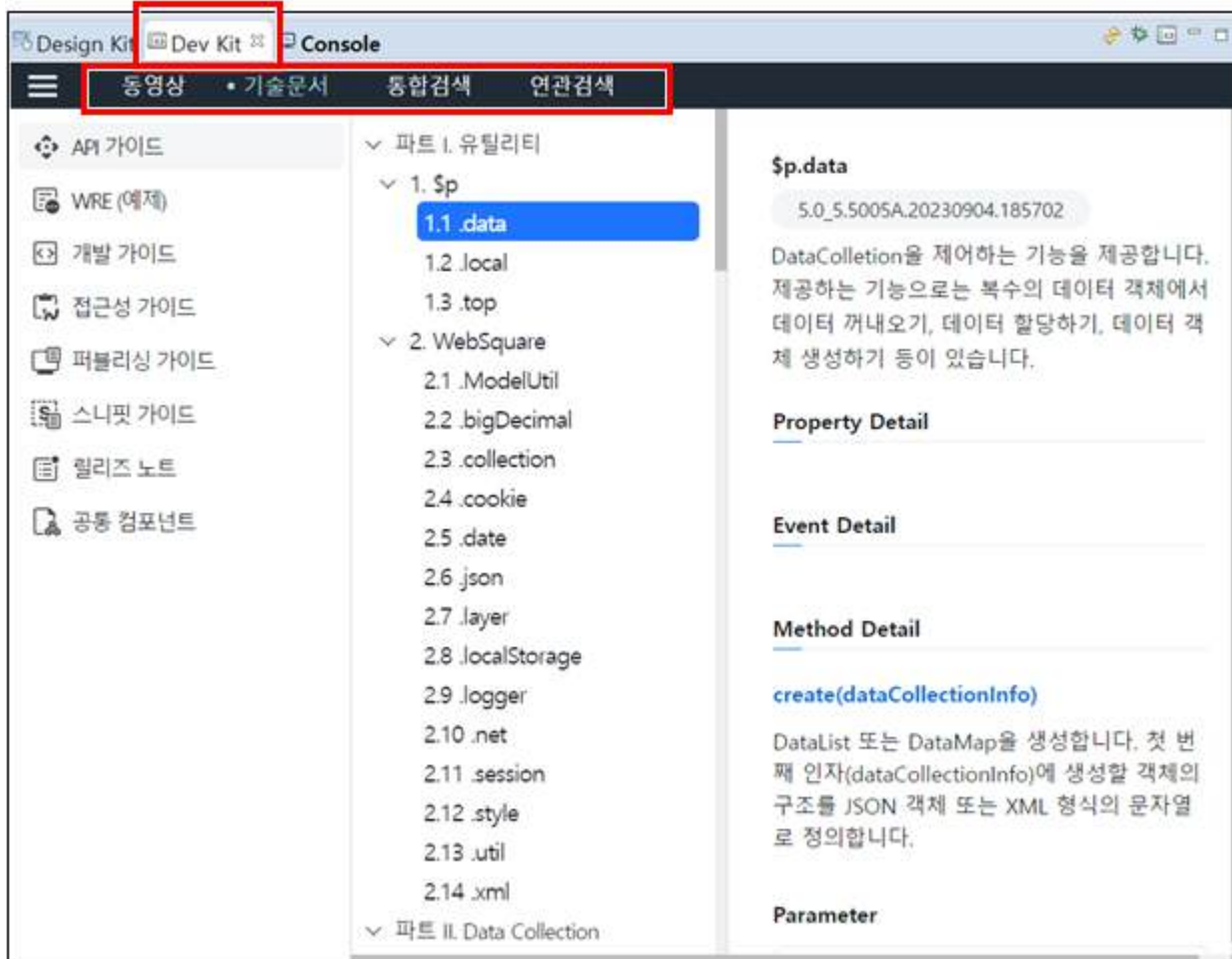
디자인 탭에서 컴포넌트를 선택한 후 디자인 가이드의 폰트 복사 버튼을 클릭하면 해당 색상을 선택한 컴포넌트에 적용할 수 있습니다.



7.8 Dev Kit

동영상, 기술 문서, 검색 등을 통해 WebSquare의 전반적인 개발 가이드를 제공합니다.

그림 7-117 Dev Kit



스튜디오 상단의 Dev Kit 아이콘을 클릭하면 별도의 창에서 확인할 수도 있습니다.

그림 7-118 Dev Kit 아이콘



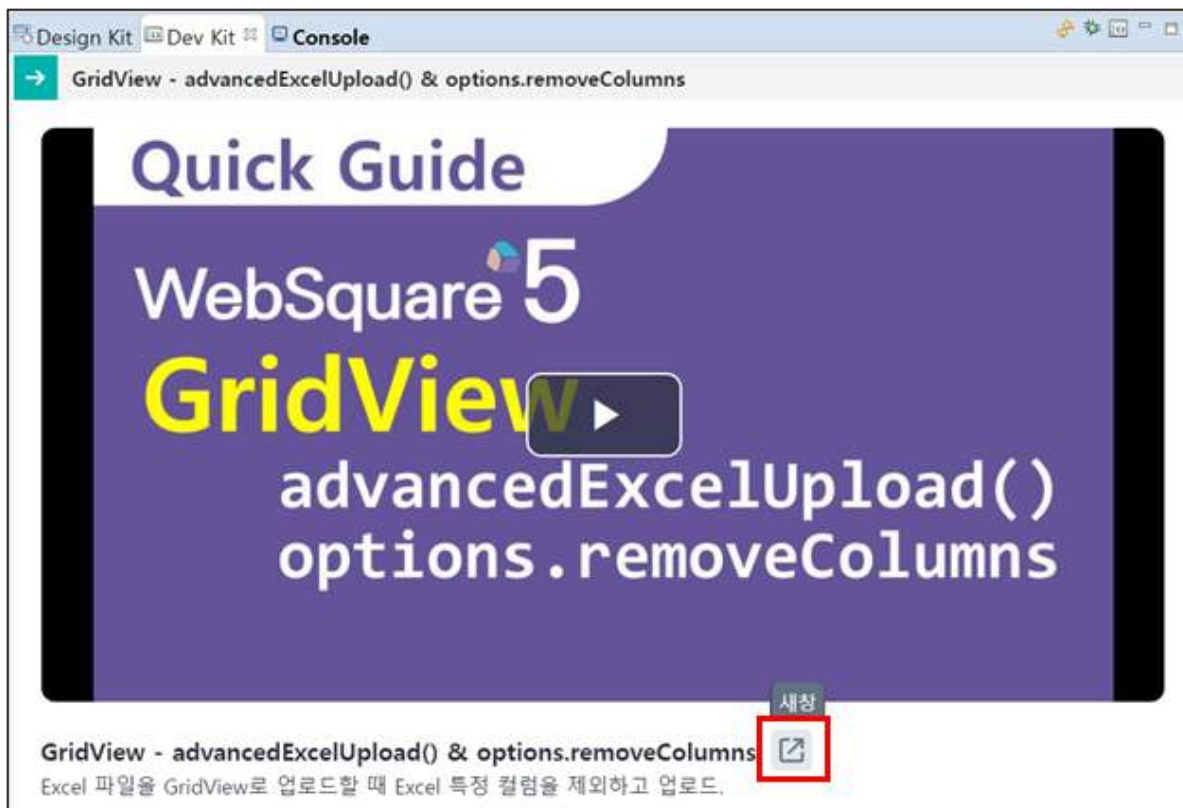
7.8.1. 동영상

웹스퀘어 컴포넌트 사용법을 설명하는 동영상 및 쇼츠 목록을 표시합니다.

그림 7-119 Dev Kit - 동영상



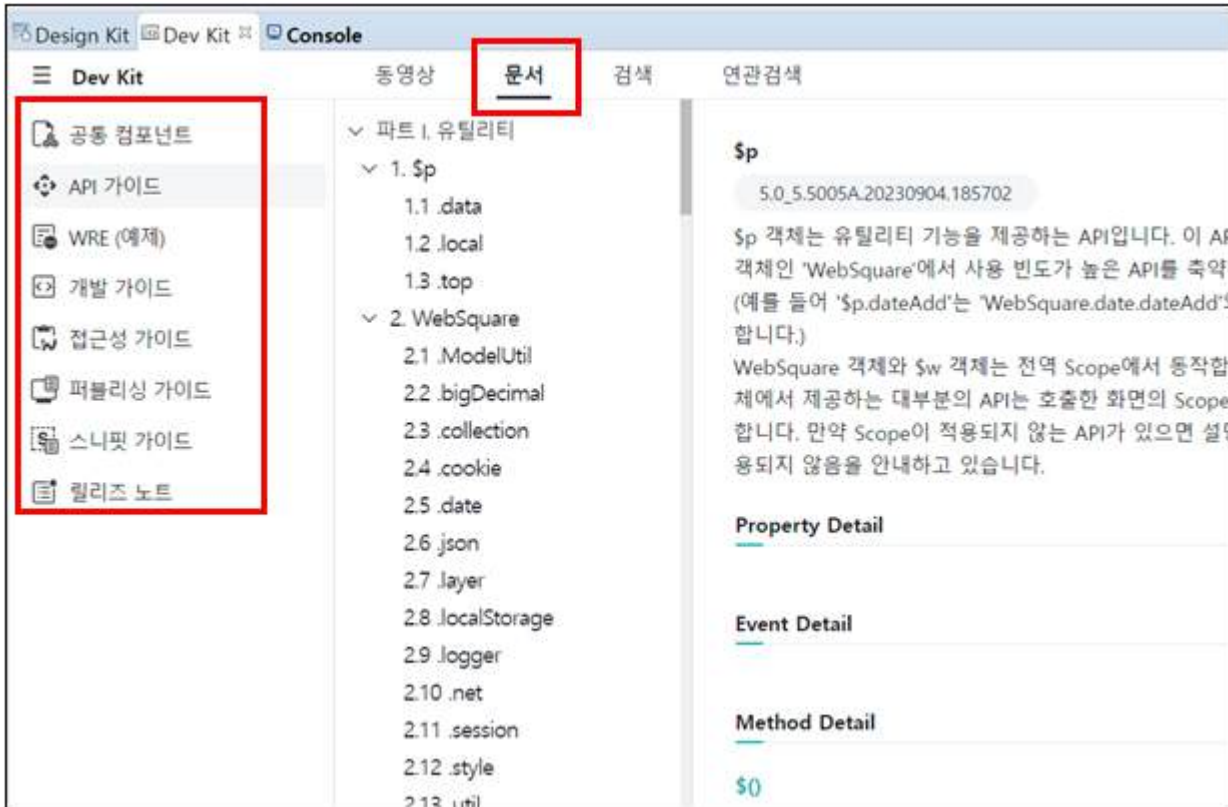
각 동영상을 클릭하여 해당 동영상을 재생할 수 있습니다. 또한 우측 상단의 **새창** 버튼을 클릭하면 재생 화면을 별도의 창으로 분리할 수 있습니다.



7.8.2. 문서

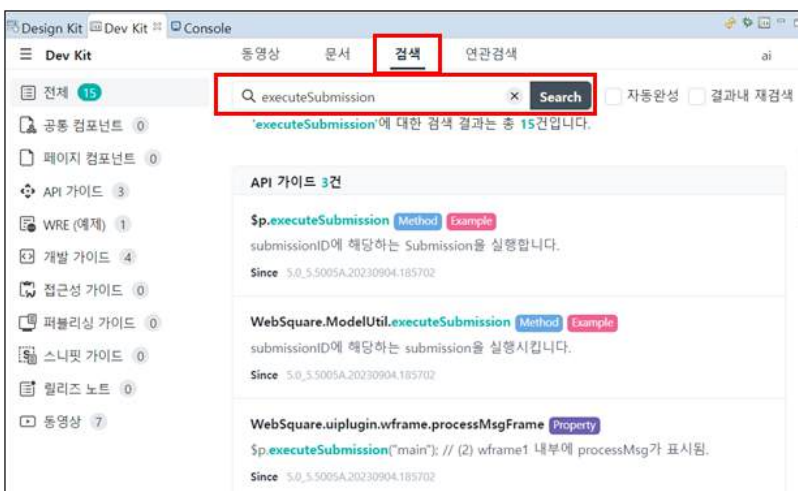
웹스퀘어 다양한 각종 기술 문서를 제공합니다.

그림 7-120 Dev Kit - 문서



7.8.3. 검색

키워드를 입력하여 해당 키워드와 관련된 웹스퀘어 기술 문서 및 동영상を検査하여 표시합니다.

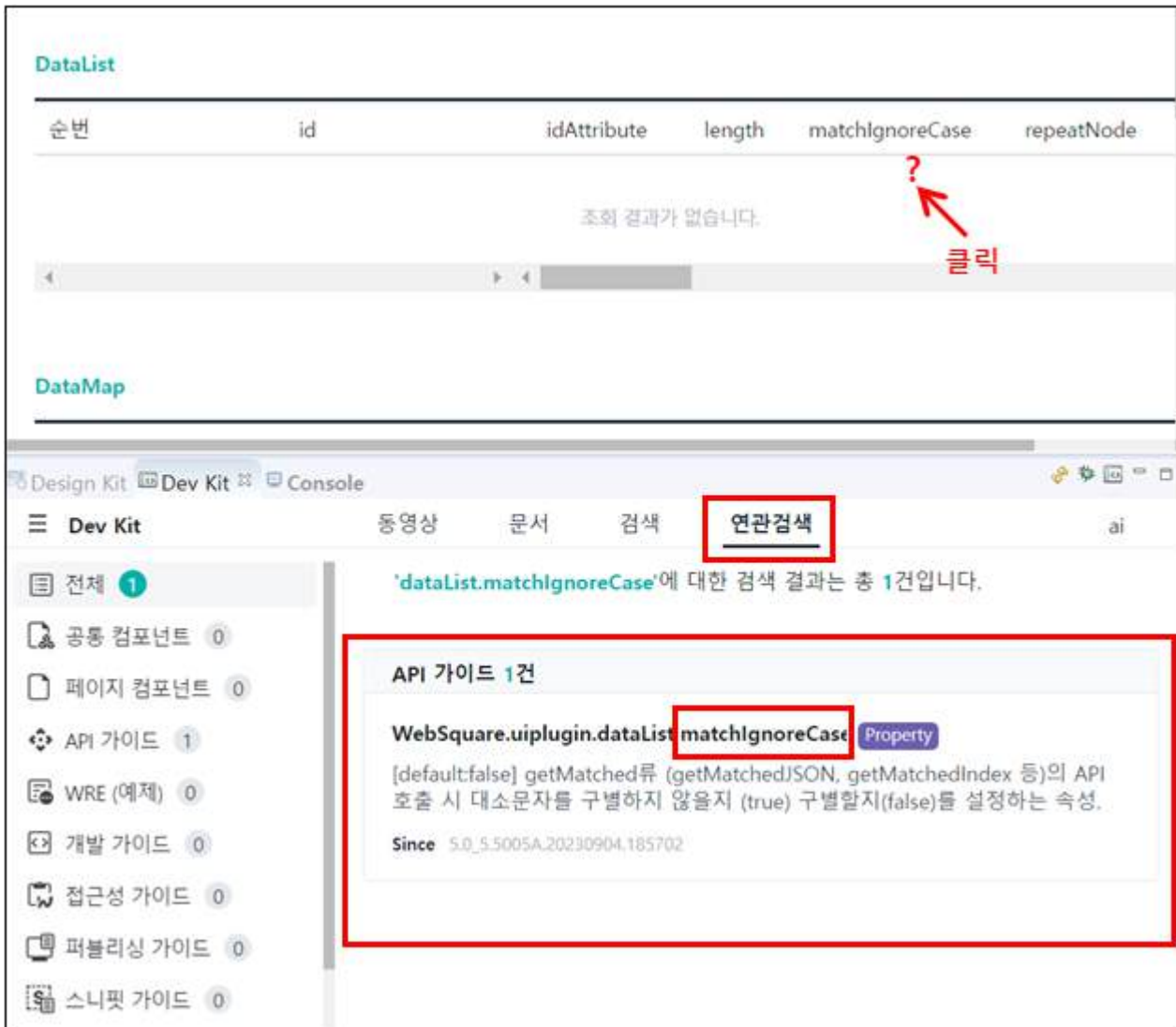


7.8.4. 연관 검색

7.8.4.1. 입력 필드

스튜디오에서 입력 필드명에 마우스 커서를 올릴 때 물음표 모양의 아이콘이 표시되는 경우, 이 물음표 아이콘을 클릭하면 웹스퀘어 기술 문서에서 입력 필드를 검색한 결과가 **연관 검색** 탭에 표시됩니다.

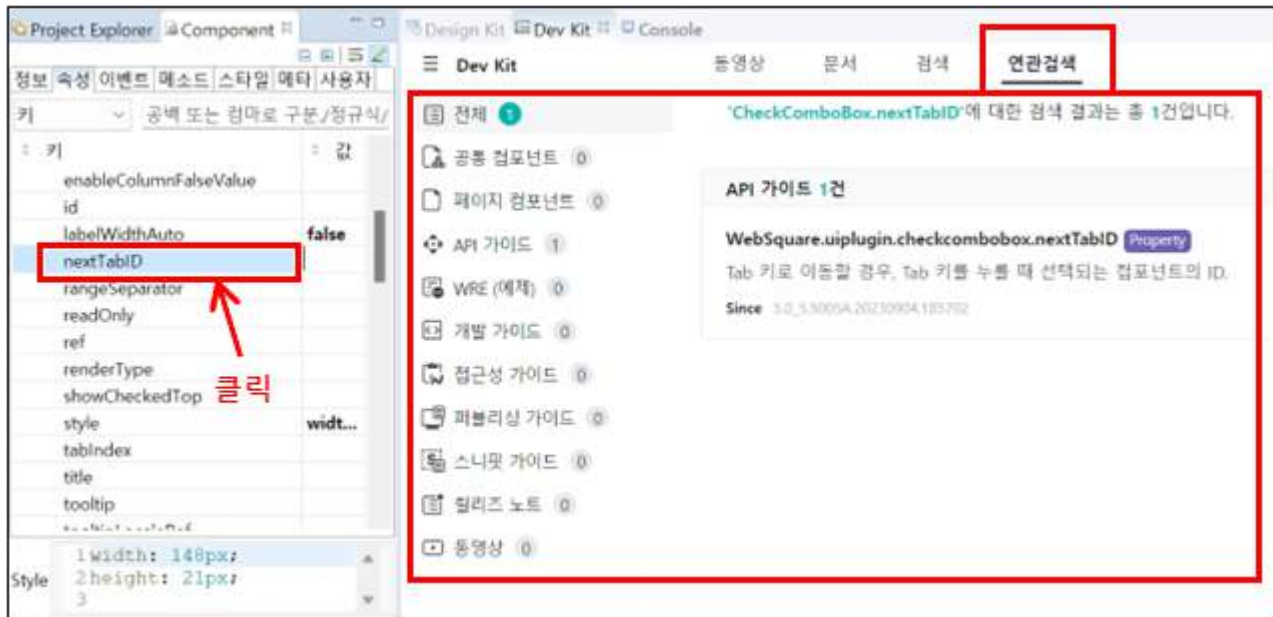
그림 7-121 연관 검색



7.8.4.2. 속성

Component 뷰의 속성 탭에서 속성을 클릭하면 관련 검색 결과가 **연관검색** 탭에 표시됩니다.

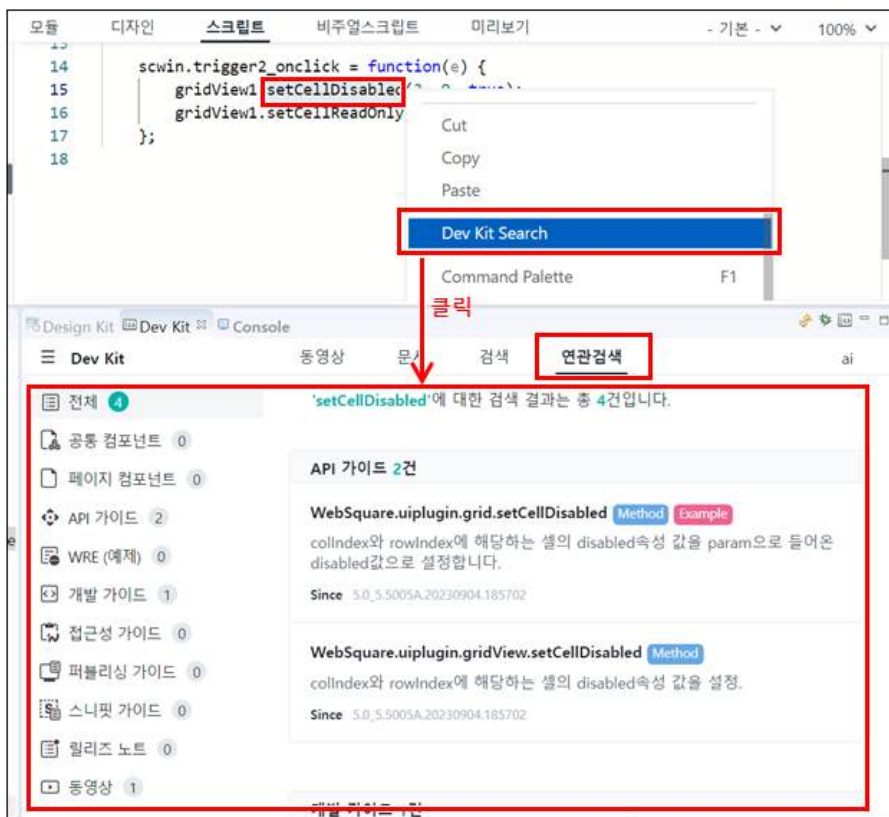
그림 7-122 속성 연관 검색



7.8.4.3. 메소드

스크립트 탭 혹은 비주얼스크립트 탭에서 메소드에 대한 설명을 검색할 수 있습니다. 메소드를 선택하고 마우스 우클릭한 후, Dev Kit Search 메뉴를 선택하십시오. 해당 메소드와 관련된 웹스퀘어 기술 문서 및 동영상에 검색되어, 그 결과가 연관검색 탭에 표시됩니다.

그림 7-123 메소드 연관 검색



8. 화면 그리기

8.1 그리기 모드

웹스퀘어 AI는 Static 및 Absolute 그리기 모드를 제공합니다.

	Static	Absolute
방식	클릭&클릭 방식. 1. Design Kit의 컴포넌트 탭에서 추가할 컴포넌트를 클릭. 2. 디자인 탭의 팝업창에서 추가할 위치를 지정하고 클릭.	클릭&드래그 방식. 1. Design Kit의 컴포넌트 탭에서 추가할 컴포넌트를 클릭. 2. 디자인 탭에서 컴포넌트를 직접 드래그하여 그리기.
컴포넌트 위치 결정	디자인 탭에서 선택한 컴포넌트를 기준으로 새로운 컴포넌트의 위치가 결정됨.	디자인 탭에서 드래그하여 직접 위치를 결정.
렌더링	컴포넌트 생성 시 지정한 위치에 따라 렌더링 순서가 결정됨.	렌더링 순서가 컴포넌트를 그린 순서를 따라감.
권장 환경	기존에 존재하는 화면을 수정하거나 기존 화면에 새로운 컴포넌트를 추가하는 경우.	새로운 WebSquare 페이지를 생성할 경우.

상단의 그리기 모드 변경 아이콘을 클릭하여 그리기 모드를 변경할 수 있습니다.

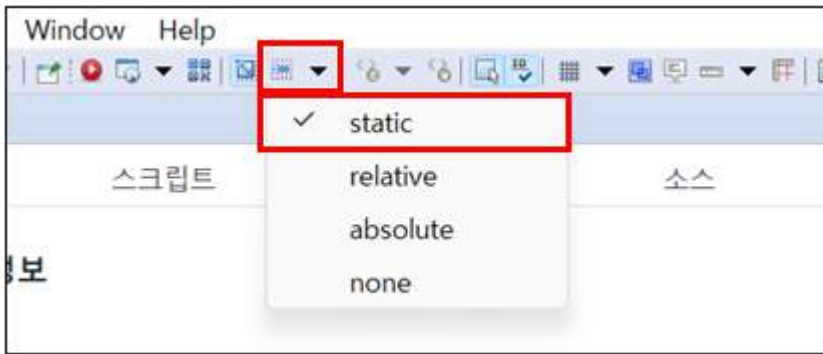
그림 8-1 그리기 모드 변경



8.2 Static 모드 (클릭 & 클릭)

1. 툴바의 그리기 모드 선택 아이콘 아이콘을 클릭하여 "static" 모드를 선택합니다.

그림 8-2 static 모드 선택



2. 혹은 그리기 모드 변경 아이콘을 클릭하여 바로 "static" 모드로 변경할 수도 있습니다.

그림 8-3 그리기 모드 변경

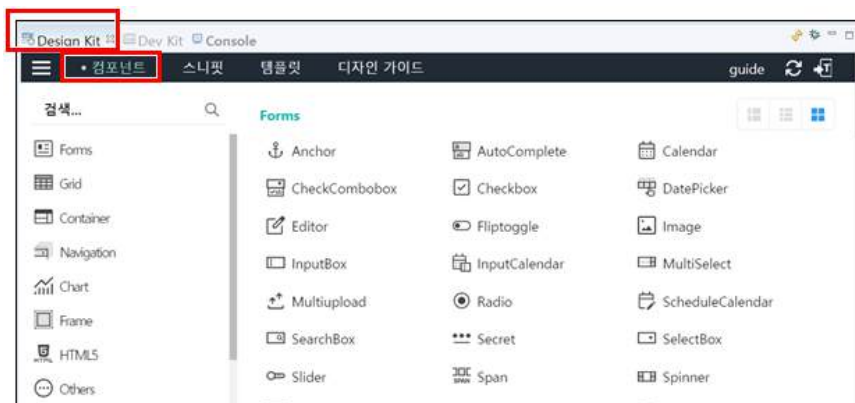


그림 8-4 static 모드 선택 확인



3. Design Kit의 컴포넌트 탭에서 (컴포넌트를 검색하여) 추가할 컴포넌트를 선택합니다.

그림 8-5 Design Kit - 컴포넌트



4. 선택한 컴포넌트를 추가할 위치를 선택합니다.

그림 8-6 컴포넌트를 추가할 위치 결정

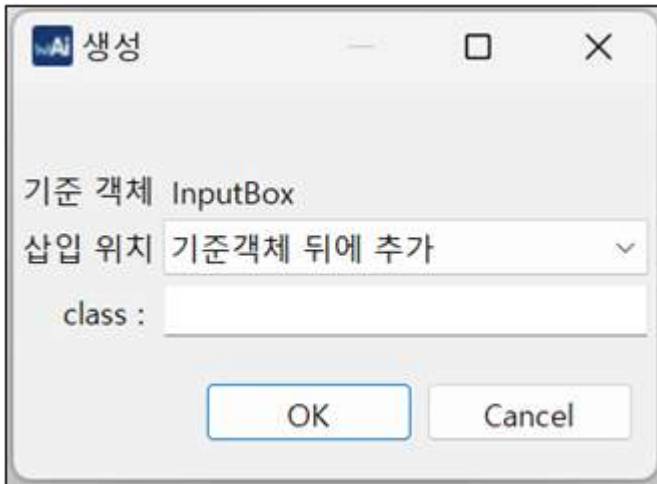
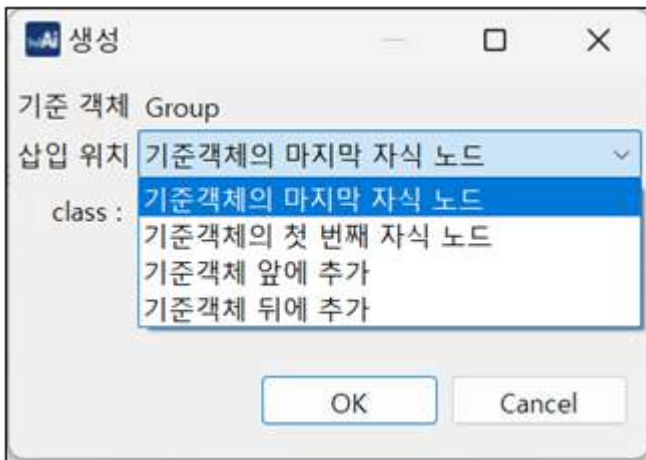


그림 8-7 컴포넌트를 추가할 위치 결정



5. 디자인 탭에서 컴포넌트가 추가되었는지 확인합니다.



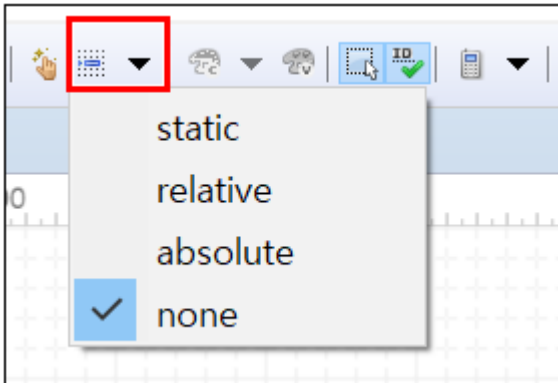
Static 모드 권장

대부분의 경우 Static 모드 사용을 권장합니다.

8.2.1. 스타일 제거

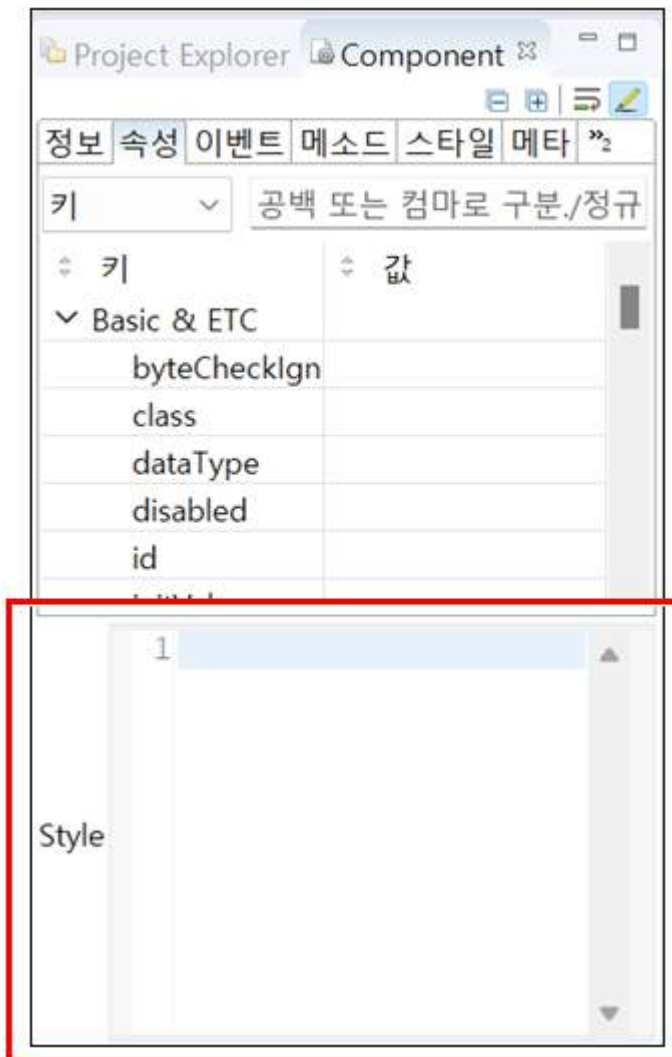
Static 모드 (클릭 & 클릭) 모드에서 컴포넌트를 그릴 경우, 스타일 정보 없이 컴포넌트 그리기가 가능합니다. **inline Style** 버튼을 클릭한 후 **none**을 선택하십시오.

그림 8-8 스타일 제거 기능 (Inline Style - none)



디자인 탭에서 컴포넌트를 그린 후, 해당 컴포넌트를 선택해도 **Style** 뷰에는 아무 정보도 표시되지 않습니다.

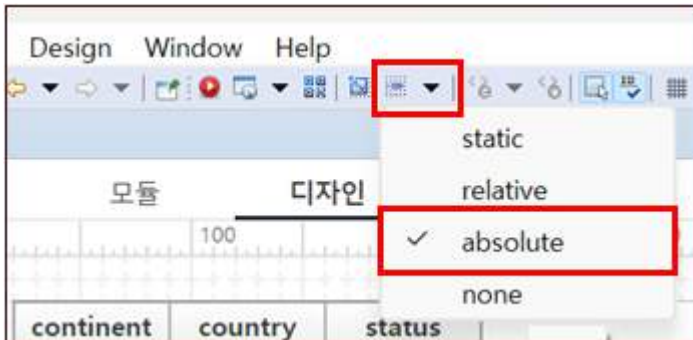
그림 8-9 스타일 정보가 없는 Style 뷰



8.3 Absolute 모드 (클릭 & 드래그)

1. 툴바의 그리기 모드 선택 아이콘 아이콘을 클릭하여 "absolute" 모드를 선택합니다.

그림 8-10 absolute 모드 선택



2. 혹은 그리기 모드 변경 아이콘을 클릭하여 바로 "absolute" 모드로 변경할 수도 있습니다.

그림 8-11 그리기 모드 변경

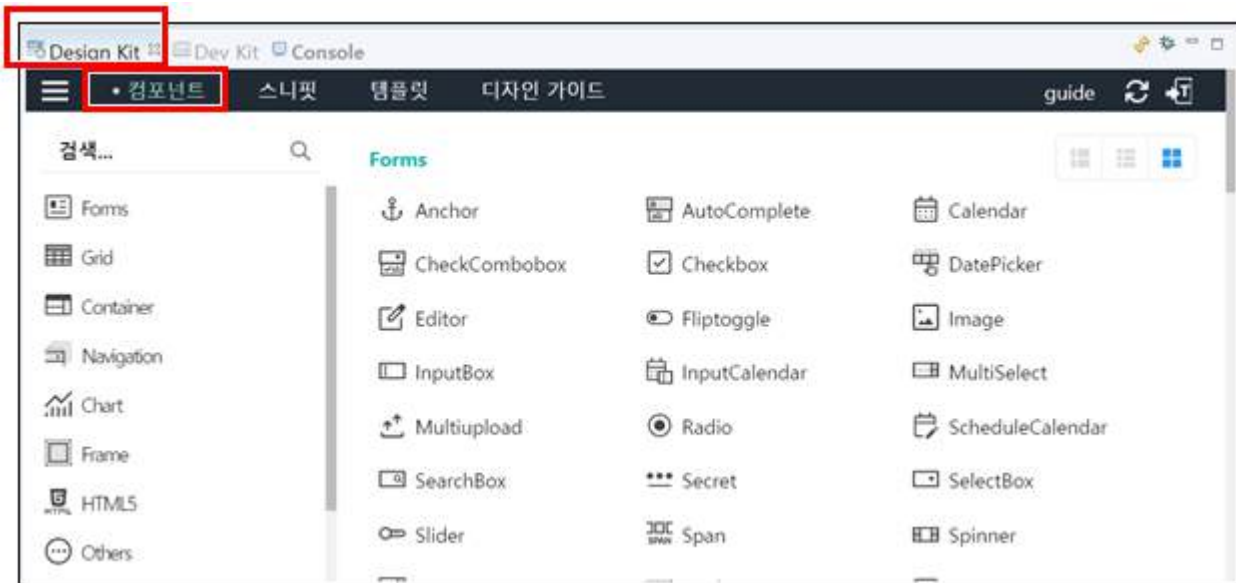


그림 8-12 absolute 모드 선택 확인



3. Design Kit의 컴포넌트 탭에서 (컴포넌트를 검색하여) 추가할 컴포넌트를 선택합니다.

그림 8-13 Design Kit - 컴포넌트



4. 선택한 컴포넌트를 추가할 위치를 선택합니다.

그림 8-14 컴포넌트를 추가할 위치 결정

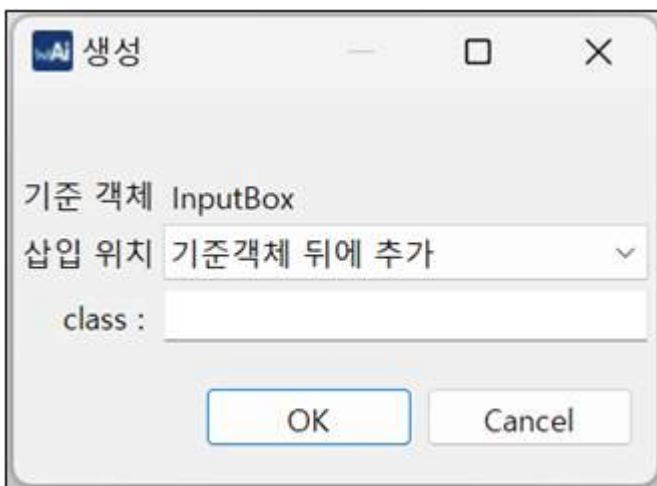
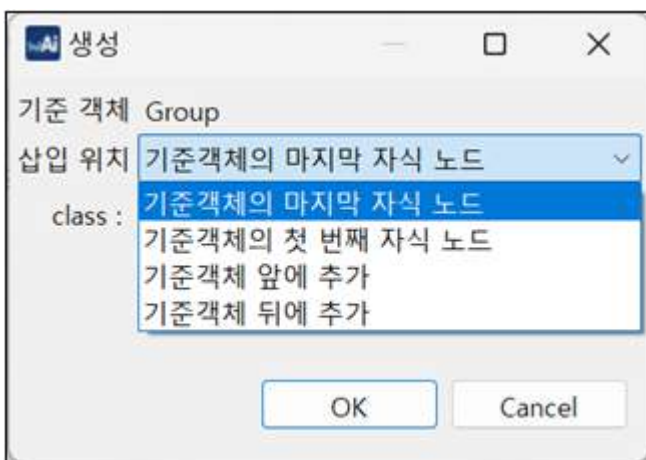
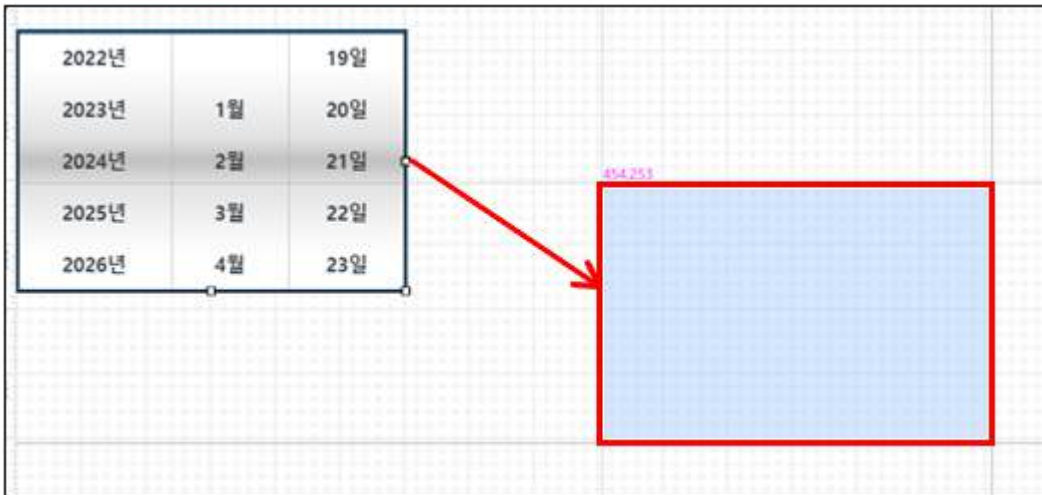


그림 8-15 컴포넌트를 추가할 위치 결정



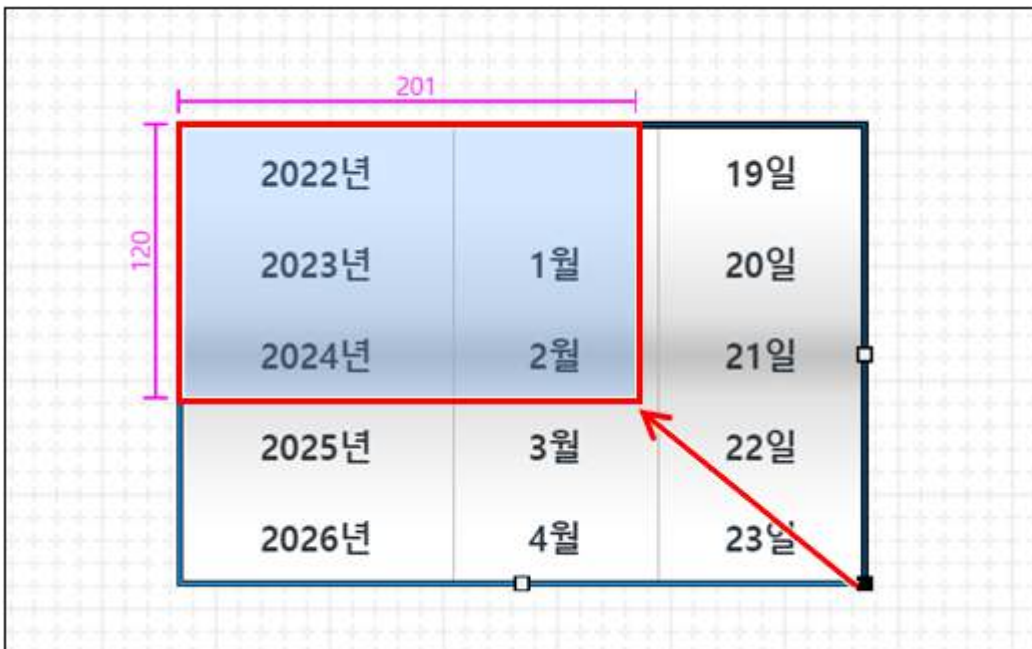
5. 디자인 탭에서 컴포넌트가 추가되었는지 확인합니다.
6. 추가한 컴포넌트를 원하는 위치에 원하는 위치로 드래깅하여 이동합니다.

그림 8-16 원하는 위치로 이동



7. 추가한 컴포넌트를 드래깅하여 원하는 크기로 조정합니다.

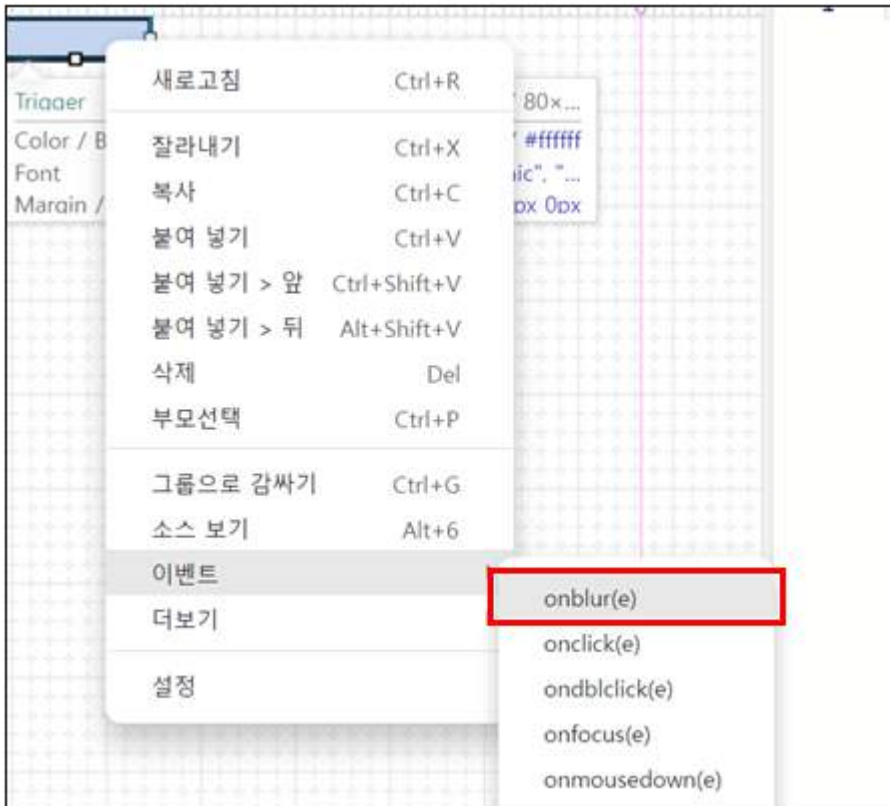
그림 8-17 원하는 크기로 조정



8.4 이벤트 추가

1. 컴포넌트를 우클릭합니다.
2. 이벤트를 클릭한 후 원하는 이벤트를 선택합니다. (각 컴포넌트에 별로 등록 가능한 이벤트는 API 가이드를 참고하십시오.)

그림 8-18 이벤트 추가

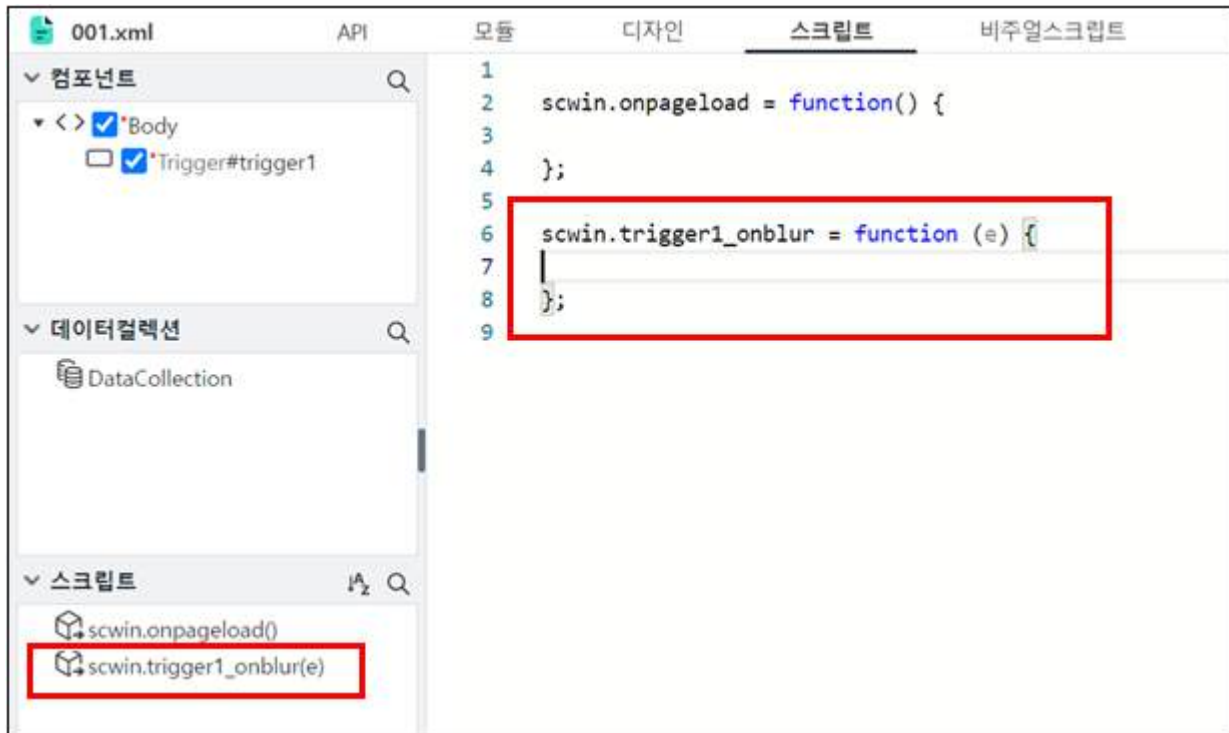


3. Script 탭이 열리고 해당 이벤트에 대한 함수가 아래와 같이 생성되는지 확인합니다.

Code 8-1 이벤트 함수명

```
1 // scwin.컴포넌트명_이벤트
```

그림 8-19 이벤트 추가



4. 이벤트 핸들러 함수를 정의합니다.

Code 8-2 함수 정의 (예시)

```
1 scwin.trigger1_onblur = function(e) {
2     var name = input1.getValue();
3     output1.setValue(name);
4 };
```

8.5 컴포넌트 동적 생성

`$p.dynamicCreate()` 함수를 이용하여 컴포넌트를 동적으로 생성할 수 있습니다.

Code 8-3 사용 예

```
1 scwin.trigger1_onclick = function(e) {
2
3     // Group 생성
4     $p.dynamicCreate("group1", "group", {
5         style: "width: 200px; height: 40px; margin: 50px"
6     });
```

```

7
8 // Group 내에 TextBox 생성 및 값 설정.
9 $p.dynamicCreate("textbox1", "textbox", {
10     style: "border: solid black 2px; float: left; width: 200px; height: 20px; float: left;
text-align: center; margin-bottom: 20px"
11 }, group1);
12
13 var comObj = WebSquare.util.getComponentById("textbox1");
14 comObj.setValue("Select the date.");
15
16 // Group 내에 Calendar 생성.
17 $p.dynamicCreate("calendar1", "calendar", {
18     style: "width: 200px; float: left; height: 40px; float: left"
19 }, group1);
20 };

```

8.6 객체 추가

Script 탭에서 아래와 같은 객체를 정의할 수 있습니다.

8.6.1. 브라우저 객체

Code 8-4 브라우저 제공 객체

```

1 window.open('http://');

```

8.6.2. 사용자 정의

변수, 함수, 객체 등을 직접 정의할 수 있습니다.

```

1 var userName = "WebSquare";
2
3 function getUser_name() {
4     return userName;

```

```

5  };
6
7  var info = {
8      name: "WebSquare",
9      city: "Seoul"
10 };
11
12 var common = {
13     isNumber: function(tmpVal) {
14         return Number(tmpVal);
15     }
16 };

```

8.6.3. WebSquare 객체



WebSquare 객체는 전역 공간에 정의됨.

Browser 객체 혹은 사용자 정의 객체의 이름과 중복되지 않아야 함.

8.6.3.1. 유틸리티

Code 8-5 유틸리티

```

1  var curDate = $p.getCurrentServerDate('yyyy-MM-dd');
2  // WAS에서 오늘 날짜를 꺼내는 예

```

8.6.3.2. UI 객체

Code 8-6 UI 객체

```

1  inputbox1.setValue("WebSquare");
2  // "inputbox1" ID에 해당하는 컴포넌트의 value를 "WebSquare"로 설정

```

8.6.3.3. 데이터 객체

Code 8-7 데이터 객체

```

1 curRowDataObj = datalist1.getRowJSON(0);
2 //"datalis1" ID에 해당하는 DataCollection 객체(DataList)에 접근한 후,
3 // 0번째 행의 데이터를 꺼내오는 예

```

8.7 브라우저에서 확인하기

완성한 페이지는 브라우저에서 직접 확인할 수 있습니다. 빨간색 화살표 모양의 아이콘을 클릭하여 페이지 컴포넌트가 렌더링된 모습을 직접 확인하십시오.

그림 8-20 기본 브라우저로 확인

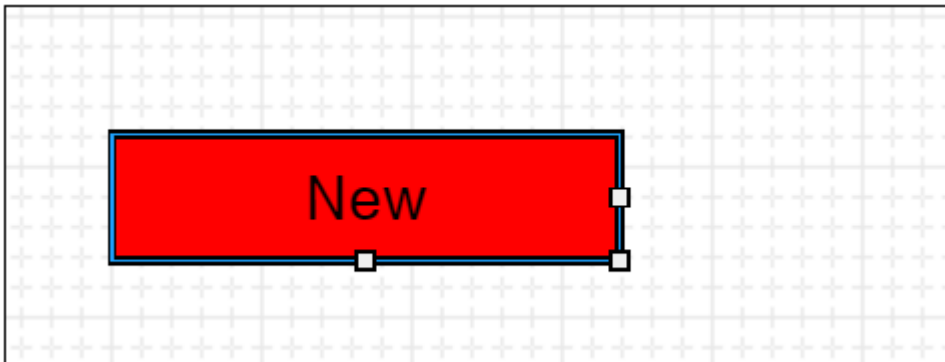


9. 유용한 기능

9.1 드래그 & 드롭을 통한 컴포넌트 제어

사용자는 드래그 & 드롭을 통해 컴포넌트를 쉽게 선택, 이동, 리사이징할 수 있습니다. 선택한 컴포넌트는 아래와 같이 테두리에 선과 점이 표시됩니다.

그림 9-1 컴포넌트 선택



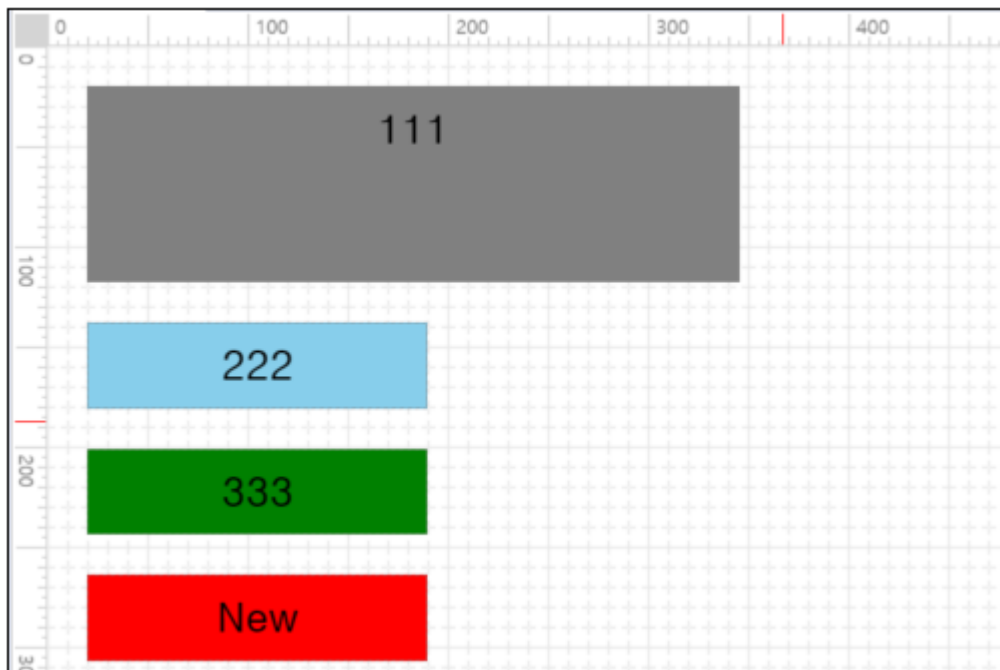
9.1.1. 리사이징

컴포넌트를 클릭 후 원하는 크기로 드래깅하여 크기를 변경할 수 있습니다. 크기 정보도 같이 표시됩니다.

그림 9-2 사이즈 조정을 위한 드래깅



그림 9-3 사이즈 조정 후



9.1.2. 여러 컴포넌트 선택

여러 컴포넌트를 한번에 선택하여 이동하는 것도 가능합니다.

그림 9-4 여러 컴포넌트를 드래깅하여 선택



그림 9-5 여러 컴포넌트가 선택된 모습



그림 9-6 여러 컴포넌트를 드래킹하여 선택

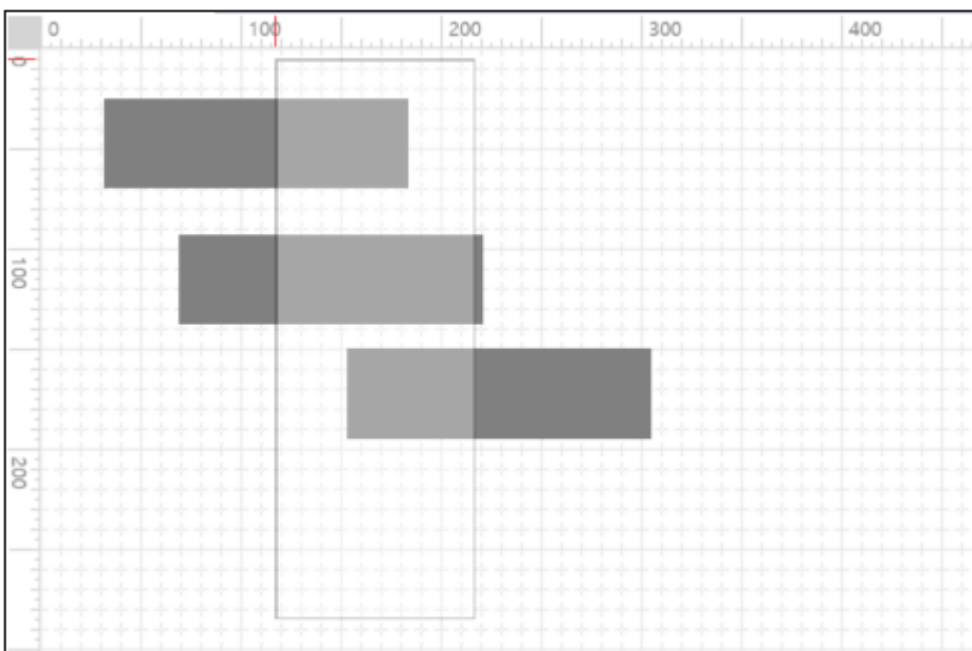
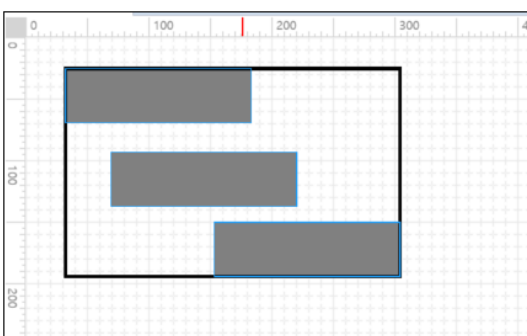


그림 9-7 여러 컴포넌트가 동시에 선택된 모습



9.1.3. 앞/뒤 이동



동일 모드 사용

컴포넌트 생성 및 이동 시 동일한 모드를 사용하십시오.

즉, Static 모드에서 생성한 컴포넌트의 경우 Static 모드에서 이동하고, Absolute 모드에서 생성한 컴포넌트의 경우 Absolute 모드에서 이동할 것을 권장합니다.

9.1.3.1. Static 모드

특정 컴포넌트를 드래킹하여 다른 컴포넌트의 앞/뒤로 이동할 수 있습니다.

9.1.3.1.1. 뒤로 이동

원하는 컴포넌트를 이동 기준 컴포넌트의 뒷 부분에 그림자가 표시되도록 드래킹합니다.

아래 예제에서는:

- (1) New 컴포넌트를 222 컴포넌트(이동 기준 컴포넌트)로 드래킹한 후,
- (2) 222 컴포넌트 뒤에 그림자가 표시되는 지점에서 드롭합니다.
- (3) New 컴포넌트는 222 컴포넌트 뒤로 이동됩니다.

그림 9-8 "222" 뒤로 이동



9.1.3.1.2. 앞으로 이동

원하는 컴포넌트를 이동 기준 컴포넌트의 앞 부분에 그림자가 표시되도록 드래킹합니다.

아래 예제에서는:

- (1) New 컴포넌트를 222 컴포넌트(이동 기준 컴포넌트)로 드래깅한 후,
- (2) 222 컴포넌트 앞에 그림자가 표시되는 지점에서 드롭합니다.
- (3) New 컴포넌트는 222 컴포넌트 앞으로 이동됩니다.

그림 9-9 "222" 앞으로 이동



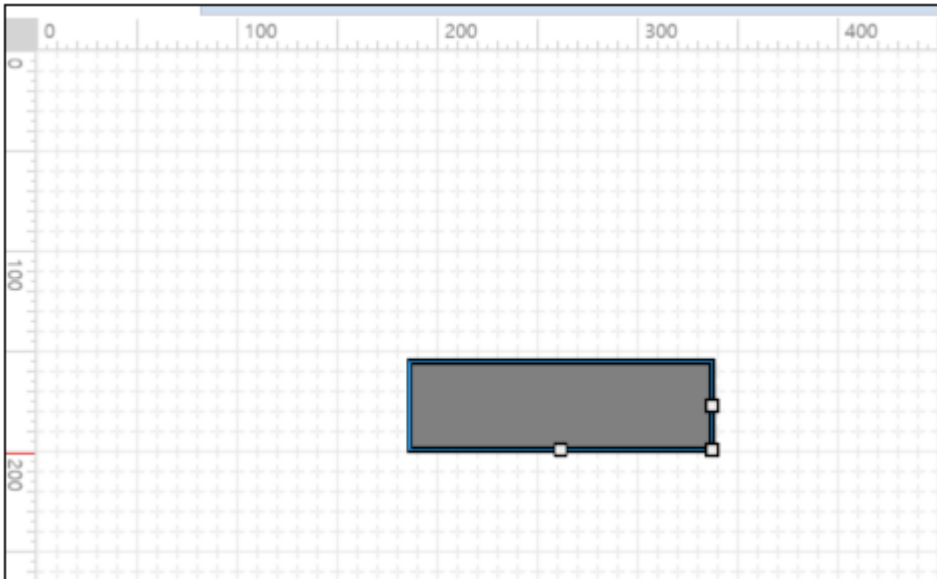
9.1.3.2. Absolute 모드

Absolute 모드에서는 컴포넌트를 직접 드래깅하여 원하는 위치로 이동할 수 있습니다.

그림 9-10 이동할 위치로 드래깅



그림 9-11 드래킹 후 이동 완료된 모습



9.1.4. 여러 컴포넌트를 한번에 이동

여러 컴포넌트를 한번에 선택하여 이동하는 것도 가능합니다.



동일 모드 사용

컴포넌트 생성 및 이동 시 동일한 모드를 사용하십시오.

즉, Static 모드에서 생성한 컴포넌트의 경우 Static 모드에서 이동하고, Absolute 모드에서 생성한 컴포넌트의 경우 Absolute 모드에서 이동할 것을 권장합니다.

9.1.4.1. Static 모드

그림 9-12 New 컴포넌트 뒤로 한번에 이동



그림 9-13 이동 후



9.1.4.2. Absolute 모드

그림 9-14 드래깅

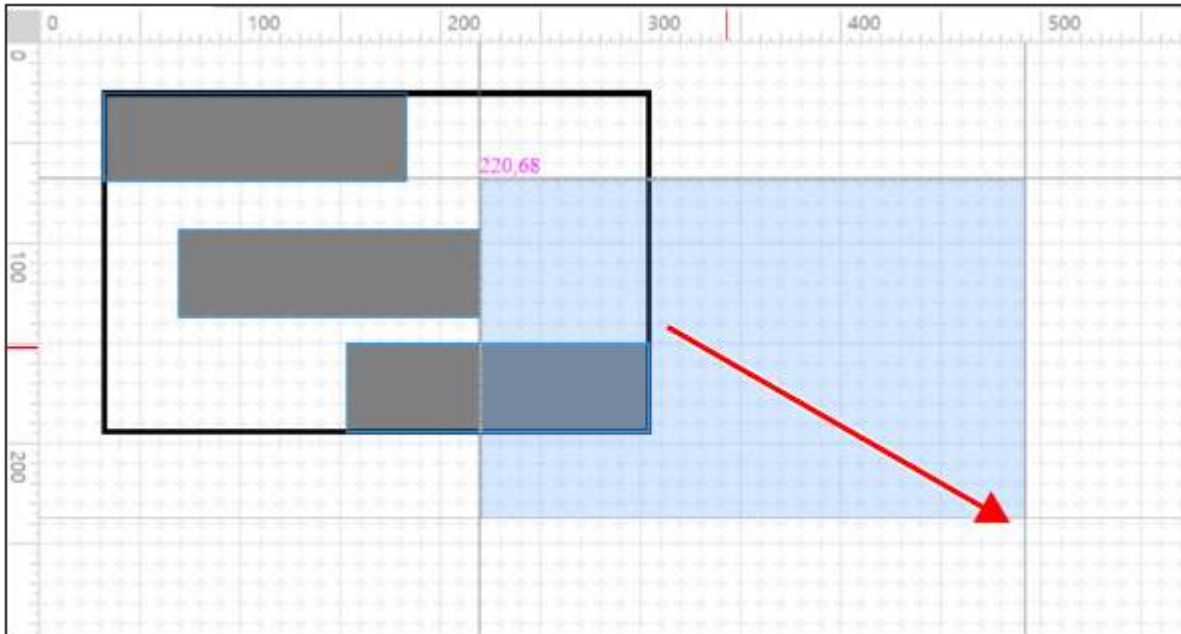
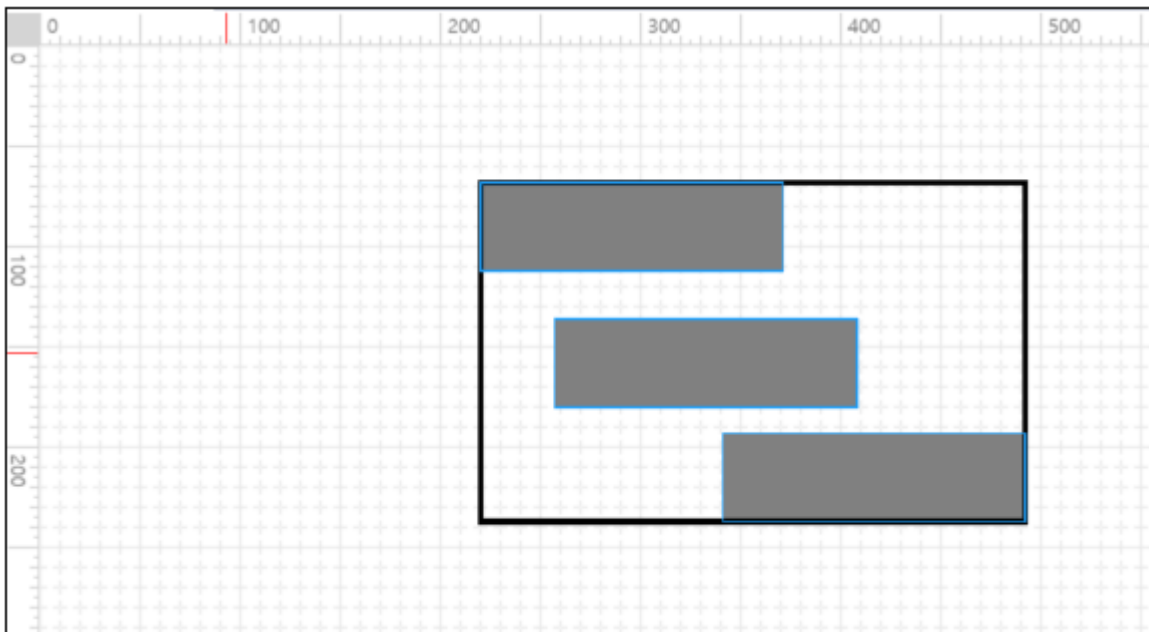


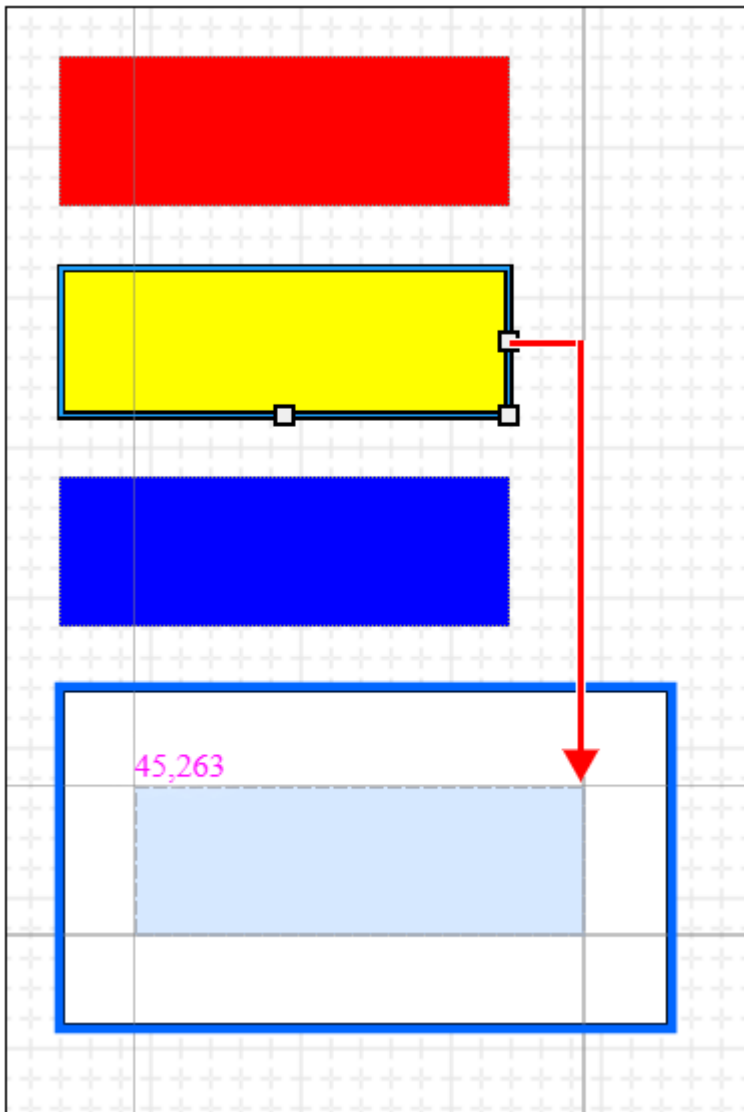
그림 9-15 이동 후



9.1.5. 하위로 이동

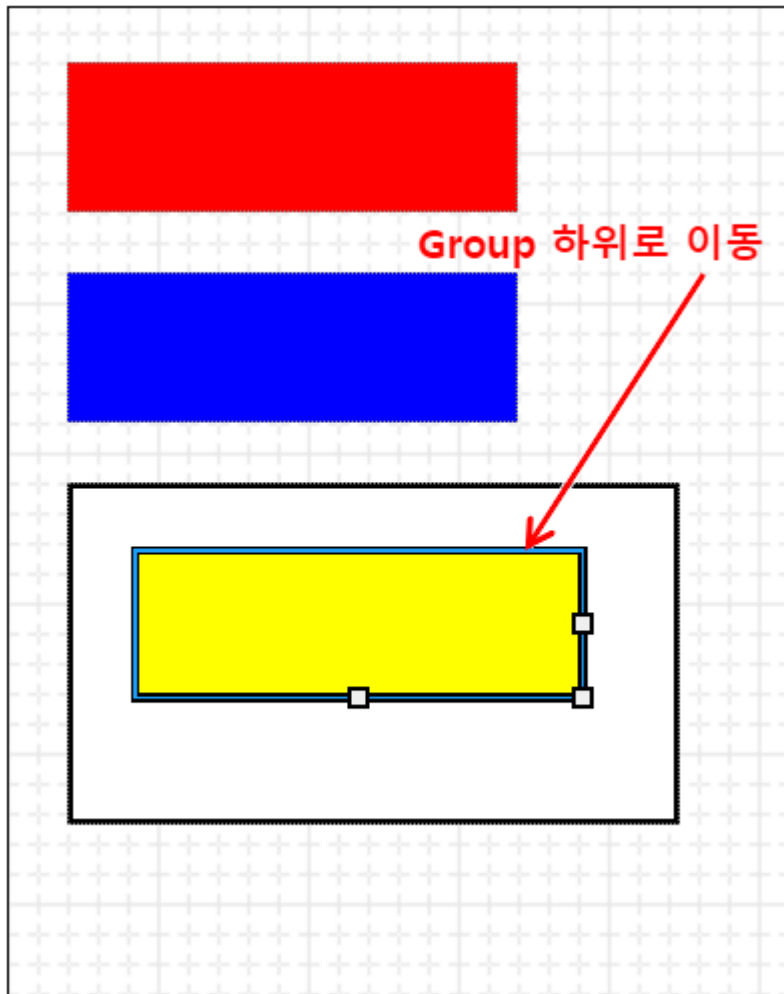
Group과 같이 하위에 컴포넌트를 포함할 수 있는 경우, 하위 이동도 가능합니다. 아래 그림은 TextBox를 Group 내부로 드래깅 하여 Group 하위로 이동하는 예시입니다.

그림 9-16 이동 전



아래 그림은 이동 후의 모습입니다. Group 하위로 이동된 것을 디자인 탭 및 Component 뷰에서 확인할 수 있습니다.

그림 9-17 이동 후



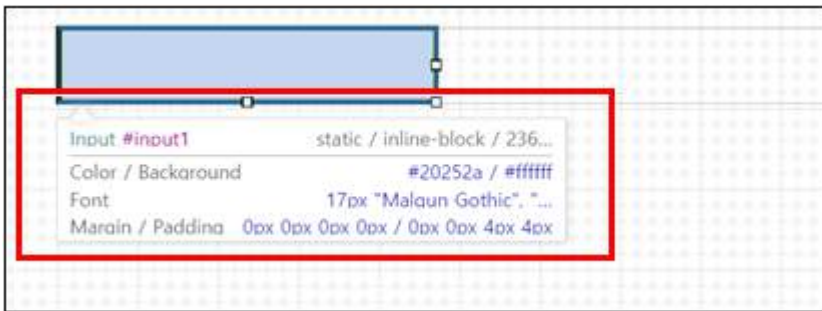
9.2 컴포넌트

9.2.1. 컴포넌트 정보

9.2.1.1. 툴팁

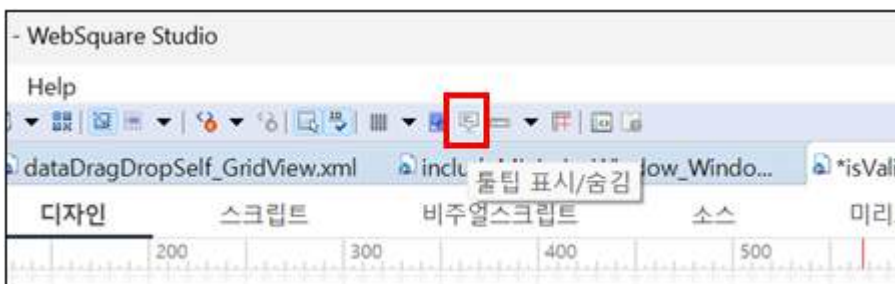
컴포넌트를 클릭하거나 컴포넌트 상에 마우스를 가져다 대면 아래와 같이 툴팁이 표시됩니다.

그림 9-18 컴포넌트 정보 표시



툴팁 표시/숨김 아이콘을 클릭하거나 **툴팁 표시/숨김** 키를 눌러 툴팁을 표시하거나 숨길 수 있습니다.

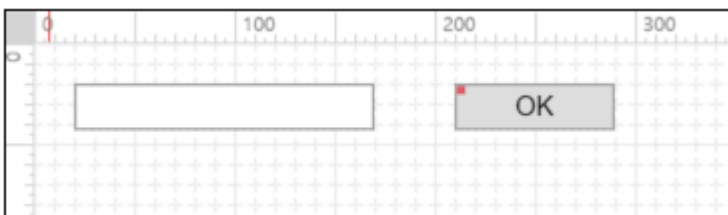
그림 9-19 툴팁 표시/숨김



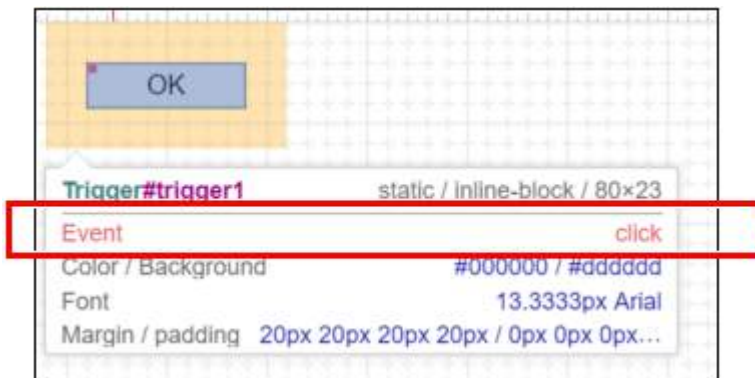
9.2.1.2. 이벤트 표시

컴포넌트에 이벤트 및 이벤트 함수가 정의된 경우 아래와 같이 좌측 상단에 붉은 점이 표시됩니다.

그림 9-20 이벤트가 추가된 컴포넌트 (OK 버튼)



마우스 커서를 올리면 툴팁이 표시되고 해당 이벤트 정보를 확인할 수 있습니다.



9.2.1.3. **ref** 바인딩 표시

컴포넌트가 **ref** 속성을 통해 바인딩된 경우, 아래와 같이 좌측 상단에 초록색 점이 표시됩니다. 이벤트가 존재할 경우, 빨간색 점과 함께 표시됩니다.

그림 9-21 이벤트 및 ref 바인딩 표시



마우스 커서를 올리면 툴팁이 표시되고 **ref** 속성을 통한 바인딩 정보를 확인할 수 있습니다.

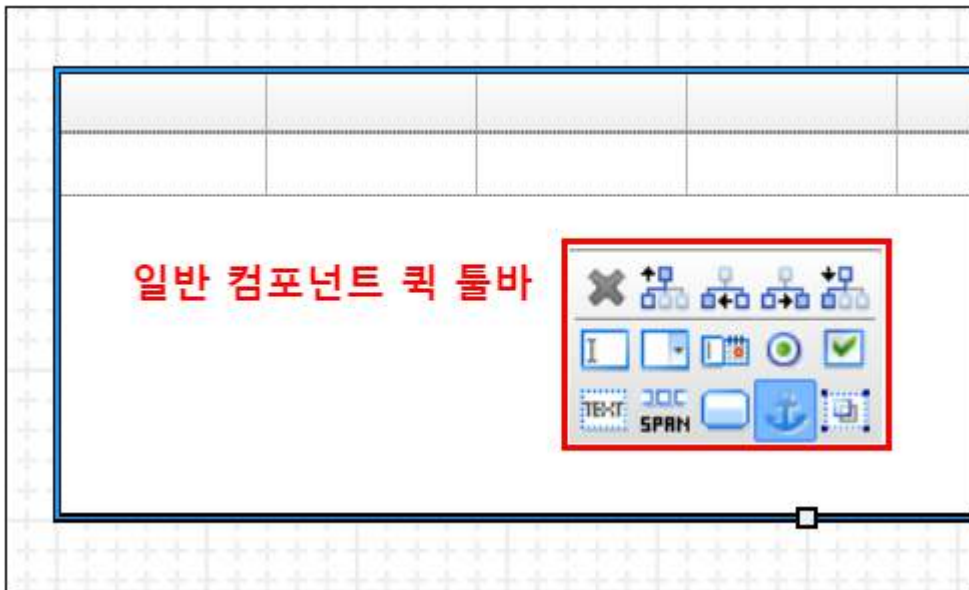
그림 9-22 이벤트 및 바인딩 정보 확인



9.2.2. 컴포넌트 킷 툴바

디자인 탭에서 특정 컴포넌트를 선택한 후 스페이스바를 누르면 아래와 같은 킷 툴바가 나타납니다. 사용자는 방향키를 사용하여 킷 툴바의 각 아이콘을 선택할 수 있습니다.

그림 9-23 컴포넌트 킷 툴바



GridView의 경우, 컬럼을 선택한 후 스페이스바를 누르면 아래와 같은 킷 툴바가 나타납니다. 위의 일반 컴포넌트 킷 툴바와 달리 GridView 컬럼의 `inputType` 속성을 변경할 수 있습니다.

그림 9-24 GridView 컬럼 킷 툴바



킷 툴바 상의 각 아이콘에 대한 설명은 아래와 같습니다.

Table 9-1 킷 툴바 설명

아이콘	설명	
	컴포넌트 킷 툴바	GridView 컬럼 킷 툴바
	킷 툴바 닫기.	(동일.)
	현재 선택한 컴포넌트를 기준으로 관련 컴포넌트 선택.	

아이콘	설명	
	컴포넌트 킷 톨바	GridView 컬럼 킷 톨바
	- 부모 선택. - 왼쪽 형제 선택. - 오른쪽 형제 선택. - 자식 선택.	(동일.)
	InputBox 컴포넌트 추가.	(해당 없음.)
	SelectBox 컴포넌트 추가.	GridView 컬럼에 <code>inputType="select"</code> 적용.
	InputCalendar 컴포넌트 추가.	GridView 컬럼에 <code>inputType="inputcalendar"</code> 적용.
	Radio 컴포넌트 추가.	GridView 컬럼에 <code>inputType="radio"</code> 적용.
	CheckBox 컴포넌트 추가.	GridView 컬럼에 <code>inputType="checkbox"</code> 적용.
	TextBox 컴포넌트 추가.	(해당 없음.)
	Span 컴포넌트 추가.	(해당 없음.)
	Trigger 컴포넌트 추가.	GridView 컬럼에 <code>inputType="button"</code> 적용.
	Anchor 컴포넌트 추가.	GridView 컬럼에 <code>inputType="link"</code> 적용.
	Group 컴포넌트 추가.	(해당 없음.)
	(해당 없음.)	GridView 컬럼에 <code>inputType="image"</code> 적용.
	(해당 없음.)	GridView 컬럼에 <code>inputType="textImage"</code> 적용.
	(해당 없음.)	GridView 컬럼에 <code>inputType="custom"</code> 적용.

9.2.3. 표 편집

GridView나 TableLayout과 같은 표 형식 컴포넌트의 경우, 아래의 아이콘을 사용하여 표 편집을 쉽게할 수 있습니다.



(1) Insert Column into Left **CTRL + Shift + L**

왼쪽에 열 삽입

(2) Insert Column into Right **CTRL + Shift + R**

오른쪽에 열 삽입

(3) Add Row **CTRL + Shift + R**

행 추가

(4) Delete Column **CTRL + Alt + C**

열 삭제

(5) Delete Row **CTRL + Alt + R**

행 삭제

(6) Merge Cell **CTRL + M**

셀 병합

(7) Divide Cell **CTRL + Shift + M**

병합 해제

(8) Move Column Left

왼쪽으로 열 이동

(9) Move Column Right

오른쪽으로 열 이동

9.2.4. 컴포넌트 복사

디자인 탭에서 **CTRL+C**와 **CTRL+V**를 사용하여 컴포넌트의 복사&붙여넣기가 가능합니다. 단축키는 아래와 같습니다.

Table 9-2 복사 관련 단축키

기능	단축키
부모 컴포넌트 선택	F2
복사	CTRL+C
붙여넣기	CTRL+V
잘라내기	CTRL+X
컴포넌트 앞에 붙여넣기	CTRL+Shift+V
컴포넌트 뒤에 붙여넣기	CTRL+ALT+V

9.3 컨텍스트 메뉴

9.3.1. 메뉴 구성

컴포넌트를 우클릭하면 아래와 같이 컨텍스트 메뉴가 표시됩니다.

- ❶ 모든 컴포넌트에 공통적으로 적용되는 메뉴입니다.
- ❷ 현재 선택한 컴포넌트에 적용 가능한 메뉴만 표시됩니다.



다음은 GridView를 우클릭할 때 표시되는 컨텍스트 메뉴입니다.

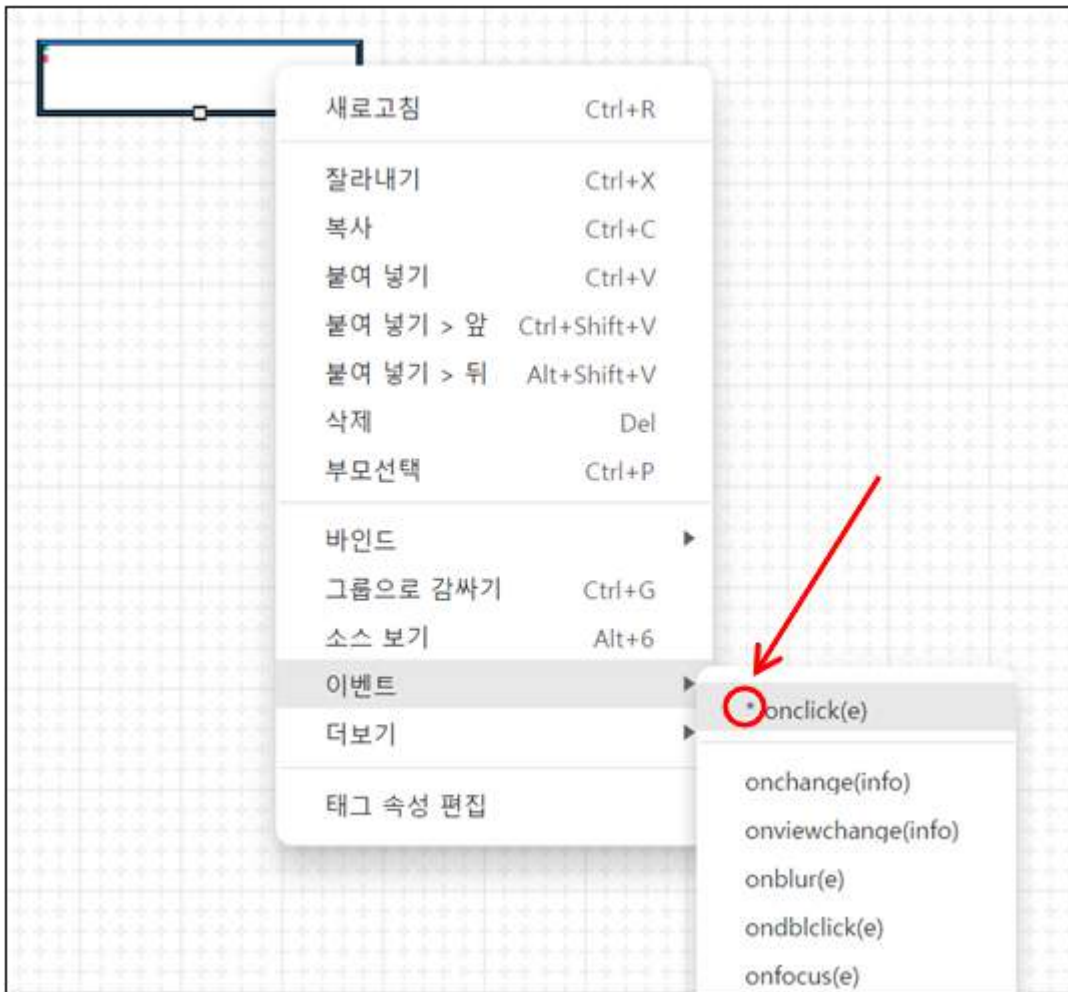
그림 9-25 GridView 컨텍스트 메뉴



9.3.2. 등록 이벤트 표시

컴포넌트 우클릭 후 **이벤트** - 이벤트 종류를 선택하여 이벤트를 추가할 수 있습니다. 단, 이미 추가된 이벤트는 아래와 같이 " * " 기호와 함께 이벤트 목록 상단에 표시됩니다.

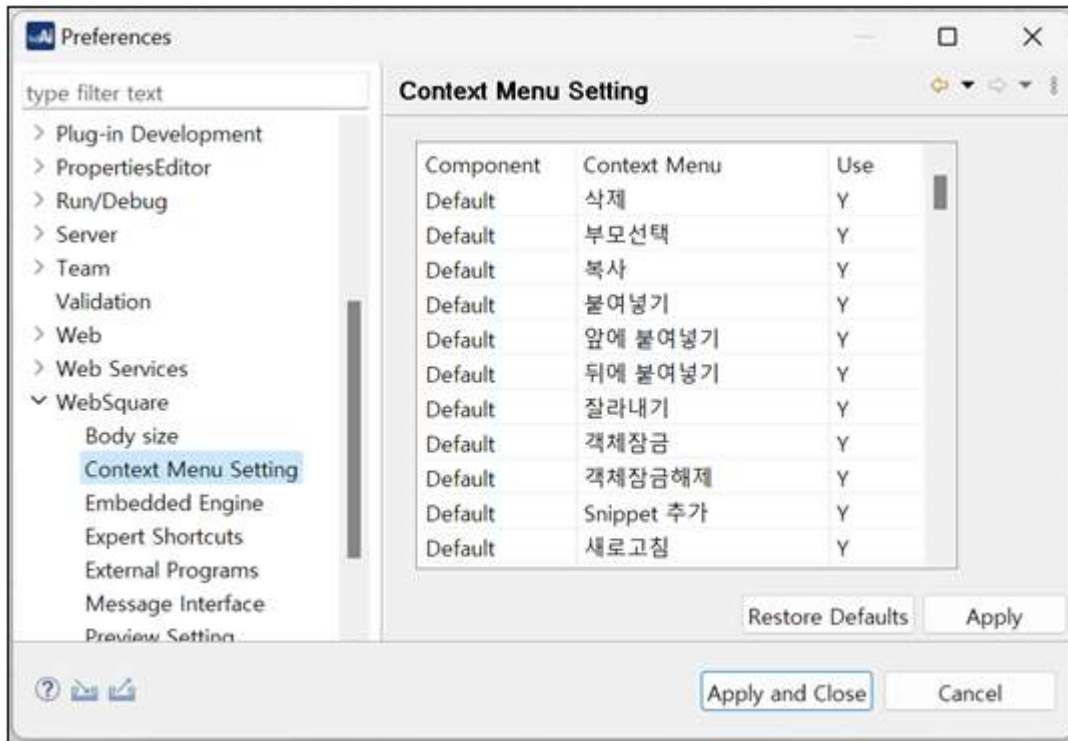
그림 9-26 이벤트 추가



9.3.3. 컨텍스트 메뉴 편집

1. **Window - Preferences - WebSquare - Context Menu Setting**을 선택합니다.
2. 마우스 우클릭 시 표시되는 컨텍스트 메뉴의 보임/숨김을 지정할 수 있습니다.

그림 9-27 컨텍스트 메뉴 설정



Studio 재시작

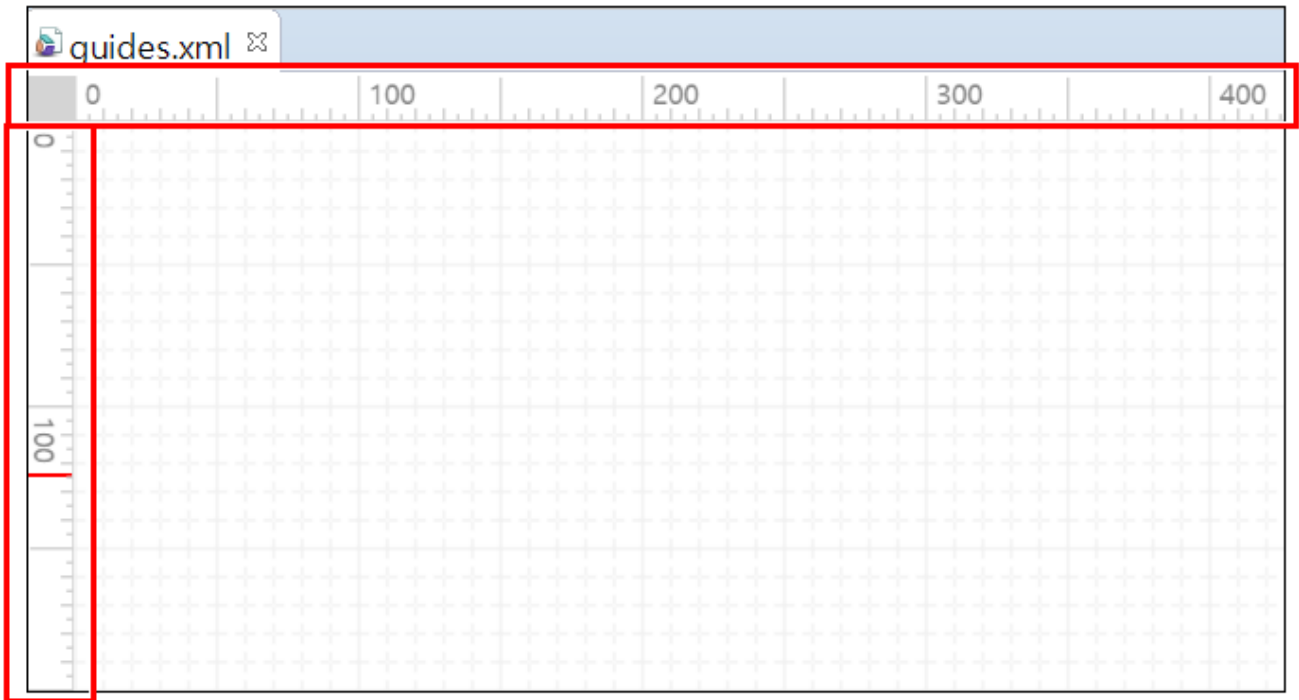
컨텍스트 메뉴 변경 후 Studio를 재시작하십시오.
Studio 재시작 후, 새로 편집한 컨텍스트 메뉴가 적용됩니다.

9.4 눈금선, 눈금자, 안내선

9.4.1. 눈금자 (Ruler)

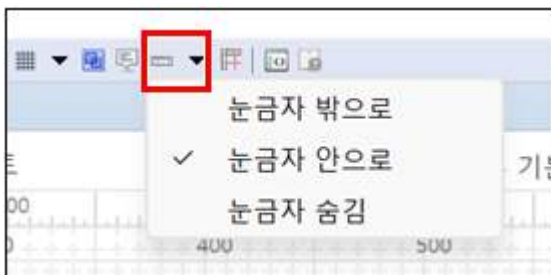
눈금자는 아래와 같이 디자인 탭 외곽에 표시됩니다.

그림 9-28 눈금자 밖으로



눈금자 표시 변경 아이콘을 사용하여 눈금자를 표시하거나 숨길 수 있습니다.

그림 9-29 눈금자 표시하기/숨기기



9.4.2. 눈금선 (Gridlines)

눈금선은 디자인 탭 바탕에 격자 무늬로 표시됩니다.

눈금선 표시 변경 아이콘을 선택하여 눈금선을 표시하거나 숨길 수 있습니다

그림 9-30 눈금선 표시/숨김

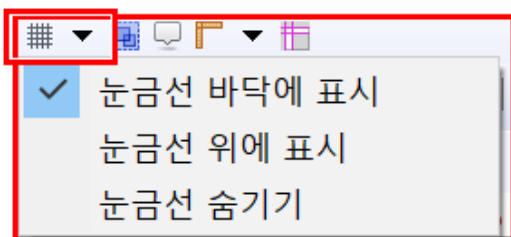
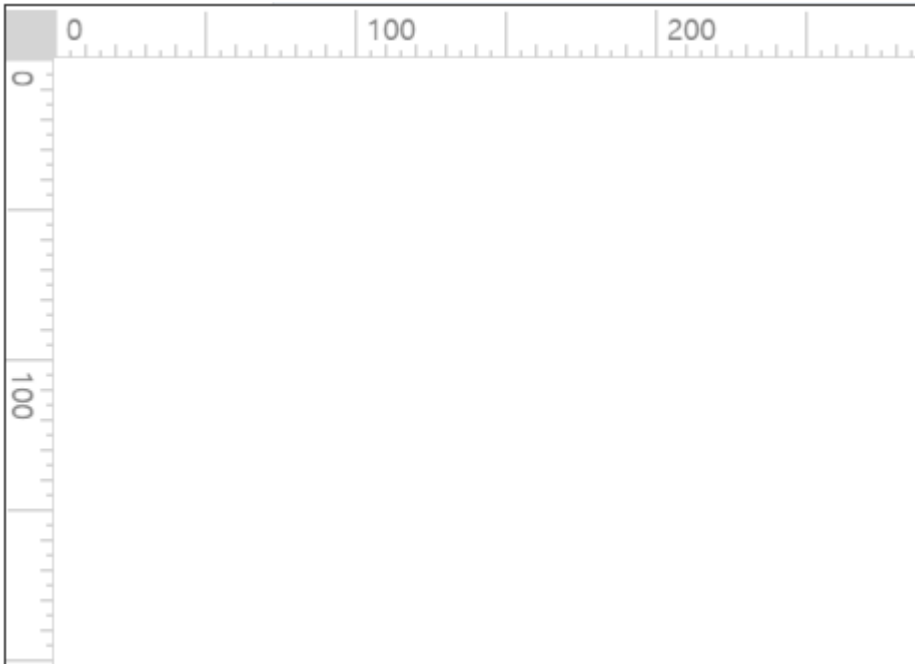


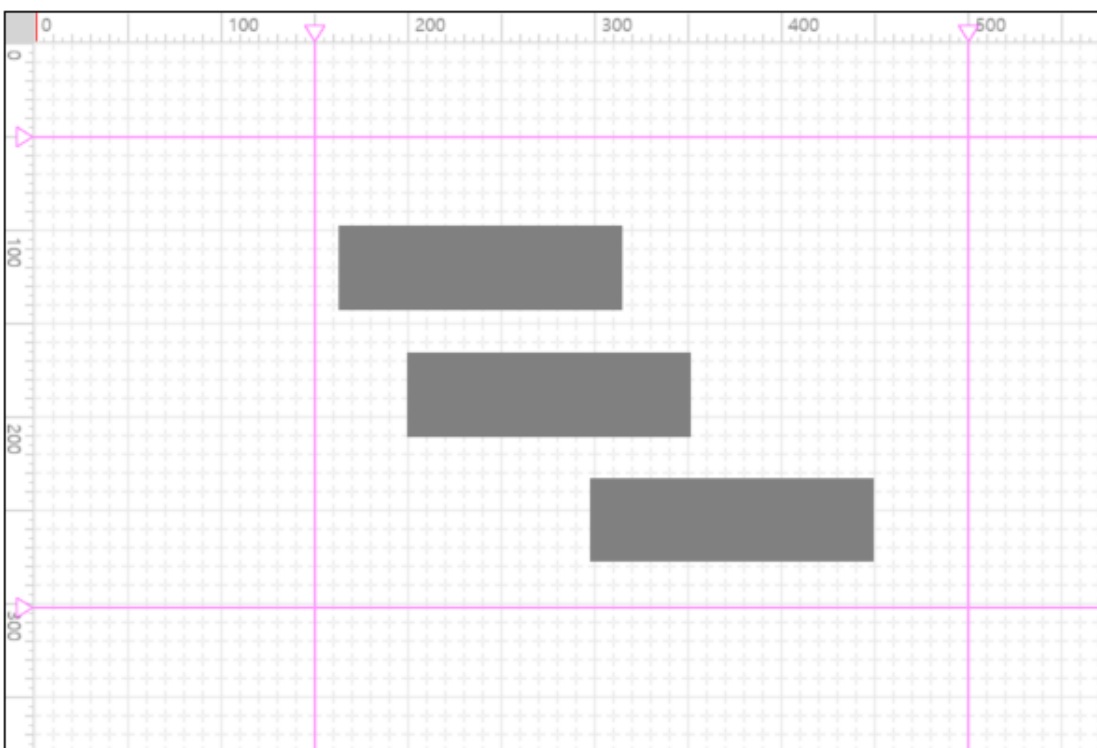
그림 9-31 눈금선 숨기기



9.4.3. 안내선 (Guides)

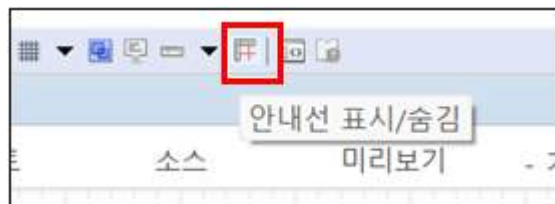
눈금자 위의 원하는 지점에서 클릭할 경우, 아래와 같이 안내선을 생성할 수 있습니다. 이미 생성한 안내선을 다시 클릭하면 안내선을 삭제할 수 있습니다.

그림 9-32 안내선

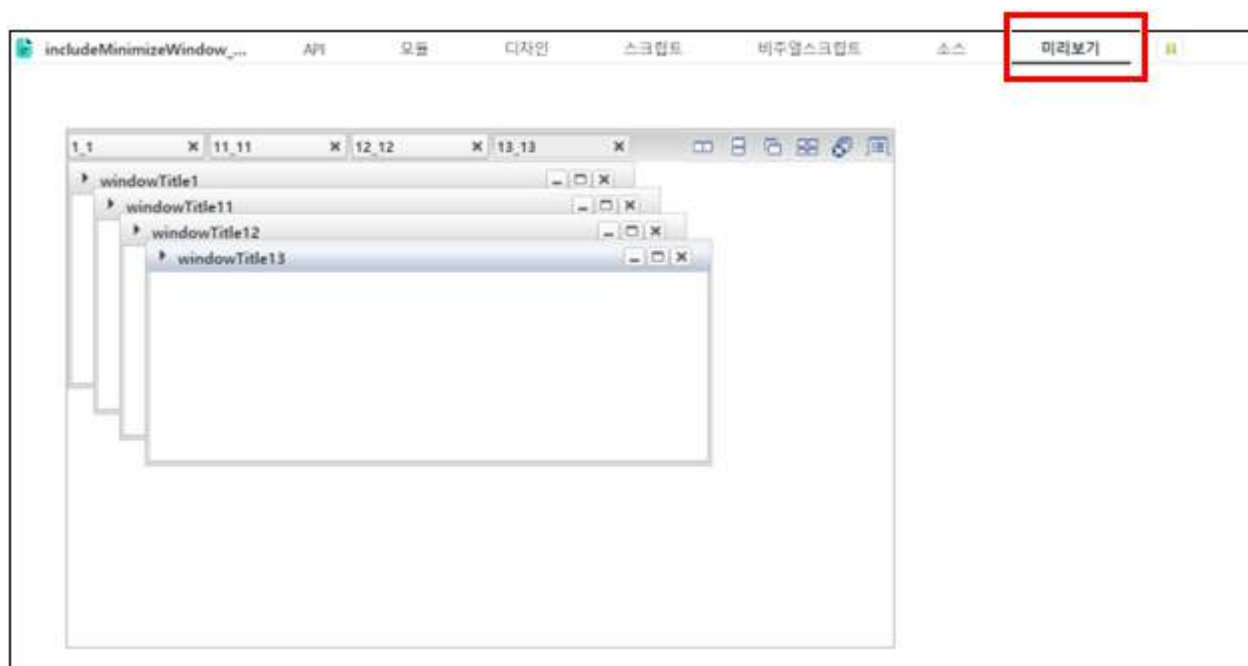


안내선 표시/숨김 아이콘을 클릭하여 안내선을 숨기거나 표시할 수 있습니다.

그림 9-33 안내선 표시/숨김



9.5 미리 보기



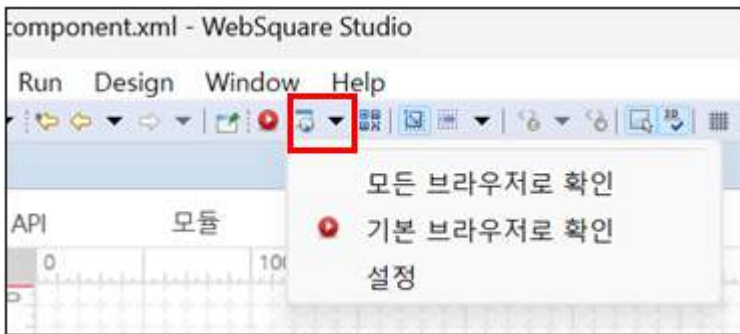
9.6 브라우저로 보기

빨간색 화살표 모양의 아이콘을 클릭하면 페이지 컴포넌트가 렌더링된 모습을 사전에 설정한 기본 브라우저에서 직접 확인할 수 있습니다.

그림 9-34 기본 브라우저로 확인



그림 9-35 브라우저로 확인



모든 브라우저로 확인

[Windows] - [Preferences] - [External Programs]에 추가한 모든 브라우저에서 개발자가 작업한 파일을 표시합니다.

기본 브라우저로 확인

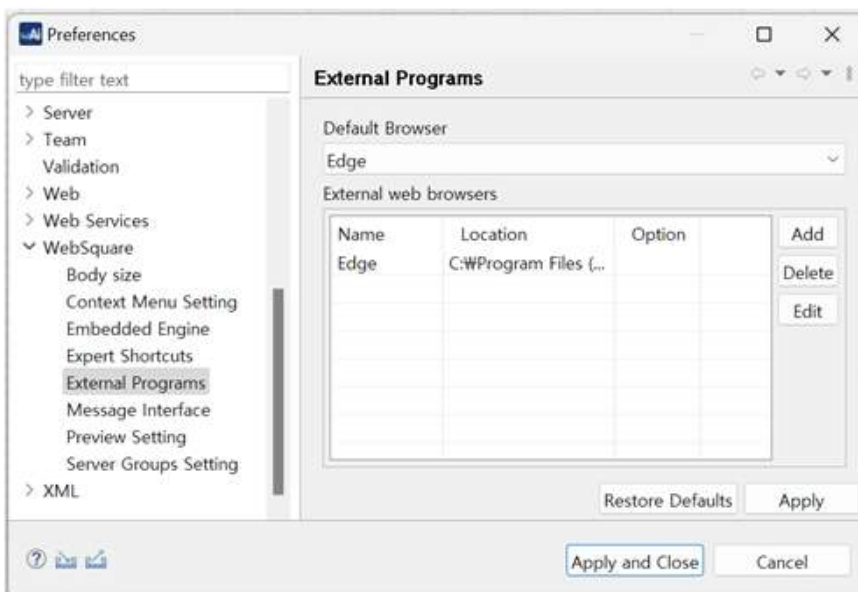
개발자가 PC에 설정한 기본 브라우저에서 개발자가 작업한 파일을 표시합니다.

설정

기본 브라우저를 설정할 수 있습니다.

또한 페이지 컴포넌트를 확인할 브라우저는 **브라우저 확인 - 설정** 아이콘을 클릭하여 브라우저를 설정하십시오.

그림 9-36 브라우저 등록

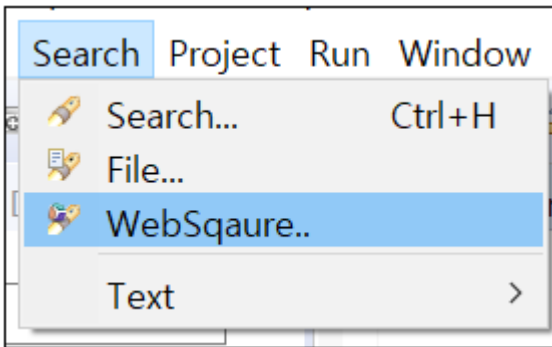


9.7 파일 검색

WebSquare Search 기능을 이용하여 WebSquare 페이지 파일을 효율적으로 검색할 수 있습니다.

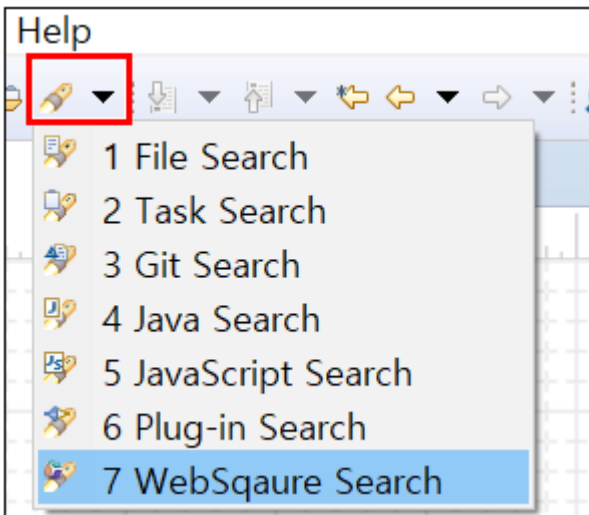
Search - WebSquare 메뉴를 선택하십시오.

그림 9-37 Search - WebSquare



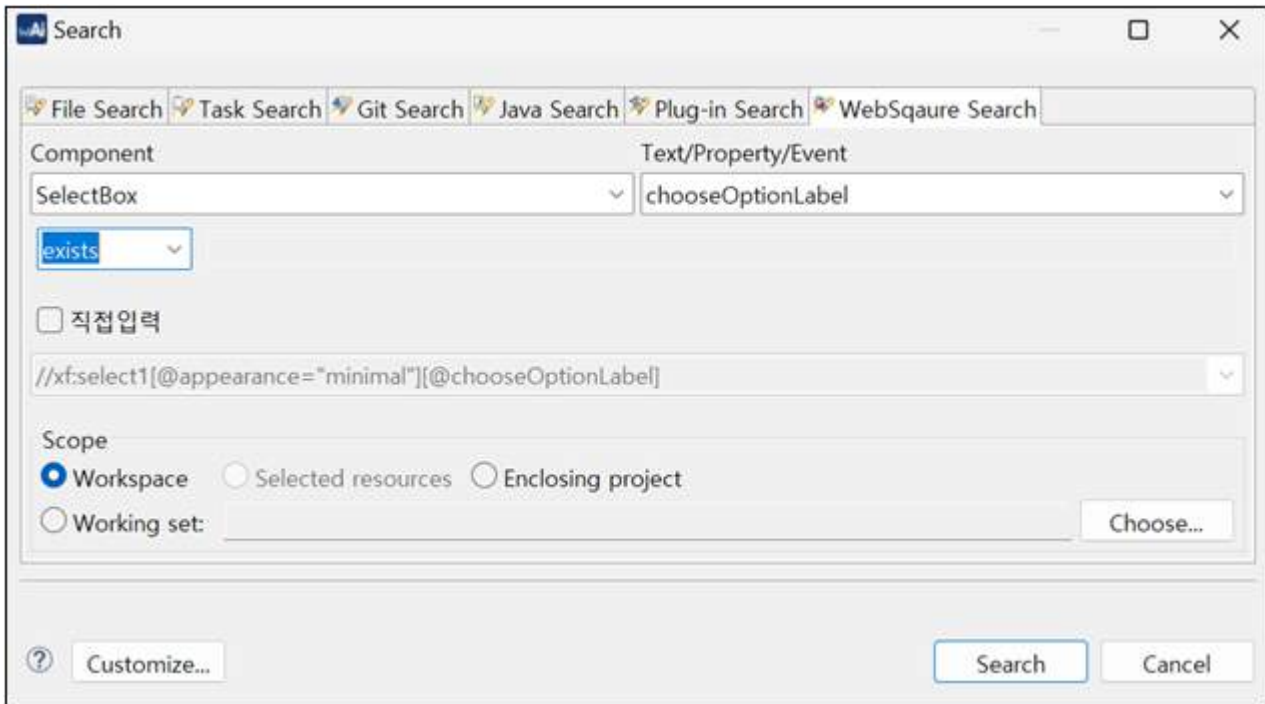
혹은, 아래와 같이 툴바의 아이콘을 클릭하여 **WebSquare Search**를 선택하십시오.

그림 9-38 웹스퀘어 검색



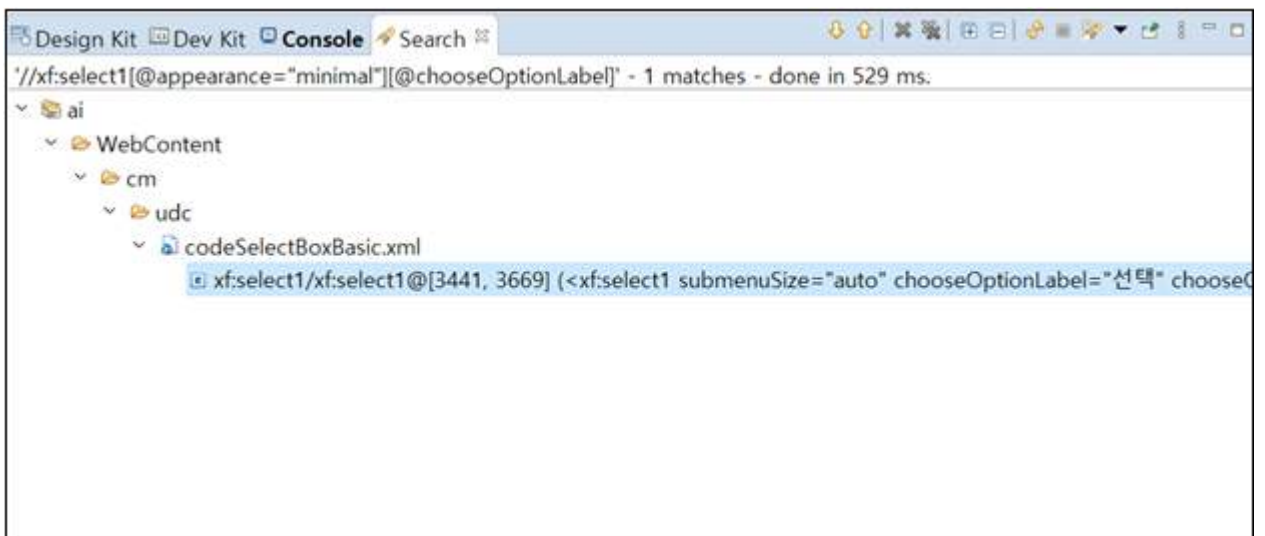
다음은 **SelectBox**를 검색하되, SelectBox의 `chooseOptionLabel` 속성 값이 "city"를 포함한 WebSquare 페이지 파일을 찾는 예제입니다.

그림 9-39 WebSquare 페이지 파일 찾기 예



아래와 같이 Search 뷰에 검색 결과가 표시되고, 파일을 클릭하여 해당 부분을 확인할 수 있습니다.

그림 9-40 WebSquare 페이지 검색 결과



9.8 기타

9.8.1. 고유 ID 부여

새로 생성되는 컴포넌트에 기존 ID와 중복되지 않는 ID를 부여하는 기능입니다. 초기 상태에서 이 기능은 비활성화되어 있습니다. 아래와 같은 방법으로 본 기능을 사용할 수 있습니다.



컴포넌트 복사

복사를 통해 컴포넌트를 새로 생성할 경우 **컴포넌트 복사 ID 자동 생성** 기능을 사용하십시오. 동일한 ID의 컴포넌트가 여러 개 존재할 경우, 미리보기를 할 수 없습니다.

9.8.2. 디자인 탭 확대/축소

디자인 탭 우측 상단에서 **디자인 탭** 화면을 확대하거나 축소할 수 있습니다.

그림 9-41 디자인 탭 확대 및 축소



9.8.3. 청사진

컴포넌트가 겹쳐져 있거나 화면이 복잡하게 구성된 경우, 청사진 기능을 사용하십시오. 아래와 같이 컴포넌트의 가시성이 높아집니다.

그림 9-42 청사진 기능을 사용하지 않은 모습

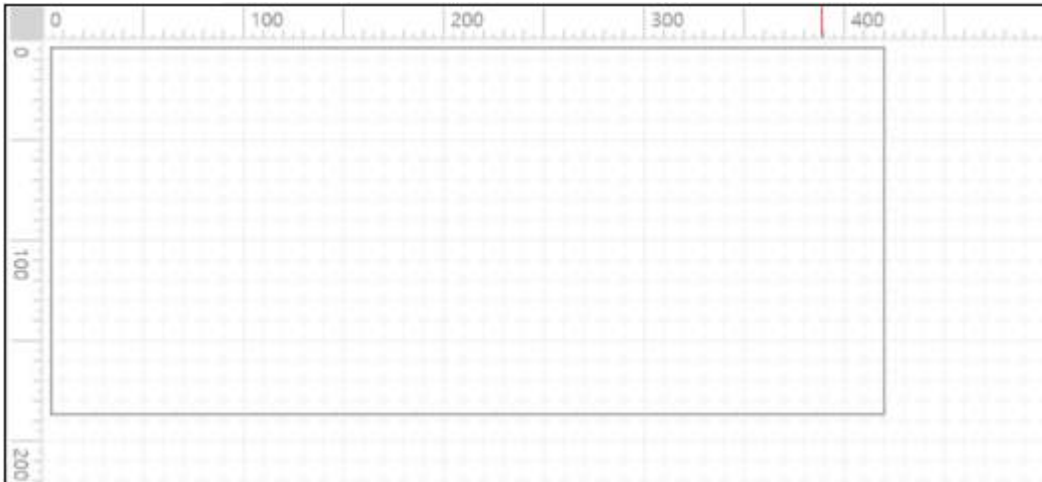


그림 9-43 청사진 기능을 적용한 모습

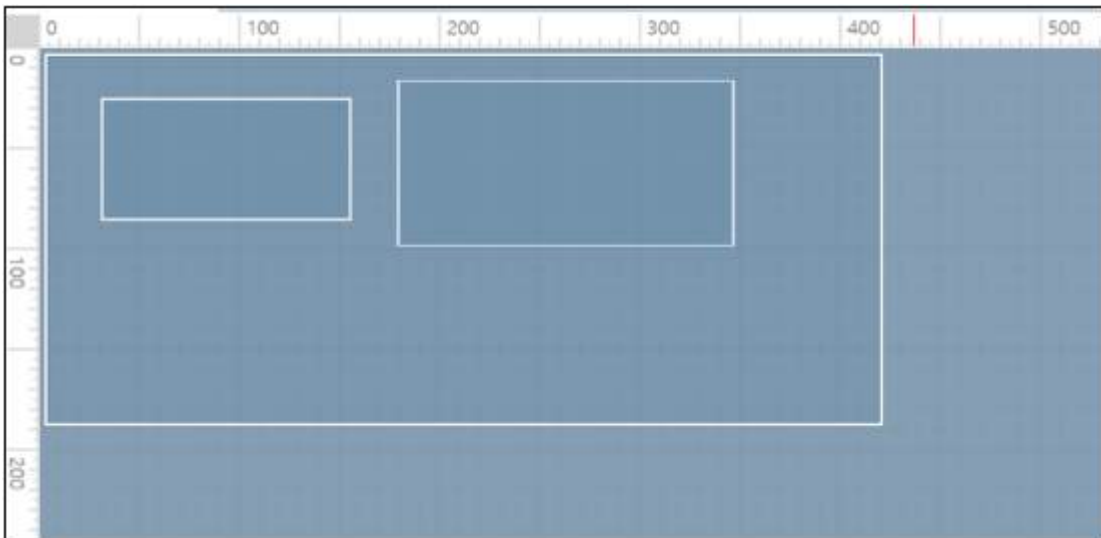


그림 9-44 청사진 아이콘



9.8.4. 신규 아이콘

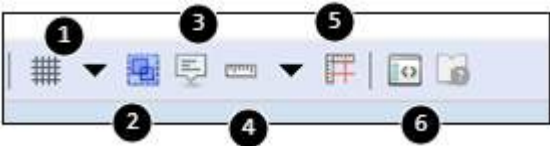
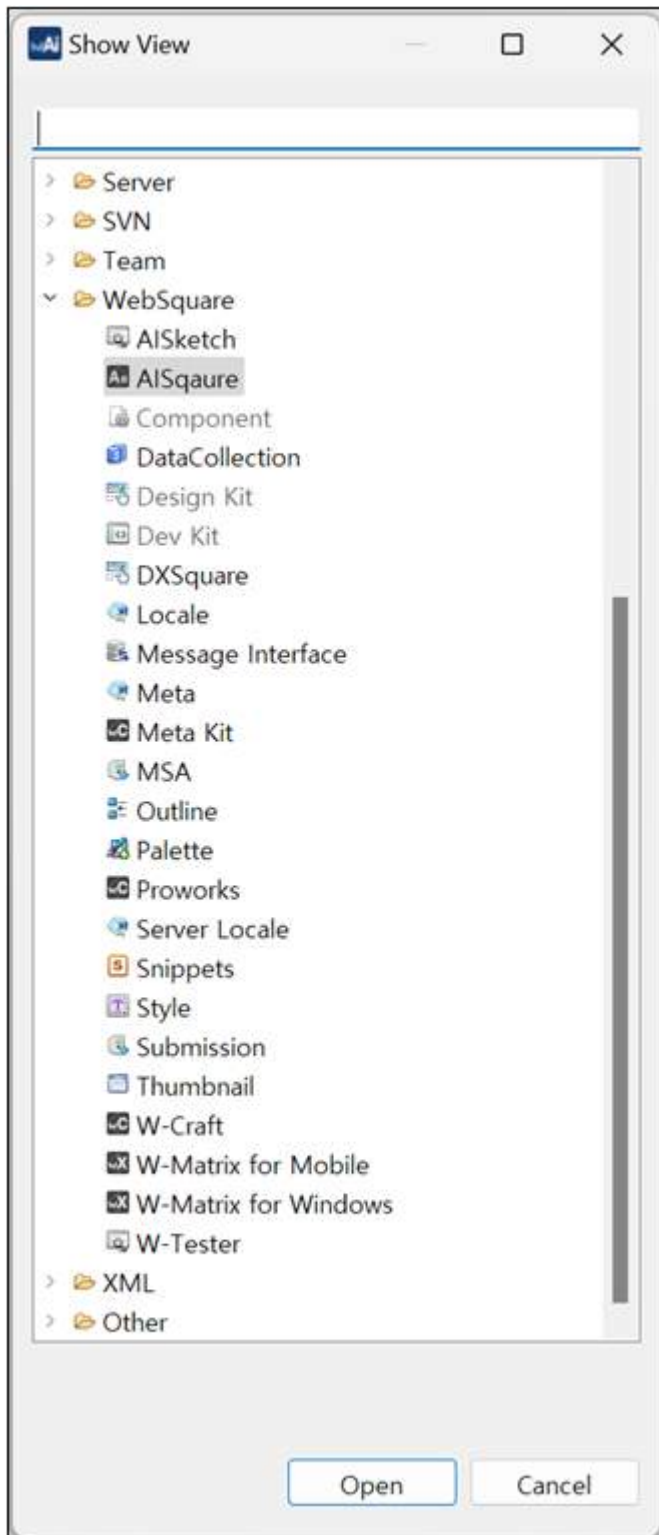


Table 9-3 신규 아이콘

아이콘	설명
1	9.4.2. 눈금선 (Gridlines)
2	9.8.3. 청사진
3	9.2.1.1. 툴팁
4	9.4.1. 눈금자 (Ruler)
5	9.4.3. 안내선 (Guides)
6	7.8 Dev Kit
	9.2.2. 컴포넌트 킵 툴바 (일반 컴포넌트)
	9.2.2. 컴포넌트 킵 툴바 (GridView 컬럼)

9.8.5. SVN 및 Git 플러그인



웹스퀘어 AI Studio는 SVN 플러그인과 Git 플러그인을 포함합니다. 아래와 같이 **Window - Show View**를 선택 후, SVN 혹은 Git을 검색해서 사용할 수 있습니다.

그림 9-45 플러그인 찾기

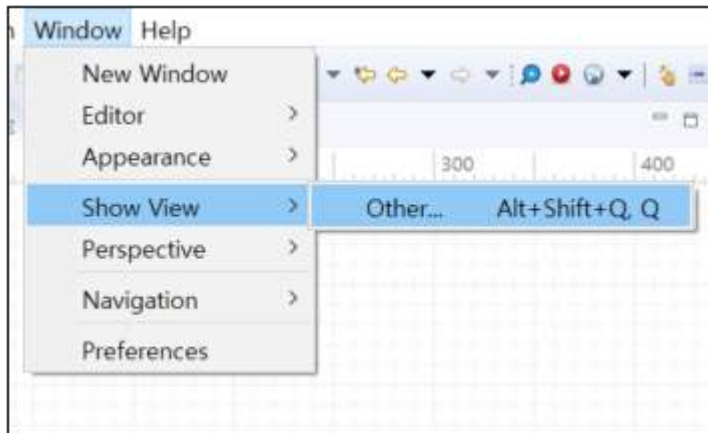
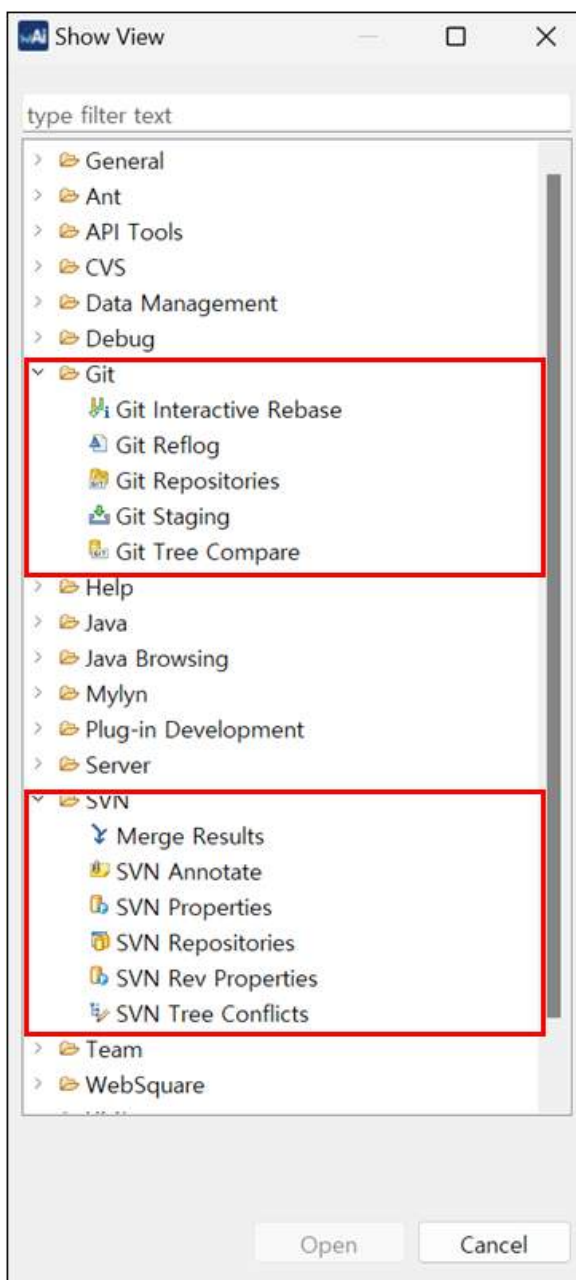


그림 9-46 SVN 및 Git 플러그인



9.8.6. 메시지 표시

작업 상태가 변경되거나 오류가 발생할 경우 이를 알려주는 메시지가 스튜디오 우측 하단에 표시됩니다.

9.8.6.1. 저장 확인 메시지

그림 9-47 저장 확인 메시지



9.8.6.2. 오류 메시지

그림 9-48 오류 메시지



그림 9-49 오류 메시지



Part III. 개발 프로세스

10. 프로젝트 생성

10.1 WebSquare Project

그림 10-1 YouTube 동영상 (<https://youtu.be/ROf-Xqbg2TY>)

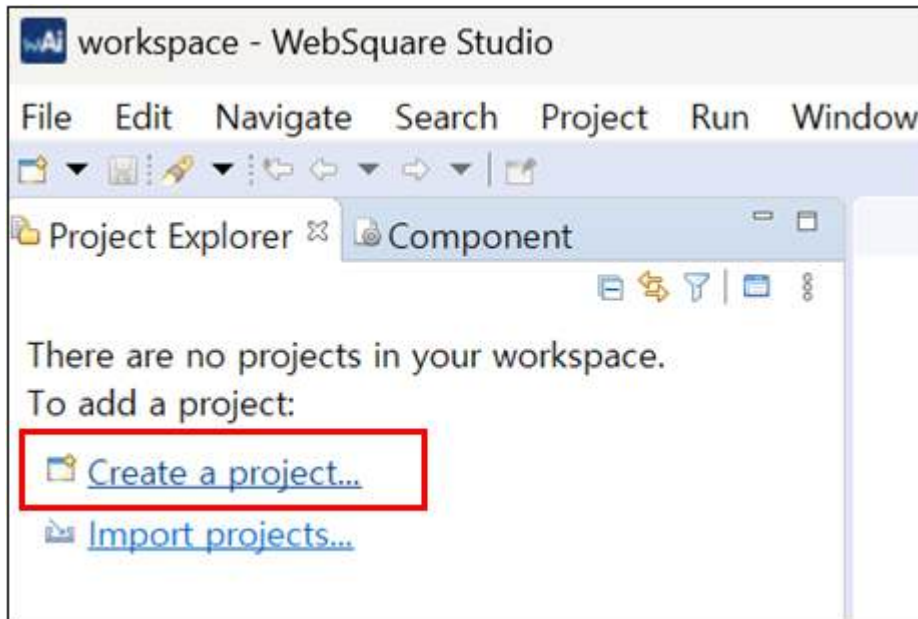


10.1.1. 생성

스튜디오를 시작한 후, 웹스퀘어 프로젝트를 아래의 방법으로 생성하십시오.

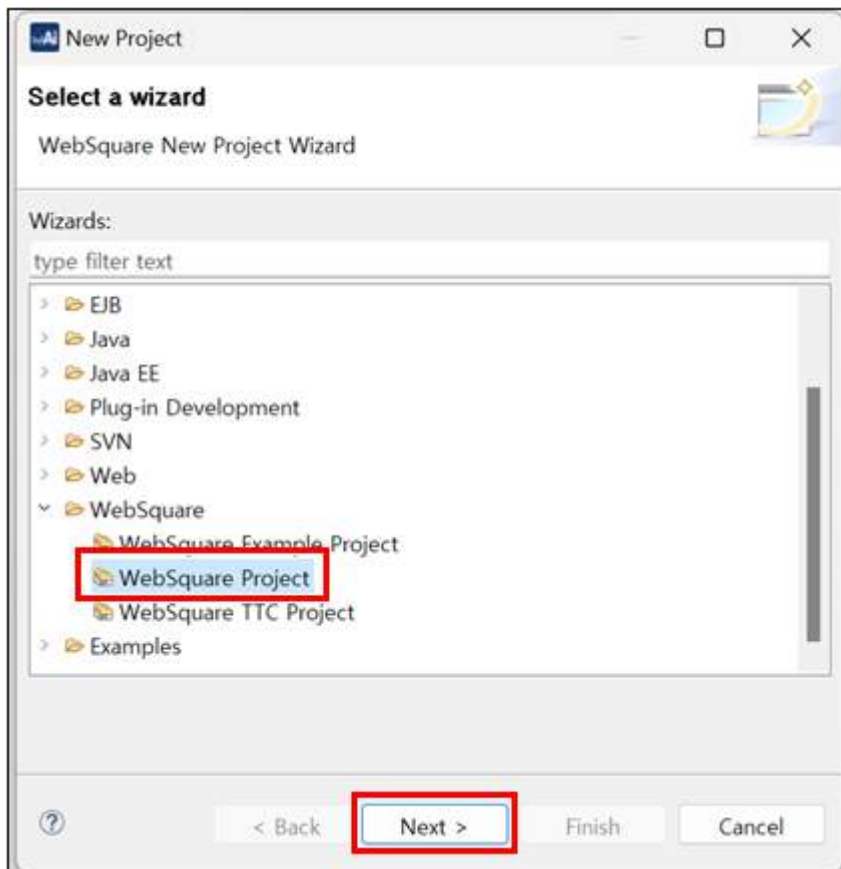
1. Create a project를 선택합니다.

그림 10-2 프로젝트 생성



2. **WebSquare - WebSquare Project**를 선택합니다.
3. **Next**를 클릭합니다.

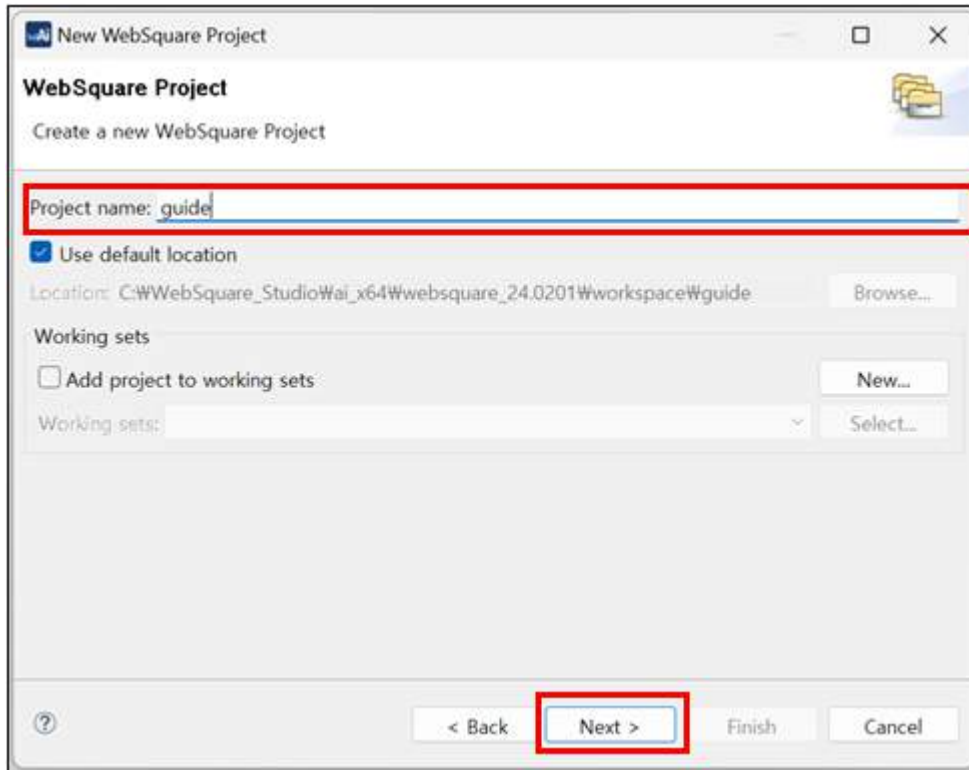
그림 10-3 프로젝트 생성



(혹은, Project Explorer를 우클릭한 후, **New - WebSquare Project**를 선택합니다.)

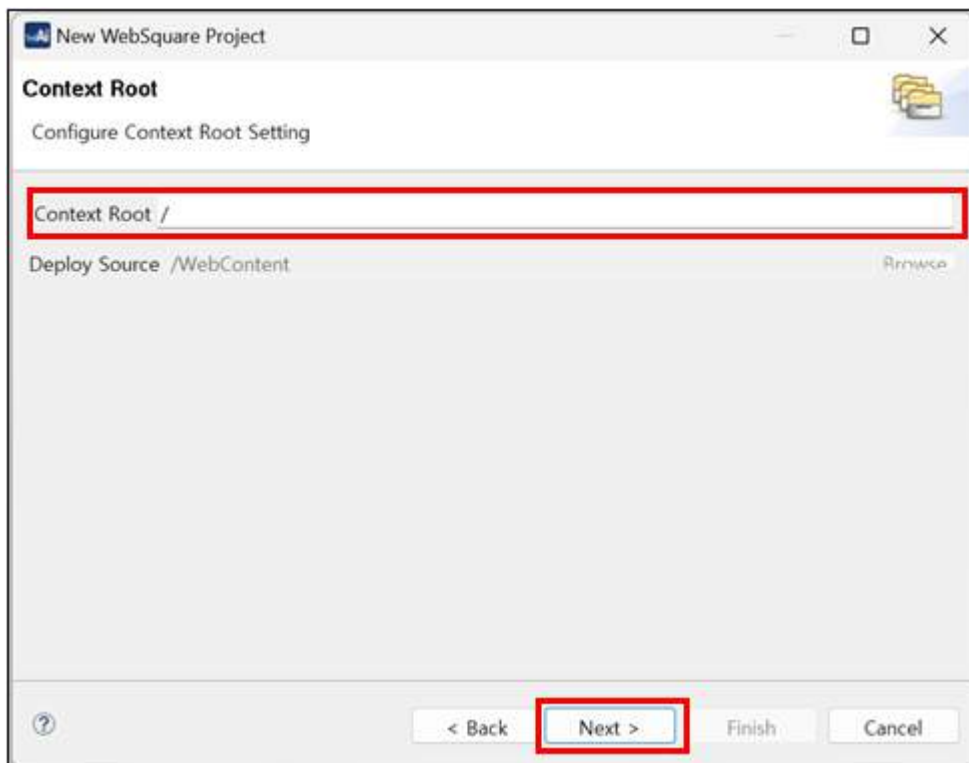
4. 프로젝트 이름을 입력하고 **Next** 버튼을 클릭합니다.

그림 10-4 프로젝트 이름 입력



5. Context Root를 지정합니다. 기본 값은 루트("/ ")입니다.

그림 10-5 Context Root 지정



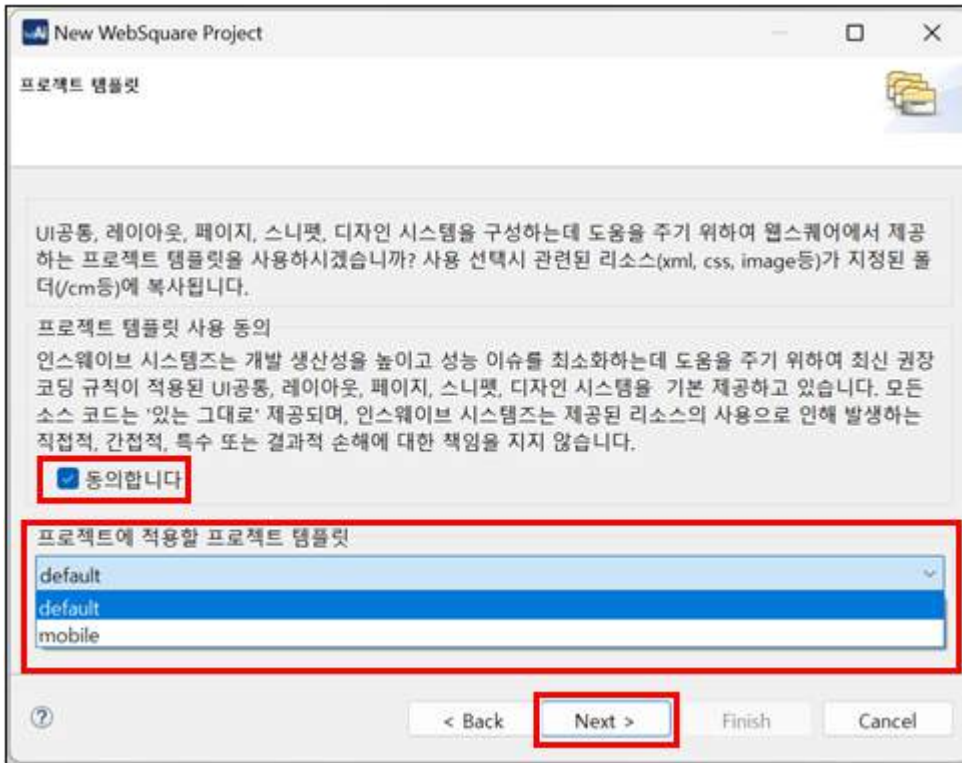


Context Root

Context Root는 *client.config.xml* 및 *server.config.xml* 설정 파일에 영향을 줍니다.

6. 동의합니다 항목을 선택하고, Next를 클릭합니다. (권장)

그림 10-6 프로젝트 템플릿 사용 동의



WRM 컴포넌트 사용 동의

웹스퀘어 AI는 공통 업무용 UDC(User Developed Component)를 WRM 컴포넌트로 제공합니다. WRM 컴포넌트 사용에 동의할 경우, 웹스퀘어 AI가 제공하는 WebSquare Reference model이 “/WebContent/cm” 폴더에 자동으로 생성됩니다.

단, 사용자가 WRM 컴포넌트를 수정할 경우, (주)인스웨이브시스템즈는 정상 작동을 보장하지 않습니다.

Table 10-1 프로젝트에 적용할 WRM 컴포넌트

default	공통업무용 UDC
mobile	모바일 환경에 특화된 공통업무용 UDC

7. Preview 설정 화면이 표시되면, Finish 버튼을 클릭합니다.

그림 10-7 Preview 설정

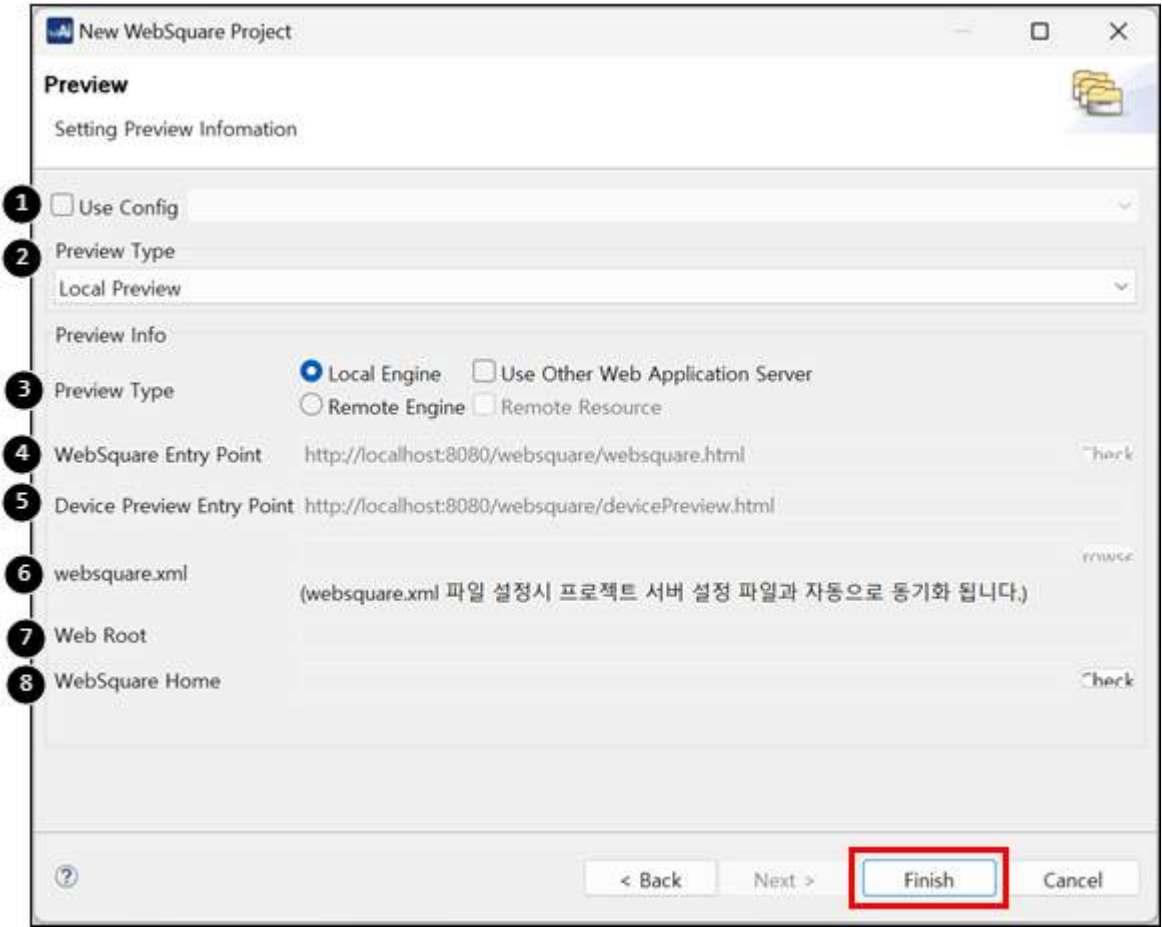


Table 10-2 Preview 항목 설명

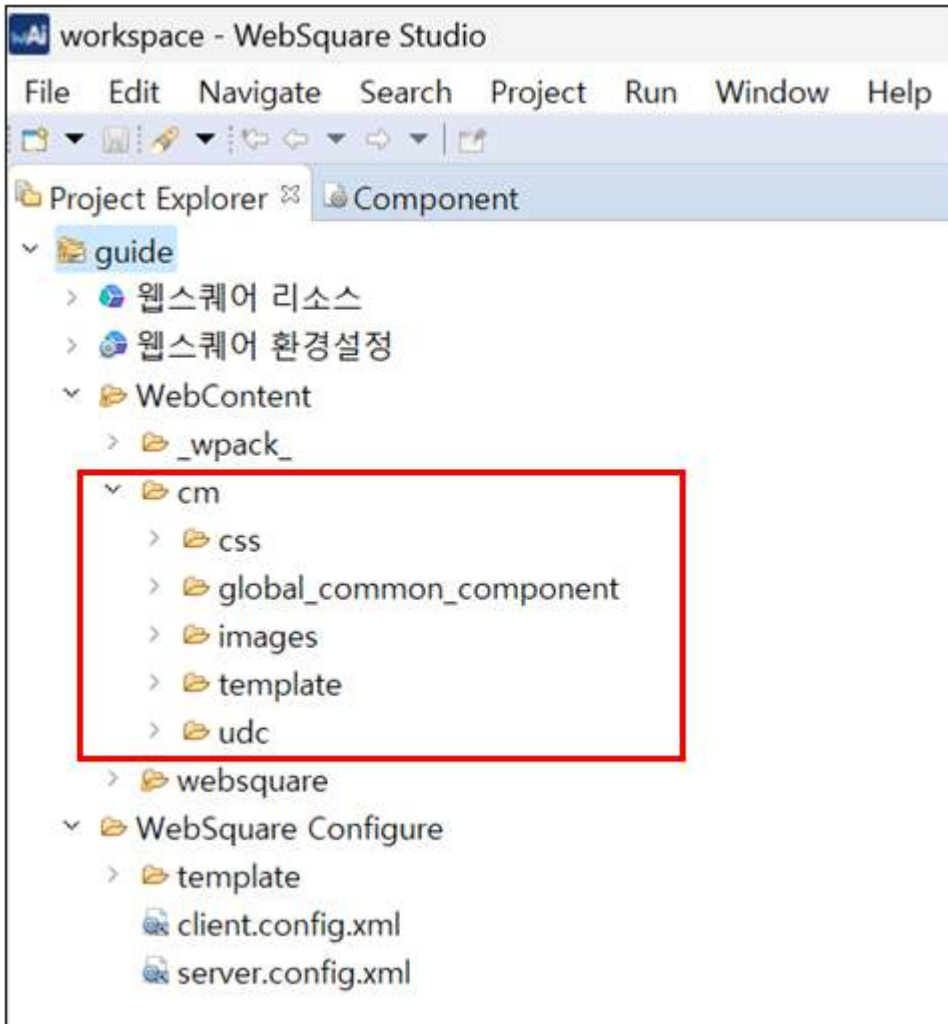
1 Use Config		
2 Preview Type		• Local Preview • Remove CIFS Preview • Remote FTP Preview • Remote SFTP Preview
Preview Info	3 Preview Type	• Local Engine • Use Other Web Application Server • Remote Engine • Remote Resource
	4 WebSquare Entry Point	
	5 Device Preview Entry Point	
	6 websquare.xml	
	7 Web Root	
	8 WebSquare Home	



서버 환경 설정 파일 편집 시 동기화할 websquare.xml 파일의 위치를 지정할 수 있습니다.
단, **Use Other Web Application Server** 옵션이 선택된 경우에만 활성화됩니다.

WRM 컴포넌트 사용에 동의한 경우, **WebContent** 폴더 아래에 공통 리소스를 포함한 **cm** 폴더가 생성됩니다.

그림 10-8 프로젝트 폴더



단, WRM 컴포넌트 사용에 동의하지 않을 경우, **WebContent** 폴더 아래에 추가 리소스를 포함하는 **cm** 폴더가 생성되지 않습니다.

10.1.2. 폴더 구조

다음은 웹스퀘어 프로젝트를 생성한 모습입니다. 폴더를 통해 공통 모듈을 포함한 웹스퀘어 AI의 기능을 직관적으로 확인할 수 있습니다.

그림 10-9 폴더 구조

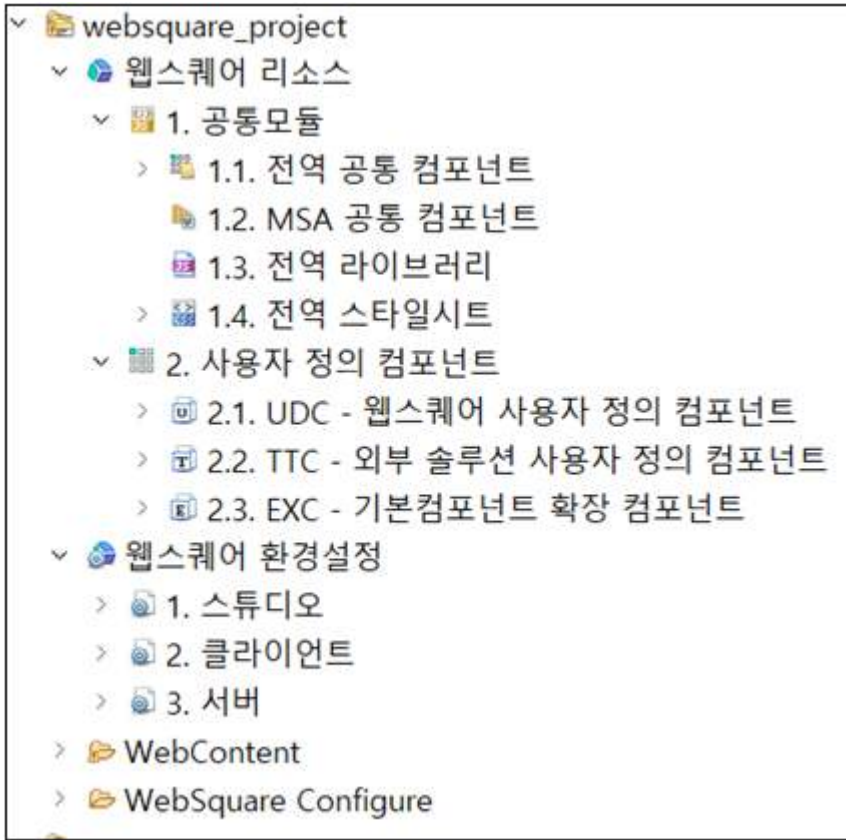


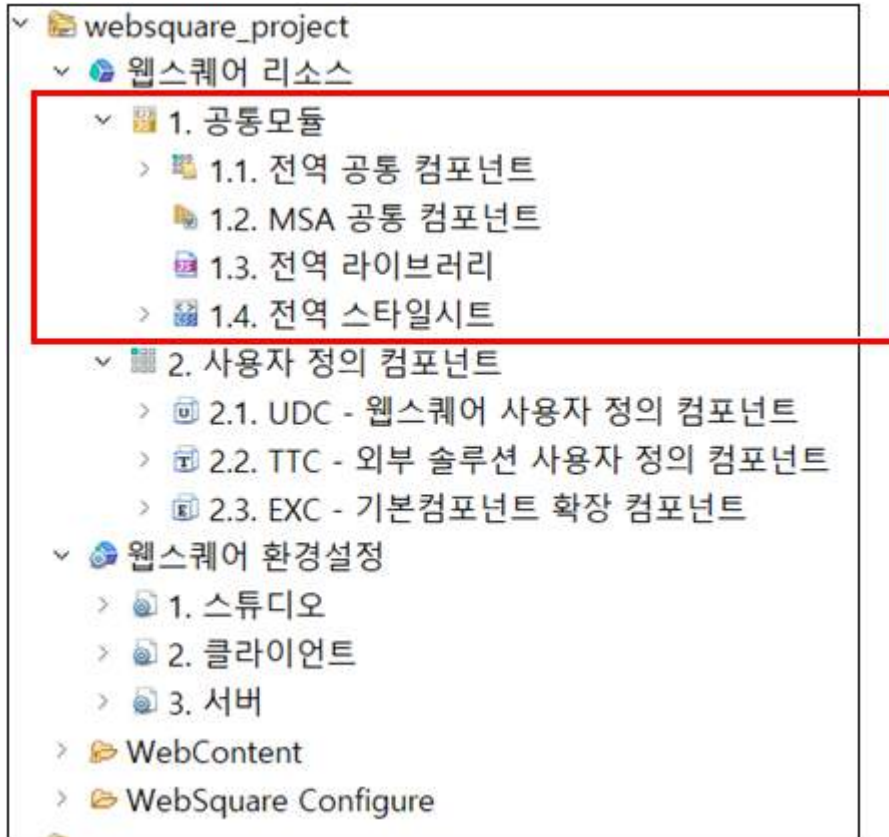
Table 10-3 폴더 설명

폴더			설명
웹스퀘어 리소스			공통 모듈 및 사용자 정의 컴포넌트를 표시.
웹스퀘어 환경설정			스튜디오, 클라이언트, 서버 버튼을 클릭하면 각 항목을 설정할 수 있는 화면을 표시.
WebContent			
	wpack		WebContent 폴더 아래에 생성하는 XML 화면 파일을 실시간으로 JS 파일로 변환하여 저장.
	cm	css	공통 리소스에 사용되는 CSS 파일 모음
		global_common_component	
		images	공통 리소스에 사용되는 이미지 모음
		template	페이지 템플릿 (화면 생성 시 선택)
		udc	Design Kit - 컴포넌트 탭에 등록할 UDC가 저장된 폴더
WebSquare Configure			- client.config.xml (클라이언트 환경 설정 파일) - server.config.xml (서버 환경 설정 파일)

10.1.2.1. 웹스퀘어 리소스

10.1.2.1.1. 공통 모듈

그림 10-10 웹스퀘어 리소스



10.1.2.1.1.1. 전역 공통 컴포넌트

[7.4.2.1.1. 전역 공통 컴포넌트 \(config.js\)](#) 참고.

10.1.2.1.1. MSA 공통 컴포넌트

[7.4.2.1.2. MSA 공통 컴포넌트 \(config.js\)](#) 참고.

10.1.2.1.1. 전역 라이브러리

[7.4.2.2.1. 전역 라이브러리 \(config.js\)](#) 참고.

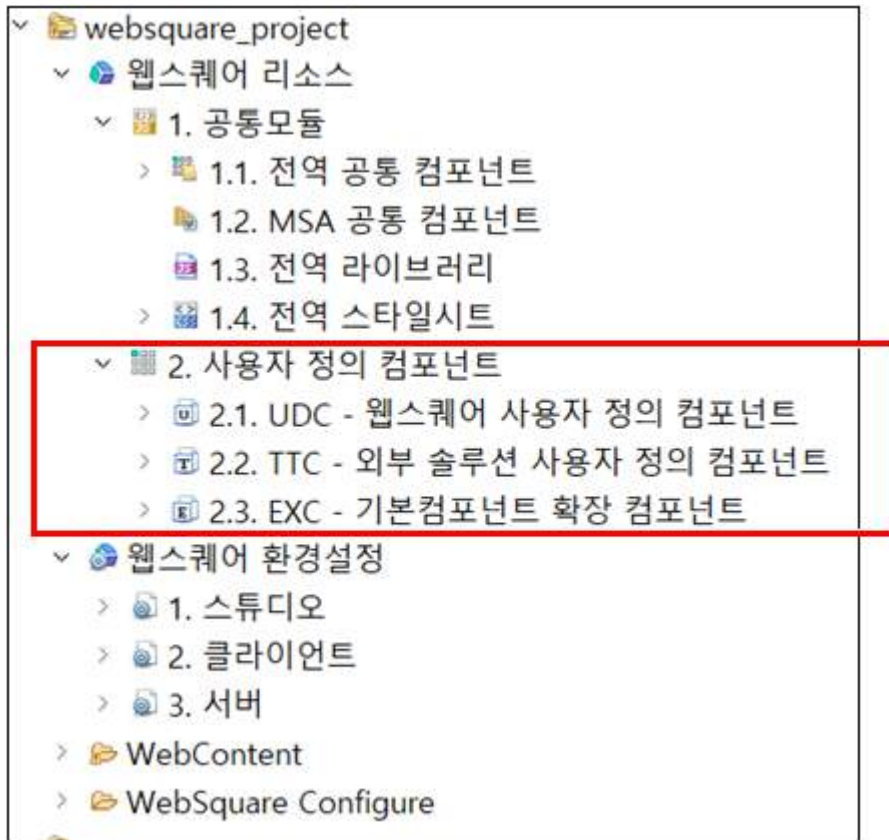
10.1.2.1.1. 전역 스타일시트

[7.4.2.3.1. 전역 스타일 시트 \(config.js\)](#) 참고.

10.1.2.1.2. 사용자 정의 컴포넌트

현재 프로젝트에 포함된 UDC, TTC, EXC 컴포넌트를 확인할 수 있습니다. 각 컴포넌트를 생성하는 방법은 [11.3 User-Defined Component \(UDC\)](#), [11.4 Extended Component \(EXC\)](#) 및 [11.5 Trusted Third-party Component \(TTC\)](#) 부분을 참조하십시오.

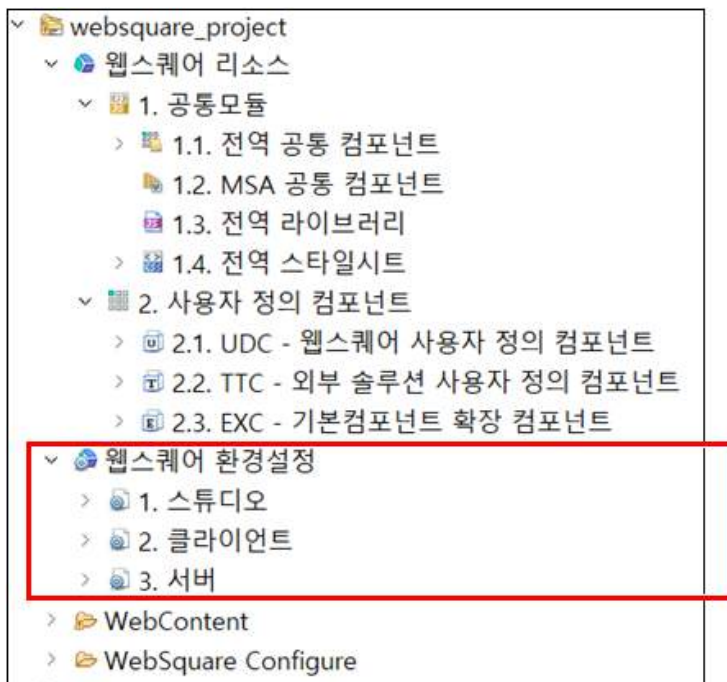
그림 10-11 사용자 정의 컴포넌트



10.1.2.2. 웹스퀘어 환경 설정

각 항목을 클릭하여 스튜디오, 클라이언트, 서버를 설정할 수 있습니다.

그림 10-12 웹스퀘어 환경 설정



10.1.2.2.1. 스튜디오

25. 스튜디오 설정 참고.

10.1.2.2.2. 클라이언트

26. 클라이언트 설정 (client.config.xml) 참고.

10.1.2.2.3. 서버

27. 서버 설정 (server.config.xml) 참고.

10.2 WebSquare Example Project

10.2.1. 생성

WebSquare Reference Example 프로젝트는 외부 네트워크의 사용이 불가능한 환경에서도 예제 파일을 쉽게 확인할 수 있는 환경을 제공합니다. 웹스퀘어 스튜디오에서 서버 구성 없이 쉽게 생성하고 실행할 수 있습니다.

WRE 프로젝트를 생성하려면 **New - WRE Project**를 선택하십시오.

그림 10-13 New - WebSquare Example Project

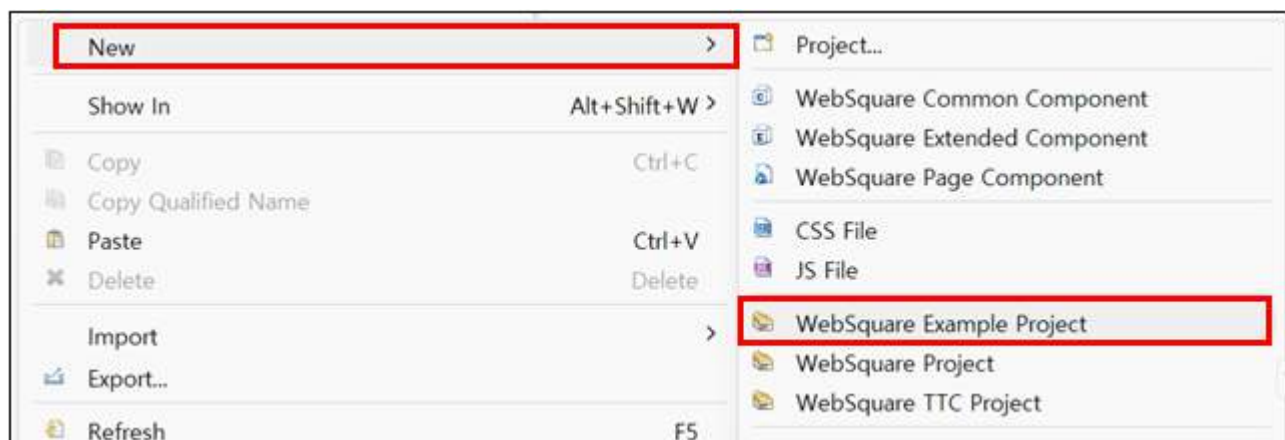


그림 10-14 WebSquare Example Project

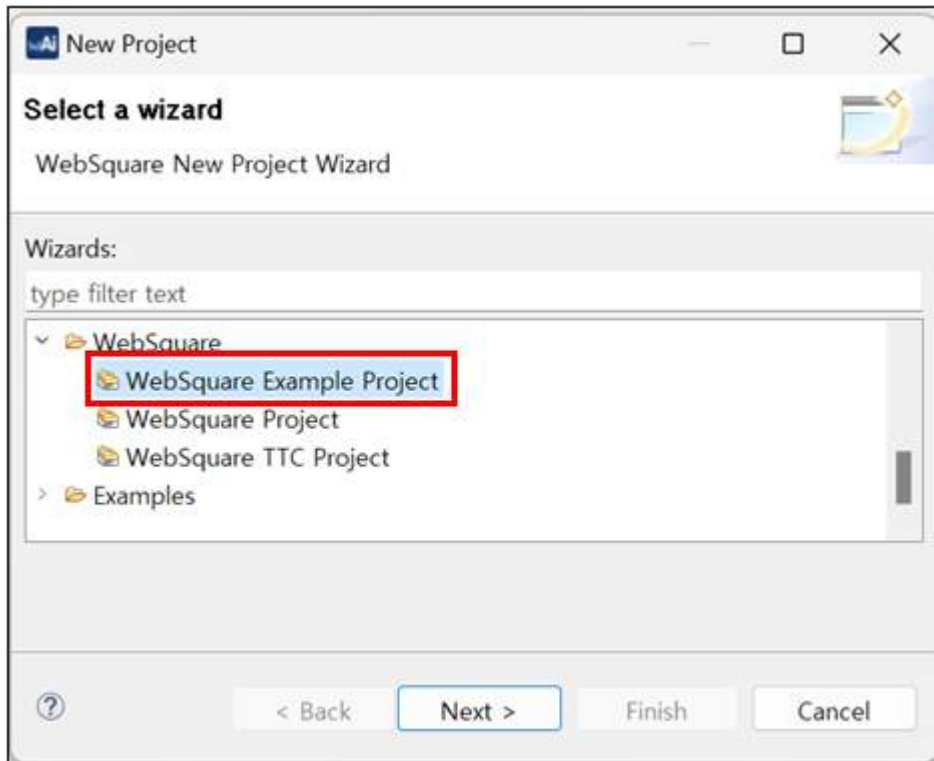
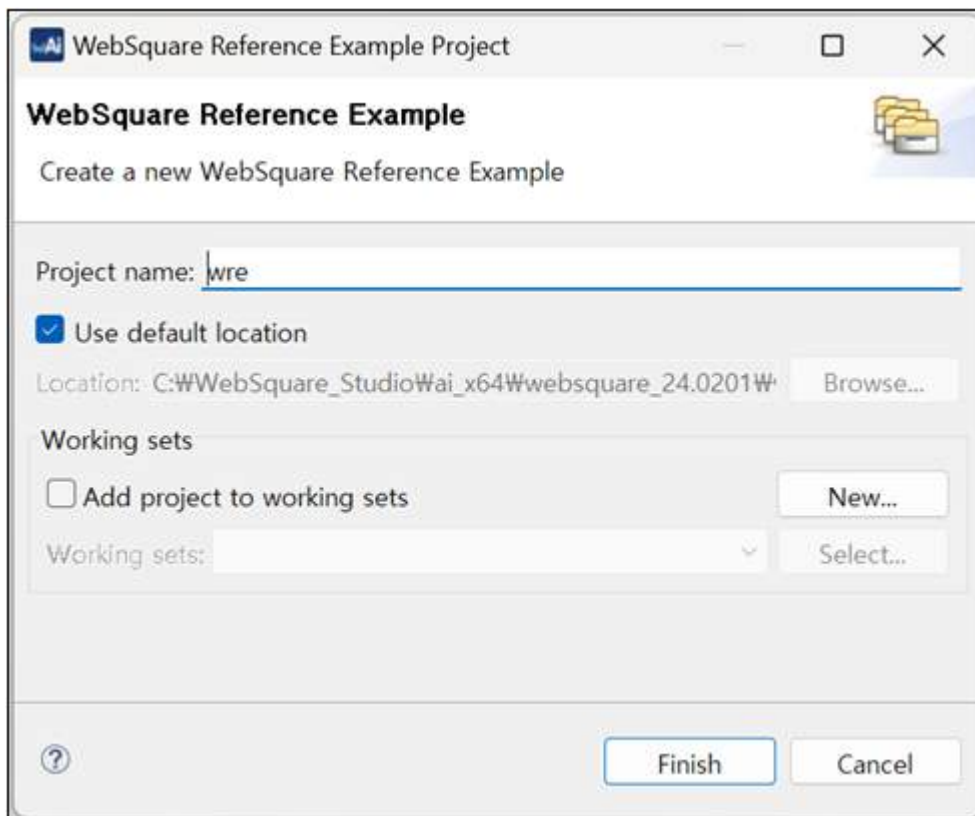
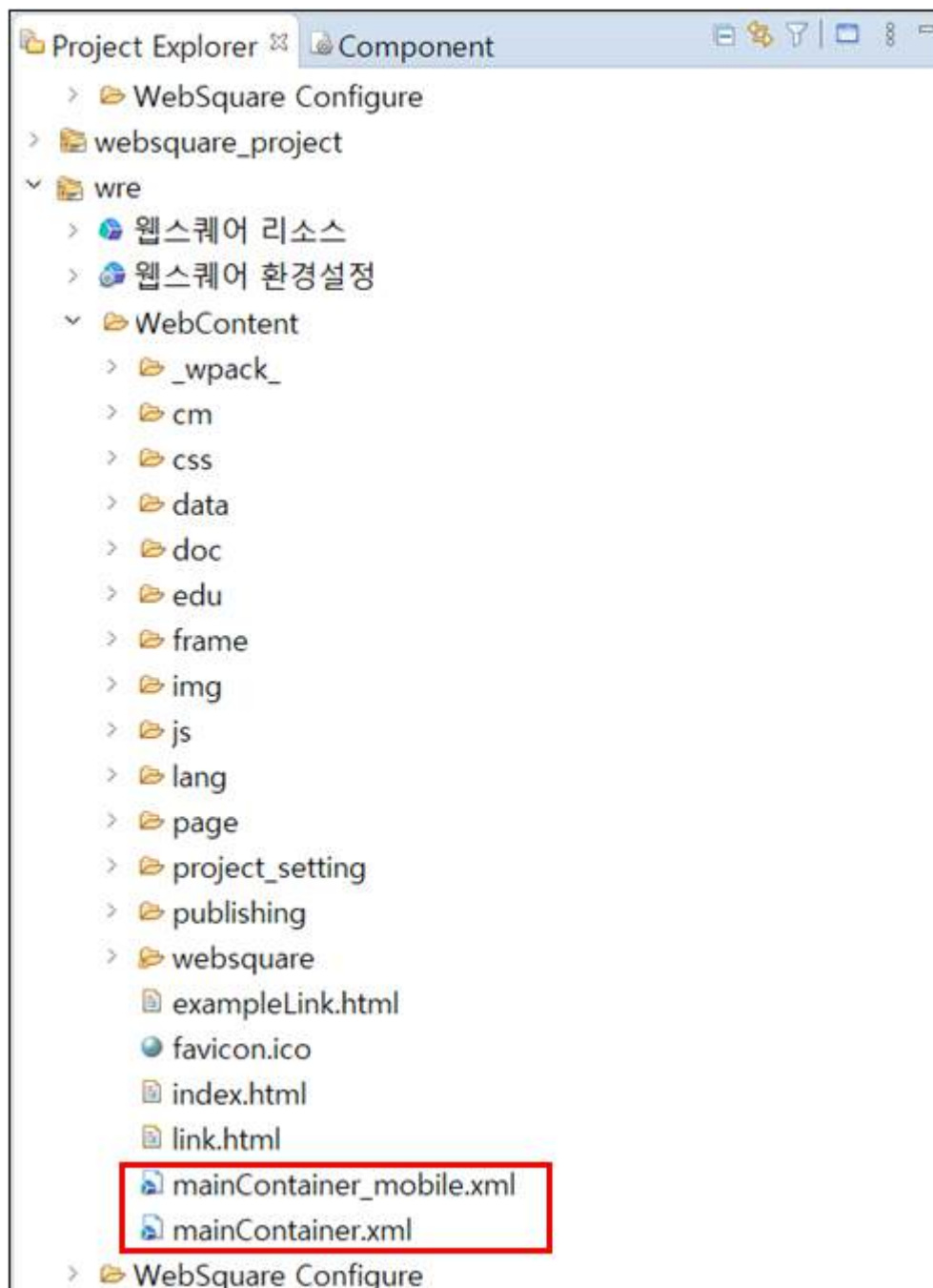


그림 10-15 WebSquare Example Project



"/wre/WebContent/" 폴더 하위의 mainContainer.xml 및 mainContainer_mobile.xml 파일을 선택하여 PC 및 모바일 기기용 WebSquare Reference Example을 확인할 수 있습니다.

그림 10-16 mainContainer.xml 및 mainContainer_mobile.xml 파일



10.3 WebSquare TTC Project

10.3.1. 생성

WebSquar TTC Project는 TTC 컴포넌트를 생성하기 위한 프로젝트입니다. 개발자는 TTC 컴포넌트 생성에 필요한 폴더 구성을 확인할 수 있고 직접 TTC를 임포트할 수도 있습니다. WebSquar TTC Project는 아래의 순서로 생성하십시오.

1. Project Explorer를 마우스로 우클릭한 후 마우스 **New - WebSquare TTC Project**를 선택합니다.

그림 10-17 New - WebSquare TTC Project

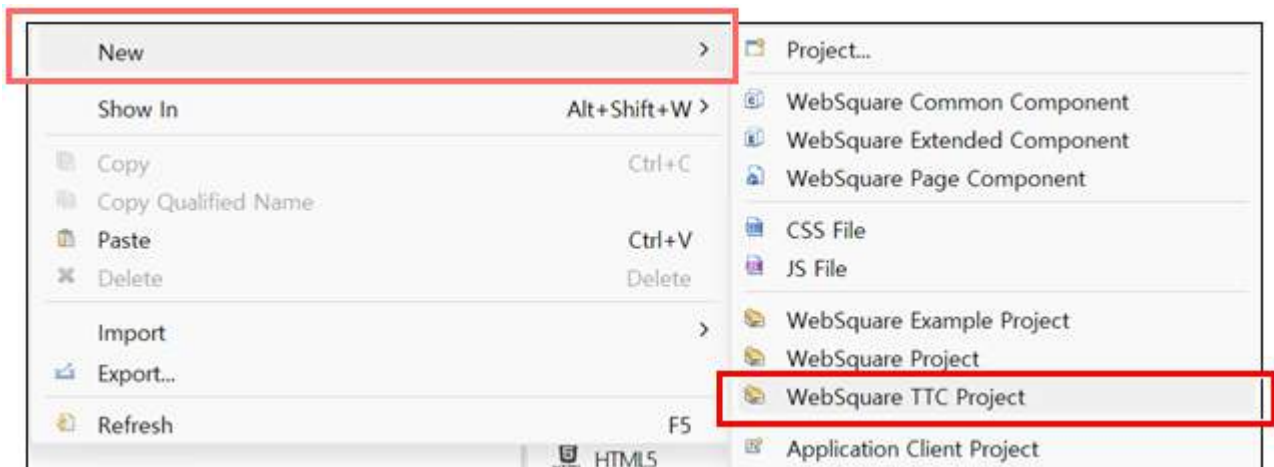
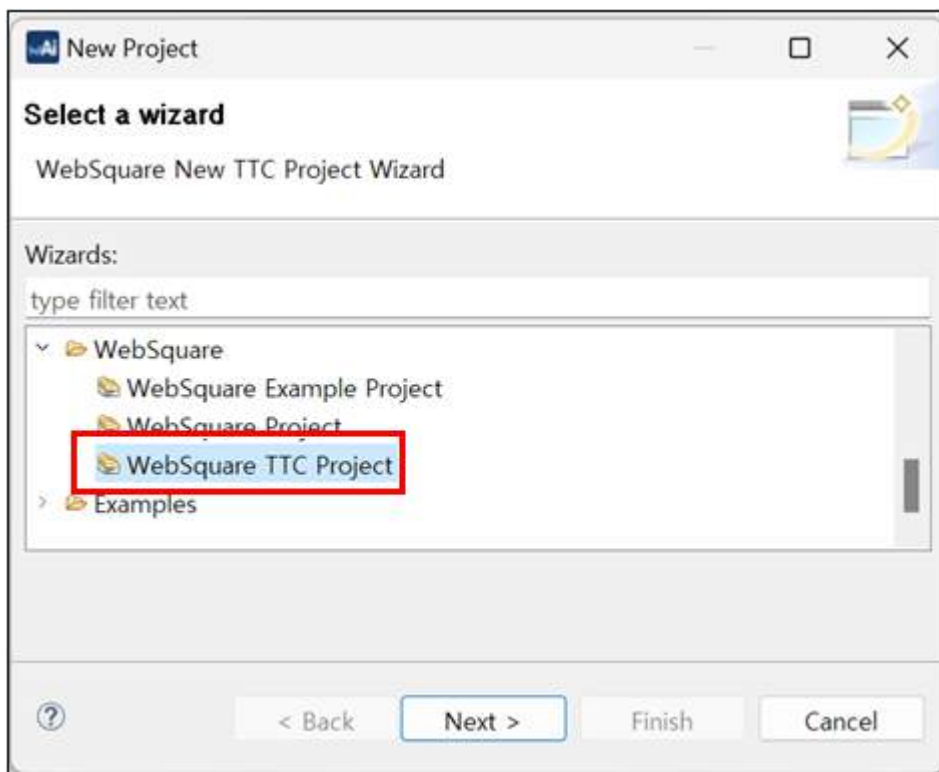
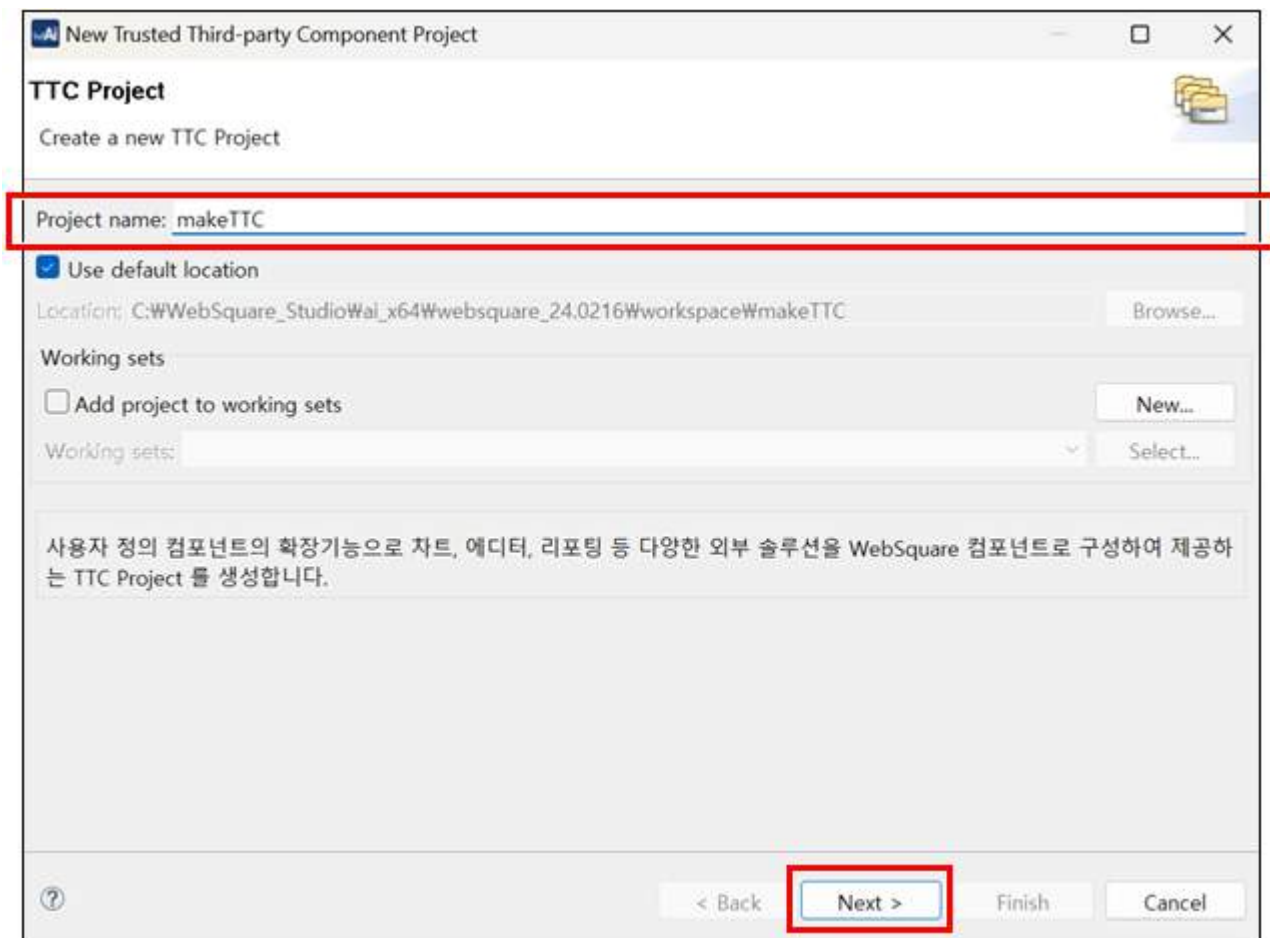


그림 10-18 WebSquare TTC Project



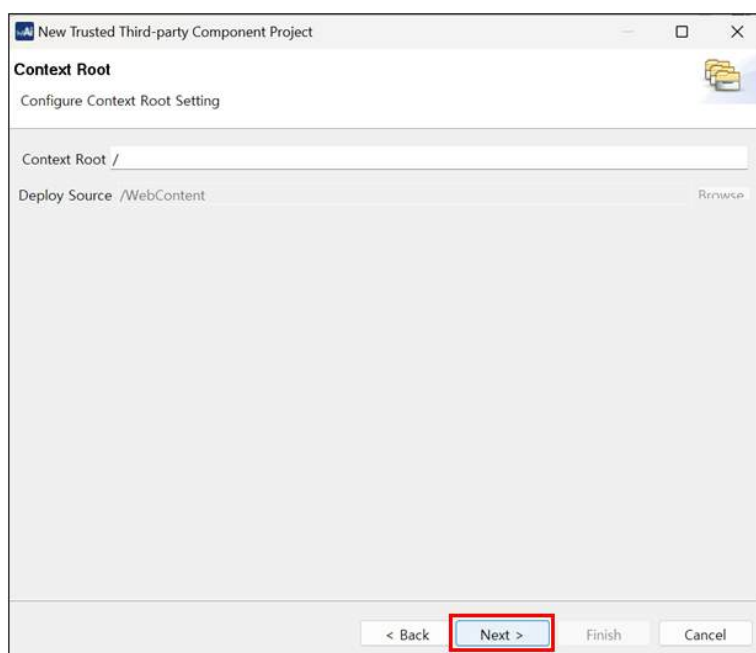
2. 프로젝트 이름을 입력하고 **Next** 버튼을 클릭합니다.

그림 10-19 WebSquare TTC Project



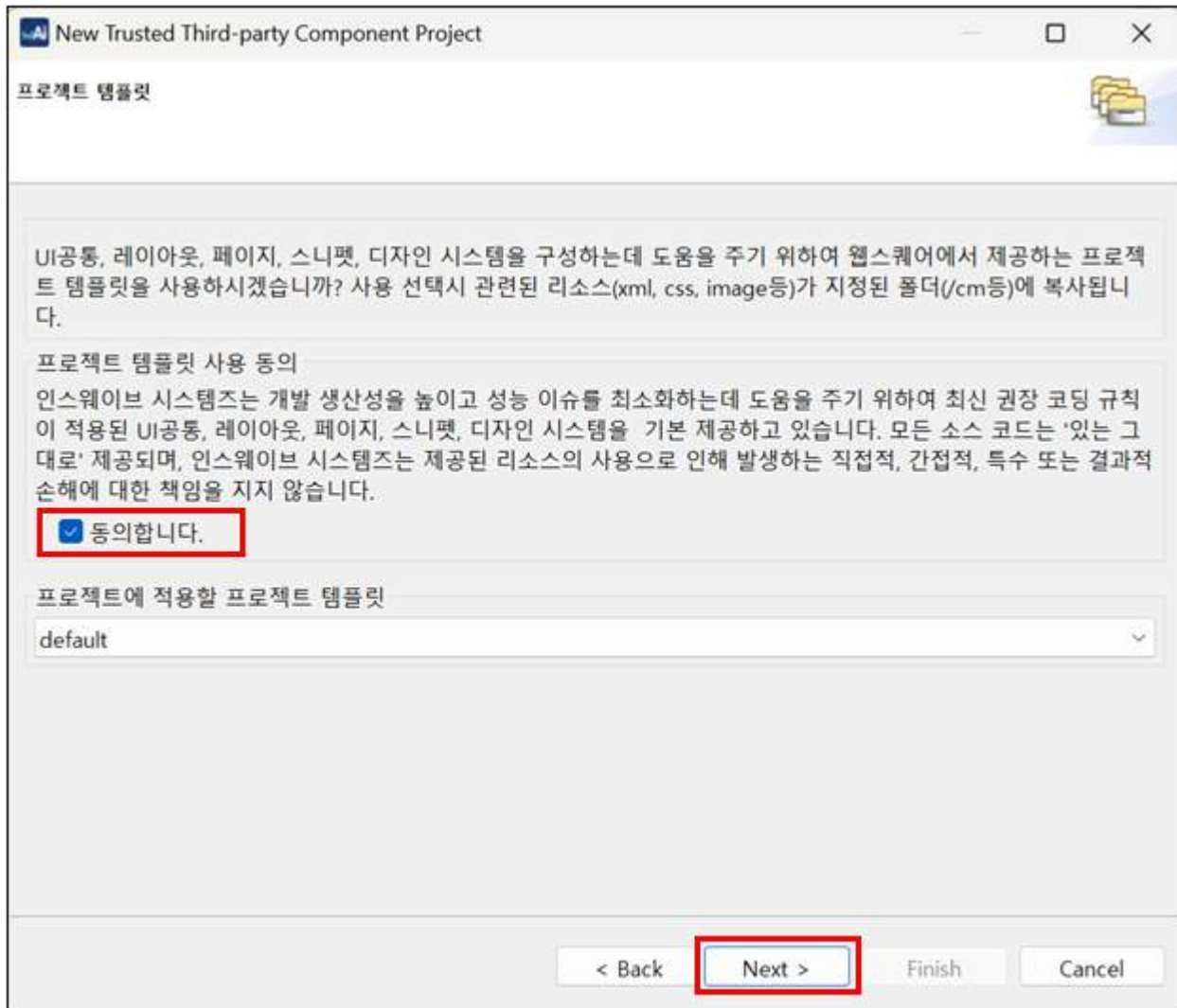
3. Context Root를 지정합니다. 기본 값은 루트("/ ")입니다.
4. **Next** 버튼을 클릭합니다.

그림 10-20 Context Root 지정



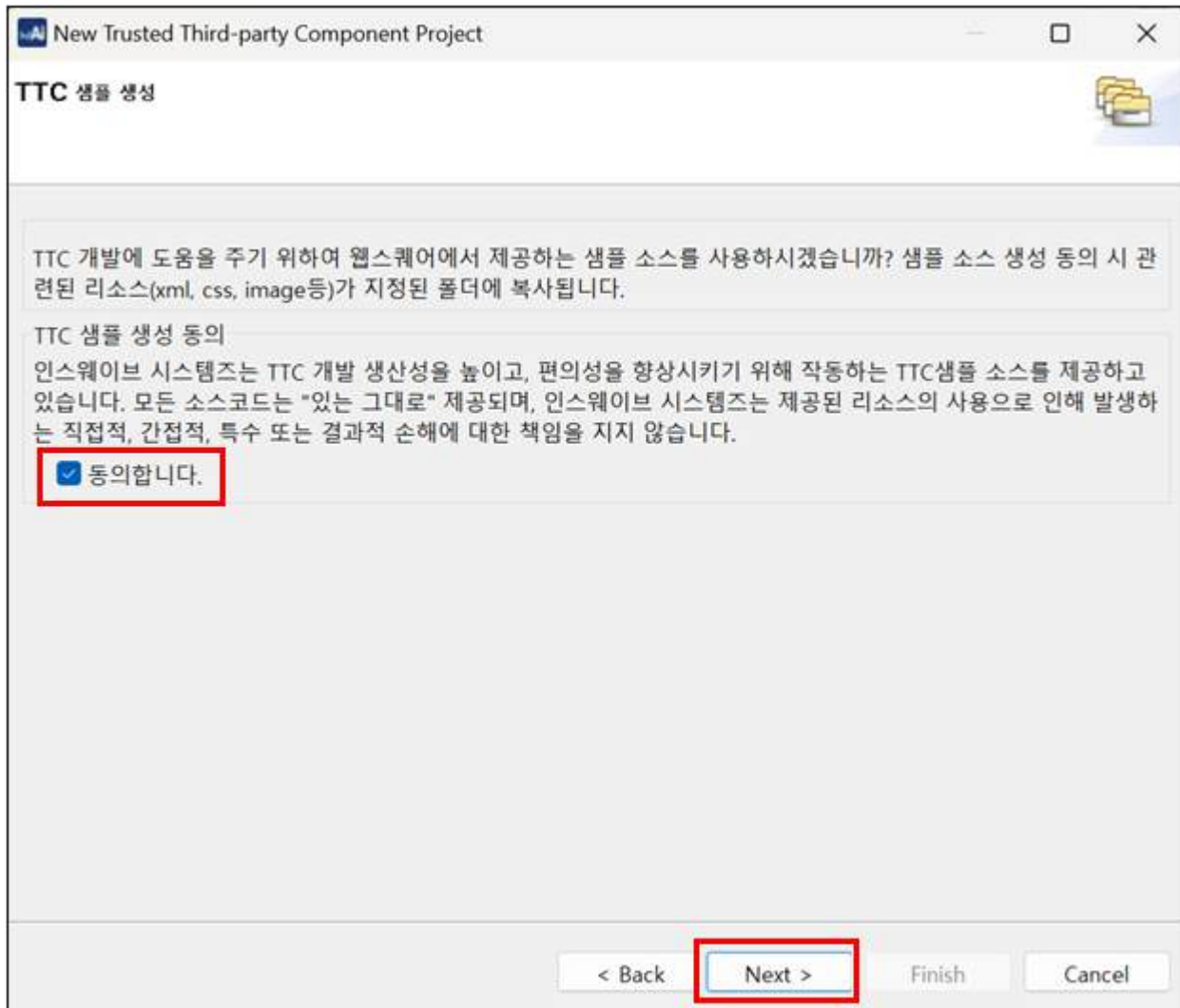
5. 프로젝트 템플릿 생성 여부를 선택할 수 있는 화면이 나타납니다. "동의합니다." 항목을 선택하고, **Next** 버튼을 클릭합니다. (권장)

그림 10-21 프로젝트 템플릿 사용 동의



6. TTC 샘플 생성 여부를 선택할 수 있는 화면이 나타납니다. "동의합니다." 항목을 선택하고, **Next** 버튼을 클릭합니다. (권장)

그림 10-22 TTC 샘플 생성 동의

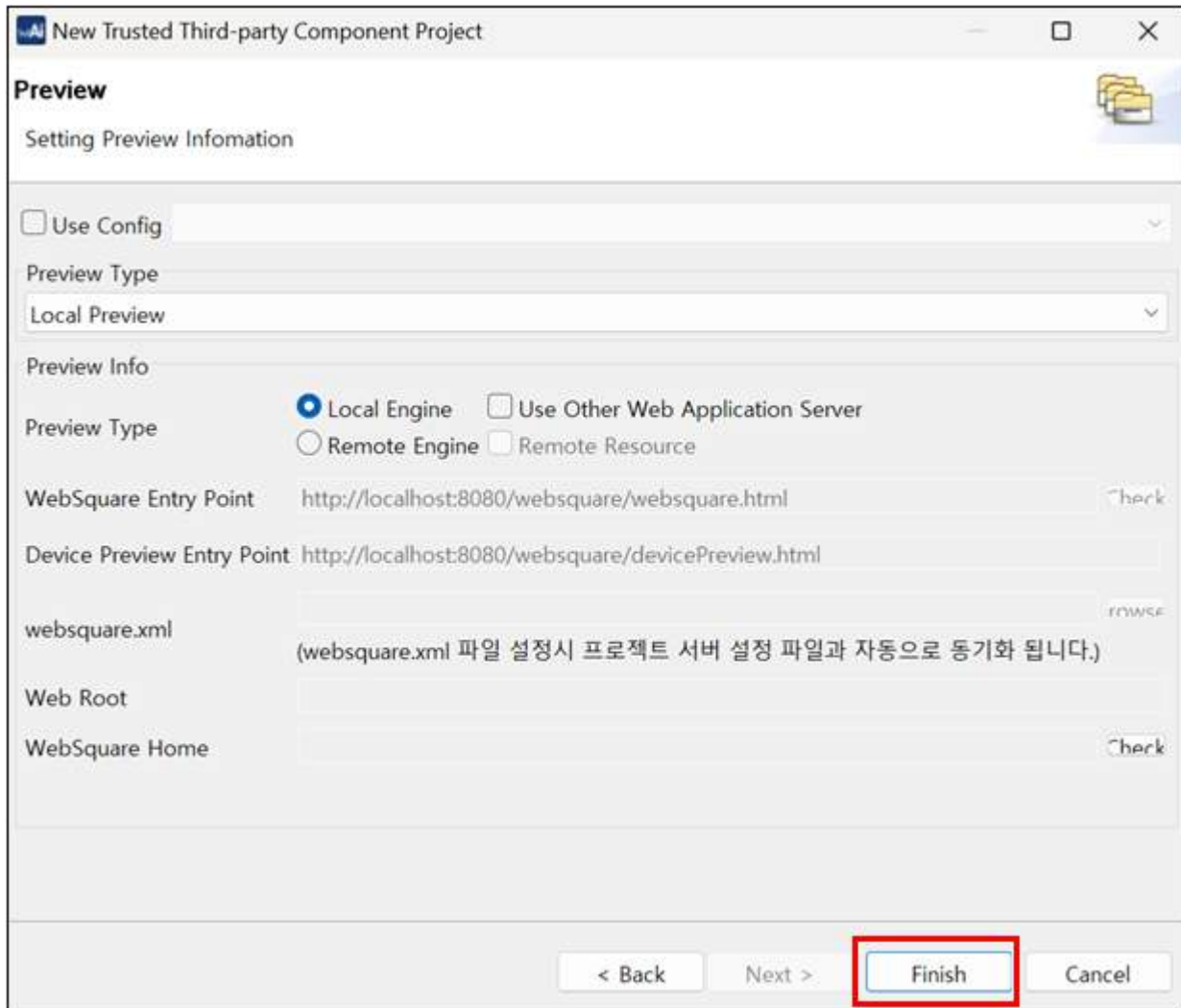


TTC 샘플 생성

“동의합니다.” 항목을 선택하지 않을 경우, TTC 샘플이 임포트되지 않고 TTC 컴포넌트 생성시 필요한 폴더 구조만 생성됩니다.

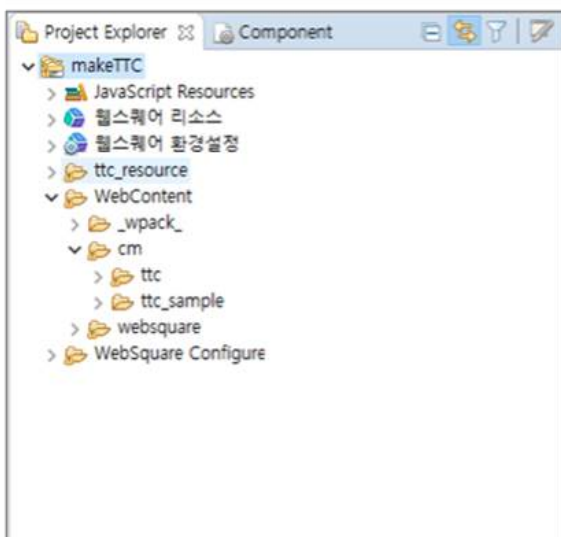
7. **Finish** 버튼을 클릭합니다.

그림 10-23 WebSquare TTC Project



8. 아래와 같이 WebSquare TTC Project가 구성되는지 확인합니다.

그림 10-24 WebSquare TTC Project



10.3.2. 폴더 구조

WebSquare TTC Project의 폴더 구조는 아래와 같습니다.

Table 10-4 TTC 폴더 구조

TTC 프로젝트				
	/ttc_resource			TTC에 필요한 리소스 • 라이선스 파일 • 활성화/비활성화 아이콘
	/WebContent			
		/cm		
			/ttc	TTC로 배포할 파일
			/ttc_sample	TTC 샘플

10.3.3. 라이선스 및 아이콘 등록

1. /ttc_resource/ 폴더 하위에 TTC 이름의 폴더를 생성합니다.

그림 10-25 /ttc_resource/ 폴더

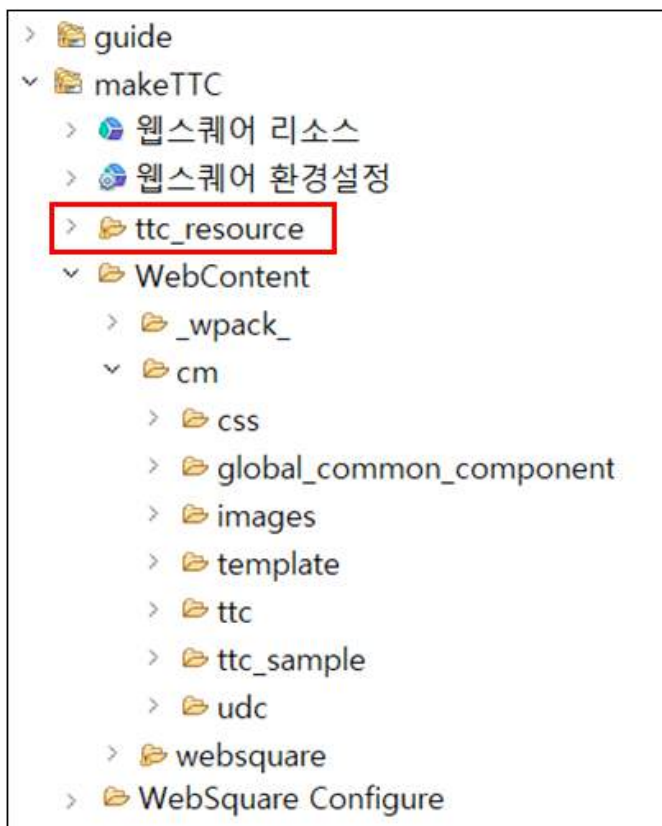
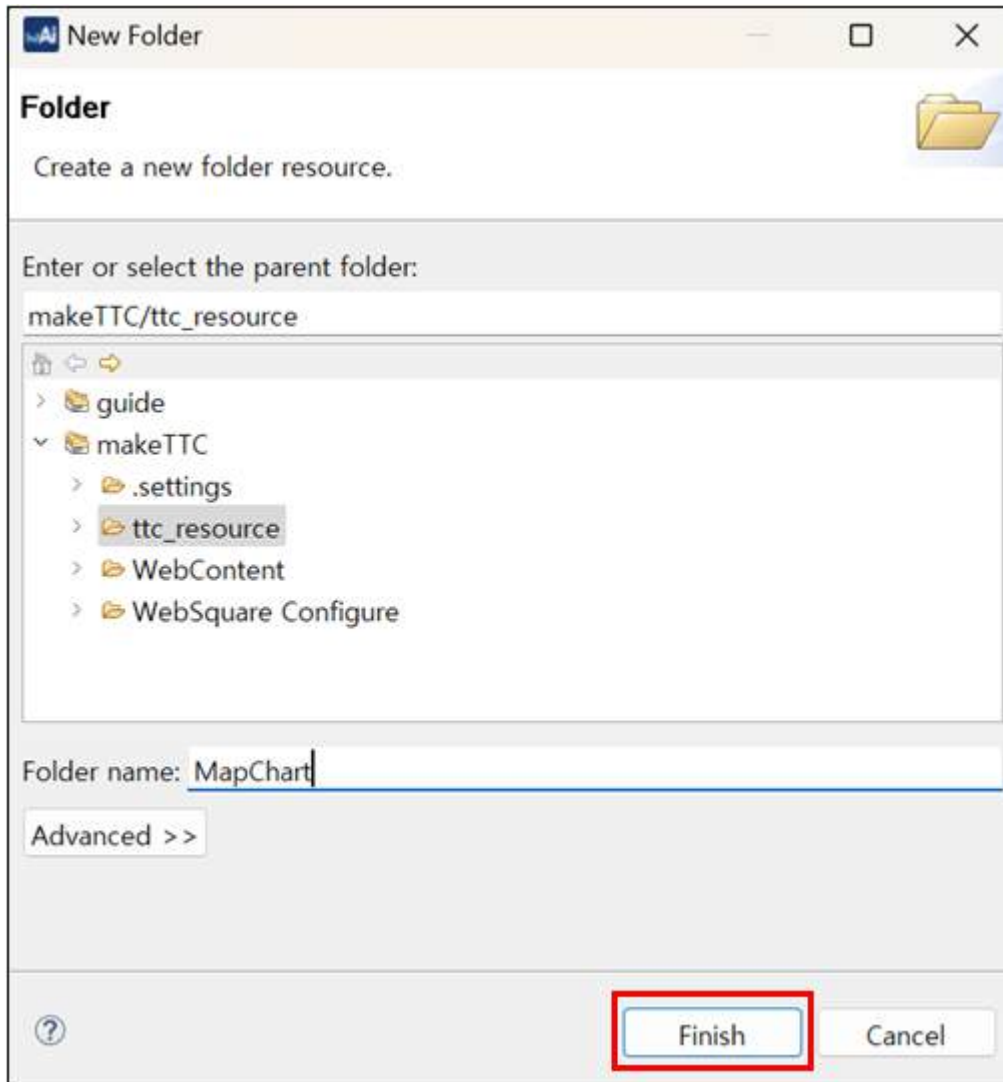
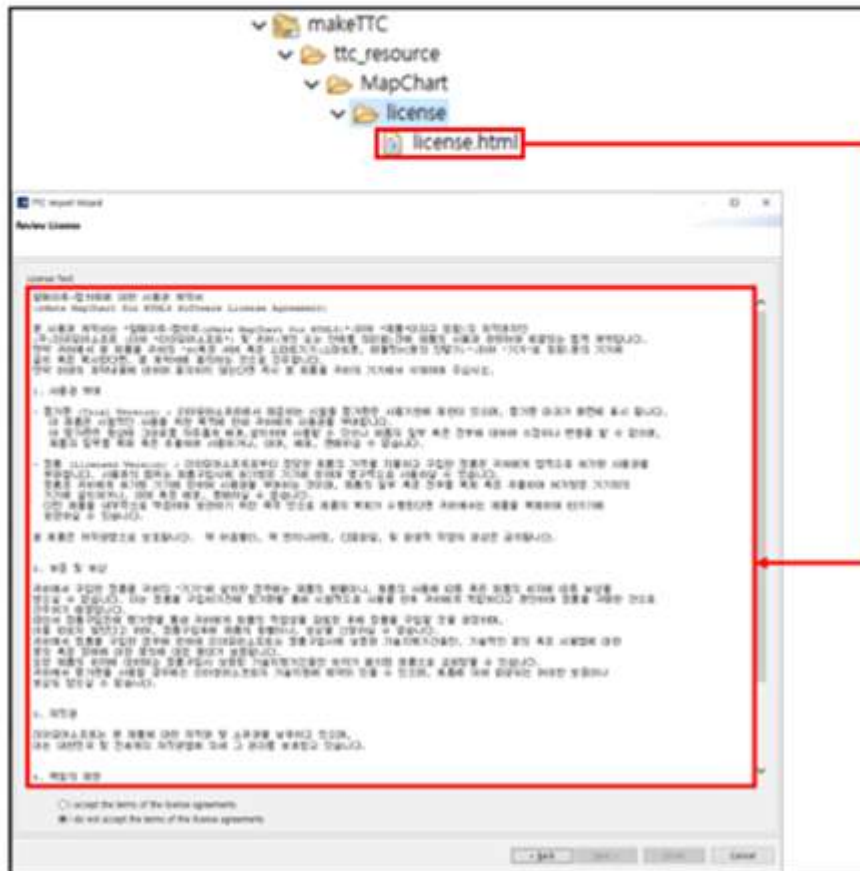
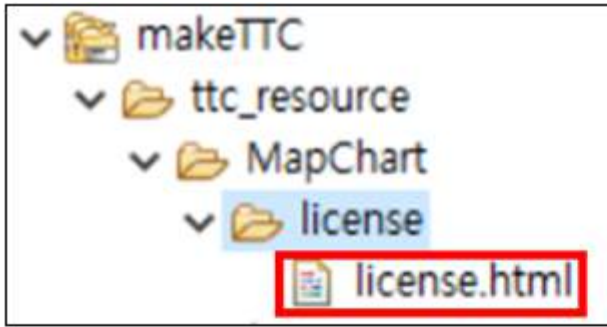


그림 10-26 TTC 이름의 폴더 생성



- 위에서 생성한 폴더 아래에 **/license/** 폴더를 생성하고 라이선스 파일(HTML)을 복사합니다.

그림 10-27 라이선스 파일 복사

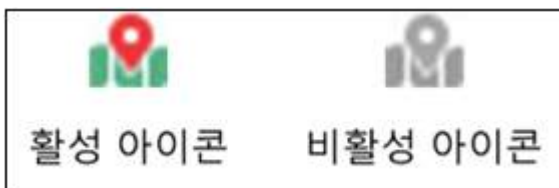
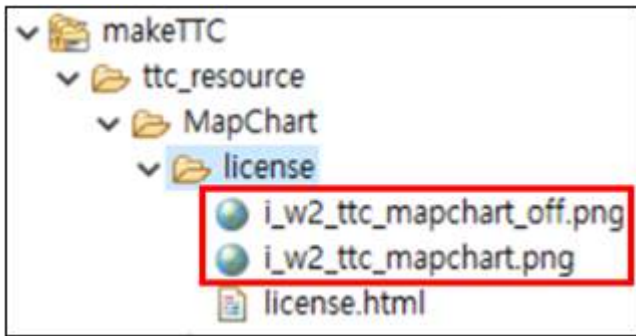


라이선스 파일

등록한 라이선스 파일은 TTC를 임포트할 때 표시됩니다.

- 위에서 생성한 license 폴더에 TTC가 활성화인 상태와 비활성인 상태 때 표시할 (16X16 크기의) 아이콘의 파일들도 복사하십시오.

그림 10-28 Palette에 등록할 때 표시할 아이콘



10.3.4. TTC 등록

1. `/ttc/` 폴더 하위에 TTC 이름의 폴더를 생성합니다.
2. 생성한 폴더에 TTC의 XML 파일을 복사합니다.

그림 10-29 /WebContent/cm/ttc/ 폴더

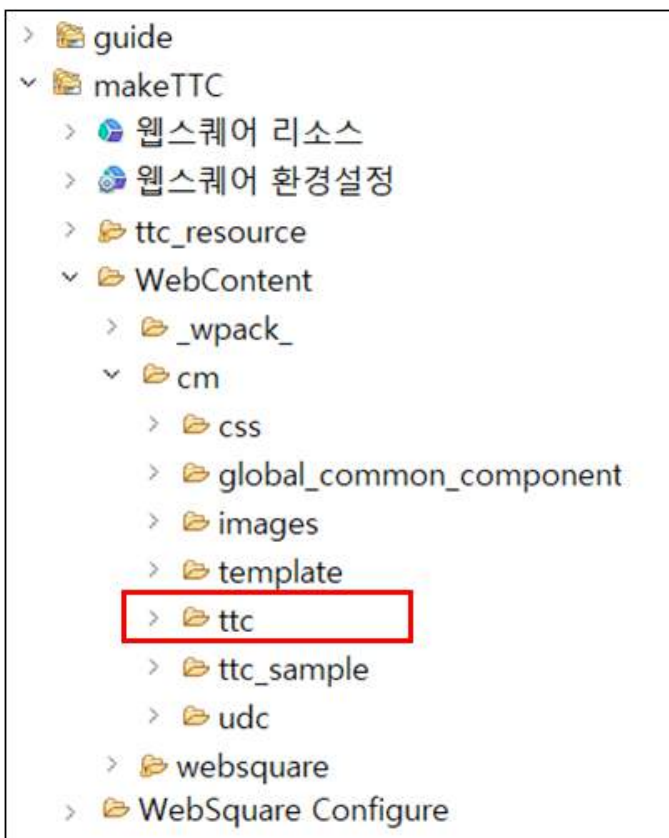
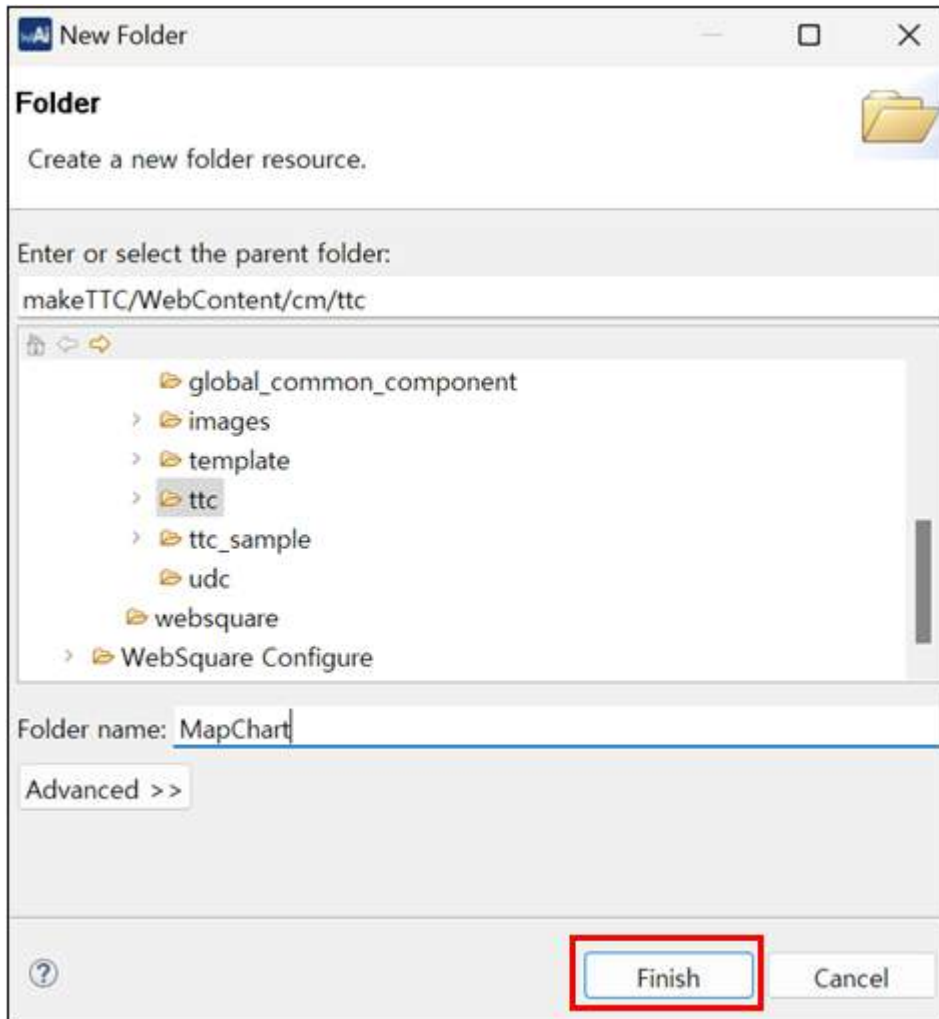


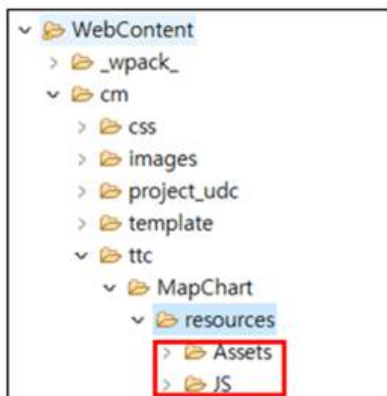
그림 10-30 TTC 이름의 폴더 생성



10.3.5. TTC 자원 등록

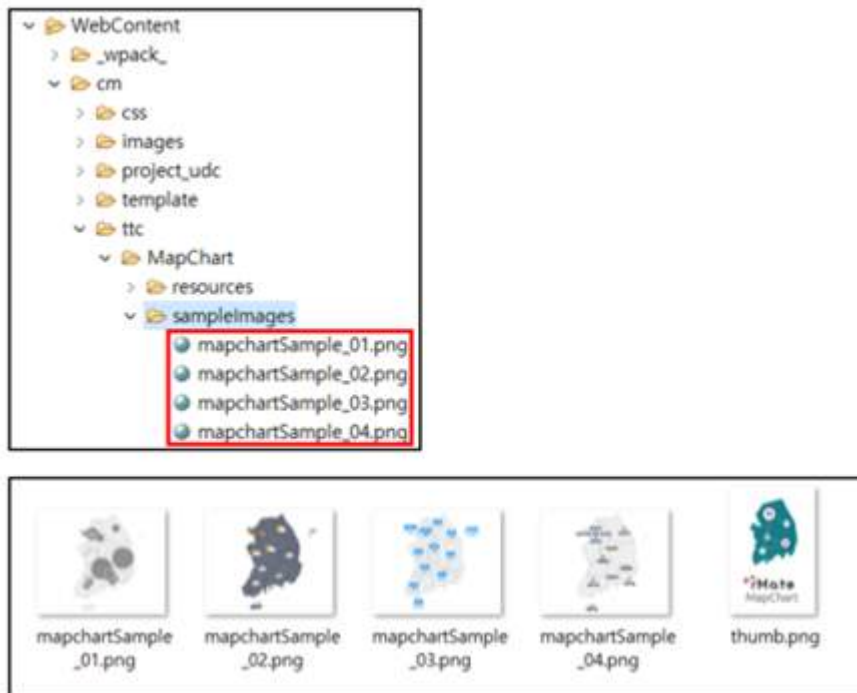
1. /ttc/ttc_이름/ 폴더 하위에 /resources/ 폴더를 생성하고, 필요한 파일을 복사합니다.

그림 10-31 /ttc/ttc_이름/resources/



2. /ttc/ttc_이름/ 폴더 하위에 /sampleImages/ 폴더를 생성하고, 샘플 이미지로 사용할 파일들을 복사합니다.

그림 10-32 /ttc/ttc_이름/sampleImages/



샘플 이미지들은 TTC 가져오기, TTC 미리보기 또는 TTC 컴포넌트를 생성할 때 사용됩니다.

그림 10-33 샘플 이미지 사용 예시 (1)

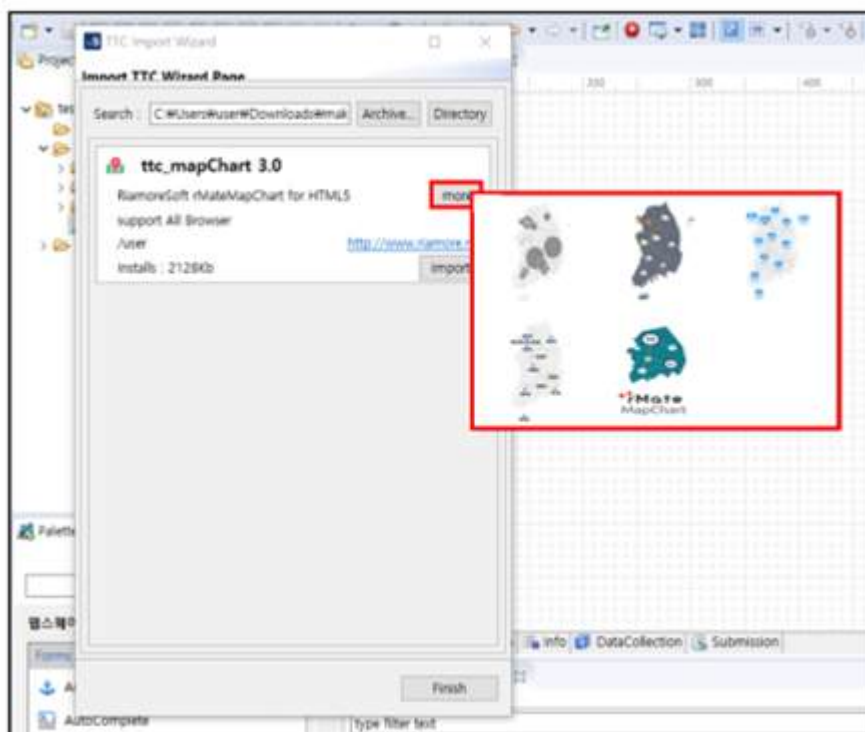
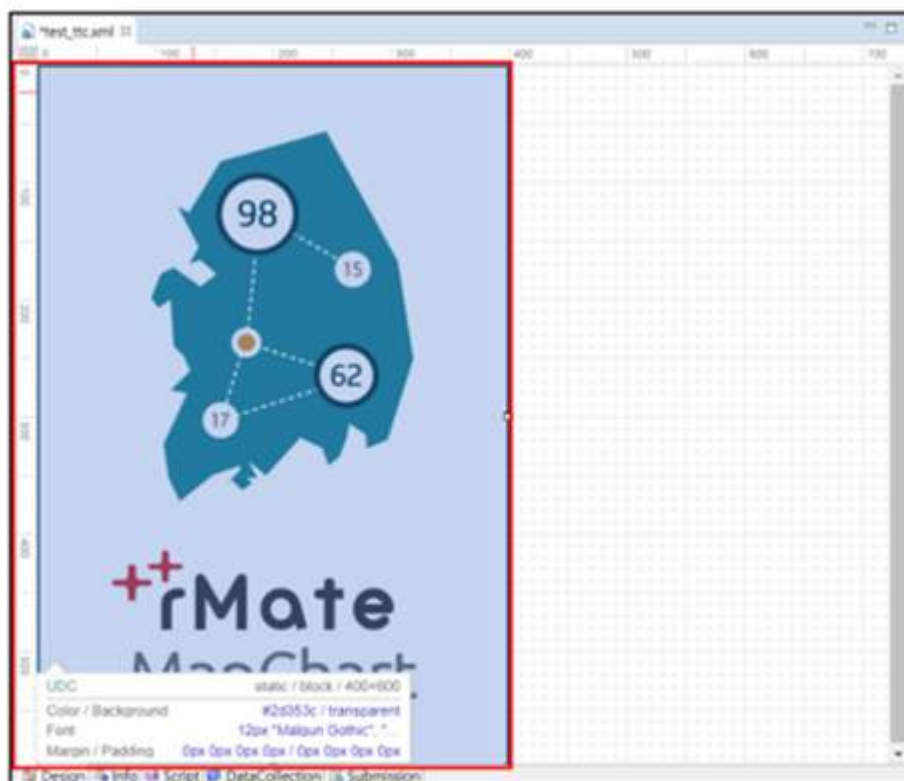
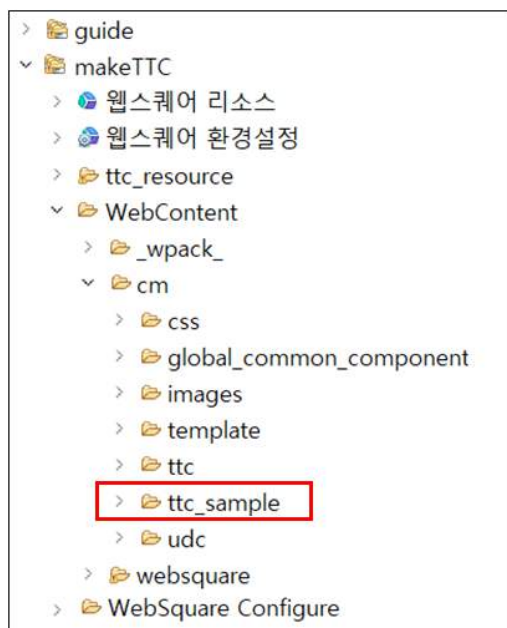


그림 10-34 샘플 이미지 사용 예시 (2)



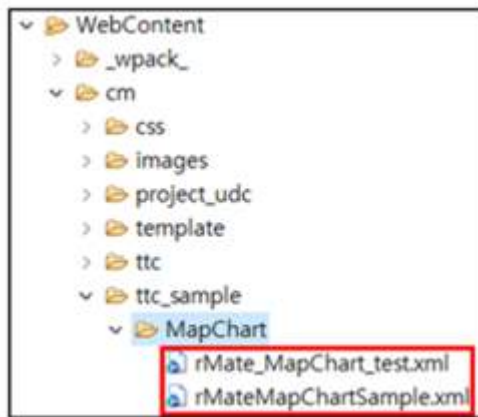
10.3.6. TTC 샘플 등록

1. `/ttc_sample/` 폴더 하위에 TTC 이름의 폴더를 생성합니다.

그림 10-35 `/WebContent/cm/ttc_sample/` 폴더

2. 생성한 폴더에 샘플 파일을 복사합니다.

그림 10-36 샘플 파일



샘플 및 테스트용 파일

샘플 파일이 없을 경우 등록하지 않아도 무방합니다.

단, 샘플 및 테스트 용도의 XML 파일을 등록할 경우 **API - 컴포넌트** 탭에서 TTC 컴포넌트 항목을 미사용으로 선택하십시오.

10.4 프로젝트 설정

25. 스튜디오 설정 참고.

11. 컴포넌트 생성

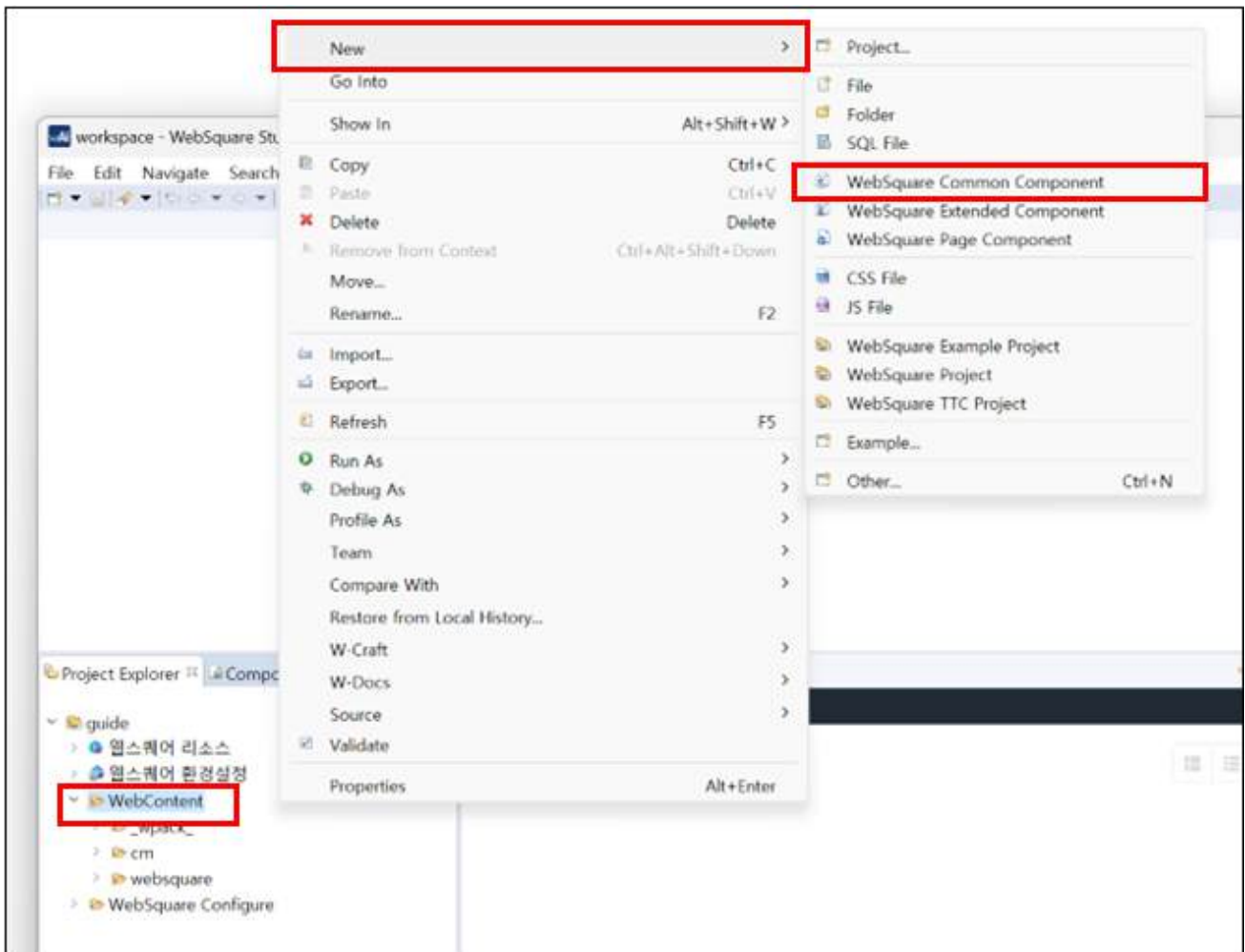
11.1 WebSquare Common Component

프로젝트 내부에서 공통으로 사용할 수 있는 컴포넌트를 생성하기 위한 용도로 사용됩니다.

11.1.1. 생성하기

1. 프로젝트를 우클릭하고, **New - WebSquare Common Component** 메뉴를 선택합니다.

그림 11-1 New - WebSquare Common Component



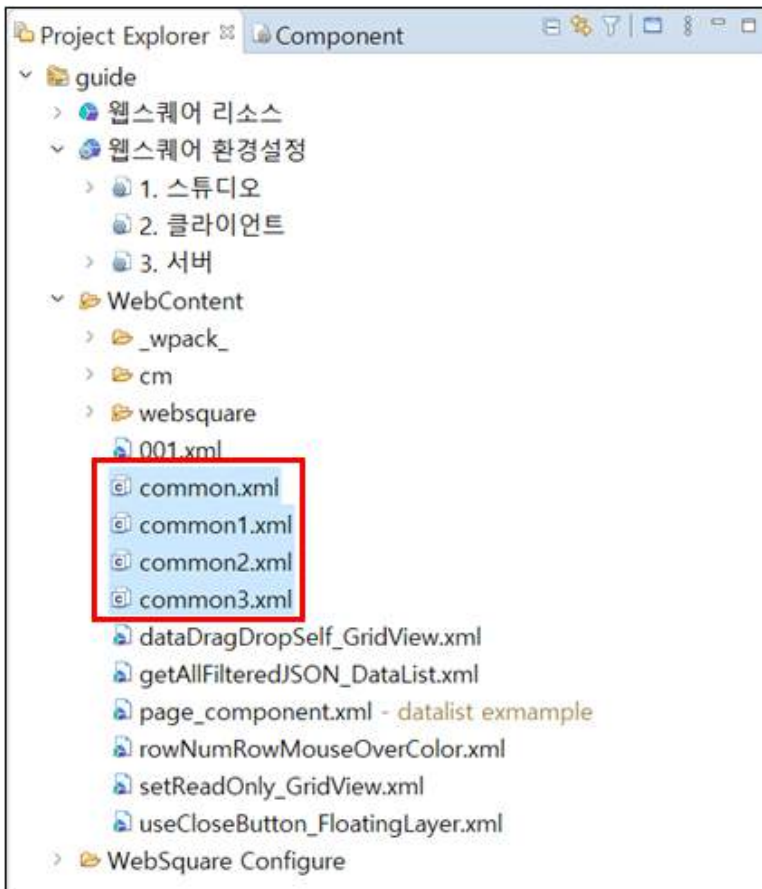
2. 스크립트 탭에서 등록할 공통 컴포넌트 함수를 구현합니다.

그림 11-2 공통 컴포넌트 함수 구현



3. 아래와 같이 생성된 페이지 공통 컴포넌트 (WebSquare Common Component) 목록이 표시되는지 확인합니다.

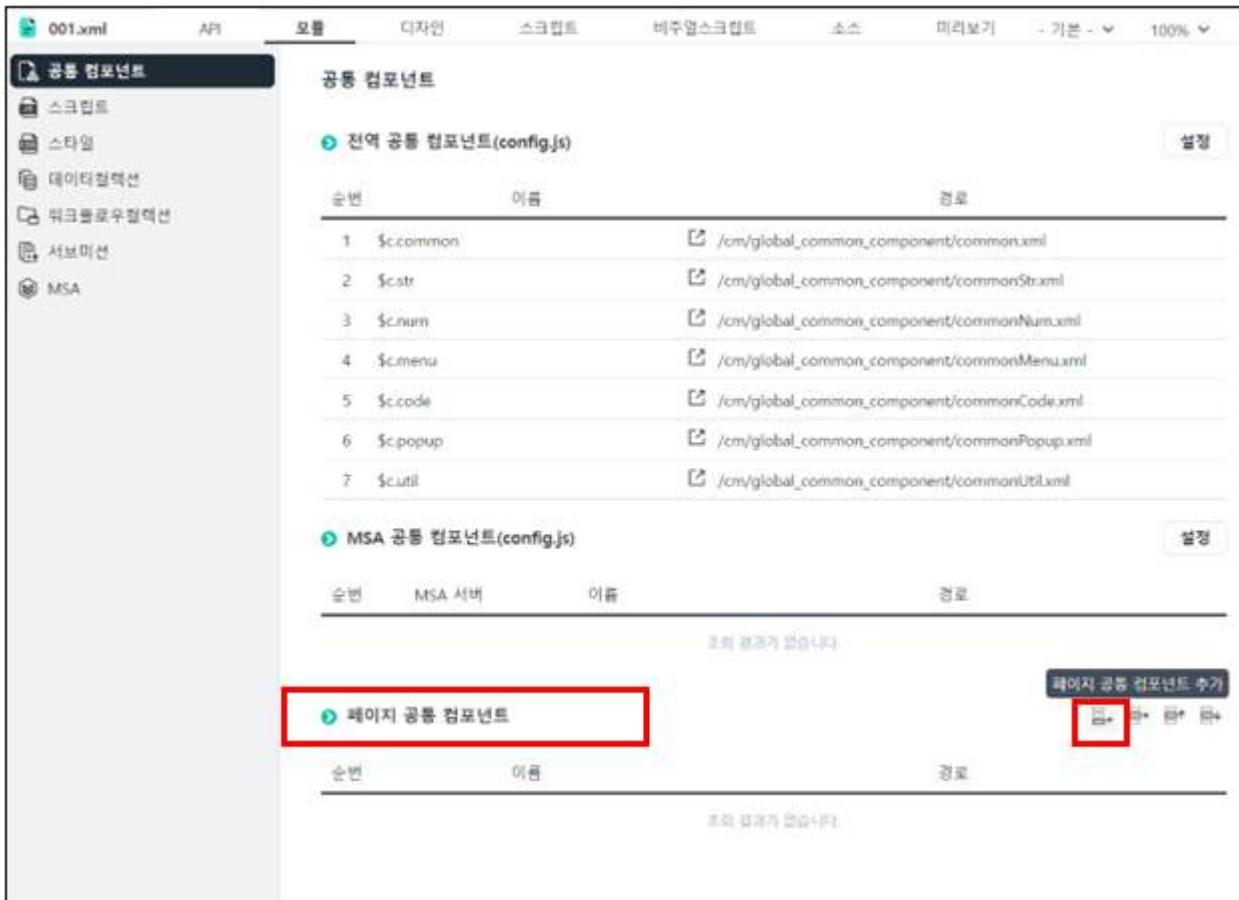
그림 11-3 프로젝트 공통 컴포넌트 (WebSquare Common Component)



11.1.2. 사용하기

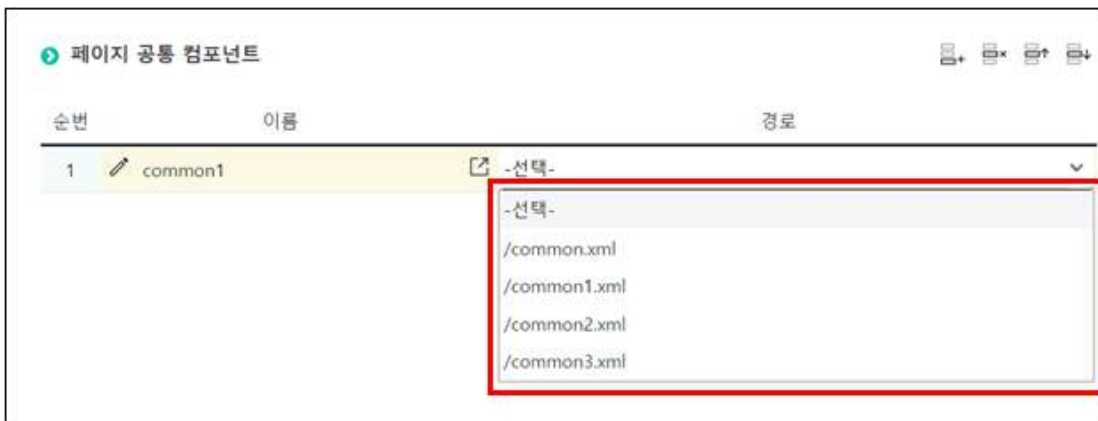
1. WebSquare Page Component를 생성합니다.
2. 모듈 - 공통 컴포넌트 탭으로 이동합니다.
3. 우측의 페이지 공통 컴포넌트 추가 버튼을 클릭합니다.

그림 11-4 페이지 공통 컴포넌트



4. 프로젝트에 포함된 페이지 공통 컴포넌트들의 목록에서 사용할 페이지 공통 컴포넌트를 선택합니다.

그림 11-5 공통 컴포넌트 목록



5. 이름 항목을 더블 클릭하여 페이지의 공통 컴포넌트의 이름을 변경할 수 있습니다.

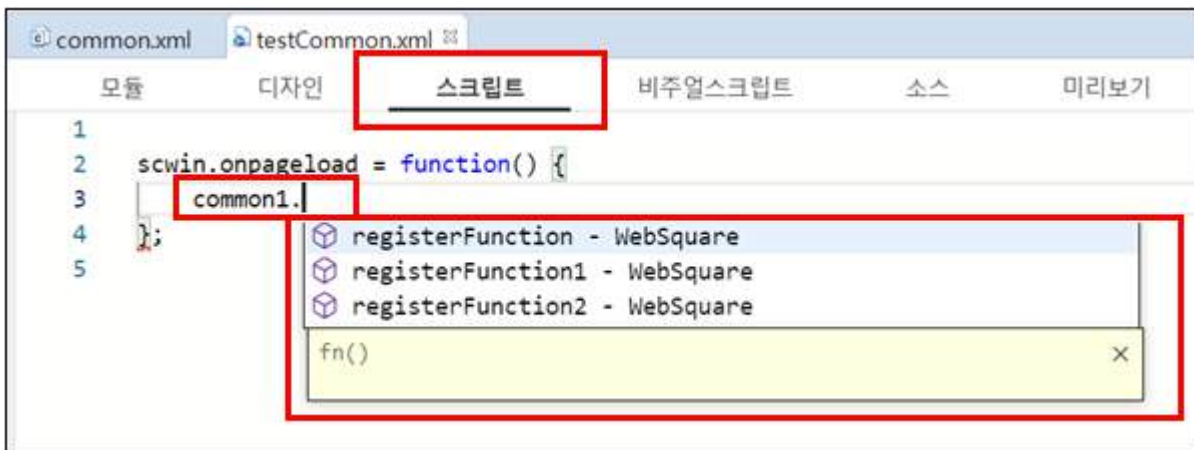
그림 11-6 페이지 공통 컴포넌트의 이름 변경



6. 스크립트 탭으로 이동합니다.

7. 등록된 페이지 공통 컴포넌트의 함수를 사용하여 코드를 작성합니다.

그림 11-7 코드 작성



11.2 WebSquare Page Component

11.2.1. 소개

화면 파일은 웹스퀘어가 제공하는 80여개의 기본 컴포넌트와 UDC와 같은 공통 개발 컴포넌트의 속성, 함수, 이벤트 등을 설정하고, 웹스퀘어가 제공하는 공통 디자인 리소스(페이지 템플릿, 레이아웃 템플릿, 스니펫)를 이용하여 개발할 수 있습니다. 상세한 스튜디오 사용법은 [화면 그리기](#)를 참조하십시오.

화면 개발용 리소스

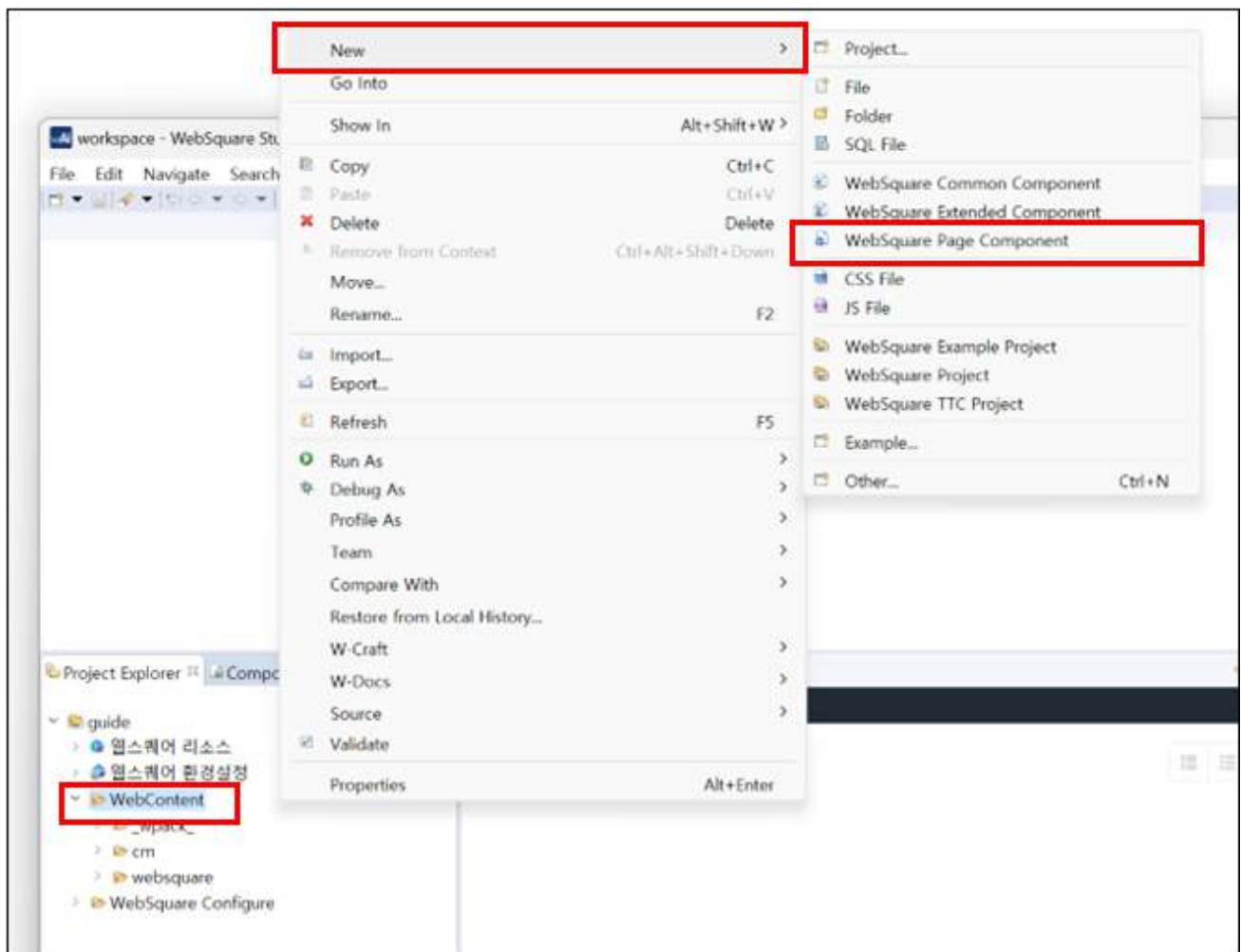
- (1) 80여개의 기본 컴포넌트
- (2) UDC

- (3) 페이지 템플릿
- (4) 레이아웃 템플릿
- (5) 스니펫

11.2.2. 생성하기

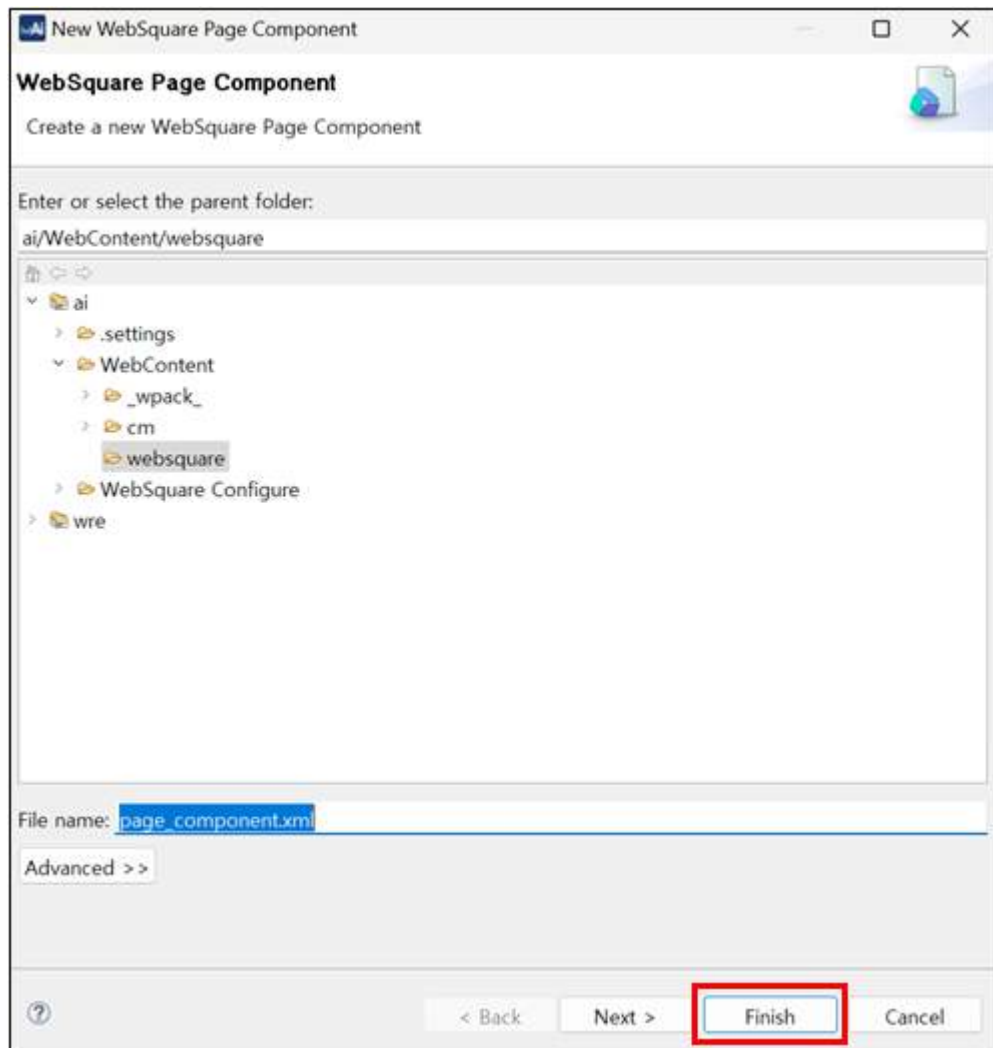
1. **WebContent** 폴더를 우클릭한 후, **New - WebSquare Page Component**를 선택하십시오. (**WebContent** 폴더 아래에 새로운 페이지 컴포넌트를 생성하는 것을 권장합니다.)

그림 11-8 New - WebSquare Page Component

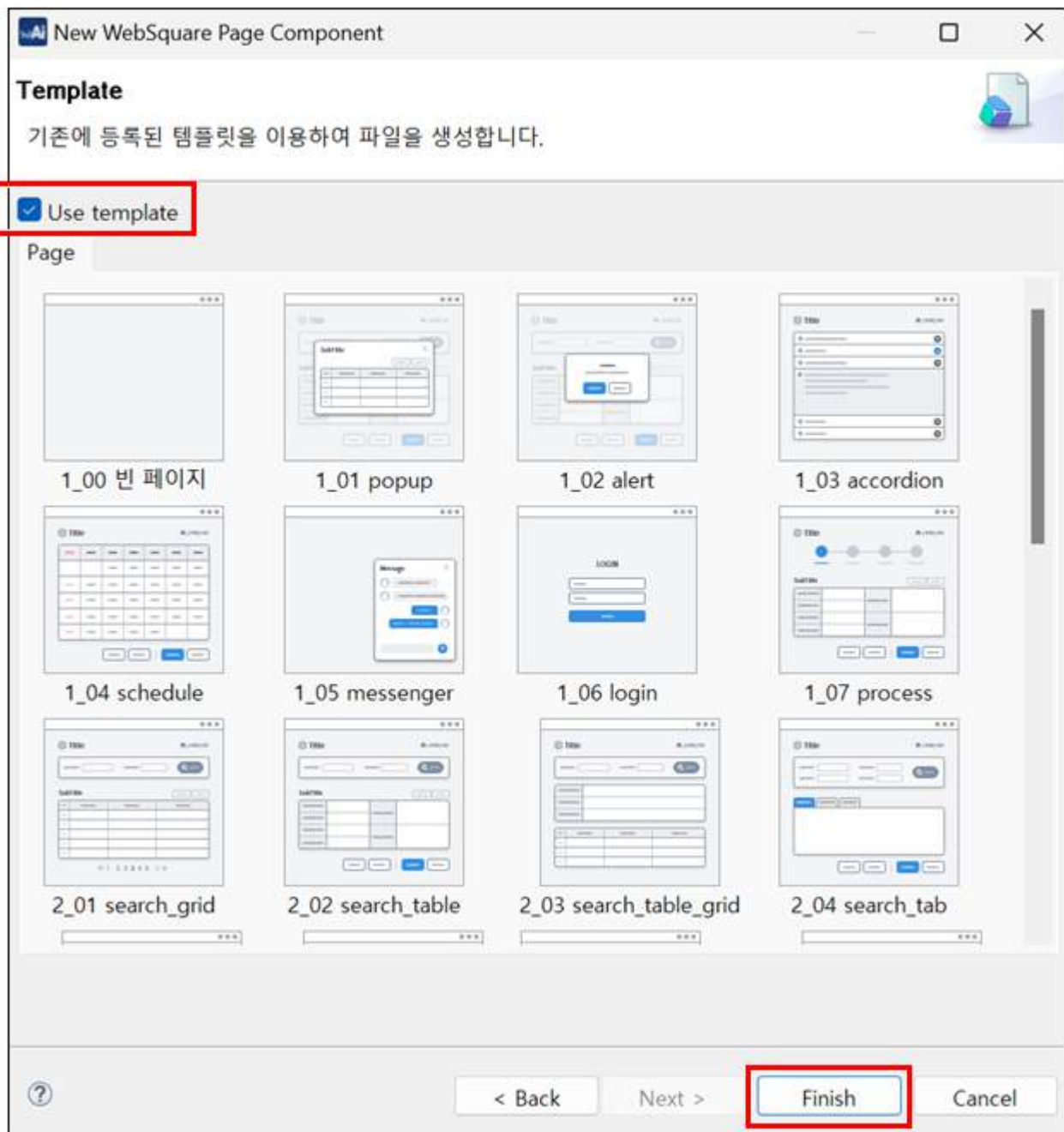


2. 파일명을 입력하고, **Next** 혹은 **Finish**를 클릭합니다.

그림 11-9 파일명 지정



3. **Next**를 클릭한 경우, 아래와 같이 템플릿을 선택하는 창이 나타납니다. 템플릿을 사용할 경우, **Use template** 옵션을 선택하고 사용할 템플릿을 클릭합니다.
4. **Finish** 버튼을 클릭합니다.



5. 위에서 선택한 템플릿이 Design 탭에 표시되는지 확인합니다.

그림 11-10 템플릿 적용

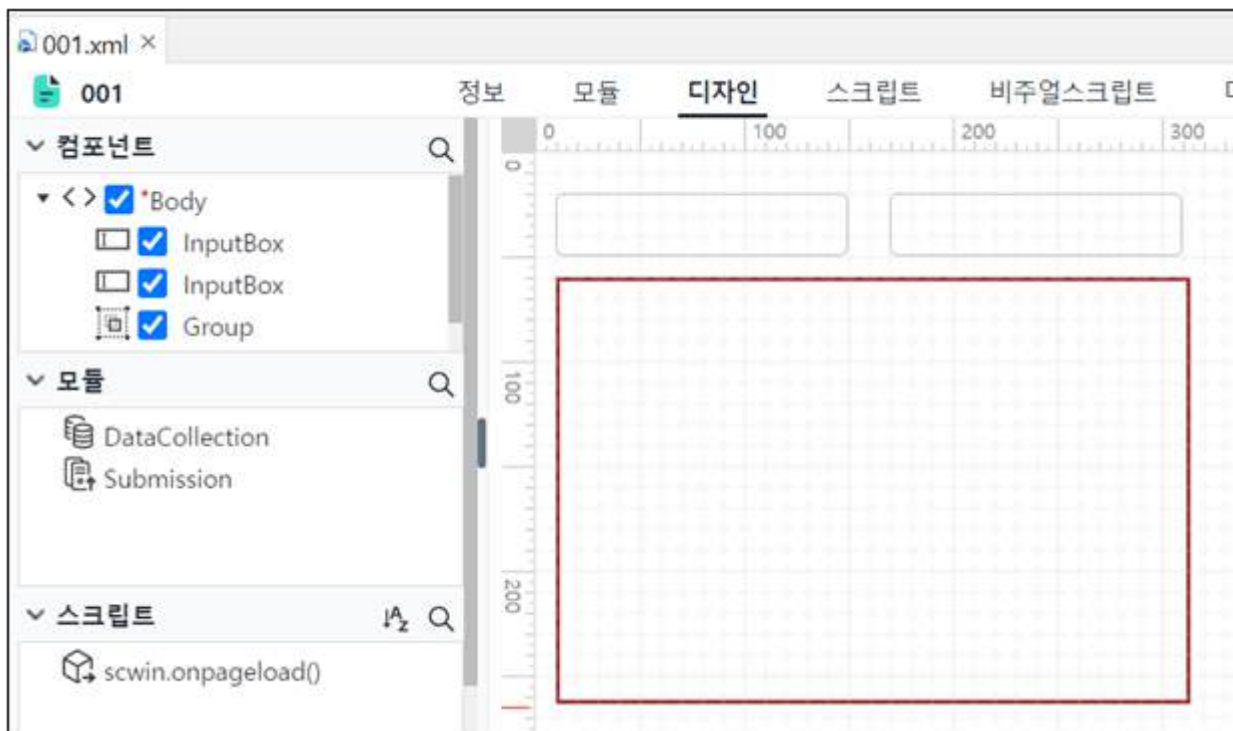


6. 8. [화면 그리기](#)를 참고하여 화면을 작성합니다.

11.2.3. 메소드 추가하기

1. 화면 파일을 작성합니다.

그림 11-11 화면 파일



2. 정보 - 메소드를 선택합니다.
3. 우측 상단의 메소드 정의 추가 버튼을 클릭합니다.



4. 메소드 정보 창이 표시되면 추가할 메소드의 이름과 메소드의 정보를 입력합니다.
5. 좌측 상단의 → 버튼을 클릭하여 메소드 정보 창을 닫습니다.

그림 11-12 메소드 정보 입력

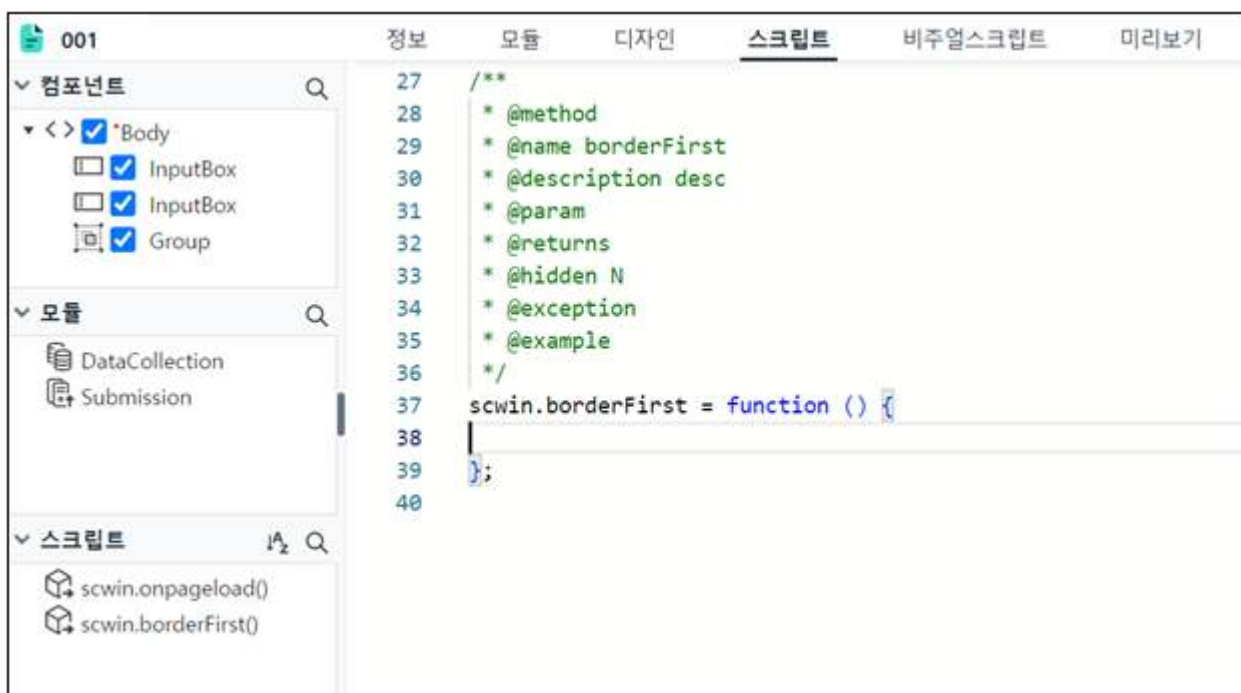


6. 우측 상단의 메소드 생성 버튼을 클릭합니다.

그림 11-13 메소드 생성



7. 아래와 같이 추가한 메소드를 구현할 수 있는 스크립트 화면이 자동으로 표시됩니다.



8. 추가한 메소드를 작성합니다.



9. 동일한 방법으로 메소드를 여러 개 생성할 수 있습니다.

메소드				메소드 마법사	메소드 생성	+	x
순번	숨김		메소드 서명	설명			
1			<code>borderFirst()</code>	Add border to first input box.			
2			<code>borderSecond()</code>	Add border to the second input box.			
3			<code>setBrown()</code>	Set the background color of the group.			
4			<code>showAll()</code>	Display values of the two input boxes.			

10. 메소드를 추가한 화면 파일을 PageFrame의 소스(src)로 사용할 경우, 해당 메소드에 직접 접근할 수 있습니다.
(11.2.4. PageFrame 소스로 사용하기 참고.)

Code 11-1 추가한 메소드 사용하기 (예시)

```

1 // PageFrame의 src가 위에서 메소드를 추가한 화면 파일이고,
2 // PageFrame의 id가 "page1"인 경우,
3 // 아래와 같이 추가한 메소드에 직접 접근 가능.
4
5 page1.borderFirst();

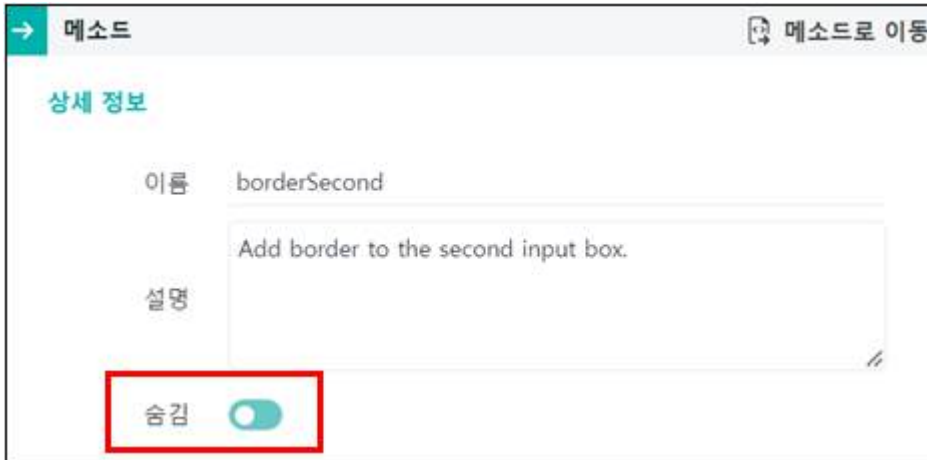
```

11. 또한 추가한 메소드를 비공개하여 실행을 막을 수도 있습니다. 메소드를 비공개하려면 메소드 앞의 수정 버튼을 클릭하고, 숨김 옵션을 선택하십시오.

그림 11-14 메소드 수정

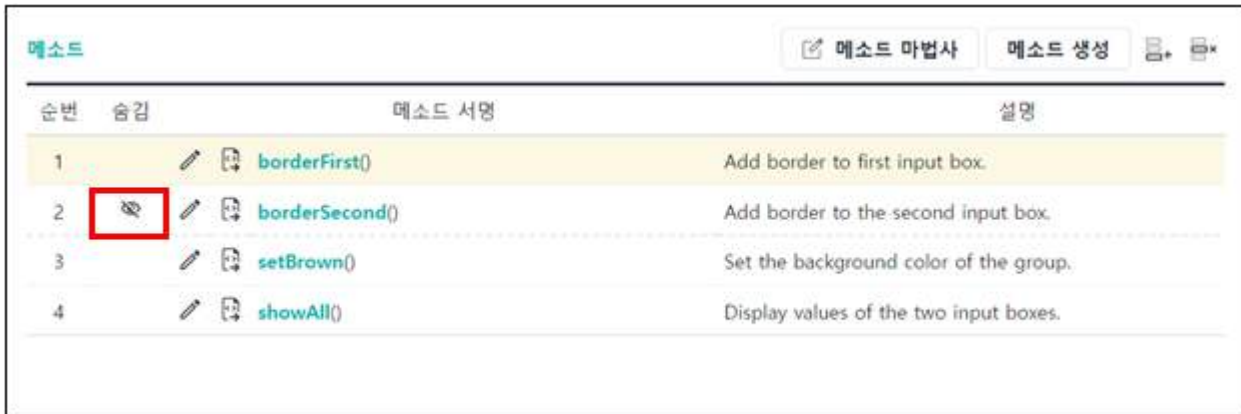
메소드			
순번	숨김		메소드 서명
1			<code>borderFirst()</code>
2			<code>borderSecond()</code>
3			<code>setBrown()</code>
4			<code>showAll()</code>

그림 11-15 메소드 숨김



12. 숨김 처리한 메소드 앞에는 숨김 아이콘이 표시됩니다.

그림 11-16 숨김 처리된 메소드



11.2.4. PageFrame 소스로 사용하기

메소드를 추가한 화면 파일을 PageFrame 컴포넌트의 소스(src)로 사용할 경우, 해당 메소드에 직접 접근할 수 있습니다.

Code 11-2 추가한 메소드 사용하기 (예시)

```

1 // PageFrame의 id가 "pageFrame1"이고,
2 // PageFrame의 src로 사용된 UDC 화면 파일이 "aaa()"라는 메소드를 포함할 경우,
3 // 아래와 같이 UDC의 메소드에 직접 접근 가능.
4
5 pageFrame1.aaa();

```



예제 파일

인터넷에서 다운로드 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/_Page_Component/method_Page_Componentmethod_Page_Component/)

그림 11-17 YouTube 동영상 (<https://youtu.be/q5B8xYNWFJA>)

Quick Guide WebSquare AI Page Component Adding Methods 메소드 추가하기	Goal / 목표 Add methods to Page Component to be accessible via PageFrame. / PageFrame에서 접근할 수 있는 메소드를 Page Component에 추가.
Example / 사용 예 Page_Component <pre>scwin.addedMethod = function () { ... };</pre> PageFrame <pre>pageFrame1.addedMethod();</pre>	YouTube  https://youtu.be/q5B8xYNWFJA



다른 컴포넌트에서의 사용

PageFrame을 사용하는 다른 컴포넌트에서도 동일한 방법으로 소스(src) 화면 파일의 메소드에 접근 가능합니다.

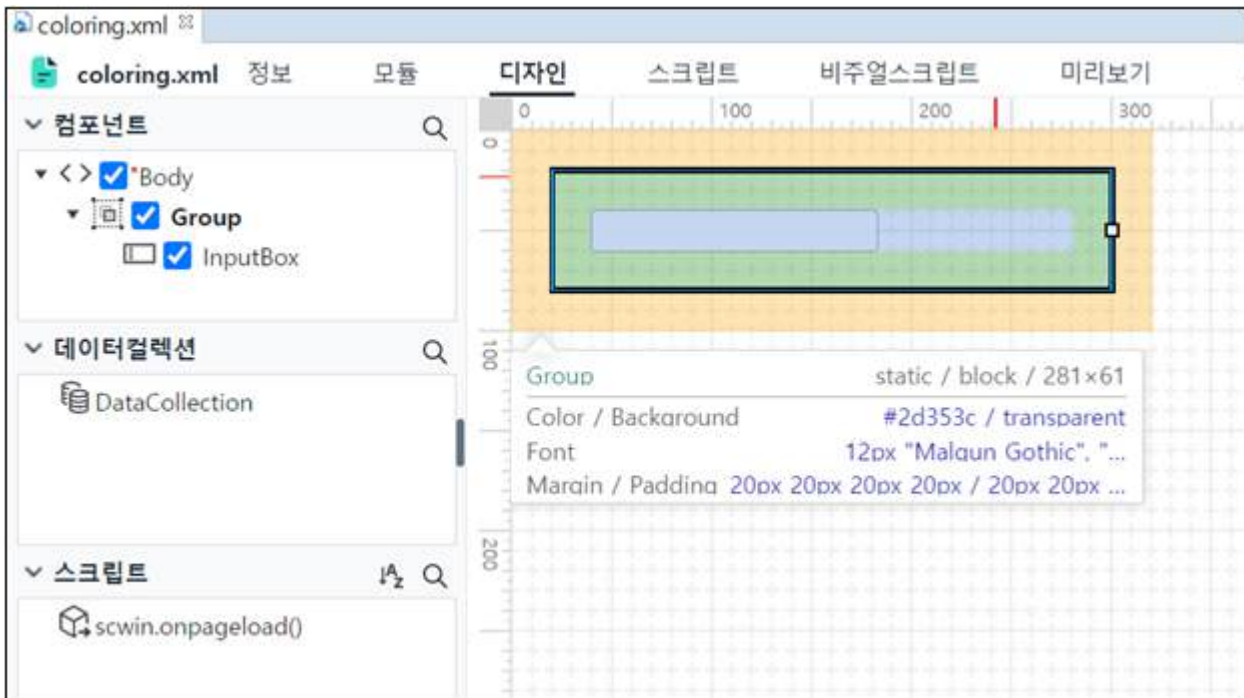
- TabControl - [PageFrame 탭](#)
- WindowContainer - [PageFrame 윈도우](#)
- 팝업 - [PageFrame 팝업](#)

11.3 User-Defined Ccomponent (UDC)

11.3.1. 생성하기

1. UDC로 사용할 화면 파일을 작성합니다.

그림 11-18 화면 파일 작성



2. 디자인 에디터의 정보 탭에서 문서화 정보 - 이름 항목을 포함한 문서화에 필요한 컴포넌트 정보를 입력합니다.

그림 11-19 정보 - 문서화 정보



3. 디자인 에디터의 정보 - 컴포넌트 화면에서 팔레트 등록 항목을 선택합니다.

그림 11-20 정보 - 컴포넌트



4. 디자인 에디터의 정보 - 속성 탭에서 행을 추가합니다.

그림 11-21 정보 - 속성



5. UDC에 추가할 속성을 입력하십시오.

그림 11-22 속성 정보

→ 속성

상세 정보

이름	color
형식	string
구분	01.Basic & ETC
기본값	
옵션	
바인딩 대상	
설명	배경색 지정
필수여부	☐

6. `$p.getOptions()` 함수를 사용하여 Script 탭에서 속성 값을 구현하십시오. (API 혹은 이벤트를 추가한 후 `met hod` 생성 혹은 `event` 생성 버튼을 클릭할 경우 추가한 속성에 대한 주석이 Script 탭에 자동 생성됩니다.)

그림 11-23 color 속성 구현 예시

```

58  /**
59   * @property
60   * @name color
61   * @category 05.Design
62   * @type string
63   * @option
64   * @default orange
65   * @necessary N
66   * @bindparent
67   * @description desc
68   */
69
70
71  scwin.onpageload = function() {
72      var options = $p.getOptions();
73      var background = options.color;
74      group1.setStyle("background-color", background);
75  };
76

```

7. 디자인 에디터의 정보 - 메소드 화면에서 행을 추가합니다.

그림 11-24 메소드 행 추가



8. 생성할 함수명과 설명을 입력합니다.

그림 11-25 함수명과 설명을 입력

→ 메소드

상세 정보

이름

addBorder

설명

경계선을 동적으로 설정

숨김

☐

매개 변수

+

×

↑

↓

순번	형식	이름	설명
조회 결과가 없습니다.			

반환 값

형식

이름

설명

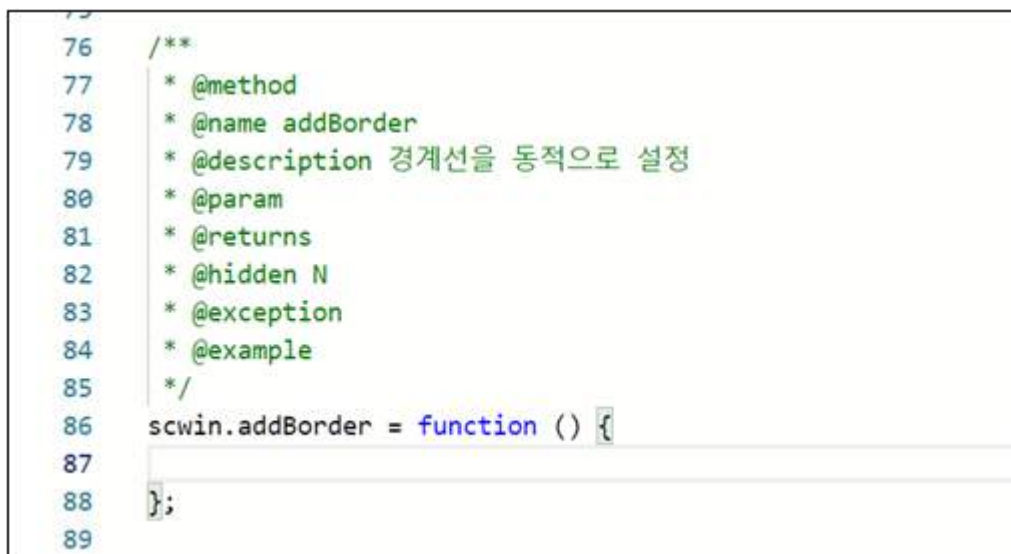
9. 메소드 생성 버튼을 클릭합니다.

그림 11-26 메소드 생성 버튼



10. 앞에서 추가한 메소드에 대한 주석과 코드가 자동으로 생성됩니다.

그림 11-27 자동 생성된 주석과 코드



11. 해당 메소드를 구현합니다.

그림 11-28 UDC 메소드 구현 예시



12. 디자인 에디터의 정보 - 이벤트 탭에서 행을 추가합니다.

그림 11-29 정보 - 이벤트



13. 생성할 이벤트 명과 설명을 입력합니다.

그림 11-30 이벤트 정보 입력

→ 이벤트

상세 정보

이름 oncolorchange

설명

배경색이 변경될 때 발생.

매개 변수

+

×

↑

↓

순번	형식	이름	설명
조회 결과가 없습니다.			

예제

14. 이벤트 생성 버튼을 클릭합니다.

그림 11-31 이벤트 생성



15. 앞에서 추가한 이벤트에 대한 주석과 코드가 자동으로 생성된 것을 확인합니다.

그림 11-32 이벤트 발생 함수 정의

```

90  /**
91   * @event
92   * @name oncolorchange
93   * @description 배경색이 변경될 때 발생.
94   * @param
95   * @example
96   */
97  scwin.emit_oncolorchange = function () {
98      $p.emit("oncolorchange");
99  };
100

```

16. 이벤트 발생 시점을 정의합니다.

그림 11-33 이벤트 발생 시점 정의 (예시)

```

101  scwin.input1_oneditenter = function() {
102      var color = input1.getValue();
103      group1.setStyle( "background-color", color);
104      $p.emit("oncolorchange");
105  };
106

```

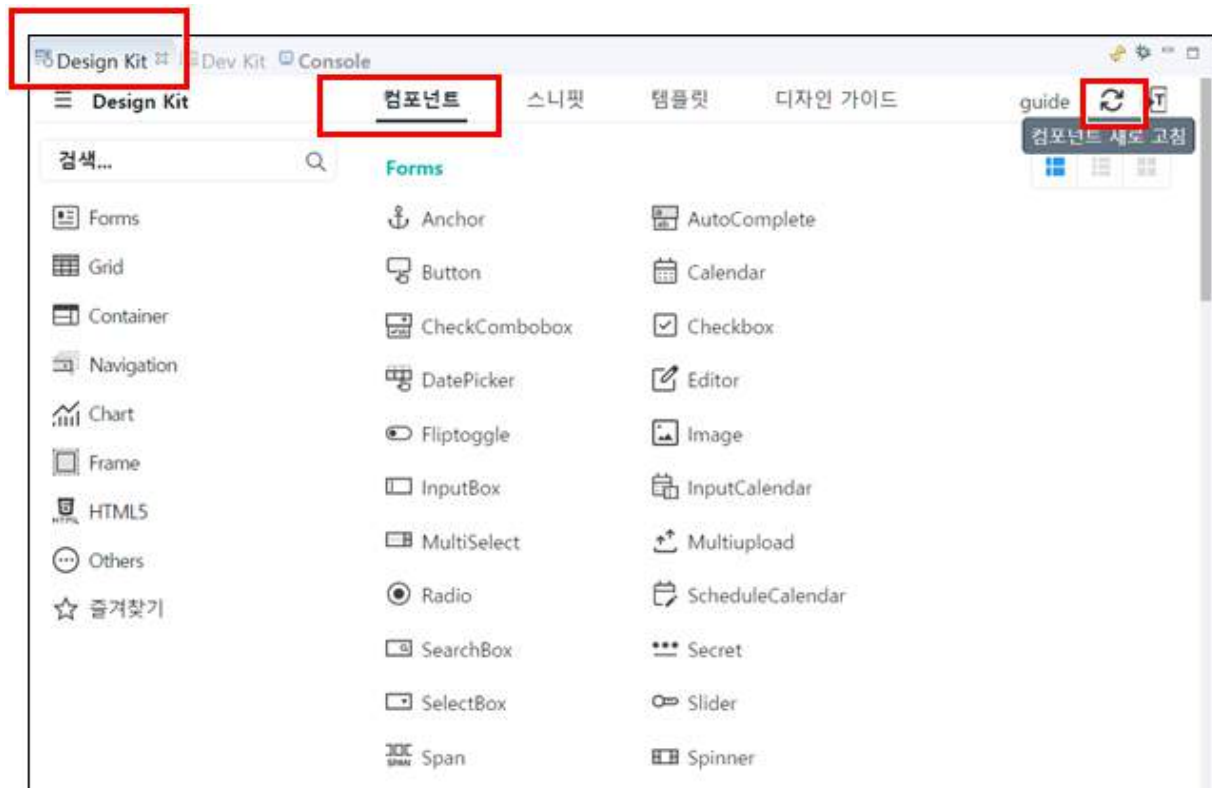
17. 화면 파일을 /cm/udc/ 폴더로 이동합니다.

그림 11-34 /cm/udc/ 폴더



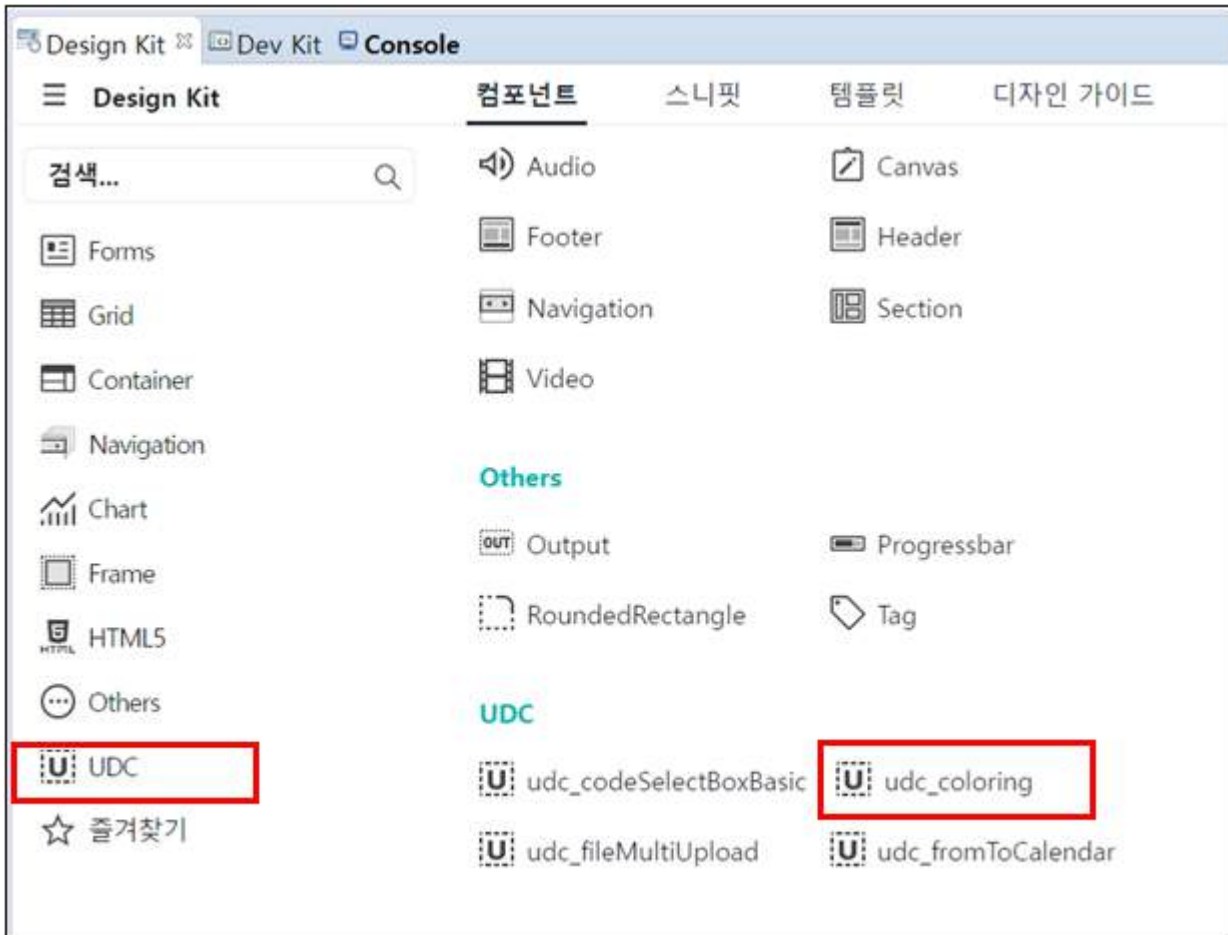
18. Design Kit의 컴포넌트 탭에서 컴포넌트 새로 고침 버튼을 클릭합니다.

그림 11-35 컴포넌트 새로 고침



19. `/cm/udc/` 폴더로 이동한 화면 파일이 UDC로 표시되는지 확인합니다.

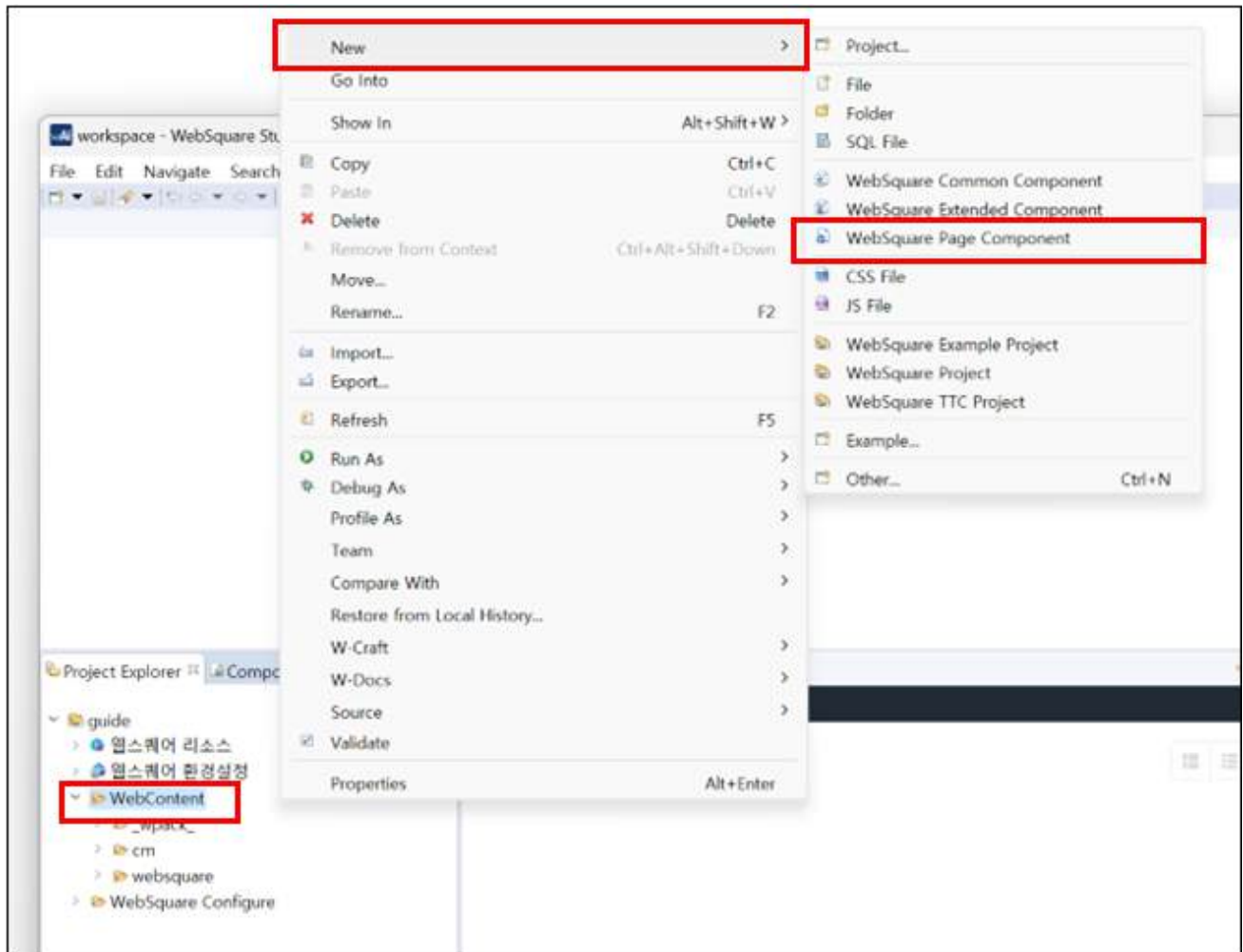
그림 11-36 추가된 UDC



11.3.2. 사용하기

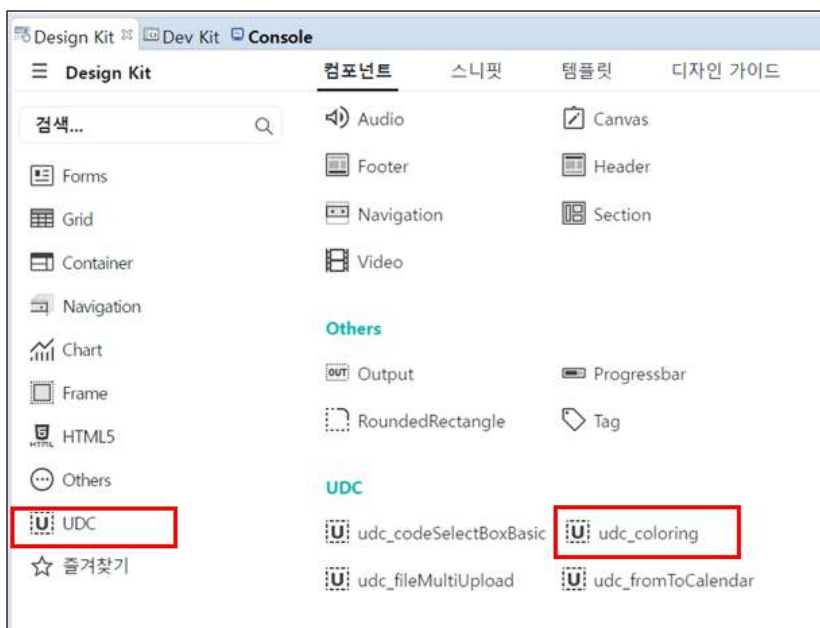
1. WebSquare Page Component를 새로 생성합니다.

그림 11-37 New - WebSquare Page Component



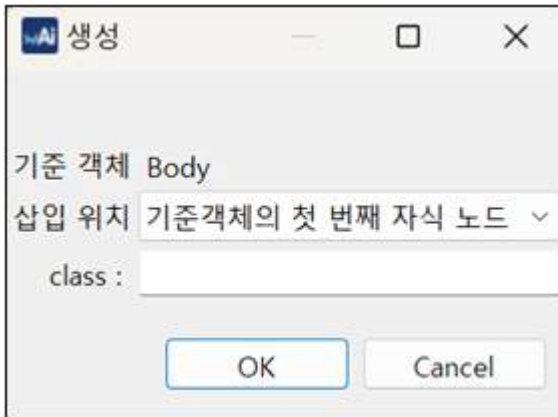
2. Design Kit의 컴포넌트 탭에 추가한 UDC가 표시되는지 확인합니다.

그림 11-38 추가된 UDC



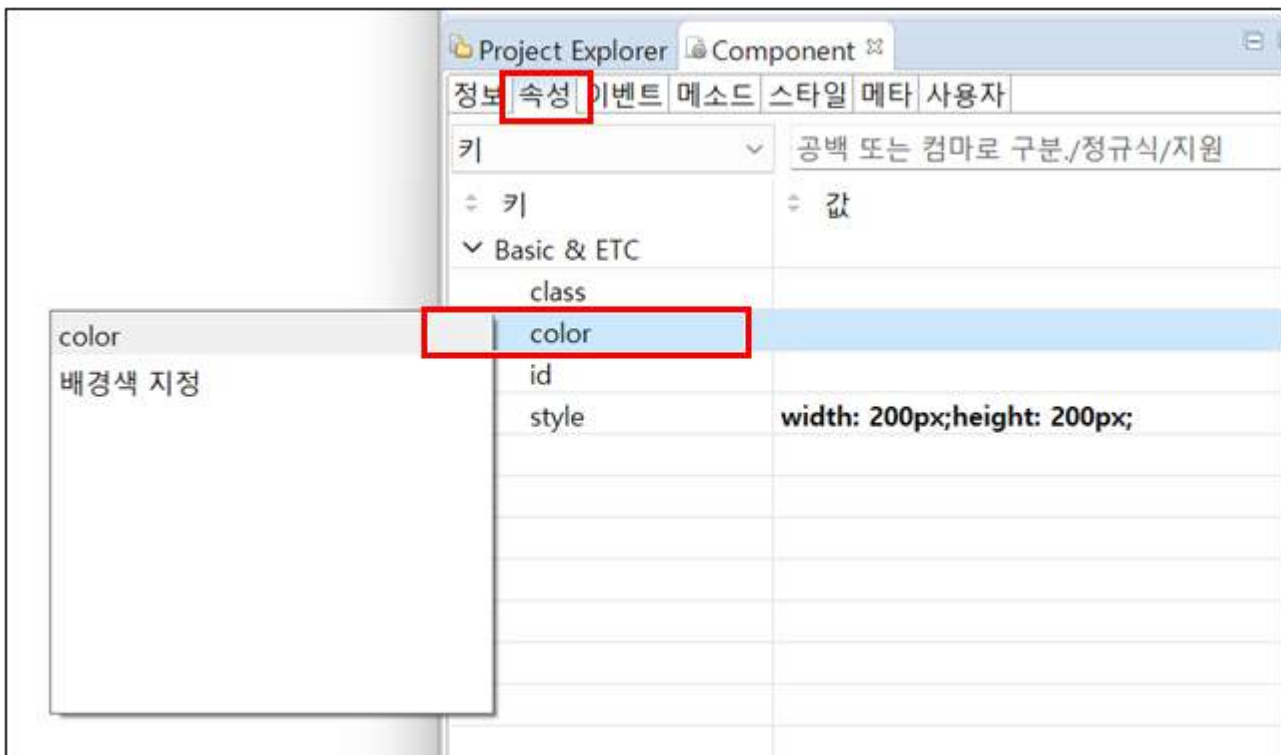
3. 해당 UDC를 클릭하여 화면 파일에 추가합니다.

그림 11-39 UDC 추가



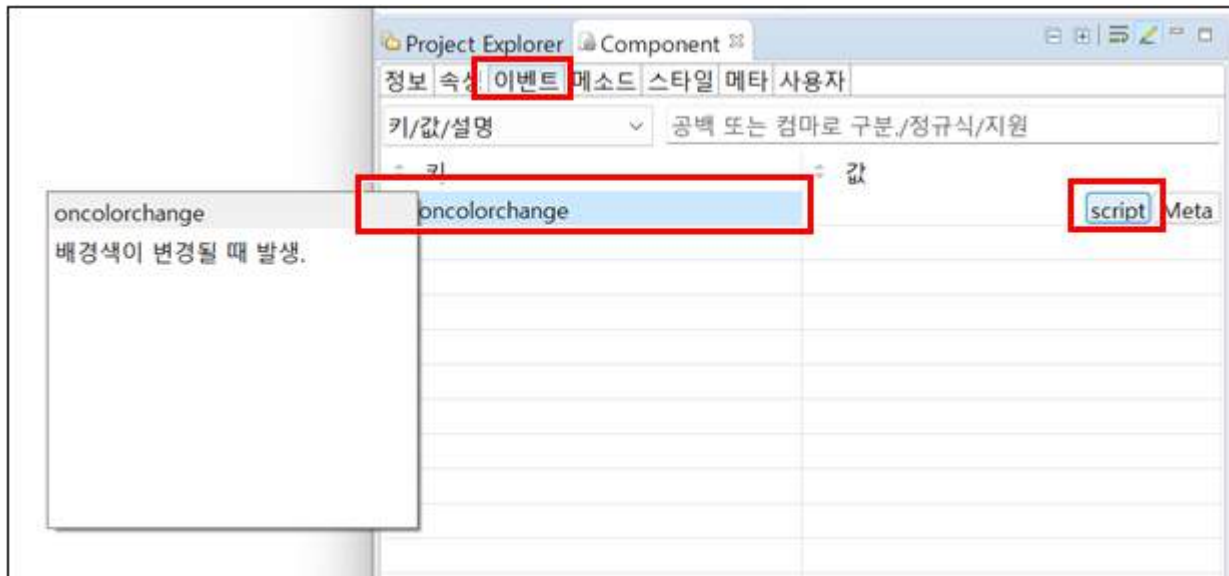
4. UDC 생성 시 추가한 속성이 **Component** 뷰의 속성 탭에 표시되는지 확인합니다.

그림 11-40 UDC 속성



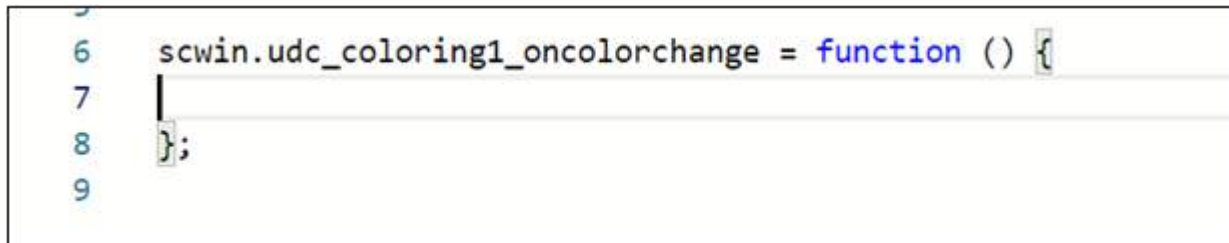
5. UDC 생성 시 추가한 이벤트가 **Component** 뷰의 이벤트 탭에 표시되는지 확인합니다.

그림 11-41 UDC 이벤트



6. **script** 버튼을 클릭하면 Script 탭에 이벤트 추가용 코드가 아래와 같이 자동 생성됩니다.

그림 11-42 이벤트 핸들러 스크립트 자동 생성



Code 11-3 UDC 함수 사용 예시

```
1 udc_coloring1.addBorder();
```

Code 11-4 UDC 이벤트 사용 예시

```
1 scwin.udc_coloring1_oncolorchange = function () {
2     alert("배경색이 변경되었습니다.");
3 };
```

7. 추가한 UDC의 ID를 지정한 후, UDC 생성 시 추가한 함수 및 이벤트를 활용하여 화면을 개발합니다.

11.4 Extended Component (EXC)

11.4.1. 소개

EXC(Extended Component)는 사용자들이 기존 컴포넌트에 추가로 속성, 이벤트, 함수를 정의하여 사용할 수 있는 기능을 제공합니다. 기존의 웹스퀘어 컴포넌트를 EXC로 생성할 경우, 해당 컴포넌트의 기능은 그대로 상속받는 동시에 추가로 필요한 속성, 이벤트, 함수를 생성하여 사용할 수 있습니다.

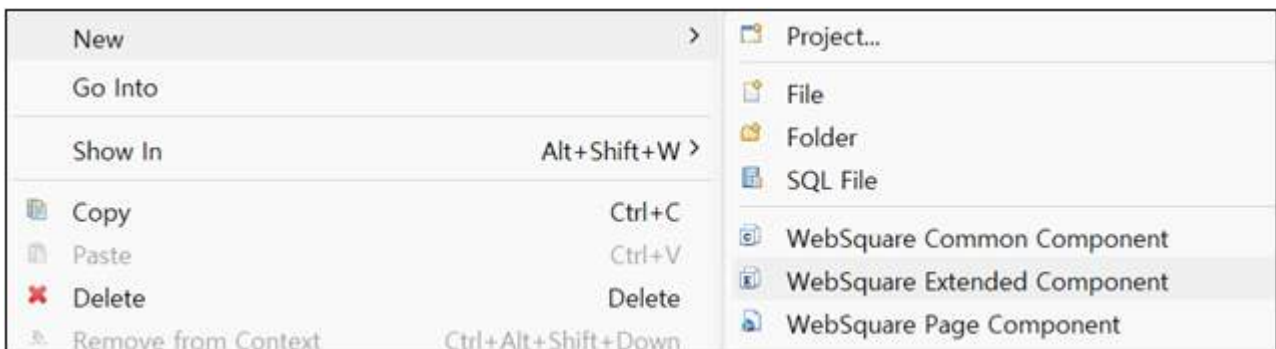
11.4.2. UDC Vs. EXC

항목	UDC	EXC
구현 방법	UDC의 모든 속성, 이벤트, 함수를 구현해야 함.	기본 컴포넌트의 모든 속성과 이벤트, 함수를 상속받음.
사용 컴포넌트	여러 컴포넌트를 조합하여 한 개의 UDC로 구현 가능.	기본 컴포넌트 한 개 만 확장 가능.
(Group 컴포넌트 관련) 제약 사항	Group 컴포넌트를 이용하여 UDC를 생성할 경우, UDC가 다른 컴포넌트를 포함할 수 없음.	Group 컴포넌트를 상속하는 경우, Group 컴포넌트로 동작 가능. 즉, 다른 컴포넌트를 포함할 수 있음.

11.4.3. 생성하기

1. 프로젝트를 우클릭한 후, **New - WebSquare Extended Component**를 선택합니다.

그림 11-43 New - WebSquare Extended Component



2. 한 개의 컴포넌트만 사용하여 EXC로 사용할 화면 파일을 작성합니다.



한 개의 컴포넌트를 확장한 것이 EXC

한 개의 컴포넌트만 EXC로 등록 가능합니다. 여러 개의 컴포넌트가 있을 경우, 첫번째 컴포넌트만 EXC로 등록됩니다.



EXC의 속성, 함수, 이벤트는 UDC 생성하기와 동일한 방법으로 추가할 수 있습니다.

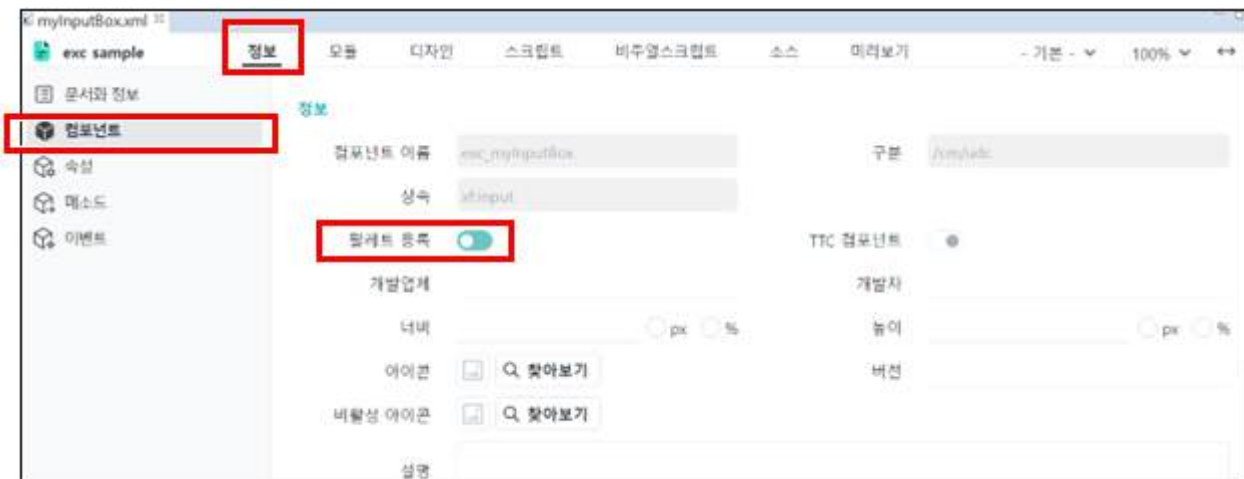
3. 디자인 에디터의 정보 탭에서 문서화 정보 - 이름 항목을 포함한 문서화에 필요한 컴포넌트 정보를 입력합니다.

그림 11-44 정보 - 문서화 정보



4. 디자인 에디터의 정보 - 컴포넌트 화면에서 팔레트 등록 항목을 선택합니다.

그림 11-45 정보 - 컴포넌트

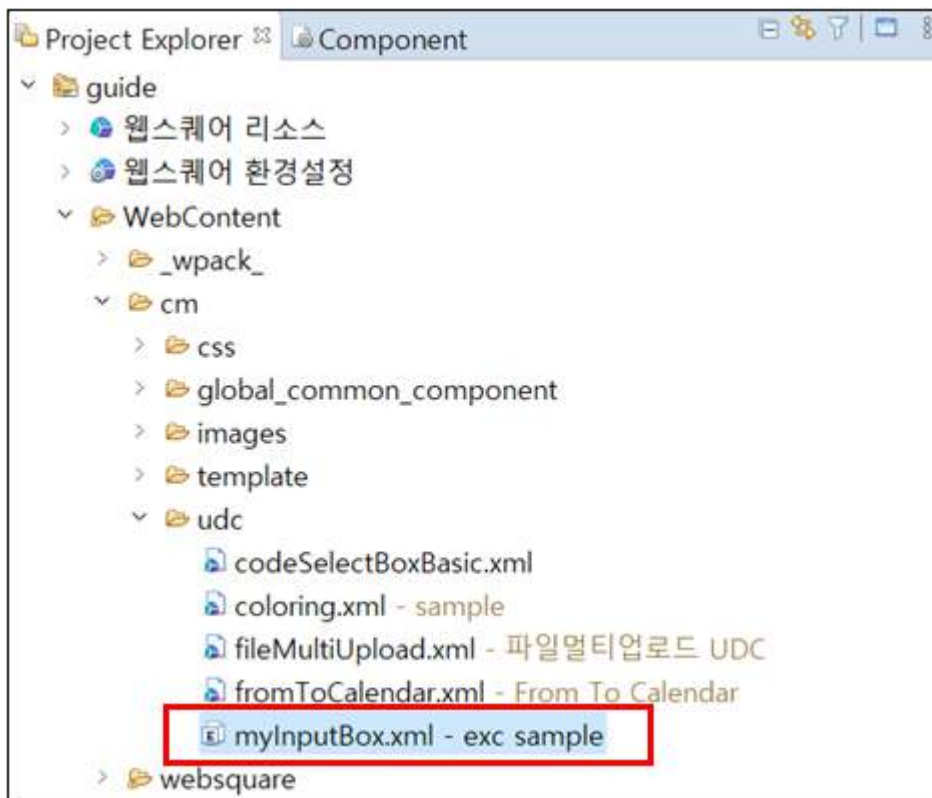


5. 디자인 에디터의 정보 탭을 클릭하여 EXC 컴포넌트에 추가할 속성, 함수, 이벤트를 정의합니다.

그림 11-46 EXC의 속성, 이벤트, 메소드 추가

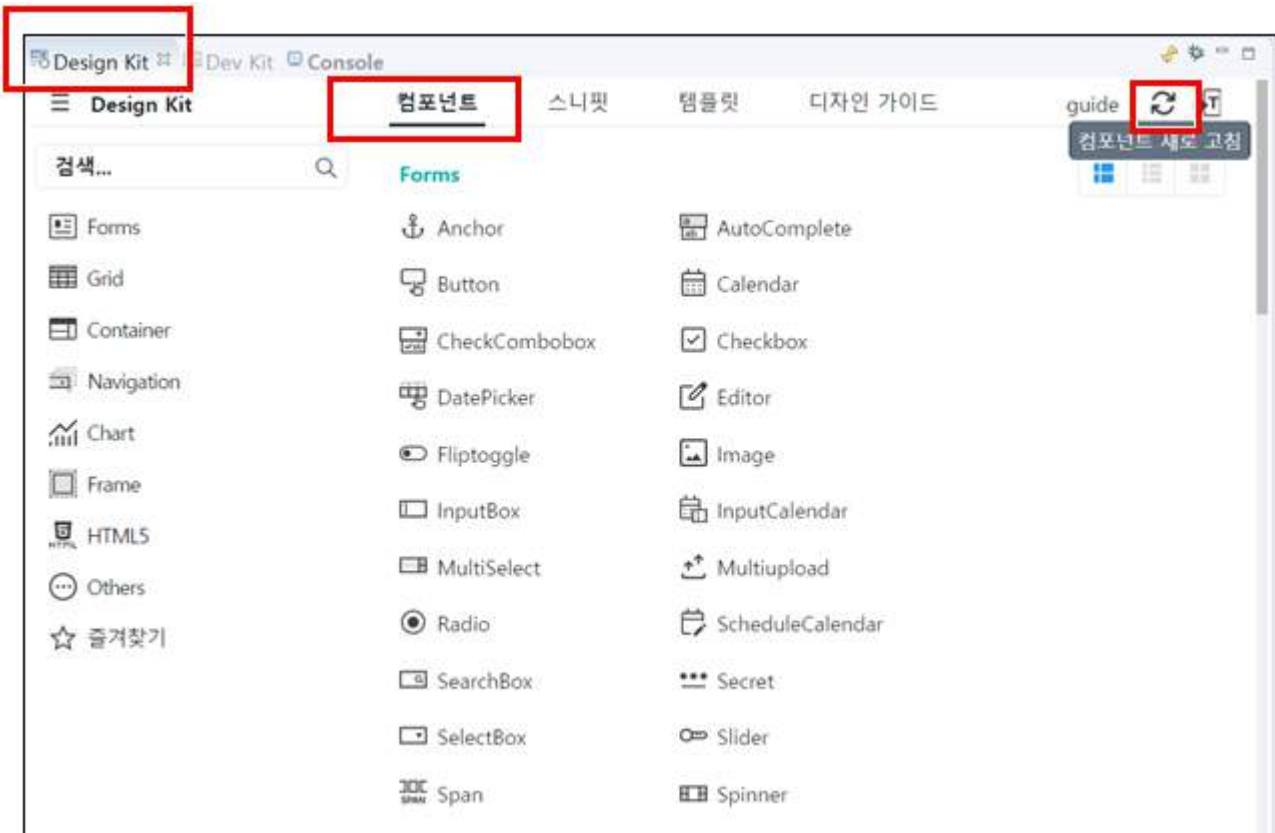


6. 화면 파일을 /cm/udc/ 폴더로 이동합니다.



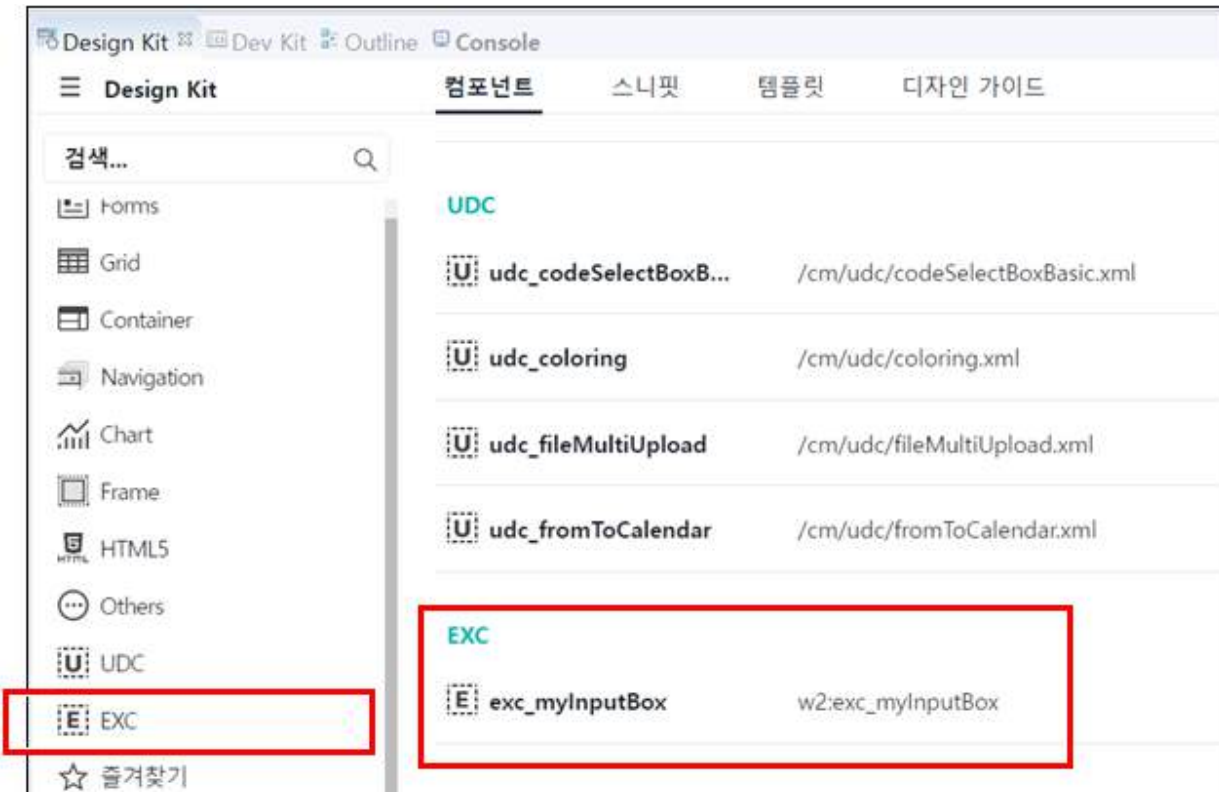
7. Design Kit의 컴포넌트 탭에서 컴포넌트 새로 고침 버튼을 클릭합니다.

그림 11-47 컴포넌트 새로 고침



8. /cm/udc/ 폴더로 이동한 화면 파일이 UDC로 표시되는지 확인합니다.

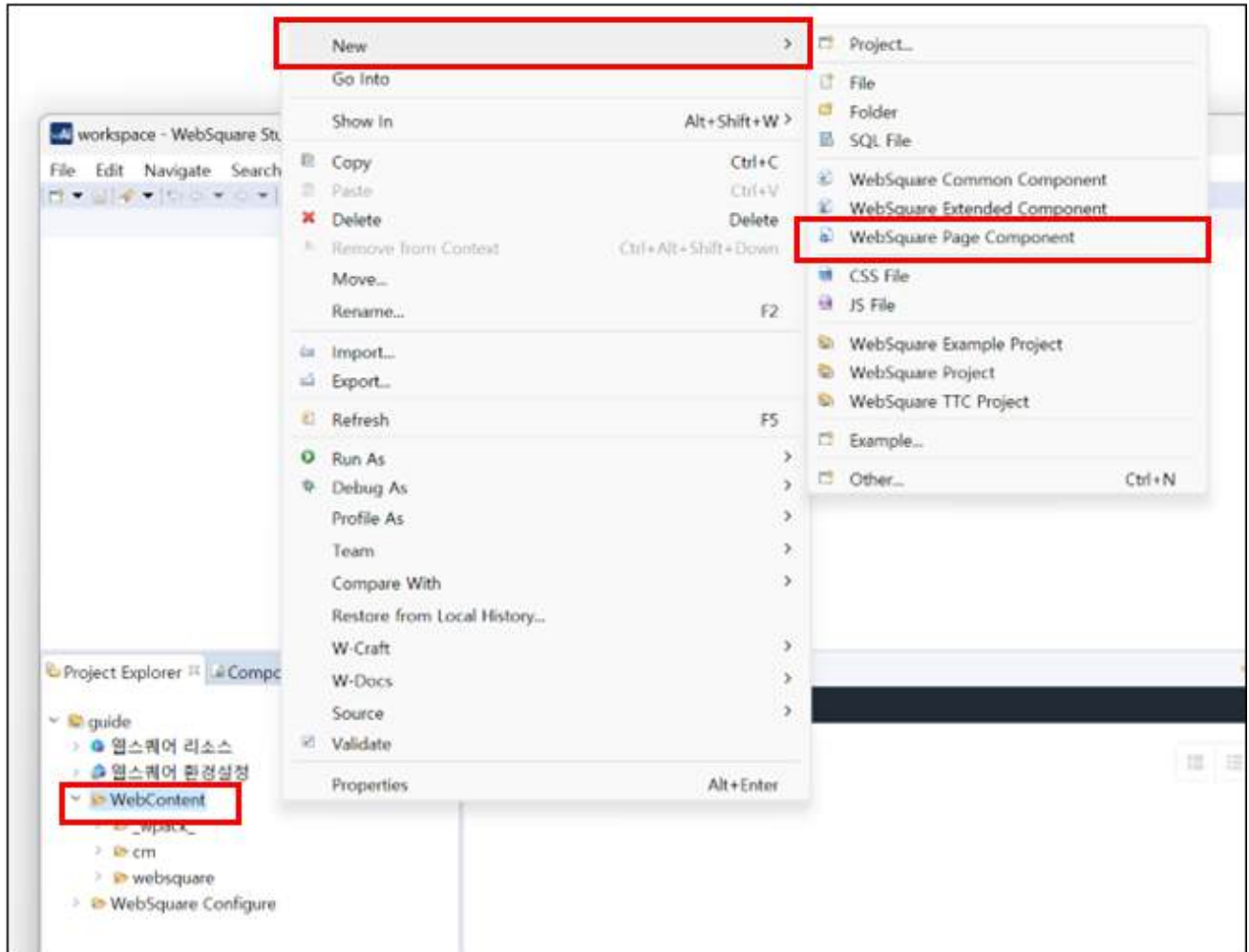
그림 11-48 추가된 EXC



11.4.4. 사용하기

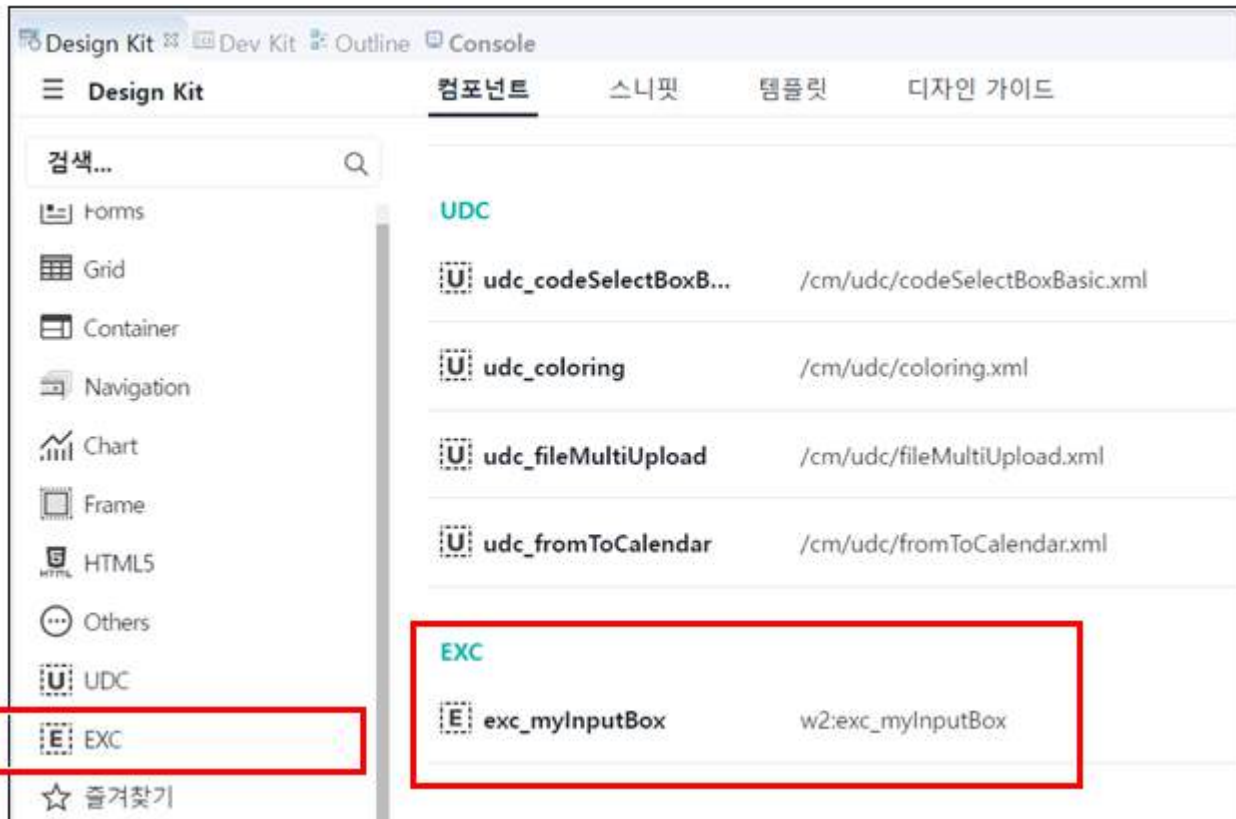
1. WebSquare Page Component를 새로 생성합니다.

그림 11-49 New - WebSquare Page Component



2. Design Kit의 컴포넌트 탭에 추가한 EXC가 표시되는지 확인합니다.

그림 11-50 추가된 EXC



3. 해당 EXCC를 클릭하여 화면 파일에 추가합니다.
4. 이후 사용 방법은 UDC 11.3.2. 사용하기 부분에 설명된 내용과 동일합니다.

11.5 Trusted Third-party Component (TTC)

11.5.1. 소개

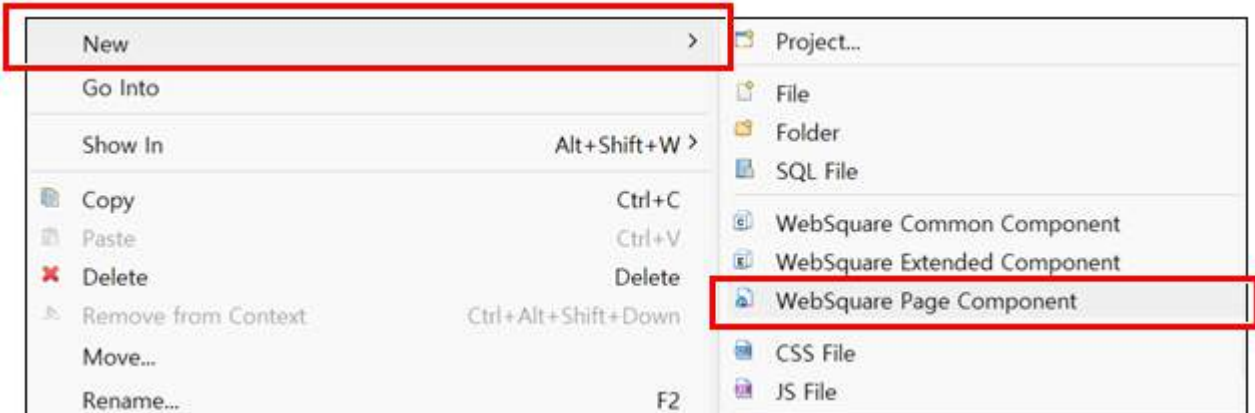
TTC는 웹스퀘어 AI 혹은 그외에 HTML5 표준을 준수하는 기능들을 웹스퀘어 AI에서 쉽게 사용할 수 있도록 제공하는 플러그인 방식의 컴포넌트입니다. 개발자는 앱스토어 처럼 TTC를 등록하고 사용할 수 있습니다.

웹스퀘어 페이지와 동일하게 구성하되, license, css, image 등 필요한 리소스를 정해진 규격에 맞게 작성해야 합니다.

11.5.2. 생성하기

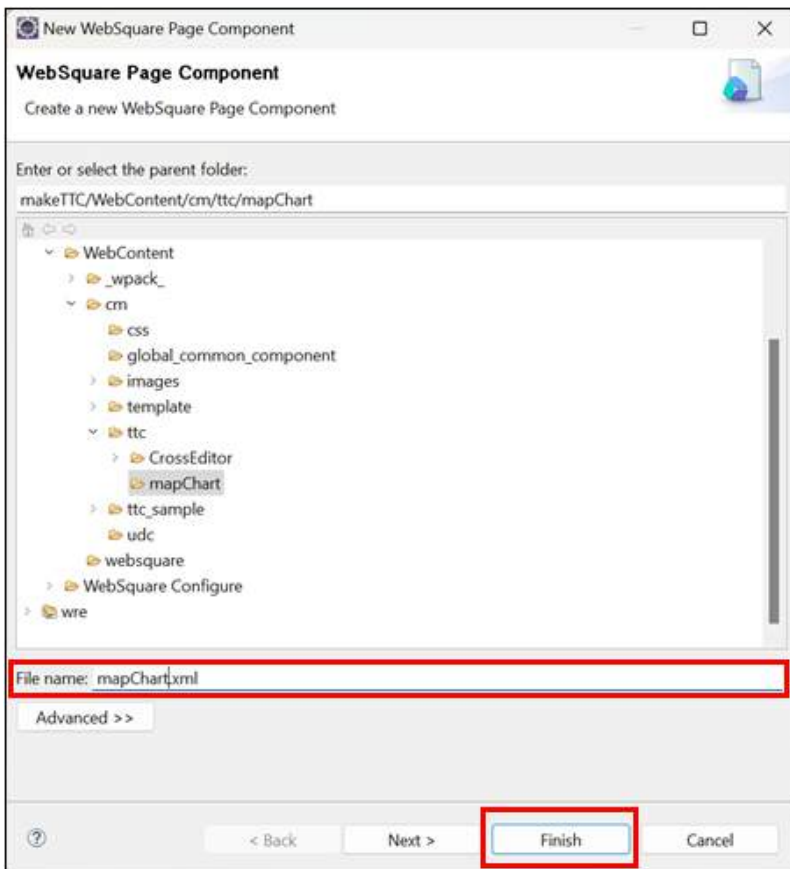
1. WebSquare TTC Project를 생성합니다. (10.3 WebSquare TTC Project 참고.) 혹은, 10.3 WebSquare TTC Project - 10.3.2. 폴더 구조를 참고하여 TTC용 폴더 구조를 직접 구성하십시오.
2. /프로젝트/WebContent/cm/ttc/TTC 이름/ 폴더를 우클릭하고 New - WebSquare Page Component 메뉴를 선택합니다.

그림 11-51 New - WebSquare Page Component



3. TTC 이름을 파일명으로 지정합니다.

그림 11-52 파일명 지정



4. 디자인 에디터에 화면이 생성되면, **API - 컴포넌트** 탭에서 팔레트 등록 및 TTC 컴포넌트 옵션을 선택합니다.

그림 11-53 API - 컴포넌트



5. 컴포넌트 정보를 입력합니다.

그림 11-54 컴포넌트 정보 입력

컴포넌트

컴포넌트 이름 구분

팔레트 등록 ☒ TTC 컴포넌트 ☒

개발업체 개발자

너비 px ☐ % 높이 px ☐ %

아이콘 ☐ 버전

비활성 아이콘 ☐

설명

HTML 렌더

라이선스

예제 이미지 경로

홈페이지

항목	설명
	찾아보기... 버튼을 클릭하여 TTC 컴포넌트를 표시할 때 사용할 이미지의 경로를 선택.

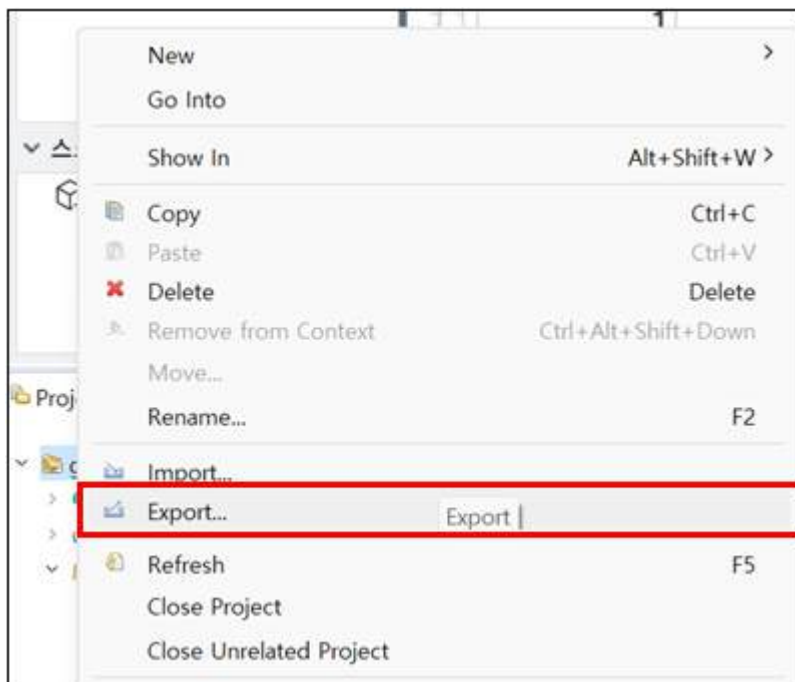
항목	설명
HTML 렌더	(예시) /cm/ttc/ttc_이름/sampleImages/ 폴더에 있는 이미지를 선택.
라이선스	찾아보기... 버튼을 클릭하여 /ttc_resource/ttc_이름//license/ 폴더에 존재하는 HTML 라이선스 파일을 선택합니다. (중요) 라이선스 파일이 있어야 TTC를 가져올 수 있습니다.
예제 이미지 경로	TTC 가져오기를 하거나 TTC 미리보기, 혹은 TTC 컴포넌트를 생성할 경우 사용할 이미지들이 저장된 폴더를 선택합니다.

6. TTC로 사용할 화면 파일을 작성합니다. (8. [화면 그리기](#) 참고.)

11.5.3. 내보내기

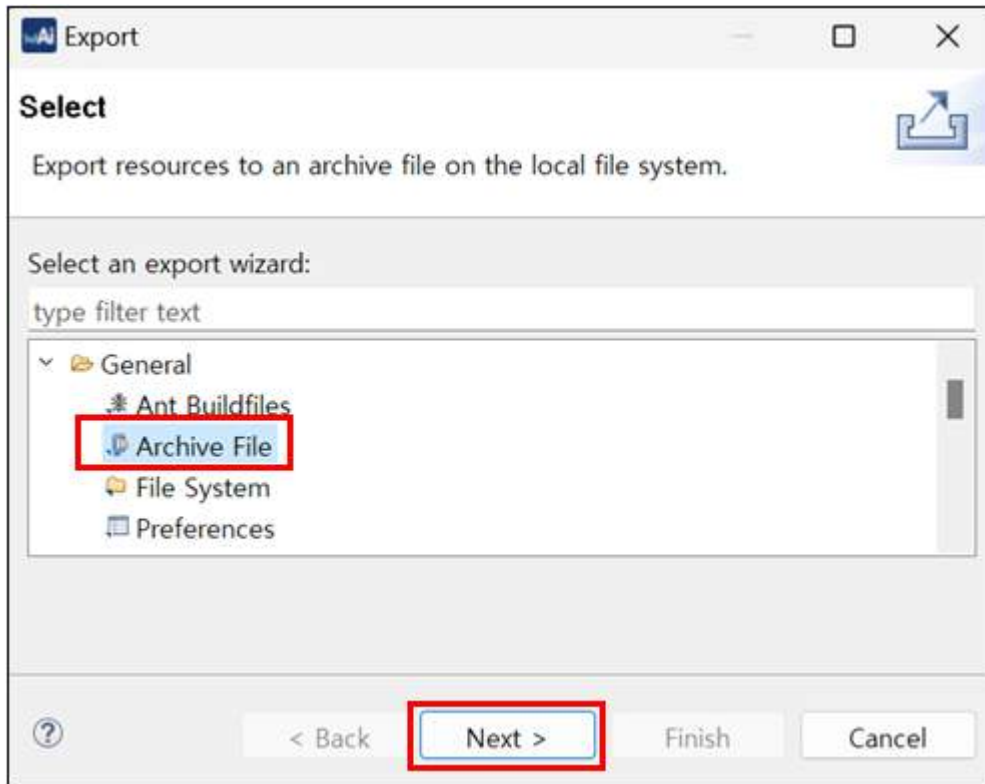
1. 프로젝트를 우클릭하고 **Export**를 선택합니다.

그림 11-55 프로젝트 우클릭 - Export



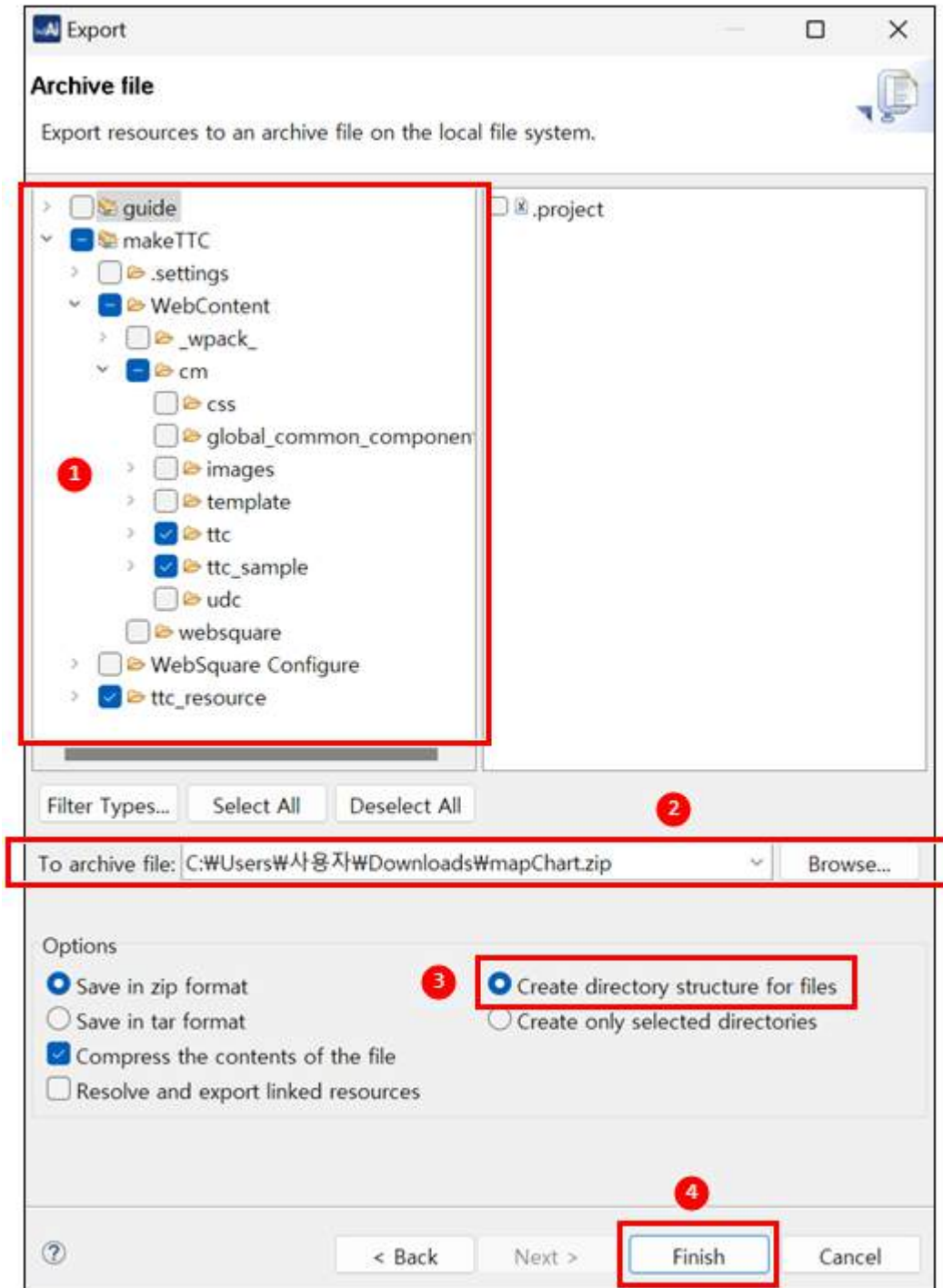
2. **General - Archive File** 선택 후, **Next** 버튼을 클릭합니다.

그림 11-56 내보내기



3. 아래의 폴더를 선택합니다.
 - I. `/ttc/`
 - II. `/ttc_sample/`
 - III. `/ttc_resource/`
4. TTC를 다운로드할 디렉토리와 ZIP 파일 이름을 지정합니다.
5. **Create directory structure for files** 옵션을 선택합니다.
6. **Finish** 버튼을 클릭합니다.

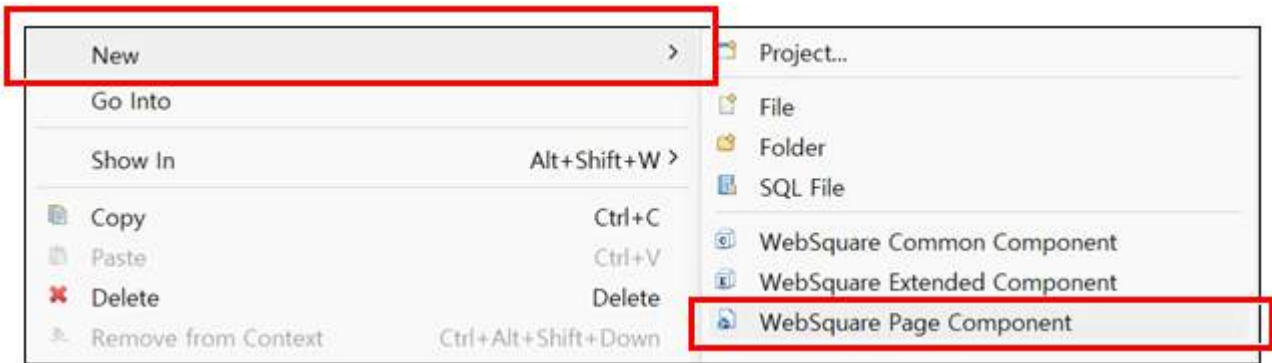
그림 11-57 내보내기



11.5.4. 가져오기

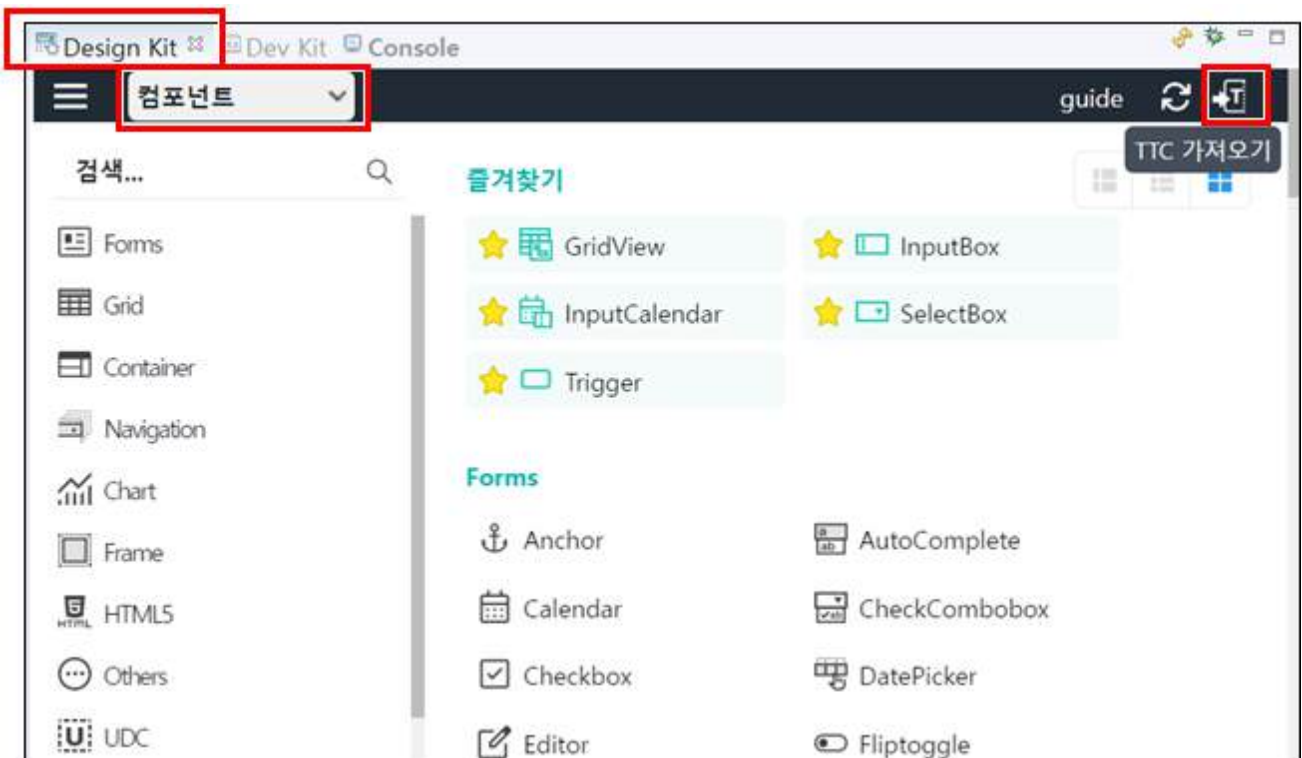
1. 웹스퀘어 페이지를 생성합니다.

그림 11-58 New - WebSquare Page Component



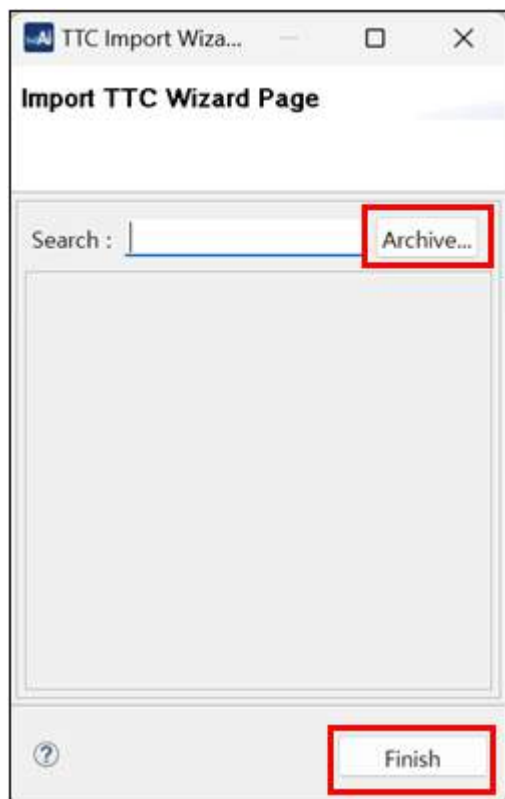
2. 디자인 탭 혹은 비주얼스크립트 탭으로 이동합니다.
3. Design Kit 우측 상단의 TTC 가져오기 버튼을 클릭합니다.

그림 11-59 TTC 가져오기



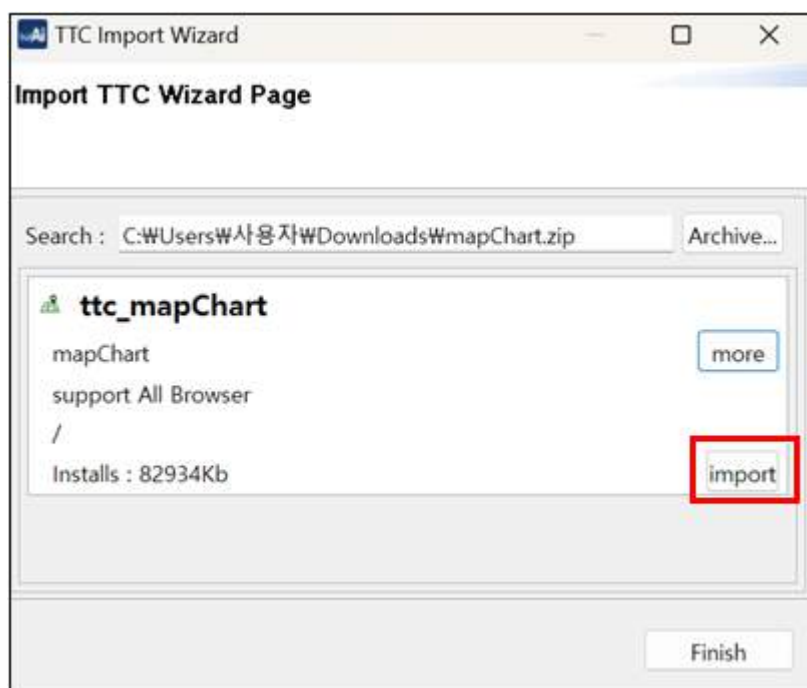
4. Archive... 버튼을 클릭합니다.
5. TTC의 ZIP파일을 선택하고, Finish 버튼을 클릭합니다.

그림 11-60 TTC 가져오기



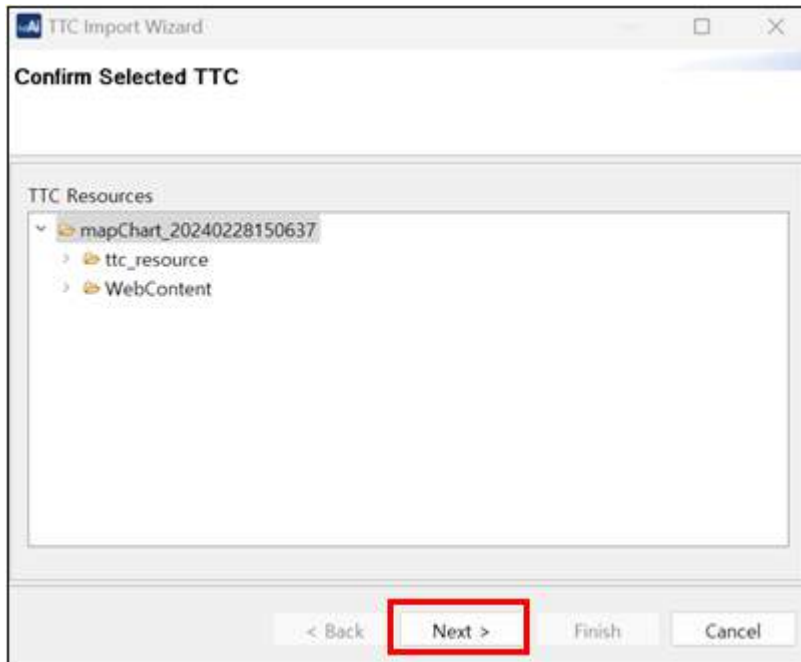
6. 아래와 같은 창이 나타나면 **Import** 버튼을 클릭합니다.

그림 11-61 TTC 가져오기



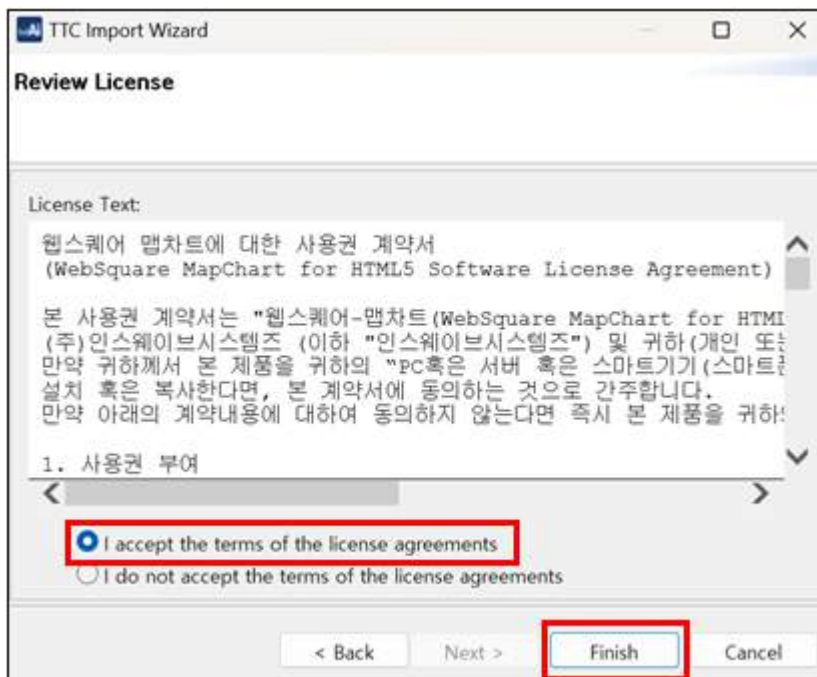
7. 등록할 TTC를 확인한 후 **Next** 버튼을 클릭합니다.

그림 11-62 TTC 가져오기



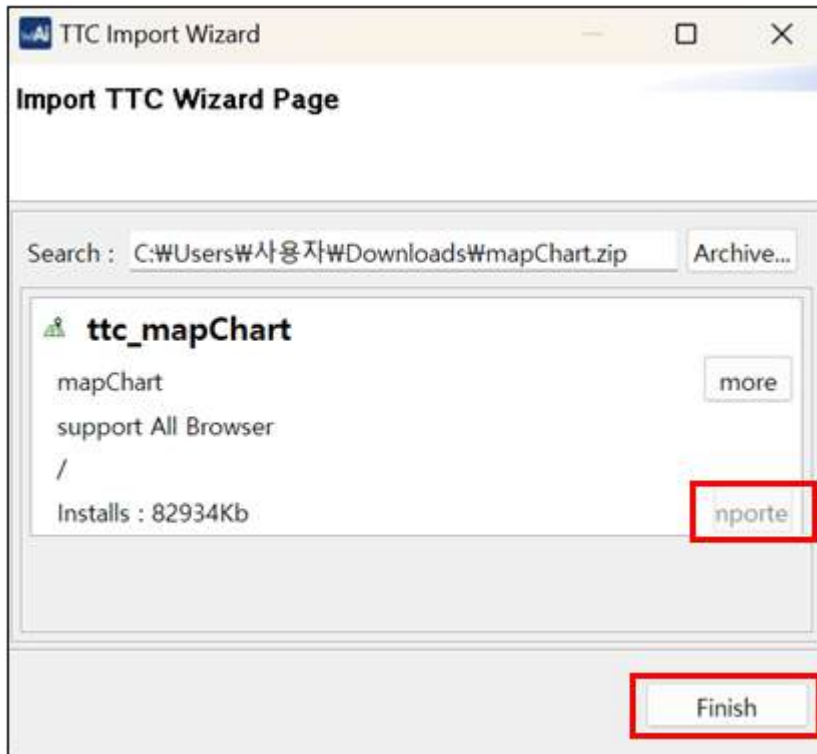
8. 아래와 같은 라이선스 창이 표시되면, 동의하고 **Finish** 버튼을 클릭합니다.

그림 11-63 라이선스 사용 동의



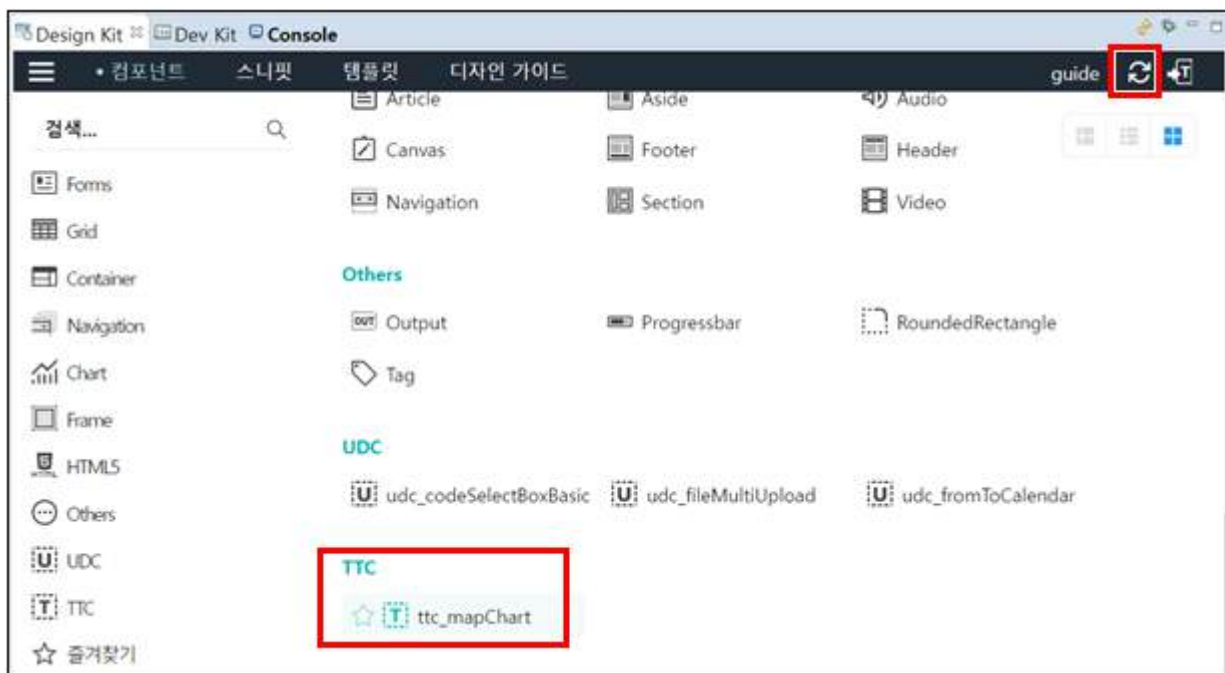
9. **Import** 버튼이 **Imported**로 변한 것을 확인하고, **Finish** 버튼을 클릭합니다.

그림 11-64 TTC 가져오기



10. Design Kit - 컴포넌트 탭 우측 상단의 새로고침 버튼을 클릭합니다.
11. 추가한 TTC 컴포넌트가 표시되는지 확인합니다.

그림 11-65 TTC 등록



12. Design Kit - 컴포넌트 탭에서 등록한 TTC를 클릭하면 화면 파일에 추가하여 사용할 수 있습니다. TTC를 생성할 때 추가한 속성, 메소드, 이벤트를 일반 컴포넌트와 동일한 방법으로 사용할 수 있습니다.

12. DataCollection

12.1 DataCollection

12.1.1. 개요

정의

브라우저의 메모리에 할당되는 JavaScript 데이터 객체.

DataCollection은 다양한 종류의 방대한 데이터를 웹 브라우저에서 쉽게 제어하기 위해 웹스퀘어 AI가 사용하는 데이터 모델.

종류

DataMap: 단건 데이터 객체

DataList: 다건 데이터 객체

LinkedDataList: DataList 객체에 Filter/Sort를 적용한 결과

Table 12-1 DataCollection 모델

모델	설명
dataMap	key/value 형태의 단일 데이터로 구성된 객체
dataList	리스트 형태의 다건의 데이터로 구성된 객체
linkedDataList	dataList를 필터링/정렬한 후, 그 결과만 제공

기능

서버에 전달할 데이터 규격을 정의 (Reference)

서버로부터 받을 데이터 규격을 정의 (Target)



브라우저와 서버 간의 데이터 통신은 [14. Submission](#) 부분을 참고하십시오.

주요 용도

화면에서 사용하고자 하는 임시 데이터

UI컴포넌트와 연동

서버로부터 받은 단일형(DataMap) 혹은 목록형(DataList) 데이터를 브라우저에 출력

웹 브라우저로부터 원격의 서버에 존재하는 데이터를 (추가, 삭제, 편집 등) 관리할 수 있는 인터페이스 제공

특징

DataCollection은 XML 구조와 XPath에 대한 이해 없이도 데이터 구조를 이해할 수 있는 직관적인 인터페이스를 제공.

디자인 에디터에서 DataCollection을 설정할 경우, 해당 코드는 **스크립트** 탭에 자동 생성됨.

각 데이터 객체의 id는 필수값.

JSON, XML, JSON Array포맷으로 설정(set)하거나 또는 반환(get) 받을 수 있음. (LinkedList는 반환만 가능.)

초기값을 정의 가능. (LinkedList 제외)

동적(Script)으로 생성 가능.

데이터의 상태(추가/수정/삭제)값을 관리 가능.

12.1.2. 동적 생성

[\\$p.data\(\)](#) - DataCollection 동적 생성 참고.

12.1.3. 현재 화면에 포함된 DataCollection 반환

[\\$p.data\(\)](#) - 화면에 포함된 DataCollection 반환 참고.

12.1.4. 복사 및 삭제

특정 Page Component의 DataCollection을 복사해서 다른 Page Component에서 사용하는 것이 가능합니다.

Studio의 디자인 - 모듈 탭에서 복사/삭제할 DataCollection을 클릭한 후 해당 메뉴를 선택하십시오.

그림 12-1 복사하기

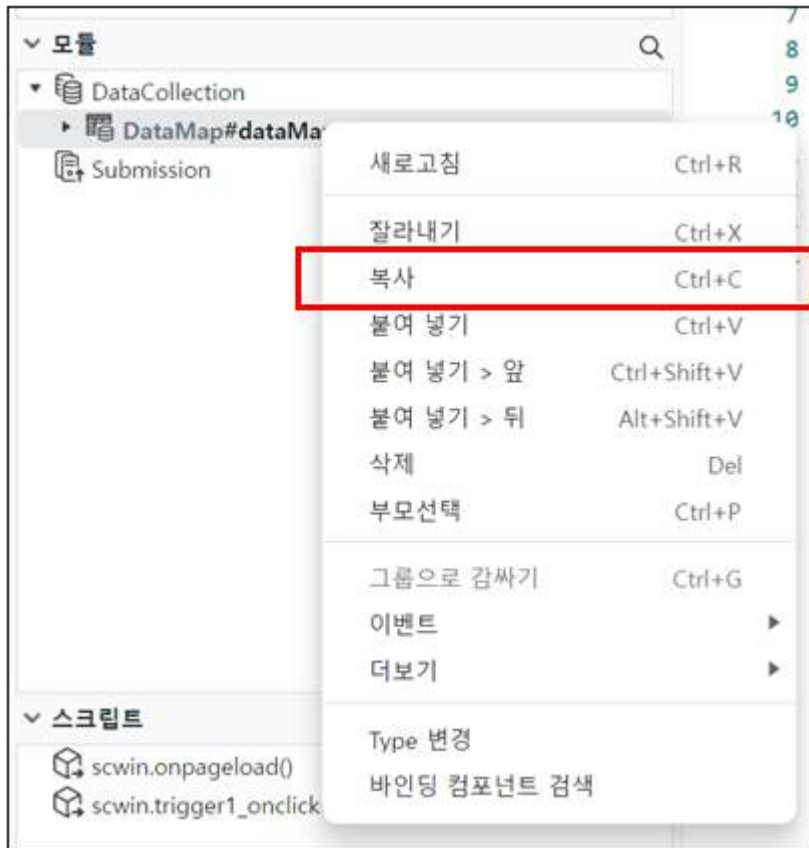
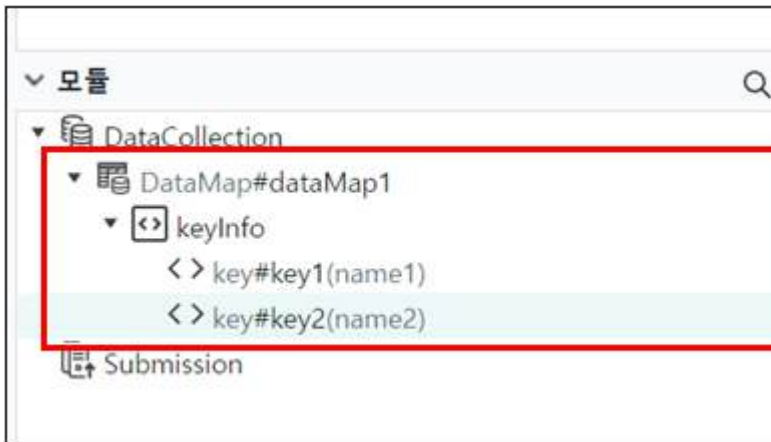


그림 12-2 붙여 넣기



그림 12-3 복사 후

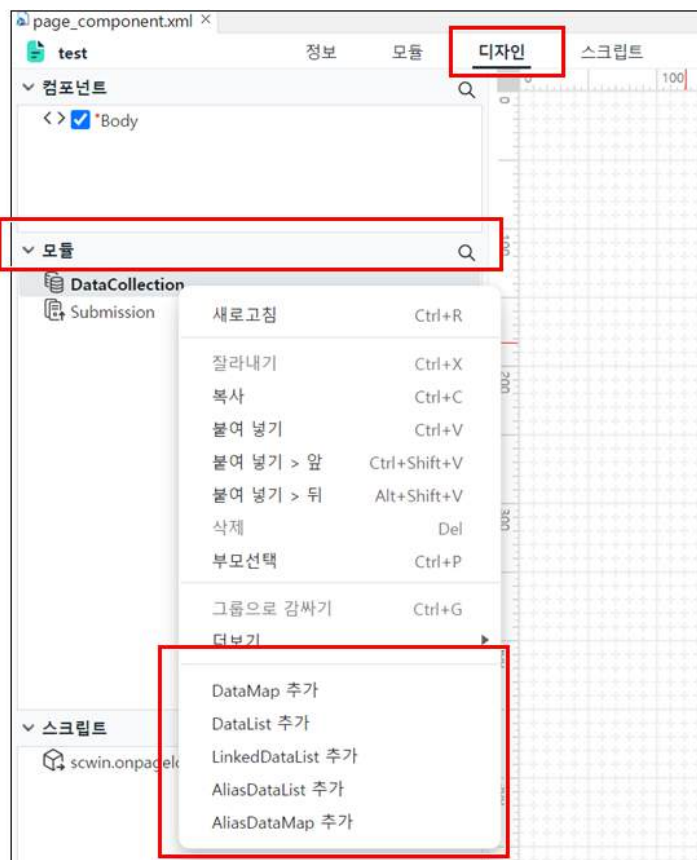


12.1.5. Studio에서 생성하고 디버깅 메뉴로 확인하기

Studio의 디자인 - 모듈 탭에서 DataCollection을 우클릭한 후, 각 메뉴를 선택하여 DataMap, DataList, LinkedDataList, AliasDataList, AliasDataMap을 생성할 수 있습니다.

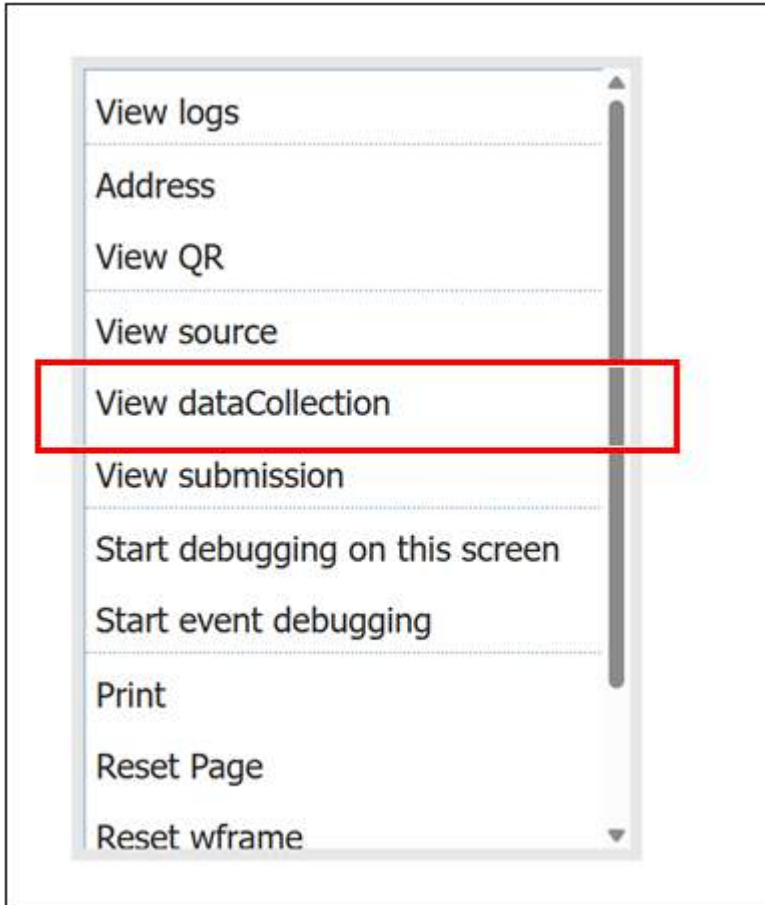
Studio에서 DataList, DataMap, LinkedDataList를 생성한 후, 웹스퀘어가 제공하는 디버깅 메뉴를 이용하여 생성한 DataList, DataMap, LinkedDataList의 데이터를 확인할 수 있습니다.

그림 12-4 DataCollection 생성



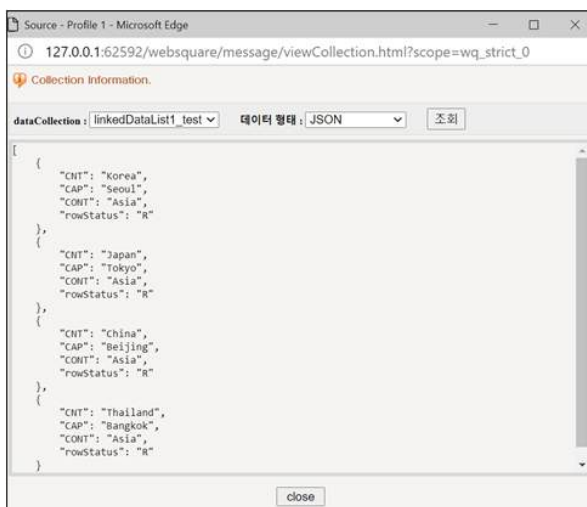
DataMap, DataList, LinkedDataList, AliasDataList, AliasDataMap를 생성한 페이지를 브라우저에 로딩한 후, Ctrl + 버튼을 누른채로 마우스를 우클릭하면 아래와 같이 디버깅 메뉴가 표시됩니다. View dataCollection 메뉴를 선택하여 생성한 DataMap, DataList, LinkedDataList, AliasDataList, AliasDataMap를 확인할 수 있습니다.

그림 12-5 디버깅 메뉴 - View dataCollection



DataCollection ID와 데이터 형태를 선택한 후, 조회 버튼을 클릭하면 아래와 같이 해당 DataCollection 객체의 데이터를 확인할 수 있습니다.

그림 12-6 DataCollection 확인



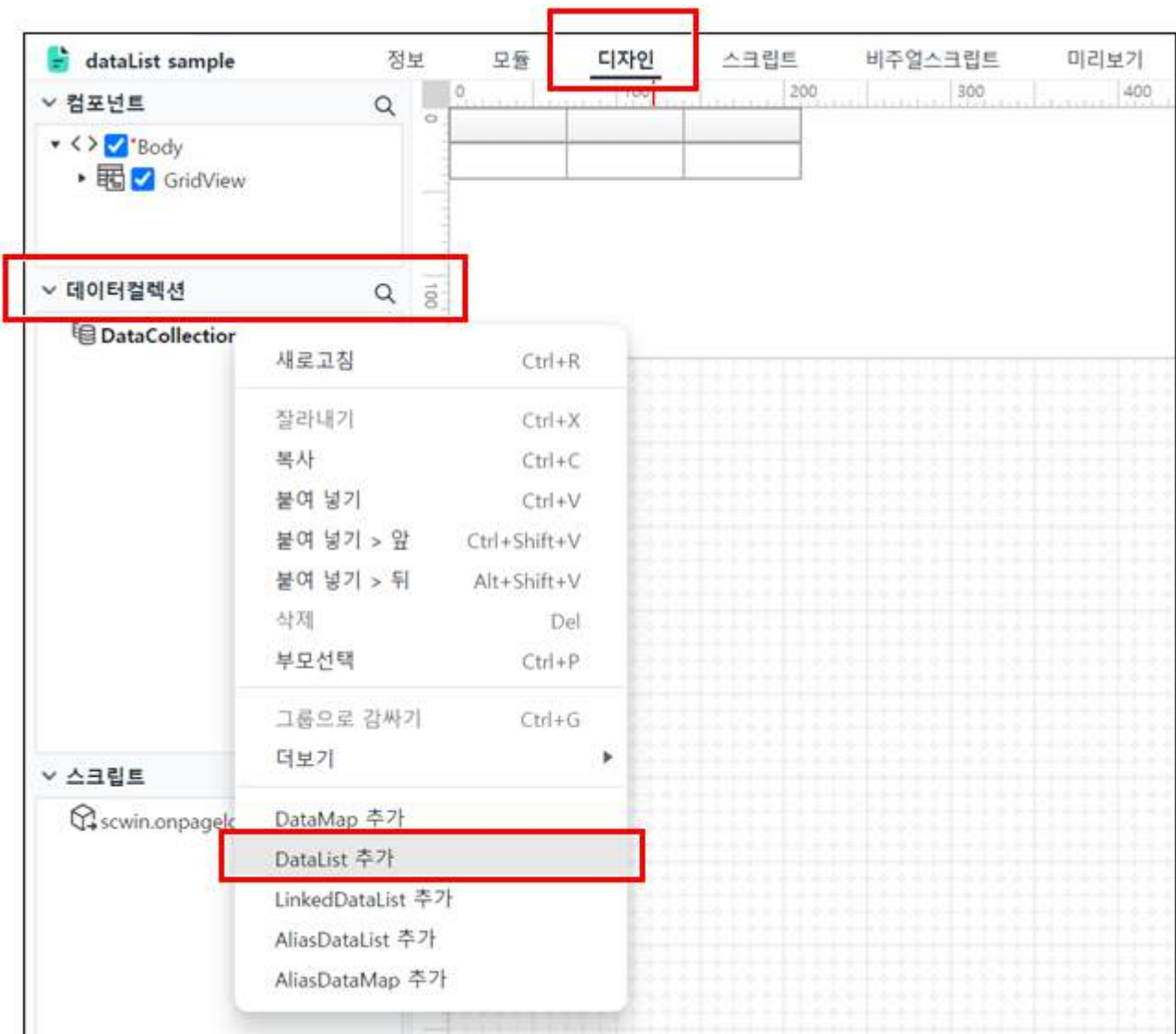
12.2 DataList

12.2.1. 생성

12.2.1.1. 디자인 탭에서 생성

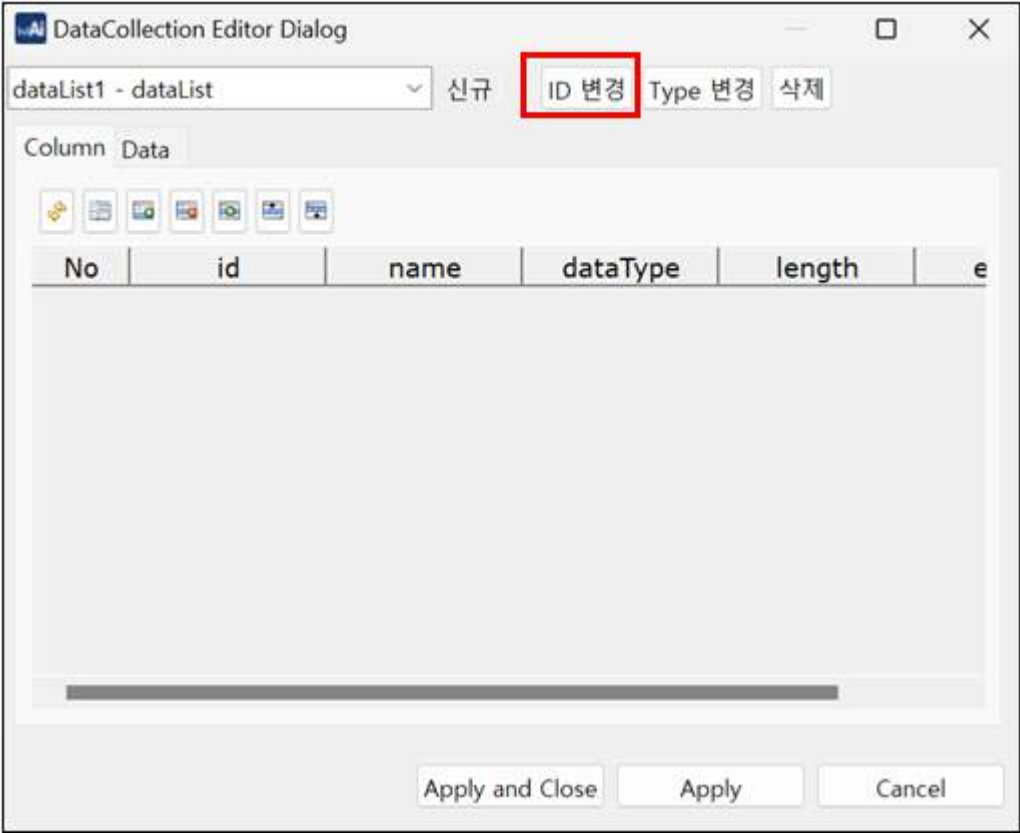
1. 디자인 에디터의 디자인 - 데이터컬렉션 탭으로 이동합니다.
2. **DataCollection** 항목을 우클릭하고 **DataList 추가** 메뉴를 선택합니다.

그림 12-7 디자인 - 데이터 컬렉션



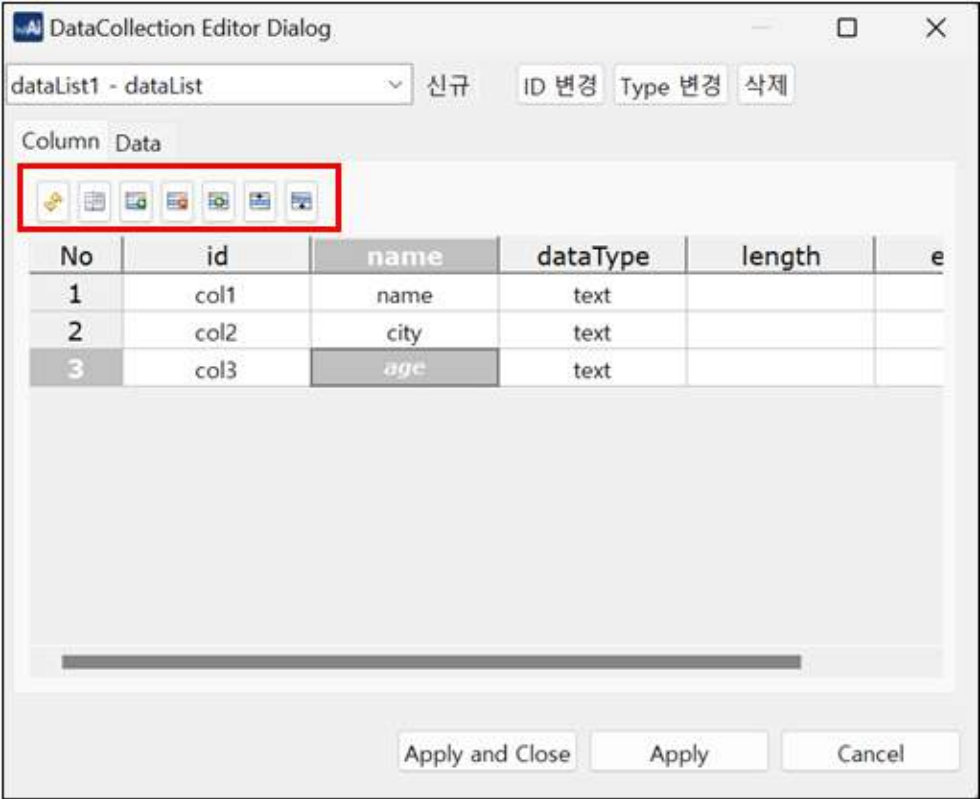
3. 아래와 같은 화면이 나타납니다. (DataList의 ID를 변경하려면 **ID 변경** 버튼을 클릭하십시오.)

그림 12-8 DataList 추가



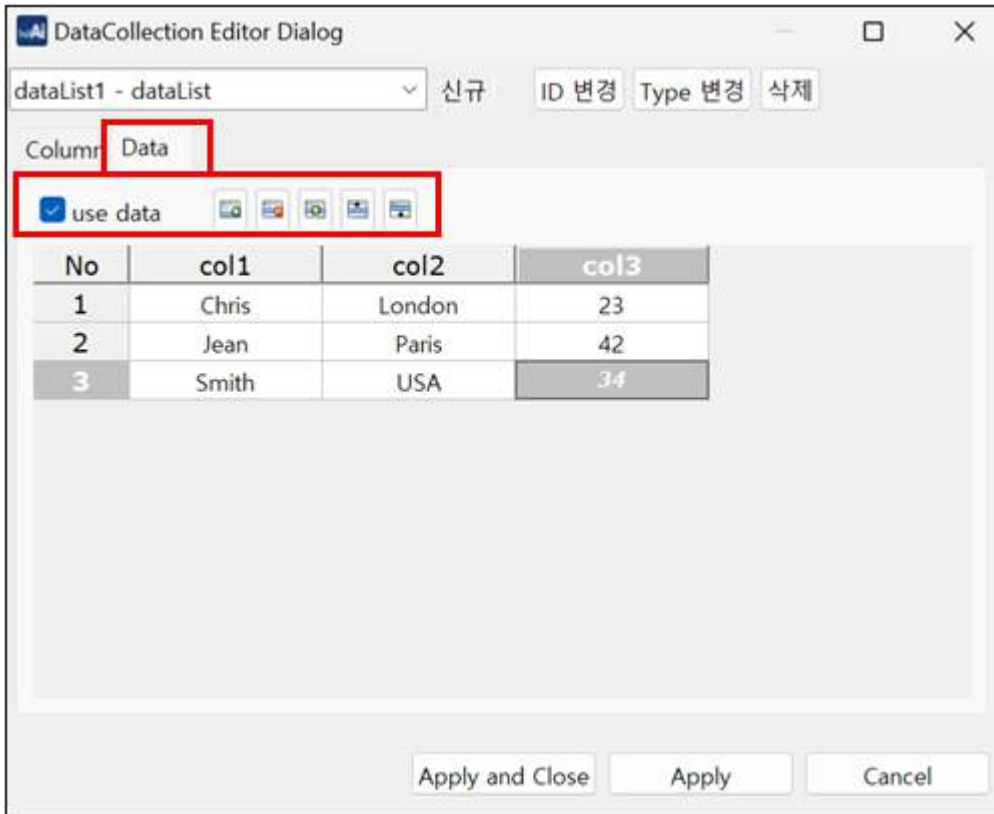
4. 상단의 행 추가 버튼을 클릭하여 컬럼을 추가합니다.

그림 12-9 컬럼 정의



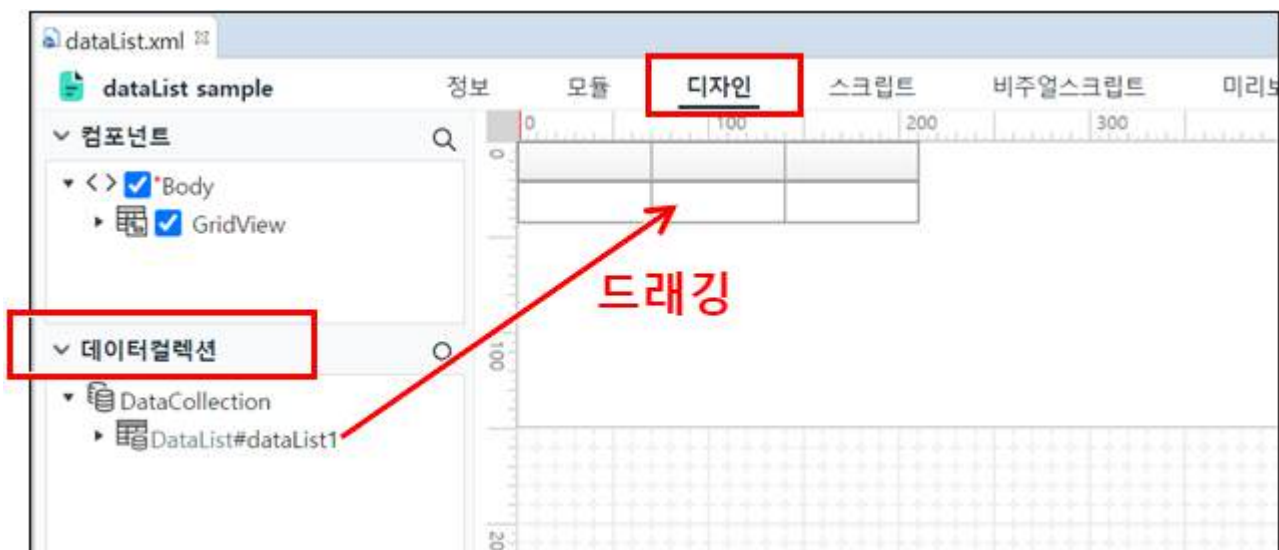
5. 데이터 통신 없이 로컬에 정의한 값을 사용할 경우, **Data** 탭을 선택한 후 **use data** 옵션을 선택합니다.
6. 상단의 행 추가 버튼을 클릭하여 위에서 정의한 각 컬럼의 데이터를 추가합니다.

그림 12-10 컬럼 데이터를 로컬에 정의



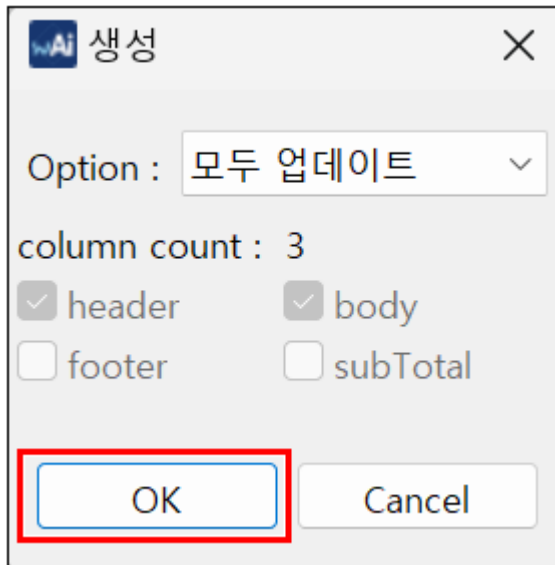
7. DataList에 정의된 형태의 데이터를 출력할 GridView를 디자인 탭에 그립니다.
8. 데이터컬렉션 탭에 추가된 DataList를 GridView로 드래그-앤-드롭합니다.

그림 12-11 DataList를 GridView로 드래킹하여 바인딩하기



9. 아래와 같은 창이 나타나면 OK를 클릭합니다.

그림 12-12 GridView와 바인딩하기



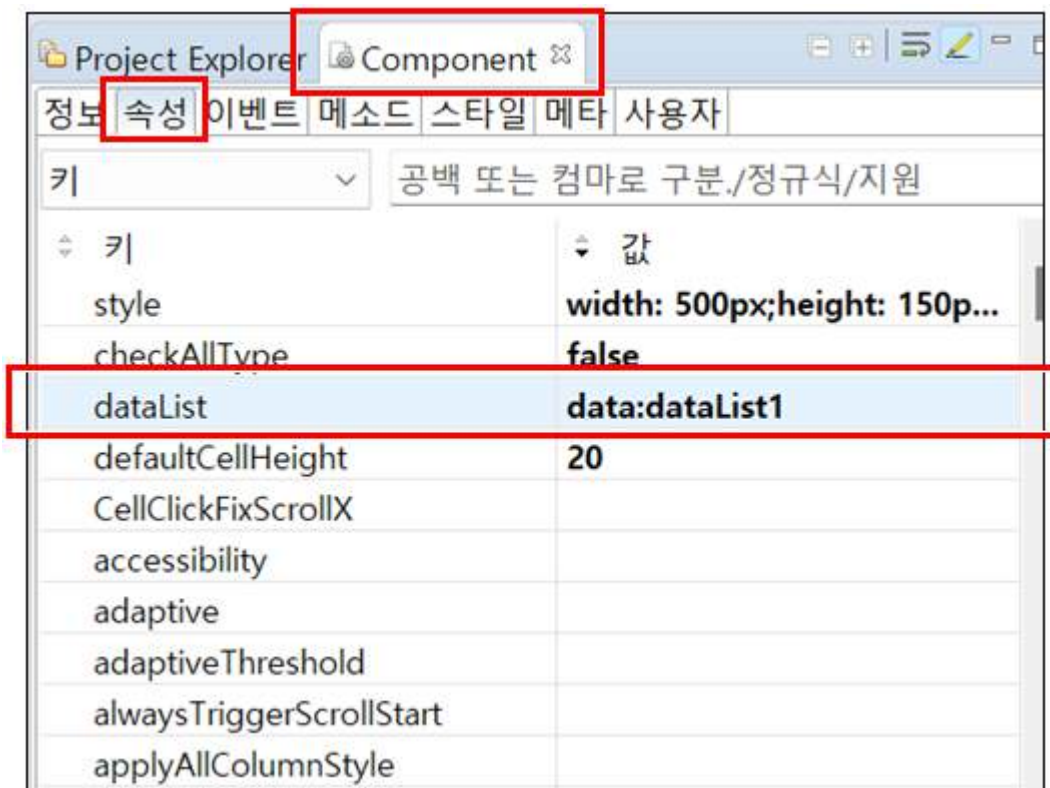
10. GridView 상의 툴팁을 통해 바인딩 상태를 확인할 수 있습니다.

그림 12-13 GridView 툴팁



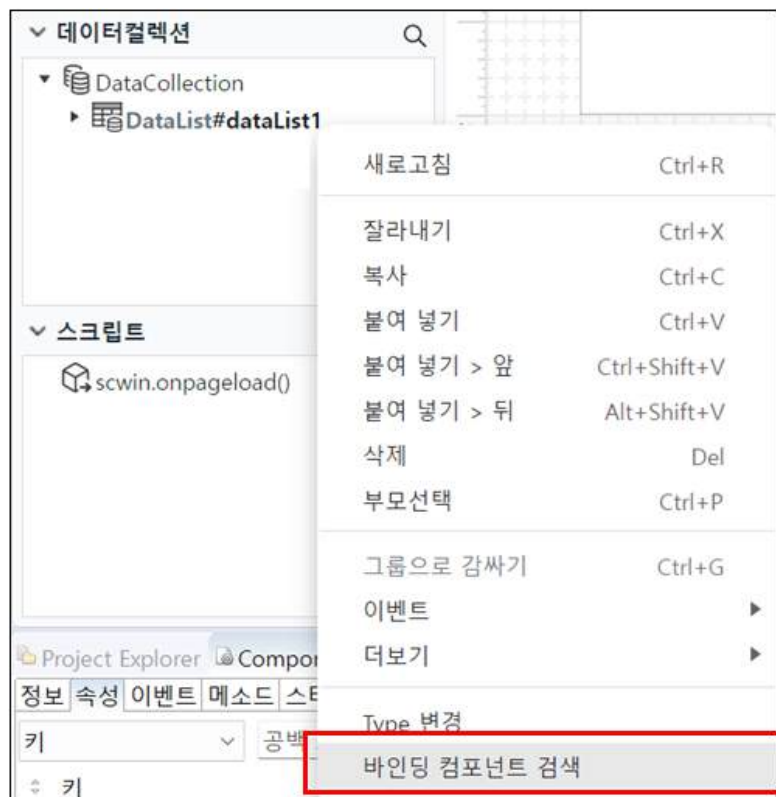
혹은, Component 뷰의 속성 탭에서 GridView의 `dataList` 속성값을 통해 바인딩 상태를 확인할 수도 있습니다.

그림 12-14 GridView의 dataList 속성 확인



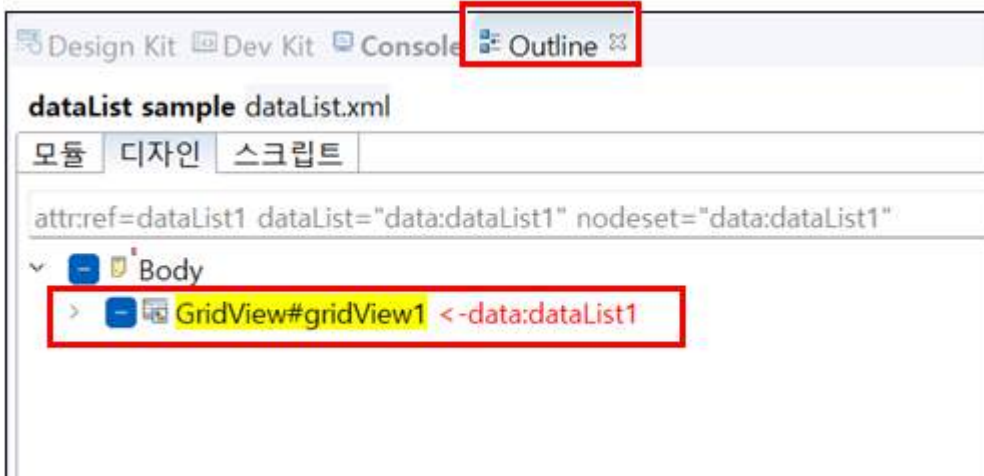
혹은, 디자인 에디터의 디자인 - 데이터컬렉션 탭에서 추가한 DataList를 우클릭한 후 바인딩 컴포넌트 검색 메뉴를 선택하여 해당 DataList와 바인딩된 컴포넌트를 확인할 수 있습니다.

그림 12-15 DataList와 바인딩된 컴포넌트 확인



검색 결과는 아래와 같이 **Outline** 뷰에 표시됩니다.

그림 12-16 바인딩된 컴포넌트 검색 결과



11. 브라우저에서 화면 파일을 실행하여 정상적으로 바인딩 되었는지 확인합니다.

그림 12-17 실제 실행 화면

name	city	age
Chris	London	23
Jean	Paris	42
Smith	USA	34

12.2.1.2. 모듈 탭에서 생성

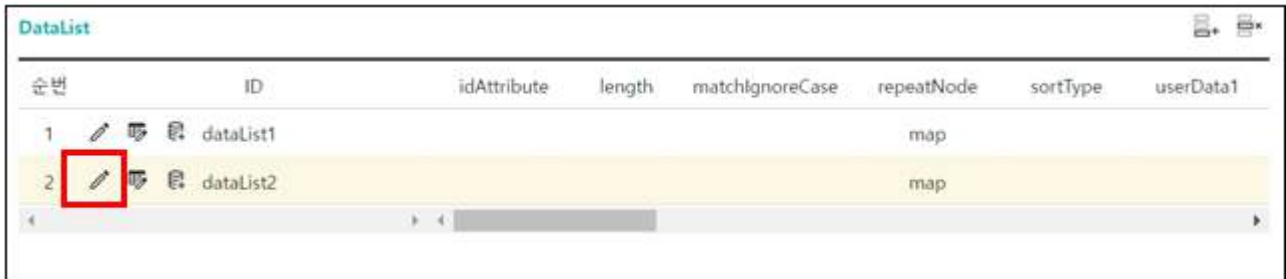
1. 디자인 에디터의 **모듈 - 데이터컬렉션** 탭으로 이동합니다.
2. DataList 우측의 추가 버튼을 클릭하여 DataList를 추가합니다.

그림 12-18 DataList 추가



3. DataList 정보를 수정하는 버튼을 클릭합니다.

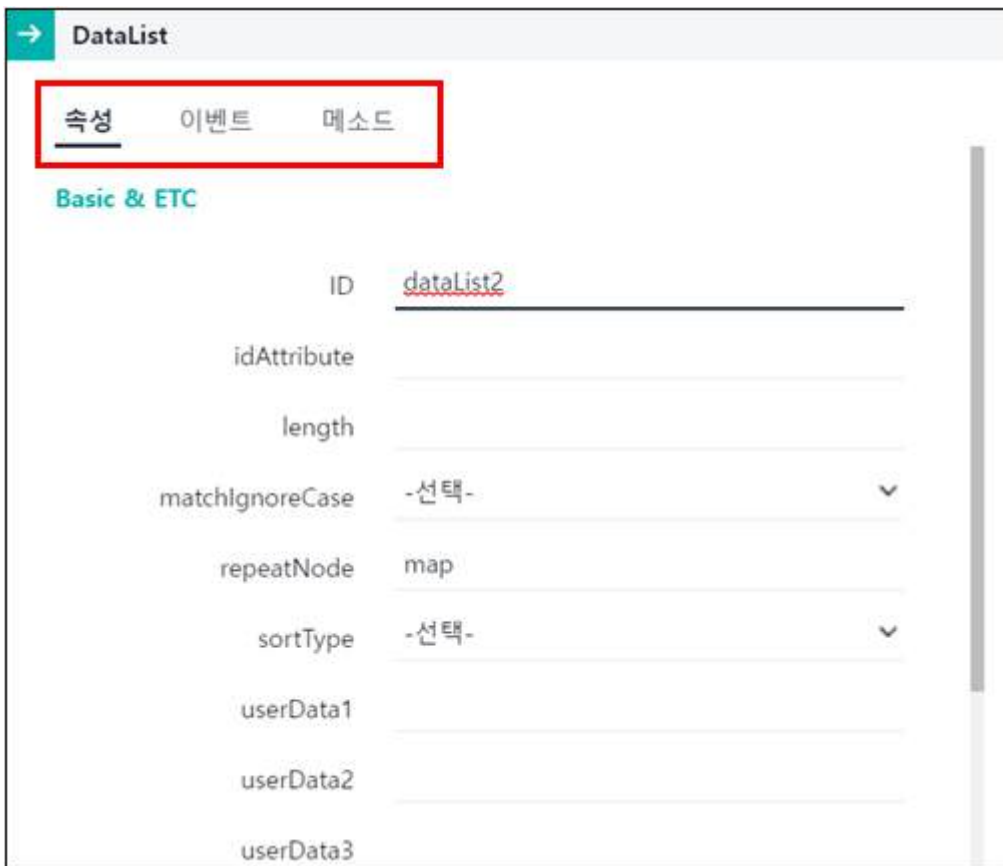
그림 12-19 DataList 정보 수정



4. DataList의 ID를 지정합니다.

5. 상단의 속성, 이벤트, 메소드 탭을 클릭하여 DataList의 속성값을 정의하거나 이벤트를 추가할 수 있고 DataList가 지원하는 함수를 확인할 수도 있습니다.

그림 12-20 DataList 정보 설정



6. DataList의 컬럼을 정의하는 버튼을 클릭합니다.

그림 12-21 DataList 컬럼 정의 버튼

순번	ID	idAttribute	length	matchIgnoreCase	repeatNode	sortType	userData1
1	dataList1				map		
2	dataList2				map		

7. 우측의 버튼을 클릭하여 DataList 컬럼을 정의합니다.

그림 12-22 DataList 컬럼 설정

DataList: dataList2

Columns

순번	ID	name	dataType	defa
1	col1	country	text	▼
2	col2	capital	text	▼
3	col3	code	text	▼

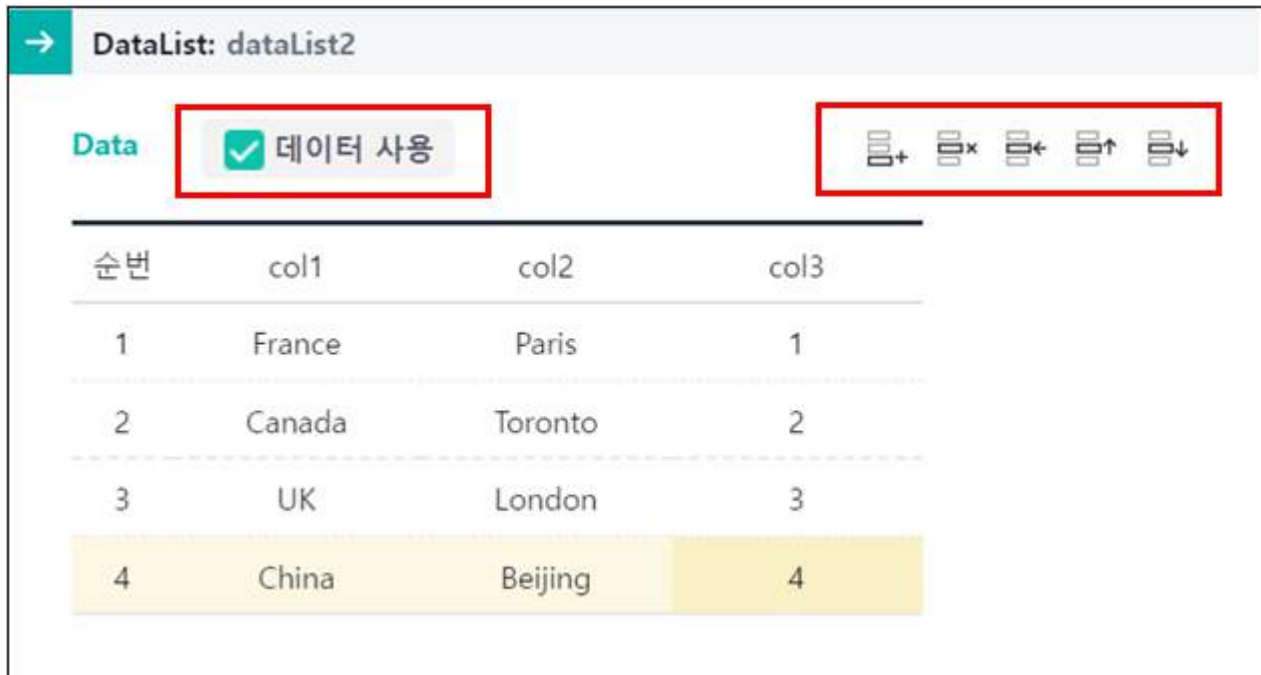
8. DataList의 컬럼에 데이터를 추가하는 버튼을 클릭합니다.

그림 12-23 DataList 데이터 추가 버튼

순번	ID	idAttribute	length	matchIgnoreCase	repeatNode	sortType	userData1
1	dataList1				map		
2	dataList2				map		

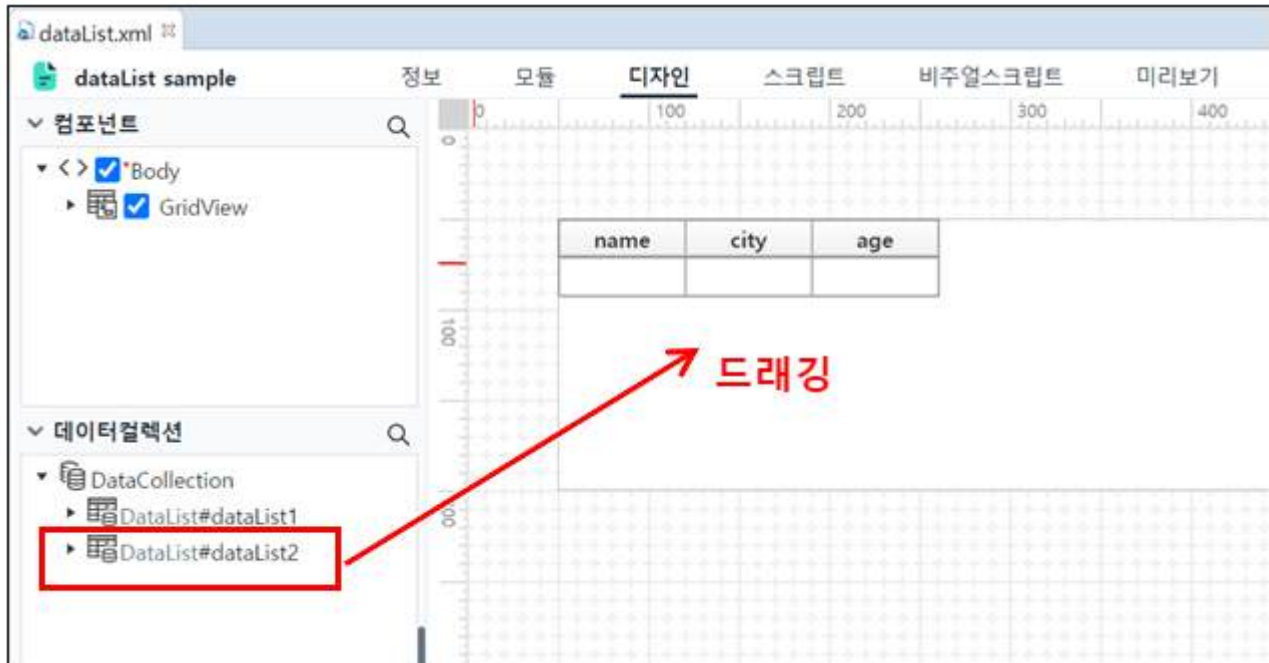
9. 로컬에서 정의한 데이터를 사용할 경우 **데이터 사용** 옵션을 선택하고, 우측의 버튼을 클릭하여 DataList 컬럼에 대한 데이터를 추가합니다.

그림 12-24 DataList 데이터를 로컬에 정의

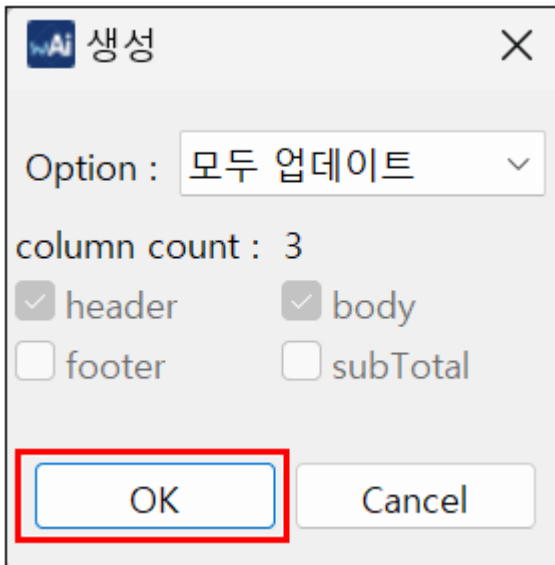


10. DataList에 정의된 형태의 데이터를 출력할 GridView를 디자인 탭에 그립니다.
11. 데이터컬렉션 탭에 추가된 DataList를 GridView로 드래그-앤-드롭합니다.

그림 12-25 DataList를 GridView로 드래킹하여 바인딩하기



12. 아래와 같은 창이 나타나면 OK를 클릭합니다.



13. 브라우저에서 화면 파일을 실행하여 정상적으로 바인딩 되었는지 확인합니다.

그림 12-26 실제 실행 화면

country	capital	code
France	Paris	1
Canada	Toronto	2
UK	London	3
China	Beijing	4

12.2.2. 데이터 타입 자동 변경

DataList에 설정하려는 데이터의 타입과 DataList 컬럼의 `dataType` 속성 값이 일치하지 않을 경우, `keepDataType` 및 `preserveType` 속성을 사용하여 DataList 컬럼의 `dataType`에 따라 데이터를 설정할 수 있습니다.

12.2.2.1. `string` 타입 데이터를 `number` 타입으로 설정 (`keepDataType`)

`string` 타입의 데이터를 DataList에 설정할 경우 DataList 컬럼의 `dataType`이 `"number"`인 경우, `string` 타입의 데이터를 `dataType="number"` 타입으로 변환하여 DataList 컬럼에 설정할 수 있습니다. 이 기능을 사용하려면 DataList

의 `keepDataType` 속성 값을 `"true"`로 설정하십시오.



예제 파일

인터넷에서 다운로드 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/_DataCollection/keepDataType_DataList.xml)

그림 12-27 YouTube 동영상 (<https://youtu.be/FRxRJ6jawyg>)



12.2.2.2. `number` 타입 데이터를 `string` 타입으로 설정 (`preserveType`)

`number` 타입의 데이터를 DataList에 설정할 경우 DataList 컬럼의 `dataType`이 `"text"`인 경우, `number` 타입의 데이터를 `dataType="text"` (`string`) 타입으로 변환하여 DataList 컬럼에 설정할 수 있습니다. 이 기능을 사용하려면 DataList의 `preserveType` 속성 값을 `"true"`로 설정하십시오.



예제 파일

인터넷에서 다운로드 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/_DataCollection/preserveType_DataList.xml)

그림 12-28 YouTube 동영상 (<https://youtu.be/fvpVt24nNhw>)



12.2.3. 필터링

12.2.3.1. `getFilterList()`

DataList가 `setColumnFilter()` 함수에 의해 필터링된 경우, `getFilterList()` 함수를 사용하여 각 칼럼의 필터링 상태를 반환할 수 있습니다.

아래는 현재 필터링 상태를 저장한 후, 필터링을 해제하고 다시 이전 필터링 상태로 돌아가는 예제입니다.

Code 12-1 사용 예

```
1 var data = [];
2 for (var i = 0; i < 20 * 3; i++) {
3     data.push(i);
4 }
5 dataList1.setData(data);
6 scwin.tempFilter = [];
```

```

7
8 scwin.trigger1_onclick = function(e) {
9     dataList1.clearFilter();
10 };
11
12 scwin.trigger2_onclick = function(e) {
13     scwin.tempFilter = dataList1.getFilterList();
14 };
15
16 scwin.trigger3_onclick = function(e) {
17     for (var i = 0; i < scwin.tempFilter.length; i++) {
18         dataList1.setColumnFilter(scwin.tempFilter[i]);
19     }
20 };

```

12.2.3.2. getFilteredRowIndex()

`getFilteredRowIndex( realRowIndex )` 함수를 통해 필터링 후 행의 변경된 인덱스를 확인할 수 있습니다.

Code 12-2 사용 예

```

1 // 실제 인덱스가 "5인 행이 필터링된 이후의 인덱스를 반환.
2
3 dataList1.getFilteredRowIndex(5);

```

12.2.4. 데이터 반환

12.2.4.1. DataList의 전체 데이터 반환

`getAllJSON()` 함수를 사용하여 DataList의 전체 데이터를 JSON으로 변환하여 1차원 배열로 반환할 수 있습니다.

Code 12-3 사용 예

```

1 // DataList의 전체 데이터를 rowStatus를 제외하고 JSON으로 반환
2
3 dataList1.getAllJSON({ "rowStatusData": false } );

```

12.2.4.2. DataList의 행 수 반환

`getRowCount()` 함수를 사용하여 현재 DataList와 바인딩된 GridView에 표시된 행 수를 반환할 수 있습니다.

Code 12-4 사용 예

```
1 // 로딩된 데이터 행수를 얼럿으로 표시
2
3 var count = dataList1.getRowCount();
4 alert("Total " + count + " data has been loaded.");
```

12.2.4.3. 셀 데이터 반환

`getCellData()` 함수를 사용하여 특정 셀의 데이터를 반환할 수 있습니다.

Code 12-5 Example

```
1 // 인덱스가 1인 행의 "ID" 컬럼의 값을 반환
2
3 dataList1.getCellData(1, "ID")
```

12.2.5. 사용자 데이터 설정 및 반환

`setUserData()` 및 `getUserData()` 함수를 이용하여 DataList의 `userData1`, `userData2`, 혹은 `userData3` 속성 값을 설정하거나 반환할 수 있습니다. 모든 DataCollection 컴포넌트(`aliasDataMap`, `aliasDataList`, `aliasLinkedDataList`, `dataMap`, `linkedDataList` 포함)에 대해 동일하게 사용할 수 있습니다.

Code 12-6 사용 예

```
1 dataList1.setUserData(
2     "userData1 ", " 12345 "
3 );
4
5 dataList1.getUserData(
6     "userData1 "
7 );
```

12.2.6. 여러 DataList를 Excel 파일로 다운로드

DataList 여러 개 → Excel 다운로드 참조.

12.2.7. 공백 데이터

12.2.7.1. 공백 데이터 처리

DataList가 공백 데이터를 포함한 경우, 공백 데이터를 공백("")으로 치환하여 반환하거나, 공백 데이터를 제외하고 반환하거나, 혹은, null 값으로 반환할 수 있습니다. DataList의 `nullYNTYPE` 속성 값을 아래와 같이 설정하십시오.

- "default" (기본 값) : 공백("")으로 치환.
- "exclude" : 공백 데이터를 제외하고 반환.
- "null" : null로 반환.



DataList 컬럼(dataList.column)의 nullYN 속성 설정

DataList의 `nullYNTYPE` 속성 설정을 적용하려면, 각 DataList의 컬럼(dataList.column)의 `nullYN` 속성 값을 "true"로 설정하십시오. `nullYN="true"` 속성 설정이 적용된 DataList 컬럼(dataList.column)만 DataList의 `nullYNTYPE` 속성 설정에 따라 공백 데이터가 처리됩니다.

아래 예제는 DataList에 `nullYNTYPE="true"` 설정을 적용한 후, DataList의 col2, col3, col4 컬럼에 대해서만 `nullYN="true"` 설정을 적용한 경우입니다. col1의 공백 데이터는 공백("")으로 반환되지만, col2, col3, col4 컬럼의 공백 데이터는 반환되지 않는 것을 확인할 수 있습니다.

그림 12-29 `nullYNTYPE="exclude"` (DataList) `nullYN="true"` (col2, col3 & col4 of DataList)

	col1	col2	col3	col4
1				a
2			b	

127.0.0.1:60074 says

```
[{"col1": "", "col4": "a", "rowStatus": "R"}, {"col1": "", "col3": "b", "rowStatus": "R"}]
```

OK



예제 파일

[인터넷에서 다운로드](#) 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/DataList/nullYNTType_DataList.xml)

12.2.7.2. null 값과 공백("") 값 구분

DataList의 useNull 속성을 사용하여 null 값과 공백("") 값을 구분하여 설정할 수 있습니다.

useNull

"true" : null 값과 공백("") 값을 구분하여 설정.

"false" (기본 값) : 구분 없이 모두 공백("")으로 설정.



예제 파일

[인터넷에서 다운로드](#) 혹은

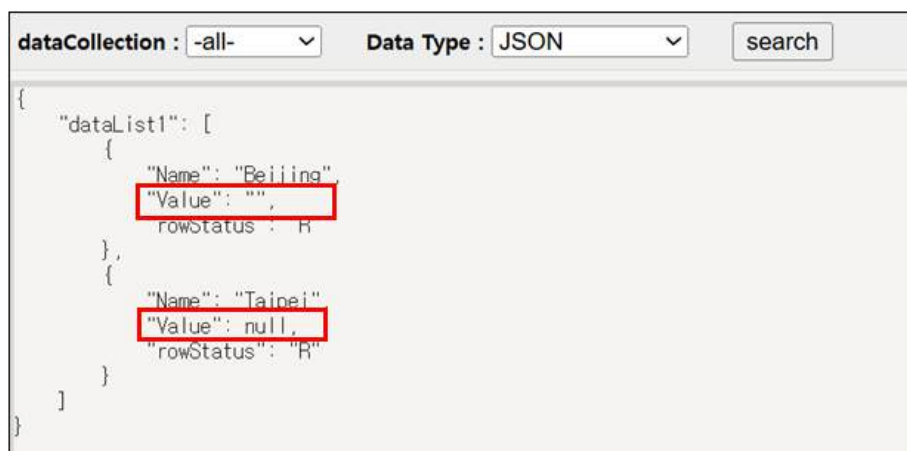
WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/DataList/useNull_DataList.xml)

Code 12-7 DataList에 공백("")과 null 값을 설정 (예시)

```
1 // DataList에 useNull="true" 속성 설정을 적용할 경우,
2 // null 값과 공백 값을 구분해서 설정 가능.
3
4 // 공백("")과 null 값을 설정.
5 dataList1.setData(["Beijing", "", "Taipei", null]);
```

그림 12-30 실제 설정된 값



12.2.8. 컬럼 데이터 포맷 (dataList.column - importFormatter)

dataList.column의 importFormatter 속성으로 사용자 정의 함수를 지정한 후, 해당 함수를 통해 컬럼의 데이터를 변경할 수 있습니다.

Code 12-8 사용 예

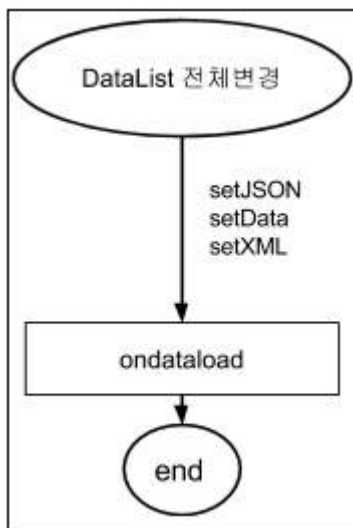
```
1 // importFormatter="scwin.statusFormatter"인 경우, 해당 함수를 구현
2
3 scwin.statusFormatter = function(rowJson, rowIndex, colIndex) {
4     data = rowJson.Status;
5     return data.substring(0, 1);
6 }
```

12.2.9. 이벤트

12.2.9.1. ondataload

DataList의 ondataload 이벤트는 DataList의 데이터가 새로 설정되거나 추가될 때 발생합니다.

그림 12-31 ondataload



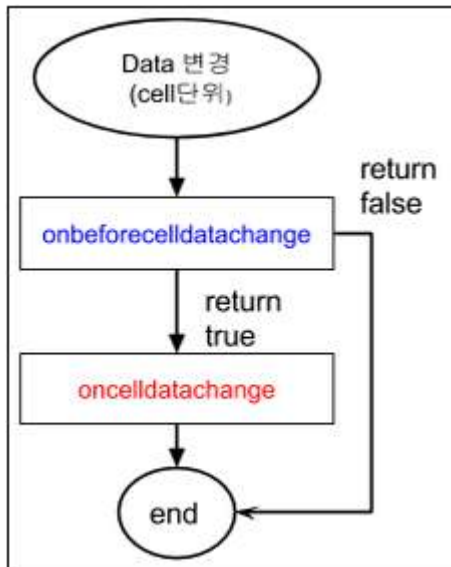
Code 12-9 사용 예

```
1 scwin.dataList1_ondataload = function() {
2     alert(" Loading done.");
3 };
```


12.2.9.2. onbeforecelldatachange

DataList의 `onbeforecelldatachange` 이벤트는 DataList와 바인딩된 GridView의 셀 데이터가 변경되기 직전에 발생합니다. 이벤트 함수가 `false`를 반환하면 GridView에서 변경한 데이터가 DataList에 적용되지 않습니다.

그림 12-32 onbeforerowpositionchange & oncelldatachange



Code 12-10 사용 예

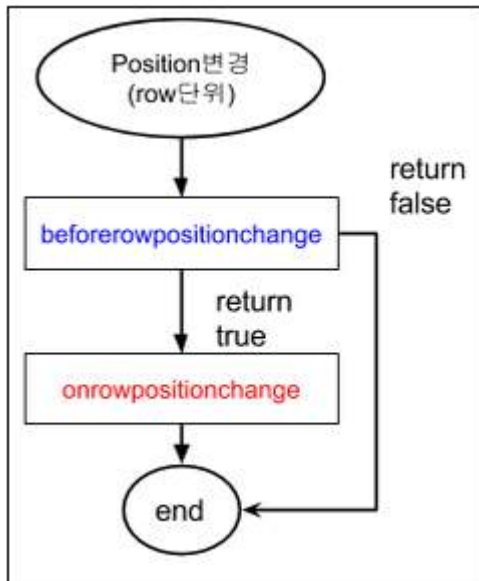
```

1  scwin.dataList1_onbeforecelldatachange = function(info) {
2      if (info.colID == "Hobby") {
3          if(info.newValue == "") {
4              return false;
5          }
6      }
7  };

```

12.2.9.3. onbeforerowpositionchange & onrowpositionchange

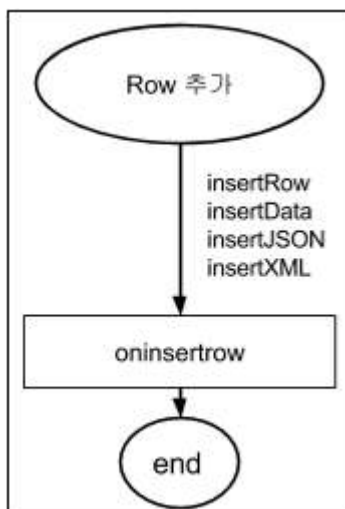
DataList의 `onbeforerowposition` 이벤트는 DataList와 바인딩된 GridView의 행 위치가 변경되기 전에 발생하며, `onbeforerowposition` 이벤트 함수가 `true`를 반환하면 `onrowposition` 이벤트가 발생합니다.

그림 12-33 `onbeforerowpositionchange` & `onrowpositionchange`

12.2.9.4. `oninsertrow`

DataList의 `oninsertrow` 이벤트는 아래의 함수를 사용하여 DataList와 바인딩된 GridView의 행을 추가할 때 발생합니다.

- `insertRow()`
- `insertData()`
- `insertJSON()`
- `insertXML()`

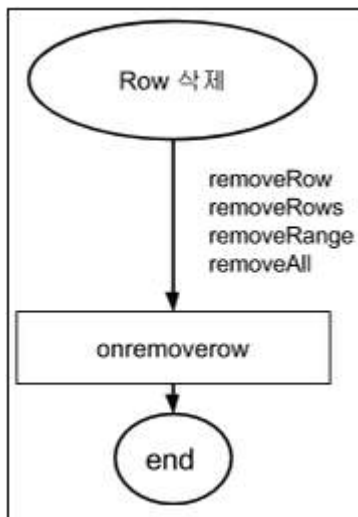
그림 12-34 `oninsertrow`

12.2.9.5. onremoverow

DataList의 `onremoverow` 이벤트는 아래의 함수를 사용하여 DataList와 바인딩된 GridView의 행을 제거할 때 발생합니다.

- `removeRow()`
- `removeRows()`
- `removeRange()`
- `removeAll()`

그림 12-35 onremoverow



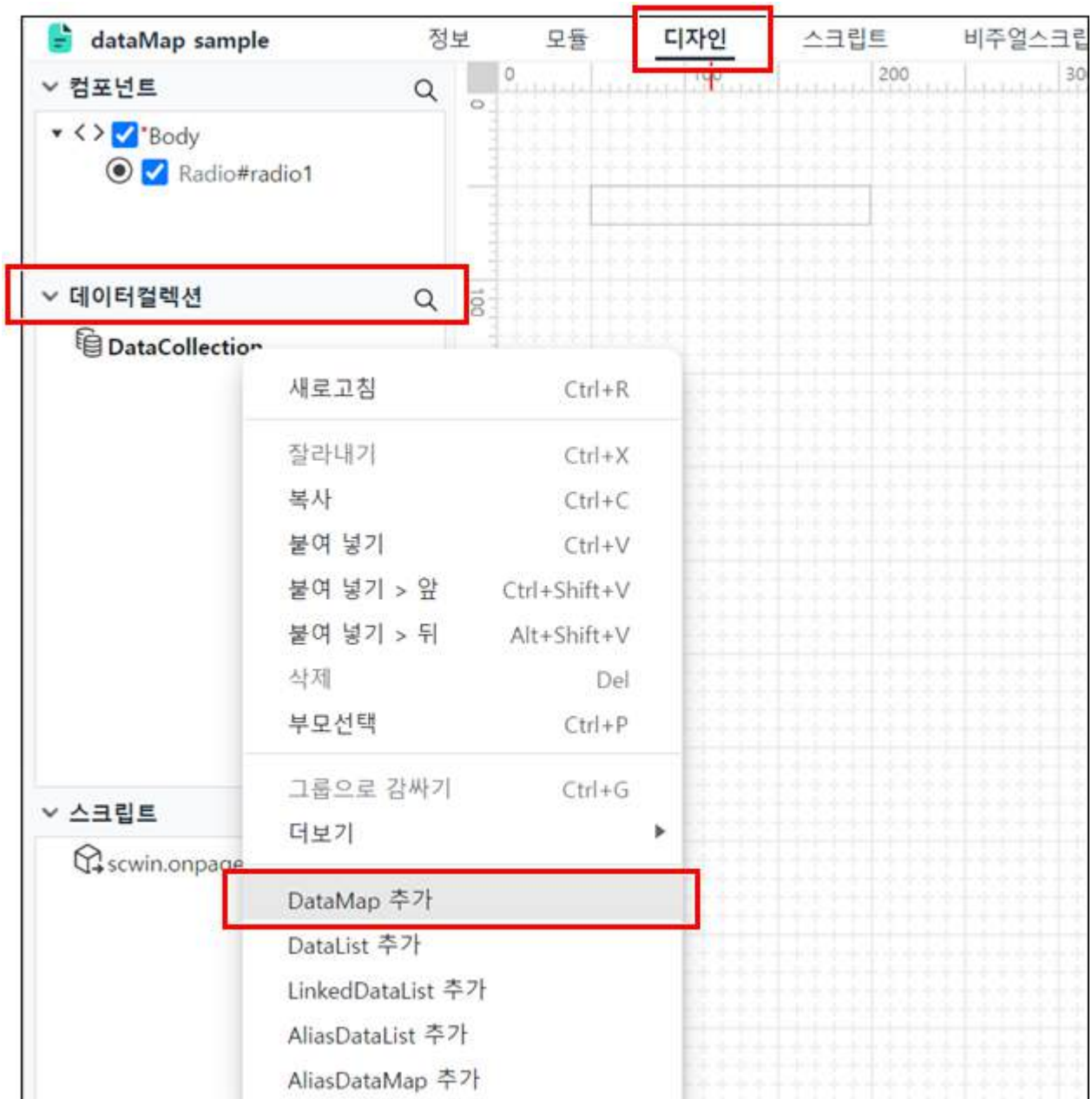
12.3 DataMap

12.3.1. 생성

12.3.1.1. 디자인 탭에서 생성

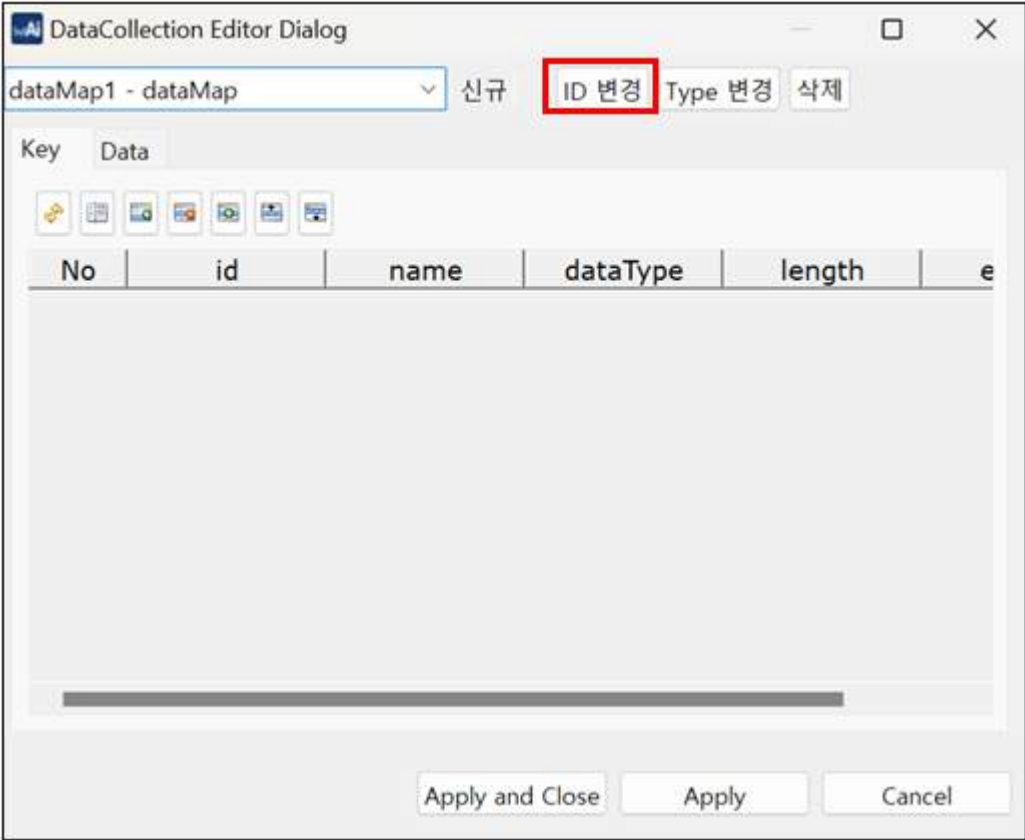
1. 디자인 에디터의 디자인 - 데이터컬렉션 탭으로 이동합니다.
2. **DataCollection** 항목을 우클릭하고 **DataMap 추가** 메뉴를 선택합니다.

그림 12-36 디자인 - 데이터 컬렉션



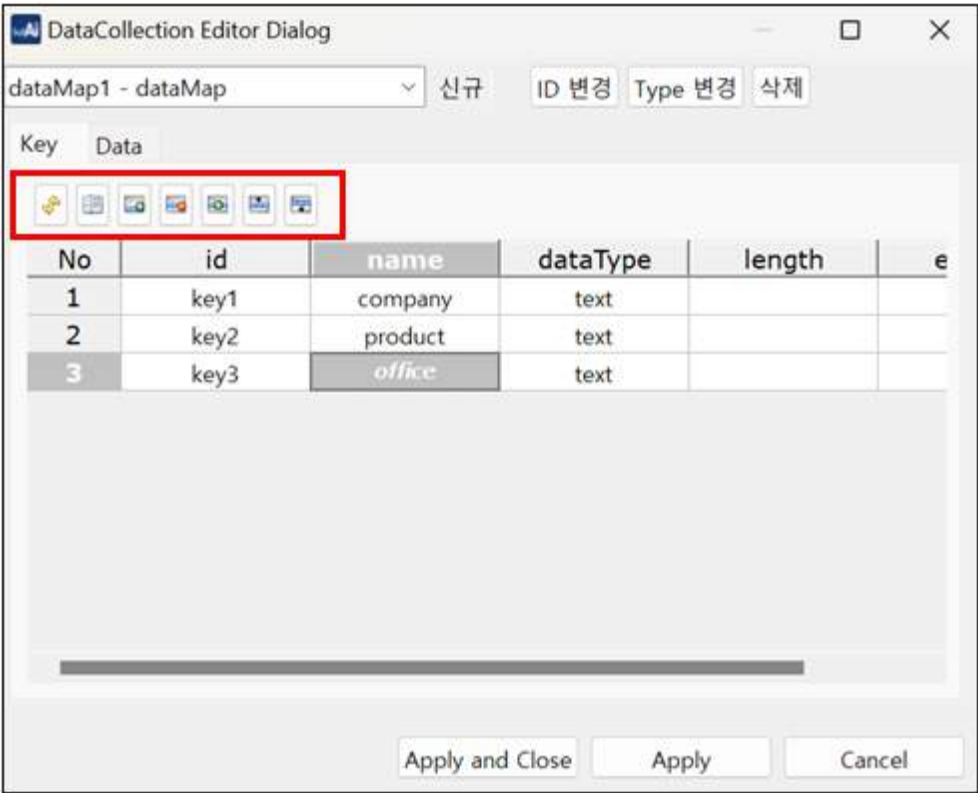
3. 아래와 같은 화면이 나타납니다. (DataMap의 ID를 변경하려면 ID 변경 버튼을 클릭하십시오.)

그림 12-37 DataMap 추가



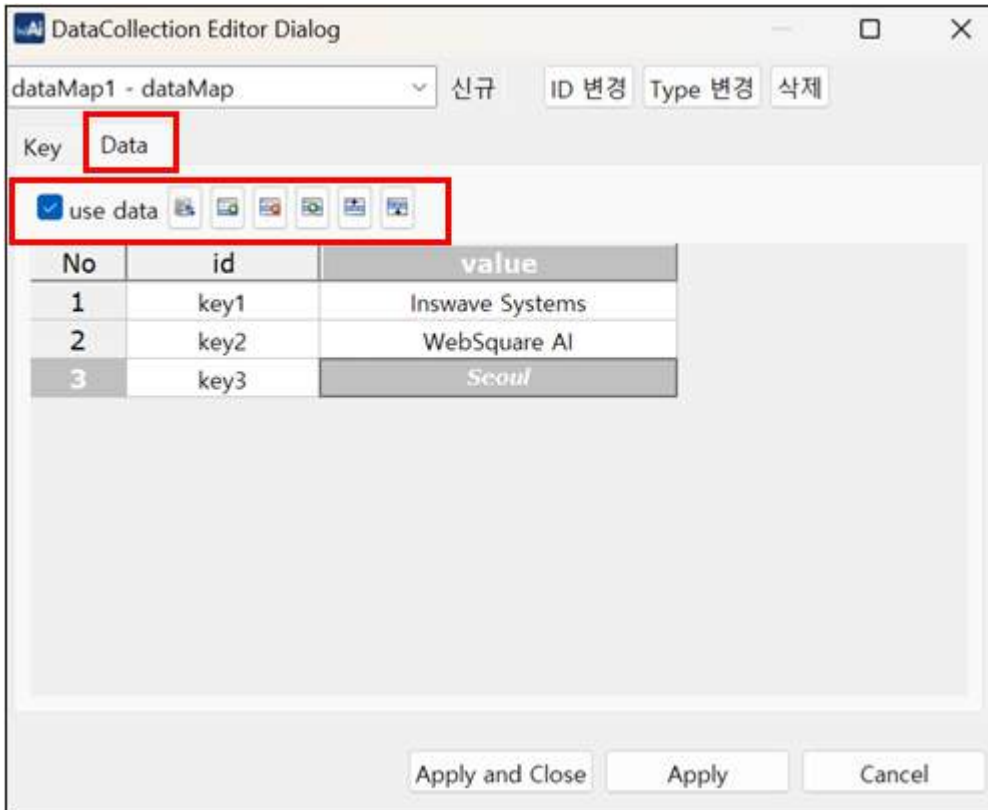
4. 행 추가 버튼을 클릭하여 Key 정보를 입력합니다.

그림 12-38 Key 정의



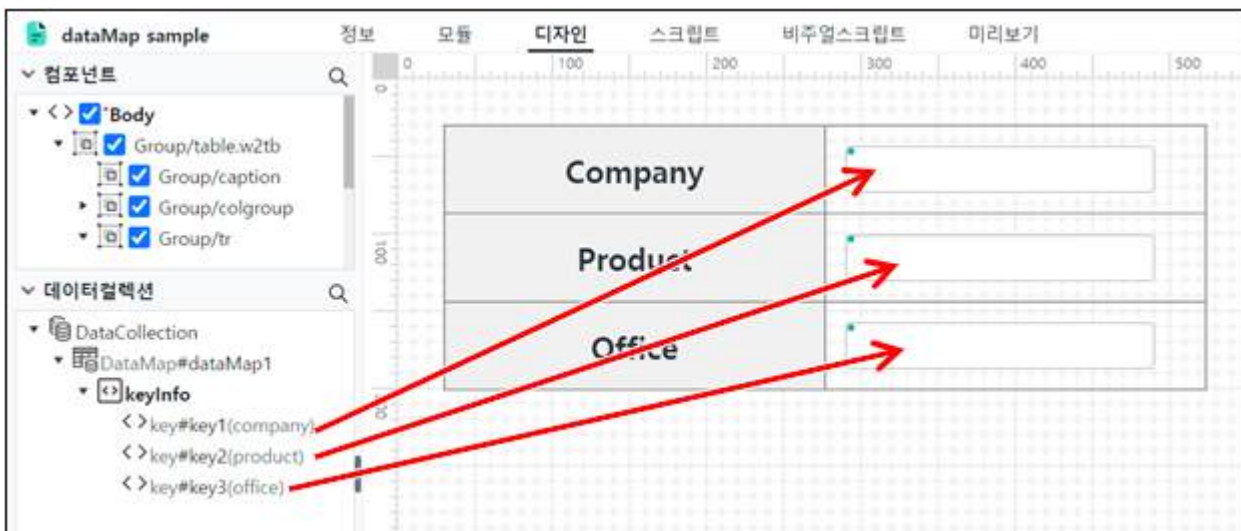
5. 데이터 통신 없이 로컬에 정의한 값을 사용할 경우, **Data** 탭을 선택한 후 **use data** 옵션을 선택합니다.
6. 상단의 행 추가 버튼을 클릭하여 위에서 정의한 각 키의 데이터를 추가합니다.

그림 12-39 키 데이터를 로컬에 정의



7. DataMap에 정의된 형태의 데이터를 출력할 화면을 Design 탭에서 구성합니다.
8. 데이터컬렉션 탭에 추가된 DataMap의 Key를 바인딩할 컴포넌트로 드래그-앤-드롭합니다.
9. 바인딩된 컴포넌트에 초록색 표시가 나타나는지 확인합니다.

그림 12-40 바인딩



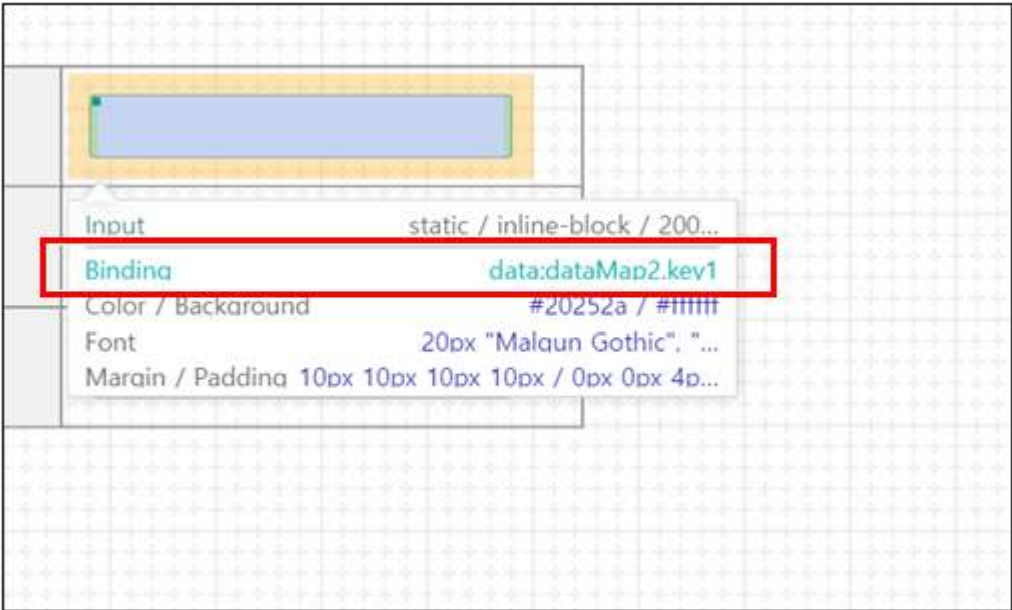
10. 브라우저에서 화면 파일을 실행하여 정상적으로 바인딩 되었는지 확인합니다.

그림 12-41 실행 화면

Company	Inswave Systems
Product	WebSquare AI
Office	Seoul

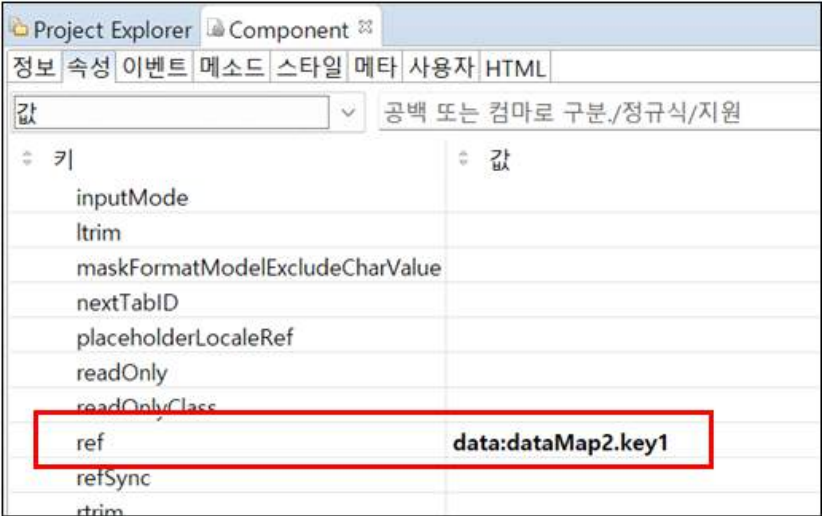
11. DataMap과 바인딩된 컴포넌트 상의 툴팁을 통해 바인딩 상태를 확인할 수 있습니다.

그림 12-42 컴포넌트 툴팁



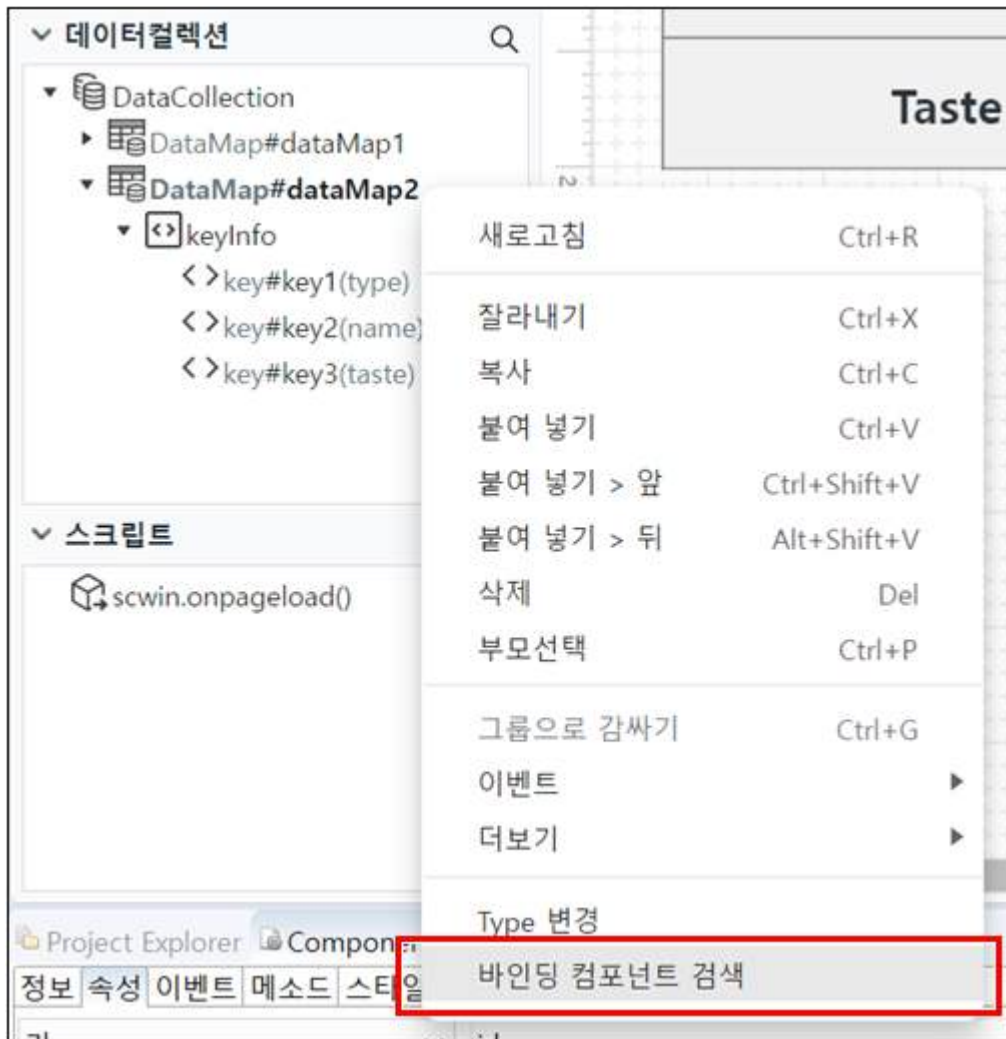
혹은, Component 뷰의 속성 탭에서 해당 컴포넌트의 ref 속성값을 통해 바인딩 상태를 확인할 수도 있습니다.

그림 12-43 ref 속성 확인



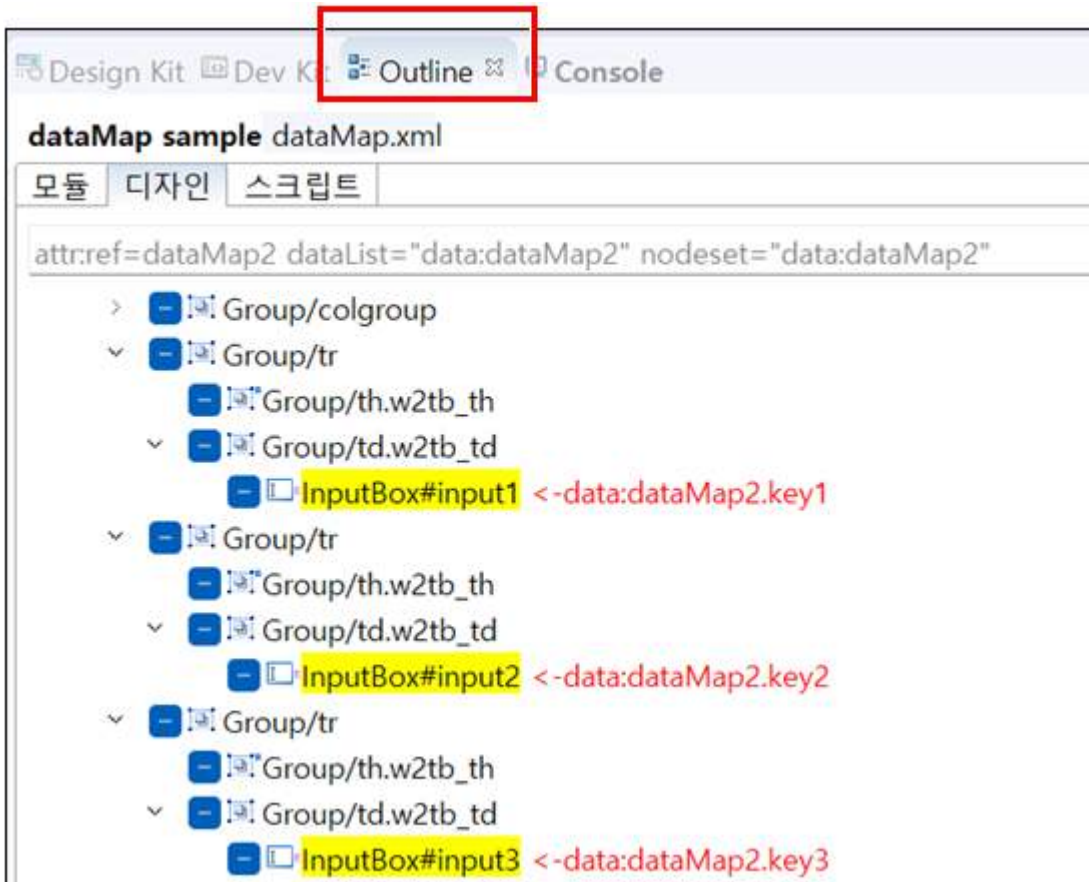
혹은, 디자인 에디터의 디자인 - 데이터컬렉션 탭에서 추가한 DataMap을 우클릭한 후 바인딩 컴포넌트 검색 메뉴를 선택하여 해당 DataMap과 바인딩된 컴포넌트를 확인할 수 있습니다.

그림 12-44 DataMap과 바인딩된 컴포넌트 확인



검색 결과는 아래와 같이 Outline 뷰에 표시됩니다.

그림 12-45 바인딩된 컴포넌트 검색 결과



12.3.1.2. 모듈 탭에서 생성

1. 디자인 에디터의 모듈 - 데이터컬렉션 탭으로 이동합니다.
2. DataMap 우측의 추가 버튼을 클릭하여 DataMap을 추가합니다.

그림 12-46 DataMap 추가



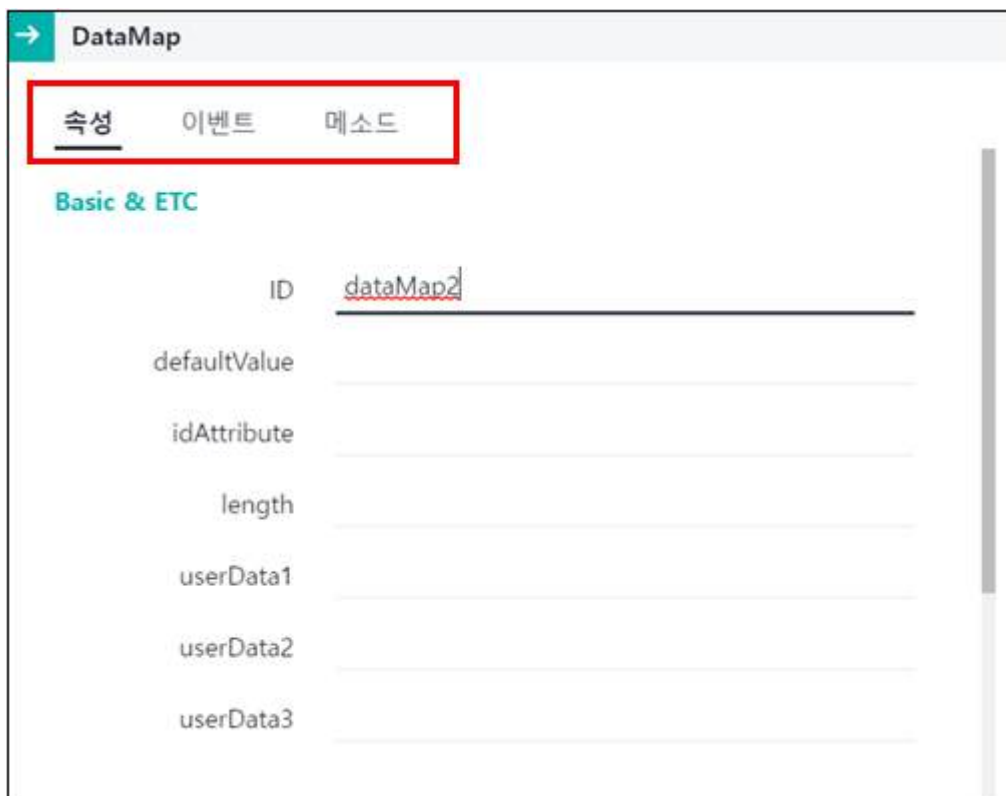
3. DataList 정보를 수정하는 버튼을 클릭합니다.

그림 12-47 DataMap 정보 수정 버튼



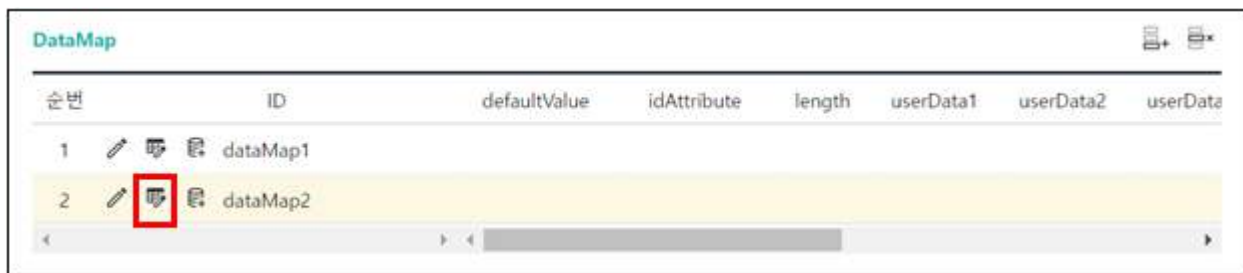
4. DataMap의 ID와 속성값을 설정합니다.
5. 상단의 속성, 이벤트, 메소드 탭을 클릭하여 DataMap의 속성값을 정의하거나 이벤트를 추가할 수 있고 DataMap이 지원하는 함수를 확인할 수도 있습니다.

그림 12-48 DataMap 정보 설정



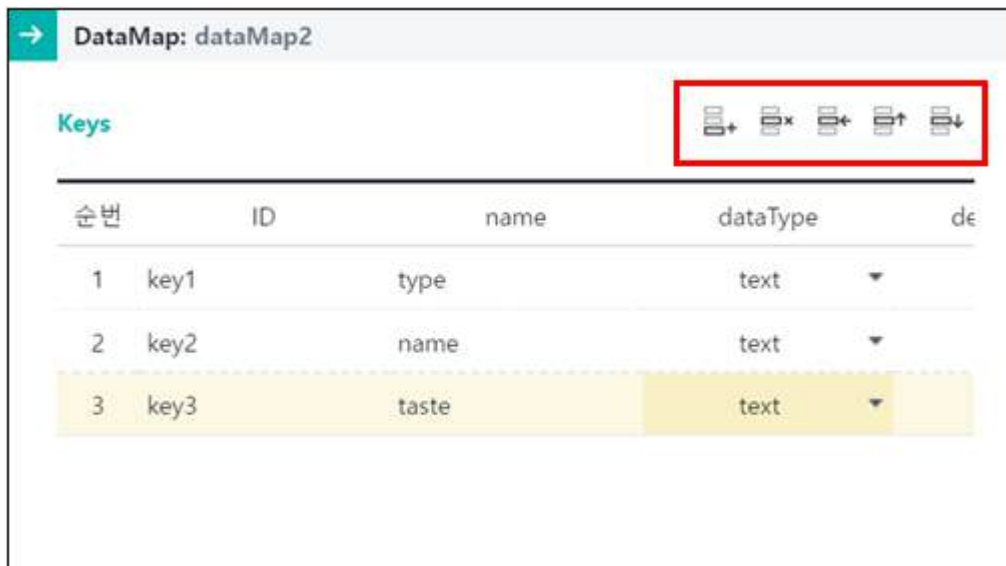
6. DataMap의 키를 정의하는 버튼을 클릭합니다.

그림 12-49 DataMap 키 추가 버튼



7. 우측의 버튼을 클릭하여 DataMap 키를 정의합니다.

그림 12-50 DataMap 키 정의



8. DataMap의 키에 데이터를 추가하는 버튼을 클릭합니다.

그림 12-51 DataMap 데이터 추가 버튼



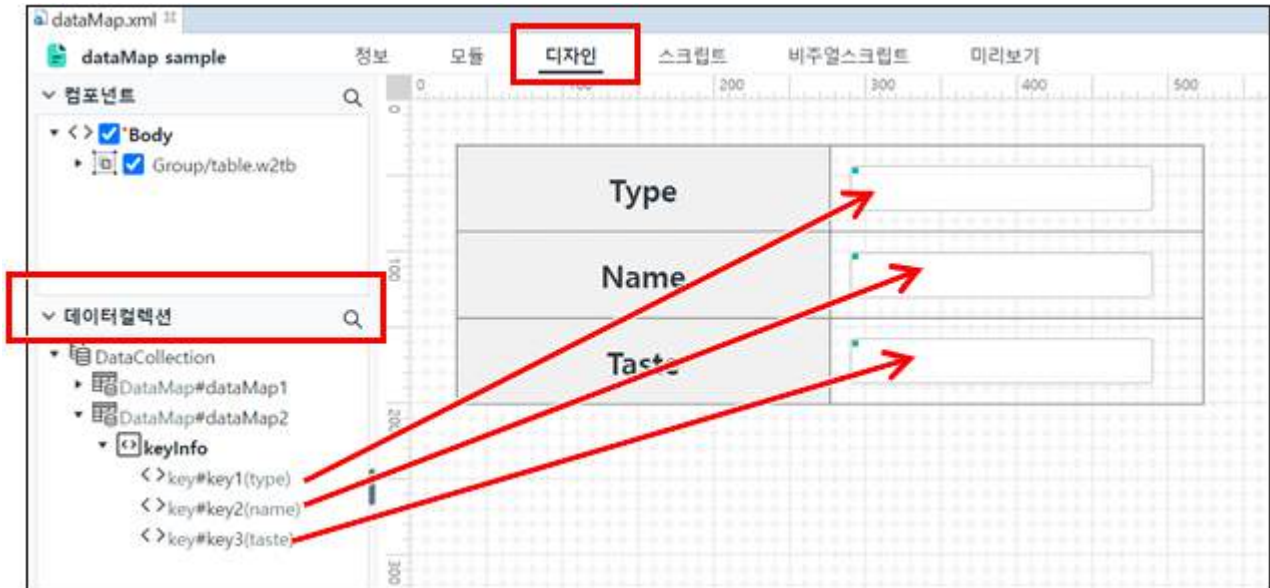
9. 로컬에서 정의한 데이터를 사용할 경우 **데이터 사용** 옵션을 선택하고, 우측의 버튼을 클릭하여 DataMap 키에 대한 데이터를 추가합니다.

그림 12-52 DataMap 데이터를 로컬에 정의



10. DataMap에 정의된 형태의 데이터를 출력할 화면을 Design 탭에서 구성합니다.
11. 데이터컬렉션 탭에 추가된 DataMap의 Key를 바인딩할 컴포넌트로 드래그-앤-드롭합니다.
12. 바인딩된 컴포넌트에 초록색 표시가 나타나는지 확인합니다.

그림 12-53 바인딩



13. 브라우저에서 화면 파일을 실행하여 정상적으로 바인딩 되었는지 확인합니다.

그림 12-54 실행 화면



12.3.2. 데이터 타입 자동 변경

DataMap에 설정하려는 데이터의 타입과 DataMap 키의 `dataType` 속성 값이 일치하지 않을 경우, `keepDataType` 및 `preserveType` 속성을 사용하여 DataMap 키의 `dataType`에 따라 데이터를 설정할 수 있습니다.

12.3.2.1. **string** 타입 데이터를 **number** 타입으로 설정 (**keepDataType**)

string 타입의 데이터를 DataMap에 설정할 때 DataMap 키의 **dataType**이 **"number"**인 경우, **string** 타입의 데이터를 **dataType="number"** 타입으로 변환하여 DataMap 키에 설정할 수 있습니다. 이 기능을 사용하려면 DataMap의 **keepDataType** 속성 값을 **"true"**로 설정하십시오.



예제 파일

인터넷에서 다운로드 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/_DataCollection/keepDataType_DataMap.xml)

그림 12-55 YouTube 동영상 (<https://youtu.be/G8l864qjL6M>)



12.3.2.2. **number** 타입 데이터를 **string** 타입으로 설정 (**preserveType**)

number 타입의 데이터를 DataMap에 설정할 때 DataMap 키의 **dataType**이 **"text"**인 경우, **number** 타입의 데이터를 **dataType="text"** (string) 타입으로 변환하여 DataMap 키에 설정할 수 있습니다. 이 기능을 사용하려면 DataMap의 **preserveType** 속성 값을 **"true"**로 설정하십시오.



예제 파일

[인터넷에서 다운로드](#) 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/_DataCollection/preserveType_DataMap.xml)

그림 12-56 [YouTube 동영상](https://youtu.be/9LetTM_qW-A) (https://youtu.be/9LetTM_qW-A)



12.3.3. 동적 바인딩

`setRef()` 함수를 사용하여 DataMap과 컴포넌트를 동적으로 바인딩할 수 있습니다.

Code 12-11 사용 예

```
1 setRef("data:dataMap2.key1");
```



예제 파일

[인터넷에서 다운로드](#) 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기
 (/SearchBox/setRef_SearchBox.xml)

그림 12-57 YouTube 동영상 (<https://youtu.be/bVHR842dBRQ>)



12.3.4. 바인딩된 컴포넌트 확인

`getRefComponent()` 함수를 DataMap과 바인딩된 컴포넌트(들)의 ID를 반환할 수 있습니다.



예제 파일

인터넷에서 다운로드 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기
 (/DataCollection/getRefComponent_DataMap.xml)

그림 12-58 YouTube 동영상 (<https://youtu.be/DahV6YmncI4>)

12.3.5. 공백 데이터

12.3.5.1. 공백 데이터 처리

DataMap 공백 데이터를 포함한 경우, 공백 데이터를 공백("")으로 치환하여 반환하거나, 공백 데이터를 제외하고 반환하거나, 혹은, `null` 값으로 반환할 수 있습니다. DataMap의 `nullYNType` 속성 값을 아래와 같이 설정하십시오.

- "`default`" (기본 값) : 공백("")으로 치환.
- "`exclude`" : 공백 데이터를 제외하고 반환.
- "`null`" : `null`로 반환.



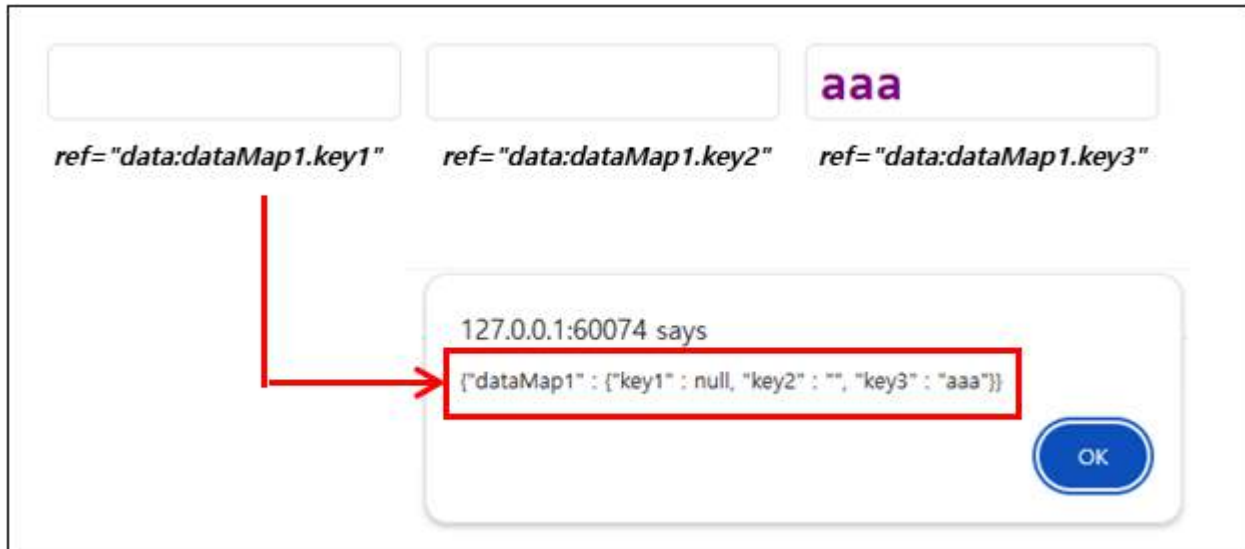
DataMap 키(`dataMap.key`)의 `nullYN` 속성 설정

DataMap의 `nullYNType` 속성 설정을 적용하려면, 각 DataMap 키의 (`dataMap.key`)의 `nullYN` 속성 값을 "`true`"로 설정하십시오. `nullYN="true"` 속성 설정이 적용된 DataMap 키(`dataMap.key`)만 DataMap의 `nullYNType` 속성 설정에 따라 공백 데이터가 처리됩니다.

아래 예제는 DataMap과 바인딩된 세 개의 InputBox입니다. DataMap에 `nullYNType="true"` 설정을 적용한 후, D

ataMap의 **key1** 키에 대해서만 **nullYN="true"** 설정을 적용하여 데이터를 반환할 경우, **key1**의 공백 데이터는 **"null"**로 반환되고, **key2**의 공백 데이터는 공백("")으로 반환되는 것을 확인할 수 있습니다.

그림 12-59 **nullYNType="null"** (DataMap) **nullYN="true"** (**key1** of DataMap)



예제 파일

[인터넷에서 다운로드](#) 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/DataMap/nullYNType_DataMap.xml)

12.3.5.2. **null** 값과 공백("") 값 구분

DataMap의 **useNull** 속성을 사용하여 **null** 값과 공백("") 값을 구분하여 설정할 수 있습니다.

useNull

"true" : **null** 값과 공백("") 값을 구분하여 설정.

"false" (기본 값) : 구분 없이 모두 공백("")으로 설정.



예제 파일

[인터넷에서 다운로드](#) 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/DataMap/useNull_DataMap.xml)

Code 12-12 DataMap에 공백("")과 **null** 값을 설정 (예시)

```

1 // DataMap에 useNull="true" 속성 설정을 적용할 경우,
2 // null 값과 공백 값을 구분해서 설정 가능.
3
4 // "key1", "key2", "key3"에 각각 3, 공백(""), null 값을 설정.
5
6 dataMap1.setArray([3, "", null]);

```

그림 12-60 실제 설정된 값



12.4 LinkedDataList

12.4.1. 권장 사용법

LinkedDataList는 DataList의 데이터를 필터링하거나 정렬한 결과만을 저장하는 객체입니다. DataList 데이터를 필터링하여 아래와 같은 컴포넌트의 선택항목으로 표시할 때 LinkedDataList를 사용하십시오

- AutoComplete
- CheckBox
- CheckComboBox
- MultiSelect
- Radio
- SelectBox



권장 사용법

DataList의 데이터는 1,000 건 이하를 권장하며 최대 10,000건까지 지원합니다.

LinkedDataList는 행 추가, 데이터 일괄 추가, 데이터 전체 삭제 기능을 제외한 DataList에서 지원하는 함수를 모두 지원합니다.

Table 12-2 linkedDataList 속성

속성	설명
bind	원본 데이터가 저장된 DataList
filterCondition	필터링 조건식
sortCondition	정렬 조건식

12.4.2. 필터링 및 정렬 조건 설정

LinkedDataList는 DataList와 바인딩되어 바인딩된 DataList의 데이터를 정렬하거나 필터링하여 사용합니다. LinkedDataList를 생성할 경우, 바인딩할 DataList를 지정하고 아래와 같이 필터링 및 정렬 조건을 설정하십시오.

Table 12-3 필터링 조건 설정

필터	함수 조건식	설명
val	Bno==val('selectbox1')	bind된 data가 selectBox1의 value와 같은 데이터만 표현
ref	Bno == ref('data:dataList1.Bno')	bind된 data가 dataList1의 colId 'Bno'의 데이터 값과 같은 데이터만 표현
default	Bno == "1"	bind된 data의 colId 'Bno'가 1인 데이터만 표현

Table 12-4 비교문 관계 & 논리 연산자

비교문 조건식	설명
A == B	A와 B가 일치하는 데이터 반환
A != B	A와 B가 일치하지 않는 데이터 반환
A >= B	A가 B보다 크거나 같은 데이터 반환
A <= B	A가 B보다 작거나 같은 데이터 반환
A > B	A가 B보다 큰 데이터 반환
A < B	A가 B보다 작은 데이터 반환
(A) && (B)	A 조건과 B 조건이 모두 참인 데이터 반환
(A) (B)	A 조건 또는 B 조건 둘 중 어느 하나가 참이면 데이터 반환

정렬 조건 설정 예시

ASC('col0') : col0 컬럼의 데이터를 오름차순으로 정렬하여 사용.

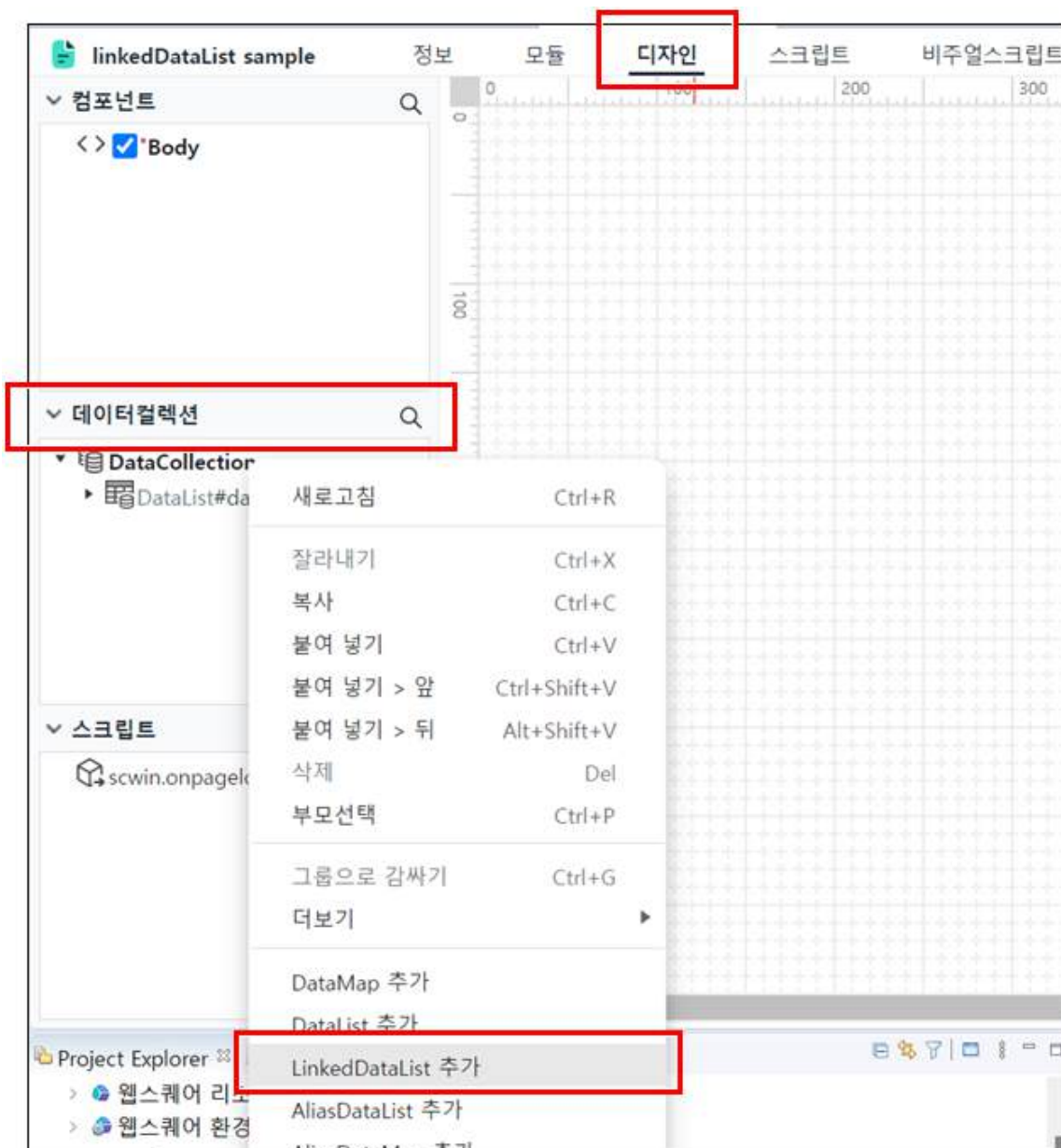
DESC('col0') : col0 컬럼의 데이터를 내림차순으로 정렬하여 사용.

DESC('col0') && DESC('col1') : col0 컬럼 및 col1 컬럼의 데이터를 내림차순으로 이중 정렬하여 사용.

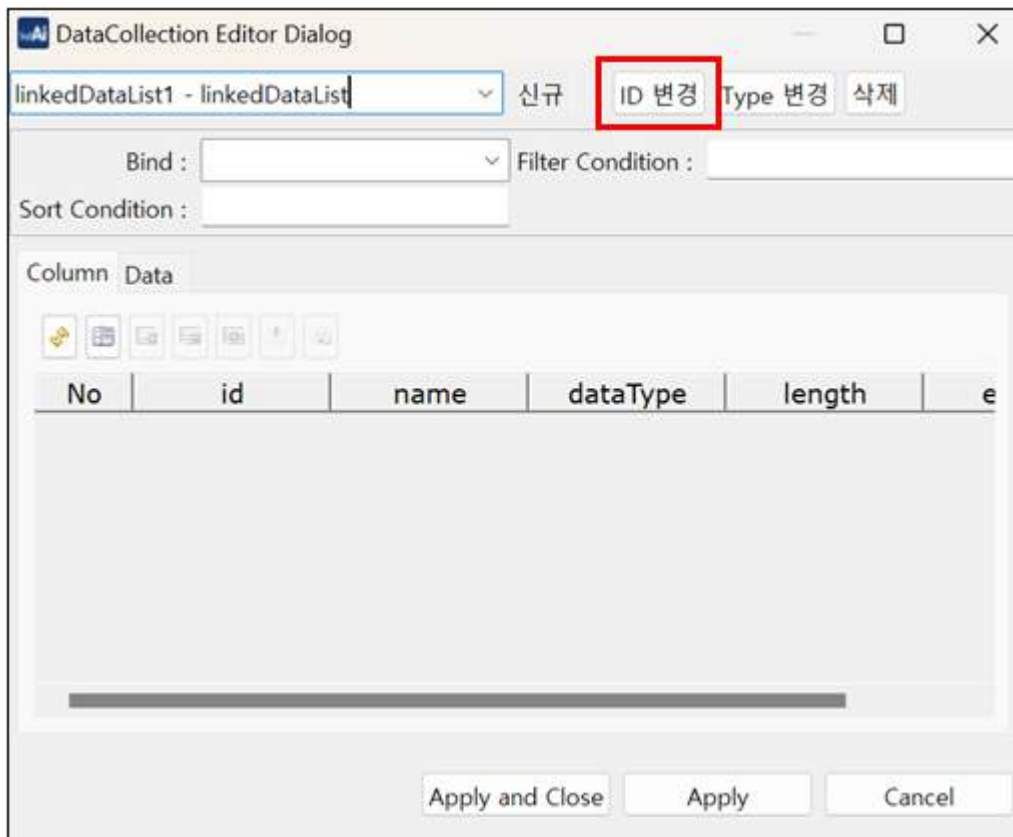
12.4.3. 생성

12.4.3.1. 디자인 탭에서 생성

1. 디자인 에디터의 디자인 - 데이터컬렉션 탭으로 이동합니다.
2. **DataCollection** 항목을 우클릭하고 **LinkedList** 추가 메뉴를 선택합니다.



3. 아래와 같은 화면이 나타납니다. (LinkedDataList의 ID를 변경하려면 **ID 변경** 버튼을 클릭하십시오.)



4. 바인딩할 DataList를 선택합니다.

5. 필터링 조건 혹은 정렬 조건을 입력합니다.

그림 12-61 필터링 조건 설정 (dataList1의 데이터 중 Country 컬럼의 값이 "KOREA"인 데이터만 사용)

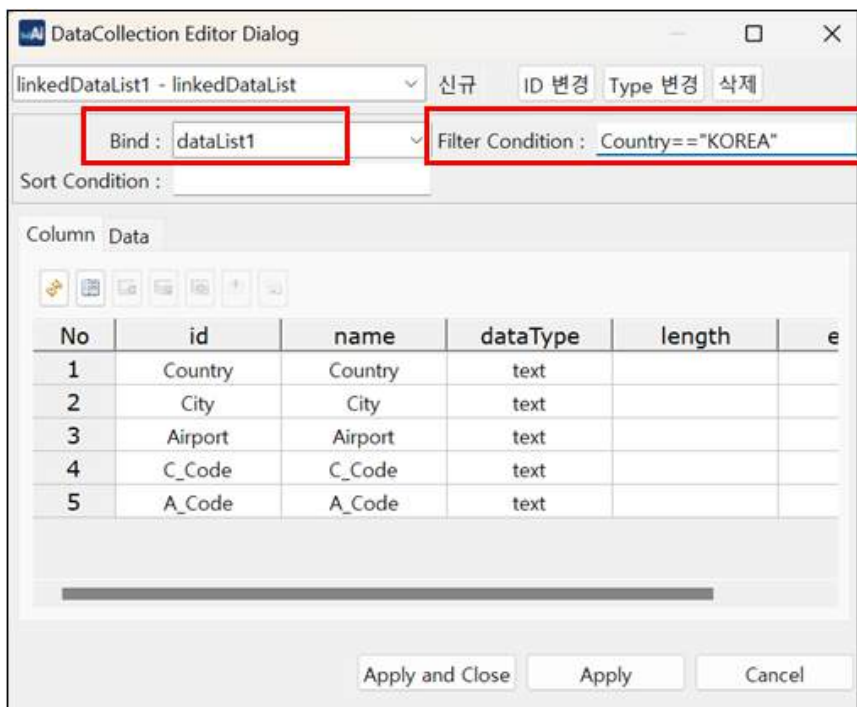
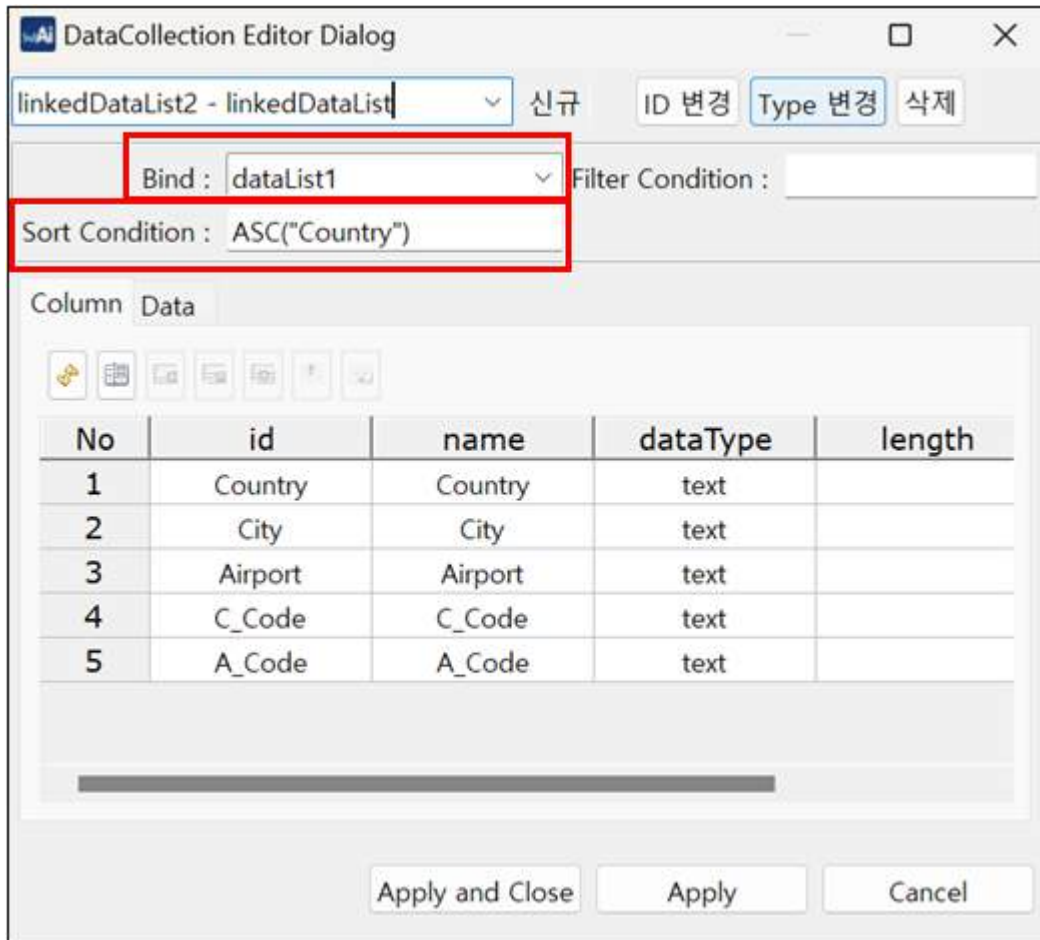
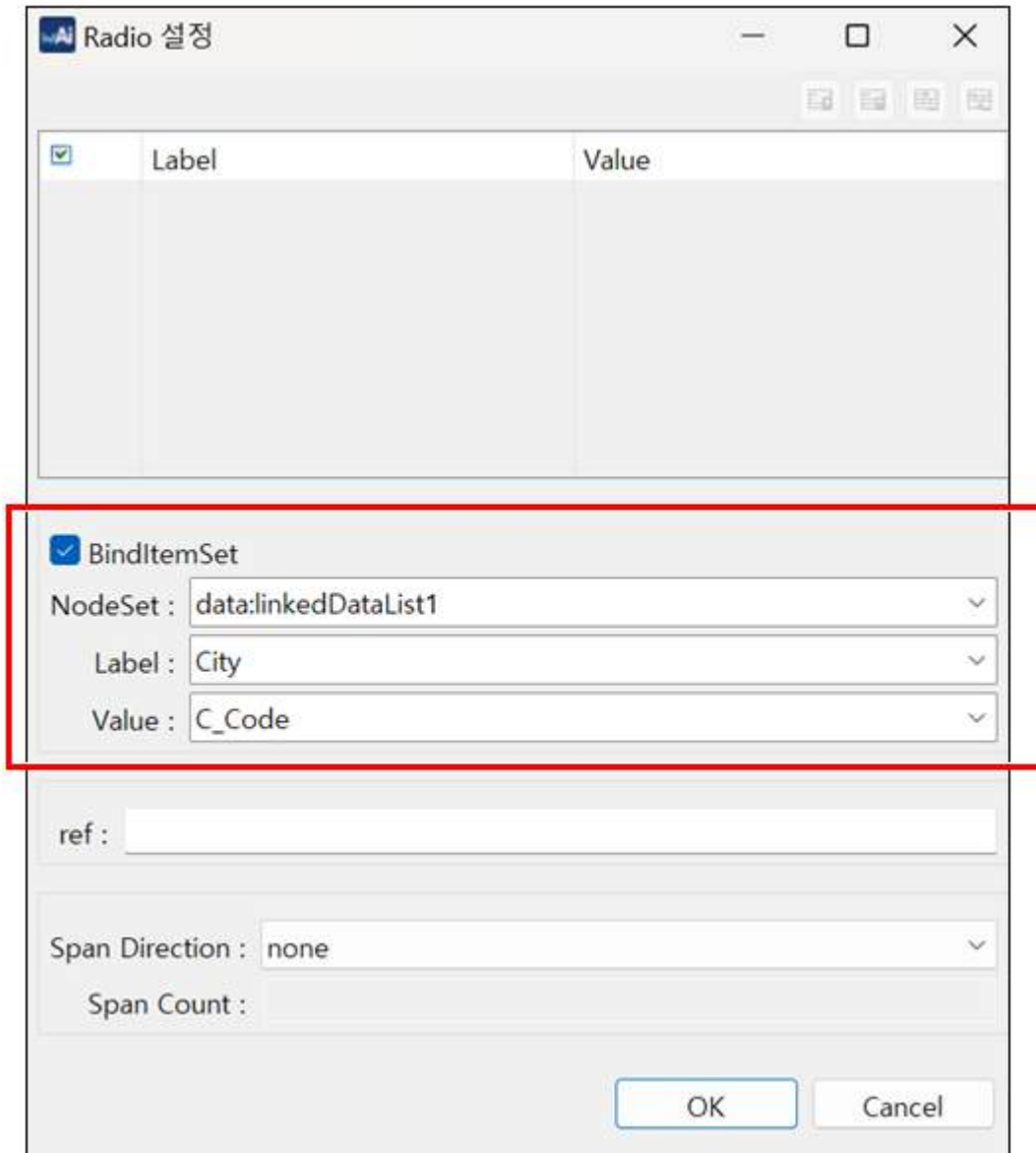


그림 12-62 정렬 조건 설정 (dataList1의 데이터 중 Country 컬럼을 오름 차순으로 정렬하여 사용)



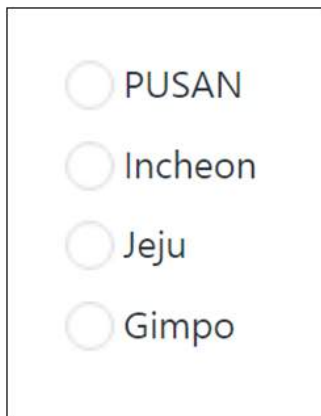
6. 디자인 탭에서 LinkedDataList와 바인딩할 수 있는 컴포넌트를 추가합니다. (LinkedDataList와 바인딩할 수 있는 컴포넌트는 AutoComplete, CheckBox, CheckComboBox, MultiSelect, Radio, 그리고 SelectBox입니다.)
7. 추가한 컴포넌트를 더블 클릭합니다.
8. 아래와 같은 창이 나타나면 **BindItemSet** 항목을 선택하고 바인딩할 LinkedDataList를 **NodeSet**으로 선택합니다.

그림 12-63 Radio 컴포넌트의 바인딩 설정 (1)



9. 브라우저에서 화면 파일을 실행하여 정상적으로 바인딩 되었는지 확인합니다. (위의 5단계에서 설정한 것처럼 한국에 있는 도시(Country=="KOREA")만 표시됩니다.)

그림 12-64 실제 실행 화면



아래의 예제는 위의 5단계에서 dataList1의 데이터 중 Country 컬럼을 오름 차순으로 정렬하여 사용linkedDataList 2를 바인딩합니다.

그림 12-65 Radio 컴포넌트의 바인딩 설정 (2)

The screenshot shows the 'Radio 설정' (Radio Settings) dialog box. At the top, there is a table with columns 'Label' and 'Value'. Below this, the 'BindItemSet' checkbox is checked and highlighted with a red box. Under 'BindItemSet', the following settings are visible:

- NodeSet : data:linkedDataList2
- Label : Country
- Value : C_Code

Below these settings, there are fields for 'ref', 'Span Direction' (set to 'none'), and 'Span Count'. At the bottom, there are 'OK' and 'Cancel' buttons.

이 경우 국가 이름이 오름차순으로 정렬되어 표시되는 것을 확인할 수 있습니다.

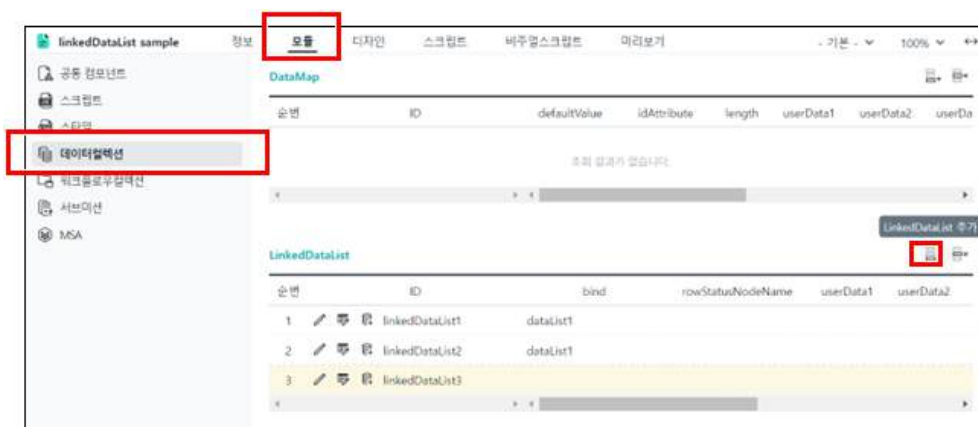
그림 12-66 실제 실행 화면



12.4.3.2. 모듈 탭에서 생성

1. 디자인 에디터의 모듈 - 데이터컬렉션 탭으로 이동합니다.
2. LinkedList 우측의 추가 버튼을 클릭하여 LinkedList를 추가합니다.

그림 12-67 LinkedList 추가



3. DataList 정보를 수정하는 버튼을 클릭합니다.

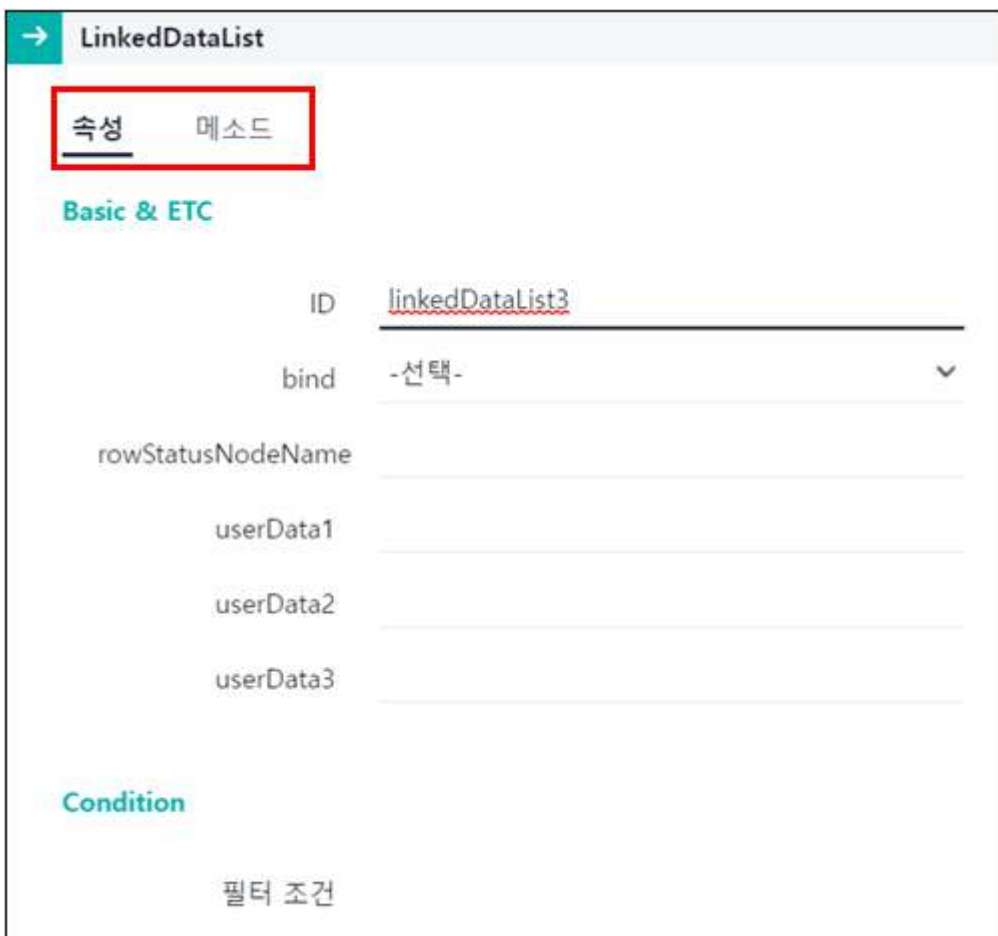


순번	ID	bind	rowStatusNodeName	userData1	userData2
1	linkedDataList1	dataList1			
2	linkedDataList2	dataList1			
3	linkedDataList3				

4. LinkedDataList의 ID를 지정합니다.

5. 상단의 속성, 이벤트, 메소드 탭을 클릭하여 Linked DataList의 속성값을 정의하고 DataList가 지원하는 함수를 확인할 수도 있습니다.

그림 12-68 LinkedDataList 정보 설정



→ **LinkedDataList**

속성 메소드

Basic & ETC

ID: linkedDataList3

bind: -선택- ▼

rowStatusNodeName: _____

userData1: _____

userData2: _____

userData3: _____

Condition

필터 조건: _____

6. LinkedDataList의 컬럼을 정의하는 버튼을 클릭합니다.

LinkedDataList						
순번		ID	bind	rowStatusNodeName	userData1	userData2
1	  	linkedDataList1	dataList1			
2	  	linkedDataList2	dataList1			
3	  	linkedDataList3				

7. 바인딩할 DataList와 필터링 조건 혹은 정렬 조건을 입력합니다.

그림 12-69 필터링 조건 설정 (dataList1의 데이터 중 Country 컬럼의 값이 "U.S.A."인 데이터만 사용) 비주얼

→

LinkedDataList: linkedDataList3

바인드 정보

바인드

dataList1

▼

필터 조건

Country=="U.S.A."

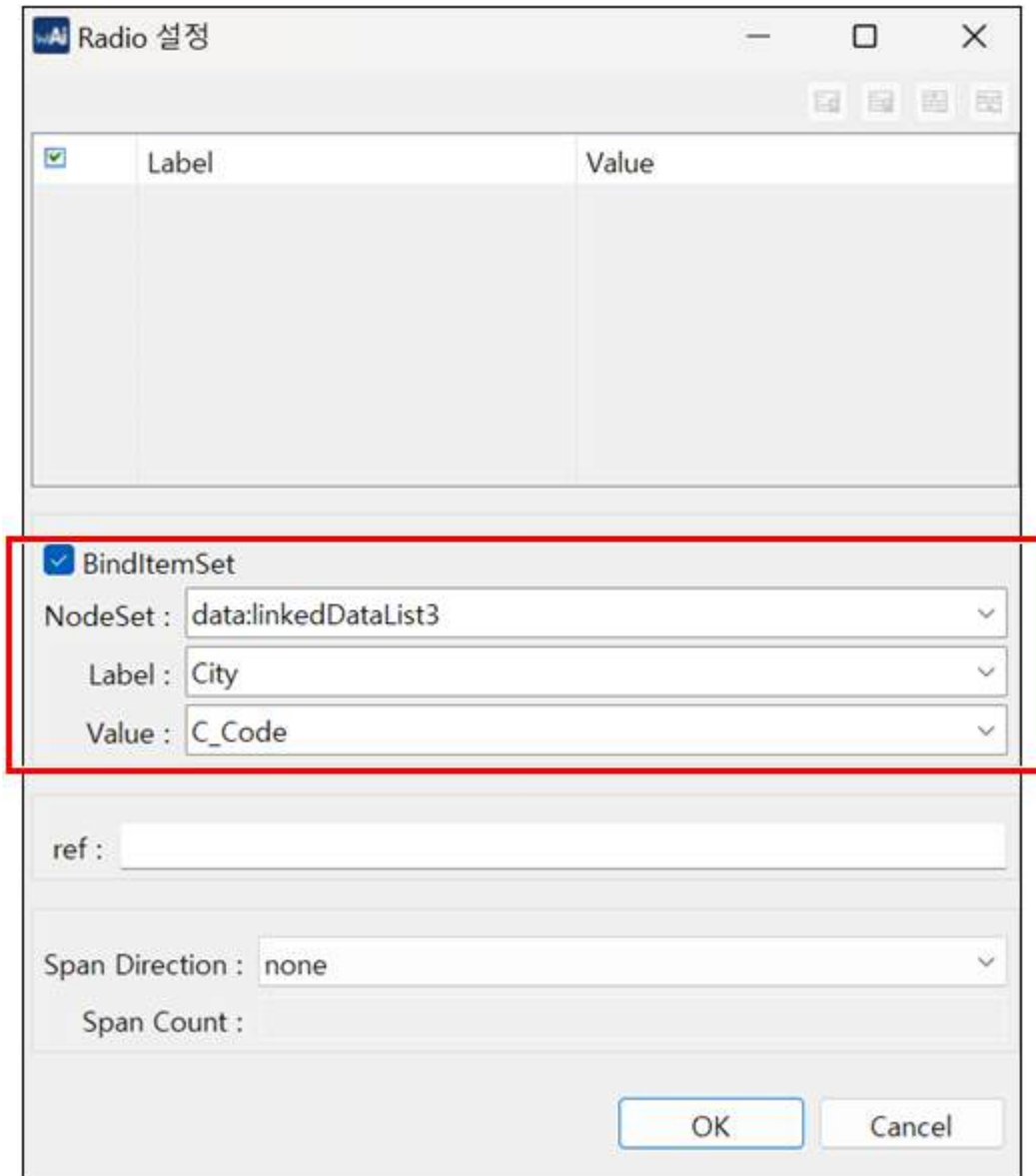
정렬 조건

Columns

순번	ID	name	dataType	defaultValue
1	Country	Country	text	
2	City	City	text	
3	Airport	Airport	text	
4	C_Code	C_Code	text	
5	A_Code	A_Code	text	

- 디자인 탭에서 AutoComplete, CheckBox, CheckComboBox, MultiSelect, Radio, 혹은 SelectBox 컴포넌트를 추가합니다.
- 추가한 컴포넌트를 더블 클릭합니다.
- 아래와 같은 창이 나타나면 **BindItemSet** 항목을 선택하고 바인딩할 LinkedDataList를 **NodeSet**으로 선택합니다.

그림 12-70 Radio 컴포넌트의 바인딩 설정



11. 브라우저에서 화면 파일을 실행하여 정상적으로 바인딩 되었는지 확인합니다. (위의 7 단계에서 설정한 것처럼 미국에 있는 도시(Country=="U.S.A.")만 표시됩니다.)

그림 12-71 실제 실행 화면

☐ PORTLAND
 ☐ PHILADELPHIA
 ☐ PHOENIX
 ☐ PITTSBURGH

13. AliasDataCollection

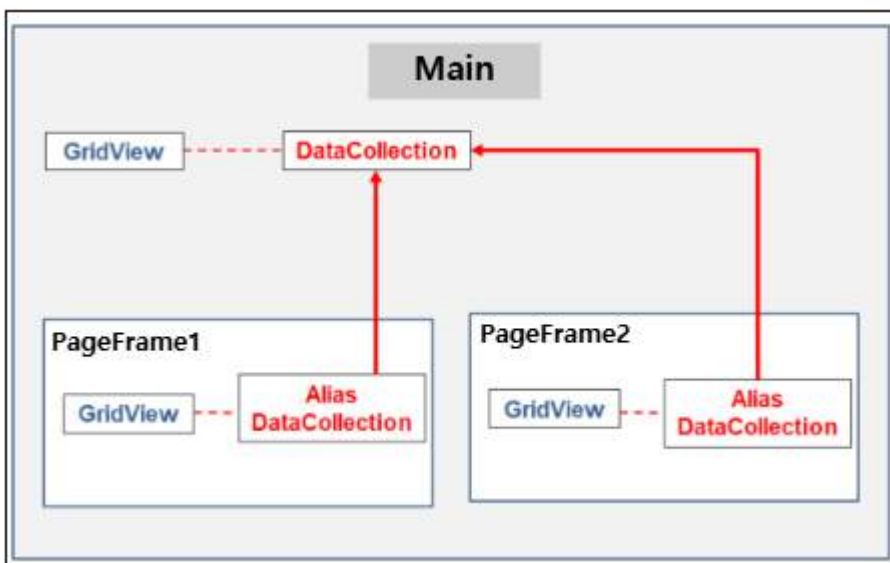
13.1 정의

PageFrame을 사용할 경우, 모든 동작은 Scope 내에서 실행됩니다. 즉, PageFrame 내에서 GridView를 사용할 경우 바인딩 되는 DataCollection 역시 동일한 PageFrame 내에 존재해야 합니다. 단, 실제 업무 환경에서는 메인 화면에 존재하는 DataCollection을 여러 화면의 GridView가 참조하는 경우가 많습니다. 이 같은 요구를 해결하기 위한 것이 Alias DataCollection입니다.

각 PageFrame 내에 AliasDataCollection을 정의한 후, 실제 바인딩되는 메인 (혹은 부모) 화면의 DataCollection 과 연결하여 이 같은 업무 환경을 지원할 수 있습니다.

DataCollection이 제공하는 모든 API를 AliasDataCollection에서 동일하게 사용할 수 있습니다.

그림 13-1 AliasDataCollection



제약사항

부모 화면에 있는 DataCollection 참조만 지원합니다.
자식 및 형제 참조는 지원하지 않습니다.



예제 파일

[인터넷에서 다운로드](#) 혹은
WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/_DataCollection/scope_aliasDataMap/)

그림 13-2 YouTube 동영상 (<https://youtu.be/VOn4wslrBTA>)



13.2 주요 API

13.2.1. 속성

Table 13-1 주요 속성

속성	설명
<code>id</code>	PageFrame 내에서 사용할 객체 ID
<code>scope</code>	부모 페이지에 존재하는 (링크 대상) DataCollection의 ID - 상대 경로 지원 (예: <code>scope="../dataList1"</code>) - 절대 경로 지원 (예: <code>scope="main/dataList1"</code>)

13.2.2. 함수

Table 13-2 주요 함수

함수	설명
<code>getObj</code>	scope에 지정된 객체를 반환 사용 예: <code>var parentDataList = aliasDataList1.getObj();</code>
<code>setScope</code>	참조할 DataCollection을 동적으로 변경. DataCollection 변경시 페이지에 바인딩된 컴포넌트도 자동 갱신됨. 사용 예: <code>aliasDataList1.setScope("/dataList1");</code>
그 외	참조하는 DataCollection에서 제공하는 API 모두 사용 가능. 사용 예: <code>aliasDataList1.getRowCount();</code>

13.3 제약 사항

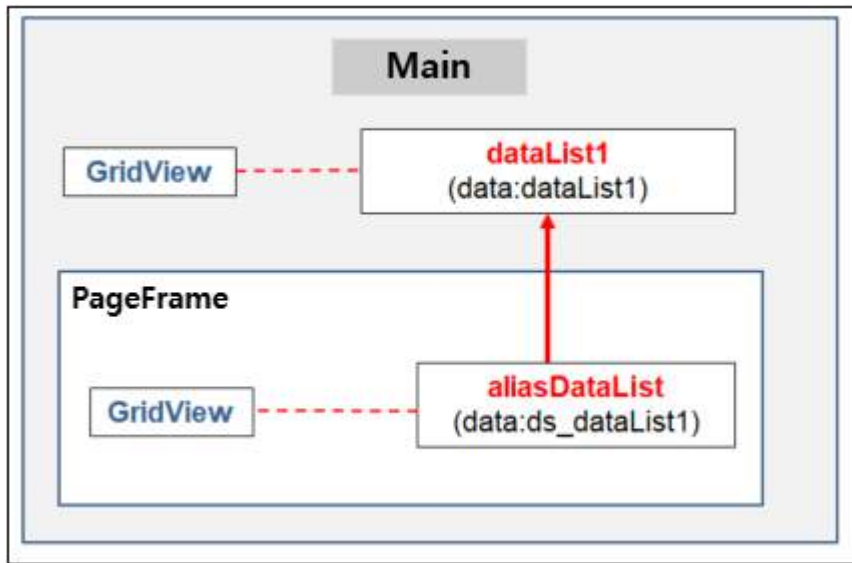
AliasDataCollection은 아래와 같은 제약사항을 갖습니다.

1. 부모 페이지만 참조할 수 있습니다.
2. 동일한 유형만 링크할 수 있습니다. (예를 들어, aliasDataList는 DataList만 참조 가능합니다.)
3. aliasDataList는 다른 화면의 aliasDataList를 참조할 수 없습니다.
4. Scope에 지정된 객체가 존재하지 않는 경우, 빈 DataCollection이 생성됩니다. 이 경우 `setScope`를 이용하여 참조할 DataCollection을 다시 지정하십시오.

13.4 생성

다음은 PageFrame을 사용하여 화면을 구성한 예입니다. PageFrame 내부의 GridView는 Alias DataList를 통해 메인 화면의 GridView와 바인딩된 DataList를 참조합니다.

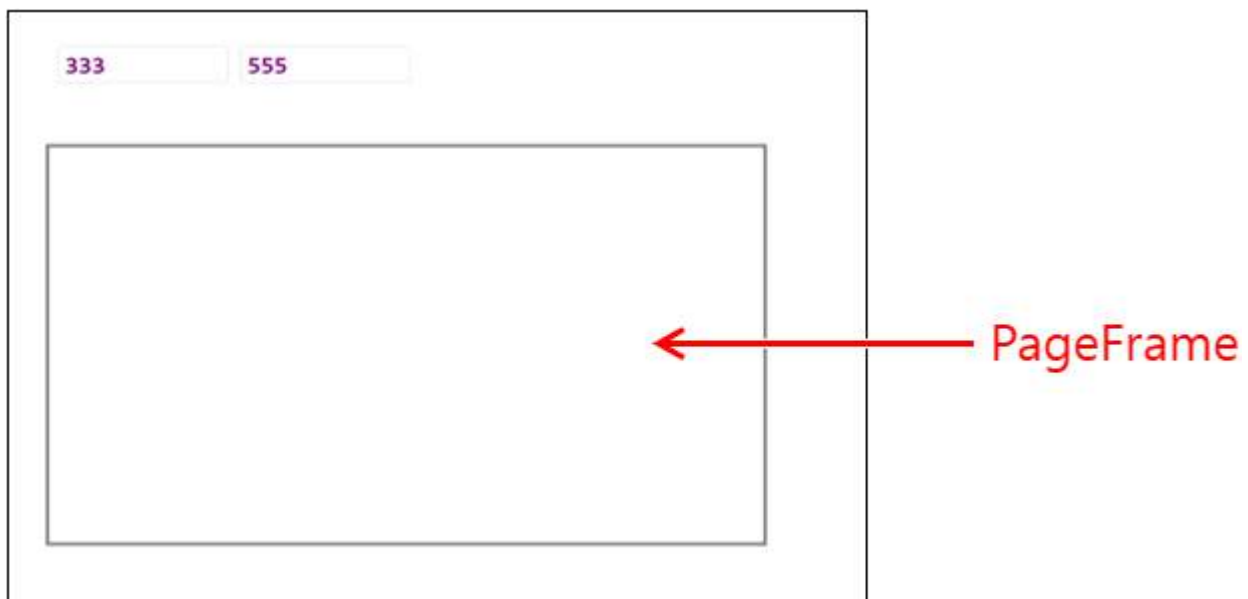
그림 13-3 AliasDataCollection 작성 예



13.4.1. Studio에서 생성

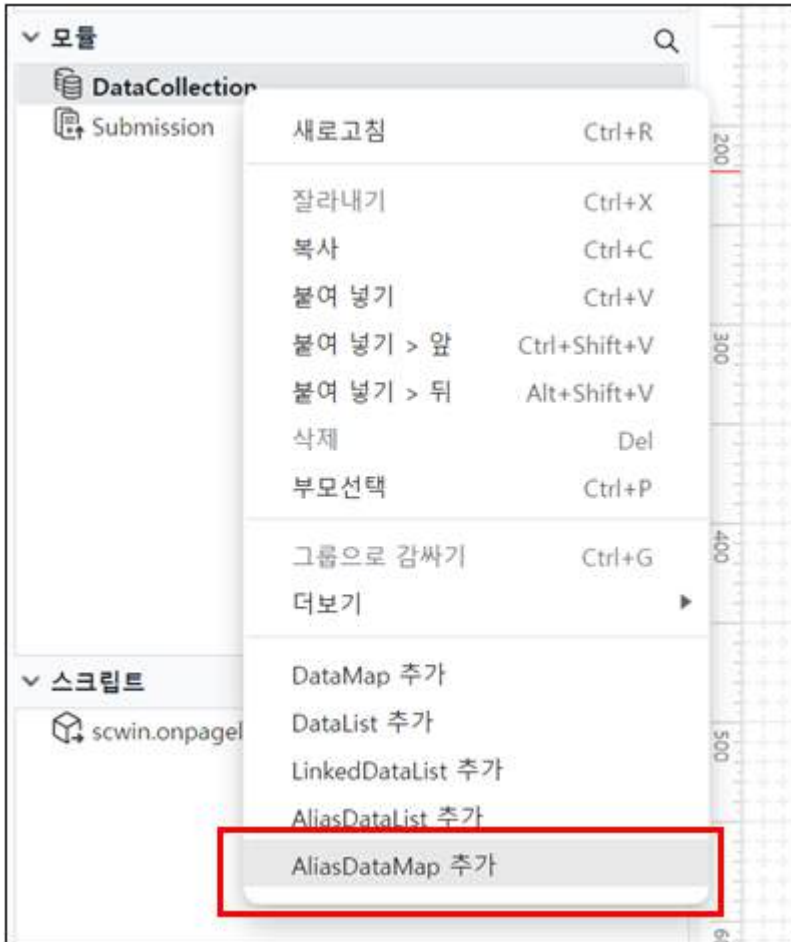
다음은 메인 화면의 DataMap을 참조하는 AliasDataMap을 작성하여 이를 PageFrame 내부의 InputBox와 바인딩하는 예제입니다.

5. 메인 화면을 생성합니다. (*main.xml*)
 - I. 메인 화면에 id가 "dataMap1"인 DataMap을 추가합니다.
 - II. 메인 화면에 InputBox를 추가하고, DataMap과 바인딩합니다.
 - III. 메인 화면에 PageFrame을 추가합니다.

그림 13-4 *main.xml*

6. 위에서 생성한 PageFrame의 소스(src)로 사용할 화면을 생성합니다. (aliasDataMap.xml)
7. # 위의 4단계에서 생성한 화면 파일(aliasDataMap.xml)을 열고, 디자인 - 모듈 탭에서 DataCollection을 우클릭합니다.
8. AliasDataMap 추가를 클릭합니다.

그림 13-5 AliasDataMap 추가 (aliasDataMap.xml)



9. 생성하려는 AliasDataList의 id와 scope 속성을 아래와 같이 지정합니다.

id	AliasDataList의ID 지정. 예: aliasDataMap1
scope	부모 화면(메인 화면)에 있는 참조 대상 DataMap을 지정. 절대 경로 및 상대 경로 모두 지원. (예) - 상대 경로: scope="../dataMap1" - 절대 경로: scope="main/dataMap1"

10. PageFrame의 소스(src) 화면(aliasDataMap.xml)에 InputBox를 추가합니다.
11. 추가한 InputBox의 ref 속성을 위의 제 5단계에서 설정한 AliasDataMap으로 선택합니다.

그림 13-6 *aliasDataMap.xml*

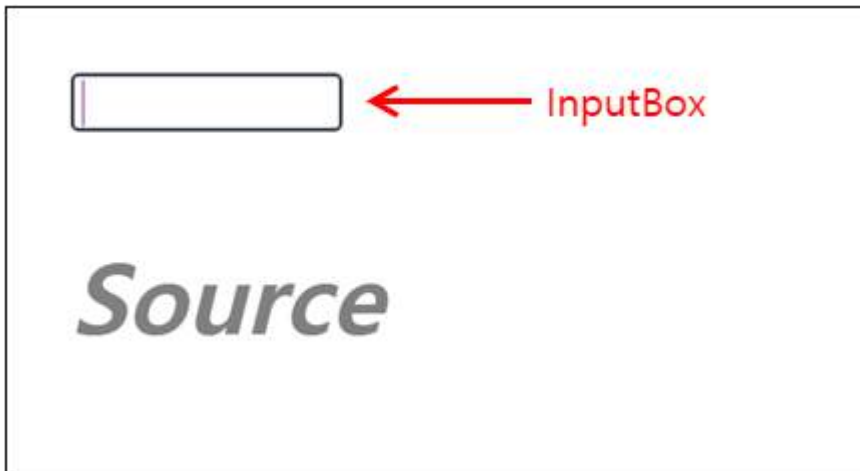
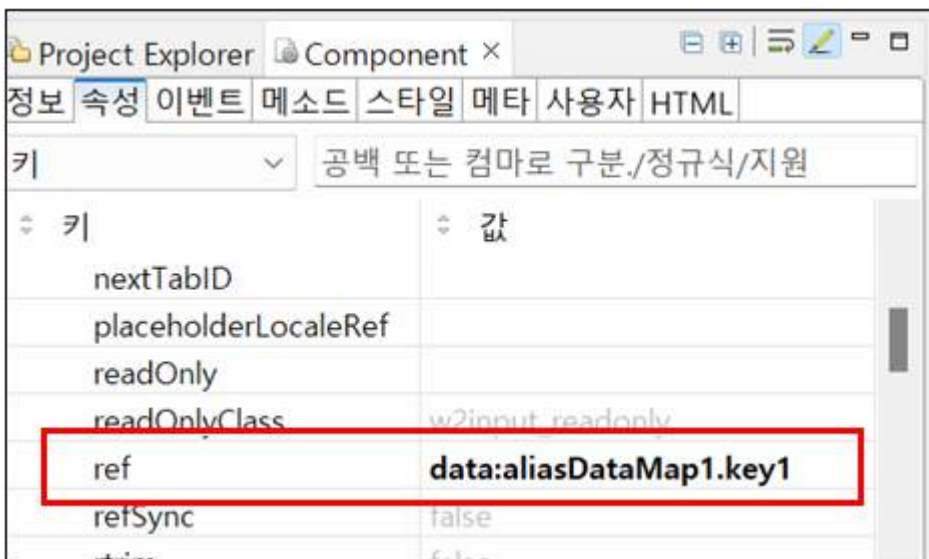
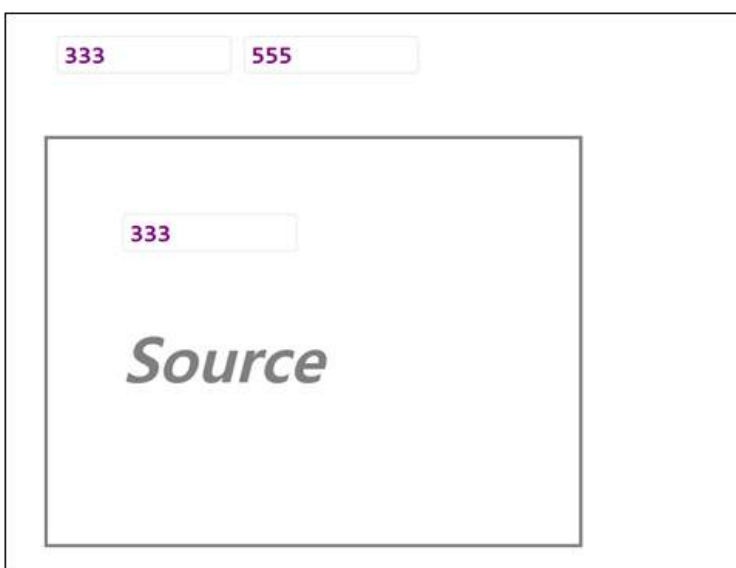


그림 13-7 ref 속성을 통해 InputBox와 aliasDataMap을 바인딩



12. 메인 화면(*main.xml*)에 화면이 정상적으로 표시되는지 확인합니다.



13. 메인 화면(*main.xml*) 및 PageFrame(*aliasDataMap.xml*) 내부에서 Ctrl 버튼을 누르고 마우스 우클릭하여 디버깅 메뉴를 표시한 뒤 View dataCollection 메뉴를 클릭하여 현재 DataCollection 및 AliasDataCollection을 확인할 수 있습니다.

그림 13-8 디버깅 메뉴 - View dataCollection

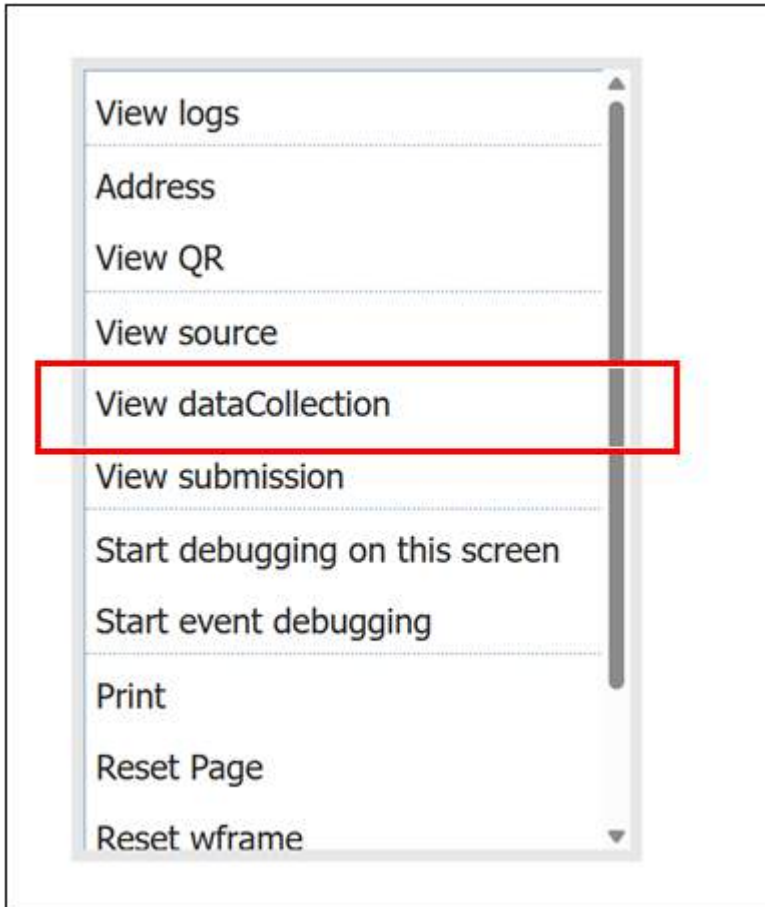


그림 13-9 *main.xml*

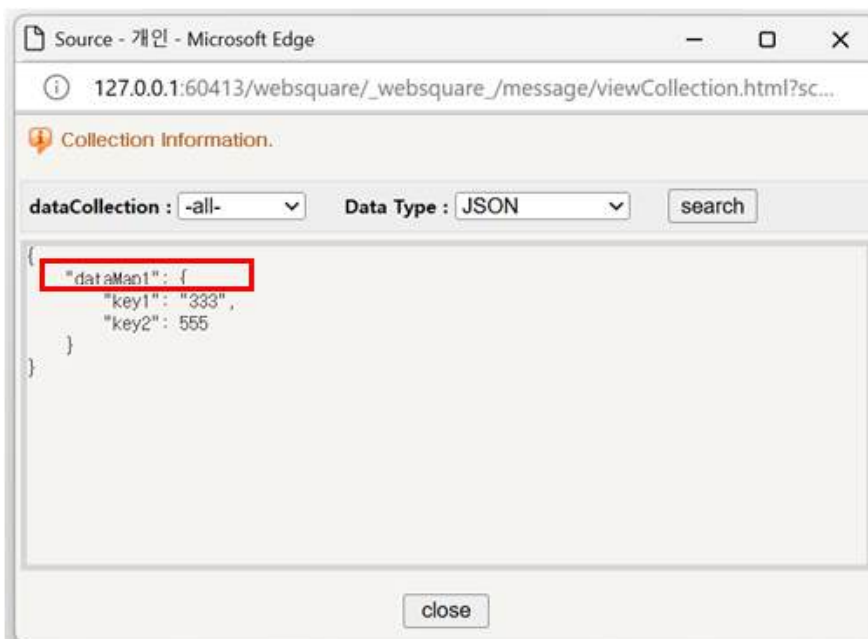
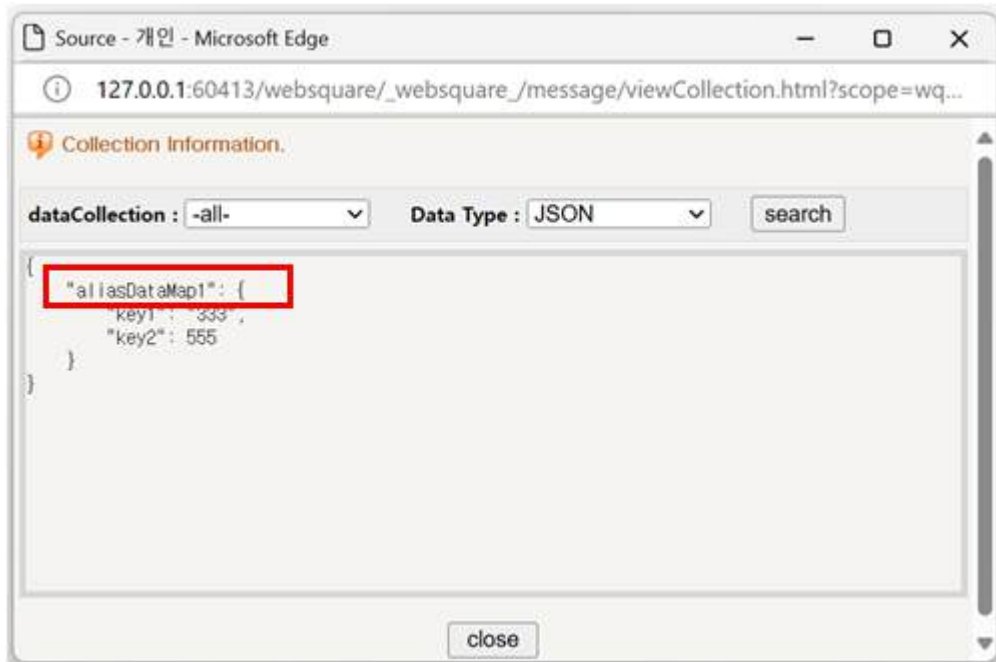


그림 13-10 *aliasDataMap.xml*

13.4.2. 동적 생성 (Script)

DataCollection 생성과 동일한 방법으로 script를 이용하여 aliasDataCollection을 생성할 수 있습니다. 단, option에 `scope` 속성을 추가해야 합니다.

Code 13-1 aliasDataList 생성 예제

```

1 $p.data.create({
2     "id" : "dynamicAliasDataList1",
3     "type" : "aliasDataList",
4     "option" : {
5         "scope" : "../dataList1"
6     }
7 });

```

DataCollection 바인딩 하는 방법과 동일한 방식으로 컴포넌트에 바인딩할 수 있습니다.

Code 13-2 aliasDataList 바인딩 예제

```

1 <w2:gridView dataList="data:aliasDataList1" ... />
2 <xf:select1>
3     <xf:choices>
4         <xf:itemset nodeset="data:aliasDataList1"/>
5     </xf:choices>

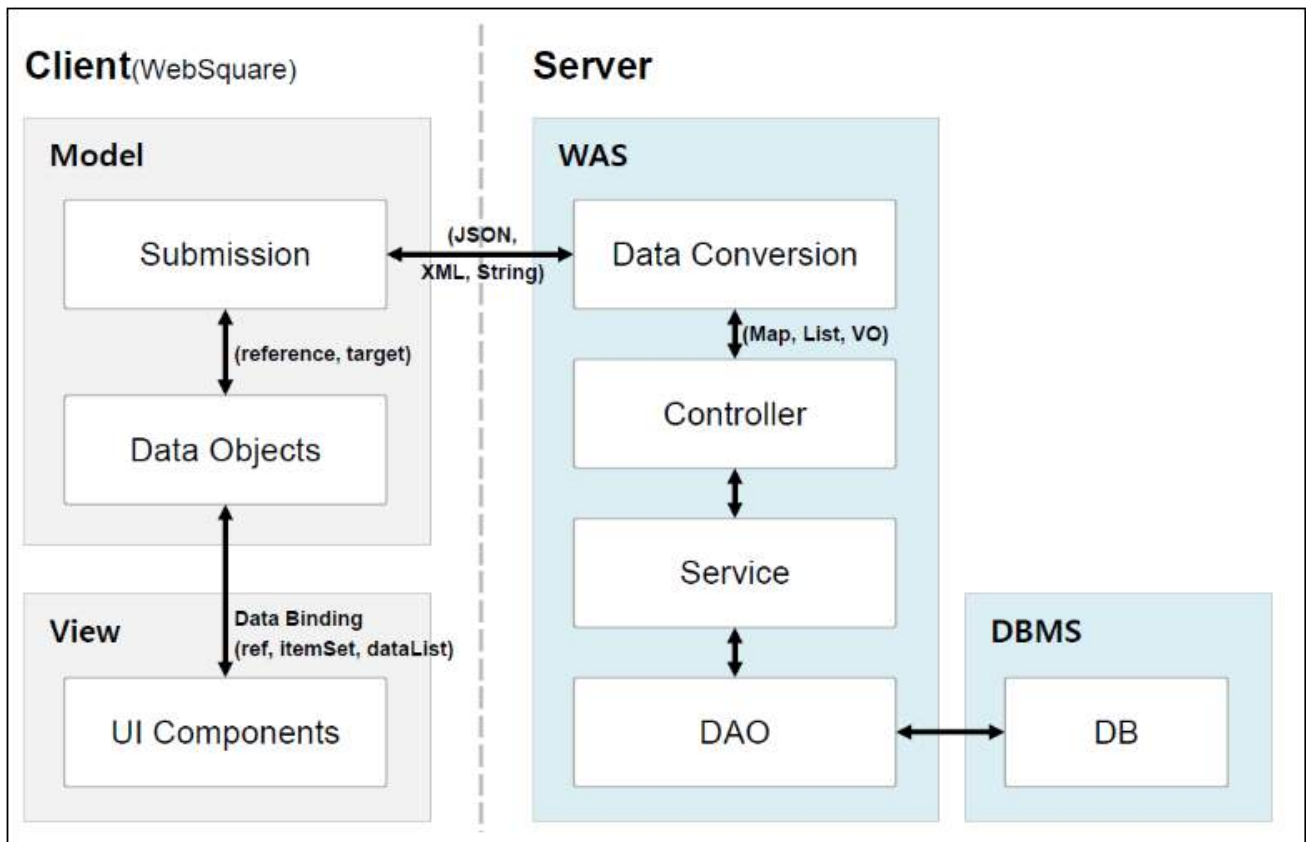
```

6 </xf:select1>

14. Submission

14.1 정의

그림 14-1 서버-클라이언트 통신



Submission은 서버와의 데이터 통신을 위한 모듈로서, AJAX로 구현되어 있습니다. Submission의 특징은 아래와 같습니다.

특징

서버와의 통신용 전송/수신 데이터를 DataCollection 형식으로 정의해야 함.

화면의 전환 없이 데이터만 전송.

동기/비동기 통신 선택 가능.

통신의 전/후 처리를 위해 각 이벤트에 호출할 함수 정의 가능.

Script를 통해 동적 Submission 생성 가능.

ID를 제외한 속성은 동적 변경이 가능.

권장 내용

- 용도별로 Submission을 등록하는 것을 권장. (유지보수 편의성)
- 화면의 코드성 데이터 통신은 공통 모듈을 만들어 이용할 것을 권장

일반적인 순서

- (1) 화면 UI 작성
- (2) 서버와 주고 받는 데이터 형식 정의 (DataCollection 정의)
- (3) 화면 UI와 DataCollection 바인딩
- (4) Submission 생성
- (5) Submission 실행 이벤트 정의

14.2 Submission vs. AJAX

개발자는 데이터 통신을 위해 Submission은 정의하거나 AJAX를 통해 직접 통신 모듈을 개발할 수도 있습니다. Ajax는 Submission을 생성하지 않고 서버를 호출할 수 있습니다. 다음은 Submission과 AJAX를 비교한 것입니다.

Table 14-1 Submission vs. AJAX

	Submission	AJAX
특징	- 웹 페이지 별로 정의 - WebSquare5가 제공하는 형식 사용	- 공통 모듈로 작성하여 일괄 적용이 가능 - 개발자가 원하는 형식을 유연하게 사용 가능
장단점	- 코드의 가독성이 높아짐 - 유지보수가 쉬움	- 코드의 가독성이 떨어짐 - 유지 보수가 어려움
적용	각 웹페이지의 대표적인 기능을 수행하는 데이터 통신 (코드의 가독성을 높이고 유지보수를 쉽게 함)	반복적으로 사용되는 데이터 통신

```

1  var reqstr = '<request>'
2    + '<data value="efg">ABCD</data>'
3    + '</request>';
4
5  $p.ajax({
6    action : "http://localhost:8090/developGuide/getAjax.jsp",
7    mode : "asynchronous",
8    mediatype : "application/xml",

```



```

9      method : "post",
10     requestData : reqstr,
11     type : "xml",
12     beforeAjax : function(e) {
13         WebSquare.logger.printLog("beforeAjax call");
14     },
15     success : function(e) {
16         alert(e.responseText);
17     },
18     error : function(e) {
19         WebSquare.logger.printLog("error call");
20     },
21     requestHeader : {
22         sendData1 : "123",
23         sendData2 : "456"
24     }
25 });

```

Table 14-2 ajax options

options 속성	설명
<code>options.action</code>	ajax 요청주소
<code>options.mode</code>	asynchronous(default)/synchronous
<code>options.mediatype</code>	mediatype
<code>options.method</code>	get/post/put/delete
<code>options.requestData</code>	요청 본문으로 문자열 텍스트 데이터
<code>options.timeout</code>	ajax요청후 timeout 시간. 이시간이 초과해도 응답이 오지 않는 경우 error callback함수를 실행합니다.
<code>options.type</code>	xml/json. xml인 경우, success callback함수의 인자객체의 responseBody속성에 xml객체가 설정되고, json인 경우 자바스크립트 객체가 설정됩니다. 나머지 경우는 text형식이 설정.
<code>options.beforeAjax</code>	요청 전에 실행되는 함수로 이 함수내에서 false를 return하면 ajax요청을 하지 않는다
<code>options.success</code>	요청이 성공한 경우 실행되는 callback함수
<code>options.error</code>	요청이 실패한 경우 실행되는 callback함수
<code>options.requestHeader</code>	request head에 값을 설정하는 객체

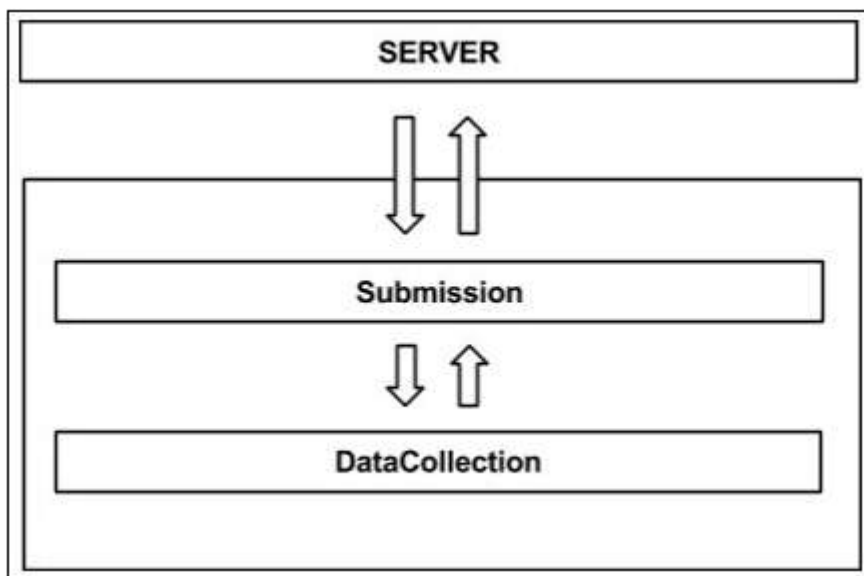
14.3 Submission & DataCollection

Table 14-3 비교

DataCollection	Submission
다양한 종류의 방대한 데이터를 웹 브라우저에서 쉽게 출력하기 위해 WebSquare5가 사용하는 데이터 모델.	DataCollection을 이용하여 서버와의 데이터 통신을 정의

즉, 웹 브라우저와 서버와의 데이터 통신이 필요한 경우 DataCollection을 생성하고 Submission을 정의해야 합니다. (본 장은 Submission을 위주로 설명합니다. DataCollection에 대한 자세한 설명은 본 매뉴얼의 DataCollection 부분을 참조하십시오.)

그림 14-2 Submission Vs. DataCollection



14.4 Submission 생성

14.4.1. Studio에서 생성

1. 디자인 에디터의 모듈 - 서브미션 탭으로 이동합니다.
2. 서브미션 우측의 추가 버튼을 클릭하여 서브미션을 추가합니다.

그림 14-3 서브미션 추가



3. 아래와 같은 창이 표시되면 해당 정보를 입력합니다.

그림 14-4 서브미션 설정 창

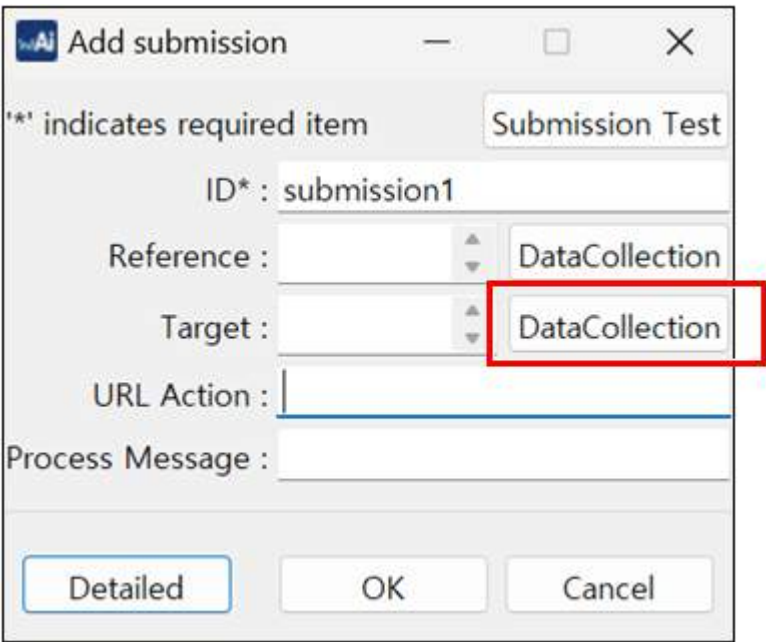


Table 14-4 Submission 속성

항목	설명	비고
ID	Submission ID.	필수.
Reference	서버로 전송할 데이터 형식. (DataCollection 버튼 클릭 후 선택. 단, 해당 DataCollection을 미리 설정해야 함.)	
Target	서버로부터 수신할 데이터 형식. (DataCollection 버튼 클릭 후 선택. 단, 해당 DataCollection을 미리 설정해야 함.)	필수.
URL Action	수신할 데이터가 있는 서버 (혹은 파일의) URL	필수.
Process Message	Submission 실행 중 상태 창에 표시할 메시지.	

그림 14-5 Target 선택

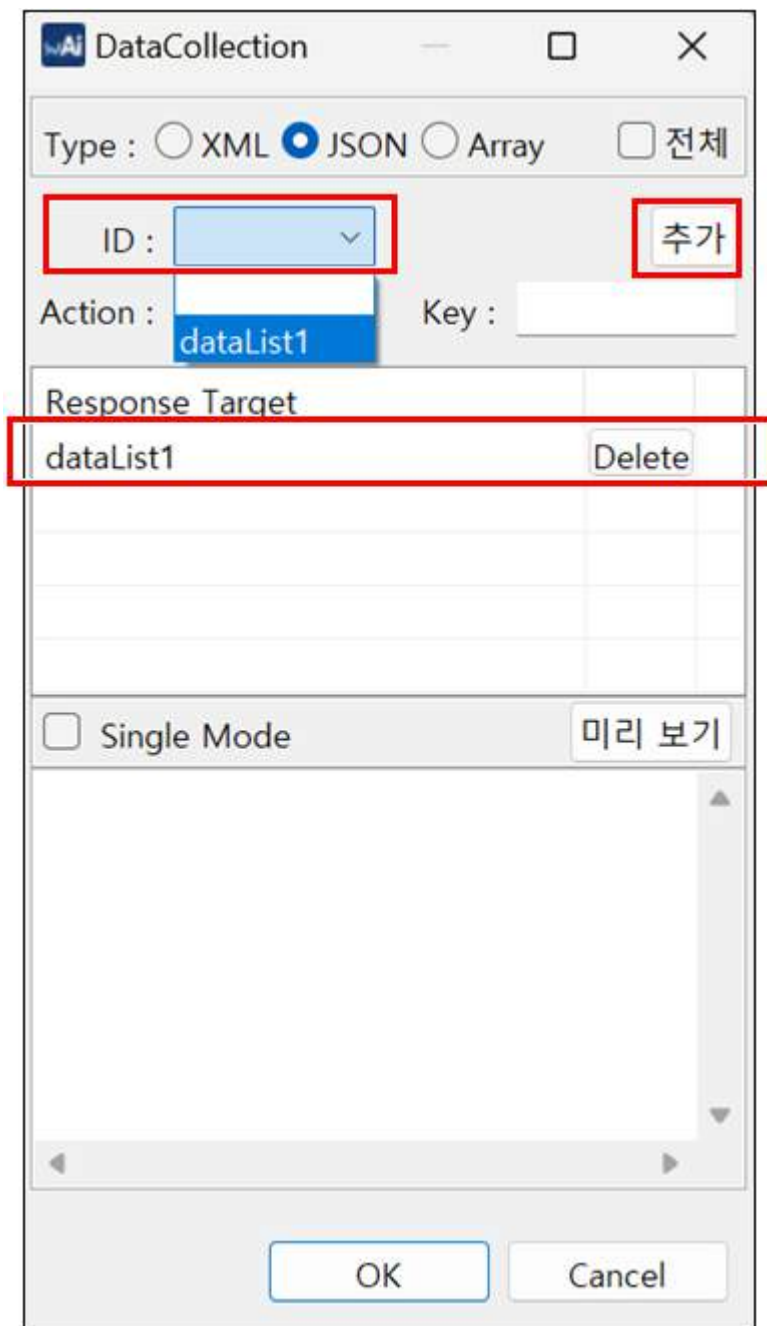


그림 14-6 URL Action 정의

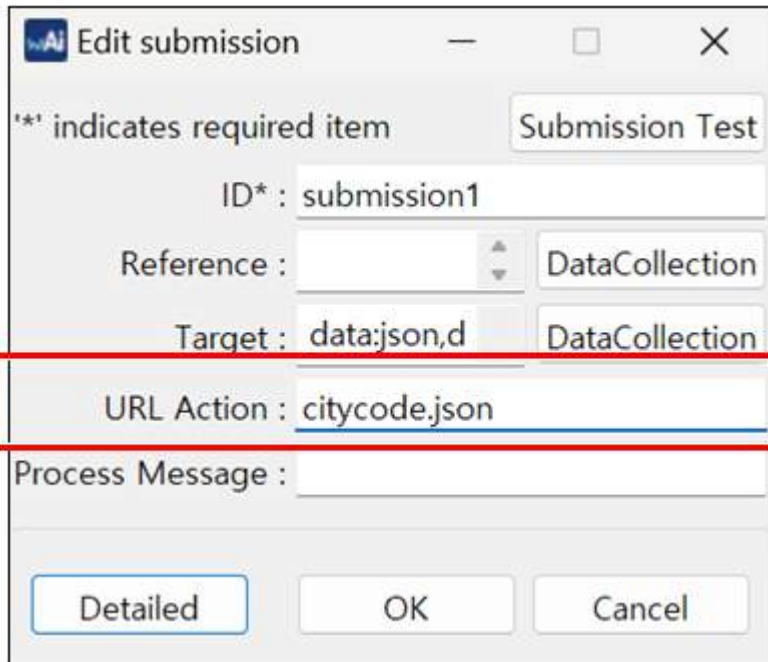


Diagram illustrating the 'Edit submission' dialog box. The dialog box contains the following fields and buttons:

- Header: Edit submission
- Legend: '**' indicates required item
- Submission Test button
- ID*: submission1
- Reference: (empty field) DataCollection button
- Target: data:json,d DataCollection button
- URL Action: citycode.json (highlighted with a red box)
- Process Message: (empty field)
- Buttons: Detailed, OK, Cancel

4. `$p.executeSubmission()` 함수를 사용하여 아래와 같이 서브미션을 실행합니다.

그림 14-7 서브미션 실행

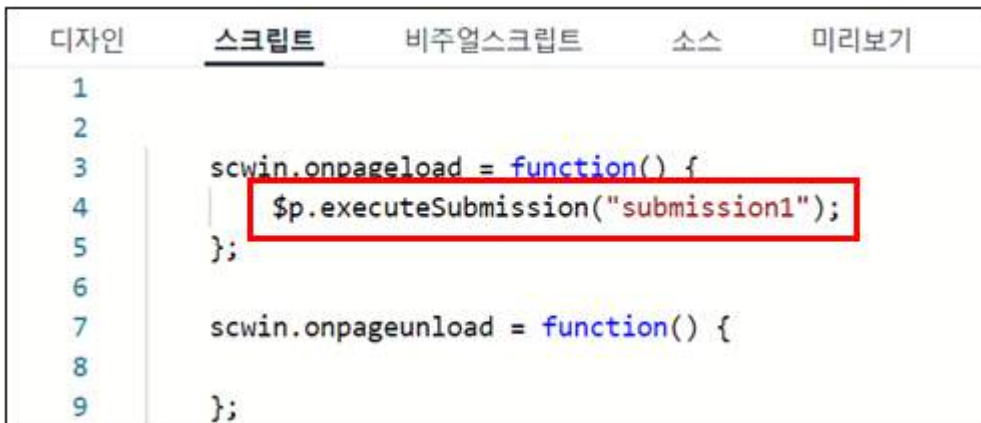


Diagram illustrating the script editor showing the execution of the submission. The script is as follows:

```

1
2
3  scwin.onpageload = function() {
4      $p.executeSubmission("submission1");
5  };
6
7  scwin.onpageunload = function() {
8
9  };

```

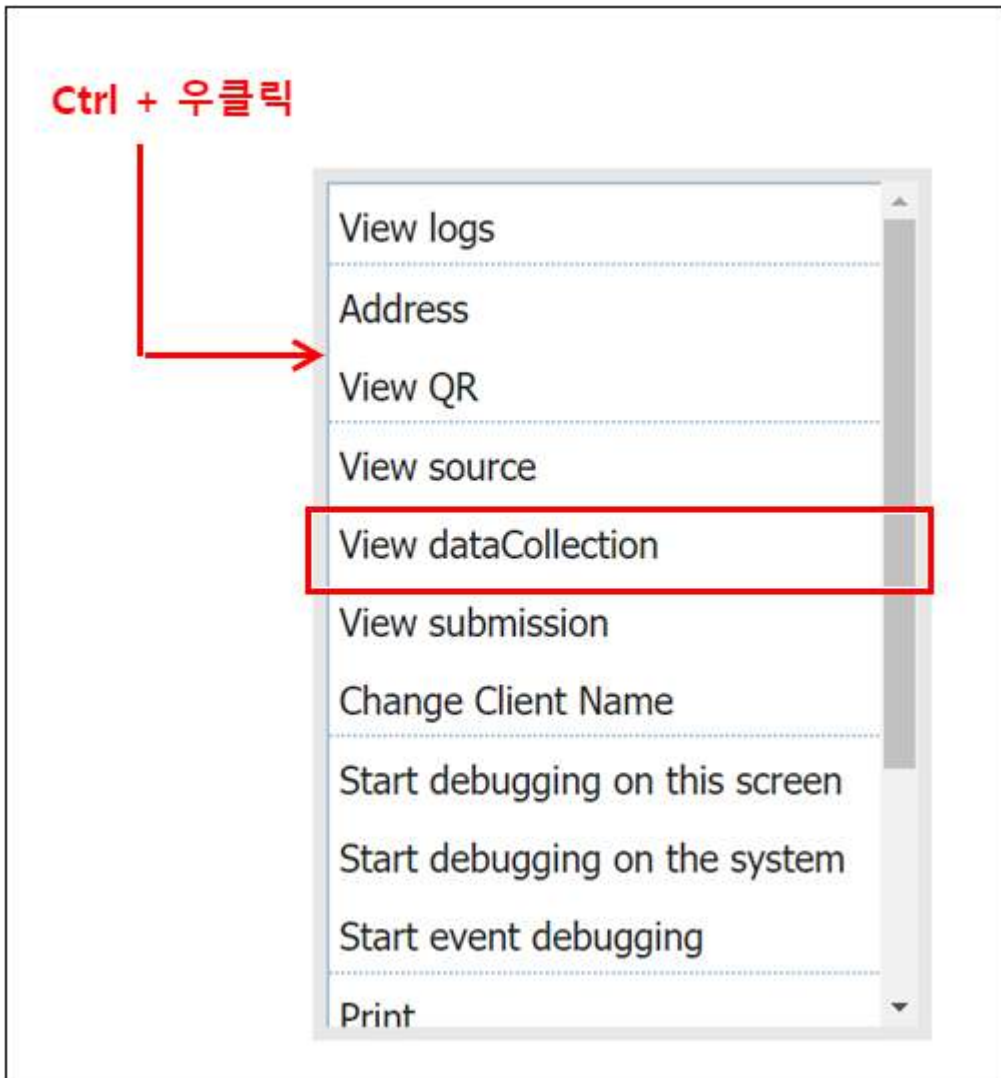
The line `$p.executeSubmission("submission1");` is highlighted with a red box.

Code 14-1 executeSubmission() 사용 예

```
1 $p.executeSubmission("submission1");
```

5. 화면 파일을 브라우저에서 실행합니다.
6. 브라우저의 화면 파일에서 Ctrl + 우클릭하여 디버깅 메뉴를 표시합니다.
7. **View dataCollection** 메뉴를 선택합니다.

그림 14-8 디버깅 메뉴



8. 서브미션이 정상 실행되어 DataList에 데이터가 로딩되었는지 확인합니다.

그림 14-9 서브미션 실행 결과 확인



14.4.2. 동적 생성

아래와 같이 API를 활용하여 동적으로 Submission을 생성할 수도 있습니다.

```
1 $p.createSubmission ( temOpt );
```

아래와 같이 오브젝트 형태로 Submission 객체를 생성하십시오. 속성은 Submission 생성창의 속성과 동일합니다.

```
1 var tmpOpt = {
2   id : "sub_getUserXmlMapData"
3   , ref : "data:xml,dc_reqCode"
4   , target : "data:xml,dc_resUserMapData"
5   , action : "/data/sampleData_xml_dc_map.data"
```

```

6      , mediatype : "text/plain"
7      , submitDoneHandler : function(e){
8          alert(e.responseText);
9      }
10 };
11
12 // Submission 생성
13 $p.createSubmission( tmpOpt );

```

Submission의 생성만으로는 데이터를 주고 받을 수 가 없습니다.

14.5 Submission 실행, 중단, 확인

14.5.1. Submission 실행

데이터를 주고받기 위해서는 생성된 Submission 을 실행시켜야 합니다.

`$p.executeSubmission` API를 활용하여 아래와 같이 Submission을 실행하십시오.

Code 14-2 사용 예

```

1 $p.executeSubmission ( submission id );

```

14.5.2. Submission 중단

`abortTrigger="true"` 속성 설정을 통해 Submission을 중단할 수 있는 취소 버튼을 화면에 출력할 수 있습니다. 사용자가 이 버튼을 클릭하면 현재 실행중인 Submission은 중단됩니다.

지정한 ID의 Submission이 현재 실행 중일 경우 해당 Submission을 중단할 수 있습니다.

Code 14-3 형식

```

1 WebSquare.ModelUtil.abort( "서브미션ID" );

```


14.5.3. Submission 확인

`$p.getSubmissionList()` 함수를 호출할 경우, 해당 함수가 호출된 화면에 포함된 submission 객체를 반환할 수 있습니다. 또한, 화면이 PageFrame을 포함할 경우, PageFrame의 src 화면에 포함된 submission 객체를 함께 반환할 수도 있습니다.

```

1 // main.xml 아래에 pageframe1.xml과 wframe2.xml이 있고
2 // 각 화면에 submission1, submission2가 정의된 경우
3
4 // pageframe1.xml에서 아래를 실행하면
5 $p.getSubmissionList();
6 // pageframe1.xml에 있는 submission1, submission2 객체가 array로 반환됨.
7
8 // main.xml에서 아래를 실행하면,
9 $p.getSubmissionList({"returnType": "json", {"all" : true}});
10
11 // 아래의 결과가 반환됨.
12 // {"mf":["submission1","submission2"], "mf_pageframe1":["submission1","submission2"],
"mf_wframe2":["submission1","submission2"]}
```

14.6 이벤트

Studio의 디자인 - 모듈 탭에서 Submission을 우클릭한 후, 이벤트를 선택하여 원하는 이벤트를 추가할 수 있습니다.

그림 14-10 Submission 이벤트

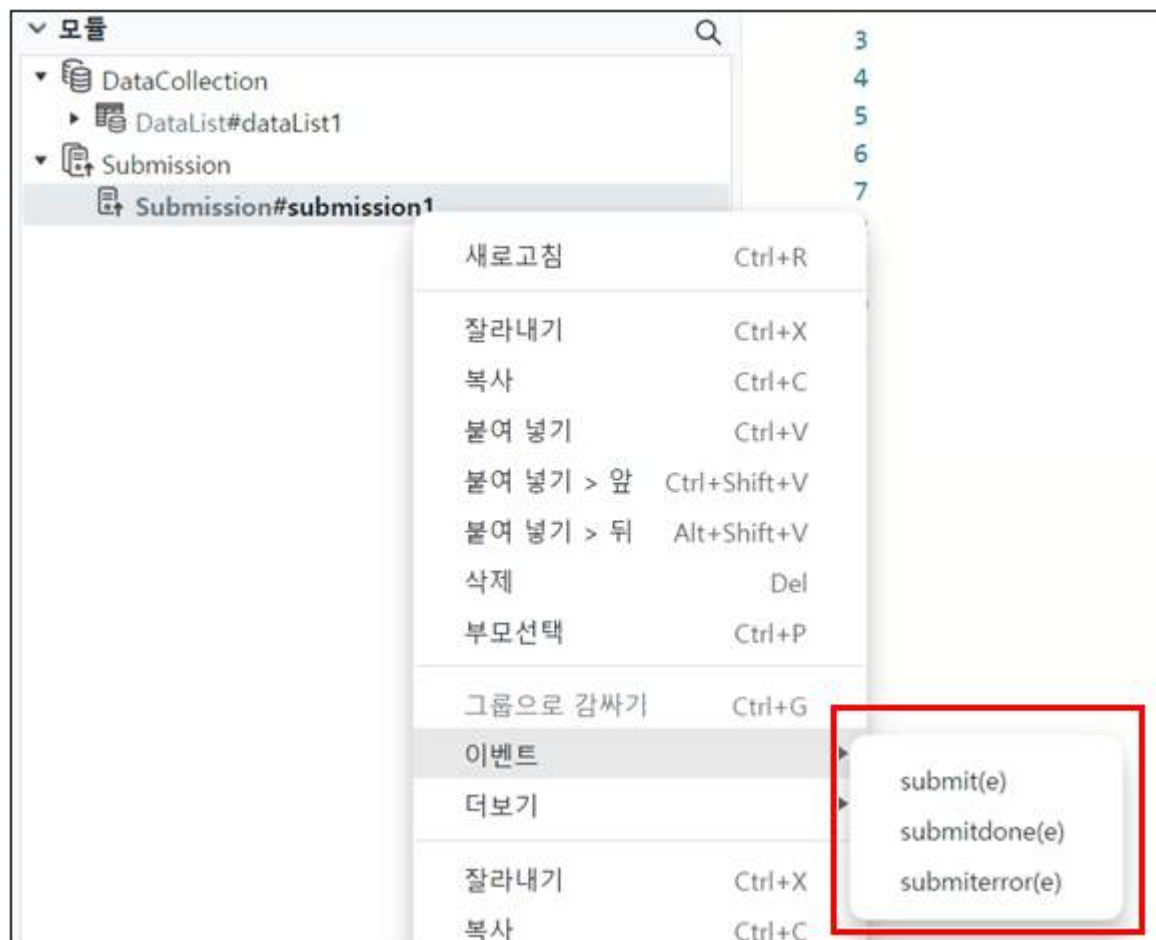


Table 14-5 Submission 이벤트

이벤트	설명
Submit	Submission 요청시 선택리 동작
Submit-done	Submission 동작
Submit-error	Submission 오류시 동작

14.6.1. submitdone(e)



xforms-submit-done : Response Status 코드 값이 정상 일 경우 동작.

Table 14-6 xforms-submit-done

e 객체 key	설명
e.id	Submission 객체의 ID 반환.
e.resourceUri	통신 URI 반환.

e 객체 key	설명
e.responseHeaders	• Response Headers 내용 반환.
e.responseStatusCode	• Response Status Code 반환.
e.responseReasonPhrase	• Response Status Text 반환.
e.responseBody	• Response Data를 XML 데이터로 parse 한 XML 객체 반환. • Response Content-Type이 JSON 인 경우 json객체를 XML로 파싱.
e.responseText	• Response Data 원본으로 String 형태 반환.
e.responseHeadersJSON	• Response Headers를 JSON으로 반환.
e.responseJSON	• responseText를 JSON으로 변환. • response header의 content-type에 "json" 문자열이 있는 경우에만 변환을 수행.

14.6.2. submiterror(e)



xforms-submit-error : Response Status 코드가 오류 (200 미만 300 이상) 일 경우 동작.

Table 14-7 xforms-submit-error

e 객체 key	설명
e.id	• Submission 객체의 ID 반환.
e.errorType	• target-error 로 고정값 반환.
e.resourceUri	• 통신 URI 반환.
e.responseHeaders	• Response Headers 내용 반환.
e.responseStatusCode	• Response Status Code 반환.
e.responseReasonPhrase	• Response Status Text 반환.
e.requestBody	• Request Data로 String 형태 반환.
e.responseText	• Response Data 원본으로 String 형태 반환.
e.responseHeadersJSON	• Response Headers를 JSON으로 반환.

14.7 기타

14.7.1. request header 설정

`setRequestHeader()` 함수를 사용하여 각 서브미션에 대해 request header를 설정할 수 있습니다.

Code 14-4 사용 예

```
1 var subObj = $p.getSubmission("submission01");  
2 subObj.setRequestHeader( {"userKey": "userValue"} );
```

14.7.2. 데이터가 없는 경우 전송 데이터에서 제외

`dataType= "bigDecimal"` 혹은 `"number"` 컬럼의 데이터가 없는 경우 전송 데이터에서 제외할 수 있습니다. `config.xml` 파일에서 `<exceptEmptyValue>` 항목을 아래와 같이 설정하십시오.

Code 14-5 사용 예

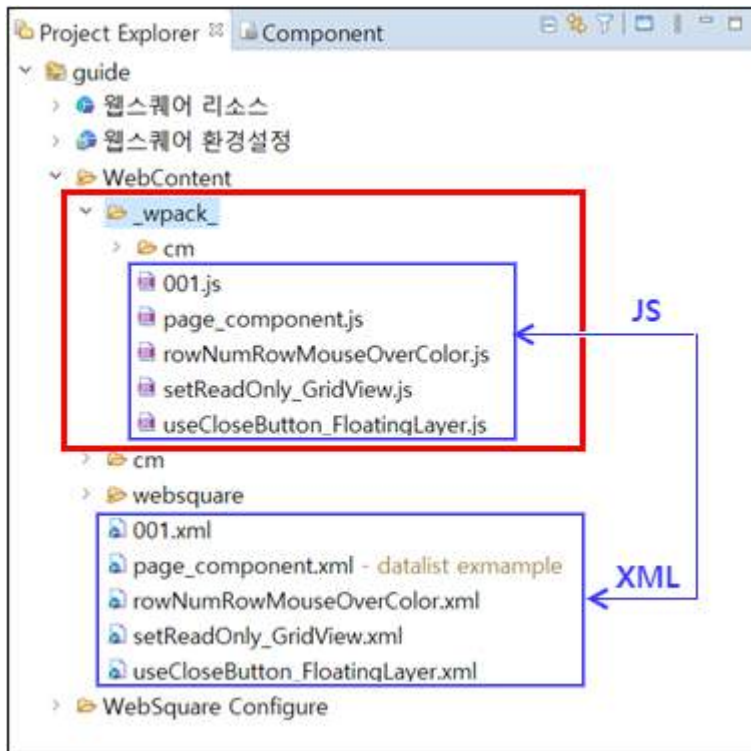
```
1 <WebSquare>  
2   <submission>  
3     <exceptEmptyValue value="true" />  
4   </submission>  
5 </WebSquare>
```

15. JS로 전환

15.1 JS 파일 생성 폴더

XML 화면 파일은 JS 파일로 변환되어 `/WebContent/_wpack_/` 폴더 아래에 저장됩니다. 화면 파일 개발에 사용한 공통 리소스도 JS 파일로 변환되어 `/WebContent/_wpack_/cm/` 폴더 아래에 저장됩니다.

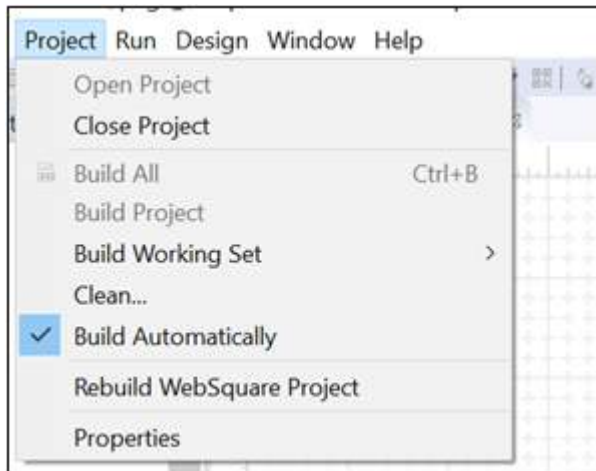
그림 15-1 W-Pack 폴더



15.2 스튜디오에서 변환하기

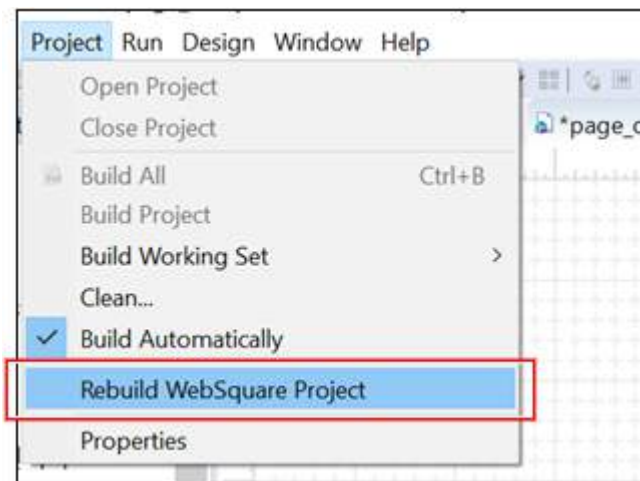
Project - Build Automatically 옵션을 설정하면 XML로 생성되는 화면 파일을 JS로 자동 변환할 수 있습니다.

그림 15-2 Project - Build Automatically



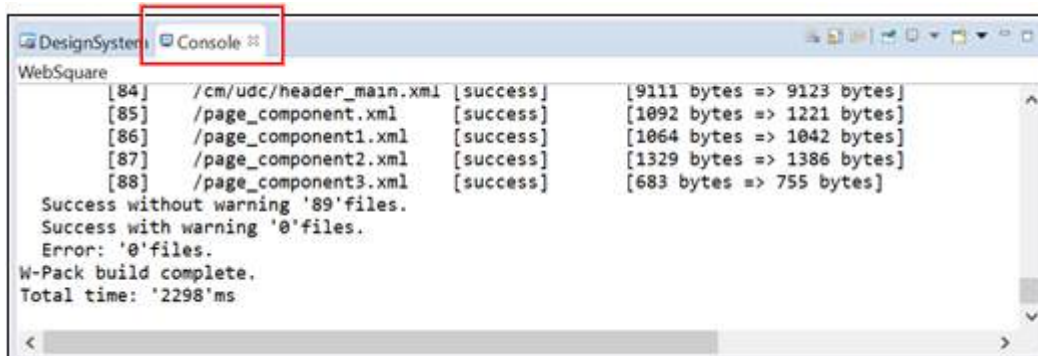
웹스퀘어 프로젝트를 다시 JS 파일로 빌드하려면 **Project - Rebuild WebSquare Project** 메뉴를 사용하십시오.

그림 15-3 Project - Rebuild WebSquare Project



W-Pack의 로그는 **Console** 탭에서 확인할 수 있습니다.

그림 15-4 Console 탭



15.3 Stand-alone 모듈 사용하기

W-Pack은 CI 서버 연동 및 서버에서의 일괄 변환을 지원하기 위해 Stand-alone 모듈을 지원합니다. W-Pack Stand-alone 모듈은 스튜디오 내장된 W-Pack과 동일하게 동작하며 Mac, Linux, Window (32bit, 64bit) 운영 체제를 지원합니다.

W-Pack Stand-alone 모듈은 아래 순서로 실행하십시오.

1. cmd 나 terminal과 같은 CLI 도구에서 W-pack을 실행합니다.

Code 15-1 Windows에서 실행 (cmd를 실행 후 모듈이 있는 위치로 이동)

```
1 filename.exe --src path --dest path --baseDir path(...생략)
```

Code 15-2 Mac 혹은 Linux에서 실행 (terminal을 실행 후 모듈이 있는 위치로 이동)

```
1 filename --src path --dest path --baseDir path(...생략)
```

2. 활성화된 명령창에 아래를 참조하여 명령어를 작성합니다.

Table 15-1 명령어 종류

명령어	인자	필수 여부	기본 값	설명
<code>--baseDir</code> <code>-b</code>	절대 경로	O	없음	프로젝트 폴더까지의 디렉토리 경로. posix, Windows 방식 모두 지원.
<code>--src</code> <code>-s</code>	절대 경로	O	없음	빌드할 XML 파일의 디렉토리 혹은 파일 경로. posix, Windows 방식 모두 지원.
<code>--dest</code> <code>-d</code>	절대 경로	O	없음	빌드 후 파일이 생성될 디렉토리 혹은 파일 경로. posix, Windows 방식 모두 지원.
<code>--js</code> <code>-j</code>	0~2	(선택)	1	0 : JS 코드 minify하지 않음. 1 : JS 코드 주석과 공백을 제거. 2 : JS 코드 주석과 공백을 제거하고, 로컬 변수를 치환.
<code>--css</code> <code>-c</code>	0~1	(선택)	1	0 : CSS 코드를 minify 하지 않음. 1 : CSS 코드의 주석과 공백을 제거하고, RGB → Hex 변환 수행.
<code>--ignore</code> <code>-i</code>	절대 경로	(선택)	없음	빌드 시 제외할 디렉토리 경로. posix, Windows 방식 모두 지원.
<code>--noDebug</code> <code>-nd</code>	true, false	(선택)	true	- true : debugger 코드 제거. - false : debugger 코드 제거하지 않음.

15.3.1. 디렉토리를 빌드할 경우

다음은 C:\Program Files\project\ 웹스퀘어 프로젝트 경로 전체를 JS로 빌드하는 경우의 예시입니다.

Code 15-3 example.sh

```
1 ... 중략 ...
2 --baseDir C:\Program Files\project\ --src C:\Program Files\project\ --dest C:\Program
  Files\project\_wpack\_ --js 1 --css 1 --ignore C:\Program Files\project\engine\ --noDebug false
3 ... 중략 ...
```

위의 예제에 따른 JS 변환 설정은 아래와 같습니다.

Table 15-2 JS 변환 설정

절대 경로 --baseDir	C:\Program Files\project\
변환 대상 --src	C:\Program Files\project\ 하위에 있는 모든 웹스퀘어 XML 파일
빌드된 파일들을 생성할 위치 --dest	C:\Program Files\project_wpack_
Minify 조건 --js 1 --css 1	• JS : 1 레벨로 Minify 수행. • CSS: 1 레벨로 Minify 수행.
제외 대상 --ignore	project 폴더 하위의 engine 폴더를 변환 대상에서 제외.
debugger 코드 제거 여부 --noDebug false	Minify 작업 중에 debugger 코드는 삭제하지 않음.

15.3.2. 여러 파일을 빌드할 경우

다음은 파일 여러 개를 JS로 빌드하는 경우의 예시입니다.

Code 15-4 example1.sh

```
1 ... 중략 ...
2 -b C:\Program Files\project\ -s C:\Program Files\project\index.xml,c:\Program
  Files\project\sub.xml -d C:\Program Files\project\_wpack\_ ,C:\program Files\project\_wpack\_ -j
2
3 ... 중략 ...
```

위의 예제에 따른 JS 변환 설정은 아래와 같습니다.

Table 15-3 JS 변환 설정

절대 경로 -b	C:\Program Files\project\
변환 대상 -s	C:\Program Files\project\ 하위의 index.xml 및 sub.xml 파일만 변환.

빌드된 파일들을 생성할 위치 - d	C:\Program Files\project\wpack\w
Minify 조건 -j 2	• CSS는 명령어를 입력하지 않았으므로 설정 기본값인 1레벨로 Minify 수행. • JS는 2레벨로 Minify 수행.
제외 대상	ignore 명령어를 입력하지 않았으므로 제외할 대상은 없음.
debugger 코드 제거 여부	noDebug 명령어를 입력하지 않았으므로 설정 기본값인 true 를 적용하여 debugger 코드를 제거.



제약 사항

- (1) 타겟이 다중 파일인 경우, (a) 파일 단위 경로는 공백 없는 ,(coma)로 구분하고 (b) **src** 및 **dest**의 인자는 반드시 파일 단위 경로를 모두 입력.
- (2) **baseDir**, **src**, **ignore** 경로가 실재 존재하지 않을 경우, 비정상적 수행을 막기 위해 동작이 중단됨.
- (3) 명령어와 인자는 반드시 하나의 공백으로 구분하여 입력.

그림 15-5 Jenkins Job 실행 시 작성 한 example.sh 파일을 Execute shell 해당 경로를 작성하는 예제



16. 디버깅

16.1 로그

16.1.1. 기본 로그

아래와 같이 `$p.log` API를 이용하여 기본 로그를 출력할 수 있습니다.

Code 16-1 로그 출력 예

```
1 $p.log( ":::: WebSquare :::" );
```

그림 16-1 기본 로그 출력

```
16:22:58.547 00002 000548 websquare] :::: WebSquare :::: javascriptPluginAll.wq:10
```

`$p.log()` 함수를 이용한 로그는 다음과 같은 형태로 생성됩니다.

Code 16-2 로그 형태

```
1 시간 이전로그출력후경과시간 첫로그출력후경과시간 로그출력기] 로그메시지
```

Code 16-3 로그 예시

```
1 06:03:13.838 00032 013248 websquare] convertTypeB
```

위의 로그는 아래와 같이 해석할 수 있습니다.

Table 16-1 로그 해석

항목	설명
06:03:13.838	로그생성시간
32ms	이전로그출력 후 경과시간
13.248s	첫로그출력 후 경과시간
websquare	로그출력기
convertTypeB	메시지

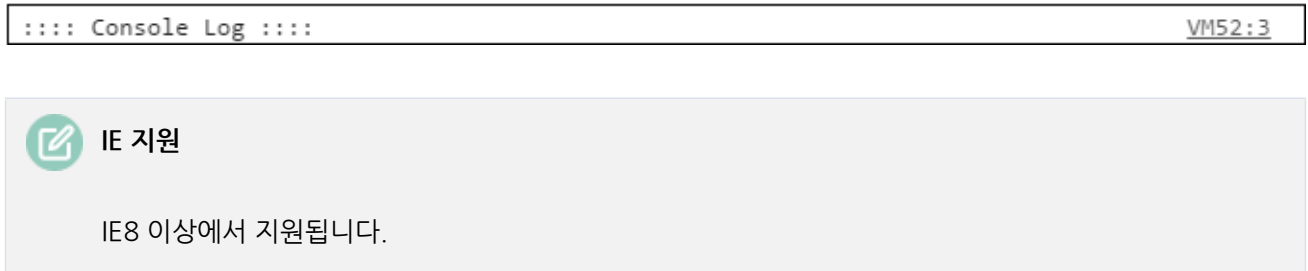
16.1.2. 브라우저 콘솔 로그

브라우저에서 console 객체를 지원해야 확인 가능합니다. 아래와 같이 `console.log` 메소드를 이용하십시오.

Code 16-4 로그 확인

```
1 console.log( ":::: Console Log :::" );
```

그림 16-2 브라우저 콘솔 로그

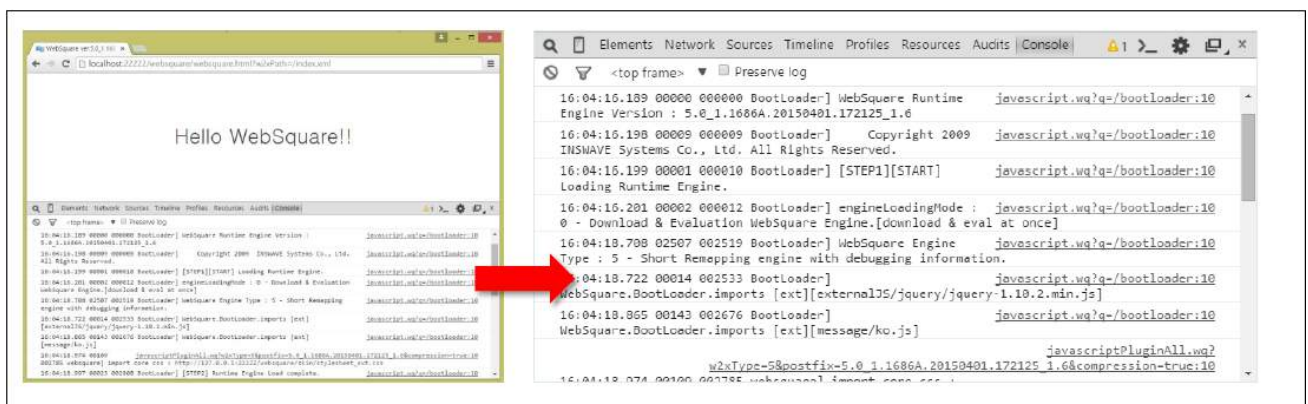


16.1.3. 브라우저에서 로그 확인 (Chrome & IE 기준)

브라우저 실행 후 [F12]를 누르거나 브라우저 메뉴를 통해 개발자 도구를 실행할 수 있습니다. 아래의 방법으로 브라우저에서 로그를 확인할 수 있습니다.

1. 아래 경로에 있는 `config.xml` 파일을 엽니다.
 - 파일 경로: `/WebContent/websquare/config.xml`
2. 아래와 같이 설정하십시오.
 - `<debug value="true" remoteConsole="false" console="true"/>`
3. 브라우저에서 F12 단축키를 누릅니다.
4. 화면을 실행한 후 Console 탭을 선택합니다.

그림 16-3 로그 확인



16.1.4. 확인할 로그 타입 지정

클라이언트 환경 설정을 통해 개발자 도구 Console 창에서 출력할 로그 타입을 지정할 수 있습니다. 스튜디오에서 client.config.xml 파일을 아래와 같이 설정하십시오.



Table 16-2 로그 타입 지정

정상 로그와 오류 로그를 모두 출력	• debug : true • console : true
오류 로그만 출력	• debug : true • errorConsole : true

16.2 웹스퀘어 AI 제공 기능

16.2.1. 디버깅용 컨텍스트 메뉴

웹스퀘어 AI로 작성한 화면의 스크립트는 웹스퀘어 AI 엔진 위에서 실행됩니다.

화면 로그는 브라우저 경고 창으로 표현되지 않고 별도로 관리됩니다.

아래의 방법으로 로그 메시지를 확인하십시오.

1. 웹스퀘어 AI로 작성한 화면을 브라우저에 실행합니다.
2. 브라우저 상에서 [Ctrl+마우스 우클릭]합니다.
3. [로그보기]를 선택합니다.

그림 16-4 [Ctrl + 마우스 우클릭]

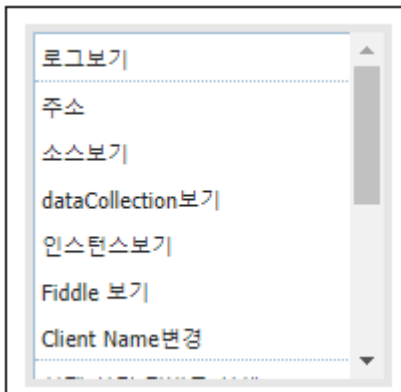


Table 16-3 주요 메뉴 설명

메뉴	설명
로그 보기	Script 오류를 확인하거나 Script에서 출력한 로그를 확인.
dataCollection 보기	데이터 객체의 현재 값을 확인.
소스보기	WebSquare화면 소스를 확인
현재 화면 디버그 실행	Submission을 이용한 통신의 Input, Output, Header정보를 확인
전체 새로고침	이벤트가 발생한 위치와 관계 없이 최상위(top) 페이지를 SPA 방식으로 재로딩.
새로고침	Debug 메뉴 호출 이벤트가 발생한 위치의 페이지를 SPA 방식으로 재로딩. (URL 변경없이 body 및 script를 재로딩.)



브라우저 상의 URL 재로딩 및 IFrame의 재로딩은 브라우저에서 제공하는 기능을 이용하십시오.

16.2.2. 디버깅용 컨텍스트 메뉴 허용 및 차단

WebSquare.logger의 `enableContextMenu()` 및 `disableContextMenu()` 함수를 사용하여 디버깅용 컨텍스트 메뉴 사용을 허용하거나 차단할 수 있습니다.

Code 16-5 사용 예

```

1 // 디버깅용 컨텍스트 메뉴 허용
2 WebSquare.logger.disableContextMenu();
3
4
5 // 디버깅용 컨텍스트 메뉴 차단
6 WebSquare.logger.enableContextMenu();

```

위의 함수들은 `config.xml` 설정 파일의 `<debugMenu>` 항목의 설정 값에 상관 없이 사용할 있습니다. 즉, `<debugMenu`

`value="use"/>` 혹은 `<debugMenu value="block"/>`으로 설정된 경우 모두 사용 가능합니다.



예제 파일

인터넷에서 다운로드 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/WebSquare.logger/enableContextMenu_disableContextMenu_WebSquare_logger.xml)

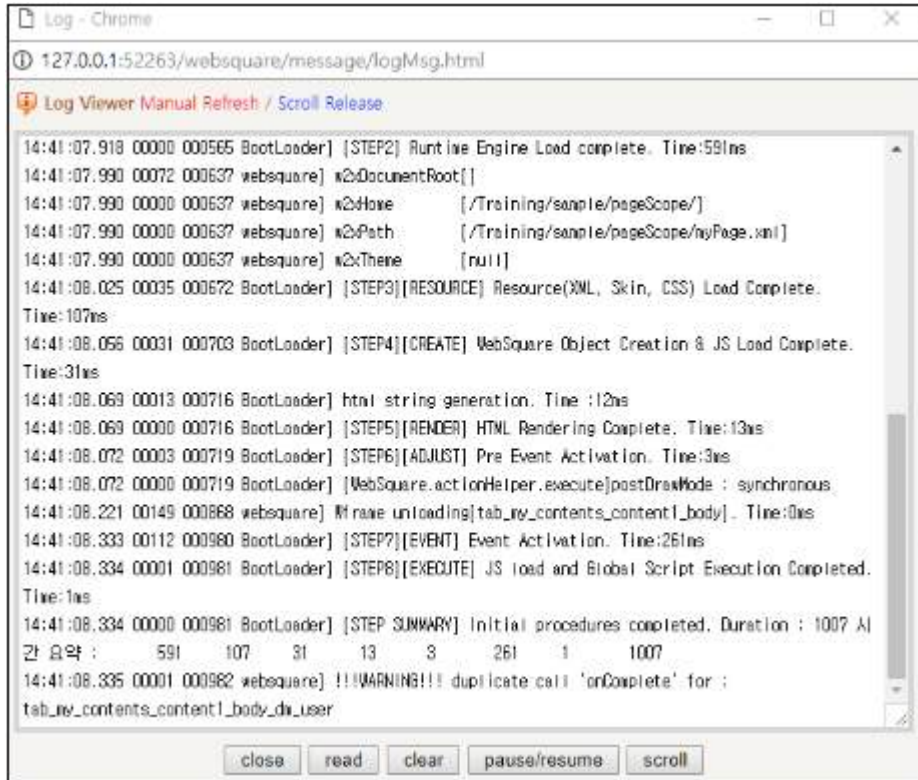
그림 16-5 YouTube 동영상 (<https://youtu.be/nGZQwuYFjkg>)



16.2.3. 각 디버깅 메뉴 설명

16.2.3.1. 로그 보기

그림 16-6 로그 보기



16.2.3.2. DataCollection 보기

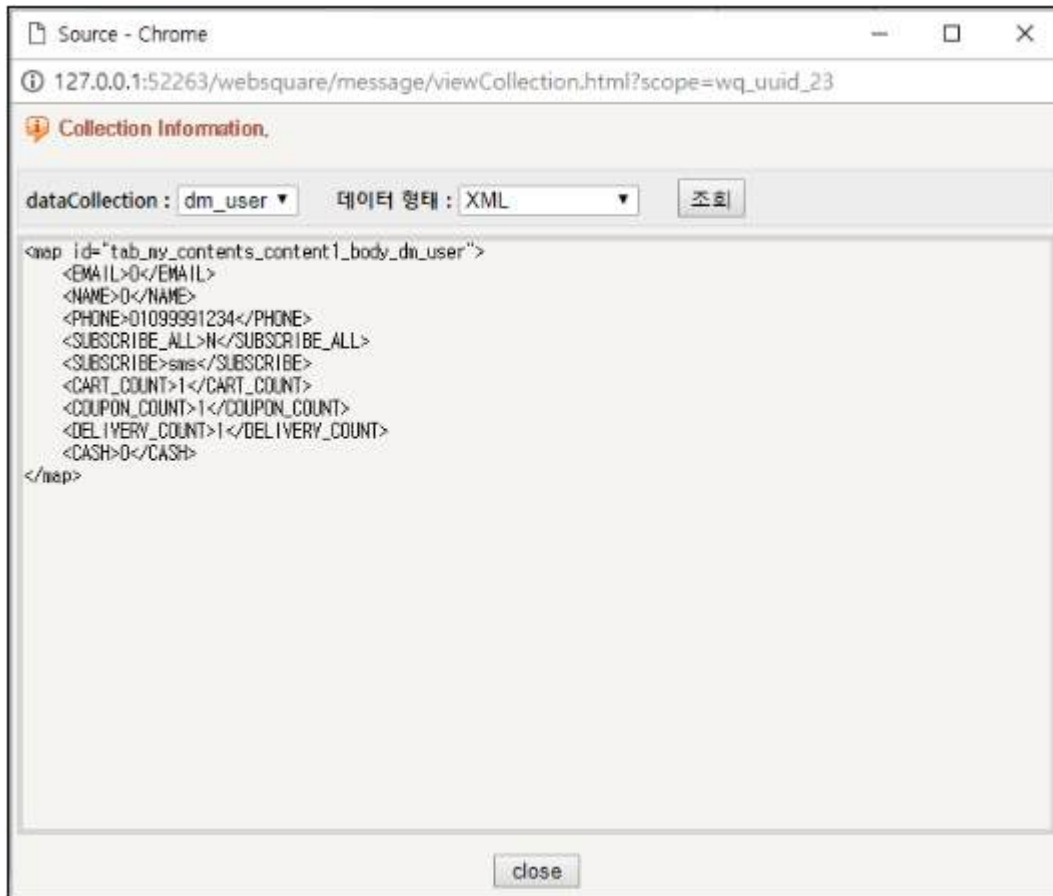
현재 컴포넌트와 바인딩된 DataCollection을 확인할 수 있습니다. 아래의 방법으로 실제 메모리에 할당된 데이터를 확인하십시오.

1. 웹스퀘어 시로 작성한 화면을 브라우저에 실행합니다.
2. 브라우저 상에서 [Ctrl+마우스 우클릭]합니다.
3. [dataCollection 보기]를 선택합니다.

그림 16-7 Ctrl + 마우스 우클릭 시



그림 16-8 현재 바인딩된 DataCollection 확인

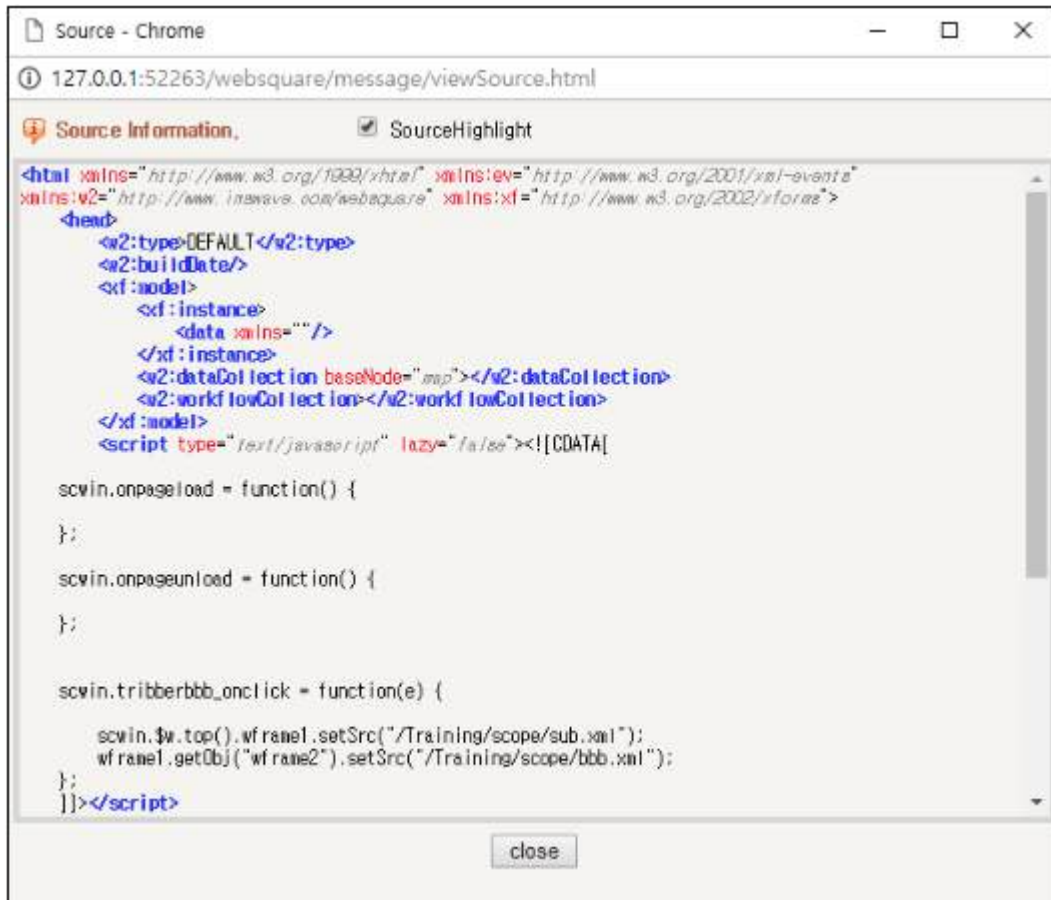


16.2.3.3. 페이지 소스 보기

1. 웹스퀘어 AI로 작성한 화면을 브라우저에 실행합니다.
2. 브라우저 상에서 [Ctrl+마우스 우클릭]합니다.
3. [소스보기]를 선택합니다.



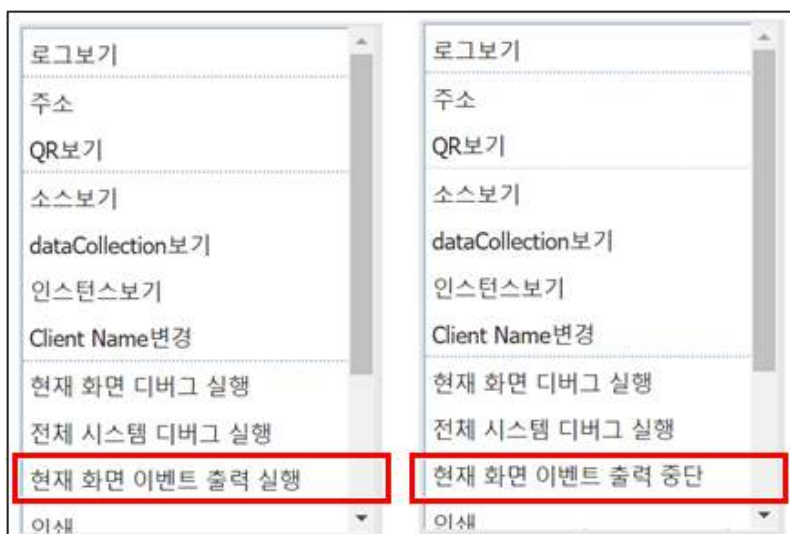
그림 16-9 소스 보기



16.2.3.4. 현재 화면 이벤트 출력 실행/중단

디버깅 메뉴 중 "현재 화면 이벤트 출력 실행" 및 "현재 화면 이벤트 출력 중단" 메뉴를 사용하면, 현재 화면에서 발생하는 이벤트 관련 로그들을 생성하거나 로그 생성을 중단할 수 있습니다. 이벤트와 바인딩된 컴포넌트, 스코프, 이벤트 이름, 이벤트 함수를 확인할 수 있습니다.

그림 16-10 디버깅 메뉴



Code 16-6 로그 모양

```
1 [EVENT LOG] id: [trigger3] scope id: [] ev: [onclick] : ev name: [scwin.trigger3_onclick]
```



예제 파일

[인터넷에서 다운로드](#) 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/_Debugging/Event_Debugging.xml)

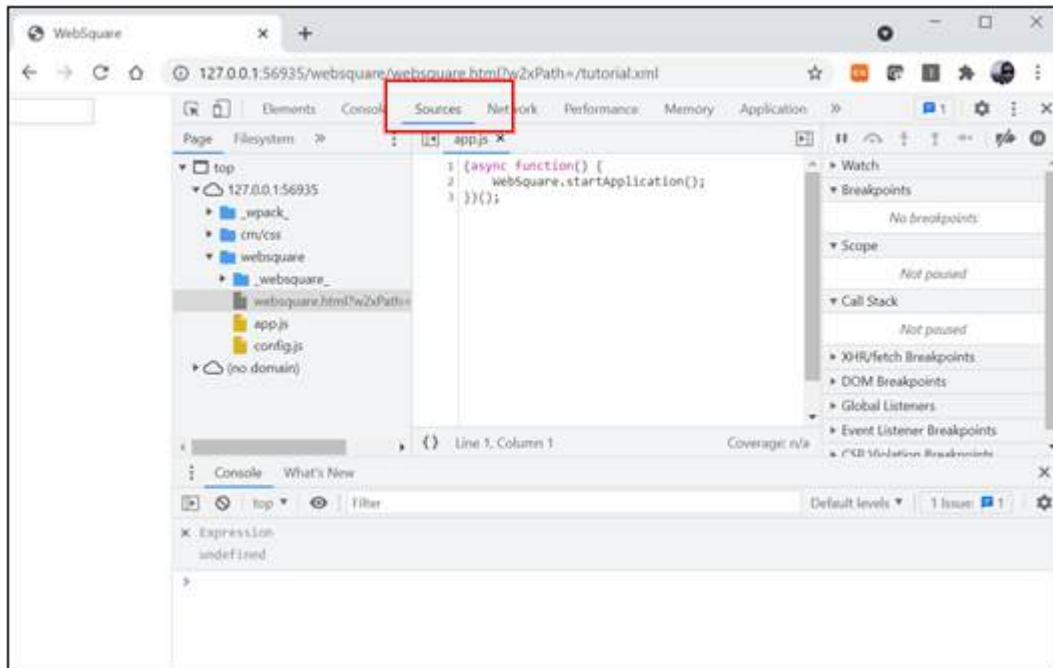
그림 16-11 [YouTube 동영상](https://youtu.be/l8O6aKN-NwQ) (https://youtu.be/l8O6aKN-NwQ)



16.3 브라우저 개발자 도구를 활용한 디버깅 (Chrome 기준)

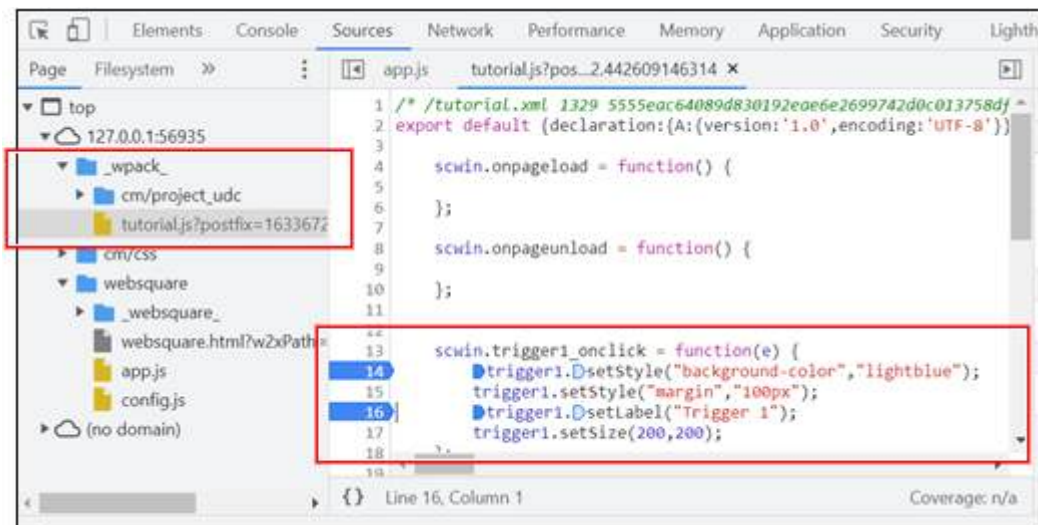
다음은 Chrome 브라우저에서 F12를 눌러 개발자 도구를 실행한 모습입니다. **Source** 탭에서 관련 파일을 확인할 수 있습니다.

그림 16-12 Chrome 개발자 도구



웹스퀘어 AI로 개발한 화면의 JS 파일은 _wpack_ 폴더 아래에 위치합니다. 화면에 해당하는 JS 파일을 선택한 후, 브레이크 포인트를 설정하여 개발자 도구에서 직접 디버깅을 수행할 수 있습니다.

그림 16-13 개발자 도구에서 디버깅 수행



Part IV. 컴포넌트

17. Forms

17.1 Anchor

Table 17-1 컴포넌트 설명

요약	HTML의 <code><a></code> 태그와 같이 하이퍼링크 역할을 수행
기능 및 특징	스타일 및 클래스 변경을 통해 다양한 형태 지원 가능
주요 용도	- 클릭 이벤트 제공 - 웹 접근성 지원 (<code>outerDiv</code>를 <code>false</code>로 지정한 후 <code><a></code> 태그만 표현)

17.2 AutoComplete

그림 17-1 AutoComplete



Table 17-2 컴포넌트 설명

요약	- SelectBox 기능을 확장한 컴포넌트 - 입력된 텍스트에 따라 자동 완성 기능을 제공
기능 및 특징	- 대소문자 구분 (<code>caseSensitive</code>) - 입력 텍스트 강조 (<code>useKeywordHighlight</code>) - 키워드 포함 검색 및 키워드 일치 검색 모두 지원 (<code>search</code>)
주요 용도	검색용 키워드 입력 상자
YouTube 동영상	AutoComplete 동영상 모음 https://www.youtube.com/playlist?list=PL7a9HhkvOVb001wZO87T-kbHtU3Ikbn AutoComplete 쇼츠 동영상 모음

- https://youtube.com/playlist?list=PL7a9HhkvOVb0wJ0b_Y7e4BMrzcOGT-ftI

Table 17-3 주요 API

<code>interactionMode</code>	• 주요 포털의 검색 기능처럼 검색어 리스트를 자동완성 기능을 통해 출력할 때 사용 • 사용자 입력에 따라 서버로부터 데이터를 동적으로 받아옴.
------------------------------	---

그림 17-2 `caseSensitive="true"`그림 17-3 `search = "contain" caseSensitive = "true"`

17.3 Calendar

그림 17-4 Calendar



Table 17-4 컴포넌트 설명

요약	• 달력 이미지를 출력하여 날짜를 선택할 수 있는 인터페이스 • 기본적으로 InputCalendar와 동일. (단, 직접적인 날짜 입력은 불가능.)
----	---

기능 및 특징	• 날짜 형식 지정 (<code>calendarValueType</code>) • 최소/최대 연도 설정 (<code>maxYear</code>, <code>minYear</code>) • 시작 요일 설정 (<code>weekStartsOn</code>) • 스킨 디자인 선택 가능 (<code>calendarClass</code>) • 연도/월 표시 순서 지정 (<code>useMonthYearFormat</code>)
주요 용도	날짜 선택 수단
YouTube Video	• Calendar 동영상 모음 ◦ https://www.youtube.com/playlist?list=PL7a9HhkvOVb2l8SJmid0SkcQH6gvSKEuq • Calendar 쇼츠 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb0ZVV_GIRJpp_z36eH8hmz8&si=8UZOVJoHuE0wdJXV

Table 17-5 주요 API

API	설명
<code>calendarClass</code>	• 총 5가지 디자인 제공.
<code>calendarValueType</code>	• Today 버튼 옆 날짜 표시 방법
<code>maxYear</code> / <code>minYear</code>	• Calendar 컴포넌트에 표현할 수 있는 최대/최소 연도를 설정
<code>weekStartsOn</code>	• 한주의 시작일을 설정 (0: 일요일, 1: 월요일 ...)
<code>topLayout</code>	• CSS나 jQuery 조작 없이 Calendar 컴포넌트의 Today 버튼을 상단으로 이동
<code>disableBeforeToday</code> & <code>disableAfterToday</code>	• 오늘 날짜를 기준으로 날짜 선택을 제한 • Video 보기

17.4 CheckBox

그림 17-5 CheckBox

☐ 딸기
☐ 바나나
☐ 자몽
☐ 천혜향

Table 17-6 컴포넌트 설명

요약	• 선택 대상 항목을 선택 박스와 함께 출력 • HTML에서 제공되는 Input 객체의 <code>type="checkbox"</code>와 유사한 인터페이스를 제공
----	---

	• Radio와 달리 여러 항목을 동시에 선택 가능
주요 용도	• 다중 선택이 가능한 메뉴
YouTube 동영상	• CheckBox 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb0tRjIUsmhMZz00xfAHg4-R

17.5 CheckComboBox

Table 17-7 컴포넌트 설명

요약	• CheckBox와 SelectBox를 혼합한 형태의 컴포넌트 • 입력된 텍스트에 따라 자동 완성 기능을 제공
기능 및 특징	• 다중 선택 가능 • 선택 박스와 함께 드롭다운 리스트 형태의 메뉴를 제공.
YouTube 동영상	• CheckComboBox 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb00FkvHROGneUGph492q6wo • CheckComboBox 쇼츠 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb2ovkNxgMERFSwoudvTF0PA

17.6 DatePicker

그림 17-6 DatePicker



Table 17-8 컴포넌트 설명

요약	모바일 전용 달력 컴포넌트.
기능 및 특징	• 모바일 환경에서 Calendar 컴포넌트와 InputCalendar 컴포넌트 대체. • 컴포넌트의 최상위는 HTML <div>를 사용. • <code>calendarValueType</code> 속성을 이용하여 연월일시간, 연월일, 연월등을 선택할 수 있도록 지원.

17.7 Editor

그림 17-7 Editor

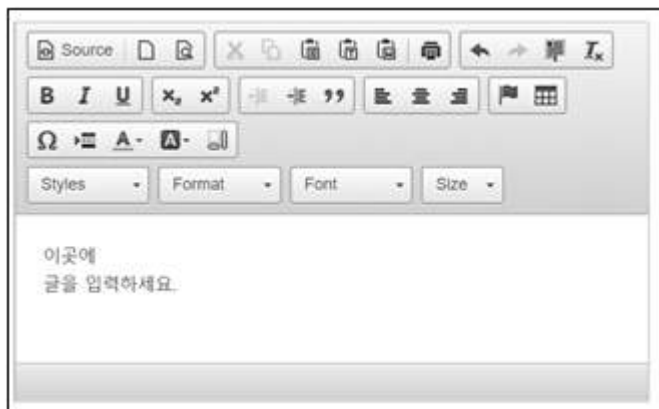


Table 17-9 컴포넌트 설명

요약	웹 에디터(CKeditor) 제공.
기능 및 특징	CKEditor를 생성하고 옵션을 통해서 다양한 기능을 지원.
YouTube 동영상	Editor 동영상 모음 https://www.youtube.com/playlist?list=PL7a9HhkvOVb0DgmObHSYewthL1Als2dBK Editor 쇼츠 동영상 모음 https://www.youtube.com/playlist?list=PL7a9HhkvOVb2pcuCI55zM-XYP-Pi9XDQy

17.8 FlipToggle

그림 17-8 FlipToggle



Table 17-10 컴포넌트 설명

요약	터치 드래깅을 통한 토글링 지원.
기능 및 특징	Label 지정 가능
주요 용도	모바일 장치용 토글링 인터페이스

17.9 Image

Table 17-11 컴포넌트 설명

요약	화면에 이미지를 출력
기능 및 특징	- 경로 설정을 통한 이미지 추가 (<code>src</code>) - 이미지 크기 및 스타일 지정 가능 (<code>style</code>)
주요 용도	화면 이미지 삽입

17.10 InputBox

그림 17-9 InputBox

The figure shows three input boxes stacked vertically. The first is labeled 'text' and contains the placeholder text '이름을 입력하세요.' (Enter name). The second is labeled 'number' and contains the value '1.299.68'. The third is labeled 'date' and contains the value '01/01/2020'.

Table 17-12 컴포넌트 설명

요약	- 텍스트 입력 상자 - 가장 기본적인 텍스트 입력 컴포넌트
기능 및 특징	- 입력 가능한 텍스트 제한 (<code>allowChar</code>) - 입력 포맷 지정 (<code>applyFormat</code>) - 입력 값 종류 지정 (<code>dataType</code>) - 출력 포맷 지정 (<code>displayFormat</code>) - 날짜 출력 포맷 선택 가능 (<code>dateMask</code>) - 대문자 변환 등 입력값 출력 함수 추가 (<code>displayFormatter</code>) - 복사&붙여넣기 방지 (<code>preventCopyPaste</code>)
주요 용도	간단한 텍스트 입력 상자
YouTube 동영상	InputBox 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb05RQ5h7SX85cVKxVeRJ6Ri InputBox 쇼츠 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb06wbWFP3qUNey8KNLqMi1L

17.11 InputCalendar

그림 17-10 InputCalendar



Table 17-13 컴포넌트 설명

요약	- 날짜를 직접 입력하거나 달력 이미지에서 날짜를 선택할 수 있는 인터페이스를 제공 - Calendar와 동일한 기능을 제공 (단, 직접 날짜 입력 가능)
기능 및 특징	- 날짜 입력 - 연도/월 표시 순서 지정 (<code>useMonthYearFormat</code>)
YouTube 동영상	InputCalendar 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb3NribEDGchbQCftN8QYptf InputCalendar 쇼츠 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb2ZPgbquW-o3GXN5IID755t

Table 17-14 주요 API

API	설명
<code>displayFormat</code>	입력한 날짜에 대한 표시방법을 제어
<code>dateValidCheck</code>	<code>dateValidCheck="true"</code> 속성 설정을 통해 입력된 날짜 값의 유효성을 확인하고 유효하지 않은 값이 입력된 경우 경우 이전 값으로 복귀
<code>topLayout</code>	CSS나 jQuery 조작 없이 Calendar 컴포넌트의 Today 버튼을 상단으로 이동
<code>disableBeforeToday</code> & <code>disableAfterToday</code>	- 오늘 날짜를 기준으로 날짜 선택을 제한 동영상 보기

날짜값 종류(`calendarValueType`) 별로 날짜 표시방식(`displayFormat`) 속성을 설정하는 대신 아래와 같이 *config.xml* 파일에서 일괄 설정할 수 있습니다.

Code 17-1 config.xml 설정

```

1 <displayFormat valueType="yearMonth" defaultValue="yyyy::MM"/>
2 <displayFormat valueType="yearMonthDate" defaultValue="yyyy/MM/dd"/>
3 <displayFormat valueType="yearMonthDateTime" defaultValue="yyyy/MM/dd HH:mm"/>
4 <displayFormat valueType="yearMonthDateTimeSec" defaultValue="yyyy/MM/dd HH:mm:ss"/>

```

17.12 MultiSelect

Table 17-15 컴포넌트 설명

요약	사용자의 다중 선택을 지원.
기능 및 특징	SelectBox 형태로 다중 선택을 지원하는 컴포넌트.
YouTube 동영상	MultiSelect 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb2JcEdNgxhbqNRq0yE42az1 MultiSelect 쇼츠 동영상 모음 https://www.youtube.com/playlist?list=PL7a9HhkvOVb3c-d-xME6bEvX7A3ozOOUn

17.13 MultiUpload

그림 17-11 Multiupload

#	파일	크기	상태

추가
삭제
취소
전송

Table 17-16 컴포넌트 설명

요약	파일 여러 개를 한번에 업로드할 수 있는 기능을 제공
기능 및 특징	- 파일 확장자를 통한 업로드 파일 종류 제한 (filter) - 최대 업로드 파일 개수 지정 (maxcount) - 최대 업로드 파일 크기 지정 (maxsize)
주요 용도	파일 첨부 메뉴
YouTube 동영상	MultiUpload 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb2GupMXHTIGuU2f-Z8I2oAJ

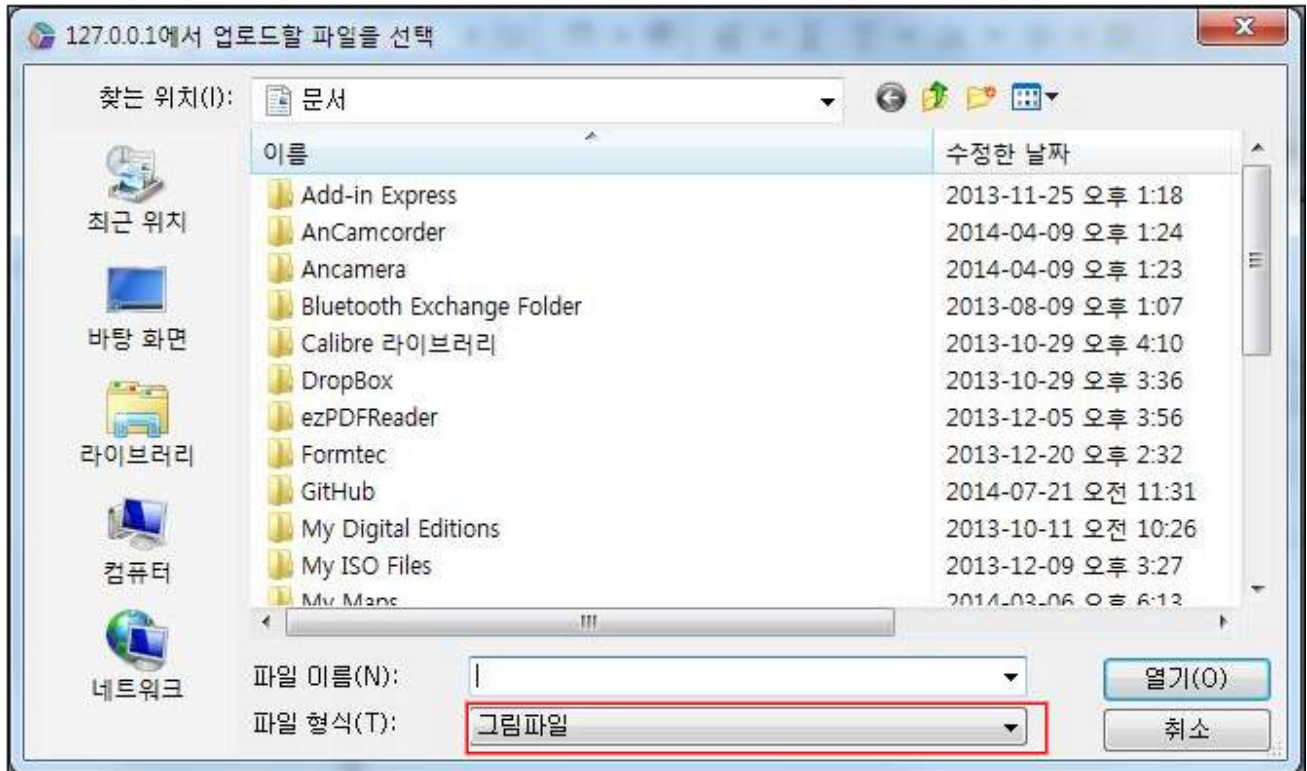
주요 속성

filter: 업로드 가능한 파일 확장자를 지정. (형식: [파일 형식 이름] : [확장자 명], 사용 예: `AutoCad:*.dxf;*`

.dws)

maxcount: 업로드 가능한 최대 파일 갯수**maxsize**: 업로드 가능한 최대 파일 크기 (단위: Kb)

그림 17-12 filter



17.14 Radio

그림 17-13 Radio

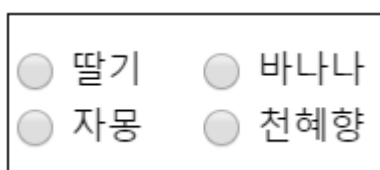


Table 17-17 컴포넌트 설명

요약	선택 대상 항목을 선택용 버튼과 함께 제공 (CheckBox와 달리 단일 항목만 선택 가능)
기능 및 특징	- 단일 선택만 허용 - 기본 선택 항목 지정
주요 용도	단일 선택만 허용되는 메뉴
	Radio 동영상 모음

YouTube 동영상

- <https://youtube.com/playlist?list=PL7a9HhkvOVb0iXbjE8n9EWoyRyLch5iU4>
- [Radio 쇼츠 동영상 모음](#)
 - <https://youtube.com/playlist?list=PL7a9HhkvOVb0RL0SdV-l29yJgVRhw9FR2>

17.15 ScheduleCalendar

그림 17-14 ScheduleCalendar



Table 17-18 컴포넌트 설명

요약	달력 형태로 일정을 표시하는 컴포넌트. 일정은 추가, 삭제, 편집이 가능.
YouTube 동영상	• ScheduleCalendar 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb3XyMUJI0jpJB70Xk0CP0rx • ScheduleCalendar 쇼츠 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb3pxTNX43cmOZYS667ZQlhv

17.16 SearchBox

그림 17-15 SearchBox



Table 17-19 컴포넌트 설명

요약	검색용 텍스트 입력 상자
기능 및 특징	기본적으로 InputBox와 동일한 기능을 제공하며, 추가적으로 전체 텍스트를 삭제할 수 있는 버튼을 함께 제공
주요 용도	검색용 텍스트 입력 상자
YouTube 동영상	SearchBox 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb3Rw2O2NAckQvPYA4-zc3nx

17.17 Secret

그림 17-16 Secret



Table 17-20 컴포넌트 설명

요약	텍스트 입력 상자
기능 및 특징	기본적으로 InputBox 컴포넌트와 동일한 기능을 제공하며 입력된 텍스트를 별도의 지정된 기호로 표시
주요 용도	비밀번호 입력 상자
YouTube 동영상	Secret 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb0zYq_ga5RFDAGk88szL8aM

17.18 SelectBox

그림 17-17 SelectBox



Table 17-21 컴포넌트 설명

요약	콤보 박스를 통해 선택 대상을 목록으로 표시
기능 및 특징	단일 선택만 허용
주요 용도	단일 선택만 허용되는 드롭다운 리스트 형태의 메뉴
YouTube 동영상	SelectBox 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb3iilZREIS7CzsjwtFW4S6h SelectBox 쇼츠 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb23uYA3N77-iGBUEyFDN3Q4

17.19 Slider

그림 17-18 Slider

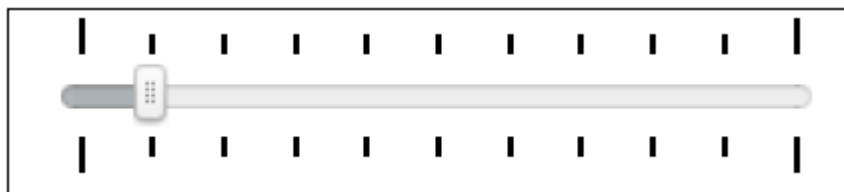


Table 17-22 컴포넌트 설명

요약	상하, 좌우 슬라이딩을 통해 값을 선택할 수 있는 인터페이스를 제공
기능 및 특징	슬라이딩 방향 지정 (vertical)

Table 17-23 주요 API

API	설명
<code>setMinValue(최소값);</code>	슬라이더 상의 최소값을 설정.
<code>setMaxValue(최대값);</code>	슬라이더 상의 최대값을 설정.

API	설명
<code>onthumbmove</code>	Slider 컴포넌트의 핸들을 이동할 때 발생하는 이벤트
<code>onmovedone</code>	Slider 컴포넌트의 핸들 이동 후 발생하는 이벤트

17.20 Span

Table 17-24 컴포넌트 설명

요약	<code>tagname</code> 속성이 <code>span</code> 인 TextBox 컴포넌트
기능 및 특징	Output과 동일한 기능을 제공.
주요 용도	웹 브라우저 전체 폭에 걸쳐 문자열을 출력

17.21 Spinner

그림 17-19 Spinner

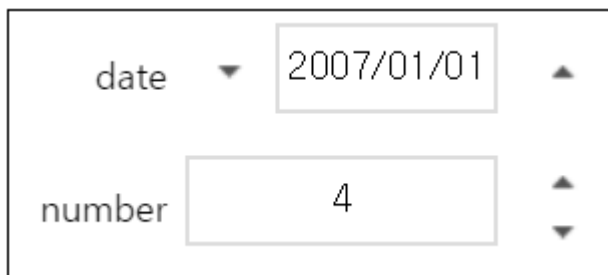


Table 17-25 컴포넌트 설명

요약	화살표로 값을 선택.
기능 및 특징	사용자가 화살표를 통해 값을 선택할 수 있는 기능을 제공.
YouTube 동영상	Spinner 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb38HNjcVUgLO85g_5DLX6qA Spinner 쇼츠 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb2n_A6zCWf-yvRGE4oUxNHC&si=XmCn_zAJhFwLnac

17.22 TextArea

그림 17-20 TextArea

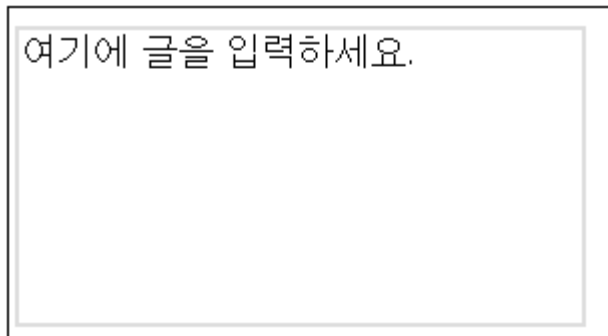


Table 17-26 컴포넌트 설명

요약	개행 텍스트의 입력/출력을 지원.
기능 및 특징	- HTML <textarea>와 유사한 인터페이스를 제공. - InputBox는 한 줄의 텍스트 입력을 허용하지만, TextArea는 여러 줄의 텍스트 입력이 가능. <textarea> 태그로 구성됨.
YouTube 동영상	TextArea 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb3QoNos0n8Xv3Axfc21i6Pa

17.23 TextBox

그림 17-21 TextBox

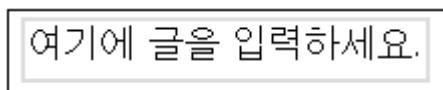


Table 17-27 컴포넌트 설명

요약	간단한 텍스트 출력을 위한 Output 컴포넌트의 경량화 버전.
기능 및 특징	- Output과 비교 속도의 이점이 있으나, Formatter, Validator, Align 등은 지원하지 않음. - HTML <div> 태그로 표현됨. - WebSquare에서 제공하지 않은 HTML 태그(li, dt, dd, h1, p 등)는 tagName 속성을 이용하여 표현 가능. - TextBox는 자식을 가질 수 없음. 자식을 포함하는 태그의 경우 Group 컴포넌트와 함께 사용해야 함.

YouTube 동영상	• TextBox 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb0GIVTlzmeQxgSiZCw27uXm • TextBox 쇼츠 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb0DidKNmCjRX_1BhRFNsTZf
-------------	---

17.24 Trigger

그림 17-22 Trigger



Table 17-28 컴포넌트 설명

요약	• 버튼 UI를 제공. • HTML <input> 태그의 type=button과 유사한 인터페이스를 제공.
기능 및 특징	• type 속성에 따라 HTML의 <input>, <table>, <a> 태그로 구성된 다양한 버튼을 생성할 수 있음. ◦ button : HTML <input> 태그의 type=button 사용. ◦ image : 최상위는 table 태그로 구성되고 left, center, right에 각각에 CSS 적용이 가능. ◦ anchor : 최상위는 anchor 태그로 구성되고, 버튼의 텍스트는 span 태그로 구성.
YouTube 동영상	• Trigger 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb2vYP1KWIKGoIBhTYeOrjQ3

17.25 Upload

그림 17-23 Upload



Table 17-29 컴포넌트 설명

요약	파일을 업로드
기능 및 특징	MultiUpload와 달리 파일 한 개만 업로드 가능
주요 용도	파일 1개만 첨부할 수 있는 메뉴
YouTube 동영상	• Upload 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb2Cko8W5beHHOi79JJGWSmz

18. Grid

18.1 GridView

Table 18-1 컴포넌트 설명

요약	테이블형 자료를 그리드로 표시
기능 및 특징	- 크게 Body와 Header로 구분 - 다양한 데이터 형식 지원 - 자유로운 그리드 편집 - 페이징, 정렬, 필터링, 그룹핑 지원 - 각 칼럼 별로 데이터 형식 지정 - 순차적으로 데이터를 표시하여 대량의 데이터 표시에 적합 & 데이터 출력 속도 개선 (무한 스크롤링) - Total, Subtotal, Footer 지원 - 클립보드를 지원하여 데이터 바인딩이 용이 (드래그-앤-드롭으로 바로 GridView와 바인딩 가능)
주요 용도	- 복잡한 테이블 구성 - 대량의 데이터를 처리하는 테이블 구성 - 대용량 파일 업로드/다운로드 (Excel, CVS, 한셀/넥셀)
	GridView 동영상 a모듬 전체 https://youtube.com/playlist?list=PL7a9HhkvOVb3rbVfPAsS3Pq_rvxmdM1Ru 일반 https://youtube.com/playlist?list=PL7a9HhkvOVb2_7IKQf54XwX4X3f4bqOAJ 드래깅 https://youtube.com/playlist?list=PL7a9HhkvOVb0-ysABiKKfhnxLU6ZIQTMR 이벤트 https://youtube.com/playlist?list=PL7a9HhkvOVb1VEHd2qwiF0oOR7IO5-J1P Excel https://youtube.com/playlist?list=PL7a9HhkvOVb1BDBKKQEIEcTBfyZ9VaKHh 필터링 https://youtube.com/playlist?list=PL7a9HhkvOVb2-tNuJab1i_nQHNE-pM4p3 gridView.footer https://youtube.com/playlist?list=PL7a9HhkvOVb2THHGg30Kn-U5AUb

YouTube 동영상

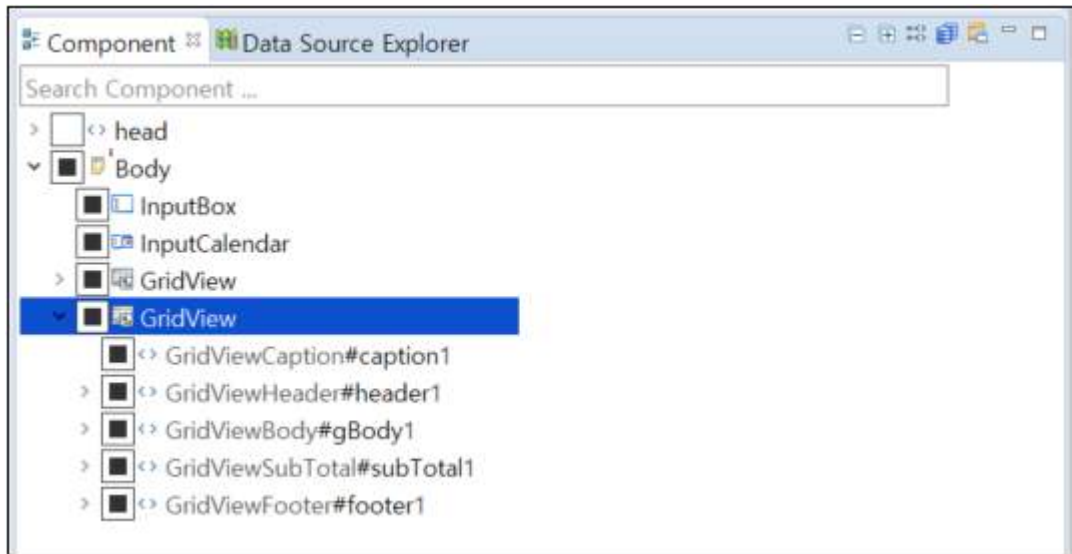
- Zwq1gk
- [gridView.column](#)
 - https://youtube.com/playlist?list=PL7a9HhkvOVb3_4QpWiErS_eYjxisFzbYc
- [그룹핑](#)
 - <https://youtube.com/playlist?list=PL7a9HhkvOVb1QjeJxZV8aqb8Bd1tHH4W->
- [inputType](#)
 - <https://youtube.com/playlist?list=PL7a9HhkvOVb0BNrcR6LP1WUuYSunz6hid>
- [병합](#)
 - <https://www.youtube.com/playlist?list=PL7a9HhkvOVb377prDaJ6x5Ftc83JGSmeW>
- [모바일](#)
 - <https://youtube.com/playlist?list=PL7a9HhkvOVb3cPXzBklG-tj2ZFUMcvslb>
- [정렬](#)
 - <https://youtube.com/playlist?list=PL7a9HhkvOVb1cQbkNF0aLMRjKTW0vFmT2>
- [스타일](#)
 - https://www.youtube.com/playlist?list=PL7a9HhkvOVb24GxSq-ydV6_ByyU7ROv22
- [gridView.subTotal](#)
 - <https://youtube.com/playlist?list=PL7a9HhkvOVb37YKuc4nJVEzSyHfY-3yQC>
- **GridView 쇼츠 동영상 모음**
 - [전체](#)
 - <https://youtube.com/playlist?list=PL7a9HhkvOVb213UWbVH8U1V2api8n2JjU>
 - [일반](#)
 - https://youtube.com/playlist?list=PL7a9HhkvOVb2Xq_ISOfr84D5Dx9BdoBY1
 - [Excel](#)
 - <https://www.youtube.com/playlist?list=PL7a9HhkvOVb1ZfQk1xBUXrMWtNCwymbhP>
 - [필터링](#)
 - <https://youtube.com/playlist?list=PL7a9HhkvOVb0h95oDiKX2t1yWCckWDmjf>
 - [그룹핑](#)
 - <https://youtube.com/playlist?list=PL7a9HhkvOVb2y2vhlov918FcHXvrXH3kk>
 - [inputType](#)
 - https://youtube.com/playlist?list=PL7a9HhkvOVb2zeARHkcB8A_qH7o35Tbny

- 병합
 - <https://www.youtube.com/playlist?list=PL7a9HhkvOVb2Roe4saQO-aRHfompiY2Su>
- 정렬
 - <https://youtube.com/playlist?list=PL7a9HhkvOVb2oH1otZwCuMToel8P4tLSy>
- 스타일
 - <https://www.youtube.com/playlist?list=PL7a9HhkvOVb3EHaedVgt4M3M9oNzHchbk>

Table 18-2 주요 API

속성	기능 및 설명
<code>autoFit</code>	화면 전체 폭에 그리드를 출력. • <code>lastColumn</code>: 마지막 열만 확장 • <code>allColumn</code>: 모든 열의 폭을 균등하게 확장
<code>visibleRowNum</code>	페이지 당 표시할 행을 지정. • <code>all</code>: 모든 행을 표시. (단, 최적의 성능을 위해 50행 이하를 권장.)
<code>defaultValue</code>	새로운 행이 추가될 때 표시할 기본 값을 지정.
<code>hidden</code>	열 표시/숨김을 지정.
<code>textAlign</code>	셀에 표시할 텍스트의 정렬 방향을 지정합니다.
<code>readOnly</code>	각 셀 및 그리드 전체에 대해 데이터의 수정을 허용하거나 금지합니다.
<code>sortable</code>	해당 열의 정렬 가능 여부를 지정합니다.
<code>resize</code>	열의 너비를 사용자가 조정할 수 있도록 허가합니다.
<code>hideHeader</code>	헤더를 표시하거나 숨깁니다.
<code>fixedColumn</code>	틀고정을 위해 고정할 열의 개수를 지정합니다.
<code>focusMode</code> <code>focusMove</code> <code>keyMoveEditMode</code> <code>editModeEvent</code>	포커스 이동, 키 이동, 편집 모드 등을 설정할 수 있습니다.
<code>tooltipDisplay</code>	각 셀의 표시되지 못한 데이터를 사용자의 마우스 오버시 풍선 도움말 형태로 보여줍니다.
<code>colMerge</code> <code>upperColumn</code>	<code>colMerge="true"</code> 는 하나의 열에 있는 공통된 값의 행을 하나의 셀로 병합하도록 설정합니다. <code>upperColumn</code> 은 하나의 셀 병합 열을 기준으로 다른 열을 셀 병합하도록 기준을 설정합니다.

그림 18-1 GridView 구조



18.2 Pivot

Table 18-3 컴포넌트 설명

요약	기존 GridView에서 제한적으로 제공했던 Pivot 기능을 전문적으로 제공하는 컴포넌트. Excel의 Pivot Table 기능과 유사.
기능 및 특징	- 주요 기능 - 다차원 데이터 분석 - 드릴다운 - 축 변환 - 무한 가로축 스크롤링 - 통계 함수 - Sum, Avg, Count, Minimum, Maximum
YouTube 동영상	- Pivot 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb0J32Z13mRPC4xqhA4Yo8tg
비고	현재 IE10 버전 이상만 지원.

Table 18-4 주요 기능 설명

주요 기능	설명
축 변환	드래그&드롭을 통해 행과 열 사이의 축 변환이 가능하고, 새로운 축을 사용하여 테이블을 다시 생성.
컬럼 리사이징	드래그&드롭을 통해 헤더 사이즈 변경 가능.
Pivot 너비에 따른 컬럼 폭 자동 조절	테이블 전체 혹은 마지막 열의 너비를 화면 크기에 맞게 자동 조절 가능.

주요 기능		설명
		관련 속성: <code>autoFit</code>
파일 Import/Export	Excel (xls, xsls)	파일 내보내기/가져가오 가능.
	CSV	
컨텍스트 메뉴 제공		고정할 행 및 열을 지정할 수 있음.
포매팅 기능		사용자 정의 메뉴 지정 가능.
통계 함수 (행, 열, 및 전체 데이터에 대한 통계)		숫자 포맷에 필요한 옵션 제공. <code>digitsAfterDecimal</code> <code>scaler</code> <code>showZero</code> <code>decimalSep</code> <code>thousandsSep</code> <code>prefix</code> <code>suffix</code>
헤더 데이터 필터링		화면에 출력하지 않을 헤더 데이터 지정 가능.
DataCollection 바인딩		DataCollection과 바인딩 가능.
텍스트 정렬		<code>textAlign</code> (각 항목) <code>bodyTextAlign</code> (바디) <code>headerTextAlign</code> (헤더) <code>totalTextAlign</code> (전체)

19. Container

19.1 FloatingLayer

그림 19-1 FloatingLayer



Table 19-1 컴포넌트 설명

요약	화면 안에 팝업 형태로 출력되는 새로운 창을 생성
기능 및 특징	• 화면 최상위 층에 출력 (useModal) • 이동 제어 (movable)
주요 용도	• 팝업 창
YouTube 동영상	• FloatingLayer 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb3ebAYwTkrKQnjeCMkqUpoz&si=E9Q0sTCsT086KJkD • FloatingLayer 쇼츠 동영상 모음 ◦ https://www.youtube.com/playlist?list=PL7a9HhkvOVb22iiojjmse52vSRkprERug

19.2 Generator

Table 19-2 컴포넌트 설명

요약	여러 컴포넌트로 구성된 컴포넌트 그룹을 한번에 생성하기 위한 컴포넌트
기능 및 특징	• 여러 개의 컴포넌트로 구성된 컴포넌트 그룹을 동시에 생성 • 반복 생성할 그룹에 동적으로 컴포넌트를 추가 및 삭제 • API를 통해 그룹에 포함된 컴포넌트를 제어

주요 용도	여러 컴포넌트로 구성된 웹 UI를 생성하는 인터페이스 (예: 입력 상자 추가)
YouTube 동영상	• Generator 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb2hTiHa41XfIVgW8560YDej

그림 19-2 Generator

Table 19-3 주요 API

API	설명
<code>getChild(index, id)</code>	• Generator의 index 번째 반복부에서 id에 해당하는 웹스퀘어 컴포넌트를 반환
<code>getLength()</code>	• Generator가 몇 개의 반복부를 가지고 있는지를 반환
<code>insert()</code>	• Generator의 반복부에 해당하는 부분을 맨 뒤에 하나 더 추가
<code>removeAll()</code>	• Generator로 출력된 항목을 모두 삭제하고 초기화



아래 컴포넌트는 Generator를 통해 반복 생성할 수 없습니다.

Chart
 GridView
 Menu
 Switch
 TabControl
 TreeView
 WindowContainer

19.3 Group

Table 19-4 컴포넌트 설명

요약	여러 컴포넌트를 그룹핑하는 컴포넌트.
기능 및 특징	- 그룹에 속한 컴포넌트 전체를 초기화 - 그룹에 속한 컴포넌트 전체에 대한 함수 호출 - HTML5 tag 지정
주요 용도	- 전체 레이아웃을 구성 - 복잡한 계층 구조 구현
YouTube 동영상	Group 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb1G2Q_9dW7L7hRyq0RTF3ml Group 쇼츠 동영상 모음 https://www.youtube.com/playlist?list=PL7a9HhkvOVb0SQ_QTKXdijeONh_bjcVOx

Table 19-5 주요 API

API	설명
<code>init()</code>	- Group 컴포넌트 내에 있는 컴포넌트들의 값을 초기화 - 사용 예: <code>group1.init();</code>
<code>invoke()</code>	- Group 컴포넌트 하위의 모든 컴포넌트의 함수를 동일하게 호출 - 사용 예: <code>group1.invoke("setValue", "WebSquare");</code>

19.4 ScrollView

그림 19-3 ScrollView



Table 19-6 컴포넌트 설명

요약	모바일 전용 터치 스크롤링을 제공.
기능 및 특징	Group 컴포넌트처럼 하위에 여러 컴포넌트를 포함.

19.5 SlideHide

그림 19-4 SlideHide



Table 19-7 컴포넌트 설명

요약	슬라이딩을 통한 숨김/표시 기능을 제공.
기능 및 특징	이벤트에 따라 특정 layer를 animation으로 표시하는 컴포넌트. 처음에는 정해진 크기(<code>showSize</code>)만큼만 표시. 이후 마우스 오버 등 특정 이벤트(<code>slideEvent</code>)가 발생할 경우, 정해진 방향(<code>slideDirection</code>)으로 정해진 크기(<code>moveSize</code>)만큼 이동하면서 내부 내용을 표시.
주요 용도	퀵 메뉴와 같이 마우스를 올리면 UI를 잠시 표시하는 상황에서 사용.

19.6 Switch

그림 19-5 Switch

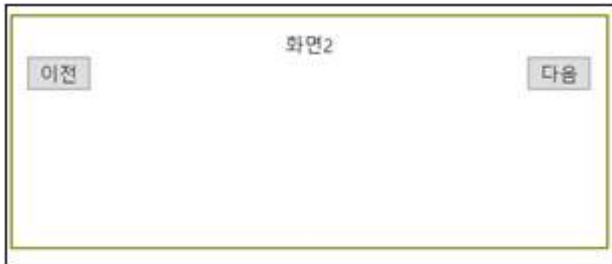


Table 19-8 컴포넌트 설명

요약	여러 상황이 가능한 경우, 각 상황에 대한 화면을 구성하고 해당 상황에 맞는 화면을 출력
기능 및 특징	• 각 상황 별 동적 화면 표시 • 슬라이딩 효과 • 기본 표시 화면 설정 가능 • 드래그 앤 드롭 (브라우저) • 터치 (모바일)
주요 용도	• 페이지 이동, 목록 이동 등
응용	• 모바일 접근성

19.7 TabControl

그림 19-6 TabControl



Table 19-9 컴포넌트 설명

요약	탭 형태의 화면을 구성. TabControl 컴포넌트를 그리면 최초 1개의 Tab과 해당 Content 영역이 생성됨. 각 Content는 Group처럼 하위에 컴포넌트를 포함할 수 있음.
----	---

기능 및 특징	• <code>src</code> 속성을 이용하여 외부의 XML을 포함 가능 • <code>addTab()</code> 함수로 탭 추가시 프레임 종류 (IFrame 혹은 WFrame) 선택 가능 • 탭 닫기 버튼 • 웹 접근성 지원 • 키보드 제어 지원
주요 용도	여러 탭으로 이루어진 화면 구성
YouTube 동영상	• TabControl 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb2icS8hs4SXmAE-rs_JGrhb

19.8 TableLayout

Table 19-10 컴포넌트 설명

요약	Group 컴포넌트의 <code>tagname</code> 속성에 HTML5 태그(<code><colgroup></code> , <code><tr></code> , <code><td></code> , <code><th></code>)를 추가하여 <code><table></code> 태그로 묶음
기능 및 특징	• 각 셀에 컴포넌트 추가 가능 • 테이블 편집이 용이
주요 용도	• 폼 양식 구성
YouTube 동영상	• TableLayout 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb0VqTzmRAyr7tXJZpCxrXbv • TabControl 쇼츠 동영상 모음 ◦ https://www.youtube.com/playlist?list=PL7a9HhkvOVb2-1F4lhG4W0DYu_zPynva7

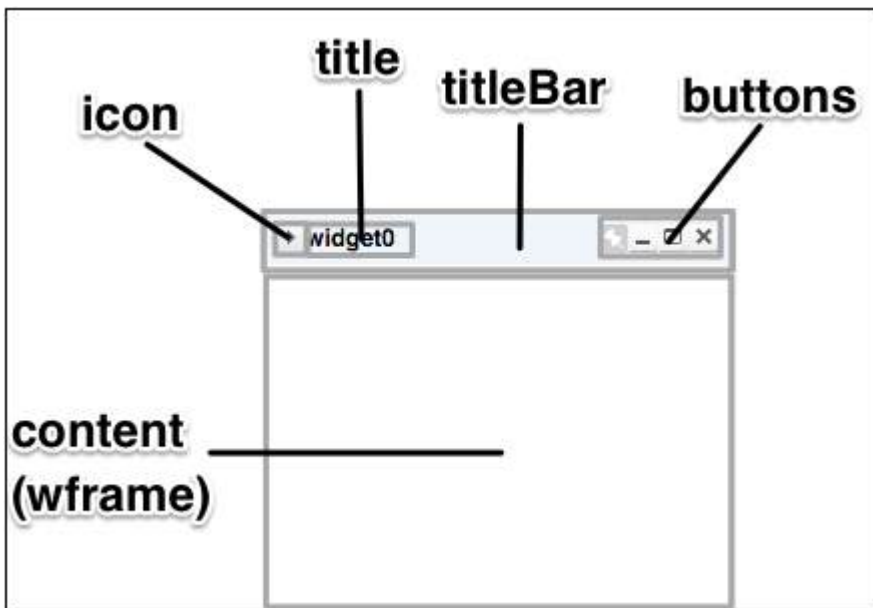
19.9 WidgetContainer

Table 19-11 컴포넌트 설명

요약	• 여러 개의 웹 위젯을 한 화면에서 화면에서 제공하는 컴포넌트. • 서버 통신 등을 포함한 업무 화면을 웹 위젯으로 사용. • 업무 화면 및 유틸리티 사용 환경을 개인화.
	• 좌표계를 바탕으로 위젯 위치 결정 (<code>cols</code>, <code>unitHeightPixel</code>) • 동적 위젯 생성/추가 (<code>addWidget()</code>)

기능	- 위젯 위치 및 크기 제어 (이동, 고정, 리사이징, 최대화/최소화) - 위젯 추가/내보내기 <code>importWidgets()</code> <code>exportWidgets()</code> <code>importWidget()</code> <code>exportWidget()</code>
기능 및 특징	- 각 위젯은 서로 다른 URL을 통해 관리. - 사용자가 원하는 콘텐츠만 한 화면에 배치 가능. - 위젯 이동, 리사이징, 최대화/최소화 가능. - WFrame 사용으로 SPA 구현 및 효율적인 리소스 관리 가능.
관련 컴포넌트	WFrame
YouTube 동영상	WidgetContainer 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb1Nf9NRubR0GgqtgBRFUKLX WidgetContainer 쇼츠 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb0vm2s4YIRkoUZ_icoutHkQ

그림 19-7 위젯 구조



19.10 WindowContainer

그림 19-8 WindowContainer

Table 19-12 컴포넌트 설명

요약	툴바를 이용하여 여러 개의 윈도우를 제어
기능 및 특징	하나의 부모 윈도우가 여러 개의 자식 윈도우가 포함하고, 윈도우 간 계층 구조를 갖는 MDI(Multiple Document Interface)를 지원
YouTube 동영상	- WindowContainer 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb3XTGQ1f53wizoLORdND0Yf - WindowContainer 쇼츠 동영상 모음 https://www.youtube.com/playlist?list=PL7a9HhkvOVb3y9O3_FBxMW6bZqJxh9WDW

20. Navigation

20.1 Accordion

그림 20-1 Accordion

Table 20-1 컴포넌트 설명

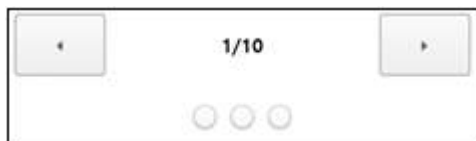
요약	여러 개의 패널에 콘텐츠를 표시
기능 및 특징	- 서로 다른 콘텐츠를 담은 패널 여러 개를 포함 - 각 패널의 추가, 삭제, 열기/닫기가 가능
주요 용도	서로 다른 콘텐츠를 패널 별로 분리해서 한 화면에 표시할 때 유용
YouTube 동영상	Accordion 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb23rTGpwGL_4V4tB3rNGr_x Accordion 쇼츠 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb06f6oKRnEXIGic72JVb1lm

20.2 PageControl

Table 20-2 컴포넌트 설명

요약	페이지 이동용 UI를 제공.
기능 및 특징	단순히 앞으로 가기 및 뒤로 가기의 기능만 제공.
주요 용도	전체 페이지 수 및 페이지 당 표시하고 싶은 데이터 수는 직접 구현해야 함.

그림 20-2 PageControl



20.3 PageList

그림 20-3 PageList



Table 20-3 컴포넌트 설명

요약	페이지 수와 페이지를 선택하는 인터페이스를 제공
YouTube 동영상	• PageList 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb2CEgeDt3y4LY6ybU2ZOQr5

20.4 TreeView

그림 20-4 TreeView

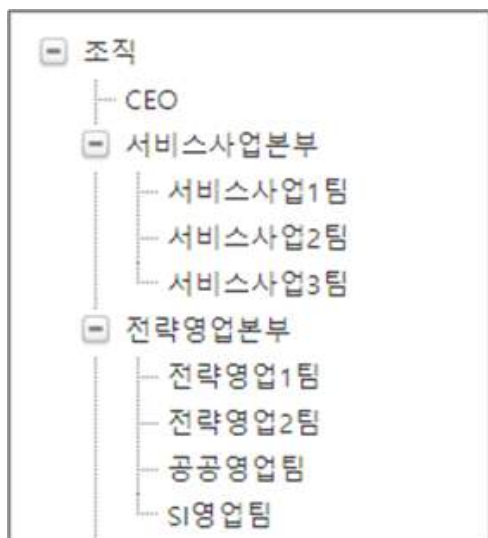


Table 20-4 컴포넌트 설명

요약	계층적인 데이터를 트리 형태로 표시
기능 및 특징	• 드래그-앤-드롭을 통한 동적 편집 가능 • 일반 데이터를 계층적인 데이터 형태로 변환 가능
YouTube 동영상	• TreeView 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb31luBtsoqrcJh6FoCb0N0Y • TreeView 쇼츠 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb3pbEdWSJ4kuo1SAZiwYC9i

21. Chart

21.1 FusionChart

그림 21-1 FusionChart

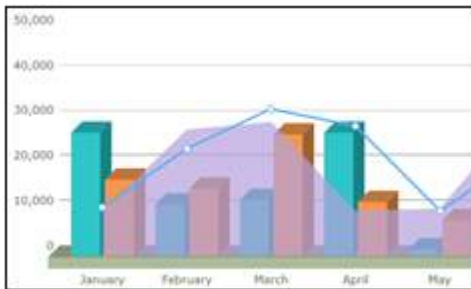


Table 21-1 컴포넌트 설명

요약	FusionChart는 다양한 Device, Browser, Platform에서 작동하는 JavaScript기반 차트입니다. WebSquare5는 FusionChart 기능을 사용할 수 있는 환경을 제공합니다.
지원 브라우저	거의 모든 브라우저를 지원합니다. 단, flash 렌더링 시 flash를 지원하지 않는 브라우저는 지원하지 않습니다. • Internet Explorer 6/7/8/9/10/11 • Chrome • Safari • Opera • Firefox
차트 종류	• 선형 차트 (line chart) • 세로막대차트 (2D/3D Column Chart) • 영역차트 (Area Chart) • 가로막대차트 (2D/3D Bar Chart) • 원형차트 (2D/3D Pie Chart) • 도넛차트(2D/3D Doughnut Chart) • 스택차트 (Stacked Chart) • 조합차트(Combination Chart) • 스크롤차트(Scroll Chart) • 버블차트(Bubble Chart) • xy(Scatter)차트 (xy Chart) SSGrid
지원 데이터	• json • xml (FusionChart native API 이용)
YouTube 동영상	• YouTube 동영상 모음 ◦ https://www.youtube.com/playlist?list=PL7a9HhkvOVb0xvpS56yhKjv8hemJU9SKU

- [YouTube 쇼츠 동영상 모음](#)
 - <https://youtube.com/playlist?list=PL7a9HhkvOVb0Soc06Pj6MBTgyORkHi0q&si=7sFcbqiwuBDaNBz>

21.2 MapChart

그림 21-2 MapChart



Table 21-2 컴포넌트 설명

요약	d3 데이터를 이용하여 대한민국의 전국 지도를 다양한 형태로 시각화하여 표현.
기능 및 특징	• 각 지명 위에 마커 표시 가능. • 각 마커 별 스타일 지정 가능.
YouTube 동영상	• MapChart 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb0kc2GAWbX_jT0zE_wWh ec6 • MapChart 쇼츠 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb2pJmbXYNxpjDLA9LIN_z0 0

22. Frame

22.1 IFrame

Table 22-1 컴포넌트 설명

요약	HTML의 IFrame과 유사. IFrame을 통해 WebSquare 파일, 일반 웹 콘텐츠, 다른 사이트들을 화면에 출력 가능.
기능 및 특징	SPA 모드 적용이 가능한 컴포넌트이며. 단, SPA 적용을 위한 제약사항을 반드시 준수해야 함.
YouTube 동영상	- IFrame 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb3gLxM2tVmMkCuBEvBb12a6

22.2 PageFrame

Table 22-2 컴포넌트 설명

요약	(UDC로 작성된) Page Component 화면을 PageFrame의 src 로 등록하여 Page Component의 API에 직접 접근할 수 있는 기능을 제공.
기능 및 특징	- Page Component 화면 파일을 별도의 UDC로 등록하지 않고 컴포넌트로 활용할 수 있는 효과 제공. - WFrame의 모든 기능을 제공하기 때문에 WFrame을 대체할 수 있음.
YouTube 동영상	- PageFrame 동영상 모음 https://www.youtube.com/playlist?list=PL7a9HhkvOVb0gRxSMm0EdPR0G42iuByhz

PageFrame은 WFrame에서 제공하는 모든 기능을 포함하기 때문에 WFrame 대신에 PageFrame을 사용할 수 있습니다. 단, WFrame이 사용하는 **onwframeonload** 및 **onwframeunload** 이벤트 대신, PageFrame은 **onpageframeonload** 및 **onpageframeunload** 이벤트를 사용해야 합니다.

Table 22-3 WFrame Vs. PageFrame

	WFrame	PageFrame
소스(src)로 Page Component를 등록	O	O
소스(src)로 Page Component를 등록하여 Page Component의 API에 직접 접근	X	O
로딩/언로딩 이벤트	onwframeonload	onpageframeonload

	WFrame	PageFrame
	onwframeunload	onpageframeunload

22.3 WFrame

Table 22-4 컴포넌트 설명

요약	WebSquare 화면을 삽입 (WFrame에 포함된 페이지를 상위 페이지와 함께 렌더링.)
기능 및 특징	DataCollection, 인스턴스, JavaScript, CSS 공통화
YouTube 동영상	- WFrame 동영상 모음 https://youtube.com/playlist?list=PL7a9HhkvOVb0IRHVCdtYkN59bIPyHPH9e

Table 22-5 IFrame Vs. WFrame

	페이지와의 Merge 여부	호출 방식
IFrame	X	IFrame을 따로 호출.
WFrame	O	WFrame과 페이지를 함께 호출.



IFrame과의 차이

IFrame 컴포넌트에 삽입된 객체는 현재 웹 페이지와 무관한 독립적인 객체입니다.

반면, WFrame에 삽입된 객체는 현재 페이지에 포함(merge)되어 전체가 하나의 소스로 간주됩니다.

즉, IFrame 컴포넌트를 포함한 페이지를 로딩할 경우, 해당 페이지와 IFrame에 포함된 페이지는 각각 따로 호출됩니다. WFrame을 포함한 페이지는 해당 페이지와 WFrame에 포함된 객체가 함께 호출됩니다.

23. 기타 컴포넌트

23.1 ActiveX

Table 23-1 컴포넌트 설명

요약	ActiveX 사용을 지원.
----	-----------------

23.2 Applet

Table 23-2 컴포넌트 설명

요약	Applet 사용을 지원.
----	----------------

23.3 Flash

Table 23-3 컴포넌트 설명

요약	Flash 사용을 지원.
----	---------------

23.4 Output

Table 23-4 컴포넌트 설명

요약	• 브라우저에 문자열을 출력. • 최상위에 <code><div></code> 태그를 사용하며, 텍스트 출력 및 포맷 적용 등의 기능을 제공. • 현재 Deprecated 상태. TextBox 사용을 권장.
----	--

23.5 ProgressBar

그림 23-1 ProgressBar

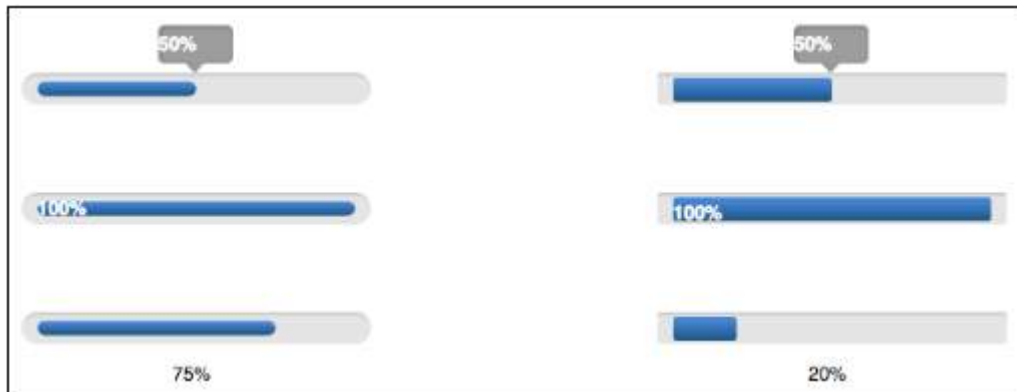


Table 23-5 컴포넌트 설명

요약	진행 상태를 표시하는 컴포넌트.
주요 용도	특정 동작의 진행 상태를 표시.

23.6 RoundedRectangle

Table 23-6 컴포넌트 설명

요약	- 둥근 모서리의 사각형을 제공. - 사각형 내부에 색을 채우거나 gradient를 설정할 수 있음. - SVG를 활용하여 간단한 라운드 박스를 그릴 수 있으나, 다른 컴포넌트에 비해 속도가 느리므로 꼭 필요한 경우에만 사용 권장.
----	---

23.7 Tag

Table 23-7 컴포넌트 설명

요약	일반 HTML 태그를 WebSquare에서 관리하는 상위 객체
----	------------------------------------

23.8 XHTML

Table 23-8 컴포넌트 설명

요약	• XHTML 파일을 레이어에 표시. • XHTML 페이지를 임베드할 수 있는 플러그인. • HTML 파일의 경로는 src 속성을 통해 설정 가능.
----	--

23.9 XSL

Table 23-9 컴포넌트 설명

요약	XSL을 통해 웹스퀘어 XML을 출력.
----	-----------------------

23.10 HTML5 태그 지원

Table 23-10 컴포넌트 설명

요약	아래와 같은 HTML5 태그를 컴포넌트 형태로 지원. • Article • Aside • Audio • Canvas • Footer • Header • Navigation • Section • Video
----	--

24. 유틸리티

24.1 \$p

Table 24-1 컴포넌트 설명

요약	자주 사용하는 유틸리티 객체를 축약하여 제공. (예를 들어 <code>\$p.dateAdd()</code> 함수는 <code>WebSquare.date.dateAdd()</code> 함수와 동일.)
YouTube 동영상	• \$p 비디오 동영상 모음 ◦ 일반 ▪ https://youtube.com/playlist?list=PL7a9HhkvOVb3ZLLI7_F-O1L0c0FdLHS6B ◦ Data ▪ https://www.youtube.com/playlist?list=PL7a9HhkvOVb0C3zGOIrelvcK6niQiuNDk ◦ 팝업 ▪ https://youtube.com/playlist?list=PL7a9HhkvOVb1ijrxDMsk6eDkjwy1Fgb_X ◦ 서브미션 ▪ https://youtube.com/playlist?list=PL7a9HhkvOVb1J5nm94k4wl0Mx4gb0b7U6 • \$p 쇼츠 동영상 모음 ◦ 일반 ▪ https://www.youtube.com/playlist?list=PL7a9HhkvOVb3IKV3laAytirDXP57Z2tbW ◦ 팝업 ▪ https://www.youtube.com/playlist?list=PL7a9HhkvOVb2h5n79Lwv-cr_bzx7oiJlM

24.2 WebSquare.date

Table 24-2 설명

요약	날짜 및 시간 관리 기능을 제공.
YouTube 동영상	• WebSquare.date 비디오 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb3tIEPN7BWS6svzZXJh5Tmb

- [WebSquare.date 쇼츠 동영상 모음](#)
 - <https://youtube.com/playlist?list=PL7a9HhkvOVb0eUbNc2mtVr4PzPVxrCE8T>

24.3 WebSquare.util

Table 24-3 설명

요약	자주 사용하는 기능을 API로 제공.
YouTube 동영상	• WebSquare.util 비디오 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb0U1iZ0u573o7mh7oVa3UT6

24.4 팝업

Table 24-4 컴포넌트 설명

요약	\$p.openPopup() 함수를 사용하여 생성하는 팝업 객체.	
기능 및 특징	• 모달 팝업 지원. • 프레임 모달 팝업 지원. • 부모 - 자식 관계의 팝업 제어 (함께 열기/닫기, 상호 참조 등) 가능. • 팝업을 통한 파라미터 전달.	
YouTube 동영상	• 팝업 비디오 동영상 모음 ◦ https://youtube.com/playlist?list=PL7a9HhkvOVb1ijrxDMsk6eDkjwy1Fgb_X • 팝업 쇼츠 동영상 모음 ◦ https://www.youtube.com/playlist?list=PL7a9HhkvOVb2h5n79Lwv-cr_bzx7oijIM	

Table 24-5 팝업 종류 (type)

옵션	설명	비고
wframePopup	• WFrame 내에 팝업 화면을 호출. • Scope 기능 지원.	• 권장.
pageFramePopup	• PageFrame 내에 팝업 화면을 호출. • Scope 기능 지원.	• 권장. • AI 버전 이후 지원.
		• 비권장.

옵션	설명	비고
iframePopup	IFrame 내에 팝업 화면을 호출.	- 외부 도메인과 연동시 사용. - 외부 솔루션과 연동 시 사용.
browserPopup	- 브라우저가 제공하는 native 윈도우에 팝업 화면을 호출. - 팝업 호출을 위해 별도의 프로세스를 생성. - URL 주소가 노출됨.	- 비권장. - 별도 창이 필요한 경우 사용.

Part V. **설정**

25. 스튜디오 설정

25.1 개요

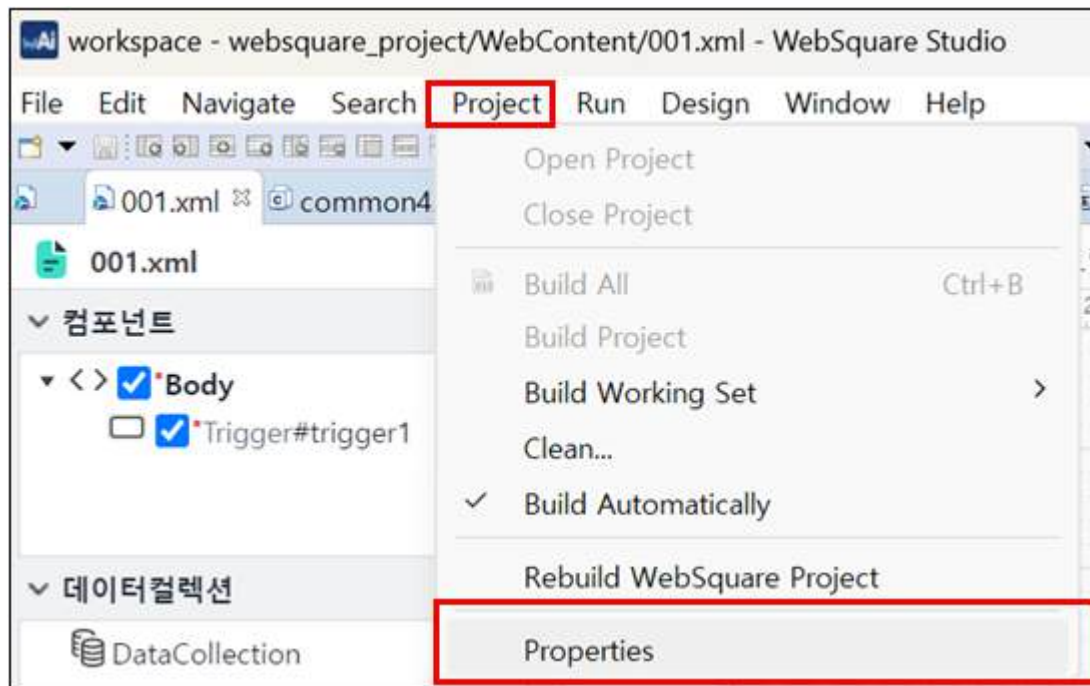
Project Explorer에서 웹스퀘어 환경설정 - 스튜디오 아래의 각 항목을 클릭하여 해당 부분을 설정할 수 있습니다.

그림 25-1 웹스퀘어 환경설정 - 스튜디오



혹은 해당 프로젝트의 Properties를 선택해도 동일한 화면을 표시할 수 있습니다.

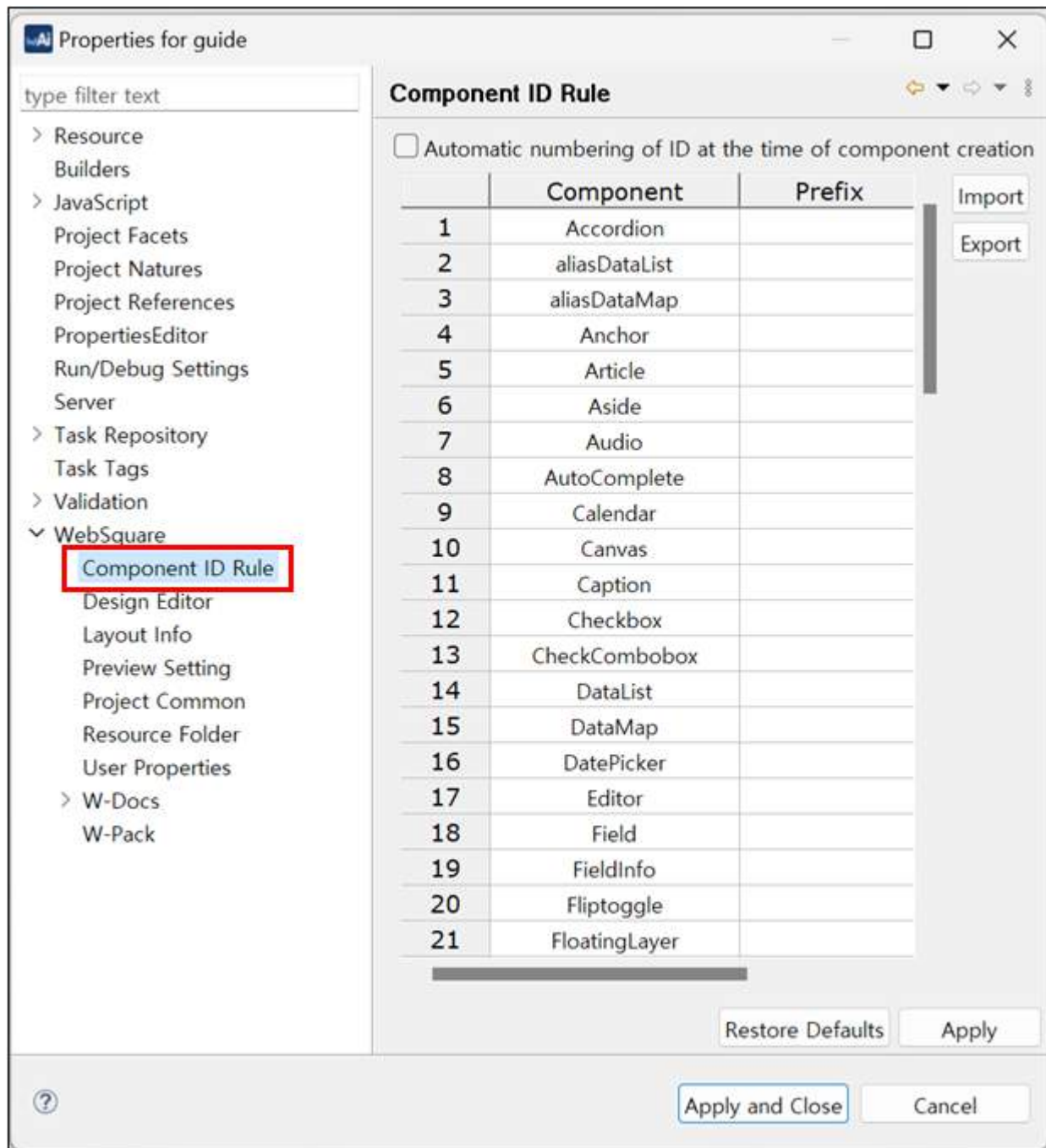
그림 25-2 Project - Properties



25.2 Component ID Rule

ID 생성 규칙을 설정할 수 있습니다.

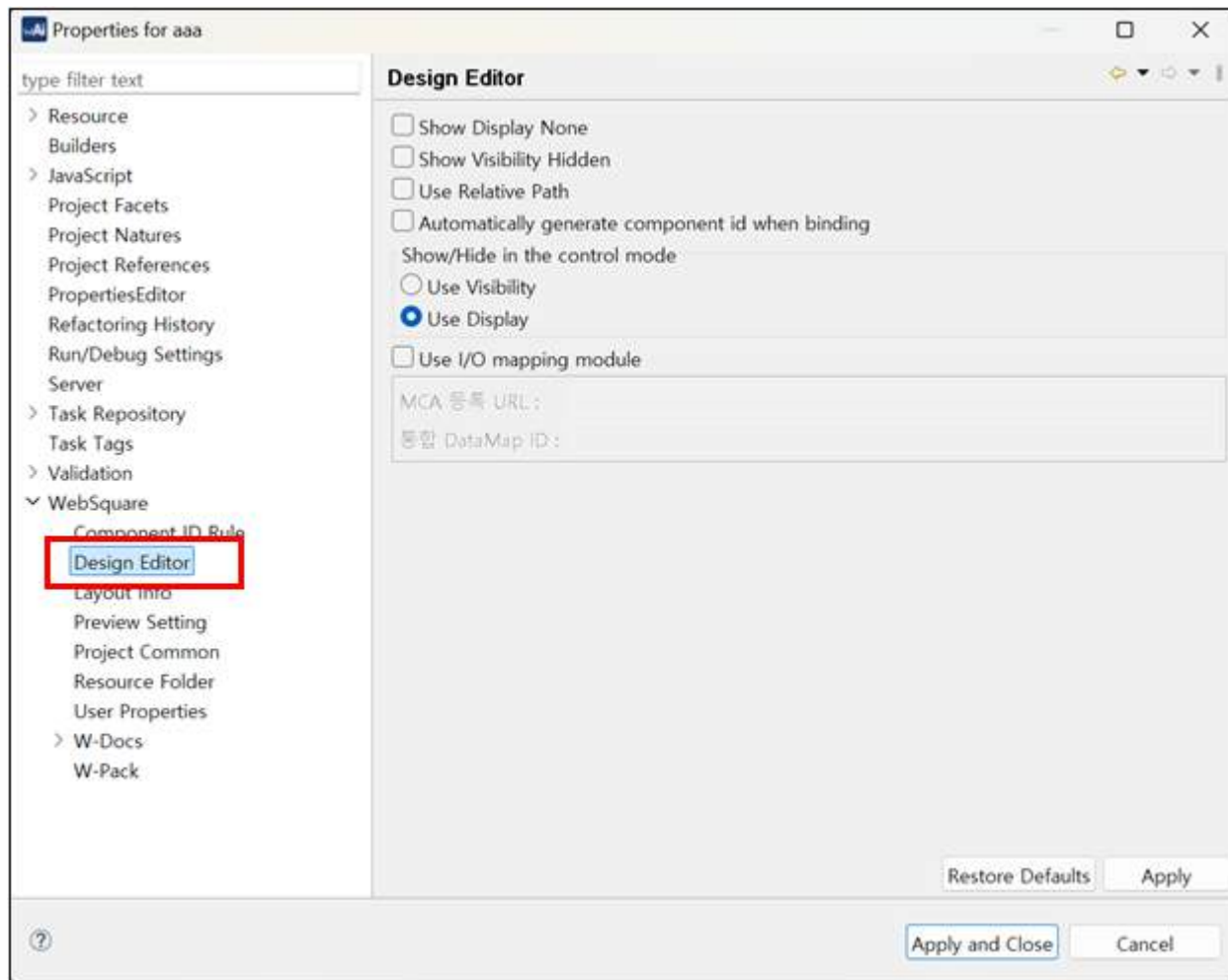
그림 25-3 Component ID Rule



25.3 Design Editor

디자인 에디터와 관련된 각종 내용을 설정할 수 있습니다.

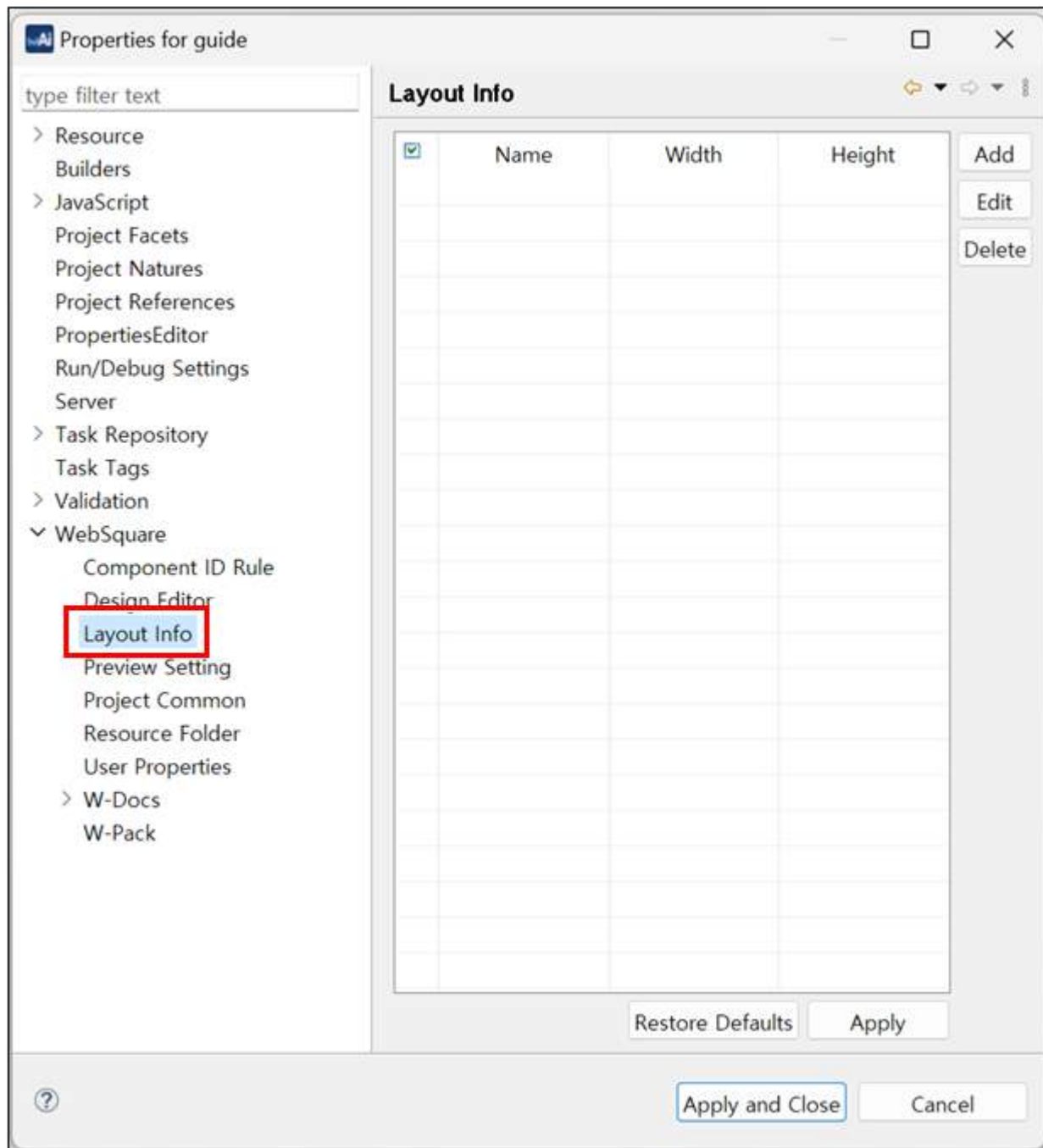
그림 25-4 Design Editor



25.4 Layout Info

7.4.1.2.2. 레이아웃 및 7.4.1.2.2. 레이아웃 생성 참고.

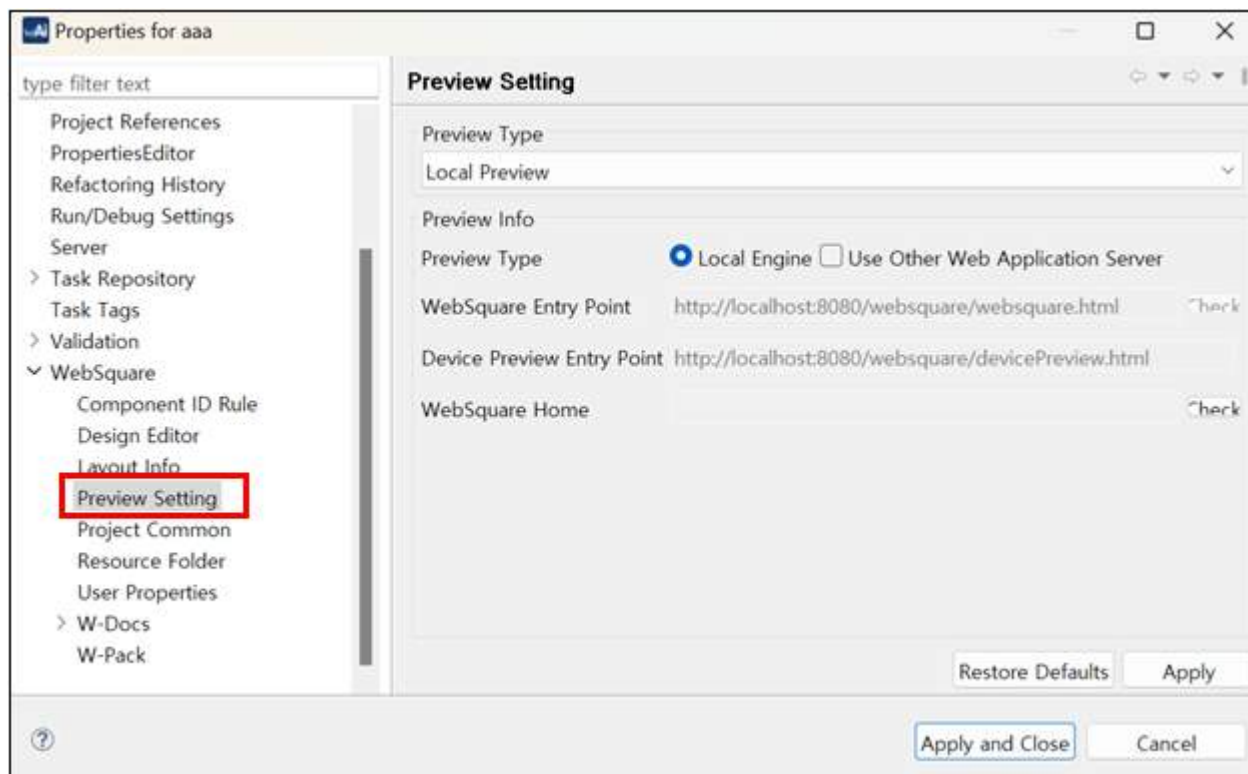
그림 25-5 Layout Info



25.5 Preview Setting

화면 미리보기 관련 설정을 할 수 있습니다.

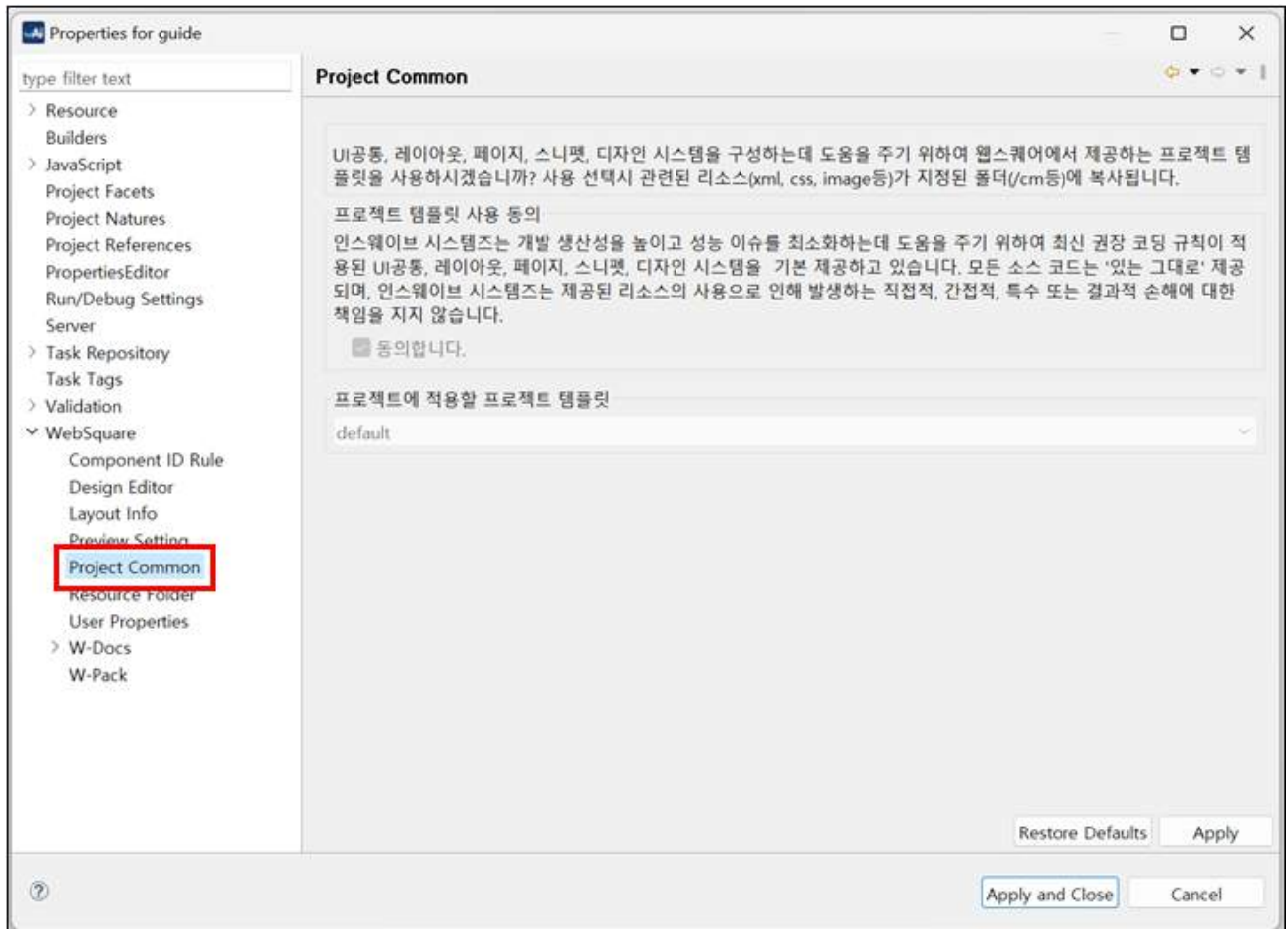
그림 25-6 Preview Setting



25.6 Project Common

최초 프로젝트 생성시 WRM 컴포넌트 사용에 동의하지 않은 경우, Project Common 설정을 통해 WRM 컴포넌트를 사용할 수 있습니다.

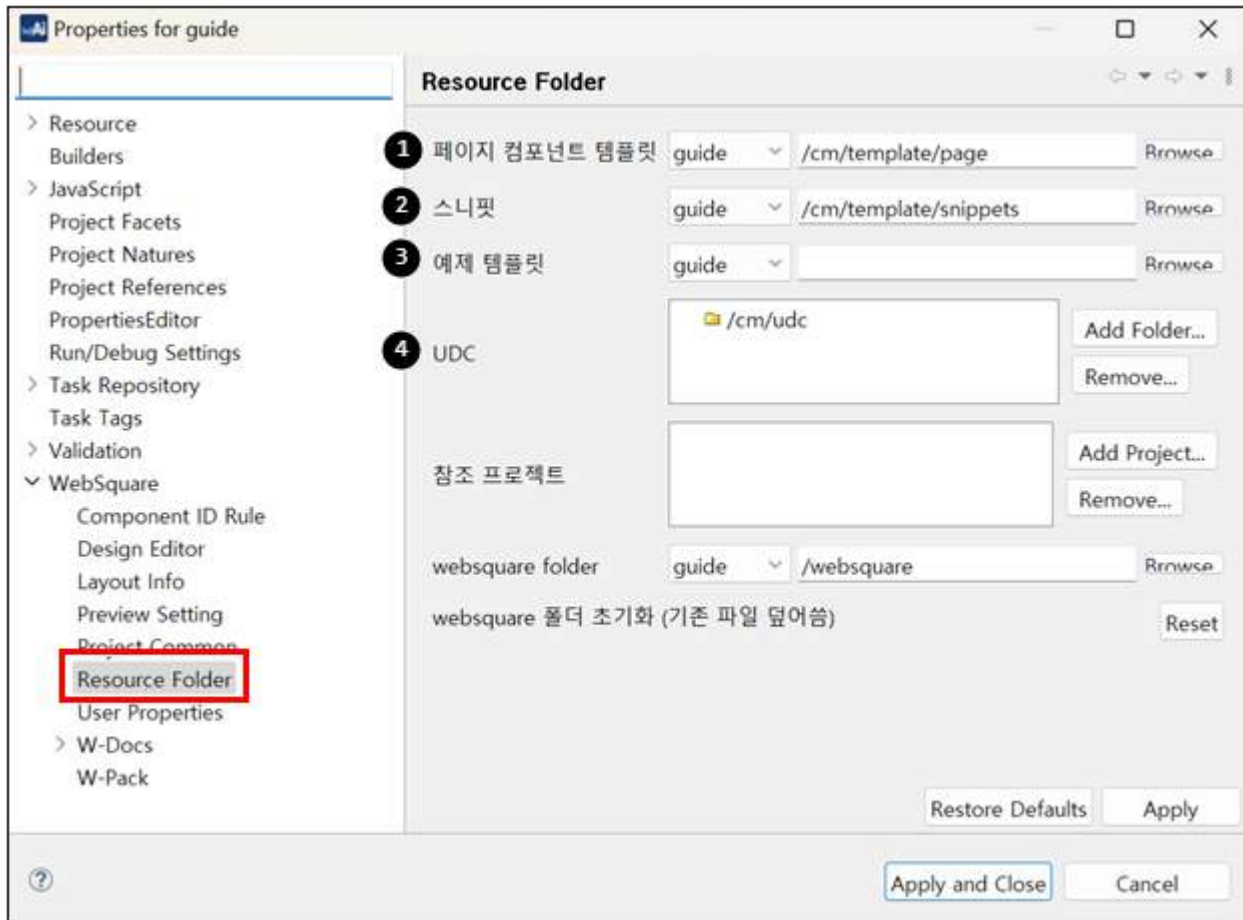
그림 25-7 Project Common



25.7 Resource Folder

폴더를 통해 공통 모듈을 포함한 웹스퀘어 시에 추가된 기능을 직관적으로 확인할 수 있습니다.

그림 25-8 Resource Folder

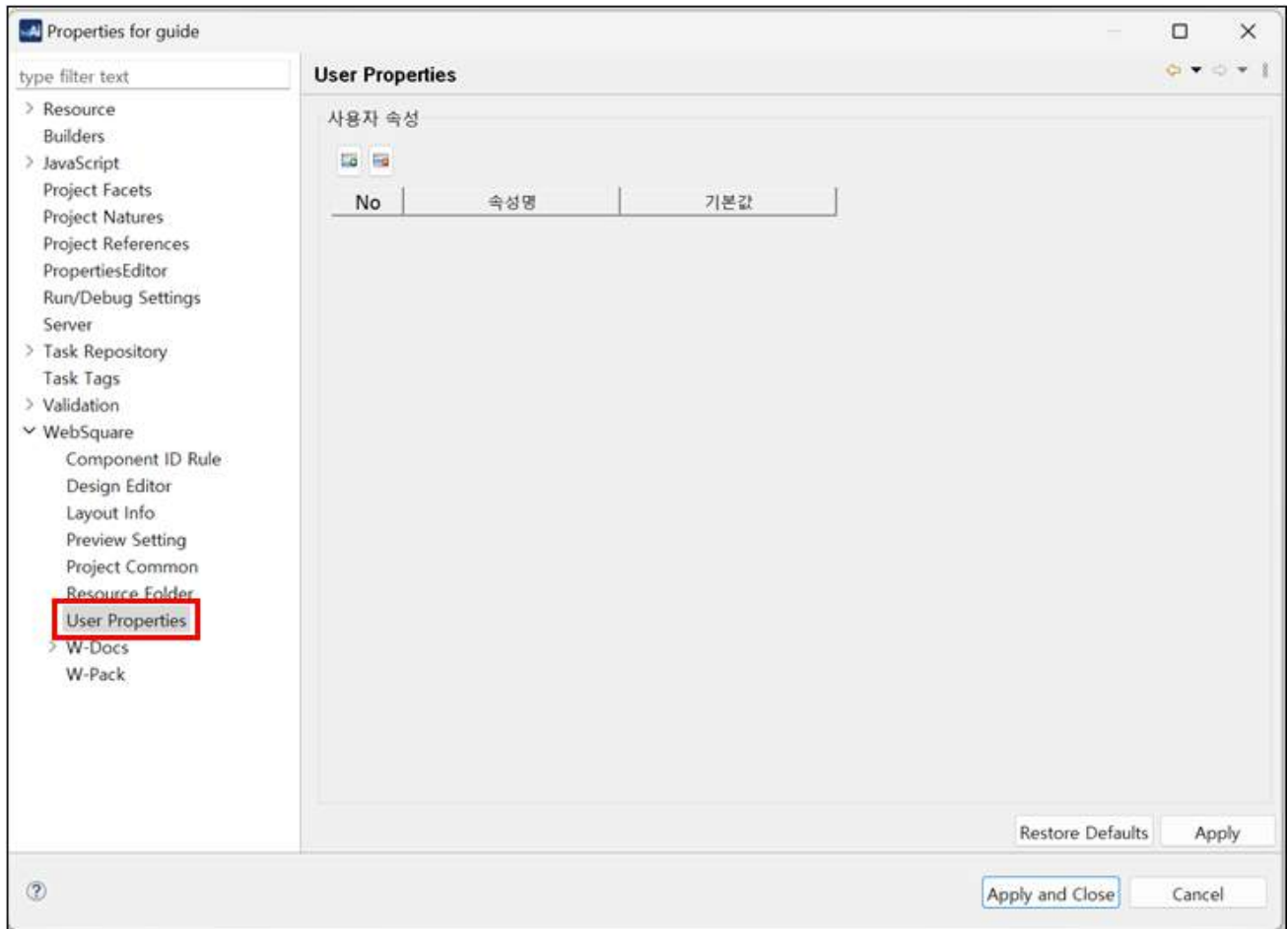


No.	리소스	설명
①	페이지 컴포넌트 템플릿	페이지 템플릿 (화면 생성 시 선택)
②	Snippets	Snippet 리소스
③	Example 템플릿	예제 템플릿
④	UDC	Design Kit에 등록되는 UDC

25.8 User Properties

사용자 정의 속성을 추가할 수 있습니다.

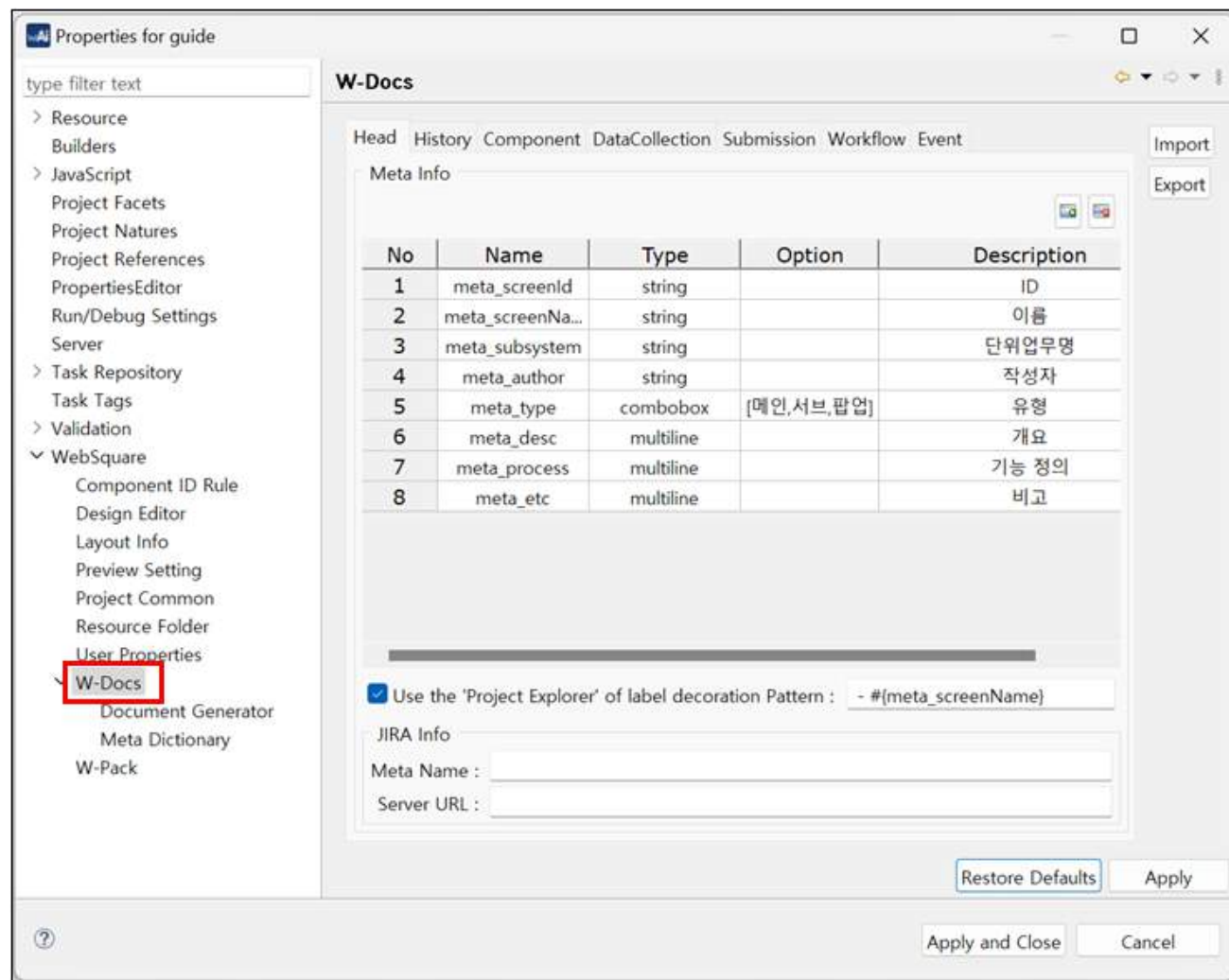
그림 25-9 User Properties



25.9 W-Docs

W-Docs 관련 속성을 추가할 수 있습니다.

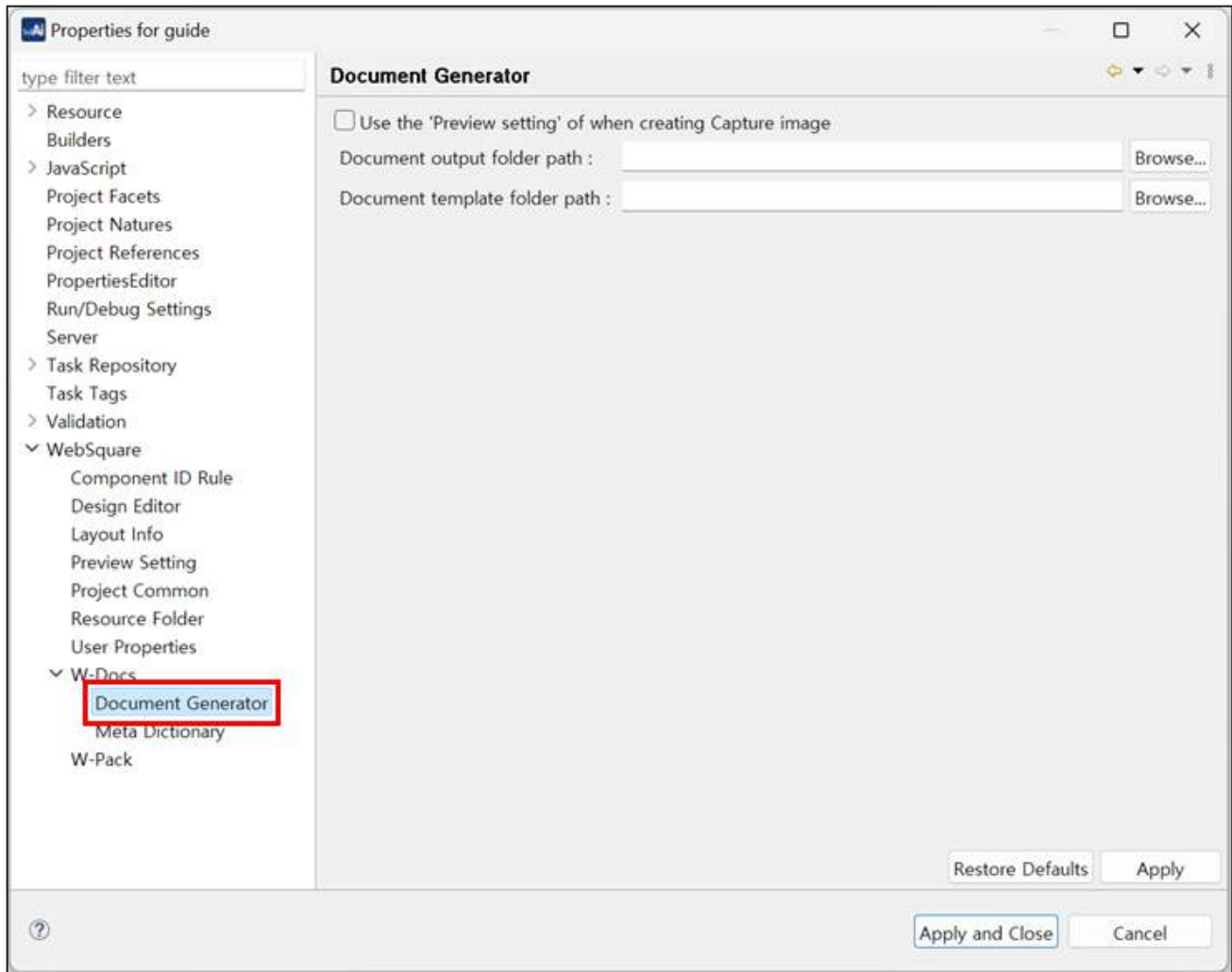
그림 25-10 W-Docs



25.9.1. Document Generator

W-Docs의 문서 생성 경로 및 템플릿 경로를 추가할 수 있습니다.

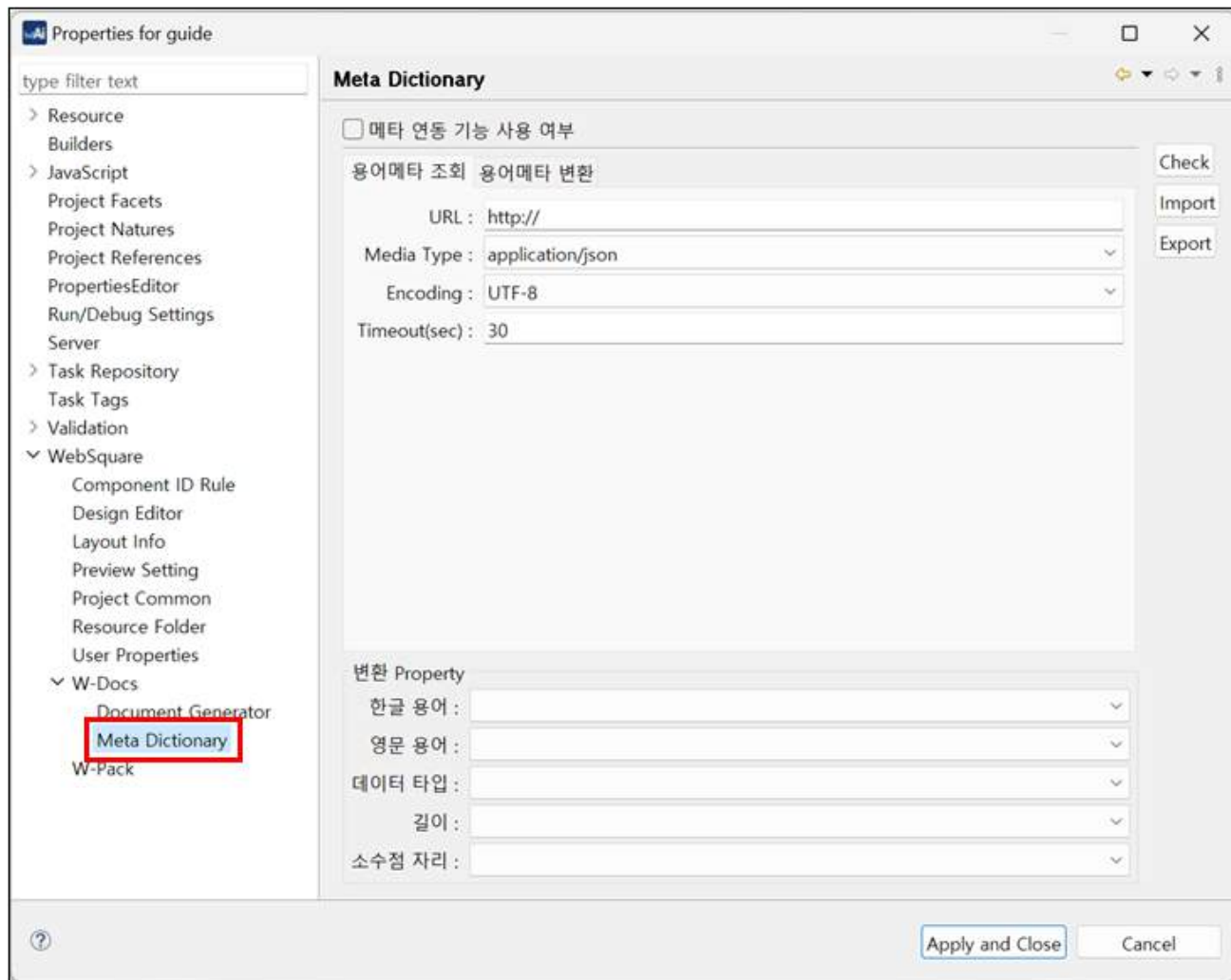
그림 25-11 W-Docs - Document Generator



25.9.2. Meta Dictionary

W-Docs의 메타 연동 기능 관련 속성을 추가할 수 있습니다.

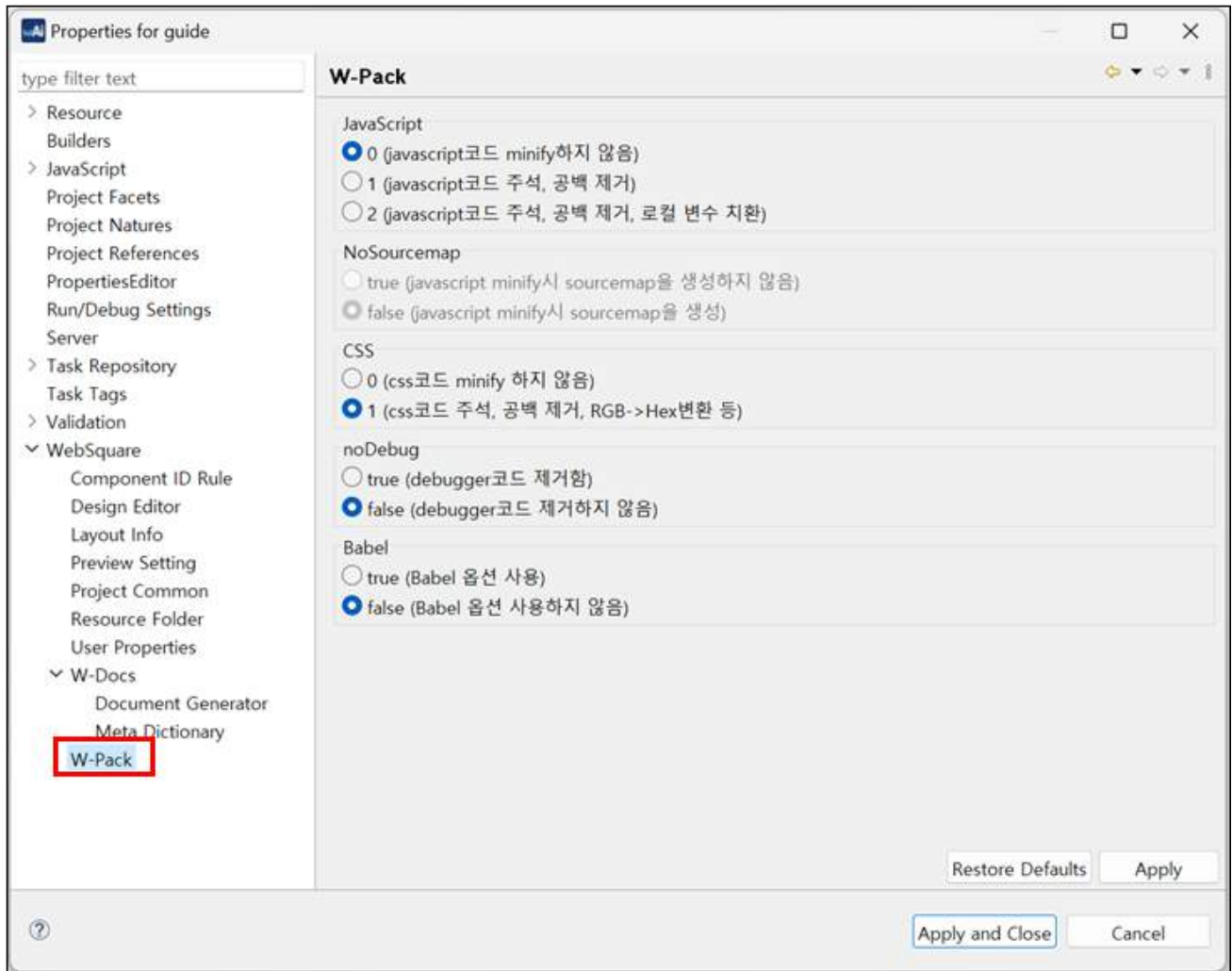
그림 25-12 W-Docs - Meta Dictionary



25.10 W-Pack

W-Pack 관련 속성을 추가할 수 있습니다.

그림 25-13 W-Pack

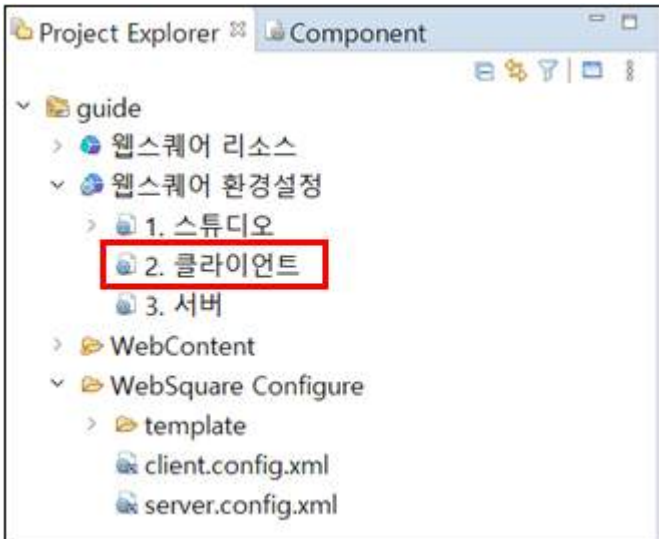


26. 클라이언트 설정 (client.config.xml)

26.1 개요

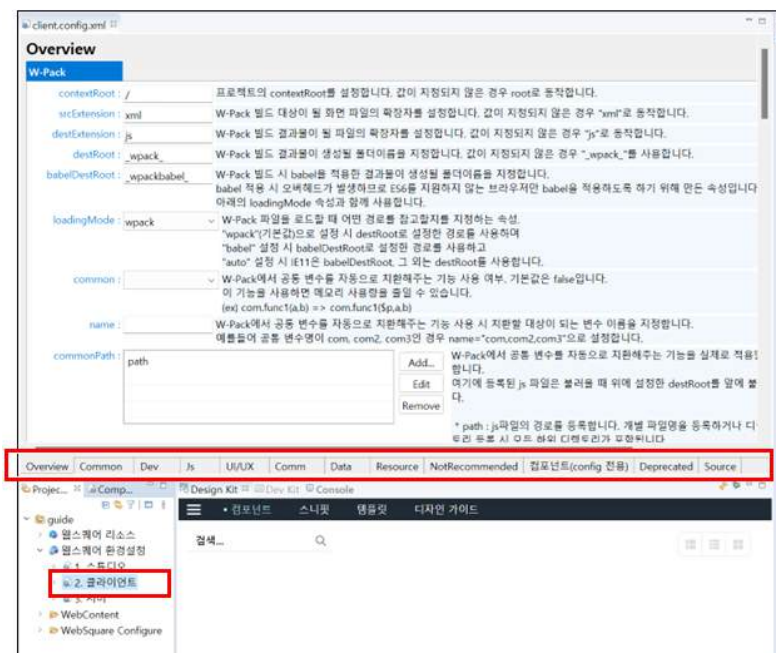
클라이언트 설정 파일은 JavaScript에서 참고하는 파일로 항상 서버에서 읽어옵니다. Project Explorer에서 클라이언트 설정 파일을 열어 설정할 수 있습니다.

그림 26-1 웹스퀘어 환경설정 - 클라이언트



클라이언트 설정 파일을 열고 하단의 각 탭을 클릭하여 해당 항목들을 설정할 수 있습니다.

그림 26-2 클라이언트 설정 (client.config.xml)



26.2 Overview

26.2.1. W-Pack

26.2.1.1. contextRoot

기본 값	설명
/	프로젝트의 contextRoot를 설정. 값이 지정되지 않은 경우 root로 동작.

26.2.1.2. srcExtension

기본 값	설명
xml	W-Pack 빌드 대상이 될 화면 파일의 확장자를 설정. 값이 지정되지 않은 경우, "xml"로 동작.

26.2.1.3. destExtension

기본 값	설명
js	W-Pack 빌드 결과물이 될 파일의 확장자를 설정. 값이 지정되지 않은 경우, "js"로 동작.

26.2.1.4. destRoot

기본 값	설명
wpack	W-Pack 빌드 결과물이 생성될 폴더 이름을 지정. 값이 지정되지 않은 경우 "_wpack_"을 사용.

26.2.1.5. babelDestRoot

기본 값	설명
wpackbabel	W-Pack 빌드 시 babel을 적용한 결과물이 생성될 폴더 이름을 지정. babel 적용 시 오버헤드가 발생하므로 ES6를 지원하지 않는 브라우저 만 babel을 적용하도록 하기 위해 만든 속성임. 아래의 loadingMode 속성과 함께 사용.

26.2.1.6. loadingMode

기본 값	설명
wpack	W-Pack 파일을 로드할 때 참고할 경로를 지정. "wpack"(기본 값) : destRoot로 설정한 경로를 사용. "babel" : babelDestRoot로 설정한 경로를 사용. "auto" : IE11은 babelDestRoot, 그 외는 destRoot를 사용.

26.2.1.7. common

기본 값	설명
false	W-Pack에서 공통 변수를 자동으로 치환하는 기능을 사용할지 여부를 지정. 이 기능을 사용하면 메모리 사용량을 줄일 수 있습니다. (사용 예) com.func1(a,b) → com.func1(\$p,a,b)

26.2.1.8. name

기본 값	설명
	W-Pack에서 공통 변수를 자동으로 치환해주는 기능 사용 시 치환할 대상이 되는 변수 이름을 지정. (사용 예) 공통 변수명이 com, com2, com3인 경우, name="com,com2,com3"으로 설정.

26.2.1.9. commonPath

기본 값	설명
	W-Pack에서 공통 변수를 자동으로 치환해주는 기능을 실제로 적용할 JS 파일들의 경로를 등록. 여기에 등록된 JS 파일은 위에 설정한 destRoot를 앞에 붙인 상태로 경로로 호출됨.

26.2.1.9.1. path

기본 값	설명
	JS 파일의 경로를 등록. 개별 파일 이름이나 특정 폴더를 등록 가능. (사용 예)

기본 값	설명
	• 개별 파일 - <code>/com/js/commonGlobal.js</code> • 폴더 전체 - <code>/com/js/</code>

26.2.2. strictMode

26.2.2.1. strictMode

기본 값	설명
true	WFrame을 사용하는 경우 자신의 프레임 영역을 벗어나는 자원들을 암시적으로 접근하는 것을 차단. "true" 설정 시 모든 화면을 눈에 보이지 않는 WFrame으로 감쌀 수 있음. WFrame 내부 화면과 WFrame이 없는 화면의 코딩 방식을 통일하기 위해 도입한 설정. 즉 WFrame 내부 페이지와 일반 페이지의 경계를 없애기 위한 설정. 〈비고〉 SP3부터는 "true"로 설정할 것을 권장.

26.2.2.2. scope variable

기본 값	설명
mf	strict mode에서 사용할 TOP 스코프의 scope variable 명.

26.2.2.3. scriptPrecedence

기본 값	설명
true	initScript, onpageload, postScript의 이벤트 순서를 지정. "true"일 경우 아래의 순서로 실행. 1. initScript 2. onpageload 3. postScript 〈비고〉 SP3부터는 "true"로 설정할 것을 권장.

26.2.3. page (SP3) / SPA (SP2)

26.2.3.1. variable

기본 값	설명
scwin	- SP2: SPA로 페이지 이동 시 자동으로 삭제할 객체를 string 형태로 등록. - SP3 이후: 페이지에서 사용할 변수명을 정의.

26.2.3.2. clone

기본 값	설명
	SP3 페이지 구조에서 외부 JS 로딩 시 page scope로 감쌀 변수를 지정.

26.2.3.3. spa

기본 값	설명
	페이지 이동 시 브라우저를 갱신하지 않고 body 영역만 새로 그려서 마치 페이지가 실제로 이동된 것처럼 보이게 하는 기능. 즉, 한번 로딩된 WebSquare 엔진을 재사용하여 시스템 전체가 한 페이지에서 동작. 기존 WebSquare 페이지 표시 방법에 비해 적은 작업을 진행하므로 엔진이 로딩되고 생성되는 시간을 절약할 수 있음.

26.2.4. welcome-file

기본 값	설명
	w2xPath가 지정되지 않았을 때 기본적으로 표시할 파일의 경로.

26.2.5. clearMemory

26.2.5.1. clearMemory

기본 값	설명
false	<code>onbeforeunload</code> 이벤트 발생 시 메모리 leak 방지 코드를 실행 여부. IFrame 사용 시 "true" 권장.

26.2.5.2. userFunction

기본 값	설명
	clearMemory 기능에서 사용자 공통 clearFunction을 설정. clearMemory 처리 전에 발생.

26.2.5.3. userAfterFunction

기본 값	설명
	개발자가 글로벌 영역에 직접 정의한 변수 및 객체들은 직접 정리해야 함. <code>WebSquare.clearMemory()</code> 가 호출된 이후 <code>userAfterFunction</code> 에 정의된 함수가 호출됨.

26.2.6. Process Message

26.2.6.1. processMsgURL

기본 값	설명
	상태 메시지로 표시할 HTML 파일의 URL.

26.2.6.2. processMsgHeight

기본 값	설명
81	상태 메시지 창의 높이.

26.2.6.3. processMsgWidth

기본 값	설명
295	상태 메시지 창의 너비

26.2.6.4. useProcessMsgBackground

기본 값	설명
true	프로세스 메시지 창의 모달의 배경색을 사용할지를 설정. • true : 배경색 지정 가능. • false : 배경색 지정할 수 없음.

26.2.6.5. processMsgBackground

기본 값	설명
#112233	useProcessMsgBackground 값이 true인 경우, ProcessMessage의 배경색.

26.2.6.6. processMsgType

기본 값	설명
iframe	메시지 창을 div로 출력할지, iframe형태로 출력할지를 결정. • wframe • iframe

26.2.6.7. processMsgFrameURL

기본 값	설명
message/processMsg.xml	processMsgType="wframe"인 경우 출력하는 화면의 URL을 설정.

26.2.7. date

26.2.7.1. maxYear

기본 값	설명
	가장 큰 년도 값.

26.2.7.2. minYear

기본 값	설명
	가장 적은 년도 값.

26.2.7.3. enable

기본 값	설명
true	<code>\$p.getCurrentServerDate</code> 호출시 서버 날짜를 조회여부. <code>true</code>: 서버 날짜 조회. <code>false</code>: 브라우저 Client 날짜 조회.

26.2.7.4. autoupdateinterval

기본 값	설명
3600	<code>\$p.getCurrentServerDate()</code> 함수를 호출하는 시간 단위. 정해진 시간 내에 <code>\$p.getCurrentServerDate()</code> 함수가 반복 호출될 경우 "처음 호출 시 서버 시간 + 이후 호출까지 흘러간 시간"이 반환됨. 서버 요청을 최소화하여 성능을 개선하기 위한 설정. 단, 서버 시간과 클라이언트 시간의 간격 차이가 비정상임을 인지할 경우 이 설정값을 무시하고 네트워크 요청을 보내 서버 시간을 조회.

26.2.8. pageFaultMsg

26.2.8.1. pageFaultMsg

기본 값	설명
	w2xPath에 설정되어 있는 XML 파일을 읽다가 에러가 발생할 경우 오류 메시지 표시 여부. false로 설정되어 있으면 메시지를 표시하지 않음.

26.2.9. cloneTab

26.2.9.1. cloneTab

기본 값	설명
	Ctrl + K 단축키 동작 허용 여부. <code>"true"</code>: 단축키 동작 허용 <code>"false"</code> (기본 값): 단축키 동작 차단. Ctrl + K 단축키 동작은 IE계열 브라우저에서 탭을 복제하는 동작을 수행.

26.2.10. editor

26.2.10.1. version

기본 값	설명
4.11.3	editor에 사용될 버전을 설정함. 값을 설정하지 않은 경우, IE계열 브라우저를 제외한 나머지 브라우저에서는 3.6.6.1버전으로 동작. [IE는 브라우저 버전 별로 상이]

26.2.11. fusionchart

26.2.11.1. version

기본 값	설명
	fusionChart에서 사용할 버전을 설정.

26.2.12. encoding

26.2.12.1. byteCheckEncoding

기본 값	설명
euc-kr	string의 Byte 길이를 셀 때의 기본 인코딩을 설정. 아래 함수에서 사용. - WebSquare.util.getStringByteSize - WebSquare.util.getStringByByteSize

26.2.13. encodeParameter

26.2.13.1. value

기본 값	설명
	w2xPath 문자열과 파일 경로를 쉽게 알아볼 수 없도록 경로를 인코딩.

26.2.13.2. fallback

기본 값	설명
	인코딩한 문자열과 기존 문자열을 모두 받을 것인지에 대한 속성.

26.2.13.3. onlyParam

기본 값	설명
	w2xPath 나 w2xHome 등 parameter로 넘기는 문자열만 인코딩.

26.2.13.4. key

기본 값	설명
	인코딩할 때 넣어주는 key 값. key에 따라 같은 문자열이라도 인코딩해서 나오는 문자열이 다름.

26.2.14. language

26.2.14.1. language

기본 값	설명
ko	웹스퀘어에서 기본적으로 나타나는 메시지의 언어.

26.2.14.2. useLanguagePack

기본 값	설명
false	다국어 지원을 위한 언어팩을 설정. true: 브라우저의 언어코드(navigator.language navigator.browserLanguage)를 읽어서 하위 노드에 설정된 언어팩을 자동으로 로딩. 한국어의 경우 언어코드를 ko-KR을 자동으로 ko로 변경.

26.2.14.3. multilanguage

기본 값	설명
ko	url 태그를 통해서 사용할 다국어의 언어 코드.

26.2.14.3.1. lang

기본 값	설명
ko	다국어 지원을 위한 언어팩.

26.2.14.3.2. file

기본 값	설명
/langpack/ko.js	다국어 지원을 위한 언어팩. import할 다국어 파일의 경로

26.2.15. pageURLprefix

26.2.15.1. pageURLprefix

기본 값	설명
	함수명 또는 string값을 지정. type이 function인 경우 함수명을, 그 외의 경우에는 string 값을 입력. type이 function인 ignoreW2XPath 속성에 따라 처리가 결정됨.

26.2.15.2. type

기본 값	설명
	웹스퀘어 화면 XML 파일을 읽어올 때 화면 경로에 특정 prefix를 붙이는 기능. - function - string 단, function인 경우에는 입력한 함수명이 실제로 정의되어 있어야 함.

26.2.15.3. ignoreW2XPath

기본 값	설명
	type이 function인 경우에 사용되는 속성. - true : 함수를 호출할 때 w2XPath값을 인자로 전달하여 호출하며, 그 함수의 반환값을 실제 XML 경로로 사용. (사용 예) xpath = func(url) - false : 함수를 인자 없이 호출하며, 실제 XML 경로는 해당 함수의 반환값에 w2XPath값을 더한 값이 사용됨.

기본 값	설명
	(사용 예) <code>xPath = func() + url</code>

26.2.16. escape

26.2.16.1. escape

기본 값	설명
	HTML escape 문자를 그대로 표시("true")하거나 HTML 태그로 인식("false")하는 escape 속성을 일괄 설정. escape 속성 적용 우선 순위 1. 컴포넌트 설정 2. config.xml 파일의 개별 컴포넌트에 대한 설정. 3. config.xml 파일의 일괄 설정. 4. 컴포넌트 기본 값 ("true")

26.2.17. requirejs

26.2.17.1. timeout

기본 값	설명
20	requirejs의 timeout 시간을 설정. 단위 : 초 (sec) 웹스퀘어 화면 파일을 로딩할 때 네트워크 시간이 설정 시간을 초과하면 timeout 처리됨.

26.3 Common

26.3.1. Project UDC

26.3.1.1. Project UDC

기본 값	설명
	공통 모듈로 사용되는 Project UDC를 등록.

26.3.1.1.1. name

기본 값	설명
	Project UDC에 접근할 때 사용하는 변수명.

26.3.1.1.2. src

기본 값	설명
	Project UDC의 경로.

26.3.2. MSA

26.3.2.1. MSA Server

기본 값	설명
	MSA 서비스 단위인 서버 이름과 URL을 설정.

26.3.2.1.1. name

기본 값	설명
	등록된 MSA 서버의 이름.

26.3.2.1.2. url

기본 값	설명
	등록된 MSA 서버의 URL.

26.3.2.1.3. w2xHome

기본 값	설명
	등록된 MSA 서버의 XML root 경로.

26.3.2.2. MSA UDC

기본 값	설명
	공통 모듈로 사용되는 MSA UDC를 등록.

26.3.2.2.1. MSA Server

기본 값	설명
	MSA UDC가 위치한 MSA 서버 이름.

26.3.2.2.2. name

기본 값	설명
	MSA UDC에 접근할 때 사용하는 변수명.

26.3.2.2.3. src

기본 값	설명
	MSA UDC의 경로.

26.3.3. UDC

26.3.3.1. Global UDC

기본 값	설명
	프로젝트에서 사용될 전역 UDC 컴포넌트를 등록.

26.3.3.1.1. name

기본 값	설명
	Component로 사용할 UDC의 이름.

26.3.3.1.2. src

기본 값	설명
	Component로 사용할 UDC의 경로.

26.3.3.1.3. type

기본 값	설명
page	Component로 사용할 UDC의 타입.

26.4 dev

26.4.1. debug

26.4.1.1. debug

기본 값	설명
true	엔진의 디버그 모드 작동 여부.

26.4.1.2. remoteConsole

기본 값	설명
false	WebSquare.logger.sendRemoteLog 함수를 통해 웹스퀘어 로그를 WAS에 저장.

26.4.1.3. console

기본 값	설명
false	개발자 도구의 Console 창에 로그를 출력.

26.4.1.4. errorConsole

기본 값	설명
false	<code>console="false", errorConsole="true"</code> 인 경우, 개발자 도구의 Console 창에 웹스퀘어 출력하지 않고 개발자 로그만 출력.

26.4.1.5. skipLog

기본 값	설명
	<code>true</code>인 경우, 웹스퀘어 로그 출력을 건너뛸. • <code>true</code> • <code>false</code>

26.4.1.6. skipCrossOrigin

기본 값	설명
	Iframe 접근 권한 관련 로그 출력을 건너뛸. • <code>true</code> • <code>false</code>

26.4.1.7. debugKey

기본 값	설명
	엔진 디버그 모드가 작동중 일때 로그를 보고 싶은 컴포넌트를 value로 설정.

26.4.1.8. debugMenu

기본 값	설명
use	Ctrl+ 오른쪽 클릭시 디버깅 메뉴 출력 여부를 설정. • <code>use</code> (기본 값) : 출력. • <code>block</code> : 출력하지 않음.

26.4.1.9. fiddle

기본 값	설명
	컨텍스트 메뉴에서 fiddle 항목 출력.

26.4.2. scriptLoading

26.4.2.1. merge

기본 값	설명
	WFrame 및 PageInherit 컴포넌트의 내부 스크립트 eval 방법. • false : 각각 개별로 eval. • true : 한번에 eval. (주의) 개발 환경에서만 사용해야 함.

26.4.3. exceptionHandler

26.4.3.1. exceptionHandler

기본 값	설명
websquare	웹스퀘어의 exception 핸들러 사용 여부. • websquare (기본 값): 웹스퀘어의 exception 핸들러를 사용. • system: 브라우저의 기본 설정을 적용.

26.5 Js

26.5.1. script

26.5.1.1. initScriptSupport

기본 값	설명
false	• true: 화면 로딩 후, 전역 스크립트 실행 전, <code>initScript</code> 안쪽의 text 노드 스크립트를 실행. (주의) • SP2: CDATA 블록을 사용할 수 있음. • SP3: <code>scriptPrecedence="true"</code>인 경우, <code>initScript</code> → <code>on pageload</code> → <code>postScript</code> 순서로 실행됨.

26.5.1.2. initScript

기본 값	설명
	<code>initScriptSupport</code> 가 <code>true</code> 일 때 실행할 스크립트.

26.5.1.3. postScriptSupport

기본 값	설명
	<code>initScript</code>처럼 <i>config.xml</i>에 설정하여 모든 페이지 스크립트의 마지막 부분에 들어가서 실행하도록 설정. (주의) - SP2: CDATA 블록을 사용할 수 있음. - SP3: <code>scriptPrecedence="true"</code>인 경우, <code>initScript</code> → <code>onpageload</code> → <code>postScript</code> 순서로 실행됨.

26.5.1.4. postScript

기본 값	설명
	<code>postScriptSupport</code> 가 <code>true</code> 일 때 실행할 스크립트.

26.5.1.5. external

기본 값	설명
	업무 공통 JavaScript를 추가하는 경우 사용.

26.5.1.5.1. module

기본 값	설명
	설정된 JavaScript 파일을 import. 업무 공통 JavaScript를 추가하는 경우 사용.

26.5.1.6. jquery

기본 값	설명
<code>externalJS/jquery/jquery-1.10.2.min.js</code>	원하는 버전의 jQuery url를 설정.

26.6 UI/UX

26.6.1. 반응형/적응형

26.6.1.1. media

기본 값	설명
	레이아웃 컴포넌트 파일과 적응형 컴포넌트에서 사용할 미디어 정보를 입력

26.6.1.1.1. name

기본 값	설명
	미디어 이름.

26.6.1.1.2. minWidth

기본 값	설명
	mediaQuery의 minWidth 값.

26.6.1.1.3. maxWidth

기본 값	설명
	mediaQuery의 maxWidth 값.

26.6.2. stylesheet

26.6.2.1. enable

기본 값	설명
	웹스퀘어 기본 스타일시트 파일 (stylesheet.css)을 사용할지에 대한 여부.

26.6.2.2. import

기본 값	설명
link	웹스퀘어 기본 스타일시트 파일(stylesheet.css)을 <link> 방식으로 삽입할지 <style> 블록 내에 inline으로 삽입할 지를 결정. • link : <link> 방식으로 삽입하며 stylesheet_ext.css 파일을 사용. • 그 외에는 stylesheet.css 파일을 사용.

26.6.2.3. earlyImportList

기본 값	설명
	화면이 렌더링되기 전에 미리 로딩할 css들을 정의. css가 늦게 적용되어 화면이 깜빡거리는 현상을 회피하기 위해 만든 설정. 여러 개의 파일을 설정할 경우 ","로 구분. 현재는 사용하지 않음.

26.6.3. mobile

26.6.3.1. preventTouch

기본 값	설명
false	showModal시 touchMove기능을 방지. • true • false

26.6.3.2. hideAddressBar

기본 값	설명
	모바일에서 주소창을 감추기 위한 설정. • true : document.body.scrollTop = 1 로 설정. • false

26.6.3.3. bindMobileMouseEvent

기본 값	설명
	모바일에서 블루투스 마우스를 쓸 때 WFrame popup이나 tab이 클릭이 제대로 되지 않는 현상을 해결하기 위한 속성.

기본 값	설명
	블루투스 마우스를 사용해야하는 사이트에서 사용을 권장. • <code>true</code> • <code>false</code>

26.6.4. html

26.6.4.1. postDrawMode

기본 값	설명
<code>asynchronous</code>	화면을 그린 다음 이벤트를 추가하고 사용자스크립트를 수행하는 과정을 <code>setTimeout</code>을 통해 비동기로 호출할지("<code>asynchronous</code>"), 동기 호출할지("<code>synchronous</code>")를 설정.

26.6.4.2. parseHtml

기본 값	설명
<code>true</code>	웹스퀘어 XML이 HTML 태그를 포함할 경우, 이를 파싱할지 여부. • <code>true</code> • <code>false</code> : 웹스퀘어 컴포넌트만 파싱하고 나머지는 전부 무시.

26.6.4.3. emptyTag

기본 값	설명
<code>area,base,basefont,br,col,frame,hr,img,input,link,meta,param</code>	<code>closing</code> 이 필요없는 태그를 등록.

26.6.4.4. visibleHelper

기본 값	설명
	웹스퀘어 렌더링 시 추가 작업이 필요한 경우 <code>targetHandler</code>에 처리할 함수를 지정. 함수의 인자로 <code>html string</code>이 전달됨.

26.6.5. tooltip

26.6.5.1. hideTooltipDisabledComp

기본 값	설명
	disabled 상태의 컴포넌트인 경우, 툴팁을 표시하지 않음. • <code>true</code> • <code>false</code>

26.6.6. itemTable

26.6.6.1. allValue

기본 값	설명
all	itemTable의 "전체 선택"에 대한 value. • 관련 컴포넌트 ◦ SelectBox ◦ AutoComplete ◦ CheckBox

26.7 Comm

26.7.1. submission

26.7.1.1. processMSg

기본 값	설명
	submission시 표시되는 상태창에 표시할 메시지. 이 설정은 <code>processMsg</code> 속성을 별도로 지정하지 않은 모든 submission에 적용됨. 특정 submission에서 상태창을 표시하지 않도록 할 수 있으므로 주의해서 사용.

26.7.1.2. hideProcessMsg

기본 값	설명
	submission시 나타나는 <code>processMsg</code> 를 숨기는 기능.

26.7.1.3. errorHandler

기본 값	설명
	submission 노드의 <code>errorHandler</code> 속성의 기본값. 이 설정은 <code>errorHandler</code> 속성을 별도로 지정하지 않은 모든 submission에 적용됨. 모든 submission에 영향을 주기 때문에 사용시 주의를 요함.

26.7.1.4. showSubmissionTime

기본 값	설명
true	submission의 시작 시간과 끝난 시간을 '로그 보기' 창에 표시.

26.7.1.5. customHandler

기본 값	설명
	<code>executeSubmission()</code> 실행 시에 proworks와 연계된 <code>proworksSOA</code> PHandler를 사용하거나, 사용자가 직접 작성한 handler 형태로 submission이 실행되도록 설정.

26.7.1.6. action

기본 값	설명
	Instance Data를 submit 할 서버측 URI.

26.7.1.7. mode

기본 값	설명
	서버와의 통신 방식. <code>asynchronous</code> : 비동기식. <code>synchronous</code> : 동기식.

26.7.1.8. method

기본 값	설명
post	• <code>get</code> : 파라메타를 url에 붙이는 방식. (HTML과 동일) • <code>post</code> : 파라메타를 body 구간에 담는 방식. (HTML과 동일) • <code>form-data-post</code> : 파일 업로드용 요청.

26.7.1.9. mediatype

기본 값	설명
	• <code>application/x-www-form-urlencoded</code> : 웹 form 방식. (HTML 방식) • <code>application/json</code> : json 방식. • <code>application/xml</code> : XML 방식. • <code>text/xml</code> : xml방식 (application/xml 방식과의 차이는 http://stackoverflow.com/questions/4832357 참조.)

26.7.1.10. timeout

기본 값	설명
0	submission 요청 후 timeout 시간. 이 시간이 초과해도 응답이 오지 않는 경우 error로 처리. (단위: ms)

26.7.1.11. encoding

기본 값	설명
utf-8	서버 측 encoding 타입을 설정. • <code>euc-kr</code> • <code>utf-16</code> • <code>utf-8</code>

26.7.1.12. preSubmitFunction

기본 값	설명
presubmit	submission 시작 전에 공통으로 호출할 함수. 지정된 함수는 submission 객체를 첫번째 인자로 받음.

26.7.1.13. callbackSubmitFunction

기본 값	설명
	submission 종료 후에 공통으로 호출할 함수. 지정된 함수는 submission 객체와 submission 결과가 담겨 있는 객체를 인자로 받음.

26.7.1.14. submitErrorHandler

기본 값	설명
	submission에서 HTTP 에러(상태값이 300이상이거나 200 미만인 경우)가 발생할 경우, 추가 작업을 위해 공통으로 호출할 함수.

26.7.1.15. submitErrorHandlerMode

기본 값	설명
	submission error가 발생할 경우, submitErrorHandler에 등록된 함수 및 이벤트를 호출하는 방법을 지정. • "only" : 등록된 함수만 호출. • "after" : 등록된 이벤트를 호출한 후에 함수를 호출. • "before" : 등록된 함수를 호출한 후에 이벤트를 호출.

26.7.1.16. externalHandler

기본 값	설명
	submission에서 data를 전송하기 전, 암호화(외부 API)된 값으로 변환해서 서버에서 전송할 수 있도록 외부 API를 설정.

26.7.1.17. externalResponseHandler

기본 값	설명
	submission에서 data를 전송받기 전, 복호화(외부 API)된 값으로 변환해서 서버로부터 값을 전송받을 수 있도록 외부 API를 설정.

26.7.1.18. requestID

기본 값	설명
	requestID가 지정되어 있는 경우, requestHeader의 mediatype 값을 설정.

26.7.1.19. makeGlobalObject

기본 값	설명
	submission 객체를 window 전역 객체로의 노출 여부. • true: 해당 객체들을 전역 변수로 접근. (SP2 전용)

26.7.1.20. requestHeaderFunction

기본 값	설명
	request Header에 사용자 customHeader를 설정할수 있는 사용자 정의 함수.

26.7.1.21. abortTrigger

기본 값	설명
	Submission 중 사용자가 직접 통신을 중지 시킬수 있는 창을 제공. • true • false

26.7.1.22. useModal

기본 값	설명
	Submission 중에 popup이 아니라 modal을 사용.

26.7.1.23. customHeader

기본 값	설명
true	• true • false : customHeader="false"인 경우, submissionId를 설정하지 않음.

26.7.1.24. requestJSONHandler

기본 값	설명
	response로 온 데이터를 조작하는 함수를 지정. 함수 인자로 responseJSON을 받음.

26.7.2. ajax

26.7.2.1. externalHandler

기본 값	설명
	ajax통신 시 외부 API를 정의하는 externalHandler을 설정.

26.7.3. workflow

26.7.3.1. processMSg

기본 값	설명
	workflow 실행 시 적용할 processMsg 속성 값.

26.7.3.2. makeGlobalObject

기본 값	설명
true	workflow 객체를 window 전역 객체로 노출. • true : 해당 객체에 전역 변수로 접근 가능. • false

26.7.4. hybrid

26.7.4.1. submissionURL

기본 값	설명
	submission의 경로(action)을 변경하기 위한 설정. 이 값을 설정할 경우, 설정된 값에 submission의 action 값을 더한 값

기본 값	설명
	이 실제 submission호출 경로로 사용됨. pageURLPrefix와 유사한 기능. hybrid에서 경로 호출 시 action의 앞에 특수한 path를 추가해서 넣어야 하는 경우가 생겨서 만들어진 설정. (사용 예) /WebSquare/hybrid/submissionURL/@value="/aaa", submission.action="/temp/aa.jsp"인 경우 최종 경로는 "/aaa/temp/aa.jsp" 가 됩니다.

26.8 Data

26.8.1. dataList

26.8.1.1. removeDummyRowStatus

기본 값	설명
true	row의 상태값이 C에서 E로 바뀐 데이터(removeRow로 삭제 한 데이터)의 관리 여부를 설정. • true • false

26.8.1.2. removeDataMatch

기본 값	설명
true	아래 함수가 반환하는 XML 구조를 별도의 구조로 분리 여부. • getModifiedXML() • getDeleteXML() • getVoidXML() • getOnlyDeleteXML() 설정 값 • true : 하위 호환을 위해 설정. • false

26.8.1.3. decreaseGetRangeXML

기본 값	설명
	<code>getRangeXML(from, to);</code> 호출 시 to - 1 값이 아닌 to 값까지 반환함. • <code>true</code> • <code>false</code>

26.8.1.4. excelErrorHandler

기본 값	설명
	excel 다운 시 에러 발생 시 실행되는 함수를 지정.

26.8.2. dataMap

26.8.2.1. jsonArray

기본 값	설명
	jsonArray 형태의 dataMap 지원 여부. jsonArray 형태로 dataMap을 설정할 수 있고 jsonArray 형태로 반환 됨. • <code>true</code> • <code>false</code>

26.9 Resource

26.9.1. cache

26.9.1.1. cache

기본 값	설명
nocache	개발 환경에서 캐싱을 비활성화 시키기 위해서 random url을 생성하는 로직을 추가할지 설정. XML 파일을 서버에서 읽어들이기 때 이 설정을 참고하여 캐싱 여부를 결정.

26.9.1.2. postfix

기본 값	설명
day	캐싱 제어를 위한 URL을 생성하기 위한 속성. month, week, day 또는 일련의 문자열을 설정 가능.

26.9.1.3. scriptPostfix

기본 값	설명
	외부 JS의 cache 정책을 <i>config.xml</i> 의 environment 설정을 따르도록 설정. 외부 JS는 기본적으로 environment를 따라가는 것이 올바른 동작이지만, 이전에는 engineCache 설정을 따라가고 있었음. SP2 이후에서는 "environment"로 설정할 것을 권장.

26.9.2. engineCache

26.9.2.1. enable

기본 값	설명
true	웹스웨어 엔진 캐싱 여부를 설정. - true - false

26.9.2.2. compression

기본 값	설명
true	엔진을 gzip으로 압축하여 사용할 것인지 설정. - true - false

26.9.2.3. postfix

기본 값	설명
month	enable true 설정 시 postfix 속성을 참고하여 postfix를 추가.

26.9.3. engineLoadingModule

26.9.3.1. ie

기본 값	설명
0	bootlodaer에서 나머지 엔진을 로딩하는 방법을 설정. 0 : 기존 방식 (All Download) 1 : 60K분할 다운로드 + 통합 Eval 2 : 파일별 다운로드 + 통합 Eval 3 : 파일별 다운로드 + 개별 Eval

26.9.3.2. chrome

기본 값	설명
0	bootlodaer에서 나머지 엔진을 로딩하는 방법을 설정. 0 : 기존 방식 (All Download) 1 : 60K분할 다운로드 + 통합 Eval 2 : 파일별 다운로드 + 통합 Eval 3 : 파일별 다운로드 + 개별 Eval

26.9.3.3. moz

기본 값	설명
0	bootlodaer에서 나머지 엔진을 로딩하는 방법을 설정. 0 : 기존 방식 (All Download) 1 : 60K분할 다운로드 + 통합 Eval 2 : 파일별 다운로드 + 통합 Eval 3 : 파일별 다운로드 + 개별 Eval

26.9.3.4. safari

기본 값	설명
0	bootlodaer에서 나머지 엔진을 로딩하는 방법을 설정. 0 : 기존 방식 (All Download) 1 : 60K분할 다운로드 + 통합 Eval 2 : 파일별 다운로드 + 통합 Eval 3 : 파일별 다운로드 + 개별 Eval

26.9.3.5. opera

기본 값	설명
0	bootlodaer에서 나머지 엔진을 로딩하는 방법을 설정. 0 : 기존 방식 (All Download) 1 : 60K분할 다운로드 + 통합 Eval 2 : 파일별 다운로드 + 통합 Eval 3 : 파일별 다운로드 + 개별 Eval

26.9.3.6. android

기본 값	설명
0	bootlodaer에서 나머지 엔진을 로딩하는 방법을 설정. 0 : 기존 방식 (All Download) 1 : 60K분할 다운로드 + 통합 Eval 2 : 파일별 다운로드 + 통합 Eval 3 : 파일별 다운로드 + 개별 Eval

26.9.3.7. iphone

기본 값	설명
0	bootlodaer에서 나머지 엔진을 로딩하는 방법을 설정. 0 : 기존 방식 (All Download) 1 : 60K분할 다운로드 + 통합 Eval 2 : 파일별 다운로드 + 통합 Eval 3 : 파일별 다운로드 + 개별 Eval

26.9.4. userAgentPattern

26.9.4.1. xPathParser

기본 값	설명
Edge Trident MSIE	브라우저에서 제공하는 XML 및 XPath 구현을 사용하지 않을 userAgent 패턴을 등록. 예시: Opera 11.60, 11.61 등

26.9.4.2. serializer

기본 값	설명
	브라우저에서 제공하는 XML Serializer를 사용하지 않을 userAgent 패턴을 등록. ("Galaxy S2"의 일부 모델과 같이 브라우저에서 제공하는 XML Serializer 사용 시에 오류가 발생하는 스마트폰의 userAgent값을 설정)

26.9.4.3. URLEncoder

기본 값	설명
	브라우저에서 URLEncoding을 자동처리하는 userAgent 패턴을 등록.

26.10 notRecommended

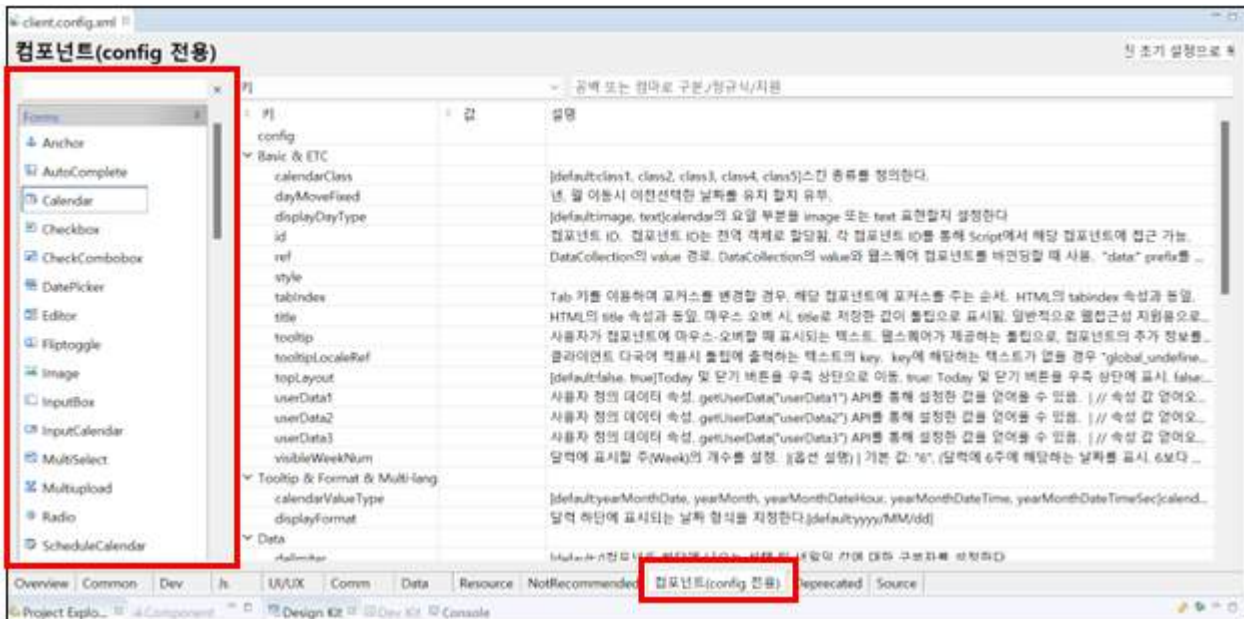
26.10.1. notRecommended

26.10.1.1. warningSync

기본 값	설명
true	submission에 sync를 사용하는 경우 경고를 표시. 설정 안 된 경우 false를 기본값으로 사용. true로 설정하는 것을 권장.

26.11 컴포넌트 (config 전용)

각 컴포넌트에 적용할 내용을 설정할 수 있습니다.



26.11.1. Forms

26.11.1.1. autoComplete

26.11.1.1.1. overrideKey

기본 값	설명
	입력에 대한 동작을 처리하지 않을 키의 keyCdoe를 설정. 입력을 무시할 키의 keyCode 숫자값을 입력. 여러 개의 키를 등록하는 경우 ", " 구분자를 이용하여 keyCode 숫자값을 나열. 모든 키를 등록하는 경우, "all"을 설정.

26.11.1.2. checkcombobox

26.11.1.2.1. overrideKey

기본 값	설명
	입력에 대한 동작을 처리하지 않을 키의 keyCdoe를 설정. 입력을 무시할 키의 keyCode 숫자값을 입력. 여러 개의 키를 등록하는 경우 ", " 구분자를 이용하여 keyCode 숫자값을 나열. 모든 키를 등록하는 경우, "all"을 설정.

26.11.1.3. inputCalendar

26.11.1.3.1. calendarDisplayFormat[yearMonthDateHour]/@defaultValue

기본 값	설명
	inputCalendar의 displayFormat 속성과는 별도로 달력에 표시되는 날짜 자체의 displayFormat을 설정함.

26.11.1.3.2. calendarDisplayFormat[yearMonthDateTimeSec]/@defaultValue

기본 값	설명
	inputCalendar의 displayFormat 속성과는 별도로 달력에 표시되는 날짜 자체의 displayFormat을 설정함.

26.11.1.3.3. calendarDisplayFormat[yearMonthDateTime]/@defaultValue

기본 값	설명
	inputCalendar의 displayFormat 속성과는 별도로 달력에 표시되는 날짜 자체의 displayFormat을 설정함.

26.11.1.3.4. calendarDisplayFormat[yearMonthDate]/@defaultValue

기본 값	설명
	inputCalendar의 displayFormat 속성과는 별도로 달력에 표시되는 날짜 자체의 displayFormat을 설정함.

26.11.1.3.5. calendarDisplayFormat[yearMonth]/@defaultValue

기본 값	설명
	inputCalendar의 displayFormat 속성과는 별도로 달력에 표시되는 날짜 자체의 displayFormat을 설정함.

26.11.1.3.6. calendarDisplayFormat[year]/@defaultValue

기본 값	설명
	inputCalendar의 displayFormat 속성과는 별도로 달력에 표시되는 날짜 자체의 displayFormat을 설정함.

26.11.1.3.7. displayFormat[yearMonthDateHour]/@defaultValue

기본 값	설명
	input에 표현될 년월일에 대한 format의 default 값을 설정.

26.11.1.3.8. displayFormat[yearMonthDateTimeSec]/@defaultValue

기본 값	설명
	input에 표현될 년월일에 대한 format의 default 값을 설정.

26.11.1.3.9. displayFormat[yearMonthDateTimeTime]/@defaultValue

기본 값	설명
	input에 표현될 년월일에 대한 format의 default 값을 설정.

26.11.1.3.10. displayFormat[yearMonthDate]/@defaultValue

기본 값	설명
	input에 표현될 년월일에 대한 format의 default 값을 설정.

26.11.1.3.11. displayFormat[yearMonth]/@defaultValue

기본 값	설명
	input에 표현될 년월일에 대한 format의 default 값을 설정.

26.11.1.3.12. displayFormat[year]/@defaultValue

기본 값	설명
	input에 표현될 년월일에 대한 format의 default 값을 설정.

26.11.1.3.13. ioFormat[yearMonthDateHour]/@defaultValue

기본 값	설명
	사용자가 data를 입력하는 순서와 format을 맞춰줌. ('y', 'M', 'd', 'H', 'm' 문자만 허용)

26.11.1.3.14. ioFormat[yearMonthDateTimeSec]/@defaultValue

기본 값	설명
	사용자가 data를 입력하는 순서와 format을 맞춰줌. ('y', 'M', 'd', 'H', 'm' 문자만 허용)

26.11.1.3.15. ioFormat[yearMonthDateTime]/@defaultValue

기본 값	설명
	사용자가 data를 입력하는 순서와 format을 맞춰줌. ('y', 'M', 'd', 'H', 'm' 문자만 허용)

26.11.1.3.16. ioFormat[yearMonthDate]/@defaultValue

기본 값	설명
	사용자가 data를 입력하는 순서와 format을 맞춰줌. (<code>'y','M','d','H','m'</code> 문자만 허용)

26.11.1.3.17. ioFormat[yearMonth]/@defaultValue

기본 값	설명
	사용자가 data를 입력하는 순서와 format을 맞춰줌. (<code>'y','M','d','H','m'</code> 문자만 허용)

26.11.1.3.18. ioFormat[year]/@defaultValue

기본 값	설명
	사용자가 data를 입력하는 순서와 format을 맞춰줌. (<code>'y','M','d','H','m'</code> 문자만 허용)

26.11.1.3.19. placeholder[yearMonthDateHour]/@defaultValue

기본 값	설명
	HTML5의 placeholder 기능과 동일. 사용자에게 알려줘야 할 텍스트 (hint) 등을 명시.

26.11.1.3.20. placeholder[yearMonthDateTimeSec]/@defaultValue

기본 값	설명
	HTML5의 placeholder 기능과 동일. 사용자에게 알려줘야 할 텍스트 (hint) 등을 명시.

26.11.1.3.21. placeholder[yearMonthDateTime]/@defaultValue

기본 값	설명
	HTML5의 placeholder 기능과 동일. 사용자에게 알려줘야 할 텍스트 (hint) 등을 명시.

26.11.1.3.22. placeholder[yearMonthDate]/@defaultValue

기본 값	설명
	HTML5의 placeholder 기능과 동일. 사용자에게 알려줘야 할 텍스트 (hint) 등을 명시.

26.11.1.3.23. placeholder[yearMonth]/@defaultValue

기본 값	설명
	HTML5의 placeholder 기능과 동일. 사용자에게 알려줘야 할 텍스트 (hint) 등을 명시.

26.11.1.3.24. placeholder[year]/@defaultValue

기본 값	설명
	HTML5의 placeholder 기능과 동일. 사용자에게 알려줘야 할 텍스트 (hint) 등을 명시.

26.11.1.4. selectBox

26.11.1.4.1. mobileRenderType

기본 값	설명
	모바일에서 렌더링되는 레이아웃 구조를 지정.

26.11.1.4.2. overrideKey

기본 값	설명
	입력에 대한 동작을 처리하지 않을 키의 keyCdoe를 설정. 입력을 무시할 키의 keyCode 숫자값을 입력. 여러 개의 키를 등록하는 경우 "," 구분자를 이용하여 keyCode 숫자값을 나열. 모든 키를 등록하는 경우, "all"을 설정.

26.11.2. Grid

26.11.2.1. GridView

26.11.2.1.1. colIdToColIndex

기본 값	설명
	editMode 진입시 column id를 column index로 변환해서 계산. (하위호환용) • true • false

26.11.2.1.2. column/chooseOptionLabel

기본 값	설명
	column의 <code>chooseOptionLabel</code> 을 표시.

26.11.2.1.3. column/displayFormat

기본 값	설명
	column의 <code>displayFormat</code> 을 설정.

26.11.2.1.4. column/editType

기본 값	설명
<code>scriptLoading</code>	column의 <code>editType</code> 을 설정.

26.11.2.1.5. column/textAlign

기본 값	설명
	column의 <code>text-align</code> 을 설정.

26.11.2.1.6. column[calendar]/dateValidCheck

기본 값	설명
	calander에서 유효성 검사를 수행.

26.11.2.1.7. column[calendar]/ioFormat[yearMonthDateTimeSec]/@defaultValue

기본 값	설명
	<code>ioFormat</code> 이 "yearMonthDateTimeSec"인 경우 열의 기본 값인 <code>defaultValue</code> 를 설정.

26.11.2.1.8. column[calendar]/ioFormat[yearMonthDateTime]/@defaultValue

기본 값	설명
	<code>ioFormat</code> 이 "yearMonthDateTime"인 경우 열의 기본 값인 <code>defaultValue</code> 를 설정.

26.11.2.1.9. column[calendar]/ioFormat[yearMonthDate]/@defaultValue

기본 값	설명
	ioFormat이 "yearMonthDate"인 경우 열의 기본 값인 defaultValue를 설정.

26.11.2.1.10. column[calendar]/ioFormat[yearMonth]/@defaultValue

기본 값	설명
	ioFormat이 "yearMonth"인 경우 열의 기본 값인 defaultValue를 설정.

26.11.2.1.11. column[expression]/NaNFormat

기본 값	설명
	값 처리중 NaN 발생 시 다른 값으로 바꾸는 속성.

26.11.2.1.12. column[text]/dataType[euro]/customModelFormatter

기본 값	설명
	dataType이 euro일때 customModelFormatter 속성을 적용.

26.11.2.1.13. column[text]/dataType[euro]/customModelUnformatter

기본 값	설명
	dataType이 euro일때 customModelUnformatter 속성을 적용.

26.11.2.1.14. column[textarea]/keepCellClass

기본 값	설명
	inputType="textarea"인 경우 cell에 class를 추가하면 textarea tag에도 추가.

**비공개 속성**

keepCellClass는 현재 비공개 속성.

26.11.2.1.15. `dataType[date]/@displayFormat`

기본 값	설명
	<code>displayFormat</code> 을 설정.

26.11.2.1.16. `excel/download/errorHandler`

기본 값	설명
	excel 다운 시 에러 발생 시 실행되는 함수를 지정. <code>errorHandler</code> 에 정의된 함수의 인자로 <code>gridView id</code> 를 전달.

26.11.2.1.17. `getColumnVisible/@useRealColIndex`

기본 값	설명
	<code>getColumnVisible</code> API 사용 시 <code>multiRow</code> 일 때도 정상 동작 하도록 하는 속성.

26.11.2.1.18. `header/column/hideCustomFilter`

기본 값	설명
	checkbox에서 "사용자 필터" 를 숨김. <code>true</code> <code>false</code>

26.11.2.1.19. `header/column/useFilter`

기본 값	설명
	column의 <code>dataType</code> 에 따라 필터를 사용할 것인지 설정. <code>true</code> <code>false</code>

26.11.2.1.20. `mainZIndex`

기본 값	설명
10	main div의 <code>zIndex</code> 를 지정.

26.11.2.1.21. `useDataDragDropGuideLine`

기본 값	설명
<code>true</code>	<code>dragDrop</code> 시 가이드라인 사용 여부.

26.11.2.1.22. xScrollZIndex

기본 값	설명
20	xScroll div의 zIndex를 지정.

26.11.2.1.23. yScrollZIndex

기본 값	설명
15	yScroll div의 zIndex를 지정.

26.11.3. Container

26.11.3.1. group

26.11.3.1.1. adaptive/trimText

기본 값	설명
	<code>true</code>: TableLayout에서 adaptive 적용시 WebSquare Studio 에서 ctrl+shift+f(줄바꿈) 실행 후 페이지 로딩 시 디자인을 유지.

26.11.3.1.2. applyScopeHeaders

기본 값	설명
	<code>strictmode="true"</code> 설정 후, 스튜디오에서 헤더 추가를 하면 header attribute에 등록되는 element id와 실제 id가 달라 추가한 속성. header attribute에 scope이 적용됨. 예시: group1 → mf_group1"

26.11.3.2. TabControl

26.11.3.2.1. overrideKey

기본 값	설명
	입력에 대한 동작을 처리하지 않을 키의 keyCode를 설정. 입력을 무시할 키의 keyCode 숫자값을 입력. 여러 개의 키를 등록하는 경우 "," 구분자를 이용하여 keyCode 숫자값을 나열. 모든 키를 등록하는 경우, <code>"all"</code>을 설정.

26.11.3.3. WindowContainer

26.11.3.3.1. getWindow/@searchTarget

기본 값	설명
windowId	getWindow() 함수 사용 시 window를 찾는 기준.

26.12 Deprecated

26.12.1. stylesheet

26.12.1.1. stylesheet

기본 값	설명
stylesheet_ext.css	value 값에 해당하는 css 파일을 읽어 적용. 파일은 websquare/skin/에 위치. • (empty) • stylesheet.css • stylesheet_ext.css • stylesheet_block.css • stylesheet_ext_block.css 현재 이 속성을 직접 컨트롤하는 경우는 없으며, import 설정에 따라 결정됨.

26.12.1.2. relativePath

기본 값	설명
	iframe의 w2xHome을 부모가 아닌 자신으로 설정해야 상대 경로가 올바르게 설정.

26.12.1.3. removeDefaultClass

기본 값	설명
true	• true : removeClass API 사용 시 컴포넌트의 class속성에 직접 정의된 class를 삭제.

26.12.2. docType

26.12.2.1. docType

기본 값	설명
standard	value 값에 따라 웹스퀘어 DocType 모드를 설정. • quirk • standard

26.12.3. mode

26.12.3.1. mode

기본 값	설명
production	• production : _alert를 실행시키지 않음.

26.12.4. debug

26.12.4.1. fixErrorLineNumber

기본 값	설명
	IE 구형 버전에서 script 오류 발생시 위치가 표시되지 않는 문제 보완 용으로 추가됨. (deprecated)

26.12.5. 선행 처리

26.12.5.1. systemPreProcessor

기본 값	설명
	웹스퀘어의 모든 화면이 로딩되기 전에 실행되는 스크립트. 웹스퀘어 엔진이 로딩 직전에 실행됨.

26.12.5.2. businessPreProcessor

기본 값	설명
	웹스퀘어의 모든 화면이 로딩되기 전에 실행되는 스크립트. 화면 렌더링이 되기 직전에 실행됨.

26.12.6. 컴포넌트

26.12.6.1. 웹스퀘어 컴포넌트

기본 값	설명
	웹스퀘어 컴포넌트를 추가로 정의하는 경우 사용. 하위노드에 설정된 JavaScript파일을 추가 uiplugin으로 사용. 특정 사이트를 위한 UI Component가 필요한 경우 사용. (deprecated)

26.12.6.1.1. id

기본 값	설명
	xml파일에서 사용되는 tag 이름.

26.12.6.1.2. namespace

기본 값	설명
	xml파일에서 사용되는 namespace.

26.12.6.1.3. className

기본 값	설명
	UI Plugin이 구현되어 있는 객체 이름.

26.12.6.1.4. value

기본 값	설명
	JavaScript 파일 위치.

26.12.7. UDC

26.12.7.1. SP1 UDC 스펙 지원

기본 값	설명
	초창기 버전의 udc 스펙을 사용해서 udc파일들을 작성한 경우 true로 설정. udc파일에서 <head> 밑에 <w2:type>UDC</w2:type> 노드가 없는 것이 초창기 버전의 udc 스펙임. (즉, 현재는 사용하지 않는 옵션.)

26.12.8. 데이터 변환

26.12.8.1. xml2jsonFunction

기본 값	설명
	입력된 XML을 JSON으로 변환하여 String으로 반환하는 함수를 설정.

26.12.8.2. json2xmlFunction

기본 값	설명
	JSON 형태의 데이터를 웹스퀘어에서 정의된 XML형태로 변환하여 반환하는 함수를 설정.

26.12.8.3. removeXmlNamespace

기본 값	설명
	네임스페이스 없이 XML Document를 XML String으로 반환하는 기능을 사용.

26.12.9. Instance

26.12.9.1. copyChildrenNodes

기본 값	설명
false	copyChildrenNodes 태그의 refresh 값이 true인 경우, WebSquare.ModelUtil.copyChildrenNodes를 실행하고나서 모델을 refresh 함.

26.12.9.2. removeInstanceNode

기본 값	설명
	removeInstanceNode태그의 값이 true인 경우, WebSquare.ModelUtil.removeInstanceNode를 실행하고 나서 모델을 refresh 함.

26.12.9.3. forcedValueSetting

기본 값	설명
true	value 값에 따라 XML 모델 인스턴스에 값을 넣으려고 할 때에 대상 인스턴스가 없을 경우 해당 instance를 만들지를 설정. • true : 새로운 인스턴스를 생성. • false : 값을 넣지 않음.

26.12.10. checkUpdate

26.12.10.1. enable

기본 값	설명
	XML 파일 갱신을 자동으로 확인. (head노드의 <w2:buildDate>2008-12-22 15:51:20</w2:buildDate> 노드 사용.)

26.12.10.2. mode

기본 값	설명
	client로 설정되면 XML을 직접 client로 가져와서 파싱한 다음 업데이트 여부를 판단. • server : 서버 URL에 XML path를 보내서 버전정보를 반환받아 업데이트 여부를 판단.

26.12.10.3. interval

기본 값	설명
	확인 간격 (단위: ms)

26.12.11. imageCache

26.12.11.1. enable

기본 값	설명
	이미지들을 JSP/Servlet을 이용하여 캐싱할 것인지를 설정. true로 설정하려면 캐싱을 위한 JSP/Servlet에서 이미지 파일에 접근 가능해야 함.

26.12.11.2. version

기본 값	설명
	캐싱 제어를 위한 URL을 생성. month, week, day 또는 일련의 문자열을 설정 가능.

26.12.11.3. serverUrl

기본 값	설명
	이미지 캐싱을 지원하는 서버 Url. 설정되지 않으면 기본값으로 resource.wq?url= 를 사용. JSP를 사용하는 경우에는 engine/servlet/image.jsp?url= 를 설정해야 함.

26.12.11.4. hostUrl

기본 값	설명
	이미지를 관리하는 서버가 있다면 포트값을 포함한 url을 지정.

26.12.12. localStorage

26.12.12.1. defer

기본 값	설명
	localStorage를 window.onload 이후시점에 생성할지(true) 이전시점에 설정할지(false)에 대한 설정입니다. defer의 기본값은 false입니다. IE9에서 팝업창을 메인으로 사용하는 구조 (빈 화면을 띄우고, 팝업창을 띄운다음 빈 화면을 닫는 구조) 팝업 위에 팝업을 띄우는 경우 localStorage를 onload이전에 생성할 경우 부모창과 자식창의 localStorage가 서로 독립적으로 생성되는 버그가 발생하여 추가한 옵션입니다. 2014년 이 속성이 추가됨.

26.12.13. SPA

26.12.13.1. onpageunload

기본 값	설명
	SPA로 페이지 이동 시 호출할 함수의 변수명을 string으로 등록. 주로 페이지를 이동한 뒤에 남은 리소스들을 처리하는 함수를 등록.

26.12.13.2. scriptCache

기본 값	설명
true	SPA로 페이지 이동 시 외부 모듈을 다시 로딩하고 실행할지 여부. • true : 한번 로딩된 외부 모듈은 다시 로딩하지 않으며 실행도 하지 않음.

26.12.13.3. autoReload

기본 값	설명
true	SPA로 페이지 이동 시 화면에 메모리 릭이 발생하는 웹스퀘어 컴포넌트(FusionChart 및 Editor)가 있을 경우 해당 컴포넌트 개수가 count 설정 값보다 많을 경우, 브라우저를 갱신.

26.12.13.4. count

기본 값	설명
50	화면에 그려진 Editor 및 FusionChart의 총 허용 개수.

26.12.13.5. editorCount

기본 값	설명
	count 설정을 editor 컴포넌트 개별로 설정.

26.12.13.6. fusionchartCount

기본 값	설명
	count 설정을 FusionChart 컴포넌트 개별로 설정.

26.12.13.7. userFunction

기본 값	설명
	화면 이동할 때 경로를 변경할 수 있는 사용자 함수를 설정. 인자로 기존 경로를 받음.

26.12.14. errorPage

26.12.14.1. errorPage

기본 값	설명
	errorPage에 대한 처리를 지정.

26.12.14.2. E0

기본 값	설명
	HTTP ERROR 00이 리턴되었을 때의 errorPage를 지정.

26.12.14.3. E400

기본 값	설명
	HTTP ERROR 400이 리턴되었을 때의 errorPage를 지정.

26.12.14.4. E401

기본 값	설명
	HTTP ERROR 401이 리턴되었을 때의 errorPage를 지정.

26.12.14.5. E402

기본 값	설명
	HTTP ERROR 402가 리턴되었을 때의 errorPage를 지정.

26.12.14.6. E403

기본 값	설명
	HTTP ERROR 403이 리턴되었을 때의 errorPage를 지정.

26.12.14.7. E404

기본 값	설명
	HTTP ERROR 404가 리턴되었을 때의 errorPage를 지정.

26.12.14.8. E405

기본 값	설명
	HTTP ERROR 400가 리턴되었을 때의 errorPage를 지정.

26.12.14.9. E500

기본 값	설명
	HTTP ERROR 500이 리턴되었을 때의 errorPage를 지정.

26.12.14.10. E501

기본 값	설명
	HTTP ERROR 501이 리턴되었을 때의 errorPage를 지정.

26.12.14.11. E502

기본 값	설명
	HTTP ERROR 502가 리턴되었을 때의 errorPage를 지정.

26.12.14.12. E503

기본 값	설명
	HTTP ERROR 503이 리턴되었을 때의 errorPage를 지정.

26.12.14.13. E504

기본 값	설명
	HTTP ERROR 504가 리턴되었을 때의 errorPage를 지정.

26.12.14.14. Else

기본 값	설명
	설정하지 않은 HTTP ERROR 코드가 리턴되었을 때의 errorPage를 지정.

26.12.15. url

26.12.15.1. DocumentRoot

기본 값	설명
	w2xDocument 경로를 지정.

26.12.15.2. logMsgMobileURL

기본 값	설명
	logMsgMobile.html 대신 호출할 페이지를 지정.

26.12.15.3. logMsgUrl

기본 값	설명
	logMsg.html 대신 호출할 페이지를 지정.

26.12.15.4. debugMsgURL

기본 값	설명
	debugMsg.html 대신 호출할 페이지를 지정.

26.12.15.5. errorMsgURL

기본 값	설명
	errorMsg.html 대신 호출할 페이지를 지정.

26.12.15.6. viewSourceURL

기본 값	설명
	viewSource.html 대신 호출할 페이지를 지정.

26.12.15.7. viewCollectionURL

기본 값	설명
	viewCollection.html 대신 호출할 페이지를 지정.

26.12.15.8. viewInstanceURL

기본 값	설명
	viewInstance.html 대신 호출할 페이지를 지정.

26.12.15.9. fiddleURL

기본 값	설명
	fiddle.html 대신 호출할 페이지를 지정.

26.12.15.10. printRangeURL

기본 값	설명
	printRange.html 대신 호출할 페이지를 지정.

26.12.15.11. redirectUrl

기본 값	설명
	redirect할 url을 지정.

26.12.16. etc

26.12.16.1. processMsgTitle

기본 값	설명
	processMessage의 title을 지정. (웹접근성 관련)

26.12.16.2. parseChild

기본 값	설명
	하위 컴포넌트 또는 순수 html 태그를 포함하는 xml을 인자로 전달할 경우 dynamicCreate로 동적 생성.

26.12.16.3. scriptEncoding

기본 값	설명
	js의 인코딩 설정.

26.12.16.4. showLogPopup

기본 값	설명
	내부 테스트용 파일 설정. 테스트 완료 후 로그 팝업을 표시.

26.12.16.5. license

기본 값	설명
true	라이센스를 사용 여부. 외부 배포 엔진에서 false로 설정하면 정상 동작하지 않음.

26.12.16.6. requestHeaderFunction

기본 값	설명
	XML 요청 시 사용자 정의 request header를 설정. config 설정 후 전역에 함수를 등록해야 함.

26.12.16.7. preserveWhiteSpace

기본 값	설명
	IE에서 XML을 파싱할 때 preserveWhiteSpace 옵션 사용.

26.12.16.8. codebase

기본 값	설명
	InternetExplorer 6, 7에서 HTTP동시 연결 개수와 페이지 설정 변경을 위한 activeX의 codebase를 설정. 설정하지 않으면 engine/cab/WebSquareConfig.cab 를 기본값으로 사용. 버전이 변경되거나 배포 위치가 변경된 경우 codebase 속성에 URL을 추가해야 함.

26.12.16.9. globalExposure

기본 값	설명
	false : 전역으로 websquare 객체를 생성하는 것을 금지.

26.12.16.10. layerForceClose

기본 값	설명
	MouseWheelEvent 발생 시, layer들을 숨김.

26.12.16.11. blurAll

기본 값	설명
	Input 컴포넌트에 포커스가 있는 상태에서 휠 이벤트 발생하고 input 컴포넌트가 화면에서 안보이게 될 때 blur가 발생.

26.12.16.12. nextTabFunction

기본 값	설명
	input에서 autoFocus 기능 사용 시 nextTabId가 지정되어 있지 않은 경우 config.xml에 설정된 nextTabFunction 함수가 지정되었는지를 확인. 지정된 경우, 해당 함수를 호출.

26.12.16.13. loadErrorHandler

기본 값	설명
	w2xPath경로가 잘못된 경우, 실행할 사용자 지정 함수를 설정.

26.12.16.14. useAppConfig

기본 값	설명
	head 태그에서 config 설정을 허가. (내부 테스트용 속성)

26.12.16.15. domainFrameURL

기본 값	설명
	domainFrame.html 대신 호출할 URL을 설정.

26.12.16.16. imageURLPrefix

기본 값	설명
	별도의 이미지 서버가 존재하는 경우 이미지 처리용 prefix를 설정.

26.12.16.17. clearMemoryCloseChildPopup

기본 값	설명
	팝업을 IFrame으로 사용할 경우메모리릭 문제가 발생하여 clearMemory="true"인 경우 부모 창이 닫히면 popup도 자동으로 닫히는 처리가 SP2에 추가됨. 이 때 의도하지 않게 popup이 browser와 별도로 실행되는 경우에도 자동으로 닫히게 되었는데, 하위 호환을 고려하여 popup이 browser인 경우에는 config.xml의 특정 설정을 통해 부모 창이 닫히더라도 자식 창이 닫히지 않는 설정.

26.12.16.18. serverTimeURL

기본 값	설명
	serverTime 관련 API 호출 시 URL을 지정.

26.12.16.19. modal

기본 값	설명
	showModal시 touchMove 기능이 동작을 방지.

26.12.16.20. forcedValueSetting

기본 값	설명
	value 값에 따라 XML 모델 인스턴스에 값을 넣으려고 할 때 대상 인스턴스가 없을 경우 해당 instance 생성 여부를 지정. • true : 새로운 인스턴스를 생성. • false : 값을 넣지 않음.

26.12.16.21. parseHtml

기본 값	설명
	XML 내에 script 태그를 파싱해서 실행.

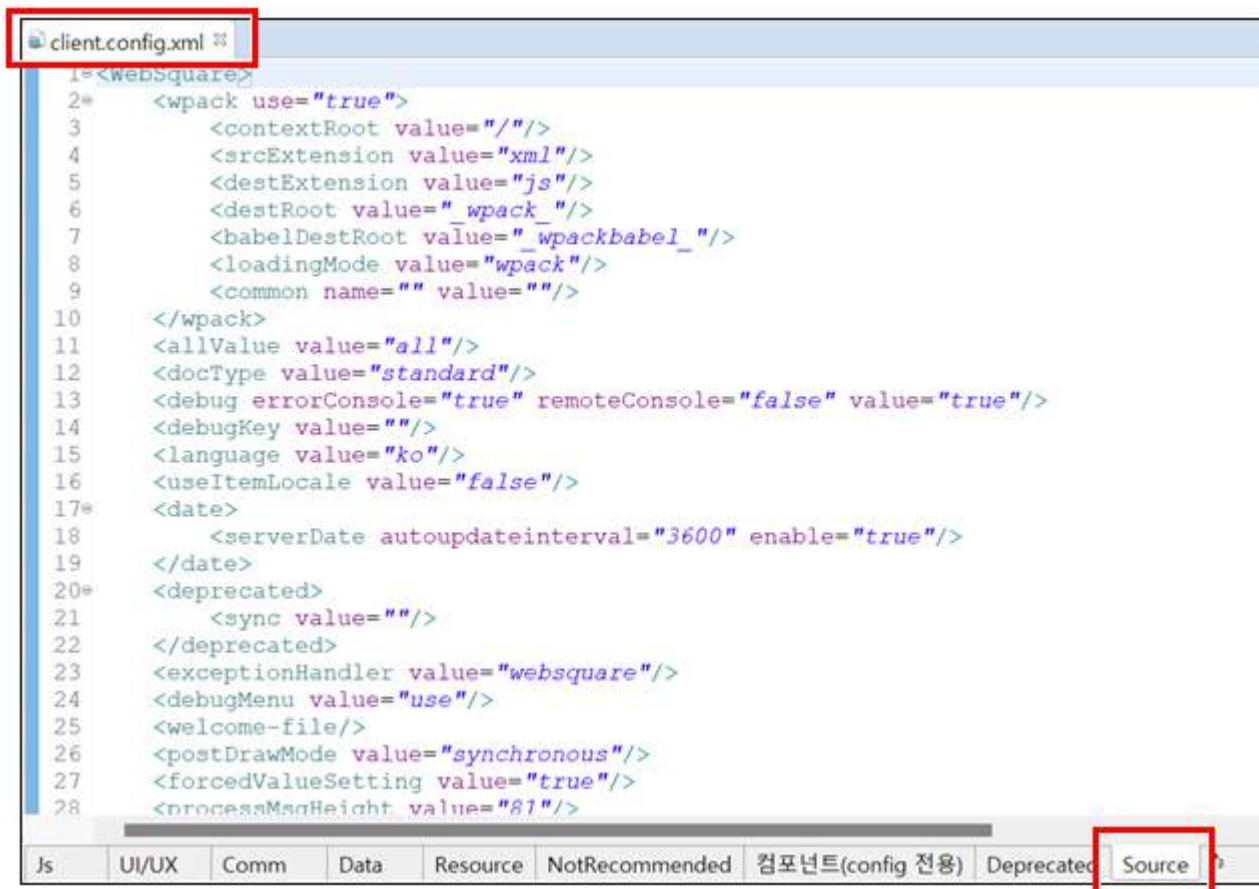
26.12.16.22. postDrawMode

기본 값	설명
	화면을 그린 다음 이벤트를 추가하고 사용자 스크립트를 수행하는 과정을 setTimeout을 통해 비동기로 호출할지("asynchronous"), 동기 로 호출할지("synchronous")를 설정.

26.13 Source

Source 탭은 전체 클라이언트 설정 내용을 XML 파일 형식으로 보여줍니다.

그림 26-3 클라이언트 설정

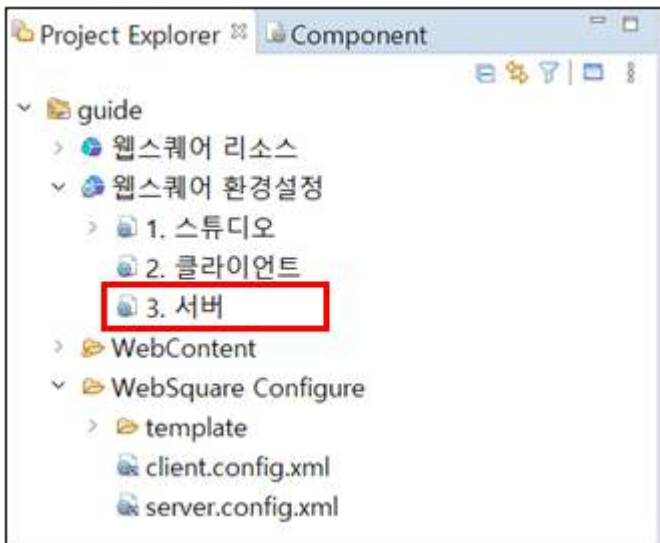


27. 서버 설정 (server.config.xml)

27.1 개요

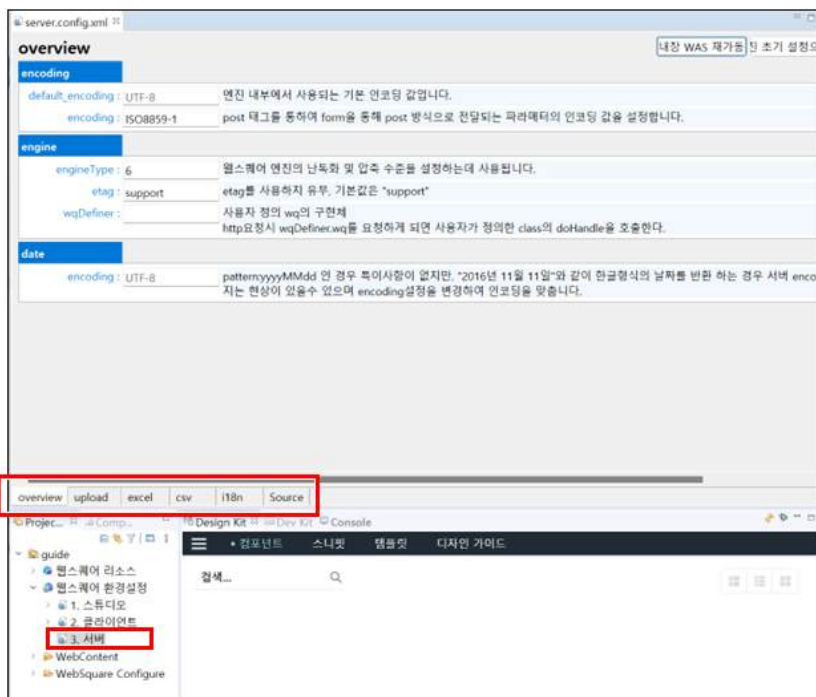
Project Explorer에서 서버 설정 파일을 열어 직접 설정할 수 있습니다.

그림 27-1 웹스퀘어 환경설정 - 서버



서버 설정 파일을 열고 하단의 각 탭을 클릭하여 해당 항목들을 설정할 수 있습니다.

그림 27-2 서버 설정 (server.config.xml)



주요 설정 항목은 아래와 같습니다.

주요 설정 항목

WebSquare 엔진 모듈 처리 설정
Excel, CSV 업로드/다운로드 설정
Framework 연계관련 adaptor 설정
다국어 처리 설정
WebSquare Hybrid 관련 설정

27.2 overview

27.2.1. encoding

27.2.1.1. default_encoding

기본 값	설명
UTF-8	엔진 내부에서 사용하는 인코딩 값.

27.2.1.2. encoding

기본 값	설명
UTF-8	post 태그를 통하여 form을 통해 post 방식으로 전달되는 파라미터의 인코딩 값.

27.2.2. engine

27.2.2.1. engineType

기본 값	설명
5	웹스퀘어 엔진의 난독화 및 압축 수준을 설정.

웹스퀘어 엔진의 난독화 및 압축 수준을 설정합니다. 값은 1~8까지 설정 가능합니다.

Table 27-1 난독화 및 압축 수준

값	구분	설명
1	기본 엔진	- 설정이 없는 경우 기본 값으로 기본 난독화 방식과 동일. - 엔진 크기: 1,126 KByte
2	기본 엔진	"WebSquare.core.getConfiguration"과 같이 함수마다 선언되어 있는 debug 정보 제거한 엔진. - 엔진 크기: 1,045 KByte
3	맵핑 엔진	- WebSquare를 _\$W라는 변수로 치환하여 크기를 축약한 엔진으로 debug 정보를 포함. - 엔진 크기: 1,055 KByte
4	맵핑 엔진	- WebSquare를 _\$W라는 변수로 치환하고 debug 정보를 제거한 엔진. - 엔진 크기: 974 KByte
5	맵핑 엔진	- WebSquare를 _라는 변수로 치환하여 크기를 축약한 엔진으로 debug 정보를 포함. 만일 _가 사용중이어서 오류가 발생하면 3번 엔진을 이용. - 엔진 크기: 1,040 KByte
6	맵핑 엔진	- WebSquare를 _\$W라는 변수로 치환하고 debug 정보를 제거한 엔진. 만일 _가 사용중이어서 오류가 발생하면 4번 엔진을 이용. - 엔진 크기: 959 KByte
7	맵핑 엔진	4번 엔진에서 WebSquare.exception.printStackTrace와 WebSquare.logger.printLog를 제거한 버전. 운영환경에서 로그 출력이 없어야 하는 경우 사용. - 엔진 크기: 879 KByte
8	맵핑 엔진	(_)(debug 정보 제거) (log출력 제거)



개발환경은 5를 권장하고, 운영환경은 6을 권장합니다.

w2xType이 포함된 URL을 이용하여 직접 지정할 수 있습니다.

1 <http://localhost:9999/websquare/websquare.html?w2xPath=/preview2.xml&w2xType=3>

Table 27-2 엔진 타입 별 로그

WebSquare Engine Type	로그
1	Default engine with debugging information.
2	Default engine without debugging information.
3	Remapping engine with debugging information.
4	Remapping engine without debugging information.
5	Short Remapping engine with debugging information.
6	Short Remapping engine without debugging information.
7	Remove logger and remapping engine.
8	Remove logger and short remapping engine.

engineType 별 소스는 아래와 같습니다.

Code 27-1 engineType: 1

```

1  WebSquare.Actions = {};
2  WebSquare.Actions.WebSquareAction = function(_1) {
3      ["WebSquare.Actions.WebSquareAction"];
4      try {
5          if (arguments.length == 0) {
6              return;
7          }
8          WebSquare.WebSquareObject.call(this, _1, true);
9      } catch (e) {
10         WebSquare.exception.printStackTrace(e);
11     }
12 };

```

Code 27-2 engineType: 2

```

1  WebSquare.Actions = {};
2  WebSquare.Actions.WebSquareAction = function(_1) {
3      try {
4          if (arguments.length == 0) {
5              return;
6          }
7          WebSquare.WebSquareObject.call(this, _1, true);
8      } catch (e) {
9          WebSquare.exception.printStackTrace(e);
10     }
11 };

```

Code 27-3 engineType: 3

```

1  _$W._E = {};
2  _$W._E.WebSquareAction = function(_1) {
3      ["WebSquare.Actions.WebSquareAction"];
4      try {
5          if (arguments.length == 0) {
6              return;
7          }
8          _$W.WebSquareObject.call(this, _1, true);
9      } catch (e) {
10         _$W.exception.printStackTrace(e);

```

```

11     }
12 };

```

Code 27-4 engineType: 4

```

1  _$. _E = {};
2  _$. _E.WebSquareAction = function(_1) {
3      try {
4          if (arguments.length == 0) {
5              return;
6          }
7          _$.WebSquareObject.call(this, _1, true);
8      } catch (e) {
9          _$.exception.printStackTrace(e);
10     }
11 };

```

Code 27-5 engineType: 5

```

1  _._E = {};
2  _._E.WebSquareAction = function(_1) {
3      ["WebSquare.Actions.WebSquareAction"];
4      try {
5          if (arguments.length == 0) {
6              return;
7          }
8          _._WebSquareObject.call(this, _1, true);
9      } catch (e) {
10         _._exception.printStackTrace(e);
11     }
12 };

```

Code 27-6 engineType: 6

```

1  _._E = {};
2  _._E.WebSquareAction = function(_1) {
3      try {
4          if (arguments.length == 0) {
5              return;
6          }
7          _._WebSquareObject.call(this, _1, true);
8      } catch (e) {
9          _._exception.printStackTrace(e);

```

```

10     }
11 };

```

Code 27-7 engineType: 7

```

1  _$.W._E = {};
2  _$.W._E.WebSquareAction = function(_1) {
3      try {
4          if (arguments.length == 0) {
5              return;
6          }
7          _$.W.WebSquareObject.call(this, _1, true);
8      } catch (e) {}
9  };

```

Code 27-8 engineType: 8

```

1  _._E = {};
2  _._E.WebSquareAction = function(_1) {
3      try {
4          if (arguments.length == 0) {
5              return;
6          }
7          _._WebSquareObject.call(this, _1, true);
8      } catch (e) {}
9  };

```

27.2.2.2. etag

기본 값	설명
support	etag 사용 유무.

27.2.2.3. wqDefiner

기본 값	설명
	사용자 정의 wq의 구현체. http 요청 시 wqDefiner.wq를 사용하면 사용자가 정의한 class의 do Handle을 호출.

27.2.3. date

27.2.3.1. encoding

기본 값	설명
UTF-8	"2016년 11월 11일"와 같은 한글 형식의 날짜를 반환 하는 경우 사용할 인코딩 값. 서버의 인코딩 설정에 따라 한글이 깨질 수 있음. 일반적인 pattern:yyyyMMdd 패턴인 경우 별도의 설정이 필요 없음.

27.3 upload

27.3.1. file upload

27.3.1.1. support

기본 값	설명
true	엔진 업로드 기능 사용 여부. false: upload.wq 및 imageUpload.wq 업로드 기능이 동작하지 않음.

27.3.1.2. useFileUpload

기본 값	설명
true	파일업로드 사용유무 false: upload.wq 요청이 동작하지 않음. 단, imageUpload.wq는 사용 가능.

27.3.1.3. encoding

기본 값	설명
UTF-8	업로드 하는 파일의 파일명 인코딩 방법.

27.3.1.4. baseDir

기본 값	설명
	파일이 올라갈 경로. (주의) 실제 서버에 존재하지 않는 경로를 baseDir 항목으로 설정할 경우, 에러가 발생.



5M 이상의 파일 업로드

파일 크기가 5M 이상인 경우,

- (1) Temp 파일은 **baseDir**로 지정한 폴더로 지정되고,
- (2) 업로드가 완료되면 **subDir**로 이동.

27.3.1.5. subDir

기본 값	설명
	하위 노드. 태그 이름은 사용자 정의가 가능.

Code 27-9 설정 예 (baseDir)

```

1 <websquare>
2   <upload>
3     <baseDir value="D:\upload\">
4       <subDir1 value="D:\upload\test1\" />
5     </baseDir>
6   </upload>
7 </websquare>

```



Upload 컴포넌트의 **subDir** 속성 값에 이 태그의 이름을 주어 세부 조정을 할 수 있습니다.

(예)

Upload 컴포넌트에 subDir 설정이 없다면 D:\upload\ 가 기본 경로가 됩니다 subDir 설정 값이 subDir1 이라면 D:\upload\test1\ 가 기본 경로로 설정됩니다.)

27.3.1.5.1. name

기본 값	설명
	Upload 컴포넌트의 baseDir 하위 경로를 지정하기 위한 이름.

27.3.1.5.2. path

기본 값	설명
	Upload 컴포넌트의 subDir 속성 값을 조정하기 위한 태그.

27.3.1.6. firstfilePostFix

기본 값	설명
false	파일명에 unique한 postfix를 추가.

27.3.1.7. duplicatedFilePostFix

기본 값	설명
	업로드 시 동일한 파일이 이미 존재할 경우, 설정한 값을 파일명 뒤에 추가되는 postfix로 사용.

27.3.1.8. sequence

기본 값	설명
false	파일명에 순차적인 순번을 추가. WAS가 재시작되면 순번이 초기화됨.

27.3.1.9. maxUploadSize

기본 값	설명
1048576	업로드 하는 파일의 최대 크기를 설정. (단위: byte) 설정하지 않을 경우 기본 값은 1Mbyte임

27.3.1.10. subDirMaxUpload

기본 값	설명
	업로드 하는 파일의 최대 크기를 설정. (단위: byte) 설정하지 않을 경우 기본 값은 1Mbyte.

27.3.1.10.1. name

기본 값	설명
	업로드 하는 파일명.

27.3.1.10.2. path

기본 값	설명
	업로드 하는 파일의 경로.

27.3.1.11. allowedExtension

기본 값	설명
gif, jpg, doc, xls, ppt, pdf, txt	업로드를 허용할 파일의 확장자

27.3.1.12. deniedExtension

기본 값	설명
exe	업로드를 허용하지 않는. 파일의 확장자

27.3.1.13. uploadMode

기본 값	설명
session	업로드 시 사용할 폴더명을 정의. - session (기본 값): session의 folderName 설정 값으로 폴더명을 정의. - random: websquare.xml 파일의 folderName 설정 값을 읽어 폴더명을 정의.

27.3.1.14. folderName

기본 값	설명
	업로드할 폴더명.

27.3.1.15. removeKeyPath

기본 값	설명
false	업로드 후 업로드 결과가 XML로 반환됨. 반환되는 결과 정보 중에 있는 키 노드에는 실제 경로를 포함. 해당 경로 정보의 삭제 여부. • true: 삭제

27.3.1.16. fileDefiner

기본 값	설명
	file명과 filePath를 정의 할 수 있는 인터페이스. 설정하지 않을 경우 WebSquare에서 지원하는 파일명과 파일 경로를 그대로 사용.

파일명과 파일 경로의 커스텀마이징이 필요할 경우, 아래와 같이 추상클래스에 정의되어 있는 method를 구현합니다. 이후, 구현한 class의 전체 package 경로를 **fileDefiner** 속성에 설정합니다.

Code 27-10 설정 예 (fileDefiner)

```

1 <websquare>
2   <upload>
3     <fileDefiner value="site.SiteUploadFileDefinerImpl" />
4   </upload>
5 </websquare>

```

Code 27-11 AbstractUploadFileDefiner 클래스

```

1 package websquare.upload.handl;
2 public abstract class AbstractUploadFileDefiner {
3
4     abstract public String getFileName( String clientFileName, String originalFileName )
5     throws Exception;
6
7     abstract public String getFilePath( String filePath ) throws Exception;
8
9 }

```

```
8 }
```

Code 27-12 구현 예시 (SiteUploadFileDefinerImpl)

```
1 package site;
2 import websquare.upload.handl.*;
3 public class SiteUploadFileDefinerImpl extends AbstractUploadFileDefiner {
4
5     public String getFileName( String clientFileName, String originalFileName ) throws
Exception {
6         // 파일명 처리
7         return originalFileName;
8     }
9
10    public String getFilePath( String filePath ) throws Exception {
11        // 경로명 처리
12        return filePath;
13    }
14 }
```

27.3.1.17. decrypt

기본 값	설명
	파일업로드시 DRM연계시 복호화를 지원하기 위한 인터페이스.

아래의 방법으로 복호화 연계를 구현할 수 있습니다.

1. 추상 클래스에 정의되어 있는 method를 구현합니다.
2. 구현한 class의 전체 package 경로를 **decrypt** 속성에 설정합니다. (필수)
3. 복호화에 사용되는 temp 디렉토리의 작업 경로를 **tempDir** 속성에 설정합니다. (필수)

Code 27-13 설정 예 (decrypt)

```
1 <websquare>
2     <upload>
3         <decrypt value="site.SiteDefinerImpl" />
4     </upload>
5 </websquare>
```

Code 27-14 AbstractUploaDecrypter 클래스

```

1 package websquare.upload.handl;
2 import java.io.*;
3 public abstract class AbstractUploaDecrypter {
4
5     abstract public File decrypt( File file ) throws Exception;
6
7 }

```

Code 27-15 구현 예 (SiteDefinerImpl)

```

1 package site;
2 import websquare.upload.handl;
3 public class SiteDefinerImpl extends AbstractUploaDecrypter {
4
5     public String decrypt( File file ) throws Exception;
6         // 복호화 처리
7     return file;
8 }
9 }

```

27.3.1.18. decryptTempDir

기본 값	설명
	복호화를 위한 temp 디렉토리 작업 경로를 설정.

27.3.2. image upload

27.3.2.1. baseDir

기본 값	설명
	파일이 올라갈 경로.

27.3.2.2. multiSubDir

기본 값	설명
	baseDir 하위의 경로. 지정할 경우, /websquare/upload/imgUpload/baseDir/subDir/@value 설정값은 무시됨.

27.3.2.3. verifyHeader

기본 값	설명
true	유효한 이미지 확장자(gif, jpg, jpeg, jpe, ico, png) 검증

27.3.2.4. allowedExtension

기본 값	설명
	업로드를 허용할 파일의 확장자

27.3.2.5. deniedExtension

기본 값	설명
	업로드를 허용하지 않는 파일의 확장자

27.3.2.6. imageDefiner

기본 값	설명
	이미지를 업로드할 경우 fileName와 filePath를 정의할 수 있는 인터페이스. 설정하지 않을 경우 WebSquare에서 지원하는 파일명과 파일 경로를 그대로 사용.

파일명과 파일 경로를 커스텀마이징해야할 경우, 추상클래스에 정의되어 있는 method를 구현한 후 imageDefiner value속성에 구현한 class의 package 전체경로를 설정해야 합니다.

27.3.2.7. subDir (deprecated)

기본 값	설명
	baseDir 하위의 경로. 지정할 경우, /websquare/upload/imgUpload/subDir/@value 설정 값은 무시됨.

27.4 excel

27.4.1. excel upload

27.4.1.1. maxRowCount

기본 값	설명
1000	엑셀 업로드 시 최대 업로드 행 수.



`maxRowCount` 속성은 `splitProvider`와 함께 동작합니다.

27.4.1.2. maxCellCount

기본 값	설명
	Excel 파일 업로드 시 업로드 가능한 최대 셀 수.

27.4.1.3. maxUploadSize

기본 값	설명
	업로드 가능한 최대 파일 크기를 지정. (단위: 바이트)

27.4.1.4. loadingMode

기본 값	설명
POI	Excel 파일 업로드 방법. • POI: POI 라이브러리 중 객체를 생성. • SAX: XML 형태의 데이터를 읽음.

27.4.1.5. saxDecimal

기본 값	설명
	<code>loadingMode="SAX"</code> 사용 시 소수점 출력 자리 수를 지정.

27.4.1.6. decrypt

기본 값	설명
	Excel 파일 업로드 시 DRM 복호화를 지원하기 위한 interface를 지정. 복호화 연계가 필요한 경우, 추상 클래스에 정의되어 있는 method를 구현한 후 decrypt value속성에 구현한 class의 package 전체 경로를 설정.

Code 27-16 설정 예 (decrypt)

```

1 <websquare>
2     <csv>
3         <upload>
4             <decrypt value="site.SiteUploaDecryper tempDir="temp경로"/>
5         </upload>
6     </csv>
7 </websquare>

```

Code 27-17 AbstractUploaDecryper 클래스

```

1 package websquare.upload.handl;
2 import java.io.*;
3 public abstract class AbstractUploaDecryper {
4     abstract public File decrypt(File file) throws Exception;
5 }

```

Code 27-18 구현 예 (SiteUploaDecryper)

```

1 package site;
2 import java.io.*;
3 import websquare.upload.handl;
4 public class SiteUploaDecryper extends AbstractUploaDecryper {
5     public File decrypt(File file) throws Exception {
6         // 복호화 처리
7         return file;
8     }
9 }

```

27.4.1.7. temp

기본 값	설명
	복호화를 위한 임시 디렉토리 작업 경로.

27.4.1.8. dataConvertor

기본 값	설명
	Excel 파일 업로드 시 셀 값의 커스터마이징을 위한 인터페이스. 각 셀 값을 커스터마징할 경우, interface에 정의되어 있는 method를 구현 후 dataConvertor value 속성에 구현한 class의 package 전체 경로를 설정.

Code 27-19 설정 예 (dataConvertor)

```

1 <websquare>
2   <excel>
3     <upload>
4       <dataConvertor value="site.SitecellDataProvierImpl" />
5     </upload>
6   </excel>
7 </websquare>

```

Code 27-20 ICellDataProvider

```

1 package websquare.http.controller.grid.excel.read;
2 public interface ICellDataProvider {
3     public String convertValue(String cellvalue) throws Exception;
4 }

```

Code 27-21 구현 예(SitecellDataProvierImpl)

```

1 package site;
2 public class SitecellDataProvierImpl implements ICellDataProvider {
3     public String convertValue(String cellvalue) throws Exception {
4         // cellvalue 수정
5         return cellvalue;
6     }
7 }

```

27.4.1.9. validExtension

기본 값	설명
false	Excel 파일 업로드시 확장자 확인 여부. xls 혹은 xlsx 확장자에 맞는 P I를 사용.

27.4.1.10. cellDataConvertor

기본 값	설명
	업로드하는 Excel 파일의 셀 값 변경 시, 사용자 class를 구현후 구현된 class의 full package명을 설정.

27.4.1.11. applyExcelFormat

기본 값	설명
	셀 타입이 숫자이고 소수점 자리가 가변인 경우, 소수점 자리에 맞는 포맷을 생성하여 적용.

27.4.1.12. dataRefine

기본 값	설명
	Excel 파일 데이터에 내부에 ' (apos)가 있는 경우 치환 유무.

27.4.2. excel download

27.4.2.1. headerColor

기본 값	설명
	다운로드하는 Excel 파일의 header 색상.

Code 27-22 설정 예

```

1 <websquare>
2   ...
3   <excel>
4     ...
5     <headerColor value="#FF0000" /> <!-- red -->
6     ...
7   </excel>
8 </websquare>

```


27.4.2.2. bodyColor

기본 값	설명
	다운로드하는 Excel 파일의 body 색상.

Code 27-23 설정 예

```

1 <websquare>
2   ...
3   <excel>
4     ...
5     <bodyColor value="#FFFF00" /> <!-- yellow -->
6     ...
7   </excel>
8 </websquare>

```

27.4.2.3. footerColor

기본 값	설명
	다운로드하는 Excel 파일의 footer 색상.

Code 27-24 설정 예

```

1 <websquare>
2   ...
3   <excel>
4     ...
5     <subtotalColor value="#00FFFF" /> <!-- aqua -->
6     ...
7   </excel>
8 </websquare>

```

27.4.2.4. subtotalColor

기본 값	설명
	다운로드하는 Excel 파일의 subtotal 색상.

Code 27-25 설정 예

```

1 <websquare>
2   ...

```

```

3    <excel>
4        ...
5        <footerColor value="#008000" /> <!-- green -->
6        ...
7    </excel>
8 </websquare>

```

27.4.2.5. maxRowCount

기본 값	설명
	다운로드하는 Excel 파일의 최대 행 수. maxRowCount 속성은 splitProvider와 함께 동작.

27.4.2.6. allowedExtension

기본 값	설명
	다운로드를 허용할 파일의 확장자를 지정.

27.4.2.7. rowAccessWindowSize

기본 값	설명
100	POI 3.9 이상(xlsx) 지원되는 대량 건 처리를 위한 설정. 메모리에 저장할 행의 수.

27.4.2.8. encrypt

기본 값	설명
	Excel 파일 다운로드 시 DRM 연계 암호를 위한 인터페이스. 암호화 연계가 필요한 경우, 추상클래스에 정의되어 있는 method를 구현후 encrypt value속성에 구현한 class의 package 전체 경로를 설 정 .

Code 27-26 설정 예 (encrypt)

```

1 <websquare>
2     <excel>
3         <download>
4             <encrypt value="site.SitetDownloadEncrypterImpl" />
5         </download>

```

```

6    </excel>
7  </websquare>

```

Code 27-27 AbstractDownloadEncrypter 클래스

```

1  package websquare.download.handl;
2  import java.io.*;
3  import javax.servlet.http.*;
4  public abstract class AbstractDownloadEncrypter {
5      abstract public void encrypt(BufferedInputStream is, HttpServletRequest request,
6      HttpServletResponse response) throws Exception;
7  }

```

Code 27-28 구현 예 (SitetDownloadEncrypterImpl)

```

1  package site;
2  import websquare.download.handl;
3  import java.io.*;
4  import javax.servlet.http.*;
5  public class SitetDownloadEncrypterImpl extends AbstractDownloadEncrypter {
6      public void encrypt(BufferedInputStream is, HttpServletRequest request,
7      HttpServletResponse response) throws Exception {
8          // 암호화 처리
9          // response에 암호화 처리된 stream을 write합니다.
10     }
11 }

```

27.4.2.9. tempDir

기본 값	설명
	advancedExcelDownload 옵션 중 pwd를 지정한 경우 (xlsx만 지원) 비밀번호 설정을 위한 임시 폴더 경로.

27.4.2.10. dataConvertor

기본 값	설명
	Excel 파일 다운로드 시 셀 값 커스터마이징에 필요한 인터페이스. 각각의 셀 값을 커스터마이징할 경우, interface에 정의되어 있는 method를 구현 후 dataConvertor value 속성에 구현한 class의 package 전체 경로를 설정.

Code 27-29 설정 예 (dataConvertor)

```

1 <websquare>
2     <excel>
3         <download>
4             <dataConvertor value="site.SiteCellDataProviderImpl" />
5         </download>
6     </excel>
7 </websquare>

```

Code 27-30 ICellDataProvider interface

```

1 package websquare.http.controller.grid.excel.write;
2
3 public interface ICellDataProvider {
4     public String convertValue(String cellvalue) throws Exception;
5 }

```

Code 27-31 구현 예 (SiteCellDataProviderImpl)

```

1 package site;
2 import websquare.http.controller.grid.excel.write;
3
4 public SiteCellDataProviderImpl implements ICellDataProvider {
5     public String convertValue(String cellvalue) throws Exception {
6         // cellvalue 수정
7         return cellvalue;
8     }
9 }

```

27.4.2.11. dataXMLProvider

기본 값	설명
	Excel파일 다운로드를 위해 클라이언트에서 전송된 XML 정보를 사용자가 재정의 할 수 있는 인터페이스. 사용자가 구현한 class full package 명을 설정.

27.4.2.12. baseDir

기본 값	설명
	gridView inputType="image"인 경우, Excel 파일에 삽입할 이미지

기본 값	설명
	가 존재하는 서버의 절대 경로.

27.4.2.13. dataTypeDate

기본 값	설명
	셀 타입을 "날짜"로 지정하기 위해 필요한 설정. advancedExcelDownload 옵션 중 useDateFormat="true"로 지정하고 본 설정을 "calendar"로 지정하게 되면 날짜 형식으로 설정됨.

27.4.3. mobile download

27.4.3.1. keyMode

기본 값	설명
	Android에서 Excel 파일 다운로드 시 특정 기기에서 서버 요청이 두 번 발생하는 현상처리를 위해 지원. - 첫번째 요청으로 전송받은 정보를 session에 설정하고, - 두번째 요청에서 session에 저장한 정보를 조회 하여 다운로드 처리를 진행. session에 설정할 key값은 keyMode노드의 값을 설정하여 사용.

27.4.3.2. preFix

기본 값	설명
session	websquare 엔진 내부에서 사용하는 키 값 대신 사용자가 설정한 값을 사용하기 위한 설정.

Code 27-32 사용 예

```

1 <keyMode value="random" preFix="SITE" />
2
3 // "random"으로 설정된 경우,
4 // client에서 "randomKey"로 설정된 문자열(request.getParameter("randomKey"))에
5 // "SITE" 문자열을 접두어로 붙여 key로 사용.
```

27.4.3.3. userAgent

기본 값	설명
	두 번 요청하는 기기의 userAgent의 정규식을 설정.

27.4.3.4. definer

기본 값	설명
	Android에서 Excel 파일 다운로드 시 특정 기기에서 서버 요청이 두 번 발생하는 현상 처리를 위한 설정. 특정 기기에서 서버 요청이 두 번 발생하는 현상 처리용. 사용자가 정의한 사용자 class에 client에서 전송한 XML 정보를 전달하기 위한 인터페이스를 설정.

27.5 csv

27.5.1. general information

27.5.1.1. encoding

기본 값	설명
UTF-8	업로드 하는 파일의 파일명 인코딩 방법.

27.5.1.2. useBOM

기본 값	설명
false	BOM 값 설정 여부. "true" : BOM 값을 설정.

27.5.1.3. opencsv

기본 값	설명
false	csv 스펙을 구현한 opencsv 외부 모듈을 사용.

27.5.2. csv upload

27.5.2.1. maxUploadSize

기본 값	설명
50Mbyte	업로드 하는 CSV 파일의 최대 크기를 설정. (단위: byte)

27.5.2.2. decrypt

기본 값	설명
	CSV 파일 업로드 시 DRM 복호화를 지원하기 위한 인터페이스. 사용자가 복호화 연계가 필요한 경우 아래와 같이 추상클래스에 정의되어 있는 method를 구현후 decrypt value속성에 구현한 class의 package 전체경로를 설정합니다.

27.5.2.3. tempDir

기본 값	설명
	복호화를 위한 임시 디렉토리 경로.

27.5.2.4. checkEncode

기본 값	설명
false	업로드되는 파일의 BOM 확인 유무.

27.5.3. csv download

27.5.3.1. keyMode

기본 값	설명
	Android에서 CSV 파일 다운로드 시 특정 기기에서 서버 요청이 두번 발생하는 현상 처리를 위해 설정. - 첫번째 요청으로 전송받은 정보를 session에 설정하고, - 두번째 요청에서 session에 저장한 정보를 조회 하여 다운로드

기본 값	설명
	처리를 진행. session에 설정할 key값은 keyMode노드의 값을 설정하여 사용.

websquare 엔진 내부에서 사용하는 키 값 대신 사용자가 설정한 값을 사용하기 위한 설정.

Code 27-33 사용 예

```

1 <keyMode value="random" preFix="SITE" />
2
3 // "random"으로 설정된 경우,
4 // client에서 "randomKey"로 설정된 문자열(request.getParameter("randomKey"))에
5 // "SITE" 문자열을 접두어로 붙여 key로 사용.

```

27.5.3.2. prefix

기본 값	설명
	websquare 엔진 내부에서 사용하는 키 값 대신 사용자가 설정한 값을 사용하기 위한 설정.

27.5.3.3. userAgent

기본 값	설명
	두 번 요청하는 기기의 userAgent의 정규식을 설정.

27.5.3.4. dataConvertor

기본 값	설명
	CVS 파일 다운로드 시 셀 값 커스터마이징에 필요한 인터페이스. 각각의 셀 값을 커스터마이징할 경우, interface에 정의되어 있는 method를 구현 후 dataConvertor value 속성에 구현한 class의 package 전체 경로를 설정.

Code 27-34 설정 예 (dataConvertor)


```

1 <websquare>
2   <csv>
3     <download>
4       <dataConvertor value="site.SiteCellDataProviderImpl" />
5     </download>
6   </csv>
7 </websquare>

```

Code 27-35 ICellDataProvider interface

```

1 package websquare.http.controller.grid.write;
2
3 public interface ICellDataProvider {
4
5     public String convertValue(String cellvalue) throws Exception;
6
7 }

```

Code 27-36 구현 예 (SiteCellDataProviderImpl)

```

1 package Site;
2 imports websquare.http.controller.grid.write;
3
4 public SiteCellDataProviderImpl implements ICellDataProvider {
5     public String convertValue(String cellvalue) throws Exception {
6         // cellvalue 수정
7         return cellvalue;
8     }
9 }

```

27.5.3.5. encrypt

기본 값	설명
	CSV 파일 다운로드 시 DRM 연계 암호를 위한 인터페이스. 암호화 연계가 필요한 경우, 추상클래스에 정의되어 있는 method를 구현후 encrypt value속성에 구현한 class의 package 전체 경로를 설 정 .

Code 27-37 설정 예 (encrypt)

```

1 <websquare>
2     <csv>
3         <download>
4             <encrypt value="site.SiteCellDataProviderImpl" />
5         </download>
6     </csv>
7 </websquare>

```

Code 27-38 AbstractDownloadEncrypter 클래스

```

1 package websquare.download.handl;
2 import java.io.*;
3 import javax.servlet.http.*;
4
5 public abstract class AbstractDownloadEncrypter {
6     abstract public void encrypt(BufferedInputStream is, HttpServletRequest request,
7     HttpServletResponse response) throws Exception;
8 }

```

Code 27-39 구현 예 (SitetDownloadEncrypterImpl)

```

1 package site;
2
3 import java.io.*;
4 import javax.servlet.http.*;
5 import websquare.download.handl
6
7 public class SitetDownloadEncrypterImpl extends AbstractDownloadEncrypter {
8     public void encrypt(BufferedInputStream is, HttpServletRequest request,
9     HttpServletResponse response) throws Exception {
10         // 암호화 처리
11         // response에 암호화 처리된 stream을 write합니다.
12     }
13 }

```

27.5.4. mobile download

27.5.4.1. definer

기본 값	설명
	CSV 파일 다운로드 시 특정 기기에서 서버 요청이 두 번 발생하는 현상 처리를 위한 설정. 특정 기기에서 서버 요청이 두 번 발생하는 현상 처리용. 사용자가 정의한 사용자 class에 client에서 전송한 XML 정보를 전달하기 위한 인터페이스를 설정.

27.6 i18n

27.6.1. general information

27.6.1.1. baseDir

기본 값	설명
	다국어 properties 파일이 위치하는 경로.

27.6.1.2. defaultLocale

기본 값	설명
	기본 locale.

27.6.1.3. domain

27.6.1.3.1. name

기본 값	설명
	ip 혹은 도메인.

27.6.1.3.2. locale

기본 값	설명
	locale.

27.6.1.4. localeParam

기본 값	설명
	get 방식으로 locale을 넘길 파라미터 이름.

27.6.1.5. storageType

기본 값	설명
	XML 읽는 방식. • 0 : file • 1 : war • 2 : jar • 3 : http • 4 : https

27.6.1.6. baseUrl

기본 값	설명
	http 기본 경로 (사용 예) • http://localhost:8080

27.6.1.7. cache

기본 값	설명
false	cache 사용 유무

27.6.2. https

27.6.2.1. keystorePath

기본 값	설명
	storageType="4" 설정(https)을 사용할 경우, 인증서 (jks) 파일의 경로.

27.6.2.2. keystoreType

기본 값	설명
	storageType="4" 설정(https)을 사용할 경우, 인증서 형식 • PKCS12 • JKD

27.6.2.3. keystoreaPass

기본 값	설명
	storageType="4" 설정(https)을 사용할 경우, 인증서 비밀번호.

27.6.3. xmlInfo

27.6.3.1. web2fileCacheImpl

기본 값	설명
	사용자 정의 구현 팩키징 경로. 지정하지 않을 경우, WebSquare에서 기본 제공하고 있는 default 구현체를 사용.

27.6.3.2. excludeList

기본 값	설명
	storageType="0"인 경우 XML 경로(full path)에 포함되지 말아야 하는 문자열을 검증하기 위한 설정. 검증해야되는 문자열을 설정.

27.6.4. messageInfo

27.6.4.1. storageType

기본 값	설명
	Properties를 읽어오는 방식 • 0 : file • 1 : war • 2 : jar • 3 : http • 4 : https

27.6.4.2. baseDir

기본 값	설명
	Properties가 있는 절대 위치

27.6.4.3. messageImpl

기본 값	설명
websquare.i18n.impl.DefaultMessageImpl	properties 를 읽어올수 있는 사용자 정의 속성. 구현체의 full package 명을 입력. 입력하지 않으면 default로 동작.

27.6.4.4. resource

기본 값	설명
	다국어 properties 파일 경로 지정 (en)

27.6.5. obfuscate

27.6.5.1. encodeParameter

기본 값	설명
false	노출되는 경로를 난독화하기 위한 설정.

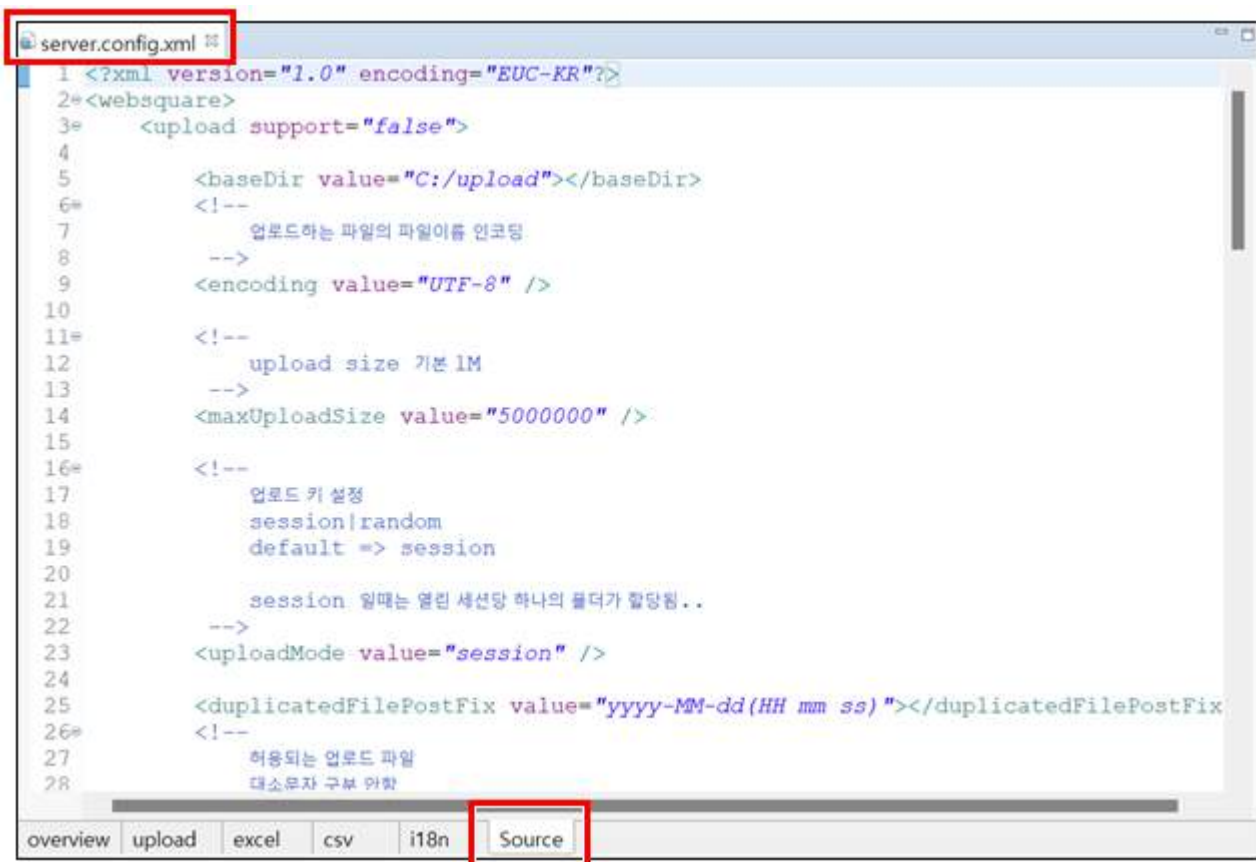
27.6.5.2. fileNameBase64Decode

기본 값	설명
false	노출되는 XML 파일명을 난독화하기 위한 설정.

27.7 Source

Source 탭은 전체 서버 설정 내용을 XML 파일 형식으로 보여줍니다.

그림 27-3 서버 설정



Part VI. 심화 가이드

28. 다국어 설정 가이드

28.1 클라이언트 다국어

설명	- 클라이언트(브라우저)에 WebSquare 페이지가 로드된 후 다국어 변환을 수행. - 브라우저의 언어를 인식하여 해당 언어의 스크립트 파일을 참조.
제약 사항	- 브라우저의 navigator 언어 설정을 따름. - 브라우저에 따라 언어의 code 값이 다를 수 있음. - 쿠키에 설정된 언어코드가 있는 경우 브라우저의 언어 설정보다 우선. 확인 방법: <code>WebSquare.cookie.getCookie("system_language");</code>

28.1.1. 언어 파일

다국어 변환시 참조할 언어 별 JS 파일은 아래의 방법으로 작성하십시오.

작성 형식

- (1) 다국어로 표현할 값을 **KEY : VALUE** 형태의 Array 형식으로 작성하십시오.
- (2) 다국어 치환에 사용되는 **KEY**는 모든 언어가 동일한 값을 사용합니다.

Code 28-1 ko.js (한국어)

```
1 WebSquare.WebSquareLang = {
2     "test": "가나다가나다",
3     "anchor": "앵커",
4     "output": "아웃풋",
5     "span": "스팬",
6     "textbox": "텍스트박스",
7     "trigger": "트리거",
8     "001": "서울",
9     "002": "부산",
10    "003": "대구",
11    "004": "제주",
12    "chooseOption": "도시를 선택하세요.",
13    "tooltip": "툴팁용 텍스트입니다."
14 }
```

Code 28-2 en.js (영어)

```

1 WebSquare.WebSquareLang = {
2     "test": "abcABC",
3     "anchor": "Anchor",
4     "output": "Output",
5     "span": "Span",
6     "textbox": "TextBox",
7     "trigger": "Trigger",
8     "001": "Seoul",
9     "002": "Busan",
10    "003": "Daegu",
11    "004": "Jeju",
12    "chooseOption": "Select a city.",
13    "tooltip": "This is tooltip text."
14 };

```

그림 28-1 ko.js 및 en.js 예시

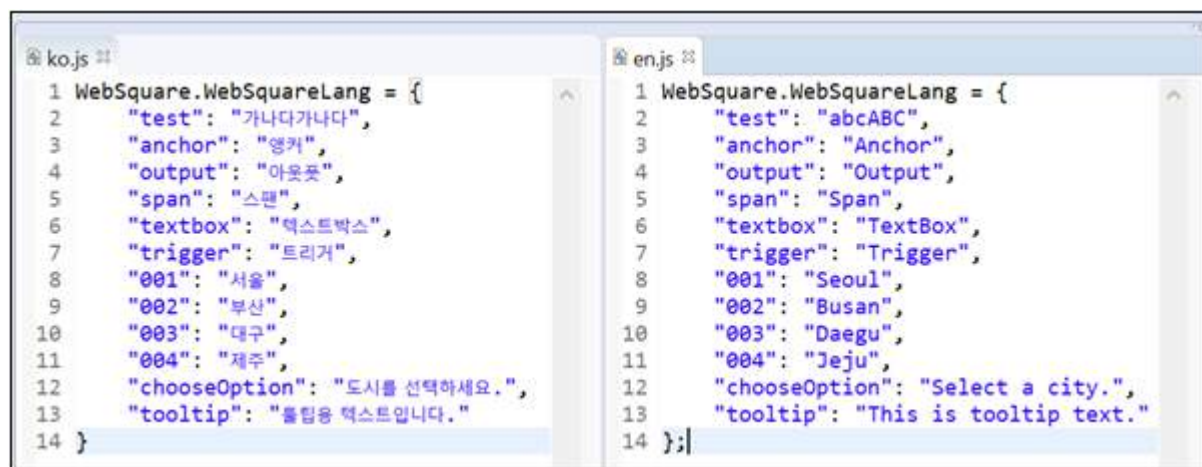


Table 28-1 ISO 639-1 언어 코드

언어	ISO 코드	언어	ISO 코드
Abkhazian	ab	Kurdish	ku
Afar	aa	Laothian	lo
Afrikaans	af	Latin	la
Albanian	sq	Latvian (Lettish)	lv
Amharic	am	Limburgish (Limburger)	li
Arabic	ar	Lingala	ln
Aragonese	an	Lithuanian	lt
Armenian	hy	Macedonian	mk
Assamese	as	Malagasy	mg
Aymara	ay	Malay	ms
Azerbaijani	az	Malayalam	ml

언어	ISO 코드	언어	ISO 코드
Bashkir	ba		
Basque	eu	Maltese	mt
Bengali (Bangla)	bn	Maori	mi
Bhutani	dz	Marathi	mr
Bihari	bh	Moldavian	mo
Bislama	bi	Mongolian	mn
Breton	br	Nauru	na
Bulgarian	bg	Nepali	ne
Burmese	my	Norwegian	no
Byelorussian (Belarusian)	be	Occitan	oc
Cambodian	km	Oriya	or
Catalan	ca	Oromo (Afan, Galla)	om
Cherokee		Papiamentu	
Chewa		Pashto (Pushto)	ps
Chinese (Simplified)	zh	Polish	pl
Chinese (Traditional)	zh	Portuguese	pt
Corsican	co	Punjabi	pa
Croatian	hr	Quechua	qu
Czech	cs	Rhaeto-Romance	rm
Danish	da	Romanian	ro
Divehi		Russian	ru
Dutch	nl	Sami (Lappish)	
Edo		Samoan	sm
English	en	Sangro	sg
Esperanto	eo	Sanskrit	sa
Estonian	et	Serbian	sr
Faeroese	fo	Serbo-Croatian	sh
Farsi	fa	Sesotho	st
Fiji	fj	Setswana	tn
Finnish	fi	Shona	sn
Flemish		Sichuan Yi	ii
French	fr	Sindhi	sd
Frisian	fy	Sinhalese	si
Fulfulde		Siswati	ss
Galician	gl	Slovak	sk
Gaelic (Scottish)	gd	Slovenian	sl
Gaelic (Manx)	gv	Somali	so
Georgian	ka	Spanish	es
German	de	Sundanese	su
Greek	el	Swahili (Kiswahili)	sw

언어	ISO 코드	언어	ISO 코드
Greenlandic	kl	Swedish	sv
Guarani	gn	Syriac	
Gujarati	gu	Tagalog	tl
Haitian Creole	ht	Tajik	tg
Hausa	ha	Tamazight	
Hawaiian		Tamil	ta
Hebrew	he, iw	Tatar	tt
Hindi	hi	Telugu	te
Hungarian	hu	Thai	th
Ibibio		Tibetan	bo
Icelandic	is	Tigrinya	ti
Ido	io	Tonga	to
Igbo		Tsonga	ts
Indonesian	id, in	Turkish	tr
Interlingua	ia	Turkmen	tk
Interlingue	ie	Twi	tw
Inuktitut	iu	Uighur	ug
Inupiak	ik	Ukrainian	uk
Irish	ga	Urdu	ur
Italian	it	Uzbek	uz
Japanese	ja	Venda	
Javanese	jv	Vietnamese	vi
Kannada	kn	Volapuk	vo
Kanuri		Wallon	wa
Kashmiri	ks	Welsh	cy
Kazakh	kk	Wolof	wo
Kinyarwanda (Ruanda)	rw	Xhosa	xh
Kirghiz	ky	Yi	
Kirundi (Rundi)	rn	Yiddish	yi, ji
Konkani		Yoruba	yo
Korean	ko	Zulu	zu

28.1.2. *config.xml*

config.xml 설정 파일에 아래와 같이 언어팩 관련 설정을 추가하십시오.

1. WebSquare5 Studio를 실행합니다.
2. 프로젝트를 우클릭한 후 **Properties**를 선택합니다.

3. 좌측에서 **WebSquare** - Engine Configuration을 선택 후, 아래와 같이 config.xml 파일을 편집합니다.
 - I. `useLanguagePack="true"` 설정을 추가합니다.
 - II. `lang`에 지정한 언어 구분 키 값을 설정합니다.
 - III. `value`에 다국어 변환시 참조할 각 언어의 스크립트 파일(`ko.js`, `en.js`, `ch.js` 등)을 Context-Root 기준으로 명시합니다.



Code 28-3 다국어 설정 예 (config.xml)

```

1 <languagePack useLanguagePack="true">
2   <url lang="ko" value="/langpack/ko.js"/>
3   <url lang="en" value="/langpack/en.js"/>
4   <url lang="ch" value="/langpack/ch.js"/>
5 </languagePack>

```

28.1.3. 텍스트 다국어 (localeRef)

각 컴포넌트에 `useLocale="true"` 설정을 추가한 후, 언어 별로 정의된 KEY 값을 `localeRef` 속성으로 설정하여 다국어를 지원할 수 있습니다.



텍스트 다국어 지원 컴포넌트

- Anchor
- TextBox
- Output
- Trigger
- Span

그림 28-2 ko.js와 en.js 파일에 각 키 별로 표시할 텍스트를 정의한 예

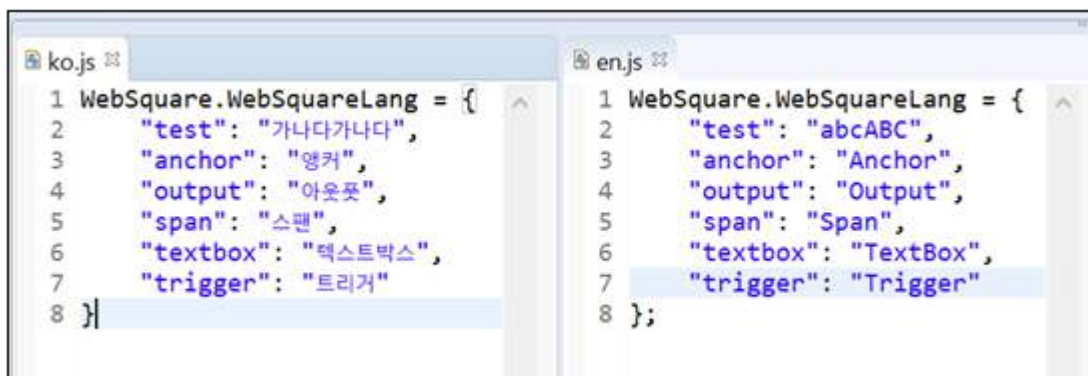
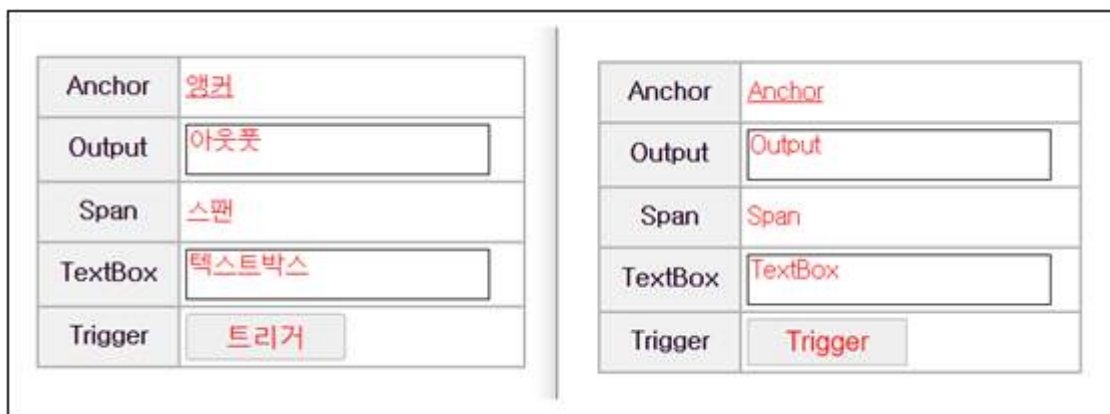


그림 28-3 useLocale="true" 및 localeRef="언어 팩에 정의한 키" 설정을 통해 다국어를 적용한 모습



예제 파일

[인터넷에서 다운로드](#) 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/_Multi-Language/useLocale_localeRef_Multi_language/)

그림 28-4 YouTube 동영상 (<https://youtu.be/ekYGymLIQCM>)

28.1.3.1. localeRef 속성 값이 언어 파일에 정의되지 않은 경우

아래 컴포넌트에 대해 useLocale="true" 설정 및 localeRef 속성 값으로 다국어를 적용할 경우, localeRef가 언어 파일에 없는 경우 "global_undefined" 대신 각 컴포넌트의 label 혹은 value 값을 표시할 수 있습니다.

- Anchor
- Output
- Span
- TextBox
- Trigger

그림 28-5 useDefaultLabel="false"

Anchor	<u>global_undefined</u>
Output	global_undefined
Span	global_undefined
TextBox	global_undefined
Trigger	global_undefi

그림 28-6 useDefaultLabel="true"

Anchor	aaa
Output	bbb
Span	ccc
TextBox	ddd
Trigger	eee

Code 28-4 config.xml 설정 예

```

1 <WebSquare>
2     <useLocale value="true" useDefaultLabel="true"/>
3 </WebSquare>

```



예제 파일

인터넷에서 다운로드 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/_Multi-language/useLocale_useDefaultLabel_Multi_language/)

그림 28-7 YouTube 동영상 (<https://youtu.be/p3FQLMpBe1g>)

Quick Guide

WebSquare 5

config.xml

<useDefaultLabel>

Goal / 목표

In case localeRef is not defined in multi-language environment, display original label or value of the component. / localeRef 속성 값이 언어 파일에 없는 경우, 컴포넌트의 value 혹은 label 값을 표시.

Components / 컴포넌트

- Anchor
- Output
- Span
- TextBox
- Trigger

```

<WebSquare>
  <useLocale value="true" useDefaultLabel="true"/>
</WebSquare>

```

YouTube

<https://youtu.be/p3FQLMpBe1g>

28.1.4. 선택 항목 다국어 (useLocaleItem)

각 컴포넌트에 `useLocale="true"` 및 `useItemLocale="true"` 설정을 추가하여, 선택 항목을 각 언어 파일에 정의된 KEY 값으로 표시할 수 있습니다.



선택 항목 다국어 지원 컴포넌트

- AutoComplete
- CheckBox
- CheckComboBox
- MultiSelect
- Radio
- SelectBox

선택 항목은 각 언어 파일에 정의할 KEY로 설정하십시오.

그림 28-8 선택 항목 설정 (CheckBox 예시)

Label	Value
<> 001	<> aaa
<> 002	<> bbb
<> 003	<> ccc
<> 004	<> ddd

☐ BindItemSet
 NodeSet :
 Label :
 Value :
 ref :
 Span Direction : rows
 Span Count : 1

OK Cancel

다음은 `useLocale` 및 `useLocaleItem` 속성을 설정하지 않았을 때 표시되는 CheckBox의 선택 항목입니다.

그림 28-9 useLocale="false"



그러나, 아래와 같이 각 언어 파일에 선택 항목 001 ~ 004 KEY에 대한 언어 별 텍스트를 정의하고, useLocale 및 useItemLocale 속성을 설정하면 각 언어에 대한 텍스트가 표시됩니다.

그림 28-10 en.js 및 ko.js

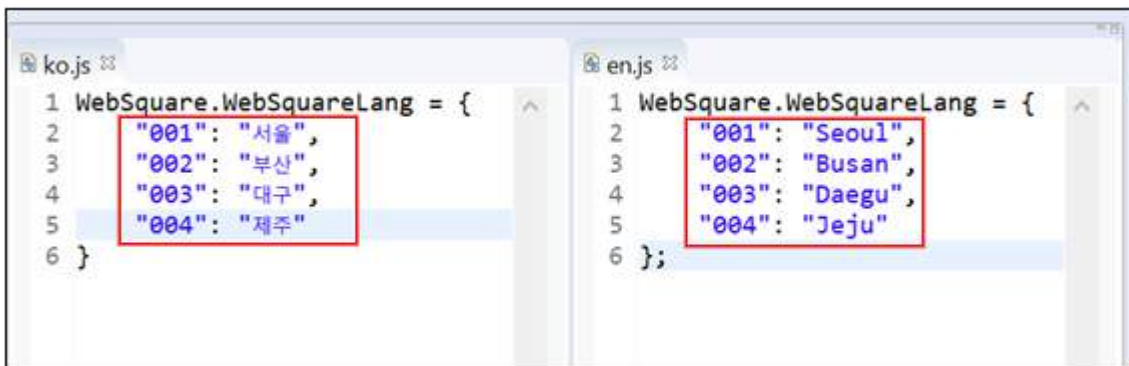


그림 28-11 useLocale="true" useItemLocale="true"

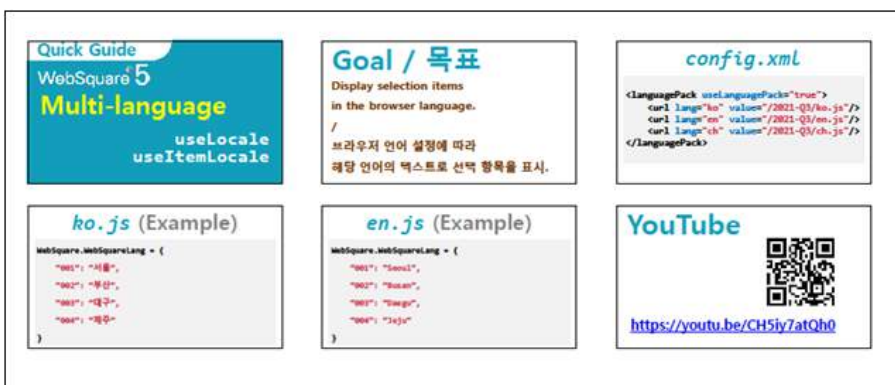


예제 파일

인터넷에서 다운로드 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/_Multi-Language/useLocale_useItemLocale_Multi_language/)

그림 28-12 YouTube 동영상 (<https://youtu.be/CH5iy7atQh0>)

28.1.5. 선택 항목 및 chooseOption 다국어 (useItemLocale & chooseOptionLabelLocaleRef)

각 컴포넌트에 `useLocale="true"` 및 `useItemLocale="true"` 설정을 추가하여, 선택 항목을 각 언어 파일에 정의된 KEY 값으로 표시하고, `chooseOption="true"`인 경우, 해당 `chooseOption` 텍스트도 `chooseOptionLabelLocaleRef` 속성을 설정하여 다국어로 표시할 수 있습니다.



선택 항목 및 옵션 다국어 관련 설정

- `useLocale="true"`
- `localeRefuseItemLocale="true"`
- `allOption="true"`
- `chooseOption="true"`
- `chooseOptionLabelLocaleRef`

아래와 같이 SelectBox 설정 창에서 `chooseOption`과 `allOption`을 선택합니다.

그림 28-13 `chooseOption` 및 `allOption` 설정 (SelectBox 예시)

Label	Value
<> 001	<> aaa
<> 002	<> bbb
<> 003	<> ccc
<> 004	<> ddd

☐ BindItemSet
 NodeSet :
 Label :
 Value :
 ref :

☒ All Option
☒ Choose Option :

OK Cancel

각 언어 파일에는 `chooseOptionLabelLocaleRef`로 지정할 KEY에 대한 각 언어 별 텍스트를 정의합니다. 아래 예시의 경우, 한국어 브라우저에서는 "도시를 선택하세요."가 표시되고, 영어 브라우저에서는 "Select a city."가 표시됩니다.

다.

그림 28-14 ko.js 및 en.js 파일에 chooseOption 텍스트용 KEY를 지정



다음은 chooseOption="true" 설정을 추가한 후, chooseOptionLabelLocaleRef 값으로 각 언어 파일에 정의한 "chooseOption" KEY 값을 설정했을 때의 모습입니다. allOption에 대해서는 "-전체-"와 "-all-"이 각각 표시됩니다.

그림 28-15 chooseOptionLabelLocaleRef="chooseOption" allOption="true"



예제 파일

인터넷에서 다운로드 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/_Multi-Language/

useLocale_useItemLocale_chooseOption_chooseOptionLabelLocaleRef_Multi_language/)

그림 28-16 YouTube 동영상 (<https://youtu.be/9jZhEDnSmlw>)



28.1.6. 툴팁 다국어 (tooltipLocaleRef)

useLocale과 tooltip 기능을 지원하는 지원하는 컴포넌트의 경우, tooltipLocaleRef 속성을 통해 툴팁을 다국어로 표시할 수 있습니다.



툴팁 관련 속성

```
useLocale="true"
tooltip
tooltipLocaleRef
```

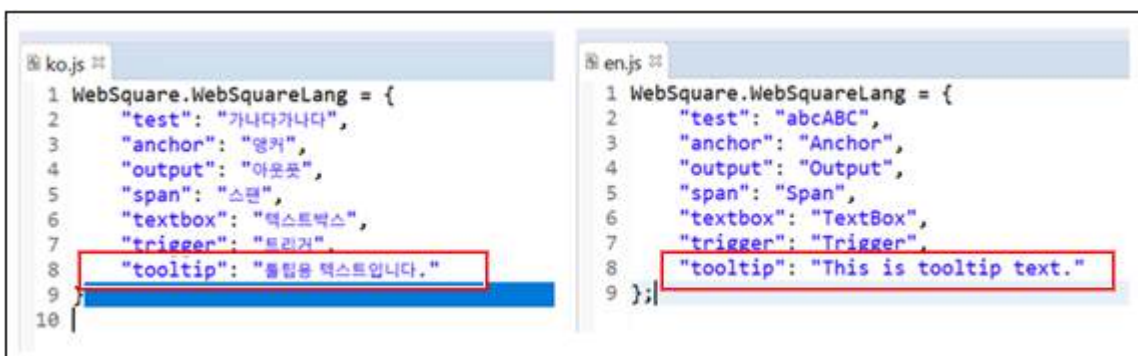


useLocale 및 tooltip 속성을 지원하는 컴포넌트

Anchor
AutoComplete
CheckBox
Group
MultiSelect
Output
Radio
SelectBox
TextBox
Trigger

각 언어 파일에는 tooltipLocaleRef로 지정할 KEY에 대한 각 언어 별 텍스트를 정의합니다. 아래 예시의 경우, 한국어 브라우저에서는 "툴팁용 텍스트입니다."가 표시되고, 영어 브라우저에서는 "This is tooltip text."가 표시됩니다.

그림 28-17 ko.js 및 en.js 파일에 tooltipLocaleRef 텍스트용 KEY를 지정



다음은 useLocale="true" 설정을 추가한 후, chooseOptionLabelLocaleRef 값으로 각 언어 파일에 정의한 "tooltip" KEY 값을 설정했을 때의 모습입니다. (tooltipLocaleRef 값이 없는 경우, tooltip 값속성으로 정의한 값을 그대로

로 사용합니다.)

그림 28-18 `useLocale="true" tooltipLocaleRef="tooltip"`



예제 파일

인터넷에서 다운로드 혹은

WEBSQUARE_DEV_PACK의 GUIDE 프로젝트에서 보기

(/_Multi-Language/useLocale_tooltipLocaleRef_Multi_language/)

그림 28-19 YouTube 동영상 (https://youtu.be/mV2H51EV_iw)



28.1.7. GridView 다국어

28.1.7.1. 행 번호 및 행 상태 컬럼의 헤더 다국어

아래 방법으로 행 번호(`rowNum`) 및 행 상태(`rowStatus`) 컬럼의 헤더를 다국어로 표시할 수 있습니다.

1. `rowStatusVisible="true"` 및 `rowNumVisible="true"` 설정을 사용하여 행 번호 및 상태를 나타내는 컬럼을 표시합니다.
2. `WebSquare.WebSquareLang`에 각 번호(`rowNum`) 및 행 상태(`rowStatus`) 컬럼의 KEY 값에 대해 표시할 헤더 값을 설정합니다.

Code 28-5 WebSquare.WebSquareLang에 Key 값 별로 헤더 값 설정

```

1 scwin.init = function() {
2     WebSquare.WebSquareLang = {
3         'rowNum': 'Number',
4         'rowStatus': 'Status'
5     };
6 };

```

3. GridView의 `rowStatusLocaleRef` 및 `rowNumLocaleRef` 속성으로 해당 KEY 값을 설정합니다.
아래 예제의 경우 행 번호 및 행 상태 컬럼의 헤더에 "순번"이나 "상태"가 아닌 "Number"와 "Status"가 표시됩니다.

Code 28-6 rowStatusLocaleRef 및 rowNumLocaleRef 속성 설정

```

1 <w2:gridView
2     ...
3     rowNumLocaleRef="rowNum" rowStatusLocaleRef="rowStatus"
4     rowNumVisible="true" rowStatusVisible="true"
5     ...
6 </w2:gridView>

```

28.1.7.2. 헤더 툴팁 다국어

아래 방법으로 헤더 툴팁을 다국어로 표시할 수 있습니다.

1. `WebSquare.WebSquareLang`에 각 언어 별로 사용할 헤더 툴팁용 KEY 값에 대한 텍스트를 설정하고,
2. `gridView.header`의 `tooltipLocaleRef` 속성으로 해당 KEY 값을 설정하십시오.



헤더 툴팁과 관련된 속성

`tooltipHeader="true"` (GridView)
`tooltipHeaderShowAlways="true"` (GridView)

28.2 서버 (WAS) 다국어

28.2.1. Properties 파일 사용

설명	- 다국어로 표시할 값이 서버에서 변환 된 후, 클라이언트(브라우저)로 전송. - 브라우저는 전송 받은 값을 표시.
제약 사항	- WebSquare5 Studio의 미리보기 기능은 CIFS, FTP Preview에서만 지원. (Local Preview 제외.) - Base Folder에 언어 별 key 값으로 폴더를 생성하여 각 언어의 Properties 파일을 생성.

28.2.1.1. 다국어 프레임워크

최초 다국어 변환을 요청하면 다국어 치환에 사용될 properties를 서버에 로딩합니다. (재로딩은 할 수 없습니다.)



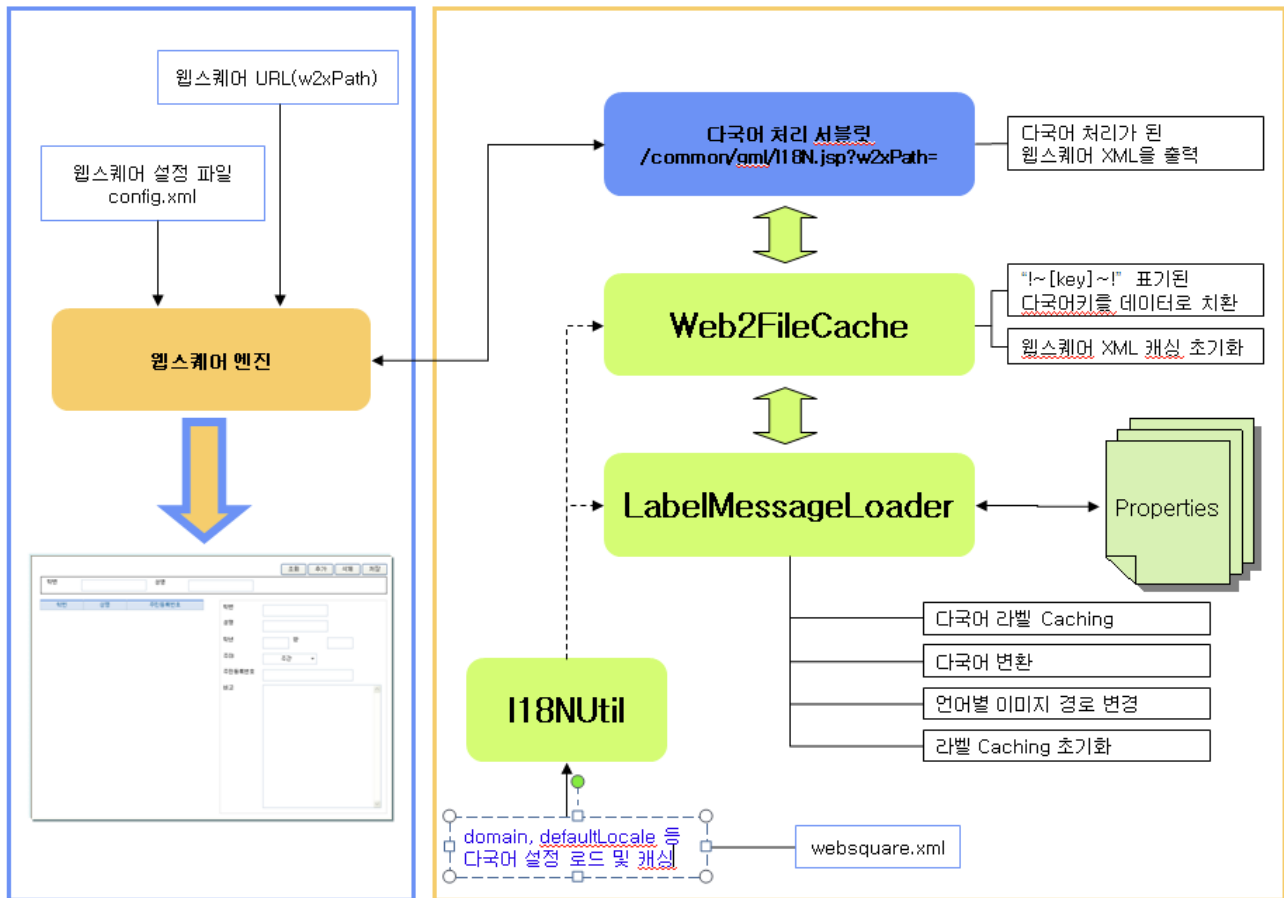
properties를 수정할 경우 아래 방법으로 재 로딩 하십시오.

1. WAS를 다시 기동합니다.
2. LabelMessageLoader의 instance를 받아 `reload();` API를 호출합니다.



다국어로 치환된 화면 XML을 캐싱할 수 있습니다. 즉, 동일한 화면 요청 시 다시 치환하는 과정을 생략할 수 있습니다.

1. **w2xPath** 경로에 다국어 처리용 Servlet을 태워서 웹스퀘어 화면 파일을 내려줍니다.
2. 다국어 처리용 Servlet은 **/websquare/config.xml**에 정의됩니다.
3. Server에 별도의 다국어 처리용 엔진이 있어야 합니다.
4. 글로벌 बैंकिंग의 경우 다국어 정보는 DB로 관리를 하고 기존 XSL 다국어 처리용 엔진을 웹스퀘어에 맞게 수정하십시오.
5. **websquare/config.xml** 파일에 아래 설정을 추가하십시오.



28.2.1.2. Property

1. 다국어로 치환될 값을 KEY=VALUE 형식의 Property로 작성합니다.
2. value는 unicode 값으로 입해야 합니다.
3. 아래는 "TEST" 키 값에 매칭될 한글명 "테스트"를 property로 작성한 예입니다.

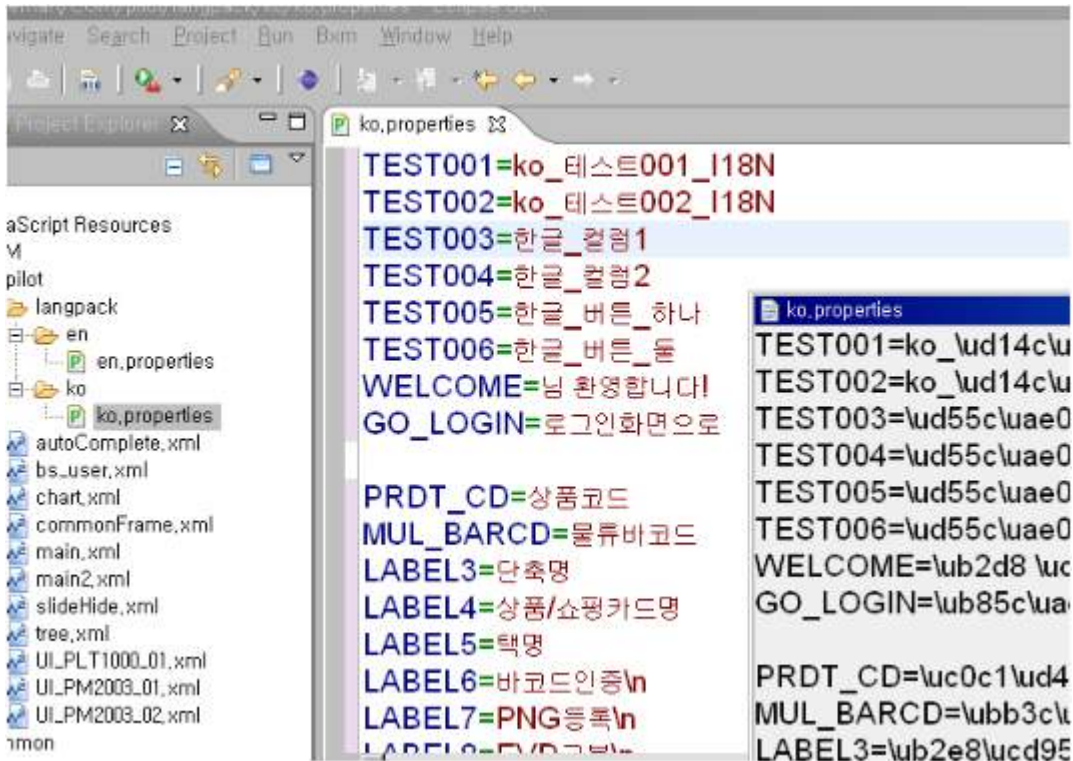
Code 28-7 property 작성 예

```
1 TEST=\ud14c\uc2a4\ud2b8
```



Eclipse의 Properties Editor 플러그인 사용을 권장합니다.
properties 파일 작성 시 자동 유니코드 변환 기능이 지원됩니다.

그림 28-20 Properties Editor



28.2.1.3. config.xml

1. 다국어 처리시 WebSquare XML 페이지를 호출할 Servlet 파일의 지정된 스크립트 Function을 지정합니다.
2. 페이지 로딩 전 WebSquare 엔진에서 참조할 스크립트를 지정합니다.
3. 별도 Servlet에서 다국어 분기 로직을 수행하고 하는 경우 `pageURLPrefix`의 value에 Context-Root를 기준으로 해당 Servlet작성합니다.

Code 28-8 설정 예 (config.xml)

```

1 <!-- w2xPath경로를 별도 Servlet으로 처리하는 경우 설정 -->
2 <!-- pageURLPrefix value="/common/gib/jsp/I18N.jsp?w2xPath=" -->
3 <pageURLPrefix type="function" value="getI18NUrl" />
4
5 <!-- WebSquare 엔진에서 참조할 스크립트 -->
6 <engine>
7     <module src="/common/gml/js/gmlCommon.js" />
8 </engine>

```

28.2.1.4. Function 작성

1. 다국어 Page를 호출할 Servlet url을 return합니다.
2. locale 값을 지정하고자 하는 경우는 url에 파라미터를 포함하여 return합니다.

Code 28-9 function 작성 예

```

1  function getI18Nurl() {
2      var locale = WebSquare.cookie.getCookie( "locale" );
3      if( locale == null || locale == '' ) {
4          return "/websquare/engine/servlet/I18N.jsp?w2xPath=";
5      } else {
6          return "/websquare/engine/servlet/I18N.jsp?locale="+unescape(locale)+"&w2xPath=";
7      }
8  }

```

28.2.1.5. websquare.xml

1. 다국어에서 필요한 설정값들을 추가합니다.

```

1  <i18n>
2      <!--
3          다국어 리소스 번들 파일이 존재하는 폴더. 설정이 없는 경우 $WEBSQUARE_HOME/locale를
4          사용한다.
5          * 한국어의 경우 baseDir/ko/ 아래 파일이 존재함
6          * 영어의 경우 baseDir/en/ 아래 파일이 존재함
7      -->
8      <baseDir value="/" />
9      <!-- 기본 locale를 설정한다. 설정이 없는 경우 en을 사용한다. -->
10     <defaultLocale value="jp">
11         <domain name="localhost" locale="en" />
12         <domain name="ko.shb.co.kr" locale="ko" />
13         <domain name="en.shb.co.kr" locale="en" />
14         <domain name="jp.shb.co.kr" locale="jp" />
15     </defaultLocale>
16     <!--
17         변환에 사용할 다국어를 서버에 넘겨주기 위해 사용할 파라미터. 설정이 없는 경우
18         locale를 사용한다.
19         다국어 판단 우선순위
20         1.url
21         2.defaultLocale설정(domain별 default locale참조 후 전체 default locale 참조)
22     -->
23     <localeParam value="locale" />
24     <!--
25         baseDir : 웹스퀘어 XML파일이 존재하는 폴더를 지정한다. 설정이 없는 경우 정상
26         동작하지 않는다.

```

```

24         cache : true | false 변환된 파일을 캐싱할지 설정한다. 기본값은 false이다.
25     -->
26     <xmlInfo baseDir="C:\TNB\webdev\webapps\ROOT" cache="false"/>
27 </i18n>

```

28.2.1.6. WebSquare 화면 파일

1. 페이지 작성시 value에 다국어로 변환할 부분을 "!~KEY~!" 형식으로 지정합니다.
2. locale 값을 지정하는 경우 파라미터로 전달합니다.

Code 28-10 설정

```

1 <w2:column id="column3" style="height:20px"
2     hidden="" inputType="text" width="161" valueType="binary"
3     value="!~TEST~!">
4 </w2:column>

```

Code 28-11 변환 결과

```

1 <w2:column id="column3" style="height:20px"
2     hidden="" inputType="text" width="161" valueType="binary"
3     value="테스트">
4 </w2:column>

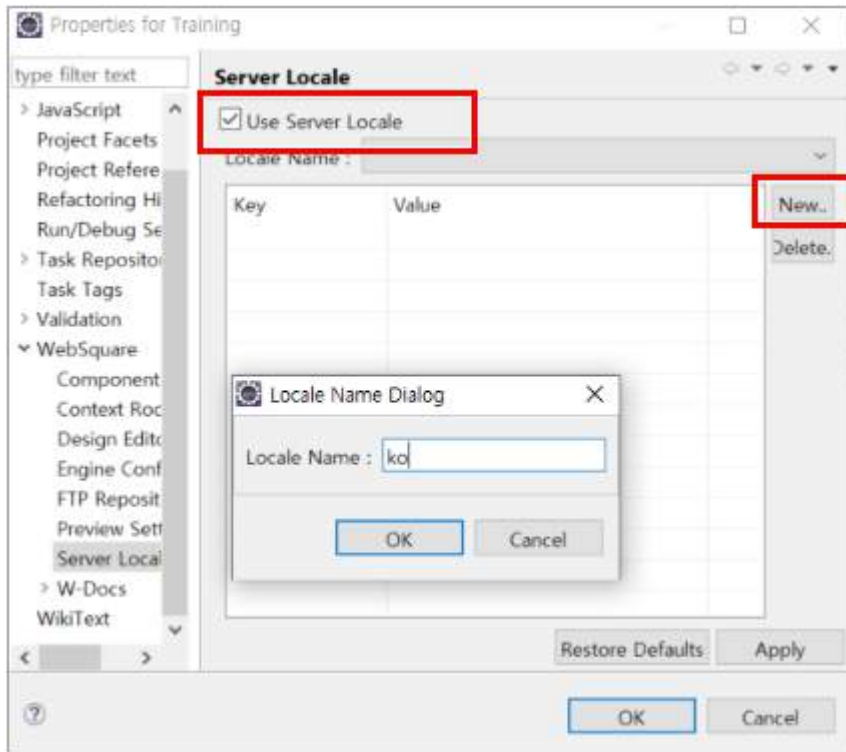
```

28.2.1.7. WebSquare5 Studio에서 보기

Project별 Server Locale 설정을 통해 Preview에 서버 다국어가 적용된 화면을 Preview에서 볼 수 있는 기능을 제공합니다.

1. WebSquare5 Studio를 실행합니다.
2. 해당 프로젝트를 우클릭하여 **Properties**를 선택합니다.
3. **WebSquare - Server Locale**을 선택합니다.

그림 28-21 Server Locale 선택



4. New 버튼을 클릭 하여 등록할 Locale Name을 입력하고 등록할 .properties 파일을 선택합니다. (여러 파일을 선택할 수 있습니다.)
5. 등록한 Locale Name 및 .properties의 내용이 화면에 표시되는지 확인합니다.
6. 여러 개의 Locale을 등록하면 Preview를 통해 적용된 화면을 볼 수 있습니다.

28.2.2. 서버 다국어 DB 사용

프로퍼티 파일이 아닌 DB를 사용할 경우 해당 파일 일부분(initializeProperty 등)을 참고하여 커스터마이징 class를 생성/등록해야 합니다.

DB를 사용하기 위해서 아래의 순서로 작업하십시오.

1. **websquare.xml** 변경. (**i18N** 노드)
2. com.inswave.common.i18n.LoadI18nMessageImpl 생성 및 등록.
3. DB 작업 (테이블 생성 및 데이터 등록)
4. **i18NCommon.js** 등록
5. **config.xml** 설정 추가. (**i18Ncommon.js**, **pageURLPrefix**)



참고용 예제 파일:

- (1) [DefaultMessageImpl.java](#): 웹스퀘어 원본 소스.

- (2) [websquare_multilang.zip](#): 특정 프로젝트에 적용한 커스터마이징 소스
- (3) [LoadI18nMessageImpl.java](#)

28.2.2.1. websquare.xml 변경

1. websquare.xml 파일에 아래 내용을 추가합니다.

Code 28-12 설정 추가 (websquare.xml)

```
1 <messageImpl value="com.inswave.common.i18n.LoadI18nMessageImpl"/>
```

Code 28-13 설정 예 (websquare.xml)

```
1 <i18n>
2   // 변환에사용할 다국어를서버에 넘겨주기위해사용할 파라미터.
3   // 설정이 없는 경우 locale을 사용.
4
5
6   // 다국어판단 우선순위
7   // 1.url
8   // 2.defaultLocale 설정(domain 별default locale 참조후 전체default locale참조)
9
10
11   <defaultLocale value="ko">
12
13   <domain name="" locale=""/>
14   </defaultLocale>
15   <localeParam value="locale"/>
16
17
18   <!-- DB를 이용한 서버 다국어 설정 -->
19   <messageInfo storageType="" baseDir="">
20   <messageImpl value="com.inswave.common.i18n.LoadI18nMessageImpl"/>
21   </messageInfo>
22   <defaultLocale value="ko">
23   <domain name="" locale=""/>
24   </defaultLocale>
25   <localeParam value="locale"/>
26
27   // baseDir : 웹스퀘어 XML이 존재하는 폴더. 필수.
28   // cache   : true | false 변환된 파일 캐싱 여부. 기본값은 false.
29   -->
```



```

30 <xmlInfo baseDir="C:\project01\webapp\xml" cache="false"/>
31 </i18n>

```

2. 위에 등록한 `LoadI18nMessageImpl.java`를 프로젝트에 맞게 수정하십시오.

28.2.2.2. com.inswave.common.i18n.LoadI18nMessageImpl 생성/등록

해당 클래스는 실제 변환될 다국어 값을 DB에서 읽어 들여 HashTable 에 올려 놓습니다.

`LoadI18nMessageImpl.java` 을 컴파일하여 클래스를 등록합니다.



첨부파일은 실제 프로젝트 파일과 상이합니다.

프로젝트에 적용된 클래스는 프로워스 DB커넥션을 이용하므로 JDBC 처리가 다릅니다.

28.2.2.3. DB 테이블 생성 / 샘플 데이터 등록 (오라클용)

Code 28-14 DB 테이블 생성 및 데이터 등록 예

```

1 CREATE TABLE I18N_TABLE (
2     LANG_KEY VARCHAR(20) NOT NULL ,
3     WORD_KO VARCHAR(150) NULL ,
4     WORD_EN VARCHAR(150) NULL ,
5     WORD_CN VARCHAR(150) NULL ,
6     I18N_TABLEcol VARCHAR(45) NULL ,
7     PRIMARY KEY (LANG_KEY) );
8 insert into i18n_table(lang_key, word_ko, word_en, word_cn)
9 values('a1', '한글_a1', 'ENG_a1', 'CHN_a1');
10 insert into i18n_table(lang_key, word_ko, word_en, word_cn)
11 values('a2', '한글_a2', 'ENG_a2', 'CHN_a2');

```

28.2.2.4. config.xml 변경

아래와 같이 `i18Ncommon.js` 및 `pageURLPrefix`를 설정합니다.

Code 28-15 설정 예 (config.xml)

```

1 <engine>
2     <!-- module src="/education/common/common.js" / -->
3     <module src="/wqtemplate/common/js/i18nCommon.js"/>

```

```
4 </engine>
5 <pageURLPrefix type="function" value="getI18NUrl" />
```

28.2.2.5. i18NCommon.js 등록

config.xml 파일에 설정된 *i18NCommon.js* 를 WebSquare 디렉토리의 해당 경로에 등록하십시오.

- [호출경로 : */wqtemplate/common/js/i18nCommon.js*]

아래 fuction을 포함해야 합니다.

```
1 function getI18NUrl() {
2     var locale = WebSquare.cookie.getCookie( "locale" );
3     if( locale == null || locale == '' ) {
4         return "/websquare/engine/servlet/I18N.jsp?w2xPath=";
5     } else{
6         return  "/websquare/engine/servlet/I18N.jsp?locale="+unescape(locale)+"&w2xPath="; }
7 }
```

28.2.2.6. 웹스퀘어 화면 파일

페이지 작성시 value에 다국어로 변환할 부분을 "!~KEY~!" 형식으로 지정합니다.

Code 28-16 설정

```
1 <w2:column id="column3" style="height:20px"
2     hidden="" inputType="text" width="161" valueType="binary"
3     value="!~TEST~!">
4 </w2:column>
```

아래와 같이 KEY에 해당하는 locale 값으로 변환됩니다.

Code 28-17 변환 결과

```
1 <w2:column id="column3" style="height:20px"
2     hidden="" inputType="text" width="161" valueType="binary"
3     value="테스트">
4 </w2:column>
```

28.2.2.7. locale 설정

locale 우선 순위는 아래와 같습니다.

- 1순위 쿠키
- 2순위 도메인 설정 (*websquare.xml*)
- 3순위 request parameter (locale) 값

키값에 해당하는 locale 값이 없으면 공백 처리됩니다.

트리거 클릭 이벤트 시 아래와 같이 쿠키에 locale을 설정하십시오. 이후 처리되는 XML 페이지는 쿠키에 셋팅된 locale에 따라 변환됩니다.

Code 28-18 locale 설정 예

```
1 WebSquare.cookie.setCookie("ko");
```


29. 동적 이벤트 가이드

29.1 `component.bind(type, function);`

- event를 등록하기 위한 함수입니다.
- type이 WebSquare 이벤트인 경우 이벤트 발생 시 WebSquare에서 전달하는 arguments가 해당 핸들러 함수로 전달됩니다.

(예제)

```
grid1.bind( ' oncellclick ', function(row, col){ ... }
```

- type이 Browser이벤트인 경우 이벤트 발생 시 브라우저 이벤트 객체(event)가 해당 핸들러 함수로 전달됩니다.

(예제)

```
input1.bind( ' onclick ', function(ev){ ... })
```

- 위 두가지에 해당하지 않는 type인 경우 자동으로 이벤트가 발생하지 않으며, trigger 함수를 통해 동작시킬 수 있습니다.

(예제)

```
calendar1.bind( ' oncustom ', function(args){ ... } );
```

```
calendar1.trigger( ' oncustom ', [args]);
```

- 하나의 type에 대해 여러 function을 bind로 등록할 경우 이벤트 발생 시 bind를 호출한 순서대로 핸들러 함수가 호출됩니다.
- WebSquare 엔진 내부에서 특정 이벤트 type에 대한 이벤트를 등록하여 처리하고 있는 경우, IE 6,7,8의 경우 엔진 내부의 핸들러 함수를 나중에 호출하며, 그 외의 브라우저의 경우 엔진 내부 핸들러 함수를 먼저 호출한 후, bind로 등록된 핸들러 함수들을 차례대로 호출합니다.

(예제)

```
grid1.handleClickEvent = function(e){alert(0); ...} // 엔진 내부 핸들러
```

```
grid1.bind( ' onclick ', function(e){alert(1);});
```

```
grid1.bind( ' onclick ', function(e){alert(2);});
```

(결과)

IE 6,7,8 : 1 -> 2 -> 0

그 외 : 0 -> 1 -> 2

(grid의 click 이벤트 핸들러에 `alert(0);` 코드를 추가했다고 가정한 경우입니다.)

29.2 `component.unbind([type], [function]);`

- bind 함수로 등록한 event 핸들러 또는 XML에서 스크립트로 등록한 핸들러를 제거하는 함수입니다. 엔진 내부 핸들러나 `document.addEventListener`로 등록한 함수와 같이 bind로 등록하지 않은 이벤트 핸들러는 제거하

지 않습니다.

- unbind인자로 type, function이 모두 주어진 경우, 해당 type으로 등록된 이벤트 핸들러 중 function과 정확히 일치하는 핸들러를 제거합니다.
- unbind인자로 type만 주어진 경우, 해당 type으로 등록된 모든 이벤트 핸들러를 제거합니다.
- unbind인자로 아무 인자도 주어지지 않을 경우 해당 component에 등록된 모든 이벤트 핸들러를 제거합니다.

29.3 component.trigger(type, [array]);

- 이벤트를 발생시키기 위해 사용하는 함수입니다.
- type이 브라우저 이벤트이면서 WebSquare component event가 아닌 경우 document.dispatchEvent 또는 document.fireEvent 함수를 통해 이벤트를 직접 발생시킵니다. 이 경우 [object](object array)형태로 추가 인자를 전달한 다음 해당 핸들러의 이벤트 객체에서 해당 인자를 가져와서 사용할 수 있습니다.

(예제)

```
grid1.bind( ' onclick ' , function(e){alert(e.a + ' , ' + e.b);});
grid1.trigger( ' onclick ' , [{a : ' arg1 ' , b : ' arg2 ' }]);
```

(결과)

arg1,arg2

- 그 외 type의 경우에는 해당 type으로 등록된 핸들러 함수들을 등록된 순서대로 호출합니다. array형태로 추가 인자를 전달하면 해당 이벤트 핸들러 함수에 추가 인자가 전달됩니다.
- [array] 인자의 type은 반드시 array여야 합니다.



1. bind, unbind, trigger 모두 WebSquare의 component 단위로 지원합니다. HTML element 단위로 는 지원하지 않습니다.

2. 각 함수 호출 시 type인자의 맨 앞의 2글자가 'on'으로 시작하지 않을 경우 자동으로 'on'을 붙입니다.

3. type의 이름이 WebSquare event와 browser event가 겹치는 경우가 있는데, 이 경우 항상 WebSquare event 형태의 type으로 간주하여 처리합니다.

(ex : treeview의 ondrop, anchor의 onclick)

30. 모바일 웹 개발 가이드

30.1 개요

본 문서는 웹스케어를 사용하여 모바일 개발 시 주의사항에 대한 가이드입니다. 유념하실 것은 웹스케어는 데스크탑용 브라우저를 위한 AJAX프레임워크로, 모바일 용으로 최적화 되어 있지는 않다는 점을 인지하시기 바랍니다.

30.2 주의사항

30.2.1. 여러가지 해상도 대응은 viewport나 css3의 @media를 사용

- 모바일 기기의 해상도가 다양하므로 @media속성을 이용해 최적화된 css를 구성하여 각 기기에 맞는 화면을 꾸밀 수 있다.

```
1  .dynamicScreen {
2    font-family: Arial, Helvetica, sans-serif;
3    height: 250px;
4    color: black;
5  }
6
7  @media all and (max-width: 320px) {
8    /* 아이폰/아이팟터치/안드로이드의 초기 스크린 해상도 설정 */
9    .dynamicScreen {background-color: orange;width:320px; }
10 }
11
12 @media all and (max-width: 768px) {
13   /* 아이패드 초기 해상도 설정 */
14   .dynamicScreen {background-color: blue;}
15 }
16
17 @media all and (min-width: 769px) {
18   /* 아이패드보다 큰 스크린 사이즈와 해상도를 갖는 모든 기기에서의 해상도 설정*/
```

```
19 .dynamicScreen {background-color: green;}
20 }
```

- meta태그의 viewport를 이용해서 기기의 최대가로크기를 정하거나 줌레벨, 사용자 확대축소 방지
- 여부등을 설정 가능하다. meta태그는 websquare.html(jsp)에 넣어야 한다.

```
1 <meta name= " viewport " content= " width=device-width " />
2 <meta name= " viewport " content= " initial-sacle=1,user-scalable=no " />
```

30.2.2. 마우스나 키보드 사용을 전제하지 말 것

- 각종 모바일 기기에 따라 키보드나 마우스 같은 장치 유무가 다르기 때문에 마우스나 키보드 사용을 전제해서는 안 된다.

30.2.3. 스크롤 사용시 주의 사항

- 모바일에서 body를 제외하고, overflow:hidden으로 사용하는 것을 권장 함.
스크롤이 필요한 경우 scrollView 컴포넌트를 사용하여 스크롤을 사용할 수 있다

30.2.4. 이벤트 사용시 주의

- 스마트 폰 들이 기본적으로 터치 기반이므로 double click 등의 이벤트를 사용하지 말아야 한다.
- 마우스 이벤트의 경우 기기마다 다른 동작을 하므로 주의 하여야 한다.

30.2.5. 파일 첨부 제한

- iPhone의 경우 보안을 위해 허용된 디렉터리의 파일만 첨부 가능.

30.2.6. 플러그인 기술 사용금지

- 기기에 따라 flash나 silverlight등의 플러그인 사용이 제한되므로 플러그인 사용을 하지 말아야 한다.

30.2.7. 모바일 화면 구성 시 iframe 사용 금지

- 기기에 따라 iframe으로 화면을 구성 하게 되면, focus, blur 관련 동작이 일반 브라우저와 다르게 동작하여 input간 focus가 정상적으로 동작 하는 않는 문제를 발생 시킬 수 있다.

30.2.8. 팝업 사용시 주의 사항

- 모바일의 경우 팝업을 띄우게 되면, 새 창(window.open)이 띄며 화면전환이 이루어 지므로 권장 하지 않는다.
- layer 형태의 팝업 및 link를 사용 하는 것을 권장 함.

30.2.9. 사용금지 컴포넌트

- ifame
- xsl
- ckEditor
- chart (iPhone에서만 동작.)
- upload
- multiupload
- flash
- applet
- active
- multiselect
- slideHide
- tabContainer
- windowContainer

30.2.10. 테스트 된 모바일 OS 및 브라우저

- iphone os3.1/4.0(safari)
- android - galaxyS(2.1-update1, 2.2, 2.3), nexusOne(2.2) => 기본내장브라우저

30.3 모바일 개발시 사용가능 컴포넌트

아래의 기준으로 (1) 권장, (2) 사용 가능, (3) 비권장, (4) 사용 금지 컴포넌트로 분류할 수 있습니다.

기준	권장	사용 가능	비권장	사용 금지
모든 플랫폼 지원 여부	지원			미지원 (일부)
기능 지원 범위	모두 지원	대부분	일부 지원	N/A
터치 화면 사용성	높음	보통	낮음	N/A
개발자 코딩에 따른 오작동 가능성	낮음	보통	높음	N/A
컴포넌트의 복잡성	보통	높음		N/A

Table 30-1 컴포넌트 분류

순 서	컴포넌트	권장	사용가능	비 권장	사용금지
1	ActiveX				●
2	Applet				●
3	AutoComplete			●	
4	Calendar		●		
5	Chart				●
6	CheckBox	●			
7	DatePicker	●			
8	Editor				●
9	Flash				●
10	FlipToggle	●			
11	FloatingLayer			●	
12	Generator		●		
13	GridView		●		
14	Group	●			
15	IFrame			●	
16	InputCalendar				●
17	InputBox	●			
18	Image	●			
19	Menu			●	
20	MultiSelect				●
21	MultiUpload				●
22	Output	●			
23	PageControl	●			
24	PageList		●		

순 서	컴포넌트	권장	사용가능	비 권장	사용금지
25	Secret	●			
26	Popup			●	
27	Radio	●			
28	Repeat			●	
29	ScrollView	●			
30	SearchBox		●		
31	SelectBox		●		
32	Slider	●			
33	SlideHide				●
34	Spinner	●			
35	Switch		●		
36	TabContainer				●
37	TabControl			●	
38	Trigger	●			
39	TextArea	●			
40	TextBox	●			
41	TreeView			●	
42	Upload				●
43	XHTML			●	
44	XSL				●
45	WindowContianer				●
46	WFrame	●			

30.3.1. 권장

모바일 웹 개발에 권장되는 컴포넌트는 아래와 같습니다.

- CheckBox
- DatePicker
- FlipToggle
- Group
- InputBox
- Image
- Output
- PageControl

- Secret
- Radio
- ScrollView
- Slider
- Spinner
- Trigger
- TextArea
- TextBox
- WFrame

30.3.2. 사용 가능

Table 30-2 사용 가능 컴포넌트

No.	사용가능	플랫폼					비고
		Android (2.1, .2)	Android(2.3)	Android(3.1)	Android(4.0)	IOS	
1	Calendar						
2	Generator						
3	GridView						
4	PageList						
5	SerachBox			• 닫기 버튼 미동작	• 닫기 버튼 미동작		
6	SelectBox						
7	Switch (터치 페이지 전환)						

30.3.3. 비권장

Table 30-3 비권장 컴포넌트

No.	컴포넌트	플랫폼					비고
		Android (2.1, .2)	Android(2.3)	Android(3.1)	Android(4.0)	IOS	
1	Auto Complete	• key 입력시 조회 되지 않음	• key 입력시 조회 되지 않음	• key 입력시 조회 되지 않음 • 그 외 기능 정상동작	• key 입력시 조회 되지 않음 • 목록 선택 되지 않음		

No.	컴포넌트	플랫폼					비고
		Android (2.1, .2)	Android(2.3)	Android(3.1)	Android(4.0)	IOS	
2	Float ingLa yer			• treeview와 z-Index 문 제	• treeview와 z-Index 문 제		
3	IFra me						
4	Men u						
5	Popu p						브라우저팝업사용
6	Repe at						
7	TabC ontr ol						데스크탑 브라우저에 최 적화된 컴포넌트
8	Tree View	• 펼치기 기능 미동작	• 펼치기 기능 미동작				
9	XHT ML						

30.3.4. 사용 금지

Table 30-4 사용 금지 컴포넌트

No.	컴포넌트	플랫폼					비고
		Android (2.1, .2)	Android(2.3)	Andr oid(3 .1)	Android(4.0)	IOS	
1	ActiveX						IE 전용
2	Applet						
3	Chart	• FF4.0 베타이상 에서 지원	• FF4.0 베타이상 에서 지원				
4	Editor						데스크탑 브라우저에 최적화된 컴포넌트
5	Flash						
6	InputCal	• 달력 아이콘 클릭			• selectbox		컴포넌트를 조합하여

No.	컴포넌트	플랫폼					비고
		Android (2.1, .2)	Android(2.3)	Android(3 .1)	Android(4.0)	IOS	
	endar	안됨			(native) 디자인		구현해야 함.
7	MultiSelect						다중선택되지 않음
8	MultiUpload						
9	SlideHide	• 복합 컴포넌트 추가시 동작하지 않음	• 복합 컴포넌트 추가시 동작하지 않음				데스크탑 브라우저에 최적화된 컴포넌트
10	TabContainer	• 탭선택시 동작하지 않음					데스크탑 브라우저에 최적화된 컴포넌트
11	Upload						
12	XSL	• 갤럭시S(2.1)만 정상동작	• 동작하지 않음				
13	Window Container	• 탭선택이 되지 않음 • IFrame이 정상동작하지 않음	• 탭선택이 되지 않음 • IFrame이 정상동작하지 않음			• 미동작	데스크탑 브라우저에 최적화된 컴포넌트

30.4 모바일 접근성

30.4.1. FlipToggle

30.4.1.1. **accessibility**

accessibility 속성 설정을 통해 iOS의 VoiceOver 및 Android의 TalkBack 기능을 지원할 수 있습니다.

accessibility="true"로 설정할 경우, 버튼을 HTML의 `<a>` 태그로 감싸고, **aria-valuenow** 및 **aria-valuetext** 같은 HTML 속성을 추가합니다.

accessibility 속성 설정을 통해 iOS의 VoiceOver 및 Android의 TalkBack 기능을 지원할 수 있습니다.

30.4.2. ScrollView

`accessibility` 속성 설정을 통해 iOS의 VoiceOver 및 Android의 TalkBack 기능을 지원할 수 있습니다.

30.4.3. Switch

30.4.3.1. `accessibility`

`accessibility` 속성 설정을 통해 iOS의 VoiceOver 및 Android의 TalkBack 기능을 지원할 수 있습니다.



`slide="false"`인 경우에만 동작합니다.

31. GridView의 senseReader 속성 사용 시 유의 사항

31.1 성능 비교 (예시)

GridView와 바인딩 된 데이터가 1,000건이고 `visibleRowNum="all"` 설정을 적용한 경우에 대한 예시입니다. `visibleRowNum="all"` 설정을 적용했기 때문에 1,000건의 데이터가 한번에 표시됩니다.

그림 31-1 `senseReader="false"` `visibleRowNum="all"`



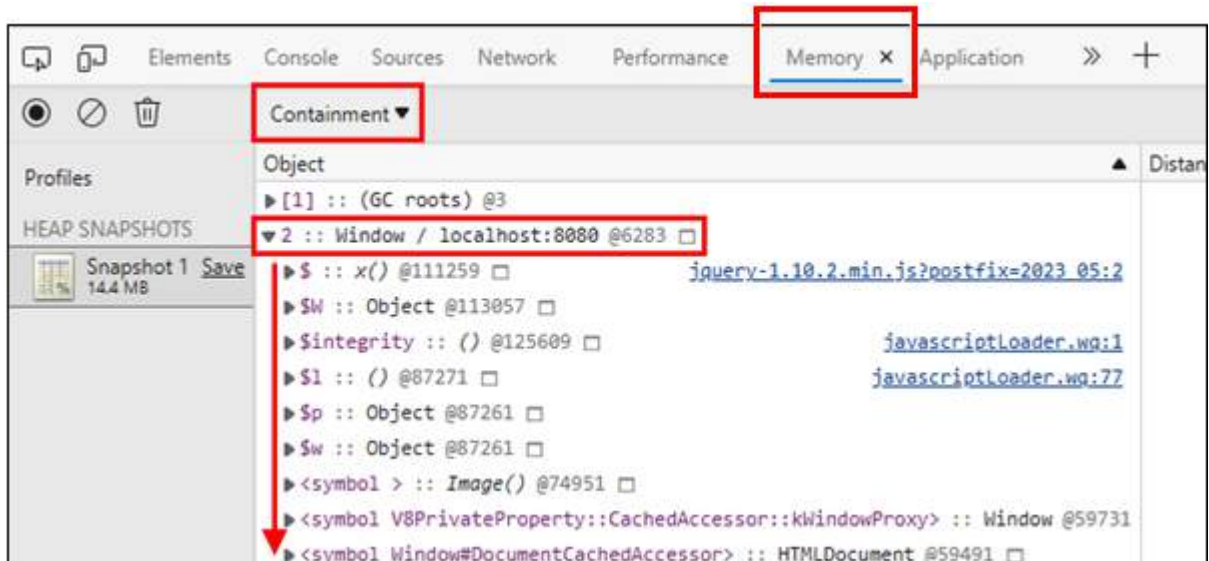
	컬럼1	컬럼2	컬럼3	컬럼4	컬럼5
1	data-001-01	data-001-02	data-001-03	data-001-04	data-001-05
2	data-002-01	data-002-02	data-002-03	data-002-04	data-002-05
3	data-003-01	data-003-02	data-003-03	data-003-04	data-003-05
4	data-004-01	data-004-02	data-004-03	data-004-04	data-004-05
5	data-005-01	data-005-02	data-005-03	data-005-04	data-005-05
6	data-006-01	data-006-02	data-006-03	data-006-04	data-006-05
7	data-007-01	data-007-02	data-007-03	data-007-04	data-007-05
8	data-008-01	data-008-02	data-008-03	data-008-04	data-008-05
9	data-009-01	data-009-02	data-009-03	data-009-04	data-009-05
10	data-010-01	data-010-02	data-010-03	data-010-04	data-010-05
11	data-011-01	data-011-02	data-011-03	data-011-04	data-011-05
...	...	...	...	...	...
995	data-995-01	data-995-02	data-995-03	data-995-04	data-995-05
996	data-996-01	data-996-02	data-996-03	data-996-04	data-996-05
997	data-997-01	data-997-02	data-997-03	data-997-04	data-997-05
998	data-998-01	data-998-02	data-998-03	data-998-04	data-998-05
999	data-999-01	data-999-02	data-999-03	data-999-04	data-999-05
1000	data-000-01	data-000-02	data-000-03	data-000-04	data-000-05

위의 결과는 겉으로만 보기에 `senseReader="true"`인 경우와 `senseReader="false"`인 경우에는 다른 차이가 없습니다. 성능 비교를 위해 메모리 사용량과 DOM 개체 수를 확인해 보겠습니다.

31.1.1. Heap 메모리

브라우저의 개발 도구에서 Memory 탭을 선택한 후 Containment를 엽니다. "mf_"로 시작하는 GridView의 개체 노드를 확인할 수 있습니다.

그림 31-2 개발자 도구 (Memory - Containment)

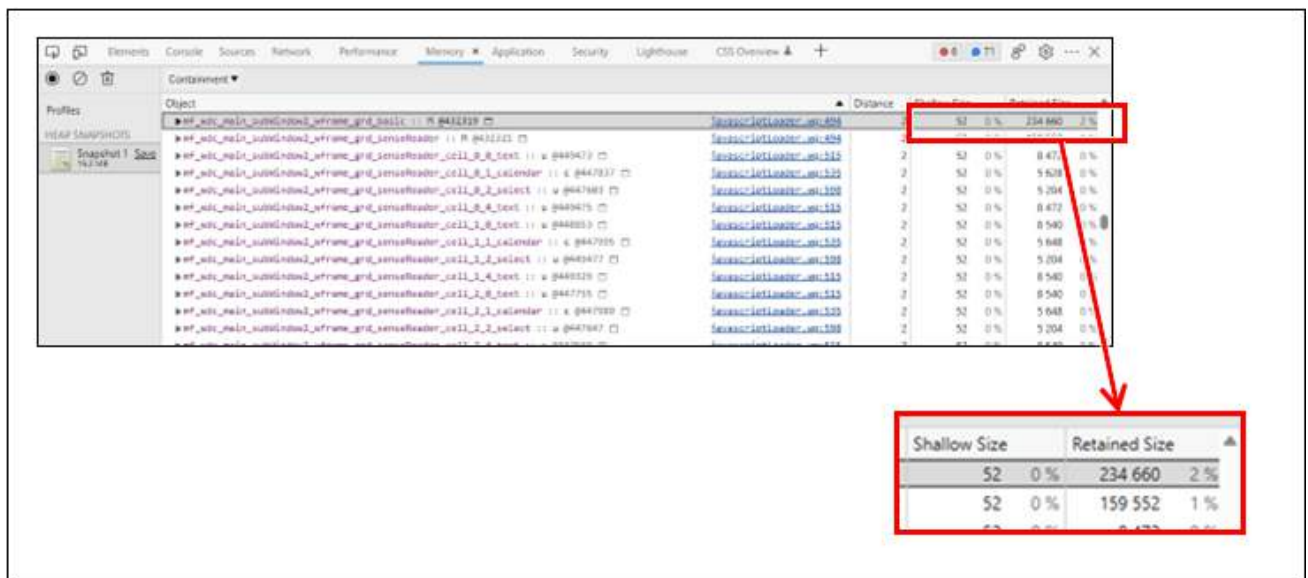


Heap 스냅 샷으로 GridView의 메모 사용량을 확인할 수 있습니다.

31.1.1.1. senseReader="false"

GridView (id="mf_wdc_main_subWindow2_wframe_grd_basic") 개체가 사용하는 Retained Size의 값이 약 234kb 임을 알 수 있습니다. Retained Size는 가비지 컬렉션이 해당 개체를 제거할 때 반환될 수 있는 Heap 메모리의 크기입니다.

그림 31-3 Heap 스냅샷 (senseReader="false")



31.1.3. senseReader 메모리 사용량

브라우저의 작업 관리자에서 메모리 사용량을 확인할 수 있습니다.

메모리

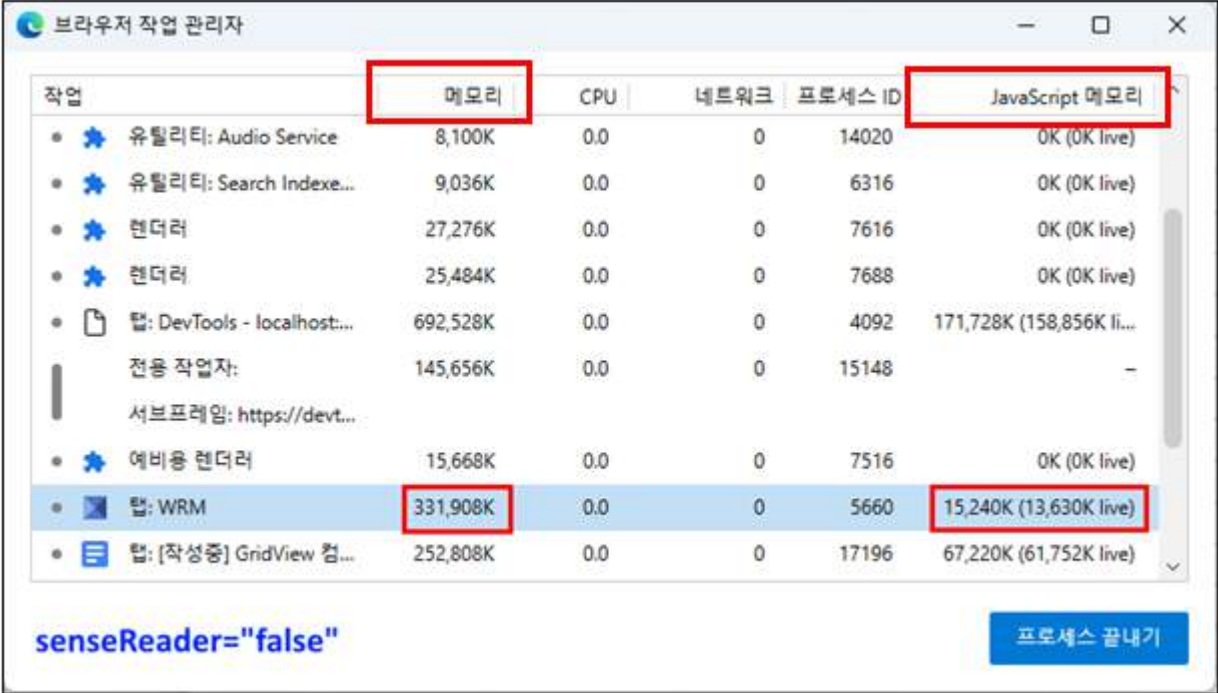
네이티브 메모리의 사용량을 보여주며 DOM 개체는 이 네이티브 메모리에 저장됩니다.

JavaScript 메모리

JavaScript Heap 메모리의 사용량을 보여줍니다. 괄호 안에 표시된 값은 페이지에서 도달 가능한 활성 개체의 메모리 사용량을 보여줍니다.

31.1.3.1. senseReader="false"

그림 31-5 브라우저 작업 관리자



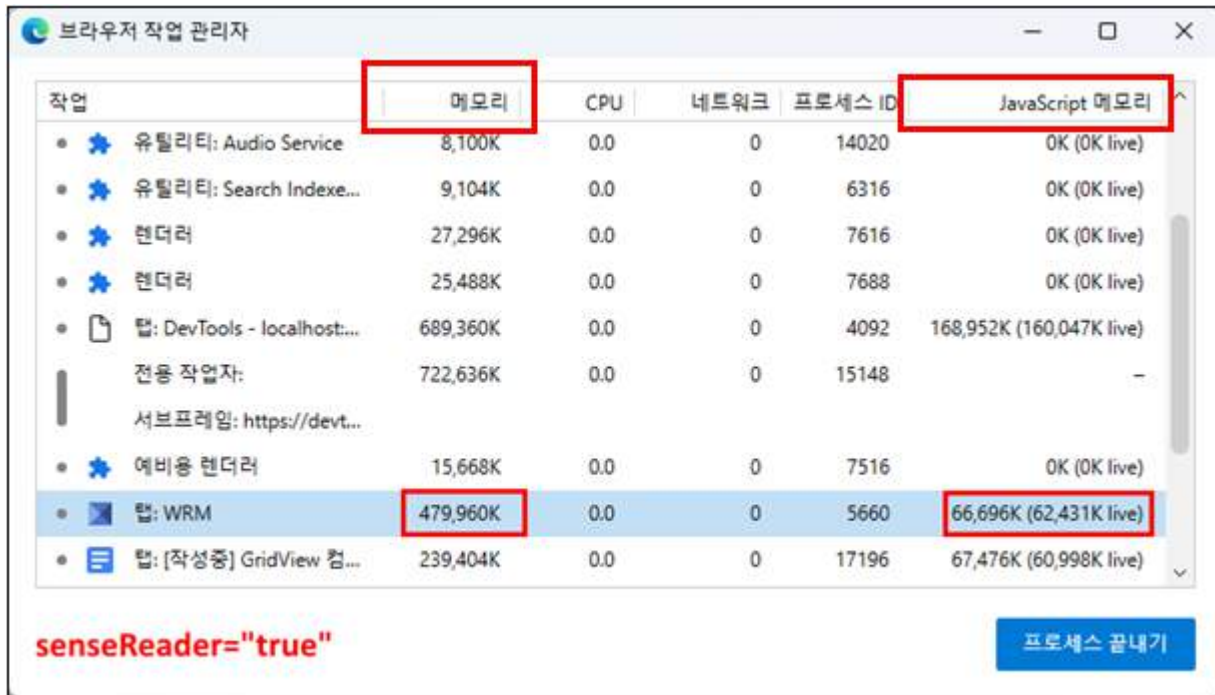
작업	메모리	CPU	네트워크	프로세스 ID	JavaScript 메모리
유틸리티: Audio Service	8,100K	0.0	0	14020	OK (OK live)
유틸리티: Search Indexe...	9,036K	0.0	0	6316	OK (OK live)
렌더러	27,276K	0.0	0	7616	OK (OK live)
렌더러	25,484K	0.0	0	7688	OK (OK live)
태: DevTools - localhost:...	692,528K	0.0	0	4092	171,728K (158,856K li...
전용 작업자:	145,656K	0.0	0	15148	-
서브프레임: https://devt...					
예비용 렌더러	15,668K	0.0	0	7516	OK (OK live)
태: WRM	331,908K	0.0	0	5660	15,240K (13,630K live)
태: [작성중] GridView 컴...	252,808K	0.0	0	17196	67,220K (61,752K live)

senseReader="false"

프로세스 끝내기

31.1.3.2. `senseReader="true"`

그림 31-6 브라우저 작업 관리자



31.1.4. 결론

다음은 위의 예제를 비교한 표입니다. `senseReader="true"`인 경우, `senseReader="false"`와 비교하여 더 많은 리소스를 사용한다는 점을 알 수 있습니다.

Table 31-1 성능 비교

항목	<code>senseReader="false"</code>	<code>senseReader="true"</code>
Heap 메모리 (Retained Size)	234kb	334kb (약 1.5배 높음.)
DOM 개수	1	5,002
(네이티브) 메모리	311,908kb	479,960kb (약 1.5배 높음.)
JavaScript 메모리	13,630kb	62,431kb (약 4.5배 높음.)

위의 비교는 단순한 기본 예제를 테스트한 것으로 실제 프로젝트에서는 `senseReader="true"` 설정이 초래하는 영향도가 훨씬 클 수 있습니다. 수천 건 이상의 데이터가 GridView와 바인딩되거나, 수십 개의 컬럼이 존재하거나, TabControl의 여러 탭에 GridView들이 존재할 수도 있습니다.

기술적인 측면에서 `senseReader="true"` 설정으로 인한 변화를 이해하고 다른 속성들과의 상호 작용을 확인한 후 `senseReader` 속성을 사용할 것을 권장합니다.

31.2 렌더링 방식 (drawType)

31.2.1. virtual 렌더링 (drawType="virtual")

다음은 컬럼 5개와 행 100개로 구성된 GridView를 화면에 표시한 예시입니다. 현재 5개의 행이 브라우저에 출력되어 있습니다.

그림 31-7 GridView (컬럼 5개 X 행 100개)

	컬럼1	컬럼2	컬럼3	컬럼4	컬럼5
1	data-001-01	data-001-02	data-001-03	data-001-04	data-001-05
2	data-002-01	data-002-02	data-002-03	data-002-04	data-002-05
3	data-003-01	data-003-02	data-003-03	data-003-04	data-003-05
4	data-004-01	data-004-02	data-004-03	data-004-04	data-004-05
5	data-005-01	data-005-02	data-005-03	data-005-04	data-005-05

브라우저의 개발자 도구를 사용하여 렌더링 결과를 확인하면 <tbody> 태그 하위에는 현재 화면에 출력된 5개의 행수와 동일하게 <tr> 태그는 5개만 존재합니다. 스크롤 아래로 이동해도 <tr> 태그 개수는 5개로 유지됩니다. 각 <tr> 태그의 rowindex는 변경되지만 전체 <tr> 태그 개수는 5개로 유지됩니다. 이와 같은 렌더링 방식을 drawType="virtual" 설정을 통한 virtual 방식이라고 합니다.

그림 31-8 GridView 렌더링 결과

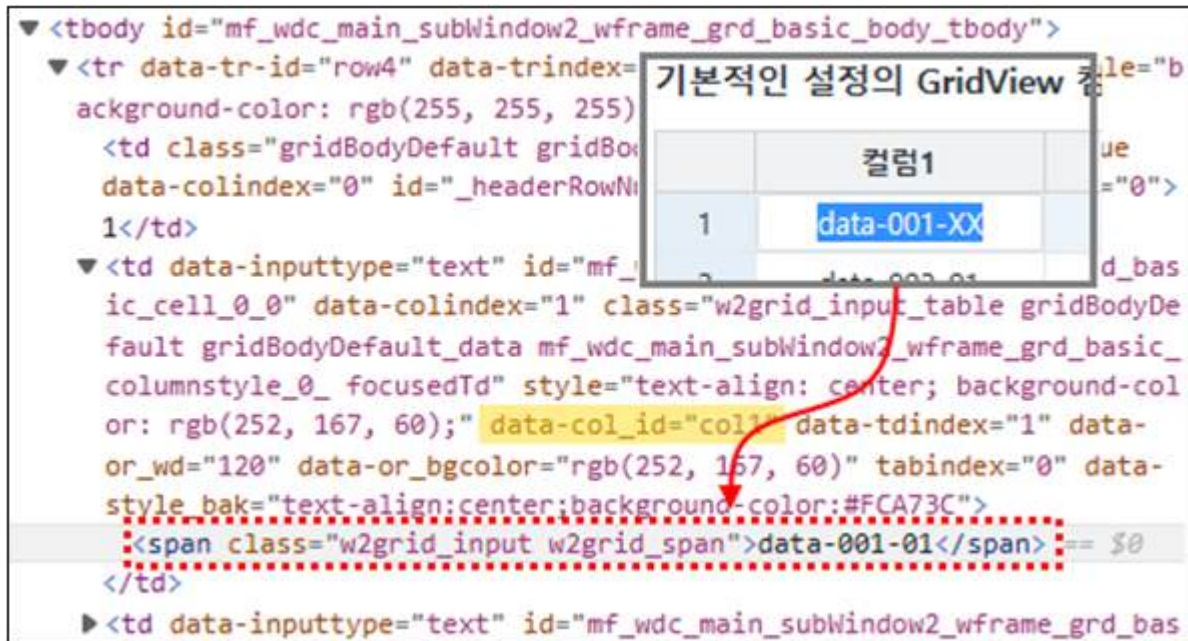
```

▼ <tbody id="mf_wdc_main_subWindow2_wframe_grd_basic_body_tbody">
  ▶ <tr data-tr-id="row4" data-trindex="0" class="grid_body_row" style="background-color: rgb(255, 255, 255);"> ... </tr>
  ▶ <tr data-tr-id="row4" data-trindex="0" class="grid_body_row" style="background-color: rgb(245, 245, 245);"> ... </tr>
  ▶ <tr data-tr-id="row4" data-trindex="0" class="grid_body_row" style="background-color: rgb(255, 255, 255);"> ... </tr>
  ▶ <tr data-tr-id="row4" data-trindex="0" class="grid_body_row" style="background-color: rgb(245, 245, 245);"> ... </tr>
  ▶ <tr data-tr-id="row4" data-trindex="0" class="grid_body_row" style="background-color: rgb(255, 255, 255);"> ... </tr>
  ▶ <tr data-tr-id="row4" data-trindex="0" class="grid_body_row" style="background-color: rgb(245, 245, 245);"> ... </tr>
  ▶ <tr data-tr-id="row4" data-trindex="0" class="grid_body_row" style="background-color: rgb(255, 255, 255);"> ... </tr>
</tbody>

```

각 셀의 값은 <input> 태그가 아닌 태그 내에 존재합니다. gridView.column의 inputType에 따라 화면 렌더링에 적합한 DOM 개체들이 사용됩니다. 참고로 아래 예제에서 gridView.column의 inputType은 "text"입니다.

그림 31-9 <tr> 노드 확장



해당 셀 데이터를 편집하는 동안에 렌더링 상태는 변경되지 않습니다. 이는 셀 편집 시 해당 셀의 DOM 구성 자체가 변경되는 것이 아니라, 편집을 위한 <input> 태그 등 별도의 DOM 개체가 개발자가 지정한 컬럼 설정에 따라 해당 위치의 Z축 위로 적절히 이동하여 나타나는 것이기 때문입니다. F12 개발자 도구를 이용해서 이를 쉽게 확인해 볼 수 있습니다. 다음은 Z축 위치를 덮고 있는 <input> 태그의 좌표를 살짝 변경하여 우하단으로 약간 이동시킨 모습입니다.

그림 31-10 <input> 태그 좌표 이동 후



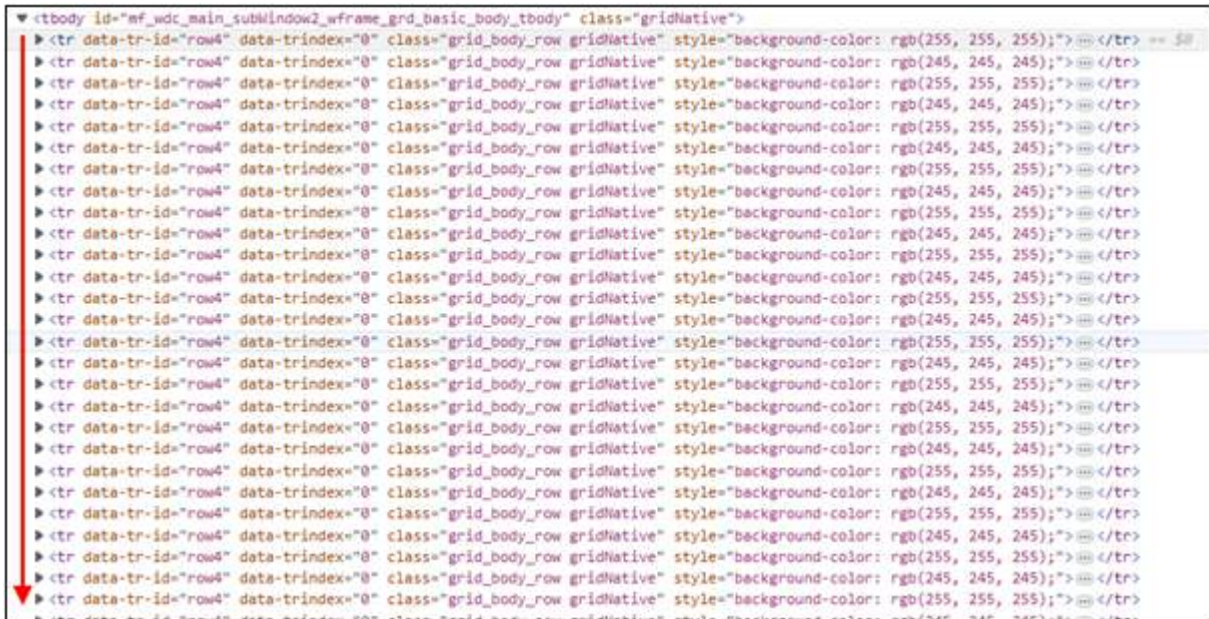
GridView의 이 같은 virtual 방식의 렌더링은 상대적으로 많은 대량의 데이터를 용이하게 관리할 수 있는 이점을 제공합니다. 불필요한 대량의 DOM 개체들을 생성하지 않기 때문에 페이징 기법을 사용하지 않고서도 웹 브라우저의 부하 없이 대량의 데이터를 다룰 수 있습니다. 보통 컬럼의 수가 많고 복잡한 편집 컴포넌트를 사용할 수록 DOM 개체에 의한 부하가 심해지지만, virtual 방식으로 렌더링할 경우 부하가 늘어는 것을 방지할 수 있습니다.

하지만 virtual 렌더링 방식의 단점도 있습니다. 웹 접근성 지원을 위한 스크린 리더 기능의 경우 시각 장애인에게 읽어줘야 할 대상이 왜곡될 가능성이 매우 높습니다. 일례로 `<input>` 태그로 인식되어야 할 셀이 `<span>` 태그로 인식될 수 있습니다.

31.2.2. native 렌더링 (drawType="native")

drawType="native" 설정을 사용하면 바인딩 된 데이터의 건수와 동일한 건수의 `<tr>` 태그를 생성합니다. 아래 그림을 보면 GridView에 바인딩된 전체 데이터 건수와 1:1로 대응하여 `<tr>` 태그가 생성된 것을 확인할 수 있습니다.

그림 31-11 drawType="native"



native 렌더링 방식은 virtual 렌더링 방식에 비해 브라우저에 가해지는 부하가 크고 무겁습니다. senseReader="true" 설정까지 적용하면 visibleRowNum="all" 설정을 적용한 것과 거의 동일한 수준의 영향을 줍니다. native 렌더링과 virtual 렌더링 방식의 차이는 virtual 렌더링 방식에서는 GridView의 높이가 자동으로 변경된다는 것입니다.

native 렌더링 방식 사용 시 고려할 점

바인딩 된 데이터 건수가 많거나 senseReader="true" 설정이 적용된 경우 높은 부하를 감당해야 함

데이터를 편집하지 않는 읽기 전용 GridView 컴포넌트인 경우 권장

데이터를 편집해야 하는 GridView 컴포넌트인 경우, 바인딩 되는 데이터 건수를 가급적 적게 유지할 것을 권장

GridView 대신 Generator 사용을 검토

native 렌더링 방식으로 해결할 수 있는 대표적인 상황은 아래와 같습니다.

(1) UI적인 측면의 불만을 해소해야 할 때

아래의 예시에서 보듯이 `drawType="virtual"` 설정을 적용할 경우 GridView 행 일부가 가려지는 경우가 발생합니다. 이 경우 native 렌더링 방식을 활용하는 것이 한 가지 해법이 될 수 있습니다.

그림 31-12 drawType="virtual"

	컬럼1	컬럼2	컬럼3	컬럼4	컬럼5
1	data-001-01	data-001-02	data-001-03	data-001-04	data-001-05
2	data-002-01	data-002-02	data-002-03	data-002-04	data-002-05
3	data-003-01	data-003-02	data-003-03	data-003-04	data-003-05
4	data-004-01	data-004-02	data-004-03	data-004-04	data-004-05
5	data-005-01	data-005-02	data-005-03	data-005-04	data-005-05

(2) 비-웹스퀘어적인 방법을 통한 GridView의 변경 사항을 유지

웹스퀘어 API가 아닌 (비-웹스퀘어적인) 방법을 통해 GridView를 변경하고, 이 변경 사항을 스크롤바 이동에 상관 없이 유지해야 할 경우 `drawType="native"` 설정을 사용하여 해결할 수 있습니다.

다음은 Vanilla JS 코드를 실행하여 특정 셀의 색상을 변경한 경우입니다.

	컬럼1	컬럼2	컬럼3	컬럼4	컬럼5
1	data-001-01	data-001-02	data-001-03	data-001-04	data-001-05
2	data-002-01	data-002-02	data-002-03	data-002-04	data-002-05
3	data-003-01	data-003-02	data-003-03	data-003-04	data-003-05
4	data-004-01	data-004-02	data-004-03	data-004-04	data-004-05
5	data-005-01	data-005-02	data-005-03	data-005-04	data-005-05

Code 31-2 Vanilla JS 코드

```
1 document.querySelector("#mf_wdc_main_subWindow2_wframe_grd_basic_cell_1_1")
2 .style
3 .setProperty("color", "red");
```

위와 같이 특정 셀의 색상을 변경한 후, 스크롤바를 내리고 다시 올리면 셀의 변경한 셀의 색상이 유지되지 않고 원래 색상으로 돌아오는 것을 볼 수 있습니다.

	컬럼1	컬럼2	컬럼3	컬럼4	컬럼5
1	data-001-01	data-001-02	data-001-03	data-001-04	data-001-05
2	data-002-01	data-002-02	data-002-03	data-002-04	data-002-05
3	data-003-01	data-003-02	data-003-03	data-003-04	data-003-05
4	data-004-01	data-004-02	data-004-03	data-004-04	data-004-05
5	data-005-01	data-005-02	data-005-03	data-005-04	data-005-05

변경된 셀 색상을 유지하려면 GridView의 `setCellColor()` 함수를 사용하여 셀 색상을 변경해야 합니다.

Code 31-3 setCellColor() 사용 예

```
1 gridView1.setCellColor(1, 1, "red");
```

그러나 WebSquare에서 필요한 API를 지원하지 않거나, jQuery 또는 Vanilla JS 등을 직접 사용할 수밖에 없는 (특정 조건에서 새로운 DOM 구조를 직접 생성하여 삽입해야 하는 등의) 경우에는, `drawType="native"` 렌더링 방식을 사용하여 스크롤바 이동에 상관없이 GridView의 변경 사항을 유지할 수 있습니다.

32. TabControl의 탭 콘텐츠 렌더링 가이드

32.1 탭 콘텐츠 렌더링

TabControl을 포함하는 화면을 로딩할 경우, 첫 번째 탭은 항상 화면에 표시됩니다. TabControl이 여러 개의 탭을 포함할 경우, 첫 번째 탭을 제외한 나머지 탭들은 사용자가 열 수도 있고 그렇지 않을 수도 있습니다. 즉, 탭이 4개 있을 때 사용자가 모든 탭을 열어보지 않는 경우가 존재할 수 있습니다. 혹은 탭이 20개 있을 경우, 대부분의 경우 일부 탭만 열어서 사용하고 20개 탭을 모두 여는 경우는 극히 적을 것입니다.

TabControl이 포함하고 있는 모든 탭을 초기에 렌더링할 경우 성능에 부담을 줄 수 있습니다. 초기 데이터 적재 측면에서도 사용하지 않을지도 모르는 데이터를 가져오기 위해 네트워크 대역폭을 낭비하고 로딩 시간을 크게 늘릴 수도 있기 때문입니다. 즉, 탭의 개수가 많고 탭 콘텐츠가 복잡할 경우, 사용하지 않을지도 모르는 탭들의 콘텐츠에 포함된 컴포넌트들을 대량으로 로딩하고 렌더링하고 이는 곧 성능 관련 문제로 이어질 수 있습니다.

특히 대량의 데이터를 사용하는 GridView를 포함하는 탭이 존재하거나, UDC 및 Generator 등 렌더링 비용이 상대적으로 높은 컴포넌트를 탭 콘텐츠가 포함할 경우, 렌더링으로 인한 부하는 더욱 증가합니다.

32.2 `alwaysDraw`

`alwaysDraw` 속성을 사용하면 TabControl이 포함하는 탭 콘텐츠의 렌더링 시점을 결정할 수 있습니다. TabControl이 포함된 화면이 처음 로딩될 경우, TabControl의 모든 탭 콘텐츠를 렌더링(`alwaysDraw="true"`)하거나 혹은 첫 번째 탭 콘텐츠만 렌더링(`alwaysDraw="false"`)할 수 있습니다. 첫 번째 탭 콘텐츠만 렌더링하는 경우, 나머지 탭 콘텐츠는 사용자 이벤트 혹은 스크립트 이벤트로 각 탭이 선택되는 시점에 해당 탭 콘텐츠가 렌더링됩니다.



`frameMode` 속성

TabControl의 `<content>` 태그의 `frameMode` 속성 설정에 따라 렌더링 방식이 일부 변경됩니다.

32.2.1. `alwaysDraw="true"`

다음은 TabControl에 `alwaysDraw="true"` 설정을 적용하고, TabControl의 `<content>` 태그에 `alwaysDraw="true"` 및 `frameMode="wframe"` 설정을 적용한 경우입니다.

Table 32-1 적용 설정

	alwaysDraw	frameMode
TabControl	"true"	
TabControl - <content>	"true"	"wframe"

Code 32-1 alwaysDraw="true" frameMode="wframe"

```

1 <w2:tabControl id="tac_main" alwaysDraw="true" class="wq_tab">
2   <w2:tabs id="tabs1" label="탭1" />
3   <w2:tabs id="tabs2" label="탭2" />
4   <w2:tabs id="tabs3" label="탭3" />
5   <w2:tabs id="tabs4" label="탭4" />
6   <w2:content id="content1"
7     alwaysDraw="true" frameMode="wframe" src="/.../tab1.xml" />
8   <w2:content id="content2"
9     alwaysDraw="true" frameMode="wframe" src="/.../tab2.xml" />
10  <w2:content id="content3"
11    alwaysDraw="true" frameMode="wframe" src="/.../tab3.xml" />
12  <w2:content id="content4"
13    alwaysDraw="true" frameMode="wframe" src="/.../tab4.xml" />
14 </w2:tabControl>

```

위의 코드를 적용하여 예제 페이지를 로딩한 직후, 아무런 추가 작업도 수행하지 않고 F12 개발자 도구를 열어보겠습니다. Elements 탭을 확인하면, `alwaysDraw="true"` 설정을 적용했기 때문에 다음과 같이 모든 탭 콘텐츠가 렌더링된 것을 확인할 수 있습니다.

그림 32-1 초기 렌더링 결과



또한 F12 개발자 도구에서 Network 탭을 확인해보면 다음과 같이 동시에 관련된 모든 JS 파일들이 다운로드 되는 것을 확인할 수 있습니다. (SP5 기준)

그림 32-2 다운로드된 JS 파일 목록

Name
 alwaysDraw.js?postfix=16817144860264930.197778996268
 contentHeader.js?postfix=16817144860495123.124709286027
 tab1.js?postfix=1681714486070255.8159623844536
 tab2.js?postfix=16817144860709151.765629124711
 tab3.js?postfix=16817144860706145.934395318133
 tab4.js?postfix=16817144860706400.756634670317

하지만 실제 프로젝트에서는 위의 예제처럼 탭 콘텐츠가 소수의 단순한 컴포넌트만 포함하는 경우는 별로 없습니다. 수많은 컴포넌트들이 대량의 데이터와 함께 복잡하게 구성되어 있거나, WFrame이 여러 단계에 걸쳐 네스팅되어 존재하는 경우도 많습니다

32.2.2. alwaysDraw="false" (기본 값)

alwaysDraw="false" 설정을 적용할 경우, 첫 번째 탭 콘텐츠만 렌더링되고 나머지 탭의 콘텐츠들은 사용자 이벤트나 스트립트 이벤트로 해당 탭을 선택할 경우 렌더링됩니다. 또한 TabControl의 <content> 태그의 frameMode 속성 설정에 따라 렌더링 방식이 추가로 변경됩니다.

TabControl의 <content> 태그의 frameMode 속성으로 아래와 같은 세 가지 값을 지정할 수 있습니다.

Table 32-2 frameMode 옵션 설명

옵션	사용 프레임	비고
"iframe"	IFrame	비권장.
"wframe"	WFrame	alwaysDraw 속성 설정에 따라 탭 콘텐츠의 렌더링 동작이 결정됨.
"wframePreload"	WFrame	초기 화면 로딩 시, 첫 번째 탭을 제외한 나머지 탭의 콘텐츠에 속하는 JS 파일을 로딩하고 개체도 생성. 단, 렌더링은 수행하지 않음. (주의! 탭 콘텐츠 scwin.onpageLoad() 이벤트 핸들러는 실행되지 않음.)



WFrameMode

SP4 이후 부터는 탭 내부의 구성이 극히 단순한 경우를 제외하면, <content> 태그의 frameMode 속성은 대부분의 경우 "wframe" 또는 "wframePreload"로 설정합니다.

또한 F12 개발자 도구에서 Network 탭을 확인해보면 TabControl이 포함된 페이지와 첫 번째 탭의 JS 파일들만 다운로드된 것을 확인할 수 있습니다.

그림 32-4 다운로드된 JS 파일 목록

Name
 alwaysDraw.js?postfix=168171487897041.69072714254707
 contentHeader.js?postfix=16817148789938033.623874567364
 tab1.js?postfix=16817148790172702.2022455151064

마우스로 네 번째 탭을 클릭한 후 (즉, 네 번째 탭을 최초로 선택한 후), 다시 F12 개발자 도구의 Elements 탭을 확인해 보겠습니다. 방금 선택한 네 번째 탭이 추가적으로 렌더링 된 것을 볼 수 있습니다.

그림 32-5 네 번째 탭 선택 후 화면 렌더링 결과

▼	<div id="mf_wdc_main_subWindow3_wframe_tac_main" class="w2tabcontrol wq_tab">
▶	<ul id="mf_wdc_main_subWindow3_wframe_tac_main_tabhost" class="w2tabcontrol_tabho
▼	<div id="mf_wdc_main_subWindow3_wframe_tac_main_container" class="w2tabcontrol_co
탭1	▼ <div id="mf_wdc_main_subWindow3_wframe_tac_main_contents_content1" class="w2tab "true" style="display: none; visibility: hidden;"> ▼ <div id="mf_wdc_main_subWindow3_wframe_tac_main_contents_content1_body" class ▶ <div id="mf_wdc_main_subWindow3_wframe_tac_main_contents_content1_body_wq_u </div> </div> 렌더된 탭1의 컴포넌트들
탭2	<div id="mf_wdc_main_subWindow3_wframe_tac_main_contents_content2" class="w2tab "true" style="display: none; visibility: hidden;"></div>
탭3	<div id="mf_wdc_main_subWindow3_wframe_tac_main_contents_content3" class="w2tab "true" style="display: none; visibility: hidden;"></div>
탭4	▼ <div id="mf_wdc_main_subWindow3_wframe_tac_main_contents_content4" class="w2tab "false" style="display: block; visibility: visible;"> ▼ <div id="mf_wdc_main_subWindow3_wframe_tac_main_contents_content4_body" class abcontrol_contents_wrapper_selected"> ▶ <div id="mf_wdc_main_subWindow3_wframe_tac_main_contents_content4_body_wq_u </div> 렌더된 탭4의 컴포넌트들

또한 F12 개발자 도구의 Network 탭을 확인하면 네 번째 탭의 JS 파일이 다운로드 된 것을 확인할 수 있습니다.

그림 32-6 다운로드된 JS 파일

Name
 alwaysDraw.js?postfix=168171487897041.69072714254707
 contentHeader.js?postfix=16817148789938033.623874567364
 tab1.js?postfix=16817148790172702.2022455151064
 tab4.js?postfix=16817150541877456.848874598973

32.2.2.2. `frameMode="wframePreload"`

다음은 TabControl에 `alwaysDraw="false"` 설정을 적용하고, TabControl의 `<content>` 태그에 `alwaysDraw="false"` 및 `frameMode="wframePreload"` 설정을 적용한 경우입니다.

Table 32-4 적용 설정

	<code>alwaysDrawFrameMode</code>	<code>frameMode</code>
TabControl	"false"	
TabControl - <code><content></code>	"false"	"wframePreload"

Code 32-3 `alwaysDraw="false" frameMode="wframePreload"`

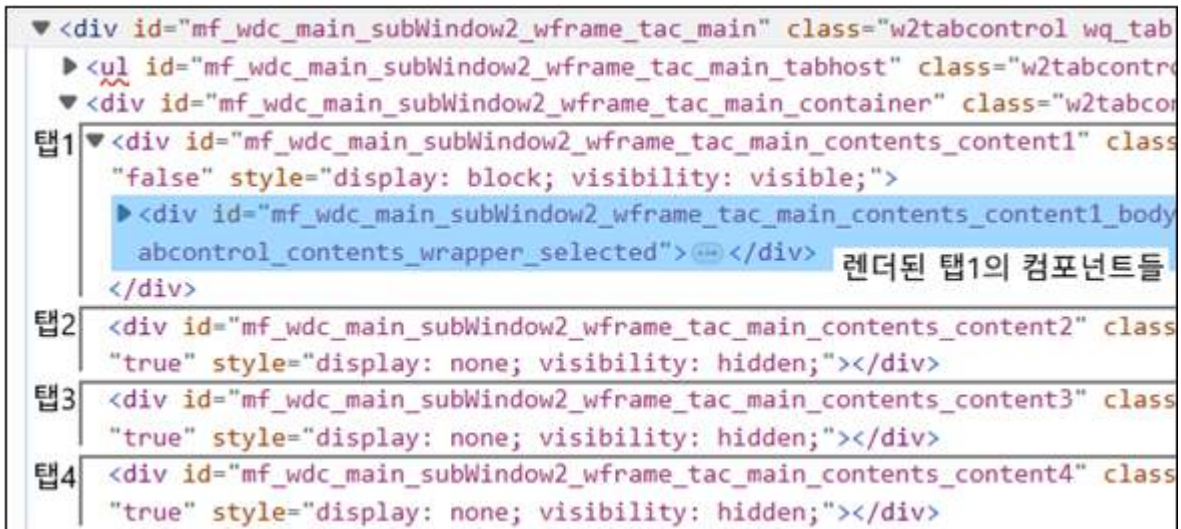
```

1 <w2:tabControl id="tac_main" alwaysDraw="false" class="wq_tab">
2   <w2:tabs id="tabs1" label="탭1" />
3   <w2:tabs id="tabs2" label="탭2" />
4   <w2:tabs id="tabs3" label="탭3" />
5   <w2:tabs id="tabs4" label="탭4" />
6   <w2:content id="content1"
7     alwaysDraw="false" frameMode="wframePreload"
8     src="/.../tab1.xml" />
9   <w2:content id="content2"
10    alwaysDraw="false" frameMode="wframePreload"
11    src="/.../tab2.xml" />
12   <w2:content id="content3"
13    alwaysDraw="false" frameMode="wframePreload"
14    src="/.../tab3.xml" />
15   <w2:content id="content4"
16    alwaysDraw="false" frameMode="wframePreload"
17    src="/.../tab4.xml" /></w2:tabControl>

```

위의 코드를 적용하여 예제 페이지를 로딩한 직후, 아무런 추가 작업도 수행하지 않고 F12 개발자 도구를 열어보겠습니다. Elements 탭을 확인하면, `alwaysDraw="false"` 설정을 적용했기 때문에 첫 번째 탭 콘텐츠만 렌더링 된 것을 확인할 수 있습니다. 두 번째 이후의 탭의 내용들은 모두 비어 있습니다.

그림 32-7 초기 렌더링 결과



그런데 F12 개발자 도구에서 Network 탭을 확인해보면 동시에 관련된 모든 JS파일들이 다운로드된 것을 확인할 수 있습니다. 즉, 아직 렌더링되지 않은 탭 콘텐츠의 JS 파일들이 다운로드되었습니다.

그림 32-8 다운로드된 JS 파일 목록

Name
alwaysDraw.js?postfix=16817144860264930.197778996268
contentHeader.js?postfix=16817144860495123.124709286027
tab1.js?postfix=1681714486070255.8159623844536
tab2.js?postfix=16817144860709151.765629124711
tab3.js?postfix=16817144860706145.934395318133
tab4.js?postfix=16817144860706400.756634670317

32.2.2.2.1. 탭 콘텐츠 렌더링과 객체 접근

`frameMode="wframePreload"`로 설정할 경우, 각 탭 콘텐츠는 아래와 같이 접근할 수 있습니다.

컴포넌트

접근 가능.

단, (id 호출 등을 통해) 컴포넌트에 직접적으로 접근하여 렌더링 작업을 할 수는 없음.

DataCollection

접근 가능.

컬럼 값을 설정할 수 있음.

탭을 선택할 경우, 탭이 렌더링되고, `ref` 속성으로 바인딩된 컴포넌트에 데이터가 정상적으로 출력됨.

scwin 객체 및 scwin 객체의 속성

접근 가능.

각 컴포넌트와 바인딩된 이벤트 핸들러 및 메소드도 접근 가능.

단, 이벤트 핸들러 내부에 컴포넌트에 직접적으로 접근하는 코드가 존재할 경우, 이벤트 핸들러 및 메소드를 호출하면 오류가 발생.

32.3 frameMode="wframe"

탭의 콘텐츠가 렌더링되지 않은 경우, 해당 탭의 콘텐츠를 구성하고 있는 컴포넌트 등에 접근할 수 없습니다.

예제 페이지에 다음과 같이 두 개의 버튼을 구성하여 테스트 해보겠습니다. 이 버튼들을 클릭하면 **탭1**과 **탭2**에 존재하는 InputBox 컴포넌트에 임의의 값이 설정됩니다.

그림 32-9 **탭1**과 **탭2**에 포함된 컴포넌트에 접근하기 위한 예제



두 버튼의 **onclick** 이벤트 핸들러를 아래와 같이 구현합니다.

Code 32-4 **탭1**과 **탭2**에 포함된 컴포넌트에 접근하기 위한 코드

```

1 // 탭1-키1 값 설정 버튼 클릭 이벤트 핸들러
2 scwin.btn_tab1SetVal_onclick = function (e) {
3     const tabs1 = tac_main.getWindow(tac_main.getTabIndex("tabs1"));
4     tabs1.ipt_key1.setValue("탭1-키1 값을 설정합니다.");
5 };
6
7
8 // 탭2-키1 값 설정 버튼 클릭 이벤트 핸들러

```

```

9  scwin.btn_tab2SetVal_onclick = function (e) {
10     const tabs2 = tac_main.getWindow(tac_main.getTabIndex("tabs2"));
11     tabs2.ipt_key1.setValue("탭2-키1 값을 설정합니다.");
12 };

```

예제 페이지를 새로 고침한 후, 아무런 작업을 수행하지 않고 두 버튼을 차례로 클릭하면 `alwaysDraw` 및 `frameMode` 설정에 따라 아래와 같이 동작합니다.

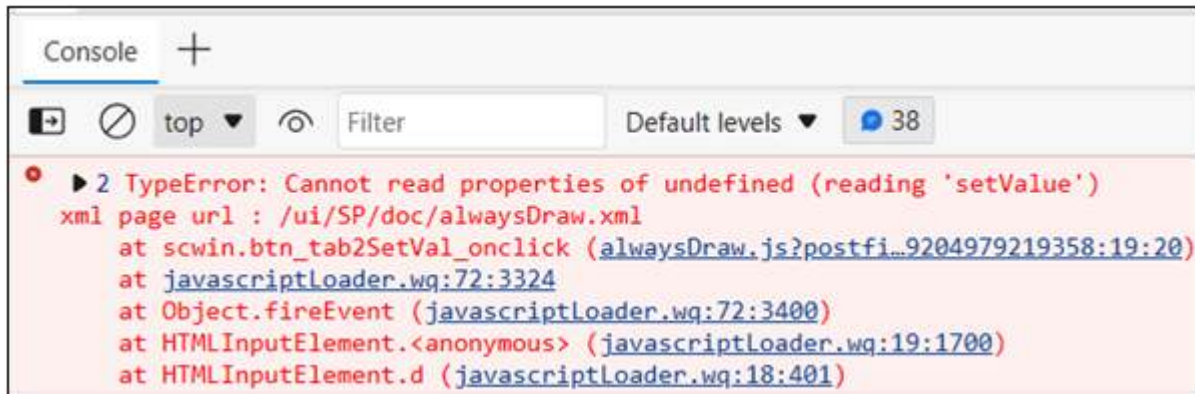
`alwaysDraw="true" frameMode="wframe"`

탭1-키1 값 설정 및 탭2-키1 값 설정 두 버튼 모두 정상적으로 동작합니다.

`alwaysDraw="false" frameMode="wframe"`

탭1-키1 값 설정 버튼은 정상적으로 동작하지만, 탭2-키1 값 설정 버튼을 클릭할 경우 다음과 같은 오류가 발생합니다.

그림 32-10 탭2-키1 값 설정 버튼을 클릭할 경우



위의 오류가 발생하는 이유는 두 번째 탭 콘텐츠가 아직 렌더링되지 않았고, 그 결과 탭 콘텐츠를 구성하는 컴포넌트들의 인스턴스도 아직 생성되지 않았기 때문입니다. 즉, `setValue()` 메소드를 제공해야 할 `ipt_key1` 컴포넌트가 아직 인스턴스화 되지 않은 상태(`undefined`)이므로 위와 같은 오류가 발생하게 됩니다.

하지만, 두 번째 탭을 클릭한 후, 탭2-키1 값 설정 버튼을 클릭하면 오류가 나타나지 않습니다.

32.4 frameMode="wframePreload"

다음은 `TabControl`에 `alwaysDraw="false"` 설정을 적용하고, `TabControl`의 `<content>` 태그에 `alwaysDraw="false"` 및 `frameMode="wframePreload"` 설정을 적용한 경우입니다.

Table 32-5 적용 설정

	alwaysDrawFrameMode	frameMode
TabControl	"false"	
TabControl - <content>	"false"	"wframePreload"

Code 32-5 alwaysDraw="false" frameMode="wframePreload"

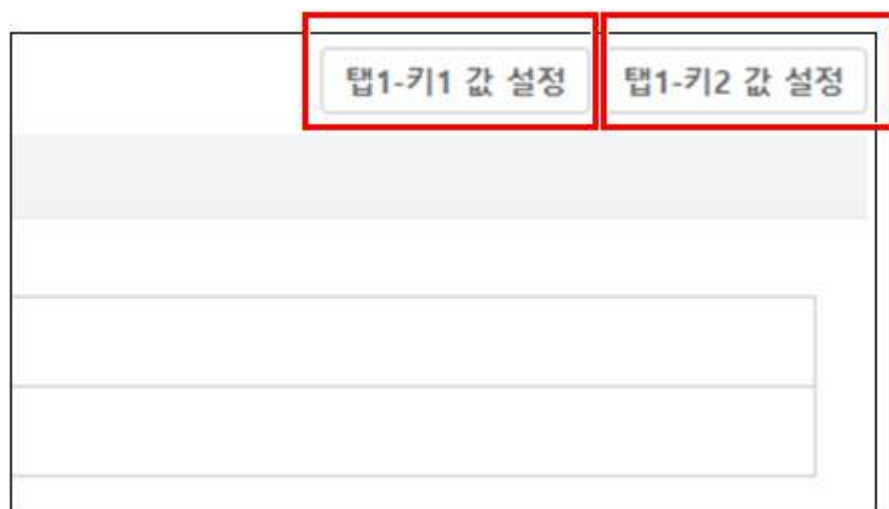
```

1 <w2:tabControl id="tac_main" alwaysDraw="false" class="wq_tab">
2   <w2:tabs id="tabs1" label="탭1" />
3   <w2:tabs id="tabs2" label="탭2" />
4   <w2:tabs id="tabs3" label="탭3" />
5   <w2:tabs id="tabs4" label="탭4" />
6   <w2:content id="content1"
7     alwaysDraw="false" frameMode="wframePreload"
8     src="/.../tab1.xml" />
9   <w2:content id="content2"
10    alwaysDraw="false" frameMode="wframePreload"
11    src="/.../tab2.xml" />
12   <w2:content id="content3"
13    alwaysDraw="false" frameMode="wframePreload"
14    src="/.../tab3.xml" />
15   <w2:content id="content4"
16    alwaysDraw="false" frameMode="wframePreload"
17    src="/.../tab4.xml" /></w2:tabControl>

```

예제 페이지에 다음과 같이 두 개의 버튼을 구성하여 테스트 해보겠습니다. 이 버튼들을 클릭하면 탭1과 탭2에 존재하는 InputBox 컴포넌트에 임의의 값이 설정됩니다.

그림 32-11 탭1과 탭2에 포함된 컴포넌트에 접근하기 위한 예제



두 버튼의 `onclick` 이벤트 핸들러를 아래와 같이 구현합니다.

Code 32-6 탭1과 탭2에 포함된 컴포넌트에 접근하기 위한 코드

```

1 // 탭1-키1 값 설정 버튼 클릭 이벤트 핸들러
2 scwin.btn_tab1SetVal_onclick = function (e) {
3     const tabs1 = tac_main.getWindow(tac_main.getTabIndex("tabs1"));
4     tabs1.ipt_key1.setValue("탭1-키1 값을 설정합니다.");
5 };
6
7
8 // 탭2-키1 값 설정 버튼 클릭 이벤트 핸들러
9 scwin.btn_tab2SetVal_onclick = function (e) {
10    const tabs2 = tac_main.getWindow(tac_main.getTabIndex("tabs2"));
11    tabs2.ipt_key1.setValue("탭2-키1 값을 설정합니다.");
12 };

```

‘탭2-키1 값 설정’ 버튼을 클릭해보면 다음과 같이 32.3 `frameMode="wframe"`에서 발생했던 것보다 더 긴 내용의 오류가 발생합니다.

그림 32-12 탭2-키1 값 설정 버튼을 클릭할 경우



단, 다음과 같이 DataCollection에 접근하여 컬럼 값을 설정하면 오류가 발생하지 않습니다.

Code 32-7 탭1과 탭2에 포함된 컴포넌트에 접근하기 위한 코드 (변경 후)

```

1 // 탭1-키1 값 설정 버튼 클릭 이벤트 핸들러
2 scwin.btn_tab1SetVal_onclick = function (e) {
3     const tabs1 = tac_main.getWindow(tac_main.getTabIndex("tabs1"));
4     tabs2.dma_map.set("key1", "탭2-키1 값을 설정합니다.");
5 };
6
7
8 // 탭2-키1 값 설정 버튼 클릭 이벤트 핸들러
9 scwin.btn_tab2SetVal_onclick = function (e) {
10    const tabs2 = tac_main.getWindow(tac_main.getTabIndex("tabs2"));
11    tabs2.dma_map.set("key1", "탭2-키1 값을 설정합니다.");
12 };

```

32.4.1. 탭 콘텐츠는 렌더링되지 않았지만 컴포넌트에 접근은 가능한 상황

컴포넌트 자체에 접근은 가능하지만 렌더링에 관련된 작업은 불가능한 상태를 살펴보겠습니다. 다음과 같이 ‘탭2-키1 값 설정’ 버튼 클릭 이벤트 핸들러 코드를 수정하여 F12 개발자 도구의 Console 탭을 살펴보겠습니다.

```

1 // 탭2-키1 값 설정 버튼 클릭 이벤트 핸들러
2 scwin.btn_tab2SetVal_onclick = function (e) {
3     const tabs2 = tac_main.getWindow(tac_main.getTabIndex("tabs2"));
4     console.log(tabs2.ipt_key1);
5 };

```

아래와 같은 출력이 나타나고, 이는 두 번째 탭의 `ipt_key1` 컴포넌트의 인스턴스는 정상적으로 생성되었다는 것을 의미합니다. 즉, 두 번째 탭이 렌더링되기 전이지만, 두 번째 컴포넌트에 포함된 컴포넌트에 접근은 가능합니다.

그림 32-13 options 및 render 속성



위 결과 이미지에서 `options`과 `render` 속성을 볼 수 있습니다.



비공개 속성

`options`과 `render` 속성은 비공개입니다. 공통 개발자와 같이 핵심적인 WebSquare 사용자들에게 보다 자세한 설명을 드리기 위한 목적으로만 언급됩니다.

`options` 속성에는 개발자가 컴포넌트를 작성하면서 지정한 각종 속성 값들이 저장됩니다. 위의 이미지를 살펴보면 `frameMode="wframePreload"`로 설정된 상태에서도 `options` 속성이 그 내용과 함께 정상적으로 생성된 것을 확인할 수 있습니다.

현재 `render` 속성에는 `null` 참조가 할당되어 있기 때문에 컴포넌트 자체에 접근은 가능하지만 렌더링에 관련된 작업은 불가능합니다. 즉, 내부적으로 `options` 속성에 접근하는 기능들은 정상적으로 동작하지만, `render` 속성에 접근하는 기능들은 대부분 오류가 발생합니다.

정상적인 경우, `render` 속성에는 해당 WebSquare 컴포넌트에 대응하는 실제 DOM 요소에 대한 참조가 저장됩니다. 다음과 같이 `<input>` 태그로 생성된 DOM 요소에 대한 참조가 설정되어 있어야 합니다.

그림 32-14 render 속성이 정상적으로 설정된 상태



32.4.1.1. 예외 상황

렌더링이 되지 않은 상태에서 DOM 개체에 접근할 경우 오류가 발생합니다. 결국 null 상태인 DOM의 속성 등에 접근하는 셈이기 때문입니다.

하지만 오류는 발생하지 않지만 정상 동작하는 경우가 존재합니다.

Code 32-8 오류 발생하지만 정상 동작

```

const tabs2 = tac_main.getWindow(tac_main.getTabIndex("tabs2"));
console.log(tabs2.ipt_key1.getValue());
tabs2.ipt_key1.setValue("탭2-키1 값을 설정합니다.");
tabs2.ipt_key1.setReadOnly(true); // 오류는 발생하지만 정상 동작함.
tabs2.ipt_key1.setDisabled(true); // 오류는 발생하지만 정상 동작함.
tabs2.ipt_key1.focus();
...등...

```

또한, 오류는 발생하지 않지만 정상 동작하지 않는 경우도 존재합니다.

Code 32-9 오류 없지만 정상 동작하지 않음

```

1  const tabs2 = tac_main.getWindow(tac_main.getTabIndex("tabs2"));
2  console.log(tabs2.ipt_key1.getMandatory());
3  console.log(tabs2.ipt_key1.getDisplayFormat());
4  console.log(tabs2.ipt_key1.getAllowChar());
5  console.log(tabs2.ipt_key1.getReadOnly());
6  console.log(tabs2.ipt_key1.getRef());
7  console.log(tabs2.ipt_key1.getDisabled());
8  console.log(tabs2.ipt_key1.getID());
9  tabs2.ipt_key1.setMandatory(true);
10 tabs2.ipt_key1.show(" "); // 오류는 없지만 정상 동작 안 함.
11 tabs2.ipt_key1.hide(); // 오류는 없지만 정상 동작 안 함.
12 tabs2.ipt_key1.setStyle("display", "none"); // 오류는 없지만 정상 동작 안 함.
13 ...등...

```


32.4.1.2. 대안

현재 주어진 리소스를 최대한 활용하여 유효성 검사 등의 상황에서 활용할 수 있는 여지는 존재합니다. 예를 들어서, `frameMode="wframePreload"` 설정을 적용한 상태에서 아래의 방법으로 `ipt_key1` 컴포넌트의 유효성 검사를 수행할 수 있습니다.

1. 탭의 `ipt_key1` 컴포넌트에 접근합니다.
2. `ipt_key1.getMandatory()` 메소드로 필수 입력 컴포넌트 여부를 확인합니다.
3. 만약 `ipt_key1`이 필수 입력 컴포넌트라면, 값이 입력되어 있는지 확인해야 합니다.
4. 그러나 `ipt_key1.getValue()` 메소드는 사용할 수 없습니다. (탭이 아직 렌더링 되어 있지 않을 수도 있기 때문입니다.)
5. 이 경우, `ipt_key1.getRef()` 메소드로 바인딩 된 `DataCollection` 정보를 가져옵니다.
6. `DataCollection` 정보를 활용해서 값이 입력되었는지 여부를 확인합니다.



한계점

실제 프로젝트 현장에서 이런 로직을 구현하려면 전제 조건도 많고, WRM 등의 기존 공통 라이브러리가 이런 기능을 지원하는지 확인해야 합니다.



권장

가령 필수적으로 모든 컴포넌트와 `DataCollection`을 바인딩하여 데이터를 관리하는 것을 권장합니다. 비록 단 한 건의 항목이라도 이런 방식으로 관리하는 것이 복잡한 프로젝트일수록, 또 프로젝트 후반으로 갈수록 유리합니다. (최근 시작되는 신규 프로젝트에서는 모든 컴포넌트를 필수적으로 `DataCollection`과 바인딩하여 사용하는 추세입니다.)

32.5 주의할 점

`TabControl`에 존재하는 탭의 개수, 각 탭들의 내용을 구성하는 수많은 컴포넌트들의 복잡도, 페이지 로딩 시 화면 구성을 위해 필요한 데이터 조회량 등을 감안하지 않고, 무조건 `alwaysDraw="true"` 설정을 적용하는 것을 지양해야 합니다. `TabControl`에 존재하는 탭의 개수가 많고, 각 탭들을 구성하는 컴포넌트들의 복잡도가 높은 경우가 많습니다. 이 같은 경우 `alwaysDraw="true"` 설정을 적용하면 구현은 간단하지만 성능이 저하될 수 있습니다.

32.5.1. 사례 1

상황

TabControl를 포함하고 있는 페이지를 로딩하면서 해당 페이지 및 TabControl의 각 탭을 구성하는 모든 컴포넌트들을 함께 초기화하고 기본 데이터를 동시에 적재하고자 하지만, 메인 영역과 특정 탭 간에 또는 다수의 탭 간에 UI/데이터 적인 상호관계가 존재하는 경우.

그림 32-15 화면 예시

계약메인

구분 ☒ 외주 ☐ 자재

'구분' 값 선택에 따라 두 번째 탭의 '계약유형' SelectList의 목록이 달라진다.

상세정보

계약업체 계약상세 보증보험 ...

계약유형 -선택- or -선택-

-선택- -선택-

외주계약 유형1
외주계약 유형2
외주계약 유형3
외주계약 유형4

물품계약 유형1
물품계약 유형2
년단가계약 유형1
년단가계약 유형2

계약업체 계약상세 보증보험 ...

선급금이행보증
계약이행보증
하자이행보증

두 번째 탭의 '계약유형' 선택값에 따라 이행보증 TR이 모두 나 출력되기도 하고 특정 TR만 출력되기도 한다.

일반적으로 SP4/SP5 기반의 WebSquare 화면 개발 시에는 페이지 로딩 시 호출되는 `scwin.onpageLoad()` 이벤트 핸들러에서 컴포넌트들의 구성을 초기화하고 기본 데이터를 적재하게 됩니다. 페이지 내부에 다수의 WFrame가 포함되어 있는 경우에도, 특별한 상황이 아니라면 부모 페이지에서 자식 WFrame 페이지의 컴포넌트 구성 등을 초기화하기보다는(비록 불가능한 것은 아니지만 않지만), 가급적 각 WFrame 페이지 자체의 `scwin.onpageLoad()` 이벤트 핸들러에서 스스로 필요한 컴포넌트들의 구성을 초기화하고 기본 데이터를 적재하는 경우가 많습니다. 원칙적으로는 본문에서 살펴본 TabControl의 각 탭들도 WFrame의 한 형태이므로 위와 같은 방식으로 각 탭들이 자체적인 `scwin.onpageLoad()` 이벤트 핸들러에서 필요한 컴포넌트들의 구성을 초기화하고 기본 데이터를 적재하면 됩니다. 그러나, 즉 렌더링 시점을 예상할 수 없는 상황에서는, 언급한 방식의 컴포넌트 구성 초기화가 곤란한 경우도 있습니다.

건설사에서 전자계약 업무에 사용되는 계약 상세 관리 페이지가 존재한다고 가정해보겠습니다. 이 페이지에는 주요 계약 정보를 담고 있는 메인 영역과 `alwaysDraw="false"` 설정이 적용된 10여개의 탭으로 이루어진 TabControl가 구성되어 있습니다. 계약 상세 정보를 편집하기 위해 특정 계약 건을 선택해 이 페이지를 열면 우선적으로 메인 영역

및 TabControl의 첫 번째 탭에 대한 컴포넌트 구성 및 기본 데이터 적재 작업이 수행될 것입니다. 그런데 하필이면 메인 영역의 '외주/자재' Radio 컴포넌트의 선택 값에 따라, 두 번째 탭의 '계약유형' SelectBox 컴포넌트의 선택 목록 데이터가 변경되고, 이 SelectBox 컴포넌트의 선택 값에 따라 세 번째 탭의 테이블 레이아웃에서 '보증증권' 관련 TR의 출력 토글된다고 가정해보도록 하겠습니다.

이런 경우 (아래의 사례는 구현 방식이나 프로젝트 내부에서 결정된 DataCollection 사용 표준 등에 따라 결과가 달라질 수는 있지만, 본문의 목적은 문제가 발생하는 상황을 이해하기 위한 것이므로 무시합니다.)

- 페이지를 로딩하면서 또는 페이지 로딩 직후 두 번째 탭을 선택하지 않고 메인 영역에서 '외주/자재' Radio 컴포넌트의 선택을 변경하면, 두 번째 탭의 '계약유형' SelectBox 컴포넌트 선택 목록 데이터를 (ID를 호출하여 직접적인 방식으로) 변경하려고 시도하다가 오류가 발생할 수 있습니다.
- 페이지 로딩 직후, 두 번째 탭을 선택하지 않고 세 번째 탭을 선택하면 세 번째 탭의 컴포넌트 구성 및 기본 데이터 적재 작업이 수행되면서 '보증증권' 관련 TR 토글 처리를 위해 두 번째 탭의 '계약유형' SelectBox 컴포넌트에 접근하게 됩니다. 이때 ID를 호출하여 직접적인 방식으로 접근 시 오류가 발생할 수도 있고, ID를 직접 호출하지 않더라도 두 번째 탭의 기본 데이터가 아직 적재되지 않아서 오동작을 하거나 오류가 발생할 수 있습니다.
- 데이터(DataCollection) 관점에서 위의 사례들에 대해 예외 처리를 하기 위해서는 탭의 렌더링 여부 상태와 관련 없이 필요한 동작을 수행할 수 있어야만 합니다. 특정 사례에서는 애당초 불가능한 경우도 있고, 메인 영역과 각 탭에 동일한 정보를 여러 벌 관리해야 할 수도 있습니다. 또는 AliasData... 시리즈의 DataCollection을 활용해야 할 수도 있어서 개발 난이도 및 업무 개발자 교육 수준이 높아지게 됩니다.
- 전체 10 개의 탭 중, 이런 식으로 다른 탭이나 메인 영역과 상호 작용이 필요한 몇 개의 탭들만 alwaysDraw 속성을 true로 지정할 수도 있습니다. 그러나 경험적으로 이렇게 안내할 경우, 나중에 확인해보면 대부분의 업무 개발자들이 모든 탭들에 `alwaysDraw="true"` 설정을 적용 경우가 대부분입니다.

32.5.2. 사례 2

상황

프로젝트에서 합의된 UI/UX 표준에서 유효성 검사 수행 표준이 TabControl에 친화적이지 않은 경우.

건설사에서 전자계약 업무에 사용되는 계약 상세 관리 페이지가 존재한다고 가정해보겠습니다. 이 페이지에는 주요 계약 정보를 담고 있는 메인 영역과 `alwaysDraw="false"` 설정이 적용된 10여개의 탭으로 이루어진 TabControl이 구성되어 있습니다. 그리고 이번에도 메인 영역의 '외주/자재' Radio 컴포넌트의 선택 값에 따라, 두 번째 탭의 '계약유형' SelectBox 컴포넌트의 선택 목록 데이터가 변경되고, 이 SelectBox 컴포넌트의 선택 값에 따라 세 번째 탭의 테이블 레이아웃에서 '보증증권' 관련 TR의 출력이 토글된다고 가정해보도록 하겠습니다. 만약 프로젝트 내부 UI/UX 표준에서 '저장' 버튼이 TabControl의 각 탭마다 별도로 존재하는 것으로 합의되었다면 상대적으로 수월하게 유효성 검사를 수행할 수 있습니다. 특정 탭의 유효성 검사 수행 시, 다른 탭이나 메인 영역의 데이터를 검증하거나 컴포넌트에 접근할 필요성이 상대적으로 적기 때문입니다.

그림 32-16 화면 예시

그러나 이런 경우에도 본문의 예제와 같은 경우 세 번째 탭의 유효성 검사를 수행하려면 두 번째 탭의 컴포넌트 또는 데이터에 접근해야 할 수도 있습니다. 또한, 업무 개발자 측의 요구사항 중 두 번째 탭의 '계약유형' 값이 지정되지 않은 경우 [‘보증증권’ 정보를 설정하려면 ‘계약유형’ 값을 지정해야 합니다.]라는 메시지를 출력한 다음, 자동으로 두 번째 탭으로 이동하고 ‘계약유형’ SelectBox 컴포넌트에 포커스를 지정해야 하는 등의 요건이 존재한다면 난이도가 더욱 복잡해집니다.

그림 32-17 화면 예시

보통 프로젝트에서는 공통 개발자가 이런 유효성 검사 로직에 자동으로 대응할 수 있는 공통 유효성 검사 메소드를 요구 받는 경우가 많습니다. 물론 업무 개발자가 직접 대응한다고 하더라도 수월한 작업이 아닌 것은 당연합니다.

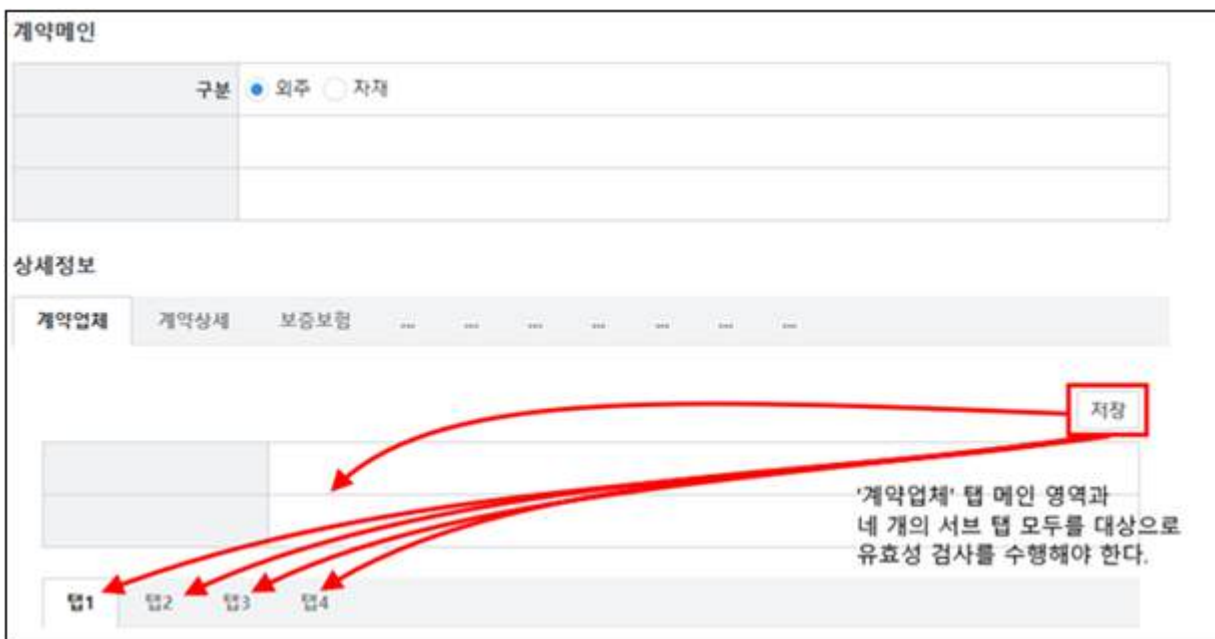
더 난감한 경우는 다음과 같이 프로젝트 내부 UI/UX 표준에서 ‘저장’ 버튼이 메인 영역에 한 개만 존재하는 것으로 합의된 경우입니다.

그림 32-18 화면 예시



이에 대응하는 유효성 검사를 수행하려면 ‘저장’ 버튼 클릭 시, 메인 영역과 10개의 모든 탭에 대해 유효성 검사를 수행해야 하는데, 이번에도 문제는 각각의 탭들이 이미 렌더링 완료된 상태인지, 또는 일부 탭만 렌더링이 완료된 상태인지, 그도 아니면 오직 첫 번째 탭만 렌더링이 완료된 상태인지 불분명하다는 점입니다. 결과적으로 유효성 검사 코드를 구현하기가 쉽지 않습니다. 다른 탭이나 메인 영역과 상호 작용이 필요한 일부 탭들은 여전히 [사례 1](#)에서 언급한 문제점까지 해결해야 함은 물론입니다. 극단적인 경우로 ‘저장’ 버튼이 TabControl의 각 탭마다 별도로 존재하는 것으로 합의된다고 하더라도, 특정 탭 내부에 또 다시 자식 TabControl이 존재할 수 있고, 이는 결과적으로 ‘저장’ 버튼이 메인 영역에 한 개만 존재하는 것으로 합의된 경우와 동일합니다.

그림 32-19 화면 예시



32.6 결론

특정 상황에서 TabControl의 `alwaysDraw="false"` 설정을 적용할 경우, 명백한 성능 향상 효과와 메모리 사용량 저하 효과를 기대할 수 있습니다. 그러나 실제 프로젝트 환경에서 극단적으로 복잡한 화면 구성을 구현할 때 `alwaysDraw="false"` 설정을 적용하면, 업무 개발자들의 구현 난이도를 크게 높이고, 심지어는 구현이 불가능 할 때도 있습니다. 따라서 `alwaysDraw="false"` 설정은 아래 내용을 기준으로 주의해서 사용할 것을 권장합니다.

- 근본적으로 WebSquare에 최적화된 UI/UX 설계를 수행하는 것이 가장 중요합니다.
- 단순히 기술적인 측면만 강조하여 `alwaysDraw="true"` 설정을 적용한 업무 개발자를 압박하는 것은 바람직하지 않습니다. (업무적으로 필요한 상황이 존재합니다.)
- 기본 시스템 레이아웃에 따라서 유연한 설계 방침을 적용하는 것도 한 방법입니다. 가령 TabControl나 WindowContainer 컴포넌트를 기반으로 메뉴-업무 페이지 레이아웃이 구성된 프로젝트에서는 `alwaysDraw="true"` 설정을 적용하는 것을 엄격하게 피하도록 가이드하고, 단일 페이지(WFrame 기반) 방식으로 메뉴-업무 페이지 레이아웃이 구성된 프로젝트에서는 `alwaysDraw="true"` 설정 적용을 어느 정도 허용하는 것도 권장합니다.

33. 서버 프레임워크 연동 가이드

33.1 일반 원칙

웹스퀘어는 웹표준 RIA 솔루션으로써 데이터 연계는 기본적으로 웹브라우저와 웹서버 간의 HTTP Protocol을 기반으로 한 통신방식을 준수합니다.

33.1.1. 표준 웹을 준수하는 데이터 구조와 통신 방식

- HTTP Protocol에서 제공하는 Text Data 기반의 통신방식을 지원하며, Ajax 기반의 통신 모듈을 제공합니다.
- 지원되는 통신 Data 구조는 일반적인 HTTP Protocol상에서 범용적으로 사용되는 XML, JSON, Plain Text, Request Parameter(key/value) 등의 구조를 기본적으로 모두 사용할 수 있으나, 가장 권장되는 것은 XML 구조입니다.
- 이하 XML 구조를 기준으로 설명합니다.
- 웹스퀘어가 강제하는 고정된 Format은 없으며, 임의의 표준 포맷을 정의하여 사용할 수 있습니다. (Well-Formed XML의 조건만 만족하면 됩니다.)
- 서버의 데이터 인터페이스 모듈 역시 고정된 모듈을 제공하고 있지 않으며, 정해진 표준 데이터 포맷을 처리할 수 있으면 어떤 형태로든 직접 구현하여 사용할 수 있습니다. (단, 기본적으로 HTTP Protocol 기반으로 연계할 수 있는 Web Application의 구조를 가져야 합니다.)
- 일반적으로 권장되는 데이터 연계와 관련된 시스템 구성은 J2EE기반의 Web Application Server하에서 Web Application Framework을 구성하여 웹스퀘어의 통신객체(Submission)와 표준포맷의 데이터를 교환하는 방식이 될 수 있습니다.

33.1.2. 웹스퀘어와 서버 간 데이터 변환

- 웹스퀘어 화면에서 일반적으로 사용되는 XML, JSON 등의 데이터 구조에 대하여 서버사이드에서 일반적으로 사용되는 데이터 구조체로의 데이터 변환 처리를 통하여 효과적인 데이터 연계 구조를 구성할 수 있습니다.
 - 웹스퀘어 화면 데이터 구조 : 표준 웹브라우저에서 표준 JavaScript로 접근가능한 데이터 Object - XML, JSON
 - 서버사이드의 데이터 구조 : J2EE기반 Web Application을 기준으로 볼 때, 일반적으로 데이터 구조체로 사용되는 Java Object - Collection(Map/List), VO(Java Beans)
- 상기에 기술한 웹스퀘어와 서버간의 데이터 변환을 수행하는 모듈이 서버단의 Web Application 구조에 포함되어 데이터 연계 시 변환처리를 수행하는 과정이 필요합니다.
 - 일반적인 MVC 기반의 Web Application Framework의 구조를 예시로,
 - Controller 영역에서 웹스퀘어 Submission으로부터 전달된 화면의 Request XML Data를 서버사이드의 Java Object 데이터 구조체로 변환

- View 영역에서 웹스퀘어 Submission으로 전달할 Response Java Object 데이터를 정해진 표준 포맷의 XML Data로 변환

33.1.3. 웹스퀘어 Adapter

- 일반적으로 널리 사용되는 MVC 기반의 OpenSource Web Application Framework과의 연동을 위하여 데이터 변환을 지원하는 WebSquare Add-on 모듈을 제공하며, 웹스퀘어 엔진 배포본에 포함되어 있습니다. (websquare_adapter_[version number].jar)
- 이 웹스퀘어 어댑터는 필수사항은 아니며, 필요에 따라 선택적으로 사용할 수 있습니다.
- 이하에 대표적인 범용 OpenSource Web Application Framework에 대하여 웹스퀘어 어댑터를 적용하여 데이터 변환 처리를 하기위한 내용을 설명합니다.
- 웹스퀘어 Adapter에서 사용되는 XML 표준 포맷

```

1  <map id="">
2    <userid>honggildong</userid>
3    <username>홍길동</username>
4    <address>서울...</address>
5    <age>17</age>
6
7    <list id="phoneList">
8      <map id="0">
9        <title>집전화</title>
10       <phoneNo>02-000-0000</phoneNo>
11     </map>
12     <map id="1">
13       <title>핸드폰</title>
14       <phoneNo>010-111-1111</phoneNo>
15     </map>
16   </list>
17 </map>

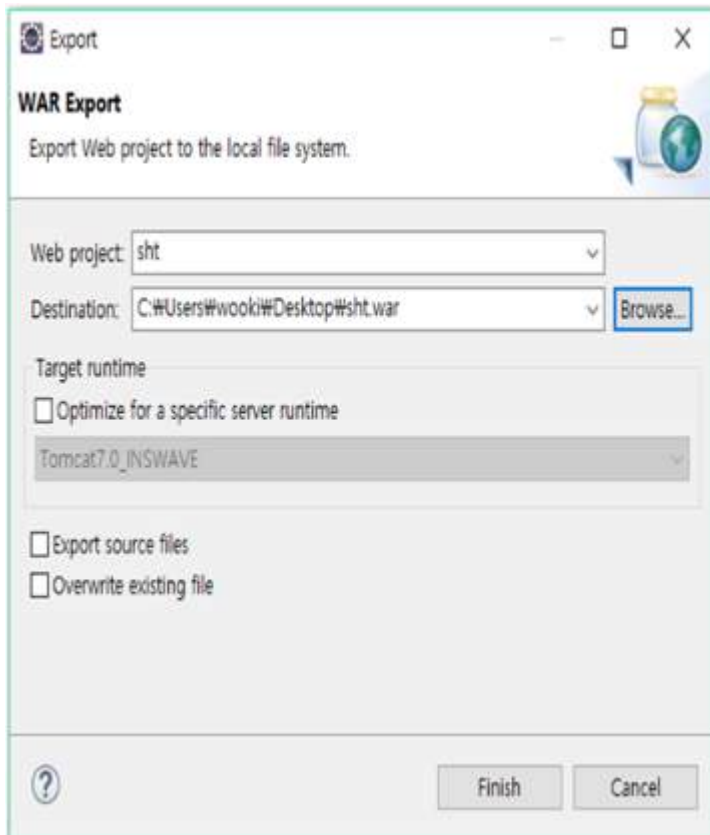
```

33.2 전자정부 프레임워크 연동

33.2.1. 배포

33.2.1.1. WAR 배포

1. [Export] - [WAR file]을 선택합니다.



2. **setenv.bat** 파일을 생성한 후, 환경변수를 설정합니다.

```
setenv.bat
set JAVA_HOME=C:\eGovFrameDev-3.5.1-32bit\jdk7_32\jdk1.7.0_80\bin
set JRE_HOME=C:\eGovFrameDev-3.5.1-32bit\jdk7_32\jre7
set JAVA_OPTS=-DWEBSQUARE_HOME=C:\eGovFrameDev-3.5.1-32bit\tomcat\apache-tomcat-7.0.82\webapps\resource\websquare_home
```

3. WAR 파일을 **webapps** 디렉토리에 넣고, 톰캣을 기동합니다.

이름	수정한 날짜	유형	크기
docs	2017-11-22 오전...	파일 폴더	
examples	2017-11-22 오전...	파일 폴더	
host-manager	2017-11-22 오전...	파일 폴더	
manager	2017-11-22 오전...	파일 폴더	
resource	2017-11-22 오전...	파일 폴더	
ROOT	2017-11-22 오전...	파일 폴더	
sht	2017-11-22 오전...	파일 폴더	
sht.war	2017-11-22 오전...	ALZip WAR File	79,925KB

33.2.1.2. Exploded 배포

1. `C:\WeGovFrameDev-3.5.1-32bit\workspace\src\main\webapp` 디렉토리의 하위 디렉토리를 `WEB-INF`의 `webapp` 디렉터리에 넣습니다.

common	2017-11-21 오전...	파일 폴더	
css	2017-11-21 오전...	파일 폴더	
images	2017-11-21 오전...	파일 폴더	
js	2017-11-21 오전...	파일 폴더	
META-INF	2017-11-21 오전...	파일 폴더	
WEB-INF	2017-11-21 오전...	파일 폴더	
websquare	2017-11-21 오전...	파일 폴더	
xml	2017-11-21 오전...	파일 폴더	
index.jsp	2017-11-01 오후...	JSP 파일	1KB

2. `C:\WeGovFrameDev-3.5.1-32bit\workspace\src\target\classes` 디렉토리의 하위 디렉토리를 `WEB-INF`의 `classes`에 넣고 톰캣을 기동합니다.

com	2017-11-21 오전...	파일 폴더	
db	2017-11-21 오전...	파일 폴더	
egovframework	2017-11-21 오전...	파일 폴더	
log4j2.xml	2017-11-21 오전...	XML 문서	1KB

33.2.2. 어플리케이션 설정

33.2.2.1. 인코딩

1. `web.xml` 파일에 아래와 같이 encoding filter를 추가합니다.

```
<filter>
  <filter-name>encodingFilter</filter-name>
  <filter-class>org.springframework.web.filter.CharacterEncodingFilter</filter-class>
  <init-param>
    <param-name>encoding</param-name>
    <param-value>utf-8</param-value>
  </init-param>
</filter>
<filter-mapping>
  <filter-name>encodingFilter</filter-name>
  <url-pattern>*.do</url-pattern>
</filter-mapping>
```

33.2.2.2. Auto Deploy

1. *Server.xml* 파일의 설정을 변경합니다.

- **unpackWARs**: WAR 파일 자동 언팩 (true/false)
- **autoDeploy**: 자동 배포 (true/false)

```
<Host name="localhost"  appBase="webapps"  unpackWARs="true" autoDeploy="true">

  <!-- SingleSignOn valve, share authentication between web applications -->
  <!-- Documentation at: /docs/config/valve.html -->
  <!-->
  <Valve className="org.apache.catalina.authenticator.SingleSignOn" />

  <!-- Access log processes all example. -->
  <!-- Documentation at: /docs/config/valve.html -->
  <!-- Note: The pattern used is equivalent to using pattern="common" -->
  <Valve className="org.apache.catalina.valves.AccessLogValve" directory="logs"
    prefix="localhost_access_log." suffix=".txt"
    pattern="%h %l %u %t &quot;%r&quot; %s %b" />

</Host>
```

33.2.3. DataSource 설정

1. *Globals.properties*에 각 DB에 맞는 JDBC 설정을 입력합니다.

```
# 운영서버 타입(WINDOWS, UNIX)
Globals.OsType = WINDOWS

# G4C 연결용 IP (localhost)
Globals.LocalIp = 127.0.0.1

# DB서버 타입(mysql,oracle,altibase,tibero) - datasource 및 sqlMap 파일 지정에 사용됨
#Globals.DbType = hsql
Globals.DbType = mysql
Globals.UserName=root
Globals.Password=

# maria
Globals.DriverClassName=com.mysql.jdbc.Driver
Globals.Url=jdbc:mysql://localhost:3306/test

# MainPage Setting
Globals.MainPage = /cmm/main/mainPage.do
```

2. `context-datasource.xml` 파일에 각 DB에 맞는 JDBC 설정을 입력합니다.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:jdbc="http://www.springframework.org/schema/jdbc"
    xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
        http://www.springframework.org/schema/jdbc http://www.springframework.org/schema/jdbc/spring-jdbc-4.0.xsd">

    <!-- 환경설정 기본정보를 globals.properties 에서 참조하도록 PropertyConfigurer 설정 -->
    <bean id="propertyConfigurer"
        class="org.springframework.beans.factory.config.PropertyPlaceholderConfigurer">
        <property name="locations">
            <list>
                <value>classpath:/egovframework/egovProps/globals.properties</value>
                <!-- value>file:/product/jeus/egovProps/globals.properties</value-->
            </list>
        </property>
    </bean>

    <!-- datasource 설정(propertyConfigurer 참조) -->
    <alias name="dataSource-${Globals.DbType}" alias="dataSource" />
    <alias name="dataSource-${Globals.DbType}" alias="egov.dataSource" />

    <!-- mysql -->
    <bean id="dataSource-mysql" class="org.apache.commons.dbcp.BasicDataSource" destroy-method="close">
        <property name="driverClassName" value="${Globals.DriverClassName}" />
        <property name="url" value="${Globals.Url}" />
        <property name="username" value="${Globals.UserName}" />
        <property name="password" value="${Globals.Password}" />
    </bean>
```

33.2.4. 로컬 개발 환경 설정

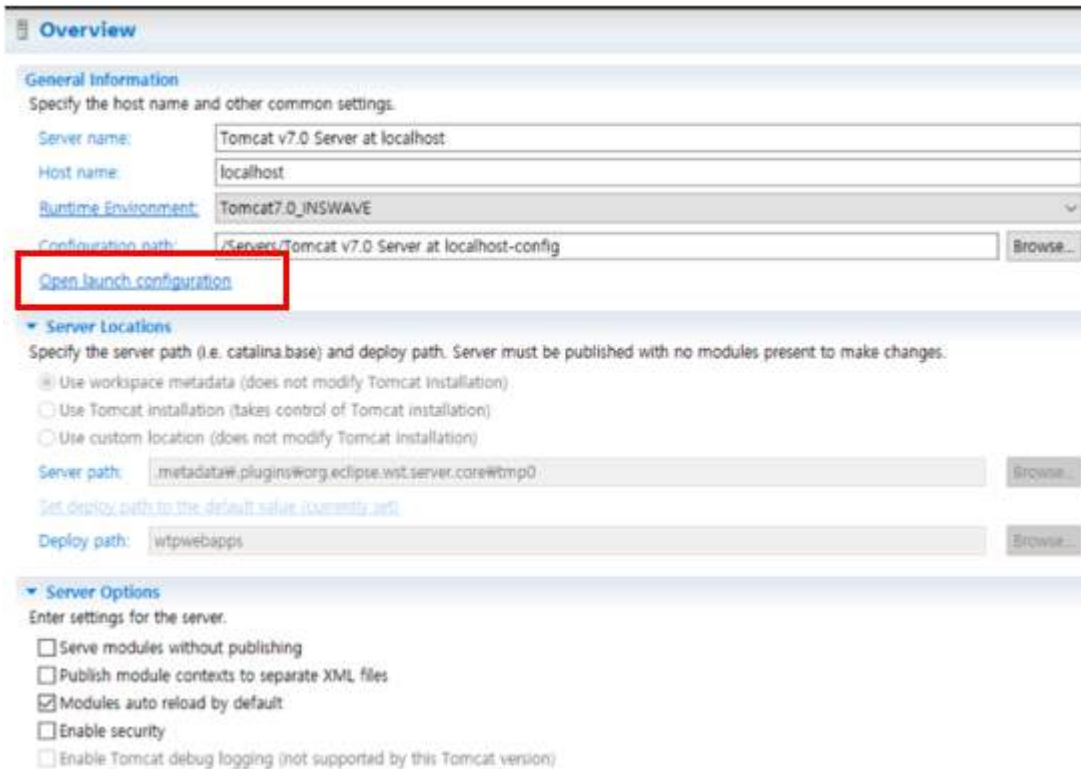
표준프레임워크 적용 후 아래와 같이 로컬 개발 환경을 설정하십시오.

33.2.4.1. WAS

1. 추가된 WAS를 더블클릭 합니다.



2. Open launch configuration을 클릭 합니다.



3. VM Arguments에 다음 옵션을 추가합니다. (setenv 파일의 JAVA_OPTS에 해당.)

Code 33-1 옵션 추가

1 DWEBSQUARE_HOME= “ websquare5_home 디렉토리경로 ”



33.2.4.2. Web.xml

1. **web.xml** 파일에 다음 내용을 추가합니다.

Code 33-2 설정 추가

```

1 <servlet>
2     <servlet-name>websquareDispatcher</servlet-name>
3     <servlet-class>websquare.http.DefaultRequestDispatcher</servlet-class>
4 </servlet>
5 <servlet-mapping>
6     <servlet-name>websquareDispatcher</servlet-name>
7     <url-pattern>*.wq</url-pattern>
8 </servlet-mapping>

```

```

<display-name>egovframework.sht</display-name>
<filter>
    <filter-name>encodingFilter</filter-name>
    <filter-class>org.springframework.web.filter.CharacterEncodingFilter</filter-class>
    <init-param>
        <param-name>encoding</param-name>
        <param-value>utf-8</param-value>
    </init-param>
</filter>
<filter-mapping>
    <filter-name>encodingFilter</filter-name>
    <url-pattern>*.do</url-pattern>
</filter-mapping>
<servlet>
    <servlet-name>websquareDispatcher</servlet-name>
    <servlet-class>websquare.http.DefaultRequestDispatcher</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>websquareDispatcher</servlet-name>
    <url-pattern>*.wq</url-pattern>
</servlet-mapping>
<context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>
        classpath*:egovframework/spring/com/context-*.xml
    </param-value>
</context-param>
<listener>
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<servlet>
    <servlet-name>action</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <init-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>/WEB-INF/config/egovframework/springmvc/*.xml</param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>action</servlet-name>
    <url-pattern>*.do</url-pattern>
</servlet-mapping>

```


33.2.4.3. Spring config

33.2.4.3.1. WebSquareAdapter 사용

1. WebSquareAdapter를 사용할 경우 Application 설정에 다음 내용을 추가합니다.

```

1  <bean
class="org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerMapping"
2      p:order="0"/>
3
4  <bean class="com.inswave.adapter.CustomRequestMappingHandlerAdapter"
5      p:order="1">
6      <property name="webBindingInitializer">
7          <bean class="egovframework.com.cmm.web.EgovBindingInitializer" />
8      </property>
9      <property name="customArgumentResolvers">
10         <list>
11             <bean class="com.inswave.adapter.WebSquareArgumentResolver">
12                 <property name="uiAdapter">
13                     <ref bean="wqAdapter" />
14                 </property>
15             </bean>
16         </list>
17     </property>
18 </bean>
19
20 <bean id='wqAdapter' class="com.inswave.adapter.WqAdapter" />
21 <bean id='wqView' class="com.inswave.adapter.WqAdapterViewByMap" />
22
23 <bean class="org.springframework.web.servlet.view.BeanNameViewResolver" p:order="2" />

```

```

<!-- 패키지 내 Controller, Service, Repository 클래스의 auto detect를 위한 mvc 설정 -->
<context:component-scan base-package="egovframework, com">
    <context:include-filter type="annotation" expression="org.springframework.stereotype.Controller"/>
    <context:exclude-filter type="annotation" expression="org.springframework.stereotype.Service"/>
    <context:exclude-filter type="annotation" expression="org.springframework.stereotype.Repository"/>
</context:component-scan>

<bean class="org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerMapping" p:order="0"/>

<bean class="com.inswave.adapter.CustomRequestMappingHandlerAdapter" p:order="1">
    <description>
        Spring에서 사용자가 등록한 Custom HandlerArgumentResolver의 우선순위가 Spring에서 기본으로 제공하는 Map을 처리하는 HandlerMethodArgumentResolver보다 낮다.
        즉, Custom Annotation을 이용하여 Map으로 사용자가 보낸 데이터를 받을때 이 작업을 위해 등록된 Custom HandlerArgumentResolver보다
        Spring에서 Map을 처리하는 HandlerMethodArgumentResolver를 먼저 만나기 때문에 사용자가 등록한 Custom HandlerArgumentResolver가 이를 처리할수 없다.
        이 부분을 구현하는 것은 Spring에서 제공하는 태그만으로는 할 수가 없다. Spring에서 사용하는 클래스를 직접 커스터마이징 해야 한다.
    </description>
    <property name="webBindingInitializer">
        <bean class="egovframework.com.cmm.web.EgovBindingInitializer" />
    </property>
    <property name="customArgumentResolvers">
        <list>
            <bean class="com.inswave.adapter.WebSquareArgumentResolver">
                <property name="uiAdapter">
                    <ref bean="wqAdapter" />
                </property>
            </bean>
        </list>
    </property>
</bean>

<bean id='wqAdapter' class="com.inswave.adapter.WqAdapter" />
<bean id='wqView' class="com.inswave.adapter.WqAdapterViewByMap" />

<bean class="org.springframework.web.servlet.view.BeanNameViewResolver" p:order="2" />

```

2. WebSquareAdapter를 사용할 경우 Application 설정에서 다음 내용을 수정합니다.

```

1 <!-- <mvc:annotation-driven/> --> <! - 주석처리 또는 삭제.-->

```



```

<bean class="org.springframework.web.servlet.view.BeanNameViewResolver" p:order="2" />

<!-- 화면처리용 JSP 파일명의 prefix, suffix 처리에 대한 mvc 설정 -->
<bean class="org.springframework.web.servlet.view.UrlBasedViewResolver" p:order="3"
    p:viewClass="org.springframework.web.servlet.view.JstlView"
    p:prefix="/WEB-INF/jsp/" p:suffix=".jsp"/>
<!-- 서버유틸리티네이션상의 exception에 대한 오류 페이지를 연결하는 mvc 설정-->
<bean class="org.springframework.web.servlet.handler.SimpleMappingExceptionResolver">
    <property name="defaultErrorView" value="cmm/error/egovError"/>
    <property name="exceptionMappings">
        <props>
            <prop key="org.springframework.dao.DataAccessException">cmm/error/dataAccessFailure</prop>
            <prop key="org.springframework.transaction.TransactionException">cmm/error/transactionFailure</prop>
            <prop key="egovframework.rte.fdl.cmmn.exception.EgovBizException">cmm/error/egovError</prop>
            <prop key="org.springframework.security.AccessDeniedException">cmm/error/accessDenied</prop>
        </props>
    </property>
</bean>

<!-- 로그인 체크가 필요한 URL과 로그인 여부를 체크해준다 -->
<mvc:interceptors>
    <mvc:interceptor>
        <mvc:mapping path="/cop/com/*.do"/>
        <mvc:mapping path="/cop/bbs/*Master*.do"/>
        <mvc:mapping path="/uat/uia/*.do"/>
        <mvc:exclude-mapping path="/uat/uia/actionLogin.do"/>
        <mvc:exclude-mapping path="/uat/uia/egovLoginUsr.do"/>
        <bean class="egovframework.com.cmm.interceptor.AuthenticInterceptor" />
    </mvc:interceptor>
</mvc:interceptors>

<!-- Annotation 을 사용하지 않는 경우에 대한 MVC 처리 설정 -->
<mvc:view-controller path="/cmmn/validator.do" view-name="cmmn/validator"/>

```

33.2.4.3.2. MessageConverter 사용

1. MessageConverter - Jackson을 사용할 경우 Application 설정에 다음 내용을 추가합니다.

```

1  <bean
class="org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerMapping"
2      p:order="0"/>
3
4  <bean
class="org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdapter"
5      p:order="1">
6      <property name="messageConverters">
7          <list>
8              <ref bean="mappingJackson2HttpMessageConverter" />
9          </list>
10     </property>
11 </bean>
12

```

```

13 <bean class="org.springframework.http.converter.json.MappingJackson2HttpMessageConverter "
14     id="mappingJackson2HttpMessageConverter">
15     <property name="supportedMediaTypes">
16         <list>
17             <value>application/json</value>
18         </list>
19     </property>
20 </bean>

```

```

<!-- 패키지 내 Controller, Service, Repository 클래스의 auto detect를 위한 mvc 설정 -->
<context:component-scan base-package="egovframework, com">
    <context:include-filter type="annotation" expression="org.springframework.stereotype.Controller"/>
    <context:exclude-filter type="annotation" expression="org.springframework.stereotype.Service"/>
    <context:exclude-filter type="annotation" expression="org.springframework.stereotype.Repository"/>
</context:component-scan>

<bean class="org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerMapping" p:order="0"/>

<bean class="org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdapter" p:order="1">
    <property name="messageConverters">
        <list>
            <ref bean="mappingJackson2HttpMessageConverter" />
        </list>
    </property>
</bean>

<bean id="mappingJackson2HttpMessageConverter" class="org.springframework.http.converter.json.MappingJackson2HttpMessageConverter">
    <property name="supportedMediaTypes">
        <list>
            <value>application/json</value>
        </list>
    </property>
</bean>

<bean class="org.springframework.web.servlet.view.BeanNameViewResolver" p:order="2" />

```

2. MessageConverter - Jackson을 사용할 경우 Application 설정에서 다음 내용을 수정합니다.

```

1 <!-- <mvc:annotation-driven/> --> <! - 주석처리 또는 삭제.-->

```

```

<bean class="org.springframework.web.servlet.view.BeanNameViewResolver" p:order="2" />

<!-- 화면처리용 JSP 파일명의 prefix, suffix 처리에 대한 mvc 설정 -->
<bean class="org.springframework.web.servlet.view.UrlBasedViewResolver" p:order="3"
    p:viewClass="org.springframework.web.servlet.view.JstlView"
    p:prefix="/WEB-INF/jsp/" p:suffix=".jsp"/>
<!-- 서버관련네이밍상의 exception에 대한 오류 페이지를 연결하는 mvc 설정-->
<bean class="org.springframework.web.servlet.handler.SimpleMappingExceptionResolver">
    <property name="defaultErrorView" value="cmm/error/egovError"/>
    <property name="exceptionMappings">
        <props>
            <prop key="org.springframework.dao.DataAccessException">cmm/error/dataAccessFailure</prop>
            <prop key="org.springframework.transaction.TransactionException">cmm/error/transactionFailure</prop>
            <prop key="egovframework.rte.fdl.cmmn.exception.EgovBizException">cmm/error/egovError</prop>
            <prop key="org.springframework.security.AccessDeniedException">cmm/error/accessDenied</prop>
        </props>
    </property>
</bean>

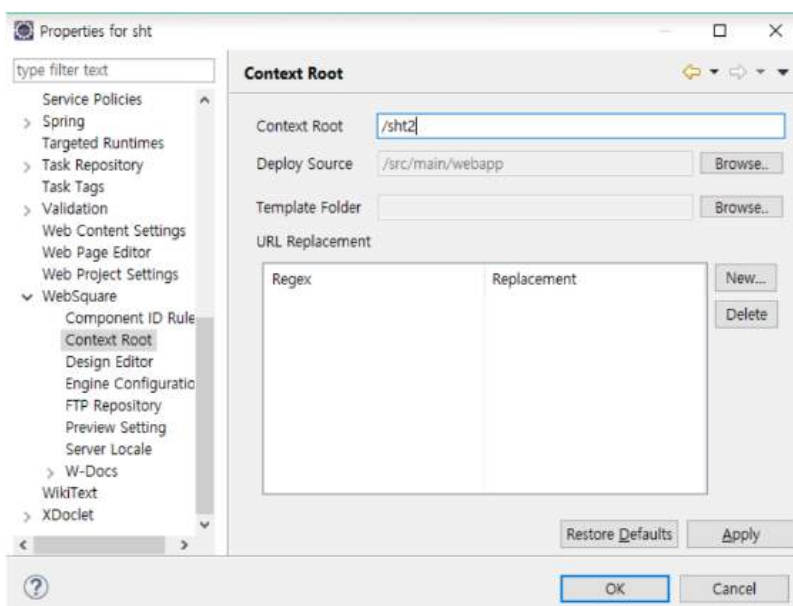
<!-- 로그인 체크가 필요한 URL과 로그인 여부를 체크해준다 -->
<mvc:interceptors>
    <mvc:interceptor>
        <mvc:mapping path="/cop/com/*.do"/>
        <mvc:mapping path="/cop/bbs/*Master*.do"/>
        <mvc:mapping path="/uat/uia/*.do"/>
        <mvc:exclude-mapping path="/uat/uia/actionLogin.do"/>
        <mvc:exclude-mapping path="/uat/uia/egovLoginUsr.do"/>
        <bean class="egovframework.com.cmm.interceptor.AuthenticInterceptor" />
    </mvc:interceptor>
</mvc:interceptors>

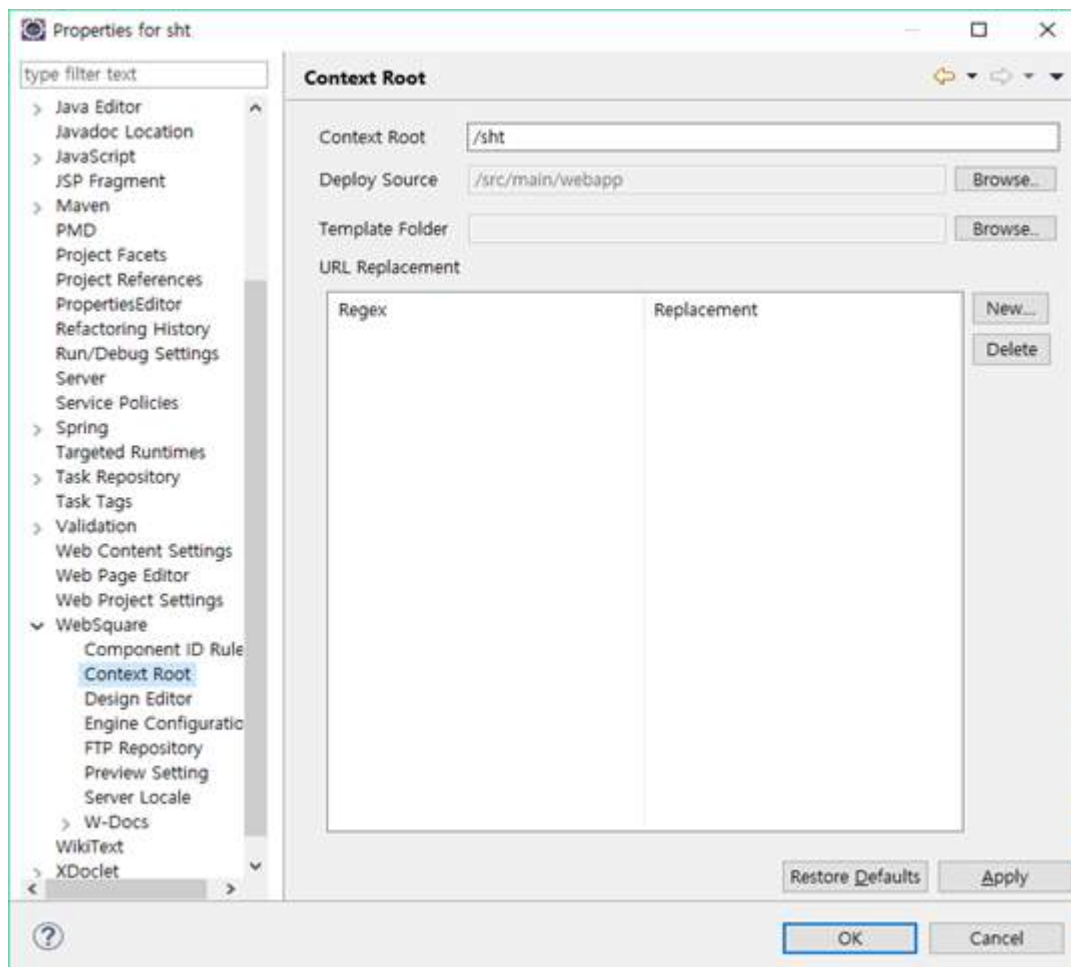
<!-- Annotation 을 사용하지 않는 경우에 대한 MVC 처리 설정 -->
<mvc:view-controller path="/cmmn/validator.do" view-name="cmmn/validator"/>

```

33.2.4.4. Context ROOT

1. Context root를 설정합니다.





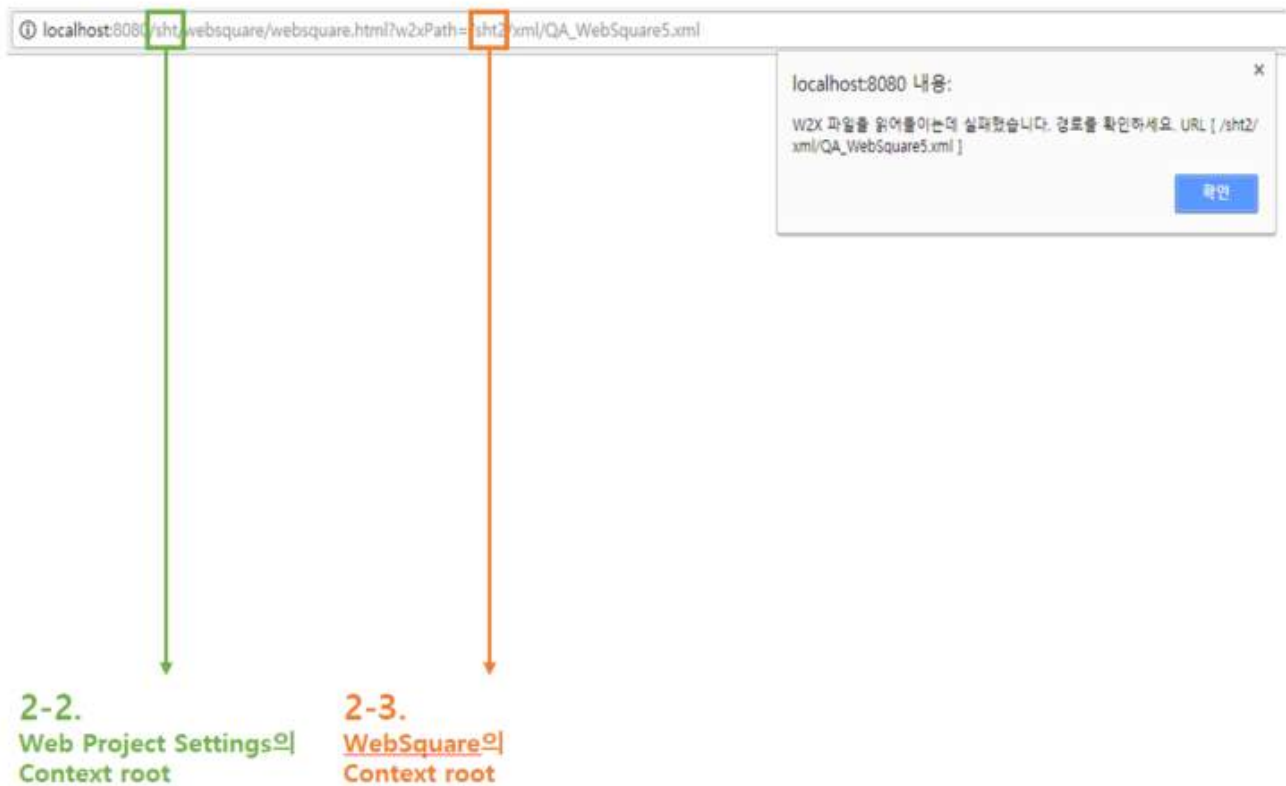
33.2.4.4.1. 주의

WebSquare의 Context root를 Web Project Settings의 Context root와 아래와 같이 다르게 설정할 경우, 프리뷰가 정상 동작하지 않습니다.

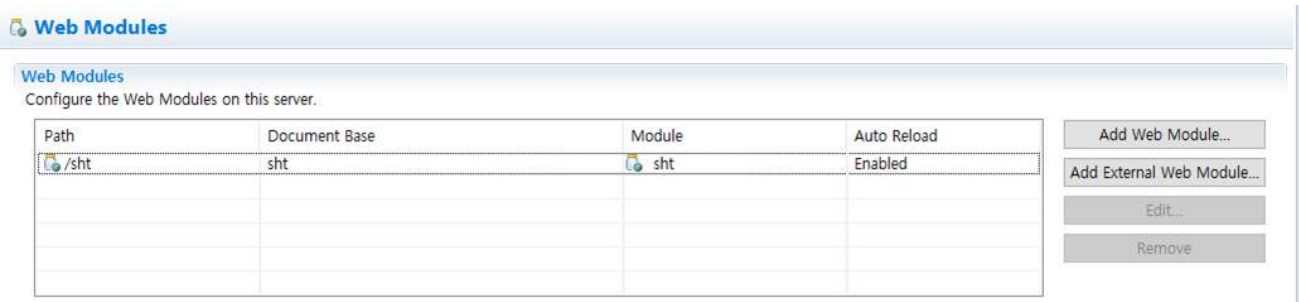
다른 설정을 사용

Web Project Settings = /sht

Websquare Context RootProject Settings = /sht2



2. 톰캣의 Context root를 설정합니다.

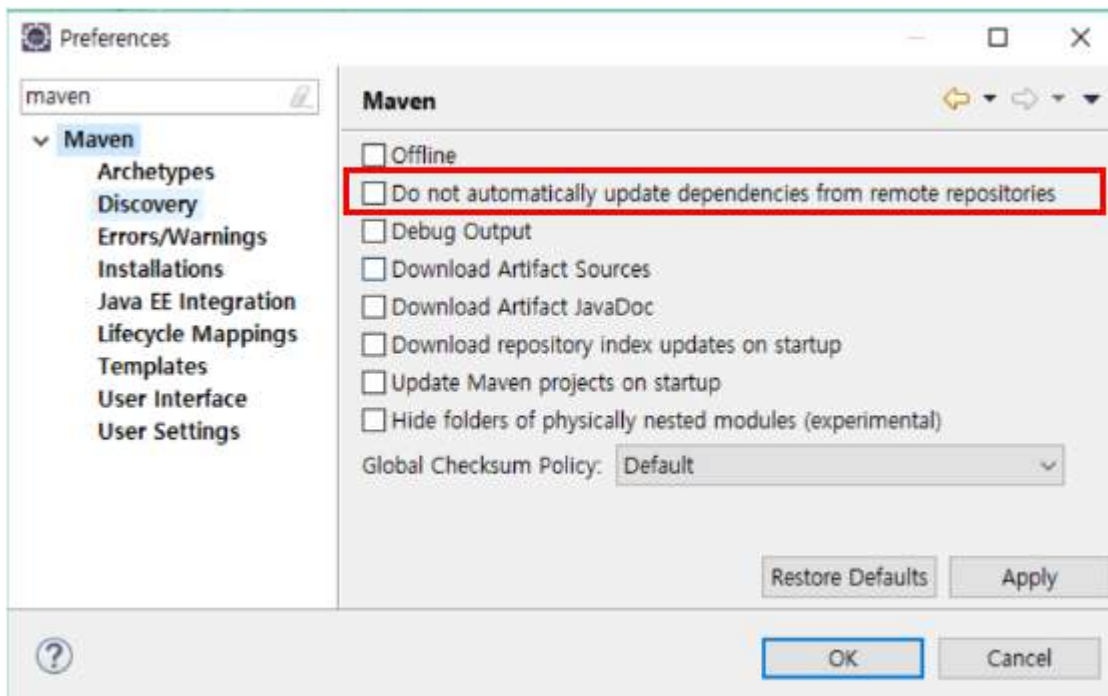


33.2.4.5. Maven

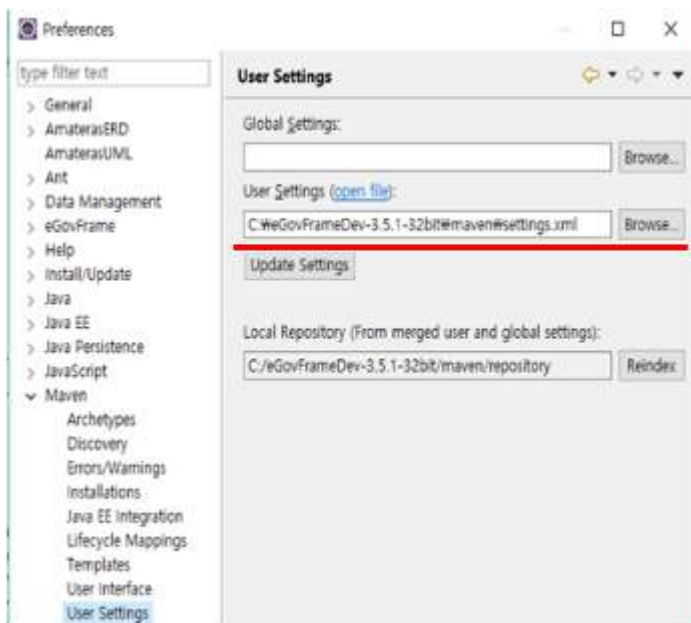
1. C:\WeGovFrameDev-3.5.1-32bit\maven\settings.xml 파일을 열고 `localRepository` 값을 설정합니다.

```
<localRepository>/path/to/local/repo</localRepository>
-->
<localRepository>C:/eGovFrameDev-3.5.1-32bit/maven/repository</localRepository>
```

2. 아래 표시한 항목을 선택 해제합니다.



3. User Settings의 값을 변경한 settings.xml의 파일로 설정합니다.



4. 웹스퀘어5 엔진과 xbean.jar는 localPath로 추가합니다.

```

<!-- WebSquare 구동시 오류가 많이 발생할. 해당 라이브러리는 WAS 공통 라이브러리 등. 반드시 확인 -->
<dependency>
  <groupId>xbean</groupId>
  <artifactId>xbean</artifactId>
  <version>2.5</version>
  <scope>system</scope>
  <systemPath>${basedir}/src/main/webapp/WEB-INF/lib/xbean.jar</systemPath>
</dependency>
<!-- dependency for WebSquare //-->

<!--// WebSquare Engine -->
<dependency>
  <groupId>websquare</groupId>
  <artifactId>websquare</artifactId>
  <version>5.0_2.2822B</version>
  <scope>system</scope>
  <systemPath>${basedir}/src/main/webapp/WEB-INF/lib/websquare_5.0_2.2822B.20171023.120057_1.5.jar</systemPath>
</dependency>
<dependency>
  <groupId>websquare</groupId>
  <artifactId>websquare_adapter</artifactId>
  <version>5.0_2.2822B</version>
  <scope>system</scope>
  <systemPath>${basedir}/src/main/webapp/WEB-INF/lib/websquare_adapter_5.0_2.2822B.20171023.120057_1.5.jar</systemPath>
</dependency>

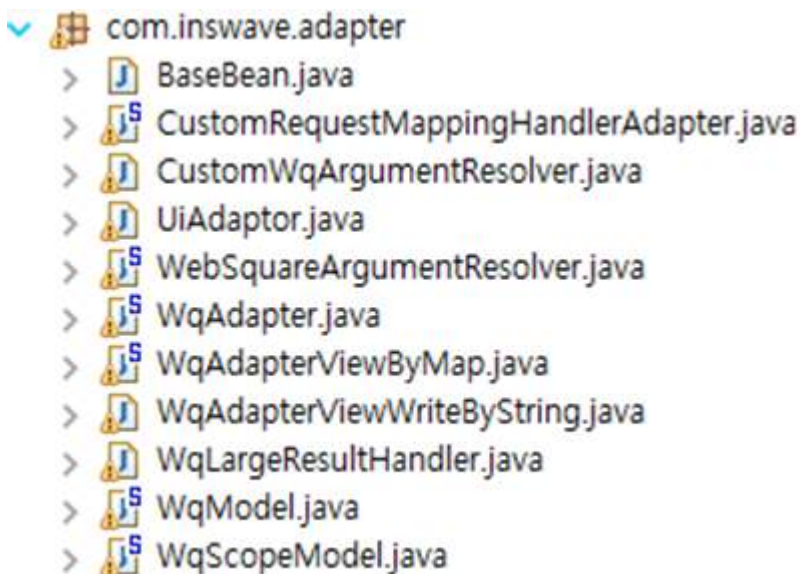
```

33.2.4.6. Adpater 추가

1. Adapter는 Spring 3.1.x이하 버전과 3.1.x이상 버전이 모두 포함되어있습니다.

Table 33-1 Adapter

3.0.x	CustomWqArgumentResolver
3.1.x	WebSquareArgumentResolver



33.2.4.7. DB Persistence

1. Map 타입을 사용하기 위해 미리 정의된 `egovMap`을 사용합니다.

```

<typeAlias alias="egovMap" type="egovframework.rte.psl.dataaccess.util.EgovMap"/>
<typeAlias alias="Board" type="egovframework.let.cop.bbs.service.Board"/>
<typeAlias alias="BoardVO" type="egovframework.let.cop.bbs.service.BoardVO"/>

```

2. 꼭 필요한 경우가 아니면 `parameterClass`는 명시하지 않고 `resultClass`는 입력합니다.

```

<!-- 웹소켓이S SP2 게시판 글 등록 -->
<insert id="BBSManageDAO.QA_BoardInsert">
  <![CDATA[
    insert into WQ_EGOV_BOARD
    (sTitle, sAuthor, sDate, sContent, sEmail, sPhone, sEmailReceive)
    VALUES
    (#sTitle#, #sAuthor#, now(), #sContent#, 'abc@inswave.com', '01012345678', #sEmailReceive# )
  ]]>
</insert>

<!-- 웹소켓이S SP2 게시판 조회 -->
<select id="BBSManageDAO.QA_BoardSelect" resultClass="egovMap">
  <![CDATA[
    SELECT * FROM WQ_EGOV_BOARD
    WHERE 1=1
  ]]>
  <isEqual prepend="AND" property="sType" compareValue="title">
    <![CDATA[stitle LIKE CONCAT ('%', #keyword#, '%') ]]>
  </isEqual>
  ORDER BY iSeq DESC
</select>

- 웹소켓이S SP2 게시판 글 갯수 조회 -->
<select id="BBSManageDAO.QA_BoardSelectCount" resultClass="java.lang.Integer">
  <![CDATA[
    SELECT count(*) as cnt FROM WQ_EGOV_BOARD
    WHERE 1=1
  ]]>
  <isEqual prepend="AND" property="sType" compareValue="title">
    <![CDATA[stitle LIKE CONCAT ('%', #keyword#, '%') ]]>
  </isEqual>
</select>

<update id="BBSManageDAO.QA_BoardViewIncrement">
  update wq_egov_board set
  sView = sView+1
  where iSeq = #keyword#
</update>

- 웹소켓이S SP2 게시판 댓글 조회 -->
<select id="BBSManageDAO.QA_BoardSelectDetail" resultClass="egovMap">
  <![CDATA[
    SELECT * FROM WQ_EGOV_BOARDDETAIL
    WHERE iSeq = #keyword#
  ]]>
  ORDER BY iDetailSeq DESC
</select>

```

3. WebSquareAdapter를 사용할 경우,
 I. Controller에서 ModelAndView를 리턴합니다.

그림 33-1 <Controller> - WebSquareAdapter

```

@RequestMapping(value = "/cop/bbs/QABoard_Insert.do")
public ModelAndView QABoardInsert(HttpServletRequest request, ModelMap model, Map param)
    throws Exception{
    ModelAndView mv = new ModelAndView("wqView");
    mv.addObject( "dlt_Board", bbsMngService.QA_BoardInsert((Map) param.get("dma_BoardWrite_QABoard")));
    mv.addObject( "S", "성공");

    return mv;
}

@RequestMapping(value = "/cop/bbs/QABoard_Select.do")
public ModelAndView QABoard(HttpServletRequest request, ModelMap model, Map param)
    throws Exception{
    ModelAndView mv = new ModelAndView("wqView");
    mv.addObject( "dlt_Board", bbsMngService.QA_BoardSelect((Map) param.get("dma_Search")) );
    mv.addObject( "S", "리스트가 조회 되었습니다.");

    return mv;
}

@RequestMapping(value = "/cop/bbs/QABoard_SelectDetail.do")
public ModelAndView QABoardDetail(HttpServletRequest request, ModelMap model, Map param)
    throws Exception{
    ModelAndView mv = new ModelAndView("wqView");
    mv.addObject( "dlt_BoardDetail", bbsMngService.QA_BoardSelectDetail((Map) param.get("dma_Search")));
    mv.addObject( "dlt_BoardComment", bbsMngService.QA_BoardViewIncrement((Map) param.get("dma_Search")));
    mv.addObject( "S", "리스트가 조회 되었습니다.");

    return mv;
}

@RequestMapping(value = "/cop/bbs/QABoard_CommentInsert.do")
public ModelAndView QABoardCommentInsert(HttpServletRequest request, ModelMap model, Map param)
    throws Exception{
    ModelAndView mv = new ModelAndView("wqView");
    mv.addObject( "dlt_BoardComment", bbsMngService.QA_BoardCommentInsert((Map) param.get("dma_AddComment")));
    mv.addObject( "S", "댓글이 추가되었습니다.");

    return mv;
}

```

4. MessageConverter를 사용할 경우,
 - I. Controller에서 Map을 리턴합니다. (본 방법은 Jackson-databind, Jackson-core, Jackson-annotations 라이브러리를 필요로 합니다.)

그림 33-2 <Controller> - MessageConverter

```

@RequestMapping(value = "/cop/bbs/QABoard_Insert.do")
public @ResponseBody Map QABoardInsert(@RequestBody Map param)
    throws Exception {

    Map hash = new HashMap();
    hash.put( "dlt_Board", bbsMngService.QA_BoardInsert((Map) param.get("dma_BoardWrite_QABoard")) );
    hash.put( "S", "성공" );

    return hash;
}

@RequestMapping(value = "/cop/bbs/QABoard_Select.do")
public @ResponseBody Map QABoard_Select(@RequestBody Map param)
    throws Exception{

    System.out.println("통장");
    System.out.println(param);

    Map hash = new HashMap();
    hash.put( "dlt_Board", bbsMngService.QA_BoardSelect((Map) param.get("dma_Search")) );
    hash.put( "S", "리스트가 조회 되었습니다." );

    System.out.println("끝");

    return hash;
}

@RequestMapping(value = "/cop/bbs/QABoard_SelectDetail.do")
public @ResponseBody Map QABoard_SelectDetail(@RequestBody Map param)
    throws Exception{

    Map hash = new HashMap();
    hash.put( "dlt_BoardDetail", bbsMngService.QA_BoardSelectDetail((Map) param.get("dma_Search")) );
    hash.put( "dlt_BoardComment", bbsMngService.QA_BoardViewIncrement((Map) param.get("dma_Search"))); //조회
    hash.put( "S", "리스트가 조회 되었습니다." );

    return hash;
}

@RequestMapping(value = "/cop/bbs/QABoard_CommentInsert.do")
public @ResponseBody Map QABoard_CommentInsert(@RequestBody Map param)
    throws Exception{

    Map hash = new HashMap();
    hash.put( "dlt_BoardComment", bbsMngService.QA_BoardCommentInsert((Map) param.get("dma_AddComment")) );
    hash.put( "S", "댓글을 작성했습니다." );

    return hash;
}

```

5. DAO에서 입력된 Persistence의 id를 입력합니다.

그림 33-3 <DAO>

```

/**
 * 웹스퀘어5 SP2 - 게시글 댓글을 등록한다.
 *
 * @param board
 * @return
 * @throws Exception
 */
public int QA_BoardCommentInsert(Map param) throws Exception {
    insert("BBSManageDAO.QA_BoardCommentInsert", param);
    int CNT = 1;

    return CNT;
}

/**
 * 웹스퀘어5 SP2 - 게시글 댓글을 삭제한다.
 *
 * @param board
 * @return
 * @throws Exception
 */
public int QA_BoardCommentDelete(Map param) throws Exception {
    System.out.println("DAO :: ");
    System.out.println(param);
    int uCnt = update("BBSManageDAO.QA_BoardCommentDelete", param);
    System.out.println("DAO END");
    return uCnt;
}

/**
 * 웹스퀘어5 SP2 - 게시글 댓글을 삭제한다.
 *
 * @param board
 * @return
 * @throws Exception
 */
public List QA_BoardSelect(Map param) throws Exception {
    public int QA_BoardViewIncrement(Map param) throws Exception {
        System.out.println("DAO :: ");
        System.out.println(param);
        int uCnt = update("BBSManageDAO.QA_BoardViewIncrement", param);
        System.out.println("DAO END");
        return uCnt;
    }
}

```

6. Service를 상속 받고 구현합니다.

그림 33-4 Service Impl

```

public Map QA_BoardSelectDetail(Map param) throws Exception {
    Map hash = new HashMap();
    hash.put("list", bbsMngDAO.QA_BoardSelectDetail(param));
    hash.put("cnt", bbsMngDAO.QA_BoardSelectCount(param));
    return hash;
}

public Map QA_BoardCommentInsert(Map param) throws Exception {
    int cCnt = bbsMngDAO.QA_BoardCommentInsert(param);
    Map hash = new HashMap();
    hash.put("cCnt", cCnt);
    return hash;
}

public Map QA_BoardCommentDelete(Map param) throws Exception {
    int uCnt = bbsMngDAO.QA_BoardCommentDelete(param);
    Map hash = new HashMap();
    hash.put("uCnt", uCnt);
    return hash;
}

public Map QA_BoardViewIncrement(Map param) throws Exception {
    int uCnt = bbsMngDAO.QA_BoardViewIncrement(param);
    Map hash = new HashMap();
    hash.put("uCnt", uCnt);
    return hash;
}

```

7. 추상메소드를 선언합니다.

그림 33-5 Service

```
/**
 * 웹스퀘어5 SP2 - 게시글 댓글을 조회한다.
 *
 * @param Board
 * @return
 * @exception Exception Exception
 */
public Map QA_BoardSelectDetail(Map param) throws Exception;

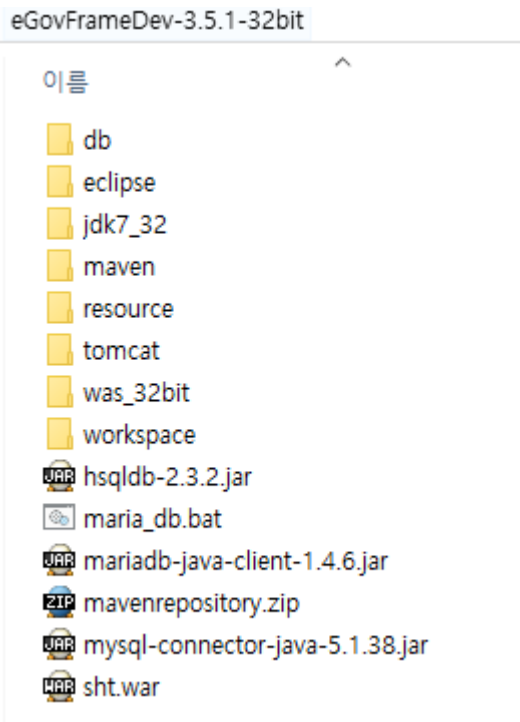
/**
 * 웹스퀘어5 SP2 - 게시글 댓글을 등록한다.
 *
 * @param Board
 * @return
 * @exception Exception Exception
 */
public Map QA_BoardCommentInsert(Map param) throws Exception;

/**
 * 웹스퀘어5 SP2 - 게시글 댓글을 등록한다.
 *
 * @param Board
 * @return
 * @exception Exception Exception
 */
public Map QA_BoardCommentDelete(Map param) throws Exception;

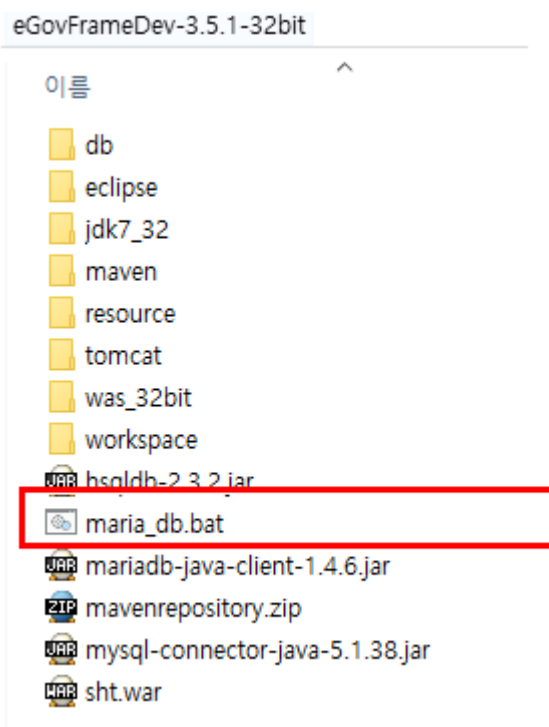
/**
 * 웹스퀘어5 SP2 - 게시글 조회수를 수정한다.
 *
 * @param Board
 * @return
 * @exception Exception Exception
 */
public Map QA_BoardViewIncrement(Map param) throws Exception;
```

33.2.5. 화면 구동

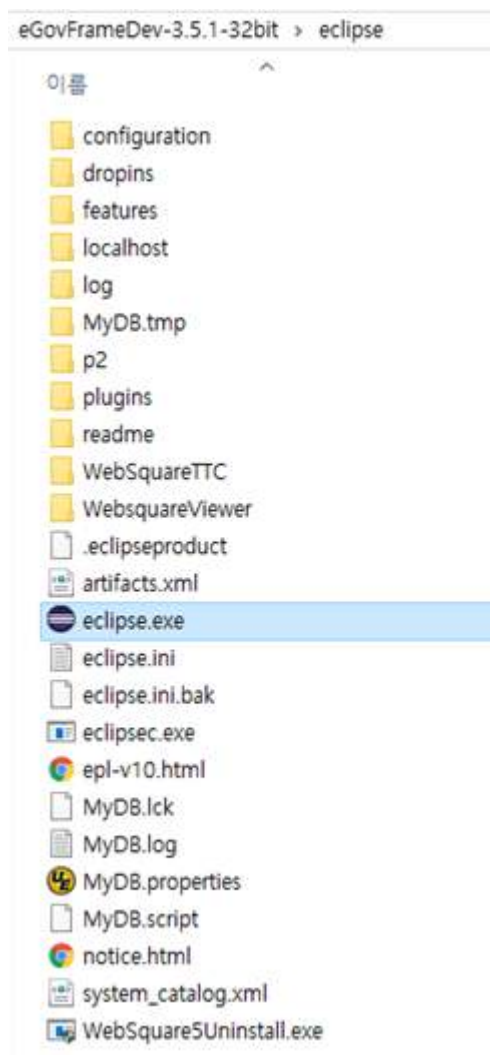
1. *eGovFrameDev-XXX* 형식의 ZIP 파일을 C: 드라이브에서 압축해제 합니다.
2. 아래와 같은 구조로 압축이 풀리는지 확인합니다.



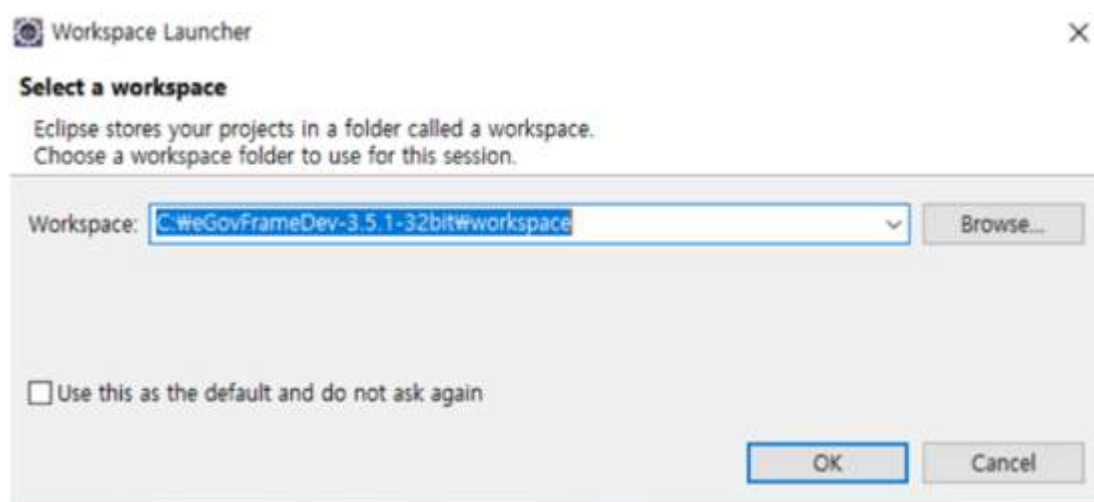
3. **Maria_db.bat** 파일을 실행합니다.



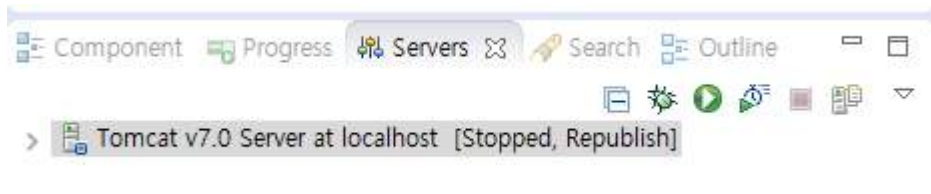
4. **eclipse.exe** 파일을 실행합니다.



5. *Maria_db.bat* 파일을 실행합니다.



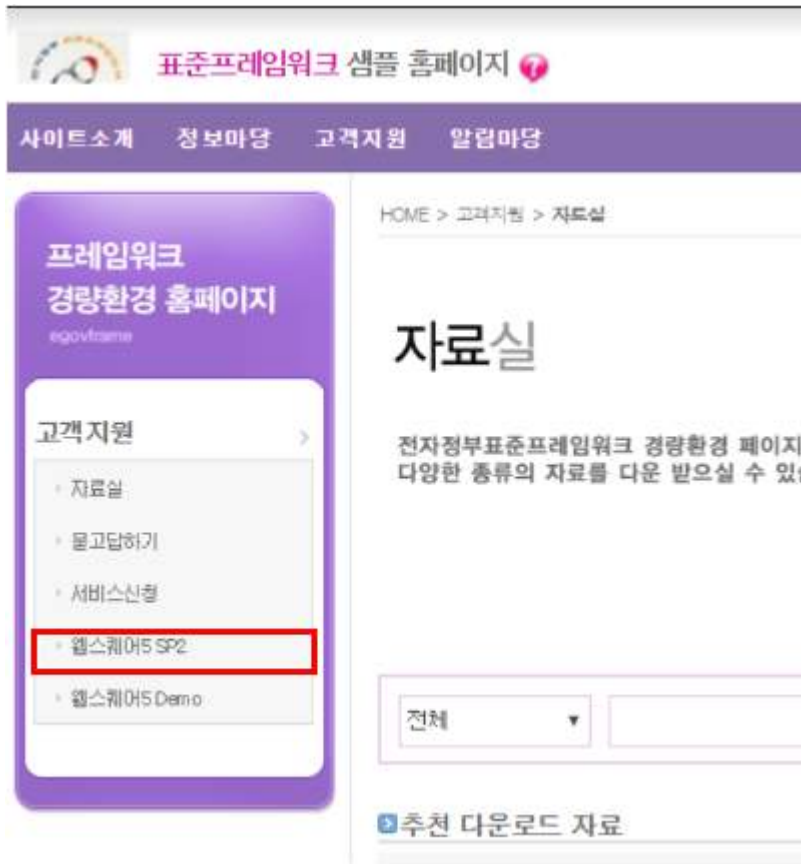
6. WTP Server를 실행합니다.



7. <http://localhost:8080/sht> 를 주소창에 입력 합니다.
8. 아래와 같은 화면이 표시되면, 상단의 **고객지원** 탭을 클릭합니다.



9. 고객지원 탭의 **웹스퀘어5 SP2**를 클릭합니다.



10. 우측 하단의 글쓰기 버튼을 클릭합니다.

1. 묻고 답하기(Q&A)

전체 검색

번호	제목	작성자	조회수	등록일
96	1234	인스웨이브	4	2018-01-10 17:28:23
95	작성	인스웨이브	12	2018-01-10 17:22:00
94	ㅋㅋㅋ	인스웨이브	1	2018-01-10 16:43:53
93	sad	인스웨이브	13	2018-01-10 11:17:00
92	99	인스웨이브	0	2018-01-10 10:46:02
91	455	인스웨이브	0	2018-01-10 10:45:08
90	33	인스웨이브	0	2018-01-10 10:44:12
89	11	인스웨이브	0	2018-01-10 10:42:58
88	제목	인스웨이브	2	2018-01-10 10:31:46
87	쓰기	인스웨이브	2	2017-12-21 14:05:38

1 2 3 4 5

글쓰기

11. 내용 입력 후 작성 버튼을 클릭합니다.

문의내용 작성하기

1. 질문하기

제목 첨부파일 작성

첨부파일

질문내용

이메일 답변여부 답변요청

첨부파일 C:\fakepath\bg_btn_close.png

작성 취소

12. 작성한 글 확인 및 첨부파일 다운로드와 답변을 달 수 있습니다.

Q&A 상세조회
목록

제목	첨부파일 작성	이메일	abc@inswave.com
이메일답변여부	답변요청	등록일자	
작성자	인스웨이브	전화	010-1234-5678
작성일	2017-12-07 15:36:39	조회	46
처리상태	접수대기		
첨부파일	/sht/upload/up/bg_btn_close_2017-12-07(15 36 38).png		

질문

첨부파일!

답변달기

13. 작성한 글을 제목을 기준으로 검색할 수 있습니다. (현재는 제목으로만 검색이 가능합니다.)

문고 답하기(Q&A)

전체
첨부

번호	제목	작성자	조회수	등록일
86	첨부파일 작성	인스웨이브	46	2017-12-07 15:36:39
6	첨부파일이 올라가지 않음	인스웨이브	63	2017-11-03 17:21:07

글쓰기

33.2.6. websquare.xml 로그

각 로그 속성에 대한 설명은 아래와 같습니다.

Table 33-2 로그 속성

항목	설명
console	true로 설정하면 stdout으로도 로그 내용이 출력
description	로그 설명 작성

항목	설명
dir	로그 파일을 저장할 경로
filename	로그 파일명
level	OFF, SEVERE (highest value), WARNING, INFO, CONFIG, FINE, FINER, FINEST (lowest value), ALL
lineNumber	로그 기록 라인 번호 저장 여부 (시스템 리소스를 많이 사용하기 때문에 운영시에는 반드시 false로 설정 해야함)
name	websquare - 웹스퀘어 로그명, exception - 예외사항 로그명
retentionPeriod	로그 저장 기간 (단위: 일)
thread	쓰레드 번호 저장 여부 (시스템 리소스를 많이 사용하기 때문에 운영시에는 반드시 false로 설정 해야함)

1. **Websquare.xml** 파일에 아래 내용을 입력합니다.

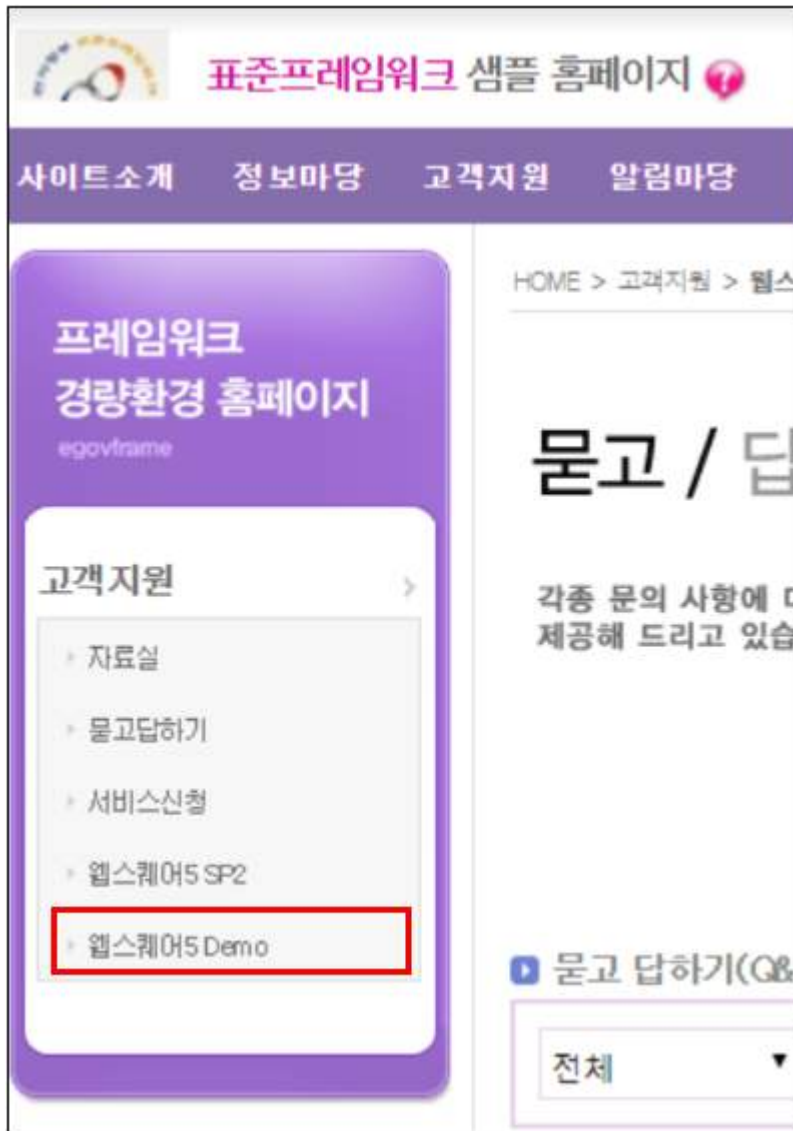
```

1 <log>
2   <target console="false" description="기본 로그"
3   dir="C:\eGovFrameDev-3.5.1-32bit\resource\websquare_home\log"
4   filename="websquare" level="ALL" lineNumber="false" name="websquare" retentionPeriod="3"
5   thread="false" />
6   <target console="false" description="에러 로그"
7   dir="C:\eGovFrameDev-3.5.1-32bit\resource\websquare_home\log"
8   filename="exception" level="ALL" lineNumber="false" name="exception" retentionPeriod="3"
9   thread="false" />
10 </log>

```

33.2.7. 웹스퀘어5 데모

1. 웹스퀘어5 Demo를 클릭하면 새로운 탭에서 화면이 호출됩니다.



2. 대량데이터 조회를 클릭합니다.



3. 아래와 같이 설정합니다. (10만 건 & 100)

4. FIND 버튼을 클릭합니다.

EDI 판매 (SFOMRA240) ? 🔍 ★

10만건 FIND EXPORT

EDI GROUP 100

5. 그리드의 스크롤을 최하단까지 내리면 5만건씩 추가됩니다.

No	ed1	ed2	ed3	ed4	ed5	ed6	ed7	ed8	ed9	ed10	ed11
144327	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144328	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144329	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144330	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144331	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144332	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144333	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144334	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144335	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144336	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144337	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144338	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144339	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144340	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144341	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144342	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144343	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144344	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144345	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144346	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144347	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144348	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144349	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144350	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144351	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144352	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144353	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144354	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144355	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144356	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144357	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144358	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144359	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144360	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC
144361	가나다	123	abc	정렬순	1 123	가 나	가 나	가 나	가 나	가 나	ABC

33.3 SpringBoot Jackson 연동

33.3.1. 개발 환경 구성

1. SpringBoot 프로젝트 구성시 SpringStartProject로 생성합니다.
(참고 URL: <http://www.adeveloperdiary.com/java/spring-boot/create-spring-boot-application-step-step/>)
2. WebSquare Studio Plug-in을 설치합니다.

SpringBoot 1.5.2를 기준으로 작성한 환경정보는 아래와 같습니다.

Table 33-3 환경정보 (SpringBoot 1.5.2를 기준)

항목	설명	비고
OS	Windows7, Vista 이상	

항목	설명	비고
JDK	SpringBoot 내장 JDK	
IDE	Eclipse 4.4(Luna) 이상	구현도구에 포함



1. WebSquare Studio는 Mozilla에서 배포한 Xulrunner Plug-in을 사용합니다. 해당 배포 버전 Plug-in이 64-bit 시스템에서 자주 충돌을 일으키는 것으로 보고되어 32-bit IDE 사용을 권장합니다.

33.3.2. SpringBoot와 WebSquare 연계

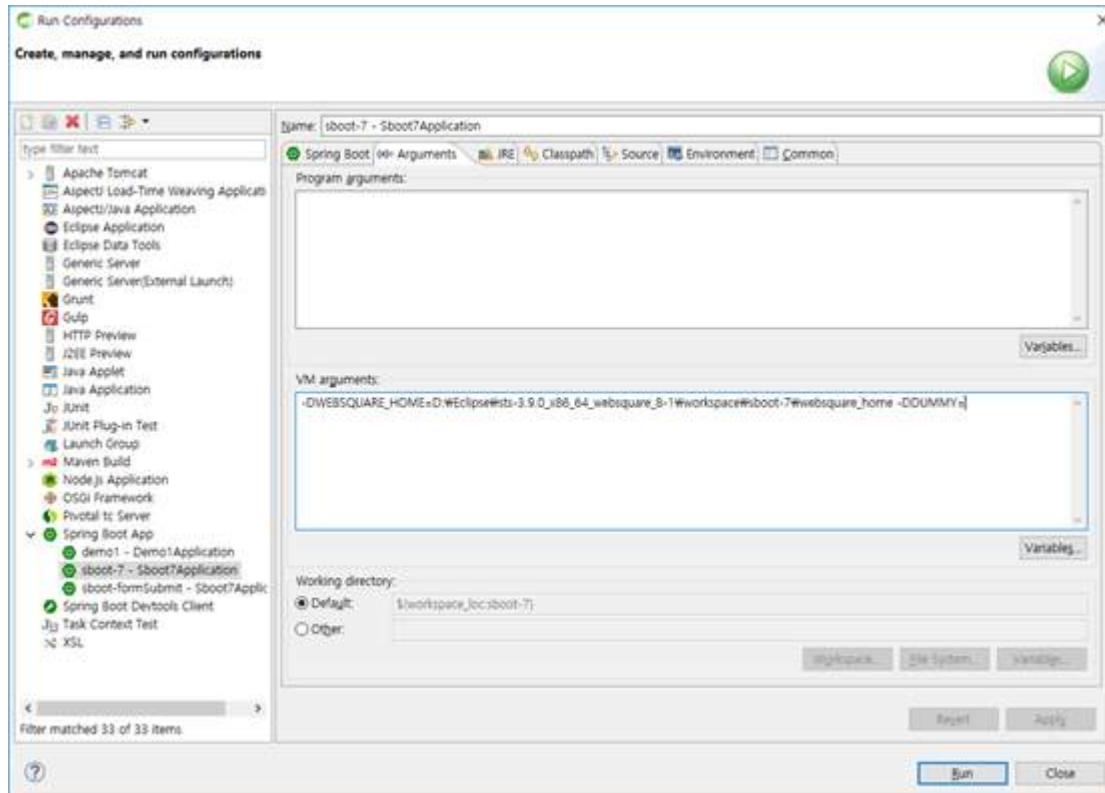
아래는 개발 환경이 이미 구축되어 있다고 가정 하에 기본적인 Spring Start Project 로 생성된 프로젝트를 기준으로 한 내용입니다. 또한 연계를 위해 SpringBoot의 messageConverters를 사용했습니다.



websquare_home 설정 및 Java Options 등록은 방법은 [WebSquare5 Studio](#)를 참고하십시오.

1. SpringBoot에서 **websquare_home**을 아래와 같이 설정하십시오.

그림 33-6 websquare_home 설정





websquare_home 설정 및 Java Options 등록은 방법은 [WebSquare5 Studio](#)를 참고하십시오.

2. WebSquare 엔진을 설치하십시오.

그림 33-7 WebSquare 엔진의 압축을 해제한 디렉토리 구조

이름	수정된 날짜	유형	크기
API	2018. 3. 19. 오후...	파일 폴더	
WEB-INF	2018. 3. 19. 오후...	파일 폴더	
websquare	2018. 3. 19. 오후...	파일 폴더	
websquare_5.0_2.2994B.20180319.163...	2018. 3. 19. 오후...	Executable Jar File	9,593KB
websquare_5.0_2.2994B.20180319.163...	2018. 3. 19. 오후...	Executable Jar File	9,792KB
websquare_adapter_5.0_2.2994B.20180...	2018. 3. 19. 오후...	Executable Jar File	46KB
websquare_common_5.0_2.2994B.2018...	2018. 3. 19. 오후...	Executable Jar File	9,245KB
websquare_hybrid_5.0_2.2994B.201803...	2018. 3. 19. 오후...	Executable Jar File	5,097KB

3. WebSquare 리소스(B) 를 *eGovFrame Project > src > main > resources > static* 하위로 복사합니다.

그림 33-8 **websquare** 복사



4. *{Project}/pom.xml* 파일을 편집 하여 WebSquare 실행에 필요한 *lib* 폴더 하위 Library 를 등록합니다.

Code 33-3 **pom.xml**

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  http://maven.apache.org/xsd/maven-4.0.0.xsd">
4     <modelVersion>4.0.0</modelVersion>
5     <groupId>com.inswave</groupId>

```

```

6    <artifactId>WebSquareSpringBoot</artifactId>
7    <version>0.0.1-SNAPSHOT</version>
8    <packaging>jar</packaging>
9    <name>WebSquareSpringBoot</name>
10   <description>Demo project for Spring Boot</description>
11   <parent>
12       <groupId>org.springframework.boot</groupId>
13       <artifactId>spring-boot-starter-parent</artifactId>
14       <version>1.5.8.RELEASE</version>
15       <relativePath/> <!-- lookup parent from repository -->
16   </parent>
17   <properties>
18       <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
19       <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
20       <java.version>1.8</java.version>
21   </properties>
22   <dependencies>
23       <dependency>
24           <groupId>org.springframework.boot</groupId>
25           <artifactId>spring-boot-starter-web</artifactId>
26       </dependency>
27       <dependency>
28           <groupId>org.springframework.boot</groupId>
29           <artifactId>spring-boot-starter-test</artifactId>
30           <scope>test</scope>
31       </dependency>
32       <dependency>
33           <groupId>org.springframework.boot</groupId>
34           <artifactId>spring-boot-starter-actuator</artifactId>
35       </dependency>
36       <!-- WebSquare Engine Dependency Library ===== -->
37       <dependency>
38           <groupId>org.antlr</groupId>
39           <artifactId>antlr</artifactId>
40           <version>3.5.2</version>
41       </dependency>
42       <dependency>
43           <groupId>commons-fileupload</groupId>
44           <artifactId>commons-fileupload</artifactId>
45           <version>1.2</version>
46       </dependency>

```



```

47     <dependency>
48         <groupId>commons-io</groupId>
49         <artifactId>commons-io</artifactId>
50         <version>1.4</version>
51     </dependency>
52     <dependency>
53         <groupId>commons-logging</groupId>
54         <artifactId>commons-logging</artifactId>
55         <version>1.0.4</version>
56     </dependency>
57     <dependency>
58         <groupId>dom4j</groupId>
59         <artifactId>dom4j</artifactId>
60         <version>1.6.1</version>
61     </dependency>
62     <dependency>
63         <groupId>com.googlecode.json-simple</groupId>
64         <artifactId>json-simple</artifactId>
65         <version>1.1</version>
66     </dependency>
67     <dependency>
68         <groupId>net.sf.opencsv</groupId>
69         <artifactId>opencsv</artifactId>
70         <version>1.8</version>
71     </dependency>
72     <dependency>
73         <groupId>org.apache.poi</groupId>
74         <artifactId>poi</artifactId>
75         <version>3.9</version>
76     </dependency>
77     <dependency>
78         <groupId>org.apache.poi</groupId>
79         <artifactId>poi-ooxml</artifactId>
80         <version>3.9</version>
81     </dependency>
82     <dependency>
83         <groupId>org.apache.poi</groupId>
84         <artifactId>poi-excelant</artifactId>
85         <version>3.9</version>
86     </dependency>
87     <dependency>

```

```
88         <groupId>org.apache.poi</groupId>
89         <artifactId>poi-scratchpad</artifactId>
90         <version>3.9</version>
91     </dependency>
92     <dependency>
93         <groupId>organtlr</groupId>
94         <artifactId>ST4</artifactId>
95         <version>4.0.8</version>
96     </dependency>
97     <dependency>
98         <groupId>xalan</groupId>
99         <artifactId>xalan</artifactId>
100        <version>2.7.0</version>
101    </dependency>
102    <dependency>
103        <groupId>xmlbeans</groupId>
104        <artifactId>xbean_xpath</artifactId>
105        <version>2.2.0</version>
106    </dependency>
107    <dependency>
108        <groupId>xerces</groupId>
109        <artifactId>xercesImpl</artifactId>
110        <version>2.7.1</version>
111    </dependency>
112    <dependency>
113        <groupId>xml-apis</groupId>
114        <artifactId>xml-apis</artifactId>
115        <version>1.4.01</version>
116    </dependency>
117    <dependency>
118        <groupId>org.apache.xmlbeans</groupId>
119        <artifactId>xmlbeans</artifactId>
120        <version>2.5.0</version>
121    </dependency>
122    <dependency>
123        <groupId>org.apache.xmlbeans</groupId>
124        <artifactId>xmlbeans-qname</artifactId>
125        <version>2.3.0</version>
126    </dependency>
127    <dependency>
128        <groupId>org.apache.xmlbeans</groupId>
```

```

129         <artifactId>xmlbeans-xmlpublic</artifactId>
130         <version>2.5.0</version>
131     </dependency>
132     <!--         <dependency>
133             <groupId>websquare</groupId>
134             <artifactId>websquare</artifactId>
135             <version>5.0_2.2743B</version>
136         </dependency> -->
137
138     <dependency>
139         <groupId>websquare</groupId>
140         <artifactId>websquare</artifactId>
141         <version>5.0_2.2743B.20170823.183931_1.5</version>
142         <scope>system</scope>
143         <systemPath>C:\STS\apache-maven-3.5.2\repository\websquare\websquare\5.0_2.274
144         3B\websquare_5.0_2.2743B.20170823.183931_1.5.jar</systemPath>
145     </dependency>
146
147     <!-- jackson -->
148     <dependency>
149         <groupId>com.fasterxml.jackson.core</groupId>
150         <artifactId>jackson-databind</artifactId>
151         <version>2.9.0</version>
152     </dependency>
153     <dependency>
154         <groupId>com.fasterxml.jackson.core</groupId>
155         <artifactId>jackson-core</artifactId>
156         <version>2.9.0</version>
157     </dependency>
158     <dependency>
159         <groupId>com.fasterxml.jackson.core</groupId>
160         <artifactId>jackson-annotations</artifactId>
161         <version>2.9.0</version>
162     </dependency>
163 </dependencies>
164 <build>
165     <plugins>
166         <plugin>
167             <groupId>org.springframework.boot</groupId>
168             <artifactId>spring-boot-maven-plugin</artifactId>

```

```

169         </plugin>
170     </plugins>
171 </build>
172 </project>

```

5. WebSquare Servlet을 등록합니다. SpringBoot에서 Servlet 등록 시 @configuration 어노테이션으로 해당 Servlet을 java 클래스로 작성해야 합니다.

Code 33-4 *ServletRegistrationConfig.java*

```

1  package com.inswave.sample;
2
3  import org.springframework.boot.web.servlet.ServletRegistrationBean;
4  import org.springframework.context.annotation.Bean;
5  import org.springframework.context.annotation.Configuration;
6
7  @Configuration
8  public class ServletRegistrationConfig
9  {
10     @Bean
11     public ServletRegistrationBean getServletRegistrationBean()
12     {
13         ServletRegistrationBean websquareDispatcher = new ServletRegistrationBean(new
14 websquare.http.DefaultRequestDispatcher());
15         websquareDispatcher.addUrlMappings("*.wq");
16         return websquareDispatcher;
17     }
18 }

```



JackSon 연계시에는 Jackson 라이브러리만 pom.xml로 선언하십시오. @ResponseBody를 통해 파라미터를 받을 수 있습니다.

6. Controller Class를 아래와 같이 구현하십시오.

Code 33-5 *TestController.java*

```

1  ... 중략 ...
2  // JSON타입으로 200응답을 받기 위해서는 consumes = MediaType.APPLICATION_JSON_VALUE 해당
선언이 필요함.
3  // 만약 해당 선언이 없다면 HTTP Error 400 - Bad Request로 응답을 받는 현상이 있음.
4  @RequestMapping(value="/testJsonV0", consumes = MediaType.APPLICATION_JSON_VALUE)
5  public @ResponseBody DataBean<UserBean> testJsonV0() {

```

```

6      DataBean<UserBean> data = new DataBean<UserBean>();
7      List<UserBean> userList = new ArrayList<UserBean>();
8      HeaderBean header = new HeaderBean();
9      header.setResultCode("001");
10     header.setResultMessage("성공했습니다.");
11
12     UserBean user1 = new UserBean();
13     user1.setId("001");
14     user1.setName("홍길동");
15     user1.setTel("010-4232-7431");
16     userList.add(user1);
17     UserBean user2 = new UserBean();
18     user2.setId("002");
19     user2.setName("허균");
20     user2.setTel("010-4421-5421");
21     userList.add(user2);
22     data.setHeader(header);
23     data.setData(userList);
24     return data;
25 }
26 ... 중략 ...

```

7. 아래 URL을 호출하여 화면이 정상적으로 표시되는지 확인합니다.

- 호출 URL: localhost:8080/websquare/websquare.html?w2xPath=/sample1.xml



샘플 파일 다운로드

33.4 Spring 연동

전자정부 프레임워크는 Spring 기반으로 적성되었습니다. Spring 프레임워크 연동은 전자정부 프레임워크 연동과 매우 유사합니다. Spring 프레임워크 연동은 [33.2 전자정부 프레임워크 연동](#) 부분을 참조하십시오.

33.5 스트러츠 연동

스트러츠 연계는 데이터의 구조체로서 ActionForm(Java Beans구조)을 사용하는 Struts 1 버전을 기준으로 설명합니다.

33.5.1. 개요

- 스트러츠 프레임워크는 MVC 기반의 Web Application Framework로서, 웹스퀘어와의 연계를 위해서는 다음과 같은 연계 작업이 필요합니다.
- RequestProcessor class를 구현하여 Controller단의 request data 변환 및 View단의 response data 변환을 처리할 수 있도록 합니다.
- 스트러츠에서 데이터의 구조체로 사용되는 VO인 ActionForm에 데이터가 연계되도록 Annotation 방식으로 Action Class에 지정합니다.

33.5.2. 구성

33.5.2.1. RequestProcessor 구현

```
1 package sample.struts.processor;
2
3 import java.io.IOException;
4 import java.lang.annotation.Annotation;
5 import java.lang.reflect.Method;
6 import java.util.Iterator;
7 import java.util.Map;
8
9 import javax.servlet.ServletException;
10 import javax.servlet.http.HttpServletRequest;
11 import javax.servlet.http.HttpServletResponse;
12 import javax.servlet.http.HttpSession;
13
14 import org.apache.struts.action.ActionError;
15 import org.apache.struts.action.ActionErrors;
16 import org.apache.struts.action.ActionForm;
17 import org.apache.struts.action.ActionMapping;
18 import org.apache.struts.action.RequestProcessor;
```

```

19 import org.apache.struts.config.FormBeanConfig;
20 import org.apache.struts.config.ForwardConfig;
21 import org.apache.struts.util.PropertyMessageResources;
22
23 import sample.struts.form.ErrorForm;
24 import websquare.system.adapter.RequestAdapter;
25 import websquare.system.adapter.ResponseAdapter;
26 import websquare.system.adapter.annotation.BEAN_DEF;
27 import websquare.system.adapter.vo.BeanDefinitionCreatorFactory;
28
29 /**
30  * Struts의 controller의 기능을 확장
31  *
32  * @author Inswave
33  *
34  */
35 public class WQRequestProcessor extends RequestProcessor {
36
37     /**
38      * Action Form을 판단하여 request data mapping을 수행한다.
39      *
40      */
41     protected ActionForm processActionForm(HttpServletRequest request, HttpServletResponse
response, ActionMapping mapping) {
42
43         String reqContentType = request.getContentType();
44
45         // xml data, 즉, WebSquare Request가 요청되었을 시 Action Form 매핑을 수행한다.
46         if(reqContentType != null && (reqContentType.indexOf("application/xml") >= 0 ||
reqContentType.indexOf("application/json") >= 0)) {
47
48             ActionForm instance = null;
49
50             String name = mapping.getName();
51             request.setAttribute("WQ_MAPPING_NAME", name);
52             FormBeanConfig config = moduleConfig.findFormBeanConfig(name);
53             if (config == null) {
54                 //log.warn("No FormBeanConfig found under '" + name + "'");
55                 return null;
56             }
57

```

```

58         String actionClassName = mapping.getType();
59         String formClassName = config.getType();
60
61         try {
62             Method actionMethod = getActionMethod(actionClassName);
63             Annotation annotation = actionMethod.getAnnotation(BEAN_DEF.class);
64             Map beanDef = BeanDefinitionCreatorFactory.getInstance("annotation"
).create(annotation);
65             beanDef.put("root", formClassName);
66             instance = (ActionForm)new RequestAdapter().convert(request, beanDef);
67         } catch (Exception e) {
68             e.printStackTrace();
69             return null;
70         }
71
72         if (instance == null) {
73             return null;
74         }
75
76         if ("request".equals(mapping.getScope())) {
77             request.setAttribute(mapping.getAttribute(), instance);
78         } else {
79             HttpSession session = request.getSession();
80             session.setAttribute(mapping.getAttribute(), instance);
81         }
82         return instance;
83
84     } else {
85         return super.processActionForm(request, response, mapping);
86     }
87 }
88
89
90 protected void processForwardConfig(HttpServletRequest request,
91     HttpServletResponse response, ForwardConfig forward)
92     throws IOException, ServletException {
93
94     String reqContentType = request.getContentType();
95     if(reqContentType != null && (reqContentType.indexOf("application/xml") >= 0 ||
reqContentType.indexOf("application/json") >= 0)) {
96

```



```

97         String forwardName = forward.getName();
98
99         if(forwardName == null) throw new ServletException("Undefined Forward Name");
100
101         String name = (String)request.getAttribute("WQ_MAPPING_NAME");
102         if(name == null) {
103             throw new ServletException("No mapping name...");
104         }
105
106         ActionForm form = (ActionForm)request.getAttribute(name);
107         if(form == null) {
108             throw new ServletException("No mapping ActionForm...");
109         }
110
111         // Model단에서 오류 메시지를 전송하는 경우 Error Action Form을 전송한다.
112         ActionErrors errors = (ActionErrors)request.getAttribute("errors");
113         if(errors != null && errors.size() > 0) {
114
115             PropertyMessageResources mresources =
116             (PropertyMessageResources)request.getAttribute("org.apache.struts.action.MESSAGE");
117
118             ActionError error = null;
119             String messageKey = null;
120             String messageValue = null;
121             for (Iterator iterator = errors.get(); iterator.hasNext();) {
122                 error = (ActionError)iterator.next();
123                 messageKey = error.getKey();
124                 messageValue = mresources.getMessage(messageKey, error.getValues());
125                 if(messageValue == null || messageValue.trim().equals(""))
126                     messageValue = messageKey;
127
128                 break;
129             }
130
131             ErrorForm errorForm = new ErrorForm();
132             errorForm.setCode(messageKey);
133             errorForm.setMessage(messageValue);
134
135             try {
136                 new ResponseAdapter().convertAndSend(request, response, errorForm);
137             } catch (Exception e) {

```

```

136         throw new ServletException(e);
137     }
138
139     return;
140 }
141
142 try {
143     new ResponseAdapter().convertAndSend(request, response, form);
144 } catch (Exception e) {
145     throw new ServletException(e);
146 }
147
148 } else {
149     super.processForwardConfig(request, response, forward);
150 }
151 }
152
153 private Method getActionMethod(String className) throws SecurityException,
NoSuchMethodException, ClassNotFoundException {
154     Class actionClass = Class.forName(className);
155     Method actionMethod = actionClass.getMethod("execute", ActionMapping.class,
ActionForm.class, HttpServletRequest.class, HttpServletResponse.class);
156     return actionMethod;
157 }
158 }

```

33.5.2.2. Action Class 구현

```

1  package sample.struts.action;
2
3  import javax.servlet.http.HttpServletRequest;
4  import javax.servlet.http.HttpServletResponse;
5
6  import org.apache.struts.action.Action;
7  import org.apache.struts.action.ActionError;
8  import org.apache.struts.action.ActionErrors;
9  import org.apache.struts.action.ActionForm;
10 import org.apache.struts.action.ActionForward;
11 import org.apache.struts.action.ActionMapping;
12

```

```

13 import sample.struts.form.TestForm;
14 import websquare.system.adapter.annotation.BEAN_DEF;
15
16 public class TestAction extends Action {
17
18     @BEAN_DEF (beans={ "test=sample.struts.form.TestForm" })
19     public ActionForward execute(ActionMapping mapping, ActionForm form,
20         HttpServletRequest request, HttpServletResponse response)
21         throws Exception {
22         ActionErrors errors = new ActionErrors();
23
24         try {
25
26             TestForm tform = (TestForm) form;
27
28             String userid = tform.getUserid();
29
30             // 업무 구현
31
32             tform.setUsername("홍길동");
33
34         } catch (Exception e) {
35             // error message는 struts의 message properties를 그대로 사용한다.
36             errors.add(ActionErrors.GLOBAL_ERROR, new ActionError("error.access.message"));
37             System.err.println("execute(): " + e.toString());
38
39         }
40
41         request.setAttribute("errors", errors);
42         return (mapping.findForward("wqXml"));
43     }
44 }
45 }

```

33.5.2.3. Action Form Class 구현

```

1 package sample.struts.form;
2
3 import org.apache.struts.action.ActionForm;
4

```

```

5  @SuppressWarnings("serial")
6  public class TestForm extends ActionForm {
7
8      String userid;
9      String username;
10
11     public String getUserid() {
12         return userid;
13     }
14     public void setUserid(String userid) {
15         this.userid = userid;
16     }
17     public String getUsername() {
18         return username;
19     }
20     public void setUsername(String username) {
21         this.username = username;
22     }
23 }

```

33.5.2.4. struts-config.xml 설정

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2
3  <!DOCTYPE struts-config PUBLIC
4      "-//Apache Software Foundation//DTD Struts Configuration 1.1//EN"
5      "http://jakarta.apache.org/struts/dtds/struts-config_1_1.dtd">
6
7  <struts-config>
8
9      <controller
10         contentType="application/xml; charset=UTF-8"
11         debug="3"
12         locale="true"
13         nocache="false"
14         processorClass="sample.struts.processor.WQRequestProcessor"/>
15
16     <form-beans>
17         <form-bean name="testForm" type="sample.struts.form.TestForm" />
18     </form-beans>

```

```

19
20     <action-mappings>
21
22         <action
23             scope="request"
24             path="/struts/test"
25             name="testForm"
26             type="sample.struts.action.TestAction">
27         </action>
28
29
30     </action-mappings>
31
32 </struts-config>

```

33.5.2.5. WebSquare XML 구현

```

1 <xf:submission id="test" ref="request/map" target="response"
2     action="/struts/test.do" method="post"
3     mediatype="application/xml" encoding="UTF-8" instance=""
4     replace="instance" errorHandler="" customHandler=""
5     mode="asynchronous" processMsg="">
6     <script type="javascript" ev:event="xforms-submit-done"><![CDATA[
7         alert(WebSquare.ModelUtil.getInstanceValue("response/map/username"));
8     ]]></script>
9 </xf:submission>

```

```

1 var go = function() {
2     WebSquare.ModelUtil.setInstanceValue("request/map/userid", "honggildong");
3     WebSquare.ModelUtil.executeSubmission("test");
4 };
5 </javascript>

```

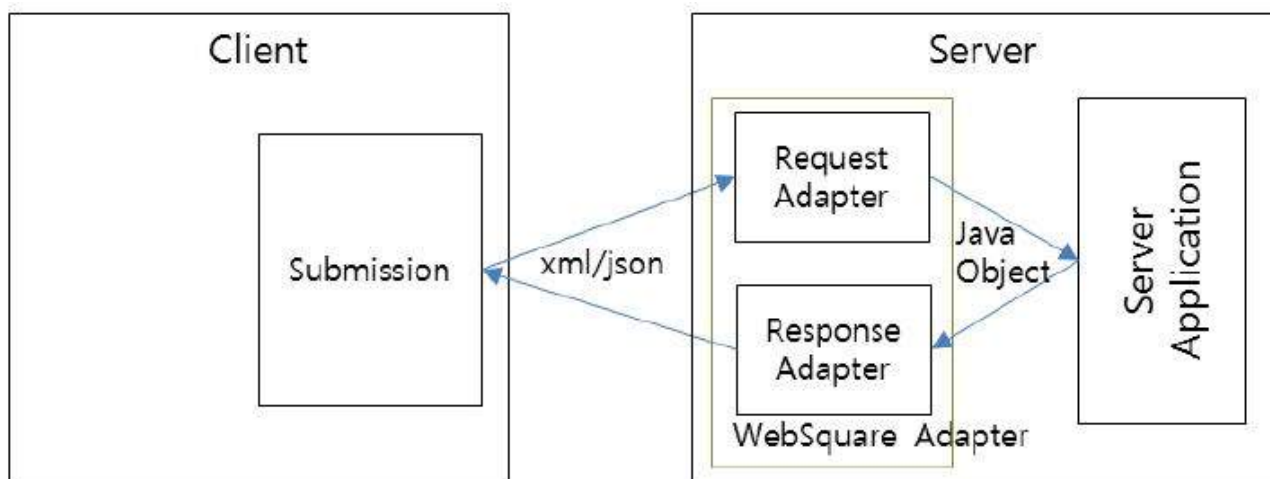
33.6 WebSquare JSON 어댑터 연동

33.6.1. Data Adapter 개요

WebSquare 는 서버와의 Data 연동을 위하여 Submission 이라는 통신 객체를 생성하고 , HTTP Ajax 방식에 기반한 표준 포맷의 데이터를 송수신 합니다. 본 가이드는 서버사이드에서 WebSquare 화면과 데이터 연동을 하는데 있어 서버측에서 사용하는 Java Object 형태의 데이터와 통신 데이터 사이의 변환을 처리하기 위하여 제공되는 WebSquare Data Adapter 의 사용법을 다룹니다.

33.6.2. 역할 및 기능

- WebSquare Data Adapter는 J2EE 표준 규약을 준수하는 Web Application Server에서 동작하는 Java Library 형태로 제공되며, HTTP 통신상의 HttpServletRequest, HttpServletResponse 객체를 핸들링하는 형태로 제공되며, 클라이언트와 연계되는 통신 데이터를 변환하는 역할을 합니다.



- 클라이언트 데이터(클라이언트-서버 간의 통신)는 HTTP기반의 Ajax 통신에서 표준방식으로 사용되는 xml과 json 타입의 텍스트 기반 데이터 Format을 지원합니다.
- 서버사이드에서 사용되는 데이터의 형태는 Java Object 형태의 Map/List 객체 타입과 POJO형태인 Java Beans (VO) 타입을 지원합니다.

33.6.3. 적용 방법

33.6.3.1. 기본 API

33.6.3.1.1. websquare.system.adapter.RequestAdapter

- HttpServletRequest에 포함되어 전달된 request data(XML or JSON String)를 추출하여 서버사이드에서 사용할 Java Object(Map or VO) 타입으로 변환처리를 수행합니다.
- 설정을 통하여 통신데이터(XML, JSON)와 서버사이드의 데이터 type(Map/List, VO)을 정의할 수 있으며, 통신데이터의 경우 설정을 하지 않으면 기본적으로 request의 Content-Type(media-type)에 따라 동작합니다.

Code 33-6 RequestAdapter 사용 예

```
1 import javax.servlet.http.HttpServletRequest;
2 import websquare.system.adapter.RequestAdapter;
3
4 ... 중략 ...
5 RequestAdapter reqAdapter = new RequestAdapter();
6 obj = reqAdapter.convert(request);
7 ... 중략 ...
```

33.6.3.1.2. websquare.system.adapter.ResponseAdapter

- 서버사이드에서 출력할 데이터를 Java Object(Map or VO) 타입에서 String(XM or JSON) 타입으로 변환처리를 수행합니다.
- RequestAdapter와 동일하게, 설정을 통하여 통신데이터(XML, JSON)와 서버사이드의 데이터 type(Map/List, VO)을 정의할 수 있으며, 통신데이터의 경우 설정을 하지 않으면 기본적으로 request의 Content-Type(media-type)에 따라 동작합니다.
- convert method를 통하여 변환된 String data를 return 받을 수 있으며, convertAndSend method를 사용하면, HttpServletResponse의 OutputStream으로 직접 write를 합니다. (설정을 통하여 GZipStream으로의 압출 전송도 지원합니다.)

Code 33-7 ResponseAdapter 사용 예

```
1 import javax.servlet.http.HttpServletRequest;
2 import javax.servlet.http.HttpServletResponse;
3 import websquare.system.adapter.ResponseAdapter;
4
5 ... 중략 ...
6 ResponseAdapter resAdapter = new ResponseAdapter();
```

```

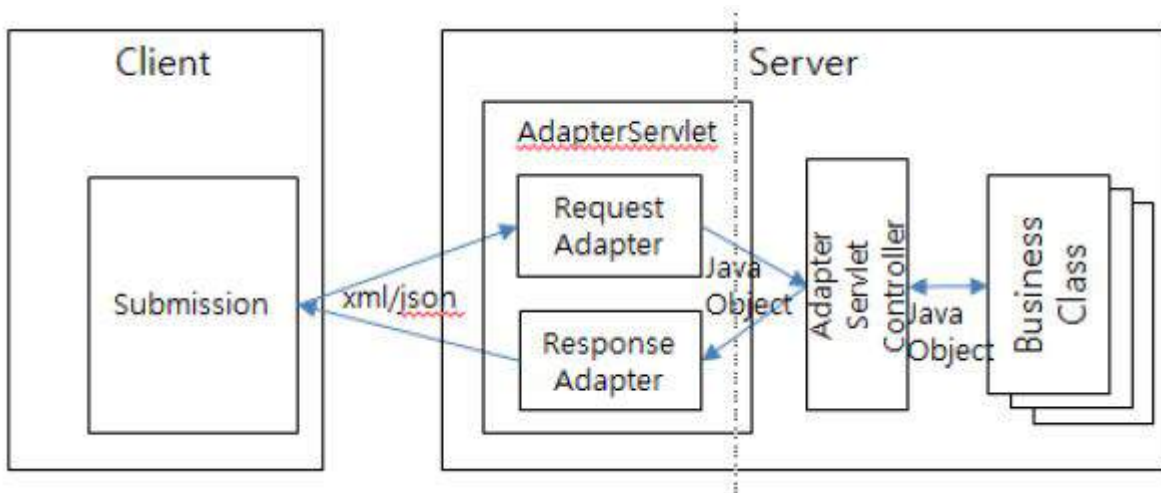
7
8 //String type return
9 String outString = resAdapter.convert(request, response, obj);
10
11 //response OutputStream 으로 write
12 resAdapter.convertAndSend(request, response, obj);
13 ... 중략 ...

```

33.6.3.2. Servlet

33.6.3.2.1. websquare.system.adapter.servlet.DefaultAdapterServlet

- DefaultAdapterServlet을 통하여 Request/Response 데이터 변환을 수행합니다.
- WebS Context의 설정파일인 web.xml에 등록하여 활성화합니다.
- 변환된 데이터를 전달하여 업무 클래스로 분기하는 것은 Adapter Servlet Controller를 구현하여야 합니다.
- Adapter Servlet Controller는 Adapter Servlet으로부터 넘어온 input data를 받아 업무클래스를 호출하고, 업무클래스로부터 넘어온 output data를 다시 Adapter Servlet으로 넘겨주는 역할을 합니다. 업무 클래스를 판단/분기하고 실행하는 처리는 Adapter Servlet Controller의 영역에 해당하며, IAdapterServletController interface를 implements하여 구현합니다.
- 구현한 Adapter Servlet Controller는 설정파일에 등록하여 Adapter Servlet이 인식할 수 있도록 합니다.
- Adapter Servlet Controller를 등록하는 방법은 두가지가 있다. (web.xml에 등록하는 방식을 권장함.)
 - WebSquare의 설정파일인 websquare.xml에 등록할 수 있으며, 모든 WebSquare Adapter Servlet에 적용됩니다.
 - Web Application Server의 context 설정파일인 web.xml에 WebSquare Adapter Servlet을 등록 시에 해당 Servlet의 init parameter로 Controller를 등록할 수 있다. 이렇게 등록된 경우에는 각 Servlet 단위로 Controller를 별개로 배치할 수 있으며, 이 설정은 websquare.xml의 설정보다 우선권을 갖습니다.



Code 33-8 DefaultAdapterServlet 등록 예 (web.xml)


```

1 <servlet>
2     <servlet-name>AdapterServlet</servlet-name>
3     <servlet-class>websquare.system.adapter.servlet.DefaultAdapterServlet</servlet-class>
4 </servlet>
5 <servlet-mapping>
6     <servlet_name>AdapterServlet</servlet-name>
7     <url-pattern>/servlet/websquare/adapterServlet</url-pattern>
8 </servlet-mapping>

```

Code 33-9 Adapter Servlet Controller 작성 예

```

1  package sample.servlet.controller;
2
3  import java.util.Map;
4
5  import javax.servlet.ServletException;
6  import javax.servlet.http.HttpServletRequest;
7  import javax.servlet.http.HttpServletResponse;
8
9  import sample.model.MapConverterSampleModel;
10 import websquare.system.adapter.servlet.IAdapterServletController;
11
12 public class SampleAdapterServletController implements IAdapterServletController {
13     public SampleAdapterServletController() {}
14
15     @Override
16     public Object control(HttpServletRequest request,
17                           HttpServletResponse response, Object input) throws Exception {
18
19         Map inputMap = (Map)input;
20         Map outputMap = null;
21
22         String submissionid = (String)inputMap.get("submissionid");
23
24         //submission ID에 따라 업무 클래스 분기 처리...
25
26         if(submissionid.equals("goSelect")){
27             outputMap = (Map)new MapConverterSampleModel().select(inputMap);
28         } else if(submissionid.equals("goModified")){
29             outputMap = (Map)new MapConverterSampleModel().modified(inputMap);
30         } else {

```

```

31         throw new ServletException("Undefined service model...");
32     }
33 }
34 }

```

Code 33-10 Adapter Servlet Controller 등록 예 #1(websquare.xml)

```

1 <adapter>
2     <servlet>
3         <controller value="Sample.servlet.controller.SampleAdapterServletController"/>
4     </servlet>
5 </adapter>

```

Code 33-11 Adapter Servlet Controller 등록 예 #2(web.xml)

```

1 <servlet>
2     <servlet-name>AdapterServlet</servlet-name>
3     <servlet-class>websquare.system.adapter.servlet.DefaultAdapterServlet</servlet-class>
4     <init-param>
5         <param-name>controller</param-name>
6
7         <param-values>sample.servlet.controller.SampleAdapterServletController</param-value>
8     </init-param>
9 </servlet>
10 <servlet-mapping>
11     <servlet-name>AdapterServlet</servlet-name>
12     <url-pattern>/servlet/websquare/adapterServlet</url-pattern>
13 </servlet-mapping>

```

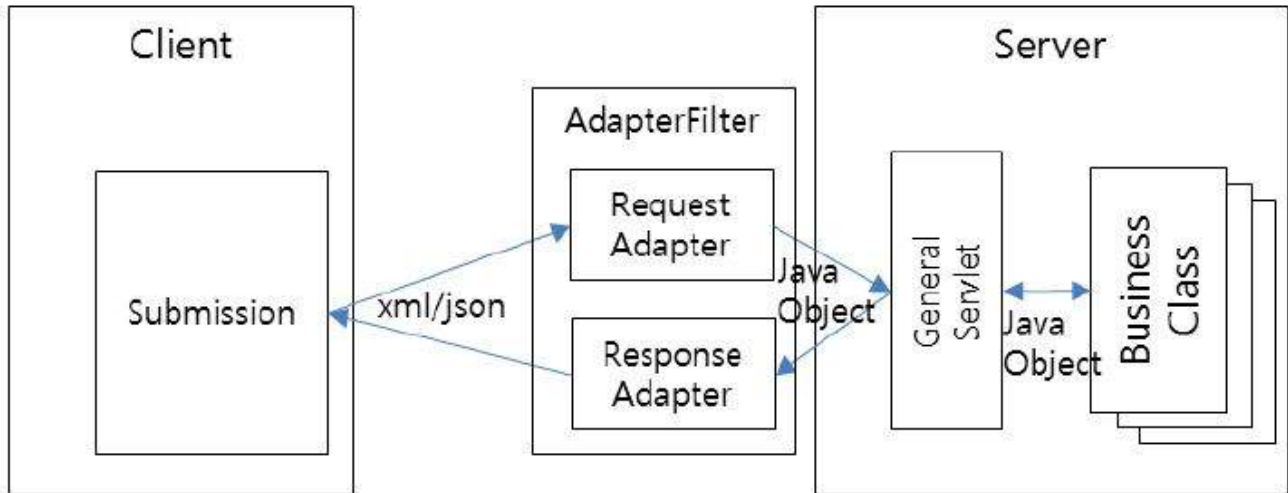
33.6.3.3. Filter

33.6.3.3.1. websquare.system.adapter.filter.AdapterFilter

- Filter는 J2EE 표준 스펙에서 제공되는 기능으로, 특정 URL 호출에서 대하여 메인프로세스에 대한 Request 전처리, Response 후처리의 기능을 제공합니다.
- AdapterFilter를 적용하면 특정 URL의 호출에 대하여 Request 메인프로세스(Servlet실행) 이전에 Data 변환을 수행하여 전달하여 주고, Response 프로세스 수행 후 Data 변환 후 Client로 데이터를 전송합니다.
- 제공되는 AdapterFilter를 Web Context 설정(web.xml)에 적용하여, URL 패턴에 따라 적용된 범위를 설정할 수 있으며 Web Application 구조에 종속되지 않고 전후처리기의 개념으로 별도의 모듈로 적용할 수 있습니다.
- AdapterFilter를 통하여 Request 데이터를 전달받는 Servlet은 이미 변환된 Java Object 타입의 데이터를 전달 받게 되며, 이 데이터는 HttpServletRequest 의 attribute로 전달된다. Response 데이터도 마찬가지로 HttpServletResponse의 attribute로 Java Object 를 전달하면 AdapterFilter에서 변환처리를 수행하여 client 로 전달

합니다.

- 주의: J2EE 표준 스펙에서 Filter는 다중으로 적용가능하며, 여러 Filter를 다중으로 설정할 경우, WebSquare AdapterFilter를 가장 앞서서 수행하도록 설정하여야 합니다. (Request Data Stream 추출에 대한 최우선처리가 되어야 함)



Code 33-12 Web Context 설정(web.xml)에 AdapterFilter 설정 예

```

1 <!-- Adapter Filter -->
2 <filter>
3   <filter-name>AdapterFilter</filter-name>
4   <filter-class>websquare.system.adapter.filter.AdapterFilter</filter-class>
5 </filter>
6 <filter-mapping>
7   <filter-name>AdapterFilter</filter-name>
8   <url-pattern>*</url-pattern><!-- 모든 url pattern에 filter 적용 -->
9 </filter-mapping>
  
```

Code 33-13 AdapterFilter를 통한 Servlet에서의 데이터 송수신 예시

```

1 package sample.filter;
2
3 import java.io.IOException;
4 import java.util.Map;
5
6 import javax.servlet.ServletException;
7 import javax.servlet.http.HttpServlet;
8 import javax.servlet.http.HttpServletRequest;
9 import javax.servlet.http.HttpServletResponse;
10
11 import sample.model.MapConverterSampleModel;
  
```

```

12
13 public class SampleFilterSelectServlet extends HttpServlet{
14
15     @Override
16     protected void doPost(HttpServletRequest request, HttpServletResponse response)
17         throws ServletException, IOException {
18         // input data 추출 (AdapterFilter 통과 후)
19         Map inputMap = (Map)request.getAttribute("WQ_VALUE_DATE");
20         Map outputMap;
21         try {
22             // business class 수행
23             outputMap = (Map)new MapConverterSampleModel().select(inputMap);
24         } catch(Exception e){
25             throw new ServletException(e);
26         }
27         //output data 전달 (AdapterFilter 통과 전)
28         request.setAttribute("WQ_VALUE_DATA", outputMap);
29     }
30 }

```

33.6.4. 지원 데이터 Format

- Request/Response Adapter는 내부적으로 Request/Response Converter를 통하여 데이터 변환을 수행합니다. 기본 Request/Response Converter는 다음과 같은 규칙으로 데이터의 변환을 수행합니다.

	Client Side(통신 Data Format)		Server Side	
	XML	JSON	Map/List	VO(Beans)
최상위 데이터 Type	"map" node	Json Object	Map	VO
개별 데이터(key, value)	<key value="value"/>	key: value	map.put(key, value)	객체의 member field (getter/setter로 연동)
반복 데이터 최상위 Type	"list" node	Json Array	List	List
반복 데이터 Row Type	"map" node	Json Object	Map	VO

주의: VO(Beans) 형태로 데이터를 연동하는 경우의 데이터 Type 제약사항

- 각 VO Class 내의 Member Field에는 대응되는 getter, setter method가 반드시 존재하여야 하며, Member Field의 Type과 setter method의 parameter Type, getter method의 return Type은 반드시 일치하여야 함

니다.

2. 사용할 수 있는 Member Field 의 Class Type은 아래와 같다. 이 외의 Type은 기본적으로 지원되지 않습니다.
 - String
 - int, Integer
 - float, Float
 - double, Double
 - BigDecimal
 - java.util.Date
 - 이 외는 기본적으로 String으로 casting된 형태로 적용되므로, 원치 않는 동작이 발생할 수 있습니다.
3. boolean Type의 Member Field의 경우 getter method의 prefix는 'is'로 시작합니다. 그 외 Type(Boolean Type 포함)의 getter method의 prefix는 'get'이다. setter method는 모두 'set' prefix를 갖습니다. (Eclipse 의 generate getter/setter의 규칙과 동일)

```
private boolean useSession;
```

public boolean getSession() (X) -> boolean type이므로 is prefix로 정의하여야 합니다.

```
public boolean isUseSession() (O)
```

예외) Member Field의 이름 자체가 소문자 'is' prefix를 가진 경우, getter method는 Member Field 와 동일한 이름을 가지며, setter method 는 'is'를 제거하고, 'set' prefix를 붙인다. (Member Field 명이 'is'가 없는 것과 동일한 형태가 된다.)

```
private boolean isSession;
```

```
public boolean isSession() (X)
```

public boolean isSession() (O) -> getter method 의 이름도 동일하게 isSession이 되어야 합니다.

```
Public void setSession(boolean session) (X)
```

```
Public void session(boolean session) (O) -> 'is'를 제거
```

```
private boolean IsSession;
```

```
public boolean isSession() (X)
```

public boolean isSession() (O) -> Member Field 가 대문자 'Is'로 시작하면, 원래의 규칙을 따른다.

- Type에 따라 혼동이 발생할 수 있으므로, 되도록이면 Member Field명 자체에 'is' prefix를 쓰지 않는 형태로 사용하는 것을 권장함.

4. Data Type 의 경우 String 형태의 문자열 데이터를 지정된 data format에 따라 변환하는 방식으로 제공하며, format은 설정을 통하여 변경할 수 있다. Format은 Java Date Format의 기준을 따른다.
(default: 'yyyMMdd', 7장 Configuration 참고)

33.6.4.1. Client Side(통신 Data Format)

33.6.4.1.1. XML

Code 33-14 XML Data Format 예

```

1  <map id= ''>
2    <userid>hong</userid>
3    <username>홍길동</username>
4    <list id= 'infoList'>
5      <map id= '0'>
6        <zip>우편번호0</zip>
7        <title>배송지0</title>
8        <address>서울0</address>
9        <telno>011-1111-0</telno>
10       <number>0</number>
11     </map>
12     <map id= '1'>
13       <zip>우편번호1</zip>
14       <title>배송지1</title>
15       <address>서울1</address>
16       <telno>011-1111-1</telno>
17       <number>1</number>
18     </map>
19     <map id= '2'>
20       <zip>우편번호2</zip>
21       <title>배송지2</title>
22       <address>서울2</address>
23       <telno>011-1111-2</telno>
24       <number>2</number>
25     </map>
26   </list>
27 </map>

```

33.6.4.1.2. JSON

Code 33-15 JSON Data Format 예

```

1  {
2    "userid": "hong",
3    "username": "홍길동",
4    "infoList": [

```

```

5      {
6          "zip":"우편번호0" ,
7          "title":"배송지0" ,
8          "address":"서울0" ,
9          "telno":"011-1111-0" ,
10         "number":"0"
11     } ,
12     {
13         "zip":"우편번호1" ,
14         "title":"배송지1" ,
15         "address":"서울1" ,
16         "telno":"011-1111-1" ,
17         "number":"1"
18     } ,
19     {
20         "zip":"우편번호2" ,
21         "title":"배송지2" ,
22         "address":"서울2" ,
23         "telno":"011-1111-2" ,
24         "number":"2"
25     }
26 ]
27 }

```

33.6.4.2. Server Side

33.6.4.2.1. Map

Code 33-16 Map/List Data 매핑 예

```

1  Map outMap = new HashMap();
2
3  outMap.put("userid", "hong");
4  outMap.put(":username", "홍길동");
5
6  List infoList = new ArrayList();
7  Map tmp = null;
8  for(int i=0; i<3; i++){
9      tmp = new HashMap();
10
11     tmp.put("number", ""+i);

```

```

12     tmp.put("title", "배송지"+i);
13     tmp.put("telno", "010-1111-"+i);
14     tmp.put("zip", "+"우편번호"+i);
15     tmp.put("address", "+서울"+i);
16
17     infoList.add(tmp);
18
19 }
20
21 outMap.put("infoList", infoList);

```

33.6.4.2.2. VO (Beans)

Code 33-17 VO Data 매핑 예

```

1  SelectBean sbean = new SelectBean();
2
3  sbean.setUserid("hong");
4  sbean.setUsername("홍길동");
5
6  // info list put
7
8  List infoList = new ArrayList();
9  InfoBean infoBean = null;
10
11  for(int i=0; i<3; i++){
12      infoBean = new InfoBean();
13
14      infoBean.setNumber(i);
15      infoBean.setTitle("배송지" + i);
16      infoBean.setTelno("011-1111-" + i);
17      infoBean.setZip("우편번호" + i);
18      infoBean.setAddress("서울" + i);
19
20      infoList.add(infoBean);
21
22  }
23
24  sbean.setInfoList(infoList);

```

Code 33-18 SelectBean 예시


```

1  package sample.beans;
2
3  import java.util.ArrayList;
4  import java.util.List;
5
6  public class SelectBean extends BaseBean {
7
8      private String userid;
9      private String username;
10
11     private List infoList = new ArrayList();
12
13     public String getUserid(){
14         return userid;
15     }
16
17     public List getInfolist() {
18         return infoList;
19     }
20
21     public void setInfolist(List infoList){
22         this.infoList = infoList;
23     }
24
25     public void setUserid(String userid){
26         this.userid = userid;
27     }
28
29     public String getUsername(){
30         return username;
31     }
32
33     public void setUsername(String username){
34         this.username = username;
35     }
36 }

```

Code 33-19 InfoBea 예시

```

1  package sample.beans;
2  public class InfoBean extends BaseBean {

```

```
3     private int number;
4     private String title;
5     private String telno;
6     private String zip;
7     private String address;
8
9     public int getNumber() {
10         return number;
11     }
12
13     public void setNumber(int number) {
14         this.number = number;
15     }
16
17     public String getTitle() {
18         return title;
19     }
20
21     public void setTitle(String title) {
22         this.title = title;
23     }
24
25     public String getTelno() {
26         return telno;
27     }
28
29     public void setTelno(String telno) {
30         this.telno = telno;
31     }
32
33     public String getZip() {
34         return zip;
35     }
36
37     public void setZip(String zip){
38         this.zip = zip;
39     }
40
41     public String getAddress(){
42         return address;
43     }
```

```

44
45     public void setAddress(String address){
46         this.address = address;
47     }
48 }

```

33.6.5. VO(Beans) Class Type 지정

- VO의 경우 데이터 변환 시에 해당 VO 객체에 대한 Class Type을 명시적으로 지정해 주어야만 VO 객체를 생성하여 값을 처리할 수 있습니다.
- Request 시에 xml/json format의 데이터를 VO 객체로 변환하기 위하여 VO Class의 type에 대한 정의를 Converter로 전달해 주어야 정상적으로 VO 객체를 생성하게 되며, 아래 3가지 방법이 제공됩니다. response VO 객체의 경우도 VO Class type을 인식해야 정상적으로 xml/json format로 변환되므로 request 용도의 VO, response 용도의 VO 모두 class type을 지정합니다.
- VO 객체의 변환은 XML의 map, node, JSON의 json object에 해당됩니다. 즉, XML의 map node, JSON의 json object에 대하여 변환된 VO 클래스를 지정하며, 반복데이터인 XML의 list node, JSON의 json array의 변환타입은 각 list node, json array의 VO 객체의 타입이 아닌 내부에 반복데이터로 존재할 row 레벨의 map node, json object에 해당하는 VO 객체의 타입을 의미합니다.

33.6.5.1. 클라이언트에서 VO Class Type 전달

- 클라이언트에서 서버로 전송될 데이터 내에 VO 클래스의 정의를 포함하여 서버로 전송합니다.
- "BEAN_DEF"라는 key로 XML의 map node, JSON의 json object에 해당하는 VO 클래스의 type을 지정합니다. "root"는 최상위 VO 객체의 type을 지정하는 key에 해당합니다.

Code 33-20 XML 데이터의 VO 타입 지정 예시

```

1  <request>
2      <map id="">
3          <map id = "BEAN_DEF">
4              <root value="sample.beans.SelectBean" />
5              <infoList value="sample.beans.InfoBean" />
6              <codeList value="sample.beans.CodeBean" />
7          </map>
8          <userid value=""/>
9          <username value=""/>
10     </map>
11 </request>

```

Code 33-21 JSON 데이터의 VO 타입 지정 예시

```

1  var requestData = {
2      userid : inputUserID.getValue(),
3      username : inputUserName.getValue(),
4      BEAN_DEF : {
5          root : "sample.bean.SelectBean",
6          infoList : "sample.beans.InfoBean",
7          codeList : "sample.beans.CodeBean"
8      }
9  };

```

33.6.5.2. 서버에서 Map 객체의 형태로 VO Class Type 전달

- 서버에서 RequestAdapter 호출 시 convert method의 parameter로 Map 객체로 정의한 VO Class 정의를 전달합니다.
- "root"는 최상위 VO 객체의 type을 지정하는 key에 해당합니다.

Code 33-22 Map 객체의 형태로 VO 타입 지정 예시

```

1  Map beanDef = new HashMap(){
2      {
3          put("root", "sample.beans.SelectBean");
4          put("infoList", "sample.beans.InfoBean");
5          put("codeList", "sample.beans.CodeBean");
6      }
7  };
8
9  RequestAdapter reqAdapter = new RequestAdapter();
10 obj = reqAdapter.convert(request, beanDef);

```

33.6.5.3. 서버에서 Annotation으로 VO Class Type 정의

- Annotation을 사용하기 위해서는 JDK1.5 이상의 환경이어야 합니다.
- Annotation의 정의를 위하여 WebSquare의 websquare.system.adapter.annotation.BEAN_DEF Annotation interface가 제공됩니다.
- MVC Web Application Framework 구조의 경우 데이터 변환을 수행할 Controller 영역에서 수행할 Model 클래스에 Annotation 을 정의하고 접근 가능한 구조여야 합니다.
(예: Spring 의 Controller 객체, Struts의 Action 객체 등)

Code 33-23 Annotation 형태로 VO 타입 지정 예시

```

1 Annotation annotation = param.getMethodAnnotation(BEAN_DEF.class)
2 RequestAdapter reqAdapter = new RequestAdapter();
3 obj = reqAdapter.convert(request, (BEAN_DEF)annotation);

```

Code 33-24 Annotation을 지정하는 Controller Class의 Method 예시

```

1 @BEAN_DEF (beans={ "root=sample.beans.ModifiedBean", "infoList=sample.beans.InfoBean",
2 "codeList=sample.beans.CodeBean"})
3
4 public ModelAndView goSelect( BaseBean ibean ) throws Exception {

```

33.6.6. Custom Converter 생성

- WebSquare Adapter의 구조에서 Request/Response Converter를 통하여 데이터 변환이 수행되며, 별도로 Converter를 명시적으로 지정하지 않으면, 기본 Converter가 사용됩니다.
- 기본 Converter에서 제공되는 Data Format이 아닌 별도의 Data Format을 적용할 목적으로 Custom Converter를 구현하여 적용할 수 있는 구조가 제공됩니다.
- 단, Custom Converter를 적용할 경우 데이터 변환에 대한 일체의 작업은 직접 구현해야 합니다.

33.6.6.1. Custom Converter 적용 방법

33.6.6.1.1. Custom RequestConverter

- IRequestConverter interface를 구현상속 받아 임의의 RequestConverter를 구현합니다.
- WebSquare의 서버 설정 파일인 websquare.xml 상에 구현한 converter를 설정합니다.
- 설정 파일에 등록하지 않고 RequestAdapter의 convert method를 호출 시 parameter로 전달하여 동적으로 호출하는 형태로도 사용 가능합니다.

Code 33-25 Customer RequestConverter 구현 예시

```

1 package sample.customconverter;
2
3 import java.util.Map;
4 import javax.servlet.http.HttpServletRequest;
5 import websquare.system.adapter.IRequestConverter;
6
7 /**
8  * Sample Request Converter
9  *
10 * XML/JSON request를 Map Object로 변환하는 예제

```

```

11  *
12  * @author inswave
13  *
14  */
15  public class SampleRequestConverter implements IRequestConverter {
16      /**
17       * request로 전달되어온 xml이나 json String type의 data를 Server에서 사용할 Map Object로
18       * 변환.
19       * method 내부에 데이터 변환을 구현하는 데에 따른 제약은 없으며,
20       * 실제 Project의 데이터 포맷 기준에 맞게 변환부를 구현하여야 합니다.
21       * 다만, return 되는 Object는 Server의 업무 Class에서 사용할 데이터의 형태로 변환되어야
22       * 합니다.
23       * 이 Sample에서는 Java Map Object로 변환하는 예시를 보여주며, RequestAdapter의 convert
24       * method를 호출하여 반환되는 데이터를 의미합니다.
25       *
26       * @param request
27       * @param HttpServletRequest
28       * @param plainData
29       * @param request data (xml or json String)
30       * @param beanDef
31       * @param V0(Beans) 형태로의 데이터 변환 시 V0 객체의 Class type을 정의한 object (Map or
32       * Annotation)
33       * @return 변환된 데이터 Object (Map/V0)
34       * @throws Exception
35       */
36      @Override
37      public Object convert(HttpServletRequest request, String plainData,
38                          Object beanDef) throws Exception {
39          Map imap = null;
40          // plainData -> imap 변환 구현...
41          return imap;
42      }
43  }

```

Code 33-26 Custom RequestConverter 설정 예시

```

1  <adapter>
2      <request>
3          <customConverter value="sample.customconverter.SampleRequestConverter" />
4      </request>
5  </adapter>

```

Code 33-27 동적 호출 예시

```

1 RequestAdapter reqAdapter = new RequestAdapter();
2 Object reqObj = reqAdapter.convert(request, new SampleRequestConverter());

```

33.6.6.1.2. Custom ResonseConverter

- IResponseConverter interface를 구현상속 받아 임의의 ResponseCOnverter를 구현합니다.
- WebSquare의 서버 설정 파일인 websquare.xml 상에 구현한 converter를 설정합니다.
- 설정 파일에 등록하지 않고 ResponseAdapter의 convert, convertAndSend method를 호출 시 parameter로 전달하여 동적으로 호출하는 형태로도 사용 가능합니다.

Code 33-28 Custom ResponseConverter 구현 예시

```

1 package sample.customconverter;
2
3 import java.io.OutputStream;
4 import java.util.Map;
5 import javax.servlet.http.HttpServletRequest;
6 import javax.servlet.http.HttpServletResponse;
7 import websquare.system.adapter.IResponseConverter;
8
9 /**
10  * Sample Response Converter
11  *
12  * Map Object를 XML/JSON resonse로 변환하는 예제
13  *
14  * @author inswave
15  *
16  */
17 public class SampleResponseConverter implements IResponseConverter {
18     public SampleResponseConverter() {
19
20     }
21
22     /**
23      * server에서 client로 전달할 데이터를 Java Object에서 xml이나 json String으로 변환.
24      * method 내부에 데이터 변환을 구현하는 데에 따른 제약은 없으며, 실제 Project의 데이터
25      * 포맷 기준에 맞게 변환부를 구현하여야 합니다.
26      * 이 Sample에서는 Map Object를 변환하는 예시를 보여주며,
27      * ResponseAdapter의 convert나 convertAndSend method를 호출하여 반환되는 데이터를
28      * 의미합니다.
29      *

```

```

28  * @param request
29  * HttpServletRequest
30  * @param response
31  * HttpServletResponse
32  * @param os
33  * OutputStream, ResponseAdapter에서 convertAndSend method를 호출 시
34  * client로의 OutputStream이 생성되어 전달되며, 해당 os를 통하여 write처리를 합니다.
35  * @param obj
36  * xml/json으로 변환할 Server Java Object (Map or VO)
37  * @return xml or json String, 단, ResponseAdapter에서 convert를 호출한 경우는 os
38  * parameter가 전달되지 않고 return값을 전달하여야 하며, convertAndSend를 호출한 경우는
39  * os parameter가 전달되며, 데이터는 os를 통하여 write하고, return값은 전달하지 않는다.
40  * @throws Exception
41  */
42  @Override
43  public String convert(HttpServletRequest request,
44                      HttpServletResponse response, OutputStream os, Object obj)
45                      throws Exception {
46
47      Map omap = (Map) obj;
48      // omap -> xml or json
49      String String str = .....;
50      If(os == null) {
51          return str; // return xml or json string...
52      } else {
53          os.write(.....); // write to OutputStream...
54      }
55  }
56  }

```

Code 33-29 Custom ResponseConverter 설정 예시

```

1  <adapter>
2      <response>
3          <customConverter value="sample.customconverter.SampleResponseConverter" />
4      </response>
5  </adapter>

```

Code 33-30 동적 호출 예시

```

1  ResponseAdapter resAdapter = new ResponseAdapter();
2  resAdapter.convertAndSend(request, response, new SampleResponseConverter(), resObj);

```


33.6.7. Configuration

- WebSquare Data Adapter의 대부분의 기능은 설정파일 상에서 설정이 가능하다, 단, 여기에서 의미 하는 설정은 기본 Converter의 동작에 대한 설정에 해당하며, Custom Converter를 구현한 경우에는 데이터 변환 관련 설정들은 적용이 되지 않습니다.
- 설정은 WebSquare의 서버 설정 파일인 websquare.xml에 적용합니다. 적용하지 않은 경우 default값으로 동작합니다.

- adapter/request/parameterKey : Form Parameter로 전송 시 parameter key(default: xmlValue)

- adapter/request/attributeKey : Filter 통과 시 HttpServletRequest attribute key
(default: WQ_VALUE_DATA)

- adapter/request/convertType: 데이터 변환 포맷 정의

- server: map/bean (default: map)
- client: xml/json/없음 (default: 없음)
-> 설정하지 않으면 request 의 Content-Type에 따라 동작합니다.
(application/xml, application/json)

- adapter/response/convertType: 데이터 변환 포맷 정의

- server: map/bean (default: map)
- client: xml/json/없음 (default: 없음)
-> 설정하지 않으면 request의 Content-Type에 따라 동작합니다.
(application/xml, application/json)

- adapter/response/charset: response data의 character set(default: UTF-8)

- adapter/response/customConverter: custom response converter 지정(default: 없음)
-> 설정하지 않으면 기본 Converter로 동작합니다.

- adapter/response/compress: Response 데이터 압축 여부 (default: false, 압축하지 않음)

- adapter/response/immediateToStream: convertAndSend method 호출 시 output String을 생성하면서 바로 OutputStream에 write 할 것인지 여부. IResponseConverter의 convert method의 argument로 OutputStream을 넘겨준 경우에 해당함. (default: true)

- adapter/xml/mapNode: xml map node명 (default: map)

- adapter/xml/listNode: xml list node명 (default: list)

- adapter/xml/valueNode: xml value attribute명 (default: value)

- use 속성: use=false인 경우 node의 text 값을 value로 사용합니다. (default: false)

- adapter/xml/convertAttributeToNode: request xml의 attribute를 node data로 치환 여부(default: true)

- adapter/xml/attributePrefix: request xml의 attribute를 node data로 치환할 경우 node명의 prefix(default: 없음)

- adapter/xml/attributePostfix: request xml의 attribute를 node data로 치환할 경우 node명의 postfix(default:

없음)

- adapter/xml/encoding: xml response xml 생성 시 encoding 처리 여부 (default: true)
- adapter/xml/useIndent: response xml 생성 시 indentation 처리 여부 (default: false)
- adapter/servlet/controller: Adapter 를 servlet 형태로 적용할 경우, Adapter Servlet에서 호출할 Adapter Servlet Controller 클래스를 지정합니다.
- Adapter/bean/dateFormat: VO 에서 Date 타입의 멤버변수에 대하여 date format을 지정
 - request: Java Date Format (default: 'yyyyMMdd')
 - response: Java Date Format (default: 'yyyyMMdd')

Code 33-31 설정 예시

```

1  <adapter>
2    <request>
3      <parameterKey value="xmlValue" /><!-- Form Parameter로 전송 시 HttpServletRequest
parameter key -->
4      <attributeKey value="WQ_VALUE_DATA" /><!-- Filter 통과 시 HttpServletRequest
attribute key -->
5      <convertType server="" client="" /><!-- server:map/bean, client:xml/json -->
6      <charset value="UTF-8" />
7      <customConverter value="" /><!-- sample.customconverter.SampleRequestConverter -->
8    </request>
9    <response>
10     <convertType server="" client="" /><!-- server:map/bean, client:xml/json -->
11     <charset value="UTF-8" />
12     <customConverter value="" /><!-- sample.customconverter.SampleResponseConverter
-->
13     <compress value="false" /><!-- response 압축 여부 (gzip output stream) -->
14     <immediateToStream value="true" /><!-- convertAndSend 호출 시 output String을
생성하면서 바로 OutputStream에 write 할 것인지 여부. true/false -->
15   </response>
16   <xml>
17     <mapNode value="map" />
18     <listNode value="list" />
19     <valueNode value="value" use="false" />
20     <convertAttributeToNode value="true" /><!-- xml의 map node의 attribute를
node레벨로 치환할 것인지 여부 -->
21     <attributePrefix value="" />
22     <attributePostfix value="" />
23     <encoding value="true" />
24     <useIndent value="false" />
25   </xml>

```

```
26     <servlet>
27         <controller value="sample.servlet.controller.SampleAdapterServletController" />
28     </servlet>
29     <bean>
30         <dateFormat request="yyyyMMdd" response="yyyyMMdd" />
31     </bean>
32 </adapter>
```


34. 성능 개선 가이드

34.1 코드 분석

34.1.1. 전역 변수

의도치 않은 전역 변수 선언이 있는지 확인하십시오.

var 선언 없이 변수를 생성할 경우, 전역 변수가 생성됩니다. 함수 내에서만 필요한 변수가 전역으로 선언될 경우, 함수 종료 후에도 참조가 가능하여 메모리 누수가 발생할 수 있습니다.

Code 34-1 수정 전

```
1  scwin.getDay = function(pYmd, format) {
2      try {
3          if (pYmd.length > 8) {
4              pYmd = pYmd.substring(0, 8);
5          }
6          if (typeof format !== "undefined" && format !== "" &&
WebSquare.util.getBoolean(format)) {
7              date = WebSquare.date.getDay(pYmd);
8              //date가 var 선언이 생략되어 있어 전역변수로 사용됨
9          } else {
10             ...
11         }
12         return date;
13     } catch (e) {
14     }
15 }
```

Code 34-2 수정 후

```
1  scwin.getDay = function(pYmd, format) {
2      try {
3          var date = -1;
4          //var를 생략하지 않고 date가 가질 수 없는 값을 초기값으로 설정한뒤,
5          // case에 따른 수식결과를 저장한뒤 리턴한다.
6          if (pYmd.length > 8) {
7              pYmd = pYmd.substring(0, 8);
```

```

8      }
9      if (typeof format !== "undefined" && format !== "" &&
WebSquare.util.getBoolean(format)) {
10         date = WebSquare.date.getDay(pYmd);
11     } else {
12         ...
13     }
14     return date;
15 } catch (e) {
16 }
17 }

```

34.1.2. 객체

객체에 적절한 API 사용 여부 및 수식 형태를 확인하십시오. API나 수식을 올바른 형태로 사용하지 않을 경우 에러가 발생하거나 잘못된 값이 반환될 수 있습니다.

34.1.2.1. API

Code 34-3 수정 전 (1)

```

1  scwin.isDate = function(sDate) {
2      try {
3          if (sDate.length == 8) {
4              sDate = sDate+ " 000000 " ;
5          }
6          var flag = true;
7          .....
8          return flag;
9      } catch (e) {
10     }
11 }

```

Code 34-4 수정 후 (1)

```

1  scwin.isDate = function(sDate) {
2      var flag = true;
3      try {
4          if (sDate.length == 8) {
5              sDate = sDate+ " 000000 " ;

```

```

6      }
7      var flag = true;
8      .....
9      return flag;
10     } catch (e) {
11         flag = false;    //sDate가 undefined인 경우 false 반환
12     }
13     return flag;
14 }

```

Code 34-5 수정 전 (2)

```

1  scwin.addDateSimpleisDate = function(sDate) {
2      try {
3          var date = " ";
4          var isDate = scwin.isDate(pYmd);    //8자리 이상의 string여부 체크
5          if (!isDate) return;
6          cYmd = pYmd.substring(0, 8);
7          var dTime = (pYmd+ " ").substring(8);
8          var isSet = Number(offset); //isDate에서 숫자인지 체크하지는 않음
9          if (isDate && typeof isSet == "number")
10             date = WebSquare.date.dateAdd(cYmd, isSet);
11             if (typeof format != "undefined" && format == "yyyyMMdd") {
12                 date = date;
13             } else {
14                 date = date + " " + dTime;
15             }
16             return date;
17         } catch (e) {
18         }
19     }

```

Code 34-6 수정 후 (2)

```

1  scwin.addDateSimpleisDate = function(sDate) {
2      try {
3          var date = " ";
4          var isDate = scwin.isDate(pYmd);
5          if (!isDate) return;
6          var cYmd = pYmd.substring(0, 8);
7          var dTime = pYmd.substring(8);
8

```

```

9      var isSet = Number(offset);
10     if (!isNaN(isSet)) date = WebSquare.date.dateAdd(cYmd, isSet);
11     // Number 변환시 NaN가 아닌 경우 dateAdd 수행
12     if (typeof format == "undefined" || format != "yyyyMMdd ")
13         date = date + dTime;
14     return date
15 } catch (e) {
16 }
17 }

```

34.1.2.2. 수식

객체 형태에 적합한 수식을 사용했는지 확인하십시오.

Code 34-7 수정 전

```

1  scwin.addYmd = function (pYmd, year, month, day) {
2      try{
3          var dY = (pYmd+"").substring(0, 4);
4          var dM = (pYmd+"").substring(4, 6);
5          var dD = (pYmd+"").substring(6, 8);
6          var dTime = (pYmd+"").substring(8);
7          if (year == "" || typeof year == "undefined")
8              year = 0;
9          if (month == "" || typeof month == "undefined")
10              month = 0;
11          if (day == "" || typeof day == "undefined")
12              day = 0;
13          var date = new Date();
14          date.setFullYear((dY*1) + year);
15          date.setMonth((dM*1) + month-1);
16          date.setDate((dD*1) + day);

```

Code 34-8 수정 후

```

1  scwin.addYmd = function (pYmd, year, month, day) {
2      try{
3          var dY = (pYmd+"").substring(0, 4);
4          var dM = (pYmd+"").substring(4, 6);
5          var dD = (pYmd+"").substring(6, 8);
6          var dTime = (pYmd+"").substring(8);
7          year = Number(year);

```



```

8      month = Number(month);
9      day = Number(day);
10     if (isNaN(year)) year = 0; // 리턴값이 숫자가 아닌 경우 0으로 변경
11     if (isNaN(month)) month = 0;
12     if (isNaN(day)) day = 0;
13     var cDate = $p.parseDate(dY+dM+dD);
14     cDate.setFullYear(cDate.getFullYear() + year); //일자 더하기 계산
15     cDate.setMonth(cDate.getMonth() + month);
16     cDate.setDate(cDate.getDate() + day);

```

34.1.3. 객체 생성

공통 소스의 경우, 사용하는 객체의 Lifecycle을 확인하여 재사용하거나 혹은 사용하지 않을 경우 삭제하십시오.

그림 34-1 수정 전



그림 34-2 수정 후



34.1.4. 스크립트 수행 시간 최소화

공통에서 자주 호출되는 함수는 최소한의 자원 사용으로 결과를 반환할 수 있도록 작성하십시오.

Code 34-9 수정 전

```

1 //일자를 계산된 월의 말일자로 변
2 if (Number(dD) >= 30) { //일자가 29일 경우 2월인 경우 윤년 체크필요
3     var chkMonth1 = [2];
4     var chkMonth2 = [4,6,9,11];
5     if (chkMonth1.indexOf(dM) > -1) {
6         if (scwin.isLeafYear(dY+ "01 " + "01 ")) {
7             //윤년체크를 위해 8자리 일자로 다시 생성
8             dD = "29 ";
9         } else {
10             dD = "28 ";
11         }
12     } else if (chkMonth2.indexOf(dM) > -1) { //31인 경우 30일로 변경
13         if (Number(dD) > 30) {
14             dD = "30 ";
15         }
16     }
17 }

```

Code 34-10 수정 후

```

1 if (Number(dD) >= 29) {
2     if (dM == 2) {
3         var dY = cDate.getFullYear();
4         var isLeafYear = ((dY % 400 == 0) || ((dY % 4 == 0) && (dY % 100 != 0))) ? true :
false;
5         if (isLeafYear) {
6             dD = "29";
7         } else {
8             dD = "28";
9         }
10     } else if (Number(dD) > 30 && (dM == 4 || dM == 6 || dM == 9 || dM == 11)) {
11         dD = "30 ";
12     }
13 }

```

34.1.5. 가독성

코드 분량과 코드 실행에 필요한 자원을 고려하여 최대한 가독성이 높은 형태로 코드를 작성하십시오.

Code 34-11 수정 전

```

1  //년월일 또는 년,월,일 또는 년,월,일,시,분,초에 해당하는 파라미터를 조합하여 일자 스트링을
   반환한다.
2  scwin.dateTime = function() {
3      try{
4          var len = arguments.length;
5          var isDate = true;
6          if (len > 1) {
7              var year = "";
8              var month = "";
9              var day = "";
10             var hour = "";
11             var minute = "";
12             var second = "";
13             if ( len > 3) {
14                 year = arguments[0] + "";
15                 month = arguments[1] + "";
16                 day = arguments[2] + "";
17                 hour = arguments[3]+"";
18                 minute = arguments[4] + "";
19                 second =arguments[5] + "";
20             } else {
21                 year = arguments[0] + "";
22                 month = arguments[1] + "";
23                 day = arguments[2] + "";
24                 hour = "0";
25                 minute = "0";
26                 second = "0";
27             }
28             if (month.length < 2) month = "0" + month;
29             if (day.length < 2) day = "0" + day;
30             if (hour.length < 2) hour = "0" + hour;
31             if (minute.length < 2) minute = "0" + minute;
32             if (second.length < 2) second = "0" + second;
33             date = year + month + day + hour + minute + second;
34             isDate = scwin.isDate(date);
35         } else {
36             if (typeof arguments[0] == "string") {
37                 date = arguments[0];
38                 var yy = date.substring(0, 4);
39                 var MM = date.substring(4, 6);

```

```

40     var dd = date.substring(6, 8);
41     var hh = date.substring(8, 10);
42     var mm = date.substring(10, 12);
43     var ss = date.substring(12);
44     if (MM.length < 2) MM = "01";
45     if (dd.length < 2) dd = "01";
46     if (hh.length < 2) hh = "00";
47     if (mm.length < 2) mm = "00";
48     if (ss.length < 2) ss = "00";
49     date = yy + MM + dd + hh + mm + ss;
50     isDate = scwin.isDate(date);
51   } else {
52   }
53 }
54 if(!isDate) {
55   date = "00000101";
56 }
57 return date;
58 } catch (e) {
59   scwin.trace("[scwin.dateTime] Exception :: ", e);
60 }
61 };

```

Code 34-12 수정 후

```

1  scwin.dateTime = function() {
2    try{
3      var len = arguments.length;
4      var yy, MM, dd, hh, mm, ss;
5      if (len == 1) {
6        var pYmd = arguments[0]+"";
7        yy = scwin.lpad(pYmd.substring(0, 4), "0", 4);
8        MM = scwin.lpad(pYmd.substring(4, 6), "0", 2);
9        dd = scwin.lpad(pYmd.substring(6, 8), "0", 2);
10       hh = scwin.lpad(pYmd.substring(8, 10), "0", 2);
11       mm = scwin.lpad(pYmd.substring(10, 12), "0", 2);
12       ss = scwin.lpad(pYmd.substring(12), "0", 2);
13     } else {
14       yy = (typeof arguments[0] == "undefined") ? "0000" :
scwin.lpad(arguments[0]+"", "0", 4);
15       MM = (typeof arguments[1] == "undefined") ? "01" : scwin.lpad(arguments[1]+"",
"0", 2);

```

```

16         dd = (typeof arguments[2] == "undefined") ? "01" : scwin.lpad(arguments[2]+ "",
"0", 2);
17         hh = (typeof arguments[3] == "undefined") ? "00" : scwin.lpad(arguments[3]+ "",
"0", 2);
18         mm = (typeof arguments[4] == "undefined") ? "00" : scwin.lpad(arguments[4]+ "",
"0", 2);
19         ss = (typeof arguments[5] == "undefined") ? "00" : scwin.lpad(arguments[5]+ "",
"0", 2);
20     }
21     if (MM == "00") MM = "01";
22     if (dd == "00") dd = "01";
23     var date = yy + MM + dd + hh + mm + ss;
24     var isDate = scwin.isDate(date);
25     if(!isDate) {
26         date = "00000101";
27     }
28     return date;
29 } catch (e) {
30     scwin.trace("[scwin.dateTime] Exception :: ", e);
31 }
32 };

```

34.1.6. 데이터 형태에 적합한 수식 사용

데이터 형태에 적합한 수식을 사용해야 합니다.

Code 34-13 수정 전

```

1  scwin.getMonthTerm = function (fromDate, toDate) {
2      try{
3          var iMonth = 0;
4          var iYear = 0;
5          var rMonth = "";
6          if (!scwin.isDate(fromDate) || !scwin.isDate(toDate)) {
7              return ;
8          }
9          if (parseInt(fromDate) <= parseInt(toDate)) {    //date를 숫자로 바꾸어 계산 -
정확하지 않음
10             iYear = parseInt(toDate.substr(0, 4)) - parseInt(fromDate.substr(0,4));
11             iMonth = parseInt(toDate.substr(4, 2), 10) - parseInt(fromDate.substr(4,2),

```

```

10);
12         rMonth = (12 * iYear) + iMonth;
13     }

```

Code 34-14 수정 후

```

1  scwin.getMonthTerm = function (fromDate, toDate) {
2      try{
3          var iMonth = 0;
4          var iYear = 0;
5          var rMonth = 0;
6          if (!scwin.isDate(fromDate) || !scwin.isDate(toDate)) return;
7          if ($p.dateDiff(fromDate.substring(0,8), toDate.substring(0,8)) >= 0)
8              {
9                  iYear = parseInt(toDate.substr(0, 4)) - parseInt(fromDate.substr(0,4));
10                 iMonth = parseInt(toDate.substr(4, 2), 10) - parseInt(fromDate.substr(4,2),
11);
12                 rMonth = (12 * iYear) + iMonth;
13             } else {
14                 return " ";
15             }
16     }

```

34.1.7. 변수 선언 및 참조 방식

선언되는 변수의 사용성에 따라 선언 위치와 사용 방법을 결정하십시오.

그림 34-3 수정 전

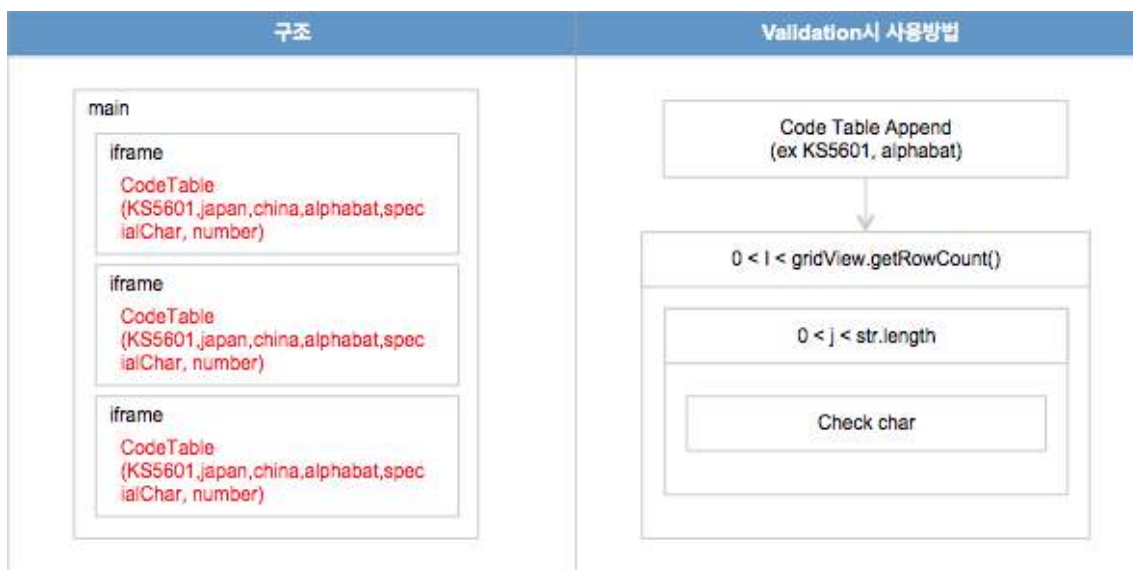
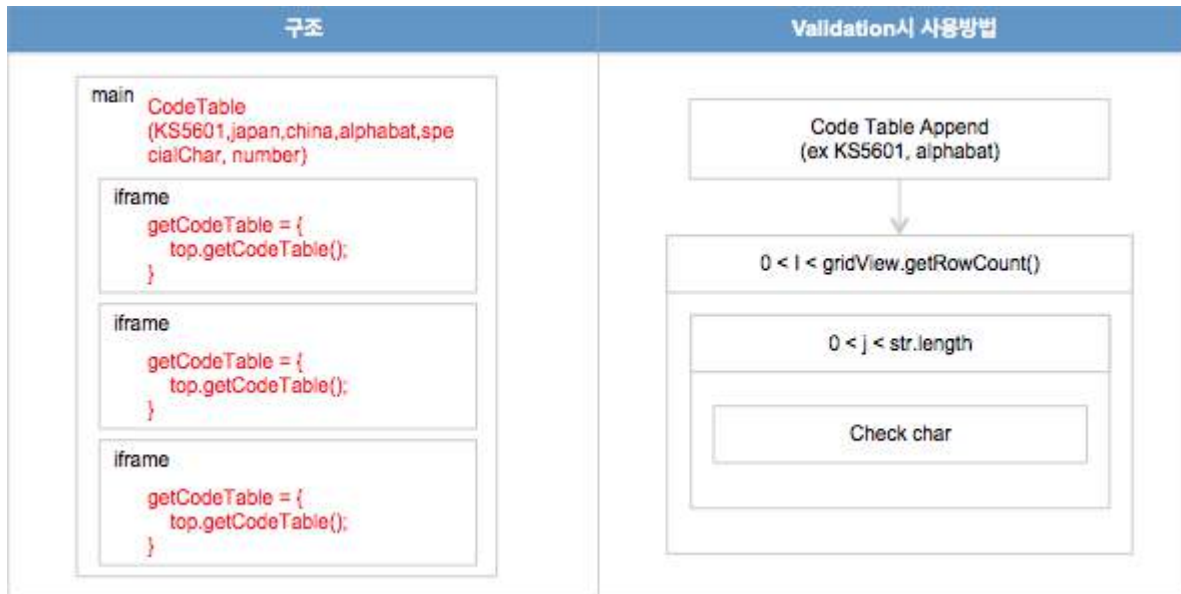


그림 34-4 수정 후



수정 내용은 아래와 같습니다.

1. CodeTable은 입력값 검증을 위해 참조되는 입력가능한 문자의 집합 스트링이므로 top에서만 선언. (메모리 절약)
2. IFrame이 참조하는 API는 top의 String을 반환하는 API로 생성. (반환값은 slice(0)을 통해 복사된 string.)
3. 컬럼 값에 허용 가능한 문자들을 IFrame 내의 API를 이용하여 하나의 String으로 생성.
4. 컬럼 값에서 하나의 char 씩 가져와서 가능한 문자 String에 존재하는지 확인.

34.1.8. 팝업 파라미터 전달

String이 아닌 Object 형식의 리턴 값은 사용하지 마십시오. 브라우저에 따라 메모리 누수가 발생할 수 있습니다.

그림 34-5 수정 전



그림 34-6 수정 후



34.1.9. 실행 시간 최소화

공통 소스는 최대한 빠른 시간에 결과 값을 반환할 수 있어야 합니다.

Code 34-15 수정 전

```

1  var windodwObj = {
2    pageUrl : "/" + Fprefix + "/xml/" + Formid + ".xml",
3    menuCd : Formid,
4    screenNm : Formid,
5    menuNm : FormName || Formid,
6    pageSize : pageSize
7  };
8  if (window.top.dlt_MenuList.getMatchedJSON("menuCd", Formid).length > 0) {
9    windodwObj.pageUrl = window.top.dlt_MenuList.getMatchedJSON("menuCd", Formid)[0].pageUrl
10   || "";
11    windodwObj.menuCd = window.top.dlt_MenuList.getMatchedJSON("menuCd", Formid)[0].menuCd
12   || "";
13    windodwObj.menuNm = window.top.dlt_MenuList.getMatchedJSON("menuCd", Formid)[0].menuNm ||
14   "";
15    windodwObj.screenNm = window.top.dlt_MenuList.getMatchedJSON("menuCd",
16   Formid)[0].screenNm || "";
17    windodwObj.parentMenu = window.top.dlt_MenuList.getMatchedJSON("menuCd",
18   Formid)[0].parentMenu || "";
19  } else if (window.top.dlt_MenuList.getMatchedJSON("screenNm", FormName).length > 0) {
20    windodwObj.pageUrl = window.top.dlt_MenuList.getMatchedJSON("screenNm",
21   FormName)[0].pageUrl || "";
22    windodwObj.menuCd = window.top.dlt_MenuList.getMatchedJSON("screenNm",
23   FormName)[0].menuCd || "";
24    windodwObj.menuNm = window.top.dlt_MenuList.getMatchedJSON("screenNm",
25   FormName)[0].menuNm || "";
26    windodwObj.screenNm = window.top.dlt_MenuList.getMatchedJSON("screenNm",

```



```

FormName)[0].screenNm || "";
19   windodwObj.parentMenu = window.top.dlt_MenuList.getMatchedJSON("screenNm",
FormName)[0].parentMenu || "";
20 }
21 if( GLV.G_DEVFG == "L" && (windodwObj.screenNm == "ZCOMSMKF00010" || windodwObj.screenNm ==
"ZCLMSVOC21410" || windodwObj.screenNm == "ZCLMSVOC11410") ) {
22   strFunction = "scwin.frmload_only_arg_call_" + Formid;
23 }
24 var openAction = "selectWindow";
25 var selectedTab = ( window.top.mainTab.getSelectedTabIndex() +1).zf(2);
26 var windowMDI = window.top.scwin.object("mdi"+selectedTab);
27 //이미 오픈되어 있으면 창호출 안함.
28 if (windowMDI && windowMDI.getWindow(windodwObj.screenNm)) {
29   return scwin._reloadNew(windodwObj.screenNm, strArg, strFunction);
30 }

```

Code 34-16 수정 후

```

1  ar reloadCheck = false; //이미 오픈되어 있으면 창호출 안함.
2  if (windowMDI) {
3    if (typeof FormName != "undefined" && FormName != "") {
4      if (windowMDI.getWindow(FormName)) {
5        reloadCheck = true;
6        Formid = windowMDI.getWindow(FormName).windowId; //window에 해당하는 id를 반환하는
api 사용
7      }
8    } else if (typeof Formid != "undefined" && Formid != "") {
9      if (windowMDI.getWindowByWindowId(Formid)) reloadCheck = true;
10    }
11    if(reloadCheck) return scwin._reloadNew(Formid, strArg, strFunction);
12  }
13  var windodwObj = {
14    pageUrl : "/" + Fprefix + "/xml/" + Formid + ".xml",
15    menuCd : Formid,
16    screenNm : Formid,
17    menuNm : FormName || Formid,
18    pageSize : pageSize
19  };
20  var menuDataList = window.top.dlt_MenuList.getMatchedJSON("menuCd", Formid);
21  if (menuDataList.length <= 0) menuDataList =
window.top.dlt_MenuList.getMatchedJSON("screenNm", FormName);
22  if (menuDataList.length > 0) {

```

```

23  windodwObj.pageUrl = menuDataList[0].pageUrl || "" ;
24  windodwObj.menuCd = menuDataList[0].menuCd || "" ;
25  windodwObj.menuNm = menuDataList[0].menuNm || "" ;
26  windodwObj.screenNm = menuDataList[0].screenNm || "" ;
27  windodwObj.parentMenu = menuDataList[0].parentMenu || "";
28  }

```

34.1.10. DataCollection 관련 API의 반복 수행

DataList나 DataMap의 경우, 데이터 변경이 발생할 때마다 이벤트도 함께 발생합니다. 다른 컴포넌트에 영향을 주는 이벤트일 경우, 한번에 데이터를 변경할 것을 권장합니다.

Code 34-17 수정 전

```

1  scwin.addBodyLink = function(ds_obj){
2    var lv_ARow = ds_obj.insertRow();
3    ds_obj.setCellData(lv_ARow, "link_nm", "주민등록증 진위확인/잠김해제");
4    ds_obj.setCellData(lv_ARow, "link_url",
"http://www.egov.go.kr/main?a=AA090UserJuminIsCertApp");
5    ds_obj.setCellData(lv_ARow, "add_gubun", 1);
6
7    lv_ARow = ds_obj.insertRow();
8    ds_obj.setCellData(lv_ARow, "link_nm", "운전면허증 진위여부 조회");
9    ds_obj.setCellData(lv_ARow, "link_url",
"http://dls.koroad.or.kr/jsp/ool/olq/OL_LcnsTruthYnRtvV.jsp");
10   ds_obj.setCellData(lv_ARow, "add_gubun", 1);
11
12   lv_ARow = ds_obj.insertRow();
13   ds_obj.setCellData(lv_ARow, "link_nm", "외국인등록증 진위 확인");
14   ds_obj.setCellData(lv_ARow, "link_url",
"http://www.hikorea.go.kr/pt/FrnRegIdChkPopupR_kr.pt");
15   ds_obj.setCellData(lv_ARow, "add_gubun", 1);
16   .....

```

Code 34-18 수정 후

```

1  scwin.addBodyLink = function(ds_obj){
2    var linkData = [{"link_nm" : "주민등록증 진위확인/잠김해제 ",
3                      "link_url" : "http://www.egov.go.kr/main?a=AArtApp ",
4                      "add_gubun" : " 1 " },
5                      {"link_nm" : "운전면허증 진위여부 조회 ",

```

```

6      "link_url " : "http://dls.koroad.or.kr/jsp/oolRtvV.jsp " ,
7      " add_gubun " : " 1 " },
8      .....
9  };
10  ds_obj.setJSON(linkData, true);

```

34.2 주요 확인 사항

34.2.1. 기본

Table 34-1 주요 확인 항목

대분류	소분류
이슈를 제기한 부서	
업무 환경	• 내부 업무 • 콜센타 시스템 등 시스템 집중 사용자가 있는 환경 • 대고객 업무
현재 개발/상태	• 개발 • 테스트 • 운영
시스템 환경	• 웹스퀘어 버전 • 웹스퀘어 스튜디오 버전 (빌드일) • 사용자 OS, 브라우저 • PC 사양 • 네트워크
구체적인 문제	• 속도: 34.2.2. 속도 참조. • 메모리 누수: 34.2.3. 메모리 참조. • 백화현상: 34.2.4. 백화 현상 참조.

34.2.2. 속도

Table 34-2 주요 확인 항목

대분류	소분류
초기 화면 로딩 속도 이슈인가?	
초기화 이후 조회 속도 이슈인가?	

대분류	소분류
화면의 조작이 느린 상황인가?	
기타 상황	• 메모리 누수 의심 • 네트워크 속도 의심 • 구형 브라우저 이슈 • 모바일 이슈 • SSL 관련 이슈
WEB서버와 WAS서버 종류 및 버전은?	

34.2.3. 메모리

Table 34-3 주요 확인 항목

대분류	소분류
브라우저가 종료되는 현상이 발생하는가?	
메모리가 얼마까지 증가하는가?	
메모리 릴리즈 여부	• 창 새로고침 • 창 닫기 • 브라우저 닫기 • OS 재기동
각종 환경	• 콜센터 상담직원 처럼 하루종일 동일한 화면을 사용하는가? • MDI 사용 여부 • 발생 브라우저 • 창간 객체 전달 여부 • ActiveX나 플러그인 사용 여부 • 문서보안/PC보안 모듈 사용 여부

34.2.4. 백화 현상

Table 34-4 주요 확인 항목

대분류	소분류
백화 현상이 발생했을 때 화면은 조작 가능한가?	
화면이 조작 가능한 경우	• 메모리와 CPU 점유율 확인 • 개발자 도구를 이용하여 화면의 객체 확인 • 다른 탭을 닫았을 경우 동작 확인
화면이 조작 가능하지 않은 경우	• 메모리와 CPU 점유율 확인 • 시스템 오류 메시지 확인

대분류	소분류
각종 환경	• 콜센터 상담직원 처럼 하루종일 동일한 화면을 사용하는가? • MDI 사용여부 • 발생 브라우저 • 창간 객체 전달 여부 • ActiveX나 플러그인 사용 여부 • 문서보안/PC보안 모듈 사용 여부

34.3 메모리 관리

메모리와 성능을 고려한 개발은 프로젝트의 성공 여부를 결정하는 요소 중 하나입니다. 특히, 특히 공통 소스는 모든 업무 화면에 영향을 줄 수 있기 때문에 메모리 및 성능에 영향이 없도록 최대한 간결하게 작성하십시오.

메모리 사용량이 증가는 메모리 부족을 의미하며 이는 곧 스크립트 수행 속도 저하로 이어집니다. 브라우저에 HTML 태그가 느리게 렌더링되어 브라우저의 백화 현상이 발생하거나 심할 경우 브라우저가 정상 동작하지 않습니다.

메모리 관리는 브라우저 사용성 및 개발 프로그램의 성능의 핵심입니다.

34.3.1. 메모리 누수 방지

34.3.1.1. WebSquare 공개 API 사용

- WebSquare5는 각 컴포넌트 별로 API를 제공하며 개발자는 각 컴포넌트에 대해 이벤트를 등록할 수 있습니다. 제공된 API를 사용하여 개발할 것을 권장합니다.
- 공개 API와 이벤트는 [WebSquare 기술문서](#)에서 확인할 수 있습니다.
- API 문서에 공개되지 않았지만 브라우저 개발자 도구에서만 확인된 API는 향후 변경 가능성이 있습니다. 해당 API 사용시 스크립트 에러가 발생할 수 있으며 사이트에서 문제 확인 및 디버깅이 어렵고 프로그램 수정이 필요한 경우도 발생할 수 있습니다.

34.3.1.2. jQuery의 제한적 사용

- jQuery에서 제공하는 API를 이용하여 컴포넌트를 검색하는 것은 간편하지만 검색 대상이 많은 경우 속도 저하가 발생할 수 있습니다. 또한 검색 후 수행되는 스크립트에 따라 검색을 위해 사용한 Object가 릴리즈 되지 않을 수도 있습니다.
- jQuery를 이용하여 이벤트를 등록할 경우 일부 브라우저에서는 컴포넌트(HTML Tag)에 등록된 이벤트를 먼저 삭제해야 컴포넌트 삭제시 컴포넌트에 연계된 Object들이 릴리즈 되는 경우가 있습니다.

그림 34-7 jQuery Component Search 유형별 성능 테스트 결과 (검색방식에 따른 속도 테스트) ([출처보기](#))

Testing in Chrome 65.0.3325 / Mac OS X 10.13.3		
Test		Ops/sec
find method (node context)	<code>var has = \$(article).find('ins').length > 0 ? true : false;</code>	1,321,051 ±1.38% fastest
find method (jQuery context)	<code>var has = \$(\$article).find('ins').length > 0 ? true : false;</code>	41,974 ±1.23% 97% slower
context node	<code>var has = \$('ins', article).length > 0 ? true : false;</code>	1,079,317 ±1.20% 18% slower
context jQuery	<code>var has = \$('ins', \$article).length > 0 ? true : false;</code>	43,204 ±1.24% 97% slower
context jQuery[0]	<code>var has = \$('ins', \$article[0]).length > 0 ? true : false;</code>	1,102,385 ±1.20% 16% slower
cascade	<code>var has = \$('article ins').length > 0 ? true : false;</code>	47,141 ±1.19% 96% slower
context (no cached)	<code>var has = \$('ins', 'article').length > 0 ? true : false;</code>	39,220 ±1.21% 97% slower

34.3.1.3. 최상위 scope에 함수 및 변수 선언 제한

- WebSquare에서는 업무 화면에서 함수 또는 상수 값 선언 시 객체의 속성으로 설정하여 사용할 것을 권장합니다. 화면 unload 시점에 선언된 객체의 속성으로 등록된 함수 및 상수를 제거하여 메모리 누수를 방지하십시오.
- 업무화면 내에서 사용하는 전역 변수 또한 지정된 객체의 속성으로 등록해서 사용하십시오.

Code 34-19 비권장 사용 예제

```

1 var GlobalValue = 1;
2
3 function doInit() {
4     //Something
5 }

```

Code 34-20 권장 사용 예제

```

1 scvar.GlobalValue = 1;
2
3 scwin.doInit = function() {
4     //Something
5 }

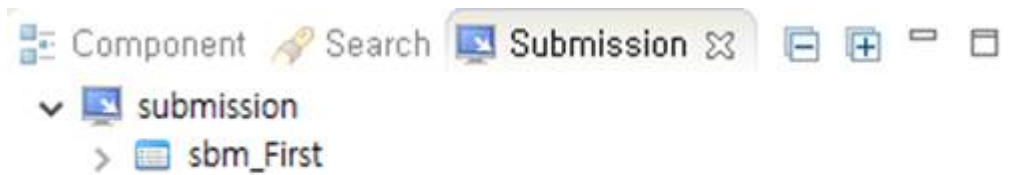
```

34.3.2. Submission

34.3.2.1. 업무화면에서 Submission 생성

업무화면에서 통신 객체(Submission)를 생성하여 필요한 통신 객체를 수행하는 것을 권장합니다.

그림 34-8 Submission 생성



업무 화면에서 필요한 통신 객체를 미리 생성해 놓는 것은 소스 가독성이나 Workflow 연계를 위해서도 필요한 중요한 작업입니다.

동적으로 Submission을 생성하는 경우 동일한 id의 Submission이 존재하는지 확인하십시오. 동일한 ID를 사용할 경우 기존 Submission의 ID를 수정하거나 삭제하십시오.

Code 34-21 비권장 사용 예제

```

1  common.executeSubmission = function(options) {
2
3      // ...
4
5      var submissionObj = {
6          "id" : options.id,
7          ...
8      }
9
10     $p.createSubmission(submissionObj);
11 }
```

Code 34-22 권장 사용 예제

```

1  common.executeSubmission = function(options) {
2
3      // ...
4
5      var submissionObj = {
6          "id" : options.id,
7          ...
8      }
```

```

8      }
9
10     if ($p.getSubmission(submissionObj.id) != null) {
11         $p.deleteSubmission(submissionObj.id);
12     }
13
14     $p.createSubmission(submissionObj);
15 }

```

34.3.2.2. Async 통신 권장 (Sync 모드 경고)

Submission 실행시 비동기(Asynchronous) 모드의 통신을 권장합니다.

그림 34-9 Async 모드 권장

The screenshot shows a 'w5 Add submission' dialog box. At the top right is a 'Submission Test' button. Below it, a note says '** indicates required item'. The 'ID*' field contains 'submission1'. The 'Reference' and 'Target' fields are empty, each with a 'DataCollection' button to its right. There is a 'Single Mode' checkbox. The 'URL Action' field is empty. The 'Mode' dropdown menu is open, showing 'asynchronous' selected, and this row is highlighted with a red border. Below it are 'Method*' (post), 'Media Type' (application/json), 'Encoding' (UTF-8), 'Instance', 'Replace', 'AbortTrigger', and 'Error Handler' fields. At the bottom are 'Simple', 'OK', and 'Cancel' buttons.

config.xml 파일에 `<warningSyn>` 항목이 "true"로 설정된 경우, 동기(synchronous) 모드의 Submission에 대해 경고 메시지가 표시됩니다.

Code 34-23 사용 예

```

1 <WebSquare>
2   <notRecommended>
3     <warningSyn value="true"/>

```



```
4    </notRecommended>
5  </WebSquare>
```



동영상

동기 통신 모드 경고 (비동기 통신 권장) - config.xml

그림 34-10 Studio에서 sync 모드를 선택할 때 표시되는 경고 메시지

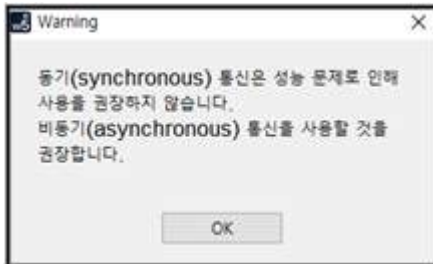
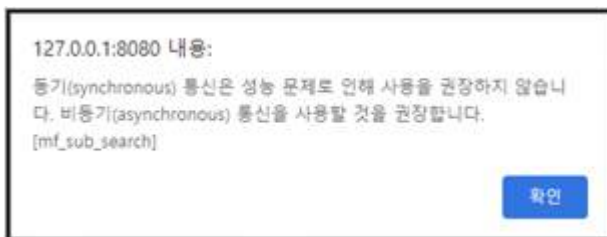


그림 34-11 sync 모드 사용 시 실행 화면에 표시되는 경고 메시지



34.3.3. \$p.ajax

34.3.3.1. 응답 객체의 직접 할당 금지

WebSquare는 Submission을 이용한 통신을 권장합니다. 그러나 사용자가 직접 통신 관련 설정을 해야하는 경우를 위해 `$p.ajax` API를 제공합니다.

아래 예제의 경우, 사용자가 직접 제어할 수 있도록 응답 결과 객체를 반환하고 있습니다. 그러나 업무 화면의 콜백 함수로 그대로 넘기기 때문에 업무 화면에서 변수 등에 직접 할당할 경우 통신 완료 후 응답 객체가 메모리에서 릴리즈되지 않을 수 있습니다.

Code 34-24 비권장 사용 예

```
1  $p.ajax({
2    action : action,
3    mode : mode,
4    mediatype : mediatype,
5    method : method,
```

```

6    processMsg : processMsg,
7    requestData : reqData,
8    requestHeader : requestHeader,
9    beforeAjax : function(e) {
10   },
11   success : function(e) {
12       var resBody = e.responseBody;
13       if (callbackFnc != '') {
14           eval(callbackFnc + '(resBody)');
15       }
16   },
17   error : function(e) {
18       return _requestErrCallback(e);
19   }
20 });

```

Code 34-25 권장 사용 예

```

1  $p.ajax({
2      action : action,
3      mode : mode,
4      mediatype : mediatype,
5      method : method,
6      processMsg : processMsg,
7      requestData : reqData,
8      requestHeader : requestHeader,
9      beforeAjax : function(e) {
10   },
11   success : function(e) {
12       if (callbackFnc != '') {
13           window[callbackFnc](JSON.parse(e.responseText));
14       }
15   },
16   error : function(e) {
17       return _requestErrCallback(e);
18   }
19 });

```

34.3.3.2. Closure 내부에서 객체 참조

Closure는 전역이 아닌 지역 변수 선언이 가능하여 공통 모듈에서 많이 사용합니다. 단, Closure 안에서 Object 형태의 외부 객체를 직접 참조하지 않도록 주의하십시오. 메모리에서 릴리즈 되지 않을 수 있습니다.

특히 Closure 함수를 비동기 형태로 호출할 경우, 객체를 직접 다른 변수에 할당하면 참조 관계가 발생합니다. 이 경우, 함수 실행이 완료되더라도 메모리에서 제거되지 않을 수 있습니다.

아래 예제에서 **test2**는 **test**와 직접 참조 관계가 이루어져 **test** 값 변경에 따라 **test2** 값이 변경됩니다. 즉 **test2**와 참조 관계가 발생하여 **test** 사용이 끝난 뒤에도 메모리에서 제거되지 않습니다.

Code 34-26 직접 참조 관계 예

```

1  //CASE1
2
3  var test = {a: "abc", b: "bbc"};
4  scwin.functionTest = function(value) {
5      scwin.test2 = value;
6      console.log(value);
7      // 결과 : {a: "abc", b: "bbc"}
8      console.log(scwin.test2);
9      // 결과 : {a: "abc", b: "bbc"}
10
11     for (var item in value) { value[item] = null; }
12     console.log(value);
13     // 결과 : {a: null, b: null}
14     console.log(scwin.test2);
15     // 결과 : {a: null, b: null}
16 };
17
18 scwin.functionTest(test);
19 console.log(test);
20     // 결과 : {a: null, b: null}
21
22 //CASE2
23
24 var test = {a: "abc", b: "bbc"};
25 scwin.functionTest = function(value) {
26     scwin.test2 = value;
27     console.log(value);
28     // 결과 : {a: "abc", b: "bbc"}
29     console.log(scwin.test2);
30     // 결과 : {a: "abc", b: "bbc"}

```

```

31  };
32  scwin.functionTest(test);
33  for (var item in test) { test[item] = null; }
34  console.log(test);
35      // 결과 : {a: null, b: null}
36  console.log(scwin.test2);
37      // 결과 : {a: null, b: null}

```

따라서 일반적으로 String 타입으로 파라미터를 전달하나, 직접 Object형식으로 파라미터를 전달해야만 하는 경우 반드시 초기화를 수행하여 호출된 함수에서 전역변수로 재사용되는 것을 방지하십시오.

Code 34-27 비권장 사용 예

```

1  $p.ajax({
2
3      // ...
4
5      success :    function(e) {
6                      resBody = e.responseBody;
7                      if (callbackFnc != '') {
8                          eval(callbackFnc + '(resBody, reqAjaxOptions)');
9                      }
10                     }
11  });
12  }
13
14  scwin.callback1 = function(resBody, reqAjaxOptions) {
15      if (reqAjaxOptions.mode) {
16
17          // ...
18
19      }
20      scwin.orgResData = resBody;
21  }

```

Code 34-28 권장 사용 예

```

1  $p.ajax({
2
3      //...
4
5      success :    function(e) {

```

```

6         resBody = e.responseBody;
7         if (callbackFnc != '') {
8             eval(callbackFnc + '(resBody, reqAjaxOptions)');
9             for(var item in resBody){
10                resBody[item] = null;
11            }
12            for(var item in reqAjaxOptions){
13                reqAjaxOptions[item] = null;
14            }
15        }
16    }
17 });
18 }
19
20 scwin.callback1 = function(resBody, reqAjaxOptions) {
21     if (reqAjaxOptions.mode) {
22
23         // ...
24
25     }
26
27     scwin.orgResData = JSON.parse(JSON.stringify(resBody));
28 }

```

34.3.4. DataCollection

34.3.4.1. 생성

DataCollection은 개발도구에서 생성하거나 API를 통해서 생성할 수 있습니다. 단, 업무화면에서는 유지보수를 위해 개발도구에서 DataCollection 선언 한 뒤 스크립트에서 사용하는 것을 권장합니다.



DataCollection을 API를 이용하여 생성할 때 동일한 id가 이미 존재하는 경우 삭제 후 생성하십시오.

Code 34-29 비권장 사용 예

```

1 common.searchCondition = function(targetCollectionId) {
2     var option = {};
3     option.id = targetCollectionId;
4     option.type = "dataList";

```

```

5    option.option = {"baseNode": "list", "repeatNode": "map"};
6    option.columnInfo = [{"id": "title", "name": "title", "dataType": "text"},
7                          {"id": "value", "name": "value", "dataType": "text"}];
8    $p.data.create(option);
9
10   //...
11
12  }

```

Code 34-30 권장 사용 예

```

1  common.searchCondition = function(targetCollectionId) {
2    $p.data.remove(targetCollectionId)
3    var option = {};
4    option.id = targetCollectionId;
5    option.type = "dataList";
6    option.option = {"baseNode": "list", "repeatNode": "map"};
7    option.columnInfo = [{"id": "title", "name": "title", "dataType": "text"},
8                          {"id": "value", "name": "value", "dataType": "text"}];
9    $p.data.create(option);
10
11   // ...
12
13  }

```

34.3.4.2. 초기화

DataList는 `removeAll`, DataMap은 `reset` API를 이용하여 등록된 데이터를 삭제하십시오.

Code 34-31 권장 사용 예

```

1  common.clearData = function(collectionObj) {
2    switch (collectionObj.initializeType) {
3      case "dataList":
4        collectionObj.removeAll();
5      case "dataMap":
6        collectionObj.reset();
7    }
8  }

```

34.3.5. 변수명

일반적인 네이밍 규칙에 어긋난 변수명 역시 메모리 누수를 유발합니다. 일부 브라우저(Internet Explorer)에서 변수명이 숫자로 이루어진 경우 메모리 누수 현상이 발생하는 경우가 있습니다. 일반적인 변수명 사용 규칙에 따라 영문으로 시작하는 변수명을 사용하십시오.

WebSquare에서 제공하는 컴포넌트는 id를 지정해야 합니다. 단, 이벤트 등록이나 스크립트에서 호출할 필요가 없는 경우 id를 생략할 수 있습니다.

Code 34-32 비권장 사용 예

```
1 <w2:anchor class="todolist_refresh" id="1" outerDiv="true">
```

Code 34-33 권장 사용 예

```
1 <w2:anchor class="todolist_refresh" id="anchor01" outerDiv="true">
2
3 // 혹은 id 삭제 가능
4
5 <w2:anchor class="todolist_refresh" outerDiv="true">
```

34.3.6. 팝업

34.3.6.1. 팝업의 Lifecycle

- 팝업에서 opener의 함수를 호출하는 경우, 호출된 함수가 정상적으로 완료되어 참조 관계가 끝난 뒤에 팝업을 종료해야 합니다.
- 팝업의 호출 순서에 따라 역으로 팝업을 종료하십시오 즉 A → B → C 순서로 호출했다면 C → B → A 순서로 종료해야 합니다.
- Popup 결과는 기본 자료형(일반적으로 String)으로 반환하십시오. Object (JSON 등) 형식으로 직접 전달할 경우 메모리 누수가 발생할 수 있다. ([34.3.6.2. 타 Frame으로 객체 전달](#) 참조.)
- 최신 ECMAScript 표준은 아래의 6개의 자료형을 정의합니다.
 - 기본 자료형(Primitive)인 여섯가지 데이터 타입: Boolean, Null, Undefined, Number, String, Symbol

34.3.6.2. 타 Frame으로 객체 전달

- 다른 Frame의 함수 호출 시 String 타입의 객체 형태로 파라미터를 전달하십시오. String 타입이 아닌 JSON 객체 등을 직접 전달하여 다른 Frame의 객체에 할당할 경우 일부 브라우저에서 메모리 누수 현상이 발생합니다.
- 다른 Frame간 전달되는 Object의 타입은 팝업 형식의 호출에만 국한된 것이 아니고 IFrame으로 구성된 화면에도 적용됩니다. Frame 간의 파라미터 전달 방식은 String 타입의 객체를 파라미터로 전달하십시오.

Code 34-34 비권장 사용 예

```

1  //Child
2
3  comf.onPopupClose = function(retValue) {
4
5  //...
6
7  comf.onPopupClose(retValue);
8  }
9  comf.onPopupClose = function(retValue) {
10     opener.comf._onPopupClose(retValue);
11  }
12
13 //Parent
14 comf._onPopupClose = function(retValue) {
15     scwin.callUserCallback(retValue);
16 }

```

Code 34-35 권장 사용 예

```

1  //Child
2
3  scwin.onPopupClose = function() {
4
5  // ...
6
7  comf.onPopupClose(retValue);
8  }
9  comf.onPopupClose = function(retValue) {
10     var str = JSON.stringify(retValue);
11     opener.comf._onPopupClose(str);
12  }
13
14
15 //Parent
16 comf._onPopupClose = function(str) {
17     var retValue = JSON.parse(str);
18     scwin.callUserCallback(retValue);
19 }

```


34.3.7. 객체 제어

JavaScript는 느슨한 타입(loosely typed) 언어, 혹은 동적(dynamic) 언어입니다. 즉, 변수 타입을 미리 선언할 필요가 없고 프로그램이 처리되는 과정에서 자동으로 파악 변환되기도 합니다. 공통함수가 객체 타입에 따른 대응을 못하고 에러가 발생하는 경우 스크립트가 중지되고 브라우저에 따라 성능 및 메모리에 영향을 받습니다.

Code 34-36 비권장 사용 예

```

1  scwin.addDateSimpleisDate = function(sDate) {
2      try {
3          var date = " ";
4          var isDate = scwin.isDate(pYmd); //8자리 이상의 string여부 체크
5          if (!isDate) return;
6          cYmd = pYmd.substring(0, 8);
7          var dTime = (pYmd+ " ").substring(8);
8          var isSet = Number(offset); //isDate에서 숫자인지 체크하지는 않음
9          if (isDate && typeof isSet == "number") date = $p.dateAdd(cYmd, isSet);
10         if (typeof format != "undefined" && format == "yyyyMMdd") {
11             date = date;
12         } else {
13             date = date + " " + dTime;
14         }
15         return date;
16     } catch (e) { }
17 }
```

Code 34-37 권장 사용 예

```

1  scwin.addDateSimpleisDate = function(sDate) {
2      try {
3          var date = "";
4          var isDate = scwin.isDate(pYmd);
5          if (!isDate) return;
6          var cYmd = pYmd.substring(0, 8);
7          var dTime = pYmd.substring(8);
8          var isSet = Number(offset);
9          if (!isNaN(isSet)) date = $p.dateAdd(cYmd, isSet);
10         // Number 변환시 NaN가 아닌 경우 dateAdd 수행
11         if (typeof format == "undefined" || format != "yyyyMMdd")
12             date = date + dTime;
13         return date;
14     } catch (e) { }
```

```
15 }
```

34.3.8. 전역 변수 선언

JavaScript는 느슨한 타입(loosely typed) 언어로 변수 선언을 위해 사용하는 `var` 문자를 누락해도 스크립트는 정상적으로 수행됩니다. 단, 이 경우 해당 변수는 전역에 선언되어, 함수 종료 후에도 계속 참조가 가능하고 자동 릴리즈가 되지 않습니다. SPA 방식으로 이동하는 화면인 경우 이는 메모리 누수 현상으로 이어집니다.

Code 34-38 비권장 사용 예

```
1 scwin.getDay = function(pYmd, format) {
2     try {
3         if (pYmd.length > 8) {
4             pYmd = pYmd.substring(0, 8);
5         }
6         if (typeof format !== "undefined" && format !== "" &&
7             WebSquare.util.getBoolean(format)) {
8             date = WebSquare.date.getDay(pYmd);
9             //date가 var 선언이 생략되어 있어 전역변수로 사용됨
10        } else {
11            // ...
12        }
13        return date;
14    } catch (e) {
15    }
16 }
```

Code 34-39 권장 사용 예

```
1 scwin.getDay = function(pYmd, format) {
2     try {
3         var date = -1;
4         //var를 생략하지 않고
5         // date가 가질 수 없는 값을 초기값으로 설정한뒤
6         // case에 따른 수식결과를 저장한뒤 리턴한다.
7         if (pYmd.length > 8) {
8             pYmd = pYmd.substring(0, 8);
9         }
10        if (typeof format !== "undefined" && format !== "" &&
11            WebSquare.util.getBoolean(format)) {
12            date = WebSquare.date.getDay(pYmd);
```

```

13     } else {
14         // ...
15     }
16     return date;
17 } catch (e) {
18 }
19 }

```

34.3.9. 공통 코드 조회

각 업무 페이지의 SelectBox나 CheckBox와 같은 공통 코드(혹은 코드성 데이터)가 존재합니다. 이 같은 공통 코드는 업무 페이지의 데이터를 불러오기 전에 먼저 조회해야 합니다. 비동기(asynchronous) 통신을 사용할 경우 공통 코드 조회가 완료되기 전에 업무 페이지의 데이터가 먼저 로드될 수 있습니다. 이 경우 SelectBox, CheckBox와 같은 공통 코드가 제대로 표현되지 않습니다.

아래의 방법을 권장합니다.

- orkflow 혹은 동기 통신을 사용하십시오.
- 동기 통신을 통해 함수 구현 시 하나의 request에 (각 업무 페이지에서 필요한) 코드 여러 개를 한번에 조회하십시오.
- MDI 혹은 WindowContainer를 사용할 경우 한번 조회한 공통 코드는 공통 영역에 저장하십시오.

34.3.10. 웹사이트 성능 최적화

WebSquare5로 렌더링된 화면은 대부분의 작업을 브라우저에 위임합니다. 즉, XML 파일을 HTML Tag 로 변환하여 브라우저가 변환된 HTML을 해석하도록 위임합니다.

아래와 같은 최적화 작업을 수행하여 웹사이트 성능을 최적화하십시오.

Expires 헤더 추가

정적인 자원(stylessheet, image, script 등)의 비효율적인 처리는 성능 저하로 연결됩니다.

해당 자원들 헤더에 Expires 정보를 추가하여 캐시 처리를 하십시오.

Gzip 압축

외부 파일 및 Response 데이터를 압축하여 성능을 높일 수 있습니다.

대부분의 브라우저가 Gzip 압축 전송을 지원하므로 Gzip 압축을 권장합니다.

34.3.11. 기타 주의할 내용

34.3.11.1. WFrame

IFrame 수가 지나치게 많은 경우 메모리 점유율이 올라갑니다. IFrame 대신 WFrame을 사용을 권장합니다.

34.3.11.2. TabControl

TabControl 컴포넌트는 `frameMode` 속성값으로 `iframe`과 `wframe`을 제공하며, 해당 속성값에 따라 콘텐츠를 렌더링 합니다. 단, `iframe` 을 설정할 경우 탭 수에 비례하여 브라우저의 메모리 점유가 증가합니다. 따라서 메인 층에서 사용하는 경우에만 속성값을 `iframe`으로 지정하길 권장합니다. `src` 속성은 콘텐츠에 웹스퀘어5 컴포넌트를 직접 구성 하지 않고 content 영역을 XML 파일을 불러 렌더링합니다.

다음은 잘못 사용한 코드 예입니다.

Code 34-40 잘못된 frameMode 설정 (예)

```

1  <w2:tabControl confirmTrueAction="exist" useTabKeyOnly="true" id="tabControl1"
useConfirmMessage="false" confirmFalseAction="new"
2      alwaysDraw="false">
3      <w2:tabs id="tabs1" style="width:70px;height:30px" label="" disabled="false"></w2:tabs>
4      <w2:tabs id="tabs2" style="width:70px;height:30px" label="" disabled="false"></w2:tabs>
5      <w2:tabs id="tabs3" style="width:70px;height:30px" label="" disabled="false"></w2:tabs>
6      <w2:tabs id="tabs4" style="width:70px;height:30px" label="" disabled="false"></w2:tabs>
7      <!-- 올바른 예 -->
8      <w2:content id="content1" alwaysDraw="false" frameMode="wframe"
src="/tabChild_001.xml"></w2:content>
9      <!-- 잘못된 예. 브라우저 리소스를 많이 사용하게 될 iframe 를 사용 -->
10     <w2:content id="content2" alwaysDraw="false" frameMode="iframe"
src="/tabChild_002.xml"></w2:content>
11     <!-- 잘못된 예. frameMode를 사용하지 않음. wframe 를 사용하여 해당 컴포넌트에 src 사용.
-->
12     <w2:content id="content3" alwaysDraw="false">
13         <w2:wframe src="/tabChild_003.xml"></w2:wframe>
14     </w2:content>
15     <!-- 잘못된 예. frameMode 를 사용하지 않음. iframe 를 사용 해당 컴포넌트에 src 사용.
-->
16     <w2:content id="content4" alwaysDraw="false">
17         <w2:iframe id="" src="/tabChild_004.xml" spa="false"></w2:iframe>
18     </w2:content>
19 </w2:tabControl>

```

34.3.11.3. 동기 요청 최소화

대부분의 프로젝트에서는 현재 시간 (서버 시간), 사용자 정보, 또는 사용자 권한을 조회하기 위해 동기 요청을 사용합니다. 한 화면에서 동기 요청을 여러 번 호출하는 것은 성능 저하로 이어질 수 있습니다.

하나의 요청으로 처리 가능한 함수를 구현하십시오.

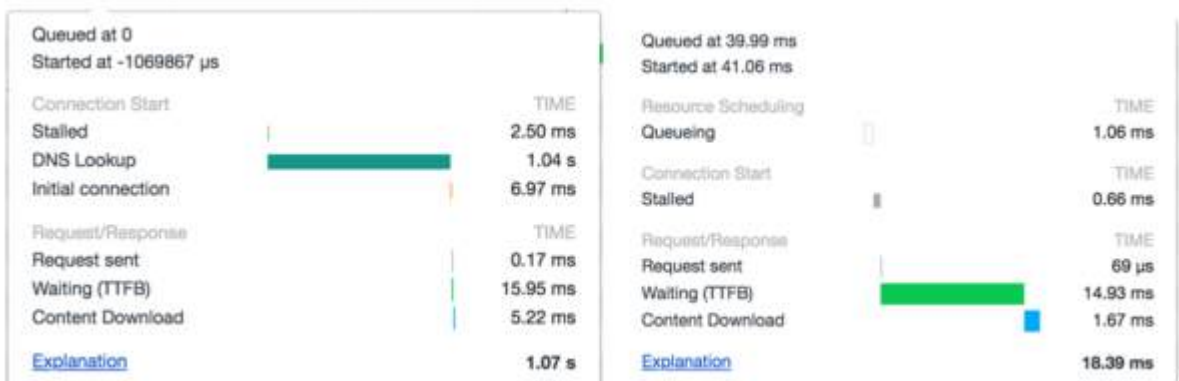
34.4 성능 관리

34.4.1. 초기 화면 네트워크 튜닝

초기 화면은 시스템 관점에서는 리소스 다운로드와 실행 및 그 외 준비로 시간이 많이 소요되는 단계이지만, 사용자 입장에서는 시스템의 진입 단계로 빠른 응답속도가 제공되어야 하는 중요한 화면입니다. 따라서 네트워크를 통해 리소스가 수집되는 단계를 이해하여 리소스 로딩에 대한 튜닝 작업이 반드시 필요합니다.

34.4.1.1. 리소스 로딩

Network 패널의 주어진 입력 항목에 대한 전체 타이밍 정보를 확인할 수 있습니다.



위와 같은 정보는 아래의 console 명령으로 추출할 수 있습니다.

```
performance.getEntriesByType('resource').filter(item => item.name.includes("config.xml"))
```

```

> performance.getEntriesByType('resource').filter(item => item.name.includes("config.xml"))
< ▼ [PerformanceResourceTiming] ⓘ
  ▼ 0: PerformanceResourceTiming
    connectEnd: 1112.8000000026077
    connectStart: 1112.8000000026077
    decodedBodySize: 16261
    domainLookupEnd: 1112.8000000026077
    domainLookupStart: 1112.8000000026077
    duration: 18.40000005904585
    encodedBodySize: 5263
    entryType: "resource"
    fetchStart: 1112.8000000026077
    initiatorType: "xmlhttprequest"
    name: "http://www.inswave.com/websquare/config.xml?postfix=15253214535595758.785215844062"
    nextHopProtocol: ""
    redirectEnd: 0
    redirectStart: 0
    requestStart: 1114.5000000251457
    responseEnd: 1131.2000000616536
    responseStart: 1129.5000000391155
    secureConnectionStart: 0
    ▶ serverTiming: []
      startTime: 1112.8000000026077
      transferSize: 5606
      workerStart: 0
    ▶ __proto__: PerformanceResourceTiming
  length: 1
  ▶ __proto__: Array(0)

```

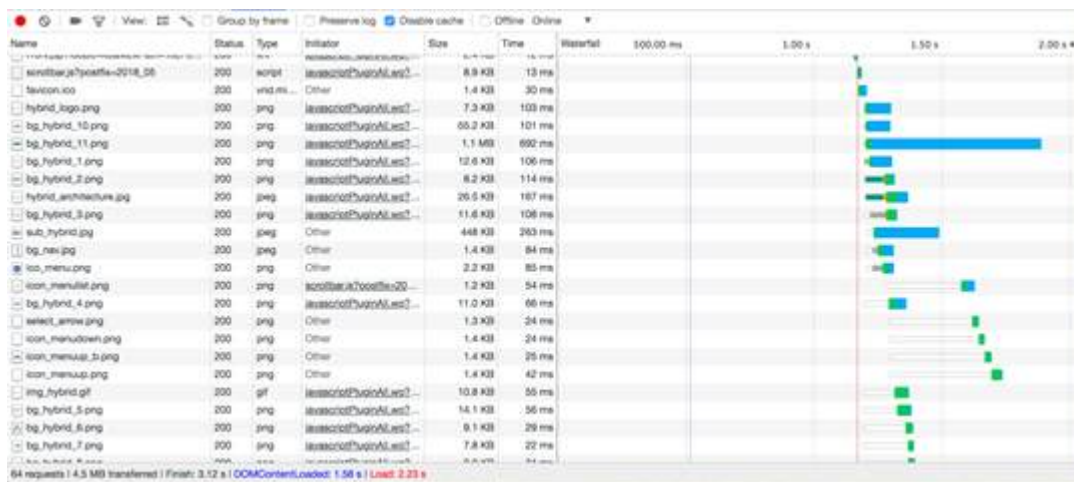
Table 34-5 각 단계 별 설명

단계	설명
Queuing	- 요청이 큐에 저장될 경우, 다음을 의미합니다. - 랜더링 엔진이 요청을 연기한 것으로 이는 이 요청이 중요 리소스 (예: 스크립트/스타일)보다 우선순위가 낮다고 간주됨. 주로 이미지 파일에서 자주 발생. - 곧 사용 가능한 TCP 소켓을 기다리도록 보류되었음. - 브라우저가 HTTP 1에서 각 출처마다 6개의 TCP 연결만 허용하기 때문에 요청이 보류됨. - 디스크 캐시 항목을 만드는데 걸린 시간. (대부분의 경우 매우 빠름.)
Stalled/Blocking	- 요청을 전송할 수 있게 되기까지 기다리는 데 소비한 시간. 발생 원인은 위의 Queueing 원인 중 하나가 될 수 있음. - 또한 이 시간은 프록시 협상에 소비한 시간도 모두 포함.
Proxy Negotiation	프록시 서버 연결과 협상하는데 소비한 시간.
DNS Lookup	- DNS 검색을 수행하는데 소비한 시간. - 페이지의 새 도메인마다 DNS 검색을 수행하는데 한번의 전체 왕복이 필요함.
Initial Connection/Connecting	- 연결을 설정하는 데 소요된 시간. - TCP 핸드셰이크/재시도와 SSL 협상 등이 포함됨.
SSL	SSL 핸드셰이크를 완료하는데 소요된 시간.
Request Sent/Sending	- 네트워크 요청을 발급하는데 걸린 시간. - 일반적으로 밀리초의 극히 짧은 시간.
Waiting (TTFB)	최초 응답 대기 시간. TTFB(Time To FirstByte)라고도 함.

단계	설명
	(서버까지 왕복에 소요된 시간) + (서버 응답을 기다린 시간)
Content Download/Downloading	응답 데이터 수신에 소요된 시간.

34.4.1.2. 네트워크 문제 진단

34.4.1.2.1. Queue에 저장되거나 지연된 시리즈



- 확인된 가장 일반적인 문제는 큐에 저장되거나 지연된 일련의 항목으로 이런 현상이 나타나면 단일 클라이언트에서 너무 많은 리소스를 가져오고 있다는 의미입니다.
- HTTP 1.0/1.1 연결의 경우, Chrome은 호스트당 최대 6개의 TCP 연결을 적용합니다. 한번에 12개의 항목을 요청하는 경우, 첫 6개부터 먼저 시작되고 나머지 절반은 큐에 저장됩니다.
- 첫 절반 중 하나가 완료되면 큐에 대개중인 첫 번째 항목이 요청 프로세스를 시작합니다.

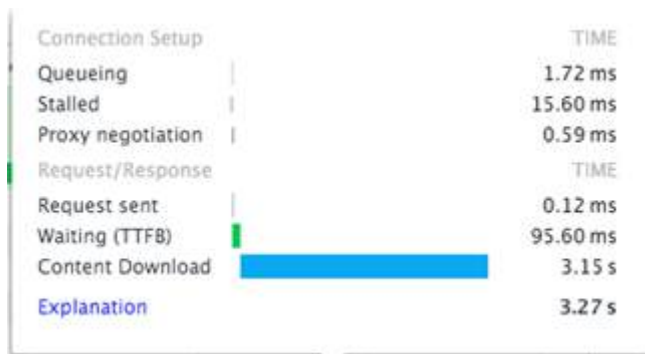
34.4.1.2.2. 느린 TTFB



느린 TTFB는 긴 대기 시간 즉 녹색이 많이 나타나는 경우입니다. 이 시간은 200ms 미만이 좋습니다. 이 시간이 길다면, 그 원인은 아래와 같습니다.

1. 클라이언트와 서버 사이의 네트워크 상태가 잘못된 경우
2. 서버 애플리케이션의 응답이 느린 경우

34.4.1.2.3. 처리 용량 확보



34.4.1.3. 해결 방안

Table 34-6 해결 방안

해결 방안	설명
네트워크 요청 횟수 튜닝	- js: 공통으로 제공되는 js 파일이 여러개로 분리된 경우, 하나의 파일로 합쳐서 호출. minify하여 js의 용량을 줄이는 것도 좋은 방안임. - CSS Sprites: 아이콘등 여러 이미지를 개별로 요청하지 않고 한 이미지 파일로 통합한 후 Background로 Position을 잡아주어 한 이미지만 로딩하면 트래픽 절약이 가능.
브라우저 기본 폰트 사용	웹폰트를 사용하는 경우 웹폰트 리소스 다운로드를 위한 트래픽과 폰트 사용을 위한 약간의 지연이 브라우저에서 발생 가능. 반드시 사용해야 한다면, 용량이 적은 웹폰트를 권장.
시스템 사용을 위한 외부 프레임워크 연계	- 시스템 사용 시 외부 프레임워크는 초기 화면이 아닌 필요한 시점에 로드하는 것을 권장. 로딩 부하를 줄일 수 있음. (예) 보안, 인증, 레포트
단일 frame 사용	- 화면 구성 프레임(frameset, iframe)은 하나로 처리하는 것을 권장. - 각 프레임은 각각의 윈도우 객체를 소유하고 웹스퀘어 엔진이나 공통 JS와 같은 리소스를 요청.

34.4.2. jQuery 셀렉터 사용

34.4.2.1. id 셀렉터

34.4.2.1.1. 사용 예 (1)

Object에 지정된 id를 이용하여 검색하는 것이 가장 빠른 속도를 보장합니다.

Code 34-41 예제 1

```
1 <xf:group class="grp_shbox" id="">
2   <w2:anchor id="btn_shbox " class="btn_shbox_hide" outerDiv="false"
3     style="" ev:onclick="scwin.btn_shbox_hide onclick">
4     <xf:label><![CDATA[검색영역 숨기기]]></xf:label>
5   </w2:anchor>
6 </xf:group>
```

Anchor 컴포넌트 접근 방법

`".grp_shbox. btn_shbox_hide "`

`".btn_shbox_hide "`

`"#btn_shbox "`

`"a.btn_shbox_hide "`

가장 빨리 접근하는 방법: `"#btn_shbox "`

34.4.2.1.2. 사용 예 (2)

아래와 같이 컴포넌트에 id 속성이 없는 경우에는 가장 가까운 상위 컴포넌트의 id 속성으로부터 자식 셀렉터를 이용하여 `"#link_area .link_text "`로 지정하는 방법이 더 빠릅니다.

Code 34-42 예제 2

```
1 <xf:group id="link_area" class="btn_shbox" tagName="ul">
2   <xf:group class="" tagName="li">
3     <w2:anchor class="link_text" outerDiv="false" style="">
4     <xf:label><![CDATA[고객센터]]></xf:label>
5     </w2:anchor>
6   </xf:group>
7 </xf:group>
```

34.4.2.2. 태그 셀렉터

클래스 셀렉터는 jquery 셀렉터 중 검색 시간이 가장 많이 소요됩니다. 태그 명을 사용하거나 클래스로 셀렉터를 지정해야 하는 경우, 태그 명 추가를 권장합니다.

Code 34-43 예제

```

1 <xf:group id="link_area" class="btn_shbox" tagName="ul">
2   <xf:group class="" tagName="li">
3     <w2:anchor class="link_text" outerDiv="false" style="">
4       <xf:label><![CDATA[고객센터]]></xf:label>
5     </w2:anchor>
6   </xf:group>
7 </xf:group>

```

위의 예제에서 “#link_area .link_text”보다 “#link_area a”가 더 빠릅니다. 따라서 “#link_area a.link_text”와 같이 태그 명을 추가하는 것이 좋습니다.

34.4.2.3. 메서드 체인

동일한 Object에 여러 개의 작업을 실행하는 경우 검색된 결과에 여러 개의 메소드를 붙여서 순차적으로 호출하는 “메서드 체인”을 사용하십시오. 태그를 다시 검색하는 과정을 생략할 수 있어 상당한 속도 개선을 기대할 수 있습니다.

Code 34-44 예제

```

1 $("#link_area").css("color", "red").slideUp(2000).slideDown(2000);

```

34.4.2.4. 셀렉터 캐싱

메서드 체인을 이용할 수 없는 경우, 셀렉터 검색 결과를 캐싱(변수에 저장 후 재사용)하여 재검색을 생략하고 그 결과 처리 속도를 올릴 수 있습니다. 아래는 검색 결과를 “linkText”에 저장하여 사용하는 예제입니다.

Code 34-45 예제

```

1 var linkText = $("#link_area a.link_text");
2 linkText.css("...");
3   ...중략...
4 linkText.toggle("...");
5   ...중략...
6 linkText.click("...");

```

34.4.2.5. 셀렉터 범위 지정

아래 예제와 같이 셀렉터의 검색 범위를 지정하여 검색 속도를 개선할 수도 있습니다.

Code 34-46 예제 (1)

```

1 $("#link_area li a.link_text").css("...");
2     ...중략...
3 $("#link_area li span").toggle("...");
4     ...중략...
5 $("#link_area li a").click("...");

```

아래는 공통 셀렉터인 `$("#link_area li")`를 변수 `"linkText"`에 저장하여 셀렉터 뒤에 검색대상인 `linkText`를 지정하여 변수의 범위 내에서 검색하는 예제입니다. HTML 전체에서 검색하는 보통의 지정 방법과 비교하여 검색 속도를 개선할 수 있습니다.

Code 34-47 예제 (2)

```

1 var linkText = $("#link_area li");
2 $("a.link_text", linkText).css("...");
3     ...중략...
4 $("span", linkText).toggle("...");
5     ...중략...
6 $("a", linkText).click("...");

```

34.4.3. 재귀 호출과 반복문 튜닝

재귀 호출형 함수는 호출할 때 마다 스택 메모리가 생성되며, 이는 재귀 조건이 달성된 후에야 제거됩니다. 그 결과 메모리가 계속 쌓이는 문제가 있습니다. 재귀로 호출된 함수가 많은 경우 스택 메모리 한계로 인해 스택오버플로우(stack overflow) 및 브라우저 멈춤현상이 발생합니다. 일반적인 반복문은 스택에 쌓이지 않지만 일정 시간 후에는 스크립트 수행 속도가 점점 떨어지고 스크립트 타임아웃(script timeout) 현상이 발생할 수 있습니다. 특히 모바일 브라우저는 5초 이내에 스크립트 타임아웃이 발생하기 때문에 무거운 로직을 반복문으로 처리하는 것은 자제해야 합니다.

34.4.3.1. DataCollection 제어

34.4.3.1.1. 반복적인 DataCollection 변경

DataCollection의 데이터 변경은 이벤트 발생으로 이어지며, 이는 곧 (ref로 연결된 Component 등) 관련된 객체들에 대해서도 추가 작업이 필요함을 의미합니다. 따라서 셀단위로 데이터를 반복 설정하는 것보다 데이터를 한번에 설정할 것을 권장합니다.

Code 34-48 비권장 사용 예

```

1 scwin.setBasicData = function() {
2
3     // ...

```

```

4
5     dataCollection1.setCellData("name","WebSquare");
6     dataCollection1.setCellData("address","서울시 구로구");
7     dataCollection1.setCellData("telephone","01011112222");
8 }

```

Code 34-49 일반 사용 예

```

1  scwin.setBasicData = function() {
2      var basicData = {};
3      basicData["name"] = "WebSquare";
4      basicData["address"] = "서울시 구로구";
5      basicData["telephone"] = "01011112222";
6
7      dataCollection1.setJSON(basicData);
8  }

```

Code 34-50 권장 사용 예

```

1  scwin.setBasicData = function() {
2      var basicData = {
3          "name":"WebSquare",
4          "address":"서울시 구로구",
5          "telephone":"01011112222"
6      };
7      dataCollection1.setJSON(basicData);
8  }

```

34.4.3.1.2. 여러행 추가시 Array를 이용하여 처리

여러 행을 추가하는 경우 Array를 만들어 Data를 추가한 후 한번에 DataCollection에 추가하는 것이 가능합니다.

Code 34-51 비권장 사용 예

```

1  scwin.addBodyLink = function(ds_obj){
2      var lv_ARow = ds_obj.insertRow();
3      ds_obj.setCellData(lv_ARow, "link_nm", "주민등록증 진위확인/잠김해제");
4      ds_obj.setCellData(lv_ARow, "link_url",
5          "http://www.egov.go.kr/main?a=AA090UserJuminIsCertApp");
6      ds_obj.setCellData(lv_ARow, "add_gubun", 1);
7
8      lv_ARow = ds_obj.insertRow();
9      ds_obj.setCellData(lv_ARow, "link_nm", "운전면허증 진위여부 조회");
10     ds_obj.setCellData(lv_ARow, "link_url",
11         "http://dls.koroad.or.kr/jsp/ool/olq/OL_LcnsTruthYnRtvV.jsp");

```

```

12  ds_obj.setCellData(lv_ARow, "add_gubun", 1);
13
14  lv_ARow = ds_obj.insertRow();
15  ds_obj.setCellData(lv_ARow, "link_nm", "외국인등록증 진위 확인");
16  ds_obj.setCellData(lv_ARow, "link_url",
17  "http://www.hikorea.go.kr/pt/FrnRegIdChkPopupR_kr.pt");
18  ds_obj.setCellData(lv_ARow, "add_gubun", 1);
19
20  // ...
21
22  ds_obj.setJSON(linkData, true);

```

Code 34-52 권장 사용 예

```

1  scwin.addBodyLink = function(ds_obj){
2      var linkData =
3          [{"link_nm" : "주민등록증 진위확인/잠김해제 ",
4            "link_url" : "http://www.egov.go.kr/main?a=AA090UserJuminIsCertApp ",
5            "add_gubun" : " 1 " },
6          {"link_nm" : "운전면허증 진위여부 조회 ",
7            "link_url" : "http://dls.koroad.or.kr/jsp//0L_LcnsTruthYnRtvV.jsp ",
8            "add_gubun" : " 1 " },
9          // ...
10 };
11 ds_obj.setJSON(linkData, true);

```

34.4.3.1.3. 반복문 내에 사용시 이벤트 중지 후 처리

반복문 내에서 건당 작업이 필요한 경우, DataCollection 변경 시 발생하는 이벤트를 중지하고 변경이 완료된 후 이벤트가 발생하도록 처리하십시오.

Code 34-53 사용 예

```

1  //broadcast 비활성화에 의해 dataList를 변경해도 연동된 컴포넌트 UI는 갱신되지 않음.
2  $p.data.setBroadcast(false);
3      for(var i = 0; i < 10; i++){
4          dataList1.insertRow(i);
5      };
6      for(var i = 0; i < 10; i++){
7          dataList2.insertRow(i);
8      };
9
10 //dataList1, dataList2의 최종데이터로 연동된 컴포넌트 ui가 갱신된다.

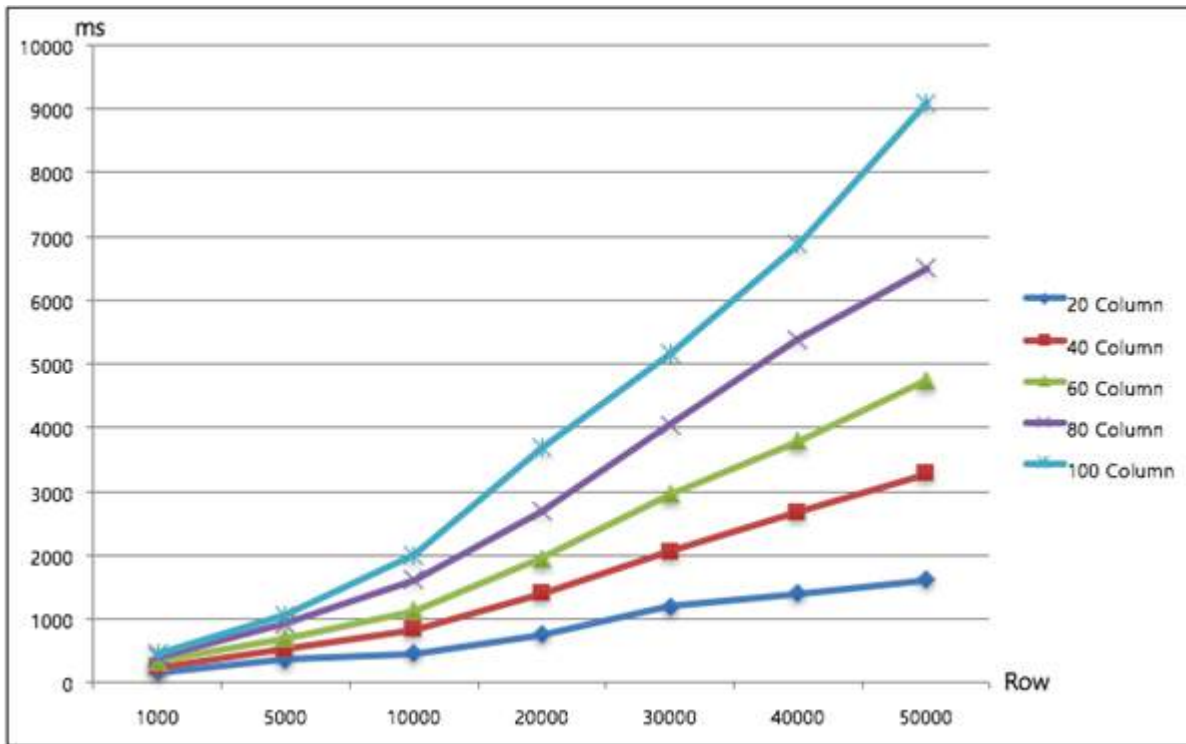
```

```
11 $p.data.setBroadcast(true);
```

34.4.3.2. GridView 튜닝

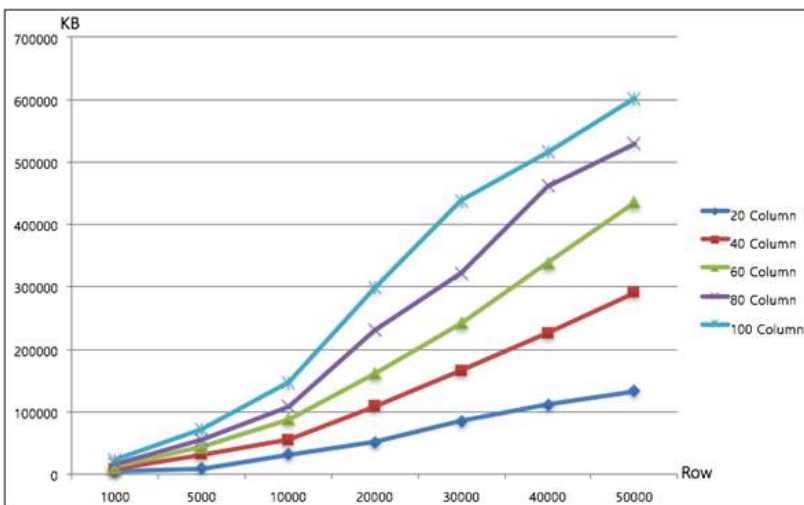
- GridView 컴포넌트는 많은 양의 데이터를 DataCollection에 저장하고 이를 화면에 Table 형식으로 표시합니다. 이를 위해 각 셀에 지정된 타입에 따라 스트립트 수행 및 렌더링이 반복적으로 발생합니다. 데이터 요청부터 GridView에 데이터가 렌더링되는 시간은 셀이 많은 수록 증가하고 증가폭도 늘어납니다.

그림 34-12 전체 렌더링 소요 시간 (통신 + 렌더링)



메모리의 사용량 역시 셀이 많을수록 증가폭이 늘어납니다.

그림 34-13 메모리 사용량

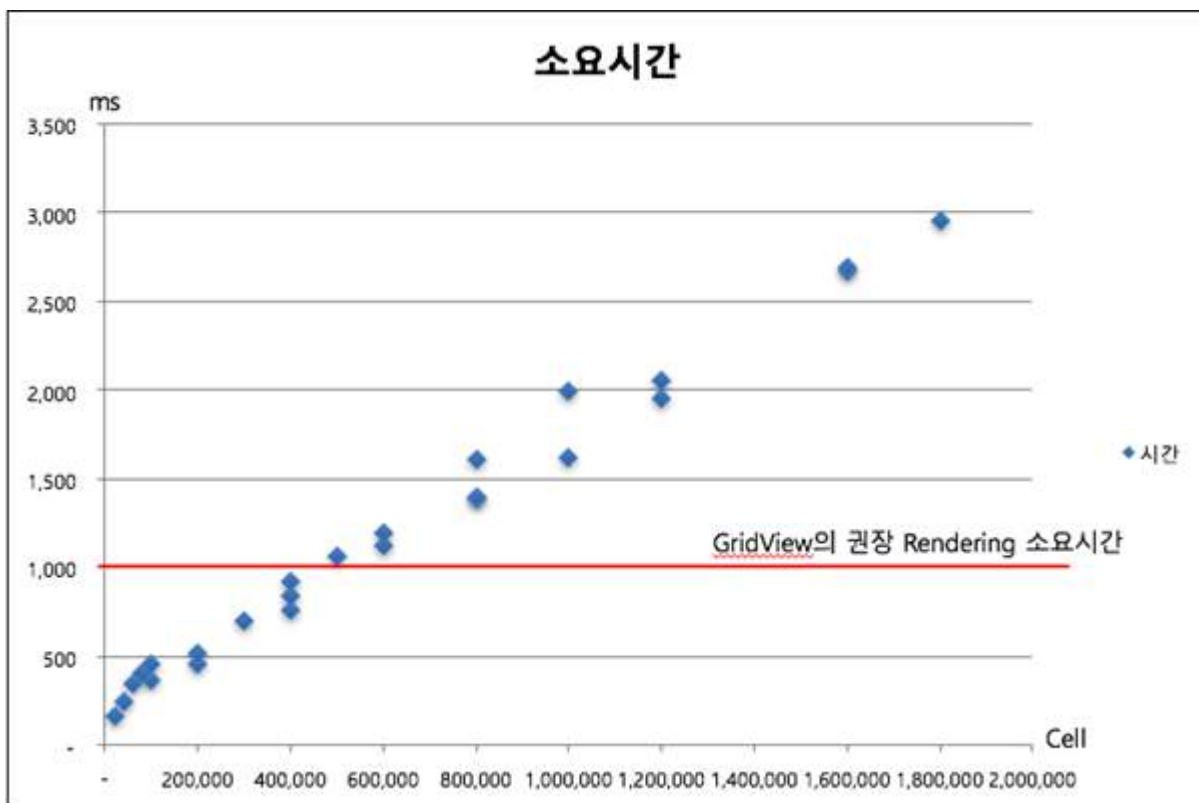


아래는 1초 내 메모리 증가분은 100M를 기준으로 허용하는 것입니다. 아래 테이블에 따라, 약 400,000 Cell을 기준으로 제한하는 것을 권장합니다.

구분	20 Column	40 Column	60 Column	80 Column	100 Column
1,000 Row	20,000 Cell ★ ○ ● ■	40,000 Cell ★ ○ ● ■	60,000 Cell ★ ○ ● ■	80,000 Cell ★ ○ ● ■	100,000 Cell ★ ○ ● ■
5,000 Row	100,000 Cell ★ ○ ● ■	200,000 Cell ★ ○ ● ■	300,000 Cell ★ ○ ● ■	400,000 Cell ★ ○ ● ■	500,000 Cell ○ ● ■
10,000 Row	200,000 Cell ★ ○ ● ■	400,000 Cell ★ ○ ● ■	600,000 Cell ○ ● ■	800,000 Cell ○ ●	1,000,000 Cell ○
20,000 Row	400,000 Cell ★ ○ ● ■	800,000 Cell ○ ●	1,200,000 Cell ○	1,600,000 Cell	2,000,000 Cell
30,000 Row	600,000 Cell ○ ● ■	120,000 Cell ○	1,800,000 Cell	2,400,000 Cell	3,000,000 Cell
40,000 Row	800,000 Cell ○ ●	160,000 Cell ○	2,400,000 Cell	3,200,000 Cell	4,000,000 Cell
50,000 Row	1,000,000 Cell ○	200,000 Cell	3,000,000 Cell	4,000,000 Cell	5,000,000 Cell

★ 전체 소요시간 ○ 네트워크 속도 ● Rendering 속도 ■ 메모리 증가분(100M)

그림 34-14 권장 렌더링 소요 시간



34.4.4. 성능 개선을 위한 추가 기능

34.4.4.1. SPA

WebSquare 엔진, 공통 CSS, 공통 JavaScript 등 여러 화면에서 공통으로 사용되는 모듈과 화면 소스의 Lifecycle을 분리하여 화면 로딩 및 전환 속도를 높이는 기능입니다. WindowContainer 및 TabControl은 WebSquare 엔진, 공통 CSS, 공통 JavaScript 등 여러 화면에서 공통으로 사용되는 모듈을 미리 로딩하여 화면 로딩 및 전환 속도를 높일 수 있습니다.

34.4.4.2. WFrame

IFrame은 분리된 전역메모리 영역(window)을 사용하여 개발은 용이하지만 메모리 및 성능이 떨어질 수 있습니다. 이를 보완하기 위해 WebSquare5 SP2는 WFrame을 제공합니다. 개발 생산성 향상과 메모리 및 성능 향상을 위해 WFrame들은 전역메모리 영역(window)을 공유하지만 각 WFrame은 분리된 지역변수를 참조합니다. ([내부 링크가 잘못되었습니다.](#) 참조.)

34.5 성능 저하 원인 및 해결 방안

본 단락은 성능 저하가 발생할 경우 그 원인 및 해결 방안을 분류하여 정리한 것입니다.

34.5.1. 웹 서버 설정

34.5.1.1. JSON, JS, XML, CSS 다운로드 속도

원인	해결 방안
gzip 설정이 빠져 있는 경우 주로 발생. (모바일 환경에서는 속도에 더 큰 영향을 줌.)	웹서버 설정 변경. (gzip 설정 추가.)

34.5.1.2. 정적인 자원에 대한 Caching 설정 오류

원인	해결 방안
JS 및 이미지 파일에 대한 캐싱 오류.	- 웹서버 설정 변경 - etag 설정 추가 - lastmodified date 설정 추가

34.5.1.3. SSL Handshake

원인	해결 방안
Keepalive가 설정되지 않은 경우 모든 요청시 SSL Handshake가 수행되어 속도 저하.	- 웹서버 설정에 변경 - SSL Keep Alive 설정 추가

34.5.2. WebSquare 설정

34.5.2.1. Cache 설정 및 압축 설정

원인	해결 방안
- 웹스퀘어 엔진 압축 미적용 으로 인한 성능 저하 - 웹스퀘어 XML 파일 캐싱 미적용 으로 인한 성능 저하	<code><engineCache compression="true" ... /></code> 설정 적용 <code><environment ... cache="cache" ... /></code> 설정 적용

34.5.2.2. engineType 설정

원인	해결 방안
개발용 웹스퀘어 엔진(engineType="5") 사용	- 웹스퀘어 설정 파일에서 운영환경에 적절한 웹스퀘어 엔진(engineType="6")을 사용 JS크기를 최소화 하여 성능 향상을 할 수 있음. engineType 참조.

Code 34-54 WebSquare engineType 설정 (예)

```

1 <!-- 파일명 : {websquare_home}/config/websquare.xml -->
2 <!-- 웹스퀘어 엔진의 난독화 및 압축 수준을 설정하는데 사용 개발환경에는 5, 운영환경에는 6을 권장 -->
3 <engineType value="6"/

```

그림 34-15 <engineCache compression="true" ... />

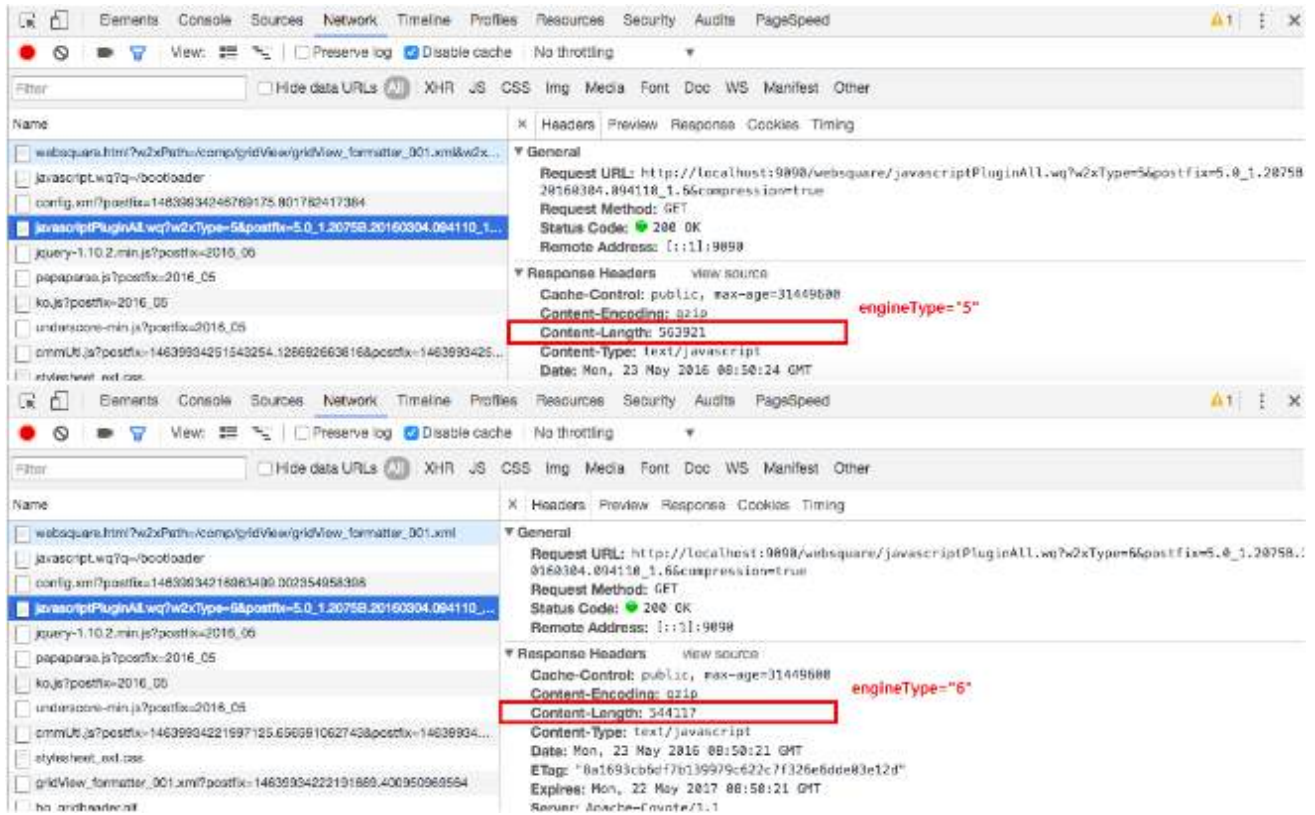
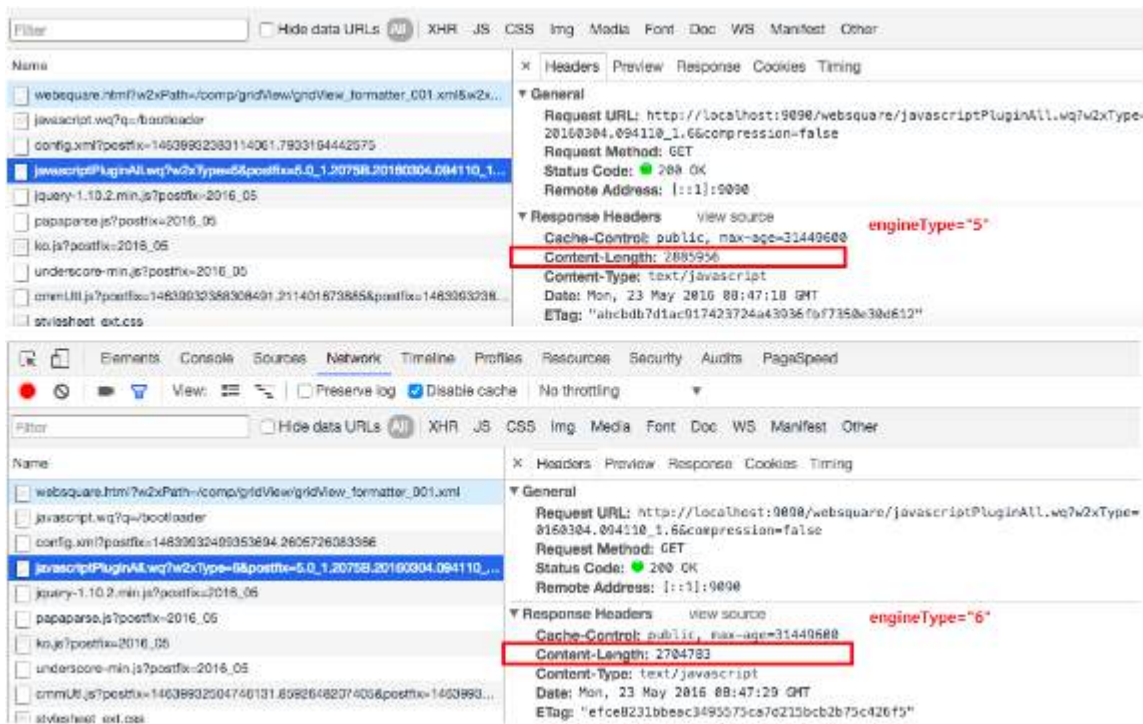


그림 34-16 <engineCache compression="false" ... />



34.5.3. 통신 최적화

34.5.3.1. 화면 프리징

원인	해결 방안
synchronous 방식의 HTTP Request 사용시 화면 프리징이 발생	업무 프로그램 튜닝

34.5.3.2. HTTP 오류 발생 리소스

원인	해결 방안
404, 500과 같은 HTTP 오류가 발생하면 화면 처리 속도가 저하됨 - 원인 : 오류 처리 및 리소스가 캐싱	업무 프로그램 수정

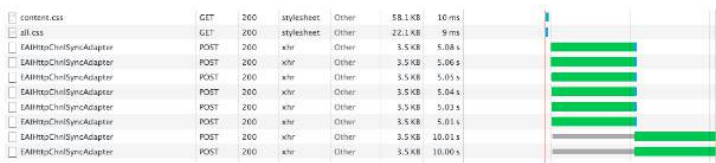
34.5.3.3. HTTP 1.1 Max Connection 초과시

원인	해결 방안
브라우저별 동일한 도메인으로 맺을수 있는 최대 Connection 수를 초과	업무 프로그램 튜닝

Table 34-7 브라우저 별 동일한 도메인으로 맺을수 있는 최대 Connection 수

브라우저	호스트 당 최대 병렬 연결 개수
IE 6 and 7	2
IE 8	6
IE 9	6
IE 10	8
Firefox 2	2
Firefox 4 ~ 17	6
Opera 9.63	4
Opera 10	8
Opera 11 & 12	6
Chrome 1 & 2	6
Chrome 3	4
Chrome 4 ~ 23	6
Safari 3 & 4	4

그림 34-17 브라우저 별 동일한 도메인으로 맺을수 있는 최대 Connection 수



34.5.3.4. 동시에 다수의 HTTP 요청

원인	해결 방안
단일 처리용 서비스를 동시 처리하도록 업무를 변경하면서 발생. (최대 100건 이상의 HTTP 요청이 발생.)	업무 프로그램 튜닝

34.5.3.5. 다수의 JS 파일 요청

원인	해결 방안
공통 JS파일을 지나치게 세분화 하여 발생	JS 파일 통합 및 난독화 기능을 형상 관리에 추가



외부 자바스크립트 파일에 대한 HTTP 요청시 성능 부담이 증가합니다.

크기가 25kb인 파일 4개보다 100kb 파일 하나를 더 빨리 내려받기 때문에 외부 자바스크립트 파일 개수를 최소화할 것을 권장합니다.

34.5.4. 데이터 튜닝

34.5.4.1. XML 방식의 대량 데이터 사용

원인	해결 방안
IE에서 MSXML Parser가 없어서 발생	MSXML Parser 설치

원인	해결 방안
XML 파서의 속도 저하 및 메모리 과다 사용으로 발생	JSON 방식으로 변경

34.5.5. 리소스 중복 로딩

34.5.5.1. CSS, JS 중복 로딩 오류

원인	해결 방안
동일한 CSS, JS파일을 중복 로딩하여 속도 저하 발생	중복 CSS, JS 로딩 여부 검토

34.5.6. 화면 튜닝

34.5.6.1. TabControl이 여러 Tab을 포함

TabControl 컴포넌트는 여러 화면을 탭으로 구성할 때 사용하는 컴포넌트입니다.

각 탭에 표시하는 화면의 복잡도(컴포넌트의 수)를 바탕으로 브라우저가 그려야 하는 모든 컴포넌트를 고려해야 합니다.

원인	해결 방안
숨겨진 Tab 영역까지 한번에 Rendering	업무 프로그램 튜닝

alwaysDraw 속성은 아래와 같이 TabControl 컴포넌트가 포함하는 모든 탭의 컴포넌트의 렌더링 여부를 결정합니다.

Table 34-8 **alwaysDraw**

alwaysDraw="true"	alwaysDraw="false" (권장)
- TabControl 컴포넌트가 포함하는 모든 탭의 컴포넌트를 초기 화면 로딩 시 렌더링. - 탭 및 컴포넌트 개수에 따라 브라우저의 렌더링 부하가 증가.	- 초기 로딩 시, 첫번째 탭의 컴포넌트만 렌더링. - 사용자 이벤트 혹은 스크립트 이벤트로 각 탭이 선택되는 시점에 해당 탭의 화면을 렌더링.



alwaysDraw="false" 설정을 사용할 경우, 초기 렌더링 후 첫 번째 탭을 제외한 나머지 탭의 컴포넌트는 객체화가 되지 않습니다.

34.5.6.2. 화면 로딩 속도

원인	해결 방안
메뉴 클릭 또는 CTI inbound call시 화면 전환 속도	공통 및 업무 프로그램 튜닝

34.5.6.3. 외부 컴포넌트 수

원인	해결 방안
RD, Web Editor, Chart 등 리소스를 많이 사용하는 외부 컴포넌트를 다수 사용하는 경우	측정 후 외부 컴포넌트 사용을 적절하게 제한

34.5.6.4. 그리드 엑셀 업/다운로드

원인	해결 방안
대용량 엑셀 업/다운로드로 인한 서버 메모리 부족	측정 후 결과에 따른 적절한 메모리 사용량 계산

34.5.6.5. 그리드 컬럼 개수

원인	해결 방안
그리드 컬럼이 100개를 초과 하는 경우	환경에 맞게 적절한 수로 제한

34.5.6.6. 그리드 대량 데이터

원인	해결 방안
10만셀 이상의 데이터를 사용하는 경우	환경에 맞게 적절한 수로 제한

34.5.6.7. IE 하위 호환성

원인	해결 방안
IE 브라우저 하위 호환성을 유지하기 위하여 meta-tag로 구형 브라우저 설정 추가	하위 브라우저만 지원하는 외부 모듈 수정

34.5.6.8. 성능을 고려하지 않은 개발

원인	해결 방안
동시 처리가 가능한 API대신 단일 처리 API를 반복하여 호출하는 경우	업무 프로그램 튜닝

34.5.6.9. IFrame 개수

원인	해결 방안
지나치게 많은 IFrame 사용	IFrame 사용을 적절하게 제한

34.5.6.10. WFrame 개수

원인	해결 방안
WFrame을 너무 많이 분할 해서 로딩 성능 저하	WFrame 사용을 적절하게 제한

34.5.6.11. 화면당 컴포넌트 수

원인	해결 방안
화면에 너무 많은 컴포넌트를 사용하는 경우 성능 저하	측정 후 컴포넌트 개수를 적절하게 제한

34.5.7. IE 브라우저에서의 렌더링 속도

34.5.7.1. 속도 저하 원인

IE 브라우저에서 JavaScript 기반 파서(xmlsax.js, xmlw3cdom.js)를 사용할 경우, ActiveX 기반 내장 XML 파서와 비교하여 렌더링 속도가 최대 30배 저하됩니다.

IE 브라우저의 렌더링 속도 저하 원인

- (1) JavaScript 기반 파서를 사용하도록 **config.xml** 파일에 설정된 경우
- (2) IE 브라우저의 ActiveX 기반 내장 XML 파서를 사용하지만, 실행 시 오류가 발생한 경우

34.5.7.2. 확인 방법

현재 사용 중인 XML 파서는 WebSquare 로그를 통해 확인할 수 있습니다.

IE 브라우저 화면에서 Ctrl키 + 마우스 우클릭 후 **로그보기**를 선택하십시오.

그림 34-18 WebSquare 로그 보기

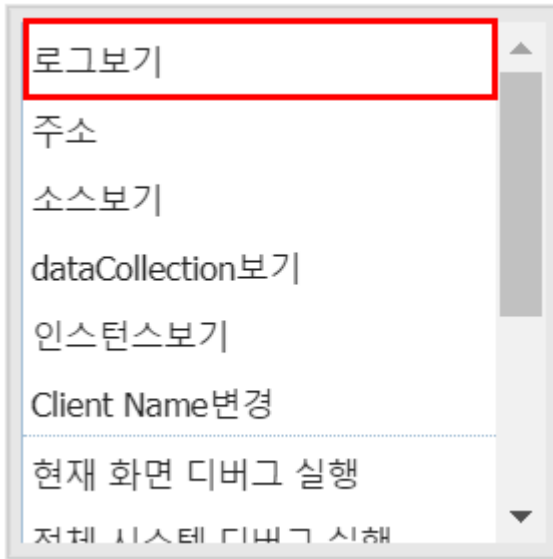


그림 34-19 권장 - IE11 브라우저 내장 XML 파서 사용

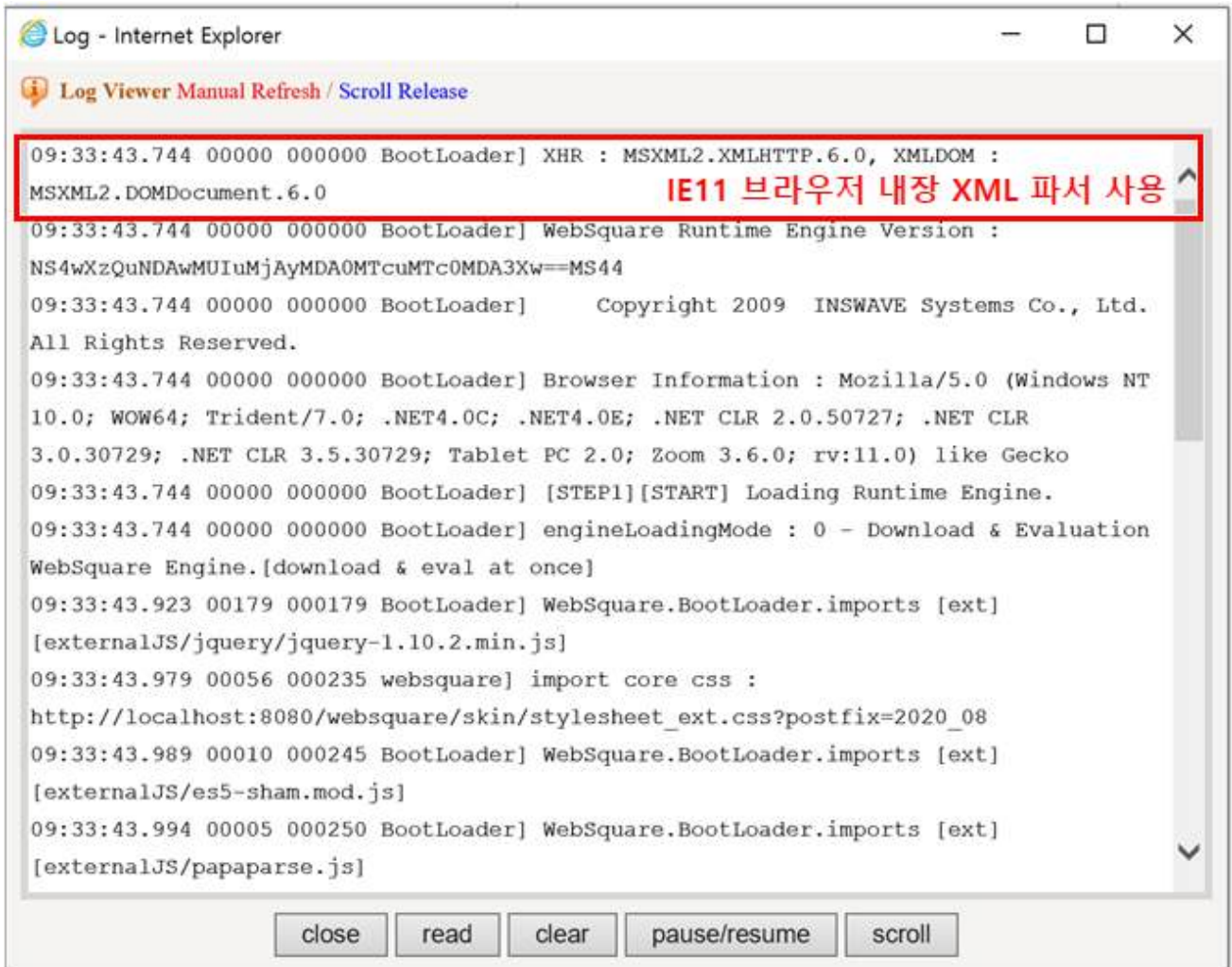
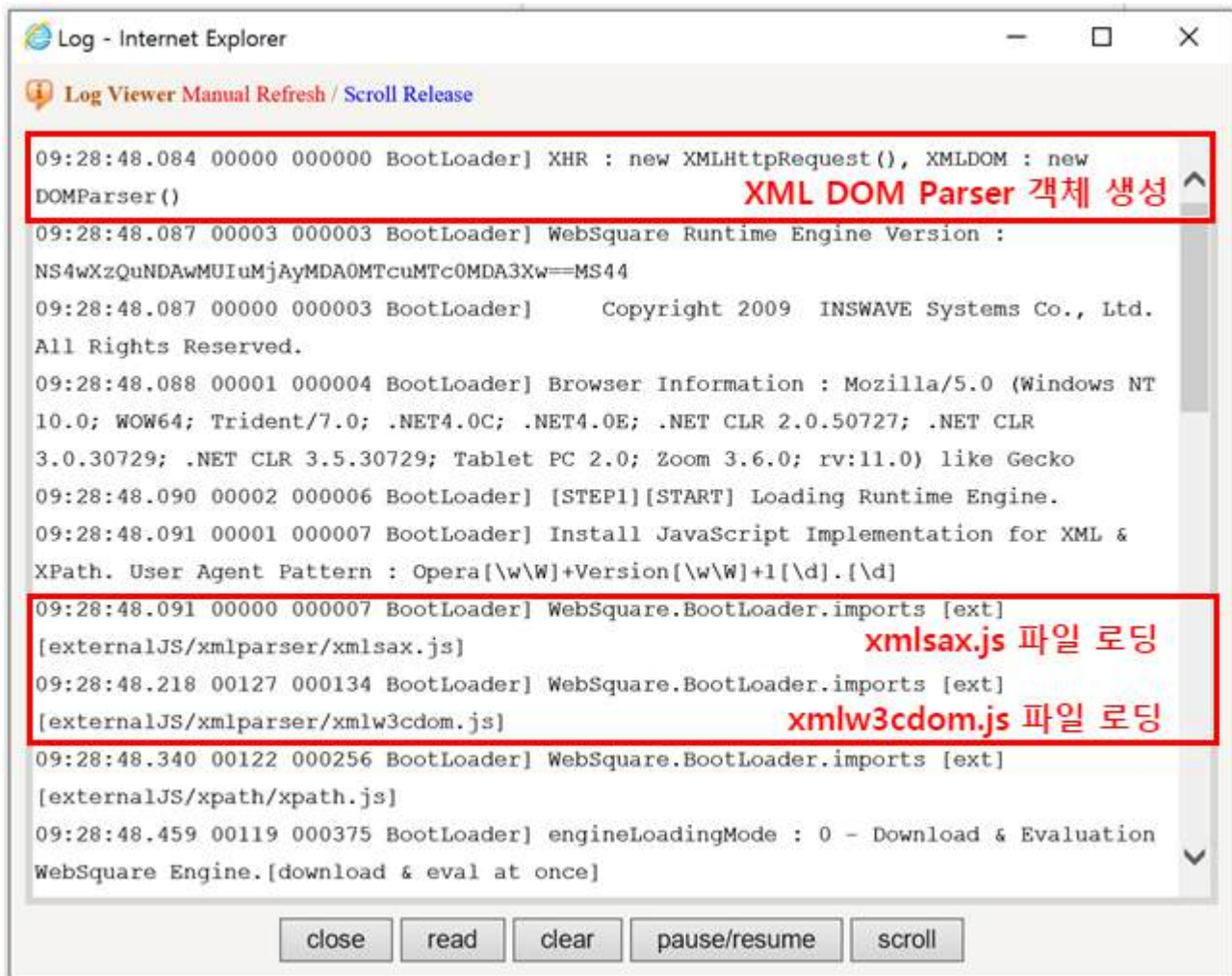


그림 34-20 비권장 - JavaScript 기반 파서 사용



34.5.7.3. 해결 방법

34.5.7.3.1. config.xml

config.xml 파일에서 `useActiveXObject` 설정을 확인합니다.

아래와 같이 `useActiveXObject="false"`인 경우, IE 브라우저는 JavaScript 기반 파서(xmlsax.js, xmlw3cdom.js)를 사용합니다.

Code 34-55 config.xml 설정 (외부 JS XML 파서를 사용)

```
1 <WebSquare>
2   <xhr useActiveXObject="false" useResponseXML="true"/>
3 </WebSquare>
```

`useActiveXObject` 설정을 `true`로 변경하거나 다음과 같이 해당 설정을 주석으로 처리하십시오.

Code 34-56 config.xml 설정 (useActiveXObject 설정 주석 처리)

```
1 <!-- <xhr useActiveXObject="true" useResponseXML="true"/> -->
```

34.5.7.3.2. 클라이언트

JavaScript 기반 파서(xmlsax.js, xmlw3cdom.js) 사용 설정이 **config.xml** 파일에 없어도, 일부 PC에서 렌더링 속도가 느릴 수 있습니다.

이는 주로 IE 브라우저의 ActiveX 기반 내장 XML 파서에 문제가 있는 경우입니다.

위의 비권장인 경우와 동일한 로그가 확인될 경우 IE 브라우저를 초기화하거나 재설치하십시오.

34.6 WebSquare 로그 분석

34.6.1. 로그 조회

화면에서 Ctrl + 마우스 오른쪽 버튼을 클릭 후, **[로그보기]**를 선택합니다.

그림 34-21 [Ctrl + 마우스 우클릭]

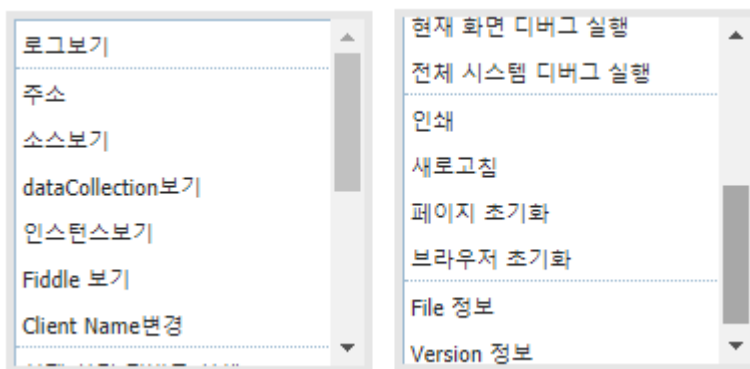
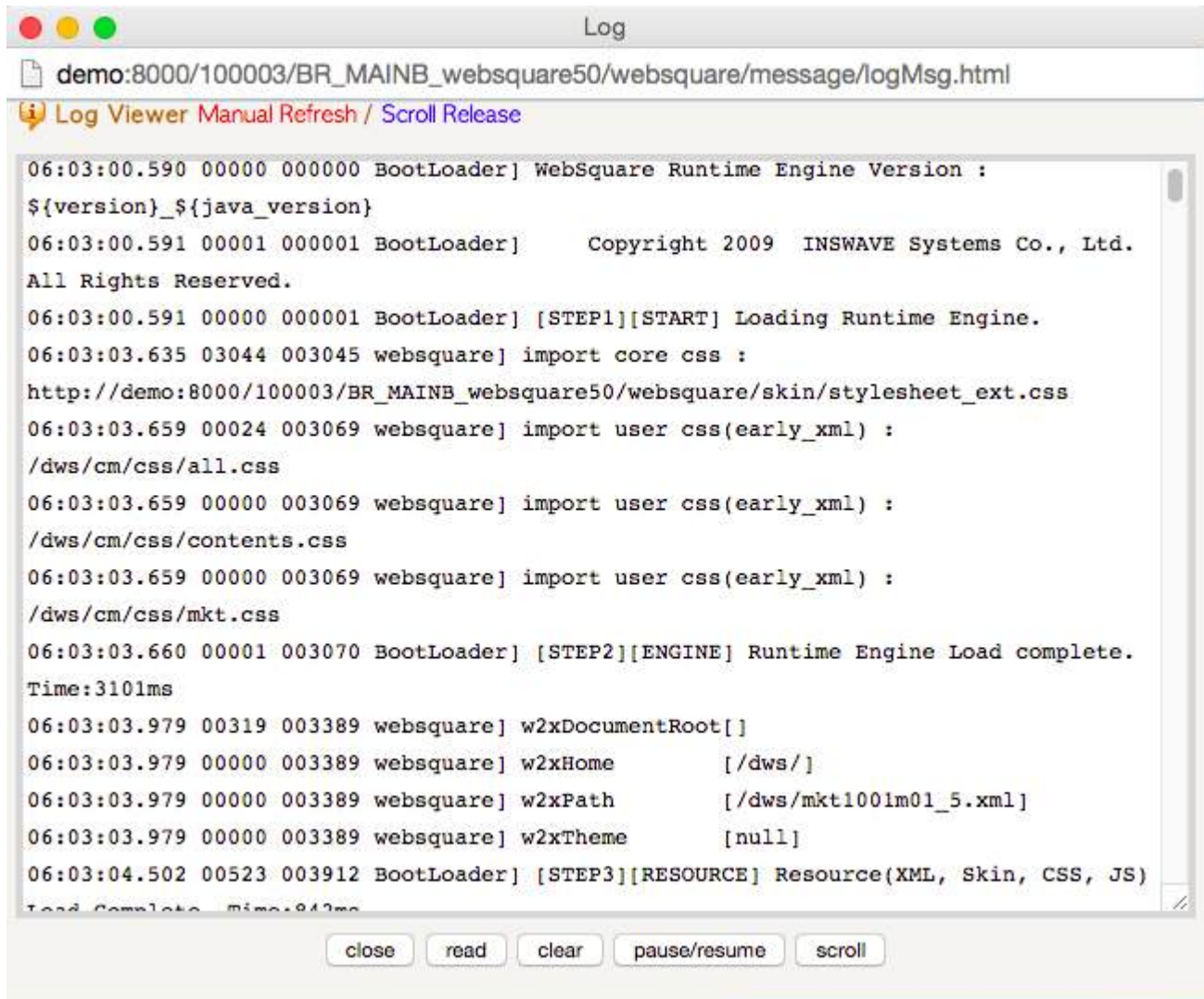


그림 34-22 웹스퀘어 로그 확인 창



Code 34-57 일반적인 로그 (예)

```

1 06:03:00.590 00000 000000 BootLoader] WebSquare Runtime Engine Version :
  ${version}_${java_version}
2
3 06:03:00.591 00001 000001 BootLoader] Copyright 2009 INSWAVE Systems Co., Ltd. All
  Rights Reserved.
4
5 06:03:00.591 00000 000001 BootLoader] [STEP1][START] Loading Runtime Engine.
6
7 06:03:03.635 03044 003045 websquare] import core css :
  http://demo:8000/100003/BR_MAINB_websquare50/websquare/skin/styleSheet_ext.css
8
9 06:03:03.659 00024 003069 websquare] import user css(early_xml) : /dws/cm/css/all.css
10
11 06:03:03.659 00000 003069 websquare] import user css(early_xml) : /dws/cm/css/contents.css
  
```

```

12
13 06:03:03.659 00000 003069 websquare] import user css(early_xml) : /dws/cm/css/mkt.css
14
15 06:03:03.660 00001 003070 BootLoader] [STEP2][ENGINE] Runtime Engine Load complete.
Time:3101ms
16
17 06:03:03.979 00319 003389 websquare] w2xDocumentRoot[ ]
18
19 06:03:03.979 00000 003389 websquare] w2xHome          [/dws/]
20
21 06:03:03.979 00000 003389 websquare] w2xPath          [/dws/mkt1001m01_5.xml]
22
23 06:03:03.979 00000 003389 websquare] w2xTheme          [null]
24
25 06:03:04.502 00523 003912 BootLoader] [STEP3][RESOURCE] Resource(XML, Skin, CSS, JS) Load
Complete. Time:842ms
26
27 06:03:05.838 01336 005248 BootLoader] [STEP4][CREATE] WebSquare Object Creation Complete.
Time:1336ms
28
29 06:03:05.839 00001 005249 websquare] [WebSquare.util.js]script
loading[script_ordered_async]
30
31 06:03:06.716 00877 006126 websquare] [WebSquare.util.js]script
loading[script_ordered_async]
32
33 06:03:06.719 00003 006129 BootLoader] controlCollection.toHTML - before
34
35 06:03:06.760 00041 006170 BootLoader] controlCollection.toHTML - after
36
37 06:03:06.764 00004 006174 BootLoader] [STEP5][RENDER] HTML Rendering Complete. Time:926ms
38
39 06:03:06.865 00101 006275 BootLoader] [STEP6][ADJUST] Pre Event Activation. Time:101ms
40
41 06:03:06.865 00000 006275 BootLoader] Asynchronous call process
42
43 06:03:07.772 00907 007182 BootLoader] [STEP7][EVENT] Event Activation. Time:907ms
44
45 06:03:07.821 00049 007231 websquare] WebSquare.layer.showProcessMessage [processing..]
46
47 06:03:07.824 00003 007234 websquare] /mkt/mkt1/mkt1001m01/selectBascInfo.do

```

```

48
49 06:03:07.826 00002 007236 websquare] /session/getSessionData.json
50
51 06:03:07.831 00005 007241 websquare] /session/getSessionData.json
52
53 06:03:07.832 00001 007242 websquare] /dws/view/com/data/dc_movePageMenuList.json
54
55 06:03:07.832 00000 007242 websquare] /memorydata/selectCustMenu.json
56
57 06:03:07.833 00001 007243 websquare] /dws/view/com/data/dc_menuList1.json
58
59 06:03:07.834 00001 007244 websquare] /dws/view/com/data/dc_menuList2.json
60
61 06:03:07.834 00000 007244 websquare] /dws/view/com/data/dc_menuList3.json
62
63 06:03:07.835 00001 007245 websquare] /dws/view/com/data/dc_menuList4.json
64
65 06:03:07.836 00001 007246 websquare] /session/getSessionData.json
66
67 06:03:07.839 00003 007249 BootLoader] [STEP8][EXECUTE] JS load and Global Script Execution
Completed. Time:67ms
68
69 06:03:07.839 00000 007249 BootLoader] [STEP SUMMARY] Initial procedures completed. Duration
: 7280, 시간 요약 : 3101 842 1336 926 101 907 67 7280
70
71 06:03:07.857 00018 007267 websquare] ajax complete. Status[200] async[true]
url[/session/getSessionData.json]
72
73 06:03:07.859 00002 007269 websquare] ajax complete. Status[200] async[true]
url[/mkt/mkt1/mkt1001m01/selectBascInfo.do]
74
75 06:03:07.995 00136 007405 websquare] ajax complete. Status[200] async[true]
url[/session/getSessionData.json]
76
77 06:03:07.996 00001 007406 websquare] ajax complete. Status[200] async[true]
url[/memorydata/selectCustMenu.json]
78
79 06:03:08.006 00010 007416 websquare] ajax complete. Status[200] async[true]
url[/dws/view/com/data/dc_movePageMenuList.json]
80
81 06:03:08.009 00003 007419 websquare] ajax complete. Status[200] async[true]

```

```

url[/dws/view/com/data/dc_menuList1.json]
82
83 06:03:08.026 00017 007436 websquare] ajax complete. Status[200] async[true]
url[/dws/view/com/data/dc_menuList2.json]
84
85 06:03:08.038 00012 007448 websquare] ajax complete. Status[200] async[true]
url[/dws/view/com/data/dc_menuList3.json]
86
87 06:03:08.044 00006 007454 websquare] ajax complete. Status[200] async[true]
url[/session/getSessionData.json]
88
89 06:03:08.045 00001 007455 websquare] ajax complete. Status[200] async[true]
url[/dws/view/com/data/dc_menuList4.json]
90
91 06:03:13.804 05759 013214 websquare] testFun.hogaList - 1
92
93 06:03:13.806 00002 013216 websquare] convertTypeB - 2 [{ "o_FluctAmt21": "-18236", "o_RemOffer
Vol21": "2", "hoga": "2", "o_RemOfferVol21_Color": "1", "hoga_Color": "1", "gubun": "5"}, { "o_FluctAmt21"
: "-39898", "o_RemOfferVol21": "2", "hoga": "2", "o_RemOfferVol21_Color": "1", "hoga_Color": "1", "gubun"
: "5"}, { "o_FluctAmt21": "-80343", "o_RemOfferVol21": "2", "hoga": "2", "o_RemOfferVol21_Color": "1", "ho
ga_Color": "1", "gubun": "3"}, { "o_FluctAmt21": "-133879", "o_RemOfferVol21": "2", "hoga": "2", "o_RemOff
erVol21_Color": "1", "hoga_Color": "1", "gubun": "3"}, { "o_FluctAmt21": "-91962", "o_RemOfferVol21": "2"
, "hoga": "2", "o_RemOfferVol21_Color": "1", "hoga_Color": "1", "gubun": "3"}, { "o_FluctAmt22": "-39607", "
hoga": "2", "o_RemBidVol22": "2", "o_RemBidVol22_Color": "4", "hoga_Color": "1", "gubun": "3"}, { "o_Fluct
Amt22": "-8555", "hoga": "2", "o_RemBidVol22": "2", "o_RemBidVol22_Color": "4", "hoga_Color": "1", "gubun"
: "3"}, { "o_FluctAmt22": "-7913", "hoga": "2", "o_RemBidVol22": "2", "o_RemBidVol22_Color": "4", "hoga_Co
lor": "1", "gubun": "3"}, { "o_FluctAmt22": "-27099", "hoga": "2", "o_RemBidVol22": "2", "o_RemBidVol22_Co
lor": "4", "hoga_Color": "1", "gubun": "5"}, { "o_FluctAmt22": "-24134", "hoga": "2", "o_RemBidVol22": "2", "
o_RemBidVol22_Color": "4", "hoga_Color": "1", "gubun": "5"}]
94
95 06:03:13.838 00032 013248 websquare] convertTypeB - 3
96
97 06:03:13.842 00004 013252 websquare] testFun.hogaList - 2
98
99 06:03:13.843 00001 013253 websquare] 39 1 starthogaList fps : 25.6

```

34.6.2. 로그 구조

`$p.log()` 함수를 이용한 로그는 다음과 같은 형태로 생성됩니다.

Code 34-58 로그 형태

```
1 시간 이전로그출력후경과시간 첫로그출력후경과시간 로그출력기] 로그메시지
```

`06:03:13.838 00032 013248 websquare] convertTypeB` 라는 로그는 아래와 같이 해석할 수 있습니다.

Table 34-9 로그 해석

No.	항목	설명
1	로그생성시간	06:03:13.838
2	이전로그출력 후 경과시간	32ms
3	첫로그출력 후 경과시간	13.248s
4	로그출력기	websquare
5	메시지	convertTypeB

34.6.2.1. Step 로그

Code 34-59 Step 로그 (예)

```
1 06:03:00.591 00000 000001 BootLoader] [STEP1][START] Loading Runtime Engine.
2
3 06:03:03.660 00001 003070 BootLoader] [STEP2][ENGINE] Runtime Engine Load complete.
Time:3101ms
4
5 06:03:04.502 00523 003912 BootLoader] [STEP3][RESOURCE] Resource(XML, Skin, CSS, JS) Load
Complete. Time:842ms
6
7 06:03:05.838 01336 005248 BootLoader] [STEP4][CREATE] WebSquare Object Creation Complete.
Time:1336ms
8
9 06:03:06.764 00004 006174 BootLoader] [STEP5][RENDER] HTML Rendering Complete. Time:926ms
10
11 06:03:06.865 00101 006275 BootLoader] [STEP6][ADJUST] Pre Event Activation. Time:101ms
12
13 06:03:07.772 00907 007182 BootLoader] [STEP7][EVENT] Event Activation. Time:907ms
14
15 06:03:07.839 00003 007249 BootLoader] [STEP8][EXECUTE] JS load and Global Script Execution
Completed. Time:67ms
```

```

16
17 06:03:07.839 000000 007249 BootLoader] [STEP SUMMARY] Initial procedures completed. Duration
: 7280, 시간 요약 : 3101 842 1336 926 101 907 67 7280

```

34.6.2.2. 오류 로그

Code 34-60 오류 로그 (예)

```

1 19:27:23.626 000000 007026 websquare] 작업 도중 오류가 발생하였습니다.
2
3 Exception Occured at element id [tradingGridView2]
4
5 * message      : Cannot read property 'getRealRowIndex' of undefined
6
7 * stack
8
9   TypeError: Cannot read property 'getRealRowIndex' of undefined
10
11   at WebSquare.uiplugin.gridViewApiController.getCellStyleClassName (eval at <anonymous>
(http://demo:8000/100003/BR_MAINB_websquare50/websquare/bootloader.js:267:28),
<anonymous>:3931:35)
12
13   at WebSquare.uiplugin.styleController.setCellStyle (eval at <anonymous>
(http://demo:8000/100003/BR_MAINB_websquare50/websquare/bootloader.js:267:28),
<anonymous>:395:32)
14
15   at Object.PageFunc.setCurrentBorder (<anonymous>:613:50)
16
17   at Object.PageFunc.convertTypeB (<anonymous>:852:26)
18
19   at Object.PageFunc.Realmkt1001m01_applyRTSData (<anonymous>:1003:34)
20
21   at Object.testFun.hogaList (<anonymous>:1701:30)
22
23   at Object.testFun.starthogaList (<anonymous>:1733:29)
24
25   at Object.testFun.testStart (<anonymous>:1713:29)
26
27   at <anonymous>:1:9
28
29 * caller

```



```

30
31   at WebSquare.uiplugin.gridViewApiController.prototype.getCellStyleClassName
32
33   at WebSquare.uiplugin.styleController.prototype.setCellStyle
34
35   at function ()
36
37   at function (data)
38
39   at function (data)
40
41   at l0fferVol_
42
43   at function ()
44
45   at function ()
46
47
48
49 <<Internal JavaScript Source Block [/dws/mkt1001m01_9.xml][0] Start>>
50
51 [/dws/mkt1001m01_9.xml:0001]
52
53 [/dws/mkt1001m01_9.xml:0002]
54
55 [/dws/mkt1001m01_9.xml:0003]
56
57 [/dws/mkt1001m01_9.xml:0004]           // page function object
58
59 [/dws/mkt1001m01_9.xml:0005]           FFFF = 0;
60
61 [/dws/mkt1001m01_9.xml:0006]           var PageFunc = $.extend({}, PageFunc );
62
63 [/dws/mkt1001m01_9.xml:0007]
64
65
66 ... 생략 ...
67
68
69 [/dws/mkt1001m01_9.xml:0611]           } else {
70

```

```

71 [/dws/mkt1001m01_9.xml:0612] tradingGridView.setCellStyle
(i-5 , 2 , 'border', '');
72
73 [/dws/mkt1001m01_9.xml:0613] tradingGridView2.setCellStyle(i
, 2 , 'border', '');
74
75 [/dws/mkt1001m01_9.xml:0614]
76
77 [/dws/mkt1001m01_9.xml:0615] if( i == 11) {
78
79 [/dws/mkt1001m01_9.xml:0616]
tradingGridView.setCellStyle (5 , 2 , 'border-bottom', '');
80
81 [/dws/mkt1001m01_9.xml:0617]
tradingGridView2.setCellStyle(10 , 2 , 'border-bottom', '');
82
83 [/dws/mkt1001m01_9.xml:0618] }
84
85 [/dws/mkt1001m01_9.xml:0619] }

```

34.7 점검 목록

성능 관련 문제가 발생할 경우 아래 내용을 확인하십시오.

34.7.1. 일반

34.7.1.1. JavaScript

번호	확인 사항	설명
1	JavaScript 함수 작성 시 지역 변수를 적절하게 사용했는가?	함수 내에서 변수 선언시 var 선언을 생략하면 전역변수로 생성되어 메모리 과다 점유 현상 발생.
2	JavaScript 로직 내에 오류가 발생하는가?	JavaScript 내 오류 발생시 화면 렌더링 속도 저하. 스크립트 수행이 중지되어 함수가 정상적으로 종료되지 않는 경우가 있는지 확인.
		하나의 XML 파일 내의 JavaScript 양이 많을 경우 렌더링 속도 저하.

번호	확인 사항	설명
3	XML 파일 내의 업무 로직 처리를 위한 JavaScript 분량이 적절한가?	- 권장: - 공통으로 사용되는 함수 등은 xml 파일 별로 넣지 않고 별도의 .js 파일로 분리하여 관리.
4	<code>offsetHeight()</code> 등 상대적으로 느린 기능의 과도한 사용하고 있지 않은가?	상대적으로 느린 기능의 과도한 사용을 지양.
5	반복적으로 발생하는 이벤트를 적절하게 처리하고 있는가?	onresize, onscroll 등의 일부 이벤트는 반복적으로 수십번 이상 이벤트 핸들러를 호출하므로 적절한 처리가 필요함.
6	부적절한 반복문은 없는가?	- 권장: - 반복과 관련이 없는 변수 등은 외부로 이동. - 반복문 내부에서 break, continue 문을 적절히 사용.
7	변수 및 함수가 SPA 방식에 맞게 선언되어 있는가?	올바르지 않은 변수 및 함수 선언은 메모리 누수를 초래 수 있음. - 잘못된 변수 선언 예시 <code>var status = "S";</code> <code>rowCount = dataList1.getRowCount();</code> <code>function updateData() { ...</code>

34.7.1.2. 통신

번호	확인 사항	설명
1	sync 및 async 통신 방식이 적절히 사용되었는가?	sync 방식은 완료시까지 스크립트 정지 현상이 발생하므로 async 방식을 권장.
2	통신 관련 공통 함수가 객체 대신 string을 사용했는가?	callback 함수에 객체 형태로 전달하면 메모리 누수 발생. - 예시 - IE8 브라우저 다운현상이 발생할 수 있음
3	통신 객체의 동적 생성시 중복 문제는 없는가?	동일한 ID로 중복하여 생성할 경우 메모리 누수 발생.
4	통신 함수의 응답 객체 사용에 문제가 없는가?	통신 함수에서 사용자 함수 호출 시 응답 결과가 다른 곳과 참조 관계가 없는지 확인.

34.7.1.3. 네트워크

번호	확인 사항	설명
1	Keep-Alive 관련 설정을 확인했는가?	Keep-Alive 기능의 활성화/비활성이 무조건 성능에 긍정적인 영향을 주지는 않음.

번호	확인 사항	설명
		반드시 해당 프로젝트의 여건에 따라서 서버 관리자와 협의 후 적용 여부를 결정할 것을 권장.
2	텍스트 형식(HTML, XML, CSS, JavaScript, JSON 등)의 자원을 gzip으로 압축해서 전송하고 있는가?	텍스트 형식의 자원을 gzip 압축 전송할 경우, 전송 데이터의 크기를 줄여 네트워크 속도를 개선할 수 있음.

34.7.1.4. CSS

번호	확인 사항	설명
1	웹 폰트 사용시 부모 페이지 외의 자식 페이지에서 웹 폰트를 중복 호출하지 않는가?	중복 호출 시 IE8 등에서 백화 현상 또는 웹 폰트 미적용 현상 발생.
2	IE8을 지원해야 하는 프로젝트에서 웹 폰트를 사용하는가?	IE8의 경우 웹 폰트를 내려 받기 위한 브라우저 처리 속도가 현저하게 느려지기 때문에 웹 폰트 사용을 권장하지 않음. - 예시 - ie8 + iframe + 웹 폰트 사용시 백화 현상이 발생할 수 있음
3	무거운 웹 폰트를 사용하는가?	IE에서 웹 폰트를 사용할 경우 브라우저 메모리 사용량이 크게 증가함. 특히 IE에서 웹 폰트를 다수의 IFrame 및 Window Popup과 병행하여 사용할 경우, 메모리의 과도한 사용과 누수가 발생하여 속도 저하, 백화 현상, 브라우저 비정상 종료 현상으로 이어질 수 있음. - 권장: - 무거운 웹 폰트의 사용은 지양할 것을 권장.
4	border-radius 속성을 과도하게 사용하지 않는가?	과도한 border-radius 속성의 사용은 구형 스마트폰 기기에서 문제를 발생시킬 수 있음.

34.7.1.5. jQuery

번호	확인 사항	설명
1	jQuery 1.7.2를 사용하는가?	해당 버전 사용시 메모리 누수 발생
2	jQuery 사용 시 \$.... 기능을 사용하고 있는가?	WebSquare에서 제공하는 최적화된 \$.p.\$.... 기능 사용을 권장.

34.7.1.6. 메모리 누수

번호	확인 사항	설명
1	메모리 누수를 야기할 수 있는 잠재적 요인들을 확인했는가?	대표적인 메모리 누수 요인을 확인. - Closure 내부 개체 참조로 인한 메모리 누수. - Top 영역에 무분별하게 데이터를 계속 저장하여 발생하는 메모리 누수. - 부적절한 jQuery 사용으로 개체가 해제되지 발생하는 메모리 누수.
2	JavaScript Exception으로 인한 메모리 누수는 없는가?	WebSquare 로그를 조회하여 JavaScript Exception이 발생하는 코드가 존재할 경우, 로직 보완 및 예외 처리가 필요함. Exception이 발생할 경우 화면 로딩 속도가 저하됨.
3	팝업으로 인자 전달 시에 객체 대신 string 을 사용했는가?	객체 형태로 전달시 메모리 누수 발생.
4	데이터 객체의 동적 생성시 중복 문제는 없는가?	동일한 ID로 중복하여 생성할 경우 누수 발생.
5	변수 및 함수가 SPA 방식에 맞게 선언되어 있는가?	올바르지 않은 변수 및 함수 선언은 메모리 누수를 초래 수 있음. - 잘못된 변수 선언 예시. <code>var status = "S";</code> <code>rowCount = dataList1.getRowCount();</code> <code>function updateData() { ...</code>
6	통신 관련 공통 함수가 객체 대신 string을 사용했는가?	callback 함수에 객체 형태로 전달하면 메모리 누수 발생. - 예시 - IE8 브라우저 다운현상이 발생할 수 있음
7	통신 객체의 동적 생성시 중복 문제는 없는가?	동일한 ID로 중복하여 생성할 경우 메모리 누수 발생.
8	jQuery 1.7.2를 사용하는가?	해당 버전 사용시 메모리 누수 발생
9	IFrame이 야기할 수 있는 메모리 누수 가능성을 확인했는가?	대표적인 메모리 누수 원인을 확인. - IFrame 사이에서 개체 참조 시 - IFrame 사이에서 데이터 전달 시 - IFrame 사이에서 함수 호출 시 인자로 개체를 전달하는 경우 - 다른 IFrame 영역의 DOM에 동적으로 이벤트를 생성할 경우
10	데이터 객체의 동적 생성시 중복 문제는 없는가?	동일한 ID로 중복하여 생성할 경우 메모리 누수 발생.
11	외부 모듈 연계 시 해당 외부 모듈의 메모리 누수 여부를 확인했는가?	브라우저 전체 메모리에 영향을 미침.
12	Fasoo DRM 모듈을 사용하는가?	Fasoo DRM Client for LIG NoNX Wix, FSW Client 류의 제품들이 메모리 사용시 팝업을 열고 닫을 때 메모리 사용량이 증가할 수 있음.
13	외부 라이브러리를 점검했는가?	- Import된 외부 라이브러리의 메모리 점유량 확인. - IFrame, Window.Popup, SPA 방식이 메모리 누수를 야기하는지 확인.

번호	확인 사항	설명
		인.

34.7.1.7. 기타

번호	확인 사항	설명
1	공통코드, 공통 메시지 등 업무 공통 데이터가 시스템 로딩 초기에 일괄적으로 메모리에 적재되는가?	소규모 프로젝트에서는 이런 방식의 데이터 적재 패턴이 별다른 문제를 야기하지 않을 수도 있음. 단, 중대형 프로젝트 이상에서는 심각한 문제의 원인이 될 수 있으므로 설계자와 협의하여 업무에 적합한 업무 공통 데이터 로딩 방식을 적용해야 함.
2	SSL 환경에서 멀티업로드(플래시 모드)를 사용하는가?	업로드 성능이 저하될 수 있음. • 권장: ◦ html5_transparent 모드를 권장. (IE 10 이상만 사용 가능)
3	캐시가 적절히 사용되고 있는지 확인했는가? (웹스퀘어 환경설정, 웹 서버 만료일 설정, 브라우저의 캐시 설정 등)	웹스퀘어 config.xml 내의 캐시 관련 설정(웹서버 만료일, 브라우저 캐시 설정 등 관련)이 웹페이지 속도에 큰 영향을 미침. 개발/운영 환경에 따라 웹스퀘어 <i>config.xml</i> 내의 캐시 관련 설정이 적용되어 있는지와 서버 설정을 확인. • 예시 ◦ 웹스퀘어 환경설정, 웹 서버 만료일 설정, 브라우저의 캐시 설정 등
4	(갤럭시노트3, 갤럭시탭10.1과 같은) 특정 모바일 장비를 사용하는가?	해당 장비에서는 자체 xml parser가 내장되어 있지 않음. config.xml에서 <code><userAgentPattern XPathParser="Android 4.3[WwWW]+N900 Android 4.3[WwWW]+E300 Android 4.4"/></code> 을 추가. <i>/websquare/externalJS/xpath/xpath.js</i> 를 로딩하여 사용 시 성능 저하 발생. • 권장: ◦ 근본적인 처리 방안 없음으로 가급적 해당 기기를 사용하지 않을 것.
5	MDI(WindowContainer) 사용시 화면 중복 오픈 금지 및 오픈 창 개수 제한이 있는가?	MDI 개별 Window가 지나치게 많아지게 될 경우 메모리 과다로 인해 화면 조작이 불가.

34.7.2. 컴포넌트

34.7.2.1. 일반

번호	확인 사항	설명
1	컴포넌트 및 함수의 네이밍은 문제가 없는가?	가이드된 네이밍 규칙을 준수하는지 확인.

34.7.2.2. Generator

번호	확인 사항	설명
1	대량 건을 반복해서 출력하고 있는가?	렌더링 되는 DOM 개수가 많아지면 그만큼 성능 저하가 발생. 실제로 화면에 출력되는 영역만 렌더링하고, 출력되지 않는 영역은 아예 DOM 자체가 생성되지 않도록 처리하는 것을 권장. - 사례 - WFrame이 포함된 컴포넌트를 Generator로 500건 이상 그렸을 때 초기 로딩 속도 10초 이상 소요됨. WFrame을 500개 그리는 대신 1개만 사용하여, 사용자가 클릭할 때마다 <code>wframe.setSrc()</code> 함수를 호출하는 구조로 변경하여 초기 로딩 속도를 3초까지 줄일 수 있었음.

34.7.2.3. GridView

번호	확인 사항	설명
1	GridView의 각 셀에 서로 다른 서식을 적용하고 있는가?	GridView의 <code>formatter</code> 및 <code>setCellColor()</code>, <code>setCellBackgroundColor()</code> 함수 등을 빈번하게 실행할수록 화면 렌더링 속도 및 스크롤 속도가 느려질 수 있음. - 권장: - 서버에서 서식을 처리하여 내려주는 것을 권장. - 한행의 셀 표현 방식을 변경하기 위해 <code>formatter</code>류가 많이 사용된 경우 렌더링 속도에 영향을 줄 수 있으므로, 행이 많은 경우는 서버에서 서식을 적용하는 것을 권장.
2	<code>setCellStyle()</code> 함수를 남용하고 있지 않은가?	셀 단위로 스타일을 적용하기 보다는, 행 단위 혹은 컬럼 단위로 스타일을 적용할 것을 권장.
3	드릴다운 기능을 사용할 경우 컬럼 혹은 데이터가 너무 많지는 않은가?	컬럼이나 데이터가 너무 많은 경우 기하급수적인 성능 저하가 발생할 수 있음.
		<code>visibleRowNum</code> 속성값이 "all" 혹은 "100" 이상으로 설정하고 GridView와 바인딩된 DataList가 많은 데이터를 포함할 경우, GridView에 모든 데이터가 렌더링되

번호	확인 사항	설명
4	<code>visibleRowNum</code> 속성값이 너무 크게 설정되어 있지 않은가?	어 CPU 및 메모리 과다 사용이 발생할 수 있고, 그 결과 속도 저하, 백화 현상, 브라우저 비정상 종료 등이 초래될 수 있음. - 권장: <code>visibleRowNum</code> 속성은 업무 화면에서 스크롤이 생기지 않는 수준에서 설정하거나 "100" 미만으로 설정할 것을 권장.
5	<code>customerFormatter</code> 혹은 <code>typeGetter</code> 속성값으로 지정된 함수가 <code>DataList</code> 혹은 UI를 조작하는 함수를 호출하고 있는가?	<code>customFormatter</code> 혹은 <code>typeGetter</code> 가 <code>GridView</code> 와 바인딩 된 <code>DataList</code> 를 조작할 경우, <code>GridView</code> 가 갱신되면서 <code>customFormatter</code> 및 <code>typeGetter</code> 가 다시 호출되어 무한 루프에 빠질 수 있음. <code>GridView</code> UI 조작 API 중에서도 <code>GridView</code> 를 갱신하여 <code>customFormatter</code> 및 <code>typeGetter</code> 를 다시 호출하도록 유도하는 API가 있으므로 가급적 이를 피해야 함. - 피해야 할 대표적인 API <code>DataList</code> <code>setCellData()</code> <code>GridView</code> <code>setCellReadOnly()</code> <code>setRowVisible()</code>
6	렌더링 되는 컬럼의 갯수(<code><td></code> 태그)가 과도하게 많지는 않은가?	렌더링 되는 <code><td></code> 태그의 수는 "컬럼 수 * 현재 화면에 표시된 행의 수"와 동일. 메모리 사용량 및 렌더링 시간은 <code><td></code> 태그의 수에 비례함. <code>senseReader= " true "</code> 또는 <code>visibleRowNum= " all "</code> 속성 설정을 사용할 경우 특히 주의해야 함.

34.7.2.4. IFrame

번호	확인 사항	설명
1	전체 Layout 화면 구성 시 IFrame 개수가 지나치게 많이 사용되지는 않았는가?	IFrame 사용 개수가 많아지면 화면 렌더링 속도 저하 및 메모리 과다 점유 현상 발생 가능. - 권장: - 가능한 IFrame을 WFrame로 변환해서 처리할 것을 권장.
2	IFrame이 야기할 수 있는 메모리 누수 가능성을 확인했는가?	대표적인 메모리 누수 원인을 확인. - IFrame 사이에서 개체 참조 시 - IFrame 사이에서 데이터 전달 시 - IFrame 사이에서 함수 호출 시 인자로 개체를 전달하는 경우 - 다른 IFrame 영역의 DOM에 동적으로 이벤트를 생성할 경우

34.7.2.5. Submission

번호	확인 사항	설명
1	sync 통신을 무분별하게 사용하고 있지 않은가?	sync 방식으로 서비스를 요청할 경우, 데이터 응답이 반환될 때까지 브라우저가 다른 작업을 수행할 수 없기 때에 사용자 경험에 영향을 줌. 특히, 서버 응답이 지연되거나 데이터 양이 큰 경우, 사용자 경험을 크게 손상시킴.

34.7.2.6. TabControl

번호	확인 사항	설명
1	TabControl 사용 시 다수의 탭을 사용하는가?	다수의 탭 중 닫혀진 탭 내부 콘텐츠를 렌더링 하는 방법이 <code>alwaysDraw="true"</code>인 경우, 최초 로딩 시 다수의 탭이 있을수록 화면 로딩 속도가 저하. - 권장: <code>alwaysDraw="false"</code>를 사용하여 탭이 선택될 때 로딩하는 방식 권장.
2	TabControl 내부 Content 항목의 화면 구성 시 별도의 xml 파일 연결을 확인했는가?	TabControl 내부 Content 항목이 IFrame을 포함하여 IFrame이 중복으로 사용되는 현상 발생. - 권장: - TabControl 내의 content 속성 중 src 에서 별도 xml 파일을 선택하도록 해야함.
3	<code>alwaysDraw="true"</code> 속성 설정을 사용하고 있는가?	<code>alwaysDraw="true"</code> 사용 지양.

34.7.2.7. 팝업

번호	확인 사항	설명
1	Window 팝업을 alert이나 IFrame형태의 팝업으로 처리했는가?	2단 팝업 등에서 메모리 미반환 현상 발생.
2	팝업(프레임)간 Object를 전달하고 있는가?	타 프레임에 Primitives형(예: String)이 아닌 Object를 직접 전달하고 있는지 확인.
3	팝업으로 인자 전달 시에 객체 대신 string을 사용했는가?	객체 형태로 전달시 메모리 누수 발생.

34.7.3. 데이터

번호	확인 사항	설명
1	대용량 Excel 다운로드가 있는가?	클라이언트 환경에서 데이터를 수집하고 서버로 보내는 경우 시간이 오래 소요될 수 있음. - 권장: - 서버에서 데이터를 직접 처리하도록 Provider를 사용할 것을 권장.
2	DataCollection의 Data를 처리하기 위한 API들이 효율적으로 사용되었는가?	Data 처리를 비효율적으로 API를 반복하여 사용하는 경우 성능 저하 발생 - 예시 1개의 행 변경하기 위해 <code>setRowJson()</code>을 사용하지 않고 각 셀에 대해 <code>setCellData()</code>를 반복적으로 사용
3	대량 Data 처리시 Data 구조체를 적절하게 사용했는가?	array > json > xml 순으로 속도 처리가 빠름
4	데이터 객체의 동적 생성시 중복 문제는 없는가?	동일한 ID로 중복하여 생성할 경우 메모리 누수 발생.
5	대량의 데이터를 적재하고 있는 DataList의 데이터 값을 읽거나 수정할 때, 적절한 코드를 사용하는가?	값을 수정할 경우, DataList에 바인딩 되어 있는 GridView 나 LinkDataList가 많을 수록 영향도가 증가함. - 권장: <code>setCellData()</code>, <code>getCellData()</code>, <code>getAllJSON()</code> 등의 호출 패턴 검토가 필요함. <code>setBroadcast()</code> 함수 활용.

34.7.4. 외부 연동

번호	확인 사항	설명
1	외부 모듈 연계시 document mode(DocType)의 지원 범위가 표준을 준수하는가?	외부모듈 지원 시 DocType이 비표준 모드인 경우, 브라우저 하위 버전으로 설정되어 렌더링 속도에 영향을 미침.
2	외부 모듈 연계 시 해당 외부 모듈의 메모리 누수 여부를 확인했는가?	브라우저 전체 메모리에 영향을 미침.
3	외부 모듈(bootstrap 사용시)의 버전 별 메모리 해제 여부를 확인	bootstrap-2.1.1.min.js 해당버전이 메모리 해제가 되지 않는 문제 발생.

번호	확인 사항	설명
	했는가?	
4	Fasoo DRM 모듈을 사용하는가?	Fasoo DRM Client for LIG NoNX Wix, FSW Client 류의 제품들이 메모리 사용시 팝업을 열고 닫을 때 메모리 사용량이 증가할 수 있음.
5	외부 라이브러리를 점검했는가?	- Import된 외부 라이브러리의 메모리 점유량 확인. - IFame, Window Popup, SPA 방식이 메모리 누수를 야기하는지 확인.

35. 파일 업로드/다운로드 가이드

35.1 업로드

35.1.1. Upload/MultiUpload

35.1.1.1. 기본 설정

Upload나 MultiUpload 컴포넌트를 이용하여 파일을 업로드할 경우, **WEBSQUARE_HOME** 의 **config** 폴더에 있는 **websquare.xml** 에 아래와 같은 설정을 추가하십시오.

Table 35-1 파일 업로드를 위한 **websquare.xml** 설정 항목

항목	설명
baseDir	파일이 업로드되는 기본 경로. - 실제 파일이 생성되는 경로는 "baseDir+folderName"입니다. folderName은 uploadMode에 따라 결정됩니다.
subDir1	baseDir 의 하위 태그. (각 사이트 별로) 하위 경로를 추가로 설정할 수 있습니다. - 화면 개발자는 baseDir 대신 subDir1을 Upload 또는 MultiUpload 컴포넌트의 subDir 속성으로 지정해야 합니다. (아래 예제에서, Upload 컴포넌트의 subDir 속성이 지정되지 않은 경우, D:WuploadW가 파일 업로드 경로가 됩니다. subDir 속성이 "SubDir1"인 경우, D:WuploadWtest1W이 업로드 경로가 됩니다.)
encoding	업로드할 파일의 인코딩 방법
duplicatedFilePostFix	동일한 이름의 파일이 존재할 경우 업로드 하는 파일에 추가할 postfix
maxUploadSize	업로드할 파일의 최대 크기 (단위: Byte)
subSize1	maxUploadSize 하위 태그. (각 사이트 별로) 최대 크기를 변경 가능합니다. - 화면 개발자는 사이트 별로 지정된 최대 크기를 사용할 경우 Upload 또는 MultiUpload 컴포넌트의 subSize 속성으로 subSize1을 지정해야 합니다. (아래의 예제에서 Upload 컴포넌트의 subSize 속성이 지정되지 않은 경우, maxUploadSize와 동일한 500000을 사용합니다. subSize속성이 subSize1인 경우, 100000을 사용합니다.)
deniedExtension	업로드에서 제외할 파일의 확장자 allowedExtension 보다 우선합니다.
allowedExtension	업로드 가능한 파일의 확장자
	폴더 이름 생성 규칙 - session 혹은 random (기본값: session) session : request 세션에 folderName 값이 등록된 경우 해당 값을 폴더명으로 사용. (예: sessi

항목	설명
uploadMode	<code>on.getAttribute(" folderName ")</code> - request 세션에 folderName 값이 없는 경우, 해당 값을 폴더명으로 사용. foderName이 없는 경우 폴더명을 임의로 생성. random: folderName으로 지정된 값을 폴더명으로 사용. folderName이 없는 경우 폴더명을 임의로 생성.
folderName	파일을 업로드할 폴더명. (uploadMode에서 참조.)
fileDefiner	파일 업로드 경로 및 파일 이름을 정의하는 사용자 정의 클래스의 전체 경로. - 사용자 정의 클래스는 AbstractUploadFileDefiner를 상속하여 (폴 패키지 포함하여) 구현. - 지정하지 않는 경우, <code>websquare.upload.handl.impl.DefaultUploadFileDefinerImpl</code>를 사용. (fileDefiner 구현 참조.)
decrypt	파일을 업로드 후 DRM 암호화 해제에 필요한 클래스의 전체 경로. 암호화 해제 클래스는 <code>websquare.upload.handl.AbstractUploaDecrypter</code>를 상속 받아 생성함. (decrypt 구현 참조.)
decrypt/@tempDir	DRM 암호화 해제를 위한 임시 폴더.
removeKeyPath	업로드 결과 반환 시 경로 정보 삭제 여부. (true 혹은 false)

Code 35-1 websquare.xml 설정 예

```

1  <upload>
2    <baseDir value="D:\upload\">
3      <subDir1 value="D:\uplaod\test1\"/>
4      <subDir2 value="D:\uplaod\test2\"/>
5      <!-- ... subDir is changeable. -->
6    </baseDir>
7    <encoding value="UTF-8"/>
8    <duplicatedFilePostFix value="yyyy-MM-dd(HH mm ss)"/>
9    <maxUploadSize value="50000000">
10      <subSize1 value="100000"/>
11      <subSize2 value="1000000"/>
12    </maxUploadSize>
13    <deniedExtension>
14      cmd,exe,none
15    </deniedExtension>
16    <allowedExtension>
17      gif, jpg, doc, xls, ppt, pdf, txt, csv, xlsx
18    </allowedExtension>

```

```

19 <uploadMode value="session"/>
20 <folderName value="uploadFolerName"/>
21 <fileDefiner value="websquare.upload.handl.impl.DefaultUploadFileDefinerImpl"/>
22 <decrypt value="com.inswave.Decrypt" tempDir="/TEMPDIR" />
23 <callback>
24     <removeKeyPath value="true" />
25 </callback>
26 </upload>

```

35.1.1.2. fileDefiner 구현

파일 업로드 경로를 기본 설정으로 충족할 수 없는 경우, 아래의 방법으로 경로를 재정의하십시오.

1. `websquare.upload.handl.AbstractUploadFileDefiner`를 상속받습니다.
2. 리턴값이 `String`인 `getFileName`과 `getFilePath`를 구현합니다.

실제 생성되는 파일은 "getFilePath의 결과값 + File.separator+getFileName의 결과값"입니다. 아래 예제의 useDir과 같은 사용자 파라미터를 화면에서 파라미터로 전달할 수 있으며, Server는 MultipartParameterMap을 통해 String Array로 반환하여 사용합니다.

Code 35-2 Upload 컴포넌트 용 스크립트 예

```

1 upload1.addParam( "useDir" , "useDir_value" );
2 upload1.submit();

```

Code 35-3 MultiUpload 컴포넌트 용 스크립트 예

```

1 multiupload1.setParam( 0 , "useDir" , "useDir_value_m")
2 multiupload1.startUpload();

```

Code 35-4 fileDefiner 구현 예

```

1 import javax.servlet.http.HttpServletRequest;
2
3 import websquare.http.MultiPartHttpServletRequest;
4 import websquare.http.WebSquareContext;
5 import websquare.upload.handl.AbstractUploadFileDefiner;
6
7 public class DefaultUploadFileDefinerImpl extends AbstractUploadFileDefiner {
8     /**
9      * @param <String:Y> clientFileName      : 클라이언트에서 선택한 파일명
10     * @param <String:Y> originalFileName    : 물리적으로 저장될 파일명
11     * @desc 파일명을 변경한다.

```

```

12     */
13     @Override
14     public String getFileName(String clientFileName, String originalFileName) throws
Exception {
15         // 파일명에 대한 별도 변경 처리시 추가한다.
16         System.out.println("\nclientFileName : [" + clientFileName + "]");
17         System.out.println("\noriginalFileName : [" + originalFileName + "]");
18         return originalFileName;
19     }
20     /**
21     * @param <String:Y> filePath          : 물리적으로 저장될 파일 경로
22     * @desc 파일 저장 경로를 변경한다.
23     */
24     @Override
25     public String getFilePath(String filePath) throws Exception {
26
27         // 파일업로드는 WAS의 공유볼륨에 저장된다. 파일 저장위치는
{websquare_home}/config/websquare.xml 의 baseDir에 위치한다.)
28         // 업로드컴퍼넌트의 useDir 파라메타가 있을때 baseDir 하위에 폴더를 생성하고
저장한다.
29         HttpServletRequest request =
WebSquareContext.getContext().getRequest();
30         MultiPartHttpServletRequest multiPartRequest =
(MultiPartHttpServletRequest)request;
31
32         Map parameterMap = multiPartRequest.getMultipartParameters();
33
34         String[] useDir = (String[])parameterMap.get("useDir");
35
36         String _filePath = filePath.substring(0, filePath.lastIndexOf(File.separator));
37         StringBuffer rt = new StringBuffer(_filePath);
38
39         if (useDir != null) {
40             int iCnt = useDir.length;
41             for (int idx = 0; idx < iCnt; idx++) {
42                 if (idx == 0) rt.append(File.separator);
43                 rt.append(useDir[idx]);
44             }
45         }
46
47         System.out.println("filePath ==: " + _filePath);

```



```

48         System.out.println("rt.toString() ==: " + rt.toString());
49
50         return rt.toString();
51     }
52 }

```

35.1.1.3. decrypt 구현

DRM과의 연계가 필요한 경우, DRM 암호화 해제 후 파일을 반환하는 클래스를 아래와 같이 구현하십시오.

1. `websquare.upload.handl.AbstractUploaDecrypter` 를 상속받습니다.
2. 리턴값이 `File`인 `decrypt`를 구현합니다.

Code 35-5 decrypt 구현 예

```

1  import java.io.File;
2  import websquare.upload.handl.AbstractUploaDecrypter;
3  import com.fasoo.adk.packager.WorkPackager;
4
5  public class ExcelUpDecripterWithFasoo extends AbstractUploaDecrypter {
6      /**
7       * @param <File:Y> file          : 클라이언트에서 선택한 파일명
8       * @desc 엑셀파일 -> WebSquare GridView 업로드시 복호화 연계
9       */
10     public File decrypt(File file) throws Exception {
11         // 1. 클라이언트에서 upload한 파일을 복호화 연계 처리를 구현 한다.(넘겨받는 file은
extractFilePath에 저장된다.)
12         // 2. 복호화된 파일을 return 한다.
13         String strFsdinitDir = "/TMAX/wq/FSD/fsdinit";          //
설정파일경로
14         String serverid      = "0123456789";                    // 사이트 고유 코드
15         String extractFilePath = "/TMAX/wq/tempUpload/excel/extractFilePath"; //
복호화파일경로
16         // String packageFilePath = "/TMAX/wq/tempUpload/excel/packageFilePath"; //
암호화파일경로
17
18         String fileName = file.getName();
19         String packageFile = extractFilePath + File.separator + fileName;
20
21         WorkPackager oWorkPackager = new WorkPackager();
22         oWorkPackager.setOverWriteFlag(true);                    //
덮어쓰기

```

```

23      // oWorkPackager.setCharset("UTF-8");                                //
캐릭터셋 설정
24      oWorkPackager.setCharset("eucKR");
25      // oWorkPackager.setCharset("KSC5601");
26
27      if (oWorkPackager.IsPackageFile(packageFile)) {
28
29          boolean bret = oWorkPackager.DoExtract(strFsdinitDir, serverid, packageFile,
packageFile);
30
31          if (bret) {
32              System.out.println("복호화 성공.[" + packageFile + "]");
33              return new File(packageFile);
34          } else {
35              System.out.println("복호화 중 오류입니다.");
36              System.out.println(" 오류 정보..");
37              System.out.println(" [" + oWorkPackager.getLastErrorNum() + "] " +
oWorkPackager.getLastErrorStr());
38          }
39      }
40      return file;
41  }
42  }

```

35.1.2. Editor (이미지)

35.1.2.1. 기본 설정

Editor 컴포넌트를 이용하여 이미지를 업로드하거나 추가할 경우, **WEBSQUARE_HOME** 의 **config** 폴더에 있는 **w ebsquare.xml** 에 아래와 같은 설정을 추가하십시오.

Table 35-2 이미지 업로드를 위한 **websquare.xml** 설정 항목

항목	설명
baseDir	파일이 업로드되는 기본 경로. - 실제 파일이 생성되는 경로는 "baseDir+folderName"입니다. folderName은 uploadMode에 따라 결정됩니다.
	baseDir의 하위 태그. (각 사이트 별로) 하위 경로를 추가로 설정할 수 있습니다. - 화면 개발자는 baseDir 대신 subDir1을 Editor 컴포넌트의 subDir 속성으로 지정해야

항목	설명
subDir	합니다. (아래의 예제에서, Editor 컴포넌트의 subDir 속성이 빈 값인 경우, D:\upload\test1가 이미지 업로드 경로가 됩니다. subDir 속성이 "SubDir1"인 경우, D:\upload\SubDir1이 업로드 경로가 됩니다.) Editor에 포함된 이미지는 subDir 속성값만 사용합니다. (즉, Editor를 통해 업로드한 이미지가 baseDir과 subDir로 업로드되지 않고 동일한 폴더로 업로드됩니다.)
deniedExtension	업로드에서 제외할 파일의 확장자 allowedExtension 보다 우선합니다.
allowedExtension	업로드 가능한 파일의 확장자
imageDefiner	- 파일 업로드 경로 및 파일 이름을 정의하는 사용자 정의 클래스의 전체 경로. - 사용자 정의 클래스는 AbstractUploadImageDefiner를 상속하여 (폴 패키지를 포함하여) 구현. - 지정하지 않은 경우, websquare.upload.handl.impl.DefaultUploadImageDefinerImpl를 사용.

Code 35-6 websquare.xml 설정 예

```

1 <upload>
2   <imgUpload>
3     <baseDir value="D:\upload\image"
4       <subDir1 value="D:\upload\image\test1" />
5     </baseDir>
6     <allowedExtension>
7       png,jpg,gif,jpeg,tiff,bmp
8     </allowedExtension>
9     <deniedExtension>
10    </deniedExtension>
11    <!--
12      파일명을 변경하기 위해 업로드 경로 및 사용자 정의 클래스를 선언
13    -->
14    <imageDefiner value="websquare.upload.handl.impl.DefaultUploadImageDefinerImpl"/>
15  </imgUpload>
16 </upload>

```

35.1.2.2. imageDefiner 구현

이미지를 업로드 경로를 기본 설정으로 충족할 수 없는 경우, 아래의 방법으로 경로를 다시 정의하십시오.

1. **websquare.upload.handl.AbstractUploadImageDefiner**를 상속받습니다.
2. 리턴값이 **String**인 **getFileName**과 **getFilePath**를 구현합니다.

실제 생성되는 파일은 "**getFilePath**의 결과값+**File.separator**+**getFileName**의 결과값"입니다.

Code 35-7 imageDefiner 구현 예

```

1  import javax.servlet.http.HttpServletRequest;
2
3  import websquare.http.MultiPartHttpServletRequest;
4  import websquare.http.WebSquareContext;
5  import websquare.upload.handl.AbstractUploadImageDefiner;
6
7  public class DefaultUploadFileDefinerImpl extends AbstractUploadImageDefiner {
8      /**
9       * @param <String:Y> clientFileName      : 클라이언트에서 선택한 파일명
10      * @param <String:Y> originalFileName    : 물리적으로 저장될 파일명
11      * @desc 파일명을 변경한다.
12      */
13      @Override
14      public String getFileName(String clientFileName, String originalFileName) throws
Exception {
15          // 파일명에 대한 별도 변경 처리시 추가한다.
16          System.out.println("\nclientFileName : [" + clientFileName + "]");
17          System.out.println("\noriginalFileName : [" + originalFileName + "]");
18          return originalFileName;
19      }
20      /**
21      * @param <String:Y> filePath              : 물리적으로 저장될 파일 경로
22      * @desc 파일 저장 경로를 변경한다.
23      */
24      @Override
25      public String getFilePath(String filePath) throws Exception {
26
27          // 파일업로드는 WAS의 공유볼륨에 저장된다. 파일 저장위치는
28          // {websquare_home}/config/websquare.xml 의 baseDir에 위치한다.)
29          // 업로드컴퍼넌트의 useDir 파라메타가 있을때 baseDir 하위에 폴더를 생성하고
30          // 저장한다.
31
32          HttpServletRequest request =
WebSquareContext.getContext().getRequest();
33
34          MultiPartHttpServletRequest multiPartRequest =
(MultiPartHttpServletRequest)request;
35
36          Map parameterMap = multiPartRequest.getMultipartParameters();
37
38          String[] useDir = (String[])parameterMap.get("useDir");

```

```

36     String _filePath = filePath.substring(0, filePath.lastIndexOf(File.separator));
37     StringBuffer rt = new StringBuffer(_filePath);
38
39     if (useDir != null) {
40         int iCnt = useDir.length;
41         for (int idx = 0; idx < iCnt; idx++) {
42             if (idx == 0) rt.append(File.separator);
43             rt.append(useDir[idx]);
44         }
45     }
46
47     System.out.println("filePath ==: " + _filePath);
48     System.out.println("rt.toString() ==: " + rt.toString());
49
50     return rt.toString();
51 }
52 }

```

35.1.3. GridView

35.1.3.1. Excel

35.1.3.1.1. 기본 설정

GridView 컴포넌트에 Excel 파일을 업로드할 경우, **WEBSQUARE_HOME** 의 **config** 폴더에 있는 **websquare.xml** 에 아래의 설정을 추가하십시오.

Table 35-3 Excel 파일 업로드를 위한 **websquare.xml** 설정 항목

항목	설명
decrypt	Excel/CSV 파일을 업로드 후 DRM 암호화 해제에 필요한 클래스의 전체 경로. 암호화 해제 클래스는 websquare.upload.handl.AbstractUploaDecrypter를 상속 받아 구현. (35.1.3.1.2. decrypt 구현 참조.)
decrypt/@tempDir	DRM 암호화 해제를 위한 임시 폴더.
dataConvertor	각 셀의 데이터를 가공할 경우, 가공된 셀 데이터를 반환하는 클래스의 전체 경로. 셀 데이터 가공용 클래스는 websquare.http.controller.grid.excel.read.ICellDataProvider를 상속받아 구현. (35.1.3.1.3. dataConverter 구현 참조.)
loadingMode	셀 데이터 읽는 방법 - SAX 혹은 POI. (기본값: POI)

Code 35-8 **websquare.xml** 설정 예

```

1 <WebSquare>
2     <excel>
3         <upload>
4             <decrypt value="com.inswave.Decrypt" tempDir="/TEMPDIR" />
5             <dataConvertor value="com.inswave.CellConvertor" />
6             <loadingMode value="" />
7         </upload>
8     </excel>
9 </WebSquare>

```

35.1.3.1.2. decrypt 구현

DRM과의 연계가 필요한 경우, DRM 암호화 해제 후 파일을 반환하는 클래스를 아래와 같이 구현하십시오.

1. `websquare.upload.handl.AbstractUploaDecrypter` 를 상속받습니다.
2. 리턴값이 `File`인 `decrypt`를 구현합니다.

Code 35-9 구현 예

```

1 import java.io.File;
2 import websquare.upload.handl.AbstractUploaDecrypter;
3 import com.fasoo.adk.packager.WorkPackager;
4
5 public class ExcelUpDecripterWithFasoo extends AbstractUploaDecrypter {
6     /**
7      * @param <File:Y> file      : 클라이언트에서 선택한 파일명
8      * @desc 엑셀파일 -> WebSquare GridView 업로드시 복호화 연계
9      */
10    public File decrypt(File file) throws Exception {
11        // 1. 클라이언트에서 upload한 파일을 복호화 연계 처리를 구현 한다.(넘겨받는 file은
12        // 2. 복호화된 파일을 return 한다.
13        String strFsdinitDir = "/TMAX/wq/FSD/fsdinit"; //
14        String serverid = "0123456789"; // 사이트 고유 코드
15        String extractFilePath = "/TMAX/wq/tempUpload/excel/extractFilePath"; //
16        // String packageFilePath = "/TMAX/wq/tempUpload/excel/packageFilePath"; //
17        String fileName = file.getName();

```

```

19      String packageFile = extractFilePath + File.separator + fileName;
20
21      WorkPackager oWorkPackager = new WorkPackager();
22      oWorkPackager.setOverWriteFlag(true);                                //
덮어쓰기
23      // oWorkPackager.setCharset("UTF-8");                                //
캐릭터셋 설정
24      oWorkPackager.setCharset("eucKR");
25      // oWorkPackager.setCharset("KSC5601");
26
27      if (oWorkPackager.IsPackageFile(packageFile)) {
28
29          boolean bret = oWorkPackager.DoExtract(strFsdinitDir, serverid, packageFile,
packageFile);
30
31          if (bret) {
32              System.out.println("복호화 성공.[" + packageFile + "]");
33              return new File(packageFile);
34          } else {
35              System.out.println("복호화 중 오류입니다.");
36              System.out.println(" 오류 정보..");
37              System.out.println(" [" + oWorkPackager.getLastErrorNum() + "] " +
oWorkPackager.getLastErrorStr());
38          }
39      }
40      return file;
41  }
42  }

```

35.1.3.1.3. dataConverter 구현

셀 데이터의 가공이 필요한 경우 아래의 방법으로 dataConvertor를 구현하십시오.

1. websquare.http.controller.grid.excel.write.ICellDataProvider를 상속받습니다.
2. 리턴값이 String인 convertValue를 구현합니다.

다음은 셀 데이터가 tag를 포함한 경우 tag를 삭제하는 예제입니다.

Code 35-10 셀 데이터 가공 (tag 삭제)

```

1  import websquare.http.controller.grid.excel.write.ICellDataProvider;
2
3  /*

```

```

4  * Excel Download시 tag내용을 삭제한다.
5  * "계좌닉네임<a href=\"javascript:gib0bj.grdView01_onheaderclick()\" class=\"btn_help\"
title=\"도움말\">도움말</a>" => 계좌닉네임
6  */
7  public class WebSquareExcelDataConverter implements ICellDataProvider {
8
9      @Override
10     public String convertValue(String cellValue) throws Exception {
11         String value = cellValue.replaceAll("<(\\s|\\s*)*(\\s|\\s*)*>", "\\n");
12         //value =
value.replaceAll("<(\\s|\\s*)*(\\s|\\s*)*(\\s|\\s*)*>", "");
13         value = value.replaceAll("<[>]*>", "");
14         return value;
15     }
16 }

```

35.1.3.1.4. 위변조 방지¹

관련 클래스

AbstractCellDataProvider: GridView에 엑셀 파일을 업로드할 때, 위변조 검증 로직이 참조할 수 있는 셀 정보(행/열 인덱스) 및 HTTP Servlet Request 정보를 제공.

구현 방법

AbstractCellDataProvider 클래스를 extend하여 **convertValue** 함수를 구현.

이후 관련 설정을 **websquare.xml**에 추가.

Code 35-11 **AbstractCellDataProvider**

```

1  package websquare.http.controller.grid.excel.read;
2
3  import javax.servlet.http.HttpServletRequest;
4
5  public abstract class AbstractCellDataProvider {
6      public abstract String convertValue( int rowIndex, int colIndex,
7      String cellvalue, HttpServletRequest request ) throws Exception;
8
9  }

```

Code 35-12 구현 예시

1. 5.0_1.2596A.20170406.123456 버전부터 지원


```

1 package websquare.http.controller.grid.excel.read;
2
3 import javax.servlet.http.HttpServletRequest;
4
5 public class TEST extends AbstractCellDataProvider {
6     public String convertValue(int rowIndex, int colIndex, String cellvalue,
7     HttpServletRequest request) throws Exception {
8
9         return cellvalue;
10    }
11 }

```

websquare.xml에 아래 설정을 추가하십시오.

Code 35-13 **websquare.xml**

```

1 <websquare>
2     <excel>
3         <upload>
4             <cellDataConvertor value="구현팩키지명" />
5         </upload>
6     </excel>
7 </websquare>

```



GridView의 엑셀 파일 업로드/다운로드시 셀 데이터를 가공에 사용되는 **ICellDataProvider**는 현재의 셀의 값만 제공.

35.1.3.2. CSV

GridView 컴포넌트에 CSV 파일을 업로드할 경우, **WEBSQUARE_HOME** 의 **config** 폴더에 있는 **websquare.xml**에 아래의 설정을 추가하십시오.

Table 35-4 CSV 파일 업로드를 위한 **websquare.xml** 설정 항목

항목	설명
encoding	CSV 파일 일코딩 방법 (기본값: UTF-8)
useBOM	BOM 사용 여부 - true 혹은 false. 인코딩 방법이 UTF-8인 경우, 한글이 정상 추력되지 않습니다. 이 경우 Editor 등을 통해 BOM 사용 여부를 확인한 후, 확인 결과를 본 항목에 설정하십시오.
decrypt	Excel/CSV 파일을 업로드 후 DRM 암호화 해제에 필요한 클래스의 전체 경로. 암호화 해제 클래스는 websquare.upload.handl.AbstractUploaDecrypter를 상속

항목	설명
	받아 생성함. (35.1.3.1.2. decrypt 구현 참조.)
maxUploadSize	업로드 가능한 최대 파일 크기를 지정. (단위: 바이트) 해당 크기를 초과할 경우 오류 메시지 출력. 브라우저가 files 속성을 지원하지 않을 경우 사용.
decrypt/@tempDir	DRM 암호화 해제를 위한 임시 폴더.

Code 35-14 **websquare.xml** 설정 예

```

1 <WebSquare>
2     <csv>
3         <encoding value="UTF-8" />
4         <useBOM value="true" />
5         <upload>
6             <decrypt value="com.inswave.Decrypt" tempDir="/TEMPDIR" />
7             <maxUploadSize value="바이트수" />
8         </upload>
9     </csv>
10 </WebSquare>

```

35.1.3.2.1. 최대 업로드 사이즈²

Table 35-5 **options.maxFileSize** (gridView1.readCSV (options))

설명	업로드 가능한 최대 파일 크기를 지정. (단위: 바이트) 해당 크기를 초과할 경우 오류 메시지 출력.
비고	- 브라우저가 files 속성을 지원할 경우에만 유효. (IE9 이상) - 브라우저가 files 속성을 지원하지 않을 경우, 아래 항목을 이용. websquare.xml 파일의 <maxUploadSize>

Code 35-15 사용 예

```

1 var option = {};
2 option.maxFileSize = "바이트수";
3 gridView1.readCSV( option );

```

35.1.4. Upload시 파일명 처리

35.1.4.1. 중복 방지용 날짜 추가

Code 35-16 websquare.xml

```

1 <websquare>
2   <upload>
3     <uploadMode value="random" />
4     <baseDir value="D:\test\upload" />
5     <folderName value="up" />
6     <duplicatedFilePostFix value="yyyy-MM-dd(HH:mm:ss)" />
7     <firstFilePostFix value="true" />
8   </upload>
9 </websquare>

```

35.1.4.1.1. 설정 설명

물리적으로 저장되는 경로는 baseDir / [subDir] / folderName 이 되며, uploadMode 에 따른 folderName 다음과 같습니다.

- uploadMode : session, random 값을 갖는다.
 - I. session : session에 folderName이라면 변수로 값이 존재 할 경우 해당 값으로 폴더명을 정의합니다. session에 folderName이 존재 하지 않을 경우, websquare.xml에서 folderName 설정값을 읽어 폴더명을 정의합니다. folderName 설정이 존재 하지 않은 경우는 random으로 폴더명을 정의합니다. (ex) folderName_3939393939)
 - II. random : websquare.xml에서 folderName 설정값을 읽어 폴더명을 정의합니다.
- folderName 설정이 존재 하지 않은 경우는 random으로 폴더명을 정의합니다
- (ex) folderName_3939393939)
- baseDir : 기본 업로드 폴더
- baseDir의 하위노드 : baseDir의 하위에 폴더를 지정하려고 할 때 노드를 정의 하고 value값을 설정합니다. 업무적으로 업로드 폴더를 분리하는 경우 사용합니다. 하위디렉토리 정의 후 w2:upload 컴포넌트에 subDir Attribute를 정의합니다.
- folderName : 업로드 되는 폴더명. uploadMode와 연계해서 사용 한다.
- duplicatedFilePostFix : upload시 동일한 파일이 있을 경우 파일명 뒤에 붙여지는 prefix
- firstFilePostFix : true인 경우 신규파일이라도 duplicatedFilePostFix를 붙입니다.

35.1.4.1.2. 처리 방법

- firstFilePostFix 설정값이 true인 경우 신규파일이라도 duplicatedFilePostFix를 붙입니다. false이면 중복파일명이 없는 신규파일명은 원래 파일명을 따르고, 중복파일명이면 duplicatedFilePostFix값을 붙여 파일명을 정의합니다

35.1.4.2. 업로드 경로를 업무별로 분리하는 방법

websquare.xml이 아래와 같이 설정되어 있을 경우, subDir은 w2:upload 컴포넌트의 속성을 선택합니다.

```

1 <websquare>
2   <upload>
3     <baseDir value="D:\upload\">
4       <subDir1 value="D:\uptest1\" />
5       <subDir2 value="D:\uptest2\" />
6     </baseDir>
7     <folderName value="up" />
8   </upload>
9 </websquare>

```

35.1.4.2.1. 처리 방법

Code 35-17 업무 1

```

1 <w2:upload type="image" style="position:absolute;vertical-align:middle;word-wrap:break-
2 word;left:70px;top:100px;width:220px;height:50px;" imageStyle=
3 "position:absolute;vertical-align:middle;
4 word-wrap:break-word" inputStyle=
5 "position:absolute;vertical-align:middle;word-wrap:break-word"
6 id="upload1" subDir="subDir1">

```

Code 35-18 업무 2

```

1 <w2:upload type="image" style="position:absolute;vertical-align:middle;word-wrap:break-
2 word;left:70px;top:100px;width:220px;height:50px;" imageStyle=
3 "position:absolute;vertical-align:middle;
4 word-wrap:break-word" inputStyle=
5 "position:absolute;vertical-align:middle;word-wrap:break-word"
6 id="upload1" subDir="subDir2">

```

이 경우 업무1은 D:\Wuptest1\Wup\W 경로로 파일이 올라 가고, 업무2는 D:\Wuptest2\Wup\W 경로로 파일이 업로드 됩니다.

subDir속성에 값이 없는 경우에는 D:\Wupload\Wup\W의 경로로 업로드 됩니다

35.1.4.3. 사용자 정의 fileDefiner 설정

```

1 <websquare>
2   <upload>
3     ...
4     <fileDefiner value="com.sample.CustomUploadFileDefiner" />
5   </upload>
6 </websquare>

```

- fileDefiner : websquare 에서 제공하는 업로드 설정이 아닌 파일 경로와, 파일명을 재설정하는 사용자 정의 class (full-package)

35.1.4.3.1. 처리 방법

- AbstractUploadFileDefiner 을 상속 구현한다.

```

1 package com.sample;
2
3 import websquare.upload.handl.*;
4
5 public class CustomUploadFileDefiner extends AbstractUploadFileDefiner {
6
7     public String getFileName( String clientFileName, String originalFileName ) throws
Exception {
8         //파일명 변경 처리
9         return originalFileName;
10    }
11    public String getFilePath( String filePath ) throws Exception {
12        //파일경로 변경 처리
13        return filePath;
14    }
15 }

```

35.1.5. 업로드 이미지 파일명 (Editor)

editor 컴포넌트 사용시 이미지를 업로드하는 경우 참조하는 설정으로 Upload 설정과 동일합니다.

```

1 <websquare>
2   <upload>
3     ...
4     <imgUpload>
5       <baseDir value="D:\uptest1\image" />
6       <subDir1 value="D:\uptest1\image" />
7       ...
8       <imageDefiner value="com.sample.CustomImageUploadFileDefiner" />
9     </imgUpload>
10    ...
11  </upload>
12 </websquare>

```

35.2 다운로드

35.2.1. `$p.download` (Excel 및 기타 파일)

WebSquare5는 `$p.download(url경로)` API를 제공합니다. 웹에서 Excel 파일을 다운로드하거나 모바일에서 Excel 파일을 보기를 위해 사용하며, Excel 파일 경로는 아래와 같이 context-root부터의 절대 경로로 지정해야 합니다.

Code 35-19 사용 예

```
1 $p.download( " /download/sample/test.xls " )
```

Excel 이외의 파일을 다운로드하거나 URL로 접근할 수 없거나 혹은 추가 로직이 필요한 경우, 이를 JSP나 servlet으로 구현할 수 있습니다. 모바일은 Cordova가 plugin으로 제공하는 `FileTransfer`를 이용하십시오.

Code 35-20 스크립트 작성 예

```

1 //url : file의 full 경로+file명, filename : 경로를 제외한 실제 파일의 이름(test.txt)
2 common.fileDownload = function(url, fileName) {
3   if (typeof url == "undefined" || url == "") {
4     alert("url is null");
5     return;
6   }
7   if (typeof fileName == "undefined") {
8     alert("filename is null");

```

```

9     return;
10 }
11
12 var submitUrl = "";
13 if (url.indexOf(".jsp") < 0) {
14     submitUrl = "/common/jsp/fileDownload.jsp?fileUrl="+url+"&fileName="+fileName;
15 } else {
16     submitUrl = url;
17 }
18 if (WebSquare.hybridApp) {
19     var fileTransfer = new FileTransfer();
20     var filePath = "";
21     if (WebSquare.core.browserCheck.iphone || WebSquare.core.browserCheck.ipad) {
22         if (cordova.file.tempDirectory == null || cordova.file.tempDirectory == "") {
23             filePath = cordova.file.dataDirectory+fileName;
24         } else {
25             filePath = cordova.file.tempDirectory+fileName;
26         }
27         console.log(filePath);
28     } else {
29         filePath = cordova.file.externalRootDirectory+fileName;
30     }
31
32     fileTransfer.download(gibLangObj.getServerUrl(url), filePath,
33         function(entr) {
34             console.log("filedownload success");
35             if (WebSquare.core.browserCheck.iphone || WebSquare.core.browserCheck.ipad) {
36                 filePath =
37 "cdvfile://localhost/temporary/"+filePath.substr(filePath.indexOf("/tmp")+5);
38                 window.open(filePath, "_blank", "");
39             } else {
40                 window.open(filePath, "_system", "");
41             }
42             function(error) {
43                 console.log("filedownload fail!!");
44             }
45         });
46     } else {
47         $p.download(submitUrl);
48     }

```

```
49 };
```

Code 35-21 JSP 작성 예

```
1 <%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2     request.setCharacterEncoding("UTF-8");
3
4     String fileUrl = request.getParameter("fileUrl");
5     String fileName = request.getParameter("fileName");
6
7     ServletContext context = pageContext.getServletContext();
8     String applicationRoot = context.getRealPath("/");
9
10    try {
11        File file = new File(applicationRoot+"/"+fileUrl);
12        if ( file.exists() ) {
13            if ( fileName == null || "".equals(fileName) ) {
14                fileName = file.getName();
15            }
16            response.reset();
17            response.setContentType("application/octet-stream; charset=UTF-8");
18            response.setHeader("Content-Transfer-Encoding", "binary;");
19            response.setHeader("Pragma", "no-cache;");
20            response.setHeader("Expires", "-1;");
21
22            String userAgent = request.getHeader("User-Agent");
23            if ( userAgent.indexOf("MSIE 5.5") > -1 ) { // MS IE 5.5 이하
24                fileName = URLEncoder.encode(fileName, "UTF-8").replaceAll("\\+", "%20");
25            } else if ( userAgent.indexOf("MSIE") > -1 ) { // MS IE (보통은 6.x 이상 가정)
26                fileName = URLEncoder.encode(fileName, "UTF-8").replaceAll("\\+", "%20");
27            } else if ( userAgent.indexOf("Chrome") > -1 ) { // Chrome
28                fileName = new String(fileName.getBytes("UTF-8"), "ISO-8859-1");
29            } else if ( userAgent.indexOf("Firefox") > -1 ) { // Firefox
30                fileName = new String(fileName.getBytes("UTF-8"), "ISO-8859-1");
31            } else { // 모질라나 오페라
32                fileName = new String(fileName.getBytes("UTF-8"), "latin1");
33            }
34            response.setHeader("Content-Disposition", "attachment;filename=\""+ fileName
35                +"\"");
36
37            response.setHeader("Content-Length", String.valueOf((int)file.length()));
38
39            FileInputStream fis = new FileInputStream(file.getAbsolutePath());
```



```

38      ServletOutputStream sos = response.getOutputStream();
39
40      out.clear();
41      out = pageContext.pushBody();
42
43      byte b[] = new byte[4096];
44      int nRead = 0;
45      while ((nRead = fis.read(b, 0, b.length)) != -1) {
46          sos.write(b, 0, nRead);
47      }
48      sos.flush();
49      sos.close();
50      fis.close();
51  } else {
52      Logger.fine(">> File does not exist !! [" + fileUrl + "]");
53      response.reset();
54      response.setContentType("text/html; charset=UTF-8;");
55      response.setHeader("Content-Transfer-Encoding", "UTF-8");
56      response.setHeader("Content-Disposition", "");
57      out.print("<script>alert(\"file not found [" + fileUrl + "]\")</script>");
58  }
59  } catch (Exception e) {
60      Logger.exception("[fileDownload.jsp] Exception.", e);
61  } finally {
62  }
63  %>

```

35.2.2. **WebSquare.net.download**

`WebSquare.net.download( actionUrl , XML , sendMethod )`를 통해 다운로드를 수행할 경우, 아래의 방법으로 JSP로 파라미터를 전달할 수 있습니다.

1. 내부적으로 IFrame을 만들고 form-submit 하는 방식으로 서블릿을 호출합니다.
2. 두번째 인자로 파라미터(string)를 전달할 수 있습니다.
3. 해당 인자는 xmlValue의 값으로 전달되기 때문에 서블릿에서는 아래 형태로 패러미터값을 가져올 수 있습니다.
`String req = request.getParameter("xmlValue");`
4. 패러미터는 xmlValue의 값으로만 전달됩니다. 2개 이상의 정보를 전달할 경우에는 XML/JSON 형태 문자열로 전달합니다. 서버에서는 해당 문자열을 객체화하여 사용할 수 있습니다.

Code 35-22 사용 예

```

1  ...
2  <script type="text/javascript"><![CDATA[
3  var jsonData = [{"col1":"a", "col2":"b", "col3":"c", "col4":"d"},
4      {"col1":"aa", "col2":"bb", "col3":"cc", "col4":"dd"},
5      {"col1":"aaa", "col2":"bbb", "col3":"ccc", "col4":"ddd"},
6      {"col1":"aaaa", "col2":"bbbb", "col3":"cccc", "col4":"dddd"}];
7
8  dataList1.setJSON( jsonData );
9
10 scwin.trigger1_onclick = function(e) {
11     //debugger;
12     // 데이터리스트를 json으로 반환 후 해당 내용을 인코딩
13
14     var jsondata
15     = WebSquare.text.BASE64Encoder(JSON.stringify(dataList1.getAllJSON()));
16     console.log(jsondata);
17     WebSquare.net.download( "/JSP/FileDown.jsp" , jsondata , "post" );
18
19     //var name = dataList1.getCellData( 0 , 0 );
20     //WebSquare.text.BASE64Encoder(name);
21     // XML 형태로 만들어 주는구조
22     //var xmldata = "<data><name>"+ name + "</name></data>";
23     // JSON 형태로 만들어 주는구조    };
24 ]]></script>
25 ...

```

Code 35-23 사용 예(JSP)

```

1  <%@ page language="java" contentType="text/html; charset=EUC-KR"
2      pageEncoding="EUC-KR"
3      import="    org.w3c.dom.*, java.net.*, websquare.util.*,
4          org.json.simple.*,org.json.simple.parser.*" %><%
5      request.setCharacterEncoding("UTF-8");
6      String req =
7      URLDecoder.decode(HttpUtil.getBase64Parameter(request, "xmlValue"), "UTF-8");
8      application.log(req);
9      System.out.println(req);
10      JSONParser jsonParser = new JSONParser();
11
12      //JSONParser를 이용해서 던진 문자열 반환 dataList일 경우에는
13      // [{},{},{}] 해당형태로 반환하기 때문에 JSONArray 형태로 받아야함.

```

```

14 // 만약 dataMap을 사용한다면 JSONObject로 받아도 가능.
15 // WebSquare는 WebSquare.text.BASE64Encoder로 전달해야함.
16
17 JSONArray jsonArray = (JSONArray)jsonParser.parse(req);
18
19 // JSONObject에서 원하는 데이터 뽑아오기.
20 for(int i = 0; i < jsonArray.size(); i++) {
21     JSONObject jsonObject = (JSONObject)jsonArray.get(i);
22     String col = (String)jsonObject.get("col1");
23     application.log(col);
24     System.out.println(col);
25 }
26
27 // XML로 던지는 JSON 구조
28 // jsp 상에서 xmlValue라는 노드를 붙여서 전달 받음
29 // Document xmlValueDoc =
XMLUtil.getDocument(URLDecoder.decode(HttpUtil.getParameter(request, "xmlValue"), "UTF-8"));
30 // String name = XMLUtil.getText(xmlValueDoc, "name");
31 // application.log(name);
32 %>

```

35.2.3. GridView

35.2.3.1. Excel

35.2.3.1.1. 기본 설정

GridView 컴포넌트에서 Excel 파일을 다운로드할 경우, **WEBSQUARE_HOME** 의 **config** 폴더에 있는 **websquare.xml** 에 아래의 설정을 추가하십시오.

Table 35-6 Excel 파일 다운로드를 위한 **websquare.xml** 설정 항목

항목	설명
encrypt	Excel 파일을 DRM으로 암호화 후 stream으로 write하는 클래스의 전체 경로. websquare.download.handl.AbstractDownloadEncrypter 를 상속. - JDK 1.5 이상 DRM과 연계. (35.2.3.1.2. encrypt 구현 참조.)
encrypt14	DRM 암호화된 파일을 다운로드하는 클래스의 전체 경로. websquare.download.handl.AbstractDownloadEncrypter14 를 상속. - JDK 1.4 DRM과 연계.

항목	설명
userAgent	Excel 파일 다운로드 중 요청이 2번 발생하는 버그가 발생하는 일부 Android 장치에 agent의 패턴. Android 4.0 기본브라우저와 AndroidDownloadManager 사용시 발생.
keyMode	모바일에서 Excel 파일 다운로드 request를 위한 ID 생성 규칙. (기본값은 없음.) random을 지정하는 경우 ID는 "WEBSQUARE_EXCEL_INFO" + "request.getParameter("randomKey")로 생성.
keyMode@preFix	request ID는 기본적으로 "WEBSQUARE_EXCEL_INFO"로 생성되는데, 이를 변경할 경우 prefix에 지정.
definer	중복호출 시 Excel 파일 정보를 get/set하는 클래스의 전체 경로. (기본적으로 중복호출 시 Excel 파일 정보를 참조하기 위해 세션에 request ID를 저장합니다. Excel 정보를 다른 곳으로부터 참조할 경우 이같은 클래스를 구현할 수 있습니다.) 해당 클래스는 websquare.download.handl.AbstractExcelDownload를 상속
rowAccessWindowSize	POI 3.8부터 대량 Excel 파일 전송 기능이 추가되어, 대용량 그리드의 경우 그리드에 Excel 데이터를 쓴 후 메모리에 올라간 Excel 생성 정보를 flush 할 수 있습니다. - 기본값: 100 (100개의 행당 flush됨.) - 각 사이트의 특성에 따라 가감한 숫자를 지정.

Code 35-24 **websquare.xml** 설정 예

```

1  <WebSquare>
2      <excel>
3          <download>
4              <encrypt value="com.inswave.Encrypt"/>
5  </mobile>
6      <keyMode value="" preFix="" />
7      <definer value="" />
8      <userAgent
value="\w\W]+Android[\w\W]+Version/4.0[\w\W]+|[\w\W]*AndroidDownloadManager[\w\W]*" />
9  </mobile>
10 </download>
11 </excel>
12 </WebSquare>

```

35.2.3.1.2. encrypt 구현

DRM 암호화가 필요한 경우, DRM 암호화 클래스를 아래와 같이 구현하십시오.

1. **websquare.upload.handl.AbstractDownloadEncrypter** 를 상속합니다.
2. 파일을 암호화하여 stream에 file을 write 합니다.

Code 35-25 encrypt 구현 예

```

1  import java.io.BufferedInputStream;
2
3  import java.io.File;
4  import java.io.FileInputStream;
5  import java.io.FileOutputStream;
6  import java.io.IOException;
7  import java.io.InputStream;
8  import java.io.OutputStream;
9
10 import javax.servlet.http.HttpServletRequest;
11 import javax.servlet.http.HttpServletResponse;
12
13 import websquare.download.handl.AbstractDownloadEncrypter;
14
15 import com.fasoo.adk.packager.*;
16
17 public class ExcelDownEncrypterWithFasoo extends AbstractDownloadEncrypter {
18     /**
19      * @param <BufferedInputStream:Y> is          : Excel Data
20      * @param <HttpServletRequest:Y> request      : HttpServletRequest
21      * @param <HttpServletResponse:Y> response    : HttpServletResponse
22      * @desc WebSquare GridView -> 엑셀 다운로드시 암호화 연계. is을 암호화 하여 response에
write처리하도록 구현한다.
23      */
24     public void encrypt(BufferedInputStream is, HttpServletRequest request,
HttpServletResponse response) throws Exception {
25         // 1. 클라이언트에서 upload한 파일을 복호화 연계 처리를 구현 한다.
26         // 2. 복호화된 파일을 return 한다.
27         OutputStream outputStream = null;
28         InputStream inputStream = null;
29         FileOutputStream fos = null;
30
31         String excelFileName = null;
32
33         String strFsdinitDir = "/TMAX/wq/FSD/fsdinit"; // 설정파일경로
34         String serverid = "0123456789"; // 사이 고유 코드
35         String extractFilePath = "/TMAX/wq/tempUpload/excel/extractFilePath"; //
복호화파일경로
36         String packageFilePath = "/TMAX/wq/tempUpload/excel/packageFilePath"; //
암호화파일경로
37

```

```

38     String oUserID          = "admin";
39     String oUserName        = "관리자";
40     String oWriterID        = "admin";
41     String oWriterName      = "관리자";
42     String oWriterDeptCode   = "COMPANY";
43     String oWriterDeptName   = "사이트명";
44     String oOwnerID         = "admin";
45     String oOwnerName        = "관리자";
46     String oOwnerDeptCode    = "COMPANY";
47     String oOwnerDeptName    = "사이트명";
48     String SecurityLevel     = "1";
49
50     try {
51         // 1. WebSquare excel 생성(is 를 직접 읽어 DRM솔루션 업체에서 직접 엑셀 생성이
가능하면 1.0 은 필요없다.)
52         excelFileName = "tempExcel_" + System.currentTimeMillis() + ".xlsx";
53
54         File file = new File(extractFilePath + File.separator + excelFileName);
55         fos = new FileOutputStream(file);
56
57         byte[] readBuff = new byte[4096];
58         int readLen = -1;
59
60         try {
61             while ((readLen = is.read(readBuff)) != -1) {
62                 fos.write(readBuff, 0, readLen);
63             }
64         } finally {
65             if (fos != null) {
66                 try {
67                     fos.flush();
68                     fos.close();
69                 } catch (IOException e) {
70                     e.printStackTrace();
71                 }
72             }
73         }
74
75         file.deleteOnExit();
76
77         // 2. 1에서 생성한 엑셀 파일을 파수연계 암호화 처리

```

```

78     WorkPackager oWorkPackager = new WorkPackager();
79
80     boolean bret = oWorkPackager.DoPackagingFsn2(
81         strFsdinitDir,                // FSD 홈디렉토리
82         serverid,                    // Server ID
83         extractFilePath + File.separator + excelFileName,    // 암호화를 하고자 하는
원본 파일
84         packageFilePath + File.separator + excelFileName,    // 암호화 후 생성 하고자
하는 파일
85         excelFileName,                // 원본 파일명
86         oUserID,                      // 생성자(등록자) ID
87         oUserName,                    // 생성자(등록자) 이름
88         oWriterID,                    // 작성자 ID
89         oWriterName,                  // 작성자 Name
90         oWriterDeptCode,              // 작성자 부서코드
91         oWriterDeptName,              // 작성자 부서명
92         oOwnerId,                     // 소유자 ID
93         oOwnerName,                   // 소유자 Name
94         oOwnerDeptCode,               // 소유자 부서코드
95         oOwnerDeptName,               // 소유자 부서명
96         SecurityLevel                 // 보안등급
97     );
98
99     // 3. 다운로드 시작
100    if (bret) {
101        File encFile = new File(packageFilePath + File.separator + excelFileName);
102
103        outputStream = response.getOutputStream();
104        inputStream = new FileInputStream(encFile);
105
106        byte[] writeBuff = new byte[4096];
107        int writeLen = -1;
108
109        response.setHeader("Content-Length", encFile.length() + "");
110
111        while ((writeLen = inputStream.read(writeBuff)) != -1) {
112            response.getOutputStream().write(writeBuff, 0, writeLen);
113        }
114
115        encFile.deleteOnExit();
116    } else {

```

```

117         System.out.println("암호화 중 오류입니다.");
118         System.out.println("오류 정보..");
119         System.out.println("[ "+ oWorkPackager.getLastErrorNum()+"]
"+oWorkPackager.getLastErrorStr());
120     }
121     } catch (Exception e) {
122         e.printStackTrace();
123     } finally {
124         if (fos != null) {
125             try {fos.close();} catch (Exception ex) {ex.printStackTrace();}
126         }
127         if (inputStream != null) {
128             try {inputStream.close();} catch (IOException ei) {ei.printStackTrace();}
129         }
130         if (outputStream != null) {
131             try {
132                 outputStream.flush();
133                 outputStream.close();
134             } catch (IOException eio) {eio.printStackTrace();}
135         }
136     }
137 }
138 }

```

35.2.3.2. CSV

GridView 컴포넌트에서 CSV 파일을 다운로드할 경우, **WEBSQUARE_HOME** 의 **config** 폴더에 있는 **websquare.xml** 에 아래의 설정을 추가하십시오.

Table 35-7 CSV 파일 다운로드를 위한 **websquare.xml** 설정 항목

항목	설명
encrypt	CSV 파일을 DRM으로 암호화 후 stream으로 write하는 클래스의 전체 경로. • websquare.download.handl.AbstractDownloadEncrypter 를 상속. • JDK 1.5 이상 DRM과 연계. (35.2.3.1.2. encrypt 구현 참조.)

Code 35-26 **websquare.xml** 설정 예

```

1 <WebSquare>
2     <csv>
3         <encoding value="UTF-8" />

```



```
4         <useBOM value="true" />
5 <download>
6         <encry value="com.inswave.Encrypt"/>
7     </upload>
8 </csv>
9 </WebSquare>
```


36. Workflow 가이드

36.1 개요

정의

직관적인 GUI를 통해 다수의 복잡한 비동기 통신을 쉽게 호출할 수 있는 기능입니다. 사용자는 각 비동기 호출의 결과에 대한 처리 로직만 간단히 구현하면 됩니다.

용어

Workflow: 여러 건의 Submission을 처리하는 순서를 정의한 것

WorkflowCollection: Workflow의 그룹

Step: 여러 건의 작업 처리 순서를 정의한 것

특징

비동기 통신 프로그래밍이 용이

+Promise workflow 사용이 용이

Submission의 비동기 처리가 용이

기능

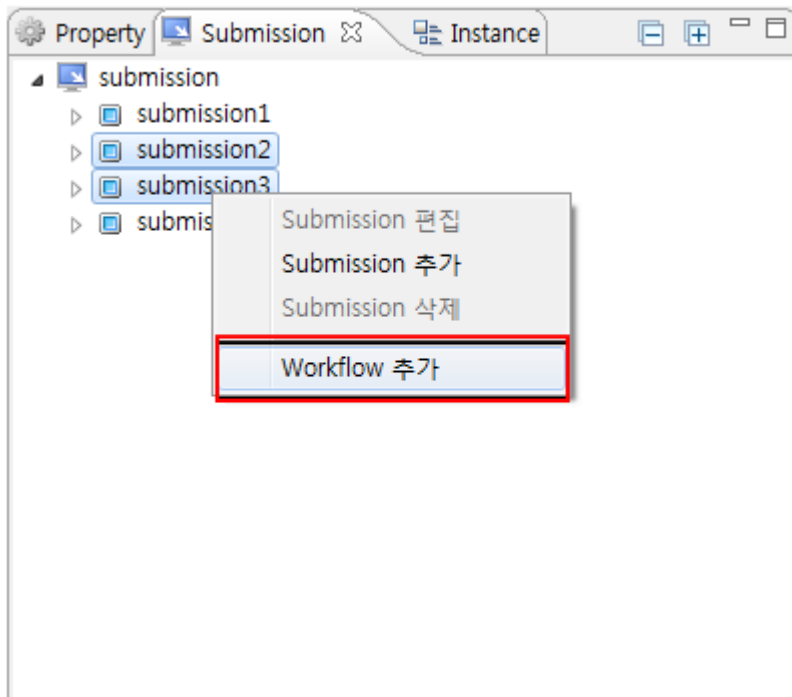
정의된 submission 및 submissionCallback의 실행순서를 보장합니다. (실행 순서에 대한 제약은 없음.)

실행 순서의 유효성을 확인합니다.

36.2 생성 방법

1. Submission 뷰를 클릭합니다.
2. Workflow에 포함할 Submission들을 우클릭한 후 [Workflow 추가]를 선택합니다.
(혹은 Component 뷰에서 WorkflowCollection 아이콘을 우클릭한 후 [Workflow 추가]를 선택합니다.)

그림 36-1 Submission View의 Workflow 추가 메뉴 화면



3. Workflow Manager 창에서 Workflow의 기본 정보를 설정하고 및 Service List를 생성합니다.

그림 36-2 Workflow Manager 화면

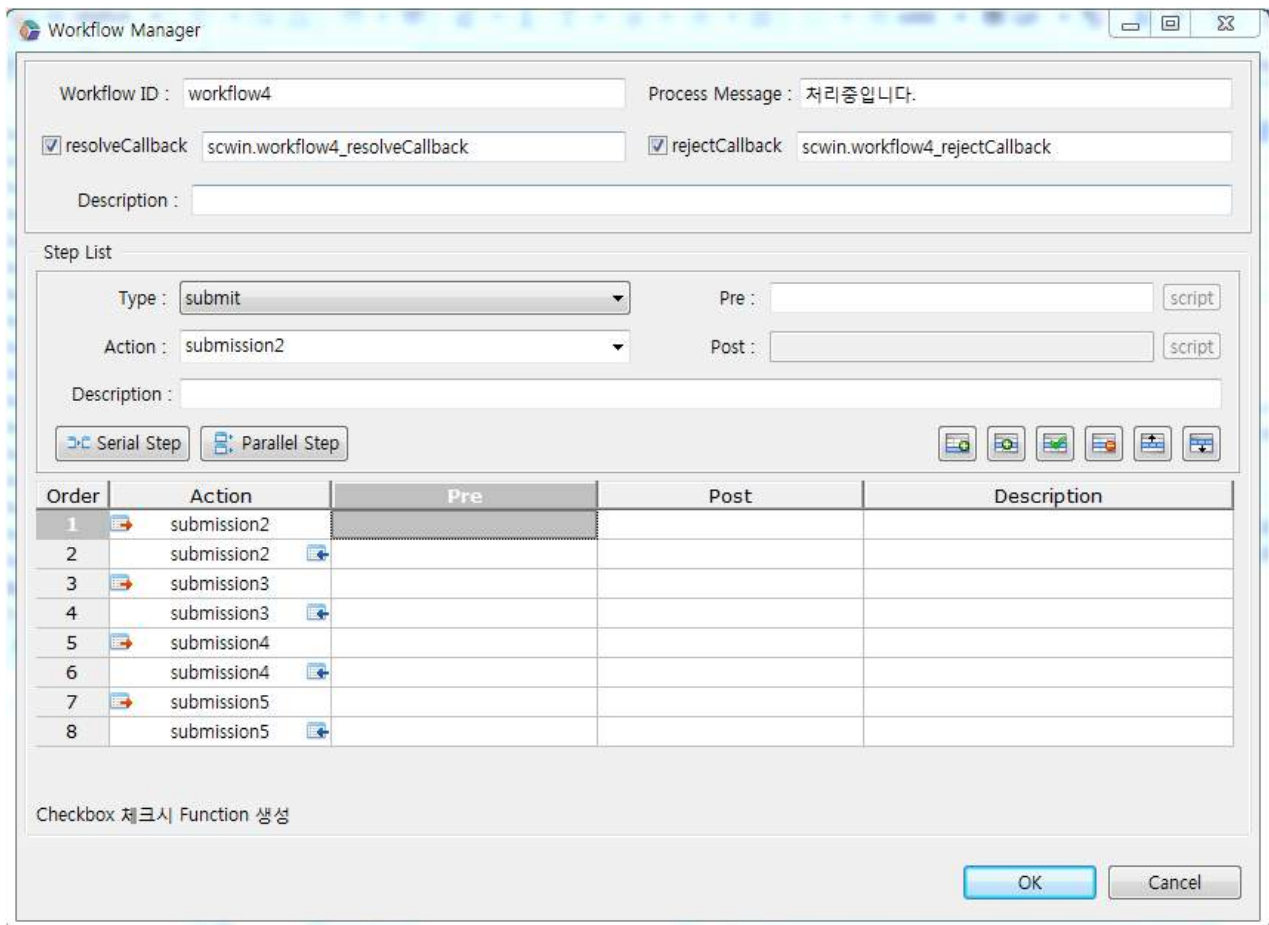


Table 36-1 Workflow 입력 항목

항목		설명
Workflow ID		workflow ID (자동 부여)
Process Msg		workflow 기동 시 표현될 메시지
resolveCallback		작업 완료 후 호출할 함수의 이름
rejectCallback		작업 처리 도중 에러 발생시 또는 workflowObj.reject() 함수가 호출된 다음 호출할 함수의 이름
Step List	Type	작업 종류 (기본값: submit) function은 실행 전에 처리할 내용을 작성합니다.
	id	type 선택에 따라 선택 및 입력 가능
	Function	type이 'function' 인 경우 입력 가능
	Pre	type이 'submit' 인 경우 선행처리용 함수명을 입력합니다. (Script 버튼을 클릭하면 해당 함수가 Script 탭 에 자동 생성됩니다.)
	Post	type이 'submitDone' 인 경우 선행처리용 함수명을 입력합니다. (Script 버튼을 클릭하면 해당 함수가 Script 탭 에 자동 생성됩니다.)
	Serial	등록된 Step을 직렬화 시킵니다.
	Parallel	등록된 Step을 병렬화 시킵니다. 우측 상단 아이콘을 이용하여 Step을 추가, 삭제, 수정, 이동할 수 있습니다.

4. [Script 탭](#)을 클릭합니다.
5. **resolveCallback** 및 **rejectCallback** 함수를 정의합니다.

36.3 관련 API

Table 36-2 `$p.workflow` API

API	설명
getWorkflow(id)	workflow Result 객체를 반환
executeWorkflow(id)	model에 선언된 workflow id를 지정하여 WebSquare workflow를 실행
executeWorkflow(json)	workflow json을 인자로 전달하여 WebSquare workflow 실행
getRunningWorkflow()	실행중인 workflow 객체 반환id 가 인자로 넘어온 경우에는 실행중인 workflowID와 동일한 경우에만 반환
isRunningWorkflow(id)	workflow 가 실행 중인지 여부 true/false 반환id 가 인자로 넘어온 경우에는 실행중인 workflowID와 동일한 경우에만 true 반환
getRunningWorkflowID()	실행중인 workflow id 반환
rejectWorkflow(msg,id)	실행중인 workflow를 reject 시킴
executeSerial(array,processMsg,resolveCallback,rejectCallback)	
	: workflow 생성없이 array 의 submission 을 serial 하게 실행
executeParallel(array,processMsg,resolveCallback,rejectCallback)	

API	설명
	: workflow 생성없이 array 의 submission 을 parallel 하게 실행
<code>executeWhilst(condFn,maxCount, submissionId,processMsg,resolveCallback,rejectCallback)</code>	
	: dataList의 자료를 add 하는 경우 사용, dataList add 기능 선택인 필요
<code>reject()</code>	workflow를 중단한다. <code>reject()</code> 가 호출된 이후 workflow는 실행하지 않고 <code>rejectCallback</code> 으로 이동
<code>result</code>	실행된 submissionId별 결과객체(e) 관리process 가 종료되는 시점에 삭제
<code>executeWorkflow(id)</code>	model에 선언된 workflow id를 지정하여 WebSquare workflow 실행
<code>executeWorkflow(json)</code>	workflow json을 인자로 전달하여 WebSquare workflow를 실행
<code>executeSerial(array, resolveCallback, rejectCallback)</code>	array의 submission을 serial 하게 실행
<code>executeParallel(array, resolveCallback, rejectCallback)</code>	array의 submission을 parallel하게 실행



각 API에 대한 좀 더 자세한 설명 및 사용법은 [API 가이드](#)를 참고하십시오.

37. 비권장 사례 모음

37.1 일반

37.1.1. 변수명

영문으로 시작하는 변수명을 사용하십시오.

Code 37-1 비권장 사용 예

```
1 <w2:anchor class="todolist_refresh" id="1" outerDiv="true">
```

Code 37-2 권장 사용 예

```
1 <w2:anchor class="todolist_refresh" id="anchor01" outerDiv="true">
2
3 // 혹은 id 삭제 가능
4
5 <w2:anchor class="todolist_refresh" outerDiv="true">
```

37.1.2. 객체 제어

JavaScript는 느슨한 타입(loosely typed) 언어, 혹은 동적(dynamic) 언어입니다. 즉, 변수 타입을 미리 선언할 필요가 없고 프로그램이 처리되는 과정에서 자동으로 파악 변환되기도 합니다. 공통함수가 객체 타입에 따른 대응을 못하고 에러가 발생하는 경우 스크립트가 중지되고 브라우저에 따라 성능 및 메모리에 영향을 받습니다.

Code 37-3 비권장 사용 예

```
1 scwin.addDateSimpleisDate = function(sDate) {
2     try {
3         var date = " ";
4         var isDate = scwin.isDate(pYmd); //8자리 이상의 string여부 체크
5         if (!isDate) return;
6         cYmd = pYmd.substring(0, 8);
7         var dTime = (pYmd+ " ").substring(8);
8         var isSet = Number(offset); //isDate에서 숫자인지 체크하지는 않음
9         if (isDate && typeof isSet == "number") date = $p.dateAdd(cYmd, isSet);
10        if (typeof format != "undefined" && format == "yyyyMMdd ") {
```

```

11         date = date;
12     } else {
13         date = date + " " + dTime;
14     }
15     return date;
16 } catch (e) { }
17 }

```

Code 37-4 권장 사용 예

```

1  scwin.addDateSimpleisDate = function(sDate) {
2      try {
3          var date = "";
4          var isDate = scwin.isDate(pYmd);
5          if (!isDate) return;
6          var cYmd = pYmd.substring(0, 8);
7          var dTime = pYmd.substring(8);
8          var isSet = Number(offset);
9          if (!isNaN(isSet)) date = $p.dateAdd(cYmd, isSet);
10         // Number 변환시 NaN가 아닌 경우 dateAdd 수행
11         if (typeof format == "undefined" || format != "yyyyMMdd")
12             date = date + dTime;
13         return date
14     } catch (e) { }
15 }

```

37.1.3. 전역 변수 선언

JavaScript는 느슨한 타입(loosely typed) 언어로 변수 선언을 위해 사용하는 `var` 문자를 누락해도 스크립트는 정상적으로 수행됩니다. 단, 이 경우 해당 변수는 전역에 선언되어, 함수 종료 후에도 계속 참조가 가능하고 자동 릴리즈가 되지 않습니다. SPA 방식으로 이동하는 화면인 경우 이는 메모리 누수 현상으로 이어집니다.

Code 37-5 비권장 사용 예

```

1  scwin.getDay = function(pYmd, format) {
2      try {
3          if (pYmd.length > 8) {
4              pYmd = pYmd.substring(0, 8);
5          }
6          if (typeof format !== "undefined" && format !== "" &&
7              WebSquare.util.getBoolean(format)) {

```



```

8      date = WebSquare.date.getDay(pYmd);
9      //date가 var 선언이 생략되어 있어 전역변수로 사용됨
10   } else {
11       // ...
12   }
13   return date;
14 } catch (e) {
15 }
16 }

```

Code 37-6 권장 사용 예

```

1  scwin.getDay = function(pYmd, format) {
2      try {
3          var date = -1;
4          //var를 생략하지 않고
5          // date가 가질 수 없는 값을 초기값으로 설정한뒤
6          // case에 따른 수식결과를 저장한뒤 리턴한다.
7          if (pYmd.length > 8) {
8              pYmd = pYmd.substring(0, 8);
9          }
10         if (typeof format !== "undefined" && format !== "" &&
11             WebSquare.util.getBoolean(format)) {
12             date = WebSquare.date.getDay(pYmd);
13         } else {
14             // ...
15         }
16         return date;
17     } catch (e) {
18     }
19 }

```

37.1.4. 동기 요청 최소화

대부분의 프로젝트에서는 현재 시간 (서버 시간), 사용자 정보, 또는 사용자 권한을 조회하기 위해 동기 요청을 사용합니다. 한 화면에서 동기 요청을 여러 번 호출하는 것은 성능 저하로 이어질 수 있습니다.

하나의 요청으로 처리 가능한 함수를 구현하십시오.

37.1.5. 최상위 scope에 함수 및 변수 선언 제한

- WebSquare에서는 업무 화면에서 함수 또는 상수 값 선언 시 객체의 속성으로 설정하여 사용할 것을 권장합니다.
- 화면 unload 시점에 선언된 객체의 속성으로 등록된 함수 및 상수를 제거하여 메모리 누수를 방지하십시오.
- 업무화면 내에서 사용하는 전역 변수 또한 지정된 객체의 속성으로 등록해서 사용하십시오.

Code 37-7 비권장 사용 예제

```
1 var GlobalValue = 1;
2
3 function doInit() {
4     //Something
5 }
```

Code 37-8 권장 사용 예제

```
1 scvar.GlobalValue = 1;
2
3 scwin.doInit = function() {
4     //Something
5 }
```

37.1.6. 금지 사항

웹스퀘어를 통한 웹 표준 개발 시, 아래 내용을 준수하십시오.

- 브라우저 객체(DOM)를 직접 제어하는 모든 API의 사용을 **금지합니다**.
 - 예를 들어 웹스퀘어에서 제공된 컴포넌트로 그려진 객체(DOM)에 대해 `getElementById();` 혹은 `createElement();` 함수를 사용하지 마십시오.
 - `document.write` 와 같은 함수를 사용하지 마십시오.
- 브라우저 Event의 직접 제어를 **금지합니다**.
- XMLHttpRequest를 직접 생성하고 조작하는 API는 사용하지 마십시오.
- 웹스퀘어가 제공하는 Submission을 이용하십시오. API를 이용할 경우, `WebSquare.ModelUtil.executeSubmission();`을 이용하십시오.
- prototype.js나 jquery.js와 같은 오픈소스 JavaScript Framework를 **동시에 사용할 수 없습니다**.
- prototype **확장은 금지합니다**.



프로그램 구현을 위해 아래 내용이 꼭 필요한 경우, 웹스퀘어 지원 인력과 먼저 확인하십시오.

37.2 메모리

최상의 메모리 및 성능을 위해 아래 내용을 확인하십시오.

번호	확인 사항	설명
1	Window 팝업을 alert이나 iframe형태의 팝업으로 처리했는가?	2단 팝업 등에서 메모리 미반환 현상 발생.
2	통신 관련 공통 함수가 객체 대신 string을 사용했는가?	callback 함수에 객체 형태로 전달하면 메모리 누수가 발생. 예) IE8 브라우저 다운현상이 발생할 수 있음.
3	팝업으로 인자 전달 시에 객체 대신 string 을 사용했는가?	객체 형태로 전달시 메모리 누수 발생.
4	웹 폰트 사용시 부모 페이지 외의 자식 페이지에서 중복 호출되지는 않는가?	중복 호출 시 IE8 등에서 백화현상 또는 웹 폰트 미적용 현상 발생.
5	외부 모듈 연계시 document mode(DocType)의 지원 범위가 표준을 준수하는가?	외부모듈 지원 시 DocType이 비표준 모드인 경우 브라우저 하위버전으로 설정되어 렌더링 속도의 영향을 미침.
6	외부 모듈 연계 시 해당 외부 모듈의 메모리 누수 여부를 확인했는가?	브라우저 전체 메모리에 영향을 미침.
7	외부 모듈(bootstrap 사용시)의 버전 별 메모리 해제 여부를 확인했는가?	bootstrap-2.1.1.min.js 해당버전이 메모리 해제가 되지 않는 문제 발생.
8	SSL 환경에서 멀티업로드 (플래시 모드)를 사용하는가?	업로드 성능이 저하되는 현상이 발견. html5_transparent 모드를 권장. (IE 10 이상만 사용 가능)
9	대용량 엑셀 다운로드가 있는가?	Client 환경에서 Data를 수집하고 Server로 보내는 경우 시간이 오래 소요될 수 있음. 권장: Server에서 Data를 직접 처리하도록 Provider를 사용할 것을 권장.
10	IE8을 지원해야 하는 프로젝트에서 웹 폰트를 사용하	IE8의 경우 웹 폰트를 내려 받기 위한 브라우저 처리 속도가 현저하게 느리기 때문에 웹 폰트 사용을 권장하지 않음.

번호	확인 사항	설명
	는가?	예) ie8 + iframe + 웹 폰트 사용시 백화 현상이 발생할 수 있음.
11	캐시가 적절히 사용되고 있는지 확인했는가? (웹스퀘어 환경설정, 웹 서버 만료일 설정, 브라우저의 캐시 설정 등)	웹스퀘어 config.xml 내의 캐시 관련 설정(웹서버 만료일, 브라우저 캐시 설정 등 관련)이 웹페이지 속도에 큰 영향을 미침.
12	DataCollection의 Data를 처리하기 위한 API들이 효율적으로 사용되었는가?	Data 처리를 비효율적으로 API를 반복하여 사용하는 경우 성능 저하 발생. 예) 1개의 Row를 변경하기 위해 <code>setRowJson</code> 을 사용하지 않고 각 셀에 대해 <code>setCellData</code> 를 반복적으로 사용.
13	대량 Data 처리시 Data 구조체를 적절하게 사용했는가?	array > json > xml 순으로 속도 처리가 빠름.
14	Fasoo DRM 모듈을 사용하는가?	Fasoo DRM Client for LIG NoNX Wix, FSW Client 류의 제품들이 메모리 사용시 팝업을 열고 닫을 때 메모리 사용량이 증가할 수 있음.
15	그리드의 Cell 별로 다양한 서식을 적용하는 사례가 있는가?	GridView의 formatter 및 <code>setCellColor</code> , <code>setCellBackgroundColor</code> 등이 붙을 수록 화면 렌더링 속도 및 스크롤 속도가 느려질 수 있음. 권장: 서버에서 서식을 처리하여 내려주도록 권장.
16	TabControl 사용 시 다수의 탭을 사용하는가?	다수의 탭 중 닫혀진 탭 내부 콘텐츠를 렌더링 하는 방법이 <code>alwaysDraw="true"</code> 인 경우, 최초 로딩 시 다수의 탭이 있을수록 화면 로딩 속도가 저하. 권장: <code>alwaysDraw="false"</code> 를 사용하여 탭이 선택될 때 로딩하는 방식 권장.
17	TabControl 내부 Content 항목의 화면 구성 시 별도의 xml 파일 연결을 확인했는가?	TabControl 내부 Content 항목이 iframe을 포함하여 iframe이 중복으로 사용되는 현상 발생. 권장: TabControl 내의 content 속성 중 src 에서 별도 xml 파일을 선택하도록 해야함.
18	JavaScript 함수 작성 시 지역변수를 적절하게 사용했는가?	함수 내에서 변수 선언시 var 선언을 생략하면 전역변수로 생성되어 메모리 과다 점유 현상 발생.
19	전체 Layout 화면 구성 시 iframe 개수가 지나치게 많이 사용되지는 않았는가?	iframe 사용개수가 많아질수록 화면 렌더링 속도 저하 및 메모리 과다 점유 현상 발생. 권장: 가능한 iframe을 wframe로 변환해서 처리할 수 있도록 권장
20	sync 및 async 통신 방식이 적절히 사용되었는가?	Sync 방식은 완료시까지 스크립트 정지 현상이 발생하므로 async 방식을 권장.
21	MDI(WindowContainer) 사용시 화면 중복 오픈 금지 및 오픈 창 개수 제한이 있는가?	MDI 개별 Window가 지나치게 많아지게 될 경우 메모리 과다로 인해 화면 조작이 불가.

번호	확인 사항	설명
	는가?	
22	JavaScript 로직 내의 오류가 발생하는가?	JS 내 오류 발생시 화면 렌더링 속도 저하.
23	xml 파일 내의 업무 로직 처리를 위한 JavaScript 분량이 적절한가?	하나의 xml 파일 내의 JavaScript 양이 많을 경우 렌더링 속도 저하. 권장: 공통으로 사용되는 함수 등은 xml 파일 별로 넣지 않고 별도의 .js 파일로 분리하여 관리.
24	(갤럭시노트3, 갤럭시탭10.1과 같은) 특정 모바일 장비를 사용하는가?	해당 장비에서는 자체 xml parser가 내장되어 있지 않음 config.xml에서 <code><userAgentPattern XPathParser="Android 4.3[WwWW]+N900 Android 4.3[WwWW]+E300 Android 4.4"/></code> 을 추가. <code>/websquare/externalJS/xpath/xpath.js</code> 를 로딩하여 사용 시 성능저하 발생. 권장: 근본적인 처리 방안 없음으로 가급적 해당 기기를 사용하지 않을 것.
25	jQuery 1.7.2를 사용하는가?	해당 버전 사용시 메모리 누수 발생.
26	통신 객체의 동적 생성시 중복 문제는 없는가?	동일한 id로 중복하여 생성할 경우 누수 발생.
27	데이터 객체의 동적 생성시 중복 문제는 없는가?	동일한 id로 중복하여 생성할 경우 누수 발생.
28	통신 함수의 응답 객체 사용에 문제가 없는가?	통신 함수에서 사용자 함수 호출 시 응답 결과가 다른 곳과 참조 관계가 없는지 확인.
29	컴포넌트 및 함수의 네이밍은 문제가 없는가?	가이드된 네이밍 규칙을 준수하는지 확인.
30	팝업(프레임)간 Object를 전달하고 있는가?	타 프레임에 Primitives형(예: String)이 아닌 Object를 직접전달하고 있는지 확인.
31	스크립트 오류가 발생하지 않는가?	스크립트 수행이 중지되어 함수가 정상적으로 종료되지 않는 경우가 있는지 확인.
32	웹폰트 사용시 부모 페이지 외의 자식 페이지에서 중복 호출되는가?	중복 호출 시 IE8 등에서 백화 현상 또는 웹 폰트 미적용 현상 발생.
33	외부 모듈 연계시 document mode(DocType)의 지원 범위가 표준을 준수하는가?	외부 모듈 지원 시 DocType이 비표준 모드인 경우 브라우저 하위 버전으로 설정되어 렌더링 속도의 영향을 미침.
34	외부 모듈 연계 시 해당 외부 모듈의 메모리 누수 여부를 확인했는가?	특히 SPA기능 사용 시 외부 모듈 추가 로드 또는 사용 완료 시에 메모리에 사용에 문제가 없는지 확인.

번호	확인 사항	설명
35	캐시가 적절히 사용되고 있는지 확인했는가?	개발/운영 환경에 따라 웹스퀘어 <i>config.xml</i> 내의 캐시 관련 설정이 적용되어 있는지와 서버 설정을 확인. (예) 웹스퀘어 환경설정, 웹 서버 만료일 설정, 브라우저의 캐시 설정 등
36	그리드의 Cell 별로 다양한 서식을 적용하는 사례가 있는가?	한 Row에 셀표현 방식 변경을 위해 formatter류가 많이 사용된 경우 렌더링 속도에 영향을 줄 수 있으므로 Row가 많은 경우는 Server에서 서식을 적용하는 것을 권장.
37	JavaScript 함수 작성 시 지역변수를 적절하게 사용했는가?	var 누락 또는 가이드에 따라 변수/함수가 지정되어 있는지 확인.

37.3 모바일

37.3.1. 여러가지 해상도 대응은 viewport나 css3의 @media를 사용

모바일 기기의 해상도가 다양하므로 @media속성을 이용해 최적화된 css를 구성하여 각 기기에 맞는 화면을 구성하십시오.

```

1  .dynamicScreen {
2    font-family: Arial, Helvetica, sans-serif;
3    height: 250px;
4    color: black;
5  }
6
7  @media all and (max-width: 320px) {
8    /* 아이폰/아이팟터치/안드로이드의 초기 스크린 해상도 설정 */
9    .dynamicScreen {background-color: orange;width:320px; }
10 }
11
12 @media all and (max-width: 768px) {
13   /* 아이패드 초기 해상도 설정 */
14   .dynamicScreen {background-color: blue;}
15 }
16

```

```

17 @media all and (min-width: 769px) {
18     /* 아이폰보다 큰 스크린 사이즈와 해상도를 갖는 모든 기기에서의 해상도 설정*/
19     .dynamicScreen {background-color: green;}
20 }

```

meta태그의 viewport를 이용해서 기기의 최대가로크기를 정하거나 줌레벨, 사용자 확대축소 방지 여부 등을 설정할 수 있습니다. meta 태그는 websquare.html(jsp)에 넣어야 합니다.

```

1 <meta name= " viewport " content= " width=device-width " />
2 <meta name= " viewport " content= " initial-sacle=1,user-scalable=no " />

```

37.3.2. 마우스나 키보드 사용을 전제하지 말 것

각종 모바일 기기에 따라 키보드나 마우스 같은 장치 유무가 다르기 때문에 마우스나 키보드 사용을 전제하지 마십시오.

37.3.3. 스크롤 사용시 주의 사항

모바일에서 body를 제외하고, overflow:hidden으로 사용하는 것을 권장합니다. 스크롤이 필요한 경우 scrollView 컴포넌트를 사용하여 스크롤을 사용할 수 있습니다.

37.3.4. 이벤트 사용시 주의

스마트 폰들은 기본적으로 터치 기반입니다. 더블-클릭 등의 이벤트를 사용하지 마십시오. 마우스 이벤트의 경우 기기마다 다르게 동작할 수 있습니다.

37.3.5. 파일 첨부 제한

iPhone의 경우 보안을 위해 허용된 디렉터리의 파일만 첨부 가능합니다.

37.3.6. 플러그인 기술 사용금지

기기에 따라 flash나 silverlight 등의 플러그인 사용이 제한됩니다.

37.3.7. 모바일 화면 구성 시 iframe 사용 금지

기기에 따라 IFrame으로 화면을 구성 하게 되면, focus, blur 관련 동작이 일반 브라우저와 다르게 동작합니다. 경우에 따라 Input 간 포커싱이 정상적으로 동작하지 않을 수 있습니다.

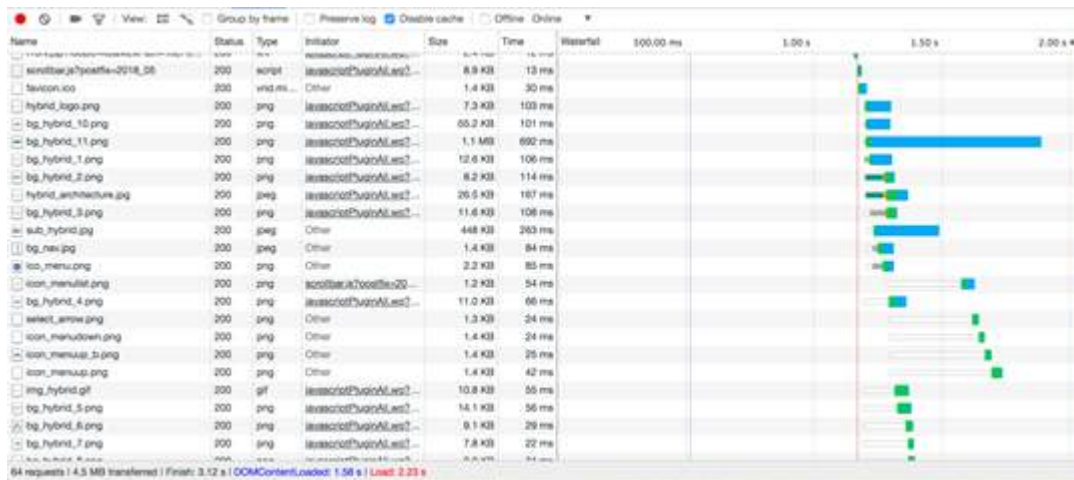
37.3.8. 팝업 사용시 주의 사항

모바일의 경우 팝업을 권장하지 않습니다. 팝업은 새 창(window.open)으로 생성되어 화면 전환이 이루어집니다. 대신 레이어 형태어 팝업 및 링크를 사용하십시오.

37.3.9. 사용 금지 컴포넌트

- ifame
- xsl
- ckEditor
- chart(iphone에서만 동작함)
- upload
- muliupload
- flash
- applet
- active
- multiselect
- slideHide
- tabContainer
- windowContainer

37.4 네트워크



- 확인된 가장 일반적인 문제는 큐에 저장되거나 지연된 일련의 항목으로 이런 현상이 나타나면 단일 클라이언트에서 너무 많은 리소스를 가져오고 있다는 의미입니다.
- HTTP 1.0/1.1 연결의 경우, Chrome은 호스트당 최대 6개의 TCP 연결을 적용합니다. 한번에 12개의 항목을 요청하는 경우, 첫 6개부터 먼저 시작되고 나머지 절반은 큐에 저장됩니다.
- 첫 절반 중 하나가 완료되면 큐에 대개중인 첫 번째 항목이 요청 프로세스를 시작합니다.

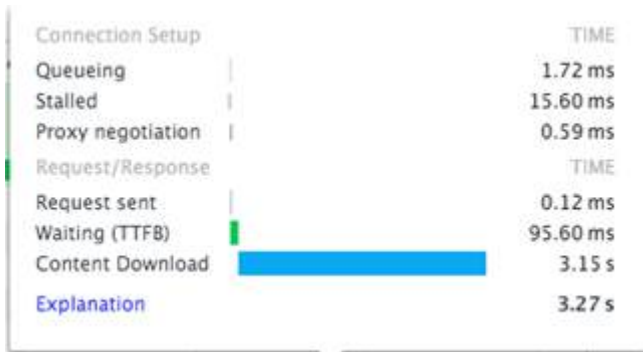
37.4.1. 느린 TTFB



느린 TTFB는 긴 대기 시간 즉 녹색이 많이 나타나는 경우입니다. 이 시간은 200ms 미만이 좋습니다. 이 시간이 길다면, 그 원인은 아래와 같습니다.

1. 클라이언트와 서버 사이의 네트워크 상태가 잘못된 경우
2. 서버 애플리케이션의 응답이 느린 경우

37.4.2. 처리 용량 확보



37.4.3. 네트워크 문제 해결

Table 37-1 해결 방안

해결 방안	설명
네트워크 요청 횟수 튜닝	- js: 공통으로 제공되는 js 파일이 여러개로 분리된 경우, 하나의 파일로 합쳐서 호출. minify하여 js의 용량을 줄이는 것도 좋은 방안임. - CSS Sprites: 아이콘등 여러 이미지를 개별로 요청하지 않고 한 이미지 파일로 통합한 후 Background로 Position을 잡아주어 한 이미지만 로딩하면 트래픽 절약이 가능.
브라우저 기본 폰트 사용	웹폰트를 사용하는 경우 웹폰트 리소스 다운로드를 위한 트래픽과 폰트 사용을 위한 약간의 지연이 브라우저에서 발생 가능. 반드시 사용해야 한다면, 용량이 적은 웹폰트를 권장.
시스템 사용을 위한 외부 프레임워크 연계	- 시스템 사용 시 외부 프레임워크는 초기 화면이 아닌 필요한 시점에 로드하는 것을 권장. 로딩 부하를 줄일 수 있음. (예) 보안, 인증, 레포트
단일 frame 사용	- 화면 구성 프레임(frameset, iframe)은 하나로 처리하는 것을 권장. - 각 프레임은 각각의 윈도우 객체를 소유하고 웹스퀘어 엔진이나 공통 JS와 같은 리소스를 요청.

37.5 기타

37.5.1. `$p.ajax`

37.5.1.1. 응답 객체의 직접 할당 금지

WebSquare는 Submission을 이용한 통신을 권장합니다. 그러나 사용자가 직접 통신 관련 설정을 해야하는 경우를 위해 `$p.ajax` API를 제공합니다.

아래 예제의 경우, 사용자가 직접 제어할 수 있도록 응답 결과 객체를 반환하고 있습니다. 그러나 업무 화면의 콜백 함수로 그대로 넘기기 때문에 업무 화면에서 변수 등에 직접 할당할 경우 통신 완료 후 응답 객체가 메모리에서 릴리즈 되지 않을 수 있습니다.

Code 37-9 비권장 사용 예

```

1  $p.ajax({
2      action : action,
3      mode : mode,
4      mediatype : mediatype,
5      method : method,
6      processMsg : processMsg,
7      requestData : reqData,
8      requestHeader : requestHeader,
9      beforeAjax : function(e) {
10     },
11     success : function(e) {
12         var resBody = e.responseBody;
13         if (callbackFnc != '') {
14             eval(callbackFnc + '(resBody)');
15         }
16     },
17     error : function(e) {
18         return _requestErrCallback(e);
19     }
20 });

```

Code 37-10 권장 사용 예

```

1  $p.ajax({
2      action : action,
3      mode : mode,
4      mediatype : mediatype,
5      method : method,
6      processMsg : processMsg,

```

```

7   requestData : reqData,
8   requestHeader : requestHeader,
9   beforeAjax : function(e) {
10  },
11  success : function(e) {
12      if (callbackFnc != '') {
13          window[callbackFnc](JSON.parse(e.responseText));
14      }
15  },
16  error : function(e) {
17      return _requestErrCallback(e);
18  }
19  });

```

37.5.1.2. Closure 내부에서 객체 참조

Closure는 전역이 아닌 지역 변수 선언이 가능하여 공통 모듈에서 많이 사용됩니다. 단, Closure 안에서 Object 형태의 외부 객체를 직접 참조하지 않도록 주의하십시오. 메모리에서 릴리즈 되지 않을 수 있습니다.

특히 Closure 함수를 비동기 형태로 호출할 경우, 객체를 직접 다른 변수에 할당하면 참조 관계가 발생합니다. 이 경우, 함수 실행이 완료되더라도 메모리에서 제거되지 않을 수 있습니다.

아래 예제에서 **test2**는 **test**와 직접 참조 관계가 이루어져 **test** 값 변경에 따라 **test2** 값이 변경됩니다. 즉 **test2**와 참조 관계가 발생하여 **test** 사용이 끝난 뒤에도 메모리에서 제거되지 않습니다.

Code 37-11 직접 참조 관계 예

```

1  //CASE1
2
3  var test = {a: "abc", b: "bbc"};
4  scwin.functionTest = function(value) {
5      scwin.test2 = value;
6      console.log(value);
7      // 결과 : {a: "abc", b: "bbc"}
8      console.log(scwin.test2);
9      // 결과 : {a: "abc", b: "bbc"}
10
11     for (var item in value) { value[item] = null; }
12     console.log(value);
13     // 결과 : {a: null, b: null}
14     console.log(scwin.test2);
15     // 결과 : {a: null, b: null}
16 };

```

```

17
18 scwin.functionTest(test);
19 console.log(test);
20     // 결과 : {a: null, b: null}
21
22 //CASE2
23
24 var test = {a: "abc", b: "bbc"};
25 scwin.functionTest = function(value) {
26     scwin.test2 = value;
27     console.log(value);
28     // 결과 : {a: "abc", b: "bbc"}
29     console.log(scwin.test2);
30     // 결과 : {a: "abc", b: "bbc"}
31 };
32 scwin.functionTest(test);
33 for (var item in test) { test[item] = null; }
34 console.log(test);
35     // 결과 : {a: null, b: null}
36 console.log(scwin.test2);
37     // 결과 : {a: null, b: null}

```

Code 37-12 비권장 사용 예

```

1  $.p.ajax({
2
3      // ...
4
5      success :    function(e) {
6                      resBody = e.responseBody;
7                      if (callbackFnc != '') {
8                          eval(callbackFnc + '(resBody, reqAjaxOptions)');
9                      }
10                     }
11  });
12 }
13
14 scwin.callback1 = function(resBody, reqAjaxOptions) {
15     if (reqAjaxOptions.mode) {
16
17         // ...
18

```

```

19     }
20     scwin.orgResData = resBody;
21 }

```

Code 37-13 권장 사용 예

```

1  $p.ajax({
2
3      //...
4
5      success :    function(e) {
6
7          resBody = e.responseBody;
8          if (callbackFnc != '') {
9              eval(callbackFnc + '(resBody, reqAjaxOptions)');
10             for(var item in resBody){
11                 resBody[item] = null;
12             }
13             for(var item in reqAjaxOptions){
14                 reqAjaxOptions[item] = null;
15             }
16         }
17     });
18 }
19
20 scwin.callback1 = function(resBody, reqAjaxOptions) {
21     if (reqAjaxOptions.mode) {
22
23         // ...
24
25     }
26
27     scwin.orgResData = JSON.parse(JSON.stringify(resBody));
28 }

```

37.5.2. LinkedDataList

LinkedDataList는 DataList 데이터를 필터링하여 아래와 같은 컴포넌트의 선택 항목으로 표시할 때 주로 사용하십시오.

- AutoComplete
- CheckBox
- CheckComboBox
- MultiSelect
- Radio
- SelectBox



권장 사용법

DataList의 데이터는 1,000 건 이하를 권장하며 최대 10,000건까지 지원합니다.

LinkedDataList는 GridView와 직접 바인딩하는 것을 권장하지 않습니다.

Code 37-14 비권장

```

1  <xf:model>
2    <w2:dataCollection baseNode="map">
3      ...
4    <w2:linkedDataList bind="dataList1" id="linkedDataList1">
5      <w2:condition type="filter"><![CDATA[]]></w2:condition>
6      <w2:condition type="sort"><![CDATA[]]></w2:condition>
7    </w2:linkedDataList>
8    ...
9    <w2:gridView scrollByColumn="false" defaultCellHeight="20"
10   scrollByColumnAdaptive="false" dataList="data:linkedDataList1">
11      ...
12   </w2:gridView>

```

Code 37-15 권장

```

1  <w2:dataList baseNode="list" repeatNode="map" id="dataList1"
2  saveRemovedData="true">
3    <w2:columnInfo>
4      <w2:column id="col1" name="name1" dataType="text"></w2:column>
5      <w2:column id="col2" name="name2" dataType="text"></w2:column>
6    </w2:columnInfo>
7  </w2:dataList>
8
9  ...
10 <w2:gridView scrollByColumn="false" defaultCellHeight="20"
11 scrollByColumnAdaptive="false" dataList="data:dataList1">
12   ...

```

37.5.3. GridView

37.5.3.1. 조회 건 수

GridView 조회시 허용 가능한 메모리의 크기는 임계치의 50%내로 사용하는 것을 권장.

그리드의 조회 건수는 약 400,000 Cell을 기준으로 제한하는 것을 권장

37.5.3.2. `visibleRowNum`

`visibleRowNum` 속성을 통해 페이지 당 표시할 행을 지정할 수 있습니다. 단, 최적의 성능을 위해 50행 이하를 권장합니다.

37.5.4. 팝업

37.5.4.1. 팝업 종류

아래와 같이 세 가지 종류의 팝업을 생성할 수 있습니다. 단, `type`을 `wframePopup`으로 설정한 WFrame 팝업을 대부분의 경우 권장합니다.

Table 37-2 `type` 설정

옵션	설명	비고
<code>wframePopup</code>	- WFrame s내에 팝업 화면을 호출. - Scope 기능 지원.	권장.
<code>iframePopup</code>	IFrame 내에 팝업 화면을 호출.	- 비권장. - 외부 도메인과 연동시 사용. - 외부 솔루션과 연동 시 사용.
<code>browserPopup</code>	- 브라우저가 제공하는 native 윈도우에 팝업 화면을 호출. - 팝업 호출을 위해 별도의 프로세스를 생성. - URL 주소가 노출됨.	- 비권장. - 별도 창이 필요한 경우 사용.



동영상

세 종류(WFrame, IFrame, Browser)의 팝업 생성

37.5.4.2. 파라미터 전달

- 다른 Frame의 함수 호출 시 String 타입의 객체 형태로 파라미터를 전달하십시오. String 타입이 아닌 JSON 객체 등을 직접 전달하여 다른 Frame의 객체에 할당할 경우 일부 브라우저에서 메모리 누수 현상이 발생합니다.
- 다른 Frame간 전달되는 Object의 타입은 팝업 형식의 호출에만 국한된 것이 아니고 IFrame으로 구성된 화면에도 적용됩니다. Frame 간의 파라미터 전달 방식은 String 타입의 객체를 파라미터로 전달하십시오.

Code 37-16 비권장 사용 예

```

1  //Child
2
3  comf.onPopupClose = function(retValue) {
4
5  //...
6
7  comf.onPopupClose(retValue);
8  }
9  comf.onPopupClose = function(retValue) {
10     opener.comf._onPopupClose(retValue);
11  }
12
13 //Parent
14 comf._onPopupClose = function(retValue) {
15     scwin.callUserCallback(retValue);
16  }

```

Code 37-17 권장 사용 예

```

1  //Child
2
3  scwin.onPopupClose = function() {
4
5  // ...
6
7  comf.onPopupClose(retValue);
8  }
9  comf.onPopupClose = function(retValue) {
10     var str = JSON.stringify(retValue);
11     opener.comf._onPopupClose(str);
12  }
13
14
15 //Parent

```

```

16  comf._onPopupClose = function(str) {
17      var retValue = JSON.parse(str);
18      scwin.callUserCallback(retValue);
19  }

```

37.5.5. TabControl

TabControl 컴포넌트는 **frameMode** 속성값으로 **iframe**과 **wframe**을 제공하며, 해당 속성값에 따라 콘텐츠를 렌더링합니다. 단, **iframe**을 설정할 경우 탭 수에 비례하여 브라우저의 메모리 점유가 증가합니다. 따라서 메인 층에서 사용하는 경우에만 속성값을 **iframe**으로 지정하길 권장합니다. **src** 속성은 콘텐츠에 웹스퀘어5 컴포넌트를 직접 구성하지 않고 content 영역을 XML 파일을 불러 렌더링합니다.

다음은 잘못 사용한 코드 예입니다.

Code 37-18 잘못된 frameMode 설정 (예)

```

1  <w2:tabControl confirmTrueAction="exist" useTabKeyOnly="true" id="tabControl1"
useConfirmMessage="false" confirmFalseAction="new"
2      alwaysDraw="false">
3      <w2:tabs id="tabs1" style="width:70px;height:30px" label="" disabled="false"></w2:tabs>
4      <w2:tabs id="tabs2" style="width:70px;height:30px" label="" disabled="false"></w2:tabs>
5      <w2:tabs id="tabs3" style="width:70px;height:30px" label="" disabled="false"></w2:tabs>
6      <w2:tabs id="tabs4" style="width:70px;height:30px" label="" disabled="false"></w2:tabs>
7      <!-- 올바른 예 -->
8      <w2:content id="content1" alwaysDraw="false" frameMode="wframe"
src="/tabChild_001.xml"></w2:content>
9      <!-- 잘못된 예. 브라우저 리소스를 많이 사용하게 될 iframe 를 사용 -->
10     <w2:content id="content2" alwaysDraw="false" frameMode="iframe"
src="/tabChild_002.xml"></w2:content>
11     <!-- 잘못된 예. frameMode를 사용하지 않음. wframe 를 사용하여 해당 컴포넌트에 src 사용.
-->
12     <w2:content id="content3" alwaysDraw="false">
13         <w2:wframe src="/tabChild_003.xml"></w2:wframe>
14     </w2:content>
15     <!-- 잘못된 예. frameMode 를 사용하지 않음. iframe 를 사용 해당 컴포넌트에 src 사용.
-->
16     <w2:content id="content4" alwaysDraw="false">
17         <w2:iframe id="" src="/tabChild_004.xml" spa="false"></w2:iframe>
18     </w2:content>
19 </w2:tabControl>

```

37.5.6. WFrame

37.5.6.1. WFrame 사용 권장

IFrame 수가 지나치게 많은 경우 메모리 점유율이 올라갑니다. IFrame 대신 WFrame을 사용을 권장합니다.

37.5.6.2. **class** selector 사용

WFrame을 사용하여 화면을 구성할 경우, 각 화면 내 컴포넌트의 ID는 항상 변합니다. 즉, `input1` 객체는 연결되는 WFrame에 따라 `wframe1_input1`이 될 수도 있고 `wframe2_wframe22_input1`이 될 수도 있습니다. 따라서 id select or 대신 class selector 사용을 권장합니다.

Code 37-19 사용 예

```
1 // class 사용 (권장)
2 <xf:group id="group1" class="contentZ1"/> .contentZ1 { }
3
4 // id 사용 (비권장)
5 <xf:group id="group1"/> #group1 { }
```



class명 중복 주의

class selector를 사용할 경우 부모 화면에 정의된 스타일 이름과 겹치지 않도록 주의하십시오. 이름이 겹칠 경우 부모 화면의 class를 덮어 쓰게 되어 부모 화면의 스타일이 변경됩니다.
class명이 겹치지 않도록 화면 ID를 class명 뒤에 붙이는 방법을 권장합니다.